

Node.js Design Patterns

Bản dịch song ngữ Anh–Việt · Third Edition

Mario Casciaro & Luciano Mammino (nguyên tác)

2026

Mục lục

What you need for this book — Bạn cần gì cho cuốn sách này	4
Who this book is for — Cuốn sách này dành cho ai	4
What this book covers — Cuốn sách này bao gồm những gì	4
To get the most out of this book — Để tận dụng tối đa cuốn sách này	6
Get in touch — Liên hệ	9
Chapter 1: The Node.js Platform — Chương 1: Nền tảng Node.js	11
The Node.js philosophy — Triết lý Node.js	12
How Node.js works — Node.js hoạt động thế nào	16
JavaScript in Node.js — JavaScript trong Node.js	25
Summary — Tóm tắt	29
Chapter 2: The Module System — Chương 2: Hệ thống Module	31
The need for modules — Nhu cầu về module	32
Module systems in JavaScript and Node.js — Hệ thống module trong JavaScript và Node.js	33
The module system and its patterns — Hệ thống module và các mẫu của nó	35
CommonJS modules — Module CommonJS	37
Module definition patterns — Các mẫu định nghĩa module	50
ESM: ECMAScript modules — ESM: ECMAScript modules	56
ESM and CommonJS differences and interoperability — Sự khác biệt và khả năng tương tác giữa ESM và CommonJS	82
Summary — Tóm tắt	85
Chapter 3: Callbacks and Events — Chương 3: Callback và Sự kiện	86
The Callback pattern — Mẫu Callback	87
The Observer pattern — Observer pattern	103
Summary — Tóm tắt	114
Exercises — Bài tập	115
Chapter 4: Asynchronous Control Flow Patterns with Callbacks — Chương 4: Các mẫu điều khiển luồng bất đồng bộ với Callback	116
The difficulties of asynchronous programming — Những khó khăn của lập trình bất đồng bộ	117
Callback best practices and control flow patterns — Best practice về callback và các mẫu điều khiển luồng	122
The async library — Thư viện async	149
Summary — Tóm tắt	151
Exercises — Bài tập	152
Chapter 5: Asynchronous Control Flow Patterns with Promises and Async/Await — Chương 5: Các mẫu điều khiển luồng bất đồng bộ với Promise và Async/Await	154

Promises — Promise	155
Async/await — Async/await	174
The problem with infinite recursive promise resolution chains — Vấn đề với các chuỗi resolution promise đệ quy vô hạn	187
Summary — Tóm tắt	190
Exercises — Bài tập	192
Chapter 6: Coding with Streams — Chương 6: Lập trình với Stream	193
Discovering the importance of streams — Khám phá tầm quan trọng của stream	194
Getting started with streams — Bắt đầu với stream	205
Asynchronous control flow patterns with streams — Các mẫu luồng điều khiển bất đồng bộ với stream	242
Piping patterns — Các piping pattern	253
Summary — Tóm tắt	271
Exercises — Bài tập	272
Chapter 7: Creational Design Patterns — Chương 7: Các mẫu thiết kế khởi tạo (Creational Design Patterns)	274
Factory — Factory	276
Builder — Builder	283
Revealing Constructor — Revealing Constructor	291
Singleton — Singleton	296
Wiring modules — Liên kết các module	300
Summary — Tóm tắt	310
Exercises — Bài tập	311
Chapter 8: Structural Design Patterns — Chương 8: Các mẫu thiết kế cấu trúc	313
Proxy — Proxy	314
Decorator — Decorator	331
The line between proxy and decorator — Ranh giới giữa proxy và decorator	339
Adapter — Adapter	340
Summary — Tóm tắt	345
Exercises — Bài tập	347
Chapter 9: Behavioral Design Patterns — Chương 9: Các mẫu thiết kế hành vi (Behavioral Design Patterns)	348
Strategy — Strategy	349
State — State	356
Template — Template	363
Iterator — Iterator	368
Middleware — Middleware	387
Command — Command	399
Summary — Tóm tắt	405
Exercises — Bài tập	406
Chapter 10: Universal JavaScript for Web Applications — Chương 10: Universal JavaScript cho ứng dụng web	409
Sharing code with the browser — Chia sẻ mã với trình duyệt	410
Fundamentals of cross-platform development — Những nền tảng cốt lõi của phát triển đa nền tảng	426
A brief introduction to React — Giới thiệu ngắn gọn về React	435

Creating a Universal JavaScript app — Tạo một ứng dụng Universal JavaScript	448
Summary — Tóm tắt	483
Exercises — Bài tập	484
Chapter 11: Advanced Recipes — Chương 11: Các công thức nâng cao	486
Dealing with asynchronously initialized components — Xử lý các thành phần được khởi tạo bất đồng bộ	487
Asynchronous request batching and caching — Gộp lô và lưu đệm các yêu cầu bất đồng bộ	495
Canceling asynchronous operations — Hủy các thao tác bất đồng bộ	506
Running CPU-bound tasks — Chạy các tác vụ chiếm dụng CPU	514
Summary — Tóm tắt	535
Exercises — Bài tập	536
Chapter 12: Scalability and Architectural Patterns — Chương 12: Khả năng mở rộng và các mẫu kiến trúc	538
An introduction to application scaling — Giới thiệu về việc mở rộng ứng dụng	539
Cloning and load balancing — Nhân bản và cân bằng tải	544
Decomposing complex applications — Phân tách các ứng dụng phức tạp	595
Summary — Tóm tắt	612
Exercises — Bài tập	614
Chapter 13: Messaging and Integration Patterns — Chương 13: Các mẫu nhắn tin và tích hợp	616
Fundamentals of a messaging system — Các nguyên lý nền tảng của một messaging system	617
Publish/Subscribe pattern — Mẫu Publish/Subscribe	626
Task distribution patterns — Các mẫu phân phối tác vụ	659
Request/Reply patterns — Các mẫu Request/Reply	683
Summary — Tóm tắt	697
Exercises — Bài tập	698

Throughout the book, you will be presented with real-life libraries and technologies, such as LevelDB, Redis, RabbitMQ, ZeroMQ, Express, and many others. They will be used to demonstrate a pattern or technique, and besides making the example more useful, these will also give you great exposure to the Node.js ecosystem and its set of solutions.

Xuyên suốt cuốn sách, bạn sẽ được giới thiệu các thư viện và công nghệ thực tế, chẳng hạn như LevelDB, Redis, RabbitMQ, ZeroMQ, Express, và nhiều thứ khác. Chúng sẽ được dùng để minh họa một mẫu thiết kế (design pattern) hoặc kỹ thuật, và bên cạnh việc làm cho ví dụ trở nên hữu ích hơn, chúng còn giúp bạn tiếp xúc nhiều với hệ sinh thái Node.js cùng tập hợp các giải pháp của nó.

Whether you use or plan to use Node.js for your work, your side project, or for an open source project, recognizing and using well-known patterns and techniques will allow you to use a common language when sharing your code and design, and on top of that, it will help you get a better understanding of the future of Node.js and how to make your own contributions a part of it.

Dù bạn đang dùng hay dự định dùng Node.js cho công việc, cho dự án cá nhân, hay cho một dự án mã nguồn mở, việc nhận biết và sử dụng các mẫu thiết kế cùng kỹ thuật phổ biến sẽ cho phép bạn dùng một ngôn ngữ chung khi chia sẻ code và thiết kế của mình. Hơn thế nữa, nó sẽ giúp bạn hiểu rõ hơn về tương lai của Node.js và cách biến những đóng góp của riêng bạn thành một phần của nó.

What you need for this book — Bạn cần gì cho cuốn sách này

To experiment with the code, you will need a working installation of Node.js version 14 (or greater) and npm version 6 (or greater). If some examples will require you to use some extra tooling, these will be described accordingly in place. You will also need to be familiar with the command line, know how to install an npm **package**, and know how to run Node.js applications. Finally, you will need a text editor to work with the code and a modern web browser.

Để thử nghiệm code, bạn sẽ cần một bản cài đặt Node.js phiên bản 14 (hoặc cao hơn) và npm phiên bản 6 (hoặc cao hơn) đang hoạt động. Nếu một số ví dụ yêu cầu bạn dùng thêm công cụ phụ trợ nào đó, chúng sẽ được mô tả tương ứng ngay tại chỗ. Bạn cũng cần quen thuộc với dòng lệnh, biết cách cài đặt một gói (package) npm, và biết cách chạy ứng dụng Node.js. Cuối cùng, bạn sẽ cần một trình soạn thảo văn bản để làm việc với code và một trình duyệt web hiện đại.

Who this book is for — Cuốn sách này dành cho ai

This book is for developers who have already had initial contact with Node.js and now want to get the most out of it in terms of productivity, design quality, and **scalability**. You are only required to have some prior exposure to the technology through some basic examples and some degree of familiarity with the JavaScript language, since this book will cover some basic concepts as well. Developers with intermediate experience in Node.js will also find the techniques presented in this book beneficial.

Cuốn sách này dành cho các lập trình viên đã có tiếp xúc ban đầu với Node.js và giờ muốn tận dụng tối đa nó về mặt năng suất, chất lượng thiết kế và khả năng mở rộng (scalability). Bạn chỉ cần có một chút tiếp xúc trước với công nghệ này thông qua một vài ví dụ cơ bản và một mức độ quen thuộc nhất định với ngôn ngữ JavaScript, vì cuốn sách này cũng sẽ trình bày một số khái niệm cơ bản. Các lập trình viên có kinh nghiệm trung cấp với Node.js cũng sẽ thấy những kỹ thuật được trình bày trong cuốn sách này hữu ích.

Some background in software design theory is also an advantage to understand some of the concepts presented.

Có một chút nền tảng về lý thuyết thiết kế phần mềm cũng là một lợi thế để hiểu một số khái niệm được trình bày.

This book assumes that you have a working knowledge of web application development, web services, databases, and data structures.

Cuốn sách này giả định rằng bạn có kiến thức thực hành về phát triển ứng dụng web, dịch vụ web, cơ sở dữ liệu và cấu trúc dữ liệu.

What this book covers — Cuốn sách này bao gồm những gì

Chapter 1, The Node.js Platform, serves as an introduction to the world of Node.js application design by showing the patterns at the core of the platform itself. It covers the Node.js ecosystem and its philosophy, and provides a quick introduction to the Node.js internals and the **reactor pattern**.

Chương 1, The Node.js Platform, đóng vai trò giới thiệu về thế giới thiết kế ứng dụng Node.js bằng cách trình bày những mẫu thiết kế nằm ở lõi của chính nền tảng này. Nó bao quát hệ sinh thái Node.js và triết lý của nó, đồng thời cung cấp một phần giới thiệu nhanh về nội bộ (internals) của Node.js và reactor pattern.

Chapter 2, The **Module System**, dives into the **module** systems available in Node.js, underlining the differences between **CommonJS** and the more modern ES modules from the ECMAScript 2015 specification.

Chương 2, The Module System, đi sâu vào các hệ thống module có sẵn trong Node.js, nhấn mạnh sự khác biệt giữa CommonJS và các ES modules hiện đại hơn từ đặc tả ECMAScript 2015.

Chapter 3, Callbacks and Events, introduces the first steps towards learning asynchronous coding and its patterns, discussing and comparing callbacks and the **event emitter (observer pattern)**.

Chương 3, Callbacks and Events, giới thiệu những bước đầu tiên hướng tới việc học lập trình bất đồng bộ và các mẫu của nó, thảo luận và so sánh giữa callback với event emitter (Observer pattern).

Chapter 4, Asynchronous **Control Flow** Patterns with Callbacks, introduces a set of patterns and techniques for efficiently handling asynchronous control flow in Node.js using callbacks. This chapter teaches you some traditional ways to mitigate the “**callback hell**” problem using plain JavaScript.

Chương 4, Asynchronous Control Flow Patterns with Callbacks, giới thiệu một tập hợp các mẫu và kỹ thuật để xử lý luồng điều khiển (control flow) bất đồng bộ một cách hiệu quả trong Node.js bằng callback. Chương này dạy bạn một số cách truyền thống để giảm thiểu vấn đề “callback hell” bằng JavaScript thuần.

Chapter 5, Asynchronous Control Flow Patterns with Promises and **Async/Await**, progresses with the exploration of more sophisticated and modern asynchronous control flow techniques.

Chương 5, Asynchronous Control Flow Patterns with Promises and Async/Await, tiếp tục với việc khám phá những kỹ thuật xử lý luồng điều khiển bất đồng bộ tinh vi và hiện đại hơn.

Chapter 6, Coding with Streams, dives deep into one of the most important tools in Node.js: streams. It shows you how to process data with transform streams and how to combine them into different patterns.

Chương 6, Coding with Streams, đi sâu vào một trong những công cụ quan trọng nhất trong Node.js: stream. Nó cho bạn thấy cách xử lý dữ liệu với transform stream và cách kết hợp chúng thành các mẫu khác nhau.

Chapter 7, Creational Design Patterns, starts to explore the traditional design patterns in Node.js. In this chapter, you will learn about some of the most popular creational design patterns, namely the **Factory** pattern, the Revealing Constructor pattern, the **Builder** pattern, and the **Singleton** pattern.

Chương 7, Creational Design Patterns, bắt đầu khám phá các mẫu thiết kế truyền thống trong Node.js. Trong chương này, bạn sẽ tìm hiểu về một số mẫu thiết kế khởi tạo (creational) phổ biến nhất, cụ thể là Factory pattern, Revealing Constructor pattern, Builder pattern, và Singleton pattern.

Chapter 8, Structural Design Patterns, continues the exploration of traditional design patterns in Node.js, covering structural design patterns such as **Proxy**, **Decorator**, and **Adapter**.

Chương 8, Structural Design Patterns, tiếp tục khám phá các mẫu thiết kế truyền thống trong Node.js, bao quát các mẫu thiết kế cấu trúc (structural) như Proxy, Decorator, và Adapter.

Chapter 9, Behavioral Design Patterns, concludes the conversation around traditional design patterns in Node.js by introducing behavioral design patterns like **Strategy**, **State**, Template,

Middleware, Command, and Iterator.

Chương 9, Behavioral Design Patterns, kết thúc cuộc thảo luận xoay quanh các mẫu thiết kế truyền thống trong Node.js bằng cách giới thiệu các mẫu thiết kế hành vi (behavioral) như Strategy, State, Template, Middleware, Command, và Iterator.

Chapter 10, **Universal JavaScript** for Web Applications, explores one of the most interesting capabilities of modern JavaScript web applications: being able to share code between the frontend and the backend. Throughout this chapter, you will learn the basic principles of Universal JavaScript by building a simple web application using modern tools and libraries. Chapter 11, Advanced Recipes, takes a problem-solution approach to show you how some common coding and design intricacies can be approached with ready-to-use solutions.

Chương 10, Universal JavaScript for Web Applications, khám phá một trong những khả năng thú vị nhất của các ứng dụng web JavaScript hiện đại: khả năng chia sẻ code giữa frontend và backend. Xuyên suốt chương này, bạn sẽ tìm hiểu các nguyên tắc cơ bản của Universal JavaScript bằng cách xây dựng một ứng dụng web đơn giản với các công cụ và thư viện hiện đại. Chương 11, Advanced Recipes, áp dụng cách tiếp cận vấn đề-giải pháp để cho bạn thấy cách xử lý một số rắc rối thường gặp trong code và thiết kế bằng những giải pháp sẵn dùng.

Chapter 12, Scalability and Architectural Patterns, teaches you the basic techniques and patterns for scaling a Node.js application.

Chương 12, Scalability and Architectural Patterns, dạy bạn các kỹ thuật và mẫu cơ bản để mở rộng quy mô một ứng dụng Node.js.

Chapter 13, Messaging and Integration Patterns, presents the most important messaging patterns, teaching you how to build and integrate complex distributed systems using Node.js and its ecosystem.

Chương 13, Messaging and Integration Patterns, trình bày những mẫu nhắn tin (messaging) quan trọng nhất, dạy bạn cách xây dựng và tích hợp các hệ thống phân tán phức tạp bằng Node.js và hệ sinh thái của nó.

To get the most out of this book — Để tận dụng tối đa cuốn sách này

To get the most out of this book you can download the example code files and the color images as per the instructions below.

Để tận dụng tối đa cuốn sách này, bạn có thể tải xuống các tệp code ví dụ và các hình ảnh màu theo hướng dẫn bên dưới.

Download the example code files — Tải xuống các tệp code ví dụ

You can download the example code files for this book from your account at www.packt.com/. If you purchased this book elsewhere, you can visit [www.packtpub.com/ support](http://www.packtpub.com/support) and register to have the files emailed directly to you.

Bạn có thể tải xuống các tệp code ví dụ cho cuốn sách này từ tài khoản của mình tại www.packt.com/. Nếu bạn mua cuốn sách này ở nơi khác, bạn có thể truy cập [www.packtpub.com/ support](http://www.packtpub.com/support) và đăng ký để các tệp được gửi trực tiếp qua email cho bạn.

You can download the code files by following these steps:

Bạn có thể tải xuống các tệp code bằng cách làm theo các bước sau:

1. Log in or register at <http://www.packt.com>.
2. Select the Support tab.
3. Click on Code Downloads.
4. Enter the name of the book in the Search box and follow the on-screen instructions.
 1. Đăng nhập hoặc đăng ký tại <http://www.packt.com>.
 2. Chọn tab Support.
 3. Nhấp vào Code Downloads.
 4. Nhập tên cuốn sách vào ô Search và làm theo hướng dẫn trên màn hình.

Once the file is downloaded, please make sure that you unzip or extract the folder using the latest version of:

Sau khi tệp được tải xuống, vui lòng đảm bảo rằng bạn giải nén thư mục bằng phiên bản mới nhất của:

- WinRAR / 7-Zip for Windows
- Zipeg / iZip / UnRarX for macOS
- 7-Zip / PeaZip for Linux
 - WinRAR / 7-Zip cho Windows
 - Zipeg / iZip / UnRarX cho macOS
 - 7-Zip / PeaZip cho Linux

The code bundle for the book is also hosted on GitHub at [nodejsdp.link/repo](https://github.com/PacktPublishing/nodejsdp.link/repo). In case there's an update to the code, it will be updated on the existing GitHub repository.

Gói code của cuốn sách cũng được lưu trữ trên GitHub tại [nodejsdp.link/repo](https://github.com/PacktPublishing/nodejsdp.link/repo). Trong trường hợp có cập nhật cho code, nó sẽ được cập nhật trên kho GitHub hiện có.

We also have other code bundles from our rich catalog of books and videos available at <https://github.com/PacktPublishing/>. Check them out!

Chúng tôi cũng có các gói code khác từ kho sách và video phong phú của mình tại <https://github.com/PacktPublishing/>. Hãy xem qua chúng!

Download the color images — Tải xuống các hình ảnh màu

We also provide a PDF file that has color images of the screenshots/diagrams used in this book. You can download it here: https://static.packt-cdn.com/downloads/9781839214110_ColorImages.pdf.

Chúng tôi cũng cung cấp một tệp PDF chứa các hình ảnh màu của ảnh chụp màn hình/sơ đồ được dùng trong cuốn sách này. Bạn có thể tải nó tại đây: https://static.packt-cdn.com/downloads/9781839214110_ColorImages.pdf.

Conventions used — Các quy ước được sử dụng

In this book, you will find a number of text styles that distinguish between different kinds of information. Here are some examples of these styles and an explanation of their meaning:

Trong cuốn sách này, bạn sẽ thấy một số kiểu định dạng văn bản phân biệt giữa các loại thông tin khác nhau. Dưới đây là một số ví dụ về các kiểu này cùng giải thích ý nghĩa của chúng:

- Code words in text: `server.listen(handle)`
- Pathname: `src/app.js`
- Dummy URL: `http://localhost:8080`
 - Từ ngữ code trong văn bản: `server.listen(handle)`
 - Tên đường dẫn: `src/app.js`
 - URL giả: `http://localhost:8080`

A block of code is generally formatted using StandardJS conventions ([nodejsdp.link/standard](https://nodejs.org/en/docs/standard/)) and it is set as follows:

Một khối code thường được định dạng theo các quy ước StandardJS ([nodejsdp.link/standard](https://nodejs.org/en/docs/standard/)) và được trình bày như sau:

```
import zmq from 'zeromq'
async function main () {
  const sink = new zmq.Pull()
  await sink.bind('tcp://*:5017')
  for await (const rawMessage of sink) {
    console.log('Message from worker: ', rawMessage.toString())
  }
}
```

When we wish to draw your attention to a particular part of a code block, the relevant lines or items are highlighted in bold:

Khi chúng tôi muốn thu hút sự chú ý của bạn vào một phần cụ thể của khối code, các dòng hoặc mục liên quan sẽ được tô đậm:

```
const wss = new ws.Server({ server })
wss.on('connection', client => {
  console.log('Client connected')
  client.on('message', msg => {
    console.log(`Message: ${msg}`)
    redisPub.publish('chat_messages', msg)
  })
})
```

Any command-line input or output is written as follows:

Mọi đầu vào hoặc đầu ra trên dòng lệnh được viết như sau:

```
node replier.js
node requestor.js
```

New terms and important words are shown in bold. Words that you see on the screen, for example, in menus or dialog boxes, appear in the text like this: “To explain the problem, we will create a little web spider, a command-line application that takes in a web URL as the input and downloads its contents locally into a file.”

Các thuật ngữ mới và các từ quan trọng được hiển thị in đậm. Những từ mà bạn thấy trên màn hình, ví dụ trong menu hoặc hộp thoại, xuất hiện trong văn bản như thế này: “Để giải thích vấn đề, chúng ta sẽ tạo một web spider nhỏ, một ứng dụng dòng lệnh nhận vào một URL web và tải nội dung của nó về một tệp cục bộ.”

Warnings or important notes appear like this.

Các cảnh báo hoặc lưu ý quan trọng xuất hiện như thế này.

Tips and tricks appear like this.

Các mẹo và thủ thuật xuất hiện như thế này.

Most URLs are linked through our own short URL system to make it easier for readers coming through the print edition to access them. These links are in the form `nodejsdp.link/some-descriptive-id`.

Hầu hết các URL được liên kết thông qua hệ thống URL rút gọn của riêng chúng tôi để giúp những độc giả đến từ bản in dễ dàng truy cập chúng hơn. Các liên kết này có dạng `nodejsdp.link/some-descriptive-id`.

Get in touch — Liên hệ

Feedback from our readers is always welcome.

Phản hồi từ độc giả của chúng tôi luôn được hoan nghênh.

General feedback: If you have questions about any aspect of this book, mention the book title in the subject of your message and email us at `customercare@packtpub.com`.

Phản hồi chung: Nếu bạn có thắc mắc về bất kỳ khía cạnh nào của cuốn sách này, hãy đề cập đến tên sách trong tiêu đề thư của bạn và gửi email cho chúng tôi tại `customercare@packtpub.com`.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you could report this to us. Please visit `www.packtpub.com/support/errata`, select your book, click on the Errata Submission Form link, and enter the details.

Đính chính (Errata): Mặc dù chúng tôi đã hết sức cẩn thận để đảm bảo tính chính xác của nội dung, các lỗi vẫn có thể xảy ra. Nếu bạn tìm thấy một lỗi trong cuốn sách này, chúng tôi sẽ rất biết ơn nếu bạn có thể báo cáo cho chúng tôi. Vui lòng truy cập `www.packtpub.com/support/errata`, chọn cuốn sách của bạn, nhấp vào liên kết Errata Submission Form, và nhập chi tiết.

Piracy: If you come across any illegal copies of our works in any form on the Internet, we would be grateful if you could provide us with the location address or website name. Please contact us at `copyright@packt.com` with a link to the material.

Vi phạm bản quyền (Piracy): Nếu bạn bắt gặp bất kỳ bản sao bất hợp pháp nào của các tác phẩm của chúng tôi dưới bất kỳ hình thức nào trên Internet, chúng tôi sẽ rất biết ơn nếu bạn có thể cung cấp cho chúng tôi địa chỉ vị trí hoặc tên trang web. Vui lòng liên hệ với chúng tôi tại `copyright@packt.com` kèm một liên kết đến tài liệu đó.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit `authors.packtpub.com`.

Nếu bạn quan tâm đến việc trở thành tác giả: Nếu có một chủ đề mà bạn có chuyên môn và bạn quan tâm đến việc viết hoặc đóng góp cho một cuốn sách, vui lòng truy cập `authors.packtpub.com`.

Reviews — Đánh giá

Please leave a review. Once you have read and used this book, why not leave a review on the site that you purchased it from? Potential readers can then see and use your unbiased opinion to make

purchase decisions, we at Packt can understand what you think about our products, and our authors can see your feedback on their book. Thank you!

Vui lòng để lại một đánh giá. Một khi bạn đã đọc và sử dụng cuốn sách này, sao không để lại một đánh giá trên trang mà bạn đã mua nó? Những độc giả tiềm năng khi đó có thể xem và dùng ý kiến khách quan của bạn để đưa ra quyết định mua hàng, chúng tôi tại Packt có thể hiểu bạn nghĩ gì về sản phẩm của mình, và các tác giả của chúng tôi có thể thấy phản hồi của bạn về cuốn sách của họ. Cảm ơn bạn!

For more information about Packt, please visit packt.com.

Để biết thêm thông tin về Packt, vui lòng truy cập packt.com.

Chapter 1: The Node.js Platform —

Chương 1: Nền tảng Node.js

Some principles and design patterns literally define the developer experience with the Node.js platform and its ecosystem. The most peculiar one is probably its asynchronous nature, which makes heavy use of asynchronous constructs such as callbacks and promises. In this introductory chapter, we will explore where Node.js gets its asynchronous behavior from. This is not just good-to-know theoretical information: knowing how Node.js works at its core will give you a strong foundation for understanding the reasoning behind more complex topics and patterns that we will cover later in the book.

Một số nguyên tắc và mẫu thiết kế (design pattern) gần như định hình trải nghiệm của lập trình viên với nền tảng Node.js và hệ sinh thái của nó. Đặc trưng kỳ lạ nhất có lẽ là bản chất bất đồng bộ của nó, vốn sử dụng rất nhiều cấu trúc bất đồng bộ như callback và promise. Trong chương mở đầu này, chúng ta sẽ khám phá xem hành vi bất đồng bộ của Node.js bắt nguồn từ đâu. Đây không chỉ là kiến thức lý thuyết “biết thì tốt”: hiểu cách Node.js hoạt động ở phần lõi sẽ cho bạn một nền tảng vững chắc để nắm bắt lý do đằng sau những chủ đề và pattern phức tạp hơn mà chúng ta sẽ đề cập về sau trong cuốn sách.

Another important aspect that characterizes Node.js is its philosophy. Approaching Node.js is, in fact, far more than simply learning a new technology: it’s also embracing a culture and a community. You will see how this greatly influences the way we design our applications and components, and the way they interact with those created by the community.

Một khía cạnh quan trọng khác đặc trưng cho Node.js là triết lý của nó. Tiếp cận Node.js thực ra không chỉ đơn thuần là học một công nghệ mới: đó còn là việc đón nhận một văn hóa và một cộng đồng. Bạn sẽ thấy điều này ảnh hưởng lớn thế nào đến cách chúng ta thiết kế ứng dụng và các thành phần của mình, cũng như cách chúng tương tác với những thứ do cộng đồng tạo ra.

In this chapter, you will learn about the following:

Trong chương này, bạn sẽ tìm hiểu những nội dung sau:

- The Node.js philosophy or the “Node way”
- The **reactor pattern**—the mechanism at the heart of the Node.js asynchronous **event-driven** architecture
- What it means to run JavaScript on the server compared to the browser
 - Triết lý Node.js hay “phong cách Node” (the “Node way”)
 - Reactor pattern—cơ chế nằm ở trung tâm của kiến trúc bất đồng bộ hướng sự kiện (event-driven) của Node.js
 - Việc chạy JavaScript trên máy chủ có ý nghĩa gì so với chạy trên trình duyệt

The Node.js philosophy — Triết lý Node.js

Every programming platform has its own philosophy, a set of principles and guidelines that are generally accepted by the community, or an ideology for doing things that influence both the evolution of the platform and how applications are developed and designed. Some of these principles arise from the technology itself, some of them are enabled by its ecosystem, some are just trends in the community, and others are evolutions of ideologies borrowed from other platforms. In Node.js, some of these principles come directly from its creator—Ryan Dahl—while others come from the people who contribute to the core or from charismatic figures in the community, and, finally, some are inherited from the larger JavaScript movement.

Mỗi nền tảng lập trình đều có triết lý riêng, một tập hợp các nguyên tắc và hướng dẫn được cộng đồng chấp nhận rộng rãi, hay một hệ tư tưởng về cách làm việc ảnh hưởng đến cả sự tiến hóa của nền tảng lẫn cách ứng dụng được phát triển và thiết kế. Một số nguyên tắc này nảy sinh từ chính công nghệ, một số được hệ sinh thái của nó tạo điều kiện, một số chỉ là xu hướng trong cộng đồng, và số khác là sự tiến hóa của những tư tưởng vay mượn từ các nền tảng khác. Trong Node.js, một số nguyên tắc đến trực tiếp từ người tạo ra nó—Ryan Dahl—trong khi số khác đến từ những người đóng góp cho phần lõi hoặc từ các nhân vật có sức ảnh hưởng trong cộng đồng, và cuối cùng, một số được kế thừa từ phong trào JavaScript rộng lớn hơn.

None of these rules are imposed and they should always be applied with common sense; however, they can prove to be tremendously useful when we are looking for a source of inspiration while designing our software.

Không có quy tắc nào trong số này bị áp đặt và chúng luôn nên được áp dụng một cách hợp lý; tuy nhiên, chúng có thể tỏ ra cực kỳ hữu ích khi ta tìm kiếm nguồn cảm hứng trong lúc thiết kế phần mềm.

You can find an extensive list of software development philosophies on Wikipedia at [nodejsdp.link/dev-philosophies](https://en.wikipedia.org/wiki/Software_development_philosophies).

Bạn có thể tìm thấy một danh sách phong phú các triết lý phát triển phần mềm trên Wikipedia tại [nodejsdp.link/dev-philosophies](https://en.wikipedia.org/wiki/Software_development_philosophies).

Small core — Lõi nhỏ (Small core)

The Node.js core—understood as the Node.js **runtime** and built-in modules—has its foundations built on a few principles. One of these is having the smallest possible set of functionalities, while leaving the rest to the so-called **userland** (or userspace), which is the ecosystem of modules living outside the core. This principle has an enormous impact on the Node.js culture, as it gives freedom to the community to experiment and iterate quickly on a broader set of solutions within the **scope** of the userland modules, instead of having one slowly evolving solution that is built into the more tightly controlled and stable core. Keeping the core set of functionalities to the bare minimum, then, is not only convenient in terms of maintainability, but also in terms of the positive cultural impact that it brings to the evolution of the entire ecosystem.

Phần lõi của Node.js—được hiểu là runtime Node.js và các module dựng sẵn—được xây dựng trên nền vài nguyên tắc. Một trong số đó là giữ tập chức năng nhỏ nhất có thể, đồng thời để phần còn lại cho cái gọi là userland (hay userspace), tức hệ sinh thái các module sống bên ngoài phần lõi. Nguyên tắc này có tác động to lớn đến văn hóa Node.js, vì nó trao cho cộng đồng quyền tự do thử nghiệm và lặp lại nhanh chóng trên một tập giải pháp rộng hơn trong phạm vi các module userland, thay vì có một giải pháp tiến hóa chậm chạp

được tích hợp vào phần lõi vốn được kiểm soát chặt chẽ và ổn định hơn. Như vậy, giữ tập chức năng lõi ở mức tối thiểu không chỉ thuận tiện về mặt khả năng bảo trì, mà còn về tác động văn hóa tích cực mà nó mang lại cho sự tiến hóa của toàn bộ hệ sinh thái.

Small modules — Module nhỏ (Small modules)

Node.js uses the concept of a **module** as the fundamental means for structuring the code of a program. It is the building block for creating applications and reusable libraries. In Node.js, one of the most evangelized principles is designing small modules (and packages), not only in terms of raw code size, but, most importantly, in terms of scope. This principle has its roots in the Unix philosophy, and particularly in two of its precepts, which are as follows:

Node.js sử dụng khái niệm module làm phương tiện cơ bản để cấu trúc mã của một chương trình. Đó là khối xây dựng để tạo ra ứng dụng và các thư viện tái sử dụng được. Trong Node.js, một trong những nguyên tắc được cổ vũ nhiều nhất là thiết kế các module (và gói (package)) nhỏ, không chỉ về kích thước mã thô, mà quan trọng nhất là về phạm vi (scope). Nguyên tắc này bắt nguồn từ triết lý Unix, và đặc biệt là hai trong số các châm ngôn của nó, như sau:

- “Small is beautiful.”
- “Make each program do one thing well.”
 - “Nhỏ là đẹp.” (Small is beautiful)
 - “Hãy để mỗi chương trình làm tốt một việc.” (Make each program do one thing well)

Node.js has brought these concepts to a whole new level. Along with the help of its module managers—with npm and yarn being the most popular—Node.js helps to solve the **dependency** hell problem by making sure that two (or more) packages depending on different versions of the same **package** will use their own installations of such a package, thus avoiding conflicts. This aspect allows packages to depend on a high number of small, well-focused dependencies without the risk of creating conflicts. While this can be considered unpractical or even totally unfeasible in other platforms, in Node.js, this practice is the norm. This enables extreme levels of reusability; they are so extreme, in fact, that sometimes we can find packages comprising of a single module containing just a couple of lines of code—for example, a regular expression for matching emails such as `nodejsdp.link/email-regex`.

Node.js đã đưa những khái niệm này lên một tầm hoàn toàn mới. Cùng với sự trợ giúp của các trình quản lý module của nó—mà npm và yarn là phổ biến nhất—Node.js giúp giải quyết vấn đề “địa ngục phụ thuộc” (dependency hell) bằng cách đảm bảo rằng hai (hoặc nhiều) gói phụ thuộc vào các phiên bản khác nhau của cùng một gói sẽ dùng bản cài đặt riêng của gói đó, qua đó tránh được xung đột. Khía cạnh này cho phép các gói phụ thuộc vào một số lượng lớn các phụ thuộc (dependency) nhỏ, tập trung tốt mà không có nguy cơ tạo ra xung đột. Trong khi điều này có thể bị xem là không thực tế hoặc thậm chí hoàn toàn bất khả thi ở các nền tảng khác, thì ở Node.js, thực hành này là chuẩn mực. Nó cho phép đạt mức tái sử dụng cực cao; thực ra cao đến mức đôi khi ta có thể bắt gặp những gói chỉ gồm một module duy nhất chứa vài dòng mã—ví dụ, một biểu thức chính quy để khớp email như `nodejsdp.link/email-regex`.

Besides the clear advantage in terms of reusability, a small module is also:

Bên cạnh lợi thế rõ ràng về khả năng tái sử dụng, một module nhỏ còn:

- Easier to understand and use
- Simpler to test and maintain

- Small in size and perfect for use in the browser
 - Dễ hiểu và dễ dùng hơn
 - Đơn giản hơn để kiểm thử và bảo trì
 - Nhỏ về kích thước và hoàn hảo để dùng trong trình duyệt

Having smaller and more focused modules empowers everyone to share or reuse even the smallest piece of code; it's the Don't Repeat Yourself (DRY) principle applied at a whole new level.

Việc có những module nhỏ hơn và tập trung hơn giúp mọi người có khả năng chia sẻ hoặc tái sử dụng dù chỉ là mẩu mã nhỏ nhất; đó là nguyên tắc Don't Repeat Yourself (DRY) được áp dụng ở một tầm hoàn toàn mới.

Small surface area — Bề mặt nhỏ (Small surface area)

In addition to being small in size and scope, a desirable characteristic of Node.js modules is exposing a minimal set of functionalities to the outside world. This has the effect of producing an **API** that is clearer to use and less susceptible to erroneous usage. In fact, most of the time the user of a component is only interested in a very limited and focused set of features, without needing to extend its functionality or tap into more advanced aspects.

Ngoài việc nhỏ về kích thước và phạm vi, một đặc tính đáng mong muốn của các module Node.js là chỉ phơi bày một tập chức năng tối thiểu ra thế giới bên ngoài. Điều này có tác dụng tạo ra một API rõ ràng hơn để sử dụng và ít có nguy cơ bị dùng sai. Thực tế, phần lớn thời gian người dùng một thành phần chỉ quan tâm đến một tập tính năng rất hạn chế và tập trung, mà không cần mở rộng chức năng của nó hay đào sâu vào các khía cạnh nâng cao hơn.

In Node.js, a very common pattern for defining modules is to expose only one functionality, such as a function or a class, for the simple fact that it provides a single, unmistakably clear entry point. Another characteristic of many Node.js modules is the fact that they are created to be used, rather than extended. Locking down the internals of a module by forbidding any possibility of an extension might sound inflexible, but it actually has the advantage of reducing use cases, simplifying implementation, facilitating maintenance, and increasing usability. In practice, this means preferring to expose functions instead of classes, and being careful not to expose any internals to the outside world.

Trong Node.js, một pattern rất phổ biến để định nghĩa module là chỉ phơi bày một chức năng duy nhất, chẳng hạn một hàm hoặc một lớp, vì lý do đơn giản là nó cung cấp một điểm vào (entry point) duy nhất, rõ ràng không thể nhầm lẫn. Một đặc tính khác của nhiều module Node.js là chúng được tạo ra để được sử dụng, chứ không phải để được mở rộng. Việc khóa kín phần nội bộ của một module bằng cách cấm mọi khả năng mở rộng nghe có vẻ cứng nhắc, nhưng thực ra nó có lợi thế là giảm số trường hợp sử dụng, đơn giản hóa hiện thực, tạo thuận lợi cho việc bảo trì và tăng khả năng dùng được. Trong thực tế, điều này nghĩa là ưu tiên phơi bày hàm thay vì lớp, và cẩn thận không phơi bày bất kỳ phần nội bộ nào ra thế giới bên ngoài.

Simplicity and pragmatism — Tính đơn giản và thực dụng (Simplicity and pragmatism)

Have you ever heard of the Keep It Simple, Stupid (KISS) principle? Richard P. Gabriel, a prominent computer scientist, coined the term “worse is better” to describe the model whereby less and simpler functionality is a good design choice for software. In his essay The Rise of “Worse is Better” he says:

Bạn đã bao giờ nghe đến nguyên tắc Keep It Simple, Stupid (KISS) chưa? Richard P. Gabriel, một nhà khoa học máy tính nổi bật, đã đặt ra thuật ngữ “worse is better” (tệ hơn lại tốt hơn) để mô tả mô hình theo đó chức năng ít hơn và đơn giản hơn là một lựa chọn thiết kế tốt cho phần mềm. Trong tiểu luận The Rise of “Worse is Better” của mình, ông viết:

“The design must be simple, both in implementation and interface. It is more important for the implementation to be simple than the interface. Simplicity is the most important consideration in a design.”

“Thiết kế phải đơn giản, cả về hiện thực lẫn giao diện. Việc hiện thực đơn giản còn quan trọng hơn việc giao diện đơn giản. Tính đơn giản là yếu tố quan trọng nhất trong một thiết kế.”

Designing simple, as opposed to perfect, fully featured software is a good practice for several reasons: it takes less effort to implement, it allows shipping faster with fewer resources, it’s easier to adapt, and, finally, it’s easier to maintain and understand. The positive effects of these factors encourage community contributions and allow the software itself to grow and improve.

Thiết kế phần mềm đơn giản, trái với phần mềm hoàn hảo và đầy đủ tính năng, là một thực hành tốt vì nhiều lý do: nó tốn ít công sức hơn để hiện thực, cho phép xuất xưởng nhanh hơn với ít tài nguyên hơn, dễ thích nghi hơn, và cuối cùng, dễ bảo trì và dễ hiểu hơn. Những hiệu ứng tích cực của các yếu tố này khuyến khích sự đóng góp của cộng đồng và cho phép bản thân phần mềm lớn lên và cải thiện.

In Node.js, the adoption of this principle is also facilitated by JavaScript, which is a very pragmatic language. In fact, it’s common to see simple classes, functions, and closures replacing complex class hierarchies. Pure object-oriented designs often try to replicate the real world using the mathematical terms of a computer system without considering the imperfection and complexity of the real world itself. Instead, the truth is that our software is always an approximation of reality, and we will probably have more success by trying to get something working sooner and with reasonable complexity, instead of trying to create near-perfect software with huge effort and tons of code to maintain.

Trong Node.js, việc áp dụng nguyên tắc này cũng được JavaScript tạo điều kiện, vì đây là một ngôn ngữ rất thực dụng. Thực tế, ta thường thấy các lớp, hàm và closure đơn giản thay thế cho những hệ thống phân cấp lớp phức tạp. Các thiết kế hướng đối tượng thuần túy thường cố gắng tái tạo thế giới thực bằng các thuật ngữ toán học của một hệ thống máy tính mà không xét đến sự không hoàn hảo và phức tạp của chính thế giới thực. Thay vào đó, sự thật là phần mềm của chúng ta luôn là một sự xấp xỉ của hiện thực, và ta có lẽ sẽ thành công hơn khi cố gắng làm cho thứ gì đó hoạt động sớm hơn với độ phức tạp hợp lý, thay vì cố tạo ra phần mềm gần như hoàn hảo bằng nỗ lực khổng lồ và hàng tấn mã phải bảo trì.

Throughout this book, you will see this principle in action many times. For example, a considerable number of traditional design patterns, such as **Singleton** or **Decorator**, can have a trivial, even if sometimes not bulletproof, implementation, and you will see how an uncomplicated, practical approach is (most of the time) preferred to a pure, flawless design.

Xuyên suốt cuốn sách này, bạn sẽ thấy nguyên tắc này được áp dụng nhiều lần. Ví dụ, một số lượng đáng kể các mẫu thiết kế truyền thống, như Singleton hay Decorator, có thể có một cách hiện thực tầm thường, dù đôi khi không thật sự kín kẽ, và bạn sẽ thấy cách một hướng tiếp cận đơn giản, thực dụng (phần lớn thời gian) được ưa chuộng hơn một thiết kế thuần túy, không tì vết.

Next, we will take a look inside the Node.js core to reveal its internal patterns and event-driven architecture.

Tiếp theo, chúng ta sẽ nhìn vào bên trong phần lõi của Node.js để hé lộ các pattern nội bộ và kiến trúc hướng sự kiện của nó.

How Node.js works — Node.js hoạt động thế nào

In this section, you will gain an understanding of how Node.js works internally and be introduced to the reactor pattern, which is the heart of the asynchronous nature of Node.js. We will go through the main concepts behind the pattern, such as the **single-threaded** architecture and the **non-blocking I/O**, and you will see how this creates the foundation for the entire Node.js platform.

Trong phần này, bạn sẽ hiểu được cách Node.js hoạt động bên trong và được giới thiệu về reactor pattern, vốn là trái tim của bản chất bất đồng bộ của Node.js. Chúng ta sẽ đi qua các khái niệm chính đằng sau pattern này, chẳng hạn kiến trúc đơn luồng (single-threaded) và I/O không chặn (non-blocking I/O), và bạn sẽ thấy điều này tạo nên nền tảng cho toàn bộ nền tảng Node.js như thế nào.

I/O is slow — I/O thì chậm

I/O (short for input/output) is definitely the slowest among the fundamental operations of a computer. Accessing the RAM is in the order of nanoseconds (10E-9 seconds), while accessing data on the disk or the network is in the order of milliseconds (10E-3 seconds). The same applies to the bandwidth. RAM has a transfer rate consistently in the order of GB/s, while the disk or network varies from MB/s to optimistically GB/s. I/O is usually not expensive in terms of CPU, but it adds a delay between the moment the **request** is sent to the device and the moment the operation completes. On top of that, we have to consider the human factor. In fact, in many circumstances, the input of an application comes from a real person—a mouse click, for example—so the speed and frequency of I/O doesn't only depend on technical aspects, and it can be many orders of magnitude slower than the disk or network.

I/O (viết tắt của input/output) chắc chắn là phép toán chậm nhất trong số các thao tác cơ bản của một máy tính. Truy cập RAM ở mức nano giây (10E-9 giây), trong khi truy cập dữ liệu trên đĩa hoặc trên mạng ở mức mili giây (10E-3 giây). Điều tương tự áp dụng cho băng thông. RAM có tốc độ truyền ổn định ở mức GB/s, trong khi đĩa hoặc mạng dao động từ MB/s đến lạc quan nhất là GB/s. I/O thường không tốn kém về mặt CPU, nhưng nó thêm một độ trễ giữa thời điểm yêu cầu được gửi đến thiết bị và thời điểm thao tác hoàn tất. Trên hết, ta còn phải xét đến yếu tố con người. Thực tế, trong nhiều tình huống, đầu vào của một ứng dụng đến từ một con người thật—một cú nhấp chuột, chẳng hạn—nên tốc độ và tần suất I/O không chỉ phụ thuộc vào các khía cạnh kỹ thuật, và nó có thể chậm hơn nhiều bậc so với đĩa hoặc mạng.

Blocking I/O — I/O chặn (Blocking I/O)

In traditional **blocking I/O** programming, the function call corresponding to an I/O request will block the execution of the **thread** until the operation completes. This can range from a few milliseconds, in the case of disk access, to minutes or even more, in the case of data being generated from user actions, such as pressing a key. The following pseudocode shows a typical blocking thread performed against a socket:

Trong lập trình I/O chặn (blocking I/O) truyền thống, lời gọi hàm tương ứng với một yêu cầu I/O sẽ chặn việc thực thi của luồng (thread) cho đến khi thao tác hoàn tất. Khoảng thời gian này có thể từ vài mili giây, trong trường hợp truy cập đĩa, đến vài phút hoặc thậm chí lâu hơn, trong trường hợp dữ liệu được tạo ra từ hành động của người dùng, như nhấn một phím. Đoạn mã giả sau minh họa một luồng chặn điển hình được thực hiện trên một socket:

```
// blocks the thread until the data is available
data = socket.read()
// data is available
print(data)
```

It is trivial to notice that a web server that is implemented using blocking I/O will not be able to handle multiple connections in the same thread. This is because each I/O operation on a socket will block the processing of any other connection. The traditional approach to solving this problem is to use a separate thread (or process) to handle each concurrent connection. This way, a thread blocked on an I/O operation will not impact the availability of the other connections, because they are handled in separate threads.

Dễ dàng nhận thấy rằng một máy chủ web được hiện thực bằng I/O chặn sẽ không thể xử lý nhiều kết nối trong cùng một luồng. Đó là vì mỗi thao tác I/O trên một socket sẽ chặn việc xử lý của bất kỳ kết nối nào khác. Cách tiếp cận truyền thống để giải quyết vấn đề này là dùng một luồng (hoặc tiến trình) riêng để xử lý mỗi kết nối đồng thời. Theo cách này, một luồng bị chặn trên một thao tác I/O sẽ không ảnh hưởng đến tính khả dụng của các kết nối khác, bởi chúng được xử lý ở các luồng riêng biệt.

The following illustrates this scenario:

Hình sau minh họa kịch bản này:

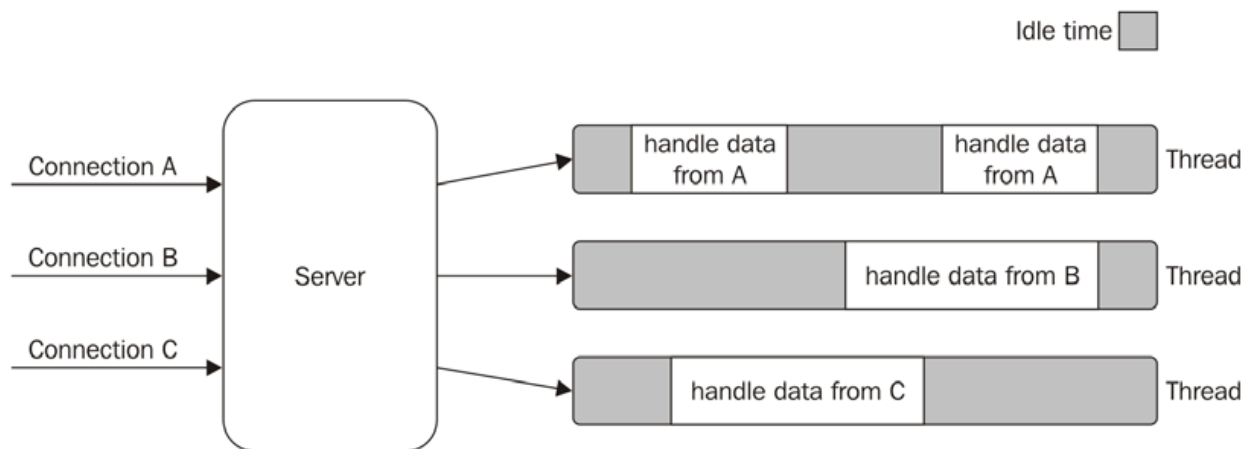


Figure 1.1: Using multiple threads to process multiple connections

Hình 1.1: Dùng nhiều luồng để xử lý nhiều kết nối

Figure 1.1 lays emphasis on the amount of time each thread is idle and waiting for new data to be received from the associated connection. Now, if we also consider that any type of I/O can possibly block a request—for example, while interacting with databases or with the filesystem—we will soon realize how many times a thread has to block in order to wait for the result of an I/O operation. Unfortunately, a thread is not cheap in terms of system resources—it consumes memory and causes

context switches—so having a long-running thread for each connection and not using it for most of the time means wasting precious memory and CPU cycles.

Hình 1.1 nhấn mạnh lượng thời gian mỗi luồng nhận rồi và chờ dữ liệu mới được nhận từ kết nối tương ứng. Bây giờ, nếu ta cũng xét rằng bất kỳ loại I/O nào cũng có thể chặn một yêu cầu—ví dụ, trong khi tương tác với cơ sở dữ liệu hoặc với hệ thống tệp—ta sẽ sớm nhận ra một luồng phải chặn bao nhiêu lần để chờ kết quả của một thao tác I/O. Đáng tiếc, một luồng không hề rẻ về mặt tài nguyên hệ thống—nó tiêu tốn bộ nhớ và gây ra chuyển ngữ cảnh (context switch)—nên việc có một luồng chạy lâu dài cho mỗi kết nối mà phần lớn thời gian không dùng đến nghĩa là lãng phí bộ nhớ quý giá và các chu kỳ CPU.

Non-blocking I/O — I/O không chặn (Non-blocking I/O)

In addition to blocking I/O, most modern operating systems support another mechanism to access resources, called non-blocking I/O. In this operating mode, the system call always returns immediately without waiting for the data to be read or written. If no results are available at the moment of the call, the function will simply return a predefined constant, indicating that there is no data available to return at that moment.

Ngoài I/O chặn, hầu hết các hệ điều hành hiện đại đều hỗ trợ một cơ chế khác để truy cập tài nguyên, gọi là I/O không chặn (non-blocking I/O). Ở chế độ vận hành này, lời gọi hệ thống luôn trả về ngay lập tức mà không chờ dữ liệu được đọc hoặc ghi. Nếu không có kết quả nào sẵn sàng tại thời điểm gọi, hàm sẽ chỉ đơn giản trả về một hằng số định trước, cho biết hiện thời không có dữ liệu để trả về.

For example, in Unix operating systems, the `fcntl()` function is used to manipulate an existing file descriptor (which in Unix represents the reference used to access a local file or a network socket) to change its operating mode to non-blocking (with the `O_NONBLOCK` flag). Once the resource is in non-blocking mode, any read operation will fail with the return code `EAGAIN` if the resource doesn't have any data ready to be read. The most basic pattern for dealing with this type of non-blocking I/O is to actively poll the resource within a loop until some actual data is returned. This is called busywaiting. The following pseudocode shows you how it's possible to read from multiple resources using non-blocking I/O and an active polling loop:

Ví dụ, trong các hệ điều hành Unix, hàm `fcntl()` được dùng để thao tác trên một file descriptor đang tồn tại (mà trong Unix biểu diễn tham chiếu dùng để truy cập một tệp cục bộ hoặc một network socket) nhằm đổi chế độ vận hành của nó sang không chặn (với cờ `O_NONBLOCK`). Một khi tài nguyên ở chế độ không chặn, mọi thao tác đọc sẽ thất bại với mã trả về `EAGAIN` nếu tài nguyên không có dữ liệu nào sẵn sàng để đọc. Pattern cơ bản nhất để xử lý loại I/O không chặn này là chủ động thăm dò (poll) tài nguyên trong một vòng lặp cho đến khi có dữ liệu thực sự được trả về. Việc này gọi là busy-waiting (chờ bận). Đoạn mã giả sau cho bạn thấy cách có thể đọc từ nhiều tài nguyên bằng I/O không chặn và một vòng lặp thăm dò chủ động:

```
resources = [socketA, socketB, fileA]
while (!resources.isEmpty()) {
    for (resource of resources) {
        // try to read
        data = resource.read()
        if (data == NO_DATA_AVAILABLE) {
            // there is no data to read at the moment
            continue
        }
    }
}
```

```

if (data === RESOURCE_CLOSED) {
    // the resource was closed, remove it from the list
    resources.remove(i)
} else {
    //some data was received, process it
    consumeData(data)
}
}
}

```

As you can see, with this simple technique, it is possible to handle different resources in the same thread, but it's still not efficient. In fact, in the preceding example, the loop will only consume precious CPU for iterating over resources that are unavailable most of the time. Polling algorithms usually result in a huge amount of wasted CPU time.

Như bạn thấy, với kỹ thuật đơn giản này, ta có thể xử lý nhiều tài nguyên khác nhau trong cùng một luồng, nhưng nó vẫn chưa hiệu quả. Thực tế, trong ví dụ trên, vòng lặp sẽ chỉ tiêu tốn CPU quý giá để lặp qua các tài nguyên mà phần lớn thời gian không sẵn sàng. Các thuật toán thăm dò thường dẫn đến một lượng thời gian CPU bị lãng phí khổng lồ.

Event demultiplexing — Phân kênh sự kiện (Event demultiplexing)

Busy-waiting is definitely not an ideal technique for processing non-blocking resources, but luckily, most modern operating systems provide a native mechanism to handle concurrent non-blocking resources in an efficient way. We are talking about the synchronous event demultiplexer (also known as the event notification interface).

Busy-waiting chắc chắn không phải là kỹ thuật lý tưởng để xử lý các tài nguyên không chặn, nhưng may mắn thay, hầu hết các hệ điều hành hiện đại đều cung cấp một cơ chế gốc (native) để xử lý các tài nguyên không chặn đồng thời một cách hiệu quả. Chúng ta đang nói đến bộ phân kênh sự kiện đồng bộ (synchronous event demultiplexer) (còn được gọi là giao diện thông báo sự kiện).

If you are unfamiliar with the term, in telecommunications, multiplexing refers to the method by which multiple signals are combined into one so that they can be easily transmitted over a medium with limited capacity. Demultiplexing refers to the opposite operation, whereby the signal is split again into its original components. Both terms are used in other areas (for example, video processing) to describe the general operation of combining different things into one and vice versa.

Nếu bạn chưa quen với thuật ngữ này, thì trong viễn thông, multiplexing (ghép kênh) đề cập đến phương pháp mà nhiều tín hiệu được kết hợp thành một để có thể dễ dàng truyền qua một môi trường có dung lượng hạn chế. Demultiplexing (phân kênh) đề cập đến thao tác ngược lại, theo đó tín hiệu lại được tách trở lại thành các thành phần ban đầu. Cả hai thuật ngữ đều được dùng trong các lĩnh vực khác (ví dụ, xử lý video) để mô tả thao tác chung của việc kết hợp những thứ khác nhau thành một và ngược lại.

The synchronous event demultiplexer that we were talking about watches multiple resources and returns a new event (or set of events) when a read or write operation executed over one of those resources completes. The advantage here is that the synchronous event demultiplexer is, of course, synchronous, so it blocks until there are new events to process. The following is the pseudocode of an algorithm that uses a generic synchronous event demultiplexer to read from two different resources:

Bộ phân kênh sự kiện đồng bộ mà chúng ta đang nói đến theo dõi nhiều tài nguyên và trả về một sự kiện mới (hoặc một tập sự kiện) khi một thao tác đọc hoặc ghi được thực thi

trên một trong các tài nguyên đó hoàn tất. Lợi thế ở đây là bộ phân kênh sự kiện đồng bộ, dĩ nhiên, là đồng bộ, nên nó chặn cho đến khi có sự kiện mới để xử lý. Sau đây là mã giả của một thuật toán dùng một bộ phân kênh sự kiện đồng bộ tổng quát để đọc từ hai tài nguyên khác nhau:

```
watchedList.add(socketA, FOR_READ) // (1)
watchedList.add(fileB, FOR_READ)
while (events = demultiplexer.watch(watchedList)) { // (2)
    // event loop
    for (event of events) { // (3)
        // This read will never block and will always return data
        data = event.resource.read()
        if (data === RESOURCE_CLOSED) {
            // the resource was closed, remove it from the watched list
            demultiplexer.unwatch(event.resource)
        } else {
            // some actual data was received, process it
            consumeData(data)
        }
    }
}
```

Let's see what happens in the preceding pseudocode:

Hãy xem điều gì xảy ra trong đoạn mã giả trên:

1. The resources are added to a data structure, associating each one of them with a specific operation (in our example, a read).
2. The demultiplexer is set up with the group of resources to be watched. The call to `demultiplexer.watch()` is synchronous and blocks until any of the watched resources are ready for read. When this occurs, the event demultiplexer returns from the call and a new set of events is available to be processed.
3. Each event returned by the event demultiplexer is processed. At this point, the resource associated with each event is guaranteed to be ready to read and to not block during the operation. When all the events are processed, the flow will block again on the event demultiplexer until new events are again available to be processed. This is called the **event loop**.

1. Các tài nguyên được thêm vào một cấu trúc dữ liệu, gắn mỗi tài nguyên với một thao tác cụ thể (trong ví dụ của chúng ta là một read).
2. Bộ phân kênh được thiết lập với nhóm tài nguyên cần theo dõi. Lệnh gọi `demultiplexer.watch()` là đồng bộ và chặn cho đến khi bất kỳ tài nguyên nào trong số đang được theo dõi sẵn sàng cho read. Khi điều này xảy ra, bộ phân kênh sự kiện trở về từ lệnh gọi và một tập sự kiện mới sẵn sàng để xử lý.
3. Mỗi sự kiện do bộ phân kênh sự kiện trả về được xử lý. Tại điểm này, tài nguyên gắn với mỗi sự kiện được đảm bảo là sẵn sàng để đọc và sẽ không chặn trong suốt thao tác. Khi tất cả sự kiện đã được xử lý, luồng sẽ lại chặn trên bộ phân kênh sự kiện cho đến khi có sự kiện mới sẵn sàng để xử lý. Việc này gọi là vòng lặp sự kiện (event loop).

It's interesting to see that, with this pattern, we can now handle several I/O operations inside a single thread, without using the busy-waiting technique. It should now be clearer why we are talking about demultiplexing; using just a single thread, we can deal with multiple resources. Figure 1.2 will help you visualize what's happening in a web server that uses a synchronous event demultiplexer and a

single thread to handle multiple concurrent connections:

Điều thú vị là, với pattern này, giờ đây ta có thể xử lý nhiều thao tác I/O bên trong một luồng duy nhất, mà không cần dùng kỹ thuật busy-waiting. Giờ thì lý do ta nói về phân kênh hẳn đã rõ hơn; chỉ dùng một luồng duy nhất, ta có thể xử lý nhiều tài nguyên. Hình 1.2 sẽ giúp bạn hình dung điều gì đang xảy ra trong một máy chủ web dùng một bộ phân kênh sự kiện đồng bộ và một luồng duy nhất để xử lý nhiều kết nối đồng thời:

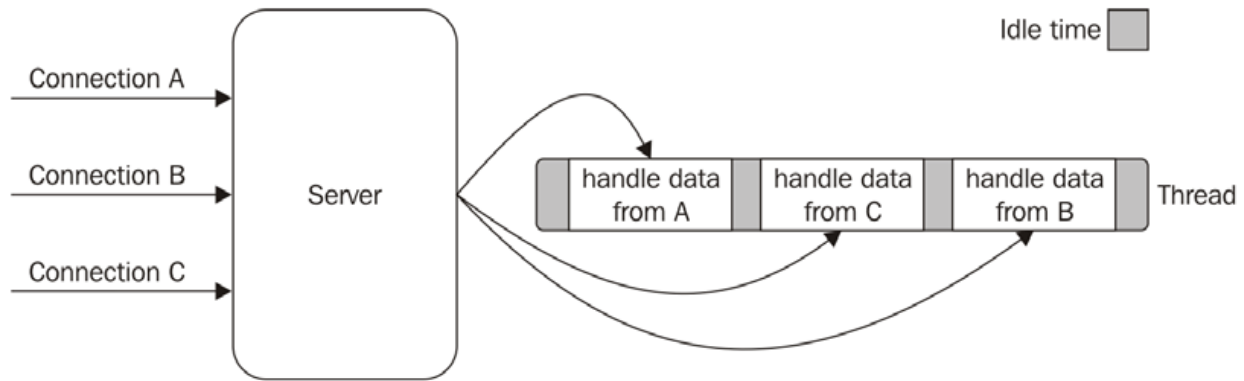


Figure 1.2: Using a single thread to process multiple connections

Hình 1.2: Dùng một luồng duy nhất để xử lý nhiều kết nối

As this shows, using only one thread does not impair our ability to run multiple I/O-bound tasks concurrently. The tasks are spread over time, instead of being spread across multiple threads. This has the clear advantage of minimizing the total idle time of the thread, as is clearly shown in Figure 1.2.

Như hình này cho thấy, việc chỉ dùng một luồng không làm suy giảm khả năng chạy nhiều tác vụ ràng buộc I/O (I/O-bound) đồng thời của chúng ta. Các tác vụ được trải ra theo thời gian, thay vì được trải ra trên nhiều luồng. Điều này có lợi thế rõ ràng là giảm thiểu tổng thời gian nhàn rỗi của luồng, như được thể hiện rõ trong Hình 1.2.

But this is not the only reason for choosing this I/O model. In fact, having a single thread also has a beneficial impact on the way programmers approach **concurrency** in general. Throughout the book, you will see how the absence of in-process race conditions and multiple threads to synchronize allows us to use much simpler concurrency strategies.

Nhưng đây không phải là lý do duy nhất để chọn mô hình I/O này. Thực tế, việc có một luồng duy nhất cũng có tác động có lợi đến cách lập trình viên tiếp cận tính đồng thời (concurrency) nói chung. Xuyên suốt cuốn sách, bạn sẽ thấy việc không có race condition trong tiến trình và không có nhiều luồng để đồng bộ hóa cho phép chúng ta dùng những chiến lược xử lý đồng thời đơn giản hơn nhiều.

The reactor pattern — Reactor pattern

We can now introduce the reactor pattern, which is a specialization of the algorithms presented in the previous sections. The main idea behind the reactor pattern is to have a handler associated with each I/O operation. A handler in Node.js is represented by a **callback** (or cb for short) function. The handler will be invoked as soon as an event is produced and processed by the event loop. The structure of the reactor pattern is shown in Figure 1.3:

Giờ chúng ta có thể giới thiệu reactor pattern, vốn là một sự chuyên biệt hóa của các thuật toán đã trình bày ở những phần trước. Ý tưởng chính đằng sau reactor pattern là gắn một handler với mỗi thao tác I/O. Một handler trong Node.js được biểu diễn bằng một hàm callback (hay viết tắt là cb). Handler sẽ được gọi ngay khi một sự kiện được tạo ra và được vòng lặp sự kiện xử lý. Cấu trúc của reactor pattern được thể hiện trong Hình 1.3:

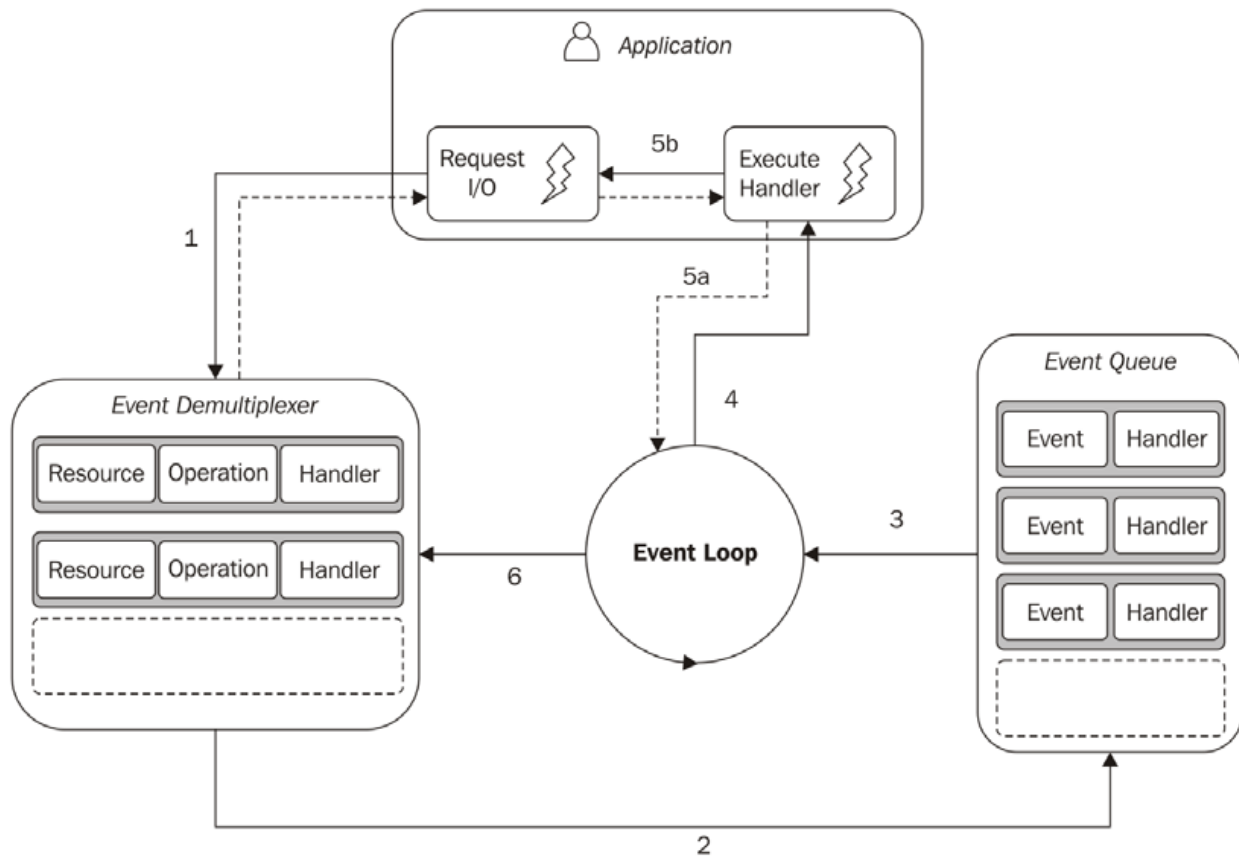


Figure 1.3: The reactor pattern

Hình 1.3: Reactor pattern

This is what happens in an application using the reactor pattern:

Đây là những gì xảy ra trong một ứng dụng dùng reactor pattern:

1. The application generates a new I/O operation by submitting a request to the Event Demultiplexer. The application also specifies a handler, which will be invoked when the operation completes. Submitting a new request to the Event Demultiplexer is a non-blocking call and it immediately returns control to the application.
2. When a set of I/O operations completes, the Event Demultiplexer pushes a set of corresponding events into the Event Queue.
3. At this point, the Event Loop iterates over the items of the Event Queue.
4. For each event, the associated handler is invoked.
5. The handler, which is part of the application code, gives back control to the Event Loop when its execution completes (5a). While the handler executes, it can request new asynchronous operations (5b), causing new items to be added to the Event Demultiplexer (1).
6. When all the items in the Event Queue are processed, the Event Loop blocks again on the Event Demultiplexer, which then triggers another cycle when a new event is available.

1. Ứng dụng tạo ra một thao tác I/O mới bằng cách gửi một yêu cầu đến Event Demultiplexer. Ứng dụng cũng chỉ định một handler, sẽ được gọi khi thao tác hoàn tất. Việc gửi một yêu cầu mới đến Event Demultiplexer là một lời gọi không chặn và nó trả quyền điều khiển về cho ứng dụng ngay lập tức.
2. Khi một tập thao tác I/O hoàn tất, Event Demultiplexer đẩy một tập sự kiện tương ứng vào Event Queue.
3. Tại điểm này, Event Loop lặp qua các phần tử của Event Queue.
4. Với mỗi sự kiện, handler tương ứng được gọi.
5. Handler, vốn là một phần của mã ứng dụng, trả quyền điều khiển về cho Event Loop khi việc thực thi của nó hoàn tất (5a). Trong khi handler thực thi, nó có thể yêu cầu các thao tác bất đồng bộ mới (5b), khiến các phần tử mới được thêm vào Event Demultiplexer (1).
6. Khi tất cả phần tử trong Event Queue đã được xử lý, Event Loop lại chặn trên Event Demultiplexer, rồi nó kích hoạt một chu kỳ khác khi có một sự kiện mới sẵn sàng.

The asynchronous behavior has now become clear. The application expresses interest in accessing a resource at one point in time (without blocking) and provides a handler, which will then be invoked at another point in time when the operation completes.

Hành vi bất đồng bộ giờ đã trở nên rõ ràng. Ứng dụng bày tỏ ý định truy cập một tài nguyên tại một thời điểm (mà không chặn) và cung cấp một handler, handler này sau đó sẽ được gọi tại một thời điểm khác khi thao tác hoàn tất.

A Node.js application will exit when there are no more pending operations in the event demultiplexer, and no more events to be processed inside the event queue.

Một ứng dụng Node.js sẽ thoát khi không còn thao tác nào đang chờ trong bộ phân kênh sự kiện, và không còn sự kiện nào để xử lý bên trong hàng đợi sự kiện.

We can now define the pattern at the heart of Node.js:

Giờ ta có thể định nghĩa pattern nằm ở trái tim của Node.js:

The reactor pattern

Reactor pattern

Handles I/O by blocking until new events are available from a set of observed resources, and then reacts by dispatching each event to an associated handler.

Xử lý I/O bằng cách chặn cho đến khi có sự kiện mới sẵn sàng từ một tập tài nguyên được quan sát, rồi phản ứng bằng cách điều phối mỗi sự kiện đến một handler tương ứng.

Libuv, the I/O engine of Node.js — Libuv, động cơ I/O của Node.js

Each operating system has its own interface for the event demultiplexer: `epoll` on Linux, `kqueue` on macOS, and the I/O completion port (IOCP) API on Windows. On top of that, each I/O operation can behave quite differently depending on the type of resource, even within the same operating system. In Unix operating systems, for example, regular filesystem files do not support non-blocking operations, so in order to simulate non-blocking behavior, it is necessary to use a separate thread outside the event loop.

Mỗi hệ điều hành có giao diện riêng cho bộ phân kênh sự kiện: `epoll` trên Linux, `kqueue` trên macOS, và API I/O completion port (IOCP) trên Windows. Trên hết, mỗi thao tác I/O có thể hành xử khá khác nhau tùy theo loại tài nguyên, ngay cả trong cùng một hệ điều hành. Ví dụ, trong các hệ điều hành Unix, các tệp hệ thống tệp thông thường không hỗ

trợ thao tác không chặn, nên để mô phỏng hành vi không chặn, cần dùng một luồng riêng nằm ngoài vòng lặp sự kiện.

All these inconsistencies across and within the different operating systems required a higher-level abstraction to be built for the event demultiplexer. This is exactly why the Node.js core team created a native library called libuv, with the objective to make Node.js compatible with all the major operating systems and normalize the nonblocking behavior of the different types of resource. Libuv represents the low-level I/O engine of Node.js and is probably the most important component that Node.js is built on.

Tất cả những sự không nhất quán giữa các hệ điều hành khác nhau và ngay trong từng hệ điều hành này đòi hỏi phải xây dựng một lớp trừu tượng cao hơn cho bộ phân kênh sự kiện. Đây chính là lý do nhóm phát triển lõi Node.js đã tạo ra một thư viện gốc gọi là libuv, với mục tiêu làm cho Node.js tương thích với tất cả các hệ điều hành lớn và chuẩn hóa hành vi không chặn của các loại tài nguyên khác nhau. Libuv đại diện cho động cơ I/O cấp thấp của Node.js và có lẽ là thành phần quan trọng nhất mà Node.js được xây dựng dựa trên.

Other than abstracting the underlying system calls, libuv also implements the reactor pattern, thus providing an API for creating event loops, managing the event queue, running asynchronous I/O operations, and queuing other types of task. A great resource to learn more about libuv is the free online book created by Nikhil Marathe, which is available at nodejsdp.link/uvbook.

Ngoài việc trừu tượng hóa các lời gọi hệ thống bên dưới, libuv còn hiện thực reactor pattern, qua đó cung cấp một API để tạo vòng lặp sự kiện, quản lý hàng đợi sự kiện, chạy các thao tác I/O bất đồng bộ, và xếp hàng các loại tác vụ khác. Một nguồn tài liệu tuyệt vời để tìm hiểu thêm về libuv là cuốn sách trực tuyến miễn phí do Nikhil Marathe tạo ra, có sẵn tại nodejsdp.link/uvbook.

The recipe for Node.js — Công thức tạo nên Node.js

The reactor pattern and libuv are the basic building blocks of Node.js, but we need three more components to build the full platform:

Reactor pattern và libuv là những khối xây dựng cơ bản của Node.js, nhưng chúng ta cần thêm ba thành phần nữa để dựng nên nền tảng hoàn chỉnh:

- A set of bindings responsible for wrapping and exposing libuv and other low-level functionalities to JavaScript.
- V8, the JavaScript engine originally developed by Google for the Chrome browser. This is one of the reasons why Node.js is so fast and efficient. V8 is acclaimed for its revolutionary design, its speed, and for its efficient memory management.
- A core JavaScript library that implements the high-level Node.js API.
 - Một tập các binding chịu trách nhiệm bọc và phơi bày libuv cùng các chức năng cấp thấp khác cho JavaScript.
 - V8, động cơ JavaScript ban đầu được Google phát triển cho trình duyệt Chrome. Đây là một trong những lý do khiến Node.js nhanh và hiệu quả đến vậy. V8 được ca ngợi vì thiết kế mang tính cách mạng, tốc độ, và khả năng quản lý bộ nhớ hiệu quả.
 - Một thư viện JavaScript lõi hiện thực API Node.js cấp cao.

This is the recipe for creating Node.js, and the following image represents its final architecture:

Đây là công thức để tạo nên Node.js, và hình ảnh sau đại diện cho kiến trúc cuối cùng của nó:

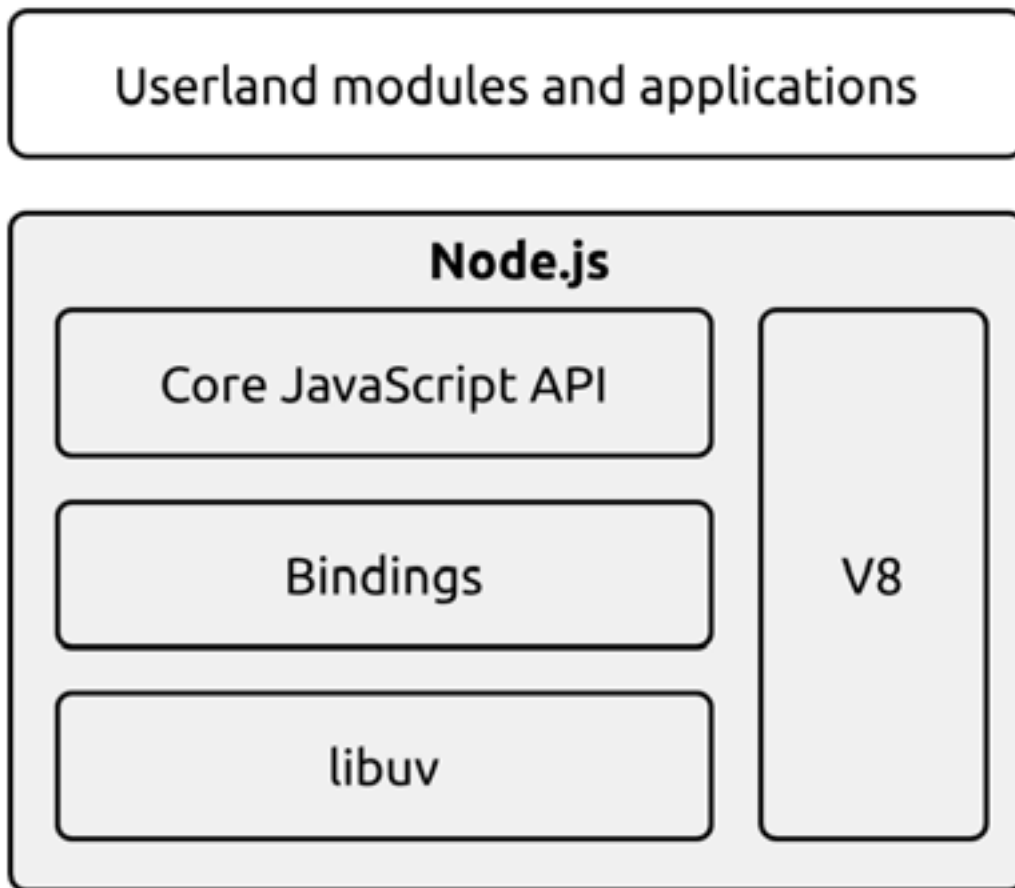


Figure 1.4: The Node.js internal components

Hình 1.4: Các thành phần nội bộ của Node.js

This concludes our journey through the internal mechanisms of Node.js. Next, we'll take a look at some important aspects to take into consideration when working with JavaScript in Node.js.

Đến đây kết thúc hành trình của chúng ta qua các cơ chế nội bộ của Node.js. Tiếp theo, chúng ta sẽ xem xét một số khía cạnh quan trọng cần lưu tâm khi làm việc với JavaScript trong Node.js.

JavaScript in Node.js — JavaScript trong Node.js

One important consequence of the architecture we have just analyzed is that the JavaScript we use in Node.js is somewhat different from the JavaScript we use in the browser.

Một hệ quả quan trọng của kiến trúc mà ta vừa phân tích là JavaScript mà chúng ta dùng trong Node.js phần nào khác với JavaScript mà chúng ta dùng trong trình duyệt.

The most obvious difference is that in Node.js we don't have a DOM and we don't have a window or a document. On the other hand, Node.js has access to a set of services offered by the underlying operating system that are not available in the browser. In fact, the browser has to implement a set of safety measures to make sure that the underlying system is not compromised by a rogue web application. The browser provides a higher-level abstraction over the operating system resources,

which makes it easier to control and contain the code that runs in it, which will also inevitably limit its capabilities. In turn, in Node.js we can virtually have access to all the services exposed by the operating system.

Khác biệt rõ ràng nhất là trong Node.js chúng ta không có DOM và không có window hay document. Mặt khác, Node.js có quyền truy cập vào một tập dịch vụ do hệ điều hành bên dưới cung cấp mà không có sẵn trong trình duyệt. Thực tế, trình duyệt phải hiện thực một tập biện pháp an toàn để đảm bảo hệ thống bên dưới không bị một ứng dụng web độc hại làm tổn hại. Trình duyệt cung cấp một lớp trừu tượng cao hơn trên các tài nguyên của hệ điều hành, giúp dễ kiểm soát và bao chứa mã chạy trong nó hơn, nhưng điều này cũng tất yếu giới hạn khả năng của nó. Ngược lại, trong Node.js chúng ta gần như có thể truy cập tất cả dịch vụ mà hệ điều hành phơi bày.

In this overview, we'll take a look at some key facts to keep in mind when using JavaScript in Node.js.

Trong phần tổng quan này, chúng ta sẽ xem xét một số điều then chốt cần ghi nhớ khi dùng JavaScript trong Node.js.

Run the latest JavaScript with confidence — Chạy JavaScript mới nhất một cách tự tin

One of the main pain points of using JavaScript in the browser is that our code will likely run on a variety of devices and browsers. Dealing with different browsers means dealing with JavaScript runtimes that may miss some of the newest features of both the language or the web platform. Luckily, today this problem can be somewhat mitigated by the use of transpilers and polyfills. Nonetheless, this brings its own set of disadvantages and not everything can be polyfilled.

Một trong những điểm gây đau đầu chính khi dùng JavaScript trong trình duyệt là mã của chúng ta có khả năng sẽ chạy trên đủ loại thiết bị và trình duyệt khác nhau. Phải đối phó với các trình duyệt khác nhau nghĩa là phải đối phó với những runtime JavaScript có thể thiếu một số tính năng mới nhất của cả ngôn ngữ lẫn nền tảng web. May thay, ngày nay vấn đề này có thể được giảm nhẹ phần nào nhờ việc dùng các transpiler và polyfill. Tuy nhiên, điều này mang theo những bất lợi riêng và không phải thứ gì cũng có thể polyfill được.

All these inconveniences don't apply when developing applications on Node.js. In fact, our Node.js applications will most likely run on a system and a Node.js runtime that are well known in advance. This makes a huge difference as it allows us to target our code for a specific JavaScript and Node.js version, with the absolute guarantee that we won't have any surprises when we run it on production.

Tất cả những bất tiện này không xuất hiện khi phát triển ứng dụng trên Node.js. Thực tế, các ứng dụng Node.js của chúng ta nhiều khả năng sẽ chạy trên một hệ thống và một runtime Node.js đã được biết rõ trước. Điều này tạo nên khác biệt khổng lồ vì nó cho phép chúng ta nhắm mã của mình vào một phiên bản JavaScript và Node.js cụ thể, với sự đảm bảo tuyệt đối rằng sẽ không có bất ngờ nào khi chạy nó trên môi trường production.

This factor, in combination with the fact that Node.js ships with very recent versions of V8, means that we can use with confidence most of the features of the latest ECMAScript specification (ES for short; this is the standard on which the JavaScript language is based) without the need for any extra transpilation step. Please bear in mind, though, that if we are developing a library meant to be used by third parties, we still have to take into account that our code may run on different versions of Node.js. The general pattern in this case is to target the oldest active longterm support (LTS) release and specify the engines section in our package.json, so that the package manager will warn the user if they are trying to install a package that is not compatible with their version of Node.js.

Yếu tố này, kết hợp với việc Node.js đi kèm các phiên bản V8 rất mới, nghĩa là chúng ta có thể tự tin dùng hầu hết các tính năng của đặc tả ECMAScript mới nhất (viết tắt là ES; đây là chuẩn mà ngôn ngữ JavaScript dựa trên) mà không cần bất kỳ bước transpile bổ sung nào. Tuy vậy, xin lưu ý rằng nếu chúng ta đang phát triển một thư viện dành cho bên thứ ba sử dụng, ta vẫn phải tính đến việc mã của mình có thể chạy trên các phiên bản Node.js khác nhau. Pattern chung trong trường hợp này là nhắm vào bản phát hành hỗ trợ dài hạn (long-term support, LTS) còn hoạt động cũ nhất và chỉ định phần engines trong package.json của chúng ta, để trình quản lý gói sẽ cảnh báo người dùng nếu họ đang cố cài một gói không tương thích với phiên bản Node.js của họ.

You can find out more about the Node.js release cycles at [nodejsdp.link/node-releases](https://nodejs.org/en/about/releases). Also, you can find the reference for the engines section of package.json at [nodejsdp.link/package-engines](https://nodejs.org/en/about/package-engines). Finally, you can get an idea of what ES feature is supported by each Node.js version at [nodejsdp.link/node-green](https://nodejs.org/en/about/node-green).

Bạn có thể tìm hiểu thêm về các chu kỳ phát hành của Node.js tại [nodejsdp.link/node-releases](https://nodejs.org/en/about/releases). Ngoài ra, bạn có thể tìm tài liệu tham khảo cho phần engines của package.json tại [nodejsdp.link/package-engines](https://nodejs.org/en/about/package-engines). Cuối cùng, bạn có thể nắm được tính năng ES nào được hỗ trợ bởi mỗi phiên bản Node.js tại [nodejsdp.link/node-green](https://nodejs.org/en/about/node-green).

The module system — Hệ thống module

From its inception, Node.js shipped with a **module system**, even when JavaScript still had no official support for any form of it. The original Node.js module system is called **CommonJS** and it uses the `require` keyword to import functions, variables, and classes exported by built-in modules or other modules located on the device's filesystem.

Ngay từ buổi đầu, Node.js đã đi kèm một hệ thống module, ngay cả khi JavaScript còn chưa có hỗ trợ chính thức cho bất kỳ dạng nào của nó. Hệ thống module ban đầu của Node.js được gọi là CommonJS và nó dùng từ khóa `require` để import các hàm, biến và lớp được export bởi các module dựng sẵn hoặc các module khác nằm trên hệ thống tệp của thiết bị.

CommonJS was a revolution for the JavaScript world in general, as it started to get popular even in the client-side world, where it is used in combination with a module **bundler** (such as Webpack or Rollup) to produce code bundles that are easily executable by the browser. CommonJS was a necessary component for Node.js to allow developers to create large and better organized applications on a par with other server-side platforms.

CommonJS là một cuộc cách mạng cho thế giới JavaScript nói chung, vì nó bắt đầu trở nên phổ biến ngay cả trong thế giới phía client, nơi nó được dùng kết hợp với một module bundler (như Webpack hay Rollup) để tạo ra các gói mã (code bundle) mà trình duyệt có thể dễ dàng thực thi. CommonJS là một thành phần cần thiết để Node.js cho phép lập trình viên tạo ra những ứng dụng lớn và được tổ chức tốt hơn, ngang tầm với các nền tảng phía máy chủ khác.

Today, JavaScript has the so-called ES modules syntax (the `import` keyword may be more familiar) from which Node.js inherits just the syntax, as the underlying implementation is somewhat different from that of the browser. In fact, while the browser mainly deals with remote modules, Node.js, at least for now, can only deal with modules located on the local filesystem.

Ngày nay, JavaScript có cú pháp gọi là ES modules (từ khóa `import` có lẽ quen thuộc hơn), từ đó Node.js chỉ kế thừa cú pháp, vì phần hiện thực bên dưới phần nào khác với của trình

duyet. Thực tế, trong khi trình duyệt chủ yếu xử lý các module từ xa, thì Node.js, ít nhất là cho đến hiện tại, chỉ có thể xử lý các module nằm trên hệ thống tệp cục bộ.

We'll talk about modules in more detail in the next chapter.

Chúng ta sẽ nói về module chi tiết hơn ở chương tiếp theo.

Full access to operating system services — Toàn quyền truy cập các dịch vụ của hệ điều hành

As we already mentioned, even if Node.js uses JavaScript, it doesn't run inside the boundaries of a browser. This allows Node.js to have bindings for all the major services offered by the underlying operating system. For example, we can access any file on the filesystem (subject to any operating system-level permission) thanks to the `fs` module, or we can write applications that use low-level TCP or UDP sockets thanks to the `net` and `dgram` modules. We can create HTTP(S) servers (with the `http` and `https` modules) or use the standard encryption and hashing algorithms of OpenSSL (with the `crypto` module). We can also access some of the V8 internals (the `v8` module) or run code in a different V8 context (with the `vm` module).

Như đã đề cập, dù Node.js dùng JavaScript, nó không chạy trong ranh giới của một trình duyệt. Điều này cho phép Node.js có các binding cho tất cả các dịch vụ lớn do hệ điều hành bên dưới cung cấp. Ví dụ, ta có thể truy cập bất kỳ tệp nào trên hệ thống tệp (tùy thuộc vào quyền ở cấp hệ điều hành) nhờ module `fs`, hoặc ta có thể viết các ứng dụng dùng socket TCP hoặc UDP cấp thấp nhờ các module `net` và `dgram`. Ta có thể tạo các máy chủ HTTP(S) (với các module `http` và `https`) hoặc dùng các thuật toán mã hóa và băm chuẩn của OpenSSL (với module `crypto`). Ta cũng có thể truy cập một số phần nội bộ của V8 (module `v8`) hoặc chạy mã trong một ngữ cảnh V8 khác (với module `vm`).

We can also run other processes (with the `child_process` module) or retrieve our own application's process information using the `process` global variable. In particular, from the `process` global variable, we can get a list of the environment variables assigned to the process (with `process.env`) or the command-line arguments passed to the application at the moment of its launch (with `process.argv`).

Ta cũng có thể chạy các tiến trình khác (với module `child_process`) hoặc lấy thông tin tiến trình của chính ứng dụng mình bằng biến toàn cục `process`. Đặc biệt, từ biến toàn cục `process`, ta có thể lấy danh sách các biến môi trường được gán cho tiến trình (với `process.env`) hoặc các đối số dòng lệnh được truyền cho ứng dụng tại thời điểm khởi chạy (với `process.argv`).

Throughout the book, you'll have the opportunity to use many of the modules described here, but for a complete reference, you can check the official Node.js documentation at [nodejsdp.link/node-docs](https://nodejs.org/docs).

Xuyên suốt cuốn sách, bạn sẽ có cơ hội dùng nhiều module được mô tả ở đây, nhưng để có tài liệu tham khảo đầy đủ, bạn có thể xem tài liệu chính thức của Node.js tại [nodejsdp.link/node-docs](https://nodejs.org/docs).

Running native code — Chạy mã gốc (native code)

One of the most powerful capabilities offered by Node.js is certainly the possibility to create userland modules that can bind to native code. This gives to the platform a tremendous advantage as it allows us to reuse existing or new components written in C/C++. Node.js officially provides great support for implementing native modules thanks to the N-API interface.

Một trong những khả năng mạnh mẽ nhất mà Node.js cung cấp chắc chắn là khả năng tạo các module userland có thể gắn kết (bind) với mã gốc (native code). Điều này mang lại cho nền tảng một lợi thế to lớn vì nó cho phép chúng ta tái sử dụng các thành phần hiện có hoặc mới được viết bằng C/C++. Node.js chính thức cung cấp sự hỗ trợ tuyệt vời cho việc hiện thực các module gốc nhờ giao diện N-API.

But what's the advantage? First of all, it allows us to reuse with little effort a vast amount of existing open source libraries, and most importantly, it allows a company to reuse its own C/C++ legacy code without the need to migrate it.

Nhưng lợi thế là gì? Trước hết, nó cho phép chúng ta tái sử dụng với chút ít công sức một lượng khổng lồ các thư viện mã nguồn mở hiện có, và quan trọng nhất, nó cho phép một công ty tái sử dụng mã C/C++ kế thừa của chính mình mà không cần phải di chuyển (migrate) nó.

Another important consideration is that native code is still necessary to access low-level features such as communicating with hardware drivers or with hardware ports (for example, USB or serial). In fact, thanks to its ability to link to native code, Node.js has become popular in the world of the Internet of things (IoT) and homemade robotics.

Một điều quan trọng khác cần xét là mã gốc vẫn còn cần thiết để truy cập các tính năng cấp thấp như giao tiếp với trình điều khiển phần cứng hoặc với các cổng phần cứng (ví dụ, USB hay cổng nối tiếp). Thực tế, nhờ khả năng liên kết với mã gốc, Node.js đã trở nên phổ biến trong thế giới Internet vạn vật (Internet of Things, IoT) và chế tạo robot tại gia.

Finally, even though V8 is very (very) fast at executing JavaScript, it still has a performance penalty to pay compared to executing native code. In everyday computing, this is rarely an issue, but for CPU-intensive applications, such as those with a lot of data processing and manipulation, delegating the work to native code can make tons of sense. We should also mention that, nowadays, most JavaScript virtual machines (VMs) (and also Node.js) support WebAssembly (Wasm), a low-level instruction format that allows us to compile languages other than JavaScript (such as C++ or Rust) into a format that is “understandable” by JavaScript VMs. This brings many of the advantages we have mentioned, without the need to directly interface with native code.

Cuối cùng, dù V8 thực thi JavaScript rất (rất) nhanh, nó vẫn phải chịu một hình phạt về hiệu năng so với việc thực thi mã gốc. Trong tính toán hàng ngày, điều này hiếm khi là vấn đề, nhưng với các ứng dụng tốn nhiều CPU (CPU-intensive), chẳng hạn những ứng dụng có nhiều xử lý và thao tác dữ liệu, việc giao phó công việc cho mã gốc có thể rất hợp lý. Ta cũng nên đề cập rằng, ngày nay, hầu hết các máy ảo JavaScript (VM) (và cả Node.js) đều hỗ trợ WebAssembly (Wasm), một định dạng lệnh cấp thấp cho phép chúng ta biên dịch các ngôn ngữ khác ngoài JavaScript (như C++ hay Rust) thành một định dạng “có thể hiểu được” đối với các VM JavaScript. Điều này mang lại nhiều lợi thế mà chúng ta đã đề cập, mà không cần phải trực tiếp giao tiếp với mã gốc.

You can learn more about Wasm on the official website of the project at nodejs.org/en/docs/guides/webassembly/.

Bạn có thể tìm hiểu thêm về Wasm trên trang web chính thức của dự án tại nodejs.org/en/docs/guides/webassembly/.

Summary — Tóm tắt

In this chapter, you have seen how the Node.js platform is built upon a few important principles that shape both its internal architecture and the code we write. You have learned that Node.js has a minimal core, and that embracing the “Node way” means writing modules that are smaller, simpler, and that expose only the minimum functionality necessary.

Trong chương này, bạn đã thấy nền tảng Node.js được xây dựng dựa trên một vài nguyên tắc quan trọng định hình cả kiến trúc nội bộ của nó lẫn mã mà chúng ta viết. Bạn đã học được rằng Node.js có một phần lõi tối giản, và rằng đón nhận “phong cách Node” (the “Node way”) nghĩa là viết những module nhỏ hơn, đơn giản hơn, và chỉ phơi bày tối thiểu chức năng cần thiết.

Next, you discovered the reactor pattern, which is the pulsating heart of Node.js, and dissected the internal architecture of the platform runtime to reveal its three pillars: V8, libuv, and the core JavaScript library.

Tiếp theo, bạn đã khám phá reactor pattern, vốn là trái tim đập của Node.js, và mổ xẻ kiến trúc nội bộ của runtime nền tảng để hé lộ ba trụ cột của nó: V8, libuv, và thư viện JavaScript lõi.

Finally, we analyzed some of the main characteristics of using JavaScript in Node.js compared to the browser.

Cuối cùng, chúng ta đã phân tích một số đặc tính chính của việc dùng JavaScript trong Node.js so với trong trình duyệt.

Besides the obvious technical advantages enabled by its internal architecture, Node.js is attracting so much interest because of the principles you have just discovered and the community orbiting around it. For many, grasping the essence of this world feels like returning to the origins, to a more humane way of programming in both size and complexity, and that’s why developers end up falling in love with Node.js.

Bên cạnh những lợi thế kỹ thuật hiển nhiên mà kiến trúc nội bộ của nó mang lại, Node.js thu hút nhiều sự quan tâm đến vậy là nhờ những nguyên tắc bạn vừa khám phá và cộng đồng xoay quanh nó. Với nhiều người, nắm bắt được tinh thần của thế giới này giống như trở về với cội nguồn, với một cách lập trình nhân văn hơn cả về kích thước lẫn độ phức tạp, và đó là lý do các lập trình viên rất cuộc đem lòng yêu mến Node.js.

In the next chapter, we will go deep into one of the most fundamental and important topics of Node.js, its module system.

Ở chương tiếp theo, chúng ta sẽ đi sâu vào một trong những chủ đề nền tảng và quan trọng nhất của Node.js, hệ thống module của nó.

Chapter 2: The Module System —

Chương 2: Hệ thống Module

In Chapter 1, The Node.js Platform, we briefly introduced the importance of modules in Node.js. We discussed how modules play a fundamental role in defining some of the pillars of the Node.js philosophy and its programming experience. But what do we actually mean when we talk about modules and why are they so important?

Trong Chương 1, Nền tảng Node.js, chúng ta đã giới thiệu sơ qua về tầm quan trọng của module trong Node.js. Chúng ta đã bàn về việc module đóng vai trò nền tảng trong việc định hình một số trụ cột của triết lý Node.js và trải nghiệm lập trình của nó. Nhưng khi nói về module thì thực ra chúng ta đang muốn nói đến điều gì, và tại sao chúng lại quan trọng đến vậy?

In generic terms, modules are the bricks for structuring non-trivial applications. Modules allow you to divide the codebase into small units that can be developed and tested independently. Modules are also the main mechanism to enforce information hiding by keeping private all the functions and variables that are not explicitly marked to be exported.

Nói một cách tổng quát, module là những viên gạch để cấu trúc nên các ứng dụng phi tầm thường. Module cho phép bạn chia codebase thành các đơn vị nhỏ có thể được phát triển và kiểm thử một cách độc lập. Module cũng là cơ chế chính để thực thi việc che giấu thông tin (information hiding) bằng cách giữ riêng tư tất cả các hàm và biến không được đánh dấu xuất ra một cách tường minh.

If you come from other languages, you have probably seen similar concepts being referred to with different names: **package** (Java, Go, PHP, Rust, or Dart), assembly (.NET), library (Ruby), or unit (Pascal dialects). The terminology is not perfectly interchangeable because every language or ecosystem comes with its own unique characteristics, but there is a significant overlap between these concepts.

Nếu bạn đến từ các ngôn ngữ khác, có lẽ bạn đã từng thấy những khái niệm tương tự được gọi bằng những cái tên khác nhau: package (Java, Go, PHP, Rust hay Dart), assembly (.NET), library (Ruby) hay unit (các phương ngữ Pascal). Thuật ngữ không hoàn toàn thay thế cho nhau được vì mỗi ngôn ngữ hay hệ sinh thái đều có những đặc trưng riêng, nhưng có sự chồng lấp đáng kể giữa các khái niệm này.

Interestingly enough, Node.js currently comes with two different **module** systems: **CommonJS** (CJS) and ECMAScript modules (**ESM** or ES modules). In this chapter, we will discuss why there are two alternatives, we will learn about their pros and cons, and, finally, we will analyze several common patterns that are relevant when using or writing Node.js modules. By the end of this chapter, you should be able to make pragmatic choices about how to use modules effectively and how to write

your own custom modules.

Điều thú vị là Node.js hiện tại đi kèm với hai hệ thống module khác nhau: CommonJS (CJS) và ECMAScript modules (ESM hay ES modules). Trong chương này, chúng ta sẽ bàn về lý do tại sao lại có hai lựa chọn, chúng ta sẽ tìm hiểu ưu và nhược điểm của chúng, và cuối cùng, chúng ta sẽ phân tích một số mẫu phổ biến có liên quan khi sử dụng hoặc viết module Node.js. Đến cuối chương này, bạn sẽ có thể đưa ra những lựa chọn thực dụng về cách sử dụng module một cách hiệu quả và cách viết các module tùy chỉnh của riêng mình.

Getting a good grasp of Node.js' module systems and module patterns is very important as we will rely on this knowledge in all the other chapters of this book. In short, these are the main topics we will be discussing throughout this chapter:

Nắm vững các hệ thống module và các mẫu module của Node.js là điều rất quan trọng vì chúng ta sẽ dựa vào kiến thức này trong tất cả các chương khác của cuốn sách. Tóm lại, đây là những chủ đề chính mà chúng ta sẽ bàn xuyên suốt chương này:

- Why modules are necessary and the different module systems available in Node.js
- CommonJS internals and module patterns
- ES modules (ESM) in Node.js
- Differences and interoperability between CommonJS and ESM
 - Tại sao module là cần thiết và các hệ thống module khác nhau có sẵn trong Node.js
 - Cơ chế nội bộ của CommonJS và các mẫu module
 - ES modules (ESM) trong Node.js
 - Sự khác biệt và khả năng tương tác giữa CommonJS và ESM

Let's begin with why we need modules.

Hãy bắt đầu với lý do tại sao chúng ta cần module.

The need for modules — Nhu cầu về module

A good **module system** should help with addressing some fundamental needs of software engineering:

Một hệ thống module tốt nên giúp giải quyết một số nhu cầu cơ bản của kỹ thuật phần mềm:

- Having a way to split the codebase into multiple files. This helps with keeping the code more organized, making it easier to understand but also helps with developing and testing various pieces of functionality independently from each other.
- Allowing code reuse across different projects. A module can, in fact, implement a generic feature that can be useful for different projects. Organizing such functionality within a module can make it easier to bring it into the different projects that may want to use it.
- **Encapsulation** (or information hiding). It is generally a good idea to hide implementation complexity and only expose simple interfaces with clear responsibilities. Most module systems allow to selectively keep the private part of the code hidden, while exposing a public interface, such as functions, classes, or objects that are meant to be used by the consumers of the module.
- Managing dependencies. A good module system should make it easy for module developers to build on top of existing modules, including third-party ones. A module system should also make it easy for module users to import the chain of dependencies that are necessary for a given module to run (transient dependencies).

- Có một cách để chia codebase thành nhiều file. Điều này giúp giữ cho code có tổ chức hơn, dễ hiểu hơn nhưng cũng giúp phát triển và kiểm thử các phần chức năng khác nhau một cách độc lập với nhau.
- Cho phép tái sử dụng code giữa các dự án khác nhau. Thực tế, một module có thể hiện thực một tính năng tổng quát hữu ích cho nhiều dự án khác nhau. Tổ chức chức năng như vậy bên trong một module có thể giúp đưa nó vào các dự án khác nhau muốn sử dụng nó dễ dàng hơn.
- Đóng gói (encapsulation) (hay che giấu thông tin). Nhìn chung, ẩn đi sự phức tạp của phần hiện thực và chỉ phơi bày các interface đơn giản với trách nhiệm rõ ràng là một ý tưởng tốt. Hầu hết các hệ thống module cho phép giữ ẩn phần riêng tư của code một cách có chọn lọc, đồng thời phơi bày một interface công khai, chẳng hạn như các hàm, lớp hay đối tượng dành cho những bên tiêu thụ module sử dụng.
- Quản lý các phụ thuộc (dependency). Một hệ thống module tốt nên giúp các nhà phát triển module dễ dàng xây dựng dựa trên các module có sẵn, bao gồm cả các module của bên thứ ba. Một hệ thống module cũng nên giúp người dùng module dễ dàng import chuỗi các phụ thuộc cần thiết để một module nhất định có thể chạy (các phụ thuộc bắc cầu - transient dependencies).

It is important to clarify the distinction between a module and a module system. We can define a module as the actual unit of software, while a module system is the syntax and the tooling that allows us to define modules and to use them within our projects.

Điều quan trọng là phải làm rõ sự phân biệt giữa module và hệ thống module. Chúng ta có thể định nghĩa module là đơn vị phần mềm thực tế, trong khi hệ thống module là cú pháp và bộ công cụ cho phép chúng ta định nghĩa các module và sử dụng chúng trong các dự án của mình.

Module systems in JavaScript and Node.js — Hệ thống module trong JavaScript và Node.js

Not all programming languages come with a built-in module system, and JavaScript had been lacking this feature for a long time.

Không phải mọi ngôn ngữ lập trình đều đi kèm với một hệ thống module dựng sẵn, và JavaScript đã thiếu tính năng này trong một thời gian dài.

In the browser landscape, it is possible to split the codebase into multiple files and then import them by using different `<script>` tags. For many years, this approach was good enough to build simple interactive websites, and JavaScript developers managed to get things done without having a fully-fledged module system.

Trong bối cảnh trình duyệt, có thể chia codebase thành nhiều file rồi import chúng bằng cách dùng nhiều thẻ `<script>` khác nhau. Trong nhiều năm, cách tiếp cận này đủ tốt để xây dựng các website tương tác đơn giản, và các nhà phát triển JavaScript đã xoay xở hoàn thành công việc mà không cần đến một hệ thống module đầy đủ.

Only when JavaScript browser applications became more complicated and frameworks like jQuery, Backbone, and AngularJS took over the ecosystem, the JavaScript community came up with several initiatives aimed at defining a module system that could be effectively adopted within JavaScript projects. The most successful ones were asynchronous module definition (AMD), popularized by RequireJS (nodejsdp.link/requirejs), and later Universal Module Definition (UMD – nodejsdp.link/umd).

Chỉ khi các ứng dụng JavaScript trên trình duyệt trở nên phức tạp hơn và các framework như jQuery, Backbone và AngularJS chiếm lĩnh hệ sinh thái, cộng đồng JavaScript mới đưa ra một số sáng kiến nhằm định nghĩa một hệ thống module có thể được áp dụng hiệu quả trong các dự án JavaScript. Những sáng kiến thành công nhất là asynchronous module definition (AMD), được phổ biến bởi RequireJS (nodejsdp.link/requirejs), và sau đó là Universal Module Definition (UMD – nodejsdp.link/umd).

When Node.js was created, it was conceived as a server **runtime** for JavaScript with direct access to the underlying filesystem so there was a unique opportunity to introduce a different way to manage modules. The idea was not to rely on HTML `<script>` tags and resources accessible through URLs. Instead, the choice was to rely purely on JavaScript files available on the local filesystem. For its module system, Node.js came up with an implementation of the CommonJS specification (sometimes also referred to as CJS, nodejsdp.link/commonjs), which was designed to provide a module system for JavaScript in browserless environments.

Khi Node.js được tạo ra, nó được hình dung như một runtime phía máy chủ cho JavaScript với quyền truy cập trực tiếp vào hệ thống tập tin bên dưới, do đó có một cơ hội độc đáo để giới thiệu một cách quản lý module khác. Ý tưởng là không dựa vào các thẻ `<script>` của HTML và các tài nguyên có thể truy cập thông qua URL. Thay vào đó, lựa chọn là dựa hoàn toàn vào các file JavaScript có sẵn trên hệ thống tập tin cục bộ. Đối với hệ thống module của mình, Node.js đã đưa ra một hiện thực của đặc tả CommonJS (đôi khi cũng được gọi là CJS, nodejsdp.link/commonjs), vốn được thiết kế để cung cấp một hệ thống module cho JavaScript trong các môi trường không có trình duyệt.

CommonJS has been the dominant module system in Node.js since its inception and it has become very prominent also in the browser landscape thanks to module bundlers like Browserify (nodejsdp.link/browserify) and webpack (nodejsdp.link/webpack).

CommonJS đã là hệ thống module thống trị trong Node.js kể từ khi nó ra đời và cũng đã trở nên rất nổi bật trong bối cảnh trình duyệt nhờ các module bundler như Browserify (nodejsdp.link/browserify) và webpack (nodejsdp.link/webpack).

In 2015, with the release of ECMAScript 6 (also called ECMAScript 2015 or ES2015), there was finally an official proposal for a standard module system: ESM or ECMAScript modules. ESM brings a lot of innovation in the JavaScript ecosystem and, among other things, it tries to bridge the gap between how modules are managed on browsers and servers.

Năm 2015, với sự ra mắt của ECMAScript 6 (còn gọi là ECMAScript 2015 hay ES2015), cuối cùng đã có một đề xuất chính thức cho một hệ thống module chuẩn: ESM hay ECMAScript modules. ESM mang lại rất nhiều đổi mới trong hệ sinh thái JavaScript và, cùng với những điều khác, nó cố gắng thu hẹp khoảng cách giữa cách module được quản lý trên trình duyệt và máy chủ.

ECMAScript 6 defined only the formal specification for ESM in terms of syntax and semantics, but it didn't provide any implementation details. It took different browser companies and the Node.js community several years to come up with solid implementations of the specification. Node.js ships with stable support for ESM starting from version 13.2. At the time of writing, the general feeling is that ESM is going to become the de facto way to manage JavaScript modules in both the browser and the server landscape. The reality today, though, is that the majority of projects are still heavily relying on CommonJS and it will take some time for ESM to catch up and eventually become the dominant standard.

ECMAScript 6 chỉ định nghĩa đặc tả chính thức cho ESM về mặt cú pháp và ngữ nghĩa, nhưng nó không cung cấp bất kỳ chi tiết hiện thực nào. Phải mất vài năm để các công ty

trình duyệt khác nhau và cộng đồng Node.js đưa ra được những hiện thực vững chắc cho đặc tả này. Node.js đi kèm hỗ trợ ổn định cho ESM bắt đầu từ phiên bản 13.2. Tại thời điểm viết sách, cảm nhận chung là ESM sẽ trở thành cách thực tế (de facto) để quản lý module JavaScript trong cả bối cảnh trình duyệt lẫn máy chủ. Tuy nhiên, thực tế ngày nay là phần lớn các dự án vẫn phụ thuộc nặng nề vào CommonJS và sẽ mất một thời gian để ESM bắt kịp và cuối cùng trở thành chuẩn thống trị.

To provide a comprehensive overview of module-related patterns in Node.js, in the first part of this chapter, we will discuss them in the context of CommonJS, and then, in the second part of the chapter, we will revisit our learnings using ESM.

Để cung cấp một cái nhìn tổng quan toàn diện về các mẫu liên quan đến module trong Node.js, trong phần đầu của chương này, chúng ta sẽ bàn về chúng trong bối cảnh CommonJS, và sau đó, trong phần thứ hai của chương, chúng ta sẽ xem lại những gì đã học bằng ESM.

The goal of this chapter is to make you comfortable with both module systems, but in the rest of the book, we will only be using ESM for our code examples. The idea is to encourage you to leverage ESM as much as possible so that your code will be more future-proof.

Mục tiêu của chương này là giúp bạn cảm thấy thoải mái với cả hai hệ thống module, nhưng trong phần còn lại của cuốn sách, chúng ta sẽ chỉ sử dụng ESM cho các ví dụ code. Ý tưởng là khuyến khích bạn tận dụng ESM càng nhiều càng tốt để code của bạn được chuẩn bị tốt hơn cho tương lai.

If you are reading this chapter a few years after its publication, you are probably not too worried about CommonJS, and you might want to jump straight into the ESM part. This is probably fine, but we still encourage you to go through the entire chapter, because understanding CommonJS and its characteristics will certainly be beneficial in helping you to understand ESM and its strengths in much more depth.

Nếu bạn đang đọc chương này vài năm sau khi nó được xuất bản, có lẽ bạn không quá lo lắng về CommonJS, và bạn có thể muốn nhảy thẳng vào phần ESM. Điều này có lẽ ổn, nhưng chúng tôi vẫn khuyến khích bạn đọc qua toàn bộ chương, bởi vì việc hiểu CommonJS và các đặc điểm của nó chắc chắn sẽ có ích trong việc giúp bạn hiểu ESM và những điểm mạnh của nó một cách sâu sắc hơn nhiều.

The module system and its patterns — Hệ thống module và các mẫu của nó

As we said, modules are the bricks for structuring non-trivial applications and the main mechanism to enforce information hiding by keeping private all the functions and variables that are not explicitly marked to be exported.

Như đã nói, module là những viên gạch để cấu trúc nên các ứng dụng phi tầm thường và là cơ chế chính để thực thi việc che giấu thông tin bằng cách giữ riêng tư tất cả các hàm và biến không được đánh dấu xuất ra một cách tường minh.

Before getting into the specifics of CommonJS, let's discuss a generic pattern that helps with information hiding and that we will be using for building a simple module system, which is the revealing module pattern.

Trước khi đi vào cụ thể của CommonJS, hãy bàn về một mẫu tổng quát giúp ích cho việc che giấu thông tin và mà chúng ta sẽ dùng để xây dựng một hệ thống module đơn giản, đó

là revealing module pattern.

The revealing module pattern — Revealing module pattern

One of the bigger problems with JavaScript in the browser is the lack of namespacing. Every script runs in the global **scope**; therefore, internal application code or third-party dependencies can pollute the scope while exposing their own pieces of functionality. This can be extremely harmful. Imagine, for instance, that a third-party library instantiates a global variable called `utils`. If any other library, or the application code itself, accidentally overrides or alters `utils`, the code that relies on it will likely crash in some unpredictable way. Unpredictable side effects can also happen if other libraries or the application code accidentally invoke a function of another library meant for internal use only. In short, relying on the global scope is a very risky business, especially as your application grows and you have to rely more and more on functionality implemented by other individuals.

Một trong những vấn đề lớn hơn với JavaScript trong trình duyệt là thiếu cơ chế không gian tên (namespacing). Mọi script đều chạy trong phạm vi toàn cục; do đó, code nội bộ của ứng dụng hoặc các phụ thuộc của bên thứ ba có thể làm ô nhiễm phạm vi này trong khi phơi bày các phần chức năng của riêng chúng. Điều này có thể cực kỳ tai hại. Ví dụ, hãy tưởng tượng rằng một thư viện của bên thứ ba khởi tạo một biến toàn cục tên là `utils`. Nếu bất kỳ thư viện nào khác, hoặc chính code của ứng dụng, vô tình ghi đè hoặc thay đổi `utils`, thì code dựa vào nó có thể sẽ sập theo một cách khó lường nào đó. Các tác dụng phụ khó lường cũng có thể xảy ra nếu các thư viện khác hoặc code của ứng dụng vô tình gọi một hàm của một thư viện khác vốn chỉ dành cho mục đích sử dụng nội bộ. Tóm lại, dựa vào phạm vi toàn cục là một việc rất rủi ro, đặc biệt là khi ứng dụng của bạn lớn dần và bạn phải ngày càng dựa vào các chức năng được hiện thực bởi những cá nhân khác.

A popular technique to solve this class of problems is called the revealing module pattern, and it looks like this:

Một kỹ thuật phổ biến để giải quyết lớp vấn đề này được gọi là revealing module pattern, và nó trông như thế này:

```
const myModule = (() => {
  const privateFoo = () => {}
  const privateBar = []
  const exported = {
    publicFoo: () => {},
    publicBar: () => {}
  }
  return exported
})(); // once the parenthesis here are parsed, the function
      // will be invoked
console.log(myModule)
console.log(myModule.privateFoo, myModule.privateBar)
```

This pattern leverages a self-invoking function. This type of function is sometimes also referred to as Immediately Invoked Function Expression (IIFE) and it is used to create a private scope, exporting only the parts that are meant to be public.

Mẫu này tận dụng một hàm tự gọi (self-invoking function). Loại hàm này đôi khi cũng được gọi là Immediately Invoked Function Expression (IIFE) và nó được dùng để tạo ra một phạm vi riêng tư, chỉ xuất ra những phần được dành để công khai.

In JavaScript, variables created inside a function are not accessible from the outer scope (outside the function). Functions can use the `return` statement to selectively propagate information to the outer scope.

Trong JavaScript, các biến được tạo bên trong một hàm thì không thể truy cập được từ phạm vi bên ngoài (bên ngoài hàm). Các hàm có thể dùng câu lệnh `return` để truyền thông tin một cách có chọn lọc ra phạm vi bên ngoài.

This pattern is essentially exploiting these properties to keep the private information hidden and export only a public-facing **API**.

Mẫu này về cơ bản đang khai thác các thuộc tính này để giữ ẩn thông tin riêng tư và chỉ xuất ra một API hướng ra công khai.

In the preceding code, the `myModule` variable contains only the exported API, while the rest of the module content is practically inaccessible from outside.

Trong đoạn code trên, biến `myModule` chỉ chứa API được xuất ra, trong khi phần còn lại của nội dung module trên thực tế không thể truy cập được từ bên ngoài.

The `log` statement is going to print something like this:

Câu lệnh `log` sẽ in ra một thứ gì đó như thế này:

```
{ publicFoo: [Function: publicFoo],  
  publicBar: [Function: publicBar] }  
undefined undefined
```

This demonstrates that only the exported properties are directly accessible from `myModule`.

Điều này chứng minh rằng chỉ các thuộc tính của `exported` mới có thể truy cập trực tiếp từ `myModule`.

As we will see in a moment, the idea behind this pattern is used as a base for the CommonJS module system.

Như chúng ta sẽ thấy ngay sau đây, ý tưởng đằng sau mẫu này được dùng làm nền tảng cho hệ thống module CommonJS.

CommonJS modules — Module CommonJS

CommonJS is the first module system originally built into Node.js. Node.js' CommonJS implementation respects the CommonJS specification, with the addition of some custom extensions.

CommonJS là hệ thống module đầu tiên vốn được dựng sẵn trong Node.js. Hiện thực CommonJS của Node.js tuân theo đặc tả CommonJS, có bổ sung thêm một số phần mở rộng tùy chỉnh.

Let's summarize two of the main concepts of the CommonJS specification:

Hãy tóm tắt hai trong số các khái niệm chính của đặc tả CommonJS:

- `require` is a function that allows you to import a module from the local filesystem
- `exports` and `module.exports` are special variables that can be used to export public functionality from the current module
 - `require` là một hàm cho phép bạn import một module từ hệ thống tập tin cục bộ

- `exports` và `module.exports` là các biến đặc biệt có thể được dùng để xuất các chức năng công khai từ module hiện tại

This information is sufficient for now; we will learn more details and some of the nuances of the CommonJS specification in the next few sections.

Thông tin này hiện tại là đủ; chúng ta sẽ tìm hiểu thêm chi tiết và một số sắc thái của đặc tả CommonJS trong vài phần tiếp theo.

A homemade module loader — Một bộ nạp module tự chế

To understand how CommonJS works in Node.js, let's build a similar system from scratch. The code that follows creates a function that mimics a subset of the functionality of the original `require()` function of Node.js.

Để hiểu cách CommonJS hoạt động trong Node.js, hãy xây dựng một hệ thống tương tự từ đầu. Đoạn code sau đây tạo ra một hàm mô phỏng một tập con chức năng của hàm `require()` gốc của Node.js.

Let's start by creating a function that loads the content of a module, wraps it into a private scope, and evaluates it:

Hãy bắt đầu bằng cách tạo một hàm nạp nội dung của một module, gói nó vào một phạm vi riêng tư, rồi đánh giá (evaluate) nó:

```
function loadModule (filename, module, require) {
  const wrappedSrc =
    `(function (module, exports, require) {
      ${fs.readFileSync(filename, 'utf8')}
    })(module, module.exports, require)`
  eval(wrappedSrc)
}
```

The source code of a module is essentially wrapped into a function, as it was for the revealing module pattern. The difference here is that we pass a list of variables to the module, in particular, `module`, `exports`, and `require`. Make a note of how the `exports` argument of the wrapping function is initialized with the content of `module.exports`, as we will talk about this later. Another important detail to mention is that we are using `readFileSync` to read the module's content. While it is generally not recommended to use the synchronous version of the filesystem APIs, here it makes sense to do so. The reason for that is that loading modules in CommonJS are deliberately synchronous operations. This approach makes sure that, if we are importing multiple modules, they (and their dependencies) are loaded in the right order. We will talk more about this aspect later in the chapter.

Mã nguồn của một module về cơ bản được gói vào trong một hàm, giống như đối với revealing module pattern. Điểm khác biệt ở đây là chúng ta truyền một danh sách các biến vào module, cụ thể là `module`, `exports` và `require`. Hãy lưu ý cách tham số `exports` của hàm bao bọc được khởi tạo bằng nội dung của `module.exports`, vì chúng ta sẽ bàn về điều này sau. Một chi tiết quan trọng khác cần đề cập là chúng ta đang dùng `readFileSync` để đọc nội dung của module. Mặc dù nhìn chung không nên dùng phiên bản đồng bộ của các API hệ thống tập tin, nhưng ở đây làm vậy là hợp lý. Lý do là việc nạp các module trong CommonJS là các thao tác đồng bộ một cách có chủ đích. Cách tiếp cận này đảm bảo rằng, nếu chúng ta đang import nhiều module, thì chúng (và các phụ thuộc của chúng) được nạp theo đúng thứ tự. Chúng ta sẽ bàn thêm về khía cạnh này ở phần sau của chương.

Bear in mind that this is only an example, and you will rarely need to evaluate some source code in a real application. Features such as `eval()` or the functions of the `vm` module ([nodejsdp.link/vm](https://nodejs.org/docs/latest/api/vm)) can be easily used in the wrong way or with the wrong input, thus opening a system to code injection attacks. They should always be used with extreme care or avoided altogether.

Hãy nhớ rằng đây chỉ là một ví dụ, và bạn sẽ hiếm khi cần phải đánh giá một đoạn mã nguồn nào đó trong một ứng dụng thực tế. Các tính năng như `eval()` hay các hàm của module `vm` ([nodejsdp.link/vm](https://nodejs.org/docs/latest/api/vm)) rất dễ bị dùng sai cách hoặc với đầu vào sai, do đó mở ra cho hệ thống các cuộc tấn công tiêm code (code injection). Chúng luôn phải được dùng với sự cẩn trọng cực độ hoặc tránh dùng hoàn toàn.

Let's now implement the `require()` function:

Bây giờ hãy hiện thực hàm `require()`:

```
function require (moduleName) {
  console.log(`Require invoked for module: ${moduleName}`)
  const id = require.resolve(moduleName)           // (1)
  if (require.cache[id]) {                         // (2)
    return require.cache[id].exports
  }
  // module metadata
  const module = {                                // (3)
    exports: {},
    id
  }
  // Update the cache
  require.cache[id] = module                       // (4)
  // load the module
  loadModule(id, module, require)                  // (5)
  // return exported variables
  return module.exports                            // (6)
}
require.cache = {}
require.resolve = (moduleName) => {
  /* resolve a full module id from the moduleName */
}
```

The previous function simulates the behavior of the original `require()` function of Node.js, which is used to load a module. Of course, this is just for educational purposes and does not accurately or completely reflect the internal behavior of the real `require()` function, but it's great to understand the internals of the Node.js module system, including how a module is defined and loaded.

Hàm trước đó mô phỏng hành vi của hàm `require()` gốc của Node.js, vốn được dùng để nạp một module. Tất nhiên, điều này chỉ nhằm mục đích giáo dục và không phản ánh một cách chính xác hay đầy đủ hành vi nội bộ của hàm `require()` thực sự, nhưng nó rất hữu ích để hiểu cơ chế nội bộ của hệ thống module Node.js, bao gồm cách một module được định nghĩa và nạp.

What our homemade module system does is explained as follows:

Những gì hệ thống module tự chế của chúng ta làm được giải thích như sau:

1. A module name is accepted as input, and the very first thing that we do is resolve the full path of the module, which we call `id`. This task is delegated to `require.resolve()`, which implements

a specific resolving algorithm (we will talk about it later).

2. If the module has already been loaded in the past, it should be available in the cache. If this is the case, we just return it immediately.
3. If the module has never been loaded before, we set up the environment for the first load. In particular, we create a module object that contains an `exports` property initialized with an empty object literal. This object will be populated by the code of the module to export its public API.
4. After the first load, the module object is cached.
5. The module source code is read from its file and the code is evaluated, as we saw before. We provide the module with the module object that we just created, and a reference to the `require()` function. The module exports its public API by manipulating or replacing the `module.exports` object.
6. Finally, the content of `module.exports`, which represents the public API of the module, is returned to the caller.

1. Một tên module được nhận làm đầu vào, và điều đầu tiên chúng ta làm là phân giải (resolve) đường dẫn đầy đủ của module, mà chúng ta gọi là `id`. Nhiệm vụ này được ủy thác cho `require.resolve()`, vốn hiện thực một thuật toán phân giải cụ thể (chúng ta sẽ bàn về nó sau).
2. Nếu module đã được nạp trước đó, nó sẽ có sẵn trong cache. Nếu đúng như vậy, chúng ta chỉ cần trả về nó ngay lập tức.
3. Nếu module chưa bao giờ được nạp trước đó, chúng ta thiết lập môi trường cho lần nạp đầu tiên. Cụ thể, chúng ta tạo một đối tượng module chứa một thuộc tính `exports` được khởi tạo bằng một object literal rỗng. Đối tượng này sẽ được code của module điền vào để xuất ra API công khai của nó.
4. Sau lần nạp đầu tiên, đối tượng module được lưu vào cache.
5. Mã nguồn của module được đọc từ file của nó và code được đánh giá, như chúng ta đã thấy trước đó. Chúng ta cung cấp cho module đối tượng module mà chúng ta vừa tạo, và một tham chiếu đến hàm `require()`. Module xuất ra API công khai của nó bằng cách thao tác hoặc thay thế đối tượng `module.exports`.
6. Cuối cùng, nội dung của `module.exports`, vốn đại diện cho API công khai của module, được trả về cho bên gọi.

As we can see, there is nothing magical behind the workings of the Node.js module system. The trick is all in the wrapper we create around a module's source code and the artificial environment in which we run it.

Như chúng ta có thể thấy, không có gì kỳ diệu đằng sau cách hoạt động của hệ thống module Node.js. Mẹo nằm hết ở lớp bao bọc mà chúng ta tạo quanh mã nguồn của module và môi trường nhân tạo mà chúng ta chạy nó trong đó.

Defining a module — Định nghĩa một module

By looking at how our custom `require()` function works, we should now be able to understand how to define a module. The following code gives us an example:

Bằng cách xem cách hàm `require()` tùy chỉnh của chúng ta hoạt động, giờ đây chúng ta sẽ có thể hiểu cách định nghĩa một module. Đoạn code sau đây cho chúng ta một ví dụ:

```
// load another dependency
const dependency = require('./anotherModule')
// a private function
function log() {
```

```

    console.log(`Well done ${dependency.username}`)
  }
  // the API to be exported for public use
  module.exports.run = () => {
    log()
  }

```

The essential concept to remember is that everything inside a module is private unless it's assigned to the `module.exports` variable. The content of this variable is then cached and returned when the module is loaded using `require()`.

Khái niệm cốt yếu cần nhớ là mọi thứ bên trong một module đều là riêng tư trừ khi nó được gán cho biến `module.exports`. Nội dung của biến này sau đó được lưu vào cache và trả về khi module được nạp bằng `require()`.

module.exports versus exports — module.exports so với exports

For many developers who are not yet familiar with Node.js, a common source of confusion is the difference between using `exports` and `module.exports` to expose a public API. The code of our custom `require()` function should again clear any doubt. The `exports` variable is just a reference to the initial value of `module.exports`. We have seen that such a value is essentially a simple object literal created before the module is loaded.

Đối với nhiều nhà phát triển chưa quen với Node.js, một nguồn gây nhầm lẫn phổ biến là sự khác biệt giữa việc dùng `exports` và `module.exports` để phơi bày một API công khai. Code của hàm `require()` tùy chỉnh của chúng ta một lần nữa sẽ xua tan mọi nghi ngờ. Biến `exports` chỉ là một tham chiếu đến giá trị ban đầu của `module.exports`. Chúng ta đã thấy rằng giá trị như vậy về cơ bản là một object literal đơn giản được tạo trước khi module được nạp.

This means that we can only attach new properties to the object referenced by the `exports` variable, as shown in the following code:

Điều này có nghĩa là chúng ta chỉ có thể gắn thêm các thuộc tính mới vào đối tượng được tham chiếu bởi biến `exports`, như được minh họa trong đoạn code sau:

```

exports.hello = () => {
  console.log('Hello')
}

```

Reassigning the `exports` variable doesn't have any effect, because it doesn't change the content of `module.exports`. It will only reassign the variable itself. The following code is therefore wrong:

Việc gán lại biến `exports` không có tác dụng gì, bởi vì nó không làm thay đổi nội dung của `module.exports`. Nó chỉ gán lại chính biến đó. Do đó, đoạn code sau là sai:

```

exports = () => {
  console.log('Hello')
}

```

If we want to export something other than an object literal, such as a function, an instance, or even a string, we have to reassign `module.exports` as follows:

Nếu chúng ta muốn xuất ra một thứ gì đó khác ngoài một object literal, chẳng hạn như một hàm, một instance, hay thậm chí một chuỗi, chúng ta phải gán lại `module.exports` như sau:


```
module.exports = () => {  
  console.log('Hello')  
}
```

The require function is synchronous — Hàm require là đồng bộ

A very important detail that we should take into account is that our homemade `require()` function is synchronous. In fact, it returns the module contents using a simple direct style, and no **callback** is required. This is true for the original Node.js `require()` function too. As a consequence, any assignment to `module.exports` must be synchronous as well. For example, the following code is incorrect and it will cause trouble:

Một chi tiết rất quan trọng mà chúng ta nên lưu ý là hàm `require()` tự chế của chúng ta là đồng bộ. Thực tế, nó trả về nội dung của module bằng một phong cách trực tiếp đơn giản, và không cần callback nào cả. Điều này cũng đúng với hàm `require()` gốc của Node.js. Hệ quả là, bất kỳ phép gán nào cho `module.exports` cũng phải đồng bộ. Ví dụ, đoạn code sau là không đúng và nó sẽ gây ra rắc rối:

```
setTimeout(() => {  
  module.exports = function() {...}  
}, 100)
```

The synchronous nature of `require()` has important repercussions on the way we define modules, as it limits us to mostly using synchronous code during the definition of a module. This is one of the most important reasons why the core Node.js libraries offer synchronous APIs as an alternative to most of the asynchronous ones.

Bản chất đồng bộ của `require()` có những hệ quả quan trọng đối với cách chúng ta định nghĩa module, vì nó giới hạn chúng ta chủ yếu phải dùng code đồng bộ trong quá trình định nghĩa một module. Đây là một trong những lý do quan trọng nhất khiến các thư viện lõi của Node.js cung cấp các API đồng bộ như một lựa chọn thay thế cho hầu hết các API bất đồng bộ.

If we need some asynchronous initialization steps for a module, we can always define and export an uninitialized module that is initialized asynchronously at a later time. The problem with this approach, though, is that loading such a module using `require()` does not guarantee that it's ready to be used. In Chapter 11, Advanced Recipes, we will analyze this problem in detail and present some patterns to solve this issue elegantly.

Nếu chúng ta cần một số bước khởi tạo bất đồng bộ cho một module, chúng ta luôn có thể định nghĩa và xuất ra một module chưa được khởi tạo, sau đó được khởi tạo một cách bất đồng bộ vào lúc muộn hơn. Tuy nhiên, vấn đề với cách tiếp cận này là việc nạp một module như vậy bằng `require()` không đảm bảo rằng nó đã sẵn sàng để sử dụng. Trong Chương 11, Các công thức nâng cao, chúng ta sẽ phân tích vấn đề này một cách chi tiết và trình bày một số mẫu để giải quyết vấn đề này một cách thanh lịch.

For the sake of curiosity, you might want to know that in its early days, Node.js used to have an asynchronous version of `require()`, but it was soon removed because it was overcomplicating a functionality that was actually only meant to be used at initialization time and where asynchronous I/O brings more complexities than advantages.

Vì sự tò mò, có thể bạn muốn biết rằng trong những ngày đầu, Node.js từng có một phiên bản bất đồng bộ của `require()`, nhưng nó sớm bị loại bỏ vì nó làm phức tạp hóa quá mức

một chức năng vốn thực ra chỉ nhằm được dùng vào lúc khởi tạo và là nơi mà I/O bất đồng bộ mang lại nhiều phức tạp hơn là lợi ích.

The resolving algorithm — Thuật toán phân giải (resolving)

The term **dependency hell** describes a situation whereby two or more dependencies of a program in turn depend on a shared dependency, but require different incompatible versions. Node.js solves this problem elegantly by loading a different version of a module depending on where the module is loaded from. All the merits of this feature go to the way Node.js package managers (such as npm or yarn) organize the dependencies of the application, and also to the resolving algorithm used in the `require()` function. Let's now give a quick overview of this algorithm. As we saw, the `resolve()` function takes a module name (which we will call `moduleName`) as input and it returns the full path of the module. This path is then used to load its code and also to identify the module uniquely. The resolving algorithm can be divided into the following three major branches:

Thuật ngữ dependency hell mô tả tình huống mà trong đó hai hay nhiều phụ thuộc của một chương trình lại phụ thuộc vào một phụ thuộc chung, nhưng đòi hỏi các phiên bản khác nhau và không tương thích với nhau. Node.js giải quyết vấn đề này một cách thanh lịch bằng cách nạp một phiên bản module khác nhau tùy thuộc vào nơi module được nạp. Mọi công lao của tính năng này thuộc về cách mà các trình quản lý gói của Node.js (như npm hay yarn) tổ chức các phụ thuộc của ứng dụng, và cũng thuộc về thuật toán phân giải được dùng trong hàm `require()`. Bây giờ hãy điểm qua nhanh về thuật toán này. Như chúng ta đã thấy, hàm `resolve()` nhận một tên module (mà chúng ta sẽ gọi là `moduleName`) làm đầu vào và nó trả về đường dẫn đầy đủ của module. Đường dẫn này sau đó được dùng để nạp code của nó và cũng để định danh module một cách duy nhất. Thuật toán phân giải có thể được chia thành ba nhánh chính sau đây:

- File modules: If `moduleName` starts with `/`, it is already considered an absolute path to the module and it's returned as it is. If it starts with `./`, then `moduleName` is considered a relative path, which is calculated starting from the directory of the requiring module.
- Core modules: If `moduleName` is not prefixed with `/` or `./`, the algorithm will first try to search within the core Node.js modules.
- Package modules: If no core module is found matching `moduleName`, then the search continues by looking for a matching module in the first `node_modules` directory that is found navigating up in the directory structure starting from the requiring module. The algorithm continues to search for a match by looking into the next `node_modules` directory up in the directory tree, until it reaches the root of the filesystem.
 - File modules: Nếu `moduleName` bắt đầu bằng `/`, nó đã được coi là một đường dẫn tuyệt đối đến module và được trả về nguyên trạng. Nếu nó bắt đầu bằng `./`, thì `moduleName` được coi là một đường dẫn tương đối, được tính bắt đầu từ thư mục của module đang yêu cầu.
 - Core modules: Nếu `moduleName` không có tiền tố `/` hay `./`, thuật toán trước tiên sẽ thử tìm kiếm trong các module lõi của Node.js.
 - Package modules: Nếu không tìm thấy module lõi nào khớp với `moduleName`, thì việc tìm kiếm tiếp tục bằng cách tìm một module khớp trong thư mục `node_modules` đầu tiên được tìm thấy khi điều hướng đi lên trong cấu trúc thư mục bắt đầu từ module đang yêu cầu. Thuật toán tiếp tục tìm kiếm một kết quả khớp bằng cách xem xét thư mục `node_modules` tiếp theo đi lên trong cây thư mục, cho đến khi nó đạt đến gốc của hệ thống tập tin.

For file and package modules, both files and directories can match `moduleName`. In particular, the

algorithm will try to match the following:

Đối với file modules và package modules, cả file lẫn thư mục đều có thể khớp với `moduleName`. Cụ thể, thuật toán sẽ thử khớp những điều sau:

- `<moduleName>.js`
- `<moduleName>/index.js`
- The directory/file specified in the main property of `<moduleName>/package.json`
 - `<moduleName>.js`
 - `<moduleName>/index.js`
 - Thư mục/file được chỉ định trong thuộc tính main của `<moduleName>/package.json`

The complete, formal documentation of the resolving algorithm can be found at [nodejsdp.link/resolve](https://nodejs.org/en/docs/guides/modules-resolving).

Tài liệu chính thức, đầy đủ về thuật toán phân giải có thể được tìm thấy tại [nodejsdp.link/resolve](https://nodejs.org/en/docs/guides/modules-resolving).

The `node_modules` directory is actually where the package managers install the dependencies of each package. This means that, based on the algorithm we just described, each package can have its own private dependencies. For example, consider the following directory structure:

Thư mục `node_modules` thực ra là nơi các trình quản lý gói cài đặt các phụ thuộc của mỗi gói. Điều này có nghĩa là, dựa trên thuật toán chúng ta vừa mô tả, mỗi gói có thể có các phụ thuộc riêng tư của riêng nó. Ví dụ, hãy xem xét cấu trúc thư mục sau:

```
myApp
├── foo.js
├── node_modules
│   ├── depA
│   │   └── index.js
│   ├── depB
│   │   ├── bar.js
│   │   └── node_modules
│   │       └── depA
│   │           └── index.js
│   └── depC
│       ├── foobar.js
│       └── node_modules
│           └── depA
│               └── index.js
```

In the previous example, `myApp`, `depB`, and `depC` all depend on `depA`. However, they all have their own private version of the dependency! Following the rules of the resolving algorithm, using `require('depA')` will load a different file depending on the module that requires it, for example:

Trong ví dụ trước, `myApp`, `depB` và `depC` đều phụ thuộc vào `depA`. Tuy nhiên, tất cả chúng đều có phiên bản riêng tư của riêng mình của phụ thuộc đó! Tuân theo các quy tắc của thuật toán phân giải, việc dùng `require('depA')` sẽ nạp một file khác nhau tùy thuộc vào module yêu cầu nó, ví dụ:

- Calling `require('depA')` from `/myApp/foo.js` will load `/myApp/node_modules/depA/index.js`
- Calling `require('depA')` from `/myApp/node_modules/depB/bar.js` will load `/myApp/node_modules/depB/node_modules/depA/index.js`
- Calling `require('depA')` from `/myApp/node_modules/depC/foobar.js` will load `/myApp/node_modules/depC/node_modules/depA/index.js`
 - Gọi `require('depA')` từ `/myApp/foo.js` sẽ nạp `/myApp/node_modules/depA/index.js`
 - Gọi `require('depA')` từ `/myApp/node_modules/depB/bar.js` sẽ nạp `/myApp/node_modules/depB/node_modules/depA/index.js`

- Gọi `require('depA')` từ `/myApp/node_modules/depC/foobar.js` sẽ nạp `/myApp/node_modules/depC/node_modules/depA/index.js`

The resolving algorithm is the core part behind the robustness of the Node.js dependency management, and it makes it possible to have hundreds or even thousands of packages in an application without having collisions or problems of version compatibility.

Thuật toán phân giải là phần lõi đằng sau tính vững chắc của việc quản lý phụ thuộc của Node.js, và nó giúp có thể có hàng trăm hoặc thậm chí hàng nghìn gói trong một ứng dụng mà không xảy ra xung đột hay vấn đề về tương thích phiên bản.

The resolving algorithm is applied transparently for us when we invoke `require()`. However, if needed, it can still be used directly by any module by simply invoking `require.resolve()`.

Thuật toán phân giải được áp dụng một cách trong suốt cho chúng ta khi chúng ta gọi `require()`. Tuy nhiên, nếu cần, nó vẫn có thể được dùng trực tiếp bởi bất kỳ module nào chỉ bằng cách gọi `require.resolve()`.

The module cache — Module cache

Each module is only loaded and evaluated the first time it is required, since any subsequent call of `require()` will simply return the cached version. This should be clear by looking at the code of our homemade `require()` function. Caching is crucial for performance, but it also has some important functional implications:

Mỗi module chỉ được nạp và đánh giá vào lần đầu tiên nó được `require`, vì bất kỳ lần gọi `require()` nào sau đó cũng sẽ chỉ trả về phiên bản đã được cache. Điều này sẽ rõ ràng khi xem code của hàm `require()` tự chế của chúng ta. Việc cache rất quan trọng đối với hiệu năng, nhưng nó cũng có một số hệ quả chức năng quan trọng:

- It makes it possible to have cycles within module dependencies
- It guarantees, to some extent, that the same instance is always returned when requiring the same module from within a given package
 - Nó giúp có thể có các vòng (cycle) bên trong các phụ thuộc của module
 - Nó đảm bảo, ở một mức độ nào đó, rằng cùng một instance luôn được trả về khi `require` cùng một module từ bên trong một gói nhất định

The module cache is exposed via the `require.cache` variable, so it is possible to directly access it if needed. A common use case is to invalidate any cached module by deleting the relative key in the `require.cache` variable, a practice that can be useful during testing but very dangerous if applied in normal circumstances.

Module cache được phơi bày thông qua biến `require.cache`, nên có thể truy cập trực tiếp vào nó nếu cần. Một trường hợp sử dụng phổ biến là làm mất hiệu lực bất kỳ module đã cache nào bằng cách xóa key tương ứng trong biến `require.cache`, một thực hành có thể hữu ích trong quá trình kiểm thử nhưng rất nguy hiểm nếu áp dụng trong các tình huống bình thường.

Circular dependencies — Phụ thuộc vòng (Circular dependencies)

Many consider circular dependencies an intrinsic design issue, but it is something that might actually happen in a real project, so it's useful for us to know at least how this works with CommonJS. If we look again at our homemade `require()` function, we immediately get a glimpse of how this might work and what its caveats are.

Nhiều người coi phụ thuộc vòng (circular dependency) là một vấn đề thiết kế cố hữu, nhưng nó là điều thực sự có thể xảy ra trong một dự án thực tế, nên việc biết ít nhất là cách nó hoạt động với CommonJS là hữu ích cho chúng ta. Nếu chúng ta nhìn lại hàm `require()` tự chế của mình, chúng ta sẽ ngay lập tức thấy được thoáng qua về cách điều này có thể hoạt động và những điều cần lưu ý của nó là gì.

But let's walk together through an example to see how CommonJS behaves when dealing with circular dependencies. Let's suppose we have the scenario represented in Figure 2.1:

Nhưng hãy cùng đi qua một ví dụ để xem CommonJS hành xử thế nào khi xử lý phụ thuộc vòng. Giả sử chúng ta có kịch bản được biểu diễn trong Hình 2.1:

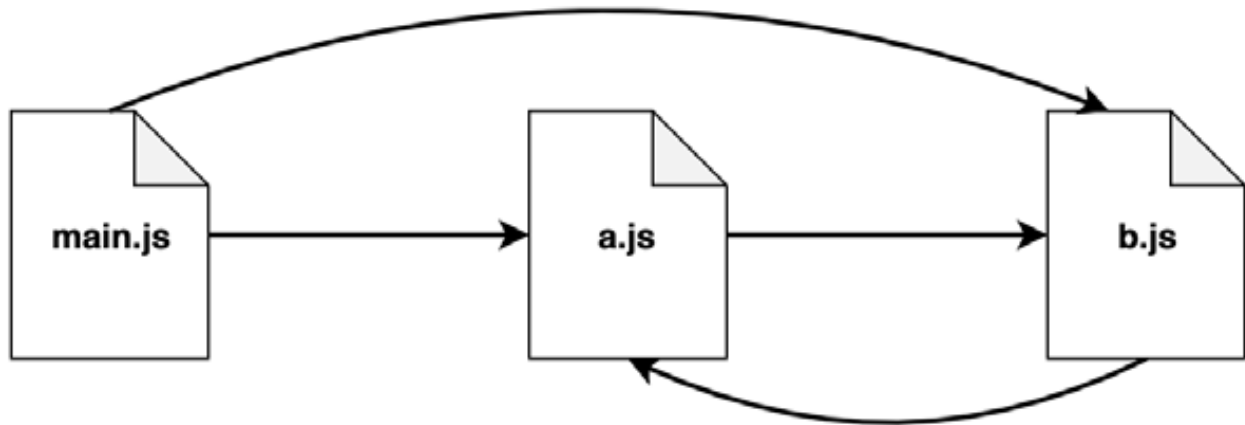


Figure 2.1: An example of **circular dependency**

Hình 2.1: Một ví dụ về phụ thuộc vòng

A module called `main.js` requires `a.js` and `b.js`. In turn, `a.js` requires `b.js`. But `b.js` relies on `a.js` as well! It's obvious that we have a circular dependency here as module `a.js` requires module `b.js` and module `b.js` requires module `a.js`. Let's have a look at the code of these two modules:

Một module tên là `main.js` require `a.js` và `b.js`. Đến lượt nó, `a.js` require `b.js`. Nhưng `b.js` cũng dựa vào `a.js`! Rõ ràng là ở đây chúng ta có một phụ thuộc vòng vì module `a.js` require module `b.js` và module `b.js` require module `a.js`. Hãy xem code của hai module này:

- Module `a.js`:

- Module `a.js`:

```
exports.loaded = false
const b = require('./b')
module.exports = {
  b,
  loaded: true // overrides the previous export
}
```

- Module `b.js`:

- Module `b.js`:

```
exports.loaded = false
const a = require('./a')
module.exports = {
```

```
  a,  
  loaded: true  
}
```

Now, let's see how these modules are required by `main.js`:

Bây giờ, hãy xem các module này được require bởi `main.js` như thế nào:

```
const a = require('./a')  
const b = require('./b')  
console.log('a ->', JSON.stringify(a, null, 2))  
console.log('b ->', JSON.stringify(b, null, 2))
```

If we run `main.js`, we will see the following output:

Nếu chúng ta chạy `main.js`, chúng ta sẽ thấy kết quả sau:

```
a -> {  
  "b": {  
    "a": {  
      "loaded": false  
    },  
    "loaded": true  
  },  
  "loaded": true  
}  
b -> {  
  "a": {  
    "loaded": false  
  },  
  "loaded": true  
}
```

This result reveals the caveats of circular dependencies with CommonJS, that is, different parts of our application will have a different view of what is exported by module `a.js` and module `b.js`, depending on the order in which those dependencies are loaded. While both the modules are completely initialized as soon as they are required from the module `main.js`, the `a.js` module will be incomplete when it is loaded from `b.js`. In particular, its **state** will be the one that it reached the moment `b.js` was required.

Kết quả này tiết lộ những điều cần lưu ý của phụ thuộc vòng với CommonJS, đó là, các phần khác nhau của ứng dụng của chúng ta sẽ có một góc nhìn khác nhau về những gì được xuất ra bởi module `a.js` và module `b.js`, tùy thuộc vào thứ tự mà các phụ thuộc đó được nạp. Trong khi cả hai module đều được khởi tạo hoàn toàn ngay khi chúng được require từ module `main.js`, thì module `a.js` sẽ không đầy đủ khi nó được nạp từ `b.js`. Cụ thể, trạng thái của nó sẽ là trạng thái mà nó đạt được vào thời điểm `b.js` được require.

In order to understand in more detail what happens behind the scenes, let's analyze step by step how the different modules are interpreted and how their local scope changes along the way:

Để hiểu chi tiết hơn về những gì xảy ra đằng sau hậu trường, hãy phân tích từng bước cách các module khác nhau được diễn giải và cách phạm vi cục bộ của chúng thay đổi dọc theo quá trình:

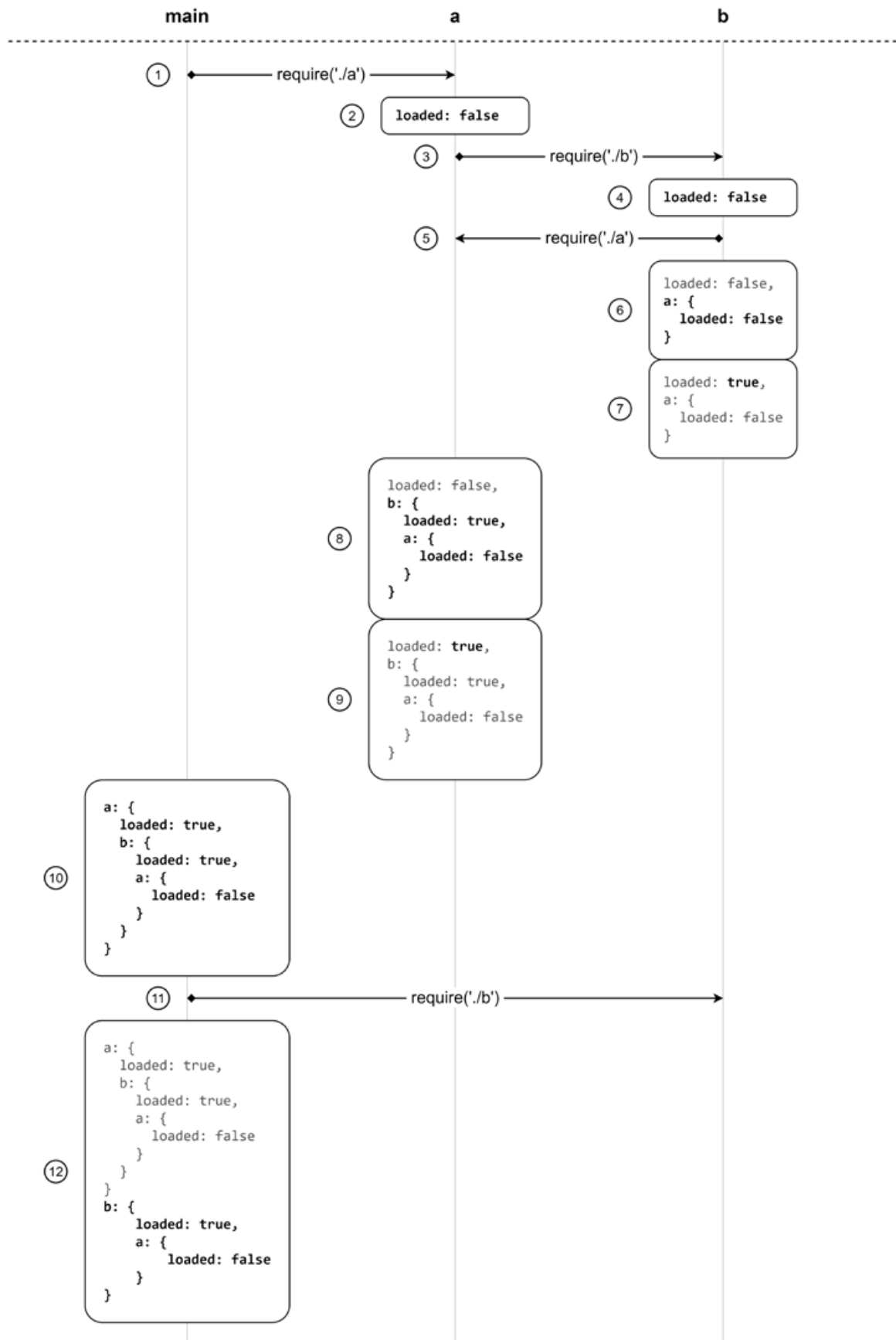


Figure 2.2: A visual representation of how a dependency loop is managed in Node.js The steps are as follows:

Hình 2.2: Một biểu diễn trực quan về cách một vòng lặp phụ thuộc được quản lý trong Node.js. Các bước như sau:

1. The processing starts in `main.js`, which immediately requires `a.js`
2. The first thing that module `a.js` does is set an exported value called `loaded` to `false`
3. At this point, module `a.js` requires module `b.js`
4. Like `a.js`, the first thing that module `b.js` does is set an exported value called `loaded` to `false`
5. Now, `b.js` requires `a.js` (cycle)
6. Since `a.js` has already been traversed, its currently exported value is immediately copied into the scope of module `b.js`
7. Module `b.js` finally changes the `loaded` value to `true`
8. Now that `b.js` has been fully executed, the control returns to `a.js`, which now holds a copy of the current state of module `b.js` in its own scope
9. The last step of module `a.js` is to set its `loaded` value to `true`
10. Module `a.js` is now completely executed and the control returns to `main.js`, which now has a copy of the current state of module `a.js` in its internal scope
11. `main.js` requires `b.js`, which is immediately loaded from cache
12. The current state of module `b.js` is copied into the scope of module `main.js` where we can finally see the complete picture of what the state of every module is

1. Quá trình xử lý bắt đầu trong `main.js`, vốn ngay lập tức require `a.js`
2. Điều đầu tiên mà module `a.js` làm là đặt một giá trị được xuất ra tên là `loaded` thành `false`
3. Tại thời điểm này, module `a.js` require module `b.js`
4. Giống như `a.js`, điều đầu tiên mà module `b.js` làm là đặt một giá trị được xuất ra tên là `loaded` thành `false`
5. Bây giờ, `b.js` require `a.js` (vòng lặp)
6. Vì `a.js` đã được duyệt qua, giá trị hiện đang được xuất ra của nó ngay lập tức được sao chép vào phạm vi của module `b.js`
7. Module `b.js` cuối cùng thay đổi giá trị `loaded` thành `true`
8. Giờ đây `b.js` đã được thực thi hoàn toàn, quyền điều khiển trả về cho `a.js`, vốn giờ đây giữ một bản sao của trạng thái hiện tại của module `b.js` trong phạm vi của riêng nó
9. Bước cuối cùng của module `a.js` là đặt giá trị `loaded` của nó thành `true`
10. Module `a.js` giờ đây đã được thực thi hoàn toàn và quyền điều khiển trả về cho `main.js`, vốn giờ đây có một bản sao của trạng thái hiện tại của module `a.js` trong phạm vi nội bộ của nó
11. `main.js` require `b.js`, vốn được nạp ngay lập tức từ cache
12. Trạng thái hiện tại của module `b.js` được sao chép vào phạm vi của module `main.js`, nơi cuối cùng chúng ta có thể thấy bức tranh đầy đủ về trạng thái của mọi module

As we said, the issue here is that module `b.js` has a partial view of module `a.js`, and this partial view gets propagated over when `b.js` is required in `main.js`. This behavior should spark an intuition which can be confirmed if we swap the order in which the two modules are required in `main.js`. If you actually try this, you will see that this time it will be the `a.js` module that will receive an incomplete version of `b.js`.

Như đã nói, vấn đề ở đây là module `b.js` có một góc nhìn không đầy đủ về module `a.js`, và góc nhìn không đầy đủ này được lan truyền tiếp khi `b.js` được require trong `main.js`. Hành vi này hẳn sẽ khơi gợi một trực giác mà có thể được xác nhận nếu chúng ta hoán đổi

thứ tự mà hai module được require trong `main.js`. Nếu bạn thực sự thử điều này, bạn sẽ thấy rằng lần này sẽ là module `a.js` nhận được một phiên bản không đầy đủ của `b.js`.

We understand now that this can become quite a fuzzy business if we lose control of which module is loaded first, which can happen quite easily if the project is big enough.

Bây giờ chúng ta hiểu rằng đây có thể trở thành một việc khá mơ hồ nếu chúng ta mất kiểm soát về module nào được nạp trước, điều có thể xảy ra khá dễ dàng nếu dự án đủ lớn.

Later in this chapter, we will see how ESM can deal with circular dependencies in a more effective way. Meanwhile, if you are using CommonJS, be very careful about this behavior and the way it can affect your application.

Ở phần sau của chương này, chúng ta sẽ thấy ESM có thể xử lý phụ thuộc vòng một cách hiệu quả hơn như thế nào. Trong khi đó, nếu bạn đang dùng CommonJS, hãy hết sức cẩn thận về hành vi này và cách nó có thể ảnh hưởng đến ứng dụng của bạn.

In the next section, we will discuss some patterns to define modules in Node.js.

Trong phần tiếp theo, chúng ta sẽ bàn về một số mẫu để định nghĩa module trong Node.js.

Module definition patterns — Các mẫu định nghĩa module

The module system, besides being a mechanism for loading dependencies, is also a tool for defining APIs. Like any other problem related to API design, the main factor to consider is the balance between private and public functionality. The aim is to maximize information hiding and API usability, while balancing these with other software qualities, such as extensibility and code reuse.

Hệ thống module, ngoài việc là một cơ chế để nạp các phụ thuộc, còn là một công cụ để định nghĩa các API. Giống như bất kỳ vấn đề nào khác liên quan đến thiết kế API, yếu tố chính cần cân nhắc là sự cân bằng giữa chức năng riêng tư và chức năng công khai. Mục tiêu là tối đa hóa việc che giấu thông tin và khả năng sử dụng của API, đồng thời cân bằng những điều này với các phẩm chất phần mềm khác, chẳng hạn như khả năng mở rộng và tái sử dụng code.

In this section, we will analyze some of the most popular patterns for defining modules in Node.js, such as named exports, exporting functions, classes and instances, and monkey patching. Each one has its own balance of information hiding, extensibility, and code reuse.

Trong phần này, chúng ta sẽ phân tích một số mẫu phổ biến nhất để định nghĩa module trong Node.js, chẳng hạn như named exports, exporting functions, classes và instances, và monkey patching. Mỗi mẫu có sự cân bằng riêng giữa che giấu thông tin, khả năng mở rộng và tái sử dụng code.

Named exports — Named exports

The most basic method for exposing a public API is using named exports, which involves assigning the values we want to make public to properties of the object referenced by `exports` (or `module.exports`). In this way, the resulting exported object becomes a container or namespace for a set of related functionalities.

Phương pháp cơ bản nhất để phơi bày một API công khai là dùng named exports, vốn liên quan đến việc gán các giá trị mà chúng ta muốn công khai vào các thuộc tính của đối

tượng được tham chiếu bởi `exports` (hay `module.exports`). Theo cách này, đối tượng được xuất ra trở thành một bộ chứa hoặc không gian tên cho một tập các chức năng liên quan.

The following code shows a module implementing this pattern:

Đoạn code sau cho thấy một module hiện thực mẫu này:

```
// file logger.js
exports.info = (message) => {
  console.log(`info: ${message}`)
}
exports.verbose = (message) => {
  console.log(`verbose: ${message}`)
}
```

The exported functions are then available as properties of the loaded module, as shown in the following code:

Các hàm được xuất ra sau đó có sẵn dưới dạng các thuộc tính của module đã nạp, như được thể hiện trong đoạn code sau:

```
// file main.js
const logger = require('./logger')
logger.info('This is an informational message')
logger.verbose('This is a verbose message')
```

Most of the Node.js core modules use this pattern. However, the CommonJS specification only allows the use of the `exports` variable to expose public members. Therefore, the named exports pattern is the only one that is really compatible with the CommonJS specification. The use of `module.exports` is an extension provided by Node.js to support a broader range of module definition patterns, which we are going to see next.

Hầu hết các module lõi của Node.js đều dùng mẫu này. Tuy nhiên, đặc tả CommonJS chỉ cho phép dùng biến `exports` để phơi bày các thành viên công khai. Do đó, mẫu named exports là mẫu duy nhất thực sự tương thích với đặc tả CommonJS. Việc dùng `module.exports` là một phần mở rộng do Node.js cung cấp để hỗ trợ một phạm vi rộng hơn các mẫu định nghĩa module, mà chúng ta sẽ thấy tiếp theo.

Exporting a function — Xuất ra một hàm

One of the most popular module definition patterns consists of reassigning the whole `module.exports` variable to a function. The main strength of this pattern is the fact that it allows you to expose only a single functionality, which provides a clear entry point for the module, making it simpler to understand and use; it also honors the principle of small surface area very well. This way of defining modules is also known in the community as the substack pattern, after one of its most prolific adopters, James Halliday (nickname substack – <https://github.com/substack>). Have a look at this pattern in the following example:

Một trong những mẫu định nghĩa module phổ biến nhất bao gồm việc gán lại toàn bộ biến `module.exports` cho một hàm. Điểm mạnh chính của mẫu này là việc nó cho phép bạn chỉ phơi bày một chức năng duy nhất, vốn cung cấp một điểm vào (entry point) rõ ràng cho module, làm cho nó đơn giản hơn để hiểu và sử dụng; nó cũng tôn trọng rất tốt nguyên tắc về bề mặt nhỏ (small surface area). Cách định nghĩa module này cũng được biết đến trong cộng đồng với tên gọi substack pattern, theo tên một trong những người áp dụng nó

nhiều nhất, James Halliday (biệt danh substack – <https://github.com/substack>). Hãy xem mẫu này trong ví dụ sau:

```
// file logger.js
module.exports = (message) => {
  console.log(`info: ${message}`)
}
```

A possible extension of this pattern is using the exported function as a namespace for other public APIs. This is a very powerful combination because it still gives the module the clarity of a single entry point (the main exported function) and at the same time it allows us to expose other functionalities that have secondary or more advanced use cases. The following code shows us how to extend the module we defined previously by using the exported function as a namespace:

Một phần mở rộng có thể có của mẫu này là dùng hàm được xuất ra làm một không gian tên cho các API công khai khác. Đây là một sự kết hợp rất mạnh mẽ vì nó vẫn cho module sự rõ ràng của một điểm vào duy nhất (hàm chính được xuất ra) và đồng thời nó cho phép chúng ta phơi bày các chức năng khác có các trường hợp sử dụng thứ yếu hoặc nâng cao hơn. Đoạn code sau cho chúng ta thấy cách mở rộng module mà chúng ta đã định nghĩa trước đó bằng cách dùng hàm được xuất ra làm một không gian tên:

```
module.exports.verbose = (message) => {
  console.log(`verbose: ${message}`)
}
```

This code demonstrates how to use the module that we just defined:

Đoạn code này minh họa cách dùng module mà chúng ta vừa định nghĩa:

```
// file main.js
const logger = require('./logger')
logger('This is an informational message')
logger.verbose('This is a verbose message')
```

Even though exporting just a function might seem like a limitation, in reality, it's a perfect way to put the emphasis on a single functionality, the most important one for the module, while giving less visibility to secondary or internal aspects, which are instead exposed as properties of the exported function itself. The modularity of Node.js heavily encourages the adoption of the single-responsibility principle (SRP): every module should have responsibility over a single functionality and that responsibility should be entirely encapsulated by the module.

Mặc dù việc chỉ xuất ra một hàm có vẻ là một sự giới hạn, nhưng trên thực tế, đó là một cách hoàn hảo để đặt sự nhấn mạnh vào một chức năng duy nhất, chức năng quan trọng nhất đối với module, đồng thời cho ít hiển thị hơn đối với các khía cạnh thứ yếu hoặc nội bộ, vốn thay vào đó được phơi bày dưới dạng các thuộc tính của chính hàm được xuất ra. Tính module hóa của Node.js khuyến khích mạnh mẽ việc áp dụng nguyên tắc trách nhiệm đơn (single-responsibility principle - SRP): mỗi module nên có trách nhiệm đối với một chức năng duy nhất và trách nhiệm đó nên được đóng gói hoàn toàn bởi module.

Exporting a class — Xuất ra một lớp (class)

A module that exports a class is a specialization of a module that exports a function. The difference is that with this new pattern we allow the user to create new instances using the constructor, but we also give them the ability to extend its prototype and forge new classes. The following is an example of this pattern:

Một module xuất ra một lớp là một sự chuyên biệt hóa của một module xuất ra một hàm. Sự khác biệt là với mẫu mới này, chúng ta cho phép người dùng tạo các instance mới bằng cách dùng hàm dựng (constructor), nhưng chúng ta cũng cho họ khả năng mở rộng prototype của nó và tạo ra các lớp mới. Sau đây là một ví dụ về mẫu này:

```
class Logger {
  constructor (name) {
    this.name = name
  }
  log (message) {
    console.log(`[${this.name}] ${message}`)
  }
  info (message) {
    this.log(`info: ${message}`)
  }
  verbose (message) {
    this.log(`verbose: ${message}`)
  }
}
module.exports = Logger
```

And, we can use the preceding module as follows:

Và, chúng ta có thể dùng module trên như sau:

```
// file main.js
const Logger = require('./logger')
const dbLogger = new Logger('DB')
dbLogger.info('This is an informational message')
const accessLogger = new Logger('ACCESS')
accessLogger.verbose('This is a verbose message')
```

Exporting a class still provides a single entry point for the module, but compared to the substack pattern, it exposes a lot more of the module internals. On the other hand, it allows much more power when it comes to extending its functionality.

Việc xuất ra một lớp vẫn cung cấp một điểm vào duy nhất cho module, nhưng so với substack pattern, nó phơi bày nhiều hơn rất nhiều phần nội bộ của module. Mặt khác, nó cho phép nhiều sức mạnh hơn khi nói đến việc mở rộng chức năng của nó.

Exporting an instance — Xuất ra một instance

We can leverage the caching mechanism of `require()` to easily define stateful instances created from a constructor or a **factory**, which can be shared across different modules. The following code shows an example of this pattern:

Chúng ta có thể tận dụng cơ chế caching của `require()` để dễ dàng định nghĩa các instance có trạng thái (stateful) được tạo từ một hàm dựng hoặc một factory, vốn có thể được chia sẻ giữa các module khác nhau. Đoạn code sau cho thấy một ví dụ về mẫu này:

```
// file logger.js
class Logger {
  constructor (name) {
    this.count = 0
```



```

    this.name = name
  }
  log (message) {
    this.count++
    console.log('[' + this.name + ']' + message)
  }
}
module.exports = new Logger('DEFAULT')

```

This newly defined module can then be used as follows:

Module mới được định nghĩa này sau đó có thể được dùng như sau:

```

// main.js
const logger = require('./logger')
logger.log('This is an informational message')

```

Because the module is cached, every module that requires the logger module will actually always retrieve the same instance of the object, thus sharing its state. This pattern is very much like creating a **singleton**. However, it does not guarantee the uniqueness of the instance across the entire application, as it happens in the traditional singleton pattern. When analyzing the resolving algorithm, we have seen that a module might be installed multiple times inside the dependency tree of an application. This results in multiple instances of the same logical module, all running in the context of the same Node.js application. We will analyze the Singleton pattern and its caveats in more detail in Chapter 7, Creational Design Patterns.

Bởi vì module được cache, mọi module require module logger thực ra sẽ luôn lấy được cùng một instance của đối tượng, do đó chia sẻ trạng thái của nó. Mẫu này rất giống với việc tạo một Singleton. Tuy nhiên, nó không đảm bảo tính duy nhất của instance trên toàn bộ ứng dụng, như điều xảy ra trong mẫu Singleton truyền thống. Khi phân tích thuật toán phân giải, chúng ta đã thấy rằng một module có thể được cài đặt nhiều lần bên trong cây phụ thuộc của một ứng dụng. Điều này dẫn đến nhiều instance của cùng một module về mặt logic, tất cả đều chạy trong bối cảnh của cùng một ứng dụng Node.js. Chúng ta sẽ phân tích mẫu Singleton và những điều cần lưu ý của nó chi tiết hơn trong Chương 7, Các mẫu thiết kế khởi tạo.

One interesting detail of this pattern is that it does not preclude the opportunity to create new instances, even if we are not explicitly exporting the class. In fact, we can rely on the constructor property of the exported instance to construct a new instance of the same type:

Một chi tiết thú vị của mẫu này là nó không loại trừ cơ hội tạo các instance mới, ngay cả khi chúng ta không xuất ra lớp một cách tường minh. Thực tế, chúng ta có thể dựa vào thuộc tính constructor của instance được xuất ra để dựng một instance mới cùng kiểu:

```

const customLogger = new logger.constructor('CUSTOM')
customLogger.log('This is an informational message')

```

As you can see, by using `logger.constructor()`, we can instantiate new Logger objects. Note that this technique must be used with caution or avoided altogether. Consider that, if the module author decided not to export the class explicitly, they probably wanted to keep this class private.

Như bạn có thể thấy, bằng cách dùng `logger.constructor()`, chúng ta có thể khởi tạo các đối tượng Logger mới. Lưu ý rằng kỹ thuật này phải được dùng một cách thận trọng hoặc tránh dùng hoàn toàn. Hãy cân nhắc rằng, nếu tác giả module đã quyết định không xuất ra lớp một cách tường minh, thì có lẽ họ muốn giữ lớp này riêng tư.

Modifying other modules or the global scope — Sửa đổi các module khác hoặc phạm vi toàn cục

A module can even export nothing. This can seem a bit out of place; however, we should not forget that a module can modify the global scope and any object in it, including other modules in the cache. Please note that these are in general considered bad practices, but since this pattern can be useful and safe under some circumstances (for example, for testing) and it's sometimes used in real-life projects, it's worth knowing.

Một module thậm chí có thể không xuất ra gì cả. Điều này có vẻ hơi lạc lõng; tuy nhiên, chúng ta không nên quên rằng một module có thể sửa đổi phạm vi toàn cục và bất kỳ đối tượng nào trong đó, bao gồm cả các module khác trong cache. Xin lưu ý rằng những điều này nhìn chung được coi là các thực hành xấu, nhưng vì mẫu này có thể hữu ích và an toàn trong một số trường hợp (ví dụ, để kiểm thử) và nó đôi khi được dùng trong các dự án thực tế, nên việc biết về nó là đáng giá.

We said that a module can modify other modules or objects in the global scope; well, this is called monkey patching. It generally refers to the practice of modifying the existing objects at runtime to change or extend their behavior or to apply temporary fixes.

Chúng ta đã nói rằng một module có thể sửa đổi các module khác hoặc các đối tượng trong phạm vi toàn cục; vâng, điều này được gọi là monkey patching. Nó thường ám chỉ việc thực hành sửa đổi các đối tượng hiện có vào lúc chạy (runtime) để thay đổi hoặc mở rộng hành vi của chúng hoặc để áp dụng các bản vá tạm thời.

The following example shows us how we can add a new function to another module:

Ví dụ sau đây cho chúng ta thấy cách chúng ta có thể thêm một hàm mới vào một module khác:

```
// file patcher.js
// ./logger is another module
require('./logger').customMessage = function () {
  console.log('This is a new functionality')
}
```

Using our new patcher module is as easy as writing the following code:

Việc dùng module patcher mới của chúng ta dễ dàng như việc viết đoạn code sau:

```
// file main.js
require('./patcher')
const logger = require('./logger')
logger.customMessage()
```

The technique described here can be very dangerous to use. The main concern is that having a module that modifies the global namespace or other modules is an operation with side effects. In other words, it affects the state of entities outside their scope, which can have consequences that aren't easily predictable, especially when multiple modules interact with the same entities. Imagine having two different modules trying to set the same global variable, or modifying the same property of the same module. The effects can be unpredictable (which module wins?), but most importantly it would have repercussions on the entire application. So, again use this technique with care and make sure you understand all the possible side effects while doing so.

Kỹ thuật được mô tả ở đây có thể rất nguy hiểm khi sử dụng. Mối quan tâm chính là việc có một module sửa đổi không gian tên toàn cục hoặc các module khác là một thao tác có tác

dụng phụ (side effect). Nói cách khác, nó ảnh hưởng đến trạng thái của các thực thể bên ngoài phạm vi của chúng, điều có thể gây ra những hậu quả không dễ lường trước, đặc biệt là khi nhiều module tương tác với cùng các thực thể đó. Hãy tưởng tượng có hai module khác nhau cùng cố đặt cùng một biến toàn cục, hoặc sửa đổi cùng một thuộc tính của cùng một module. Các hệ quả có thể khó lường (module nào thắng?), nhưng quan trọng nhất là nó sẽ có ảnh hưởng đến toàn bộ ứng dụng. Vậy nên, một lần nữa, hãy dùng kỹ thuật này một cách cẩn thận và đảm bảo rằng bạn hiểu tất cả các tác dụng phụ có thể có khi làm vậy.

If you want a real-life example of how this can be useful, have a look at `nock` ([nodejsdp.link/nock](#)), a module that allows you to mock HTTP responses in your tests. The way `nock` works is by monkey patching the Node.js `http` module and by changing its behavior so that it will provide the mocked response rather than issuing a real HTTP **request**. This allows our unit test to run without hitting the actual production HTTP endpoints, something that's very convenient when writing tests for code that relies on third-party APIs.

Nếu bạn muốn một ví dụ thực tế về cách điều này có thể hữu ích, hãy xem `nock` ([nodejsdp.link/nock](#)), một module cho phép bạn giả lập (mock) các phản hồi HTTP trong các bài kiểm thử của mình. Cách `nock` hoạt động là bằng cách monkey patching module `http` của Node.js và thay đổi hành vi của nó để nó cung cấp phản hồi đã được giả lập thay vì phát đi một yêu cầu HTTP thực sự. Điều này cho phép unit test của chúng ta chạy mà không cần truy cập đến các endpoint HTTP production thực tế, một điều rất tiện lợi khi viết các bài kiểm thử cho code dựa vào các API của bên thứ ba.

At this point, we should have a quite complete understanding of CommonJS and some of the patterns that are generally used with it. In the next section, we will explore ECMAScript modules, also known as ESM.

Tại thời điểm này, chúng ta hẳn đã có một sự hiểu biết khá đầy đủ về CommonJS và một số mẫu thường được dùng với nó. Trong phần tiếp theo, chúng ta sẽ khám phá ECMAScript modules, còn được gọi là ESM.

ESM: ECMAScript modules — ESM: ECMAScript modules

ECMAScript modules (also known as ES modules or ESM) were introduced as part of the ECMAScript 2015 specification with the goal to give JavaScript an official module system suitable for different execution environments. The ESM specification tries to retain some good ideas from previous existing module systems like CommonJS and AMD. The syntax is very simple and compact. There is support for cyclic dependencies and the possibility to load modules asynchronously.

ECMAScript modules (còn được gọi là ES modules hay ESM) được giới thiệu như một phần của đặc tả ECMAScript 2015 với mục tiêu mang lại cho JavaScript một hệ thống module chính thức phù hợp với các môi trường thực thi khác nhau. Đặc tả ESM cố gắng giữ lại một số ý tưởng tốt từ các hệ thống module đã tồn tại trước đó như CommonJS và AMD. Cú pháp rất đơn giản và gọn gàng. Có hỗ trợ cho phụ thuộc vòng và khả năng nạp module một cách bất đồng bộ.

The most important differentiator between ESM and CommonJS is that ES modules are static, which means that imports are described at the top level of every module and outside any **control flow** statement. Also, the name of the imported modules cannot be dynamically generated at runtime using expressions, only constant strings are allowed.

Điểm khác biệt quan trọng nhất giữa ESM và CommonJS là ES modules có tính tĩnh (static), nghĩa là các import được mô tả ở cấp cao nhất (top level) của mỗi module và bên ngoài bất

kỳ câu lệnh điều khiển luồng nào. Ngoài ra, tên của các module được import không thể được tạo ra một cách động vào lúc chạy bằng các biểu thức, chỉ cho phép các chuỗi hằng (constant string).

For instance, the following code wouldn't be valid when using ES modules:

Ví dụ, đoạn code sau sẽ không hợp lệ khi dùng ES modules:

```
if (condition) {
  import module1 from 'module1'
} else {
  import module2 from 'module2'
}
```

While in CommonJS, it is perfectly fine to write something like this:

Trong khi đó với CommonJS, viết một thứ gì đó như thế này là hoàn toàn ổn:

```
let module = null
if (condition) {
  module = require('module1')
} else {
  module = require('module2')
}
```

At a first glance, this characteristic of ESM might seem an unnecessary limitation, but in reality, having static imports opens up a number of interesting scenarios that are not practical with the dynamic nature of CommonJS. For instance, static imports allow the static analysis of the dependency tree, which allows optimizations such as dead code elimination (tree shaking) and more.

Thoạt nhìn, đặc điểm này của ESM có vẻ là một sự giới hạn không cần thiết, nhưng trên thực tế, việc có các import tĩnh mở ra một số kịch bản thú vị vốn không khả thi với bản chất động của CommonJS. Ví dụ, các import tĩnh cho phép phân tích tĩnh cây phụ thuộc, vốn cho phép các tối ưu hóa như loại bỏ code chết (tree shaking) và hơn thế nữa.

Using ESM in Node.js — Dùng ESM trong Node.js

Node.js will consider every .js file to be written using the CommonJS syntax by default; therefore, if we use the ESM syntax inside a .js file, the interpreter will simply throw an error.

Node.js sẽ coi mọi file .js là được viết bằng cú pháp CommonJS theo mặc định; do đó, nếu chúng ta dùng cú pháp ESM bên trong một file .js, trình thông dịch sẽ chỉ đơn giản ném ra một lỗi.

There are several ways to tell the Node.js interpreter to consider a given module as an ES module rather than a CommonJS module:

Có một số cách để báo cho trình thông dịch Node.js coi một module nhất định là một ES module thay vì một module CommonJS:

- Give the module file the extension .mjs
- Add to the nearest parent package.json a field called "type" with a value of "module"
 - Đặt cho file module phần mở rộng .mjs
 - Thêm vào file package.json cha gần nhất một trường gọi là "type" với giá trị là "module"

Throughout the rest of this book and in the code examples provided, we will keep using the `.js` extension to keep the code more easily accessible to most text editors, so if you are copying and pasting examples straight from the book, make sure that you also create a `package.json` file with the `"type": "module"` entry.

Xuyên suốt phần còn lại của cuốn sách này và trong các ví dụ code được cung cấp, chúng tôi sẽ tiếp tục dùng phần mở rộng `.js` để giữ cho code dễ tiếp cận hơn đối với hầu hết các trình soạn thảo văn bản, nên nếu bạn đang sao chép và dán các ví dụ thẳng từ sách, hãy đảm bảo rằng bạn cũng tạo một file `package.json` với mục `"type": "module"`.

Let's now have a look at the ESM syntax.

Bây giờ hãy xem qua cú pháp ESM.

Named exports and imports — Named exports và imports

ESM allows us to export functionality from a module through the `export` keyword.

ESM cho phép chúng ta xuất ra chức năng từ một module thông qua từ khóa `export`.

Note that ESM uses the singular word `export` as opposed to the plural (`exports` and `module.exports`) used by CommonJS. In an ES module, everything is private by default and only exported entities are publicly accessible from other modules.

Lưu ý rằng ESM dùng từ ở dạng số ít `export` trái với dạng số nhiều (`exports` và `module.exports`) được dùng bởi CommonJS. Trong một ES module, mọi thứ đều là riêng tư theo mặc định và chỉ các thực thể được xuất ra mới có thể truy cập công khai từ các module khác.

The `export` keyword can be used in front of the entities that we want to make available to the module users. Let's see an example:

Từ khóa `export` có thể được dùng đặt phía trước các thực thể mà chúng ta muốn làm cho có sẵn cho người dùng module. Hãy xem một ví dụ:

```
// logger.js
// exports a function as `log`
export function log (message) {
  console.log(message)
}
// exports a constant as `DEFAULT_LEVEL`
export const DEFAULT_LEVEL = 'info'
// exports an object as `LEVELS`
export const LEVELS = {
  error: 0,
  debug: 1,
  warn: 2,
  data: 3,
  info: 4,
  verbose: 5
}
// exports a class as `Logger`
export class Logger {
  constructor (name) {
    this.name = name
  }
}
```

```

    }
    log (message) {
      console.log(`[${this.name}] ${message}`)
    }
  }
}

```

If we want to import entities from a module we can use the `import` keyword. The syntax is quite flexible, and it allows us to import one or more entities and even to rename imports. Let's see some examples:

Nếu chúng ta muốn import các thực thể từ một module, chúng ta có thể dùng từ khóa `import`. Cú pháp khá linh hoạt, và nó cho phép chúng ta import một hoặc nhiều thực thể và thậm chí đổi tên các import. Hãy xem một số ví dụ:

```

import * as loggerModule from './logger.js'
console.log(loggerModule)

```

In this example, we are using the `*` syntax (also called namespace import) to import all the members of the module and assign them to the local `loggerModule` variable. This example will output something like this:

Trong ví dụ này, chúng ta đang dùng cú pháp `*` (còn gọi là namespace import) để import tất cả các thành viên của module và gán chúng cho biến cục bộ `loggerModule`. Ví dụ này sẽ xuất ra một thứ gì đó như thế này:

```

[Module] {
  DEFAULT_LEVEL: 'info',
  LEVELS: { error: 0, debug: 1, warn: 2, data: 3, info: 4,
    verbose: 5 },
  Logger: [Function: Logger],
  log: [Function: log]
}

```

As we can see, all the entities exported in our module are now accessible in the `loggerModule` namespace. For instance, we could refer to the `log()` function through `loggerModule.log`.

Như chúng ta có thể thấy, tất cả các thực thể được xuất ra trong module của chúng ta giờ đây có thể truy cập trong không gian tên `loggerModule`. Ví dụ, chúng ta có thể tham chiếu đến hàm `log()` thông qua `loggerModule.log`.

It's very important to note that, as opposed to CommonJS, with ESM we have to specify the file extension of the imported modules. With CommonJS we can use either `./logger` or `./logger.js`, with ESM we are forced to use `./logger.js`.

Điều rất quan trọng cần lưu ý là, trái với CommonJS, với ESM chúng ta phải chỉ định phần mở rộng file của các module được import. Với CommonJS chúng ta có thể dùng `./logger` hoặc `./logger.js`, còn với ESM chúng ta buộc phải dùng `./logger.js`.

If we are using a large module, most often we don't want to import all of its functionality, but only one or few entities from it:

Nếu chúng ta đang dùng một module lớn, thường thì chúng ta không muốn import tất cả chức năng của nó, mà chỉ một hoặc vài thực thể từ nó:

```

import { log } from './logger.js'
log('Hello World')

```

If we want to import more than one entity, this is how we would do that:

Nếu chúng ta muốn import nhiều hơn một thực thể, đây là cách chúng ta sẽ làm điều đó:

```
import { log, Logger } from './logger.js'
log('Hello World')
const logger = new Logger('DEFAULT')
logger.log('Hello world')
```

When we use this type of import statement, the entities are imported into the current scope, so there is a risk of a name clash. The following code, for example, would not work:

Khi chúng ta dùng loại câu lệnh import này, các thực thể được import vào phạm vi hiện tại, nên có nguy cơ xảy ra xung đột tên. Ví dụ, đoạn code sau sẽ không hoạt động:

```
import { log } from './logger.js'
const log = console.log
```

If we try to execute the preceding snippet, the interpreter fails with the following error:

Nếu chúng ta thử thực thi đoạn trên, trình thông dịch thất bại với lỗi sau:

```
SyntaxError: Identifier 'log' has already been declared
```

In situations like this one, we can resolve the clash by renaming the imported entity with the `as` keyword:

Trong những tình huống như thế này, chúng ta có thể giải quyết xung đột bằng cách đổi tên thực thể được import với từ khóa `as`:

```
import { log as log2 } from './logger.js'
const log = console.log
log('message from log')
log2('message from log2')
```

This approach can be particularly useful when the clash is generated by importing two entities with the same name from different modules, and therefore changing the original names is outside the consumer's control.

Cách tiếp cận này có thể đặc biệt hữu ích khi xung đột được tạo ra bởi việc import hai thực thể cùng tên từ các module khác nhau, và do đó việc thay đổi tên gốc nằm ngoài tầm kiểm soát của bên tiêu thụ.

Default exports and imports — Default exports và imports

One widely used feature of CommonJS is the ability to export a single unnamed entity through the assignment of `module.exports`. We saw that this is very convenient as it encourages module developers to follow the single-responsibility principle and expose only one clear interface. With ESM, we can do something similar through what's called a default export. A default export makes use of the `export default` keywords and it looks like this:

Một tính năng được dùng rộng rãi của CommonJS là khả năng xuất ra một thực thể không tên duy nhất thông qua việc gán cho `module.exports`. Chúng ta đã thấy rằng điều này rất tiện lợi vì nó khuyến khích các nhà phát triển module tuân theo nguyên tắc trách nhiệm đơn và chỉ phơi bày một interface rõ ràng. Với ESM, chúng ta có thể làm một điều tương tự thông qua cái được gọi là default export. Một default export sử dụng các từ khóa `export default` và nó trông như thế này:

```
// logger.js
export default class Logger {
```

```

constructor (name) {
  this.name = name
}
log (message) {
  console.log(`[${this.name}] ${message}`)
}
}

```

In this case, the name `Logger` is ignored, and the entity exported is registered under the name `default`. This exported name is handled in a special way, and it can be imported as follows:

Trong trường hợp này, cái tên `Logger` bị bỏ qua, và thực thể được xuất ra được đăng ký dưới cái tên `default`. Tên được xuất ra này được xử lý theo một cách đặc biệt, và nó có thể được import như sau:

```

// main.js
import MyLogger from './logger.js'
const logger = new MyLogger('info')
logger.log('Hello World')

```

The difference with named ESM imports is that here, since the default export is considered unnamed, we can import it and at the same time assign it a local name of our choice. In this example, we can replace `MyLogger` with anything else that makes sense in our context. This is very similar to what we do with CommonJS modules. Note also that we don't have to wrap the import name around brackets or use the `as` keyword when renaming.

Sự khác biệt với named ESM imports là ở đây, vì default export được coi là không tên, chúng ta có thể import nó và đồng thời gán cho nó một tên cục bộ do chúng ta chọn. Trong ví dụ này, chúng ta có thể thay thế `MyLogger` bằng bất cứ thứ gì khác có ý nghĩa trong bối cảnh của chúng ta. Điều này rất giống với những gì chúng ta làm với các module CommonJS. Cũng lưu ý rằng chúng ta không phải bọc tên import trong dấu ngoặc nhọn hoặc dùng từ khóa `as` khi đổi tên.

Internally, a default export is equivalent to a named export with `default` as the name. We can easily verify this statement by running the following snippet of code:

Về mặt nội bộ, một default export tương đương với một named export với tên là `default`. Chúng ta có thể dễ dàng xác minh phát biểu này bằng cách chạy đoạn code sau:

```

// showDefault.js
import * as loggerModule from './logger.js'
console.log(loggerModule)

```

When executed, the previous code will print something like this:

Khi được thực thi, đoạn code trên sẽ in ra một thứ gì đó như thế này:

```
[Module] { default: [Function: Logger] }
```

One thing that we cannot do, though, is import the default entity explicitly. In fact, something like the following will fail:

Tuy nhiên, một điều mà chúng ta không thể làm là import thực thể default một cách tường minh. Thực tế, một thứ gì đó như sau sẽ thất bại:

```
import { default } from './logger.js'
```

The execution will fail with a `SyntaxError: Unexpected reserved word` error. This happens because the `default` keyword cannot be used as a variable name. It is valid as an object attribute, so in the previous example, it is okay to use `loggerModule.default`, but we can't have a variable named `default` directly in the scope.

Việc thực thi sẽ thất bại với lỗi `SyntaxError: Unexpected reserved word`. Điều này xảy ra vì từ khóa `default` không thể được dùng làm tên biến. Nó hợp lệ khi làm thuộc tính của đối tượng, nên trong ví dụ trước, dùng `loggerModule.default` là ổn, nhưng chúng ta không thể có một biến tên là `default` trực tiếp trong phạm vi.

Mixed exports — Mixed exports

It is possible to mix named exports and a default export within an ES module. Let's have a look at an example:

Có thể trộn lẫn named exports và một default export trong một ES module. Hãy xem một ví dụ:

```
// logger.js
export default function log (message) {
  console.log(message)
}
export function info (message) {
  log(`info: ${message}`)
}
```

The preceding code is exporting the `log()` function as a default export and a named export for a function called `info()`. Note that `info()` can reference `log()` internally. It would not be possible to replace the call to `log()` with `default()` to do that, as it would be a syntax error (`Unexpected token default`).

Đoạn code trên đang xuất ra hàm `log()` dưới dạng một default export và một named export cho một hàm tên là `info()`. Lưu ý rằng `info()` có thể tham chiếu đến `log()` một cách nội bộ. Sẽ không thể thay thế lời gọi đến `log()` bằng `default()` để làm điều đó, vì đó sẽ là một lỗi cú pháp (`Unexpected token default`).

If we want to import both the default export and one or more named exports, we can do it using the following format:

Nếu chúng ta muốn import cả default export và một hoặc nhiều named exports, chúng ta có thể làm điều đó bằng định dạng sau:

```
import mylog, { info } from './logger.js'
```

In the preceding example, we are importing the default export from `logger.js` as `mylog` and also the named export `info`.

Trong ví dụ trên, chúng ta đang import default export từ `logger.js` dưới dạng `mylog` và cũng cả named export `info`.

Let's now discuss some key details and differences between the default export and named exports:

Bây giờ hãy bàn về một số chi tiết và khác biệt then chốt giữa default export và named exports:

- Named exports are explicit. Having predetermined names allows IDEs to support the developer with automatic imports, autocomplete, and refactoring tools. For instance, if we

type `writeFileSync`, the editor might automatically add `import { writeFileSync } from 'fs'` at the beginning of the current file. Default exports, on the contrary, make all these things more complicated as a given functionality could have different names in different files, so it's harder to make inferences on which module might provide a given functionality based only on a given name.

- The default export is a convenient mechanism to communicate what is the single most important functionality for a module. Also, from the perspective of the user, it can be easier to import the obvious piece of functionality without having to know the exact name of the binding.
- In some circumstances, default exports might make it harder to apply dead code elimination (tree shaking). For example, a module could provide only a default export, which is an object where all the functionality is exposed as properties of such an object. When we import this default object, most module bundlers will consider the entire object being used and they won't be able to eliminate any unused code from the exported functionality.
 - Named exports là tường minh. Việc có các tên định sẵn cho phép các IDE hỗ trợ nhà phát triển với các import tự động, autocomplete và các công cụ tái cấu trúc. Ví dụ, nếu chúng ta gõ `writeFileSync`, trình soạn thảo có thể tự động thêm `import { writeFileSync } from 'fs'` vào đầu file hiện tại. Ngược lại, default exports làm cho tất cả những điều này phức tạp hơn vì một chức năng nhất định có thể có những cái tên khác nhau trong các file khác nhau, nên khó hơn để suy luận xem module nào có thể cung cấp một chức năng nhất định chỉ dựa trên một cái tên nhất định.
 - Default export là một cơ chế tiện lợi để truyền đạt đâu là chức năng quan trọng nhất duy nhất của một module. Ngoài ra, từ góc nhìn của người dùng, có thể dễ dàng hơn để import phần chức năng hiển nhiên mà không cần phải biết tên chính xác của binding.
 - Trong một số trường hợp, default exports có thể làm cho việc loại bỏ code chết (tree shaking) khó áp dụng hơn. Ví dụ, một module có thể chỉ cung cấp một default export, vốn là một đối tượng nơi tất cả chức năng được phơi bày dưới dạng các thuộc tính của đối tượng đó. Khi chúng ta import đối tượng default này, hầu hết các module bundler sẽ coi toàn bộ đối tượng là đang được sử dụng và chúng sẽ không thể loại bỏ bất kỳ code không dùng đến nào khỏi chức năng được xuất ra.

For these reasons, it is generally considered good practice to stick with named exports, especially when you want to expose more than one functionality, and only use default exports if it's one clear functionality you want to export.

Vì những lý do này, nhìn chung được coi là thực hành tốt khi gắn bó với named exports, đặc biệt khi bạn muốn phơi bày nhiều hơn một chức năng, và chỉ dùng default exports nếu đó là một chức năng rõ ràng duy nhất mà bạn muốn xuất ra.

This is not a hard rule and there are notable exceptions to this suggestion. For instance, all Node.js core modules have both a default export and a number of named exports. Also, React (nodejsdp.link/react) uses mixed exports. Consider carefully what the best approach for your specific module is and what you want the developer experience to be for the users of your module.

Đây không phải là một quy tắc cứng và có những ngoại lệ đáng chú ý đối với gợi ý này. Ví dụ, tất cả các module lõi của Node.js đều có cả một default export và một số named exports. Ngoài ra, React (nodejsdp.link/react) dùng mixed exports. Hãy cân nhắc kỹ đâu là cách tiếp cận tốt nhất cho module cụ thể của bạn và bạn muốn trải nghiệm của nhà phát triển dành cho người dùng module của bạn là gì.

Module identifiers — Module identifiers

Module identifiers (also called module specifiers) are the different types of values that we can use in our `import` statements to specify the location of the module we want to load.

Module identifiers (còn gọi là module specifiers) là các loại giá trị khác nhau mà chúng ta có thể dùng trong các câu lệnh `import` để chỉ định vị trí của module mà chúng ta muốn nạp.

So far, we have seen only relative paths, but there are several other possibilities and some nuances to keep in mind. Let's list all the possibilities:

Cho đến giờ, chúng ta mới chỉ thấy các đường dẫn tương đối, nhưng còn một số khả năng khác và một số sắc thái cần lưu ý. Hãy liệt kê tất cả các khả năng:

- Relative specifiers like `./logger.js` or `../logger.js`. They are used to refer to a path relative to the location of the importing file.
- Absolute specifiers like `file:///opt/nodejs/config.js`. They refer directly and explicitly to a full path. Note that this is the only way with ESM to refer to an absolute path for a module, using a `/` or a `///` prefix won't work. This is a significant difference with CommonJS.
- Bare specifiers are identifiers like `fastify` or `http`, and they represent modules available in the `node_modules` folder and generally installed through a package manager (such as `npm`) or available as core Node.js modules.
- Deep import specifiers like `fastify/lib/logger.js`, which refer to a path within a package in `node_modules` (`fastify`, in this case).
 - Relative specifiers như `./logger.js` hay `../logger.js`. Chúng được dùng để tham chiếu đến một đường dẫn tương đối với vị trí của file đang import.
 - Absolute specifiers như `file:///opt/nodejs/config.js`. Chúng tham chiếu trực tiếp và tường minh đến một đường dẫn đầy đủ. Lưu ý rằng đây là cách duy nhất với ESM để tham chiếu đến một đường dẫn tuyệt đối cho một module, dùng tiền tố `/` hay `///` sẽ không hoạt động. Đây là một khác biệt đáng kể so với CommonJS.
 - Bare specifiers là các định danh như `fastify` hay `http`, và chúng đại diện cho các module có sẵn trong thư mục `node_modules` và nhìn chung được cài đặt thông qua một trình quản lý gói (như `npm`) hoặc có sẵn dưới dạng các module lõi của Node.js.
 - Deep import specifiers như `fastify/lib/logger.js`, vốn tham chiếu đến một đường dẫn bên trong một gói trong `node_modules` (`fastify`, trong trường hợp này).

In browser environments, it is possible to import modules directly by specifying the module URL, for instance, `https://unpkg.com/lodash`. This feature is not supported by Node.js.

Trong các môi trường trình duyệt, có thể import các module trực tiếp bằng cách chỉ định URL của module, ví dụ, `https://unpkg.com/lodash`. Tính năng này không được hỗ trợ bởi Node.js.

Async imports — Async imports

As we have seen in the previous section, the `import` statement is static and therefore subject to two important limitations:

Như chúng ta đã thấy trong phần trước, câu lệnh `import` là tĩnh và do đó chịu hai giới hạn quan trọng:

- A module identifier cannot be constructed at runtime

- Module imports are declared at the top level of every file and they cannot be nested within control flow statements
 - Một module identifier không thể được dựng vào lúc chạy
 - Các module import được khai báo ở cấp cao nhất của mỗi file và chúng không thể được lồng bên trong các câu lệnh điều khiển luồng

There are some use cases when these limitations can become a little bit too restrictive. Imagine, for instance, if we have to import a specific translation module for the current user language, or a variation of a module that depends on the user's operating system. Also, what if we want to load a given module, which might be particularly heavy, only if the user is accessing the piece of functionality that requires that module?

Có một số trường hợp sử dụng mà các giới hạn này có thể trở nên hơi quá hạn chế. Ví dụ, hãy tưởng tượng nếu chúng ta phải import một module dịch thuật cụ thể cho ngôn ngữ hiện tại của người dùng, hoặc một biến thể của một module phụ thuộc vào hệ điều hành của người dùng. Ngoài ra, sẽ thế nào nếu chúng ta muốn nạp một module nhất định, vốn có thể đặc biệt nặng, chỉ khi người dùng đang truy cập vào phần chức năng đòi hỏi module đó?

To allow us to overcome these limitations ES modules provides **async** imports (also called dynamic imports).

Để cho phép chúng ta vượt qua các giới hạn này, ES modules cung cấp async imports (còn gọi là dynamic imports).

Async imports can be performed at runtime using the special `import()` operator.

Async imports có thể được thực hiện vào lúc chạy bằng toán tử đặc biệt `import()`.

The `import()` operator is syntactically equivalent to a function that takes a module identifier as an argument and it returns a **promise** that resolves to a module object.

Toán tử `import()` về mặt cú pháp tương đương với một hàm nhận một module identifier làm đối số và nó trả về một promise phân giải (resolve) thành một đối tượng module.

We will learn more about promises in Chapter 5, Asynchronous Control Flow Patterns with Promises and Async/**Await**, so don't worry too much about understanding all the nuances of the specific promise syntax for now.

Chúng ta sẽ tìm hiểu thêm về promise trong Chương 5, Các mẫu điều khiển luồng bất đồng bộ với Promise và Async/Await, nên đừng quá lo lắng về việc hiểu hết các sắc thái của cú pháp promise cụ thể vào lúc này.

The module identifier can be any module identifier supported by static imports as discussed in the previous section. Now, let's see how to use dynamic imports with a simple example.

Module identifier có thể là bất kỳ module identifier nào được hỗ trợ bởi static imports như đã bàn trong phần trước. Bây giờ, hãy xem cách dùng dynamic imports với một ví dụ đơn giản.

We want to build a command line application that can print "Hello World" in different languages. In the future, we will probably want to support many more phrases and languages, so it makes sense to have one file with the translations of all the user-facing strings for each supported language.

Chúng ta muốn xây dựng một ứng dụng dòng lệnh có thể in "Hello World" bằng các ngôn ngữ khác nhau. Trong tương lai, chúng ta có lẽ sẽ muốn hỗ trợ nhiều cụm từ và ngôn ngữ

hơn nữa, nên việc có một file với các bản dịch của tất cả các chuỗi hướng tới người dùng cho mỗi ngôn ngữ được hỗ trợ là hợp lý.

Let's create some example modules for some of the languages we want to support:

Hãy tạo một số module ví dụ cho một vài ngôn ngữ mà chúng ta muốn hỗ trợ:

```
// strings-el.js
export const HELLO = 'Γεια σου κόσμε'
// strings-en.js
export const HELLO = 'Hello World'
// strings-es.js
export const HELLO = 'Hola mundo'
// strings-it.js
export const HELLO = 'Ciao mondo'
// strings-pl.js
export const HELLO = 'Witaj świecie'
```

Now let's create the main script that takes a language code from the command line and prints “Hello World” in the selected language:

Bây giờ hãy tạo script chính nhận một mã ngôn ngữ từ dòng lệnh và in “Hello World” bằng ngôn ngữ được chọn:

```
// main.js
const SUPPORTED_LANGUAGES = ['el', 'en', 'es', 'it', 'pl'] // (1)
const selectedLanguage = process.argv[2] // (2)
if (!SUPPORTED_LANGUAGES.includes(selectedLanguage)) { // (3)
  console.error('The specified language is not supported')
  process.exit(1)
}
const translationModule = `./strings-${selectedLanguage}.js` // (4)
import (translationModule) // (5)
  .then((strings) => { // (6)
    console.log(strings.HELLO)
  })
```

The first part of the script is quite simple. What we do there is:

Phần đầu của script khá đơn giản. Những gì chúng ta làm ở đó là:

1. Define a list of supported languages.
2. Read the selected language from the first argument passed in the command line.
3. Finally, we handle the case where the selected language is not supported.
 1. Định nghĩa một danh sách các ngôn ngữ được hỗ trợ.
 2. Đọc ngôn ngữ được chọn từ đối số đầu tiên được truyền vào trong dòng lệnh.
 3. Cuối cùng, chúng ta xử lý trường hợp ngôn ngữ được chọn không được hỗ trợ.

The second part of the code is where we actually use dynamic imports:

Phần thứ hai của code là nơi chúng ta thực sự dùng dynamic imports:

4. First of all, we dynamically build the name of the module we want to import based on the selected language. Note that the module name needs to be a relative path to the module file, that's why we are prepending `./` to the filename.
5. We use the `import()` operator to trigger the dynamic import of the module.

6. The dynamic import happens asynchronously, so we can use the `.then()` hook on the returned promise to get notified when the module is ready to be used. The function passed to `then()` will be executed when the module is fully loaded and `strings` will be the module namespace imported dynamically. After that, we can access `strings.HELLO` and print its value to the console.

4. Trước tiên, chúng ta dựng một cách động tên của module mà chúng ta muốn import dựa trên ngôn ngữ được chọn. Lưu ý rằng tên module cần phải là một đường dẫn tương đối đến file module, đó là lý do tại sao chúng ta đặt `./` ở đầu tên file.
5. Chúng ta dùng toán tử `import()` để kích hoạt việc import động của module.
6. Việc import động xảy ra một cách bất đồng bộ, nên chúng ta có thể dùng hook `.then()` trên promise được trả về để được thông báo khi module sẵn sàng để sử dụng. Hàm được truyền vào `then()` sẽ được thực thi khi module được nạp hoàn toàn và `strings` sẽ là namespace của module được import động. Sau đó, chúng ta có thể truy cập `strings.HELLO` và in giá trị của nó ra console.

Now we can execute this script like this:

Bây giờ chúng ta có thể thực thi script này như thế này:

```
node main.js it
```

And we should see `Ciao mondo` being printed to our console.

Và chúng ta sẽ thấy `Ciao mondo` được in ra console của mình.

Module loading in depth — Nạp module một cách chuyên sâu

To understand how ESM actually works and how it can deal effectively with circular dependencies, we have to deep dive a little bit more into how JavaScript code is parsed and evaluated when using ES modules.

Để hiểu ESM thực sự hoạt động như thế nào và cách nó có thể xử lý phụ thuộc vòng một cách hiệu quả, chúng ta phải đi sâu hơn một chút vào cách code JavaScript được phân tích cú pháp và đánh giá khi dùng ES modules.

In this section, we will learn how ECMAScript modules are loaded, we will present the idea of read-only live bindings, and, finally, we will discuss an example with circular dependencies.

Trong phần này, chúng ta sẽ tìm hiểu cách các ECMAScript module được nạp, chúng ta sẽ trình bày ý tưởng về các read-only live binding, và cuối cùng, chúng ta sẽ bàn về một ví dụ với phụ thuộc vòng.

Loading phases — Các pha nạp (Loading phases)

The goal of the interpreter is to build a graph of all the necessary modules (a dependency graph).

Mục tiêu của trình thông dịch là xây dựng một đồ thị (graph) của tất cả các module cần thiết (một đồ thị phụ thuộc - dependency graph).

In generic terms, a dependency graph can be defined as a directed graph (nodejsdp.link/directed-graph) representing the dependencies of a group of objects. In the context of this section, when we refer to a dependency graph, we want to indicate the dependency relationship between ECMAScript modules. As we will see, using a dependency graph allows us to determine the order in which all the necessary modules should be loaded in a given project.

Nói một cách tổng quát, một đồ thị phụ thuộc có thể được định nghĩa là một đồ thị có hướng ([nodejsdp.link/directed-graph](https://nodejs.org/en/docs/glossary/#directed-graph)) biểu diễn các phụ thuộc của một nhóm đối tượng. Trong bối cảnh của phần này, khi chúng ta đề cập đến một đồ thị phụ thuộc, chúng ta muốn chỉ mối quan hệ phụ thuộc giữa các ECMAScript module. Như chúng ta sẽ thấy, dùng một đồ thị phụ thuộc cho phép chúng ta xác định thứ tự mà tất cả các module cần thiết nên được nạp trong một dự án nhất định.

Essentially, the dependency graph is needed by the interpreter to figure out how modules depend on each other and in what order the code needs to be executed. When the node interpreter is launched, it gets passed some code to execute, generally in the form of a JavaScript file. This file is the starting point for the dependency resolution, and it is called the entry point. From the entry point, the interpreter will find and follow all the `import` statements recursively in a depth-first fashion, until all the necessary code is explored and then evaluated.

Về cơ bản, đồ thị phụ thuộc được trình thông dịch cần đến để tìm ra cách các module phụ thuộc vào nhau và theo thứ tự nào code cần được thực thi. Khi trình thông dịch node được khởi chạy, nó được truyền vào một số code để thực thi, nhìn chung là dưới dạng một file JavaScript. File này là điểm khởi đầu cho việc phân giải phụ thuộc, và nó được gọi là điểm vào (entry point). Từ điểm vào, trình thông dịch sẽ tìm và đi theo tất cả các câu lệnh `import` một cách đệ quy theo kiểu duyệt theo chiều sâu (depth-first), cho đến khi tất cả code cần thiết được khám phá và sau đó được đánh giá.

More specifically, this process happens in three separate phases:

Cụ thể hơn, quá trình này xảy ra trong ba pha riêng biệt:

- Phase 1 - Construction (or parsing): Find all the imports and recursively load the content of every module from the respective file.
 - Phase 2 - Instantiation: For every exported entity, keep a named reference in memory, but don't assign any value just yet. Also, references are created for all the `import` and `export` statements tracking the dependency relationship between them (linking). No JavaScript code has been executed at this stage.
 - Phase 3 - Evaluation: Node.js finally executes the code so that all the previously instantiated entities can get an actual value. Now running the code from the entry point is possible because all the blanks have been filled.
- Pha 1 - Construction (hay parsing): Tìm tất cả các `import` và nạp đệ quy nội dung của mỗi module từ file tương ứng.
 - Pha 2 - Instantiation: Đối với mỗi thực thể được xuất ra, giữ một tham chiếu có tên trong bộ nhớ, nhưng chưa gán bất kỳ giá trị nào. Ngoài ra, các tham chiếu được tạo cho tất cả các câu lệnh `import` và `export` theo dõi mối quan hệ phụ thuộc giữa chúng (linking). Chưa có code JavaScript nào được thực thi ở giai đoạn này.
 - Pha 3 - Evaluation: Node.js cuối cùng thực thi code để tất cả các thực thể đã được instantiate trước đó có thể nhận một giá trị thực sự. Bây giờ việc chạy code từ điểm vào là khả thi vì tất cả các chỗ trống đã được điền vào.

In simple terms, we could say that Phase 1 is about finding all the dots, Phase 2 connects those creating paths, and, finally, Phase 3 walks through the paths in the right order.

Nói một cách đơn giản, chúng ta có thể nói rằng Pha 1 là về việc tìm tất cả các điểm, Pha 2 nối các điểm đó để tạo ra các đường đi, và cuối cùng, Pha 3 đi qua các đường đi theo đúng thứ tự.

At first glance, this approach doesn't seem very different from what CommonJS does, but there's a fundamental difference. Due to its dynamic nature, CommonJS will execute all the files while the

dependency graph is explored. We have seen that every time a new `require` statement is found, all the previous code has already been executed. This is why you can use `require` even within `if` statements or loops, and construct module identifiers from variables.

Thoạt nhìn, cách tiếp cận này có vẻ không khác lắm so với những gì CommonJS làm, nhưng có một sự khác biệt cơ bản. Do bản chất động của nó, CommonJS sẽ thực thi tất cả các file trong khi đồ thị phụ thuộc được khám phá. Chúng ta đã thấy rằng mỗi khi một câu lệnh `require` mới được tìm thấy, tất cả code trước đó đã được thực thi rồi. Đây là lý do tại sao bạn có thể dùng `require` ngay cả bên trong các câu lệnh `if` hoặc vòng lặp, và dựng các module identifier từ các biến.

In ESM, these three phases are totally separate from each other, no code can be executed until the dependency graph has been fully built, and therefore module imports and exports have to be static.

Trong ESM, ba pha này hoàn toàn tách biệt với nhau, không code nào có thể được thực thi cho đến khi đồ thị phụ thuộc được xây dựng hoàn toàn, và do đó các module import và export phải là tĩnh.

Read-only live bindings — Read-only live binding

Another fundamental characteristic of ES modules, which helps with cyclic dependencies, is the idea that imported modules are effectively read-only live bindings to their exported values.

Một đặc điểm cơ bản khác của ES modules, vốn giúp ích cho phụ thuộc vòng, là ý tưởng rằng các module được import thực chất là các read-only live binding đến các giá trị được xuất ra của chúng.

Let's clarify what this means with a simple example:

Hãy làm rõ điều này nghĩa là gì với một ví dụ đơn giản:

```
// counter.js
export let count = 0
export function increment () {
  count++
}
```

This module exports two values: a simple integer counter called `count` and an `increment` function that increases the counter by one.

Module này xuất ra hai giá trị: một bộ đếm số nguyên đơn giản tên là `count` và một hàm `increment` làm tăng bộ đếm lên một.

Let's now write some code that uses this module:

Bây giờ hãy viết một số code dùng module này:

```
// main.js
import { count, increment } from './counter.js'
console.log(count) // prints 0
increment()
console.log(count) // prints 1
count++ // TypeError: Assignment to constant variable!
```

What we can see in this code is that we can read the value of `count` at any time and change it using the `increment()` function, but as soon as we try to mutate the `count` variable directly, we get an error as if we were trying to mutate a `const` binding.

Những gì chúng ta có thể thấy trong code này là chúng ta có thể đọc giá trị của `count` vào bất kỳ lúc nào và thay đổi nó bằng hàm `increment()`, nhưng ngay khi chúng ta cố biến đổi (mutate) biến `count` một cách trực tiếp, chúng ta nhận được một lỗi như thể chúng ta đang cố biến đổi một binding `const`.

This proves that when an entity is imported in the scope, the binding to its original value cannot be changed (read-only binding) unless the bound value changes within the scope of the original module itself (live binding), which is outside the direct control of the consumer code.

Điều này chứng minh rằng khi một thực thể được import vào phạm vi, binding đến giá trị gốc của nó không thể bị thay đổi (read-only binding) trừ khi giá trị được bind thay đổi trong phạm vi của chính module gốc (live binding), điều nằm ngoài tầm kiểm soát trực tiếp của code tiêu thụ.

This approach is fundamentally different from CommonJS. In fact, in CommonJS, the entire exports object is copied (shallow copy) when required from a module. This means that, if the value of primitive variables like numbers or string is changed at a later time, the requiring module won't be able to see those changes.

Cách tiếp cận này khác biệt cơ bản so với CommonJS. Thực tế, trong CommonJS, toàn bộ đối tượng `exports` được sao chép (shallow copy) khi được require từ một module. Điều này có nghĩa là, nếu giá trị của các biến nguyên thủy như số hay chuỗi bị thay đổi vào một thời điểm muộn hơn, thì module đang require sẽ không thể thấy được những thay đổi đó.

Circular dependency resolution — Phân giải phụ thuộc vòng (Circular dependency resolution)

Now to close the circle, let's reimplement the circular dependency example we saw in the CommonJS modules section using the ESM syntax:

Bây giờ để khép lại vòng tròn, hãy hiện thực lại ví dụ phụ thuộc vòng mà chúng ta đã thấy trong phần module CommonJS bằng cú pháp ESM:

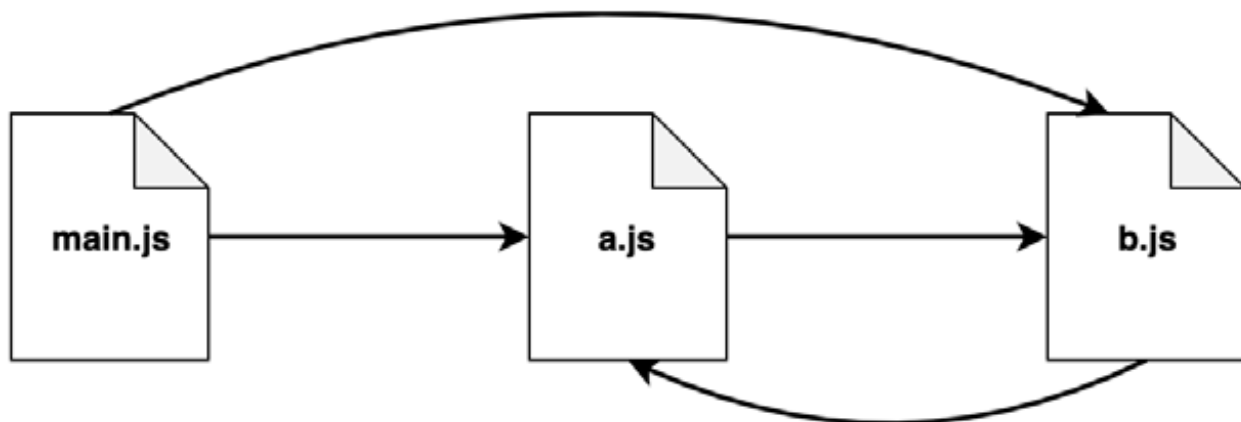


Figure 2.3: An example scenario with circular dependencies

Hình 2.3: Một kịch bản ví dụ với phụ thuộc vòng

Let's have a look at the modules `a.js` and `b.js` first:

Hãy xem các module `a.js` và `b.js` trước:

```
// a.js
import * as bModule from './b.js'
export let loaded = false
export const b = bModule
loaded = true
// b.js
import * as aModule from './a.js'
export let loaded = false
export const a = aModule
loaded = true
```

And now let's see how to import those two modules in our `main.js` file (the entry point):

Và bây giờ hãy xem cách import hai module này trong file `main.js` của chúng ta (điểm vào):

```
// main.js
import * as a from './a.js'
import * as b from './b.js'
console.log('a ->', a)
console.log('b ->', b)
```

Note that this time we are not using `JSON.stringify` because that will fail with a `TypeError: Converting circular structure to JSON`, since there's an actual circular reference between `a.js` and `b.js`.

Lưu ý rằng lần này chúng ta không dùng `JSON.stringify` bởi vì điều đó sẽ thất bại với lỗi `TypeError: Converting circular structure to JSON`, vì có một tham chiếu vòng thực sự giữa `a.js` và `b.js`.

When we run `main.js`, we will see the following output:

Khi chúng ta chạy `main.js`, chúng ta sẽ thấy kết quả sau:

```
a -> <ref *1> [Module] {
  b: [Module] { a: [Circular *1], loaded: true },
  loaded: true
}
b -> <ref *1> [Module] {
  a: [Module] { b: [Circular *1], loaded: true },
  loaded: true
}
```

The interesting bit here is that the modules `a.js` and `b.js` have a complete picture of each other, unlike what would happen with CommonJS, where they would only hold partial information of each other. We can see that because all the loaded values are set to `true`. Also, `b` within `a` is an actual reference to the same `b` instance available in the current scope, and the same goes for `a` within `b`. That's the reason why we cannot use `JSON.stringify()` to serialize these modules. Finally, if we swap the order of the imports for the modules `a.js` and `b.js`, the final outcome does not change, which is another important difference in comparison with how CommonJS works

Điểm thú vị ở đây là các module `a.js` và `b.js` có một bức tranh đầy đủ về nhau, không giống như những gì sẽ xảy ra với CommonJS, nơi chúng sẽ chỉ giữ thông tin không đầy đủ về nhau. Chúng ta có thể thấy điều đó vì tất cả các giá trị `loaded` đều được đặt thành `true`. Ngoài ra, `b` bên trong `a` là một tham chiếu thực sự đến cùng instance `b` có sẵn trong

phạm vi hiện tại, và điều tương tự cũng đúng với a bên trong b. Đó là lý do tại sao chúng ta không thể dùng `JSON.stringify()` để tuần tự hóa các module này. Cuối cùng, nếu chúng ta hoán đổi thứ tự của các import cho các module a.js và b.js, kết quả cuối cùng không thay đổi, đây là một khác biệt quan trọng khác so với cách CommonJS hoạt động

It's worth spending some more time observing what happens in the three phases of the module resolution (parsing, instantiation, and evaluation) for this specific example.

Đáng để dành thêm chút thời gian quan sát những gì xảy ra trong ba pha của việc phân giải module (parsing, instantiation và evaluation) cho ví dụ cụ thể này.

Phase 1: Parsing — Pha 1: Parsing During the parsing phase, the code is explored starting from the entry point (`main.js`). The interpreter looks only for `import` statements to find all the necessary modules and to load the source code from the module files. The dependency graph is explored in a depth-first fashion, and every module is visited only once. This way the interpreter builds a view of the dependencies that looks like a tree structure, as shown in Figure 2.4:

Trong pha parsing, code được khám phá bắt đầu từ điểm vào (`main.js`). Trình thông dịch chỉ tìm các câu lệnh `import` để tìm tất cả các module cần thiết và nạp mã nguồn từ các file module. Đồ thị phụ thuộc được khám phá theo kiểu duyệt theo chiều sâu, và mỗi module chỉ được thăm một lần. Theo cách này, trình thông dịch xây dựng một góc nhìn về các phụ thuộc trông giống như một cấu trúc cây, như được thể hiện trong Hình 2.4:

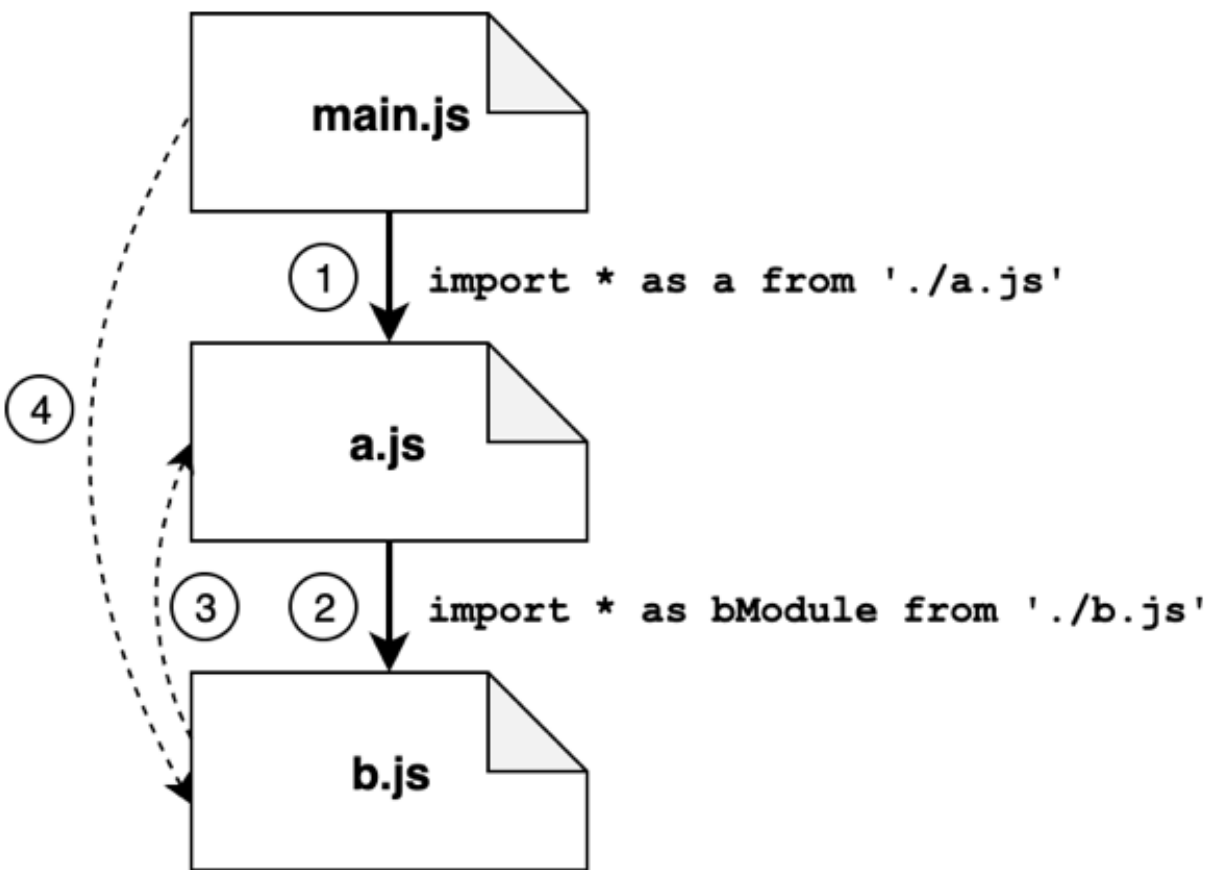


Figure 2.4: Parsing of cyclic dependencies with ESM

Hình 2.4: Parsing các phụ thuộc vòng với ESM

Given the example in Figure 2.4, let's discuss the various steps of the parsing phase:

Với ví dụ trong Hình 2.4, hãy bàn về các bước khác nhau của pha parsing:

1. From `main.js`, the first import found leads us straight into `a.js`.
2. In `a.js` we find an import pointing to `b.js`.
3. In `b.js`, we also have an import back to `a.js` (our cycle), but since `a.js` has already been visited, this path is not explored again.
4. At this point, the exploration starts to wind back: `b.js` doesn't have other imports, so we go back to `a.js`; `a.js` doesn't have other import statements so we go back to `main.js`. Here we find another import pointing to `b.js`, but again this module has been explored already, so this path is ignored.

1. Từ `main.js`, import đầu tiên được tìm thấy dẫn chúng ta thẳng vào `a.js`.
2. Trong `a.js` chúng ta tìm thấy một import trỏ đến `b.js`.
3. Trong `b.js`, chúng ta cũng có một import trở lại `a.js` (vòng lặp của chúng ta), nhưng vì `a.js` đã được thăm rồi, đường đi này không được khám phá lại.
4. Tại thời điểm này, việc khám phá bắt đầu lùi lại: `b.js` không có import nào khác, nên chúng ta quay lại `a.js`; `a.js` không có câu lệnh `import` nào khác nên chúng ta quay lại `main.js`. Ở đây chúng ta tìm thấy một import khác trỏ đến `b.js`, nhưng một lần nữa module này đã được khám phá rồi, nên đường đi này bị bỏ qua.

At this point, our depth-first visit of the dependency graph has been completed and we have a linear view of the modules, as shown in Figure 2.5:

Tại thời điểm này, việc duyệt theo chiều sâu đồ thị phụ thuộc của chúng ta đã hoàn tất và chúng ta có một góc nhìn tuyến tính về các module, như được thể hiện trong Hình 2.5:

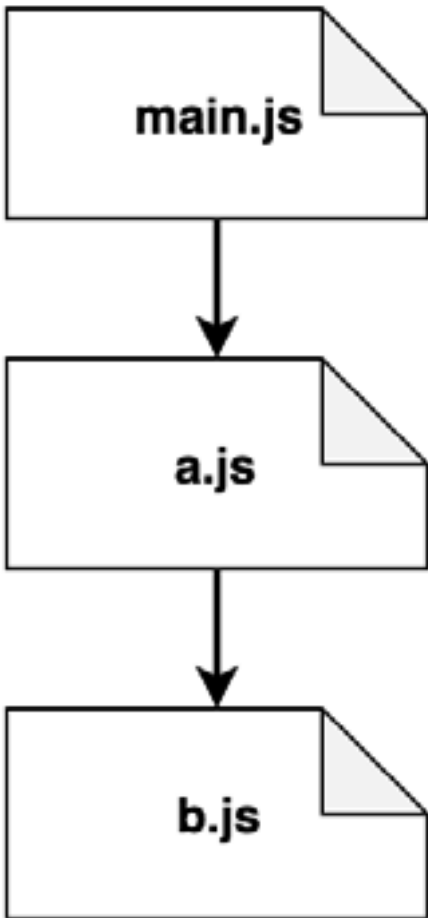


Figure 2.5: A linear view of the module graph where cycles have been removed

Hình 2.5: Một góc nhìn tuyến tính về đồ thị module nơi các vòng đã được loại bỏ

This particular view is quite simple. In more realistic scenarios with a lot more modules, the view will look more like a tree structure.

Góc nhìn cụ thể này khá đơn giản. Trong các kịch bản thực tế hơn với nhiều module hơn rất nhiều, góc nhìn sẽ trông giống một cấu trúc cây hơn.

Phase 2: Instantiation — Pha 2: Instantiation In the instantiation phase, the interpreter walks the tree view obtained from the previous phase from the bottom to the top. For every module, the interpreter will look for all the exported properties first and build out a map of the exported names in memory:

Trong pha instantiation, trình thông dịch đi qua góc nhìn dạng cây thu được từ pha trước từ dưới lên trên. Đối với mỗi module, trình thông dịch sẽ tìm tất cả các thuộc tính được xuất ra trước và xây dựng một bản đồ (map) các tên được xuất ra trong bộ nhớ:

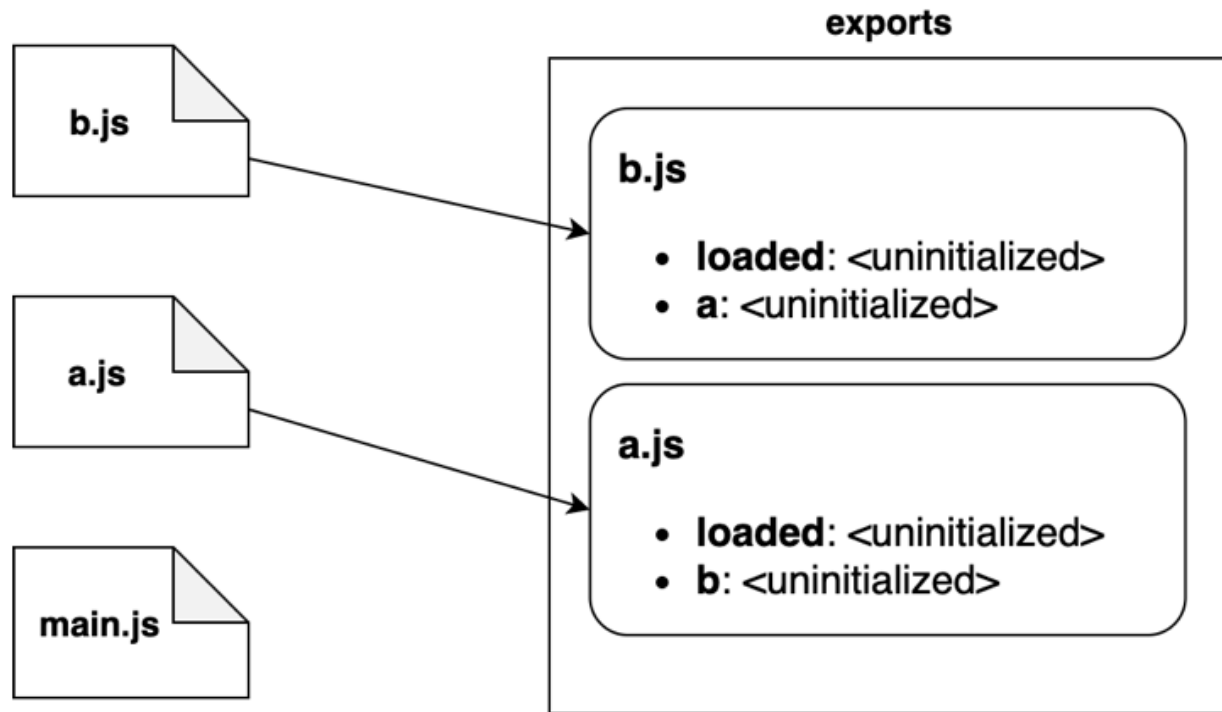


Figure 2.6: A visual representation of the instantiation phase Figure 2.6 describes the order in which every module is instantiated:

Hình 2.6: Một biểu diễn trực quan về pha instantiation. Hình 2.6 mô tả thứ tự mà mỗi module được instantiate:

1. The interpreter starts from `b.js` and discovers that the module exports `loaded` and `a`.
 2. Then, the interpreter moves to `a.js`, which exports `loaded` and `b`.
 3. Finally, it moves to `main.js`, which does not export any functionality.
 4. Note that, in this phase, the exports map keeps track of the exported names only; their associated values are considered uninitialized for now.
1. Trình thông dịch bắt đầu từ `b.js` và phát hiện rằng module xuất ra `loaded` và `a`.
 2. Sau đó, trình thông dịch chuyển sang `a.js`, vốn xuất ra `loaded` và `b`.
 3. Cuối cùng, nó chuyển sang `main.js`, vốn không xuất ra bất kỳ chức năng nào.
 4. Lưu ý rằng, trong pha này, bản đồ exports chỉ theo dõi các tên được xuất ra; các giá trị liên quan của chúng hiện tại được coi là chưa khởi tạo.

After this sequence of steps, the interpreter will do another pass to link the exported names to the modules importing them, as shown in Figure 2.7:

Sau chuỗi các bước này, trình thông dịch sẽ thực hiện thêm một lượt nữa để liên kết các tên được xuất ra với các module import chúng, như được thể hiện trong Hình 2.7:

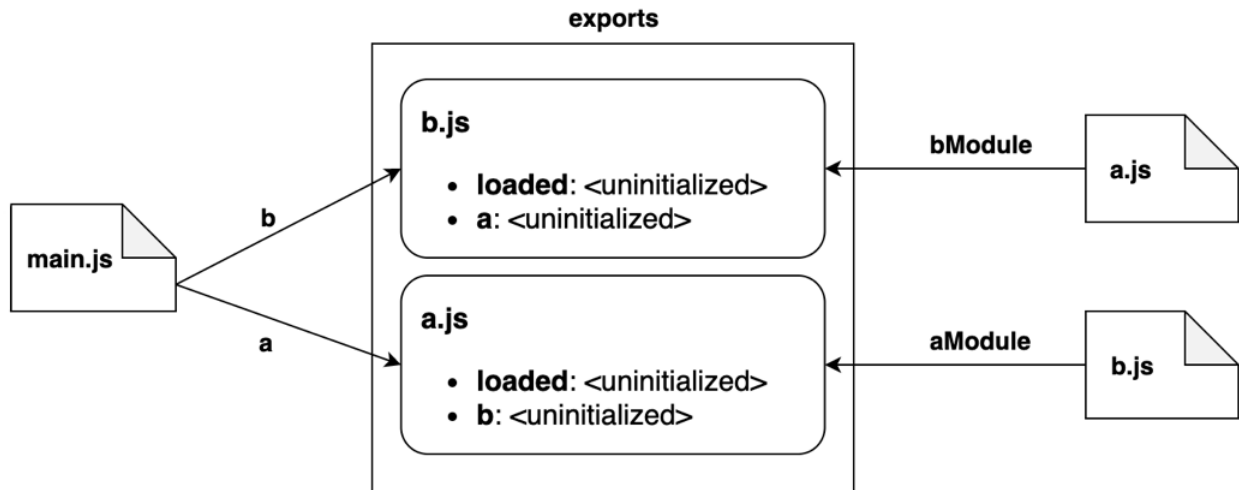


Figure 2.7: Linking exports with imports across modules

Hình 2.7: Liên kết exports với imports giữa các module

We can describe what we see in Figure 2.7 through the following steps:

Chúng ta có thể mô tả những gì chúng ta thấy trong Hình 2.7 thông qua các bước sau:

1. Module `b.js` will link the exports from `a.js`, referring to them as `aModule`.
2. In turn, `a.js` will link to all the exports from `b.js`, referring to them as `bModule`.
3. Finally, `main.js` will import all the exports in `b.js`, referring to them as `b`; similarly, it will import everything from `a.js`, referring to them as `a`.
4. Again, it's important to note that all the values are still uninitialized. In this phase, we are only linking references to values that will be available at the end of the next phase.
 1. Module `b.js` sẽ liên kết các exports từ `a.js`, tham chiếu đến chúng dưới dạng `aModule`.
 2. Đến lượt mình, `a.js` sẽ liên kết đến tất cả các exports từ `b.js`, tham chiếu đến chúng dưới dạng `bModule`.
 3. Cuối cùng, `main.js` sẽ import tất cả các exports trong `b.js`, tham chiếu đến chúng dưới dạng `b`; tương tự, nó sẽ import mọi thứ từ `a.js`, tham chiếu đến chúng dưới dạng `a`.
 4. Một lần nữa, điều quan trọng cần lưu ý là tất cả các giá trị vẫn chưa được khởi tạo. Trong pha này, chúng ta chỉ đang liên kết các tham chiếu đến các giá trị sẽ có sẵn vào cuối pha tiếp theo.

Phase 3: Evaluation — Pha 3: Evaluation The last step is the evaluation phase. In this phase, all the code in every file is finally executed. The execution order is again bottom-up respecting the post-order depthfirst visit of our original dependency graph. With this approach, `main.js` is the last file to be executed. This way, we can be sure that all the exported values have been initialized before we start executing our main business logic:

Bước cuối cùng là pha evaluation. Trong pha này, tất cả code trong mọi file cuối cùng được thực thi. Thứ tự thực thi một lần nữa là từ dưới lên trên, tôn trọng việc duyệt theo chiều sâu hậu thứ tự (post-order depth-first) đồ thị phụ thuộc gốc của chúng ta. Với cách tiếp cận này, `main.js` là file cuối cùng được thực thi. Theo cách này, chúng ta có thể chắc chắn rằng tất cả các giá trị được xuất ra đã được khởi tạo trước khi chúng ta bắt đầu thực thi

logic nghiệp vụ chính của mình:

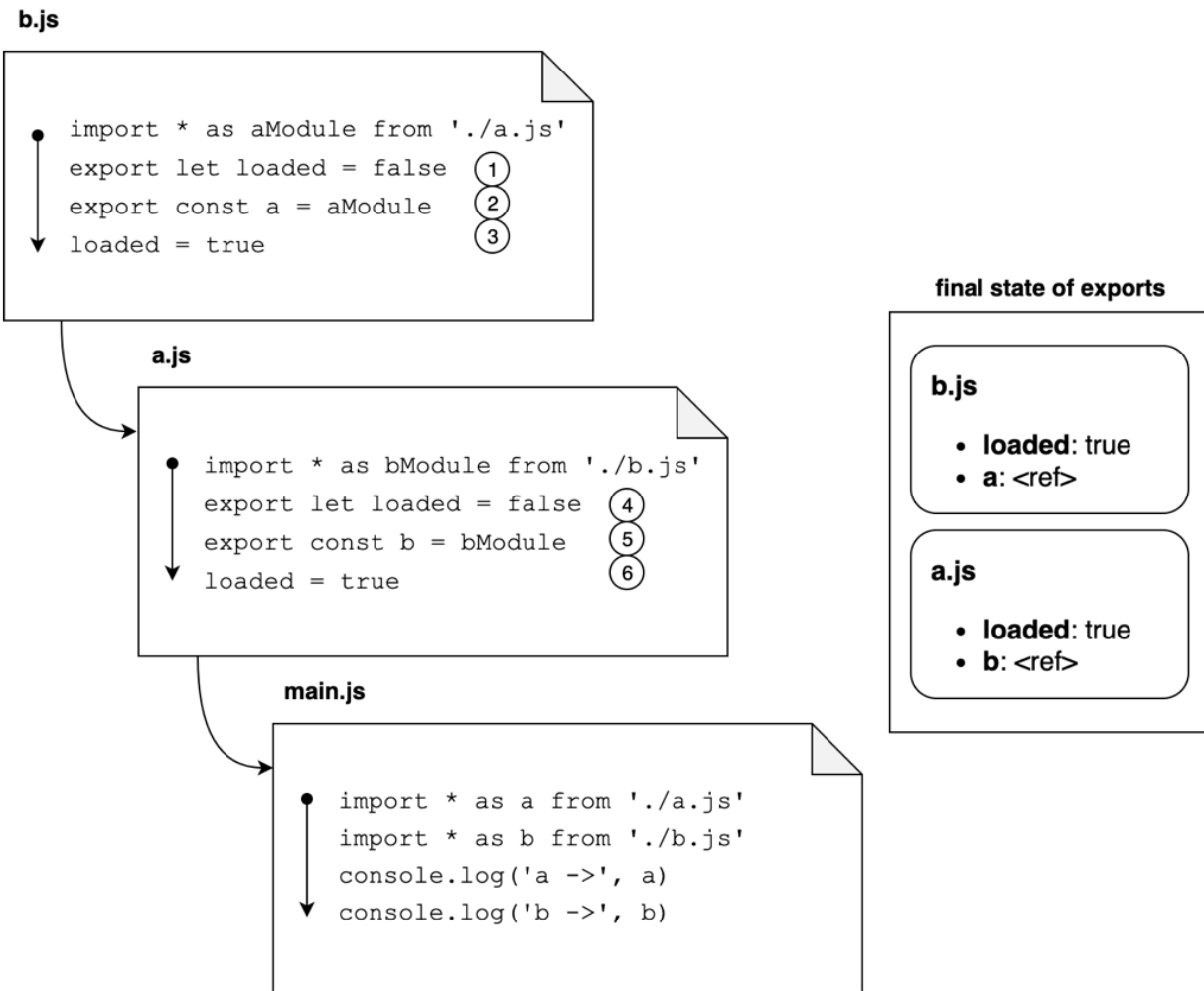


Figure 2.8: A visual representation of the evaluation phase

Hình 2.8: Một biểu diễn trực quan về pha evaluation

Following along from the diagram in Figure 2.8, this is what happens:

Theo dõi từ sơ đồ trong Hình 2.8, đây là những gì xảy ra:

1. The execution starts from `b.js` and the first line to be evaluated initializes the `loaded` export to `false` for the module.
2. Similarly, here the exported property `a` gets evaluated. This time, it will be evaluated to a reference to the module object representing module `a.js`.
3. The value of the `loaded` property gets changed to `true`. At this point, we have fully evaluated the state of the exports for module `b.js`.
4. Now the execution moves to `a.js`. Again, we start by setting `loaded` to `false`.
5. At this point, the `b` export is evaluated to a reference to module `b.js`.
6. Finally, the `loaded` property is changed to `true`. Now we have finally evaluated all the exports for `a.js` as well.

1. Việc thực thi bắt đầu từ `b.js` và dòng đầu tiên được đánh giá khởi tạo export `loaded` thành `false` cho module.

2. Tương tự, ở đây thuộc tính được xuất ra `a` được đánh giá. Lần này, nó sẽ được đánh giá thành một tham chiếu đến đối tượng module đại diện cho module `a.js`.
3. Giá trị của thuộc tính `loaded` được đổi thành `true`. Tại thời điểm này, chúng ta đã đánh giá hoàn toàn trạng thái của các exports cho module `b.js`.
4. Bây giờ việc thực thi chuyển sang `a.js`. Một lần nữa, chúng ta bắt đầu bằng việc đặt `loaded` thành `false`.
5. Tại thời điểm này, export `b` được đánh giá thành một tham chiếu đến module `b.js`.
6. Cuối cùng, thuộc tính `loaded` được đổi thành `true`. Bây giờ chúng ta cuối cùng đã đánh giá tất cả các exports cho `a.js` nữa.

After all these steps, the code in `main.js` can be executed, and at this point, all the exported properties are fully evaluated. Since imported modules are tracked as references, we can be sure every module has an up-to-date picture of the other modules, even in the presence of circular dependencies.

Sau tất cả các bước này, code trong `main.js` có thể được thực thi, và tại thời điểm này, tất cả các thuộc tính được xuất ra đã được đánh giá hoàn toàn. Vì các module được import được theo dõi dưới dạng các tham chiếu, chúng ta có thể chắc chắn rằng mỗi module có một bức tranh cập nhật về các module khác, ngay cả khi có sự hiện diện của phụ thuộc vòng.

Modifying other modules — Sửa đổi các module khác

We saw that entities imported through ES modules are read-only live bindings, and therefore we cannot reassign them from an external module.

Chúng ta đã thấy rằng các thực thể được import thông qua ES modules là các read-only live binding, và do đó chúng ta không thể gán lại chúng từ một module bên ngoài.

There's a caveat, though. It is true that we can't change the bindings of the default export or named exports of an existing module from another module, but, if one of these bindings is an object, we can still mutate the object itself by reassigning some of the object properties.

Tuy nhiên, có một điều cần lưu ý. Đúng là chúng ta không thể thay đổi các binding của default export hoặc named exports của một module hiện có từ một module khác, nhưng, nếu một trong các binding này là một đối tượng, chúng ta vẫn có thể biến đổi (mutate) chính đối tượng đó bằng cách gán lại một số thuộc tính của đối tượng.

This caveat can give us enough freedom to alter the behavior of other modules. To demonstrate this idea, let's write a module that can alter the behavior of the core `fs` module so that it prevents the module from accessing the filesystem and returns mocked data instead. This kind of module is something that could be useful while writing tests for a component that relies on the filesystem:

Điều cần lưu ý này có thể cho chúng ta đủ tự do để thay đổi hành vi của các module khác. Để minh họa ý tưởng này, hãy viết một module có thể thay đổi hành vi của module lõi `fs` để nó ngăn module truy cập hệ thống tập tin và trả về dữ liệu giả lập (mocked) thay thế. Loại module này là một thứ có thể hữu ích khi viết các bài kiểm thử cho một thành phần dựa vào hệ thống tập tin:

```
// mock-read-file.js
import fs from 'fs' // (1)
const originalReadFile = fs.readFile // (2)
let mockedResponse = null
function mockedReadFile (path, cb) { // (3)
```

```

    setImmediate(() => {
      cb(null, mockedResponse)
    })
  }
  export function mockEnable (respondWith) { // (4)
    mockedResponse = respondWith
    fs.readFile = mockedReadFile
  }
  export function mockDisable () { // (5)
    fs.readFile = originalReadFile
  }

```

Let's review the preceding code:

Hãy xem lại đoạn code trên:

1. The first thing we do is import the default export of the `fs` module. We will get back to this in a second, for now, just keep in mind that the default export of the `fs` module is an object that contains a collection of functions that allows us to interact with the filesystem.
 2. We want to replace the `readFile()` function with a mock implementation. Before doing that, we save a reference to the original implementation. We also declare a `mockedResponse` value that we will be using later.
 3. The function `mockedReadFile()` is the actual mocked implementation that we want to use to replace the original implementation. This function invokes the callback with the current value of `mockedResponse`. Note that this is a simplified implementation; the real function accepts an optional `options` argument before the callback argument and is able to handle different types of encoding.
 4. The exported `mockEnable()` function can be used to activate the mocked functionality. The original implementation will be swapped with the mocked one. The mocked implementation will return the same value passed here through the `respondWith` argument.
 5. Finally, the exported `mockDisable()` function can be used to restore the original implementation of the `fs.readFile()` function.
1. Điều đầu tiên chúng ta làm là import default export của module `fs`. Chúng ta sẽ quay lại điều này sau một giây, hiện tại, chỉ cần nhớ rằng default export của module `fs` là một đối tượng chứa một tập các hàm cho phép chúng ta tương tác với hệ thống tập tin.
 2. Chúng ta muốn thay thế hàm `readFile()` bằng một hiện thực giả lập. Trước khi làm điều đó, chúng ta lưu một tham chiếu đến hiện thực gốc. Chúng ta cũng khai báo một giá trị `mockedResponse` mà chúng ta sẽ dùng sau.
 3. Hàm `mockedReadFile()` là hiện thực giả lập thực tế mà chúng ta muốn dùng để thay thế hiện thực gốc. Hàm này gọi callback với giá trị hiện tại của `mockedResponse`. Lưu ý rằng đây là một hiện thực đơn giản hóa; hàm thực sự nhận một đối số `options` tùy chọn trước đối số callback và có thể xử lý các loại mã hóa (encoding) khác nhau.
 4. Hàm được xuất ra `mockEnable()` có thể được dùng để kích hoạt chức năng giả lập. Hiện thực gốc sẽ được hoán đổi bằng hiện thực giả lập. Hiện thực giả lập sẽ trả về cùng giá trị được truyền vào đây thông qua đối số `respondWith`.
 5. Cuối cùng, hàm được xuất ra `mockDisable()` có thể được dùng để khôi phục hiện thực gốc của hàm `fs.readFile()`.

Now let's see a simple example that uses this module:

Bây giờ hãy xem một ví dụ đơn giản dùng module này:

```
// main.js
import fs from 'fs' // (1)
import { mockEnable, mockDisable } from './mock-read-file.js'
mockEnable(Buffer.from('Hello World')) // (2)
fs.readFile('fake-path', (err, data) => { // (3)
  if (err) {
    console.error(err)
    process.exit(1)
  }
  console.log(data.toString()) // 'Hello World'
})
mockDisable()
```

Let's discuss step by step what happens in this example:

Hãy bàn từng bước những gì xảy ra trong ví dụ này:

1. The first thing that we do is import the default export of the `fs` module. Again, note that we are importing specifically the default export exactly as we did in our `mock-read-file.js` module, but more on this later.
2. Here we enable the mock functionality. We want, for every file read, to simulate that the file contains the string “Hello World.”
3. Finally, we read a file using a fake path. This code will print “Hello World” as it will be using the mocked version of the `readFile()` function. Note that, after calling this function, we restore the original implementation by calling `mockDisable()`.

1. Điều đầu tiên chúng ta làm là import default export của module `fs`. Một lần nữa, lưu ý rằng chúng ta đang import cụ thể default export đúng như cách chúng ta đã làm trong module `mock-read-file.js` của mình, nhưng sẽ nói thêm về điều này sau.
2. Ở đây chúng ta kích hoạt chức năng giả lập. Chúng ta muốn, đối với mỗi lần đọc file, mô phỏng rằng file chứa chuỗi “Hello World.”
3. Cuối cùng, chúng ta đọc một file bằng một đường dẫn giả. Code này sẽ in ra “Hello World” vì nó sẽ dùng phiên bản giả lập của hàm `readFile()`. Lưu ý rằng, sau khi gọi hàm này, chúng ta khôi phục hiện thực gốc bằng cách gọi `mockDisable()`.

This approach works, but it is very fragile. In fact, there are a number of ways in which this may not work.

Cách tiếp cận này hoạt động, nhưng nó rất mong manh. Thực tế, có một số cách mà điều này có thể không hoạt động.

On the `mock-read-file.js` side, we could have tried the two following imports for the `fs` module:

Về phía `mock-read-file.js`, chúng ta có thể đã thử hai cách import sau cho module `fs`:

```
import * as fs from 'fs' // then use fs.readFile
```

or

hoặc

```
import { readFile } from 'fs'
```

Both of them are valid imports because the `fs` module exports all the filesystem functions as named exports (other than a default export which is an object with the same collection of functions as attributes).

Cả hai đều là các import hợp lệ vì module `fs` xuất ra tất cả các hàm hệ thống tập tin dưới dạng named exports (ngoài một default export vốn là một đối tượng có cùng tập các hàm dưới dạng các thuộc tính).

There are certain issues with the preceding two import statements:

Có một số vấn đề nhất định với hai câu lệnh import trên:

- We would get a read-only live binding into the `readFile()` function, and therefore, we would be unable to mutate it from an external module. If we try these approaches, we will get an error when trying to reassign `readFile()`.
- Another issue is on the consumer side within our `main.js`, where we could use these two alternative import styles as well. In this case, we won't end up using the mocked functionality, and therefore the code will trigger an error while trying to read a nonexistent file.
 - Chúng ta sẽ nhận được một read-only live binding vào hàm `readFile()`, và do đó, chúng ta sẽ không thể biến đổi nó từ một module bên ngoài. Nếu chúng ta thử các cách tiếp cận này, chúng ta sẽ nhận được một lỗi khi cố gán lại `readFile()`.
 - Một vấn đề khác là ở phía bên tiêu thụ trong `main.js` của chúng ta, nơi chúng ta cũng có thể dùng hai kiểu import thay thế này. Trong trường hợp này, chúng ta sẽ không dùng đến chức năng giả lập, và do đó code sẽ kích hoạt một lỗi khi cố đọc một file không tồn tại.

The reason why using one of the two import statements mentioned above would not work is because our mocking utility is altering only the copy of the `readFile()` function that is registered inside the object exported as the default export, but not the one available as a named export at the top level of the module. This particular example shows us how monkey patching could be much more complicated and unreliable in the context of ESM. For this reason, testing frameworks such as Jest (nodejsdp.link/jest) provide special functionalities to be able to mock ES modules more reliably (nodejsdp.link/jest-mock).

Lý do tại sao việc dùng một trong hai câu lệnh import được đề cập ở trên sẽ không hoạt động là vì tiện ích giả lập của chúng ta chỉ đang thay đổi bản sao của hàm `readFile()` được đăng ký bên trong đối tượng được xuất ra dưới dạng default export, chứ không phải bản có sẵn dưới dạng một named export ở cấp cao nhất của module. Ví dụ cụ thể này cho chúng ta thấy monkey patching có thể phức tạp và kém tin cậy hơn nhiều như thế nào trong bối cảnh ESM. Vì lý do này, các framework kiểm thử như Jest (nodejsdp.link/jest) cung cấp các chức năng đặc biệt để có thể giả lập các ES module một cách đáng tin cậy hơn (nodejsdp.link/jest-mock).

Another approach that can be used to mock modules is to rely on the hooks available in a special Node.js core module called `module` (nodejsdp.link/module-doc). One simple library that takes advantage of this module is `mocku` (nodejsdp.link/mocku). Check out its source code if you are curious.

Một cách tiếp cận khác có thể được dùng để giả lập các module là dựa vào các hook có sẵn trong một module lõi đặc biệt của Node.js gọi là `module` (nodejsdp.link/module-doc). Một thư viện đơn giản tận dụng module này là `mocku` (nodejsdp.link/mocku). Hãy xem mã nguồn của nó nếu bạn tò mò.

We could also use the `syncBuiltinESMExports()` function from the `module` package. When this function is invoked, the value of the properties in the default exports object gets mapped again into the equivalent named exports, effectively allowing us to propagate any external change applied to the module functionality even to named exports:

Chúng ta cũng có thể dùng hàm `syncBuiltinESMExports()` từ gói `module`. Khi hàm này được gọi, giá trị của các thuộc tính trong đối tượng `default exports` được ánh xạ lại vào các `named exports` tương đương, qua đó hiệu quả cho phép chúng ta lan truyền bất kỳ thay đổi bên ngoài nào được áp dụng cho chức năng module ngay cả đến các `named exports`:

```
import fs, { readFileSync } from 'fs'
import { syncBuiltinESMExports } from 'module'
fs.readFileSync = () => Buffer.from('Hello, ESM')
syncBuiltinESMExports()
console.log(fs.readFileSync === readFileSync) // true
```

We could use this to make our small filesystem mocking utility a little bit more flexible by invoking the `syncBuiltinESMExports()` function after we enable the mock or after we restore the original functionality.

Chúng ta có thể dùng điều này để làm cho tiện ích giả lập hệ thống tập tin nhỏ của mình linh hoạt hơn một chút bằng cách gọi hàm `syncBuiltinESMExports()` sau khi chúng ta kích hoạt giả lập hoặc sau khi chúng ta khôi phục chức năng gốc.

Note that `syncBuiltinESMExports()` works only for built-in Node.js modules like the `fs` module in our example.

Lưu ý rằng `syncBuiltinESMExports()` chỉ hoạt động cho các module dựng sẵn của Node.js như module `fs` trong ví dụ của chúng ta.

This concludes our exploration of ESM. At this point, we should be able to appreciate how ESM works, how it loads modules, and how it deals with cyclic dependencies. To close this chapter, we are now ready to discuss some key differences and some interesting interoperability techniques between CommonJS and ECMAScript modules.

Điều này kết thúc việc khám phá ESM của chúng ta. Tại thời điểm này, chúng ta hẳn đã có thể nhận thức được ESM hoạt động như thế nào, cách nó nạp module, và cách nó xử lý phụ thuộc vòng. Để khép lại chương này, bây giờ chúng ta đã sẵn sàng để bàn về một số khác biệt then chốt và một số kỹ thuật tương tác thú vị giữa CommonJS và ECMAScript modules.

ESM and CommonJS differences and interoperability — Sự khác biệt và khả năng tương tác giữa ESM và CommonJS

We already mentioned several important differences between ESM and CommonJS, such as having to explicitly specify file extensions in imports with ESM, while file extensions are totally optional with the CommonJS `require` function.

Chúng ta đã đề cập đến một số khác biệt quan trọng giữa ESM và CommonJS, chẳng hạn như việc phải chỉ định tường minh phần mở rộng file trong các `import` với ESM, trong khi phần mở rộng file là hoàn toàn tùy chọn với hàm `require` của CommonJS.

Let's close this chapter by discussing some other important differences between ESM and CommonJS and how the two module systems can work together when necessary.

Hãy khép lại chương này bằng cách bàn về một số khác biệt quan trọng khác giữa ESM và CommonJS và cách hai hệ thống module có thể làm việc cùng nhau khi cần thiết.

ESM runs in strict mode — ESM chạy ở chế độ strict

ES modules run implicitly in strict mode. This means that we don't have to explicitly add the "use strict" statements at the beginning of every file. Strict mode cannot be disabled; therefore, we cannot use undeclared variables or the with statement or have other features that are only available in non-strict mode, but this is definitely a good thing, as strict mode is a safer execution mode.

ES modules chạy ngầm định ở chế độ strict. Điều này có nghĩa là chúng ta không phải thêm tường minh các câu lệnh "use strict" vào đầu mỗi file. Chế độ strict không thể bị tắt; do đó, chúng ta không thể dùng các biến chưa khai báo hoặc câu lệnh with hoặc có các tính năng khác chỉ có sẵn ở chế độ non-strict, nhưng đây chắc chắn là một điều tốt, vì chế độ strict là một chế độ thực thi an toàn hơn.

If you are curious to find out more about the differences between the two modes, you can check out a very detailed article on MDN Web Docs (<https://nodejsdp.link/strict-mode>).

Nếu bạn tò mò muốn tìm hiểu thêm về sự khác biệt giữa hai chế độ, bạn có thể xem một bài viết rất chi tiết trên MDN Web Docs (<https://nodejsdp.link/strict-mode>).

Missing references in ESM — Các tham chiếu bị thiếu trong ESM

In ESM, some important CommonJS references are not defined. These include `require`, `exports`, `module.exports`, `__filename`, and `__dirname`. If we try to use any of them within an ES module, since it also runs in strict mode, we will get a `ReferenceError`:

Trong ESM, một số tham chiếu quan trọng của CommonJS không được định nghĩa. Những tham chiếu này bao gồm `require`, `exports`, `module.exports`, `__filename` và `__dirname`. Nếu chúng ta thử dùng bất kỳ tham chiếu nào trong số chúng bên trong một ES module, vì nó cũng chạy ở chế độ strict, chúng ta sẽ nhận được một `ReferenceError`:

```
console.log(exports) // ReferenceError: exports is not defined
console.log(module) // ReferenceError: module is not defined
console.log(__filename) // ReferenceError: __filename is not defined
console.log(__dirname) // ReferenceError: __dirname is not defined
```

We already discussed at length the meaning of `exports` and `module` in CommonJS; `__filename` and `__dirname` represent the absolute path to the current module file and the absolute path to its parent folder. Those special variables can be very useful when we need to build a path relative to the current file. In ESM, it is possible to get a reference to the current file URL by using the special object `import.meta`. Specifically, `import.meta.url` is a reference to the current module file in a format similar to `file:///path/to/current_module.js`. This value can be used to reconstruct `__filename` and `__dirname` in the form of absolute paths:

Chúng ta đã bàn rất kỹ về ý nghĩa của `exports` và `module` trong CommonJS; `__filename` và `__dirname` đại diện cho đường dẫn tuyệt đối đến file module hiện tại và đường dẫn tuyệt đối đến thư mục cha của nó. Những biến đặc biệt này có thể rất hữu ích khi chúng ta cần dựng một đường dẫn tương đối với file hiện tại. Trong ESM, có thể lấy một tham chiếu đến URL của file hiện tại bằng cách dùng đối tượng đặc biệt `import.meta`. Cụ thể, `import.meta.url` là một tham chiếu đến file module hiện tại theo định dạng tương tự như `file:///path/to/current_module.js`. Giá trị này có thể được dùng để tái dựng `__filename` và `__dirname` dưới dạng các đường dẫn tuyệt đối:

```
import { fileURLToPath } from 'url'
import { dirname } from 'path'
```



```
const __filename = fileURLToPath(import.meta.url)
const __dirname = dirname(__filename)
```

It is also possible to recreate the `require()` function as follows:

Cũng có thể tái tạo hàm `require()` như sau:

```
import { createRequire } from 'module'
const require = createRequire(import.meta.url)
```

Now we can use `require()` to import functionality coming from CommonJS modules in the context of ES modules.

Bây giờ chúng ta có thể dùng `require()` để import chức năng đến từ các module CommonJS trong bối cảnh của các ES module.

Another interesting difference is the behavior of the `this` keyword.

Một khác biệt thú vị khác là hành vi của từ khóa `this`.

In the global scope of an ES module, `this` is undefined, while in CommonJS, `this` is a reference to `exports`:

Trong phạm vi toàn cục của một ES module, `this` là undefined, trong khi trong CommonJS, `this` là một tham chiếu đến `exports`:

```
// this.js - ESM
console.log(this) // undefined
// this.cjs - CommonJS
console.log(this === exports) // true
```

Interoperability — Khả năng tương tác (Interoperability)

We discussed in the previous section how to import CommonJS modules in ESM by using the `module.createRequire` function. It is also possible to import CommonJS modules from ESM by using the standard `import` syntax. This is only limited to default exports, though:

Chúng ta đã bàn trong phần trước về cách import các module CommonJS trong ESM bằng cách dùng hàm `module.createRequire`. Cũng có thể import các module CommonJS từ ESM bằng cách dùng cú pháp import chuẩn. Tuy nhiên, điều này chỉ giới hạn ở các default export:

```
import packageMain from 'commonjs-package' // Works
import { method } from 'commonjs-package' // Errors
```

Unfortunately, it is not possible to import ES modules from CommonJS modules. Also, ESM cannot import JSON files directly as modules, a feature that is used quite frequently with CommonJS. The following import statement will fail:

Đáng tiếc, không thể import các ES module từ các module CommonJS. Ngoài ra, ESM không thể import các file JSON trực tiếp dưới dạng module, một tính năng được dùng khá thường xuyên với CommonJS. Câu lệnh import sau sẽ thất bại:

```
import data from './data.json'
```

It will produce a `TypeError (Unknown file extension: .json)`.

Nó sẽ tạo ra một `TypeError (Unknown file extension: .json)`.

To overcome this limitation, we can use again the `module.createRequire` utility:

Để vượt qua giới hạn này, chúng ta có thể dùng lại tiện ích `module.createRequire`:

```
import { createRequire } from 'module'
const require = createRequire(import.meta.url)
const data = require('./data.json')
console.log(data)
```

There is ongoing work to support JSON modules natively even in ESM, so we may not need to rely on `createRequire()` in the near future for this functionality.

Có công việc đang được tiến hành để hỗ trợ các JSON module một cách nguyên bản ngay cả trong ESM, nên chúng ta có thể không cần phải dựa vào `createRequire()` trong tương lai gần cho chức năng này.

Summary — Tóm tắt

In this chapter, we explored in depth what modules are, why they are useful, and why we need a module system. We also learned about the history of modules in JavaScript and about the two module systems available today in Node.js, namely CommonJS and ESM. We also explored some common patterns that are useful when creating modules or when using third-party modules.

Trong chương này, chúng ta đã khám phá chuyên sâu module là gì, tại sao chúng hữu ích, và tại sao chúng ta cần một hệ thống module. Chúng ta cũng đã tìm hiểu về lịch sử của module trong JavaScript và về hai hệ thống module có sẵn ngày nay trong Node.js, đó là CommonJS và ESM. Chúng ta cũng đã khám phá một số mẫu phổ biến hữu ích khi tạo module hoặc khi dùng các module của bên thứ ba.

You should now be comfortable with understanding and writing code that takes advantage of the features of both CommonJS and ESM.

Bây giờ bạn hẳn đã cảm thấy thoải mái với việc hiểu và viết code tận dụng các tính năng của cả CommonJS và ESM.

In the rest of the book, we will rely mostly on ES modules, but you should now be equipped to be flexible with your choices and be able to deal with CommonJS effectively if necessary.

Trong phần còn lại của cuốn sách, chúng ta sẽ chủ yếu dựa vào ES modules, nhưng bây giờ bạn hẳn đã được trang bị để linh hoạt với các lựa chọn của mình và có thể xử lý CommonJS một cách hiệu quả nếu cần.

In the next chapter, we will start to explore the idea of asynchronous programming with JavaScript, and we will examine callbacks, events, and their patterns in depth.

Trong chương tiếp theo, chúng ta sẽ bắt đầu khám phá ý tưởng về lập trình bất đồng bộ với JavaScript, và chúng ta sẽ xem xét chuyên sâu các callback, sự kiện, và các mẫu của chúng.

Chapter 3: Callbacks and Events —

Chương 3: Callback và Sự kiện

In synchronous programming, we conceptualize code as a series of consecutive computing steps that solve a specific problem. Every operation is blocking, which means that only when an operation is completed, it is possible to execute the next one. This approach makes the code very easy to read, understand, and debug.

Trong lập trình đồng bộ, ta hình dung mã nguồn như một chuỗi các bước tính toán liên tiếp nhằm giải quyết một bài toán cụ thể. Mọi thao tác đều mang tính chặn (blocking), nghĩa là chỉ khi một thao tác hoàn tất thì mới có thể thực thi thao tác kế tiếp. Cách tiếp cận này khiến mã rất dễ đọc, dễ hiểu và dễ gỡ lỗi.

On the other side, in asynchronous programming, some operations, such as reading from a file or performing a network **request**, are launched and then executed “in the background.” When we invoke an asynchronous operation, the instruction that follows is executed immediately, even if the previous asynchronous operation has not finished yet. In this scenario, we need a way to get notified when an asynchronous operation completes, and then continue the execution flow using the results from the operation. The most basic mechanism to get notified about the completion of an asynchronous operation in Node.js is the **callback**, which is nothing more than a function invoked by the **runtime** with the result of an asynchronous operation.

Ở chiều ngược lại, trong lập trình bất đồng bộ, một số thao tác — chẳng hạn như đọc từ một tệp hoặc thực hiện một yêu cầu mạng — được khởi chạy rồi sau đó được thực thi “ở chế độ nền”. Khi ta gọi một thao tác bất đồng bộ, câu lệnh tiếp theo sẽ được thực thi ngay lập tức, ngay cả khi thao tác bất đồng bộ trước đó chưa kết thúc. Trong tình huống này, ta cần một cách để được thông báo khi một thao tác bất đồng bộ hoàn tất, rồi tiếp tục luồng thực thi bằng cách dùng kết quả từ thao tác đó. Cơ chế cơ bản nhất để được thông báo về sự hoàn tất của một thao tác bất đồng bộ trong Node.js là callback, vốn chẳng qua chỉ là một hàm được runtime gọi cùng với kết quả của một thao tác bất đồng bộ.

The callback is the most basic building block on which all other asynchronous mechanisms are based. In fact, without callbacks, we wouldn’t have promises, and therefore not even **async/await**; we also wouldn’t have streams or events. This is why it’s important to know how callbacks work.

Callback là khối xây dựng cơ bản nhất mà mọi cơ chế bất đồng bộ khác đều dựa trên đó. Thực tế, nếu không có callback, ta sẽ không có promise, và do đó cũng không có cả **async/await**; ta cũng sẽ không có stream hay sự kiện. Đây chính là lý do vì sao việc hiểu cách callback hoạt động lại quan trọng đến vậy.

In this chapter, you will learn more about the Node.js Callback pattern and understand what it means, in practice, to write asynchronous code. We will make our way through conventions, patterns, and

pitfalls, and by the end of this chapter, you will have mastered the basics of the Callback pattern. You will also learn about the **Observer** pattern, which can be considered a close relative of the Callback pattern. The Observer pattern—embodied by the `EventEmitter`—uses callbacks to deal with multiple heterogeneous events and is one of the most extensively used components in Node.js programming.

Trong chương này, bạn sẽ tìm hiểu sâu hơn về mẫu Callback của Node.js và hiểu được trên thực tế việc viết mã bất đồng bộ nghĩa là gì. Ta sẽ lần lượt đi qua các quy ước, các mẫu, và các cạm bẫy, và đến cuối chương này, bạn sẽ nắm vững các nền tảng của mẫu Callback. Bạn cũng sẽ tìm hiểu về Observer pattern, vốn có thể được xem là một họ hàng gần của mẫu Callback. Observer pattern — được hiện thân qua `EventEmitter` — sử dụng callback để xử lý nhiều sự kiện không đồng nhất và là một trong những thành phần được sử dụng rộng rãi nhất trong lập trình Node.js.

To summarize, this is what you will learn in this chapter:

Tóm lại, đây là những gì bạn sẽ học trong chương này:

- The Callback pattern, how it works, what conventions are used in Node.js, and how to deal with its most common pitfalls
- The Observer pattern and how to implement it in Node.js using the `EventEmitter` class
 - Mẫu Callback, cách nó hoạt động, các quy ước được dùng trong Node.js, và cách xử lý những cạm bẫy phổ biến nhất của nó
 - Observer pattern và cách hiện thực nó trong Node.js bằng lớp `EventEmitter`

The Callback pattern — Mẫu Callback

Callbacks are the materialization of the handlers of the **Reactor pattern** (introduced in the previous chapter). They are one of those imprints that give Node.js its distinctive programming style.

Callback là sự hiện thực hoá của các handler trong reactor pattern (đã giới thiệu ở chương trước). Chúng là một trong những dấu ấn tạo nên phong cách lập trình đặc trưng của Node.js.

Callbacks are functions that are invoked to propagate the result of an operation, and this is exactly what we need when dealing with asynchronous operations. In the asynchronous world, they replace the use of the `return` instruction, which, in turn, always executes synchronously. JavaScript is the ideal language for callbacks because functions are first-class objects and can be easily assigned to variables, passed as arguments, returned from another function invocation, or stored in data structures. Another ideal construct for implementing callbacks is closures. With closures, we can reference the environment in which a function was created; this way, we can always maintain the context in which the asynchronous operation was requested, no matter when or where its callback is invoked.

Callback là các hàm được gọi để lan truyền kết quả của một thao tác, và đây chính xác là thứ ta cần khi làm việc với các thao tác bất đồng bộ. Trong thế giới bất đồng bộ, chúng thay thế việc dùng câu lệnh `return`, vốn luôn thực thi một cách đồng bộ. JavaScript là ngôn ngữ lý tưởng cho callback bởi các hàm là đối tượng hạng nhất (first-class) và có thể dễ dàng được gán cho biến, truyền làm đối số, trả về từ một lời gọi hàm khác, hoặc lưu trong các cấu trúc dữ liệu. Một cấu trúc lý tưởng khác để hiện thực callback là closure. Với closure, ta có thể tham chiếu đến môi trường nơi một hàm được tạo ra; bằng cách này, ta luôn có thể duy trì ngữ cảnh mà tại đó thao tác bất đồng bộ được yêu cầu, bất kể callback của nó được gọi vào lúc nào hay từ đâu.

If you need to refresh your knowledge about closures, you can refer to the article on MDN Web Docs at nodejsdp.link/mdn-closures.

Nếu bạn cần ôn lại kiến thức về closure, bạn có thể tham khảo bài viết trên MDN Web Docs tại nodejsdp.link/mdn-closures.

In this section, we will analyze this particular style of programming, which uses callbacks instead of return instructions.

Trong phần này, ta sẽ phân tích phong cách lập trình đặc thù này, vốn dùng callback thay cho các câu lệnh `return`.

The continuation-passing style — Phong cách truyền tiếp tục (continuation-passing style)

In JavaScript, a callback is a function that is passed as an argument to another function and is invoked with the result when the operation completes. In functional programming, this way of propagating the result is called continuation-passing style (CPS). It is a general concept, and it is not always associated with asynchronous operations. In fact, it simply indicates that a result is propagated by passing it to another function (the callback), instead of directly returning it to the caller.

Trong JavaScript, callback là một hàm được truyền làm đối số cho một hàm khác và được gọi cùng với kết quả khi thao tác hoàn tất. Trong lập trình hàm, cách lan truyền kết quả này được gọi là phong cách truyền tiếp tục (continuation-passing style - CPS). Đây là một khái niệm tổng quát, và nó không phải lúc nào cũng gắn với các thao tác bất đồng bộ. Thực tế, nó chỉ đơn giản chỉ ra rằng một kết quả được lan truyền bằng cách truyền nó cho một hàm khác (callback), thay vì trả về trực tiếp cho bên gọi.

Synchronous CPS — CPS đồng bộ

To clarify this concept, let's take a look at a simple synchronous function:

Để làm rõ khái niệm này, hãy cùng xem một hàm đồng bộ đơn giản:

```
function add (a, b) {  
  return a + b  
}
```

If you are wondering, there is nothing special going on here. The result is passed back to the caller using the `return` instruction. This is also called direct style, and it represents the most common way of returning a result in synchronous programming.

Nếu bạn đang thắc mắc, thì ở đây chẳng có gì đặc biệt cả. Kết quả được trả lại cho bên gọi bằng câu lệnh `return`. Cách này còn được gọi là direct style (phong cách trực tiếp), và nó là cách trả về kết quả phổ biến nhất trong lập trình đồng bộ.

The equivalent CPS of the preceding function would be as follows:

Phiên bản CPS tương đương của hàm trên sẽ như sau:

```
function addCps (a, b, callback) {  
  callback(a + b)  
}
```

The `addCps()` function is a synchronous CPS function. It's synchronous because it will complete its execution only when the callback completes its execution too. The following code demonstrates this statement:

Hàm `addCps()` là một hàm CPS đồng bộ. Nó đồng bộ vì nó chỉ hoàn tất việc thực thi của mình khi callback cũng hoàn tất việc thực thi. Đoạn mã sau minh hoạ điều này:

```
console.log('before')
addCps(1, 2, result => console.log(`Result: ${result}`))
console.log('after')
```

Since `addCps()` is synchronous, the previous code will trivially print the following:

Vì `addCps()` là đồng bộ, đoạn mã trên sẽ in ra một cách hiển nhiên như sau:

```
before
Result: 3
after
```

Now, let's see how asynchronous CPS works.

Bây giờ, hãy xem CPS bất đồng bộ hoạt động ra sao.

Asynchronous CPS — CPS bất đồng bộ

Let's consider a case where the `addCps()` function is asynchronous:

Hãy xét trường hợp hàm `addCps()` là bất đồng bộ:

```
function additionAsync (a, b, callback) {
  setTimeout(() => callback(a + b), 100)
}
```

In the previous code, we used `setTimeout()` to simulate an asynchronous invocation of the callback. `setTimeout()` adds a task to the event queue that is executed after the given number of milliseconds. This is clearly an asynchronous operation. Now, let's try to use `additionAsync()` and see how the order of the operations changes:

Trong đoạn mã trên, ta dùng `setTimeout()` để mô phỏng một lời gọi callback bất đồng bộ. `setTimeout()` thêm một tác vụ vào hàng đợi sự kiện (event queue), tác vụ này được thực thi sau số mili-giây đã cho. Đây rõ ràng là một thao tác bất đồng bộ. Giờ hãy thử dùng `additionAsync()` và xem thứ tự các thao tác thay đổi ra sao:

```
console.log('before')
additionAsync(1, 2, result => console.log(`Result: ${result}`))
console.log('after')
```

The preceding code will print the following:

Đoạn mã trên sẽ in ra như sau:

```
before
after
Result: 3
```

Since `setTimeout()` triggers an asynchronous operation, it doesn't wait for the callback to be executed; instead, it returns immediately, giving the control back to `additionAsync()`, and then back again to its caller. This property in Node.js is crucial, as it gives control back to the **event**

loop as soon as an asynchronous request is sent, thus allowing a new event from the queue to be processed.

Vì `setTimeout()` kích hoạt một thao tác bất đồng bộ, nó không chờ callback được thực thi; thay vào đó, nó trả về ngay lập tức, trả quyền điều khiển lại cho `addAsync()`, rồi lại trả về cho bên gọi nó. Tính chất này trong Node.js là cốt yếu, bởi nó trả quyền điều khiển lại cho vòng lặp sự kiện (event loop) ngay khi một yêu cầu bất đồng bộ được gửi đi, qua đó cho phép một sự kiện mới từ hàng đợi được xử lý.

Figure 3.1 shows how this works:

Hình 3.1 cho thấy cách hoạt động này:

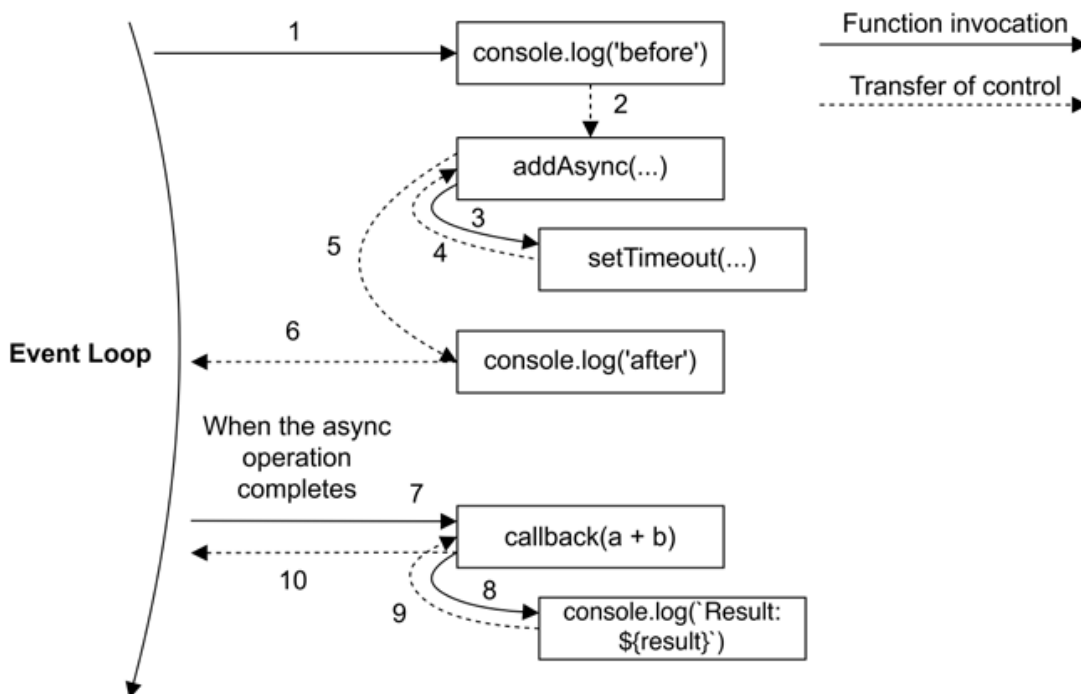


Figure 3.1: **Control flow** of an asynchronous function's invocation When the asynchronous operation completes, the execution is then resumed, starting from the callback provided to the asynchronous function that caused the unwinding. The execution starts from the event loop, so it has a fresh stack. This is where JavaScript comes in really handy. Thanks to closures, it is trivial to maintain the context of the caller of the asynchronous function, even if the callback is invoked at a different point in time and from a different location.

Hình 3.1: Luồng điều khiển của một lời gọi hàm bất đồng bộ. Khi thao tác bất đồng bộ hoàn tất, việc thực thi được tiếp tục, bắt đầu từ callback đã cung cấp cho hàm bất đồng bộ vốn gây ra sự gỡ ngăn xếp (unwinding). Việc thực thi bắt đầu từ event loop, nên nó có một stack mới tinh. Đây chính là lúc JavaScript tỏ ra cực kỳ hữu ích. Nhờ closure, việc duy trì ngữ cảnh của bên gọi hàm bất đồng bộ trở nên hết sức dễ dàng, dù callback được gọi vào một thời điểm khác và từ một vị trí khác.

To sum this up, a synchronous function blocks until it completes its operations. An asynchronous function returns immediately, and its result is passed to a handler (in our case, a callback) at a later cycle of the event loop.

Tóm lại, một hàm đồng bộ chặn cho đến khi hoàn tất các thao tác của nó. Một hàm bất đồng bộ trả về ngay lập tức, và kết quả của nó được truyền cho một handler (trong trường hợp của ta là một callback) ở một chu kỳ sau của event loop.

Non-CPS callbacks — Callback không theo CPS

There are several circumstances in which the presence of a callback argument might make us think that a function is asynchronous or is using a CPS. That's not always true. Let's take, for example, the `map()` method of an Array object:

Có một số trường hợp mà sự hiện diện của đối số callback có thể khiến ta nghĩ rằng một hàm là bất đồng bộ hoặc đang dùng CPS. Điều đó không phải lúc nào cũng đúng. Hãy lấy ví dụ phương thức `map()` của một đối tượng Array:

```
const result = [1, 5, 7].map(element => element - 1)
console.log(result) // [0, 4, 6]
```

Clearly, the callback is used just to iterate over the elements of the array, and not to pass the result of the operation. In fact, the result is returned synchronously using a direct style. There's no syntactic difference between non-CPS callbacks and CPS ones. Therefore, the intent of a callback should be clearly stated in the documentation of the **API**.

Rõ ràng, callback ở đây chỉ được dùng để lặp qua các phần tử của mảng, chứ không phải để truyền kết quả của thao tác. Thực tế, kết quả được trả về một cách đồng bộ theo direct style. Không có khác biệt cú pháp nào giữa callback không theo CPS và callback theo CPS. Do đó, ý định của một callback cần được nêu rõ trong tài liệu của API.

In the next section, we are going to discuss one of the most important pitfalls of callbacks that every Node.js developer should be aware of.

Trong phần tiếp theo, ta sẽ bàn về một trong những cạm bẫy quan trọng nhất của callback mà mọi lập trình viên Node.js cần lưu ý.

Synchronous or asynchronous? — Đồng bộ hay bất đồng bộ?

You have seen how the execution order of the instructions changes radically depending on the nature of a function—synchronous or asynchronous. This has strong repercussions on the flow of the entire application, both in terms of correctness and efficiency. The following is an analysis of these two paradigms and their pitfalls. In general, what must be avoided is creating inconsistency and confusion around the nature of an API, as doing so can lead to a set of problems that might be very hard to detect and reproduce. To drive our analysis, we will take, as an example, the case of an inconsistently asynchronous function.

Bạn đã thấy thứ tự thực thi các câu lệnh thay đổi triệt để như thế nào tùy vào bản chất của một hàm — đồng bộ hay bất đồng bộ. Điều này có tác động mạnh mẽ đến luồng của toàn bộ ứng dụng, cả về tính đúng đắn lẫn hiệu quả. Sau đây là phần phân tích về hai mô hình này và những cạm bẫy của chúng. Nhìn chung, thứ phải tránh là tạo ra sự không nhất quán và rối rắm xoay quanh bản chất của một API, bởi làm vậy có thể dẫn đến một loạt vấn đề rất khó phát hiện và tái hiện. Để dẫn dắt phần phân tích, ta sẽ lấy ví dụ về trường hợp một hàm bất đồng bộ không nhất quán.

An unpredictable function — Một hàm khó lường

One of the most dangerous situations is to have an API that behaves synchronously under certain conditions and asynchronously under others. Let's take the following code as an example:

Một trong những tình huống nguy hiểm nhất là có một API hành xử đồng bộ trong một số điều kiện và bất đồng bộ trong những điều kiện khác. Hãy lấy đoạn mã sau làm ví dụ:

```
import { readFile } from 'fs'
const cache = new Map()
function inconsistentRead (filename, cb) {
  if (cache.has(filename)) {
    // invoked synchronously
    cb(cache.get(filename))
  } else {
    // asynchronous function
    readFile(filename, 'utf8', (err, data) => {
      cache.set(filename, data)
      cb(data)
    })
  }
}
```

The preceding function uses the cache map to store the results of different file read operations. Bear in mind that this is just an example; it does not have error management, and the caching logic itself is suboptimal (in Chapter 11, Advanced Recipes, you'll learn how to handle asynchronous caching properly). But besides all this, the preceding function is dangerous because it behaves asynchronously until the file is read for the first time and the cache is set, but it is synchronous for all the subsequent requests once the file's content is already in the cache.

Hàm trên dùng map cache để lưu kết quả của các thao tác đọc tệp khác nhau. Hãy nhớ rằng đây chỉ là một ví dụ; nó không có quản lý lỗi, và bản thân logic cache cũng chưa tối ưu (trong Chương 11, Advanced Recipes, bạn sẽ học cách xử lý caching bất đồng bộ một cách đúng đắn). Nhưng bên cạnh tất cả những điều đó, hàm trên nguy hiểm bởi nó hành xử bất đồng bộ cho đến khi tệp được đọc lần đầu tiên và cache được thiết lập, nhưng lại hành xử đồng bộ với tất cả các yêu cầu tiếp theo một khi nội dung tệp đã có sẵn trong cache.

Unleashing Zalgo — Giải phóng Zalgo

Now, let's discuss how the use of an unpredictable function, such as the one that we just defined, can easily break an application. Consider the following code:

Giờ hãy bàn xem việc dùng một hàm khó lường, chẳng hạn như hàm ta vừa định nghĩa, có thể dễ dàng phá vỡ một ứng dụng như thế nào. Hãy xét đoạn mã sau:

```
function createFileReader (filename) {
  const listeners = []
  inconsistentRead(filename, value => {
    listeners.forEach(listener => listener(value))
  })
  return {
    onDataReady: listener => listeners.push(listener)
  }
}
```

```
}  
}
```

When the preceding function is invoked, it creates a new object that acts as a notifier, allowing us to set multiple listeners for a file read operation. All the listeners will be invoked at once when the read operation completes and the data is available. The preceding function uses our `inconsistentRead()` function to implement this functionality. Let's see how to use the `createFileReader()` function:

Khi hàm trên được gọi, nó tạo ra một đối tượng mới đóng vai trò như một bộ thông báo (notifier), cho phép ta đặt nhiều listener cho một thao tác đọc tệp. Tất cả các listener sẽ được gọi cùng một lúc khi thao tác đọc hoàn tất và dữ liệu đã sẵn sàng. Hàm trên dùng hàm `inconsistentRead()` của ta để hiện thực chức năng này. Hãy xem cách dùng hàm `createFileReader()`:

```
const reader1 = createFileReader('data.txt')  
reader1.onDataReady(data => {  
  console.log(`First call data: ${data}`)  
  // ...sometime later we try to read again from  
  // the same file  
  const reader2 = createFileReader('data.txt')  
  reader2.onDataReady(data => {  
    console.log(`Second call data: ${data}`)  
  })  
})
```

The preceding code will print the following:

Đoạn mã trên sẽ in ra như sau:

First call data: some data

As you can see, the callback of the second reader is never invoked. Let's see why:

Như bạn thấy, callback của reader thứ hai không bao giờ được gọi. Hãy xem tại sao:

- During the creation of `reader1`, our `inconsistentRead()` function behaves asynchronously because there is no cached result available. This means that any `onDataReady` listener will be invoked later in another cycle of the event loop, so we have all the time we need to register our listener.
- Then, `reader2` is created in a cycle of the event loop in which the cache for the requested file already exists. In this case, the inner call to `inconsistentRead()` will be synchronous. So, its callback will be invoked immediately, which means that all the listeners of `reader2` will be invoked synchronously as well. However, we are registering the listener after the creation of `reader2`, so it will never be invoked.
 - Trong quá trình tạo `reader1`, hàm `inconsistentRead()` của ta hành xử bất đồng bộ vì không có kết quả nào được cache sẵn. Điều này nghĩa là bất kỳ listener `onDataReady` nào cũng sẽ được gọi sau đó ở một chu kỳ khác của event loop, nên ta có đủ thời gian cần thiết để đăng ký listener của mình.
 - Sau đó, `reader2` được tạo trong một chu kỳ của event loop mà tại đó cache cho tệp được yêu cầu đã tồn tại. Trong trường hợp này, lời gọi bên trong tới `inconsistentRead()` sẽ là đồng bộ. Vì vậy, callback của nó sẽ được gọi ngay lập tức, nghĩa là tất cả các listener của `reader2` cũng sẽ được gọi một cách đồng bộ. Tuy nhiên, ta đang đăng ký listener sau khi tạo `reader2`, nên nó sẽ không bao giờ được

gọi.

The callback behavior of our `inconsistentRead()` function is really unpredictable as it depends on many factors, such as the frequency of its invocation, the filename passed as an argument, and the amount of time taken to load the file. The bug that you've just seen can be extremely complicated to identify and reproduce in a real application. Imagine using a similar function in a web server, where there can be multiple concurrent requests. Imagine seeing some of those requests hanging, without any apparent reason and without any error being logged. This can definitely be considered a nasty defect.

Hành vi callback của hàm `inconsistentRead()` thực sự khó lường bởi nó phụ thuộc vào nhiều yếu tố, chẳng hạn như tần suất gọi nó, tên tệp được truyền vào làm đối số, và lượng thời gian cần để nạp tệp. Lỗi mà bạn vừa thấy có thể cực kỳ phức tạp để nhận diện và tái hiện trong một ứng dụng thực tế. Hãy hình dung việc dùng một hàm tương tự trong một web server, nơi có thể có nhiều yêu cầu đồng thời. Hãy hình dung việc thấy một số yêu cầu đó bị treo, không có lý do rõ ràng nào và không có lỗi nào được ghi log. Điều này chắc chắn có thể được xem là một khiếm khuyết đáng ghét.

Isaac Z. Schlueter, the creator of npm and former Node.js project lead, in one of his blog posts, compared the use of this type of unpredictable function to unleashing Zalgo.

Isaac Z. Schlueter, người tạo ra npm và là cựu trưởng dự án Node.js, trong một bài blog của mình, đã ví việc dùng loại hàm khó lường này như việc giải phóng Zalgo.

Zalgo is an internet legend about an ominous entity believed to cause insanity, death, and the destruction of the world. If you're not familiar with Zalgo, you are invited to find out what it is.

Zalgo là một truyền thuyết internet về một thực thể đáng sợ được cho là gây ra sự điên loạn, cái chết, và sự huỷ diệt của thế giới. Nếu bạn chưa quen với Zalgo, bạn được mời tìm hiểu xem nó là gì.

You can find Isaac Z. Schlueter's original post at nodesdp.link/unleashing-zalgo.

Bạn có thể tìm bài viết gốc của Isaac Z. Schlueter tại nodesdp.link/unleashing-zalgo.

Using synchronous APIs — Dùng các API đồng bộ

The lesson to learn from the unleashing Zalgo example is that it is imperative for an API to clearly define its nature: either synchronous or asynchronous.

Bài học rút ra từ ví dụ giải phóng Zalgo là một API bắt buộc phải định nghĩa rõ bản chất của mình: hoặc đồng bộ, hoặc bất đồng bộ.

One possible fix for our `inconsistentRead()` function is to make it completely synchronous. This is possible because Node.js provides a set of synchronous direct style APIs for most basic I/O operations. For example, we can use the `fs.readFileSync()` function in place of its asynchronous counterpart. The code would become as follows:

Một cách sửa khả dĩ cho hàm `inconsistentRead()` của ta là làm cho nó hoàn toàn đồng bộ. Điều này khả thi vì Node.js cung cấp một tập các API direct style đồng bộ cho hầu hết các thao tác I/O cơ bản. Ví dụ, ta có thể dùng hàm `fs.readFileSync()` thay cho phiên bản bất đồng bộ tương ứng. Mã sẽ trở thành như sau:

```
import { readFileSync } from 'fs'
const cache = new Map()
function consistentReadSync (filename) {
  if (cache.has(filename)) {
```

```

    return cache.get(filename)
  } else {
    const data = readFileSync(filename, 'utf8')
    cache.set(filename, data)
    return data
  }
}

```

You can see that the entire function was also converted into direct style. There is no reason for a function to have a CPS if it is synchronous. In fact, it is always best practice to implement a synchronous API using a direct style. This will eliminate any confusion around its nature and will also be more efficient from a performance perspective.

Bạn có thể thấy toàn bộ hàm cũng đã được chuyển thành direct style. Không có lý do gì để một hàm dùng CPS nếu nó là đồng bộ. Thực tế, luôn là thực hành tốt nhất khi hiện thực một API đồng bộ theo direct style. Điều này sẽ loại bỏ mọi sự rối rắm xoay quanh bản chất của nó và cũng hiệu quả hơn về mặt hiệu năng.

Pattern

Mẫu (Pattern)

Always choose a direct style for purely synchronous functions.

Luôn chọn direct style cho các hàm thuần đồng bộ.

Bear in mind that changing an API from CPS to a direct style, or from asynchronous to synchronous or vice versa, might also require a change to the style of all the code using it. For example, in our case, we will have to totally change the interface of our `createFileReader()` API and adapt it so that it always works synchronously.

Hãy nhớ rằng việc đổi một API từ CPS sang direct style, hoặc từ bất đồng bộ sang đồng bộ hay ngược lại, cũng có thể đòi hỏi thay đổi phong cách của toàn bộ mã sử dụng nó. Ví dụ, trong trường hợp của ta, ta sẽ phải thay đổi hoàn toàn giao diện của API `createFileReader()` và điều chỉnh nó sao cho nó luôn hoạt động đồng bộ.

Also, using a synchronous API instead of an asynchronous one has some caveats:

Ngoài ra, việc dùng một API đồng bộ thay cho một API bất đồng bộ có một số lưu ý:

- A synchronous API for a specific functionality might not always be available.
- A synchronous API will block the event loop and put any concurrent requests on hold. This will break the Node.js **concurrency** model, slowing down the whole application. You will see later in this book what this really means for our applications.
 - Một API đồng bộ cho một chức năng cụ thể có thể không phải lúc nào cũng có sẵn.
 - Một API đồng bộ sẽ chặn event loop và khiến mọi yêu cầu đồng thời phải chờ. Điều này sẽ phá vỡ mô hình đồng thời của Node.js, làm chậm toàn bộ ứng dụng. Bạn sẽ thấy ở phần sau của cuốn sách này điều đó thực sự có ý nghĩa gì đối với các ứng dụng của ta.

In our `consistentReadSync()` function, the risk of blocking the event loop is partially mitigated because the synchronous I/O API is invoked only once per filename, while the cached value will be used for all the subsequent invocations. If we have a limited number of static files, then using `consistentReadSync()` won't have a big effect on our event loop. Things can change quickly if we have to read many files and only once.

Trong hàm `consistentReadSync()` của ta, nguy cơ chặn event loop được giảm thiểu một phần vì API I/O đồng bộ chỉ được gọi một lần cho mỗi tên tệp, trong khi giá trị được cache sẽ được dùng cho mọi lời gọi tiếp theo. Nếu ta có một số lượng hữu hạn các tệp tĩnh, thì việc dùng `consistentReadSync()` sẽ không có tác động lớn lên event loop của ta. Mọi thứ có thể thay đổi nhanh chóng nếu ta phải đọc nhiều tệp và chỉ một lần.

Using synchronous I/O in Node.js is strongly discouraged in many circumstances, but in some situations, this might be the easiest and most efficient solution. Always evaluate your specific use case in order to choose the right alternative. As an example, it makes perfect sense to use a synchronous blocking API to load a configuration file while bootstrapping an application.

Việc dùng I/O đồng bộ trong Node.js bị khuyến cáo mạnh mẽ không nên dùng trong nhiều trường hợp, nhưng trong một số tình huống, đây có thể là giải pháp dễ nhất và hiệu quả nhất. Hãy luôn đánh giá trường hợp sử dụng cụ thể của bạn để chọn phương án phù hợp. Ví dụ, việc dùng một API đồng bộ có tính chặn để nạp một tệp cấu hình trong khi khởi động (bootstrap) một ứng dụng là hoàn toàn hợp lý.

Pattern

Mẫu (Pattern)

Use blocking APIs sparingly and only when they don't affect the ability of the application to handle concurrent asynchronous operations.

Hãy dùng các API chặn một cách tiết chế và chỉ khi chúng không ảnh hưởng đến khả năng của ứng dụng trong việc xử lý các thao tác bất đồng bộ đồng thời.

Guaranteeing asynchronicity with deferred execution — Bảo đảm tính bất đồng bộ bằng thực thi hoãn lại (deferred execution)

Another alternative for fixing our `inconsistentRead()` function is to make it purely asynchronous. The trick here is to schedule the synchronous callback invocation to be executed “in the future” instead of it being run immediately in the same event loop cycle. In Node.js, this is possible with `process.nextTick()`, which defers the execution of a function after the currently running operation completes. Its functionality is very simple: it takes a callback as an argument and pushes it to the top of the event queue, in front of any pending I/O event, and returns immediately. The callback will then be invoked as soon as the currently running operation yields control back to the event loop.

Một cách khác để sửa hàm `inconsistentRead()` của ta là làm cho nó thuần bất đồng bộ. Mẹo ở đây là lên lịch để lời gọi callback đồng bộ được thực thi “trong tương lai” thay vì chạy ngay lập tức trong cùng chu kỳ event loop. Trong Node.js, điều này khả thi với `process.nextTick()`, vốn hoãn việc thực thi một hàm cho đến sau khi thao tác đang chạy hiện tại hoàn tất. Chức năng của nó rất đơn giản: nó nhận một callback làm đối số và đẩy callback đó lên đầu hàng đợi sự kiện, đứng trước bất kỳ sự kiện I/O đang chờ nào, rồi trả về ngay lập tức. Callback khi đó sẽ được gọi ngay khi thao tác đang chạy hiện tại trả quyền điều khiển lại cho event loop.

Let's apply this technique to fix our `inconsistentRead()` function, as follows:

Hãy áp dụng kỹ thuật này để sửa hàm `inconsistentRead()` của ta, như sau:

```
import { readFile } from 'fs'
const cache = new Map()
function consistentReadAsync (filename, callback) {
  if (cache.has(filename)) {
```



```

    // deferred callback invocation
    process.nextTick(() => callback(cache.get(filename)))
  } else {
    // asynchronous function
    readFile(filename, 'utf8', (err, data) => {
      cache.set(filename, data)
      callback(data)
    })
  }
}

```

Now, thanks to `process.nextTick()`, our function is guaranteed to invoke its callback asynchronously, under any circumstances. Try to use it instead of the `inconsistentRead()` function and verify that, indeed, Zalgo has been eradicated.

Giờ đây, nhờ `process.nextTick()`, hàm của ta được bảo đảm sẽ gọi callback của mình một cách bất đồng bộ, trong mọi hoàn cảnh. Hãy thử dùng nó thay cho hàm `inconsistentRead()` và xác nhận rằng, quả thực, Zalgo đã bị diệt trừ.

Pattern

Mẫu (Pattern)

You can guarantee that a callback is invoked asynchronously by deferring its execution using `process.nextTick()`. Another API for deferring the execution of code is `setImmediate()`. While its purpose is very similar to that of `process.nextTick()`, its semantics are quite different. Callbacks deferred with `process.nextTick()` are called microtasks and they are executed just after the current operation completes, even before any other I/O event is fired. With `setImmediate()`, on the other hand, the execution is queued in an event loop phase that comes after all I/O events have been processed. Since `process.nextTick()` runs before any already scheduled I/O, it will be executed faster, but under certain circumstances, it might also delay the running of any I/O callback indefinitely (also known as I/O starvation), such as in the presence of a recursive invocation. This can never happen with `setImmediate()`.

Bạn có thể bảo đảm một callback được gọi một cách bất đồng bộ bằng cách hoãn việc thực thi của nó dùng `process.nextTick()`. Một API khác để hoãn việc thực thi mã là `setImmediate()`. Tuy mục đích của nó rất giống với `process.nextTick()`, ngữ nghĩa của nó lại khá khác biệt. Các callback được hoãn bằng `process.nextTick()` được gọi là microtask và chúng được thực thi ngay sau khi thao tác hiện tại hoàn tất, thậm chí trước khi bất kỳ sự kiện I/O nào khác được kích hoạt. Với `setImmediate()` thì ngược lại, việc thực thi được xếp hàng trong một pha của event loop diễn ra sau khi tất cả các sự kiện I/O đã được xử lý. Vì `process.nextTick()` chạy trước bất kỳ I/O nào đã được lên lịch, nó sẽ được thực thi nhanh hơn, nhưng trong một số hoàn cảnh nhất định, nó cũng có thể trì hoãn việc chạy bất kỳ callback I/O nào vô hạn định (còn gọi là I/O starvation - bỏ đói I/O), chẳng hạn như khi có một lời gọi đệ quy. Điều này không bao giờ xảy ra với `setImmediate()`.

Using `setTimeout(callback, 0)` has a behavior comparable to that of `setImmediate()`, but in typical circumstances, callbacks scheduled with `setImmediate()` are executed faster than those scheduled with `setTimeout(callback, 0)`. To see why, we have to consider that the event loop executes all the callbacks in different phases; for the type of events we are considering, we have timers (`setTimeout()`) that are executed before I/O callbacks, which are, in turn, executed before `setImmediate()` callbacks. This means that if we queue a task with `setImmediate()` in a `setTimeout()` callback, in an I/O callback, or in a microtask queued after these two phases, then the callback will be executed in a phase that comes right after the phase we are currently in.

`setTimeout()` callbacks have to wait for the next cycle of the event loop.

Việc dùng `setTimeout(callback, 0)` có hành vi tương đương với `setImmediate()`, nhưng trong những hoàn cảnh điển hình, các callback được lên lịch bằng `setImmediate()` được thực thi nhanh hơn các callback được lên lịch bằng `setTimeout(callback, 0)`. Để hiểu tại sao, ta phải xét rằng event loop thực thi tất cả các callback trong các pha khác nhau; với loại sự kiện ta đang xét, ta có các timer (`setTimeout()`) được thực thi trước các callback I/O, và các callback I/O này lại được thực thi trước các callback `setImmediate()`. Điều này nghĩa là nếu ta xếp hàng một tác vụ bằng `setImmediate()` bên trong một callback `setTimeout()`, bên trong một callback I/O, hoặc trong một microtask được xếp hàng sau hai pha này, thì callback đó sẽ được thực thi trong một pha diễn ra ngay sau pha mà ta đang ở hiện tại. Các callback `setTimeout()` phải chờ chu kỳ tiếp theo của event loop.

You will better appreciate the difference between these APIs when we analyze the use of deferred invocation for running synchronous CPU-bound tasks later in this book.

Bạn sẽ hiểu rõ hơn sự khác biệt giữa các API này khi ta phân tích việc dùng lời gọi hoãn lại để chạy các tác vụ đồng bộ thiên về CPU (CPU-bound) ở phần sau của cuốn sách.

Next, we are going to explore the conventions used to define callbacks in Node.js.

Tiếp theo, ta sẽ khám phá các quy ước được dùng để định nghĩa callback trong Node.js.

Node.js callback conventions — Các quy ước callback của Node.js

In Node.js, CPS APIs and callbacks follow a set of specific conventions. These conventions apply to the Node.js core API, but they are also followed by the vast majority of the **userland** modules and applications. So, it's very important that you understand them and make sure that you comply whenever you need to design an asynchronous API that makes use of callbacks.

Trong Node.js, các API CPS và callback tuân theo một tập các quy ước cụ thể. Các quy ước này áp dụng cho API lõi của Node.js, nhưng chúng cũng được đại đa số các module và ứng dụng userland tuân theo. Vì vậy, việc bạn hiểu chúng và bảo đảm bạn tuân thủ bất cứ khi nào bạn cần thiết kế một API bất đồng bộ dùng callback là rất quan trọng.

The callback comes last — Callback đứng cuối cùng

In all core Node.js functions, the standard convention is that when a function accepts a callback as input, this has to be passed as the last argument. Let's take the following Node.js core API as an example:

Trong tất cả các hàm lõi của Node.js, quy ước chuẩn là khi một hàm nhận một callback làm đầu vào, callback này phải được truyền làm đối số cuối cùng. Hãy lấy API lõi sau của Node.js làm ví dụ:

```
readFile(filename, [options], callback)
```

As you can see from the signature of the preceding function, the callback is always put in the last position, even in the presence of optional arguments. The reason for this convention is that the function call is more **readable** in case the callback is defined in place.

Như bạn thấy từ chữ ký (signature) của hàm trên, callback luôn được đặt ở vị trí cuối cùng, ngay cả khi có các đối số tùy chọn. Lý do của quy ước này là lời gọi hàm sẽ dễ đọc hơn trong trường hợp callback được định nghĩa tại chỗ.

Any error always comes first — Lỗi luôn đứng đầu tiên

In CPS, errors are propagated like any other type of result, which means using callbacks. In Node.js, any error produced by a CPS function is always passed as the first argument of the callback, and any actual result is passed starting from the second argument. If the operation succeeds without errors, the first argument will be null or undefined. The following code shows you how to define a callback that complies with this convention:

Trong CPS, lỗi được lan truyền giống như bất kỳ loại kết quả nào khác, nghĩa là dùng callback. Trong Node.js, bất kỳ lỗi nào do một hàm CPS tạo ra luôn được truyền làm đối số đầu tiên của callback, và bất kỳ kết quả thực sự nào đều được truyền bắt đầu từ đối số thứ hai. Nếu thao tác thành công mà không có lỗi, đối số đầu tiên sẽ là null hoặc undefined. Đoạn mã sau cho bạn thấy cách định nghĩa một callback tuân thủ quy ước này:

```
readFile('foo.txt', 'utf8', (err, data) => {
  if(err) {
    handleError(err)
  } else {
    processData(data)
  }
})
```

It is best practice to always check for the presence of an error, as not doing so will make it harder for you to debug your code and discover the possible points of failure. Another important convention to take into account is that the error must always be of type Error. This means that simple strings or numbers should never be passed as error objects.

Luôn kiểm tra sự hiện diện của lỗi là một thực hành tốt nhất, vì nếu không làm vậy sẽ khiến bạn khó gỡ lỗi mã của mình hơn và khó phát hiện các điểm có thể gây lỗi. Một quy ước quan trọng khác cần lưu ý là lỗi phải luôn thuộc kiểu Error. Điều này nghĩa là không bao giờ nên truyền các chuỗi hay số đơn thuần làm đối tượng lỗi.

Propagating errors — Lan truyền lỗi

Propagating errors in synchronous, direct style functions is done with the wellknown throw statement, which causes the error to jump up in the call stack until it is caught.

Việc lan truyền lỗi trong các hàm đồng bộ, direct style được thực hiện bằng câu lệnh throw quen thuộc, vốn khiến lỗi nhảy lên trên ngăn xếp gọi (call stack) cho đến khi nó được bắt.

In asynchronous CPS, however, proper error propagation is done by simply passing the error to the next callback in the chain. The typical pattern looks as follows:

Tuy nhiên, trong CPS bất đồng bộ, việc lan truyền lỗi đúng cách được thực hiện bằng cách đơn giản truyền lỗi cho callback tiếp theo trong chuỗi. Mẫu điển hình trông như sau:

```
import { readFile } from 'fs'
function readJSON (filename, callback) {
  readFile(filename, 'utf8', (err, data) => {
    let parsed
    if (err) {
      // propagate the error and exit the current function
      return callback(err)
    }
    try {
```

```

    // parse the file contents
    parsed = JSON.parse(data)
  } catch (err) {
    // catch parsing errors
    return callback(err)
  }
  // no errors, propagate just the data
  callback(null, parsed)
})
}

```

Notice how we propagate the error received by the `readFile()` operation. We do not throw it or return it; instead, we just use the callback as if it were any other result. Also, notice how we use the `try...catch` statement to catch any error thrown by `JSON.parse()`, which is a synchronous function and therefore uses the traditional `throw` instruction to propagate errors to the caller. Lastly, if everything went well, `callback` is invoked with `null` as the first argument to indicate that there are no errors.

Hãy chú ý cách ta lan truyền lỗi nhận được từ thao tác `readFile()`. Ta không `throw` nó hay `return` nó; thay vào đó, ta chỉ dùng `callback` như thể nó là bất kỳ kết quả nào khác. Cũng hãy chú ý cách ta dùng câu lệnh `try...catch` để bắt bất kỳ lỗi nào do `JSON.parse()` ném ra, vốn là một hàm đồng bộ và do đó dùng câu lệnh `throw` truyền thống để lan truyền lỗi cho bên gọi. Cuối cùng, nếu mọi thứ suôn sẻ, `callback` được gọi với `null` làm đối số đầu tiên để chỉ ra rằng không có lỗi.

It's also interesting to note how we refrained from invoking `callback` from within the `try` block. This is because doing so would catch any error thrown from the execution of the callback itself, which is usually not what we want.

Cũng thú vị khi lưu ý cách ta tránh gọi `callback` từ bên trong khối `try`. Đó là bởi làm vậy sẽ bắt bất kỳ lỗi nào do chính việc thực thi `callback` ném ra, vốn thường không phải điều ta muốn.

Uncaught exceptions — Ngoại lệ không được bắt (uncaught exception)

Sometimes, it can happen that an error is thrown and not caught within the callback of an asynchronous function. This could happen if, for example, we had forgotten to surround `JSON.parse()` with a `try...catch` statement in the `readJSON()` function we defined previously. Throwing an error inside an asynchronous callback would cause the error to jump up to the event loop, so it would never be propagated to the next callback. In Node.js, this is an unrecoverable **state** and the application would simply exit with a non-zero exit code, printing the stack trace to the `stderr` interface. To demonstrate this, let's try to remove the `try...catch` block surrounding `JSON.parse()` from the `readJSON()` function we defined previously:

Đôi khi, có thể xảy ra việc một lỗi bị ném ra và không được bắt bên trong `callback` của một hàm bất đồng bộ. Điều này có thể xảy ra nếu, chẳng hạn, ta quên bao `JSON.parse()` bằng câu lệnh `try...catch` trong hàm `readJSON()` mà ta đã định nghĩa trước đó. Việc ném một lỗi bên trong một `callback` bất đồng bộ sẽ khiến lỗi nhảy lên tới event loop, nên nó sẽ không bao giờ được lan truyền tới `callback` tiếp theo. Trong Node.js, đây là một trạng thái không thể khôi phục và ứng dụng sẽ đơn giản thoát ra với mã thoát khác không, in dấu vết ngăn xếp (stack trace) ra giao diện `stderr`. Để minh họa điều này, hãy thử bỏ khối `try...catch` bao quanh `JSON.parse()` khỏi hàm `readJSON()` mà ta đã định nghĩa trước đó:

```
function readJSONThrows (filename, callback) {
  readFile(filename, 'utf8', (err, data) => {
    if (err) {
      return callback(err)
    }
    callback(null, JSON.parse(data))
  })
}
```

Now, in the function we just defined, there is no way of catching an eventual exception coming from `JSON.parse()`. If we try to parse an invalid JSON file with the following code:

Giờ đây, trong hàm ta vừa định nghĩa, không có cách nào để bắt một ngoại lệ có thể đến từ `JSON.parse()`. Nếu ta thử phân tích cú pháp một tệp JSON không hợp lệ bằng đoạn mã sau:

```
readJSONThrows('invalid_json.json', (err) => console.error(err))
```

This will result in the application being abruptly terminated, with a stack trace similar to the following being printed on the console:

Điều này sẽ dẫn đến việc ứng dụng bị chấm dứt một cách đột ngột, với một stack trace tương tự như sau được in ra console:

```
SyntaxError: Unexpected token h in JSON at position 1
    at JSON.parse (<anonymous>)
    at file:///.../03-callbacks-and-events/08-uncaught-errors/index.js:8:25
    at FSReqCallback.readFileAfterClose [as oncomplete] (internal/fs/read_file_context.js:61:3)
```

Now, if you look at the preceding stack trace, you will see that it starts from within the built-in `fs` **module**, and exactly from the point in which the native API has completed reading and returned its result back to the `fs.readFile()` function, via the event loop. This clearly shows that the exception traveled from our callback, up the call stack, and then straight into the event loop, where it was finally caught and thrown to the console.

Giờ đây, nếu bạn nhìn vào stack trace trên, bạn sẽ thấy nó bắt đầu từ bên trong module `fs` tích hợp sẵn, và chính xác từ điểm mà API gốc (native) đã hoàn tất việc đọc và trả kết quả của nó lại cho hàm `fs.readFile()`, thông qua event loop. Điều này cho thấy rõ ràng rằng ngoại lệ đã đi từ callback của ta, lên call stack, rồi thẳng vào event loop, nơi cuối cùng nó được bắt và ném ra console.

This also means that wrapping the invocation of `readJSONThrows()` with a `try...catch` block will not work, because the stack in which the block operates is different from the one in which our callback is invoked. The following code shows the **antipattern** that was just described:

Điều này cũng nghĩa là việc bao lời gọi `readJSONThrows()` bằng một khối `try...catch` sẽ không có tác dụng, bởi stack mà khối đó hoạt động khác với stack mà callback của ta được gọi. Đoạn mã sau cho thấy antipattern vừa được mô tả:

```
try {
  readJSONThrows('invalid_json.json', (err) => console.error(err))
} catch (err) {
  console.log('This will NOT catch the JSON parsing exception')
}
```

The preceding catch statement will never receive the JSON parsing error, as it will travel up the call stack in which the error was thrown, that is, in the event loop and not in the function that triggered the asynchronous operation.

Câu lệnh catch trên sẽ không bao giờ nhận được lỗi phân tích cú pháp JSON, bởi lỗi sẽ đi lên call stack nơi lỗi được ném ra, tức là trong event loop chứ không phải trong hàm đã kích hoạt thao tác bất đồng bộ.

As mentioned previously, the application will abort the moment an exception reaches the event loop. However, we still have the chance to perform some cleanup or logging before the application terminates. In fact, when this happens, Node.js will emit a special event called `uncaughtException`, just before exiting the process. The following code shows a sample use case:

Như đã đề cập trước đó, ứng dụng sẽ bị huỷ ngay khoảnh khắc một ngoại lệ chạm tới event loop. Tuy nhiên, ta vẫn có cơ hội thực hiện một số việc dọn dẹp hoặc ghi log trước khi ứng dụng chấm dứt. Thực tế, khi điều này xảy ra, Node.js sẽ phát ra một sự kiện đặc biệt gọi là `uncaughtException`, ngay trước khi thoát khỏi tiến trình. Đoạn mã sau cho thấy một trường hợp sử dụng ví dụ:

```
process.on('uncaughtException', (err) => {
  console.error(`This will catch at last the JSON parsing exception:
${err.message}`)
  // Terminates the application with 1 (error) as exit code.
  // Without the following line, the application would continue
  process.exit(1)
})
```

It's important to understand that an uncaught exception leaves the application in a state that is not guaranteed to be consistent, which can lead to unforeseeable problems. For example, there might still be incomplete I/O requests running or closures might have become inconsistent. That's why it is always advised, especially in production, to never leave the application running after an uncaught exception is received. Instead, the process should exit immediately, optionally after having run some necessary cleanup tasks, and ideally, a supervising process should restart the application. This is also known as the fail-fast approach and it's the recommended practice in Node.js.

Điều quan trọng cần hiểu là một ngoại lệ không được bắt để lại ứng dụng ở một trạng thái không được bảo đảm là nhất quán, vốn có thể dẫn đến những vấn đề không lường trước được. Ví dụ, có thể vẫn còn các yêu cầu I/O chưa hoàn tất đang chạy hoặc các closure có thể đã trở nên không nhất quán. Đó là lý do vì sao luôn được khuyến khích rằng, đặc biệt trong môi trường production, không bao giờ để ứng dụng tiếp tục chạy sau khi nhận được một ngoại lệ không được bắt. Thay vào đó, tiến trình nên thoát ngay lập tức, tùy chọn là sau khi đã chạy một số tác vụ dọn dẹp cần thiết, và lý tưởng nhất là một tiến trình giám sát nên khởi động lại ứng dụng. Cách này còn được gọi là cách tiếp cận fail-fast và là thực hành được khuyến nghị trong Node.js.

We'll discuss supervisors in more detail in Chapter 12, **Scalability** and Architectural Patterns.

Ta sẽ bàn về các bộ giám sát (supervisor) chi tiết hơn trong Chương 12, Scalability and Architectural Patterns.

This concludes our gentle introduction to the Callback pattern. Now, it's time to meet the Observer pattern, which is another critical component of an **event-driven** platform such as Node.js.

Phần này kết thúc phần giới thiệu nhẹ nhàng của ta về mẫu Callback. Giờ là lúc làm quen với Observer pattern, một thành phần quan trọng khác của một nền tảng hướng sự kiện

(event-driven) như Node.js.

The Observer pattern — Observer pattern

Another important and fundamental pattern used in Node.js is the Observer pattern. Together with the Reactor pattern and callbacks, the Observer pattern is an absolute requirement for mastering the asynchronous world of Node.js. The Observer pattern is the ideal solution for modeling the reactive nature of Node.js and a perfect complement for callbacks. Let's give a formal definition, as follows:

Một mẫu quan trọng và nền tảng khác được dùng trong Node.js là Observer pattern. Cùng với reactor pattern và callback, Observer pattern là một yêu cầu tuyệt đối để làm chủ thế giới bất đồng bộ của Node.js. Observer pattern là giải pháp lý tưởng để mô hình hoá bản chất phản ứng (reactive) của Node.js và là một hỗ trợ hoàn hảo cho callback. Hãy đưa ra một định nghĩa chính thức như sau:

The Observer pattern defines an object (called subject) that can notify a set of observers (or listeners) when a change in its state occurs.

Observer pattern định nghĩa một đối tượng (gọi là subject) có thể thông báo cho một tập các observer (hay listener) khi một thay đổi trong trạng thái của nó xảy ra.

The main difference from the Callback pattern is that the subject can actually notify multiple observers, while a traditional CPS callback will usually propagate its result to only one listener, the callback.

Khác biệt chính so với mẫu Callback là subject thực sự có thể thông báo cho nhiều observer, trong khi một callback CPS truyền thống thường chỉ lan truyền kết quả của nó tới một listener duy nhất, chính là callback.

The EventEmitter — EventEmitter

In traditional object-oriented programming, the Observer pattern requires interfaces, concrete classes, and a hierarchy. In Node.js, all this becomes much simpler. The Observer pattern is already built into the core and is available through the EventEmitter class. The EventEmitter class allows us to register one or more functions as listeners, which will be invoked when a particular event type is fired. Figure 3.2 visually explains this concept:

Trong lập trình hướng đối tượng truyền thống, Observer pattern đòi hỏi các interface, các lớp cụ thể, và một hệ phân cấp. Trong Node.js, tất cả những điều này trở nên đơn giản hơn nhiều. Observer pattern đã được tích hợp sẵn vào lõi và có sẵn thông qua lớp EventEmitter. Lớp EventEmitter cho phép ta đăng ký một hoặc nhiều hàm làm listener, các hàm này sẽ được gọi khi một loại sự kiện cụ thể được kích hoạt. Hình 3.2 giải thích trực quan khái niệm này:

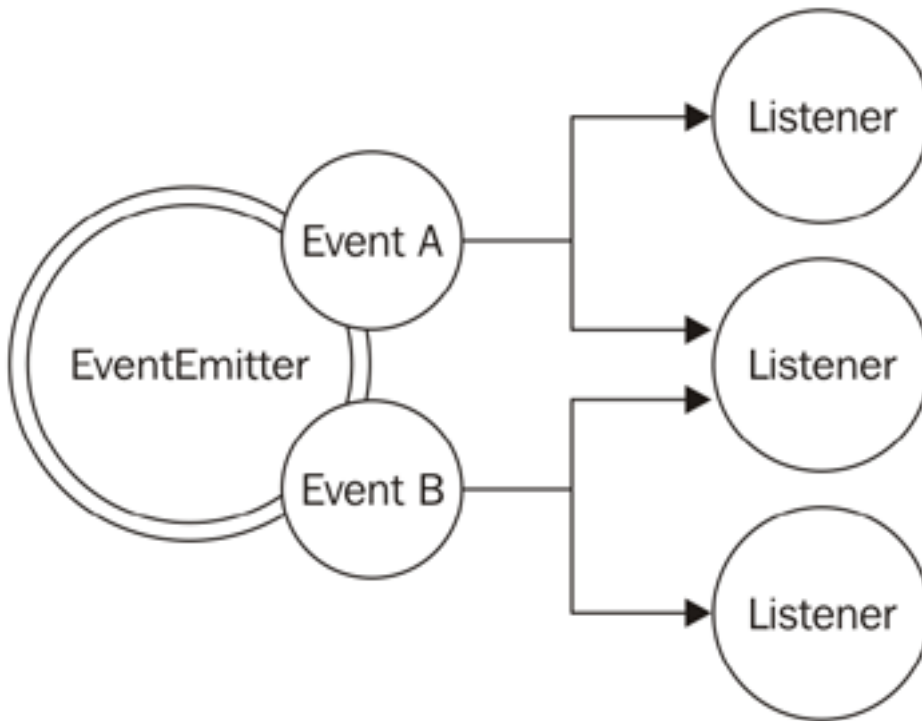


Figure 3.2: Listeners receiving events from an EventEmitter

Hình 3.2: Các listener nhận sự kiện từ một EventEmitter

The EventEmitter is exported from the events core module. The following code shows how we can obtain a reference to it:

EventEmitter được xuất ra từ module lõi events. Đoạn mã sau cho thấy cách ta có thể lấy được một tham chiếu tới nó:

```
import { EventEmitter } from 'events'
const emitter = new EventEmitter()
```

The essential methods of the EventEmitter are as follows:

Các phương thức cốt yếu của EventEmitter như sau:

- `on(event, listener)`: This method allows us to register a new listener (a function) for the given event type (a string).
- `once(event, listener)`: This method registers a new listener, which is then removed after the event is emitted for the first time.
- `emit(event, [arg1], [...])`: This method produces a new event and provides additional arguments to be passed to the listeners.
- `removeListener(event, listener)`: This method removes a listener for the specified event type.
 - `on(event, listener)`: Phương thức này cho phép ta đăng ký một listener mới (một hàm) cho loại sự kiện đã cho (một chuỗi).
 - `once(event, listener)`: Phương thức này đăng ký một listener mới, listener này sau đó sẽ bị gỡ bỏ sau khi sự kiện được phát ra lần đầu tiên.
 - `emit(event, [arg1], [...])`: Phương thức này tạo ra một sự kiện mới và cung cấp các đối số bổ sung để truyền cho các listener.

- `removeListener(event, listener)`: Phương thức này gỡ bỏ một listener cho loại sự kiện được chỉ định.

All the preceding methods will return the `EventEmitter` instance to allow chaining. The listener function has the signature `function([arg1], [...])`, so it simply accepts the arguments provided at the moment the event is emitted.

Tất cả các phương thức trên sẽ trả về thực thể `EventEmitter` để cho phép chaining (nối chuỗi). Hàm listener có chữ ký `function([arg1], [...])`, nên nó chỉ đơn giản nhận các đối số được cung cấp tại thời điểm sự kiện được phát ra.

You can already see that there is a big difference between a listener and a traditional Node.js callback. In fact, the first argument is not an error, but it can be any data passed to `emit()` at the moment of its invocation.

Bạn đã có thể thấy có một khác biệt lớn giữa một listener và một callback Node.js truyền thống. Thực tế, đối số đầu tiên không phải là một lỗi, mà có thể là bất kỳ dữ liệu nào được truyền cho `emit()` tại thời điểm gọi nó.

Creating and using the EventEmitter — Tạo và sử dụng EventEmitter

Let's now see how we can use an `EventEmitter` in practice. The simplest way is to create a new instance and use it immediately. The following code shows us a function that uses an `EventEmitter` to notify its subscribers in real time when a particular regular expression is matched in a list of files:

Giờ hãy xem ta có thể dùng một `EventEmitter` trên thực tế như thế nào. Cách đơn giản nhất là tạo một thực thể mới và dùng nó ngay lập tức. Đoạn mã sau cho ta thấy một hàm dùng `EventEmitter` để thông báo cho những người đăng ký (subscriber) của nó theo thời gian thực khi một biểu thức chính quy (regular expression) cụ thể khớp trong một danh sách tệp:

```
import { EventEmitter } from 'events'
import { readFile } from 'fs'
function findRegex (files, regex) {
  const emitter = new EventEmitter()
  for (const file of files) {
    readFile(file, 'utf8', (err, content) => {
      if (err) {
        return emitter.emit('error', err)
      }
      emitter.emit('fileread', file)
      const match = content.match(regex)
      if (match) {
        match.forEach(elem => emitter.emit('found', file, elem))
      }
    })
  }
  return emitter
}
```

The function we just defined returns an `EventEmitter` instance that will produce three events:

Hàm ta vừa định nghĩa trả về một thực thể `EventEmitter` sẽ tạo ra ba sự kiện:

- `fileread`, when a file is being read

- found, when a match has been found
- error, when an error occurs during reading the file
 - `fileread`, khi một tệp đang được đọc
 - `found`, khi một kết quả khớp được tìm thấy
 - `error`, khi một lỗi xảy ra trong quá trình đọc tệp

Let's now see how our `findRegex()` function can be used:

Giờ hãy xem hàm `findRegex()` của ta có thể được dùng như thế nào:

```
findRegex(
  ['fileA.txt', 'fileB.json'],
  /hello \w+/g
)
.on('fileread', file => console.log(`${file} was read`))
.on('found', (file, match) => console.log(`Matched "${match}" in
${file}`))
.on('error', err => console.error(`Error emitted ${err.message}`))
```

In the code we just defined, we register a listener for each of the three event types produced by the `EventEmitter` that was created by our `findRegex()` function.

Trong đoạn mã ta vừa định nghĩa, ta đăng ký một listener cho mỗi loại trong ba loại sự kiện do `EventEmitter` được tạo bởi hàm `findRegex()` của ta tạo ra.

Propagating errors — Lan truyền lỗi

As with callbacks, the `EventEmitter` can't just throw an exception when an error condition occurs. Instead, the convention is to emit a special event, called `error`, and pass an `Error` object as an argument. That's exactly what we were doing in the `findRegex()` function that we defined earlier.

Cũng như với callback, `EventEmitter` không thể chỉ đơn giản throw một ngoại lệ khi một điều kiện lỗi xảy ra. Thay vào đó, quy ước là phát ra một sự kiện đặc biệt, gọi là `error`, và truyền một đối tượng `Error` làm đối số. Đó chính xác là điều ta đã làm trong hàm `findRegex()` mà ta định nghĩa trước đó.

The `EventEmitter` treats the `error` event in a special way. It will automatically throw an exception and exit from the application if such an event is emitted and no associated listener is found. For this reason, it is recommended to always register a listener for the `error` event.

`EventEmitter` xử lý sự kiện `error` theo một cách đặc biệt. Nó sẽ tự động ném một ngoại lệ và thoát khỏi ứng dụng nếu một sự kiện như vậy được phát ra mà không tìm thấy listener tương ứng nào. Vì lý do này, được khuyến nghị là luôn đăng ký một listener cho sự kiện `error`.

Making any object observable — Biến bất kỳ đối tượng nào thành observable

In the Node.js world, the `EventEmitter` is rarely used on its own, as you saw in the previous example. Instead, it is more common to see it extended by other classes. In practice, this enables any class to inherit the capabilities of the `EventEmitter`, hence becoming an observable object. To demonstrate this pattern, let's try to implement the functionality of the `findRegex()` function in a class, as follows:

Trong thế giới Node.js, EventEmitter hiếm khi được dùng đơn lẻ, như bạn đã thấy trong ví dụ trước. Thay vào đó, phổ biến hơn là thấy nó được kế thừa bởi các lớp khác. Trên thực tế, điều này cho phép bất kỳ lớp nào kế thừa các khả năng của EventEmitter, qua đó trở thành một đối tượng observable. Để minh họa mẫu này, hãy thử hiện thực chức năng của hàm `findRegex()` trong một lớp, như sau:

```
import { EventEmitter } from 'events'
import { readFile } from 'fs'
class FindRegex extends EventEmitter {
  constructor (regex) {
    super()
    this.regex = regex
    this.files = []
  }
  addFile (file) {
    this.files.push(file)
    return this
  }
  find () {
    for (const file of this.files) {
      readFile(file, 'utf8', (err, content) => {
        if (err) {
          return this.emit('error', err)
        }
        this.emit('fileread', file)
        const match = content.match(this.regex)
        if (match) {
          match.forEach(elem => this.emit('found', file, elem))
        }
      })
    }
    return this
  }
}
```

The `FindRegex` class that we just defined extends `EventEmitter` to become a fully fledged observable class. Always remember to use `super()` in the constructor to initialize the `EventEmitter` internals. The following is an example of how to use the `FindRegex` class we just defined:

Lớp `FindRegex` mà ta vừa định nghĩa kế thừa `EventEmitter` để trở thành một lớp observable đầy đủ. Luôn nhớ dùng `super()` trong constructor để khởi tạo các phần nội bộ của `EventEmitter`. Sau đây là một ví dụ về cách dùng lớp `FindRegex` mà ta vừa định nghĩa:

```
const findRegexInstance = new FindRegex(/hello \w+/)
findRegexInstance
  .addFile('fileA.txt')
  .addFile('fileB.json')
  .find()
  .on('found', (file, match) => console.log(`Matched "${match}" in file ${file}`))
  .on('error', err => console.error(`Error emitted ${err.message}`))
```

You will now notice how the `FindRegex` object also provides the `on()` method, which is inherited from the `EventEmitter`. This is a pretty common pattern in the Node.js ecosystem. For example, the `Server` object of the core HTTP module inherits from the `EventEmitter` function, thus allowing it to produce events such as `request` (when a new request is received), `connection` (when a new connection is established), or `closed` (when the server socket is closed).

Giờ bạn sẽ để ý rằng đối tượng `FindRegex` cũng cung cấp phương thức `on()`, được kế thừa từ `EventEmitter`. Đây là một mẫu khá phổ biến trong hệ sinh thái Node.js. Ví dụ, đối tượng `Server` của module HTTP kế thừa từ hàm `EventEmitter`, qua đó cho phép nó tạo ra các sự kiện như `request` (khi một yêu cầu mới được nhận), `connection` (khi một kết nối mới được thiết lập), hoặc `closed` (khi socket của server bị đóng).

Other notable examples of objects extending the `EventEmitter` are Node.js streams. We will analyze streams in more detail in Chapter 6, Coding with Streams.

Các ví dụ đáng chú ý khác về các đối tượng kế thừa `EventEmitter` là các stream của Node.js. Ta sẽ phân tích stream chi tiết hơn trong Chương 6, Coding with Streams.

EventEmitter and memory leaks — EventEmitter và rò rỉ bộ nhớ

When subscribing to observables with a long life span, it is extremely important that we unsubscribe our listeners once they are no longer needed. This allows us to release the memory used by the objects in a listener's **scope** and prevent memory leaks. Unreleased `EventEmitter` listeners are the main source of memory leaks in Node.js (and JavaScript in general).

Khi đăng ký với các observable có vòng đời dài, việc ta huỷ đăng ký các listener của mình một khi chúng không còn cần thiết nữa là cực kỳ quan trọng. Điều này cho phép ta giải phóng bộ nhớ được dùng bởi các đối tượng trong scope của một listener và ngăn ngừa rò rỉ bộ nhớ (memory leak). Các listener `EventEmitter` không được giải phóng là nguồn chính gây rò rỉ bộ nhớ trong Node.js (và JavaScript nói chung).

A memory leak is a software defect whereby memory that is no longer needed is not released, causing the memory usage of an application to grow indefinitely. For example, consider the following code:

Rò rỉ bộ nhớ là một khiếm khuyết phần mềm theo đó bộ nhớ không còn cần thiết nữa lại không được giải phóng, khiến lượng bộ nhớ sử dụng của một ứng dụng tăng vô hạn định. Ví dụ, hãy xét đoạn mã sau:

```
const thisTakesMemory = 'A big string....'
const listener = () => {
  console.log(thisTakesMemory)
}
emitter.on('an_event', listener)
```

The variable `thisTakesMemory` is referenced in the listener and therefore its memory is retained until the listener is released from emitter, or until the emitter itself is garbage collected, which can only happen when there are no more active references to it, making it unreachable. You can find a good explanation about garbage collection in JavaScript and the concept of reachability at nodesdp.link/garbage-collection.

Biến `thisTakesMemory` được tham chiếu trong listener và do đó bộ nhớ của nó được giữ lại cho đến khi listener được giải phóng khỏi emitter, hoặc cho đến khi chính emitter được thu gom rác (garbage collected), điều chỉ có thể xảy ra khi không còn tham chiếu hoạt động nào tới nó, khiến nó không thể tiếp cận được (unreachable). Bạn có thể tìm thấy

một lời giải thích hay về thu gom rác trong JavaScript và khái niệm khả năng tiếp cận (reachability) tại nodejsdp.link/garbage-collection.

This means that if an `EventEmitter` remains reachable for the entire duration of the application, all its listeners do too, and with them all the memory they reference. If, for example, we register a listener to a “permanent” `EventEmitter` at every incoming HTTP request and never release it, then we are causing a memory leak. The memory used by the application will grow indefinitely, sometimes slowly, sometimes faster, but eventually it will crash the application. To prevent such a situation, we can release the listener with the `removeListener()` method of the `EventEmitter`:

Điều này nghĩa là nếu một `EventEmitter` vẫn còn có thể tiếp cận được trong suốt thời gian tồn tại của ứng dụng, thì tất cả các listener của nó cũng vậy, và cùng với chúng là toàn bộ bộ nhớ mà chúng tham chiếu. Ví dụ, nếu ta đăng ký một listener cho một `EventEmitter` “vĩnh viễn” ở mỗi yêu cầu HTTP đến và không bao giờ giải phóng nó, thì ta đang gây ra một rò rỉ bộ nhớ. Lượng bộ nhớ được ứng dụng sử dụng sẽ tăng vô hạn định, đôi khi chậm, đôi khi nhanh hơn, nhưng cuối cùng nó sẽ làm sập ứng dụng. Để ngăn ngừa tình huống như vậy, ta có thể giải phóng listener bằng phương thức `removeListener()` của `EventEmitter`:

```
emitter.removeListener('an_event', listener)
```

An `EventEmitter` has a very simple built-in mechanism for warning the developer about possible memory leaks. When the count of listeners registered to an event exceeds a specific amount (by default, 10), the `EventEmitter` will produce a warning. Sometimes, registering more than 10 listeners is completely fine, so we can adjust this limit by using the `setMaxListeners()` method of the `EventEmitter`.

Một `EventEmitter` có một cơ chế tích hợp rất đơn giản để cảnh báo lập trình viên về khả năng rò rỉ bộ nhớ. Khi số lượng listener được đăng ký cho một sự kiện vượt quá một mức cụ thể (mặc định là 10), `EventEmitter` sẽ tạo ra một cảnh báo. Đôi khi, việc đăng ký hơn 10 listener là hoàn toàn ổn, nên ta có thể điều chỉnh giới hạn này bằng phương thức `setMaxListeners()` của `EventEmitter`.

We can use the convenience method `once(event, listener)` in place of `on(event, listener)` to automatically unregister a listener after the event is received for the first time. However, be advised that if the event we specify is never emitted, then the listener is never released, causing a memory leak.

Ta có thể dùng phương thức tiện lợi `once(event, listener)` thay cho `on(event, listener)` để tự động huỷ đăng ký một listener sau khi sự kiện được nhận lần đầu tiên. Tuy nhiên, hãy lưu ý rằng nếu sự kiện ta chỉ định không bao giờ được phát ra, thì listener sẽ không bao giờ được giải phóng, gây ra một rò rỉ bộ nhớ.

Synchronous and asynchronous events — Sự kiện đồng bộ và bất đồng bộ

As with callbacks, events can also be emitted synchronously or asynchronously with respect to the moment the tasks that produce them are triggered. It is crucial that we never mix the two approaches in the same `EventEmitter`, but even more importantly, we should never emit the same event type using a mix of synchronous and asynchronous code, to avoid producing the same problems described in the Unleashing Zalgo section. The main difference between emitting synchronous and asynchronous events lies in the way listeners can be registered.

Cũng như với callback, sự kiện cũng có thể được phát ra một cách đồng bộ hoặc bất đồng bộ so với thời điểm các tác vụ tạo ra chúng được kích hoạt. Việc ta không bao giờ trộn lẫn

hai cách tiếp cận trong cùng một EventEmitter là cốt yếu, nhưng quan trọng hơn nữa, ta không bao giờ nên phát ra cùng một loại sự kiện bằng cách trộn lẫn mã đồng bộ và bất đồng bộ, để tránh tạo ra cùng những vấn đề được mô tả trong phần Giải phóng Zalgo. Khác biệt chính giữa việc phát ra sự kiện đồng bộ và bất đồng bộ nằm ở cách các listener có thể được đăng ký.

When events are emitted asynchronously, we can register new listeners, even after the task that produces the events is triggered, up until the current stack yields to the event loop. This is because the events are guaranteed not to be fired until the next cycle of the event loop, so we can be sure that we won't miss any events. The `FindRegex()` class we defined previously emits its events asynchronously after the `find()` method is invoked. This is why we can register the listeners after the `find()` method is invoked, without losing any events, as shown in the following code:

Khi các sự kiện được phát ra một cách bất đồng bộ, ta có thể đăng ký các listener mới, ngay cả sau khi tác vụ tạo ra các sự kiện được kích hoạt, cho tới khi stack hiện tại nhường lại cho event loop. Đó là bởi các sự kiện được bảo đảm sẽ không được kích hoạt cho đến chu kỳ tiếp theo của event loop, nên ta có thể chắc chắn rằng ta sẽ không bỏ lỡ sự kiện nào. Lớp `FindRegex()` mà ta định nghĩa trước đó phát ra các sự kiện của nó một cách bất đồng bộ sau khi phương thức `find()` được gọi. Đây là lý do vì sao ta có thể đăng ký các listener sau khi phương thức `find()` được gọi, mà không mất bất kỳ sự kiện nào, như được thể hiện trong đoạn mã sau:

```
findRegexInstance
  .addFile(...)
  .find()
  .on('found', ...)
```

On the other hand, if we emit our events synchronously after the task is launched, we have to register all the listeners before we launch the task, or we will miss all the events. To see how this works, let's modify the `FindRegex` class we defined previously and make the `find()` method synchronous:

Ở chiều ngược lại, nếu ta phát ra các sự kiện của mình một cách đồng bộ sau khi tác vụ được khởi chạy, ta phải đăng ký tất cả các listener trước khi ta khởi chạy tác vụ, nếu không ta sẽ bỏ lỡ tất cả các sự kiện. Để xem cách này hoạt động ra sao, hãy sửa lớp `FindRegex` mà ta định nghĩa trước đó và làm cho phương thức `find()` trở thành đồng bộ:

```
find () {
  for (const file of this.files) {
    let content
    try {
      content = readFileSync(file, 'utf8')
    } catch (err) {
      this.emit('error', err)
    }
    this.emit('fileread', file)
    const match = content.match(this.regex)
    if (match) {
      match.forEach(elem => this.emit('found', file, elem))
    }
  }
  return this
}
```

Now, let's try to register a listener before we launch the `find()` task, and then a second listener after

that to see what happens:

Giờ hãy thử đăng ký một listener trước khi ta khởi chạy tác vụ `find()`, rồi một listener thứ hai sau đó để xem điều gì xảy ra:

```
const findRegexSyncInstance = new FindRegexSync(/hello \w+/)
findRegexSyncInstance
  .addFile('fileA.txt')
  .addFile('fileB.json')
  // this listener is invoked
  .on('found', (file, match) => console.log(`[Before] Matched
"${match}"`))
  .find()
  // this listener is never invoked
  .on('found', (file, match) => console.log(`[After] Matched
"${match}"`))
```

As expected, the listener that was registered after the invocation of the `find()` task is never called; in fact, the preceding code will print:

Đúng như mong đợi, listener được đăng ký sau khi gọi tác vụ `find()` không bao giờ được gọi; thực tế, đoạn mã trên sẽ in ra:

```
[Before] Matched "hello world"
[Before] Matched "hello NodeJS"
```

There are some (rare) situations in which emitting an event in a synchronous fashion makes sense, but the very nature of the `EventEmitter` lies in its ability to deal with asynchronous events. Most of the time, emitting events synchronously is a telltale sign that we either don't need the `EventEmitter` at all or that, somewhere else, the same observable is emitting another event asynchronously, potentially causing a *Zalgo* type of situation.

Có một số tình huống (hiếm) mà việc phát ra một sự kiện theo cách đồng bộ là hợp lý, nhưng bản chất cốt lõi của `EventEmitter` nằm ở khả năng xử lý các sự kiện bất đồng bộ của nó. Hầu hết thời gian, việc phát ra sự kiện một cách đồng bộ là một dấu hiệu rõ ràng rằng hoặc ta không cần `EventEmitter` chút nào, hoặc ở đâu đó, cùng một observable đang phát ra một sự kiện khác một cách bất đồng bộ, có khả năng gây ra một tình huống kiểu *Zalgo*.

The emission of synchronous events can be deferred with `process.nextTick()` to guarantee that they are emitted asynchronously.

Việc phát ra các sự kiện đồng bộ có thể được hoãn lại bằng `process.nextTick()` để bảo đảm chúng được phát ra một cách bất đồng bộ.

EventEmitter versus callbacks — EventEmitter so với callback

A common dilemma when defining an asynchronous API is deciding whether to use an `EventEmitter` or simply accept a callback. The general differentiating rule is semantic: callbacks should be used when a result must be returned in an asynchronous way, while events should be used when there is a need to communicate that something has happened.

Một tình huống tiến thoái lưỡng nan phổ biến khi định nghĩa một API bất đồng bộ là quyết định liệu nên dùng một `EventEmitter` hay đơn giản là nhận một callback. Quy tắc phân biệt tổng quát mang tính ngữ nghĩa: callback nên được dùng khi một kết quả phải được

trả về theo cách bất đồng bộ, trong khi sự kiện nên được dùng khi có nhu cầu truyền đạt rằng một điều gì đó đã xảy ra.

But besides this simple principle, a lot of confusion is generated from the fact that the two paradigms are, most of the time, equivalent and allow us to achieve the same results. Consider the following code as an example:

Nhưng bên cạnh nguyên tắc đơn giản này, rất nhiều sự rối rắm được tạo ra từ thực tế rằng hai mô hình này, hầu hết thời gian, là tương đương và cho phép ta đạt được cùng những kết quả. Hãy xét đoạn mã sau làm ví dụ:

```
import { EventEmitter } from 'events'
function helloEvents () {
  const eventEmitter = new EventEmitter()
  setTimeout(() => eventEmitter.emit('complete', 'hello world'), 100)
  return eventEmitter
}
function helloCallback (cb) {
  setTimeout(() => cb(null, 'hello world'), 100)
}
helloEvents().on('complete', message => console.log(message))
helloCallback((err, message) => console.log(message))
```

The two functions `helloEvents()` and `helloCallback()` can be considered equivalent in terms of functionality. The first communicates the completion of the timeout using an event, while the second uses a callback. But what really differentiates them is the readability, the semantics, and the amount of code that is required for them to be implemented or used.

Hai hàm `helloEvents()` và `helloCallback()` có thể được xem là tương đương về mặt chức năng. Hàm đầu tiên truyền đạt sự hoàn tất của timeout bằng một sự kiện, trong khi hàm thứ hai dùng một callback. Nhưng thứ thực sự phân biệt chúng là tính dễ đọc, ngữ nghĩa, và lượng mã cần thiết để chúng được hiện thực hoặc sử dụng.

While a deterministic set of rules for you to choose between one style or the other can't be given, here are some hints to help you make a decision on which method to use:

Tuy không thể đưa ra một tập quy tắc tất định để bạn chọn giữa phong cách này hay phong cách kia, sau đây là một số gợi ý giúp bạn quyết định nên dùng phương pháp nào:

- Callbacks have some limitations when it comes to supporting different types of events. In fact, we can still differentiate between multiple events by passing the type as an argument of the callback, or by accepting several callbacks, one for each supported event. However, this can't exactly be considered an elegant API. In this situation, the `EventEmitter` can give a better interface and leaner code.
- The `EventEmitter` should be used when the same event can occur multiple times, or may not occur at all. A callback, in fact, is expected to be invoked exactly once, whether the operation is successful or not. Having a possibly repeating circumstance should make us think again about the semantic nature of the occurrence, which is more similar to an event that has to be communicated, rather than a result to be returned.
- An API that uses callbacks can notify only one particular callback, while using an `EventEmitter` allows us to register multiple listeners for the same event.
 - Callback có một số hạn chế khi nói đến việc hỗ trợ các loại sự kiện khác nhau. Thực tế, ta vẫn có thể phân biệt giữa nhiều sự kiện bằng cách truyền loại sự kiện làm đối số của callback, hoặc bằng cách nhận nhiều callback, mỗi callback cho một sự kiện

được hỗ trợ. Tuy nhiên, điều này không hẳn có thể được xem là một API thanh lịch. Trong tình huống này, EventEmitter có thể đem lại một giao diện tốt hơn và mã gọn gàng hơn.

- EventEmitter nên được dùng khi cùng một sự kiện có thể xảy ra nhiều lần, hoặc có thể không xảy ra chút nào. Một callback, thực tế, được kỳ vọng được gọi đúng một lần, dù thao tác thành công hay không. Việc có một tình huống có thể lặp lại nên khiến ta phải suy nghĩ lại về bản chất ngữ nghĩa của sự việc, vốn giống một sự kiện cần được truyền đạt hơn là một kết quả cần được trả về.
- Một API dùng callback chỉ có thể thông báo cho một callback cụ thể duy nhất, trong khi việc dùng một EventEmitter cho phép ta đăng ký nhiều listener cho cùng một sự kiện.

Combining callbacks and events — Kết hợp callback và sự kiện

There are some particular circumstances where the EventEmitter can be used in conjunction with a callback. This pattern is extremely powerful as it allows us to pass a result asynchronously using a traditional callback, and at the same time return an EventEmitter, which can be used to provide a more detailed account on the status of an asynchronous process. One example of this pattern is offered by the glob **package** (nodejsdp.link/npm-glob), a library that performs glob-style file searches. The main entry point of the module is the function it exports, which has the following signature:

Có một số tình huống cụ thể mà EventEmitter có thể được dùng kết hợp với một callback. Mẫu này cực kỳ mạnh mẽ bởi nó cho phép ta truyền một kết quả một cách bất đồng bộ dùng một callback truyền thống, và đồng thời trả về một EventEmitter, vốn có thể được dùng để cung cấp một báo cáo chi tiết hơn về trạng thái của một tiến trình bất đồng bộ. Một ví dụ về mẫu này được cung cấp bởi gói glob (nodejsdp.link/npm-glob), một thư viện thực hiện tìm kiếm tệp theo kiểu glob. Điểm vào (entry point) chính của module là hàm mà nó xuất ra, hàm này có chữ ký sau:

```
const EventEmitter = glob(pattern, [options], callback)
```

The function takes a pattern as the first argument, a set of options, and a callback that is invoked with the list of all the files matching the provided pattern. At the same time, the function returns an EventEmitter, which provides a more fine-grained report about the state of the search process. For example, it is possible to be notified in real time when a match occurs by listening to the match event, to obtain the list of all the matched files with the end event, or to know whether the process was manually aborted by listening to the abort event. The following code shows what this looks like in practice:

Hàm nhận một pattern làm đối số đầu tiên, một tập các options, và một callback được gọi với danh sách tất cả các tệp khớp với pattern được cung cấp. Đồng thời, hàm trả về một EventEmitter, vốn cung cấp một báo cáo chi tiết hơn về trạng thái của tiến trình tìm kiếm. Ví dụ, có thể được thông báo theo thời gian thực khi một kết quả khớp xảy ra bằng cách lắng nghe sự kiện match, có thể lấy danh sách tất cả các tệp khớp với sự kiện end, hoặc biết được liệu tiến trình có bị huỷ thủ công hay không bằng cách lắng nghe sự kiện abort. Đoạn mã sau cho thấy điều này trông như thế nào trên thực tế:

```
import glob from 'glob'
glob('data/*.txt',
  (err, files) => {
    if (err) {
      return console.error(err)
    }
  })
```

```

    }
    console.log(`All files found: ${JSON.stringify(files)}`)
  })
  .on('match', match => console.log(`Match found: ${match}`))

```

Combining an EventEmitter with traditional callbacks is an elegant way to offer two different approaches to the same API. One approach is usually meant to be simpler and more immediate to use, while the other is targeted at more advanced scenarios.

Việc kết hợp một EventEmitter với các callback truyền thống là một cách thanh lịch để cung cấp hai cách tiếp cận khác nhau cho cùng một API. Một cách tiếp cận thường được dùng để đơn giản hơn và dùng được ngay tức thì hơn, trong khi cách kia nhắm tới những kịch bản nâng cao hơn.

The EventEmitter can also be combined with other asynchronous mechanisms such as promises (which we will look at in Chapter 5, Asynchronous Control Flow Patterns with Promises and Async/ Await). In this case, just return an object (or array) containing both the **promise** and the EventEmitter. This object can then be destructured by the caller, like this: `{promise, events} = foo()`.

EventEmitter cũng có thể được kết hợp với các cơ chế bất đồng bộ khác như promise (mà ta sẽ xem xét trong Chương 5, Asynchronous Control Flow Patterns with Promises and Async/ Await). Trong trường hợp này, chỉ cần trả về một đối tượng (hoặc mảng) chứa cả promise và EventEmitter. Đối tượng này sau đó có thể được phá cấu trúc (destructured) bởi bên gọi, như sau: `{promise, events} = foo()`.

Summary — Tóm tắt

In this chapter, we made our first contact with the practical aspects of writing asynchronous code. You discovered the two pillars of the entire Node.js asynchronous infrastructure—the callback and the EventEmitter—and we explored in detail their use cases, conventions, and patterns. We also explored some of the pitfalls of dealing with asynchronous code and you learned about the ways to avoid them. Mastering the content of this chapter paves the way toward learning the more advanced asynchronous techniques that will be presented throughout the rest of this book.

Trong chương này, ta đã có lần tiếp xúc đầu tiên với các khía cạnh thực tiễn của việc viết mã bất đồng bộ. Bạn đã khám phá hai trụ cột của toàn bộ hạ tầng bất đồng bộ của Node.js — callback và EventEmitter — và ta đã khám phá chi tiết các trường hợp sử dụng, các quy ước, và các mẫu của chúng. Ta cũng đã khám phá một số cạm bẫy khi làm việc với mã bất đồng bộ và bạn đã học được các cách để tránh chúng. Việc làm chủ nội dung của chương này mở đường cho việc học các kỹ thuật bất đồng bộ nâng cao hơn sẽ được trình bày xuyên suốt phần còn lại của cuốn sách này.

In the next chapter, you will learn how to deal with complex asynchronous control flows using callbacks.

Trong chương tiếp theo, bạn sẽ học cách xử lý các luồng điều khiển bất đồng bộ phức tạp bằng callback.

Exercises — Bài tập

- 3.1 A simple event: Modify the asynchronous FindRegex class so that it emits an event when the find process starts, passing the input files list as an argument. Hint: beware of Zalgo!
 - 3.2 Ticker: Write a function that accepts a number and a callback as the arguments. The function will return an EventEmitter that emits an event called tick every 50 milliseconds until the number of milliseconds is passed from the invocation of the function. The function will also call the callback when the number of milliseconds has passed, providing, as the result, the total count of tick events emitted. Hint: you can use setTimeout() to schedule another setTimeout() recursively.
 - 3.3 A simple modification: Modify the function created in exercise 3.2 so that it emits a tick event immediately after the function is invoked.
 - 3.4 Playing with errors: Modify the function created in exercise 3.3 so that it produces an error if the timestamp at the moment of a tick (including the initial one that we added as part of exercise 3.3) is divisible by 5. Propagate the error using both the callback and the **event emitter**. Hint: use Date.now() to get the timestamp and the remainder (%) operator to check whether the timestamp is divisible by 5.
-
- 3.1 Một sự kiện đơn giản: Sửa lớp FindRegex bất đồng bộ sao cho nó phát ra một sự kiện khi tiến trình tìm kiếm bắt đầu, truyền danh sách các tệp đầu vào làm đối số. Gợi ý: hãy cẩn thận với Zalgo!
 - 3.2 Ticker: Viết một hàm nhận một number và một callback làm đối số. Hàm sẽ trả về một EventEmitter phát ra một sự kiện gọi là tick mỗi 50 mili-giây cho đến khi number mili-giây đã trôi qua kể từ lời gọi hàm. Hàm cũng sẽ gọi callback khi number mili-giây đã trôi qua, cung cấp, làm kết quả, tổng số sự kiện tick đã được phát ra. Gợi ý: bạn có thể dùng setTimeout() để lên lịch một setTimeout() khác một cách đệ quy.
 - 3.3 Một sửa đổi đơn giản: Sửa hàm được tạo trong bài tập 3.2 sao cho nó phát ra một sự kiện tick ngay lập tức sau khi hàm được gọi.
 - 3.4 Nghịch với lỗi: Sửa hàm được tạo trong bài tập 3.3 sao cho nó tạo ra một lỗi nếu dấu thời gian (timestamp) tại thời điểm của một tick (bao gồm cả tick ban đầu mà ta đã thêm như một phần của bài tập 3.3) chia hết cho 5. Lan truyền lỗi bằng cả callback lẫn event emitter. Gợi ý: dùng Date.now() để lấy timestamp và toán tử lấy phần dư (%) để kiểm tra liệu timestamp có chia hết cho 5 hay không.

Chapter 4: Asynchronous Control Flow Patterns with Callbacks —

Chương 4: Các mẫu điều khiển luồng bất đồng bộ với Callback

Moving from a synchronous programming style to a platform such as Node.js, where continuation-passing style (CPS) and asynchronous APIs are the norm, can be frustrating. Asynchronous code can make it hard to predict the order in which statements are executed. Simple problems such as iterating over a set of files, executing tasks in sequence, or waiting for a set of operations to complete require the developer to take on new approaches and techniques just to avoid ending up writing inefficient and unreadable code. When using callbacks to deal with asynchronous **control flow**, the most common mistake is to fall into the trap of **callback hell** and see the code growing horizontally, rather than vertically, with a nesting that makes even simple routines hard to read and maintain.

Chuyển từ phong cách lập trình đồng bộ sang một nền tảng như Node.js, nơi continuation-passing style (CPS) và các API bất đồng bộ là chuẩn mực, có thể khiến bạn nản lòng. Code bất đồng bộ có thể làm cho việc dự đoán thứ tự thực thi các câu lệnh trở nên khó khăn. Những bài toán đơn giản như duyệt qua một tập hợp các tệp, thực thi các tác vụ tuần tự, hay chờ một tập hợp các thao tác hoàn tất đều buộc lập trình viên phải tiếp nhận những hướng tiếp cận và kỹ thuật mới chỉ để tránh kết cục là viết ra code kém hiệu quả và khó đọc. Khi dùng callback để xử lý luồng điều khiển bất đồng bộ, sai lầm phổ biến nhất là rơi vào cái bẫy callback hell và chứng kiến code phình to theo chiều ngang thay vì chiều dọc, với mức độ lồng nhau khiến cho ngay cả những thủ tục đơn giản cũng trở nên khó đọc và khó bảo trì.

In this chapter, you will see how it's actually possible to tame callbacks and write clean, manageable asynchronous code by using some discipline and with the aid of some patterns. Knowing how to properly deal with callbacks will pave the way for adopting modern approaches such as promises and **async/await**.

Trong chương này, bạn sẽ thấy thực ra hoàn toàn có thể thuần hóa callback và viết code bất đồng bộ sạch sẽ, dễ quản lý bằng cách áp dụng chút kỷ luật và sự hỗ trợ của một vài mẫu thiết kế. Biết cách xử lý callback đúng đắn sẽ mở đường cho việc tiếp nhận các hướng tiếp cận hiện đại như promise và **async/await**.

In short, in this chapter, you will learn about:

Tóm lại, trong chương này, bạn sẽ học về:

- The challenges of asynchronous programming.
- Avoiding callback hell and other callback best practices.
- Common asynchronous patterns such as **sequential execution**, sequential iteration, **parallel execution**, and limited parallel execution.
 - Những thách thức của lập trình bất đồng bộ.
 - Cách tránh callback hell và các best practice khác về callback.
 - Các mẫu bất đồng bộ phổ biến như thực thi tuần tự, lập tuần tự, thực thi song song và thực thi song song có giới hạn.

The difficulties of asynchronous programming — Những khó khăn của lập trình bất đồng bộ

Losing control of asynchronous code in JavaScript is undoubtedly easy. Closures and in-place definitions of anonymous functions allow for a smooth programming experience that doesn't require the developer to jump to other points in the codebase. This is perfectly in line with the KISS principle (Keep It Simple, Stupid); it's simple, it keeps the code flowing, and we get it working in less time. Unfortunately, sacrificing qualities such as modularity, reusability, and maintainability will, sooner or later, lead to the uncontrolled proliferation of callback nesting, functions growing in size, and poor code organization. Most of the time, creating callbacks as in-place functions is not strictly required, so it's more a matter of discipline than a problem related to asynchronous programming. Recognizing that our code is becoming unwieldy or, even better, knowing in advance that it might become unwieldy and then acting accordingly with the most adequate solution, is what differentiates a novice from an expert.

Đánh mất quyền kiểm soát code bất đồng bộ trong JavaScript chắc chắn là chuyện dễ xảy ra. Closure và việc định nghĩa tại chỗ các hàm vô danh mang lại trải nghiệm lập trình mượt mà, không buộc lập trình viên phải nhảy đến những điểm khác trong codebase. Điều này hoàn toàn phù hợp với nguyên tắc KISS (Keep It Simple, Stupid); nó đơn giản, giữ cho code trôi chảy, và ta hoàn thành việc trong thời gian ngắn hơn. Tiếc thay, việc hy sinh những phẩm chất như tính module hóa, khả năng tái sử dụng và khả năng bảo trì, sớm hay muộn, sẽ dẫn đến sự sinh sôi mất kiểm soát của các lớp callback lồng nhau, các hàm phình to về kích thước, và tổ chức code kém. Phần lớn thời gian, việc tạo callback dưới dạng hàm tại chỗ không thực sự bắt buộc, nên đây là vấn đề về kỷ luật hơn là vấn đề liên quan đến lập trình bất đồng bộ. Nhận ra rằng code của ta đang trở nên khó kiểm soát hoặc, tốt hơn nữa, biết trước rằng nó có thể trở nên khó kiểm soát rồi hành động phù hợp với giải pháp thích đáng nhất, chính là điều phân biệt người mới với chuyên gia.

Creating a simple web spider — Xây dựng một web spider đơn giản

To explain this problem, we will create a little web spider, a command-line application that takes in a web URL as input and downloads its contents locally into a file. In the code presented in this chapter, we are going to use a couple of npm dependencies:

Để giải thích vấn đề này, chúng ta sẽ xây dựng một web spider nhỏ, một ứng dụng dòng lệnh nhận đầu vào là một URL web rồi tải nội dung của nó về một tệp cục bộ. Trong code trình bày ở chương này, chúng ta sẽ dùng một vài phụ thuộc npm:

- `superagent`: A library to streamline HTTP calls ([nodejsdp.link/superagent](https://www.npmjs.com/package/superagent))
- `mkdirp`: A small utility to create directories recursively ([nodejsdp.link/mkdirp](https://www.npmjs.com/package/mkdirp))

- **superagent**: Một thư viện giúp tinh giản các lời gọi HTTP (nodejsdp.link/superagent)
- **mkdirp**: Một tiện ích nhỏ để tạo thư mục theo kiểu đệ quy (nodejsdp.link/mkdirp)

Also, we will often refer to a local **module** named `./utils.js`, which contains some helpers that we will be using in our application. We will omit the contents of this file for brevity, but you can find the full implementation, along with a `package.json` file containing the full list of dependencies, in the official repository at nodejsdp.link/repo.

Ngoài ra, chúng ta sẽ thường xuyên tham chiếu đến một module cục bộ tên là `./utils.js`, chứa một số hàm trợ giúp mà ta sẽ dùng trong ứng dụng. Chúng tôi sẽ lược bỏ nội dung của tệp này cho ngắn gọn, nhưng bạn có thể tìm thấy bản hiện thực đầy đủ, cùng với một tệp `package.json` chứa danh sách đầy đủ các phụ thuộc, trong kho chính thức tại nodejsdp.link/repo.

The core functionality of our application is contained inside a module named `spider.js`. Let's see how it looks. To start with, let's load all the dependencies that we are going to use:

Chức năng cốt lõi của ứng dụng nằm trong một module tên là `spider.js`. Hãy xem nó trông như thế nào. Để bắt đầu, hãy nạp tất cả các phụ thuộc mà ta sẽ dùng:

```
import fs from 'fs'
import path from 'path'
import superagent from 'superagent'
import mkdirp from 'mkdirp'
import { urlToFilename } from './utils.js'
```

Next, let's create a new function named `spider()`, which takes in the URL to download and a callback function that will be invoked when the download process completes:

Tiếp theo, hãy tạo một hàm mới tên là `spider()`, nhận vào URL cần tải và một hàm callback sẽ được gọi khi quá trình tải hoàn tất:

```
export function spider (url, cb) {
  const filename = urlToFilename(url)
  fs.access(filename, err => {                                     // (1)
    if (err && err.code === 'ENOENT') {
      console.log(`Downloading ${url} into ${filename}`)
      superagent.get(url).end((err, res) => {                     // (2)
        if (err) {
          cb(err)
        } else {
          mkdirp(path.dirname(filename), err => {                 // (3)
            if (err) {
              cb(err)
            } else {
              fs.writeFile(filename, res.text, err => {           // (4)
                if (err) {
                  cb(err)
                } else {
                  cb(null, filename, true)
                }
              })
            }
          })
        }
      })
    }
  })
}
```

```

    }
  })
} else {
  cb(null, filename, false)
}
})
}

```

There is a lot going on here, so let's discuss in more detail what happens in every step:

Có khá nhiều thứ diễn ra ở đây, vậy hãy thảo luận chi tiết hơn về điều xảy ra ở từng bước:

1. The code checks whether the URL was already downloaded by verifying that the corresponding file was not already created. If `err` is defined and has type `ENOENT`, then the file does not exist and it's safe to create it:

1. Code kiểm tra xem URL đã được tải về hay chưa bằng cách xác minh rằng tệp tương ứng chưa được tạo. Nếu `err` được định nghĩa và có kiểu `ENOENT`, thì tệp không tồn tại và việc tạo nó là an toàn:

```
fs.access(filename, err => ...
```

2. If the file is not found, the URL is downloaded using the following line of code:

2. Nếu không tìm thấy tệp, URL được tải về bằng dòng code sau:

```
superagent.get(url).end((err, res) => ...
```

3. Then, we make sure that the directory that will contain the file exists:

3. Sau đó, ta đảm bảo rằng thư mục sẽ chứa tệp đã tồn tại:

```
mkdirp(path.dirname(filename), err => ...
```

4. Finally, we write the body of the HTTP response to the filesystem:

4. Cuối cùng, ta ghi phần thân của HTTP response vào hệ thống tệp:

```
fs.writeFile(filename, res.text, err => ...
```

To complete our web spider application, we just need to invoke the `spider()` function by providing a URL as an input (in our case, we read it from the command-line arguments). The `spider()` function is exported from the file we defined previously. Let's now create a new file called `spider-cli.js` that can be directly invoked from the command line:

Để hoàn thiện ứng dụng web spider, ta chỉ cần gọi hàm `spider()` bằng cách cung cấp một URL làm đầu vào (trong trường hợp của ta, ta đọc nó từ tham số dòng lệnh). Hàm `spider()` được export ra từ tệp mà ta đã định nghĩa trước đó. Bây giờ hãy tạo một tệp mới tên là `spider-cli.js` có thể được gọi trực tiếp từ dòng lệnh:

```

import { spider } from './spider.js'
spider(process.argv[2], (err, filename, downloaded) => {
  if (err) {
    console.error(err)
  } else if (downloaded) {
    console.log(`Completed the download of "${filename}"`)
  } else {
    console.log(`${filename} was already downloaded`)
  }
}

```

```
}  
})
```

Now, we are ready to try our web spider application, but first, make sure you have the `utils.js` module and the `package.json` file containing the full list of dependencies in your project directory. Then, install all the dependencies by running the following command:

Bây giờ, ta đã sẵn sàng thử ứng dụng web spider, nhưng trước hết, hãy đảm bảo bạn có module `utils.js` và tệp `package.json` chứa danh sách đầy đủ các phụ thuộc trong thư mục project của mình. Sau đó, cài đặt tất cả các phụ thuộc bằng cách chạy lệnh sau:

```
npm install
```

Now, let's execute the `spider-cli.js` module to download the contents of a web page with a command like this:

Bây giờ, hãy thực thi module `spider-cli.js` để tải nội dung của một trang web bằng một lệnh như sau:

```
node spider-cli.js http://www.example.com
```

Our web spider application requires that we always include the protocol (for example, `http://`) in the URL we provide. Also, do not expect HTML links to be rewritten or resources such as images to be downloaded, as this is just a simple example to demonstrate how asynchronous programming works.

Ứng dụng web spider của ta yêu cầu ta luôn phải bao gồm giao thức (ví dụ, `http://`) trong URL mà ta cung cấp. Ngoài ra, đừng kỳ vọng các liên kết HTML được viết lại hay các tài nguyên như hình ảnh được tải về, vì đây chỉ là một ví dụ đơn giản để minh họa cách lập trình bất đồng bộ hoạt động.

In the next section, you will learn how to improve the readability of this code and, in general, how to keep callback-based code as clean and **readable** as possible.

Trong phần tiếp theo, bạn sẽ học cách cải thiện khả năng đọc của đoạn code này và, nói chung, cách giữ cho code dựa trên callback sạch sẽ và dễ đọc nhất có thể.

Callback hell — Callback hell

Looking at the `spider()` function we defined earlier, you will likely notice that even though the algorithm we implemented is really straightforward, the resulting code has several levels of indentation and is very hard to read. Implementing a similar function with a direct style blocking **API** would be straightforward, and most likely, the code would be much more readable. However, using asynchronous CPS is another story, and making bad use of in-place callback definitions can lead to incredibly bad code.

Nhìn vào hàm `spider()` mà ta đã định nghĩa trước đó, bạn có lẽ sẽ nhận thấy rằng mặc dù thuật toán ta hiện thực thực sự rất đơn giản, code thu được lại có nhiều mức thụt lề và rất khó đọc. Hiện thực một hàm tương tự bằng một API chặn theo phong cách trực tiếp sẽ đơn giản, và rất có thể code sẽ dễ đọc hơn nhiều. Tuy nhiên, dùng CPS bất đồng bộ lại là một câu chuyện khác, và việc lạm dụng việc định nghĩa callback tại chỗ có thể dẫn đến code tệ đến khó tin.

The situation where the abundance of closures and in-place callback definitions transforms the code into an unreadable and unmanageable blob is known as **callback hell**. It's one of the most widely

recognized and severe anti-patterns in Node.js and JavaScript in general. The typical structure of code affected by this problem looks as follows:

Tình huống mà sự lan tràn của các closure và việc định nghĩa callback tại chỗ biến code thành một khối khó đọc và không thể quản lý được gọi là callback hell. Đây là một trong những antipattern được công nhận rộng rãi và nghiêm trọng nhất trong Node.js và JavaScript nói chung. Cấu trúc điển hình của code mắc phải vấn đề này trông như sau:

```
asyncFoo(err => {
  asyncBar(err => {
    asyncFooBar(err => {
      //...
    })
  })
})
```

You can see how code written in this way assumes the shape of a pyramid due to deep nesting, and that's why it is also colloquially known as the pyramid of doom.

Bạn có thể thấy code viết theo cách này mang hình dạng của một kim tự tháp do lồng nhau quá sâu, và đó là lý do tại sao nó cũng được gọi một cách bông đùa là pyramid of doom (kim tự tháp diệt vong).

The most evident problem with code such as the preceding snippet is its poor readability. Due to the nesting being so deep, it's almost impossible to keep track of where a function ends and where another one begins. Another issue is caused by the overlapping of the variable names used in each **scope**. Often, we have to use similar or even identical names to describe the content of a variable. The best example is the error argument received by each callback. Some people often try to use variations of the same name to differentiate the object in each scope, for example, `err`, `error`, `err1`, `err2`, and so on. Others prefer to just shadow the variable defined in the upper scope by always using the same name, for example, `err`. Both alternatives are far from perfect, and cause confusion and increase the probability of introducing defects.

Vấn đề rõ rệt nhất với code như đoạn trên là khả năng đọc kém. Do mức lồng nhau quá sâu, gần như không thể theo dõi được một hàm kết thúc ở đâu và một hàm khác bắt đầu ở đâu. Một vấn đề khác xuất phát từ sự chồng chéo của các tên biến được dùng trong mỗi phạm vi. Thường thì ta phải dùng những tên tương tự hoặc thậm chí giống hệt nhau để mô tả nội dung của một biến. Ví dụ điển hình nhất là tham số lỗi mà mỗi callback nhận được. Một số người thường cố dùng các biến thể của cùng một tên để phân biệt đối tượng trong từng phạm vi, ví dụ, `err`, `error`, `err1`, `err2`, v.v. Những người khác lại thích che (shadow) biến đã định nghĩa ở phạm vi trên bằng cách luôn dùng cùng một tên, ví dụ, `err`. Cả hai lựa chọn đều khác xa hoàn hảo, và gây nhầm lẫn cũng như làm tăng khả năng đưa vào lỗi.

Also, we have to keep in mind that closures come at a small price in terms of performance and memory consumption. In addition, they can create memory leaks that are not very easy to identify. In fact, we shouldn't forget that any context referenced by an active **closure** is retained from garbage collection.

Ngoài ra, ta phải nhớ rằng closure có một cái giá nhỏ về mặt hiệu năng và mức tiêu thụ bộ nhớ. Thêm vào đó, chúng có thể tạo ra rò rỉ bộ nhớ không dễ nhận diện. Thực vậy, ta không nên quên rằng bất kỳ ngữ cảnh nào được tham chiếu bởi một closure đang hoạt động đều được giữ lại khỏi việc thu gom rác (garbage collection).

For a great introduction to how closures work in V8, you can refer to the following blog post

by Vyacheslav Egorov, a software engineer at Google working on V8, which you can read at nodejsdp.link/v8-closures.

Để có một phần giới thiệu tuyệt vời về cách closure hoạt động trong V8, bạn có thể tham khảo bài blog sau của Vyacheslav Egorov, một kỹ sư phần mềm tại Google làm việc trên V8, mà bạn có thể đọc tại nodejsdp.link/v8-closures.

If you look at our `spider()` function, you will notice that it clearly represents a callback hell situation and has all the problems just described. That's exactly what we are going to fix with the patterns and techniques that are covered in the following sections of this chapter.

Nếu bạn nhìn vào hàm `spider()` của ta, bạn sẽ nhận thấy nó rõ ràng đại diện cho một tình huống callback hell và có tất cả các vấn đề vừa được mô tả. Đó chính xác là điều mà ta sẽ khắc phục bằng các mẫu và kỹ thuật được trình bày trong các phần tiếp theo của chương này.

Callback best practices and control flow patterns — Best practice về callback và các mẫu điều khiển luồng

Now that you have met your first example of callback hell, you know what you should definitely avoid; however, that's not the only concern when writing asynchronous code. In fact, there are several situations where controlling the flow of a set of asynchronous tasks requires the use of specific patterns and techniques, especially if we are only using plain JavaScript without the aid of any external library. For example, iterating over a collection by applying an asynchronous operation in sequence is not as easy as invoking `forEach()` over an array; it actually requires a technique similar to recursion.

Giờ đây khi bạn đã gặp ví dụ đầu tiên về callback hell, bạn biết điều mình chắc chắn nên tránh; tuy nhiên, đó không phải là mối lo duy nhất khi viết code bất đồng bộ. Thực vậy, có vài tình huống mà việc kiểm soát luồng của một tập hợp các tác vụ bất đồng bộ đòi hỏi việc dùng những mẫu và kỹ thuật cụ thể, đặc biệt nếu ta chỉ dùng JavaScript thuần mà không có sự hỗ trợ của bất kỳ thư viện ngoài nào. Ví dụ, duyệt qua một tập hợp bằng cách áp dụng một thao tác bất đồng bộ một cách tuần tự không dễ như gọi `forEach()` trên một mảng; nó thực sự đòi hỏi một kỹ thuật tương tự đệ quy.

In this section, you will learn not only about how to avoid callback hell, but also how to implement some of the most common control flow patterns, using only simple and plain JavaScript.

Trong phần này, bạn sẽ học không chỉ về cách tránh callback hell, mà còn cả cách hiện thực một số mẫu điều khiển luồng phổ biến nhất, chỉ dùng JavaScript đơn giản và thuần túy.

Callback discipline — Kỷ luật callback

When writing asynchronous code, the first rule to keep in mind is to not abuse in-place function definitions when defining callbacks. It can be tempting to do so, because it does not require any additional thinking for problems such as modularization and reusability; however, you have seen how this can have more disadvantages than advantages. Most of the time, fixing the callback hell problem does not require any libraries, fancy techniques, or changes of paradigm; you just need some common sense.

Khi viết code bất đồng bộ, quy tắc đầu tiên cần ghi nhớ là không lạm dụng việc định nghĩa hàm tại chỗ khi định nghĩa callback. Việc này có thể rất hấp dẫn, bởi nó không đòi hỏi suy nghĩ thêm cho những vấn đề như module hóa và khả năng tái sử dụng; tuy nhiên, bạn đã thấy điều này có thể có nhiều bất lợi hơn lợi ích. Phần lớn thời gian, khắc phục vấn đề callback hell không đòi hỏi bất kỳ thư viện, kỹ thuật cầu kỳ, hay thay đổi mô thức nào; bạn chỉ cần một chút lễ thường.

These are some basic principles that can help us keep the nesting level low and improve the organization of our code in general:

Đây là một số nguyên tắc cơ bản có thể giúp ta giữ mức lồng nhau thấp và cải thiện việc tổ chức code nói chung:

- Exit as soon as possible. Use `return`, `continue`, or `break`, depending on the context, to immediately exit the current statement instead of writing (and nesting) complete `if...else` statements. This will help to keep our code shallow.
- Create named functions for callbacks, keeping them out of closures and passing intermediate results as arguments. Naming our functions will also make them look better in stack traces.
- Modularize the code. Split the code into smaller, reusable functions whenever possible.
 - Thoát ra càng sớm càng tốt. Dùng `return`, `continue`, hoặc `break`, tùy ngữ cảnh, để thoát ngay lập tức khỏi câu lệnh hiện tại thay vì viết (và lồng) trọn vẹn các câu lệnh `if...else`. Điều này sẽ giúp giữ cho code của ta nông.
 - Tạo các hàm có tên cho callback, giữ chúng nằm ngoài closure và truyền các kết quả trung gian dưới dạng tham số. Việc đặt tên cho hàm của ta cũng sẽ làm chúng trông đẹp hơn trong stack trace.
 - Module hóa code. Tách code thành các hàm nhỏ hơn, có thể tái sử dụng bất cứ khi nào có thể.

Now, let's put these principles into practice.

Bây giờ, hãy đưa các nguyên tắc này vào thực hành.

Applying the callback discipline — Áp dụng kỷ luật callback

To demonstrate the power of the ideas mentioned in the previous section, let's apply them to fix the callback hell in our web spider application.

Để minh họa sức mạnh của những ý tưởng được nhắc đến trong phần trước, hãy áp dụng chúng để khắc phục callback hell trong ứng dụng web spider của ta.

For the first step, we can **refactor** our error-checking pattern by removing the `else` statement. This is made possible by returning from the function immediately after we receive an error. So, instead of having code such as the following:

Ở bước đầu tiên, ta có thể tái cấu trúc mẫu kiểm tra lỗi bằng cách loại bỏ câu lệnh `else`. Điều này khả thi nhờ việc trả về (`return`) khỏi hàm ngay sau khi ta nhận được một lỗi. Vậy, thay vì có code như sau:

```
if (err) {  
  cb(err)  
} else {  
  // code to execute when there are no errors  
}
```

We can improve the organization of our code by writing the following instead:

Ta có thể cải thiện việc tổ chức code bằng cách viết như sau thay vào đó:

```
if (err) {  
    return cb(err)  
}  
// code to execute when there are no errors
```

This is often referred to as the early return principle. With this simple trick, we immediately have a reduction in the nesting level of our functions. It is easy and doesn't require any complex refactoring.

Điều này thường được gọi là nguyên tắc early return (trả về sớm). Với mảnh đơn giản này, ta lập tức giảm được mức lồng nhau của các hàm. Nó dễ và không đòi hỏi bất kỳ việc tái cấu trúc phức tạp nào.

A common mistake when executing the optimization just described is forgetting to terminate the function after the callback is invoked. For an error-handling scenario, the following code is a typical source of defects:

Một sai lầm phổ biến khi thực hiện sự tối ưu vừa mô tả là quên kết thúc hàm sau khi callback được gọi. Đối với tình huống xử lý lỗi, đoạn code sau là một nguồn gây lỗi điển hình:

```
if (err) {  
    callback(err)  
}  
// code to execute when there are no errors.
```

We should never forget that the execution of our function will continue even after we invoke the callback. It is then important to insert a `return` instruction to block the execution of the rest of the function. Also, note that it doesn't really matter what value is returned by the function; the real result (or error) is produced asynchronously and passed to the callback. The return value of the asynchronous function is usually ignored. This property allows us to write shortcuts such as the following:

Ta không bao giờ nên quên rằng việc thực thi hàm của ta sẽ tiếp tục ngay cả sau khi ta gọi callback. Do đó, quan trọng là phải chèn một chỉ thị `return` để chặn việc thực thi phần còn lại của hàm. Ngoài ra, hãy lưu ý rằng giá trị mà hàm trả về thực ra không quan trọng; kết quả thực sự (hoặc lỗi) được tạo ra một cách bất đồng bộ và truyền vào callback. Giá trị trả về của hàm bất đồng bộ thường bị bỏ qua. Tính chất này cho phép ta viết các lối tắt như sau:

```
return callback(...)
```

Otherwise, we'd have to write slightly more verbose code, such as the following:

Ngược lại, ta sẽ phải viết code hơi dài dòng hơn, như sau:

```
callback(...)  
return
```

As a second optimization for our `spider()` function, we can try to identify reusable pieces of code. For example, the functionality that writes a given string to a file can be easily factored out into a separate function, as follows:

Là một tối ưu thứ hai cho hàm `spider()`, ta có thể thử nhận diện những đoạn code có thể tái sử dụng. Ví dụ, chức năng ghi một chuỗi cho trước vào tệp có thể dễ dàng được tách ra thành một hàm riêng, như sau:

```
function saveFile (filename, contents, cb) {
  mkdirp(path.dirname(filename), err => {
    if (err) {
      return cb(err)
    }
    fs.writeFile(filename, contents, cb)
  })
}
```

Following the same principle, we can create a generic function named `download()` that takes a URL and a filename as input, and downloads the URL into the given file. Internally, we can use the `saveFile()` function we created earlier:

Theo cùng nguyên tắc, ta có thể tạo một hàm tổng quát tên là `download()` nhận vào một URL và một tên tệp làm đầu vào, rồi tải URL về tệp đã cho. Bên trong, ta có thể dùng hàm `saveFile()` mà ta đã tạo trước đó:

```
function download (url, filename, cb) {
  console.log(`Downloading ${url}`)
  superagent.get(url).end((err, res) => {
    if (err) {
      return cb(err)
    }
    saveFile(filename, res.text, err => {
      if (err) {
        return cb(err)
      }
      console.log(`Downloaded and saved: ${url}`)
      cb(null, res.text)
    })
  })
}
```

For the last step, we modify the `spider()` function, which, thanks to our changes, will now look like the following:

Ở bước cuối cùng, ta sửa hàm `spider()`, mà nhờ những thay đổi của ta, giờ sẽ trông như sau:

```
export function spider (url, cb) {
  const filename = urlToFilename(url)
  fs.access(filename, err => {
    if (!err || err.code !== 'ENOENT') { // (1)
      return cb(null, filename, false)
    }
    download(url, filename, err => {
      if (err) {
        return cb(err)
      }
      cb(null, filename, true)
    })
  })
}
```

The functionality and the interface of the `spider()` function remained exactly the same; what changed was the way the code was organized. One important detail to notice (1) is that we inverted the check for the file's existence so that we could apply the early return principle discussed previously.

Chức năng và giao diện của hàm `spider()` vẫn giữ y nguyên; điều thay đổi là cách code được tổ chức. Một chi tiết quan trọng cần lưu ý (1) là ta đã đảo ngược phép kiểm tra sự tồn tại của tệp để có thể áp dụng nguyên tắc early return đã thảo luận trước đó.

By applying the early return principle and the other callback discipline principles, we were able to drastically reduce the nesting of our code and, at the same time, increase its reusability and testability. In fact, we could think about exporting both `saveFile()` and `download()` so that we could reuse them in other modules. This would also allow us to test their functionality as independent units.

Bằng cách áp dụng nguyên tắc early return và các nguyên tắc kỷ luật callback khác, ta đã có thể giảm đáng kể mức lồng nhau của code và, đồng thời, tăng khả năng tái sử dụng và khả năng kiểm thử của nó. Thực vậy, ta có thể nghĩ đến việc export cả `saveFile()` và `download()` để có thể tái sử dụng chúng trong các module khác. Điều này cũng sẽ cho phép ta kiểm thử chức năng của chúng như những đơn vị độc lập.

The refactoring we carried out in this section clearly demonstrates that most of the time, all we need is some discipline to make sure we do not abuse closures and anonymous functions. It works brilliantly, requires minimal effort, and it doesn't require external libraries.

Việc tái cấu trúc mà ta thực hiện trong phần này cho thấy rõ rằng phần lớn thời gian, tất cả những gì ta cần là một chút kỷ luật để đảm bảo ta không lạm dụng closure và hàm vô danh. Nó hoạt động xuất sắc, đòi hỏi nỗ lực tối thiểu, và không cần đến thư viện ngoài.

Now that you know how to write clean asynchronous code using callbacks, we are ready to explore some of the most common asynchronous patterns, such as sequential and parallel execution.

Giờ đây khi bạn đã biết cách viết code bất đồng bộ sạch sẽ bằng callback, ta đã sẵn sàng khám phá một số mẫu bất đồng bộ phổ biến nhất, như thực thi tuần tự và song song.

Sequential execution — Thực thi tuần tự

In this section, we will look at asynchronous control flow patterns and start by analyzing the sequential execution flow.

Trong phần này, ta sẽ xem xét các mẫu điều khiển luồng bất đồng bộ và bắt đầu bằng việc phân tích luồng thực thi tuần tự.

Executing a set of tasks in sequence means running them one at a time, one after the other. The order of execution matters and must be preserved, because the result of a task in the list may affect the execution of the next. Figure 4.1 illustrates this concept:

Thực thi một tập hợp các tác vụ tuần tự nghĩa là chạy chúng từng cái một, cái này sau cái kia. Thứ tự thực thi rất quan trọng và phải được bảo toàn, bởi kết quả của một tác vụ trong danh sách có thể ảnh hưởng đến việc thực thi tác vụ kế tiếp. Hình 4.1 minh họa khái niệm này:

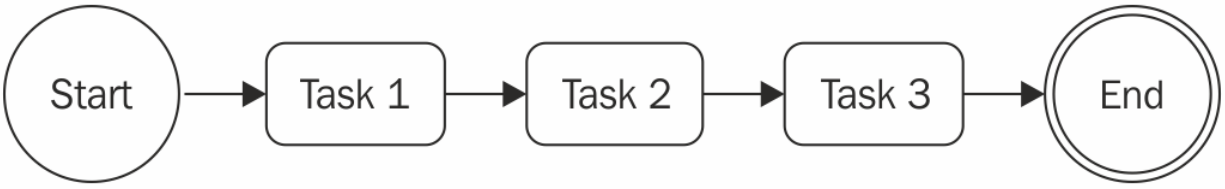


Figure 4.1: An example of sequential execution flow with three tasks

Hình 4.1: Một ví dụ về luồng thực thi tuần tự với ba tác vụ

There are different variations of this flow:

Có những biến thể khác nhau của luồng này:

- Executing a set of known tasks in sequence, without propagating data across them.
- Using the output of a task as the input for the next (also known as chain, pipeline, or waterfall).
- Iterating over a collection while running an asynchronous task on each element, one after the other.
 - Thực thi một tập hợp các tác vụ đã biết một cách tuần tự, không truyền dữ liệu giữa chúng.
 - Dùng đầu ra của một tác vụ làm đầu vào cho tác vụ kế tiếp (còn gọi là chain, pipeline, hoặc waterfall).
 - Duyệt qua một tập hợp trong khi chạy một tác vụ bất đồng bộ trên mỗi phần tử, cái này sau cái kia.

Sequential execution, despite being trivial when implemented using a direct style blocking API, is usually the main cause of the callback hell problem when using asynchronous CPS.

Thực thi tuần tự, dù tầm thường khi hiện thực bằng API chặn theo phong cách trực tiếp, thường lại là nguyên nhân chính của vấn đề callback hell khi dùng CPS bất đồng bộ.

Executing a known set of tasks in sequence — Thực thi một tập hợp tác vụ đã biết một cách tuần tự

We already looked at a sequential execution flow while implementing the `spider()` function in the previous section. By applying some simple rules, we were able to organize a set of known tasks in a sequential execution flow. Taking that code as a guideline, we can now generalize the solution with the following pattern:

Ta đã từng xem xét một luồng thực thi tuần tự khi hiện thực hàm `spider()` trong phần trước. Bằng cách áp dụng một số quy tắc đơn giản, ta đã có thể tổ chức một tập hợp các tác vụ đã biết theo một luồng thực thi tuần tự. Lấy code đó làm kim chỉ nam, giờ ta có thể tổng quát hóa giải pháp bằng mẫu sau:

```
function task1 (cb) {  
  asyncOperation(() => {  
    task2(cb)  
  })  
}  
function task2 (cb) {  
  asyncOperation(() => {  
    task3(cb)  
  })  
})
```

```

}
function task3 (cb) {
  asyncOperation(() => {
    cb() // finally executes the callback
  })
}
task1(() => {
  // executed when task1, task2 and task3 are completed
  console.log('tasks 1, 2 and 3 executed')
})

```

The preceding pattern shows how each task invokes the next upon completion of a generic asynchronous operation. The pattern puts the emphasis on the modularization of tasks, showing how closures are not always necessary to handle asynchronous code.

Mẫu trên cho thấy cách mỗi tác vụ gọi tác vụ kế tiếp khi một thao tác bất đồng bộ tổng quát hoàn tất. Mẫu này nhấn mạnh vào việc module hóa các tác vụ, cho thấy rằng closure không phải lúc nào cũng cần thiết để xử lý code bất đồng bộ.

Sequential iteration — Lặp tuần tự

The pattern described in the previous section works perfectly if we know in advance what and how many tasks are to be executed. This allows us to hardcode the invocation of the next task in the sequence, but what happens if we want to execute an asynchronous operation for each item in a collection? In cases such as this, we can't hardcode the task sequence anymore; instead, we have to build it dynamically.

Mẫu được mô tả trong phần trước hoạt động hoàn hảo nếu ta biết trước những tác vụ nào và bao nhiêu tác vụ sẽ được thực thi. Điều này cho phép ta hardcode lời gọi tác vụ kế tiếp trong chuỗi, nhưng điều gì xảy ra nếu ta muốn thực thi một thao tác bất đồng bộ cho từng phần tử trong một tập hợp? Trong những trường hợp như vậy, ta không thể hardcode chuỗi tác vụ được nữa; thay vào đó, ta phải xây dựng nó một cách động.

Web spider version 2 — Web spider phiên bản 2 To show an example of sequential iteration, let's introduce a new feature to the web spider application. We now want to download all the links contained in a web page recursively. To do that, we are going to extract all the links from the page and then trigger our web spider on each recursively and in sequence.

Để cho thấy một ví dụ về lặp tuần tự, hãy giới thiệu một tính năng mới cho ứng dụng web spider. Giờ ta muốn tải về một cách đệ quy tất cả các liên kết chứa trong một trang web. Để làm điều đó, ta sẽ trích xuất tất cả các liên kết từ trang rồi kích hoạt web spider trên mỗi liên kết một cách đệ quy và tuần tự.

The first step is modifying our `spider()` function so that it triggers a recursive download of all the links of a page by using a function named `spiderLinks()`, which we are going to create shortly.

Bước đầu tiên là sửa hàm `spider()` để nó kích hoạt việc tải về đệ quy tất cả các liên kết của một trang bằng cách dùng một hàm tên là `spiderLinks()`, mà ta sẽ tạo ngay sau đây.

Also, instead of checking whether the file already exists, we will try to read it and start spidering its links. This way, we will be able to resume interrupted downloads. As a final change, we need to make sure we propagate a new parameter, `nesting`, which will help us to limit the recursion depth. The code is as follows:

Ngoài ra, thay vì kiểm tra xem tệp đã tồn tại hay chưa, ta sẽ thử đọc nó và bắt đầu spider các liên kết của nó. Bằng cách này, ta sẽ có thể tiếp tục các lượt tải về bị gián đoạn. Là thay đổi cuối cùng, ta cần đảm bảo lan truyền một tham số mới, `nesting`, sẽ giúp ta giới hạn độ sâu đệ quy. Code như sau:

```
export function spider (url, nesting, cb) {
  const filename = urlToFilename(url)
  fs.readFile(filename, 'utf8', (err, fileContent) => {
    if (err) {
      if (err.code !== 'ENOENT') {
        return cb(err)
      }
      // The file doesn't exist, so let's download it
      return download(url, filename, (err, requestContent) => {
        if (err) {
          return cb(err)
        }
        spiderLinks(url, requestContent, nesting, cb)
      })
    }
    // The file already exists, let's process the links
    spiderLinks(url, fileContent, nesting, cb)
  })
}
```

In the next section, we will explore how the `spiderLinks()` function can be implemented.

Trong phần tiếp theo, ta sẽ khám phá cách hàm `spiderLinks()` có thể được hiện thực.

Sequential crawling of links — Thu thập (crawl) các liên kết một cách tuần tự Now, we can create the core of this new version of our web spider application, the `spiderLinks()` function, which downloads all the links of an HTML page using a sequential asynchronous iteration algorithm. Pay attention to the way we are going to define that in the following code block:

Giờ đây, ta có thể tạo phần cốt lõi của phiên bản mới này của ứng dụng web spider, hàm `spiderLinks()`, hàm này tải về tất cả các liên kết của một trang HTML bằng một thuật toán lặp bất đồng bộ tuần tự. Hãy chú ý cách ta sẽ định nghĩa điều đó trong khối code sau:

```
function spiderLinks (currentUrl, body, nesting, cb) {
  if (nesting === 0) {
    // Remember Zalgo from chapter 3?
    return process.nextTick(cb)
  }
  const links = getPageLinks(currentUrl, body) // (1)
  if (links.length === 0) {
    return process.nextTick(cb)
  }
  function iterate (index) { // (2)
    if (index === links.length) {
      return cb()
    }
    spider(links[index], nesting - 1, function (err) { // (3)
```

```

    if (err) {
      return cb(err)
    }
    iterate(index + 1)
  })
}
iterate(0) // (4)
}

```

The important steps to understand from this new function are as follows:

Những bước quan trọng cần hiểu từ hàm mới này như sau:

1. We obtain the list of all the links contained in the page using the `getPageLinks()` function. This function returns only the links pointing to an internal destination (the same hostname).
2. We iterate over the links using a local function called `iterate()`, which takes the index of the next link to analyze. In this function, the first thing we do is check whether the index is equal to the length of the links array, in which case we immediately invoke the `cb()` function, as it means we have processed all the items.
3. At this point, everything should be ready for processing the link. We invoke the `spider()` function by decreasing the nesting level and invoking the next step of the iteration when the operation completes.
4. As the last step in the `spiderLinks()` function, we bootstrap the iteration by invoking `iterate(0)`.

1. Ta lấy danh sách tất cả các liên kết chứa trong trang bằng hàm `getPageLinks()`. Hàm này chỉ trả về các liên kết trỏ đến một đích nội bộ (cùng hostname).
2. Ta duyệt qua các liên kết bằng một hàm cục bộ tên là `iterate()`, hàm này nhận vào index của liên kết kế tiếp cần phân tích. Trong hàm này, điều đầu tiên ta làm là kiểm tra xem index có bằng độ dài của mảng links hay không, trong trường hợp đó ta lập tức gọi hàm `cb()`, vì điều đó nghĩa là ta đã xử lý xong tất cả các phần tử.
3. Tại điểm này, mọi thứ nên đã sẵn sàng để xử lý liên kết. Ta gọi hàm `spider()` bằng cách giảm mức nesting và gọi bước kế tiếp của vòng lặp khi thao tác hoàn tất.
4. Là bước cuối cùng trong hàm `spiderLinks()`, ta khởi động vòng lặp bằng cách gọi `iterate(0)`.

The algorithm that was just presented allows us to iterate over an array by executing an asynchronous operation in sequence, which in our case is the `spider()` function.

Thuật toán vừa được trình bày cho phép ta duyệt qua một mảng bằng cách thực thi một thao tác bất đồng bộ một cách tuần tự, mà trong trường hợp của ta là hàm `spider()`.

Finally, we can change our `spider-cli.js` a bit so that we can specify the nesting level as an additional command-line interface (CLI) argument:

Cuối cùng, ta có thể thay đổi `spider-cli.js` một chút để có thể chỉ định mức nesting làm một tham số dòng lệnh (CLI) bổ sung:

```

import { spider } from './spider.js'
const url = process.argv[2]
const nesting = Number.parseInt(process.argv[3], 10) || 1
spider(url, nesting, err => {
  if (err) {
    console.error(err)
    process.exit(1)
  }
})

```

```

    }
    console.log('Download complete')
  })
}

```

We can now try this new version of the spider application and watch it download all the links of a web page recursively, one after the other. To interrupt the process, which can take a while if there are many links, remember that we can always use Ctrl + C. If we then decide to resume it, we can do so by launching the spider application and providing the same URL we used for the first run. Now that our web spider application can potentially trigger the download of an entire website, please consider using it carefully. For example, do not set a high nesting level or leave the spider running for more than a few seconds. It is not polite to overload a server with thousands of requests. In some circumstances, this can also be considered illegal. Spider responsibly!

Giờ ta có thể thử phiên bản mới này của ứng dụng spider và xem nó tải về tất cả các liên kết của một trang web một cách đệ quy, cái này sau cái kia. Để gián đoạn quá trình, vốn có thể mất một lúc nếu có nhiều liên kết, hãy nhớ rằng ta luôn có thể dùng Ctrl + C. Nếu sau đó ta quyết định tiếp tục, ta có thể làm vậy bằng cách khởi chạy ứng dụng spider và cung cấp cùng URL mà ta đã dùng cho lần chạy đầu tiên. Giờ đây khi ứng dụng web spider của ta có khả năng kích hoạt việc tải về cả một website, hãy cân nhắc dùng nó một cách cẩn thận. Ví dụ, đừng đặt mức nesting cao hay để spider chạy quá vài giây. Việc nhồi nhét máy chủ bằng hàng nghìn yêu cầu là không lịch sự. Trong một số hoàn cảnh, điều này còn có thể bị xem là bất hợp pháp. Hãy spider một cách có trách nhiệm!

The pattern — Mẫu The code of the `spiderLinks()` function from the previous section is a clear example of how it's possible to iterate over a collection while applying an asynchronous operation. You may also notice that it's a pattern that can be adapted to any other situation where we need to iterate asynchronously over the elements of a collection or, in general, over a list of tasks. This pattern can be generalized as follows:

Code của hàm `spiderLinks()` ở phần trước là một ví dụ rõ ràng về cách có thể duyệt qua một tập hợp trong khi áp dụng một thao tác bất đồng bộ. Bạn cũng có thể nhận thấy đó là một mẫu có thể được điều chỉnh cho bất kỳ tình huống nào khác mà ta cần duyệt bất đồng bộ qua các phần tử của một tập hợp hoặc, nói chung, qua một danh sách các tác vụ. Mẫu này có thể được tổng quát hóa như sau:

```

function iterate (index) {
  if (index === tasks.length) {
    return finish()
  }
  const task = tasks[index]
  task(() => iterate(index + 1))
}
function finish () {
  // iteration completed
}
iterate(0)

```

It's important to notice that these types of algorithms become really recursive if `task()` is a synchronous operation. In such a case, the stack will not unwind at every cycle and there might be a risk of hitting the maximum call stack size limit.

Quan trọng là phải lưu ý rằng những loại thuật toán này trở nên thực sự đệ quy nếu `task()` là một thao tác đồng bộ. Trong trường hợp đó, stack sẽ không được gỡ ra ở mỗi chu kỳ và

có thể có nguy cơ chạm tới giới hạn kích thước call stack tối đa.

The pattern that was just presented is very powerful and can be extended or adapted to address several common needs. Just to mention some examples:

Mẫu vừa được trình bày rất mạnh mẽ và có thể được mở rộng hoặc điều chỉnh để giải quyết vài nhu cầu phổ biến. Chỉ xin nêu một vài ví dụ:

- We can map the values of an array asynchronously.
- We can pass the results of an operation to the next one in the iteration to implement an asynchronous version of the `reduce` algorithm.
- We can quit the loop prematurely if a particular condition is met (asynchronous implementation of the `Array.some()` helper).
- We can even iterate over an infinite number of elements.
 - Ta có thể map các giá trị của một mảng một cách bất đồng bộ.
 - Ta có thể truyền kết quả của một thao tác sang thao tác kế tiếp trong vòng lặp để hiện thực một phiên bản bất đồng bộ của thuật toán `reduce`.
 - Ta có thể thoát khỏi vòng lặp sớm nếu một điều kiện cụ thể được thỏa mãn (bản hiện thực bất đồng bộ của hàm trợ giúp `Array.some()`).
 - Ta thậm chí có thể duyệt qua một số phần tử vô hạn.

We could also choose to generalize the solution even further by wrapping it in a function with a signature such as the following:

Ta cũng có thể chọn tổng quát hóa giải pháp thêm nữa bằng cách bọc nó trong một hàm với chữ ký như sau:

```
iterateSeries(collection, iteratorCallback, finalCallback)
```

Here, `collection` is the actual dataset you want to iterate over, `iteratorCallback` is the function to execute over every item, and `finalCallback` is the function that gets executed when all the items are processed or in case of an error. The implementation of this helper function is left to you as an exercise.

Ở đây, `collection` là tập dữ liệu thực sự bạn muốn duyệt qua, `iteratorCallback` là hàm cần thực thi trên mỗi phần tử, và `finalCallback` là hàm được thực thi khi tất cả các phần tử đã được xử lý hoặc trong trường hợp có lỗi. Việc hiện thực hàm trợ giúp này được để lại cho bạn như một bài tập.

The Sequential **Iterator** pattern

Mẫu Sequential Iterator

Execute a list of tasks in sequence by creating a function named `iterator`, which invokes the next available task in the collection and makes sure to invoke the next step of the iteration when the current task completes.

Thực thi một danh sách các tác vụ tuần tự bằng cách tạo một hàm tên là `iterator`, hàm này gọi tác vụ khả dụng kế tiếp trong tập hợp và đảm bảo gọi bước kế tiếp của vòng lặp khi tác vụ hiện tại hoàn tất.

In the next section, we will explore the parallel execution pattern, which is more convenient when the order of the various tasks is not important.

Trong phần tiếp theo, ta sẽ khám phá mẫu thực thi song song, thuận tiện hơn khi thứ tự của các tác vụ khác nhau không quan trọng.

Parallel execution — Thực thi song song

There are some situations where the order of execution of a set of asynchronous tasks is not important, and all we want is to be notified when all those running tasks are completed. Such situations are better handled using a parallel execution flow, as shown in Figure 4.2:

Có một số tình huống mà thứ tự thực thi của một tập hợp các tác vụ bất đồng bộ không quan trọng, và tất cả những gì ta muốn là được thông báo khi tất cả các tác vụ đang chạy đó hoàn tất. Những tình huống như vậy được xử lý tốt hơn bằng một luồng thực thi song song, như minh họa ở Hình 4.2:

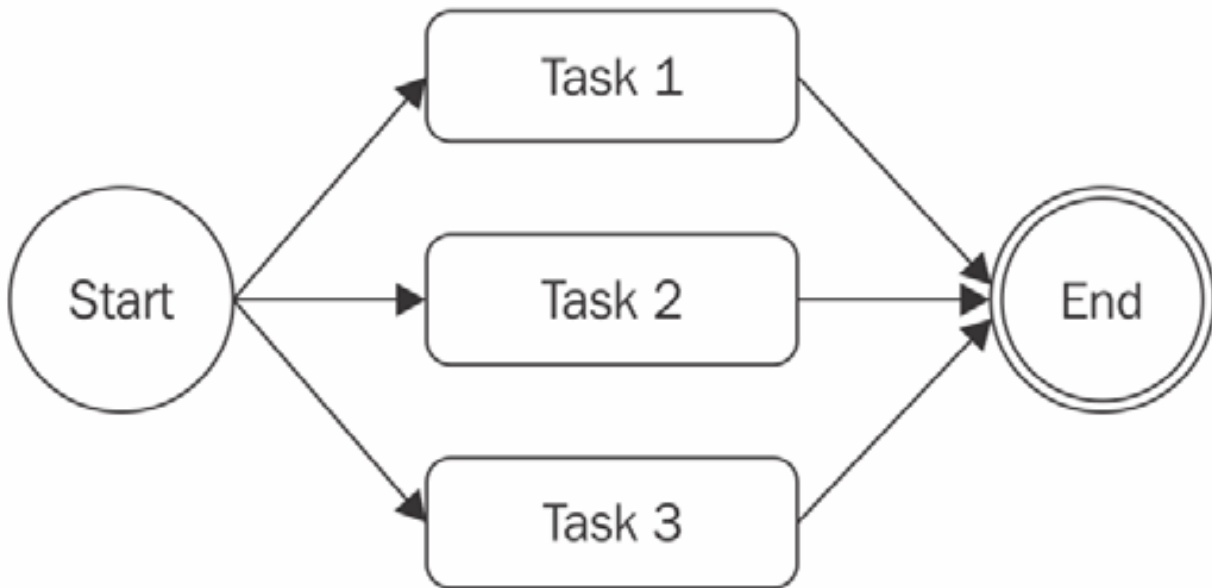


Figure 4.2: An example of parallel execution with three tasks This may sound strange if you consider that Node.js is **single-threaded**, but if you remember what we discussed in Chapter 1, The Node.js Platform, you'll realize that even though we have just one **thread**, we can still achieve **concurrency**, thanks to the non-blocking nature of Node.js. In fact, the word parallel is used improperly in this case, as it does not mean that the tasks run simultaneously, but rather that their execution is carried out by an underlying, non-blocking API and interleaved by the **event loop**.

Hình 4.2: Một ví dụ về thực thi song song với ba tác vụ. Điều này nghe có vẻ lạ nếu bạn xét rằng Node.js là đơn luồng (single-threaded), nhưng nếu bạn nhớ những gì ta đã thảo luận ở Chương 1, Nền tảng Node.js, bạn sẽ nhận ra rằng dù ta chỉ có một luồng, ta vẫn có thể đạt được tính đồng thời (concurrency), nhờ bản chất không chặn của Node.js. Thực vậy, từ “song song” được dùng không thật chính xác trong trường hợp này, vì nó không có nghĩa là các tác vụ chạy đồng thời, mà đúng hơn là việc thực thi chúng được tiến hành bởi một API không chặn ngầm bên dưới và được xen kẽ bởi vòng lặp sự kiện (event loop).

As you know, a task gives control back to the event loop when it requests a new asynchronous operation, allowing the event loop to execute another task. The proper word to use for this kind of flow is concurrency, but we will still use parallel for simplicity.

Như bạn đã biết, một tác vụ trả quyền điều khiển lại cho vòng lặp sự kiện khi nó yêu cầu một thao tác bất đồng bộ mới, cho phép vòng lặp sự kiện thực thi một tác vụ khác. Từ đúng để dùng cho loại luồng này là tính đồng thời (concurrency), nhưng ta vẫn sẽ dùng “song song” cho đơn giản.

The following diagram shows how two asynchronous tasks can run in parallel in a Node.js program:

Sơ đồ sau cho thấy cách hai tác vụ bất đồng bộ có thể chạy song song trong một chương trình Node.js:

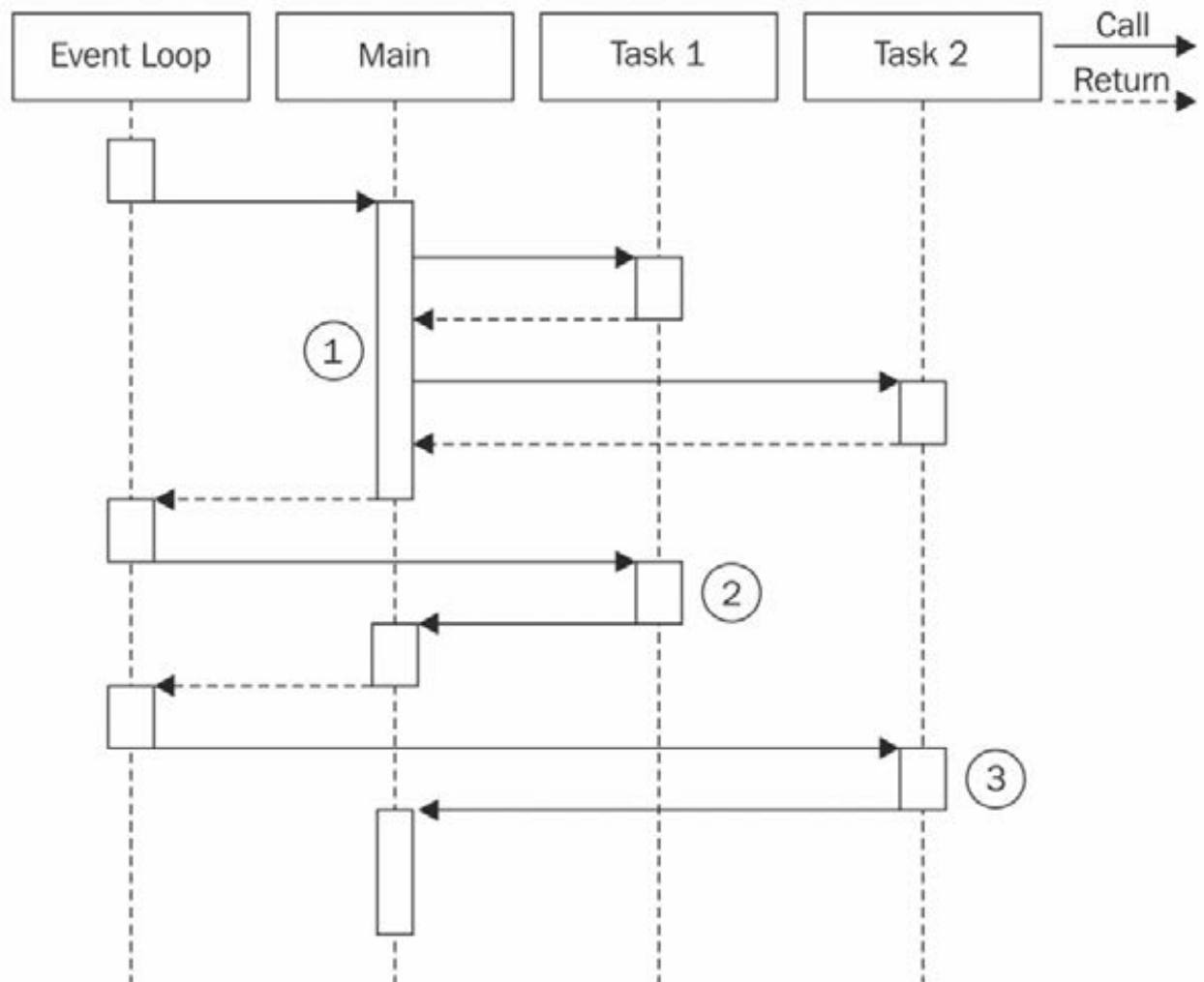


Figure 4.3: An example of how asynchronous tasks run in parallel

Hình 4.3: Một ví dụ về cách các tác vụ bất đồng bộ chạy song song

In Figure 4.3, we have a Main function that executes two asynchronous tasks:

Trong Hình 4.3, ta có một hàm Main thực thi hai tác vụ bất đồng bộ:

1. The Main function triggers the execution of Task 1 and Task 2. As they trigger an asynchronous operation, they immediately return control back to the Main function, which then returns it to the event loop.
2. When the asynchronous operation of Task 1 is completed, the event loop gives control to it. When Task 1 completes its internal synchronous processing as well, it notifies the Main function.
3. When the asynchronous operation triggered by Task 2 is complete, the event loop invokes its callback, giving control back to Task 2. At the end of Task 2, the Main function is notified once more. At this point, the Main function knows that both Task 1 and Task 2 are complete, so it can continue its execution or return the results of the operations to another callback.

1. Hàm Main kích hoạt việc thực thi Task 1 và Task 2. Vì chúng kích hoạt một thao tác bất đồng bộ, chúng lập tức trả quyền điều khiển lại cho hàm Main, hàm này sau đó trả nó lại cho vòng lặp sự kiện.
2. Khi thao tác bất đồng bộ của Task 1 hoàn tất, vòng lặp sự kiện trao quyền điều khiển cho nó. Khi Task 1 cũng hoàn tất việc xử lý đồng bộ nội bộ của mình, nó thông báo cho hàm Main.
3. Khi thao tác bất đồng bộ được kích hoạt bởi Task 2 hoàn tất, vòng lặp sự kiện gọi callback của nó, trả quyền điều khiển lại cho Task 2. Ở cuối Task 2, hàm Main lại được thông báo một lần nữa. Tại điểm này, hàm Main biết rằng cả Task 1 và Task 2 đều đã hoàn tất, nên nó có thể tiếp tục việc thực thi của mình hoặc trả kết quả của các thao tác cho một callback khác.

In short, this means that in Node.js, we can only execute asynchronous operations in parallel, because their concurrency is handled internally by the non-blocking APIs. In Node.js, synchronous (blocking) operations can't run concurrently unless their execution is interleaved with an asynchronous operation, or interleaved with `setTimeout()` or `setImmediate()`. You will see this in more detail in Chapter 11, Advanced Recipes.

Tóm lại, điều này có nghĩa là trong Node.js, ta chỉ có thể thực thi song song các thao tác bất đồng bộ, bởi tính đồng thời của chúng được xử lý nội bộ bởi các API không chặn. Trong Node.js, các thao tác đồng bộ (chặn) không thể chạy đồng thời trừ khi việc thực thi của chúng được xen kẽ với một thao tác bất đồng bộ, hoặc xen kẽ với `setTimeout()` hay `setImmediate()`. Bạn sẽ thấy điều này chi tiết hơn ở Chương 11, Các công thức nâng cao.

Web spider version 3 — Web spider phiên bản 3

Our web spider application seems like a perfect candidate to apply the concept of parallel execution. So far, our application is executing the recursive download of the linked pages in a sequential fashion. We can easily improve the performance of this process by downloading all the linked pages in parallel.

Ứng dụng web spider của ta có vẻ là một ứng viên hoàn hảo để áp dụng khái niệm thực thi song song. Cho đến nay, ứng dụng của ta đang thực hiện việc tải về đệ quy các trang được liên kết theo kiểu tuần tự. Ta có thể dễ dàng cải thiện hiệu năng của quá trình này bằng cách tải về tất cả các trang được liên kết song song.

To do that, we just need to modify the `spiderLinks()` function to make sure we spawn all the `spider()` tasks at once, and then invoke the final callback only when all of them have completed their execution. So, let's modify our `spiderLinks()` function as follows:

Để làm điều đó, ta chỉ cần sửa hàm `spiderLinks()` để đảm bảo ta sinh ra tất cả các tác vụ `spider()` cùng một lúc, rồi chỉ gọi callback cuối cùng khi tất cả chúng đã hoàn tất việc thực thi. Vậy, hãy sửa hàm `spiderLinks()` của ta như sau:

```
function spiderLinks (currentUrl, body, nesting, cb) {
  if (nesting === 0) {
    return process.nextTick(cb)
  }
  const links = getPageLinks(currentUrl, body)
  if (links.length === 0) {
    return process.nextTick(cb)
  }
  let completed = 0
```

```

let hasErrors = false
function done (err) {
  if (err) {
    hasErrors = true
    return cb(err)
  }
  if (++completed === links.length && !hasErrors) {
    return cb()
  }
}
links.forEach(link => spider(link, nesting - 1, done))
}

```

Let's discuss what we changed. As mentioned earlier, the `spider()` tasks are now started all at once. This is possible by simply iterating over the `links` array and starting each task without waiting for the previous one to finish:

Hãy thảo luận về điều ta đã thay đổi. Như đã nhắc đến trước đó, các tác vụ `spider()` giờ được khởi động cùng một lúc. Điều này khả thi bằng cách đơn giản duyệt qua mảng `links` và khởi động mỗi tác vụ mà không chờ tác vụ trước hoàn tất:

```
links.forEach(link => spider(link, nesting - 1, done))
```

Then, the trick to make our application wait for all the tasks to complete is to provide the `spider()` function with a special callback, which we call `done()`. The `done()` function increases a counter when a `spider` task completes. When the number of completed downloads reaches the size of the `links` array, the final callback is invoked:

Sau đó, mảnh để khiến ứng dụng của ta chờ tất cả các tác vụ hoàn tất là cung cấp cho hàm `spider()` một callback đặc biệt, mà ta gọi là `done()`. Hàm `done()` tăng một bộ đếm khi một tác vụ `spider` hoàn tất. Khi số lượt tải về đã hoàn tất đạt tới kích thước của mảng `links`, callback cuối cùng được gọi:

```

function done (err) {
  if (err) {
    hasErrors = true
    return cb(err)
  }
  if (++completed === links.length && !hasErrors) {
    return cb()
  }
}

```

The `hasErrors` variable is necessary because if one parallel task fails, we want to immediately call the callback with the given error. Also, we need to make sure that other parallel tasks that might still be running won't invoke the callback again.

Biến `hasErrors` là cần thiết bởi nếu một tác vụ song song thất bại, ta muốn gọi callback ngay lập tức với lỗi đã cho. Ngoài ra, ta cần đảm bảo rằng các tác vụ song song khác có thể vẫn đang chạy sẽ không gọi callback thêm lần nữa.

With these changes in place, if we now try to run our `spider` against a web page, we will notice a huge improvement in the speed of the overall process, as every download will be carried out in parallel, without waiting for the previous link to be processed.

Với những thay đổi này được áp dụng, nếu giờ ta thử chạy spider trên một trang web, ta sẽ nhận thấy sự cải thiện to lớn về tốc độ của toàn bộ quá trình, vì mỗi lượt tải về sẽ được tiến hành song song, không chờ liên kết trước được xử lý.

The pattern — Mẫu

Finally, we can extract our nice little pattern for the parallel execution flow. Let's represent a generic version of the pattern with the following code:

Cuối cùng, ta có thể trích ra mẫu nhỏ gọn gàng của mình cho luồng thực thi song song. Hãy biểu diễn một phiên bản tổng quát của mẫu bằng code sau:

```
const tasks = [ /* ... */ ]
let completed = 0
tasks.forEach(task => {
  task(() => {
    if (++completed === tasks.length) {
      finish()
    }
  })
})
function finish () {
  // all the tasks completed
}
```

With small modifications, we can adapt the pattern to accumulate the results of each task into a collection, to filter or map the elements of an array, or to invoke the `finish()` callback as soon as one or a given number of tasks complete (this last situation in particular is called competitive race).

Với những sửa đổi nhỏ, ta có thể điều chỉnh mẫu để tích lũy kết quả của mỗi tác vụ vào một tập hợp, để lọc hoặc map các phần tử của một mảng, hoặc để gọi callback `finish()` ngay khi một hoặc một số tác vụ nhất định hoàn tất (tình huống cuối cùng này nói riêng được gọi là competitive race).

The Unlimited Parallel Execution pattern

Mẫu Unlimited Parallel Execution

Run a set of asynchronous tasks in parallel by launching them all at once, and then wait for all of them to complete by counting the number of times their callbacks are invoked.

Chạy một tập hợp các tác vụ bất đồng bộ song song bằng cách khởi chạy tất cả chúng cùng một lúc, rồi chờ tất cả chúng hoàn tất bằng cách đếm số lần callback của chúng được gọi.

When we have multiple tasks running in parallel, we might have race conditions, that is, contention to access external resources (for example, files or records in a database). In the next section, we will talk about race conditions in Node.js and explore some techniques to identify and address them.

Khi ta có nhiều tác vụ chạy song song, ta có thể gặp race condition, tức là sự tranh chấp để truy cập các tài nguyên bên ngoài (ví dụ, các tệp hoặc bản ghi trong một cơ sở dữ liệu). Trong phần tiếp theo, ta sẽ nói về race condition trong Node.js và khám phá một số kỹ thuật để nhận diện và xử lý chúng.

Fixing race conditions with concurrent tasks — Khắc phục race condition với các tác vụ đồng thời

Running a set of tasks in parallel can cause issues when using **blocking I/O** in combination with multiple threads. However, you have just seen that, in Node.js, this is a totally different story. Running multiple asynchronous tasks in parallel is, in fact, straightforward and cheap in terms of resources. This is one of the most important strengths of Node.js, because it makes parallelization a common practice rather than a complex technique to only use when strictly necessary.

Chạy một tập hợp các tác vụ song song có thể gây ra vấn đề khi dùng I/O chặn (blocking I/O) kết hợp với nhiều luồng. Tuy nhiên, bạn vừa thấy rằng, trong Node.js, đây là một câu chuyện hoàn toàn khác. Chạy nhiều tác vụ bất đồng bộ song song thực ra rất đơn giản và rẻ về mặt tài nguyên. Đây là một trong những thế mạnh quan trọng nhất của Node.js, bởi nó khiến việc song song hóa trở thành một thực hành thường ngày hơn là một kỹ thuật phức tạp chỉ dùng khi thực sự cần thiết.

Another important characteristic of the concurrency model of Node.js is the way we deal with task synchronization and race conditions. In multithreaded programming, this is usually done using constructs such as locks, mutexes, semaphores, and monitors, and it can be one of the most complex aspects of parallelization, and has a considerable impact on performance. In Node.js, we usually don't need a fancy synchronization mechanism, as everything runs on a single thread. However, this doesn't mean that we can't have race conditions; on the contrary, they can be quite common. The root of the problem is the delay between the invocation of an asynchronous operation and the notification of its result.

Một đặc trưng quan trọng khác của mô hình tính đồng thời của Node.js là cách ta xử lý sự đồng bộ hóa tác vụ và race condition. Trong lập trình đa luồng, việc này thường được làm bằng các kiến trúc như lock, mutex, semaphore, và monitor, và nó có thể là một trong những khía cạnh phức tạp nhất của việc song song hóa, đồng thời có tác động đáng kể đến hiệu năng. Trong Node.js, ta thường không cần một cơ chế đồng bộ hóa cầu kỳ, vì mọi thứ chạy trên một luồng đơn. Tuy nhiên, điều này không có nghĩa là ta không thể có race condition; trái lại, chúng có thể khá phổ biến. Gốc rễ của vấn đề là độ trễ giữa lúc gọi một thao tác bất đồng bộ và lúc thông báo kết quả của nó.

To see a concrete example, we will refer again to our web spider application, and in particular, the last version we created, which actually contains a **race condition** (can you spot it?). The problem we are talking about lies in the `spider()` function, where we check whether a file already exists before we start to download the corresponding URL:

Để thấy một ví dụ cụ thể, ta sẽ lại tham chiếu đến ứng dụng web spider của mình, và cụ thể là phiên bản cuối cùng ta đã tạo, mà thực ra chứa một race condition (bạn có nhận ra nó không?). Vấn đề ta đang nói đến nằm trong hàm `spider()`, nơi ta kiểm tra xem một tệp đã tồn tại hay chưa trước khi ta bắt đầu tải về URL tương ứng:

```
export function spider (url, nesting, cb) {
  const filename = urlToFilename(url)
  fs.readFile(filename, 'utf8', (err, fileContent) => {
    if (err) {
      if (err.code !== 'ENOENT') {
        return cb(err)
      }
    }
    return download(url, filename, (err, requestContent) => {
      // ...
    })
  })
}
```


The problem is that two spider tasks operating on the same URL might invoke `fs.readFile()` on the same file before one of the two tasks completes the download and creates a file, causing both tasks to start a download. Figure 4.4 explains this situation:

Vấn đề là hai tác vụ spider thao tác trên cùng một URL có thể gọi `fs.readFile()` trên cùng một tệp trước khi một trong hai tác vụ hoàn tất việc tải về và tạo một tệp, khiến cả hai tác vụ đều bắt đầu một lượt tải về. Hình 4.4 giải thích tình huống này:

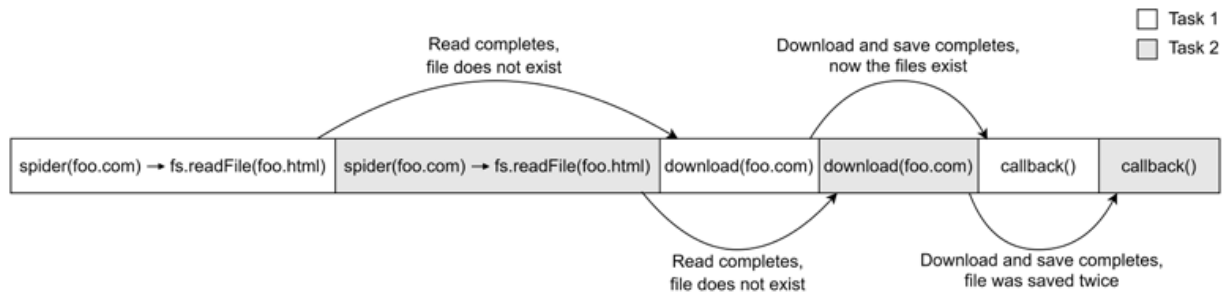


Figure 4.4: An example of a race condition in our spider() function Figure 4.4 shows how Task 1 and Task 2 are interleaved in the single thread of Node.js, as well as how an asynchronous operation can actually introduce a race condition. In our case, the two spider tasks end up downloading the same file.

Hình 4.4: Một ví dụ về race condition trong hàm spider() của ta. Hình 4.4 cho thấy cách Task 1 và Task 2 được xen kẽ trong luồng đơn của Node.js, cũng như cách một thao tác bất đồng bộ thực sự có thể đưa vào một race condition. Trong trường hợp của ta, hai tác vụ spider rốt cuộc tải về cùng một tệp.

How can we fix that? The answer is much simpler than you might think. In fact, all we need is a variable to mutually exclude multiple spider() tasks running on the same URL. This can be achieved with some code, such as the following:

Làm thế nào ta có thể khắc phục điều đó? Câu trả lời đơn giản hơn nhiều so với bạn có thể nghĩ. Thực vậy, tất cả những gì ta cần là một biến để loại trừ lẫn nhau (mutually exclude) nhiều tác vụ spider() chạy trên cùng một URL. Điều này có thể đạt được bằng một ít code, như sau:

```
const spidering = new Set()
function spider (url, nesting, cb) {
  if (spidering.has(url)) {
    return process.nextTick(cb)
  }
  spidering.add(url)
  // ...
```

The fix does not require many comments. We simply exit the function immediately if the given url is already present in the spidering set; otherwise, we add the url to the set and continue with the download. In our case, we don't need to release the lock, as we are not interested in downloading a URL twice, even if the spider tasks are executed at two completely different points in time. If you are building a spider that might have to download hundreds of thousands of web pages, removing the downloaded url from the set once a file is downloaded will help you to keep the set cardinality, and therefore the memory consumption, from growing indefinitely.

Bản sửa này không cần nhiều bình luận. Ta đơn giản thoát khỏi hàm ngay lập tức nếu url

đã cho đã hiện diện trong tập `spidering`; ngược lại, ta thêm `url` vào tập và tiếp tục với việc tải về. Trong trường hợp của ta, ta không cần giải phóng `lock`, vì ta không quan tâm đến việc tải một URL hai lần, ngay cả khi các tác vụ `spider` được thực thi ở hai thời điểm hoàn toàn khác nhau. Nếu bạn đang xây dựng một `spider` có thể phải tải về hàng trăm nghìn trang web, việc xóa `url` đã tải về khỏi tập một khi tệp tương ứng được tải về sẽ giúp bạn giữ cho số phần tử (cardinality) của tập, và do đó mức tiêu thụ bộ nhớ, khỏi tăng vô hạn.

Race conditions can cause many problems, even if we are in a single-threaded environment. In some circumstances, they can lead to data corruption and are usually very hard to debug because of their ephemeral nature. So, it's always good practice to double-check for these types of situations when running tasks in parallel.

Race condition có thể gây ra nhiều vấn đề, ngay cả khi ta ở trong một môi trường đơn luồng. Trong một số hoàn cảnh, chúng có thể dẫn đến hỏng dữ liệu và thường rất khó debug do bản chất phù du của chúng. Vậy nên, luôn là thực hành tốt khi kiểm tra kỹ những loại tình huống này khi chạy các tác vụ song song.

Also, running an arbitrary number of parallel tasks can be a dangerous practice. In the next section, you will discover why it can be a problem and how to keep the number of parallel tasks under control.

Ngoài ra, chạy một số lượng tùy ý các tác vụ song song có thể là một thực hành nguy hiểm. Trong phần tiếp theo, bạn sẽ khám phá tại sao nó có thể là một vấn đề và làm thế nào để giữ số lượng tác vụ song song trong tầm kiểm soát.

Limited parallel execution — Thực thi song song có giới hạn

Spawning parallel tasks without control can often lead to excessive load. Imagine having thousands of files to read, URLs to access, or database queries to run in parallel. A common problem in such situations is running out of resources. The most common example is when an application tries to open too many files at once, utilizing all the file descriptors available to the process. A server that spawns unbounded parallel tasks to handle a user **request** could be exploited with a denial-of-service (DoS) attack. That is when a malicious actor can forge one or more requests to push the server to consume all the available resources and become unresponsive. Limiting the number of parallel tasks is, in general, a good practice that helps with building resilient applications.

Sinh ra các tác vụ song song không kiểm soát thường có thể dẫn đến tải quá mức. Hãy tưởng tượng có hàng nghìn tệp cần đọc, URL cần truy cập, hoặc truy vấn cơ sở dữ liệu cần chạy song song. Một vấn đề phổ biến trong những tình huống như vậy là cạn kiệt tài nguyên. Ví dụ phổ biến nhất là khi một ứng dụng cố mở quá nhiều tệp cùng một lúc, dùng hết tất cả các file descriptor khả dụng cho tiến trình. Một máy chủ sinh ra các tác vụ song song không giới hạn để xử lý một yêu cầu của người dùng có thể bị khai thác bằng một cuộc tấn công từ chối dịch vụ (DoS). Đó là khi một tác nhân xấu có thể giả mạo một hoặc nhiều yêu cầu để đẩy máy chủ tới mức tiêu thụ hết tất cả các tài nguyên khả dụng và trở nên mất phản hồi. Giới hạn số lượng tác vụ song song, nói chung, là một thực hành tốt giúp xây dựng các ứng dụng có khả năng chống chịu.

Version 3 of our web spider does not limit the number of parallel tasks and therefore, it is susceptible to crashing in a number of cases. For instance, if we try to run it against a significantly big website, we might see it running for a few seconds and then failing with the error code `ECONNREFUSED`. When we are downloading too many pages concurrently from a web server, the server might decide to start rejecting new connections from the same IP. In this case, our spider would just crash and we would be forced to relaunch the process if we wanted to continue crawling the website. We could just handle

ECONNREFUSED to stop the process from crashing, but we would still be risking allocating too many parallel tasks and might run into other issues.

Phiên bản 3 của web spider của ta không giới hạn số lượng tác vụ song song và do đó, nó dễ bị crash trong một số trường hợp. Chẳng hạn, nếu ta thử chạy nó trên một website lớn đáng kể, ta có thể thấy nó chạy được vài giây rồi thất bại với mã lỗi ECONNREFUSED. Khi ta đang tải về quá nhiều trang đồng thời từ một máy chủ web, máy chủ có thể quyết định bắt đầu từ chối các kết nối mới từ cùng một IP. Trong trường hợp này, spider của ta sẽ chỉ crash và ta buộc phải khởi chạy lại tiến trình nếu ta muốn tiếp tục thu thập website. Ta có thể chỉ cần xử lý ECONNREFUSED để ngăn tiến trình khỏi crash, nhưng ta vẫn sẽ mạo hiểm cấp phát quá nhiều tác vụ song song và có thể gặp các vấn đề khác.

In this section, you will see how we can make our spider more resilient by keeping the concurrency limited.

Trong phần này, bạn sẽ thấy cách ta có thể làm cho spider của mình có khả năng chống chịu tốt hơn bằng cách giữ tính đồng thời ở mức giới hạn.

The following diagram shows a situation where we have five tasks that run in parallel with a concurrency limit of two:

Sơ đồ sau cho thấy một tình huống mà ta có năm tác vụ chạy song song với giới hạn đồng thời là hai:

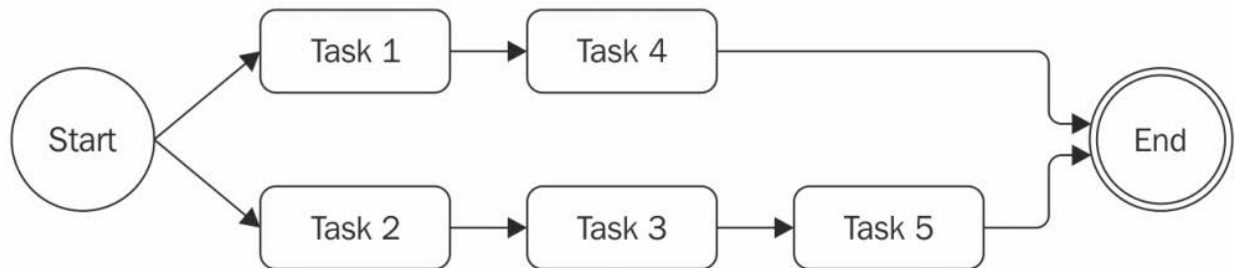


Figure 4.5: An example of how concurrency can be limited to a maximum of two parallel tasks

Hình 4.5: Một ví dụ về cách tính đồng thời có thể được giới hạn ở mức tối đa hai tác vụ song song

From Figure 4.5, it should be clear how our algorithm works:

Từ Hình 4.5, hẳn đã rõ thuật toán của ta hoạt động như thế nào:

1. Initially, we spawn as many tasks as we can without exceeding the concurrency limit.
2. Then, every time a task is completed, we spawn one or more tasks until we reach the limit again.
 1. Ban đầu, ta sinh ra càng nhiều tác vụ càng tốt mà không vượt quá giới hạn đồng thời.
 2. Sau đó, mỗi khi một tác vụ hoàn tất, ta sinh ra một hoặc nhiều tác vụ cho đến khi ta lại đạt tới giới hạn.

In the next section, we will explore a possible implementation of the limited parallel execution pattern.

Trong phần tiếp theo, ta sẽ khám phá một bản hiện thực khả dĩ của mẫu thực thi song song có giới hạn.

Limiting concurrency — Giới hạn tính đồng thời

We will now look at a pattern that will execute a set of given tasks in parallel with limited concurrency:

Giờ ta sẽ xem một mẫu thực thi một tập hợp các tác vụ cho trước song song với tính đồng thời giới hạn:

```
const tasks = [
  // ...
]
const concurrency = 2
let running = 0
let completed = 0
let index = 0
function next () { // (1)
  while (running < concurrency && index < tasks.length) {
    const task = tasks[index++]
    task(() => { // (2)
      if (++completed === tasks.length) {
        return finish()
      }
      running--
      next()
    })
    running++
  }
}
next()
function finish() {
  // all tasks finished
}
```

This algorithm can be considered a mixture of sequential execution and parallel execution. In fact, you might notice similarities with both patterns:

Thuật toán này có thể được xem là sự pha trộn giữa thực thi tuần tự và thực thi song song. Thực vậy, bạn có thể nhận thấy những điểm tương đồng với cả hai mẫu:

1. We have an iterator function, which we call `next()`, and then an inner loop that spawns as many tasks as possible in parallel while staying within the concurrency limit.
2. The next important part is the callback we pass to each task, which checks whether we completed all the tasks in the list. If there are still tasks to run, it invokes `next()` to spawn another set of tasks.
 1. Ta có một hàm iterator, mà ta gọi là `next()`, rồi một vòng lặp bên trong sinh ra càng nhiều tác vụ càng tốt song song trong khi vẫn nằm trong giới hạn đồng thời.
 2. Phần quan trọng kế tiếp là callback ta truyền cho mỗi tác vụ, callback này kiểm tra xem ta đã hoàn tất tất cả các tác vụ trong danh sách hay chưa. Nếu vẫn còn tác vụ cần chạy, nó gọi `next()` để sinh ra một tập tác vụ khác.

Pretty simple, isn't it?

Khá đơn giản, phải không?

Globally limiting concurrency — Giới hạn tính đồng thời ở phạm vi toàn cục

Our web spider application is perfect for applying what we just learned about limiting the concurrency of a set of tasks. In fact, to avoid the situation in which we have thousands of links being crawled at the same time, we can enforce a limit on the concurrency of this process by adding some predictability regarding the number of concurrent downloads.

Ứng dụng web spider của ta là hoàn hảo để áp dụng những gì ta vừa học về giới hạn tính đồng thời của một tập hợp các tác vụ. Thực vậy, để tránh tình huống mà ta có hàng nghìn liên kết bị thu thập cùng một lúc, ta có thể áp đặt một giới hạn lên tính đồng thời của quá trình này bằng cách thêm chút khả năng dự đoán về số lượt tải về đồng thời.

We could apply this implementation of the limited concurrency pattern to our `spiderLinks()` function, but by doing that, we would only be limiting the concurrency of tasks spawned from the links found within a given page. If we chose, for example, a concurrency of two, we would have, at most, two links downloaded in parallel for each page. However, as we can download multiple links at once, each page would then spawn another two downloads, causing the grand total of download operations to grow exponentially anyway.

Ta có thể áp dụng bản hiện thực này của mẫu đồng thời giới hạn cho hàm `spiderLinks()`, nhưng làm vậy, ta chỉ giới hạn tính đồng thời của các tác vụ được sinh ra từ các liên kết tìm thấy trong một trang cho trước. Nếu ta chọn, ví dụ, một mức đồng thời là hai, ta sẽ có nhiều nhất hai liên kết được tải về song song cho mỗi trang. Tuy nhiên, vì ta có thể tải về nhiều liên kết cùng một lúc, mỗi trang khi đó sẽ sinh ra hai lượt tải về khác, khiến tổng số thao tác tải về dù sao vẫn tăng theo cấp số mũ.

In general, this implementation of the limited concurrency pattern works very well when we have a predetermined set of tasks to execute, or when the set of tasks grows linearly over time. When, instead, a task can spawn two or more tasks directly, as happens with our web spider, this implementation is not suitable for limiting the global concurrency.

Nói chung, bản hiện thực này của mẫu đồng thời giới hạn hoạt động rất tốt khi ta có một tập hợp các tác vụ định trước để thực thi, hoặc khi tập hợp các tác vụ tăng tuyến tính theo thời gian. Ngược lại, khi một tác vụ có thể trực tiếp sinh ra hai hoặc nhiều tác vụ, như xảy ra với web spider của ta, bản hiện thực này không thích hợp để giới hạn tính đồng thời toàn cục.

Queues to the rescue — Hàng đợi đến giải cứu What we really want, then, is to limit the global number of download operations we can have running in parallel. We could slightly modify the pattern shown in the previous section, but this is left as an exercise for you. Instead, let's discuss another mechanism that makes use of queues to limit the concurrency of multiple tasks. Let's see how this works.

Vậy điều ta thực sự muốn là giới hạn tổng số thao tác tải về toàn cục mà ta có thể có đang chạy song song. Ta có thể sửa đổi đôi chút mẫu được trình bày ở phần trước, nhưng điều này được để lại như một bài tập cho bạn. Thay vào đó, hãy thảo luận một cơ chế khác dùng hàng đợi để giới hạn tính đồng thời của nhiều tác vụ. Hãy xem cách này hoạt động.

We are now going to implement a simple class named `TaskQueue`, which will combine a queue with the algorithm that was presented while discussing limited concurrency. Let's create a new module named `taskQueue.js`:

Giờ ta sẽ hiện thực một lớp đơn giản tên là `TaskQueue`, lớp này sẽ kết hợp một hàng đợi với thuật toán được trình bày khi thảo luận về đồng thời giới hạn. Hãy tạo một module mới tên là `taskQueue.js`:

```

export class TaskQueue {
  constructor (concurrency) {
    this.concurrency = concurrency
    this.running = 0
    this.queue = []
  }
  pushTask (task) {
    this.queue.push(task)
    process.nextTick(this.next.bind(this))
    return this
  }
  next () {
    while (this.running < this.concurrency && this.queue.length) {
      const task = this.queue.shift()
      task(() => {
        this.running--
        process.nextTick(this.next.bind(this))
      })
      this.running++
    }
  }
}

```

The constructor of this class takes, as input, only the concurrency limit, but besides that, it initializes the instance variables `running` and `queue`. The former variable is a counter used to keep track of all the running tasks, while the latter is the array that will be used as a queue to store the pending tasks.

Hàm khởi tạo (constructor) của lớp này chỉ nhận vào một đầu vào duy nhất là giới hạn đồng thời, nhưng ngoài đó, nó khởi tạo các biến thực thể `running` và `queue`. Biến đầu là một bộ đếm dùng để theo dõi tất cả các tác vụ đang chạy, còn biến sau là mảng sẽ được dùng làm hàng đợi để lưu các tác vụ đang chờ.

The `pushTask()` method simply adds a new task to the queue and then bootstraps the execution of the worker by asynchronously invoking `this.next()`. Note that we have to use `bind` because otherwise, the `next` function will lose its context when invoked by `process.nextTick`.

Phương thức `pushTask()` đơn giản thêm một tác vụ mới vào hàng đợi rồi khởi động việc thực thi worker bằng cách gọi `this.next()` một cách bất đồng bộ. Lưu ý rằng ta phải dùng `bind` bởi nếu không, hàm `next` sẽ mất ngữ cảnh của nó khi được `process.nextTick` gọi.

The `next()` method spawns a set of tasks from the queue, ensuring that it does not exceed the concurrency limit.

Phương thức `next()` sinh ra một tập hợp các tác vụ từ hàng đợi, đảm bảo rằng nó không vượt quá giới hạn đồng thời.

You may notice that this method has some similarities with the pattern presented at the beginning of the Limiting concurrency section. It essentially starts as many tasks from the queue as possible, without exceeding the concurrency limit. When each task is complete, it updates the count of running tasks and then starts another round of tasks by asynchronously invoking `next()` again. The interesting property of the `TaskQueue` class is that it allows us to dynamically add new items to the queue. The other advantage is that, now, we have a central entity responsible for the limitation of the concurrency of our tasks, which can be shared across all the instances of a function's execution. In our case, it's the `spider()` function, as you will see in a moment.

Bạn có thể nhận thấy phương thức này có một số điểm tương đồng với mẫu được trình bày ở đầu phần Giới hạn tính đồng thời. Về cơ bản nó khởi động càng nhiều tác vụ từ hàng đợi càng tốt, mà không vượt quá giới hạn đồng thời. Khi mỗi tác vụ hoàn tất, nó cập nhật số đếm các tác vụ đang chạy rồi khởi động một vòng tác vụ khác bằng cách gọi `next()` lại một lần nữa một cách bất đồng bộ. Tính chất thú vị của lớp `TaskQueue` là nó cho phép ta thêm các mục mới vào hàng đợi một cách động. Lợi thế khác là, giờ đây, ta có một thực thể trung tâm chịu trách nhiệm về việc giới hạn tính đồng thời của các tác vụ, có thể được chia sẻ xuyên suốt tất cả các lần thực thi của một hàm. Trong trường hợp của ta, đó là hàm `spider()`, như bạn sẽ thấy trong chốc lát.

Refining the TaskQueue — Tinh chỉnh TaskQueue The previous implementation of `TaskQueue` is sufficient to demonstrate the queue pattern, but in order to be used in real-life projects, it needs a couple of extra features. For instance, how can we tell when one of the tasks has failed? How do we know whether all the work in the queue has been completed?

Bản hiện thực trước của `TaskQueue` đủ để minh họa mẫu hàng đợi, nhưng để được dùng trong các dự án thực tế, nó cần một vài tính năng bổ sung. Chẳng hạn, làm thế nào ta có thể biết khi một trong các tác vụ đã thất bại? Làm thế nào ta biết liệu tất cả công việc trong hàng đợi đã hoàn tất hay chưa?

Let's bring back some of the concepts we discussed in Chapter 3, Callbacks and Events, and let's turn the `TaskQueue` into an `EventEmitter` so that we can emit events to propagate task failures and to inform any **observer** when the queue is empty.

Hãy nhắc lại một số khái niệm ta đã thảo luận ở Chương 3, Callback và Sự kiện, rồi biến `TaskQueue` thành một `EventEmitter` để ta có thể phát ra các sự kiện nhằm lan truyền các thất bại của tác vụ và để thông báo cho bất kỳ observer nào khi hàng đợi trống.

The first change we have to make is to import the `EventEmitter` class and let our `TaskQueue` extend it:

Thay đổi đầu tiên ta phải thực hiện là import lớp `EventEmitter` và cho `TaskQueue` của ta kế thừa nó:

```
import { EventEmitter } from 'events'
export class TaskQueue extends EventEmitter {
  constructor (concurrency) {
    super()
    // ...
  }
  // ...
}
```

At this point, we can use `this.emit` to fire events from within the `TaskQueue` `next()` method:

Tại điểm này, ta có thể dùng `this.emit` để kích hoạt các sự kiện từ bên trong phương thức `next()` của `TaskQueue`:

```
next () {
  if (this.running === 0 && this.queue.length === 0) {           // (1)
    return this.emit('empty')
  }
  while (this.running < this.concurrency && this.queue.length) {
    const task = this.queue.shift()
    task((err) => {                                               // (2)
```

```

    if (err) {
        this.emit('error', err)
    }
    this.running--
    process.nextTick(this.next.bind(this))
  })
  this.running++
}
}

```

Comparing this implementation with the previous one, there are two additions here:

So sánh bản hiện thực này với bản trước, có hai điểm bổ sung ở đây:

- Every time the `next()` function is called, we check that no task is running and whether the queue is empty. In such a case, it means that the queue has been drained and we can fire the empty event.
- The completion callback of every task can now be invoked by passing an error. We check whether an error is actually passed, indicating that the task has failed, and in that case, we propagate such an error with an error event.
 - Mỗi khi hàm `next()` được gọi, ta kiểm tra rằng không có tác vụ nào đang chạy và liệu hàng đợi có trống hay không. Trong trường hợp đó, điều đó nghĩa là hàng đợi đã được rút cạn và ta có thể kích hoạt sự kiện `empty`.
 - Callback hoàn tất của mỗi tác vụ giờ có thể được gọi bằng cách truyền một lỗi. Ta kiểm tra xem một lỗi có thực sự được truyền hay không, cho thấy rằng tác vụ đã thất bại, và trong trường hợp đó, ta lan truyền lỗi đó bằng một sự kiện `error`.

Notice that in case of an error, we are deliberately keeping the queue running. We are not stopping other tasks in progress, nor removing any pending tasks. This is quite common with queue-based systems. Errors are expected to happen and rather than letting the system crash on these occasions, it is generally better to identify errors and to think about retry or recovery strategies. We will discuss these concepts a bit more in Chapter 13, Messaging and Integration Patterns.

Lưu ý rằng trong trường hợp có lỗi, ta cố ý giữ cho hàng đợi tiếp tục chạy. Ta không dừng các tác vụ khác đang tiến hành, cũng không xóa bất kỳ tác vụ đang chờ nào. Điều này khá phổ biến với các hệ thống dựa trên hàng đợi. Lỗi được kỳ vọng sẽ xảy ra và thay vì để hệ thống crash trong những dịp này, nói chung tốt hơn là nhận diện lỗi và nghĩ về các chiến lược thử lại hoặc khôi phục. Ta sẽ thảo luận những khái niệm này thêm một chút ở Chương 13, Các mẫu Nhấn tin và Tích hợp.

Web spider version 4 — Web spider phiên bản 4 Now that we have our generic queue to execute tasks in a limited parallel flow, let's use it straightaway to refactor our web spider application.

Giờ đây khi ta có hàng đợi tổng quát của mình để thực thi các tác vụ trong một luồng song song giới hạn, hãy dùng nó ngay để tái cấu trúc ứng dụng web spider của ta.

We are going to use an instance of `TaskQueue` as a work backlog; every URL that we want to crawl needs to be appended to the queue as a task. The starting URL will be added as the first task, then every other URL discovered during the crawling process will be added as well. The queue will manage all the scheduling for us, making sure that the number of tasks in progress (that is, the number of pages being downloaded or read from the filesystem) at any given time is never greater than the concurrency limit configured for the given `TaskQueue` instance.

Ta sẽ dùng một thực thể của `TaskQueue` làm một danh sách công việc tồn đọng (work backlog); mỗi URL mà ta muốn thu thập cần được đẩy vào hàng đợi như một tác vụ. URL khởi đầu sẽ được thêm vào làm tác vụ đầu tiên, rồi mọi URL khác được phát hiện trong quá trình thu thập cũng sẽ được thêm vào. Hàng đợi sẽ quản lý toàn bộ việc lên lịch cho ta, đảm bảo rằng số tác vụ đang tiến hành (tức là số trang đang được tải về hoặc đọc từ hệ thống tệp) tại bất kỳ thời điểm cho trước nào không bao giờ lớn hơn giới hạn đồng thời được cấu hình cho thực thể `TaskQueue` đã cho.

We have already defined the logic to crawl a given URL inside our `spider()` function. We can consider this to be our generic crawling task. For more clarity, it's best to rename this function `spiderTask`:

Ta đã định nghĩa logic để thu thập một URL cho trước bên trong hàm `spider()` của ta. Ta có thể xem đây là tác vụ thu thập tổng quát của mình. Cho rõ ràng hơn, tốt nhất là đổi tên hàm này thành `spiderTask`:

```
function spiderTask (url, nesting, queue, cb) { // (1)
  const filename = urlToFilename(url)
  fs.readFile(filename, 'utf8', (err, fileContent) => {
    if (err) {
      if (err.code !== 'ENOENT') {
        return cb(err)
      }
      return download(url, filename, (err, requestContent) => {
        if (err) {
          return cb(err)
        }
        spiderLinks(url, requestContent, nesting, queue) // (2)
        return cb()
      })
    }
    spiderLinks(url, fileContent, nesting, queue) // (3)
    return cb()
  })
}
```

Other than renaming the function, you might have noticed that we applied some other small changes:

Ngoài việc đổi tên hàm, bạn có thể đã nhận thấy ta áp dụng một số thay đổi nhỏ khác:

- The function signature now accepts a new parameter called `queue`. This is an instance of `TaskQueue` that we need to carry over to be able to append new tasks when necessary.
- The function responsible for adding new links to crawl is `spiderLinks`, so we need to make sure that we pass the `queue` instance when we call this function after downloading a new page.
- We also need to pass the `queue` instance to `spiderLinks` when we are invoking that from an already downloaded file.
 - Chữ ký của hàm giờ chấp nhận một tham số mới tên là `queue`. Đây là một thực thể của `TaskQueue` mà ta cần mang theo để có thể đẩy thêm các tác vụ mới khi cần thiết.
 - Hàm chịu trách nhiệm thêm các liên kết mới để thu thập là `spiderLinks`, nên ta cần đảm bảo rằng ta truyền thực thể `queue` khi ta gọi hàm này sau khi tải về một trang mới.
 - Ta cũng cần truyền thực thể `queue` cho `spiderLinks` khi ta gọi nó từ một tệp đã được tải về.

Let's revisit the `spiderLinks()` function. This function can now be greatly simplified as it doesn't have to keep track of task completion anymore, as this work has been delegated to the queue. Its job becomes effectively synchronous now; it just needs to invoke the new `spider()` function (which we will define shortly) to push a new task to the queue, one for each discovered link:

Hãy xem lại hàm `spiderLinks()`. Hàm này giờ có thể được đơn giản hóa rất nhiều vì nó không còn phải theo dõi sự hoàn tất của tác vụ nữa, vì công việc này đã được ủy thác cho hàng đợi. Công việc của nó giờ thực chất trở thành đồng bộ; nó chỉ cần gọi hàm `spider()` mới (mà ta sẽ định nghĩa ngay sau đây) để đẩy một tác vụ mới vào hàng đợi, một tác vụ cho mỗi liên kết được phát hiện:

```
function spiderLinks (currentUrl, body, nesting, queue) {
  if (nesting === 0) {
    return
  }
  const links = getPageLinks(currentUrl, body)
  if (links.length === 0) {
    return
  }
  links.forEach(link => spider(link, nesting - 1, queue))
}
```

Let's now revisit the `spider()` function, which needs to act as the entry point for the first URL; it will also be used to add every new discovered URL to the queue:

Giờ hãy xem lại hàm `spider()`, hàm này cần đóng vai trò là điểm vào (entry point) cho URL đầu tiên; nó cũng sẽ được dùng để thêm mọi URL mới được phát hiện vào queue:

```
const spidering = new Set() // (1)
export function spider (url, nesting, queue) {
  if (spidering.has(url)) {
    return
  }
  spidering.add(url)
  queue.pushTask((done) => { // (2)
    spiderTask(url, nesting, queue, done)
  })
}
```

As you can see, this function now has two main responsibilities:

Như bạn có thể thấy, hàm này giờ có hai trách nhiệm chính:

1. It manages the bookkeeping of the URLs already visited or in progress by using the `spidering` set.
2. It pushes a new task to the queue. Once executed, this task will invoke the `spiderTask()` function, effectively starting the crawling of the given URL.
 1. Nó quản lý việc ghi sổ (bookkeeping) các URL đã được ghé thăm hoặc đang tiến hành bằng cách dùng tập `spidering`.
 2. Nó đẩy một tác vụ mới vào queue. Một khi được thực thi, tác vụ này sẽ gọi hàm `spiderTask()`, qua đó thực sự bắt đầu việc thu thập URL đã cho.

Finally, we can update the `spider-cli.js` script, which allows us to invoke our spider from the command line:

Cuối cùng, ta có thể cập nhật script `spider-cli.js`, cho phép ta gọi spider của mình từ dòng lệnh:

```
import { spider } from './spider.js'
import { TaskQueue } from './TaskQueue.js'
const url = process.argv[2] // (1)
const nesting = Number.parseInt(process.argv[3], 10) || 1
const concurrency = Number.parseInt(process.argv[4], 10) || 2
const spiderQueue = new TaskQueue(concurrency) // (2)
spiderQueue.on('error', console.error)
spiderQueue.on('empty', () => console.log('Download complete'))
spider(url, nesting, spiderQueue) // (3)
```

This script is now composed of three main parts:

Script này giờ gồm ba phần chính:

1. CLI arguments parsing. Note that the script now accepts a third additional parameter that can be used to customize the concurrency level.
2. A TaskQueue object is created and listeners are attached to the error and empty events. When an error occurs, we simply want to print it. When the queue is empty, that means that we've finished crawling the website.
3. Finally, we start the crawling process by invoking the spider function.
 1. Phân tích các tham số CLI. Lưu ý rằng script giờ chấp nhận một tham số bổ sung thứ ba có thể được dùng để tùy chỉnh mức độ đồng thời.
 2. Một đối tượng TaskQueue được tạo và các listener được gắn vào sự kiện error và empty. Khi một lỗi xảy ra, ta đơn giản muốn in nó ra. Khi hàng đợi trống, điều đó nghĩa là ta đã thu thập xong website.
 3. Cuối cùng, ta bắt đầu quá trình thu thập bằng cách gọi hàm spider.

After we have applied these changes, we can try to run the spider module again. When we run the following command:

Sau khi ta đã áp dụng những thay đổi này, ta có thể thử chạy lại module spider. Khi ta chạy lệnh sau:

```
node spider-cli.js https://loige.co 1 4
```

We should notice that no more than four downloads will be active at the same time.

Ta sẽ nhận thấy rằng không quá bốn lượt tải về sẽ hoạt động cùng một lúc.

With this final example, we've concluded our exploration of callback-based patterns. In the next section, we will close this chapter by looking at a famous library that provides a production-ready implementation of these patterns and many other asynchronous utilities.

Với ví dụ cuối cùng này, ta đã kết thúc cuộc khám phá các mẫu dựa trên callback của mình. Trong phần tiếp theo, ta sẽ khép lại chương này bằng cách xem xét một thư viện nổi tiếng cung cấp bản hiện thực sẵn sàng cho production của những mẫu này và nhiều tiện ích bất đồng bộ khác.

The async library — Thư viện async

If you take a look for a moment at every control flow pattern we have analyzed so far, you will see that they can be used as a base to build reusable and more generic solutions. For example, we

could wrap the unlimited parallel execution algorithm into a function that accepts a list of tasks, runs them in parallel, and invokes the given callback when all of them are complete. This way of wrapping control flow algorithms into reusable functions can lead to a more declarative and expressive way of defining asynchronous control flows, and that's exactly what `async (nodejsdp.link/async)` does.

Nếu bạn nhìn một lát vào mọi mẫu điều khiển luồng mà ta đã phân tích cho đến nay, bạn sẽ thấy rằng chúng có thể được dùng làm nền tảng để xây dựng những giải pháp có thể tái sử dụng và tổng quát hơn. Ví dụ, ta có thể bọc thuật toán thực thi song song không giới hạn vào một hàm nhận một danh sách các tác vụ, chạy chúng song song, và gọi callback đã cho khi tất cả chúng hoàn tất. Cách bọc các thuật toán điều khiển luồng vào các hàm có thể tái sử dụng này có thể dẫn đến một cách định nghĩa luồng điều khiển bất đồng bộ mang tính khai báo (declarative) và biểu cảm hơn, và đó chính xác là điều mà `async (nodejsdp.link/async)` làm.

The `async` library (not to be confused with the `async/await` keywords, which we will discuss later in this book) is a very popular solution, in Node.js and JavaScript in general, for dealing with asynchronous code. It offers a set of functions that greatly simplify the execution of tasks in different configurations, and it also provides useful helpers for dealing with collections asynchronously. Even though there are several other libraries with a similar goal, `async` is the de facto standard in Node.js due to its historic popularity, especially when using callbacks to define asynchronous tasks.

Thư viện `async` (đừng nhầm với các từ khóa `async/await`, mà ta sẽ thảo luận sau trong cuốn sách này) là một giải pháp rất phổ biến, trong Node.js và JavaScript nói chung, để xử lý code bất đồng bộ. Nó cung cấp một tập hợp các hàm giúp đơn giản hóa rất nhiều việc thực thi các tác vụ trong những cấu hình khác nhau, và nó cũng cung cấp các hàm trợ giúp hữu ích để xử lý các tập hợp một cách bất đồng bộ. Mặc dù có vài thư viện khác với mục tiêu tương tự, `async` là chuẩn mực trên thực tế (de facto) trong Node.js nhờ sự phổ biến mang tính lịch sử của nó, đặc biệt khi dùng callback để định nghĩa các tác vụ bất đồng bộ.

Just to give you an idea of some of the most important capabilities of the `async` module, here is a sample of the functionalities it exposes:

Chỉ để cho bạn một ý niệm về một số khả năng quan trọng nhất của module `async`, đây là một mẫu các chức năng mà nó phơi bày:

- Execute asynchronous functions over a collection of elements (in series or in parallel with limited concurrency).
- Execute a chain of asynchronous functions (waterfall) where the output of every function becomes the input of the next one.
- Offers a queue abstraction functionally equivalent to the one we implemented with our `TaskQueue` utility.
- Provides other interesting asynchronous patterns such as `race` (executes multiple asynchronous functions in parallel and stops when the first one completes).
 - Thực thi các hàm bất đồng bộ trên một tập hợp các phần tử (theo chuỗi hoặc song song với tính đồng thời giới hạn).
 - Thực thi một chuỗi các hàm bất đồng bộ (waterfall) nơi đầu ra của mỗi hàm trở thành đầu vào của hàm kế tiếp.
 - Cung cấp một sự trừu tượng hàng đợi tương đương về mặt chức năng với cái mà ta đã hiện thực với tiện ích `TaskQueue` của mình.
 - Cung cấp các mẫu bất đồng bộ thú vị khác như `race` (thực thi nhiều hàm bất đồng bộ song song và dừng khi hàm đầu tiên hoàn tất).

Check out the `async` documentation (`nodejsdp.link/async`) to find out more about the module

and to see some examples.

Hãy xem tài liệu của `async` (nodejsdp.link/async) để tìm hiểu thêm về module và xem một số ví dụ.

Once you've understood the fundamentals of the asynchronous patterns described in this chapter, you shouldn't rely on the simplified implementations presented here for your everyday control flow needs. Instead, it's better to adopt a broadly used and battle-tested library like `async` for your production applications, unless your use case is so advanced that you require a custom algorithm.

Một khi bạn đã hiểu các nền tảng cơ bản của các mẫu bất đồng bộ được mô tả trong chương này, bạn không nên dựa vào các bản hiện thực đơn giản hóa được trình bày ở đây cho nhu cầu điều khiển luồng hàng ngày của mình. Thay vào đó, tốt hơn là tiếp nhận một thư viện được dùng rộng rãi và đã được kiểm chứng qua thực chiến như `async` cho các ứng dụng production của bạn, trừ khi tình huống của bạn nâng cao đến mức bạn cần một thuật toán tùy chỉnh.

Summary — Tóm tắt

At the beginning of this chapter, it was stated that Node.js programming can be tough because of its asynchronous nature, especially for people used to developing on other platforms. However, throughout this chapter, you saw how asynchronous APIs can be bent to your will. You discovered that the tools at your disposal are indeed versatile and provide good solutions to most of your problems, in addition to offering a programming style for every taste.

Ở đầu chương này, đã có khẳng định rằng lập trình Node.js có thể khó khăn vì bản chất bất đồng bộ của nó, đặc biệt với những người quen phát triển trên các nền tảng khác. Tuy nhiên, xuyên suốt chương này, bạn đã thấy cách các API bất đồng bộ có thể được uốn theo ý chí của bạn. Bạn đã khám phá rằng các công cụ trong tầm tay bạn thực sự linh hoạt và cung cấp những giải pháp tốt cho hầu hết các vấn đề của bạn, ngoài việc đem lại một phong cách lập trình hợp mọi khẩu vị.

In this chapter, we also kept refactoring and improving our web crawler example. When dealing with asynchronous code, it can sometimes be challenging to figure out the right ergonomics that can keep your code simple and effective, so allow yourself some time to digest the concepts explored in this chapter and to experiment with them.

Trong chương này, ta cũng liên tục tái cấu trúc và cải thiện ví dụ web crawler của mình. Khi xử lý code bất đồng bộ, đôi khi có thể là một thách thức để tìm ra sự tiện dụng (ergonomics) phù hợp có thể giữ cho code của bạn đơn giản và hiệu quả, vậy nên hãy cho phép bản thân chút thời gian để tiêu hóa các khái niệm được khám phá trong chương này và để thử nghiệm với chúng.

Our journey with asynchronous Node.js programming has just started. In the next few chapters, you will be introduced to other broadly adopted techniques that leverage promises, and `async/await`. After you've learned all these techniques, you will be able to choose the best solution for your needs or use many of them together in the same project.

Cuộc hành trình của ta với lập trình Node.js bất đồng bộ chỉ mới bắt đầu. Trong vài chương tiếp theo, bạn sẽ được giới thiệu những kỹ thuật được tiếp nhận rộng rãi khác tận dụng promise, và `async/await`. Sau khi bạn đã học tất cả những kỹ thuật này, bạn sẽ có thể chọn giải pháp tốt nhất cho nhu cầu của mình hoặc dùng nhiều trong số chúng cùng nhau trong cùng một dự án.

Exercises — Bài tập

- 4.1 File concatenation: Write the implementation of `concatFiles()`, a callback-style function that takes two or more paths to text files in the filesystem and a destination file:

- 4.1 Nối tệp: Viết bản hiện thực của `concatFiles()`, một hàm theo phong cách callback nhận vào hai hoặc nhiều đường dẫn đến các tệp văn bản trong hệ thống tệp và một tệp đích:

```
function concatFiles (srcFile1, srcFile2, srcFile3, ... ,
                      dest, cb) {
    // ...
}
```

This function must copy the contents of every source file into the destination file, respecting the order of the files, as provided by the arguments list. For instance, given two files, if the first file contains foo and the second file contains bar, the function should write foobar (and not barfoo) in the destination file. Note that the preceding example signature is not valid JavaScript syntax: you need to find a different way to handle an arbitrary number of arguments. For instance, you could use the rest parameters syntax (nodejsdp.link/rest-parameters).

Hàm này phải sao chép nội dung của mỗi tệp nguồn vào tệp đích, tôn trọng thứ tự của các tệp, như được cung cấp bởi danh sách tham số. Chẳng hạn, cho hai tệp, nếu tệp đầu chứa foo và tệp thứ hai chứa bar, hàm nên ghi foobar (chứ không phải barfoo) vào tệp đích. Lưu ý rằng chữ ký ví dụ ở trên không phải là cú pháp JavaScript hợp lệ: bạn cần tìm một cách khác để xử lý một số lượng tham số tùy ý. Chẳng hạn, bạn có thể dùng cú pháp `rest parameters` (nodejsdp.link/rest-parameters).

- 4.2 List files recursively: Write `listNestedFiles()`, a callback-style function that takes, as the input, the path to a directory in the local filesystem and that asynchronously iterates over all the subdirectories to eventually return a list of all the files discovered. Here is what the signature of the function should look like:

- 4.2 Liệt kê tệp một cách đệ quy: Viết `listNestedFiles()`, một hàm theo phong cách callback nhận vào, làm đầu vào, đường dẫn đến một thư mục trong hệ thống tệp cục bộ và duyệt bất đồng bộ qua tất cả các thư mục con để cuối cùng trả về một danh sách tất cả các tệp được phát hiện. Đây là chữ ký mà hàm nên có:

```
function listNestedFiles (dir, cb) { /* ... */ }
```

Bonus points if you manage to avoid callback hell. Feel free to create additional helper functions if needed.

Điểm thưởng nếu bạn xoay sở tránh được callback hell. Cứ tự nhiên tạo các hàm trợ giúp bổ sung nếu cần.

- 4.3 Recursive find: Write `recursiveFind()`, a callback-style function that takes a path to a directory in the local filesystem and a keyword, as per the following signature:

- 4.3 Tìm kiếm đệ quy: Viết `recursiveFind()`, một hàm theo phong cách callback nhận vào một đường dẫn đến một thư mục trong hệ thống tệp cục bộ và một từ khóa, theo chữ ký sau:

```
function recursiveFind(dir, keyword, cb) { /* ... */ }
```

The function must find all the text files within the given directory that contain the given keyword in the file contents. The list of matching files should be returned using the callback when the search

is completed. If no matching file is found, the callback must be invoked with an empty array. As an example test case, if you have the files `foo.txt`, `bar.txt`, and `baz.txt` in `myDir` and the keyword `'batman'` is contained in the files `foo.txt` and `baz.txt`, you should be able to run the following code:

Hàm phải tìm tất cả các tệp văn bản trong thư mục cho trước có chứa từ khóa cho trước trong nội dung tệp. Danh sách các tệp khớp nên được trả về bằng callback khi việc tìm kiếm hoàn tất. Nếu không tìm thấy tệp khớp nào, callback phải được gọi với một mảng rỗng. Như một ca kiểm thử ví dụ, nếu bạn có các tệp `foo.txt`, `bar.txt`, và `baz.txt` trong `myDir` và từ khóa `'batman'` được chứa trong các tệp `foo.txt` và `baz.txt`, bạn nên có thể chạy code sau:

```
recursiveFind('myDir', 'batman', console.log)
// should print ['foo.txt', 'baz.txt']
```

Bonus points if you make the search recursive (it looks for text files in any subdirectory as well). Extra bonus points if you manage to perform the search within different files and subdirectories in parallel, but be careful to keep the number of parallel tasks under control!

Điểm thưởng nếu bạn làm cho việc tìm kiếm có tính đệ quy (nó tìm các tệp văn bản trong bất kỳ thư mục con nào nữa). Điểm thưởng thêm nếu bạn xoay sở thực hiện việc tìm kiếm trong các tệp và thư mục con khác nhau một cách song song, nhưng hãy cẩn thận giữ số lượng tác vụ song song trong tầm kiểm soát!

Chapter 5: Asynchronous Control Flow Patterns with Promises and Async/Await — Chương 5: Các mẫu điều khiển luồng bất đồng bộ với Promise và Async/Await

Callbacks are the low-level building blocks of asynchronous programming in Node.js, but they are far from being developer-friendly. In fact, in the last chapter, we learned techniques to implement different **control flow** constructs using callbacks, and we can say that they are quite complex and verbose compared to the (low) level of complexity of the tasks they try to accomplish. In particular, serial execution flow, which is the predominant control flow structure in most of the code we write, can easily lead an untrained developer to write code affected by the **callback** hell problem. On top of that, even if properly implemented, a serial execution flow seems needlessly complicated and error-prone. Let's also remember how fragile error management with callbacks is; if we forget to forward an error, then it just gets lost, and if we forget to catch any exception thrown by some synchronous code, then the program crashes. And all of this without considering that *Zalgo* is always breathing down our necks.

Callback là những khối xây dựng cấp thấp của lập trình bất đồng bộ trong Node.js, nhưng chúng còn rất xa mới thân thiện với lập trình viên. Thực tế, ở chương trước chúng ta đã học các kỹ thuật để hiện thực những cấu trúc luồng điều khiển (control flow) khác nhau bằng callback, và có thể nói chúng khá phức tạp và dài dòng so với mức độ phức tạp (thấp) của những tác vụ mà chúng cố gắng hoàn thành. Đặc biệt, luồng thực thi tuần tự, vốn là cấu trúc luồng điều khiển chiếm ưu thế trong hầu hết mã chúng ta viết, có thể dễ dàng khiến một lập trình viên thiếu kinh nghiệm viết ra mã dính phải vấn đề callback hell. Hơn nữa, ngay cả khi được hiện thực đúng cách, một luồng thực thi tuần tự vẫn trông phức tạp một cách không cần thiết và dễ phát sinh lỗi. Hãy cũng nhớ lại việc quản lý lỗi với callback mong manh đến mức nào; nếu ta quên chuyển tiếp một lỗi, nó sẽ bị mất hút, và nếu ta quên bắt một ngoại lệ do đoạn mã đồng bộ nào đó ném ra, thì chương trình sẽ sụp đổ. Và tất cả những điều này còn chưa tính đến việc *Zalgo* luôn rình rập sau gáy chúng ta.

Node.js and JavaScript have been criticized for many years for the lack of a native solution to a problem so common and ubiquitous. Luckily, over the years, the community has worked on new solutions to the problem and finally, after many iterations, discussions, and years of waiting, today we have a proper solution to the “callback issue.” The first step toward a better asynchronous code experience is the **promise**, an object that “carries” the status and the eventual result of an

asynchronous operation. A promise can be easily chained to implement serial execution flows and can be moved around like any other object. Promises simplify asynchronous code a lot; however, there was still room for improvement. So, in an attempt to make the ubiquitous serial execution flow as simple as possible, a new construct was introduced, called **async/await**, which can finally make asynchronous code look like synchronous code.

Node.js và JavaScript đã bị chỉ trích trong nhiều năm vì thiếu một giải pháp gốc (native) cho một vấn đề phổ biến và hiện diện khắp nơi đến vậy. May mắn thay, qua nhiều năm, cộng đồng đã nỗ lực tạo ra những giải pháp mới cho vấn đề này, và cuối cùng, sau nhiều lần lặp đi lặp lại, nhiều cuộc tranh luận, cùng nhiều năm chờ đợi, ngày nay chúng ta đã có một giải pháp đúng đắn cho “vấn đề callback”. Bước đầu tiên hướng tới trải nghiệm viết mã bất đồng bộ tốt hơn chính là promise, một đối tượng “mang theo” trạng thái và kết quả cuối cùng của một thao tác bất đồng bộ. Một promise có thể dễ dàng được nối chuỗi (chain) để hiện thực các luồng thực thi tuần tự và có thể được truyền đi như bất kỳ đối tượng nào khác. Promise đơn giản hóa mã bất đồng bộ đi rất nhiều; tuy nhiên, vẫn còn chỗ để cải thiện. Vì thế, trong nỗ lực làm cho luồng thực thi tuần tự phổ biến trở nên đơn giản nhất có thể, một cấu trúc mới đã được giới thiệu, gọi là `async/await`, thứ cuối cùng có thể khiến mã bất đồng bộ trông giống như mã đồng bộ.

In today’s modern Node.js programming, `async/await` is the preferred construct to use when dealing with asynchronous code. However, `async/await` is built on top of promises, as much as promises are built on top of callbacks. So, it’s important that we know and master all of them in order to tackle our asynchronous programming problems with the right approach.

Trong lập trình Node.js hiện đại ngày nay, `async/await` là cấu trúc được ưa dùng khi xử lý mã bất đồng bộ. Tuy nhiên, `async/await` được xây dựng trên nền tảng promise, cũng như promise được xây dựng trên nền tảng callback. Vì thế, điều quan trọng là chúng ta phải biết và nắm vững tất cả chúng để giải quyết các bài toán lập trình bất đồng bộ với cách tiếp cận đúng đắn.

In this chapter, you will learn the following:

Trong chương này, bạn sẽ học được những điều sau:

- How promises work and how to use them effectively to implement the main control flow constructs we already know about.
- The `async/await` syntax, which will become our main tool for dealing with asynchronous code in Node.js.
 - Cách promise hoạt động và cách dùng chúng một cách hiệu quả để hiện thực những cấu trúc luồng điều khiển chính mà chúng ta đã biết.
 - Cú pháp `async/await`, thứ sẽ trở thành công cụ chính của chúng ta khi xử lý mã bất đồng bộ trong Node.js.

By the end of the chapter, you will have learned about the two most important components that we have in JavaScript for taming asynchronous code. So, let’s get started by discovering promises.

Đến cuối chương, bạn sẽ đã học về hai thành phần quan trọng nhất mà chúng ta có trong JavaScript để thuần hóa mã bất đồng bộ. Vậy thì, hãy bắt đầu bằng việc khám phá promise.

Promises — Promise

Promises are part of the ECMAScript 2015 standard (or ES6, which is why they are also called ES6 promises) and have been natively available in Node.js since version 4. But the history of promises

goes back a few years earlier, when there were dozens of implementations around, initially with different features and behavior. Eventually, the majority of those implementations settled on a standard called Promises/A+.

Promise là một phần của chuẩn ECMAScript 2015 (hay ES6, đó là lý do chúng còn được gọi là ES6 promises) và đã có sẵn một cách gốc (native) trong Node.js kể từ phiên bản 4. Nhưng lịch sử của promise còn đi ngược lại vài năm trước nữa, khi có hàng chục bản hiện thực xung quanh, ban đầu với những tính năng và hành vi khác nhau. Cuối cùng, phần lớn các bản hiện thực đó đã thống nhất theo một chuẩn gọi là Promises/A+.

Promises represent a big step ahead toward providing a robust alternative to continuation-passing style callbacks for propagating an asynchronous result. As we will see, the use of promises will make all the major asynchronous control flow constructs easier to read, less verbose, and more robust compared to their callbackbased alternatives.

Promise đại diện cho một bước tiến lớn hướng tới việc cung cấp một giải pháp thay thế mạnh mẽ cho callback theo phong cách continuation-passing để truyền bá một kết quả bất đồng bộ. Như chúng ta sẽ thấy, việc dùng promise sẽ khiến tất cả những cấu trúc luồng điều khiển bất đồng bộ chính trở nên dễ đọc hơn, ít dài dòng hơn, và mạnh mẽ hơn so với những giải pháp thay thế dựa trên callback.

What is a promise? — Promise là gì?

A Promise is an object that embodies the eventual result (or error) of an asynchronous operation. In promises jargon, we say that a Promise is pending when the asynchronous operation is not yet complete, it's fulfilled when the operation successfully completes, and rejected when the operation terminates with an error. Once a Promise is either fulfilled or rejected, it's considered settled.

Một Promise là một đối tượng hiện thân cho kết quả cuối cùng (hoặc lỗi) của một thao tác bất đồng bộ. Theo thuật ngữ của promise, ta nói rằng một Promise đang ở trạng thái pending khi thao tác bất đồng bộ chưa hoàn tất, nó được fulfilled khi thao tác hoàn tất thành công, và rejected khi thao tác kết thúc với một lỗi. Một khi một Promise đã hoặc fulfilled hoặc rejected, nó được coi là settled.

To receive the fulfillment value or the error (reason) associated with the rejection, we can use the `then()` method of a Promise instance. The following is its signature:

Để nhận giá trị fulfillment hoặc lỗi (lý do) liên quan tới việc rejection, ta có thể dùng phương thức `then()` của một thực thể Promise. Sau đây là chữ ký của nó:

```
promise.then(onFulfilled, onRejected)
```

In the preceding signature, `onFulfilled` is a callback that will eventually receive the fulfillment value of the Promise, and `onRejected` is another callback that will receive the reason for the rejection (if any). Both are optional.

Trong chữ ký trên, `onFulfilled` là một callback rất cuộc sẽ nhận giá trị fulfillment của Promise, và `onRejected` là một callback khác sẽ nhận lý do của việc rejection (nếu có). Cả hai đều là tùy chọn.

To have an idea of how promises can transform our code, let's consider the following callback-based code:

Để hình dung về cách promise có thể biến đổi mã của chúng ta, hãy xét đoạn mã dựa trên callback sau đây:


```

asyncOperation(arg, (err, result) => {
  if(err) {
    // handle the error
  }
  // do stuff with the result
})

```

Promises allow us to transform this typical continuation-passing style code into a better structured and more elegant code, such as the following:

Promise cho phép chúng ta biến đổi đoạn mã theo phong cách continuation-passing điển hình này thành một đoạn mã có cấu trúc tốt hơn và thanh lịch hơn, như sau:

```

asyncOperationPromise(arg)
  .then(result => {
    // do stuff with result
  }, err => {
    // handle the error
  })

```

In the code above, `asyncOperationPromise()` is returning a Promise, which we can then use to receive the fulfillment value or the rejection reason of the eventual result of the function. So far, it seems that there is nothing major going on, but one crucial property of the `then()` method is that it synchronously returns another Promise. Moreover, if any of the `onFulfilled` or `onRejected` functions return a value `x`, the Promise returned by the `then()` method will:

Trong đoạn mã trên, `asyncOperationPromise()` đang trả về một Promise, mà sau đó ta có thể dùng để nhận giá trị fulfillment hoặc lý do rejection của kết quả cuối cùng của hàm. Cho tới đây, có vẻ chẳng có gì lớn lao đang diễn ra, nhưng một thuộc tính then chốt của phương thức `then()` là nó trả về một Promise khác một cách đồng bộ. Hơn nữa, nếu bất kỳ hàm `onFulfilled` hay `onRejected` nào trả về một giá trị `x`, thì Promise được trả về bởi phương thức `then()` sẽ:

- Fulfill with `x` if `x` is a value
- Fulfill with the fulfillment value of `x` if `x` is a Promise
- Reject with the eventual rejection reason of `x` if `x` is a Promise
 - Fulfill với `x` nếu `x` là một giá trị
 - Fulfill với giá trị fulfillment của `x` nếu `x` là một Promise
 - Reject với lý do rejection cuối cùng của `x` nếu `x` là một Promise

This behavior allows us to build chains of promises, allowing easy aggregation and arrangement of asynchronous operations into several configurations. Moreover, if we don't specify an `onFulfilled` or `onRejected` handler, the fulfillment value or rejection reason is automatically forwarded to the next promise in the chain. This allows us, for example, to automatically propagate errors across the whole chain until they are caught by an `onRejected` handler. With a Promise chain, the **sequential execution** of tasks suddenly becomes a trivial operation:

Hành vi này cho phép chúng ta xây dựng những chuỗi promise, cho phép dễ dàng tổng hợp và sắp xếp các thao tác bất đồng bộ thành nhiều cấu hình khác nhau. Hơn nữa, nếu ta không chỉ định một bộ xử lý `onFulfilled` hay `onRejected`, thì giá trị fulfillment hoặc lý do rejection sẽ tự động được chuyển tiếp tới promise tiếp theo trong chuỗi. Điều này cho phép chúng ta, chẳng hạn, tự động truyền bá lỗi xuyên suốt cả chuỗi cho tới khi chúng được bắt bởi một bộ xử lý `onRejected`. Với một chuỗi Promise, việc thực thi tuần tự các tác vụ bỗng nhiên trở thành một thao tác tầm thường:

```

asyncOperationPromise(arg)
  .then(result1 => {
    // returns another promise
    return asyncOperationPromise(arg2)
  })
  .then(result2 => {
    // returns a value
    return 'done'
  })
  .then(undefined, err => {
    // any error in the chain is caught here
  })

```

The following diagram provides another perspective on how a Promise chain works:

Sơ đồ sau đây cung cấp một góc nhìn khác về cách một chuỗi Promise hoạt động:

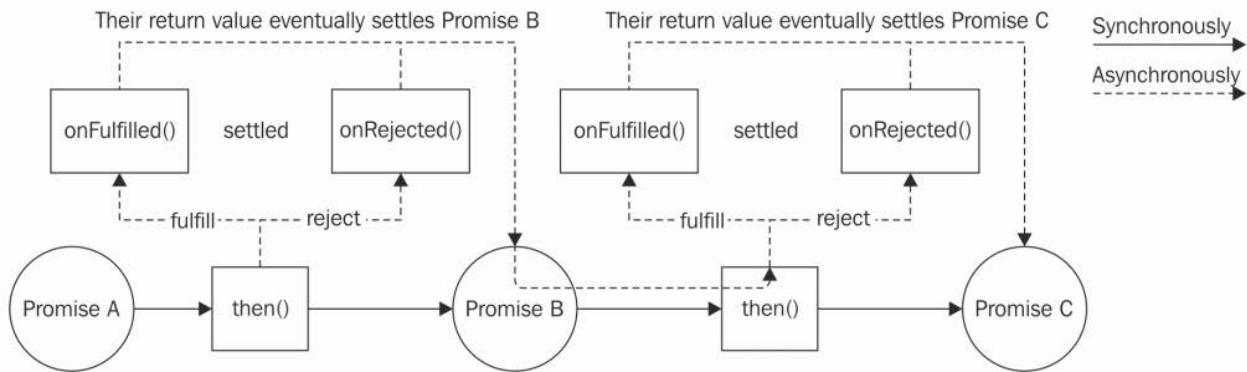


Figure 5.1: Promise chain execution flow Figure 5.1 shows how our program flows when we use a chain of promises. When we invoke `then()` on Promise A we synchronously receive Promise B as a result and when we invoke `then()` on Promise B we synchronously receive Promise C as a result. Eventually, when Promise A settles, it will either fulfill or reject, which results in the invocation of either the `onFulfilled()` or the `onRejected()` callback respectively. The result of the execution of such a callback will then fulfill or reject Promise B and such a result is, in turn, propagated to the `onFulfilled()` or the `onRejected()` callback passed to the `then()` invocation on Promise B. The execution continues similarly for Promise C and any other promise that follows in the chain.

Hình 5.1: Luồng thực thi của chuỗi promise. Hình 5.1 cho thấy chương trình của chúng ta chảy như thế nào khi ta dùng một chuỗi promise. Khi ta gọi `then()` trên Promise A, ta nhận được Promise B một cách đồng bộ như là kết quả, và khi ta gọi `then()` trên Promise B, ta nhận được Promise C một cách đồng bộ như là kết quả. Cuối cùng, khi Promise A settle, nó sẽ hoặc fulfill hoặc reject, dẫn tới việc gọi tương ứng callback `onFulfilled()` hoặc `onRejected()`. Kết quả của việc thực thi callback đó sau đó sẽ fulfill hoặc reject Promise B, và kết quả đó, đến lượt nó, được truyền bá tới callback `onFulfilled()` hoặc `onRejected()` được truyền vào lời gọi `then()` trên Promise B. Việc thực thi tiếp diễn tương tự cho Promise C và bất kỳ promise nào khác theo sau trong chuỗi.

An important property of promises is that the `onFulfilled()` and `onRejected()` callbacks are guaranteed to be invoked asynchronously and at most once, even if we resolve the Promise synchronously with a value. Not only that, the `onFulfilled()` and `onRejected()` callbacks will be invoked asynchronously even if the Promise object is already settled at the moment in which `then()` is called. This behavior shields our code against all those situations where we could

unintentionally release Zalgo (see Chapter 3, Callbacks and Events), making our asynchronous code more consistent and robust without any extra effort.

Một thuộc tính quan trọng của promise là các callback `onFulfilled()` và `onRejected()` được đảm bảo sẽ được gọi một cách bất đồng bộ và tối đa một lần, ngay cả khi ta resolve Promise một cách đồng bộ với một giá trị. Không chỉ vậy, các callback `onFulfilled()` và `onRejected()` sẽ được gọi một cách bất đồng bộ ngay cả khi đối tượng Promise đã settled tại thời điểm `then()` được gọi. Hành vi này che chắn mã của chúng ta khỏi tất cả những tình huống mà ta có thể vô tình giải phóng Zalgo (xem Chương 3, Callbacks and Events), khiến mã bất đồng bộ của chúng ta nhất quán và mạnh mẽ hơn mà không cần nỗ lực thêm gì.

Now comes the best part. If an exception is thrown (using the `throw` statement) in the `onFulfilled()` or `onRejected()` handler, the Promise returned by the `then()` method will automatically reject, with the exception that was thrown provided as the rejection reason. This is a tremendous advantage over CPS, as it means that with promises, exceptions will propagate automatically across the chain, and the `throw` statement becomes finally usable.

Bây giờ tới phần hay nhất. Nếu một ngoại lệ bị ném ra (dùng câu lệnh `throw`) trong bộ xử lý `onFulfilled()` hoặc `onRejected()`, thì Promise được trả về bởi phương thức `then()` sẽ tự động reject, với ngoại lệ đã bị ném ra được cung cấp làm lý do rejection. Đây là một lợi thế to lớn so với CPS, vì nó có nghĩa là với promise, ngoại lệ sẽ tự động truyền bá xuyên suốt chuỗi, và câu lệnh `throw` cuối cùng trở nên có thể dùng được.

Promises/A+ and thenables — Promises/A+ và thenable

Historically, there have been many different implementations of promises, and most of them were not compatible with each other, meaning that it was not possible to create chains between Promise objects coming from libraries that were using different Promise implementations.

Trong lịch sử, đã có nhiều bản hiện thực khác nhau của promise, và hầu hết chúng không tương thích với nhau, nghĩa là không thể tạo các chuỗi giữa những đối tượng Promise đến từ các thư viện sử dụng các bản hiện thực Promise khác nhau.

The JavaScript community worked very hard to address this limitation and those efforts led to the creation of the Promises/A+ specification. This specification details the behavior of the `then()` method, providing an interoperable base, which makes Promise objects from different libraries able to work with each other out of the box. Today, the majority of Promise implementations use this standard, including the native Promise object of JavaScript and Node.js. For a detailed overview of the Promises/A+ specification, you can refer to the official website at nodejsdp.link/promises-aplus.

Cộng đồng JavaScript đã làm việc rất chăm chỉ để giải quyết hạn chế này và những nỗ lực đó đã dẫn tới sự ra đời của đặc tả Promises/A+. Đặc tả này mô tả chi tiết hành vi của phương thức `then()`, cung cấp một nền tảng có khả năng tương tác, khiến những đối tượng Promise từ các thư viện khác nhau có thể làm việc với nhau ngay lập tức. Ngày nay, phần lớn các bản hiện thực Promise dùng chuẩn này, bao gồm cả đối tượng Promise gốc của JavaScript và Node.js. Để có cái nhìn tổng quan chi tiết về đặc tả Promises/A+, bạn có thể tham khảo trang web chính thức tại nodejsdp.link/promises-aplus.

As a result of the adoption of the Promises/A+ standard, many Promise implementations, including the native JavaScript Promise **API**, will consider any object with a `then()` method a Promise-like object, also called thenable. This behavior allows different Promise implementations to interact with each other seamlessly.

Do việc áp dụng chuẩn Promises/A+, nhiều bản hiện thực Promise, bao gồm cả API Promise gốc của JavaScript, sẽ coi bất kỳ đối tượng nào có một phương thức `then()` là một đối tượng kiểu Promise, còn gọi là *thenable*. Hành vi này cho phép các bản hiện thực Promise khác nhau tương tác với nhau một cách liền mạch.

The technique of recognizing (or typing) objects based on their external behavior, rather than their actual type, is called *duck typing* and is widely used in JavaScript.

Kỹ thuật nhận diện (hay định kiểu) các đối tượng dựa trên hành vi bên ngoài của chúng, thay vì kiểu thực sự của chúng, được gọi là *duck typing* và được sử dụng rộng rãi trong JavaScript.

The promise API — API của promise

Let's now take a quick look at the API of the native JavaScript Promise. This is just an overview to give you an idea of what we can do with promises, so don't worry if things are not so clear at this point yet; we will have the chance to use most of these APIs throughout the book.

Bây giờ hãy nhìn nhanh qua API của Promise gốc trong JavaScript. Đây chỉ là một cái nhìn tổng quan để cho bạn một ý niệm về những gì ta có thể làm với promise, nên đừng lo lắng nếu mọi thứ chưa rõ ràng lắm tại thời điểm này; chúng ta sẽ có dịp dùng hầu hết các API này xuyên suốt cuốn sách.

The Promise constructor (`new Promise((resolve, reject) => {})`) creates a new Promise instance that fulfills or rejects based on the behavior of the function provided as an argument. The function provided to the constructor will receive two arguments:

Hàm dựng Promise (`new Promise((resolve, reject) => {})`) tạo ra một thực thể Promise mới, thực thể này fulfill hoặc reject dựa trên hành vi của hàm được cung cấp làm đối số. Hàm được cung cấp cho hàm dựng sẽ nhận hai đối số:

- `resolve(obj)`: This is a function that, when invoked, will fulfill the Promise with the provided fulfillment value, which will be `obj` if `obj` is a value. It will be the fulfillment value of `obj` if `obj` is a Promise or a thenable.
- `reject(err)`: This rejects the Promise with the reason `err`. It is a convention for `err` to be an instance of `Error`.
 - `resolve(obj)`: Đây là một hàm mà, khi được gọi, sẽ fulfill Promise với giá trị fulfillment được cung cấp, giá trị này sẽ là `obj` nếu `obj` là một giá trị. Nó sẽ là giá trị fulfillment của `obj` nếu `obj` là một Promise hoặc một thenable.
 - `reject(err)`: Cái này reject Promise với lý do `err`. Theo quy ước, `err` nên là một thực thể của `Error`.

Now, let's take a look at the most important static methods of the Promise object:

Bây giờ, hãy xem qua những phương thức tĩnh quan trọng nhất của đối tượng Promise:

- `Promise.resolve(obj)`: This method creates a new Promise from another Promise, a thenable, or a value. If a Promise is passed, then that Promise is returned as it is. If a thenable is provided, then it's converted to the Promise implementation in use. If a value is provided, then the Promise will be fulfilled with that value.
- `Promise.reject(err)`: This method creates a Promise that rejects with `err` as the reason.
- `Promise.all(iterable)`: This method creates a Promise that fulfills with an array of fulfillment values when every item in the input `iterable` (such as an `Array`) object fulfills. If any Promise in the `iterable` object rejects, then the Promise returned by `Promise.all()` will

reject with the first rejection reason. Each item in the iterable object can be a Promise, a generic thenable, or a value.

- `Promise.allSettled(iterable)`: This method waits for all the input promises to fulfill or reject and then returns an array of objects containing the fulfillment value or the rejection reason for each input Promise. Each output object has a status property, which can be equal to 'fulfilled' or 'rejected', and a value property containing the fulfillment value, or a reason property containing the rejection reason. The difference with `Promise.all()` is that `Promise.allSettled()` will always wait for each Promise to either fulfill or reject, instead of immediately rejecting when one of the promises rejects.
- `Promise.race(iterable)`: This method returns a Promise that is equivalent to the first Promise in iterable that settles.
 - `Promise.resolve(obj)`: Phương thức này tạo một Promise mới từ một Promise khác, một thenable, hoặc một giá trị. Nếu một Promise được truyền vào, thì Promise đó được trả về nguyên trạng. Nếu một thenable được cung cấp, thì nó được chuyển đổi sang bản hiện thực Promise đang được dùng. Nếu một giá trị được cung cấp, thì Promise sẽ được fulfill với giá trị đó.
 - `Promise.reject(err)`: Phương thức này tạo một Promise reject với `err` làm lý do.
 - `Promise.all(iterable)`: Phương thức này tạo một Promise fulfill với một mảng các giá trị fulfillment khi mọi phần tử trong đối tượng iterable đầu vào (chẳng hạn một Array) fulfill. Nếu bất kỳ Promise nào trong đối tượng iterable reject, thì Promise được trả về bởi `Promise.all()` sẽ reject với lý do rejection đầu tiên. Mỗi phần tử trong đối tượng iterable có thể là một Promise, một thenable chung, hoặc một giá trị.
 - `Promise.allSettled(iterable)`: Phương thức này chờ tất cả các promise đầu vào fulfill hoặc reject rồi trả về một mảng các đối tượng chứa giá trị fulfillment hoặc lý do rejection cho mỗi Promise đầu vào. Mỗi đối tượng đầu ra có một thuộc tính status, có thể bằng 'fulfilled' hoặc 'rejected', và một thuộc tính value chứa giá trị fulfillment, hoặc một thuộc tính reason chứa lý do rejection. Điểm khác biệt so với `Promise.all()` là `Promise.allSettled()` sẽ luôn chờ mỗi Promise hoặc fulfill hoặc reject, thay vì reject ngay lập tức khi một trong các promise reject.
 - `Promise.race(iterable)`: Phương thức này trả về một Promise tương đương với Promise đầu tiên trong iterable settle.

Finally, the following are the main methods available on a Promise instance:

Cuối cùng, sau đây là những phương thức chính có sẵn trên một thực thể Promise:

- `promise.then(onFulfilled, onRejected)`: This is the essential method of a Promise. Its behavior is compatible with the Promises/A+ standard that we mentioned before.
- `promise.catch(onRejected)`: This method is just syntactic sugar ([nodejsdp. link/syntactic-sugar](https://nodejs.org/docs/latest-v10.x/api/util.html#utilpromisifyshorthand)) for `promise.then(undefined, onRejected)`.
- `promise.finally(onFinally)`: This method allows us to set up an `onFinally` callback, which is invoked when the Promise is settled (either fulfilled or rejected). Unlike `onFulfilled` and `onRejected`, the `onFinally` callback will not receive any argument as input and any value returned from it will be ignored. The Promise returned by `finally` will settle with the same fulfillment value or rejection reason of the current Promise instance. There is only once exception to all this, which is the case in which we throw inside the `onFinally` callback or return a rejected Promise. In this case, the returned Promise will reject with the error that is thrown or the rejection reason of the rejected Promise returned.
 - `promise.then(onFulfilled, onRejected)`: Đây là phương thức thiết yếu của một Promise. Hành vi của nó tương thích với chuẩn Promises/A+ mà ta đã đề cập trước

đó.

- `promise.catch(onRejected)`: Phương thức này chỉ là cú pháp đường (nodejsdp. link/syntactic-sugar) cho `promise.then(undefined, onRejected)`.
- `promise.finally(onFinally)`: Phương thức này cho phép chúng ta thiết lập một callback `onFinally`, được gọi khi Promise settle (hoặc fulfill hoặc reject). Khác với `onFulfilled` và `onRejected`, callback `onFinally` sẽ không nhận đối số nào làm đầu vào và bất kỳ giá trị nào nó trả về sẽ bị bỏ qua. Promise được trả về bởi `finally` sẽ settle với cùng giá trị fulfillment hoặc lý do rejection của thực thể Promise hiện tại. Chỉ có một ngoại lệ đối với tất cả điều này, đó là trường hợp ta throw bên trong callback `onFinally` hoặc trả về một Promise đã reject. Trong trường hợp này, Promise được trả về sẽ reject với lỗi bị ném ra hoặc lý do rejection của Promise đã reject được trả về.

Let's now see an example of how we can create a Promise from scratch using its constructor.

Bây giờ hãy xem một ví dụ về cách chúng ta có thể tạo một Promise từ đầu bằng hàm dựng của nó.

Creating a promise — Tạo một promise

Let's now see how we can create a Promise using its constructor. Creating a Promise from scratch is a low-level operation and it's usually required when we need to convert an API that uses another asynchronous style (such as a callback-based style). Most of the time we—as developers—are consumers of promises produced by other libraries and most of the promises we create will come from the `then()` method. Nonetheless, in some advanced scenarios, we need to manually create a Promise using its constructor.

Bây giờ hãy xem cách chúng ta có thể tạo một Promise bằng hàm dựng của nó. Tạo một Promise từ đầu là một thao tác cấp thấp và thường được yêu cầu khi ta cần chuyển đổi một API dùng phong cách bất đồng bộ khác (chẳng hạn phong cách dựa trên callback). Hầu hết thời gian, chúng ta—với tư cách lập trình viên—là những người tiêu thụ các promise được tạo ra bởi các thư viện khác, và hầu hết các promise mà ta tạo ra sẽ đến từ phương thức `then()`. Tuy nhiên, trong một số kịch bản nâng cao, ta cần tạo thủ công một Promise bằng hàm dựng của nó.

To demonstrate how to use the Promise constructor, let's create a function that returns a Promise that fulfills with the current date after a specified number of milliseconds. Let's take a look at it:

Để minh họa cách dùng hàm dựng Promise, hãy tạo một hàm trả về một Promise fulfill với ngày giờ hiện tại sau một số mili giây được chỉ định. Hãy cùng xem qua:

```
function delay (milliseconds) {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      resolve(new Date())  
    }, milliseconds)  
  })  
}
```

As you probably already guessed, we used `setTimeout()` to invoke the `resolve()` function of the Promise constructor. We can notice how the entire body of the function is wrapped by the Promise constructor; this is a frequent code pattern you will see when creating a Promise from scratch.

Như bạn có lẽ đã đoán ra, ta đã dùng `setTimeout()` để gọi hàm `resolve()` của hàm

dừng Promise. Ta có thể nhận thấy cách toàn bộ thân của hàm được bao bọc bởi hàm dừng Promise; đây là một mẫu mã thường gặp mà bạn sẽ thấy khi tạo một Promise từ đầu.

The `delay()` function we just created can then be used with some code like the following:

Hàm `delay()` mà chúng ta vừa tạo sau đó có thể được dùng với đoạn mã như sau:

```
console.log(`Delaying...${new Date().getSeconds()}s`)  
delay(1000)  
  .then(newDate => {  
    console.log(`Done ${newDate.getSeconds()}s`)  
  })
```

The `console.log()` within the `then()` handler will be executed approximately after 1 second from the invocation of `delay()`. The Promises/A+ specification states that the `onFulfilled` and `onRejected` callbacks of the `then()` method have to be invoked only once and exclusively (only one or the other is invoked). A compliant promises implementation makes sure that even if we call resolve or reject multiple times, the Promise is either fulfilled or rejected only once.

`console.log()` bên trong bộ xử lý `then()` sẽ được thực thi vào khoảng 1 giây sau lời gọi `delay()`. Đặc tả Promises/A+ nêu rằng các callback `onFulfilled` và `onRejected` của phương thức `then()` chỉ được gọi đúng một lần và loại trừ lẫn nhau (chỉ một trong hai được gọi). Một bản hiện thực promise tuân thủ đảm bảo rằng ngay cả khi ta gọi `resolve` hay `reject` nhiều lần, Promise chỉ hoặc fulfill hoặc reject đúng một lần.

Promisification — Promisification

When some characteristics of a callback-based function are known in advance, it's possible to create a function that transforms such a callback-based function into an equivalent function returning a Promise. This transformation is called **promisification**.

Khi một số đặc điểm của một hàm dựa trên callback được biết trước, ta có thể tạo một hàm biến đổi hàm dựa trên callback đó thành một hàm tương đương trả về một Promise. Phép biến đổi này được gọi là **promisification**.

For example, let's consider the conventions used in Node.js-style callback-based functions:

Chẳng hạn, hãy xét những quy ước được dùng trong các hàm dựa trên callback theo phong cách Node.js:

- The callback is the last argument of the function
- The error (if any) is always the first argument passed to the callback
- Any return value is passed after the error to the callback
 - Callback là đối số cuối cùng của hàm
 - Lỗi (nếu có) luôn là đối số đầu tiên được truyền cho callback
 - Bất kỳ giá trị trả về nào cũng được truyền sau lỗi cho callback

Based on these rules, we can easily create a generic function that promisifies a Node.js-style callback-based function. Let's see what this function looks like:

Dựa trên những quy tắc này, ta có thể dễ dàng tạo một hàm tổng quát **promisify** một hàm dựa trên callback theo phong cách Node.js. Hãy xem hàm này trông như thế nào:

```
function promisify (callbackBasedApi) {  
  return function promisified (...args) {
```

```

return new Promise((resolve, reject) => { // (1)
  const newArgs = [
    ...args,
    function (err, result) { // (2)
      if (err) {
        return reject(err)
      }
      resolve(result)
    }
  ]
  callbackBasedApi(...newArgs) // (3)
})
}

```

The preceding function returns another function called `promisified()`, which represents the promisified version of the `callbackBasedApi` given as the input. This is how it works:

Hàm trên trả về một hàm khác gọi là `promisified()`, đại diện cho phiên bản đã promisify của `callbackBasedApi` được cho làm đầu vào. Đây là cách nó hoạt động:

1. The `promisified()` function creates a new `Promise` using the `Promise` constructor and immediately returns it to the caller.
2. In the function passed to the `Promise` constructor, we make sure to pass to `callbackBasedApi` a special callback. Since we know that the callback always comes last, we simply append it to the arguments list (`args`) provided to the `promisified()` function. In the special callback, if we receive an error, we immediately reject the `Promise`; otherwise, we resolve it with the given result.
3. Finally, we simply invoke `callbackBasedApi` with the list of arguments we have built.
 1. Hàm `promisified()` tạo một `Promise` mới bằng hàm dựng `Promise` và trả nó về ngay lập tức cho bên gọi.
 2. Trong hàm được truyền cho hàm dựng `Promise`, ta đảm bảo truyền cho `callbackBasedApi` một callback đặc biệt. Vì ta biết callback luôn đến cuối cùng, ta chỉ đơn giản nối nó vào danh sách đối số (`args`) được cung cấp cho hàm `promisified()`. Trong callback đặc biệt, nếu ta nhận một lỗi, ta reject `Promise` ngay lập tức; nếu không, ta resolve nó với result đã cho.
 3. Cuối cùng, ta chỉ đơn giản gọi `callbackBasedApi` với danh sách đối số mà ta đã xây dựng.

Now, let's promisify a Node.js function using our newly created `promisify()` function. We can use the `randomBytes()` function of the core `crypto` **module**, which produces a **buffer** containing the specified number of random bytes. The `randomBytes()` function accepts a callback as the last argument and it follows the conventions we already know very well. Let's see what this looks like:

Bây giờ, hãy promisify một hàm Node.js bằng hàm `promisify()` mới tạo của chúng ta. Ta có thể dùng hàm `randomBytes()` của module lõi `crypto`, hàm này tạo ra một buffer chứa số byte ngẫu nhiên được chỉ định. Hàm `randomBytes()` nhận một callback làm đối số cuối cùng và nó tuân theo những quy ước mà ta đã biết rất rõ. Hãy xem nó trông như thế nào:

```

import { randomBytes } from 'crypto'
const randomBytesP = promisify(randomBytes)
randomBytesP(32)

```

```
.then(buffer => {
  console.log(`Random bytes: ${buffer.toString()}`)
})
```

The previous code should print some gibberish to the console; that's because not all generated bytes have a corresponding printable character.

Đoạn mã trên sẽ in ra console một thứ vô nghĩa nào đó; đó là vì không phải tất cả byte được tạo ra đều có một ký tự in được tương ứng.

The promisification function we created here is just for educational purposes and it's missing a few features, such as the ability to deal with callbacks returning more than one result. In real life, we would use the `promisify()` function of the `util` core module to promisify our Node.js-style callback-based functions. You can take a look at its documentation at nodejsdp.link/promisify.

Hàm promisification mà ta tạo ở đây chỉ phục vụ mục đích giáo dục và nó thiếu một vài tính năng, chẳng hạn khả năng xử lý các callback trả về nhiều hơn một kết quả. Trong thực tế, ta sẽ dùng hàm `promisify()` của module lõi `util` để promisify các hàm dựa trên callback theo phong cách Node.js của mình. Bạn có thể xem qua tài liệu của nó tại nodejsdp.link/promisify.

Sequential execution and iteration — Thực thi tuần tự và lặp

We now know enough to convert the web spider application that we created in the previous chapter to use promises. Let's start directly from version 2, the one downloading the links of a webpage in sequence.

Giờ ta đã biết đủ để chuyển đổi ứng dụng web spider mà ta đã tạo ở chương trước sang dùng promise. Hãy bắt đầu trực tiếp từ phiên bản 2, phiên bản tải các liên kết của một trang web theo tuần tự.

We can access an already promisifyed version of the core `fs` API through the `promises` object of the `fs` module. For example: `import { promises } from 'fs'`.

Ta có thể truy cập một phiên bản đã được promisify của API lõi `fs` thông qua đối tượng `promises` của module `fs`. Chẳng hạn: `import { promises } from 'fs'`.

In the `spider.js` module, the very first step required is to import our dependencies and promisify any callback-based function that we are going to use:

Trong module `spider.js`, bước đầu tiên cần làm là import các phụ thuộc (dependency) của chúng ta và promisify bất kỳ hàm dựa trên callback nào mà ta sẽ dùng:

```
import { promises as fsPromises } from 'fs' // (1)
import { dirname } from 'path'
import superagent from 'superagent'
import mkdirp from 'mkdirp'
import { urlToFilename, getPageLinks } from './utils.js'
import { promisify } from 'util'
const mkdirpPromises = promisify(mkdirp) // (2)
```

There are two main differences here compared to the `spider.js` module of the previous chapter:

Có hai khác biệt chính ở đây so với module `spider.js` của chương trước:

1. We import the `promises` object of the `fs` module to get access to all the `fs` functions already promisifyed.

2. We manually promisify the `mkdirp()` function.

1. Ta import đối tượng promises của module `fs` để truy cập tất cả các hàm `fs` đã được promisify.
2. Ta promisify thủ công hàm `mkdirp()`.

Now, we can start converting the `download()` function:

Bây giờ, ta có thể bắt đầu chuyển đổi hàm `download()`:

```
function download (url, filename) {
  console.log(`Downloading ${url}`)
  let content
  return superagent.get(url) // (1)
    .then((res) => {
      content = res.text // (2)
      return mkdirpPromises(dirname(filename))
    })
    .then(() => fsPromises.writeFile(filename, content))
    .then(() => {
      console.log(`Downloaded and saved: ${url}`)
      return content // (3)
    })
}
```

We can straightaway appreciate the elegance of implementing sequential asynchronous operations with promises. We simply have a clean and very intuitive chain of `then()` invocations.

Ta có thể ngay lập tức cảm nhận được sự thanh lịch của việc hiện thực các thao tác bất đồng bộ tuần tự bằng promise. Ta chỉ đơn giản có một chuỗi các lời gọi `then()` sạch sẽ và rất trực quan.

Compared to the previous version of the function, this time we are leveraging the out-of-the-box support for promises of the superagent **package**. Instead of invoking `end()` on the **request** object returned by `superagent.get()`, we simply invoke `then()` to send the request (1) and receive a Promise that fulfills/rejects with the result of the request.

So với phiên bản trước của hàm, lần này ta đang tận dụng sự hỗ trợ promise sẵn có của gói `superagent`. Thay vì gọi `end()` trên đối tượng `request` được trả về bởi `superagent.get()`, ta chỉ đơn giản gọi `then()` để gửi request (1) và nhận một Promise fulfill/reject với kết quả của request.

The final return value of the `download()` function is the Promise returned by the last `then()` call in the chain, which fulfills with the content of the webpage (3), which we initialized in the `onFulfilled` handler of the first `then()` call (2). This makes sure that the caller receives a Promise that fulfills with content only after all operations (`get`, `mkdirp`, and `writeFile`) have completed.

Giá trị trả về cuối cùng của hàm `download()` là Promise được trả về bởi lời gọi `then()` cuối cùng trong chuỗi, Promise này fulfill với content của trang web (3), thứ mà ta đã khởi tạo trong bộ xử lý `onFulfilled` của lời gọi `then()` đầu tiên (2). Điều này đảm bảo rằng bên gọi nhận được một Promise chỉ fulfill với content sau khi tất cả các thao tác (`get`, `mkdirp`, và `writeFile`) đã hoàn tất.

In the `download()` function that we've just seen, we have executed a known set of asynchronous operations in sequence. However, in the `spiderLinks()` function, we will have to deal with a sequential iteration over a dynamic set of asynchronous tasks. Let's see how we can achieve that:

Trong hàm `download()` mà ta vừa xem, ta đã thực thi một tập hợp các thao tác bất đồng bộ đã biết theo tuần tự. Tuy nhiên, trong hàm `spiderLinks()`, ta sẽ phải xử lý một phép lặp tuần tự trên một tập hợp động các tác vụ bất đồng bộ. Hãy xem cách ta có thể đạt được điều đó:

```
function spiderlinks (currentUrl, content, nesting) {
  let promise = Promise.resolve() // (1)
  if (nesting === 0) {
    return promise
  }
  const links = getPageLinks(currentUrl, content)
  for (const link of links) {
    promise = promise.then(() => spider(link, nesting - 1)) // (2)
  }
  return promise
}
```

To iterate over all the links of a webpage asynchronously, we had to dynamically build a chain of promises as follows:

Để lặp qua tất cả các liên kết của một trang web một cách bất đồng bộ, ta đã phải xây dựng động một chuỗi promise như sau:

1. First, we defined an “empty” Promise, which resolves to undefined. This Promise is used just as the starting point for our chain.
2. Then, in a loop, we update the promise variable with a new Promise obtained by invoking `then()` on the previous promise in the chain. This is actually our asynchronous iteration pattern using promises.
 1. Đầu tiên, ta định nghĩa một Promise “rỗng”, resolve về undefined. Promise này chỉ được dùng làm điểm khởi đầu cho chuỗi của chúng ta.
 2. Sau đó, trong một vòng lặp, ta cập nhật biến promise với một Promise mới thu được bằng cách gọi `then()` trên promise trước đó trong chuỗi. Đây thực ra chính là mẫu lặp bất đồng bộ của chúng ta dùng promise.

At the end of the for loop, the promise variable will contain the promise of the last `then()` invocation, so it will resolve only when all the promises in the chain have been resolved.

Ở cuối vòng lặp for, biến promise sẽ chứa promise của lời gọi `then()` cuối cùng, nên nó sẽ chỉ resolve khi tất cả các promise trong chuỗi đã được resolve.

Pattern (sequential iteration with promises)

Mẫu (lặp tuần tự với promise)

Dynamically build a chain of promises using a loop.

Xây dựng động một chuỗi promise bằng một vòng lặp.

Now, we can finally convert the `spider()` function:

Bây giờ, cuối cùng ta có thể chuyển đổi hàm `spider()`:

```
export function spider (url, nesting) {
  const filename = urlToFilename(url)
  return fsPromises.readFile(filename, 'utf8')
    .catch((err) => {
      if (err.code !== 'ENOENT') {
```

```

        throw err
      }
      // The file doesn't exist, so let's download it
      return download(url, filename)
    })
    .then(content => spiderLinks(url, content, nesting))
  }
}

```

In this new `spider()` function, we are using `catch()` to handle any error produced by `readFile()`. In particular, if the error has code `'ENOENT'`, it means that the file doesn't exist yet and therefore we need to download the corresponding URL. The Promise returned from `download()`, if fulfilled, will return the content at the URL. On the other hand, if the Promise produced by `readFile()` fulfills, it will skip the `catch()` handler and go straight to the next `then()`. In both cases, the `onFulfilled` handler of the last `then()` call will always receive the content of the webpage, either coming from the local file or from a fresh download.

Trong hàm `spider()` mới này, ta đang dùng `catch()` để xử lý bất kỳ lỗi nào do `readFile()` tạo ra. Cụ thể, nếu lỗi có mã `'ENOENT'`, điều đó có nghĩa là tập tin chưa tồn tại và do đó ta cần tải URL tương ứng. Promise được trả về từ `download()`, nếu fulfill, sẽ trả về content tại URL. Mặt khác, nếu Promise được tạo ra bởi `readFile()` fulfill, nó sẽ bỏ qua bộ xử lý `catch()` và đi thẳng tới `then()` tiếp theo. Trong cả hai trường hợp, bộ xử lý `onFulfilled` của lời gọi `then()` cuối cùng sẽ luôn nhận content của trang web, dù đến từ tập tin cục bộ hay từ một lần tải mới.

Now that we have converted our `spider()` function as well, we can finally modify the `spider-cli.js` module:

Giờ khi ta cũng đã chuyển đổi hàm `spider()`, cuối cùng ta có thể chỉnh sửa module `spider-cli.js`:

```

spider(url, nesting)
  .then(() => console.log('Download complete'))
  .catch(err => console.error(err))

```

The `catch()` handler here will intercept any error originating from the entire `spider()` process.

Bộ xử lý `catch()` ở đây sẽ chặn bắt bất kỳ lỗi nào bắt nguồn từ toàn bộ tiến trình `spider()`.

If we look again at all the code we have written so far, we will be pleasantly surprised by the fact that we haven't included any error propagation logic (as we would be forced to do when using callbacks). This is clearly an enormous advantage, as it greatly reduces the boilerplate in our code and the chances of missing any asynchronous errors.

Nếu ta nhìn lại tất cả đoạn mã đã viết cho tới giờ, ta sẽ ngạc nhiên một cách thú vị bởi việc ta đã không hề thêm bất kỳ logic truyền bá lỗi nào (như ta buộc phải làm khi dùng callback). Đây rõ ràng là một lợi thế to lớn, vì nó giảm đáng kể đoạn mã khuôn mẫu (boilerplate) trong mã của chúng ta và khả năng bỏ sót bất kỳ lỗi bất đồng bộ nào.

This completes the implementation of version 2 of our web spider application with promises.

Điều này hoàn tất việc hiện thực phiên bản 2 của ứng dụng web spider với promise.

An alternative of the sequential iteration pattern with promises makes use of the `reduce()` function, for an even more compact implementation:

Một giải pháp thay thế cho mẫu lặp tuần tự với promise sử dụng hàm `reduce()`, để có một bản hiện thực thậm chí còn gọn hơn:


```
const promise = tasks.reduce((prev, task) => {
  return prev.then(() => {
    return task()
  })
}, Promise.resolve())
```

Parallel execution — Thực thi song song

Another execution flow that becomes trivial with promises is the **parallel execution** flow. In fact, all that we need to do is use the built-in `Promise.all()` method. This helper function creates another Promise that fulfills only when all the promises received as input are fulfilled. If there is no causal relationship between those promises (for example, they are not part of the same chain of promises), then they will be executed in parallel.

Một luồng thực thi khác trở nên tầm thường với promise là luồng thực thi song song. Thực vậy, tất cả những gì ta cần làm là dùng phương thức tích hợp sẵn `Promise.all()`. Hàm trợ giúp này tạo một Promise khác chỉ fulfill khi tất cả các promise nhận vào làm đầu vào đều được fulfill. Nếu không có mối quan hệ nhân quả nào giữa những promise đó (chẳng hạn, chúng không thuộc cùng một chuỗi promise), thì chúng sẽ được thực thi song song.

To demonstrate this, let's consider version 3 of our web spider application, which downloads all the links of a page in parallel. Let's just update the `spiderLinks()` function again to implement a parallel execution flow using promises:

Để minh họa điều này, hãy xét phiên bản 3 của ứng dụng web spider của chúng ta, phiên bản tải tất cả các liên kết của một trang song song. Hãy chỉ cập nhật lại hàm `spiderLinks()` để hiện thực một luồng thực thi song song bằng promise:

```
function spiderLinks (currentUrl, content, nesting) {
  if (nesting === 0) {
    return Promise.resolve()
  }
  const links = getPagelinks(currentUrl, content)
  const promises = links.map(link => spider(link, nesting - 1))
  return Promise.all(promises)
}
```

The pattern here consists in starting the `spider()` tasks all at once in the `links.map()` loop. At the same time, each Promise returned by invoking `spider()` is collected in the final `promises` array. The critical difference in this loop—as compared to the sequential iteration loop—is that we are not waiting for the previous `spider()` task in the list to complete before starting a new one. All the `spider()` tasks are started in the loop at once, in the same **event loop** cycle.

Mẫu ở đây bao gồm việc khởi động tất cả các tác vụ `spider()` cùng một lúc trong vòng lặp `links.map()`. Đồng thời, mỗi Promise được trả về bởi việc gọi `spider()` được thu thập vào mảng `promises` cuối cùng. Điểm khác biệt then chốt trong vòng lặp này—so với vòng lặp tuần tự—là ta không chờ tác vụ `spider()` trước đó trong danh sách hoàn tất trước khi khởi động một tác vụ mới. Tất cả các tác vụ `spider()` được khởi động trong vòng lặp cùng một lúc, trong cùng một chu kỳ của vòng lặp sự kiện (event loop).

Once we have all the promises, we pass them to the `Promise.all()` method, which returns a new Promise that will be fulfilled when all the promises in the array are fulfilled. In other words, it fulfills when all the download tasks have completed. In addition to that, the Promise returned by

`Promise.all()` will reject immediately if any of the promises in the input array reject. This is exactly what we wanted for this version of our web spider.

Một khi ta có tất cả các promise, ta truyền chúng cho phương thức `Promise.all()`, phương thức này trả về một Promise mới sẽ được fulfill khi tất cả các promise trong mảng được fulfill. Nói cách khác, nó fulfill khi tất cả các tác vụ tải đã hoàn tất. Thêm vào đó, Promise được trả về bởi `Promise.all()` sẽ reject ngay lập tức nếu bất kỳ promise nào trong mảng đầu vào reject. Đây chính xác là điều ta muốn cho phiên bản này của web spider.

Limited parallel execution — Thực thi song song có giới hạn

So far, promises have not disappointed our expectations. We were able to greatly improve our code for both serial and parallel execution. Now, with limited parallel execution, things should not be that different, considering that this flow is just a combination of serial and parallel execution.

Cho tới giờ, promise đã không làm chúng ta thất vọng. Ta đã có thể cải thiện đáng kể mã của mình cho cả thực thi tuần tự lẫn song song. Bây giờ, với thực thi song song có giới hạn, mọi thứ cũng không nên khác biệt nhiều lắm, xét đến việc luồng này chỉ là sự kết hợp của thực thi tuần tự và song song.

In this section, we will go straight to implementing a solution that allows us to globally limit the **concurrency** of our web spider tasks. In other words, we are going to implement our solution in a class that we can use to instantiate objects that we can pass around to different functions of the same application. If you are just interested in a simple solution to locally limit the parallel execution of a set of tasks, you can still apply the same principles that we will see in this section to implement a special asynchronous version of `Array.map()`. We leave this to you as an exercise; you can find more details and hints at the end of this chapter.

Trong phần này, ta sẽ đi thẳng vào việc hiện thực một giải pháp cho phép chúng ta giới hạn một cách toàn cục tính đồng thời (concurrency) của các tác vụ web spider. Nói cách khác, ta sẽ hiện thực giải pháp của mình trong một lớp mà ta có thể dùng để khởi tạo các đối tượng có thể truyền đi tới các hàm khác nhau của cùng một ứng dụng. Nếu bạn chỉ quan tâm tới một giải pháp đơn giản để giới hạn cục bộ việc thực thi song song của một tập hợp các tác vụ, bạn vẫn có thể áp dụng những nguyên tắc tương tự mà ta sẽ thấy trong phần này để hiện thực một phiên bản bất đồng bộ đặc biệt của `Array.map()`. Ta để điều này lại cho bạn như một bài tập; bạn có thể tìm thêm chi tiết và gợi ý ở cuối chương này.

For a ready-to-use, production-ready implementation of a `map()` function supporting promises and limited concurrency, you can rely on the `p-map` package. Find out more at nodejsdp.link/p-map.

Để có một bản hiện thực sẵn sàng dùng ngay, sẵn sàng cho production của hàm `map()` hỗ trợ promise và tính đồng thời có giới hạn, bạn có thể dựa vào gói `p-map`. Tìm hiểu thêm tại nodejsdp.link/p-map.

Implementing the TaskQueue class with promises — Hiện thực lớp TaskQueue với promise

To globally limit the concurrency of our spider download tasks, we are going to reuse the `TaskQueue` class we implemented in the previous chapter. Let's start with the `next()` method, where we trigger the execution of a set of tasks until we reach the concurrency limit:

Để giới hạn một cách toàn cục tính đồng thời của các tác vụ tải của spider, ta sẽ tái sử dụng lớp `TaskQueue` mà ta đã hiện thực ở chương trước. Hãy bắt đầu với phương thức `next()`,

nơi ta kích hoạt việc thực thi một tập hợp các tác vụ cho tới khi chạm tới giới hạn đồng thời:

```
next () {
  while (this.running < this.concurrency && this.queue.length) {
    const task = this.queue.shift()
    task().finally(() => {
      this.running--
      this.next()
    })
    this.running++
  }
}
```

The core change in the `next()` method is where we invoke `task()`. In fact, now we expect that `task()` returns a Promise, so all we have to do is invoke `finally()` on that Promise so we can reset the count of running tasks if it either fulfills or rejects.

Thay đổi cốt lõi trong phương thức `next()` là chỗ ta gọi `task()`. Thực vậy, giờ ta kỳ vọng rằng `task()` trả về một Promise, nên tất cả những gì ta phải làm là gọi `finally()` trên Promise đó để có thể đặt lại bộ đếm các tác vụ đang chạy nếu nó hoặc fulfill hoặc reject.

Now, we implement a new method called `runTask()`. This method is responsible for queueing a special wrapper function and also for returning a newly built Promise. Such a Promise will essentially forward the result (fulfillment or rejection) of the Promise eventually returned by `task()`. Let's see what this method looks like:

Bây giờ, ta hiện thực một phương thức mới gọi là `runTask()`. Phương thức này chịu trách nhiệm đưa vào hàng đợi một hàm bao bọc (wrapper) đặc biệt và cũng trả về một Promise mới được xây dựng. Promise đó về cơ bản sẽ chuyển tiếp kết quả (fulfillment hoặc rejection) của Promise cuối cùng được trả về bởi `task()`. Hãy xem phương thức này trông như thế nào:

```
runTask (task) {
  return new Promise((resolve, reject) => { // (1)
    this.queue.push(() => { // (2)
      return task().then(resolve, reject) // (4)
    })
    process.nextTick(this.next.bind(this)) // (3)
  })
}
```

In the method we have just seen:

Trong phương thức ta vừa xem:

1. We create a new Promise using its constructor.
2. We add a special wrapper function to the tasks queue. This function is going to be executed at a later `next()` run, when there are enough concurrency slots left.
3. We invoke `next()` to trigger a new set of tasks to be run. We defer this to a subsequent run of the event loop to guarantee that `task` is always invoked asynchronously with respect to when `runTask()` is invoked. This prevents the problems we described in Chapter 3, Callbacks and Events (for example, Zalgo). In fact, we can notice that in the `next()` method there is another invocation of `next()` itself, in the `finally()` handler, that is always asynchronous.

4. When the wrapper function we queued is finally run, we execute the task we have received as the input, and we forward its results—fulfillment value or rejection reason—to the outer Promise, the one we return from the `runTask()` method.

1. Ta tạo một Promise mới bằng hàm dựng của nó.
2. Ta thêm một hàm bao bọc đặc biệt vào hàng đợi tác vụ. Hàm này sẽ được thực thi tại một lần chạy `next()` sau đó, khi còn đủ chỗ trống đồng thời.
3. Ta gọi `next()` để kích hoạt một tập hợp tác vụ mới được chạy. Ta trì hoãn điều này tới một lần chạy tiếp theo của vòng lặp sự kiện để đảm bảo rằng task luôn được gọi một cách bất đồng bộ so với thời điểm `runTask()` được gọi. Điều này ngăn ngừa những vấn đề mà ta đã mô tả ở Chương 3, Callbacks and Events (chẳng hạn Zalgo). Thực vậy, ta có thể nhận thấy rằng trong phương thức `next()` có một lời gọi `next()` khác của chính nó, trong bộ xử lý `finally()`, lời gọi này luôn là bất đồng bộ.
4. Khi hàm bao bọc mà ta đưa vào hàng đợi cuối cùng được chạy, ta thực thi task mà ta đã nhận làm đầu vào, và ta chuyển tiếp kết quả của nó—giá trị fulfillment hoặc lý do rejection—tới Promise bên ngoài, Promise mà ta trả về từ phương thức `runTask()`.

With this, we have completed the implementation of our new TaskQueue class using promises. Next, we'll use this new version of the TaskQueue class to implement version 4 of our web spider.

Với điều này, ta đã hoàn tất việc hiện thực lớp TaskQueue mới bằng promise. Tiếp theo, ta sẽ dùng phiên bản mới này của lớp TaskQueue để hiện thực phiên bản 4 của web spider.

Updating the web spider — Cập nhật web spider

Now it's time to adapt our web spider to implement a limited parallel execution flow using the TaskQueue class we have just created.

Bây giờ là lúc điều chỉnh web spider của chúng ta để hiện thực một luồng thực thi song song có giới hạn bằng lớp TaskQueue mà ta vừa tạo.

First, we need to split the `spider()` function into two functions, one simply initializing a new TaskQueue object and another actually executing the spidering task, which we will call `spiderTask()`. Then, we need to update the `spiderLinks()` function to invoke the newly created `spiderTask()` function and forward the task queue instance received as an input. Let's see what all this looks like:

Đầu tiên, ta cần tách hàm `spider()` thành hai hàm, một hàm chỉ khởi tạo một đối tượng TaskQueue mới và một hàm khác thực sự thực thi tác vụ spider, mà ta sẽ gọi là `spiderTask()`. Sau đó, ta cần cập nhật hàm `spiderLinks()` để gọi hàm `spiderTask()` mới tạo và chuyển tiếp thực thể queue của tác vụ nhận được làm đầu vào. Hãy xem tất cả những điều này trông như thế nào:

```
function spiderLinks (currentUrl, content, nesting, queue) {
  if (nesting === 0) {
    return Promise.resolve()
  }
  const links = getPageLinks(currentUrl, content)
  const promises = links
    .map(link => spiderTask(link, nesting - 1, queue))
  return Promise.all(promises) // (2)
}
const spidering = new Set()
function spiderTask (url, nesting, queue) {
```

```

if (spidering.has(url)) {
  return Promise.resolve()
}
spidering.add(url)
const filename = urlToFilename(url)
return queue
  .runTask(() => {                                     // (1)
    return fsPromises.readFile(filename, 'utf8')
  })
  .catch((err) => {
    if (err.code !== 'ENOENT') {
      throw err
    }
    // The file doesn't exist, so let's download it
    return download(url, filename)
  })
  .then(content => spiderLinks(url, content, nesting, queue))
}
export function spider (url, nesting, concurrency) {
  const queue = new TaskQueue(concurrency)
  return spiderTask(url, nesting, queue)
}

```

The crucial instruction in the code we have just seen is where we invoke `queue.runTask()` (1). Here, the task that we are queuing (and therefore limiting) comprises just the retrieval of the contents of the URL from either the local filesystem or the remote URL location. Only after this task has been run by the queue can we continue to spider the links of the webpage. Note that we are intentionally keeping `spiderLinks()` outside of the task that we want to limit. This is because `spiderLinks()` can trigger more `spiderTasks()` and that would create a deadlock if the depth of the spidering process is higher than the concurrency limit of the queue.

Câu lệnh `then` chốt trong đoạn mã ta vừa xem là chỗ ta gọi `queue.runTask()` (1). Ở đây, tác vụ mà ta đang đưa vào hàng đợi (và do đó giới hạn) chỉ bao gồm việc lấy nội dung của URL từ hoặc hệ thống tập tin cục bộ hoặc vị trí URL từ xa. Chỉ sau khi tác vụ này đã được chạy bởi hàng đợi thì ta mới có thể tiếp tục spider các liên kết của trang web. Lưu ý rằng ta cố ý giữ `spiderLinks()` ở ngoài tác vụ mà ta muốn giới hạn. Đó là vì `spiderLinks()` có thể kích hoạt thêm nhiều `spiderTasks()` và điều đó sẽ tạo ra một deadlock nếu độ sâu của tiến trình spider cao hơn giới hạn đồng thời của hàng đợi.

We can also notice how in `spiderLinks()` we simply continue to use `Promise.all()` (2) to download all the links of a webpage in parallel. This is because it's the responsibility of our queue to limit the concurrency of the tasks.

Ta cũng có thể nhận thấy cách trong `spiderLinks()` ta chỉ đơn giản tiếp tục dùng `Promise.all()` (2) để tải tất cả các liên kết của một trang web song song. Đó là vì việc giới hạn tính đồng thời của các tác vụ là trách nhiệm của queue của chúng ta.

In production code, you can use the package `p-limit` (available at nodejssdp.link/p-limit) to limit the concurrency of a set of tasks. The package essentially implements the pattern we have just shown but wrapped in a slightly different API. This concludes our exploration of JavaScript promises. Next, we are going to learn about the `async/await` pair, which will completely revolutionize the way we deal with asynchronous code.

Trong mã production, bạn có thể dùng gói `p-limit` (có sẵn tại nodejsdp.link/p-limit) để giới hạn tính đồng thời của một tập hợp các tác vụ. Gói này về cơ bản hiện thực mẫu mà ta vừa trình bày nhưng được gói trong một API hơi khác. Điều này kết thúc việc khám phá promise của JavaScript. Tiếp theo, ta sẽ tìm hiểu về cặp `async/await`, thứ sẽ hoàn toàn cách mạng hóa cách ta xử lý mã bất đồng bộ.

Async/await — Async/await

As we have just seen, promises are a quantum leap ahead of callbacks. They allow us to write clean and **readable** asynchronous code and provide a set of safeguards that can only be achieved with boilerplate code when working with callback-based asynchronous code. However, promises are still suboptimal when it comes to writing sequential asynchronous code. The Promise chain is indeed much better than having callback hell, but still, we have to invoke a `then()` and create a new function for each task in the chain. This is still too much for a control flow that is definitely the most commonly used in everyday programming. JavaScript needed a proper way to deal with the ubiquitous asynchronous sequential execution flow, and the answer arrived with the introduction in the ECMAScript standard of `async` functions and the `await` expression (`async/await` for short).

Như ta vừa thấy, promise là một bước nhảy vọt so với callback. Chúng cho phép ta viết mã bất đồng bộ sạch sẽ và dễ đọc, đồng thời cung cấp một tập hợp các biện pháp bảo vệ mà chỉ có thể đạt được bằng mã khuôn mẫu khi làm việc với mã bất đồng bộ dựa trên callback. Tuy nhiên, promise vẫn còn chưa tối ưu khi nói đến việc viết mã bất đồng bộ tuần tự. Chuỗi Promise thực sự tốt hơn nhiều so với việc bị callback hell, nhưng dẫu vậy, ta vẫn phải gọi một `then()` và tạo một hàm mới cho mỗi tác vụ trong chuỗi. Điều này vẫn còn quá nhiều đối với một luồng điều khiển chắc chắn là được dùng phổ biến nhất trong lập trình hằng ngày. JavaScript cần một cách thích hợp để xử lý luồng thực thi tuần tự bất đồng bộ phổ biến này, và câu trả lời đã đến với việc giới thiệu trong chuẩn ECMAScript các hàm `async` và biểu thức `await` (gọi tắt là `async/await`).

The `async/await` dichotomy allows us to write functions that appear to block at each asynchronous operation, waiting for the results before continuing with the following statement. As we will see, any asynchronous code using `async/await` has a readability comparable to traditional synchronous code.

Cặp đôi `async/await` cho phép chúng ta viết các hàm trông như thể chặn lại tại mỗi thao tác bất đồng bộ, chờ kết quả trước khi tiếp tục với câu lệnh tiếp theo. Như ta sẽ thấy, bất kỳ mã bất đồng bộ nào dùng `async/await` đều có khả năng đọc được sánh ngang với mã đồng bộ truyền thống.

Today, `async/await` is the recommended construct for dealing with asynchronous code in both Node.js and JavaScript. However, `async/await` does not replace all that we have learned so far about asynchronous control flow patterns; on the contrary, as we will see, `async/await` piggybacks heavily onto promises.

Ngày nay, `async/await` là cấu trúc được khuyến nghị để xử lý mã bất đồng bộ trong cả Node.js và JavaScript. Tuy nhiên, `async/await` không thay thế tất cả những gì ta đã học cho tới giờ về các mẫu điều khiển luồng bất đồng bộ; ngược lại, như ta sẽ thấy, `async/await` dựa rất nhiều vào promise.

Async functions and the await expression — Hàm async và biểu thức await

An `async` function is a special type of function in which it's possible to use the `await` expression to “pause” the execution on a given Promise until it resolves. Let's consider a simple example and use

the `delay()` function we implemented in the Creating a promise subsection. The Promise returned by `delay()` resolves with the current date as the value after the given number of milliseconds. Let's use this function with the `async/await` pair:

Một hàm `async` là một loại hàm đặc biệt mà trong đó ta có thể dùng biểu thức `await` để “tạm dừng” việc thực thi trên một Promise cho đến khi nó resolve. Hãy xét một ví dụ đơn giản và dùng hàm `delay()` mà ta đã hiện thực trong tiểu mục Tạo một promise. Promise được trả về bởi `delay()` resolve với ngày giờ hiện tại làm giá trị sau số mili giây đã cho. Hãy dùng hàm này với cặp `async/await`:

```
async function playingWithDelays () {
  console.log('Delaying...', new Date())
  const dateAfterOneSecond = await delay(1000)
  console.log(dateAfterOneSecond)
  const dateAfterThreeSeconds = await delay(3000)
  console.log(dateAfterThreeSeconds)
  return 'done'
}
```

As we can see from the previous function, `async/await` seems to work like magic. The code doesn't even look like it contains any asynchronous operation. However, don't be mistaken; this function does not run synchronously (they are called `async` functions for a reason!). At each `await` expression, the execution of the function is put on hold, its **state** saved, and the control returned to the event loop. Once the Promise that has been awaited resolves, the control is given back to the `async` function, returning the fulfillment value of the Promise.

Như ta có thể thấy từ hàm trên, `async/await` dường như hoạt động như có phép thuật. Đoạn mã thậm chí trông không giống như chứa bất kỳ thao tác bất đồng bộ nào. Tuy nhiên, đừng nhầm lẫn; hàm này không chạy đồng bộ (chúng được gọi là hàm `async` là có lý do!). Tại mỗi biểu thức `await`, việc thực thi của hàm bị tạm ngưng, trạng thái của nó được lưu lại, và quyền điều khiển được trả về cho vòng lặp sự kiện. Một khi Promise đã được `await` resolve, quyền điều khiển được trả lại cho hàm `async`, trả về giá trị fulfillment của Promise.

The `await` expression works with any value, not just promises. If a value other than a Promise is provided, then its behavior is similar to awaiting a value that it first passed to `Promise.resolve()`.

Biểu thức `await` hoạt động với bất kỳ giá trị nào, không chỉ promise. Nếu một giá trị khác Promise được cung cấp, thì hành vi của nó tương tự như việc `await` một giá trị mà trước tiên được truyền cho `Promise.resolve()`.

Let's now see how we can invoke our new `async` function:

Bây giờ hãy xem cách ta có thể gọi hàm `async` mới của mình:

```
playingWithDelays()
  .then(result => {
    console.log(`After 4 seconds: ${result}`)
  })
```

From the preceding code, it's clear that `async` functions can be invoked just like any other function. However, the most observant of you may have already spotted another important property of `async` functions: they always return a Promise. It's like if the return value of an `async` function was passed to `Promise.resolve()` and then returned to the caller.

Từ đoạn mã trên, rõ ràng là các hàm `async` có thể được gọi giống như bất kỳ hàm nào khác. Tuy nhiên, những người tinh ý nhất trong các bạn có lẽ đã phát hiện ra một thuộc tính

quan trọng khác của các hàm async: chúng luôn trả về một Promise. Cứ như thể giá trị trả về của một hàm async được truyền cho `Promise.resolve()` rồi sau đó được trả về cho bên gọi.

Invoking an async function is instantaneous, like any other asynchronous operation. In other words, async functions return a Promise synchronously. That Promise will then eventually settle based on the result or error produced by the function.

Việc gọi một hàm async là tức thời, giống như bất kỳ thao tác bất đồng bộ nào khác. Nói cách khác, các hàm async trả về một Promise một cách đồng bộ. Promise đó sau đó rất cuộc sẽ settle dựa trên kết quả hoặc lỗi do hàm tạo ra.

From this first encounter with async/await, we can see how dominant promises still are in our discussion. In fact, we can consider async/await just a syntactic sugar for a simpler consumption of promises. As we will see, all the asynchronous control flow patterns with async/await use promises and their API for most of the heavy-lifting operations.

Từ lần đầu tiếp xúc với async/await này, ta có thể thấy promise vẫn còn chiếm ưu thế thế nào trong cuộc thảo luận của chúng ta. Thực vậy, ta có thể coi async/await chỉ là cú pháp đường (syntactic sugar) cho việc tiêu thụ promise đơn giản hơn. Như ta sẽ thấy, tất cả các mẫu điều khiển luồng bất đồng bộ với async/await đều dùng promise và API của chúng cho hầu hết các thao tác nặng nhọc.

Error handling with async/await — Xử lý lỗi với async/await

Async/await doesn't just improve the readability of asynchronous code under standard conditions, but it also helps when handling errors. In fact, one of the biggest gains of async/await is the ability to normalize the behavior of the `try...catch` block, to make it work seamlessly with both synchronous throws and asynchronous Promise rejections. Let's demonstrate that with an example.

Async/await không chỉ cải thiện khả năng đọc của mã bất đồng bộ trong điều kiện chuẩn, mà nó còn giúp ích khi xử lý lỗi. Thực vậy, một trong những lợi ích lớn nhất của async/await là khả năng chuẩn hóa hành vi của khối `try...catch`, khiến nó hoạt động liền mạch với cả những lần ném (throw) đồng bộ lẫn những lần rejection bất đồng bộ của Promise. Hãy minh họa điều đó bằng một ví dụ.

A unified try...catch experience — Một trải nghiệm try...catch hợp nhất

Let's define a function that returns a Promise that rejects with an error after a given number of milliseconds. This is very similar to the `delay()` function that we already know very well:

Hãy định nghĩa một hàm trả về một Promise reject với một lỗi sau một số mili giây đã cho. Cái này rất giống với hàm `delay()` mà ta đã biết rất rõ:

```
function delayError (milliseconds) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      reject(new Error(`Error after ${milliseconds}ms`))
    }, milliseconds)
  })
}
```

Next, let's implement an async function that can throw an error synchronously or await a Promise that will reject. This function demonstrates how both the synchronous throw and the Promise rejection are caught by the same catch block:

Tiếp theo, hãy hiện thực một hàm async có thể throw một lỗi một cách đồng bộ hoặc await một Promise sẽ reject. Hàm này minh họa cách cả throw đồng bộ lẫn rejection của Promise đều được bắt bởi cùng một khối catch:

```
async function playingWithErrors (throwSyncError) {
  try {
    if (throwSyncError) {
      throw new Error('This is a synchronous error')
    }
    await delayError(1000)
  } catch (err) {
    console.error(`We have an error: ${err.message}`)
  } finally {
    console.log('Done')
  }
}
```

Now, invoking the function like this:

Bây giờ, gọi hàm như thế này:

```
playingWithErrors(true)
```

Will print to the console the following:

Sẽ in ra console như sau:

```
We have an error: This is a synchronous error
Done
```

While invoking the function with false as the input, like this:

Trong khi gọi hàm với false làm đầu vào, như thế này:

```
playingWithErrors(false)
```

Will produce the following output:

Sẽ tạo ra kết quả sau:

```
We have an error: Error after 1000ms
Done
```

If we remember how we had to deal with errors in Chapter 4, Asynchronous Control Flow Patterns with Callbacks, we will surely appreciate the giant improvements introduced by both promises and async/await. Now, error handling is just as it should be: simple, readable, and most importantly, supporting both synchronous and asynchronous errors.

Nếu ta nhớ lại cách ta đã phải xử lý lỗi ở Chương 4, Asynchronous Control Flow Patterns with Callbacks, chắc chắn ta sẽ trân trọng những cải tiến khổng lồ mà cả promise lẫn async/await mang lại. Giờ đây, việc xử lý lỗi đúng như nó nên là: đơn giản, dễ đọc, và quan trọng nhất, hỗ trợ cả lỗi đồng bộ lẫn bất đồng bộ.

The “return” versus “return await” trap — Cái bẫy “return” so với “return await”

One common **antipattern** when dealing with errors with `async/await` is returning a `Promise` that rejects to the caller and expecting the error to be caught by the local `try...catch` block of the function that is returning the `Promise`.

Một antipattern phổ biến khi xử lý lỗi với `async/await` là trả về một `Promise` đang reject cho bên gọi và kỳ vọng lỗi sẽ được bắt bởi khối `try...catch` cục bộ của hàm đang trả về `Promise` đó.

For example, consider the following code:

Chẳng hạn, hãy xét đoạn mã sau:

```
async function errorNotCaught () {
  try {
    return delayError(1000)
  } catch (err) {
    console.error('Error caught by the async function: ' +
      err.message)
  }
}
errorNotCaught()
  .catch(err => console.error('Error caught by the caller: ' +
    err.message))
```

The `Promise` returned by `delayError()` is not awaited locally, which means that it's returned as it is to the caller. As a consequence, the local `catch` block will never be invoked. In fact, the previous code will output:

`Promise` được trả về bởi `delayError()` không được `await` cục bộ, nghĩa là nó được trả về nguyên trạng cho bên gọi. Hệ quả là khối `catch` cục bộ sẽ không bao giờ được gọi. Thực vậy, đoạn mã trên sẽ xuất ra:

Error caught by the caller: Error after 1000ms

If our intention is catching locally any error generated by the asynchronous operation that produces the value that we want to return to the caller, then we have to use the `await` expression on that `Promise` before we return the value to the caller. The following code demonstrates this:

Nếu ý định của ta là bắt cục bộ bất kỳ lỗi nào do thao tác bất đồng bộ tạo ra giá trị mà ta muốn trả về cho bên gọi, thì ta phải dùng biểu thức `await` trên `Promise` đó trước khi ta trả giá trị về cho bên gọi. Đoạn mã sau minh họa điều này:

```
async function errorCaught () {
  try {
    return await delayError(1000)
  } catch (err) {
    console.error('Error caught by the async function: ' +
      err.message) }
}
errorCaught()
  .catch(err => console.error('Error caught by the caller: ' +
    err.message))
```

All we did was add an `await` after the `return` keyword. This is enough to cause the `async` function

to “deal” with the Promise locally and therefore also catch any rejection locally. As a confirmation, when we run the previous code, we should see the following output:

Tất cả những gì ta làm là thêm một `await` sau từ khóa `return`. Điều này đủ để khiến hàm `async` “xử lý” Promise một cách cục bộ và do đó cũng bắt bất kỳ rejection nào một cách cục bộ. Để xác nhận, khi ta chạy đoạn mã trên, ta sẽ thấy kết quả sau:

Error caught by the **async function**: Error after 1000ms

Sequential execution and iteration — Thực thi tuần tự và lặp

Our exploration of control flow patterns with `async/await` starts with sequential execution and iteration. We already mentioned a few times that the core strength of `async/await` lies in its ability to make asynchronous serial execution easy to write and straightforward to read. This was already apparent in all the code samples we have written so far; however, it will become even more obvious now that we will start converting our web spider version 2. `Async/await` is so simple to use and understand that there are really no patterns here to study. We will get straight to the code, without any preamble. So, let’s start with the `download()` function of our web spider; this is how it looks with `async/await`:

Việc khám phá các mẫu điều khiển luồng với `async/await` của chúng ta bắt đầu với thực thi tuần tự và lặp. Ta đã đề cập vài lần rằng sức mạnh cốt lõi của `async/await` nằm ở khả năng làm cho việc thực thi tuần tự bất đồng bộ dễ viết và dễ đọc. Điều này đã rõ ràng trong tất cả các mẫu mã ta đã viết cho tới giờ; tuy nhiên, nó sẽ trở nên còn hiển nhiên hơn nữa giờ khi ta bắt đầu chuyển đổi web spider phiên bản 2. `Async/await` đơn giản đến mức dễ dùng và dễ hiểu nên thực sự không có mẫu nào ở đây để nghiên cứu. Ta sẽ đi thẳng vào mã, không cần lời mở đầu nào. Vậy thì, hãy bắt đầu với hàm `download()` của web spider; đây là cách nó trông với `async/await`:

```
async function download (url, filename) {
  console.log(`Downloading ${url}`)
  const { text: content } = await superagent.get(url)
  await mkdirpPromises(dirname(filename))
  await fsPromises.writeFile(filename, content)
  console.log(`Downloaded and saved: ${url}`)
  return content
}
```

Let’s appreciate for a moment how simple and compact the `download()` function has become. Let’s just consider that the same functionality was implemented with callbacks in two different functions using a total of 19 lines of code. Now we just have seven. Plus, the code is now completely flat, with no nesting at all. This tells us a lot about the enormous positive impact that `async/await` has on our code.

Hãy dành một chút để trân trọng hàm `download()` đã trở nên đơn giản và gọn gàng như thế nào. Hãy nghĩ rằng cùng chức năng đó đã được hiện thực bằng callback trong hai hàm khác nhau với tổng cộng 19 dòng mã. Giờ ta chỉ có bảy. Hơn nữa, mã giờ hoàn toàn phẳng, không có lồng nhau nào cả. Điều này nói lên rất nhiều về tác động tích cực to lớn mà `async/await` có lên mã của chúng ta.

Now, let’s see how we can iterate asynchronously over an array using `async/await`. This is exemplified in the `spiderLinks()` function:

Bây giờ, hãy xem cách ta có thể lặp một cách bất đồng bộ trên một mảng bằng `async/await`. Điều này được minh họa trong hàm `spiderLinks()`:

```
async function spiderLinks (currentUrl, content, nesting) {
  if (nesting === 0) {
    return
  }
  const links = getPageLinks(currentUrl, content)
  for (const link of links) {
    await spider(link, nesting - 1)
  }
}
```

Even here there is no pattern to learn. We just have a simple iteration over a list of links and for each item, we await on the Promise returned by `spider()`.

Ngay cả ở đây cũng không có mẫu nào để học. Ta chỉ đơn giản có một phép lặp đơn giản trên một danh sách các links và với mỗi phần tử, ta `await` trên Promise được trả về bởi `spider()`.

The next code fragment shows the `spider()` function implemented using `async/await`. The aspect to notice here is how errors are easily dealt with using just a `try...catch` statement, making everything easier to read:

Đoạn mã tiếp theo cho thấy hàm `spider()` được hiện thực bằng `async/await`. Khía cạnh cần lưu ý ở đây là cách lỗi được xử lý dễ dàng chỉ bằng một câu lệnh `try...catch`, khiến mọi thứ dễ đọc hơn:

```
export async function spider (url, nesting) {
  const filename = urlToFilename(url)
  let content
  try {
    content = await fsPromises.readFile(filename, 'utf8')
  } catch (err) {
    if (err.code !== 'ENOENT') {
      throw err
    }
    content = await download(url, filename)
  }
  return spiderLinks(url, content, nesting)
}
```

And with the `spider()` function, we have completed the conversion of our web spider application to `async/await`. As you can see, it has been quite a smooth process but the results are quite impressive.

Và với hàm `spider()`, ta đã hoàn tất việc chuyển đổi ứng dụng web spider sang `async/await`. Như bạn có thể thấy, đó là một quá trình khá suôn sẻ nhưng kết quả thì khá ấn tượng.

Antipattern – using `async/await` with `Array.forEach` for serial execution — Antipattern – dùng `async/await` với `Array.forEach` để thực thi tuần tự

It's worth mentioning that there is a common antipattern whereby developers will try to use `Array.forEach()` or `Array.map()` to implement a sequential asynchronous iteration with `async/await`, which, of course, won't work as expected.

Đáng nói là có một antipattern phổ biến trong đó các lập trình viên cố dùng `Array.forEach()` hoặc `Array.map()` để hiện thực một phép lặp bất đồng bộ tuần tự với `async/await`, điều mà tất nhiên sẽ không hoạt động như mong đợi.

To see why, let's take a look at the following alternate implementation (which is wrong!) of the asynchronous iteration in the `spiderLinks()` function:

Để hiểu tại sao, hãy xem qua bản hiện thực thay thế sau đây (sai!) của phép lặp bất đồng bộ trong hàm `spiderLinks()`:

```
links.forEach(async function iteration(link) {  
  await spider(link, nesting - 1)  
})
```

In the previous code, the iteration function is invoked once for each element of the `links` array. Then, in the iteration function, we use the `await` expression on the Promise returned by `spider()`. However, the Promise returned by the iteration function is just ignored by `forEach()`. The result is that all the `spider()` functions are invoked in the same round of the event loop, which means they are started in parallel, and the execution continues immediately after invoking `forEach()`, without waiting for all the `spider()` operations to complete.

Trong đoạn mã trên, hàm `iteration` được gọi một lần cho mỗi phần tử của mảng `links`. Sau đó, trong hàm `iteration`, ta dùng biểu thức `await` trên Promise được trả về bởi `spider()`. Tuy nhiên, Promise được trả về bởi hàm `iteration` chỉ đơn giản bị bỏ qua bởi `forEach()`. Kết quả là tất cả các hàm `spider()` được gọi trong cùng một vòng của vòng lặp sự kiện, nghĩa là chúng được khởi động song song, và việc thực thi tiếp tục ngay lập tức sau khi gọi `forEach()`, mà không chờ tất cả các thao tác `spider()` hoàn tất.

Parallel execution — Thực thi song song

There are mainly two ways to run a set of tasks in parallel using `async/await`; one purely uses the `await` expression and the other relies on `Promise.all()`. They are both very simple to implement; however, be advised that the method relying on `Promise.all()` is the recommended (and optimal) one to use. Let's see an example of both. Let's consider the `spiderLinks()` function of our web spider. If we wanted to purely use the `await` expression to implement an unlimited parallel asynchronous execution flow, we would do it with some code like the following:

Có chủ yếu hai cách để chạy một tập hợp các tác vụ song song bằng `async/await`; một cách thuần túy dùng biểu thức `await` và cách kia dựa vào `Promise.all()`. Cả hai đều rất đơn giản để hiện thực; tuy nhiên, hãy lưu ý rằng phương pháp dựa vào `Promise.all()` là phương pháp được khuyến nghị (và tối ưu) để dùng. Hãy xem một ví dụ về cả hai. Hãy xét hàm `spiderLinks()` của web spider. Nếu ta muốn thuần túy dùng biểu thức `await` để hiện thực một luồng thực thi bất đồng bộ song song không giới hạn, ta sẽ làm như đoạn mã sau:

```
async function spiderLinks (currentUrl, content, nesting) {  
  if (nesting === 0) {  
    return  
  }  
  const links = getPageLinks(currentUrl, content)  
  const promises = links.map(link => spider(link, nesting - 1))  
  for (const promise of promises) {  
    await promise  
  }  
}
```

```
}  
}
```

That's it—very simple. In the previous code, we first start all the `spider()` tasks in parallel, collecting their promises with a `map()`. Then, we loop, and we `await` on each one of those promises.

Vậy đó—rất đơn giản. Trong đoạn mã trên, đầu tiên ta khởi động tất cả các tác vụ `spider()` song song, thu thập các promise của chúng bằng một `map()`. Sau đó, ta lặp, và ta `await` trên mỗi promise đó.

At first, it seems neat and functional; however, it has a small undesired effect. If a Promise in the array rejects, we have to wait for all the preceding promises in the array to resolve before the Promise returned by `spiderLinks()` will also reject. This is not optimal in most situations, as we usually want to know if an operation has failed as soon as possible. Luckily, we already have a built-in function that behaves exactly the way we want, and that's `Promise.all()`. In fact, `Promise.all()` will reject as soon as any of the promises provided in the input array reject. Therefore, we can simply rely on this method even for all our `async/await` code. And, since `Promise.all()` returns just another Promise, we can simply invoke an `await` on it to get the results from multiple asynchronous operations. The following code shows an example:

Thoạt nhìn, nó có vẻ gọn gàng và hoạt động tốt; tuy nhiên, nó có một tác dụng phụ nhỏ không mong muốn. Nếu một Promise trong mảng reject, ta phải chờ tất cả các promise đứng trước trong mảng resolve trước khi Promise được trả về bởi `spiderLinks()` cũng sẽ reject. Điều này không tối ưu trong hầu hết các tình huống, vì ta thường muốn biết liệu một thao tác có thất bại hay không càng sớm càng tốt. May mắn thay, ta đã có sẵn một hàm tích hợp hành xử đúng theo cách ta muốn, và đó là `Promise.all()`. Thực vậy, `Promise.all()` sẽ reject ngay khi bất kỳ promise nào được cung cấp trong mảng đầu vào reject. Do đó, ta có thể chỉ đơn giản dựa vào phương thức này ngay cả cho tất cả mã `async/await` của chúng ta. Và, vì `Promise.all()` chỉ trả về một Promise khác, ta có thể chỉ đơn giản gọi một `await` trên nó để lấy kết quả từ nhiều thao tác bất đồng bộ. Đoạn mã sau cho thấy một ví dụ:

```
const results = await Promise.all(promises)
```

So, to wrap up, our recommended implementation of the `spiderLinks()` function with parallel execution and `async/await` will look almost identical to that using promises. The only visible difference is the fact that we are now using an `async` function, which always returns a Promise:

Vậy, để tóm lại, bản hiện thực được khuyến nghị của chúng ta cho hàm `spiderLinks()` với thực thi song song và `async/await` sẽ trông gần như giống hệt với bản dùng promise. Khác biệt duy nhất nhìn thấy được là việc giờ ta đang dùng một hàm `async`, hàm này luôn trả về một Promise:

```
async function spiderLinks (currentUrl, content, nesting) {  
  if (nesting === 0) {  
    return  
  }  
  const links = getPageLinks(currentUrl, content)  
  const promises = links.map(link => spider(link, nesting - 1))  
  return Promise.all(promises)  
}
```

What we just learned about parallel execution and `async/await` simply reiterates the fact that `async/await` is inseparable from promises. Most of the utilities that work with promises will also

seamlessly work with `async/await` and we should never hesitate to take advantage of them in our `async` functions.

Những gì ta vừa học về thực thi song song và `async/await` chỉ một lần nữa khẳng định sự thật rằng `async/await` không thể tách rời khỏi `promise`. Hầu hết các tiện ích làm việc với `promise` cũng sẽ làm việc liền mạch với `async/await` và ta không bao giờ nên ngăn ngại tận dụng chúng trong các hàm `async` của mình.

Limited parallel execution — Thực thi song song có giới hạn

To implement a limited parallel execution pattern with `async/await`, we can simply reuse the `TaskQueue` class that we created in the Limited parallel execution subsection within the Promises section. We can either use it as it is or convert its internals to `async/await`. Converting the `TaskQueue` class to `async/await` is a trivial operation and we'll leave this to you as an exercise. Either way, the `TaskQueue` external interface shouldn't change; both implementations will have a `runTask()` method that returns a `Promise` that settles when the task has been run by the queue.

Để hiện thực một mẫu thực thi song song có giới hạn với `async/await`, ta có thể chỉ đơn giản tái sử dụng lớp `TaskQueue` mà ta đã tạo trong tiểu mục Thực thi song song có giới hạn trong phần Promise. Ta có thể hoặc dùng nó nguyên trạng hoặc chuyển đổi nội bộ của nó sang `async/await`. Chuyển đổi lớp `TaskQueue` sang `async/await` là một thao tác tầm thường và ta sẽ để điều này lại cho bạn như một bài tập. Dù theo cách nào, giao diện bên ngoài của `TaskQueue` cũng không nên thay đổi; cả hai bản hiện thực sẽ có một phương thức `runTask()` trả về một `Promise` settle khi tác vụ đã được chạy bởi hàng đợi.

Starting from this assumption, converting the web spider v4 from promises to `async/await` is also a trivial task and we won't show all the steps here as we wouldn't be learning anything new. Instead, what we'll do in this section is examine a third variation of the `TaskQueue` class that uses `async/await` and a producer-consumer approach.

Bắt đầu từ giả định này, việc chuyển đổi web spider v4 từ `promise` sang `async/await` cũng là một tác vụ tầm thường và ta sẽ không trình bày tất cả các bước ở đây vì ta sẽ không học được điều gì mới. Thay vào đó, điều ta sẽ làm trong phần này là xem xét một biến thể thứ ba của lớp `TaskQueue` dùng `async/await` và cách tiếp cận producer-consumer.

The general idea to apply this approach to our problem goes as follows:

Ý tưởng chung để áp dụng cách tiếp cận này cho bài toán của chúng ta như sau:

- On one side, we have an unknown set of producers adding tasks into a queue.
- On the other side, we have a predefined set of consumers, responsible for extracting and executing the tasks from the queue, one at a time.
 - Một bên, ta có một tập hợp không xác định các producer thêm tác vụ vào một hàng đợi.
 - Bên kia, ta có một tập hợp được định nghĩa trước các consumer, chịu trách nhiệm trích xuất và thực thi các tác vụ từ hàng đợi, mỗi lần một tác vụ.

The following diagram should help us understand the setup:

Sơ đồ sau đây sẽ giúp chúng ta hiểu cách bố trí:

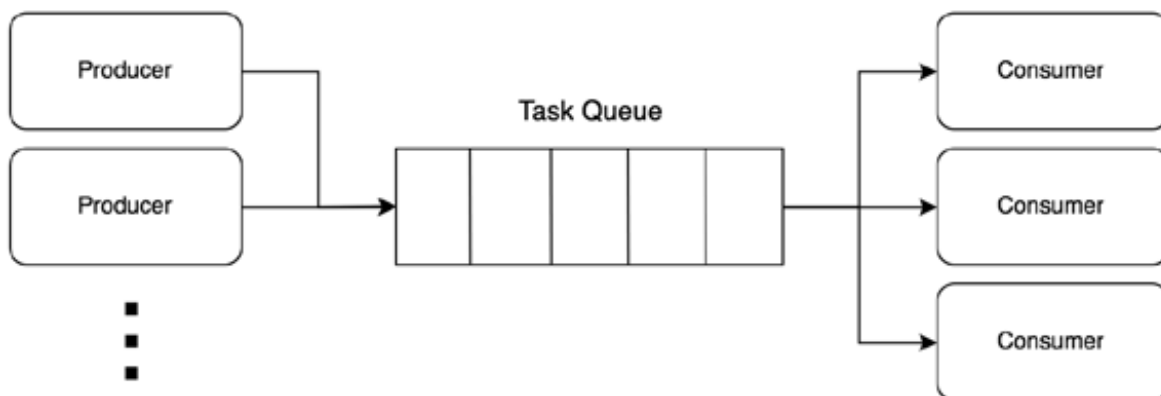


Figure 5.2: Using the Producer-Consumer pattern to implement limited parallel execution. The number of consumers will determine the concurrency with which the tasks will be executed. The challenge here is to put the consumers to “sleep” when the queue is empty and “wake them up” again when there are new tasks to run. But we are lucky, since Node.js is **single-threaded**, so putting a task to “sleep” just means giving back control to the event loop, while “resuming” a task is equivalent to invoking a callback.

Hình 5.2: Dùng mẫu Producer-Consumer để hiện thực thực thi song song có giới hạn. Số lượng consumer sẽ quyết định mức độ đồng thời mà các tác vụ được thực thi. Thách thức ở đây là đưa các consumer vào “ngủ” khi hàng đợi rỗng và “đánh thức chúng dậy” trở lại khi có tác vụ mới để chạy. Nhưng ta may mắn, vì Node.js là đơn luồng (single-threaded), nên đưa một tác vụ vào “ngủ” chỉ có nghĩa là trả lại quyền điều khiển cho vòng lặp sự kiện, trong khi “khôi phục” một tác vụ tương đương với việc gọi một callback.

With this in mind, let’s then take a look at some code. We will create a new class called TaskQueuePC with a public interface similar to one of the TaskQueue classes we implemented previously in this chapter. Taking a top-down approach, let’s see how we can implement the constructor:

Với điều này trong đầu, hãy cùng xem qua một số mã. Ta sẽ tạo một lớp mới gọi là TaskQueuePC với một giao diện công khai tương tự một trong các lớp TaskQueue mà ta đã hiện thực trước đó trong chương này. Theo cách tiếp cận từ trên xuống, hãy xem cách ta có thể hiện thực hàm dựng:

```
export class TaskQueuePC {
  constructor (concurrency) {
    this.taskQueue = []
    this.consumerQueue = []
    // spawn consumers
    for (let i = 0; i < concurrency; i++) {
      this.consumer()
    }
  }
  // ...
}
```

First of all, we can notice that we now have two queues, one to hold our tasks (taskQueue) and the other to store our sleeping consumers (consumerQueue). It will be clearer in a while how these queues will be used. In the second part of our constructor, we spawn as many consumers as the concurrency we want to attain. Let’s see what a consumer looks like:

Trước hết, ta có thể nhận thấy rằng giờ ta có hai hàng đợi, một để giữ các tác vụ của chúng ta

(taskQueue) và cái kia để lưu trữ các consumer đang ngủ của chúng ta (consumerQueue). Một lát nữa sẽ rõ hơn cách những hàng đợi này được dùng. Trong phần thứ hai của hàm dừng, ta sinh ra số consumer bằng với mức độ đồng thời mà ta muốn đạt được. Hãy xem một consumer trông như thế nào:

```
async consumer () {  
  while (true) { // (1)  
    try {  
      const task = await this.getNextTask() // (2)  
      await task() // (3)  
    } catch (err) {  
      console.error(err) // (4)  
    }  
  }  
}
```

Our consumer is an infinite while loop (1). At each iteration, we try to retrieve a new task from the queue using getNextTask() (2). As we will see, this will cause the current consumer to sleep if the queue is empty. When a new task is eventually available, we just execute it (3). Any error thrown from the above operation should not cause the consumer to stop, so we simply log it (4) and continue with the next iteration.

Consumer của chúng ta là một vòng lặp while vô hạn (1). Tại mỗi lần lặp, ta cố lấy một tác vụ mới từ hàng đợi bằng getNextTask() (2). Như ta sẽ thấy, điều này sẽ khiến consumer hiện tại ngủ nếu hàng đợi rỗng. Khi một task mới tới cuộc có sẵn, ta chỉ đơn giản thực thi nó (3). Bất kỳ lỗi nào ném ra từ thao tác trên đều không nên khiến consumer dừng lại, nên ta chỉ đơn giản ghi nhật ký nó (4) và tiếp tục với lần lặp tiếp theo.

By the look of it, it may seem that each consumer in TaskQueuePC is an actual **thread**. In fact, our consumer() function has an infinite loop and it can “pause” until awakened by some other “thread.” In reality, we should not forget that each consumer is an async function, which is nothing more than a nice syntax built around promises and callbacks. The while loop may seem to be spinning continuously consuming CPU cycles, but under the hood, the loop is more similar to an asynchronous recursion than a traditional while loop.

Nhìn qua, có vẻ như mỗi consumer trong TaskQueuePC là một luồng thực sự. Thực vậy, hàm consumer() của chúng ta có một vòng lặp vô hạn và nó có thể “tạm dừng” cho tới khi được đánh thức bởi một “luồng” nào khác. Trên thực tế, ta không nên quên rằng mỗi consumer là một hàm async, thứ chẳng qua chỉ là một cú pháp đẹp để được xây dựng quanh promise và callback. Vòng lặp while có vẻ đang quay liên tục tiêu tốn các chu kỳ CPU, nhưng dưới lớp vỏ, vòng lặp này giống một phép đệ quy bất đồng bộ hơn là một vòng lặp while truyền thống.

With the next code fragment, we should start to get an idea of what’s going on. Let’s take a look at the implementation of getNextTask():

Với đoạn mã tiếp theo, ta sẽ bắt đầu hiểu được điều gì đang diễn ra. Hãy xem qua bản hiện thực của getNextTask():

```
async getNextTask () {  
  return new Promise((resolve) => {  
    if (this.taskQueue.length !== 0) {  
      return resolve(this.taskQueue.shift()) // (1)  
    }  
  })  
}
```

```

    this.consumerQueue.push(resolve) // (2)
  })
}

```

The `getNextTask()` method returns a new `Promise` that resolves with the first task in the queue if the queue is not empty. The first task is removed from `taskQueue` and used as an argument to invoke `resolve` (1). If the queue is instead empty, we postpone the resolution of the `Promise` by queuing the `resolve` callback into the `consumerQueue`. This will effectively put the `Promise`—and the consumer that is awaiting the `Promise`—to sleep.

Phương thức `getNextTask()` trả về một `Promise` mới resolve với tác vụ đầu tiên trong hàng đợi nếu hàng đợi không rỗng. Tác vụ đầu tiên được loại bỏ khỏi `taskQueue` và được dùng làm đối số để gọi `resolve` (1). Nếu hàng đợi rỗng, ta trì hoãn việc resolution của `Promise` bằng cách đưa callback `resolve` vào `consumerQueue` (2). Điều này thực chất sẽ đưa `Promise`—và consumer đang await `Promise` đó—vào ngủ.

Now comes the “gluing” part of the whole `TaskQueuePC` class, which corresponds to the producer side of the algorithm. That’s implemented in the `runTask()` method:

Bây giờ tới phần “kết dính” của toàn bộ lớp `TaskQueuePC`, tương ứng với phía producer của thuật toán. Phần đó được hiện thực trong phương thức `runTask()`:

```

runTask (task) {
  return new Promise((resolve, reject) => {
    const taskWrapper = () => { // (1)
      const taskPromise = task()
      taskPromise.then(resolve, reject)
      return taskPromise
    }
    if (this.consumerQueue.length !== 0) { // (2)
      const consumer = this.consumerQueue.shift()
      consumer(taskWrapper)
    } else { // (3)
      this.taskQueue.push(taskWrapper)
    }
  })
}

```

First, we create a `taskWrapper` function (1) that, when executed, has the responsibility for running the input task and forwarding the status of the `Promise` returned by `task()` to the outer `Promise` returned by `runTask()`. Next, if the `consumerQueue` is not empty (2), it means that there is at least one consumer that is asleep, waiting for new tasks to run. We then extract the first consumer from the queue (remember, that’s essentially the `resolve` callback of the `Promise` returned by `getNextTask()`) and we invoke it immediately by passing our `taskWrapper`. If, instead, all the consumers are already busy (3), we push `taskWrapper` into the `taskQueue`.

Đầu tiên, ta tạo một hàm `taskWrapper` (1) mà, khi được thực thi, có trách nhiệm chạy `task` đầu vào và chuyển tiếp trạng thái của `Promise` được trả về bởi `task()` tới `Promise` bên ngoài được trả về bởi `runTask()`. Tiếp theo, nếu `consumerQueue` không rỗng (2), điều đó có nghĩa là có ít nhất một consumer đang ngủ, chờ tác vụ mới để chạy. Khi đó ta trích consumer đầu tiên ra khỏi hàng đợi (nhớ rằng, đó về cơ bản là callback `resolve` của `Promise` được trả về bởi `getNextTask()`) và ta gọi nó ngay lập tức bằng cách truyền `taskWrapper` của chúng ta. Nếu thay vào đó tất cả các consumer đã bận (3), ta đẩy `taskWrapper` vào `taskQueue`.

This concludes the implementation of our TaskQueuePC class. The public interface of the TaskQueuePC class is identical to that of the TaskQueue class that we implemented in the Promises section, so migrating the code of our web spider to the new algorithm will be a trivial task.

Điều này kết thúc việc hiện thực lớp TaskQueuePC của chúng ta. Giao diện công khai của lớp TaskQueuePC giống hệt với giao diện của lớp TaskQueue mà ta đã hiện thực trong phần Promise, nên việc di trú mã của web spider sang thuật toán mới sẽ là một tác vụ tầm thường.

This also concludes our exploration of the async/await construct. But, before we wrap up the chapter, we'll dive into a subtle problem affecting promises.

Điều này cũng kết thúc việc khám phá cấu trúc async/await của chúng ta. Nhưng, trước khi gói lại chương này, ta sẽ đào sâu vào một vấn đề tinh tế ảnh hưởng tới promise.

The problem with infinite recursive promise resolution chains — Vấn đề với các chuỗi resolution promise đệ quy vô hạn

At this point in the chapter, you should have a strong understanding of how promises work and how to use them to implement the most common control flow constructs. This is therefore the right time to discuss an advanced topic that every professional Node.js developer should know and understand. This advanced topic is about a memory leak caused by infinite Promise resolution chains. The bug seems to affect the actual Promises/A+ specification, so no compliant implementation is immune. It is quite common in programming to have tasks that don't have a predefined ending or take as an input a potentially infinite array of data. We can include in this category things like the encoding/decoding of live audio/video streams, the processing of live cryptocurrency market data, and the monitoring of IoT sensors. But we can have much more trivial situations than those, for example, when making heavy use of functional programming.

Tại thời điểm này trong chương, bạn nên đã có một hiểu biết vững chắc về cách promise hoạt động và cách dùng chúng để hiện thực những cấu trúc điều khiển luồng phổ biến nhất. Do đó đây là thời điểm thích hợp để thảo luận về một chủ đề nâng cao mà mọi lập trình viên Node.js chuyên nghiệp nên biết và hiểu. Chủ đề nâng cao này là về một rò rỉ bộ nhớ (memory leak) gây ra bởi các chuỗi resolution Promise vô hạn. Lỗi này có vẻ ảnh hưởng đến chính đặc tả Promises/A+, nên không bản hiện thực tuân thủ nào miễn nhiễm. Trong lập trình, khá phổ biến việc có những tác vụ không có điểm kết thúc được định trước hoặc nhận làm đầu vào một mảng dữ liệu có khả năng vô hạn. Ta có thể xếp vào loại này những thứ như mã hóa/giải mã các luồng âm thanh/video trực tiếp, xử lý dữ liệu thị trường tiền điện tử trực tiếp, và giám sát các cảm biến IoT. Nhưng ta cũng có thể có những tình huống tầm thường hơn nhiều so với những thứ đó, chẳng hạn, khi sử dụng nhiều lập trình hàm.

To take a simple example, let's consider the following code, which defines a simple infinite operation using promises:

Để lấy một ví dụ đơn giản, hãy xét đoạn mã sau, định nghĩa một thao tác vô hạn đơn giản dùng promise:

```
function leakingLoop () {  
  return delay(1)  
    .then(() => {  
      console.log(`Tick ${Date.now()}`)  
      return leakingLoop()  
    })  
}
```

```

    })
  }
}

```

The `leakingLoop()` function that we just defined uses the `delay()` function (which we created at the beginning of this chapter) to simulate an asynchronous operation. When the given number of milliseconds has elapsed, we print the current timestamp and we invoke `leakingLoop()` recursively to start the operation over again. The interesting part is that the Promise returned by `leakingLoop()` never resolves because its status depends on the next invocation of `leakingLoop()`, which in turn depends on the next invocation of `leakingLoop()` and so on. This situation creates a chain of promises that never settle, and it will cause a memory leak in Promise implementations that strictly follow the Promises/A+ specification, including JavaScript ES6 promises.

Hàm `leakingLoop()` mà ta vừa định nghĩa dùng hàm `delay()` (mà ta đã tạo ở đầu chương này) để mô phỏng một thao tác bất đồng bộ. Khi số mili giây đã cho trôi qua, ta in dấu thời gian (timestamp) hiện tại và ta gọi `leakingLoop()` một cách đệ quy để bắt đầu lại thao tác từ đầu. Phần thú vị là Promise được trả về bởi `leakingLoop()` không bao giờ resolve vì trạng thái của nó phụ thuộc vào lời gọi `leakingLoop()` tiếp theo, lời gọi này đến lượt nó lại phụ thuộc vào lời gọi `leakingLoop()` tiếp theo và cứ thế. Tình huống này tạo ra một chuỗi promise không bao giờ settle, và nó sẽ gây ra một rò rỉ bộ nhớ trong các bản hiện thực Promise tuân thủ chặt chẽ đặc tả Promises/A+, bao gồm cả promise ES6 của JavaScript.

To demonstrate the leak, we can try running the `leakingLoop()` function many times to accentuate the effects of the leak:

Để minh họa rò rỉ, ta có thể thử chạy hàm `leakingLoop()` nhiều lần để làm nổi bật các tác động của rò rỉ:

```

for (let i = 0; i < 1e6; i++) {
  leakingLoop()
}

```

Then we can take a look at the memory footprint of the process using our favorite process inspector and notice how it grows indefinitely until (after a few minutes) the process crashes entirely.

Sau đó ta có thể xem qua mức sử dụng bộ nhớ của tiến trình bằng trình kiểm tra tiến trình ưa thích của mình và nhận thấy cách nó tăng lên vô hạn cho tới khi (sau vài phút) tiến trình sụp đổ hoàn toàn.

The solution to the problem is to break the chain of Promise resolution. We can do that by making sure that the status of the Promise returned by `leakingLoop()` does not depend on the promise returned by the next invocation of `leakingLoop()`. We can ensure that by simply removing a return instruction:

Giải pháp cho vấn đề là phá vỡ chuỗi resolution Promise. Ta có thể làm điều đó bằng cách đảm bảo rằng trạng thái của Promise được trả về bởi `leakingLoop()` không phụ thuộc vào promise được trả về bởi lời gọi `leakingLoop()` tiếp theo. Ta có thể đảm bảo điều đó bằng cách chỉ đơn giản loại bỏ một câu lệnh return:

```

function nonLeakingLoop () {
  delay(1)
  .then(() => {
    console.log(`Tick ${Date.now()}`)
    nonLeakingLoop()
  })
}

```

```
}
```

Now, if we use this new function in our sample program, we should see that the memory footprint of the process will go up and down, following the schedule of the various runs of the garbage collector, which means that there is no memory leak.

Bây giờ, nếu ta dùng hàm mới này trong chương trình mẫu của mình, ta sẽ thấy rằng mức sử dụng bộ nhớ của tiến trình sẽ lên xuống, theo lịch trình của các lần chạy khác nhau của bộ thu gom rác (garbage collector), nghĩa là không có rò rỉ bộ nhớ.

However, the solution we have just proposed radically changes the behavior of the original `leakingLoop()` function. In particular, this new function won't propagate eventual errors produced deeply within the recursion, since there is no link between the status of the various promises. This inconvenience may be mitigated by adding some extra logging within the function. But sometimes the new behavior itself may not be an option. So, a possible solution involves wrapping the recursive function with a `Promise` constructor, such as in the following code sample:

Tuy nhiên, giải pháp ta vừa đề xuất thay đổi triệt để hành vi của hàm `leakingLoop()` ban đầu. Cụ thể, hàm mới này sẽ không truyền bá các lỗi cuối cùng được tạo ra sâu trong phép đệ quy, vì không có liên kết nào giữa trạng thái của các promise khác nhau. Sự bất tiện này có thể được giảm nhẹ bằng cách thêm một chút ghi nhật ký bổ sung bên trong hàm. Nhưng đôi khi chính hành vi mới đó có thể không phải là một lựa chọn. Vậy nên, một giải pháp khả dĩ bao gồm việc bao bọc hàm đệ quy bằng một hàm dựng `Promise`, chẳng hạn như trong đoạn mã mẫu sau:

```
function nonLeakingLoopWithErrors () {  
  return new Promise((resolve, reject) => {  
    (function internalLoop () {  
      delay(1)  
      .then(() => {  
        console.log(`Tick ${Date.now()}`)  
        internalLoop()  
      })  
      .catch(err => {  
        reject(err)  
      })  
    })()  
  })  
}
```

In this case, we still don't have any link between the promises created at the various stages of the recursion; however, the `Promise` returned by the `nonLeakingLoopWithErrors()` function will still reject if any asynchronous operation fails, no matter at what depth in the recursion that happens. A third solution makes use of `async/await`. In fact, with `async/await` we can simulate a recursive `Promise` chain with a simple infinite `while` loop, such as the following:

Trong trường hợp này, ta vẫn không có bất kỳ liên kết nào giữa các promise được tạo ra ở các giai đoạn khác nhau của phép đệ quy; tuy nhiên, `Promise` được trả về bởi hàm `nonLeakingLoopWithErrors()` vẫn sẽ reject nếu bất kỳ thao tác bất đồng bộ nào thất bại, bất kể điều đó xảy ra ở độ sâu nào trong phép đệ quy. Một giải pháp thứ ba sử dụng `async/await`. Thực vậy, với `async/await` ta có thể mô phỏng một chuỗi `Promise` đệ quy bằng một vòng lặp `while` vô hạn đơn giản, chẳng hạn như sau:

```
async function nonLeakingLoopAsync () {
```

```

while (true) {
  await delay(1)
  console.log(`Tick ${Date.now()}`)
}
}

```

In this function too, we preserve the behavior of the original recursive function, whereby any error thrown by the asynchronous task (in this case `delay()`) is propagated to the original function caller.

Trong hàm này cũng vậy, ta bảo toàn hành vi của hàm đệ quy ban đầu, theo đó bất kỳ lỗi nào ném ra bởi tác vụ bất đồng bộ (trong trường hợp này là `delay()`) đều được truyền bá tới bên gọi hàm ban đầu.

We should note that we would still have a memory leak if instead of a `while` loop, we chose to implement the `async/await` solution with an actual asynchronous recursive step, such as the following:

Ta nên lưu ý rằng ta vẫn sẽ có một rò rỉ bộ nhớ nếu thay vì một vòng lặp `while`, ta chọn hiện thực giải pháp `async/await` bằng một bước đệ quy bất đồng bộ thực sự, chẳng hạn như sau:

```

async function leakingLoopAsync () {
  await delay(1)
  console.log(`Tick ${Date.now()}`)
  return leakingLoopAsync()
}

```

The code above would still create an infinite chain of promises that never resolve and therefore it's still affected by the same memory leak issue of the equivalent promisebased implementation.

Đoạn mã trên vẫn sẽ tạo ra một chuỗi promise vô hạn không bao giờ resolve và do đó nó vẫn bị ảnh hưởng bởi cùng một vấn đề rò rỉ bộ nhớ của bản hiện thực tương đương dựa trên promise.

If you are interested in knowing more about the memory leak discussed in this section, you can check the related Node.js issue at nodejsdp.link/node-6673 or the related issue on the Promises/A+ GitHub repository at nodejsdp.link/promisesaplus-memleak.

Nếu bạn quan tâm tìm hiểu thêm về rò rỉ bộ nhớ được thảo luận trong phần này, bạn có thể kiểm tra issue Node.js liên quan tại nodejsdp.link/node-6673 hoặc issue liên quan trên kho GitHub của Promises/A+ tại nodejsdp.link/promisesaplus-memleak.

So, the next time you are building an infinite promise chain, remember to doublecheck if there are the conditions for creating a memory leak, as you learned in this section. If that's the case, you can apply one of the proposed solutions, making sure to choose the one that is best suited to your context.

Vậy nên, lần tới khi bạn đang xây dựng một chuỗi promise vô hạn, hãy nhớ kiểm tra kỹ xem có tồn tại các điều kiện để tạo ra một rò rỉ bộ nhớ hay không, như bạn đã học trong phần này. Nếu đúng vậy, bạn có thể áp dụng một trong các giải pháp được đề xuất, đảm bảo chọn giải pháp phù hợp nhất với bối cảnh của bạn.

Summary — Tóm tắt

In this chapter, we've learned how to use promises and `async/await` syntax to write asynchronous code that is more concise, cleaner, and easier to read.

Trong chương này, ta đã học cách dùng promise và cú pháp `async/await` để viết mã bất đồng bộ ngắn gọn hơn, sạch sẽ hơn, và dễ đọc hơn.

As we've seen, promises and `async/await` greatly simplify the serial execution flow, which is the most commonly used control flow. In fact, with `async/await`, writing a sequence of asynchronous operations is almost as easy as writing synchronous code. Running some asynchronous operations in parallel is also very easy thanks to the `Promise.all()` utility.

Như ta đã thấy, promise và `async/await` đơn giản hóa đáng kể luồng thực thi tuần tự, vốn là luồng điều khiển được dùng phổ biến nhất. Thực vậy, với `async/await`, viết một chuỗi các thao tác bất đồng bộ gần như dễ dàng như viết mã đồng bộ. Chạy một số thao tác bất đồng bộ song song cũng rất dễ nhờ tiện ích `Promise.all()`.

But the advantages of using promises and `async/await` don't stop here. We've learned that they provide a transparent shield against tricky situations such as code with mixed synchronous/asynchronous behavior (a.k.a. *Zalgo*, which we discussed in Chapter 3, *Callbacks and Events*). On top of that, error management with promises and `async/await` is much more intuitive and leaves less room for mistakes (such as forgetting to forward errors, which is a serious source of bugs in code using *callbacks*).

Nhưng lợi thế của việc dùng promise và `async/await` không dừng lại ở đây. Ta đã học rằng chúng cung cấp một lá chắn trong suốt chống lại những tình huống hóc búa như mã có hành vi đồng bộ/bất đồng bộ trộn lẫn (còn gọi là *Zalgo*, mà ta đã thảo luận ở Chương 3, *Callbacks and Events*). Hơn nữa, việc quản lý lỗi với promise và `async/await` trực quan hơn nhiều và ít dễ chỗ cho sai sót hơn (chẳng hạn quên chuyển tiếp lỗi, vốn là một nguồn lỗi nghiêm trọng trong mã dùng *callback*).

In terms of patterns and techniques, we should definitely keep in mind the chain of promises (to run tasks in series), *promisification*, and the *Producer-Consumer* pattern. Also, pay attention when using `Array.forEach()` with `async/await` (you are probably doing it wrong) and keep in mind the difference between a simple `return` and `return await` in *async* functions.

Về mặt các mẫu và kỹ thuật, ta chắc chắn nên ghi nhớ chuỗi promise (để chạy các tác vụ theo chuỗi), *promisification*, và mẫu *Producer-Consumer*. Ngoài ra, hãy chú ý khi dùng `Array.forEach()` với `async/await` (bạn có lẽ đang làm sai) và ghi nhớ khác biệt giữa một `return` đơn giản và `return await` trong các hàm *async*.

Callbacks are still widely used in the Node.js and JavaScript world. We find them in legacy APIs, in code that interacts with native libraries, or when there is the need to micro-optimize particular routines. That's why they are still relevant to us, Node.js developers; however, for most of our day-to-day programming tasks, promises and `async/await` are a huge step ahead compared to *callbacks* and therefore they are now the *de facto* standard for dealing with asynchronous code in Node.js. That's why we will be using promises and `async/await` throughout the rest of the book too to write our asynchronous code.

Callback vẫn được dùng rộng rãi trong thế giới Node.js và JavaScript. Ta tìm thấy chúng trong các API cũ (*legacy*), trong mã tương tác với các thư viện gốc (*native*), hoặc khi có nhu cầu vi tối ưu hóa những routine cụ thể. Đó là lý do chúng vẫn còn liên quan tới chúng ta, các lập trình viên Node.js; tuy nhiên, đối với hầu hết các tác vụ lập trình hằng ngày, promise và `async/await` là một bước tiến lớn so với *callback* và do đó giờ chúng là chuẩn *de facto* để xử lý mã bất đồng bộ trong Node.js. Đó là lý do ta cũng sẽ dùng promise và `async/await` xuyên suốt phần còn lại của cuốn sách để viết mã bất đồng bộ của mình.

In the next chapter, we will explore another fascinating topic relative to asynchronous code execution, which is also another fundamental building block in the whole Node.js ecosystem, that

is, streams.

Ở chương tiếp theo, ta sẽ khám phá một chủ đề hấp dẫn khác liên quan tới việc thực thi mã bất đồng bộ, cũng là một khối xây dựng nền tảng khác trong toàn bộ hệ sinh thái Node.js, đó là stream.

Exercises — Bài tập

- 5.1 Dissecting `Promise.all()`: Implement your own version of `Promise.all()` leveraging promises, `async/await`, or a combination of the two. The function must be functionally equivalent to its original counterpart.
- 5.2 TaskQueue with promises: Migrate the TaskQueue class internals from promises to `async/await` where possible. Hint: you won't be able to use `async/await` everywhere.
- 5.3 Producer-consumer with promises: Update the TaskQueuePC class internal methods so that they use just promises, removing any use of the `async/await` syntax. Hint: the infinite loop must become an asynchronous recursion. Beware of the recursive Promise resolution memory leak!
- 5.4 An asynchronous `map()`: Implement a parallel asynchronous version of `Array.map()` that supports promises and a concurrency limit. The function should not directly leverage the TaskQueue or TaskQueuePC classes we presented in this chapter, but it can use the underlying patterns. The function, which we will define as `mapAsync(iterable, callback, concurrency)`, will accept the following as inputs:
 - An iterable, such as an array.
 - A callback, which will receive as the input each item of the iterable (exactly like in the original `Array.map()`) and can return either a Promise or a simple value.
 - A concurrency, which defines how many items in the iterable can be processed by callback in parallel at each given time.
- 5.1 Mổ xẻ `Promise.all()`: Hiện thực phiên bản của riêng bạn của `Promise.all()` tận dụng promise, `async/await`, hoặc kết hợp cả hai. Hàm phải tương đương về mặt chức năng với bản gốc của nó.
- 5.2 TaskQueue với promise: Di trú nội bộ của lớp TaskQueue từ promise sang `async/await` ở những chỗ có thể. Gợi ý: bạn sẽ không thể dùng `async/await` ở mọi nơi.
- 5.3 Producer-consumer với promise: Cập nhật các phương thức nội bộ của lớp TaskQueuePC sao cho chúng chỉ dùng promise, loại bỏ mọi việc dùng cú pháp `async/await`. Gợi ý: vòng lặp vô hạn phải trở thành một phép đệ quy bất đồng bộ. Hãy cẩn thận với rò rỉ bộ nhớ resolution Promise đệ quy!
- 5.4 Một `map()` bất đồng bộ: Hiện thực một phiên bản bất đồng bộ song song của `Array.map()` hỗ trợ promise và một giới hạn đồng thời. Hàm này không nên trực tiếp tận dụng các lớp TaskQueue hoặc TaskQueuePC mà ta đã trình bày trong chương này, nhưng nó có thể dùng các mẫu nền tảng. Hàm này, mà ta sẽ định nghĩa là `mapAsync(iterable, callback, concurrency)`, sẽ nhận những thứ sau làm đầu vào:
 - Một iterable, chẳng hạn một mảng.
 - Một callback, sẽ nhận làm đầu vào mỗi phần tử của iterable (chính xác như trong `Array.map()` gốc) và có thể trả về hoặc một Promise hoặc một giá trị đơn giản.
 - Một concurrency, định nghĩa bao nhiêu phần tử trong iterable có thể được xử lý bởi callback song song tại mỗi thời điểm.

Chapter 6: Coding with Streams —

Chương 6: Lập trình với Stream

Streams are one of the most important components and patterns of Node.js. There is a motto in the community that goes, “**stream** all the things!”, and this alone should be enough to describe the role of streams in Node.js. Dominic Tarr, a top contributor to the Node.js community, defines streams as “Node’s best and most misunderstood idea.” There are different reasons that make Node.js streams so attractive; again, it’s not just related to technical properties, such as performance or efficiency, but it’s more about their elegance and the way they fit perfectly into the Node.js philosophy.

Stream là một trong những thành phần và mẫu thiết kế (design pattern) quan trọng nhất của Node.js. Trong cộng đồng có một câu khẩu hiệu “stream all the things!” (hãy stream mọi thứ!), và chỉ riêng câu này đã đủ mô tả vai trò của stream trong Node.js. Dominic Tarr, một trong những người đóng góp hàng đầu cho cộng đồng Node.js, định nghĩa stream là “ý tưởng tuyệt vời nhất và bị hiểu lầm nhiều nhất của Node.” Có nhiều lý do khiến stream của Node.js trở nên hấp dẫn đến vậy; một lần nữa, điều đó không chỉ liên quan tới các đặc tính kỹ thuật như hiệu năng hay hiệu quả, mà còn nhiều hơn thế ở sự thanh lịch và cách chúng ăn khớp hoàn hảo với triết lý của Node.js.

This chapter aims to provide a complete understanding of Node.js streams. The first half of this chapter serves as an introduction to the main ideas, the terminology, and the libraries behind Node.js streams. In the second half, we will cover more advanced topics and, most importantly, we will explore useful streaming patterns that can make your code more elegant and effective in many circumstances.

Chương này nhằm cung cấp cho bạn hiểu biết đầy đủ về stream của Node.js. Nửa đầu chương đóng vai trò giới thiệu các ý tưởng chính, thuật ngữ, và các thư viện đứng sau stream của Node.js. Trong nửa sau, chúng ta sẽ bàn tới các chủ đề nâng cao hơn và, quan trọng nhất, chúng ta sẽ khám phá những streaming pattern hữu ích có thể giúp mã của bạn trở nên thanh lịch và hiệu quả hơn trong nhiều tình huống.

In this chapter, you will learn about the following topics:

Trong chương này, bạn sẽ tìm hiểu về các chủ đề sau:

- Why streams are so important in Node.js
- Understanding, using, and creating streams
- Streams as a programming paradigm: leveraging their power in many different contexts and not just for I/O
- Streaming patterns and connecting streams together in different configurations
 - Vì sao stream lại quan trọng đến vậy trong Node.js
 - Hiểu, sử dụng và tạo stream

- Stream như một mô hình lập trình: tận dụng sức mạnh của chúng trong nhiều ngữ cảnh khác nhau chứ không chỉ cho I/O
- Các streaming pattern và cách kết nối các stream với nhau theo những cấu hình khác nhau

Without further ado, let's discover together why streams are one of the cornerstones of Node.js.

Không dài dòng thêm nữa, hãy cùng nhau khám phá vì sao stream là một trong những nền tảng cốt lõi của Node.js.

Discovering the importance of streams — Khám phá tầm quan trọng của stream

In an event-based platform such as Node.js, the most efficient way to handle I/O is in real time, consuming the input as soon as it is available and sending the output as soon as the application produces it.

Trên một nền tảng dựa trên sự kiện như Node.js, cách xử lý I/O hiệu quả nhất là theo thời gian thực, tiêu thụ đầu vào ngay khi nó có sẵn và gửi đầu ra ngay khi ứng dụng tạo ra nó.

In this section, we will give you an initial introduction to Node.js streams and their strengths. Please bear in mind that this is only an overview, as a more detailed analysis on how to use and compose streams will follow later in this chapter.

Trong phần này, chúng tôi sẽ giới thiệu sơ bộ về stream của Node.js và các điểm mạnh của chúng. Xin lưu ý rằng đây chỉ là một cái nhìn tổng quan, vì phần phân tích chi tiết hơn về cách sử dụng và kết hợp stream sẽ được trình bày sau trong chương này.

Buffering versus streaming — Buffering so với streaming

Almost all the asynchronous APIs that we've seen so far in this book work using **buffer** mode. For an input operation, buffer mode causes all the data coming from a resource to be collected into a buffer until the operation is completed; it is then passed back to the caller as one single blob of data. The following diagram shows a visual example of this paradigm:

Hầu hết tất cả các API bất đồng bộ mà chúng ta đã thấy cho đến giờ trong cuốn sách này đều hoạt động theo chế độ buffer. Đối với một thao tác đầu vào, chế độ buffer khiến toàn bộ dữ liệu đến từ một tài nguyên được gom vào một buffer cho đến khi thao tác hoàn tất; sau đó nó được trả lại cho bên gọi như một khối dữ liệu duy nhất. Sơ đồ sau minh họa trực quan cho mô hình này:

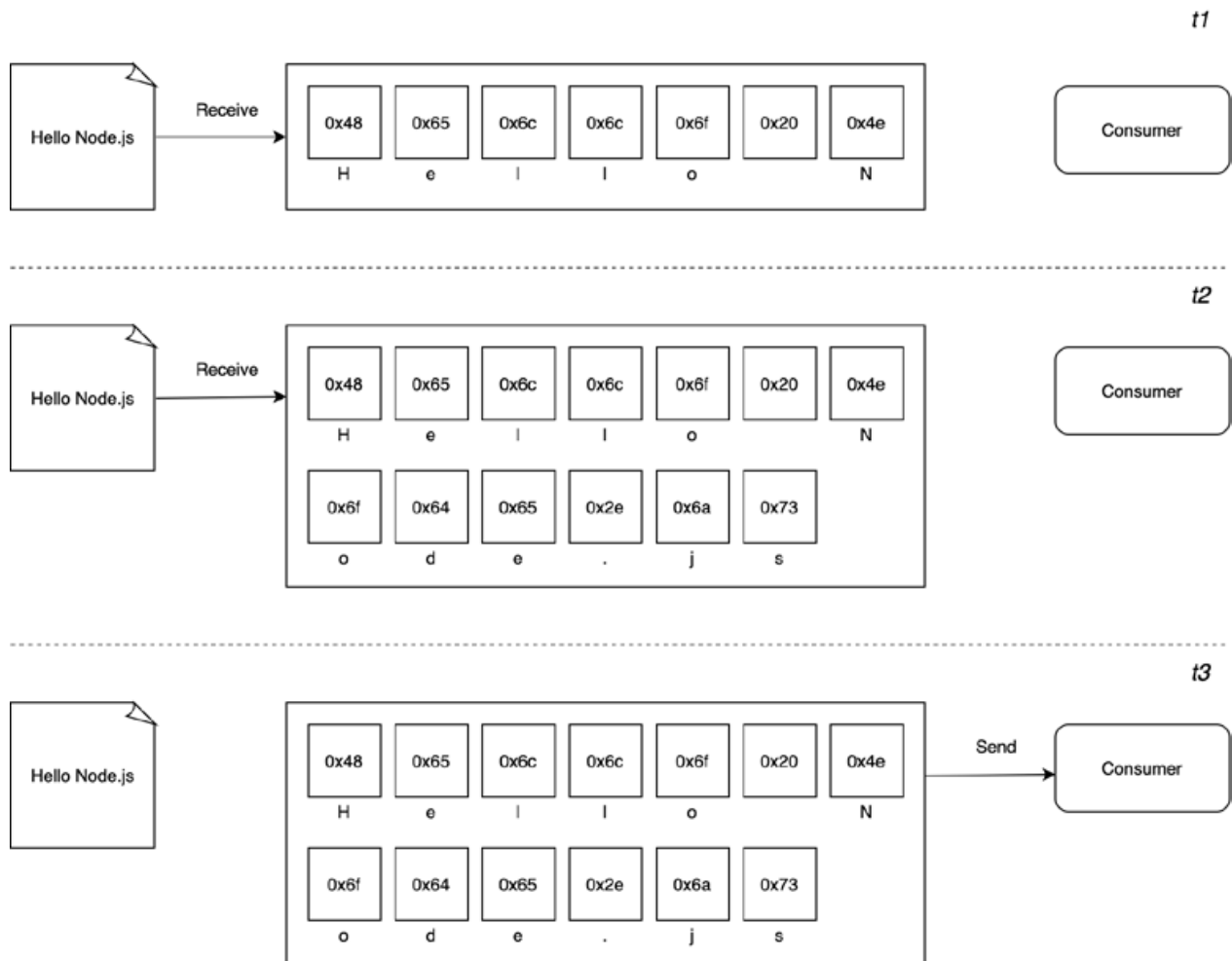


Figure 6.1: Buffering In Figure 6.1, we can see that, at time *t1*, some data is received from the resource and saved into the buffer. At time *t2*, another data chunk is received—the final one— which completes the read operation, so that, at *t3*, the entire buffer is sent to the consumer.

Hình 6.1: Buffering Trong Hình 6.1, ta có thể thấy rằng, tại thời điểm *t1*, một phần dữ liệu được nhận từ tài nguyên và lưu vào buffer. Tại thời điểm *t2*, một chunk dữ liệu khác được nhận—chunk cuối cùng—hoàn tất thao tác đọc, để rồi tại *t3*, toàn bộ buffer được gửi tới bên tiêu thụ (consumer).

On the other side, streams allow us to process the data as soon as it arrives from the resource. This is shown in the following diagram:

Ở phía bên kia, stream cho phép chúng ta xử lý dữ liệu ngay khi nó đến từ tài nguyên. Điều này được minh họa trong sơ đồ sau:

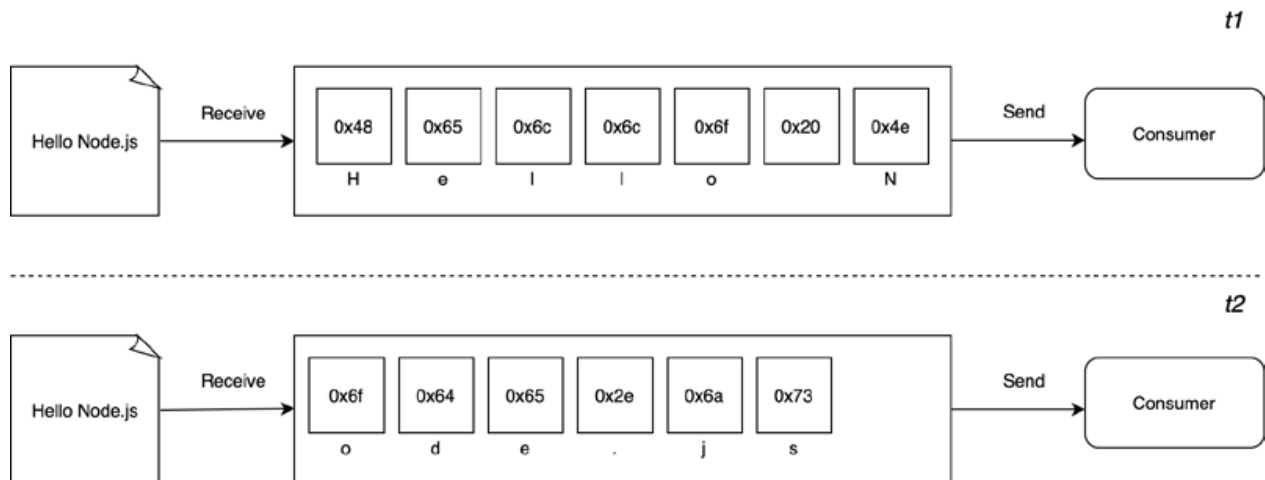


Figure 6.2: Streaming

Hình 6.2: Streaming

This time, Figure 6.2 shows you that as soon as each new chunk of data is received from the resource, it is immediately passed to the consumer, who now has the chance to process it straight away, without waiting for all the data to be collected in the buffer.

Lần này, Hình 6.2 cho thấy rằng ngay khi mỗi chunk dữ liệu mới được nhận từ tài nguyên, nó lập tức được chuyển tới consumer; người giờ đây có cơ hội xử lý nó ngay lập tức, mà không phải chờ toàn bộ dữ liệu được gom vào buffer.

But what are the differences between these two approaches? Purely from an efficiency perspective, streams can be more efficient in terms of both space (memory usage) and time (computation clock time). However, Node.js streams have another important advantage: composability. Let's now see what impact these properties have on the way we design and write our applications.

Nhưng đâu là sự khác biệt giữa hai cách tiếp cận này? Thuần túy từ góc độ hiệu quả, stream có thể hiệu quả hơn xét cả về không gian (mức sử dụng bộ nhớ) lẫn thời gian (thời gian tính toán theo đồng hồ). Tuy nhiên, stream của Node.js còn có một ưu điểm quan trọng khác: khả năng kết hợp (composability). Bây giờ hãy xem những đặc tính này tác động thế nào tới cách chúng ta thiết kế và viết ứng dụng.

Spatial efficiency — Hiệu quả về không gian

First of all, streams allow us to do things that would not be possible by buffering data and processing it all at once. For example, consider the case in which we have to read a very big file, let's say, in the order of hundreds of megabytes or even gigabytes. Clearly, using an **API** that returns a big buffer when the file is completely read is not a good idea. Imagine reading a few of these big files concurrently; our application would easily run out of memory. Besides that, buffers in V8 are limited in size. You cannot allocate more than a few gigabytes of data, so we may hit a wall way before running out of physical memory. The actual maximum size of a buffer changes across platforms and versions of Node.js. If you are curious to find out what's the limit in bytes in a given platform, you can run this code:

Trước hết, stream cho phép chúng ta làm những việc không thể thực hiện được nếu buffer toàn bộ dữ liệu rồi xử lý tất cả cùng lúc. Ví dụ, hãy xét trường hợp ta phải đọc một tập tin rất lớn, chẳng hạn cỡ hàng trăm megabyte hoặc thậm chí gigabyte. Rõ ràng, dùng một API trả về một buffer lớn khi tập tin được đọc hoàn toàn không phải là ý hay. Hãy tưởng tượng

việc đọc đồng thời vài tập tin lớn như vậy; ứng dụng của chúng ta sẽ dễ dàng cạn kiệt bộ nhớ. Bên cạnh đó, buffer trong V8 bị giới hạn về kích thước. Bạn không thể cấp phát quá vài gigabyte dữ liệu, nên ta có thể chậm tương rất lâu trước khi cạn kiệt bộ nhớ vật lý. Kích thước tối đa thực tế của một buffer thay đổi tùy theo nền tảng và phiên bản Node.js. Nếu bạn tò mò muốn biết giới hạn theo byte trên một nền tảng nhất định, bạn có thể chạy đoạn mã sau:

```
import buffer from 'buffer'
console.log(buffer.constants.MAX_LENGTH)
```

Gzipping using a buffered API — Gzip bằng API dựa trên buffer

To make a concrete example, let's consider a simple command-line application that compresses a file using the GZIP format. Using a buffered API, such an application will look like the following in Node.js (error handling is omitted for brevity):

Để có một ví dụ cụ thể, hãy xét một ứng dụng dòng lệnh đơn giản nén một tập tin bằng định dạng GZIP. Dùng một API dựa trên buffer, ứng dụng như vậy sẽ trông như sau trong Node.js (phần xử lý lỗi được lược bỏ cho gọn):

```
import { promises as fs } from 'fs'
import { gzip } from 'zlib'
import { promisify } from 'util'
const gzipPromise = promisify(gzip)
const filename = process.argv[2]
async function main () {
  const data = await fs.readFile(filename)
  const gzippedData = await gzipPromise(data)
  await fs.writeFile(`${filename}.gz`, gzippedData)
  console.log('File successfully compressed')
}
main()
```

Now, we can try to put the preceding code in a file named `gzip-buffer.js` and then run it with the following command:

Bây giờ, ta có thể thử đặt đoạn mã trên vào một tập tin tên `gzip-buffer.js` rồi chạy nó với lệnh sau:

```
node gzip-buffer.js <path to file>
```

If we choose a file that is big enough (for instance, about 8 GB), we will most likely receive an error message saying that the file that we are trying to read is bigger than the maximum allowed buffer size:

Nếu ta chọn một tập tin đủ lớn (chẳng hạn, khoảng 8 GB), nhiều khả năng ta sẽ nhận được một thông báo lỗi nói rằng tập tin ta đang cố đọc lớn hơn kích thước buffer tối đa cho phép:

```
RangeError [ERR_FS_FILE_TOO_LARGE]: File size (8130792448) is greater
than possible Buffer: 2147483647 bytes
```

That's exactly what we expected, and it's a symptom of the fact that we are using the wrong approach.

Đó chính xác là điều ta dự đoán, và nó là một triệu chứng cho thấy ta đang dùng cách tiếp cận sai.

Gzipping using streams — Gzip bằng stream

The simplest way we have to fix our Gzip application and make it work with big files is to use a streaming API. Let's see how this can be achieved. Let's write a new **module** with the following code:

Cách đơn giản nhất để sửa ứng dụng Gzip của chúng ta và khiến nó hoạt động được với tập tin lớn là dùng một API kiểu streaming. Hãy xem có thể đạt được điều này như thế nào. Hãy viết một module mới với đoạn mã sau:

```
// gzip-stream.js
import { createReadStream, createWriteStream } from 'fs'
import { createGzip } from 'zlib'
const filename = process.argv[2]
createReadStream(filename)
  .pipe(createGzip())
  .pipe(createWriteStream(`${filename}.gz`))
  .on('finish', () => console.log('File successfully compressed'))
```

“Is that it?” you may ask. Yes! As we said, streams are amazing because of their interface and composability, thus allowing clean, elegant, and concise code. We will see this in a while in more detail, but for now, the important thing to realize is that the program will run smoothly against files of any size and with constant memory utilization. Try it yourself (but consider that compressing a big file may take a while).

“Chỉ vậy thôi sao?” bạn có thể hỏi. Đúng vậy! Như đã nói, stream tuyệt vời nhờ giao diện và khả năng kết hợp của chúng, từ đó cho phép viết mã sạch, thanh lịch và súc tích. Chúng ta sẽ sớm thấy điều này chi tiết hơn, nhưng hiện giờ, điều quan trọng cần nhận ra là chương trình sẽ chạy mượt mà với các tập tin ở mọi kích thước và với mức sử dụng bộ nhớ không đổi. Hãy tự thử (nhưng hãy lưu ý rằng nén một tập tin lớn có thể mất một lúc).

Note that, in the previous example, we omitted error handling for brevity. We will discuss the nuances of proper error handling with streams later in this chapter. Until then, be aware that most examples will be lacking proper error handling.

Lưu ý rằng, trong ví dụ trước, ta đã lược bỏ phần xử lý lỗi cho gọn. Chúng ta sẽ bàn về những điểm tinh tế của việc xử lý lỗi đúng cách với stream sau trong chương này. Cho tới lúc đó, hãy nhớ rằng hầu hết các ví dụ sẽ thiếu phần xử lý lỗi đúng cách.

Time efficiency — Hiệu quả về thời gian

Let's now consider the case of an application that compresses a file and uploads it to a remote HTTP server, which, in turn, decompresses it and saves it on the filesystem. If the client component of our application was implemented using a buffered API, the upload would start only when the entire file had been read and compressed. On the other hand, the decompression would start on the server only when all the data had been received. A better solution to achieve the same result involves the use of streams. On the client machine, streams allow us to compress and send the data chunks as soon as they are read from the filesystem, whereas on the server, they allow us to decompress every chunk as soon as it is received from the remote peer. Let's demonstrate this by building the application that we mentioned earlier, starting from the server side. Let's create a module named `gzip-receive.js` containing the following code:

Bây giờ hãy xét trường hợp một ứng dụng nén một tập tin và tải nó lên một máy chủ HTTP từ xa, máy chủ này đến lượt mình giải nén nó và lưu vào hệ thống tập tin. Nếu thành phần

client của ứng dụng được hiện thực bằng một API dựa trên buffer, việc tải lên sẽ chỉ bắt đầu khi toàn bộ tập tin đã được đọc và nén. Mặt khác, việc giải nén sẽ chỉ bắt đầu trên máy chủ khi toàn bộ dữ liệu đã được nhận. Một giải pháp tốt hơn để đạt cùng kết quả là dùng stream. Trên máy client, stream cho phép ta nén và gửi các chunk dữ liệu ngay khi chúng được đọc từ hệ thống tập tin, trong khi trên máy chủ, chúng cho phép ta giải nén mỗi chunk ngay khi nó được nhận từ peer ở xa. Hãy minh họa điều này bằng cách xây dựng ứng dụng mà ta đã nhắc tới ở trên, bắt đầu từ phía máy chủ. Hãy tạo một module tên `gzip-receive.js` chứa đoạn mã sau:

```
import { createServer } from 'http'
import { createWriteStream } from 'fs'
import { createGunzip } from 'zlib'
import { basename, join } from 'path'
const server = createServer((req, res) => {
  const filename = basename(req.headers['x-filename'])
  const destFilename = join('received_files', filename)
  console.log(`File request received: ${filename}`)
  req
    .pipe(createGunzip())
    .pipe(createWriteStream(destFilename))
    .on('finish', () => {
      res.writeHead(201, { 'Content-Type': 'text/plain' })
      res.end('OK\n')
      console.log(`File saved: ${destFilename}`)
    })
})
server.listen(3000, () => console.log('Listening on http://localhost:3000'))
```

In the preceding example, `req` is a stream object that is used by the server to receive the **request** data in chunks from the network. Thanks to Node.js streams, every chunk of data is decompressed and saved to disk as soon as it is received.

Trong ví dụ trên, `req` là một đối tượng stream được máy chủ dùng để nhận dữ liệu request theo từng chunk từ mạng. Nhờ stream của Node.js, mỗi chunk dữ liệu được giải nén và lưu xuống đĩa ngay khi nó được nhận.

You might have noticed that, in our server application, we are using `basename()` to remove any possible path from the name of the received file. This is a security best practice as we want to make sure that the received file is saved exactly within our `received_files` folder. Without `basename()`, a malicious user could craft a request that could effectively override system files and inject malicious code into the server machine. Imagine, for instance, what happens if `filename` is set to `/usr/bin/node`? In such a case, the attacker could effectively replace our Node.js interpreter with any arbitrary file. The client side of our application will go into a module named `gzip-send.js`, and it looks as follows:

Bạn có thể đã để ý rằng, trong ứng dụng máy chủ, ta dùng `basename()` để loại bỏ mọi đường dẫn có thể có khỏi tên của tập tin nhận được. Đây là một thực hành bảo mật tốt vì ta muốn chắc chắn rằng tập tin nhận được được lưu đúng trong thư mục `received_files` của ta. Không có `basename()`, một người dùng độc hại có thể tạo một request có thể ghi đè các tập tin hệ thống và tiêm mã độc vào máy chủ. Hãy tưởng tượng, chẳng hạn, điều gì xảy ra nếu `filename` được đặt thành `/usr/bin/node`? Trong trường hợp đó, kẻ tấn công có thể thay thế trình thông dịch Node.js của ta bằng một tập tin tùy ý bất kỳ. Phía

client của ứng dụng sẽ nằm trong một module tên `gzip-send.js`, và nó trông như sau:

```
import { request } from 'http'
import { createGzip } from 'zlib'
import { createReadStream } from 'fs'
import { basename } from 'path'
const filename = process.argv[2]
const serverHost = process.argv[3]
const httpRequestOptions = {
  hostname: serverHost,
  port: 3000,
  path: '/',
  method: 'PUT',
  headers: {
    'Content-Type': 'application/octet-stream',
    'Content-Encoding': 'gzip',
    'X-Filename': basename(filename)
  }
}
const req = request(httpRequestOptions, (res) => {
  console.log(`Server response: ${res.statusCode}`)
})
createReadStream(filename)
  .pipe(createGzip())
  .pipe(req)
  .on('finish', () => {
    console.log('File successfully sent')
  })
```

In the preceding code, we are again using streams to read the data from the file, and then compressing and sending each chunk as soon as it is read from the filesystem.

Trong đoạn mã trên, ta lại dùng stream để đọc dữ liệu từ tập tin, rồi nén và gửi từng chunk ngay khi nó được đọc từ hệ thống tập tin.

Now, to try out the application, let's first start the server using the following command:

Bây giờ, để thử ứng dụng, trước tiên hãy khởi động máy chủ bằng lệnh sau:

```
node gzip-receive.js
```

Then, we can launch the client by specifying the file to send and the address of the server (for example, localhost):

Sau đó, ta có thể khởi chạy client bằng cách chỉ định tập tin cần gửi và địa chỉ của máy chủ (ví dụ, localhost):

```
node gzip-send.js <path to file> localhost
```

If we choose a file big enough, we can appreciate how the data flows from the client to the server. But why exactly is this paradigm—where we have flowing data—more efficient compared to using a buffered API? Figure 6.3 should make this concept easier to grasp:

Nếu ta chọn một tập tin đủ lớn, ta có thể cảm nhận được cách dữ liệu chảy từ client tới máy chủ. Nhưng vì sao chính xác mô hình này—nơi ta có dữ liệu chảy liên tục—lại hiệu

quả hơn so với việc dùng một API dựa trên buffer? Hình 6.3 sẽ giúp khái niệm này dễ nắm bắt hơn:

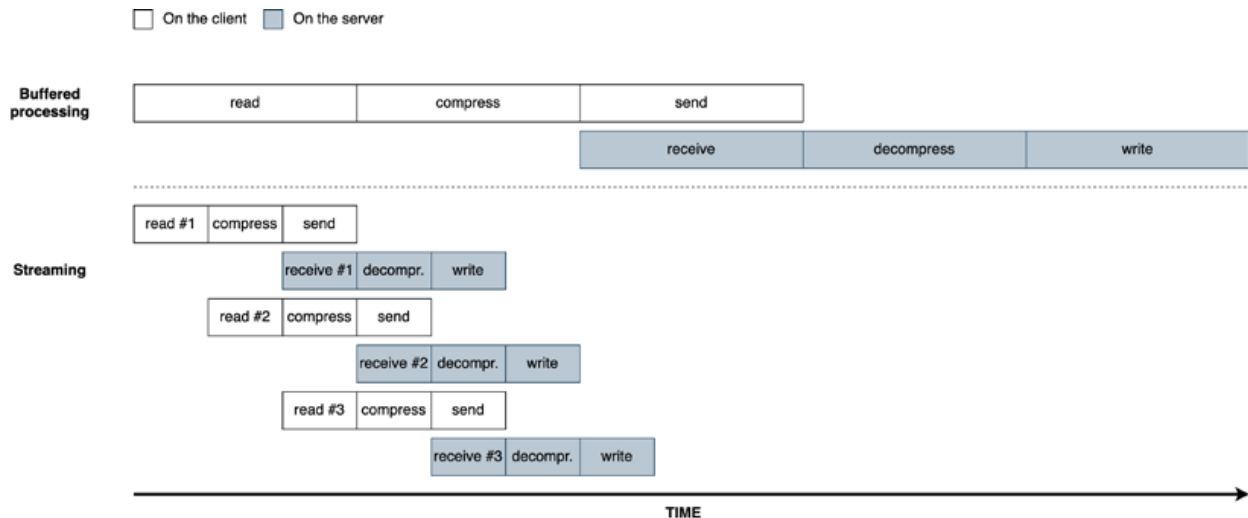


Figure 6.3: Buffering and streaming compared

Hình 6.3: So sánh buffering và streaming

When a file is processed, it goes through a number of sequential stages:

Khi một tập tin được xử lý, nó đi qua một số giai đoạn tuần tự:

1. [Client] Read from the filesystem
 2. [Client] Compress the data
 3. [Client] Send it to the server
 4. [Server] Receive from the client
 5. [Server] Decompress the data
 6. [Server] Write the data to disk
-
1. [Client] Đọc từ hệ thống tập tin
 2. [Client] Nén dữ liệu
 3. [Client] Gửi nó tới máy chủ
 4. [Server] Nhận từ client
 5. [Server] Giải nén dữ liệu
 6. [Server] Ghi dữ liệu xuống đĩa

To complete the processing, we have to go through each stage like in an assembly line, in sequence, until the end. In Figure 6.3, we can see that, using a buffered API, the process is entirely sequential. To compress the data, we first have to wait for the entire file to be read, then, to send the data, we have to wait for the entire file to be both read and compressed, and so on. Using streams, the assembly line is kicked off as soon as we receive the first chunk of data, without waiting for the entire file to be read. But more amazingly, when the next chunk of data is available, there is no need to wait for the previous set of tasks to be completed; instead, another assembly line is launched in parallel. This works perfectly because each task that we execute is asynchronous, so it can be parallelized by Node.js. The only constraint is that the order in which the chunks arrive in each stage must be preserved. The internal implementation of Node.js streams takes care of maintaining the order for us.

Để hoàn tất việc xử lý, ta phải đi qua từng giai đoạn như trên một dây chuyền lắp ráp, theo trình tự, cho đến cuối. Trong Hình 6.3, ta có thể thấy rằng, dùng một API dựa trên buffer,

quá trình hoàn toàn tuần tự. Để nén dữ liệu, trước hết ta phải chờ toàn bộ tập tin được đọc xong, rồi để gửi dữ liệu, ta phải chờ toàn bộ tập tin vừa được đọc vừa được nén xong, và cứ thế. Dùng stream, dây chuyền lắp ráp được khởi động ngay khi ta nhận được chunk dữ liệu đầu tiên, mà không phải chờ toàn bộ tập tin được đọc xong. Nhưng kỳ diệu hơn, khi chunk dữ liệu tiếp theo có sẵn, không cần phải chờ tập tác vụ trước hoàn thành; thay vào đó, một dây chuyền lắp ráp khác được khởi chạy song song. Điều này hoạt động hoàn hảo vì mỗi tác vụ ta thực thi đều bất đồng bộ, nên nó có thể được Node.js song song hóa. Ràng buộc duy nhất là thứ tự các chunk đến ở mỗi giai đoạn phải được giữ nguyên. Phần hiện thực bên trong của stream Node.js lo việc duy trì thứ tự này thay cho ta.

As we can see from Figure 6.3, the result of using streams is that the entire process takes less time, because we waste no time waiting for all the data to be read and processed all at once.

Như ta có thể thấy từ Hình 6.3, kết quả của việc dùng stream là toàn bộ quá trình tốn ít thời gian hơn, vì ta không lãng phí thời gian chờ toàn bộ dữ liệu được đọc và xử lý tất cả cùng một lúc.

Composability — Khả năng kết hợp

The code we have seen so far has already given us an overview of how streams can be composed thanks to the `pipe()` method, which allows us to connect the different processing units, each being responsible for one single functionality, in perfect Node.js style. This is possible because streams have a uniform interface, and they can understand each other in terms of API. The only prerequisite is that the next stream in the pipeline has to support the data type produced by the previous stream, which can be either binary, text, or even objects, as we will see later in this chapter.

Đoạn mã ta đã thấy đến giờ đã cho ta một cái nhìn tổng quan về cách stream có thể được kết hợp nhờ phương thức `pipe()`, cho phép ta kết nối các đơn vị xử lý khác nhau, mỗi đơn vị chịu trách nhiệm cho một chức năng duy nhất, đúng phong cách Node.js. Điều này khả thi vì stream có một giao diện đồng nhất, và chúng có thể “hiểu nhau” về mặt API. Điều kiện tiên quyết duy nhất là stream kế tiếp trong pipeline phải hỗ trợ kiểu dữ liệu do stream trước tạo ra, vốn có thể là nhị phân, văn bản, hoặc thậm chí là object, như ta sẽ thấy sau trong chương này.

To take a look at another demonstration of the power of this property, we can try to add an encryption layer to the `gzip-send/gzip-receive` application that we built previously.

Để xem thêm một minh chứng nữa về sức mạnh của đặc tính này, ta có thể thử thêm một lớp mã hóa (encryption) vào ứng dụng `gzip-send/gzip-receive` mà ta đã xây dựng trước đó.

In order to do this, we will need to apply some small changes to both our client and server.

Để làm điều này, ta sẽ cần áp dụng một số thay đổi nhỏ cho cả client và server.

Adding client-side encryption — Thêm mã hóa phía client

Let's start with the client:

Hãy bắt đầu với client:

```
// ...
import { createCipheriv, randomBytes } from 'crypto' // (1)
const filename = process.argv[2]
const serverHost = process.argv[3]
```

```
const secret = Buffer.from(process.argv[4], 'hex') // (2)
const iv = randomBytes(16) // (3)
// ...
```

Let's review what we changed here:

Hãy điểm lại những gì ta đã thay đổi ở đây:

1. First of all, we import the `createCipheriv()` Transform stream and the `randomBytes()` function from the `crypto` module.
2. We get the server's encryption secret from the command line. We expect the string to be passed as a hexadecimal string, so we read this value and load it in memory using a buffer set to hex mode.
3. Finally, we generate a random sequence of bytes that we will be using as an initialization vector for the file encryption.
 1. Trước hết, ta import Transform stream `createCipheriv()` và hàm `randomBytes()` từ module `crypto`.
 2. Ta lấy khóa mã hóa bí mật của máy chủ từ dòng lệnh. Ta kỳ vọng chuỗi được truyền vào dưới dạng chuỗi thập lục phân (hexadecimal), nên ta đọc giá trị này và nạp nó vào bộ nhớ bằng một buffer ở chế độ hex.
 3. Cuối cùng, ta tạo một dãy byte ngẫu nhiên mà ta sẽ dùng làm vector khởi tạo (initialization vector) cho việc mã hóa tập tin.

Now, we can update the piece of code responsible for creating the HTTP request:

Bây giờ, ta có thể cập nhật đoạn mã chịu trách nhiệm tạo request HTTP:

```
const http RequestOptions = {
  hostname: serverHost,
  headers: {
    'Content-Type': 'application/octet-stream',
    'Content-Encoding': 'gzip',
    'X-Filename': basename(filename),
    'X-Initialization-Vector': iv.toString('hex') // (1)
  }
}
// ...
const req = request(http RequestOptions, (res) => {
  console.log(`Server response: ${res.statusCode}`)
})
createReadStream(filename)
  .pipe(createGzip())
  .pipe(createCipheriv('aes192', secret, iv)) // (2)
  .pipe(req)
// ...
```

The main changes here are:

Những thay đổi chính ở đây là:

1. We pass the initialization vector to the server as an HTTP header.
2. We encrypt the data, just after the Gzip phase.
 1. Ta truyền vector khởi tạo tới máy chủ dưới dạng một header HTTP.
 2. Ta mã hóa dữ liệu, ngay sau giai đoạn Gzip.

That's all for the client side.

Vậy là xong cho phía client.

Adding server-side decryption — Thêm giải mã phía server

Let's now **refactor** the server. The first thing that we need to do is import some utility functions from the core crypto module, which we can use to generate a random encryption key (the secret):

Bây giờ hãy tái cấu trúc (refactor) máy chủ. Việc đầu tiên ta cần làm là import một số hàm tiện ích từ module lõi crypto, mà ta có thể dùng để tạo một khóa mã hóa ngẫu nhiên (khóa bí mật):

```
// ...
import { createDecipheriv, randomBytes } from 'crypto'
const secret = randomBytes(24)
console.log(`Generated secret: ${secret.toString('hex')}`)
```

The generated secret is printed to the console as a hex string so that we can share that with our clients.

Khóa bí mật được tạo ra được in ra console dưới dạng chuỗi hex để ta có thể chia sẻ nó với các client của mình.

Now, we need to update the file reception logic:

Bây giờ, ta cần cập nhật logic nhận tập tin:

```
const server = createServer((req, res) => {
  const filename = basename(req.headers['x-filename'])
  const iv = Buffer.from(
    req.headers['x-initialization-vector'], 'hex') // (1)
  const destFilename = join('received_files', filename)
  console.log(`File request received: ${filename}`)
  req
    .pipe(createDecipheriv('aes192', secret, iv)) // (2)
    .pipe(createGunzip())
    .pipe(createWriteStream(destFilename))
  // ...
})
```

Here, we are applying two changes:

Ở đây, ta áp dụng hai thay đổi:

1. We have to read the encryption initialization vector (`nodejsdp.link/iv`) sent by the client.
2. The first step of our streaming pipeline is now responsible for decrypting the incoming data using the `createDecipheriv` Transform stream from the crypto module.
 1. Ta phải đọc vector khởi tạo mã hóa (`nodejsdp.link/iv`) do client gửi.
 2. Bước đầu tiên trong pipeline streaming của ta giờ chịu trách nhiệm giải mã dữ liệu đến bằng Transform stream `createDecipheriv` từ module `crypto`.

With very little effort (just a few lines of code), we added an encryption layer to our application; we simply had to use some already available Transform streams (`createCipheriv` and `createDecipheriv`) and included them in the stream processing pipelines for the client and the server. In a similar way, we can add and combine other streams, as if we were playing with Lego bricks. The main advantage of this approach is reusability, but as we can see from the code so

far, streams also enable cleaner and more modular code. For these reasons, streams are often used not just to deal with pure I/O, but also as a means to simplify and modularize the code.

Với rất ít công sức (chỉ vài dòng mã), ta đã thêm một lớp mã hóa vào ứng dụng; ta chỉ cần dùng một số Transform stream đã có sẵn (`createCipheriv` và `createDecipheriv`) và đưa chúng vào các pipeline xử lý stream cho client và server. Theo cách tương tự, ta có thể thêm và kết hợp các stream khác, như thể đang chơi với các viên gạch Lego. Ưu điểm chính của cách tiếp cận này là khả năng tái sử dụng, nhưng như ta có thể thấy từ đoạn mã đến giờ, stream cũng cho phép viết mã sạch hơn và module hóa hơn. Vì những lý do này, stream thường được dùng không chỉ để xử lý I/O thuần túy, mà còn như một phương tiện để đơn giản hóa và module hóa mã.

Now that we have introduced streams, we are ready to explore, in a more structured way, the different types of streams available in Node.js.

Giờ đây, sau khi đã giới thiệu về stream, ta đã sẵn sàng khám phá, theo một cách có cấu trúc hơn, các loại stream khác nhau có sẵn trong Node.js.

Getting started with streams — Bắt đầu với stream

In the previous section, we learned why streams are so powerful, but also that they are everywhere in Node.js, starting from its core modules. For example, we have seen that the `fs` module has `createReadStream()` for reading from a file and `createWriteStream()` for writing to a file, the HTTP request and response objects are essentially streams, the `zlib` module allows us to compress and decompress data using a streaming interface and, finally, even the `crypto` module exposes some useful streaming primitives like `createCipheriv` and `createDecipheriv`.

Trong phần trước, ta đã tìm hiểu vì sao stream lại mạnh mẽ đến vậy, nhưng cũng thấy rằng chúng hiện diện ở khắp nơi trong Node.js, bắt đầu từ các module lõi của nó. Ví dụ, ta đã thấy rằng module `fs` có `createReadStream()` để đọc từ một tập tin và `createWriteStream()` để ghi vào một tập tin, các đối tượng request và response của HTTP về bản chất là các stream, module `zlib` cho phép ta nén và giải nén dữ liệu qua một giao diện streaming và, cuối cùng, ngay cả module `crypto` cũng phơi bày một số primitive streaming hữu ích như `createCipheriv` và `createDecipheriv`.

Now that we know why streams are so important, let's take a step back and start to explore them in more detail.

Giờ đây, sau khi biết vì sao stream lại quan trọng đến vậy, hãy lùi lại một bước và bắt đầu khám phá chúng chi tiết hơn.

Anatomy of streams — Giải phẫu của stream

Every stream in Node.js is an implementation of one of the four base abstract classes available in the stream core module:

Mỗi stream trong Node.js là một hiện thực của một trong bốn lớp trừu tượng cơ sở có sẵn trong module lõi stream:

- **Readable**
- Writable
- Duplex
- Transform

- Readable
- Writable
- Duplex
- Transform

Each stream class is also an instance of `EventEmitter`. Streams, in fact, can produce several types of event, such as `end` when a `Readable` stream has finished reading, `finish` when a `Writable` stream has completed writing, or `error` when something goes wrong.

Mỗi lớp stream cũng là một instance của `EventEmitter`. Trên thực tế, stream có thể phát ra nhiều loại sự kiện, chẳng hạn `end` khi một `Readable` stream đã đọc xong, `finish` khi một `Writable` stream đã ghi xong, hoặc `error` khi có gì đó sai sót.

One reason why streams are so flexible is the fact that they can handle not just binary data, but almost any JavaScript value. In fact, they support two operating modes:

Một lý do khiến stream linh hoạt đến vậy là sự thật rằng chúng có thể xử lý không chỉ dữ liệu nhị phân, mà gần như mọi giá trị JavaScript. Trên thực tế, chúng hỗ trợ hai chế độ hoạt động:

- Binary mode: To stream data in the form of chunks, such as buffers or strings
- Object mode: To stream data as a sequence of discrete objects (allowing us to use almost any JavaScript value)
 - Chế độ nhị phân (binary mode): Để stream dữ liệu dưới dạng các chunk, chẳng hạn buffer hoặc chuỗi
 - Chế độ object (object mode): Để stream dữ liệu dưới dạng một chuỗi các object rời rạc (cho phép ta dùng gần như mọi giá trị JavaScript)

These two operating modes allow us to use streams not just for I/O, but also as a tool to elegantly compose processing units in a functional fashion, as we will see later in this chapter.

Hai chế độ hoạt động này cho phép ta dùng stream không chỉ cho I/O, mà còn như một công cụ để kết hợp một cách thanh lịch các đơn vị xử lý theo phong cách hàm (functional), như ta sẽ thấy sau trong chương này.

Let's start our deep dive of Node.js streams by introducing the class of `Readable` streams.

Hãy bắt đầu chuyến đào sâu vào stream của Node.js bằng cách giới thiệu lớp `Readable` stream.

Readable streams — Readable stream

A `Readable` stream represents a source of data. In Node.js, it's implemented using the `Readable` abstract class, which is available in the `stream` module.

Một `Readable` stream đại diện cho một nguồn dữ liệu. Trong Node.js, nó được hiện thực bằng lớp trừu tượng `Readable`, có sẵn trong module `stream`.

Reading from a stream — Đọc từ một stream

There are two approaches to receive the data from a `Readable` stream: non-flowing (or paused) and flowing. Let's analyze these modes in more detail.

Có hai cách tiếp cận để nhận dữ liệu từ một `Readable` stream: không chảy (non-flowing, hay paused) và chảy (flowing). Hãy phân tích các chế độ này chi tiết hơn.

The non-flowing mode — Chế độ non-flowing The non-flowing or paused mode is the default pattern for reading from a Readable stream. It involves attaching a listener to the stream for the readable event, which signals the availability of new data to read. Then, in a loop, we read the data continuously until the internal buffer is emptied. This can be done using the `read()` method, which synchronously reads from the internal buffer and returns a Buffer object representing the chunk of data. The `read()` method has the following signature:

Chế độ non-flowing hay paused là mẫu mặc định để đọc từ một Readable stream. Nó liên quan tới việc gắn một listener vào stream cho sự kiện readable, vốn báo hiệu sự sẵn có của dữ liệu mới để đọc. Sau đó, trong một vòng lặp, ta đọc dữ liệu liên tục cho tới khi buffer nội bộ rỗng. Có thể làm điều này bằng phương thức `read()`, vốn đọc đồng bộ từ buffer nội bộ và trả về một đối tượng Buffer đại diện cho chunk dữ liệu. Phương thức `read()` có chữ ký sau:

```
readable.read([size])
```

Using this approach, the data is imperatively pulled from the stream on demand.

Dùng cách tiếp cận này, dữ liệu được kéo (pull) khỏi stream một cách mệnh lệnh (imperative) theo yêu cầu.

To show how this works, let's create a new module named `read-stdin.js`, which implements a simple program that reads from the standard input (which is also a Readable stream) and echoes everything back to the standard output:

Để minh họa cách hoạt động này, hãy tạo một module mới tên `read-stdin.js`, hiện thực một chương trình đơn giản đọc từ đầu vào chuẩn (vốn cũng là một Readable stream) và lặp lại mọi thứ ra đầu ra chuẩn:

```
process.stdin
.on('readable', () => {
  let chunk
  console.log('New data available')
  while ((chunk = process.stdin.read()) !== null) {
    console.log(
      `Chunk read (${chunk.length} bytes): "${chunk.toString()}"`
    )
  }
})
.on('end', () => console.log('End of stream'))
```

The `read()` method is a synchronous operation that pulls a data chunk from the internal buffers of the Readable stream. The returned chunk is, by default, a Buffer object if the stream is working in binary mode.

Phương thức `read()` là một thao tác đồng bộ kéo một chunk dữ liệu từ các buffer nội bộ của Readable stream. Theo mặc định, chunk được trả về là một đối tượng Buffer nếu stream đang hoạt động ở chế độ nhị phân.

In a Readable stream working in binary mode, we can read strings instead of buffers by calling `setEncoding(encoding)` on the stream, and providing a valid encoding format (for example, `utf8`). This approach is recommended when streaming UTF-8 text data as the stream will properly handle multibyte characters, doing the necessary buffering to make sure that no character ends up being split into separate chunks. In other words, every chunk produced by the stream will be a valid UTF-8 sequence of bytes.

Trong một Readable stream hoạt động ở chế độ nhị phân, ta có thể đọc chuỗi thay vì buffer bằng cách gọi `setEncoding(encoding)` trên stream, và cung cấp một định dạng encoding hợp lệ (ví dụ, `utf8`). Cách tiếp cận này được khuyến nghị khi stream dữ liệu văn bản UTF-8 vì stream sẽ xử lý đúng các ký tự nhiều byte, thực hiện buffering cần thiết để đảm bảo không ký tự nào bị tách thành các chunk riêng biệt. Nói cách khác, mỗi chunk do stream tạo ra sẽ là một chuỗi byte UTF-8 hợp lệ.

Note that you can call `setEncoding()` as many times as you want on a Readable stream, even after you've started consuming the data from the stream. The encoding will be switched dynamically on the next available chunk. Streams are inherently binary; encoding is just a view over the binary data that is emitted by the stream.

Lưu ý rằng bạn có thể gọi `setEncoding()` bao nhiêu lần tùy thích trên một Readable stream, kể cả sau khi bạn đã bắt đầu tiêu thụ dữ liệu từ stream. Encoding sẽ được chuyển đổi động ở chunk có sẵn tiếp theo. Stream vốn dĩ là nhị phân; encoding chỉ là một góc nhìn (view) trên dữ liệu nhị phân được stream phát ra.

The data is read solely from within the Readable listener, which is invoked as soon as new data is available. The `read()` method returns `null` when there is no more data available in the internal buffers; in such a case, we have to wait for another readable event to be fired, telling us that we can read again, or wait for the end event that signals the end of the stream. When a stream is working in binary mode, we can also specify that we are interested in reading a specific amount of data by passing a size value to the `read()` method. This is particularly useful when implementing network protocols or when parsing specific data formats.

Dữ liệu chỉ được đọc bên trong listener readable, vốn được gọi ngay khi dữ liệu mới có sẵn. Phương thức `read()` trả về `null` khi không còn dữ liệu nào trong các buffer nội bộ; trong trường hợp đó, ta phải chờ một sự kiện readable khác được phát ra, báo cho ta rằng có thể đọc lại, hoặc chờ sự kiện `end` báo hiệu kết thúc stream. Khi một stream hoạt động ở chế độ nhị phân, ta cũng có thể chỉ định rằng ta muốn đọc một lượng dữ liệu cụ thể bằng cách truyền một giá trị `size` cho phương thức `read()`. Điều này đặc biệt hữu ích khi hiện thực các giao thức mạng hoặc khi phân tích cú pháp các định dạng dữ liệu cụ thể.

Now, we are ready to run the `read-stdin.js` module and experiment with it. Let's type some characters into the console and then press Enter to see the data echoed back into the standard output. To terminate the stream and hence generate a graceful end event, we need to insert an EOF (end-of-file) character (using Ctrl + Z on Windows or Ctrl + D on Linux and macOS). We can also try to connect our program with other processes. This is possible using the pipe operator (`|`), which redirects the standard output of a program to the standard input of another. For example, we can run a command such as the following:

Bây giờ, ta đã sẵn sàng chạy module `read-stdin.js` và thử nghiệm với nó. Hãy gõ vài ký tự vào console rồi nhấn Enter để thấy dữ liệu được lặp lại ra đầu ra chuẩn. Để kết thúc stream và do đó tạo ra một sự kiện `end` êm đẹp, ta cần chèn một ký tự EOF (end-of-file) (dùng Ctrl + Z trên Windows hoặc Ctrl + D trên Linux và macOS). Ta cũng có thể thử kết nối chương trình của mình với các tiến trình khác. Điều này khả thi nhờ toán tử pipe (`|`), vốn chuyển hướng đầu ra chuẩn của một chương trình tới đầu vào chuẩn của một chương trình khác. Ví dụ, ta có thể chạy một lệnh như sau:

```
cat <path to a file> | node read-stdin.js
```

This is an amazing demonstration of how the streaming paradigm is a universal interface that enables our programs to communicate, regardless of the language they are written in.

Đây là một minh chứng tuyệt vời cho thấy mô hình streaming là một giao diện phổ quát

cho phép các chương trình của ta giao tiếp với nhau, bất kể chúng được viết bằng ngôn ngữ nào.

Flowing mode — Chế độ flowing Another way to read from a stream is by attaching a listener to the data event. This will switch the stream into using flowing mode, where the data is not pulled using `read()`, but instead is pushed to the data listener as soon as it arrives. For example, the `read-stdin.js` application that we created earlier will look like this using flowing mode:

Một cách khác để đọc từ một stream là gắn một listener vào sự kiện `data`. Việc này sẽ chuyển stream sang dùng chế độ flowing, nơi dữ liệu không được kéo bằng `read()`, mà thay vào đó được đẩy (push) tới listener `data` ngay khi nó đến. Ví dụ, ứng dụng `read-stdin.js` mà ta tạo trước đó sẽ trông như sau khi dùng chế độ flowing:

```
process.stdin
  .on('data', (chunk) => {
    console.log('New data available')
    console.log(
      `Chunk read (${chunk.length} bytes): "${chunk.toString()}"`
    )
  })
  .on('end', () => console.log('End of stream'))
```

Flowing mode offers less flexibility to control the flow of data compared to nonflowing mode. The default operating mode for streams is non-flowing, so to enable flowing mode, it's necessary to attach a listener to the data event or explicitly invoke the `resume()` method. To temporarily stop the stream from emitting data events, we can invoke the `pause()` method, causing any incoming data to be cached in the internal buffer. Calling `pause()` will switch the stream back to non-flowing mode.

Chế độ flowing cho ít sự linh hoạt hơn trong việc kiểm soát luồng dữ liệu so với chế độ non-flowing. Chế độ hoạt động mặc định của stream là non-flowing, nên để bật chế độ flowing, cần phải gắn một listener vào sự kiện `data` hoặc gọi tường minh phương thức `resume()`. Để tạm thời dừng stream phát ra các sự kiện `data`, ta có thể gọi phương thức `pause()`, khiến mọi dữ liệu đến được lưu cache trong buffer nội bộ. Gọi `pause()` sẽ chuyển stream trở lại chế độ non-flowing.

Async iterators — Async iterator Readable streams are also **async** iterators; therefore, we could rewrite our `read-stdin.js` example as follows:

Readable stream cũng là các async iterator; do đó, ta có thể viết lại ví dụ `read-stdin.js` như sau:

```
async function main () {
  for await (const chunk of process.stdin) {
    console.log('New data available')
    console.log(
      `Chunk read (${chunk.length} bytes): "${chunk.toString()}"`
    )
  }
  console.log('End of stream')
}
main()
```

We will discuss async iterators in greater detail in Chapter 9, Behavioral Design Patterns, so don't worry too much about the syntax in the preceding example for now. What's important to know is that if you need to write a function that consumes an entire Readable stream and returns a **Promise**, this syntax could come in very handy.

Chúng ta sẽ bàn về async iterator chi tiết hơn trong Chương 9, Behavioral Design Patterns, nên đừng quá lo về cú pháp trong ví dụ trên lúc này. Điều quan trọng cần biết là nếu bạn cần viết một hàm tiêu thụ trọn một Readable stream và trả về một Promise, cú pháp này có thể rất tiện dụng.

Implementing Readable streams — Hiện thực Readable stream

Now that we know how to read from a stream, the next step is to learn how to implement a new custom Readable stream. To do this, it's necessary to create a new class by inheriting the prototype Readable from the stream module. The concrete stream must provide an implementation of the `_read()` method, which has the following signature:

Giờ đây, sau khi biết cách đọc từ một stream, bước tiếp theo là học cách hiện thực một Readable stream tùy chỉnh mới. Để làm điều này, cần tạo một lớp mới bằng cách kế thừa prototype Readable từ module stream. Stream cụ thể phải cung cấp một hiện thực cho phương thức `_read()`, vốn có chữ ký sau:

```
readable._read(size)
```

The internals of the Readable class will call the `_read()` method, which, in turn, will start to fill the internal buffer using `push()`:

Phần nội bộ của lớp Readable sẽ gọi phương thức `_read()`, đến lượt nó sẽ bắt đầu lấp đầy buffer nội bộ bằng `push()`:

```
readable.push(chunk)
```

Please note that `read()` is a method called by the stream consumers, while `_read()` is a method to be implemented by a stream subclass and should never be called directly. The underscore usually indicates that the method is not public and should not be called directly. To demonstrate how to implement new Readable streams, we can try to implement a stream that generates random strings. Let's create a new module called `random-stream.js` that contains the code of our random string generator:

Xin lưu ý rằng `read()` là một phương thức được các consumer của stream gọi, còn `_read()` là một phương thức để một lớp con của stream hiện thực và không bao giờ nên được gọi trực tiếp. Dấu gạch dưới thường cho biết rằng phương thức không phải public và không nên được gọi trực tiếp. Để minh họa cách hiện thực Readable stream mới, ta có thể thử hiện thực một stream sinh ra các chuỗi ngẫu nhiên. Hãy tạo một module mới tên `random-stream.js` chứa mã của trình sinh chuỗi ngẫu nhiên:

```
import { Readable } from 'stream'
import Chance from 'chance'
const chance = new Chance()
export class RandomStream extends Readable {
  constructor(options) {
    super(options)
    this.emittedBytes = 0
  }
  _read(size) {
```



```

const chunk = chance.string({ length: size })           // (1)
this.push(chunk, 'utf8')                               // (2)
this.emittedBytes += chunk.length
if (chance.bool({ likelihood: 5 })) {                   // (3)
  this.push(null)
}
}
}

```

At the top of the file, we load our dependencies. There is nothing special there, except that we are loading an npm module called `chance` (`nodejsdp.link/chance`), which is a library for generating all sorts of random values, from numbers to strings to entire sentences.

Ở đầu tập tin, ta nạp các phụ thuộc (dependency) của mình. Không có gì đặc biệt ở đó, ngoại trừ việc ta nạp một module npm tên `chance` (`nodejsdp.link/chance`), một thư viện để sinh ra đủ loại giá trị ngẫu nhiên, từ số tới chuỗi tới cả những câu hoàn chỉnh.

The next step is to create a new class called `RandomStream`, which specifies `Readable` as its parent. In the preceding code, invoking `super(options)` in the `RandomStream` constructor will call the constructor of the parent class, allowing us to initialize the stream's internal **state**.

Bước tiếp theo là tạo một lớp mới tên `RandomStream`, chỉ định `Readable` là lớp cha. Trong đoạn mã trên, việc gọi `super(options)` trong constructor của `RandomStream` sẽ gọi constructor của lớp cha, cho phép ta khởi tạo trạng thái nội bộ của stream.

If you have a constructor that only invokes `super(options)`, you can remove it as you will inherit the parent constructor. Just be careful to remember to call `super(options)` every time you need to write a custom constructor. The possible parameters that can be passed through the `options` object include the following:

Nếu bạn có một constructor chỉ gọi `super(options)`, bạn có thể bỏ nó đi vì bạn sẽ kế thừa constructor của lớp cha. Chỉ cần cẩn thận nhớ gọi `super(options)` mỗi khi bạn cần viết một constructor tùy chỉnh. Các tham số có thể truyền qua đối tượng `options` bao gồm:

- The encoding argument, which is used to convert buffers into strings (defaults to `null`)
- A flag to enable object mode (`objectMode`, defaults to `false`)
- The upper limit of the data stored in the internal buffer, after which no more reading from the source should be done (`highWaterMark`, defaults to 16KB)
 - Đối số encoding, được dùng để chuyển đổi buffer thành chuỗi (mặc định là `null`)
 - Một cờ để bật chế độ object (`objectMode`, mặc định là `false`)
 - Giới hạn trên của lượng dữ liệu được lưu trong buffer nội bộ, vượt quá giới hạn này thì không nên đọc thêm từ nguồn nữa (`highWaterMark`, mặc định là 16KB)

Okay, now let's explain the `_read()` method:

Được rồi, bây giờ hãy giải thích phương thức `_read()`:

1. The method generates a random string of length equal to `size` using `chance`.
2. It pushes the string into the internal buffer. Note that since we are pushing strings, we also need to specify the encoding, `utf8` (this is not necessary if the chunk is simply a binary `Buffer`).
3. It terminates the stream randomly, with a likelihood of 5 percent, by pushing `null` into the internal buffer to indicate an EOF situation or, in other words, the end of the stream.

1. Phương thức sinh ra một chuỗi ngẫu nhiên có độ dài bằng `size` bằng cách dùng `chance`.

2. Nó đẩy chuỗi vào buffer nội bộ. Lưu ý rằng vì ta đang đẩy chuỗi, ta cũng cần chỉ định encoding, utf8 (điều này không cần thiết nếu chunk chỉ đơn giản là một Buffer nhị phân).
3. Nó kết thúc stream một cách ngẫu nhiên, với xác suất 5 phần trăm, bằng cách đẩy null vào buffer nội bộ để báo hiệu tình huống EOF hay nói cách khác là kết thúc stream.

Note that the size argument in the `_read()` function is an advisory parameter. It's good to honor it and push only the amount of data that was requested by the caller, even though it is not mandatory to do so.

Lưu ý rằng đối số size trong hàm `_read()` là một tham số mang tính khuyến nghị. Tôn trọng nó là điều tốt và chỉ nên đẩy đúng lượng dữ liệu mà bên gọi yêu cầu, dù không bắt buộc phải làm vậy.

When we invoke `push()`, we should check whether it returns `false`. When that happens, it means that the internal buffer of the receiving stream has reached the `highWaterMark` limit and we should stop adding more data to it. This is called **backpressure**, and we will be discussing it in more detail in the next section of this chapter.

Khi ta gọi `push()`, ta nên kiểm tra xem nó có trả về `false` hay không. Khi điều đó xảy ra, nghĩa là buffer nội bộ của stream nhận đã đạt giới hạn `highWaterMark` và ta nên ngừng thêm dữ liệu vào nó. Đây gọi là backpressure (áp lực ngược), và ta sẽ bàn về nó chi tiết hơn trong phần tiếp theo của chương này.

That's it for `RandomStream`, we are now ready to use it. Let's see how to instantiate a `RandomStream` object and pull some data from it:

Vậy là xong cho `RandomStream`, giờ ta đã sẵn sàng dùng nó. Hãy xem cách khởi tạo một đối tượng `RandomStream` và kéo một ít dữ liệu từ nó:

```
// index.js
import { RandomStream } from './random-stream.js'
const randomStream = new RandomStream()
randomStream
  .on('data', (chunk) => {
    console.log(`Chunk received (${chunk.length} bytes): ${chunk.toString()}`)
  })
  .on('end', () => {
    console.log(`Produced ${randomStream.emittedBytes} bytes of random data`)
  })
```

Now, everything is ready for us to try our new custom stream. Simply execute the `index.js` module as usual and watch a random set of strings flowing on the screen.

Bây giờ, mọi thứ đã sẵn sàng để ta thử stream tùy chỉnh mới. Chỉ cần thực thi module `index.js` như thường lệ và xem một tập hợp chuỗi ngẫu nhiên chảy trên màn hình.

Simplified construction — Cách dựng đơn giản hóa For simple custom streams, we can avoid creating a custom class by using the Readable stream's simplified construction approach. With this approach, we only need to invoke `new Readable(options)` and pass a method named `read()` in the set of options. The `read()` method here has exactly the same semantic as the `_read()` method

that we saw in the class extension approach. Let's rewrite our `RandomStream` using the simplified constructor approach:

Đối với các stream tùy chỉnh đơn giản, ta có thể tránh việc tạo một lớp tùy chỉnh bằng cách dùng cách dựng đơn giản hóa (simplified construction) của `Readable` stream. Với cách này, ta chỉ cần gọi `new Readable(options)` và truyền một phương thức tên `read()` trong tập `options`. Phương thức `read()` ở đây có ngữ nghĩa hoàn toàn giống phương thức `_read()` mà ta đã thấy trong cách tiếp cận mở rộng lớp. Hãy viết lại `RandomStream` bằng cách tiếp cận constructor đơn giản hóa:

```
import { Readable } from 'stream'
import Chance from 'chance'
const chance = new Chance()
let emittedBytes = 0
const randomStream = new Readable({
  read (size) {
    const chunk = chance.string({ length: size })
    this.push(chunk, 'utf8')
    emittedBytes += chunk.length
    if (chance.bool({ likelihood: 5 })) {
      this.push(null)
    }
  }
})
// now use randomStream instance directly ...
```

This approach can be particularly useful when you don't need to manage a complicated state and allows you to take advantage of a more succinct syntax. In the previous example, we created a single instance of our custom stream. If we want to adopt the simplified constructor approach but we need to create multiple instances of the custom stream, we can wrap the initialization logic in a **factory** function that we can invoke multiple times to create those instances.

Cách tiếp cận này có thể đặc biệt hữu ích khi bạn không cần quản lý một trạng thái phức tạp và cho phép bạn tận dụng một cú pháp cô đọng hơn. Trong ví dụ trước, ta tạo một instance duy nhất của stream tùy chỉnh. Nếu ta muốn dùng cách tiếp cận constructor đơn giản hóa nhưng cần tạo nhiều instance của stream tùy chỉnh, ta có thể bọc logic khởi tạo trong một hàm `factory` mà ta có thể gọi nhiều lần để tạo những instance đó.

Readable streams from iterables — Readable stream từ iterable You can easily create `Readable` stream instances from arrays or other iterable objects (that is, generators, iterators, and async iterators) using the `Readable.from()` helper.

Bạn có thể dễ dàng tạo các instance `Readable` stream từ mảng hoặc các đối tượng iterable khác (tức là, generator, iterator, và async iterator) bằng cách dùng helper `Readable.from()`.

In order to get accustomed with this helper, let's look at a simple example where we convert data from an array into a `Readable` stream:

Để làm quen với helper này, hãy xem một ví dụ đơn giản trong đó ta chuyển đổi dữ liệu từ một mảng thành một `Readable` stream:

```
import { Readable } from 'stream'
const mountains = [
```

```

    { name: 'Everest', height: 8848 },
    { name: 'K2', height: 8611 },
    { name: 'Kangchenjunga', height: 8586 },
    { name: 'Lhotse', height: 8516 },
    { name: 'Makalu', height: 8481 }
  ]
  const mountainsStream = Readable.from(mountains)
  mountainsStream.on('data', (mountain) => {
    console.log(`${mountain.name.padStart(14)}\t${mountain.height}m`)
  })

```

As we can see from this code, the `Readable.from()` method is quite simple to use: the first argument is an iterable instance (in our case, the `mountains` array). `Readable.from()` accepts an additional optional argument that can be used to specify stream options like `objectMode`.

Như ta có thể thấy từ đoạn mã này, phương thức `Readable.from()` khá đơn giản để dùng: đối số đầu tiên là một instance iterable (trong trường hợp của ta là mảng `mountains`). `Readable.from()` chấp nhận một đối số tùy chọn bổ sung có thể dùng để chỉ định các tùy chọn stream như `objectMode`.

Note that we didn't have to explicitly set `objectMode` to `true`. By default, `Readable.from()` will set `objectMode` to `true`, unless this is explicitly opted out by setting it to `false`. Stream options can be passed as a second argument to the function.

Lưu ý rằng ta không phải đặt tường minh `objectMode` thành `true`. Theo mặc định, `Readable.from()` sẽ đặt `objectMode` thành `true`, trừ khi điều này bị từ chối tường minh bằng cách đặt nó thành `false`. Các tùy chọn stream có thể được truyền làm đối số thứ hai cho hàm.

Running the previous code will produce the following output:

Chạy đoạn mã trên sẽ tạo ra kết quả sau:

Everest	8848m
K2	8611m
Kangchenjunga	8586m
Lhotse	8516m
Makalu	8481m

Try not to instantiate large arrays in memory. Imagine if, in the previous example, we wanted to list all the mountains in the world. There are about 1 million mountains, so if we were to load all of them in an array upfront, we would allocate a quite significant amount of memory. Even if we then consume the data in the array through a `Readable` stream, all the data has already been preloaded, so we are effectively voiding the memory efficiency of streams. It's always preferable to load and consume the data in chunks, and you could do so by using native streams such as `fs.createReadStream`, by building a custom stream, or by using `Readable.from` with lazy iterables such as generators, iterators, or async iterators. We will see some examples of the latter approach in Chapter 9, Behavioral Design Patterns.

Cố gắng đừng khởi tạo các mảng lớn trong bộ nhớ. Hãy tưởng tượng nếu, trong ví dụ trước, ta muốn liệt kê tất cả các ngọn núi trên thế giới. Có khoảng 1 triệu ngọn núi, nên nếu ta nạp tất cả chúng vào một mảng từ trước, ta sẽ cấp phát một lượng bộ nhớ khá đáng kể. Ngay cả khi sau đó ta tiêu thụ dữ liệu trong mảng qua một `Readable` stream, toàn bộ dữ liệu vẫn đã được nạp trước, nên ta đang vô hiệu hóa hiệu quả về bộ nhớ của stream. Luôn ưu tiên nạp và tiêu thụ dữ liệu theo từng chunk, và bạn có thể làm vậy bằng cách dùng

các stream gốc như `fs.createReadStream`, bằng cách xây dựng một stream tùy chỉnh, hoặc bằng cách dùng `Readable.from` với các iterable lười (lazy) như generator, iterator, hoặc async iterator. Chúng ta sẽ thấy một số ví dụ về cách tiếp cận sau trong Chương 9, Behavioral Design Patterns.

Writable streams — Writable stream

A Writable stream represents a data destination. Imagine, for instance, a file on the filesystem, a database table, a socket, the standard error, or the standard output interface. In Node.js, it's implemented using the Writable abstract class, which is available in the `stream` module.

Một Writable stream đại diện cho một đích đến của dữ liệu. Hãy tưởng tượng, chẳng hạn, một tập tin trên hệ thống tập tin, một bảng cơ sở dữ liệu, một socket, lỗi chuẩn (standard error), hoặc giao diện đầu ra chuẩn. Trong Node.js, nó được hiện thực bằng lớp trừu tượng Writable, có sẵn trong module `stream`.

Writing to a stream — Ghi vào một stream

Pushing some data down a Writable stream is a straightforward business; all we have to do is use the `write()` method, which has the following signature:

Đẩy một ít dữ liệu xuống một Writable stream là chuyện đơn giản; tất cả những gì ta phải làm là dùng phương thức `write()`, vốn có chữ ký sau:

```
writable.write(chunk, [encoding], [callback])
```

The `encoding` argument is optional and can be specified if `chunk` is a string (it defaults to `utf8`, and it's ignored if `chunk` is a buffer). The `**callback**` function, on the other hand, is called when the chunk is flushed into the underlying resource and is optional as well.

Đối số `encoding` là tùy chọn và có thể được chỉ định nếu `chunk` là một chuỗi (mặc định là `utf8`, và bị bỏ qua nếu `chunk` là một buffer). Mặt khác, hàm `callback` được gọi khi chunk được flush vào tài nguyên bên dưới và cũng là tùy chọn.

To signal that no more data will be written to the stream, we have to use the `end()` method:

Để báo hiệu rằng sẽ không còn dữ liệu nào được ghi vào stream nữa, ta phải dùng phương thức `end()`:

```
writable.end([chunk], [encoding], [callback])
```

We can provide a final chunk of data through the `end()` method; in this case, the `callback` function is equivalent to registering a listener to the `finish` event, which is fired when all the data written in the stream has been flushed into the underlying resource. Now, let's show how this works by creating a small HTTP server that outputs a random sequence of strings:

Ta có thể cung cấp một chunk dữ liệu cuối cùng qua phương thức `end()`; trong trường hợp này, hàm `callback` tương đương với việc đăng ký một listener cho sự kiện `finish`, vốn được phát ra khi tất cả dữ liệu ghi vào stream đã được flush vào tài nguyên bên dưới. Bây giờ, hãy minh họa cách hoạt động này bằng cách tạo một máy chủ HTTP nhỏ xuất ra một dãy chuỗi ngẫu nhiên:

```
// entropy-server.js
import { createServer } from 'http'
import Chance from 'chance'
const chance = new Chance()
```

```

const server = createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' }) // (1)
  while (chance.bool({ likelihood: 95 })) { // (2)
    res.write(`${chance.string()}\n`) // (3)
  }
  res.end('\n\n') // (4)
  res.on('finish', () => console.log('All data sent')) // (5)
})
server.listen(8080, () => {
  console.log('listening on http://localhost:8080')
})

```

The HTTP server that we created writes into the `res` object, which is an instance of `http.ServerResponse` and also a `Writable` stream. What happens is explained as follows:

Máy chủ HTTP mà ta tạo ghi vào đối tượng `res`, vốn là một instance của `http.ServerResponse` và cũng là một `Writable` stream. Những gì diễn ra được giải thích như sau:

1. We first write the head of the HTTP response. Note that `writeHead()` is not a part of the `Writable` interface; in fact, it's an auxiliary method exposed by the `http.ServerResponse` class and is specific to the HTTP protocol.
2. We start a loop that terminates with a likelihood of 5% (we instruct `chance.bool()` to return `true` 95% of the time).
3. Inside the loop, we write a random string into the stream.
4. Once we are out of the loop, we call `end()` on the stream, indicating that no more data will be written. Also, we provide a final string containing two new line characters to be written into the stream before ending it.
5. Finally, we register a listener for the `finish` event, which will be fired when all the data has been flushed into the underlying socket.
 1. Trước hết ta ghi phần head của response HTTP. Lưu ý rằng `writeHead()` không phải là một phần của giao diện `Writable`; thực tế, nó là một phương thức phụ trợ do lớp `http.ServerResponse` phơi bày và đặc thù cho giao thức HTTP.
 2. Ta bắt đầu một vòng lặp kết thúc với xác suất 5% (ta yêu cầu `chance.bool()` trả về `true` 95% số lần).
 3. Bên trong vòng lặp, ta ghi một chuỗi ngẫu nhiên vào stream.
 4. Sau khi thoát khỏi vòng lặp, ta gọi `end()` trên stream, cho biết sẽ không còn dữ liệu nào được ghi nữa. Ngoài ra, ta cung cấp một chuỗi cuối cùng chứa hai ký tự xuống dòng để ghi vào stream trước khi kết thúc nó.
 5. Cuối cùng, ta đăng ký một listener cho sự kiện `finish`, vốn sẽ được phát ra khi tất cả dữ liệu đã được flush vào socket bên dưới.

To test the server, we can open a browser at the address `http://localhost:8080` or use `curl` from the terminal as follows:

Để kiểm thử máy chủ, ta có thể mở trình duyệt ở địa chỉ `http://localhost:8080` hoặc dùng `curl` từ terminal như sau:

```
curl localhost:8080
```

At this point, the server should start sending random strings to the HTTP client that you chose (please bear in mind that some browsers might buffer the data, and the streaming behavior might not be apparent).

Tại thời điểm này, máy chủ sẽ bắt đầu gửi các chuỗi ngẫu nhiên tới client HTTP mà bạn

chọn (xin lưu ý rằng một số trình duyệt có thể buffer dữ liệu, và hành vi streaming có thể không hiển hiện rõ).

Backpressure — Backpressure

Similar to a liquid flowing in a real **pip**ing system, Node.js streams can also suffer from bottlenecks, where data is written faster than the stream can consume it. The mechanism to cope with this problem involves buffering the incoming data; however, if the stream doesn't give any feedback to the writer, we may incur a situation where more and more data is accumulated in the internal buffer, leading to undesired levels of memory usage.

Tương tự như chất lỏng chảy trong một hệ thống đường ống thực tế, stream của Node.js cũng có thể gặp nút thắt cổ chai, nơi dữ liệu được ghi nhanh hơn mức stream có thể tiêu thụ. Cơ chế để đối phó với vấn đề này liên quan tới việc buffer dữ liệu đến; tuy nhiên, nếu stream không đưa ra bất kỳ phản hồi nào cho bên ghi, ta có thể gặp tình huống ngày càng nhiều dữ liệu được tích lũy trong buffer nội bộ, dẫn tới mức sử dụng bộ nhớ không mong muốn.

To prevent this from happening, `writable.write()` will return `false` when the internal buffer exceeds the `highWaterMark` limit. In `Writable` streams, the `highWaterMark` property is the limit of the internal buffer size, beyond which the `write()` method starts returning `false`, indicating that the application should now stop writing. When the buffer is emptied, the `drain` event is emitted, communicating that it's safe to start writing again. This mechanism is called backpressure.

Để ngăn điều này xảy ra, `writable.write()` sẽ trả về `false` khi buffer nội bộ vượt giới hạn `highWaterMark`. Trong `Writable` stream, thuộc tính `highWaterMark` là giới hạn của kích thước buffer nội bộ, vượt qua giới hạn này thì phương thức `write()` bắt đầu trả về `false`, cho biết rằng ứng dụng giờ nên ngừng ghi. Khi buffer được làm rỗng, sự kiện `drain` được phát ra, thông báo rằng ghi tiếp là an toàn. Cơ chế này gọi là backpressure.

Backpressure is an advisory mechanism. Even if `write()` returns `false`, we could ignore this signal and continue writing, making the buffer grow indefinitely. The stream won't be blocked automatically when the `highWaterMark` threshold is reached; therefore, it is recommended to always be mindful and respect the backpressure.

Backpressure là một cơ chế mang tính khuyến nghị. Ngay cả khi `write()` trả về `false`, ta vẫn có thể bỏ qua tín hiệu này và tiếp tục ghi, khiến buffer phình to vô hạn. Stream sẽ không tự động bị chặn khi đạt ngưỡng `highWaterMark`; do đó, được khuyến nghị luôn lưu tâm và tôn trọng backpressure.

The mechanism described in this section is similarly applicable to `Readable` streams. In fact, backpressure exists in `Readable` streams too, and it's triggered when the `push()` method, which is invoked inside `_read()`, returns `false`. However, that's a problem specific to stream implementers, so we usually have to deal with it less frequently.

Cơ chế được mô tả trong phần này cũng tương tự áp dụng cho `Readable` stream. Trên thực tế, backpressure cũng tồn tại trong `Readable` stream, và nó được kích hoạt khi phương thức `push()`, vốn được gọi bên trong `_read()`, trả về `false`. Tuy nhiên, đó là vấn đề đặc thù cho người hiện thực stream, nên ta thường ít phải xử lý nó hơn.

We can quickly demonstrate how to take into account the backpressure of a `Writable` stream by modifying the `entropy-server.js` module that we created previously:

Ta có thể nhanh chóng minh họa cách tính đến backpressure của một `Writable` stream bằng cách sửa đổi module `entropy-server.js` mà ta đã tạo trước đó:

```
// ...
const server = createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' })
  function generateMore () { // (1)
    while (chance.bool({ likelihood: 95 })) {
      const randomChunk = chance.string({ // (2)
        length: (16 * 1024) - 1
      })
      const shouldContinue = res.write(`${randomChunk}\n`) // (3)
      if (!shouldContinue) {
        console.log('back-pressure')
        return res.once('drain', generateMore)
      }
    }
    res.end('\n\n')
  }
  generateMore()
  res.on('finish', () => console.log('All data sent'))
})
// ...
```

The most important steps of the previous code can be summarized as follows:

Các bước quan trọng nhất của đoạn mã trên có thể tóm tắt như sau:

1. We wrapped the main logic in a function called `generateMore()`.
2. To increase the chances of receiving some backpressure, we increased the size of the data chunk to 16 KB minus 1 byte, which is very close to the default `highWaterMark` limit.
3. After writing a chunk of data, we check the return value of `res.write()`. If we receive `false`, it means that the internal buffer is full and we should stop sending more data. When this happens, we exit the function and register another cycle of writes using `generateMore()` for when the `drain` event is emitted.

1. Ta bọc logic chính trong một hàm tên `generateMore()`.
2. Để tăng khả năng gặp backpressure, ta tăng kích thước chunk dữ liệu lên 16 KB trừ 1 byte, vốn rất gần với giới hạn `highWaterMark` mặc định.
3. Sau khi ghi một chunk dữ liệu, ta kiểm tra giá trị trả về của `res.write()`. Nếu nhận `false`, nghĩa là buffer nội bộ đã đầy và ta nên ngừng gửi thêm dữ liệu. Khi điều này xảy ra, ta thoát khỏi hàm và đăng ký một chu kỳ ghi khác bằng `generateMore()` cho lúc sự kiện `drain` được phát ra.

If we now try to run the server again, and then generate a client request with `curl`, there is a high probability that there will be some backpressure, as the server produces data at a very high rate, faster than the underlying socket can handle.

Nếu bây giờ ta thử chạy lại máy chủ, rồi tạo một request từ client bằng `curl`, có xác suất cao sẽ có backpressure, vì máy chủ tạo dữ liệu với tốc độ rất cao, nhanh hơn mức socket bên dưới có thể xử lý.

Implementing Writable streams — Hiện thực Writable stream

We can implement a new `Writable` stream by inheriting the class `Writable` and providing an implementation for the `_write()` method. Let's try to do it immediately while discussing the details

along the way.

Ta có thể hiện thực một Writable stream mới bằng cách kế thừa lớp Writable và cung cấp một hiện thực cho phương thức `_write()`. Hãy thử làm ngay trong khi bàn các chi tiết đọc đường.

Let's build a Writable stream that receives objects in the following format:

Hãy xây dựng một Writable stream nhận các object theo định dạng sau:

```
{
  path: <path to a file>
  content: <string or buffer>
}
```

For each one of these objects, our stream has to save the content property into a file created at the given path. We can immediately see that the inputs of our stream are objects, and not strings or buffers. This means that our stream has to work in object mode.

Với mỗi object như vậy, stream của ta phải lưu thuộc tính content vào một tập tin được tạo tại path cho trước. Ta có thể thấy ngay rằng đầu vào của stream là các object, chứ không phải chuỗi hay buffer. Điều này nghĩa là stream của ta phải hoạt động ở chế độ object.

Let's call the module `to-file-stream.js`:

Hãy đặt tên module là `to-file-stream.js`:

```
import { Writable } from 'stream'
import { promises as fs } from 'fs'
import { dirname } from 'path'
import mkdirp from 'mkdirp-promise'
export class ToFileStream extends Writable {
  constructor (options) {
    super({ ...options, objectMode: true })
  }
  _write (chunk, encoding, cb) {
    mkdirp(dirname(chunk.path))
      .then(() => fs.writeFile(chunk.path, chunk.content))
      .then(() => cb())
      .catch(cb)
  }
}
```

We created a new class for our new stream, which extends Writable from the stream module.

Ta đã tạo một lớp mới cho stream mới, lớp này mở rộng Writable từ module stream.

We had to invoke the parent constructor to initialize its internal state; we also needed to make sure that the options object specifies that the stream works in object mode (`objectMode: true`). Other options accepted by Writable are as follows:

Ta phải gọi constructor của lớp cha để khởi tạo trạng thái nội bộ của nó; ta cũng cần đảm bảo rằng đối tượng options chỉ định rằng stream hoạt động ở chế độ object (`objectMode: true`). Các tùy chọn khác mà Writable chấp nhận gồm:

- `highWaterMark` (the default is 16 KB): This controls the backpressure limit.

- `decodeStrings` (defaults to `true`): This enables the automatic decoding of strings into binary buffers before passing them to the `_write()` method. This option is ignored in object mode.
 - `highWaterMark` (mặc định là 16 KB): Điều này kiểm soát giới hạn backpressure.
 - `decodeStrings` (mặc định là `true`): Điều này bật việc tự động giải mã chuỗi thành các buffer nhị phân trước khi truyền chúng cho phương thức `_write()`. Tùy chọn này bị bỏ qua ở chế độ object.

Finally, we provided an implementation for the `_write()` method. As you can see, the method accepts a data chunk and an encoding (which makes sense only if we are in binary mode and the stream option `decodeStrings` is set to `false`). Also, the method accepts a callback function (`cb`), which needs to be invoked when the operation completes; it's not necessary to pass the result of the operation but, if needed, we can still pass an error that will cause the stream to emit an `error` event. Now, to try the stream that we just built, we can create a new module and perform some write operations against the stream:

Cuối cùng, ta cung cấp một hiện thực cho phương thức `_write()`. Như bạn thấy, phương thức nhận một chunk dữ liệu và một encoding (chỉ có ý nghĩa nếu ta ở chế độ nhị phân và tùy chọn stream `decodeStrings` được đặt thành `false`). Ngoài ra, phương thức nhận một hàm callback (`cb`), cần được gọi khi thao tác hoàn tất; không nhất thiết phải truyền kết quả của thao tác nhưng, nếu cần, ta vẫn có thể truyền một lỗi sẽ khiến stream phát ra sự kiện `error`. Bây giờ, để thử stream mà ta vừa xây dựng, ta có thể tạo một module mới và thực hiện vài thao tác ghi lên stream:

```
import { join } from 'path'
import { ToFileStream } from './to-file-stream.js'
const tfs = new ToFileStream()
tfs.write({
  path: join('files', 'file1.txt'), content: 'Hello' })
tfs.write({
  path: join('files', 'file2.txt'), content: 'Node.js' })
tfs.write({
  path: join('files', 'file3.txt'), content: 'streams' })
tfs.end(() => console.log('All files created'))
```

Here, we created and used our first custom Writable stream. Run the new module as usual and check its output. You will see that after the execution, three new files will be created within a new folder called `files`.

Ở đây, ta đã tạo và dùng Writable stream tùy chỉnh đầu tiên của mình. Chạy module mới như thường lệ và kiểm tra kết quả của nó. Bạn sẽ thấy rằng sau khi thực thi, ba tập tin mới sẽ được tạo trong một thư mục mới tên `files`.

Simplified construction — Cách dựng đơn giản hóa As we saw for Readable streams, Writable streams also offer a simplified construction mechanism. If we were to rewrite our `ToFileStream` using the simplified construction for Writable streams, it would look like this:

Như ta đã thấy với Readable stream, Writable stream cũng cung cấp một cơ chế dựng đơn giản hóa. Nếu ta viết lại `ToFileStream` bằng cách dựng đơn giản hóa cho Writable stream, nó sẽ trông như sau:

```
// ...
const tfs = new Writable({
  objectMode: true,
```

```

write (chunk, encoding, cb) {
  mkdirp(dirname(chunk.path))
  .then(() => fs.writeFile(chunk.path, chunk.content))
  .then(() => cb())
  .catch(cb)
}
})
// ...

```

With this approach, we are simply using the Writable constructor and passing a `write()` function that implements the custom logic of our Writable instance. Note that with this approach, the `write()` function doesn't have an underscore in the name. We can also pass other construction options like `objectMode`.

Với cách tiếp cận này, ta chỉ đơn giản dùng constructor `Writable` và truyền một hàm `write()` hiện thực logic tùy chỉnh của instance `Writable` của ta. Lưu ý rằng với cách tiếp cận này, hàm `write()` không có dấu gạch dưới trong tên. Ta cũng có thể truyền các tùy chọn dựng khác như `objectMode`.

Duplex streams — Duplex stream

A Duplex stream is a stream that is both Readable and Writable. It is useful when we want to describe an entity that is both a data source and a data destination, such as network sockets, for example. Duplex streams inherit the methods of both `stream`, `Readable` and `stream.Writable`, so this is nothing new to us. This means that we can `read()` or `write()` data, or listen for both `readable` and `drain` events.

Một Duplex stream là một stream vừa `Readable` vừa `Writable`. Nó hữu ích khi ta muốn mô tả một thực thể vừa là nguồn dữ liệu vừa là đích dữ liệu, chẳng hạn các network socket. Duplex stream kế thừa các phương thức của cả `stream`, `Readable` và `stream.Writable`, nên điều này không có gì mới với ta. Nghĩa là ta có thể `read()` hoặc `write()` dữ liệu, hoặc lắng nghe cả sự kiện `readable` và `drain`.

To create a custom Duplex stream, we have to provide an implementation for both `_read()` and `_write()`. The options object passed to the `Duplex()` constructor is internally forwarded to both the `Readable` and `Writable` constructors. The options are the same as those we already discussed in the previous sections, with the addition of a new one called `allowHalfOpen` (defaults to `true`) that, if set to `false`, will cause both parts (`Readable` and `Writable`) of the stream to end if only one of them does.

Để tạo một Duplex stream tùy chỉnh, ta phải cung cấp một hiện thực cho cả `_read()` và `_write()`. Đối tượng options truyền cho constructor `Duplex()` được chuyển tiếp nội bộ tới cả constructor `Readable` lẫn `Writable`. Các tùy chọn giống như những gì ta đã bàn trong các phần trước, với việc bổ sung một tùy chọn mới tên `allowHalfOpen` (mặc định là `true`) mà, nếu đặt thành `false`, sẽ khiến cả hai phần (`Readable` và `Writable`) của stream kết thúc nếu chỉ một trong hai kết thúc.

If we need to have a Duplex stream working in object mode on one side and binary mode on the other, we can use the options `readableObjectMode` and `writableObjectMode` independently.

Nếu ta cần có một Duplex stream hoạt động ở chế độ object ở một phía và chế độ nhị phân ở phía kia, ta có thể dùng các tùy chọn `readableObjectMode` và `writableObjectMode` độc lập.

Transform streams — Transform stream

Transform streams are a special kind of Duplex stream that are specifically designed to handle data transformations. Just to give you a few concrete examples, the functions `zlib.createGzip()` and `crypto.createCipheriv()` that we discussed at the beginning of this chapter create Transform streams for compression and encryption, respectively.

Transform stream là một loại đặc biệt của Duplex stream được thiết kế riêng để xử lý các phép biến đổi dữ liệu. Chỉ để cho bạn vài ví dụ cụ thể, các hàm `zlib.createGzip()` và `crypto.createCipheriv()` mà ta đã bàn ở đầu chương này tạo ra các Transform stream lần lượt cho việc nén và mã hóa.

In a simple Duplex stream, there is no immediate relationship between the data read from the stream and the data written into it (at least, the stream is agnostic to such a relationship). Think about a TCP socket, which just sends and receives data to and from the remote peer; the socket is not aware of any relationship between the input and output. Figure 6.4 illustrates the data flow in a Duplex stream:

Trong một Duplex stream đơn giản, không có mối quan hệ trực tiếp nào giữa dữ liệu đọc từ stream và dữ liệu ghi vào nó (ít nhất, stream không biết gì về mối quan hệ đó). Hãy nghĩ về một TCP socket, vốn chỉ gửi và nhận dữ liệu tới và từ peer ở xa; socket không nhận biết bất kỳ mối quan hệ nào giữa đầu vào và đầu ra. Hình 6.4 minh họa luồng dữ liệu trong một Duplex stream:



Figure 6.4: Duplex stream schematic representation On the other hand, Transform streams apply some kind of transformation to each chunk of data that they receive from their Writable side, and then make the transformed data available on their Readable side. Figure 6.5 shows how the data flows in a Transform stream:

Hình 6.4: Biểu diễn sơ đồ của Duplex stream Mặt khác, Transform stream áp dụng một kiểu biến đổi nào đó cho mỗi chunk dữ liệu mà chúng nhận từ phía Writable của mình, rồi làm cho dữ liệu đã biến đổi có sẵn ở phía Readable của chúng. Hình 6.5 cho thấy cách dữ liệu chảy trong một Transform stream:



Figure 6.5: Transform stream schematic representation

Hình 6.5: Biểu diễn sơ đồ của Transform stream

From the outside, the interface of a Transform stream is exactly like that of a Duplex stream. However, when we want to build a new Duplex stream, we have to provide both the `_read()` and `_write()` methods, while for implementing a new Transform stream, we have to fill in another pair of methods: `_transform()` and `_flush()`.

Nhìn từ bên ngoài, giao diện của một Transform stream giống hệt giao diện của một Duplex stream. Tuy nhiên, khi ta muốn xây dựng một Duplex stream mới, ta phải cung cấp cả phương thức `_read()` lẫn `_write()`, trong khi để hiện thực một Transform stream mới, ta phải điền vào một cặp phương thức khác: `_transform()` và `_flush()`.

Let's see how to create a new Transform stream with an example.

Hãy xem cách tạo một Transform stream mới qua một ví dụ.

Implementing Transform streams — Hiện thực Transform stream

Let's implement a Transform stream that replaces all the occurrences of a given string. To do this, we have to create a new module named `replaceStream.js`. Let's jump directly to the implementation:

Hãy hiện thực một Transform stream thay thế mọi lần xuất hiện của một chuỗi cho trước. Để làm điều này, ta phải tạo một module mới tên `replaceStream.js`. Hãy nhảy thẳng vào phần hiện thực:

```
import { Transform } from 'stream'
export class ReplaceStream extends Transform {
  constructor (searchStr, replaceStr, options) {
    super({ ...options })
    this.searchStr = searchStr
    this.replaceStr = replaceStr
    this.tail = ''
  }
  _transform (chunk, encoding, callback) {
    const pieces = (this.tail + chunk).split(this.searchStr) // (1)
    const lastPiece = pieces[pieces.length - 1] // (2)
    const tailLen = this.searchStr.length - 1
    this.tail = lastPiece.slice(-tailLen)
    pieces[pieces.length - 1] = lastPiece.slice(0, -tailLen)
    this.push(pieces.join(this.replaceStr)) // (3)
  }
}
```

```

        callback()
    }
    _flush (callback) {
        this.push(this.tail)
        callback()
    }
}

```

In this example, we created a new class extending the Transform base class. The constructor of the class accepts three arguments: `searchStr`, `replaceStr`, and `options`. As you can imagine, they allow us to define the text to match and the string to use as a replacement, plus an options object for advanced configuration of the underlying Transform stream. We also initialize an internal `tail` variable, which will be used later by the `_transform()` method.

Trong ví dụ này, ta đã tạo một lớp mới mở rộng lớp cơ sở Transform. Constructor của lớp chấp nhận ba đối số: `searchStr`, `replaceStr`, và `options`. Như bạn có thể hình dung, chúng cho phép ta định nghĩa văn bản cần khớp và chuỗi dùng làm thay thế, cộng với một đối tượng `options` để cấu hình nâng cao cho Transform stream bên dưới. Ta cũng khởi tạo một biến `tail` nội bộ, sẽ được phương thức `_transform()` dùng sau này.

Now, let's analyze the `_transform()` method, which is the core of our new class. The `_transform()` method has practically the same signature as the `_write()` method of the Writable stream, but instead of writing data into an underlying resource, it pushes it into the internal read buffer using `this.push()`, exactly as we would do in the `_read()` method of a Readable stream. This shows how the two sides of a Transform stream are connected.

Bây giờ, hãy phân tích phương thức `_transform()`, vốn là cốt lõi của lớp mới của ta. Phương thức `_transform()` có chữ ký gần như giống hệt phương thức `_write()` của Writable stream, nhưng thay vì ghi dữ liệu vào một tài nguyên bên dưới, nó đẩy dữ liệu vào buffer đọc nội bộ bằng `this.push()`, đúng như cách ta làm trong phương thức `_read()` của một Readable stream. Điều này cho thấy hai phía của một Transform stream được kết nối với nhau như thế nào.

The `_transform()` method of `ReplaceStream` implements the core of our algorithm. To search for and replace a string in a buffer is an easy task; however, it's a totally different story when the data is streaming, and possible matches might be distributed across multiple chunks. The procedure followed by the code can be explained as follows:

Phương thức `_transform()` của `ReplaceStream` hiện thực cốt lõi thuật toán của ta. Tìm kiếm và thay thế một chuỗi trong một buffer là một tác vụ dễ; tuy nhiên, đó lại là một câu chuyện hoàn toàn khác khi dữ liệu đang streaming, và các đoạn khớp có thể nằm rải rác qua nhiều chunk. Quy trình mà đoạn mã tuân theo có thể được giải thích như sau:

1. Our algorithm splits the data in memory (tail data and the current chunk) using `searchStr` as a separator.
2. Then, it takes the last item of the array generated by the operation and extracts the last `searchString.length - 1` characters. The result is saved in the `tail` variable and will be prepended to the next chunk of data.
3. Finally, all the pieces resulting from `split()` are joined together using `replaceStr` as a separator and pushed into the internal buffer.
 1. Thuật toán của ta tách dữ liệu trong bộ nhớ (dữ liệu `tail` và chunk hiện tại) bằng cách dùng `searchStr` làm dấu phân cách.
 2. Sau đó, nó lấy phần tử cuối cùng của mảng được tạo ra từ thao tác này và trích

`searchString.length - 1` ký tự cuối. Kết quả được lưu vào biến `tail` và sẽ được nối vào đầu chunk dữ liệu tiếp theo.

3. Cuối cùng, tất cả các mảnh thu được từ `split()` được nối lại với nhau bằng cách dùng `replaceStr` làm dấu phân cách và được đẩy vào buffer nội bộ.

When the stream ends, we might still have some content in the `tail` variable not pushed into the internal buffer. That's exactly what the `_flush()` method is for; it is invoked just before the stream is ended, and this is where we have one final chance to finalize the stream or push any remaining data before completely ending the stream. The `_flush()` method only takes in a callback, which we have to make sure to invoke when all the operations are complete, causing the stream to be terminated. With this, we have completed our `ReplaceStream` class.

Khi stream kết thúc, ta vẫn có thể còn một ít nội dung trong biến `tail` chưa được đẩy vào buffer nội bộ. Đó chính là mục đích của phương thức `_flush()`; nó được gọi ngay trước khi stream kết thúc, và đây là nơi ta có một cơ hội cuối cùng để hoàn tất stream hoặc đẩy nốt dữ liệu còn lại trước khi kết thúc hoàn toàn stream. Phương thức `_flush()` chỉ nhận một callback, mà ta phải đảm bảo gọi khi tất cả các thao tác hoàn tất, khiến stream được kết thúc. Với điều này, ta đã hoàn thành lớp `ReplaceStream` của mình.

Now, it's time to try the new stream. Let's create a script that writes some data into the stream and then reads the transformed result:

Bây giờ, đến lúc thử stream mới. Hãy tạo một script ghi một ít dữ liệu vào stream rồi đọc kết quả đã biến đổi:

```
import { ReplaceStream } from './replace-stream.js'
const replaceStream = new ReplaceStream('World', 'Node.js')
replaceStream.on('data', chunk => console.log(chunk.toString()))
replaceStream.write('Hello W')
replaceStream.write('orld!')
replaceStream.end()
```

To make life a little bit harder for our stream, we spread the search term (which is `World`) across two different chunks, then, using the flowing mode, we read from the same stream, logging each transformed chunk. Running the preceding program should produce the following output:

Để làm khó stream của ta hơn một chút, ta rải thuật ngữ tìm kiếm (vốn là `World`) qua hai chunk khác nhau, rồi, dùng chế độ flowing, ta đọc từ chính stream đó, ghi log mỗi chunk đã biến đổi. Chạy chương trình trên sẽ tạo ra kết quả sau:

```
Hel
lo Node.js
!
```

Please note that the preceding output is broken into multiple lines because we are using `console.log()` to print it out. This allows us to demonstrate that our implementation is able to replace string matches correctly, even when the matching text spans multiple chunks of data.

Xin lưu ý rằng kết quả trên bị chia thành nhiều dòng vì ta dùng `console.log()` để in nó ra. Điều này cho phép ta minh họa rằng hiện thực của ta có khả năng thay thế các đoạn chuỗi khớp một cách chính xác, ngay cả khi văn bản khớp trải dài qua nhiều chunk dữ liệu.

Simplified construction — Cách dựng đơn giản hóa Unsurprisingly, even `Transform` streams support simplified construction. At this point, we should have developed an instinct for how this API

might look, so let's get our hands dirty straight away and rewrite the previous example using this approach:

Không có gì ngạc nhiên, ngay cả Transform stream cũng hỗ trợ cách dựng đơn giản hóa. Tại thời điểm này, lẽ ra ta đã hình thành được linh cảm về việc API này có thể trông như thế nào, nên hãy bắt tay vào việc ngay và viết lại ví dụ trước bằng cách tiếp cận này:

```
const searchStr = 'World'
const replaceStr = 'Node.js'
let tail = ''
const replaceStream = new Transform({
  defaultEncoding: 'utf8',
  transform (chunk, encoding, cb) {
    const pieces = (tail + chunk).split(searchStr)
    const lastPiece = pieces[pieces.length - 1]
    const taillen = searchStr.length - 1
    tail = lastPiece.slice(-taillen)
    pieces[pieces.length - 1] = lastPiece.slice(0, -taillen)
    this.push(pieces.join(replaceStr))
    cb()
  },
  flush (cb) {
    this.push(tail)
    cb()
  }
})
// now write to replaceStream ...
```

As expected, simplified construction works by directly instantiating a new Transform object and passing our specific transformation logic through the transform() and flush() functions directly through the options object. Note that transform() and flush() don't have a prepended underscore here.

Đúng như dự đoán, cách dựng đơn giản hóa hoạt động bằng cách trực tiếp khởi tạo một đối tượng Transform mới và truyền logic biến đổi cụ thể của ta qua các hàm transform() và flush() trực tiếp thông qua đối tượng options. Lưu ý rằng transform() và flush() không có dấu gạch dưới ở đầu ở đây.

Filtering and aggregating data with Transform streams — Lọc và tổng hợp dữ liệu với Transform stream

As we mentioned in the previous section, Transform streams are the perfect building blocks for implementing data transformation pipelines. In the previous section, we illustrated an example of a Transform stream that can replace words in a stream of text. But Transform streams can be used to implement other types of data transformation as well. For instance, it's quite common to use Transform streams to implement data filtering and data aggregation.

Như đã đề cập trong phần trước, Transform stream là những khối xây dựng hoàn hảo để hiện thực các pipeline biến đổi dữ liệu. Trong phần trước, ta minh họa một ví dụ về một Transform stream có thể thay thế các từ trong một stream văn bản. Nhưng Transform stream cũng có thể được dùng để hiện thực các loại biến đổi dữ liệu khác. Chẳng hạn, việc dùng Transform stream để hiện thực lọc dữ liệu và tổng hợp dữ liệu là khá phổ biến.

Just to make a practical example, let's imagine we are asked by a Fortune 500 company to analyze a big file containing all the sales for the year 2020. The company wants us to use `data.csv`, a sales report in CSV format, to calculate the total profit for the sales made in Italy. For simplicity, let's imagine the sales data that is stored in the CSV file contains three fields per line: item type, country of sale, and profit. So, such a file could look like this:

Chỉ để có một ví dụ thực tế, hãy tưởng tượng ta được một công ty trong danh sách Fortune 500 yêu cầu phân tích một tập tin lớn chứa tất cả doanh số bán hàng của năm 2020. Công ty muốn ta dùng `data.csv`, một báo cáo doanh số ở định dạng CSV, để tính tổng lợi nhuận của các đơn hàng bán tại Ý. Cho đơn giản, hãy tưởng tượng dữ liệu doanh số được lưu trong tập tin CSV chứa ba trường mỗi dòng: loại mặt hàng, quốc gia bán, và lợi nhuận. Vậy nên, một tập tin như vậy có thể trông như sau:

```
type, country, profit
Household, Namibia, 597290.92
Baby Food, Iceland, 808579.10
Meat, Russia, 277305.60
Meat, Italy, 413270.00
Cereal, Malta, 174965.25
Meat, Indonesia, 145402.40
Household, Italy, 728880.54
[... many more lines]
```

Now, it's clear that we have to find all the records that have “Italy” as country and, in the process, sum up the profit value of the matching lines into a single number.

Bây giờ, rõ ràng ta phải tìm tất cả các bản ghi có country là “Italy” và, trong quá trình đó, cộng dồn giá trị profit của các dòng khớp vào một con số duy nhất.

In order to process a CSV file in a streaming fashion, we can use the excellent `csv-parse` module (nodejsdp.link/csv-parse).

Để xử lý một tập tin CSV theo kiểu streaming, ta có thể dùng module `csv-parse` tuyệt vời (nodejsdp.link/csv-parse).

If we assume for a moment that we have already implemented our custom streams to filter and aggregate the data, a possible solution to this task might look like this:

Nếu ta tạm giả định rằng ta đã hiện thực xong các stream tùy chỉnh để lọc và tổng hợp dữ liệu, một giải pháp khả dĩ cho tác vụ này có thể trông như sau:

```
import { createReadStream } from 'fs'
import parse from 'csv-parse'
import { FilterByCountry } from './filter-by-country.js'
import { SumProfit } from './sum-profit.js'
const csvParser = parse({ columns: true })
createReadStream('data.csv') // (1)
  .pipe(csvParser) // (2)
  .pipe(new FilterByCountry('Italy')) // (3)
  .pipe(new SumProfit()) // (4)
  .pipe(process.stdout) // (5)
```

The streaming pipeline proposed here consists of five steps:

Pipeline streaming đề xuất ở đây gồm năm bước:

1. We read the source CSV file as a stream.

2. We use the `csv-parse` library to parse every line of the document as a CSV record. For every line, this stream will emit an object containing the properties `type`, `country`, and `profit`.
3. We filter all the records by country, retaining only the records that match the country “Italy.” All the records that don’t match “Italy” are dropped, which means that they will not be passed to the other steps in the pipeline. Note that this is one of the custom Transform streams that we have to implement.
4. For every record, we accumulate the profit. This stream will eventually emit one single string, which represents the value of the total profit for products sold in Italy. This value will be emitted by the stream only when all the data from the original file has been completely processed. Note that this is the second custom Transform stream that we have to implement to complete this project.
5. Finally, the data emitted from the previous step is displayed in the standard output.

1. Ta đọc tập tin CSV nguồn dưới dạng một stream.
2. Ta dùng thư viện `csv-parse` để phân tích từng dòng của tài liệu thành một bản ghi CSV. Với mỗi dòng, stream này sẽ phát ra một object chứa các thuộc tính `type`, `country`, và `profit`.
3. Ta lọc tất cả các bản ghi theo quốc gia, chỉ giữ lại các bản ghi khớp với quốc gia “Italy.” Tất cả các bản ghi không khớp “Italy” bị loại bỏ, nghĩa là chúng sẽ không được chuyển tới các bước khác trong pipeline. Lưu ý rằng đây là một trong những Transform stream tùy chỉnh mà ta phải hiện thực.
4. Với mỗi bản ghi, ta cộng dồn lợi nhuận. Stream này cuối cùng sẽ phát ra một chuỗi duy nhất, biểu diễn giá trị tổng lợi nhuận cho các sản phẩm bán tại Ý. Giá trị này chỉ được stream phát ra khi tất cả dữ liệu từ tập tin gốc đã được xử lý hoàn toàn. Lưu ý rằng đây là Transform stream tùy chỉnh thứ hai mà ta phải hiện thực để hoàn thành dự án này.
5. Cuối cùng, dữ liệu phát ra từ bước trước được hiển thị trên đầu ra chuẩn.

Now, let’s implement the `FilterByCountry` stream:

Bây giờ, hãy hiện thực stream `FilterByCountry`:

```
import { Transform } from 'stream'
export class FilterByCountry extends Transform {
  constructor (country, options = {}) {
    options.objectMode = true
    super(options)
    this.country = country
  }
  _transform (record, enc, cb) {
    if (record.country === this.country) {
      this.push(record)
    }
    cb()
  }
}
```

`FilterByCountry` is a custom Transform stream. We can see that the constructor accepts an argument called `country`, which allows us to specify the country name we want to filter on. In the constructor, we also set the stream to run in `objectMode` because we know it will be used to process objects (records from the CSV file).

`FilterByCountry` là một Transform stream tùy chỉnh. Ta có thể thấy rằng constructor

chấp nhận một đối số tên `country`, cho phép ta chỉ định tên quốc gia mà ta muốn lọc theo. Trong constructor, ta cũng đặt stream chạy ở `objectMode` vì ta biết nó sẽ được dùng để xử lý các object (bản ghi từ tập tin CSV).

In the `_transform` method, we check if the country of the current record matches the country specified at construction time. If it's a match, then we pass the record on to the next stage of the pipeline by calling `this.push()`. Regardless of whether the record matches or not, we need to invoke `cb()` to indicate that the current record has been successfully processed and that the stream is ready to receive another record. Pattern: Transform filter

Trong phương thức `_transform`, ta kiểm tra xem quốc gia của bản ghi hiện tại có khớp với quốc gia được chỉ định lúc dựng hay không. Nếu khớp, thì ta chuyển bản ghi sang giai đoạn tiếp theo của pipeline bằng cách gọi `this.push()`. Bất kể bản ghi có khớp hay không, ta cần gọi `cb()` để cho biết rằng bản ghi hiện tại đã được xử lý thành công và stream sẵn sàng nhận một bản ghi khác. Pattern: Lọc bằng Transform

Invoke `this.push()` in a conditional way to allow only some data to reach the next stage of the pipeline.

Gọi `this.push()` theo cách có điều kiện để chỉ cho phép một số dữ liệu tới được giai đoạn tiếp theo của pipeline.

Finally, let's implement the `SumProfit` filter:

Cuối cùng, hãy hiện thực bộ lọc `SumProfit`:

```
import { Transform } from 'stream'
export class SumProfit extends Transform {
  constructor (options = {}) {
    options.objectMode = true
    super(options)
    this.total = 0
  }
  _transform (record, enc, cb) {
    this.total += Number.parseFloat(record.profit)
    cb()
  }
  _flush (cb) {
    this.push(this.total.toString())
    cb()
  }
}
```

This stream needs to run in `objectMode` as well, because it will receive objects representing records from the CSV file. Note that, in the constructor, we also initialize an instance variable called `total` and we set its value to `0`.

Stream này cũng cần chạy ở `objectMode`, vì nó sẽ nhận các object biểu diễn các bản ghi từ tập tin CSV. Lưu ý rằng, trong constructor, ta cũng khởi tạo một biến instance tên `total` và đặt giá trị của nó thành `0`.

In the `_transform()` method, we process every record and use the current profit value to increase the `total`. It's important to note that this time, we are not calling `this.push()`. This means that no value is emitted while the data is flowing through the stream. We still need to call `cb()`, though, to indicate that the current record has been processed and the stream is ready to receive another one.

Trong phương thức `_transform()`, ta xử lý mỗi bản ghi và dùng giá trị `profit` hiện tại để tăng `total`. Điều quan trọng cần lưu ý là lần này, ta không gọi `this.push()`. Nghĩa là không có giá trị nào được phát ra trong khi dữ liệu đang chảy qua stream. Tuy nhiên, ta vẫn cần gọi `cb()` để cho biết rằng bản ghi hiện tại đã được xử lý và stream sẵn sàng nhận một bản ghi khác.

In order to emit the final result when all the data has been processed, we have to define a custom flush behavior using the `_flush()` method. Here, we finally call `this.push()` to emit a string representation of the resulting `total` value. Remember that `_flush()` is automatically invoked before the stream is closed. Pattern: Streaming aggregation

Để phát ra kết quả cuối cùng khi tất cả dữ liệu đã được xử lý, ta phải định nghĩa một hành vi flush tùy chỉnh bằng phương thức `_flush()`. Ở đây, ta cuối cùng gọi `this.push()` để phát ra một biểu diễn chuỗi của giá trị `total` thu được. Hãy nhớ rằng `_flush()` được tự động gọi trước khi stream đóng. Pattern: Tổng hợp kiểu streaming

Use `_transform()` to process the data and accumulate the partial result, then call `this.push()` only in the `_flush()` method to emit the result when all the data has been processed.

Dùng `_transform()` để xử lý dữ liệu và tích lũy kết quả từng phần, rồi chỉ gọi `this.push()` trong phương thức `_flush()` để phát ra kết quả khi tất cả dữ liệu đã được xử lý.

This completes our example. Now, you can grab the CSV file from the code repository and execute this program to see what the total profit for Italy is. No surprise it's going to be a lot of money since we are talking about the profit of a Fortune 500 company!

Điều này hoàn thành ví dụ của chúng ta. Bây giờ, bạn có thể lấy tập tin CSV từ kho mã nguồn và thực thi chương trình này để xem tổng lợi nhuận cho Ý là bao nhiêu. Không có gì bất ngờ, nó sẽ là một khoản tiền rất lớn vì ta đang nói tới lợi nhuận của một công ty Fortune 500!

PassThrough streams — PassThrough stream

There is a fifth type of stream that is worth mentioning: `PassThrough`. This type of stream is a special type of `Transform` that outputs every data chunk without applying any transformation.

Có một loại stream thứ năm đáng nhắc tới: `PassThrough`. Loại stream này là một loại `Transform` đặc biệt xuất ra mọi chunk dữ liệu mà không áp dụng bất kỳ phép biến đổi nào.

`PassThrough` is possibly the most underrated type of stream, but there are actually several circumstances in which it can be a very valuable tool in our toolbelt. For instance, `PassThrough` streams can be useful for observability or to implement late piping and lazy stream patterns.

`PassThrough` có lẽ là loại stream bị đánh giá thấp nhất, nhưng thực ra có một số tình huống trong đó nó có thể là một công cụ rất giá trị trong bộ đồ nghề của ta. Chẳng hạn, `PassThrough` stream có thể hữu ích cho khả năng quan sát (observability) hoặc để hiện thực các mẫu late piping và lazy stream.

Observability — Khả năng quan sát

If we want to observe how much data is flowing through one or more streams, we could do so by attaching a data event listener to a `PassThrough` instance and then piping this instance in a given point of a stream pipeline. Let's see a simplified example to be able to appreciate this concept:

Nếu ta muốn quan sát có bao nhiêu dữ liệu đang chảy qua một hoặc nhiều stream, ta có thể làm vậy bằng cách gắn một listener cho sự kiện data vào một instance PassThrough rồi nối ống (pipe) instance này vào một điểm cho trước của một pipeline stream. Hãy xem một ví dụ đơn giản hóa để có thể cảm nhận khái niệm này:

```
import { PassThrough } from 'stream'
let bytesWritten = 0
const monitor = new PassThrough()
monitor.on('data', (chunk) => {
  bytesWritten += chunk.length
})
monitor.on('finish', () => {
  console.log(`${bytesWritten} bytes written`)
})
monitor.write('Hello!')
monitor.end()
```

In this example, we are creating a new instance of PassThrough and using the data event to count how many bytes are flowing through the stream. We also use the finish event to dump the total amount to the console. Finally we write some data directly into the stream using write() and end(). This is just an illustrative example; in a more realistic scenario, we would be piping our monitor instance in a given point of a stream pipeline. For instance, if we wanted to monitor how many bytes are written to disk in our first file compression example of this chapter, we could easily achieve that by doing something like this:

Trong ví dụ này, ta đang tạo một instance mới của PassThrough và dùng sự kiện data để đếm có bao nhiêu byte đang chảy qua stream. Ta cũng dùng sự kiện finish để in tổng lượng ra console. Cuối cùng ta ghi một ít dữ liệu trực tiếp vào stream bằng write() và end(). Đây chỉ là một ví dụ minh họa; trong một kịch bản thực tế hơn, ta sẽ nối ống instance monitor của mình vào một điểm cho trước của một pipeline stream. Chẳng hạn, nếu ta muốn theo dõi có bao nhiêu byte được ghi xuống đĩa trong ví dụ nén tập tin đầu tiên của chương này, ta có thể dễ dàng đạt được điều đó bằng cách làm như sau:

```
createReadStream(filename)
  .pipe(createGzip())
  .pipe(monitor)
  .pipe(createWriteStream(`${filename}.gz`))
```

The beauty of this approach is that we didn't have to touch any of the other existing streams in the pipeline, so if we need to observe other parts of the pipeline (for instance, imagine we want to know the number of bytes of the uncompressed data), we can move monitor around with very little effort.

Vẻ đẹp của cách tiếp cận này là ta không phải động tới bất kỳ stream hiện có nào khác trong pipeline, nên nếu ta cần quan sát các phần khác của pipeline (chẳng hạn, hãy tưởng tượng ta muốn biết số byte của dữ liệu chưa nén), ta có thể di chuyển monitor đi chỗ khác với rất ít công sức.

Note that you could implement an alternative version of the monitor stream by using a custom transform stream instead. In such a case, you would have to make sure that the received chunks are pushed without any modification or delay, which is something that a PassThrough stream would do automatically for you. Both approaches are equally valid, so pick the approach that feels more natural to you.

Lưu ý rằng bạn có thể hiện thực một phiên bản thay thế của stream monitor bằng cách

dùng một transform stream tùy chỉnh. Trong trường hợp đó, bạn sẽ phải đảm bảo rằng các chunk nhận được được đẩy đi mà không có bất kỳ sửa đổi hay trì hoãn nào, điều mà một PassThrough stream tự động làm thay bạn. Cả hai cách tiếp cận đều hợp lệ như nhau, nên hãy chọn cách nào cảm thấy tự nhiên hơn với bạn.

Late piping — Late piping

In some circumstances, we might have to use APIs that accept a stream as an input parameter. This is generally not a big deal because we already know how to create and use streams. However, it may get a little bit more complicated if the data we want to read or write through the stream is available after we've called the given API.

Trong một số tình huống, ta có thể phải dùng các API nhận một stream làm tham số đầu vào. Điều này nhìn chung không phải vấn đề lớn vì ta đã biết cách tạo và dùng stream. Tuy nhiên, nó có thể phức tạp hơn một chút nếu dữ liệu ta muốn đọc hoặc ghi qua stream chỉ có sẵn sau khi ta đã gọi API đó.

To visualize this scenario in more concrete terms, let's imagine that we have to use an API that gives us the following function to upload a file to a data storage service:

Để hình dung kịch bản này một cách cụ thể hơn, hãy tưởng tượng rằng ta phải dùng một API cung cấp cho ta hàm sau để tải một tập tin lên một dịch vụ lưu trữ dữ liệu:

```
function upload (filename, contentStream) {  
  // ...  
}
```

This function is effectively a simplified version of what is commonly available in the SDK of file storage services like Amazon Simple Storage Service (S3) or Azure Blob Storage service. Often, those libraries will provide the user with a more flexible function that can receive the content data in different formats (for example, a string, a buffer, or a Readable stream).

Hàm này thực chất là một phiên bản đơn giản hóa của những gì thường có sẵn trong SDK của các dịch vụ lưu trữ tập tin như Amazon Simple Storage Service (S3) hoặc dịch vụ Azure Blob Storage. Thường thì các thư viện đó sẽ cung cấp cho người dùng một hàm linh hoạt hơn có thể nhận dữ liệu nội dung ở các định dạng khác nhau (ví dụ, một chuỗi, một buffer, hoặc một Readable stream).

Now, if we want to upload a file from the filesystem, this is a trivial operation, and we can do something like this:

Bây giờ, nếu ta muốn tải một tập tin từ hệ thống tập tin, đây là một thao tác tầm thường, và ta có thể làm như sau:

```
import { createReadStream } from 'fs'  
upload('a-picture.jpg', createReadStream('/path/to/a-picture.jpg'))
```

But what if we want to do some processing to the file stream before the upload. For instance, let's say we want to compress or encrypt the data? Also, what if we have to do this transformation asynchronously after the upload function has been called?

Nhưng nếu ta muốn xử lý một chút lên stream tập tin trước khi tải lên thì sao? Chẳng hạn, giả sử ta muốn nén hoặc mã hóa dữ liệu? Ngoài ra, nếu ta phải thực hiện phép biến đổi này một cách bất đồng bộ sau khi hàm upload đã được gọi thì sao?

In such cases, we can provide a PassThrough stream to the upload () function, which will effectively act as a placeholder. The internal implementation of upload () will immediately try to consume data

from it, but there will be no data available in the stream until we actually write to it. Also, the stream won't be considered complete until we close it, so the `upload()` function will have to wait for data to flow through the `PassThrough` instance to initiate the upload.

Trong những trường hợp như vậy, ta có thể cung cấp một `PassThrough` stream cho hàm `upload()`, nó sẽ đóng vai trò như một placeholder. Phần hiện thực nội bộ của `upload()` sẽ lập tức thử tiêu thụ dữ liệu từ nó, nhưng sẽ không có dữ liệu nào trong stream cho đến khi ta thực sự ghi vào nó. Ngoài ra, stream sẽ không được coi là hoàn tất cho đến khi ta đóng nó, nên hàm `upload()` sẽ phải chờ dữ liệu chảy qua instance `PassThrough` để khởi tạo việc tải lên.

Let's see a possible command-line script that uses this approach to upload a file from the filesystem and also compresses it using the Brotli compression. We are going to assume that the third-party `upload()` function is provided in a file called `upload.js`:

Hãy xem một script dòng lệnh khả dĩ dùng cách tiếp cận này để tải một tập tin từ hệ thống tập tin và đồng thời nén nó bằng nén Brotli. Ta sẽ giả định rằng hàm `upload()` của bên thứ ba được cung cấp trong một tập tin tên `upload.js`:

```
import { createReadStream } from 'fs'
import { createBrotliCompress } from 'zlib'
import { PassThrough } from 'stream'
import { basename } from 'path'
import { upload } from './upload.js'
const filepath = process.argv[2] // (1)
const filename = basename(filepath)
const contentStream = new PassThrough() // (2)
upload(`${filename}.br`, contentStream) // (3)
  .then((response) => {
    console.log(`Server response: ${response.data}`)
  })
  .catch((err) => {
    console.error(err)
    process.exit(1)
  })
createReadStream(filepath) // (4)
  .pipe(createBrotliCompress())
  .pipe(contentStream)
```

In this book's repository, you will find a complete implementation of this example that allows you to upload files to an HTTP server that you can run locally.

Trong kho mã của cuốn sách này, bạn sẽ tìm thấy một hiện thực đầy đủ của ví dụ này cho phép bạn tải tập tin lên một máy chủ HTTP mà bạn có thể chạy cục bộ.

Let's review what's happening in the previous example:

Hãy điểm lại những gì đang diễn ra trong ví dụ trước:

1. We get the path to the file we want to upload from the first command-line argument and use `basename` to extrapolate the filename from the given path.
2. We create a placeholder for our content stream as a `PassThrough` instance.
3. Now, we invoke the `upload` function by passing our filename (with the added `.br` suffix, indicating that it is using the Brotli compression) and the placeholder content stream.

4. Finally, we create a pipeline by chaining a filesystem Readable stream, a Brotli compression Transform stream, and finally our content stream as the destination.

1. Ta lấy đường dẫn tới tập tin muốn tải lên từ đối số dòng lệnh đầu tiên và dùng `basename` để rút ra tên tập tin từ đường dẫn cho trước.
2. Ta tạo một placeholder cho content stream của mình dưới dạng một instance `PassThrough`.
3. Bây giờ, ta gọi hàm `upload` bằng cách truyền tên tập tin (kèm hậu tố `.br`, cho biết nó dùng nén Brotli) và content stream placeholder.
4. Cuối cùng, ta tạo một pipeline bằng cách nối chuỗi một Readable stream của hệ thống tập tin, một Transform stream nén Brotli, và cuối cùng là content stream của ta làm đích đến.

When this code is executed, the upload will start as soon as we invoke the `upload()` function (possibly establishing a connection to the remote server), but the data will start to flow only later, when our pipeline is initialized. Note that our pipeline will also close the `contentStream` when the processing completes, which will indicate to the `upload()` function that all the content has been fully consumed.

Khi đoạn mã này được thực thi, việc tải lên sẽ bắt đầu ngay khi ta gọi hàm `upload()` (có thể thiết lập một kết nối tới máy chủ ở xa), nhưng dữ liệu sẽ chỉ bắt đầu chảy sau đó, khi pipeline của ta được khởi tạo. Lưu ý rằng pipeline của ta cũng sẽ đóng `contentStream` khi việc xử lý hoàn tất, điều này sẽ báo cho hàm `upload()` biết rằng tất cả nội dung đã được tiêu thụ hoàn toàn.

Pattern

Pattern

Use a `PassThrough` stream when you need to provide a placeholder for data that will be read or written in the future. We can also use this pattern to transform the interface of the `upload()` function. Instead of accepting a Readable stream as input, we can make it return a Writeable stream, which can then be used to provide the data we want to upload:

Dùng một `PassThrough` stream khi bạn cần cung cấp một placeholder cho dữ liệu sẽ được đọc hoặc ghi trong tương lai. Ta cũng có thể dùng pattern này để biến đổi giao diện của hàm `upload()`. Thay vì chấp nhận một Readable stream làm đầu vào, ta có thể làm cho nó trả về một Writeable stream, sau đó có thể dùng để cung cấp dữ liệu mà ta muốn tải lên:

```
function createUploadStream (filename) {  
  // ...  
  // returns a writable stream that can be used to upload data  
}
```

If we were tasked to implement this function, we could achieve that in a very elegant way by using a `PassThrough` instance, as in the following example implementation:

Nếu được giao nhiệm vụ hiện thực hàm này, ta có thể đạt được điều đó một cách rất thanh lịch bằng cách dùng một instance `PassThrough`, như trong ví dụ hiện thực sau:

```
function createUploadStream (filename) {  
  const connector = new PassThrough()  
  upload(filename, connector)  
  return connector  
}
```


In the preceding code, we are using a `PassThrough` stream as a connector. This stream becomes a perfect abstraction for a case where the consumer of the library can write data at any later stage.

Trong đoạn mã trên, ta đang dùng một `PassThrough` stream làm bộ kết nối (connector). Stream này trở thành một sự trừu tượng hoàn hảo cho trường hợp người tiêu dùng thư viện có thể ghi dữ liệu ở bất kỳ giai đoạn nào sau này.

The `createUploadStream()` function can then be used as follows:

Hàm `createUploadStream()` sau đó có thể được dùng như sau:

```
const upload = createUploadStream('a-file.txt')
upload.write('Hello World')
upload.end()
```

This book's repository also contains an HTTP upload example that adopts this alternative pattern.

Kho mã của cuốn sách này cũng chứa một ví dụ tải lên qua HTTP áp dụng pattern thay thế này.

Lazy streams — Lazy stream

Sometimes, we need to create a large number of streams at the same time, for example, to pass them to a function for further processing. A typical example is when using `archiver` (nodejsdp.link/archiver), a **package** for creating archives such as TAR and ZIP. The `archiver` package allows you to create an archive from a set of streams, representing the files to add. The problem is that if we want to pass a large number of streams, such as from files on the filesystem, we would likely get an `EMFILE, too many open files` error. This is because functions like `createReadStream()` from the `fs` module will actually open a file descriptor every time a new stream is created, even before you start to read from those streams. In more generic terms, creating a stream instance might initialize expensive operations straight away (for example, open a file or a socket, initialize a connection to a database, and so on), even before we actually start to use such a stream. This might not be desirable if you are creating a large number of stream instances for later consumption.

Đôi khi, ta cần tạo một số lượng lớn stream cùng lúc, ví dụ, để truyền chúng cho một hàm xử lý tiếp. Một ví dụ điển hình là khi dùng `archiver` (nodejsdp.link/archiver), một gói để tạo các kho lưu trữ như TAR và ZIP. Gói `archiver` cho phép bạn tạo một kho lưu trữ từ một tập các stream, đại diện cho các tập tin cần thêm vào. Vấn đề là nếu ta muốn truyền một số lượng lớn stream, chẳng hạn từ các tập tin trên hệ thống tập tin, ta nhiều khả năng sẽ gặp lỗi `EMFILE, too many open files`. Điều này là vì các hàm như `createReadStream()` từ module `fs` thực ra sẽ mở một file descriptor mỗi khi một stream mới được tạo, ngay cả trước khi bạn bắt đầu đọc từ những stream đó. Nói theo cách tổng quát hơn, việc tạo một instance stream có thể khởi tạo ngay các thao tác tốn kém (ví dụ, mở một tập tin hoặc một socket, khởi tạo một kết nối tới cơ sở dữ liệu, v.v.), ngay cả trước khi ta thực sự bắt đầu dùng stream đó. Điều này có thể không mong muốn nếu bạn đang tạo một số lượng lớn instance stream để tiêu thụ sau này.

In these cases, you might want to delay the expensive initialization until you actually need to consume data from the stream.

Trong những trường hợp này, bạn có thể muốn trì hoãn việc khởi tạo tốn kém cho đến khi bạn thực sự cần tiêu thụ dữ liệu từ stream.

It is possible to achieve this by using a library like `lazystream` ([nodejsdp.link/ lazystream](https://nodejsdp.link/lazystream)). This library allows you to effectively create proxies for actual stream instances, where the proxied instance is not created until some piece of code is actually starting to consume data from the **proxy**.

Có thể đạt được điều này bằng cách dùng một thư viện như `lazystream` ([nodejsdp.link/ lazystream](https://nodejsdp.link/lazystream)). Thư viện này cho phép bạn tạo một cách hiệu quả các proxy cho các instance stream thực sự, nơi instance được proxy không được tạo cho đến khi một đoạn mã nào đó thực sự bắt đầu tiêu thụ dữ liệu từ proxy.

In the following example, `lazystream` allows us to create a lazy Readable stream for the special Unix file `/dev/urandom`:

Trong ví dụ sau, `lazystream` cho phép ta tạo một Readable stream lười cho tập tin Unix đặc biệt `/dev/urandom`:

```
import lazystream from 'lazystream'
const lazyURandom = new lazystream.Readable(function (options) {
  return fs.createReadStream('/dev/urandom')
})
```

The function we pass as a parameter to `new lazystream.Readable()` is effectively a factory function that generates the proxied stream when necessary.

Hàm mà ta truyền làm tham số cho `new lazystream.Readable()` thực chất là một hàm factory sinh ra stream được proxy khi cần thiết.

Behind the scenes, `lazystream` is implemented using a `PassThrough` stream that, only when its `_read()` method is invoked for the first time, creates the proxied instance by invoking the factory function, and pipes the generated stream into the `PassThrough` itself. The code that consumes the stream is totally agnostic of the proxying that is happening here, and it will consume the data as if it was flowing directly from the `PassThrough` stream. `lazystream` implements a similar utility to create lazy Writable streams as well.

Đằng sau hậu trường, `lazystream` được hiện thực bằng một `PassThrough` stream mà, chỉ khi phương thức `_read()` của nó được gọi lần đầu tiên, sẽ tạo instance được proxy bằng cách gọi hàm factory, và nối ống stream được sinh ra vào chính `PassThrough` đó. Đoạn mã tiêu thụ stream hoàn toàn không hay biết gì về việc proxy đang diễn ra ở đây, và nó sẽ tiêu thụ dữ liệu như thể nó chảy trực tiếp từ `PassThrough` stream. `lazystream` cũng hiện thực một tiện ích tương tự để tạo các Writable stream lười.

Creating lazy Readable and Writable streams from scratch could be an interesting exercise that is left to you. If you get stuck, have a look at the source code of `lazystream` for inspiration on how to implement this pattern.

Tạo các Readable và Writable stream lười từ đầu có thể là một bài tập thú vị dành cho bạn. Nếu bạn bí, hãy xem mã nguồn của `lazystream` để lấy cảm hứng về cách hiện thực pattern này.

In the next section, we will discuss the `.pipe()` method in greater detail and also see other ways to connect different streams to form a processing pipeline.

Trong phần tiếp theo, ta sẽ bàn về phương thức `.pipe()` chi tiết hơn và cũng xem các cách khác để kết nối các stream khác nhau tạo thành một pipeline xử lý.

Connecting streams using pipes — Kết nối các stream bằng pipe

The concept of Unix pipes was invented by Douglas McIlroy. This enabled the output of a program to be connected to the input of the next. Take a look at the following command:

Khái niệm Unix pipe được phát minh bởi Douglas McIlroy. Điều này cho phép đầu ra của một chương trình được kết nối với đầu vào của chương trình kế tiếp. Hãy xem lệnh sau:

```
echo Hello World! | sed s/World/Node.js/g
```

In the preceding command, `echo` will write `Hello World!` to its standard output, which is then redirected to the standard input of the `sed` command (thanks to the pipe `|` operator). Then, `sed` replaces any occurrence of `World` with `Node.js` and prints the result to its standard output (which, this time, is the console).

Trong lệnh trên, `echo` sẽ ghi `Hello World!` ra đầu ra chuẩn của nó, sau đó được chuyển hướng tới đầu vào chuẩn của lệnh `sed` (nhờ toán tử pipe `|`). Rồi, `sed` thay thế mọi lần xuất hiện của `World` bằng `Node.js` và in kết quả ra đầu ra chuẩn của nó (lần này là console).

In a similar way, Node.js streams can be connected using the `pipe()` method of the `Readable` stream, which has the following interface:

Theo cách tương tự, các stream của Node.js có thể được kết nối bằng phương thức `pipe()` của `Readable` stream, vốn có giao diện sau:

```
readable.pipe(writable, [options])
```

Very intuitively, the `pipe()` method takes the data that is emitted from the readable stream and pumps it into the provided writable stream. Also, the writable stream is ended automatically when the readable stream emits an end event (unless we specify `{end: false}` as options). The `pipe()` method returns the writable stream passed in the first argument, allowing us to create chained invocations if such a stream is also `Readable` (such as a `Duplex` or `Transform` stream).

Rất trực quan, phương thức `pipe()` lấy dữ liệu được phát ra từ stream `readable` và bơm nó vào stream `writable` được cung cấp. Ngoài ra, stream `writable` được tự động kết thúc khi stream `readable` phát ra sự kiện `end` (trừ khi ta chỉ định `{end: false}` làm options). Phương thức `pipe()` trả về stream `writable` được truyền ở đối số đầu tiên, cho phép ta tạo các lời gọi nối chuỗi nếu stream đó cũng là `Readable` (chẳng hạn một `Duplex` hoặc `Transform` stream).

Piping two streams together will create suction, which allows the data to flow automatically to the writable stream, so there is no need to call `read()` or `write()`, but most importantly, there is no need to control the backpressure anymore, because it's automatically taken care of.

Nối ống hai stream với nhau sẽ tạo ra một lực hút, cho phép dữ liệu tự động chảy tới stream `writable`, nên không cần gọi `read()` hay `write()`, nhưng quan trọng nhất, không cần kiểm soát backpressure nữa, vì nó được lo liệu tự động.

To provide a quick example, we can create a new module that takes a text stream from the standard input, applies the replace transformation discussed earlier when we built our custom `ReplaceStream`, and then pushes the data back to the standard output:

Để cho một ví dụ nhanh, ta có thể tạo một module mới nhận một stream văn bản từ đầu vào chuẩn, áp dụng phép biến đổi thay thế đã bàn trước đó khi ta xây dựng `ReplaceStream` tùy chỉnh, rồi đẩy dữ liệu trở lại ra đầu ra chuẩn:

```
// replace.js
import { ReplaceStream } from './replace-stream.js'
```

```
process.stdin
  .pipe(new ReplaceStream(process.argv[2], process.argv[3]))
  .pipe(process.stdout)
```

The preceding program pipes the data that comes from the standard input into an instance of `ReplaceStream` and then back to the standard output. Now, to try this small application, we can leverage a Unix pipe to redirect some data into its standard input, as shown in the following example:

Chương trình trên nối ống dữ liệu đến từ đầu vào chuẩn vào một instance của `ReplaceStream` rồi trở lại đầu ra chuẩn. Bây giờ, để thử ứng dụng nhỏ này, ta có thể tận dụng một Unix pipe để chuyển hướng một ít dữ liệu vào đầu vào chuẩn của nó, như trong ví dụ sau:

```
echo Hello World! | node replace.js World Node.js
```

This should produce the following output:

Điều này sẽ tạo ra kết quả sau:

```
Hello Node.js!
```

This simple example demonstrates that streams (and in particular, text streams) are a universal interface and that pipes are the way to compose and interconnect all these interfaces almost magically.

Ví dụ đơn giản này minh họa rằng stream (và đặc biệt là stream văn bản) là một giao diện phổ quát và rằng pipe là cách để kết hợp và liên kết tất cả các giao diện này gần như một cách kỳ diệu.

Pipes and error handling — Pipe và xử lý lỗi

The `error` events are not propagated automatically through the pipeline when using `pipe()`. Take, for example, this code fragment:

Các sự kiện `error` không được tự động lan truyền qua pipeline khi dùng `pipe()`. Lấy ví dụ, đoạn mã sau:

```
stream1
  .pipe(stream2)
  .on('error', () => {})
```

In the preceding pipeline, we will catch only the errors coming from `stream2`, which is the stream that we attached the listener to. This means that, if we want to catch any error generated from `stream1`, we have to attach another error listener directly to it, which will make our example look like this:

Trong pipeline trên, ta sẽ chỉ bắt được các lỗi đến từ `stream2`, vốn là stream mà ta gắn listener vào. Nghĩa là, nếu ta muốn bắt mọi lỗi sinh ra từ `stream1`, ta phải gắn một `error` listener khác trực tiếp vào nó, điều này sẽ khiến ví dụ của ta trông như sau:

```
stream1
  .on('error', () => {})
  .pipe(stream2)
  .on('error', () => {})
```

This is clearly not ideal, especially when dealing with pipelines with a significant number of steps. To make this worse, in the event of an error, the failing stream is only unpiped from the pipeline.

The failing stream is not properly destroyed, which might leave dangling resources (for example, file descriptors, connections, and so on) and leak memory. A more robust (yet inelegant) implementation of the preceding snippet might look like this:

Điều này rõ ràng không lý tưởng, đặc biệt khi xử lý các pipeline có số lượng bước đáng kể. Tệ hơn nữa, trong trường hợp có lỗi, stream lỗi chỉ bị tháo (unpipe) khỏi pipeline. Stream lỗi không bị hủy (destroy) đúng cách, điều này có thể để lại các tài nguyên còn lủng lảng (ví dụ, file descriptor, kết nối, v.v.) và rò rỉ bộ nhớ. Một hiện thực mạnh mẽ hơn (dù không thanh lịch) của đoạn mã trên có thể trông như sau:

```
function handleError (err) {
  console.error(err)
  stream1.destroy()
  stream2.destroy()
}
stream1
  .on('error', handleError)
  .pipe(stream2)
  .on('error', handleError)
```

In this example, we registered a handler for the error event for both `stream1` and `stream2`. When an error happens, our `handleError()` function is invoked, and we can log the error and destroy every stream in the pipeline. This allows us to ensure that all the allocated resources are properly released, and the error is handled gracefully.

Trong ví dụ này, ta đăng ký một handler cho sự kiện `error` cho cả `stream1` và `stream2`. Khi một lỗi xảy ra, hàm `handleError()` của ta được gọi, và ta có thể ghi log lỗi và hủy mọi stream trong pipeline. Điều này cho phép ta đảm bảo rằng tất cả các tài nguyên đã cấp phát được giải phóng đúng cách, và lỗi được xử lý êm đẹp.

Better error handling with `pipeline()` — Xử lý lỗi tốt hơn với `pipeline()`

Handling errors manually in a pipeline is not just cumbersome, but also errorprone—definitely something we should avoid if we can!

Xử lý lỗi thủ công trong một pipeline không chỉ cồng kềnh, mà còn dễ sai sót—chắc chắn là điều ta nên tránh nếu có thể!

Luckily, the core `stream` package offers us an excellent utility function that can make building pipelines a much safer and more enjoyable practice, which is the `pipeline()` helper function.

May mắn thay, gói lõi `stream` cung cấp cho ta một hàm tiện ích tuyệt vời có thể khiến việc xây dựng pipeline trở thành một thực hành an toàn và thú vị hơn nhiều, đó là hàm helper `pipeline()`.

In a nutshell, you can use the `pipeline()` function as follows:

Tóm lại, bạn có thể dùng hàm `pipeline()` như sau:

```
pipeline(stream1, stream2, stream3, ... , cb)
```

This helper pipes every stream passed in the arguments list to the next one. For each stream, it will also register a proper error and `close` listeners. This way, all the streams are properly destroyed when the pipeline completes successfully or when it's interrupted by an error. The last argument is an optional callback that will be called when the stream finishes. If it finishes because of an error, the callback will be invoked with the given error as the first argument.

Helper này nối ống mỗi stream được truyền trong danh sách đối số tới stream kế tiếp. Với mỗi stream, nó cũng sẽ đăng ký các listener `error` và `close` thích hợp. Bằng cách này, tất cả các stream được hủy đúng cách khi pipeline hoàn tất thành công hoặc khi nó bị gián đoạn bởi một lỗi. Đối số cuối cùng là một callback tùy chọn sẽ được gọi khi stream kết thúc. Nếu nó kết thúc do một lỗi, callback sẽ được gọi với lỗi đã cho làm đối số đầu tiên.

In order to build up some practice with this helper, let's write a simple commandline script that implements the following pipeline:

Để rèn luyện một chút với helper này, hãy viết một script dòng lệnh đơn giản hiện thực pipeline sau:

- Reads a Gzip data stream from the standard input
 - Decompresses the data
 - Makes all the text uppercase
 - Gzips the resulting data
 - Sends the data back to the standard output
- Đọc một stream dữ liệu Gzip từ đầu vào chuẩn
 - Giải nén dữ liệu
 - Viết hoa toàn bộ văn bản
 - Gzip lại dữ liệu thu được
 - Gửi dữ liệu trở lại ra đầu ra chuẩn

Let's call this module `uppercasify-gzipped.js`:

Hãy đặt tên module này là `uppercasify-gzipped.js`:

```
import { createGzip, createGunzip } from 'zlib'           // (1)
import { Transform, pipeline } from 'stream'
const uppercasify = new Transform({                       // (2)
  transform (chunk, enc, cb) {
    this.push(chunk.toString().toUpperCase())
    cb()
  }
})
pipeline(                                                  // (3)
  process.stdin,
  createGunzip(),
  uppercasify,
  createGzip(),
  process.stdout,
  (err) => {                                              // (4)
    if (err) {
      console.error(err)
      process.exit(1)
    }
  }
)
```

In this example:

Trong ví dụ này:

1. We are importing the necessary dependencies from `zlib` and the `stream` modules.

2. We create a simple Transform stream that makes every chunk uppercase.
3. We define our pipeline, where we list all the stream instances in order.
4. We add a callback to monitor the completion of the stream. In the event of an error, we print the error in the standard error interface, and we exit with error code 1.

1. Ta đang import các phụ thuộc cần thiết từ module `zlib` và `stream`.
2. Ta tạo một Transform stream đơn giản viết hoa mọi chunk.
3. Ta định nghĩa pipeline của mình, nơi ta liệt kê tất cả các instance stream theo thứ tự.
4. Ta thêm một callback để theo dõi việc hoàn tất của stream. Trong trường hợp có lỗi, ta in lỗi ra giao diện lỗi chuẩn, và ta thoát với mã lỗi 1.

The pipeline will start automatically by consuming data from the standard input and producing data for the standard output.

Pipeline sẽ tự động bắt đầu bằng cách tiêu thụ dữ liệu từ đầu vào chuẩn và tạo dữ liệu cho đầu ra chuẩn.

We could test our script with the following command:

Ta có thể kiểm thử script của mình với lệnh sau:

```
echo 'Hello World!' | gzip | node uppercasify-gzipped.js | gunzip
```

This should produce the following output:

Điều này sẽ tạo ra kết quả sau:

HELLO WORLD!

If we try to remove the `gzip` step from the preceding sequence of commands, our script will fail with an error similar to the following:

Nếu ta thử loại bỏ bước `gzip` khỏi chuỗi lệnh trên, script của ta sẽ thất bại với một lỗi tương tự như sau:

```
Error: unexpected end of file
    at Zlib.zlibOnError [as onerror] (zlib.js:180:17) {
  errno: -5,
  code: 'Z_BUF_ERROR'
}
```

This error is raised by the stream created with the `createGunzip()` function, which is responsible for decompressing the data. If the data is not actually gzipped, the decompression algorithm won't be able to process the data and it will fail. In such a case, `pipeline()` will take care of cleaning up after the error and destroy all the streams in the pipeline.

Lỗi này được nâng lên bởi stream được tạo bằng hàm `createGunzip()`, vốn chịu trách nhiệm giải nén dữ liệu. Nếu dữ liệu không thực sự được `gzip`, thuật toán giải nén sẽ không thể xử lý dữ liệu và nó sẽ thất bại. Trong trường hợp đó, `pipeline()` sẽ lo việc dọn dẹp sau lỗi và hủy tất cả các stream trong pipeline.

The `pipeline()` function can be easily promisified by using the `promisify()` helper from the `core util` module.

Hàm `pipeline()` có thể dễ dàng được promisified bằng helper `promisify()` từ module `util`.

Now that we have built a solid understanding of Node.js streams, we are ready to move into some more involved stream patterns like **control flow** and advanced piping patterns.

Giờ đây, sau khi đã xây dựng một hiểu biết vững chắc về stream của Node.js, ta đã sẵn sàng chuyển sang một số stream pattern phức tạp hơn như luồng điều khiển (control flow) và các piping pattern nâng cao.

Asynchronous control flow patterns with streams — Các mẫu luồng điều khiển bất đồng bộ với stream

Going through the examples that we have presented so far, it should be clear that streams can be useful not only to handle I/O, but also as an elegant programming pattern that can be used to process any kind of data. But the advantages do not end at its simple appearance; streams can also be leveraged to turn “asynchronous control flow” into “flow control,” as we will see in this section.

Điểm qua các ví dụ mà ta đã trình bày đến giờ, lẽ ra đã rõ rằng stream có thể hữu ích không chỉ để xử lý I/O, mà còn như một mẫu lập trình thanh lịch có thể được dùng để xử lý mọi loại dữ liệu. Nhưng lợi ích không dừng ở vẻ ngoài đơn giản của nó; stream cũng có thể được tận dụng để biến “luồng điều khiển bất đồng bộ” thành “điều khiển luồng (flow control)”, như ta sẽ thấy trong phần này.

Sequential execution — Thực thi tuần tự

By default, streams will handle data in sequence. For example, the `_transform()` function of a Transform stream will never be invoked with the next chunk of data until the previous invocation completes by calling `callback()`. This is an important property of streams, crucial for processing each chunk in the right order, but it can also be exploited to turn streams into an elegant alternative to the traditional control flow patterns. Some code is always better than too much explanation, so let's work on an example to demonstrate how we can use streams to execute asynchronous tasks in sequence. Let's create a function that concatenates a set of files received as input, making sure to honor the order in which they are provided. Let's create a new module called `concat-files.js` and define its contents as follows:

Theo mặc định, stream sẽ xử lý dữ liệu theo trình tự. Ví dụ, hàm `_transform()` của một Transform stream sẽ không bao giờ được gọi với chunk dữ liệu kế tiếp cho đến khi lời gọi trước hoàn tất bằng cách gọi `callback()`. Đây là một đặc tính quan trọng của stream, then chốt để xử lý mỗi chunk đúng thứ tự, nhưng nó cũng có thể được khai thác để biến stream thành một lựa chọn thay thế thanh lịch cho các mẫu luồng điều khiển truyền thống. Mã thì luôn tốt hơn quá nhiều lời giải thích, nên hãy làm việc với một ví dụ để minh họa cách ta có thể dùng stream để thực thi các tác vụ bất đồng bộ theo trình tự. Hãy tạo một hàm nối (concatenate) một tập các tập tin nhận làm đầu vào, đảm bảo tôn trọng thứ tự chúng được cung cấp. Hãy tạo một module mới tên `concat-files.js` và định nghĩa nội dung của nó như sau:

```
import { createWriteStream, createReadStream } from 'fs'
import { Readable, Transform } from 'stream'
export function concatFiles (dest, files) {
  return new Promise((resolve, reject) => {
    const destStream = createWriteStream(dest)
    Readable.from(files)                                // (1)
      .pipe(new Transform({                             // (2)
        objectMode: true,
        transform (filename, enc, done) {
```

```

    const src = createReadStream(filename)
    src.pipe(destStream, { end: false })
    src.on('error', done)
    src.on('end', done) // (3)
  }
}))
.on('error', reject)
.on('finish', () => { // (4)
  destStream.end()
  resolve()
})
})
}

```

The preceding function implements a sequential iteration over the `files` array by transforming it into a stream. The algorithm can be explained as follows:

Hàm trên hiện thực một phép lặp tuần tự trên mảng `files` bằng cách biến nó thành một stream. Thuật toán có thể được giải thích như sau:

1. First, we use `Readable.from()` to create a `Readable` stream from the `files` array. This stream operates in object mode (the default setting for streams created with `Readable.from()`) and it will emit filenames: every chunk is a string indicating the path to a file. The order of the chunks respects the order of the files in the `files` array.
2. Next, we create a custom `Transform` stream to handle each file in the sequence. Since we are receiving strings, we set the option `objectMode` to `true`. In our transformation logic, for each file, we create a `Readable` stream to read the file content and pipe it into `destStream` (a `Writable` stream for the destination file). We make sure not to close `destStream` after the source file finishes reading by specifying `{ end: false }` in the `pipe()` options.
3. When all the contents of the source file have been piped into `destStream`, we invoke the `done` function to communicate the completion of the current processing, which is necessary to trigger the processing of the next file.
4. When all the files have been processed, the `finish` event is fired; we can finally end `destStream` and invoke the `cb()` function of `concatFiles()`, which signals the completion of the whole operation.

1. Trước hết, ta dùng `Readable.from()` để tạo một `Readable` stream từ mảng `files`. Stream này hoạt động ở chế độ `object` (cài đặt mặc định cho các stream tạo bằng `Readable.from()`) và nó sẽ phát ra các tên tập tin: mỗi chunk là một chuỗi cho biết đường dẫn tới một tập tin. Thứ tự các chunk tôn trọng thứ tự các tập tin trong mảng `files`.
2. Tiếp theo, ta tạo một `Transform` stream tùy chỉnh để xử lý từng tập tin theo trình tự. Vì ta đang nhận các chuỗi, ta đặt tùy chọn `objectMode` thành `true`. Trong logic biến đổi của ta, với mỗi tập tin, ta tạo một `Readable` stream để đọc nội dung tập tin và nối ống nó vào `destStream` (một `Writable` stream cho tập tin đích). Ta đảm bảo không đóng `destStream` sau khi tập tin nguồn đọc xong bằng cách chỉ định `{ end: false }` trong các tùy chọn của `pipe()`.
3. Khi tất cả nội dung của tập tin nguồn đã được nối ống vào `destStream`, ta gọi hàm `done` để báo việc hoàn tất xử lý hiện tại, điều này cần thiết để kích hoạt việc xử lý tập tin tiếp theo.
4. Khi tất cả các tập tin đã được xử lý, sự kiện `finish` được phát ra; cuối cùng ta có thể kết thúc `destStream` và gọi hàm `cb()` của `concatFiles()`, vốn báo hiệu việc hoàn

tất toàn bộ thao tác.

We can now try to use the little module we just created. Let's do that in a new file, called `concat.js`:

Bây giờ ta có thể thử dùng module nhỏ mà ta vừa tạo. Hãy làm điều đó trong một tập tin mới, tên `concat.js`:

```
import { concatFiles } from './concat-files.js'
async function main () {
  try {
    await concatFiles(process.argv[2], process.argv.slice(3))
  } catch (err) {
    console.error(err)
    process.exit(1)
  }
  console.log('All files concatenated successfully')
}
main()
```

We can now run the preceding program by passing the destination file as the first command-line argument, followed by a list of files to concatenate; for example:

Bây giờ ta có thể chạy chương trình trên bằng cách truyền tập tin đích làm đối số dòng lệnh đầu tiên, theo sau là danh sách các tập tin cần nối; ví dụ:

```
node concat.js all-together.txt file1.txt file2.txt
```

This should create a new file called `all-together.txt` containing, in order, the contents of `file1.txt` and `file2.txt`.

Điều này sẽ tạo một tập tin mới tên `all-together.txt` chứa, theo thứ tự, nội dung của `file1.txt` và `file2.txt`.

With the `concatFiles()` function, we were able to obtain an asynchronous sequential iteration using only streams. This is an elegant and compact solution that enriches our toolbelt, along with the techniques we already explored in Chapter 4, *Asynchronous Control Flow Patterns with Callbacks*, and Chapter 5, *Asynchronous Control Flow Patterns with Promises and Async/Await*. Pattern

Với hàm `concatFiles()`, ta đã có thể đạt được một phép lặp tuần tự bất đồng bộ chỉ bằng stream. Đây là một giải pháp thanh lịch và gọn gàng làm phong phú thêm bộ đồ nghề của ta, bên cạnh các kỹ thuật mà ta đã khám phá trong Chương 4, *Asynchronous Control Flow Patterns with Callbacks*, và Chương 5, *Asynchronous Control Flow Patterns with Promises and Async/Await*. Pattern

Use a stream, or combination of streams, to easily iterate over a set of asynchronous tasks in sequence.

Dùng một stream, hoặc tổ hợp các stream, để dễ dàng lặp qua một tập các tác vụ bất đồng bộ theo trình tự.

In the next section, we will discover how to use Node.js streams to implement unordered parallel task execution.

Trong phần tiếp theo, ta sẽ khám phá cách dùng stream của Node.js để hiện thực việc thực thi tác vụ song song không theo thứ tự.

Unordered parallel execution — Thực thi song song không theo thứ tự

We just saw that streams process each data chunk in sequence, but sometimes, this can be a bottleneck as we would not make the most of the **concurrency** of Node.js. If we have to execute a slow asynchronous operation for every data chunk, it can be advantageous to parallelize the execution and speed up the overall process. Of course, this pattern can only be applied if there is no relationship between each chunk of data, which might happen frequently for object streams, but very rarely for binary streams.

Ta vừa thấy rằng stream xử lý mỗi chunk dữ liệu theo trình tự, nhưng đôi khi, điều này có thể là một nút thắt cổ chai vì ta sẽ không tận dụng tối đa tính đồng thời (concurrency) của Node.js. Nếu ta phải thực thi một thao tác bất đồng bộ chậm cho mỗi chunk dữ liệu, có thể có lợi khi song song hóa việc thực thi và tăng tốc toàn bộ quá trình. Tất nhiên, pattern này chỉ có thể áp dụng nếu không có mối quan hệ nào giữa từng chunk dữ liệu, điều có thể xảy ra thường xuyên với các object stream, nhưng rất hiếm với các binary stream.

Caution

Lưu ý

Unordered parallel streams cannot be used when the order in which the data is processed is important.

Stream song song không theo thứ tự không thể được dùng khi thứ tự xử lý dữ liệu là quan trọng.

To parallelize the execution of a Transform stream, we can apply the same patterns that we learned in Chapter 4, Asynchronous Control Flow Patterns with Callbacks, but with some adaptations to get them working with streams. Let's see how this works.

Để song song hóa việc thực thi của một Transform stream, ta có thể áp dụng chính các mẫu mà ta đã học trong Chương 4, Asynchronous Control Flow Patterns with Callbacks, nhưng với một số điều chỉnh để chúng hoạt động với stream. Hãy xem cách hoạt động này.

Implementing an unordered parallel stream — Hiện thực một stream song song không theo thứ tự

Let's immediately demonstrate how to implement an unordered parallel stream with an example. Let's create a module called `parallel-stream.js` and define a generic Transform stream that executes a given transform function in parallel:

Hãy minh họa ngay cách hiện thực một stream song song không theo thứ tự qua một ví dụ. Hãy tạo một module tên `parallel-stream.js` và định nghĩa một Transform stream tổng quát thực thi một hàm transform cho trước theo cách song song:

```
import { Transform } from 'stream'
export class ParallelStream extends Transform {
  constructor (userTransform, opts) { // (1)
    super({ objectMode: true, ...opts })
    this.userTransform = userTransform
    this.running = 0
    this.terminateCb = null
  }
  _transform (chunk, enc, done) { // (2)
    this.running++
```

```

    this.userTransform(
      chunk,
      enc,
      this.push.bind(this),
      this._onComplete.bind(this)
    )
    done()
  }
  _flush (done) { // (3)
    if (this.running > 0) {
      this.terminateCb = done
    } else {
      done()
    }
  }
  _onComplete (err) { // (4)
    this.running--
    if (err) {
      return this.emit('error', err)
    }
    if (this.running === 0) {
      this.terminateCb && this.terminateCb()
    }
  }
}

```

Let's analyze this new class step by step:

Hãy phân tích lớp mới này từng bước:

1. As you can see, the constructor accepts a `userTransform()` function, which is then saved as an instance variable. We invoke the parent constructor and for convenience, we enable the object mode by default.
2. Next, it is the `_transform()` method's turn. In this method, we execute the `userTransform()` function and then increment the count of running tasks. Finally, we notify the Transform stream that the current transformation step is complete by invoking `done()`. The trick for triggering the processing of another item in parallel is exactly this. We are not waiting for the `userTransform()` function to complete before invoking `done()`; instead, we do it immediately. On the other hand, we provide a special callback to `userTransform()`, which is the `this._onComplete()` method. This allows us to get notified when the execution of `userTransform()` completes.
3. The `_flush()` method is invoked just before the stream terminates, so if there are still tasks running, we can put the release of the finish event on hold by not invoking the `done()` callback immediately. Instead, we assign it to the `this.terminateCallback` variable.
4. To understand how the stream is then properly terminated, we have to look into the `_onComplete()` method. This last method is invoked every time an asynchronous task completes. It checks whether there are any more tasks running and, if there are none, it invokes the `this.terminateCallback()` function, which will cause the stream to end, releasing the finish event that was put on hold in the `_flush()` method.

1. Như bạn thấy, constructor chấp nhận một hàm `userTransform()`, sau đó được lưu làm một biến instance. Ta gọi constructor cha và để tiện, ta bật chế độ object theo mặc

- định.
2. Tiếp theo là phương thức `_transform()`. Trong phương thức này, ta thực thi hàm `userTransform()` rồi tăng số đếm các tác vụ đang chạy. Cuối cùng, ta thông báo cho `TransformStream` rằng bước biến đổi hiện tại đã hoàn tất bằng cách gọi `done()`. Mẹo để kích hoạt việc xử lý một item khác theo cách song song chính là đây. Ta không chờ hàm `userTransform()` hoàn tất trước khi gọi `done()`; thay vào đó, ta làm điều đó ngay lập tức. Mặt khác, ta cung cấp một callback đặc biệt cho `userTransform()`, đó là phương thức `this._onComplete()`. Điều này cho phép ta được thông báo khi việc thực thi `userTransform()` hoàn tất.
 3. Phương thức `_flush()` được gọi ngay trước khi stream kết thúc, nên nếu vẫn còn tác vụ đang chạy, ta có thể tạm hoãn việc phát ra sự kiện `finish` bằng cách không gọi callback `done()` ngay lập tức. Thay vào đó, ta gán nó cho biến `this.terminateCallback`.
 4. Để hiểu cách stream sau đó được kết thúc đúng cách, ta phải xem xét phương thức `_onComplete()`. Phương thức cuối cùng này được gọi mỗi khi một tác vụ bất đồng bộ hoàn tất. Nó kiểm tra xem có còn tác vụ nào đang chạy không và, nếu không còn, nó gọi hàm `this.terminateCallback()`, vốn sẽ khiến stream kết thúc, phát ra sự kiện `finish` đã bị tạm hoãn trong phương thức `_flush()`.

The `ParallelStream` class we just built allows us to easily create a `TransformStream` that executes its tasks in parallel, but there is a caveat: it does not preserve the order of the items as they are received. In fact, asynchronous operations can complete and push data at any time, regardless of when they are started. We immediately understand that this property does not play well with binary streams where the order of data usually matters, but it can surely be useful with some types of object streams.

Lớp `ParallelStream` mà ta vừa xây dựng cho phép ta dễ dàng tạo một `TransformStream` thực thi các tác vụ của nó theo cách song song, nhưng có một lưu ý: nó không bảo toàn thứ tự các item như khi chúng được nhận. Trên thực tế, các thao tác bất đồng bộ có thể hoàn tất và đẩy dữ liệu vào bất kỳ lúc nào, bất kể chúng được bắt đầu khi nào. Ta hiểu ngay rằng đặc tính này không ăn khớp tốt với các binary stream nơi thứ tự dữ liệu thường quan trọng, nhưng nó chắc chắn có thể hữu ích với một số loại object stream.

Implementing a URL status monitoring application — Hiện thực một ứng dụng giám sát trạng thái URL

Now, let's apply our `ParallelStream` to a concrete example. Let's imagine that we want to build a simple service to monitor the status of a big list of URLs. Let's imagine all these URLs are contained in a single file and are newline separated.

Bây giờ, hãy áp dụng `ParallelStream` của ta vào một ví dụ cụ thể. Hãy tưởng tượng rằng ta muốn xây dựng một dịch vụ đơn giản để giám sát trạng thái của một danh sách lớn các URL. Hãy tưởng tượng tất cả các URL này được chứa trong một tập tin duy nhất và phân tách bằng dấu xuống dòng.

Streams can offer a very efficient and elegant solution to this problem, especially if we use our `ParallelStream` class to parallelize the checking of the URLs.

Stream có thể cung cấp một giải pháp rất hiệu quả và thanh lịch cho vấn đề này, đặc biệt nếu ta dùng lớp `ParallelStream` để song song hóa việc kiểm tra các URL.

Let's build this simple application immediately in a new module called `check-urls.js`:

Hãy xây dựng ứng dụng đơn giản này ngay trong một module mới tên `check-urls.js`:

```

import { pipeline } from 'stream'
import { createReadStream, createWriteStream } from 'fs'
import split from 'split'
import superagent from 'superagent'
import { ParallelStream } from './parallel-stream.js'
pipeline(
  createReadStream(process.argv[2]),           // (1)
  split(),                                     // (2)
  new ParallelStream(                           // (3)
    async (url, enc, push, done) => {
      if (!url) {
        return done()
      }
      try {
        await superagent.head(url, { timeout: 5 * 1000 })
        push(`${url} is up\n`)
      } catch (err) {
        push(`${url} is down\n`)
      }
      done()
    }
  ),
  createWriteStream('results.txt'),           // (4)
  (err) => {
    if (err) {
      console.error(err)
      process.exit(1)
    }
    console.log('All urls have been checked')
  }
)

```

As we can see, with streams, our code looks very elegant and straightforward: everything is contained in a single streaming pipeline. Let's see how it works:

Như ta có thể thấy, với stream, mã của ta trông rất thanh lịch và rõ ràng: mọi thứ được chứa trong một pipeline streaming duy nhất. Hãy xem cách hoạt động của nó:

1. First, we create a Readable stream from the file given as input.
2. We pipe the contents of the input file through `split` (nodejsdp.link/split), a Transform stream that ensures each line is emitted in a different chunk.
3. Then, it's time to use our `ParallelStream` to check the URL. We do this by sending a head request and waiting for a response. When the operation completes, we push the result down the stream.
4. Finally, all the results are piped into a file, `results.txt`.
 1. Trước hết, ta tạo một Readable stream từ tập tin nhận làm đầu vào.
 2. Ta nối ống nội dung của tập tin đầu vào qua `split` (nodejsdp.link/split), một Transform stream đảm bảo mỗi dòng được phát ra trong một chunk khác nhau.
 3. Sau đó, đến lúc dùng `ParallelStream` của ta để kiểm tra URL. Ta làm điều này bằng cách gửi một request head và chờ một phản hồi. Khi thao tác hoàn tất, ta đẩy kết quả xuống stream.

4. Cuối cùng, tất cả các kết quả được nối ống vào một tập tin, `results.txt`.

Now, we can run the `check-urls.js` module with a command such as this:

Bây giờ, ta có thể chạy module `check-urls.js` bằng một lệnh như sau:

```
node check-urls.js urls.txt
```

Here, the file `urls.txt` contains a list of URLs (one per line); for example:

Ở đây, tập tin `urls.txt` chứa một danh sách các URL (mỗi dòng một URL); ví dụ:

```
https://mario.fyi
https://loige.co
http://thiswillbedownforsure.com
```

When the command finishes running, we will see that a file, `results.txt`, was created. This contains the results of the operation; for example:

Khi lệnh chạy xong, ta sẽ thấy rằng một tập tin, `results.txt`, đã được tạo. Tập tin này chứa kết quả của thao tác; ví dụ:

```
http://thiswillbedownforsure.com is down
https://mario.fyi is up
https://loige.co is up
```

There is a good probability that the order in which the results are written is different from the order in which the URLs were specified in the input file. This is clear evidence that our stream executes its tasks in parallel, and it does not enforce any order between the various data chunks in the stream.

Có khả năng cao là thứ tự ghi kết quả khác với thứ tự các URL được chỉ định trong tập tin đầu vào. Đây là bằng chứng rõ ràng cho thấy stream của ta thực thi các tác vụ theo cách song song, và nó không áp đặt bất kỳ thứ tự nào giữa các chunk dữ liệu khác nhau trong stream.

For the sake of curiosity, we might want to try replacing `ParallelStream` with a normal `Transform` stream and compare the behavior and performance of the two (you might want to do this as an exercise). Using `Transform` directly is way slower, because each URL is checked in sequence, but on the other hand the order of the results in the file `results.txt` is preserved.

Vì tò mò, ta có thể muốn thử thay thế `ParallelStream` bằng một `Transform` stream thông thường và so sánh hành vi cùng hiệu năng của cả hai (bạn có thể muốn làm điều này như một bài tập). Dùng `Transform` trực tiếp chậm hơn nhiều, vì mỗi URL được kiểm tra theo trình tự, nhưng mặt khác thứ tự kết quả trong tập tin `results.txt` được bảo toàn.

In the next section, we will see how to extend this pattern to limit the number of parallel tasks running at a given time.

Trong phần tiếp theo, ta sẽ xem cách mở rộng pattern này để giới hạn số tác vụ song song chạy tại một thời điểm nhất định.

Unordered limited parallel execution — Thực thi song song có giới hạn và không theo thứ tự

If we try to run the `check-urls.js` application against a file that contains thousands or millions of URLs, we will surely run into issues. Our application will create an uncontrolled number of connections all at once, sending a considerable amount of data in parallel, and potentially undermining the stability of the application and the availability of the entire system. As we already

know, the solution to keep the load and resource usage under control is to limit the concurrency of the parallel tasks.

Nếu ta thử chạy ứng dụng `check-urls.js` với một tập tin chứa hàng nghìn hoặc hàng triệu URL, ta chắc chắn sẽ gặp rắc rối. Ứng dụng của ta sẽ tạo một số kết nối không kiểm soát được cùng một lúc, gửi một lượng dữ liệu đáng kể song song, và có khả năng làm suy yếu tính ổn định của ứng dụng cũng như khả dụng của toàn bộ hệ thống. Như ta đã biết, giải pháp để giữ tải và mức sử dụng tài nguyên trong tầm kiểm soát là giới hạn tính đồng thời của các tác vụ song song.

Let's see how this works with streams by creating a `limited-parallel-stream.js` module, which is an adaptation of `parallel-stream.js` we created in the previous section. Let's see what it looks like, starting from its constructor (we will highlight the changed parts):

Hãy xem cách hoạt động này với stream bằng cách tạo một module `limited-parallel-stream.js`, vốn là một biến thể của `parallel-stream.js` mà ta đã tạo trong phần trước. Hãy xem nó trông như thế nào, bắt đầu từ constructor của nó (ta sẽ làm nổi bật các phần đã thay đổi):

```
export class LimitedParallelStream extends Transform {
  constructor (concurrency, userTransform, opts) {
    super({ ...opts, objectMode: true })
    this.concurrency = concurrency
    this.userTransform = userTransform
    this.running = 0
    this.continueCb = null
    this.terminateCb = null
  }
  // ...
```

We need a concurrency limit to be taken as input, and this time, we are going to save two callbacks, one for any pending `_transform` method (`continueCb`—more on this next) and another one for the callback of the `_flush` method (`terminateCb`).

Ta cần một giới hạn concurrency được nhận làm đầu vào, và lần này, ta sẽ lưu hai callback, một cho bất kỳ phương thức `_transform` nào đang chờ (`continueCb`—sẽ nói thêm về cái này tiếp theo) và một cho callback của phương thức `_flush` (`terminateCb`).

Next is the `_transform()` method:

Tiếp theo là phương thức `_transform()`:

```
_transform (chunk, enc, done) {
  this.running++
  this.userTransform(
    chunk,
    enc,
    this.push.bind(this),
    this._onComplete.bind(this)
  )
  if (this.running < this.concurrency) {
    done()
  } else {
    this.continueCb = done
```

```

    }
}

```

This time, in the `_transform()` method, we have to check whether we have any free execution slots before we can invoke `done()` and trigger the processing of the next item. If we have already reached the maximum number of concurrently running streams, we can simply save the `done()` callback in the `continueCb` variable so that it can be invoked as soon as a task finishes. The `_flush()` method remains exactly the same as in the `ParallelStream` class, so let's move directly to implementing the `_onComplete()` method:

Lần này, trong phương thức `_transform()`, ta phải kiểm tra xem ta có còn slot thực thi rảnh nào không trước khi ta có thể gọi `done()` và kích hoạt việc xử lý item tiếp theo. Nếu ta đã đạt số lượng tối đa các stream chạy đồng thời, ta có thể đơn giản lưu callback `done()` vào biến `continueCb` để nó có thể được gọi ngay khi một tác vụ kết thúc. Phương thức `_flush()` vẫn giữ nguyên hệt như trong lớp `ParallelStream`, nên hãy chuyển thẳng tới việc hiện thực phương thức `_onComplete()`:

```

_onComplete (err) {
  this.running--
  if (err) {
    return this.emit('error', err)
  }
  const tmpCb = this.continueCb
  this.continueCb = null
  tmpCb && tmpCb()
  if (this.running === 0) {
    this.terminateCb && this.terminateCb()
  }
}

```

Every time a task completes, we invoke any saved `continueCb()` that will cause the stream to unblock, triggering the processing of the next item.

Mỗi khi một tác vụ hoàn tất, ta gọi bất kỳ `continueCb()` nào đã lưu, vốn sẽ khiến stream được bỏ chặn, kích hoạt việc xử lý item tiếp theo.

That's it for the `LimitedParallelStream` class. We can now use it in the `check-urls.js` module in place of `ParallelStream` and have the concurrency of our tasks limited to the value that we set.

Vậy là xong cho lớp `LimitedParallelStream`. Bây giờ ta có thể dùng nó trong module `check-urls.js` thay cho `ParallelStream` và có được tính đồng thời của các tác vụ bị giới hạn ở giá trị mà ta đặt.

Ordered parallel execution — Thực thi song song theo thứ tự

The parallel streams that we created previously may shuffle the order of the emitted data, but there are situations where this is not acceptable. Sometimes, in fact, it is necessary to have each chunk emitted in the same order in which it was received. However, not all hope is lost: we can still run the transform function in parallel; all we have to do is to sort the data emitted by each task so that it follows the same order in which the data was received.

Các stream song song mà ta tạo trước đó có thể xáo trộn thứ tự dữ liệu được phát ra, nhưng có những tình huống mà điều này không chấp nhận được. Thực ra, đôi khi cần phải có mỗi chunk được phát ra theo đúng thứ tự nó được nhận. Tuy nhiên, không phải mọi hy vọng

đều mất hết: ta vẫn có thể chạy hàm transform theo cách song song; tất cả những gì ta phải làm là sắp xếp dữ liệu được phát ra bởi mỗi tác vụ để nó tuân theo đúng thứ tự dữ liệu được nhận.

This technique involves the use of a buffer to reorder the chunks while they are emitted by each running task. For brevity, we are not going to provide an implementation of such a stream, as it's quite verbose for the **scope** of this book. What we are going to do instead is reuse one of the available packages on npm built for this specific purpose, that is, `parallel-transform` (nodejsdp.link/parallel-transform).

Kỹ thuật này liên quan tới việc dùng một buffer để sắp xếp lại các chunk trong khi chúng được phát ra bởi mỗi tác vụ đang chạy. Cho ngắn gọn, ta sẽ không cung cấp một hiện thực của một stream như vậy, vì nó khá dài dòng đối với phạm vi của cuốn sách này. Thay vào đó, những gì ta sẽ làm là tái sử dụng một trong các gói có sẵn trên npm được xây dựng cho mục đích cụ thể này, đó là `parallel-transform` (nodejsdp.link/parallel-transform).

We can quickly check the behavior of an ordered **parallel execution** by modifying our existing `check-urls` module. Let's say that we want our results to be written in the same order as the URLs in the input file, while executing our checks in parallel. We can do this using `parallel-transform`:

Ta có thể nhanh chóng kiểm tra hành vi của một thực thi song song theo thứ tự bằng cách sửa đổi module `check-urls` hiện có của ta. Giả sử ta muốn kết quả của mình được ghi theo cùng thứ tự với các URL trong tập tin đầu vào, trong khi thực thi các kiểm tra của ta theo cách song song. Ta có thể làm điều này bằng `parallel-transform`:

```
//...
import parallelTransform from 'parallel-transform'
pipeline(
  createReadStream(process.argv[2]),
  split(),
  parallelTransform(4, async function (url, done) {
    if (!url) {
      return done()
    }
    console.log(url)
    try {
      await request.head(url, { timeout: 5 * 1000 })
      this.push(`${url} is up\n`)
    } catch (err) {
      this.push(`${url} is down\n`)
    }
    done()
  }),
  createWriteStream('results.txt'),
  (err) => {
    if (err) {
      console.error(err)
      process.exit(1)
    }
    console.log('All urls have been checked')
  }
)
```


In the example here, `parallelTransform()` creates a Transform stream in object mode that executes our transformation logic with a maximum concurrency of 4. If we try to run this new version of `check-urls.js`, we will now see that the `results.txt` file lists the results in the same order as the URLs appear in the input file. It is important to see that, even though the order of the output is the same as the input, the asynchronous tasks still run in parallel and can possibly complete in any order. When using the ordered parallel execution pattern, we need to be aware of slow items blocking the pipeline or growing the memory indefinitely. In fact, if there is an item that requires a very long time to complete, depending on the implementation of the pattern, it will either cause the buffer containing the pending ordered results to grow indefinitely or the entire processing to block until the slow item completes. With the first **strategy**, we are optimizing for performance, while with the second, we get predictable memory usage. `parallel-transform` implementation opts for predictable memory utilization and maintains an internal buffer that will not grow more than the specified maximum concurrency.

Trong ví dụ ở đây, `parallelTransform()` tạo một Transform stream ở chế độ object thực thi logic biến đổi của ta với tính đồng thời tối đa là 4. Nếu ta thử chạy phiên bản mới này của `check-urls.js`, ta giờ sẽ thấy rằng tập tin `results.txt` liệt kê các kết quả theo cùng thứ tự với các URL xuất hiện trong tập tin đầu vào. Điều quan trọng cần thấy là, mặc dù thứ tự đầu ra giống đầu vào, các tác vụ bất đồng bộ vẫn chạy song song và có thể hoàn tất theo bất kỳ thứ tự nào. Khi dùng pattern thực thi song song theo thứ tự, ta cần lưu ý các item chậm chặn pipeline hoặc làm phình bộ nhớ vô hạn. Trên thực tế, nếu có một item cần thời gian rất dài để hoàn tất, tùy thuộc vào hiện thực của pattern, nó sẽ hoặc khiến buffer chứa các kết quả theo thứ tự đang chờ phình to vô hạn hoặc khiến toàn bộ việc xử lý bị chặn cho đến khi item chậm đó hoàn tất. Với chiến lược thứ nhất, ta tối ưu cho hiệu năng, trong khi với chiến lược thứ hai, ta có được mức sử dụng bộ nhớ có thể dự đoán được. Hiện thực của `parallel-transform` chọn mức sử dụng bộ nhớ có thể dự đoán được và duy trì một buffer nội bộ sẽ không phình to quá mức tính đồng thời tối đa được chỉ định.

With this, we conclude our analysis of the asynchronous control flow patterns with streams. Next, we are going to focus on some piping patterns.

Với điều này, ta kết thúc phần phân tích về các mẫu luồng điều khiển bất đồng bộ với stream. Tiếp theo, ta sẽ tập trung vào một số piping pattern.

Piping patterns — Các piping pattern

As in real-life plumbing, Node.js streams can also be piped together by following different patterns. We can, in fact, merge the flow of two different streams into one, split the flow of one stream into two or more pipes, or redirect the flow based on a condition. In this section, we are going to explore the most important plumbing patterns that can be applied to Node.js streams.

Như trong hệ thống ống nước ngoài đời, các stream của Node.js cũng có thể được nối ống với nhau theo các pattern khác nhau. Thực tế, ta có thể hợp nhất luồng của hai stream khác nhau thành một, tách luồng của một stream thành hai hay nhiều pipe, hoặc chuyển hướng luồng dựa trên một điều kiện. Trong phần này, ta sẽ khám phá các pattern ống nước quan trọng nhất có thể áp dụng cho stream của Node.js.

Combining streams — Kết hợp các stream

In this chapter, we have stressed the fact that streams provide a simple infrastructure to modularize and reuse our code, but there is one last piece missing in this puzzle: what if we want to modularize

and reuse an entire pipeline? What if we want to combine multiple streams so that they look like one from the outside? The following figure shows what this means:

Trong chương này, ta đã nhấn mạnh sự thật rằng stream cung cấp một hạ tầng đơn giản để module hóa và tái sử dụng mã của ta, nhưng còn một mảnh ghép cuối cùng còn thiếu trong câu đố này: nếu ta muốn module hóa và tái sử dụng cả một pipeline thì sao? Nếu ta muốn kết hợp nhiều stream để chúng trông như một từ bên ngoài thì sao? Hình sau cho thấy điều này nghĩa là gì:

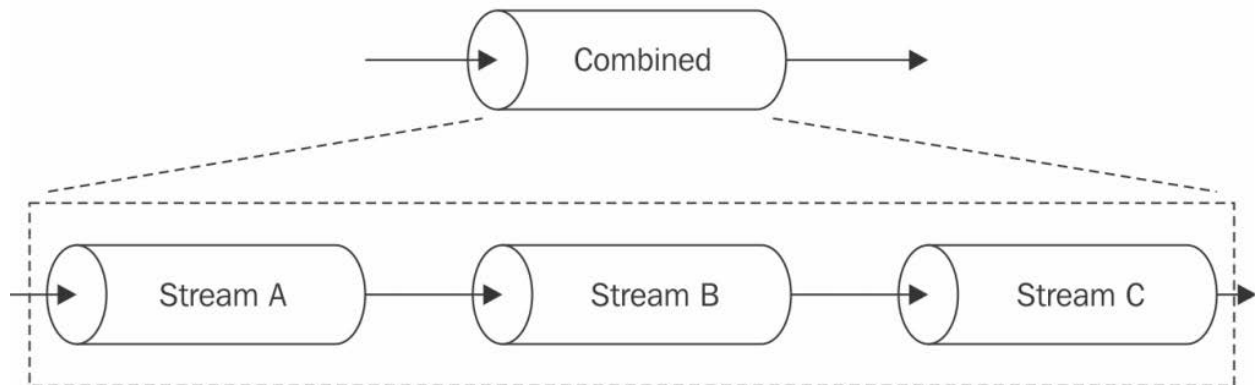


Figure 6.6: Combining streams From Figure 6.6, we should already get a hint of how this works:

Hình 6.6: Kết hợp các stream Từ Hình 6.6, ta lẽ ra đã nhận được một gợi ý về cách hoạt động của điều này:

- When we write into the combined stream, we are actually writing into the first stream of the pipeline.
- When we read from the combined stream, we are actually reading from the last stream of the pipeline.
- Khi ta ghi vào stream kết hợp, ta thực ra đang ghi vào stream đầu tiên của pipeline.
- Khi ta đọc từ stream kết hợp, ta thực ra đang đọc từ stream cuối cùng của pipeline.

A combined stream is usually a Duplex stream, which is built by connecting the first stream to its Writable side and the last stream to its Readable side.

Một stream kết hợp thường là một Duplex stream, được xây dựng bằng cách kết nối stream đầu tiên với phía Writable của nó và stream cuối cùng với phía Readable của nó.

To create a Duplex stream out of two different streams, one Writable and one Readable, we can use an npm module such as `duplexer2` (nodejsdp.link/duplexer2) or `duplexify` (nodejsdp.link/duplexify).

Để tạo một Duplex stream từ hai stream khác nhau, một Writable và một Readable, ta có thể dùng một module npm như `duplexer2` (nodejsdp.link/duplexer2) hoặc `duplexify` (nodejsdp.link/duplexify).

But that's not enough. In fact, another important characteristic of a combined stream is that it has to capture and propagate all the errors that are emitted from any stream inside the pipeline. As we already mentioned, any error event is not automatically propagated down the pipeline when we use `pipe()`, and we should explicitly attach an error listener to each stream. We saw that we could use the `pipeline()` helper function to overcome the limitations of `pipe()` with error management, but the issue with both `pipe()` and the `pipeline()` helper is that the two functions return only the last

stream of the pipeline, so we only get the (last) Readable component and not the (first) Writable component.

Nhưng như vậy chưa đủ. Trên thực tế, một đặc tính quan trọng khác của một stream kết hợp là nó phải bắt và lan truyền tất cả các lỗi được phát ra từ bất kỳ stream nào bên trong pipeline. Như ta đã đề cập, bất kỳ sự kiện error nào cũng không được tự động lan truyền xuống pipeline khi ta dùng `pipe()`, và ta nên gắn tường minh một error listener vào mỗi stream. Ta đã thấy rằng ta có thể dùng hàm helper `pipeline()` để khắc phục các hạn chế của `pipe()` về quản lý lỗi, nhưng vấn đề với cả `pipe()` lẫn `pipeline()` là cả hai hàm chỉ trả về stream cuối cùng của pipeline, nên ta chỉ nhận được thành phần (cuối) Readable chứ không phải thành phần (đầu) Writable.

We can verify this very easily with the following snippet of code:

Ta có thể kiểm chứng điều này rất dễ dàng với đoạn mã sau:

```
import { createReadStream, createWriteStream } from 'fs'
import { Transform, pipeline } from 'stream'
import { strict as assert } from 'assert'
const streamA = createReadStream('package.json')
const streamB = new Transform({
  transform (chunk, enc, done) {
    this.push(chunk.toString().toUpperCase())
    done()
  }
})
const streamC = createWriteStream('package-uppercase.json')
const pipelineReturn = pipeline(
  streamA,
  streamB,
  streamC,
  () => {
    // handle errors here
  }
)
assert.strictEqual(streamC, pipelineReturn) // valid
const pipeReturn = streamA.pipe(streamB).pipe(streamC)
assert.strictEqual(streamC, pipeReturn) // valid
```

From the preceding code, it should be clear that with just `pipe()` or `pipeline()`, it would not be trivial to construct a combined stream.

Từ đoạn mã trên, lẽ ra đã rõ rằng chỉ với `pipe()` hoặc `pipeline()`, việc xây dựng một stream kết hợp sẽ không hề đơn giản.

To recap, a combined stream has two major advantages:

Để tóm lại, một stream kết hợp có hai ưu điểm chính:

- We can redistribute it as a black box by hiding its internal pipeline.
- We have simplified error management, as we don't have to attach an error listener to each stream in the pipeline, but just to the combined stream itself.
 - Ta có thể tái phân phối nó như một hộp đen bằng cách che giấu pipeline nội bộ của nó.

- Ta đã đơn giản hóa việc quản lý lỗi, vì ta không phải gắn một error listener vào mỗi stream trong pipeline, mà chỉ vào chính stream kết hợp.

Combining streams is a pretty common practice, so if we don't have any particular need, we might just want to reuse an existing library such as `pumpify` ([nodejsdp. link/pumpify](#)).

Kết hợp các stream là một thực hành khá phổ biến, nên nếu ta không có nhu cầu đặc biệt nào, ta có thể chỉ muốn tái sử dụng một thư viện có sẵn như `pumpify` ([nodejsdp. link/pumpify](#)).

This library offers a very simple interface. In fact, all you have to do to obtain a combined stream is to call `pumpify()`, passing all the streams you want in your pipeline. This is very similar to the signature of `pipeline()`, except that there's no callback:

Thư viện này cung cấp một giao diện rất đơn giản. Thực tế, tất cả những gì bạn phải làm để có được một stream kết hợp là gọi `pumpify()`, truyền tất cả các stream mà bạn muốn trong pipeline của mình. Điều này rất giống chữ ký của `pipeline()`, ngoại trừ việc không có callback:

```
const combinedStream = pumpify(streamA, streamB, streamC)
```

When we do something like this, `pumpify` will create a pipeline out of our streams, return a new combined stream that abstracts away the complexity of our pipeline, and provide the advantages discussed previously.

Khi ta làm điều gì đó như thế này, `pumpify` sẽ tạo một pipeline từ các stream của ta, trả về một stream kết hợp mới che giấu đi sự phức tạp của pipeline, và cung cấp các ưu điểm đã bàn trước đó.

If you are curious to see what it takes to build a library like `pumpify`, you should check its source code on GitHub ([nodejsdp. link/pumpify-gh](#)). One interesting fact is that, internally, `pumpify` uses `pump` ([nodejsdp. link/pump](#)), a module that was born before the Node.js `pipeline()` helper: `pump` is effectively the module that inspired the development of `pipeline()`. If you compare their source code, you will find out that, unsurprisingly, the two modules have a lot in common.

Nếu bạn tò mò muốn xem cần những gì để xây dựng một thư viện như `pumpify`, bạn nên xem mã nguồn của nó trên GitHub ([nodejsdp. link/pumpify-gh](#)). Một sự thật thú vị là, bên trong, `pumpify` dùng `pump` ([nodejsdp. link/pump](#)), một module ra đời trước helper `pipeline()` của Node.js. `pump` thực chất là module truyền cảm hứng cho việc phát triển `pipeline()`. Nếu bạn so sánh mã nguồn của chúng, bạn sẽ nhận ra rằng, không có gì ngạc nhiên, hai module có rất nhiều điểm chung.

Implementing a combined stream — Hiện thực một stream kết hợp

To illustrate a simple example of combining streams, let's consider the case of the following two Transform streams:

Để minh họa một ví dụ đơn giản về việc kết hợp stream, hãy xét trường hợp của hai Transform stream sau:

- One that both compresses and encrypts the data
- One that both decrypts and decompresses the data
 - Một stream vừa nén vừa mã hóa dữ liệu
 - Một stream vừa giải mã vừa giải nén dữ liệu

Using a library such as pumpify, we can easily build these streams (in a file called combined-streams.js) by combining some of the streams that we already have available from the core libraries:

Dùng một thư viện như pumpify, ta có thể dễ dàng xây dựng các stream này (trong một tập tin tên combined-streams.js) bằng cách kết hợp một số stream mà ta đã có sẵn từ các thư viện lõi:

```
import { createGzip, createGunzip } from 'zlib'
import {
  createCipheriv,
  createDecipheriv,
  scryptSync
} from 'crypto'
import pumpify from 'pumpify'
function createKey (password) {
  return scryptSync(password, 'salt', 24)
}
export function createCompressAndEncrypt (password, iv) {
  const key = createKey(password)
  const combinedStream = pumpify(
    createGzip(),
    createCipheriv('aes192', key, iv)
  )
  combinedStream.iv = iv
  return combinedStream
}
export function createDecryptAndDecompress (password, iv) {
  const key = createKey(password)
  return pumpify(
    createDecipheriv('aes192', key, iv),
    createGunzip()
  )
}
```

We can now use these combined streams as if they were black boxes, for example, to create a small application that archives a file by compressing and encrypting it. Let's do that in a new module named archive.js:

Bây giờ ta có thể dùng các stream kết hợp này như thể chúng là các hộp đen, ví dụ, để tạo một ứng dụng nhỏ lưu trữ một tập tin bằng cách nén và mã hóa nó. Hãy làm điều đó trong một module mới tên archive.js:

```
import { createReadStream, createWriteStream } from 'fs'
import { pipeline } from 'stream'
import { randomBytes } from 'crypto'
import { createCompressAndEncrypt } from './combined-streams.js'
const [, , password, source] = process.argv
const iv = randomBytes(16)
const destination = `${source}.gz.enc`
pipeline(
  createReadStream(source),
  createCompressAndEncrypt(password, iv),
```

```

createWriteStream(destination),
(err) => {
  if (err) {
    console.error(err)
    process.exit(1)
  }
  console.log(`${destination} created with iv: ${iv.
toString('hex')}`)
}
)

```

Note how we don't have to worry about how many steps there are within `archiveFile`. In fact, we just treat it as a single stream within our current pipeline. This makes our combined stream easily reusable in other contexts.

Lưu ý cách ta không phải lo về việc có bao nhiêu bước bên trong `archiveFile`. Thực tế, ta chỉ coi nó như một stream duy nhất trong pipeline hiện tại của mình. Điều này khiến stream kết hợp của ta dễ dàng tái sử dụng trong các ngữ cảnh khác.

Now, to run the archive module, simply specify a password and a file in the command-line arguments:

Bây giờ, để chạy module archive, chỉ cần chỉ định một mật khẩu và một tập tin trong các đối số dòng lệnh:

```
node archive.js mypassword /path/to/a/file.txt
```

This command will create a file called `/path/to/a/file.txt.gz.enc` and it will print the generated initialization vector to the console.

Lệnh này sẽ tạo một tập tin tên `/path/to/a/file.txt.gz.enc` và nó sẽ in vector khởi tạo được sinh ra ra console.

Now, as an exercise, you could use the `createDecryptAndDecompress()` function to create a similar script that takes a password, an initialization vector, and an archived file and unarchives it. In real-life applications, it is generally preferable to include the initialization vector as part of the encrypted data, rather than requiring the user to pass it around. One way to implement this is by having the first 16 bytes emitted by the archive stream to be representing the initialization vector. The unarchive utility would need to be updated accordingly to consume the first 16 bytes before starting to process the data in a streaming fashion. This approach would add some additional complexity, which is outside the scope of this example, therefore we opted for a simpler solution. Once you feel comfortable with streams, we encourage you to try to implement as an exercise a solution where the initialization vector doesn't have to be passed around by the user.

Bây giờ, như một bài tập, bạn có thể dùng hàm `createDecryptAndDecompress()` để tạo một script tương tự nhận một mật khẩu, một vector khởi tạo, và một tập tin đã lưu trữ rồi giải lưu trữ nó. Trong các ứng dụng thực tế, nhìn chung nên đưa vector khởi tạo vào như một phần của dữ liệu đã mã hóa, thay vì yêu cầu người dùng truyền nó qua lại. Một cách để hiện thực điều này là cho 16 byte đầu tiên do stream lưu trữ phát ra đại diện cho vector khởi tạo. Tiện ích giải lưu trữ sẽ cần được cập nhật tương ứng để tiêu thụ 16 byte đầu tiên trước khi bắt đầu xử lý dữ liệu theo kiểu streaming. Cách tiếp cận này sẽ thêm một chút phức tạp, vốn nằm ngoài phạm vi của ví dụ này, do đó ta chọn một giải pháp đơn giản hơn. Một khi bạn cảm thấy thoải mái với stream, chúng tôi khuyến khích bạn thử hiện thực như một bài tập một giải pháp mà người dùng không phải truyền vector khởi tạo qua lại.

With this example, we have clearly demonstrated how important it is to combine streams. From one side, it allows us to create reusable compositions of streams, and from the other, it simplifies the error management of a pipeline.

Với ví dụ này, ta đã minh họa rõ ràng việc kết hợp stream quan trọng đến thế nào. Một mặt, nó cho phép ta tạo các tổ hợp stream có thể tái sử dụng, và mặt khác, nó đơn giản hóa việc quản lý lỗi của một pipeline.

Forking streams — Tách (fork) các stream

We can perform a fork of a stream by piping a single Readable stream into multiple Writable streams. This is useful when we want to send the same data to different destinations; for example, two different sockets or two different files. It can also be used when we want to perform different transformations on the same data, or when we want to split the data based on some criteria. If you are familiar with the Unix command `tee` ([nodejsdp.link/tee](https://nodejs.org/docs/latest-v10.x/api/stream.html#stream-link-tee)), this is exactly the same concept applied to Node.js streams!

Ta có thể thực hiện một phép fork của một stream bằng cách nối ống một Readable stream duy nhất vào nhiều Writable stream. Điều này hữu ích khi ta muốn gửi cùng một dữ liệu tới các đích đến khác nhau; ví dụ, hai socket khác nhau hoặc hai tập tin khác nhau. Nó cũng có thể được dùng khi ta muốn thực hiện các phép biến đổi khác nhau trên cùng một dữ liệu, hoặc khi ta muốn tách dữ liệu dựa trên một tiêu chí nào đó. Nếu bạn quen thuộc với lệnh Unix `tee` ([nodejsdp.link/tee](https://nodejs.org/docs/latest-v10.x/api/stream.html#stream-link-tee)), thì đây chính xác là khái niệm đó được áp dụng cho stream của Node.js!

Figure 6.7 gives us a graphical representation of this pattern:

Hình 6.7 cho ta một biểu diễn đồ họa của pattern này:

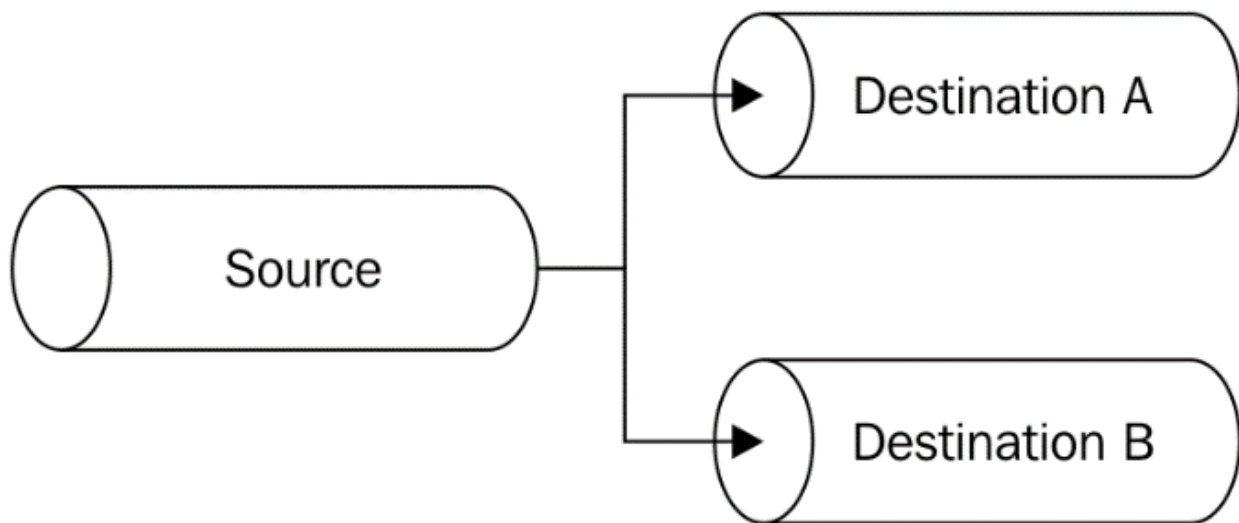


Figure 6.7: Forking a stream Forking a stream in Node.js is quite easy, but there are a few caveats to keep in mind. Let's start by discussing this pattern with an example. It will be easier to appreciate the caveats of this pattern once we have an example at hand.

Hình 6.7: Fork một stream Fork một stream trong Node.js khá dễ, nhưng có vài lưu ý cần ghi nhớ. Hãy bắt đầu bằng việc bàn về pattern này qua một ví dụ. Sẽ dễ cảm nhận các lưu ý của pattern này hơn một khi ta có một ví dụ trong tay.

Implementing a multiple checksum generator — Hiện thực một trình sinh nhiều checksum

Let's create a small utility that outputs both the sha1 and md5 hashes of a given file. Let's call this new module `generate-hashes.js`:

Hãy tạo một tiện ích nhỏ xuất ra cả hash sha1 và md5 của một tập tin cho trước. Hãy đặt tên module mới này là `generate-hashes.js`:

```
import { createReadStream, createWriteStream } from 'fs'
import { createHash } from 'crypto'
const filename = process.argv[2]
const sha1Stream = createHash('sha1').setEncoding('hex')
const md5Stream = createHash('md5').setEncoding('hex')
const inputStream = createReadStream(filename)
inputStream
  .pipe(sha1Stream)
  .pipe(createWriteStream(`${filename}.sha1`))
inputStream
  .pipe(md5Stream)
  .pipe(createWriteStream(`${filename}.md5`))
```

Very simple, right? The `inputStream` variable is piped into `sha1Stream` on one side and `md5Stream` on the other. There are a few things to note that happen behind the scenes:

Rất đơn giản, đúng không? Biến `inputStream` được nối ống vào `sha1Stream` ở một phía và `md5Stream` ở phía kia. Có vài điều cần lưu ý xảy ra ở hậu trường:

- Both `md5Stream` and `sha1Stream` will be ended automatically when `inputStream` ends, unless we specify `{ end: false }` as an option when invoking `pipe()`.
- The two forks of the stream will receive the same data chunks, so we must be very careful when performing side-effect operations on the data, as that would affect every stream that we are sending data to.
- Backpressure will work out of the box; the flow coming from `inputStream` will go as fast as the slowest branch of the fork. In other words, if one destination pauses the source stream to handle backpressure for a long time, all the other destinations will be waiting as well. Also, one destination blocking indefinitely will block the entire pipeline!
- If we pipe to an additional stream after we've started consuming the data at source (async piping), the new stream will only receive new chunks of data. In those cases, we can use a `PassThrough` instance as a placeholder to collect all the data from the moment we start consuming the stream. Then, the `PassThrough` stream can be read at any future time without the risk of losing any data. Just be aware that this approach might generate backpressure and block the entire pipeline, as discussed in the previous point.
 - Cả `md5Stream` lẫn `sha1Stream` sẽ được tự động kết thúc khi `inputStream` kết thúc, trừ khi ta chỉ định `{ end: false }` làm tùy chọn khi gọi `pipe()`.
 - Hai nhánh fork của stream sẽ nhận cùng các chunk dữ liệu, nên ta phải rất cẩn thận khi thực hiện các thao tác có tác dụng phụ (side-effect) trên dữ liệu, vì điều đó sẽ ảnh hưởng tới mọi stream mà ta đang gửi dữ liệu tới.
 - Backpressure sẽ hoạt động ngay tức thì; luồng đến từ `inputStream` sẽ đi với tốc độ bằng nhánh chậm nhất của fork. Nói cách khác, nếu một đích đến tạm dừng stream nguồn để xử lý backpressure trong một thời gian dài, tất cả các đích đến khác cũng sẽ phải chờ. Ngoài ra, một đích đến chặn vô hạn sẽ chặn toàn bộ pipeline!
 - Nếu ta nối ống tới một stream bổ sung sau khi ta đã bắt đầu tiêu thụ dữ liệu ở nguồn (nối ống bất đồng bộ), stream mới sẽ chỉ nhận các chunk dữ liệu mới. Trong những

trường hợp đó, ta có thể dùng một instance `PassThrough` làm placeholder để thu thập tất cả dữ liệu từ thời điểm ta bắt đầu tiêu thụ stream. Sau đó, `PassThrough` stream có thể được đọc vào bất kỳ thời điểm nào trong tương lai mà không có nguy cơ mất dữ liệu. Chỉ cần lưu ý rằng cách tiếp cận này có thể sinh ra backpressure và chặn toàn bộ pipeline, như đã bàn ở điểm trước.

Merging streams — Hợp nhất (merge) các stream

Merging is the opposite operation to forking and involves piping a set of Readable streams into a single Writable stream, as shown in Figure 6.8:

Hợp nhất là thao tác ngược lại với fork và liên quan tới việc nối ống một tập các Readable stream vào một Writable stream duy nhất, như trong Hình 6.8:

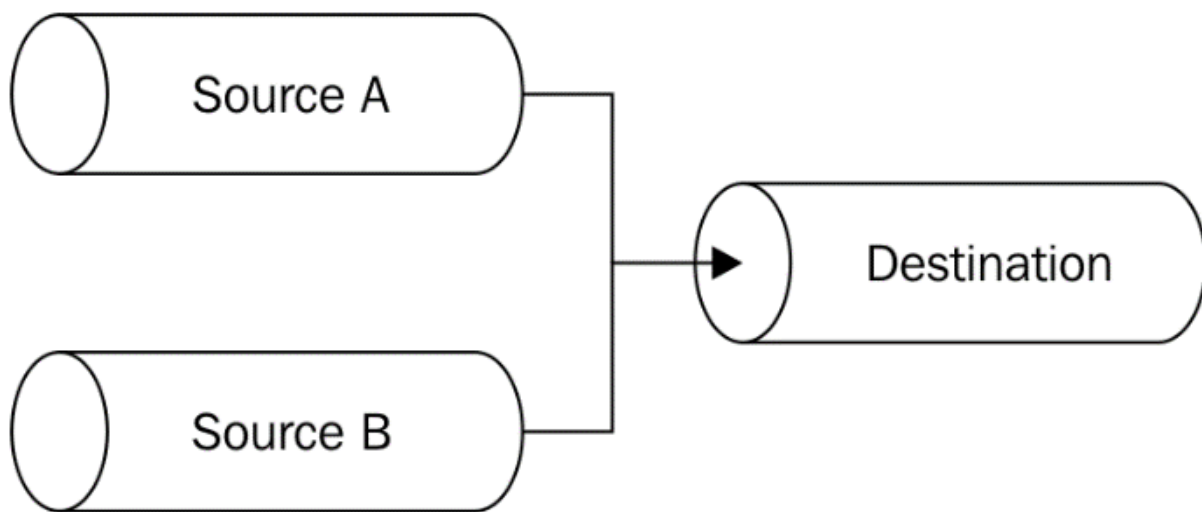


Figure 6.8: Merging streams

Hình 6.8: Hợp nhất các stream

Merging multiple streams into one is, in general, a simple operation; however, we have to pay attention to the way we handle the end event, as piping using the default options (whereby `{ end: true }`) causes the destination stream to end as soon as one of the sources ends. This can often lead to an error, as the other active sources continue to write to an already terminated stream.

Hợp nhất nhiều stream thành một nhìn chung là một thao tác đơn giản; tuy nhiên, ta phải chú ý tới cách ta xử lý sự kiện end, vì nối ống dùng các tùy chọn mặc định (theo đó `{ end: true }`) khiến stream đích kết thúc ngay khi một trong các nguồn kết thúc. Điều này thường có thể dẫn tới một lỗi, vì các nguồn còn hoạt động khác tiếp tục ghi vào một stream đã bị kết thúc.

The solution to this problem is to use the option `{ end: false }` when piping multiple sources to a single destination and then invoke `end()` on the destination only when all the sources have completed reading.

Giải pháp cho vấn đề này là dùng tùy chọn `{ end: false }` khi nối ống nhiều nguồn vào một đích duy nhất rồi gọi `end()` trên đích chỉ khi tất cả các nguồn đã đọc xong.

Merging text files — Hợp nhất các tập tin văn bản

To make a simple example, let's implement a small program that takes an output path and an arbitrary number of text files, and then merges the lines of every file into the destination file. Our new module is going to be called `merge-lines.js`. Let's define its contents, starting from some initialization steps:

Để có một ví dụ đơn giản, hãy hiện thực một chương trình nhỏ nhận một đường dẫn đầu ra và một số lượng tùy ý các tập tin văn bản, rồi hợp nhất các dòng của mỗi tập tin vào tập tin đích. Module mới của ta sẽ được gọi là `merge-lines.js`. Hãy định nghĩa nội dung của nó, bắt đầu từ một số bước khởi tạo:

```
import { createReadStream, createWriteStream } from 'fs'
import split from 'split'
const dest = process.argv[2]
const sources = process.argv.slice(3)
```

In the preceding code, we are just loading all the dependencies and initializing the variables that contain the name of the destination (`dest`) file and all the source files (`sources`).

Trong đoạn mã trên, ta chỉ đang nạp tất cả các phụ thuộc và khởi tạo các biến chứa tên của tập tin đích (`dest`) và tất cả các tập tin nguồn (`sources`).

Next, we will create the destination stream:

Tiếp theo, ta sẽ tạo stream đích:

```
const destStream = createWriteStream(dest)
```

Now, it's time to initialize the source streams:

Bây giờ, đến lúc khởi tạo các stream nguồn:

```
let endCount = 0
for (const source of sources) {
  const sourceStream = createReadStream(source, { highWaterMark: 16 })
  sourceStream.on('end', () => {
    if (++endCount === sources.length) {
      destStream.end()
      console.log(`${dest} created`)
    }
  })
  sourceStream
    .pipe(split((line) => line + '\n'))
    .pipe(destStream, { end: false })
}
```

In the preceding code, we created a Readable stream for every source file. Then, for each source stream, we attached an end listener, which will terminate the destination stream only when all the files have been read completely. Finally, we piped every source stream to `split()`, a Transform stream that makes sure that we produce a chunk for every line of text, and finally, we piped the results to our destination stream. This is when the real merge happens. We are piping multiple source streams into one single destination.

Trong đoạn mã trên, ta tạo một Readable stream cho mỗi tập tin nguồn. Sau đó, với mỗi stream nguồn, ta gắn một listener `end`, vốn sẽ kết thúc stream đích chỉ khi tất cả các tập tin đã được đọc hoàn toàn. Cuối cùng, ta nối ống mỗi stream nguồn vào `split()`, một

Transform stream đảm bảo rằng ta tạo ra một chunk cho mỗi dòng văn bản, và cuối cùng, ta nối ống các kết quả vào stream đích của ta. Đây là lúc việc hợp nhất thực sự diễn ra. Ta đang nối ống nhiều stream nguồn vào một stream đích duy nhất.

We can now execute this code with the following command:

Bây giờ ta có thể thực thi đoạn mã này với lệnh sau:

```
node merge-lines.js <destination> <source1> <source2> <source3> ...
```

If you run this code with large enough files, you will notice that the destination file will contain lines that are randomly intermingled from all the source files (a low `highWaterMark` of 16 bytes makes this property even more apparent). This kind of behavior can be acceptable in some types of object streams and some text streams split by line (as in our current example), but it is often undesirable when dealing with most binary streams. There is one variation of the pattern that allows us to merge streams in order; it consists of consuming the source streams one after the other. When the previous one ends, the next one starts emitting chunks (it is like concatenating the output of all the sources). As always, on npm, we can find some packages that also deal with this situation. One of them is `multistream` (<https://npmjs.org/package/multistream>).

Nếu bạn chạy đoạn mã này với các tập tin đủ lớn, bạn sẽ để ý rằng tập tin đích sẽ chứa các dòng được xen kẽ ngẫu nhiên từ tất cả các tập tin nguồn (một `highWaterMark` thấp 16 byte khiến đặc tính này càng rõ ràng hơn). Loại hành vi này có thể chấp nhận được trong một số loại object stream và một số stream văn bản được tách theo dòng (như trong ví dụ hiện tại của ta), nhưng nó thường không mong muốn khi xử lý hầu hết các binary stream. Có một biến thể của pattern cho phép ta hợp nhất các stream theo thứ tự; nó bao gồm việc tiêu thụ các stream nguồn lần lượt từng cái một. Khi cái trước kết thúc, cái tiếp theo bắt đầu phát ra các chunk (giống như nối đầu ra của tất cả các nguồn). Như mọi khi, trên npm, ta có thể tìm thấy một số gói cũng xử lý tình huống này. Một trong số đó là `multistream` (<https://npmjs.org/package/multistream>).

Multiplexing and demultiplexing — Ghép kênh và phân kênh (Multiplexing và demultiplexing)

There is a particular variation of the merge stream pattern in which we don't really want to just join multiple streams together but, instead, use a shared channel to deliver the data of a set of streams. This is a conceptually different operation because the source streams remain logically separated inside the shared channel, which allows us to split the stream again once the data reaches the other end of the shared channel. Figure 6.9 clarifies this situation:

Có một biến thể đặc biệt của pattern merge stream trong đó ta không thực sự muốn chỉ nối nhiều stream lại với nhau mà, thay vào đó, dùng một kênh chia sẻ để truyền dữ liệu của một tập các stream. Đây là một thao tác về mặt khái niệm khác biệt vì các stream nguồn vẫn tách biệt về mặt logic bên trong kênh chia sẻ, điều này cho phép ta tách stream trở lại một khi dữ liệu tới được đầu kia của kênh chia sẻ. Hình 6.9 làm rõ tình huống này:

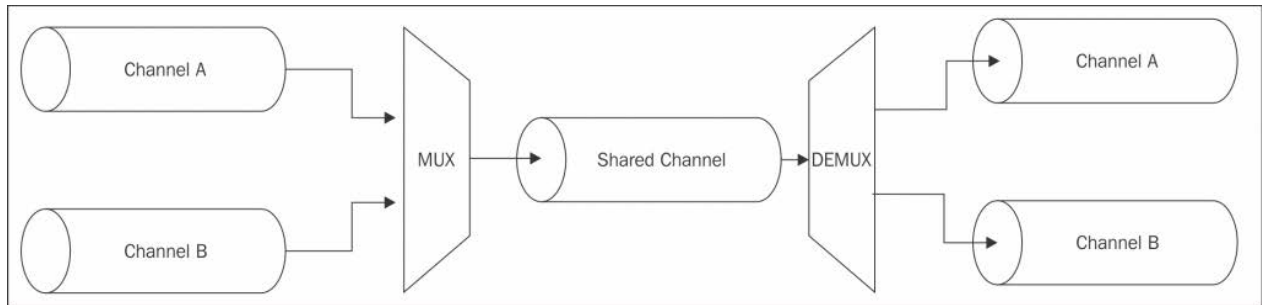


Figure 6.9: Multiplexing and demultiplexing streams

Hình 6.9: Ghép kênh và phân kênh các stream

The operation of combining multiple streams (in this case, also known as channels) to allow transmission over a single stream is called multiplexing, while the opposite operation, namely reconstructing the original streams from the data received from a shared stream, is called demultiplexing. The devices that perform these operations are called multiplexer (or mux) and demultiplexer (or demux), respectively. This is a widely studied area in computer science and telecommunications in general, as it is one of the foundations of almost any type of communication media such as telephony, radio, TV, and, of course, the Internet itself. For the scope of this book, we will not go too far with the explanations, as this is a vast topic.

Thao tác kết hợp nhiều stream (trong trường hợp này, còn được gọi là các kênh) để cho phép truyền qua một stream duy nhất được gọi là ghép kênh (multiplexing), trong khi thao tác ngược lại, cụ thể là tái dựng các stream gốc từ dữ liệu nhận được từ một stream chia sẻ, được gọi là phân kênh (demultiplexing). Các thiết bị thực hiện các thao tác này lần lượt được gọi là bộ ghép kênh (multiplexer hay mux) và bộ phân kênh (demultiplexer hay demux). Đây là một lĩnh vực được nghiên cứu rộng rãi trong khoa học máy tính và viễn thông nói chung, vì nó là một trong những nền tảng của gần như mọi loại phương tiện truyền thông như điện thoại, radio, TV, và, tất nhiên, chính Internet. Trong phạm vi của cuốn sách này, ta sẽ không đi quá xa với các lời giải thích, vì đây là một chủ đề rộng lớn.

What we want to demonstrate in this section is how it's possible to use a shared Node.js stream to transmit multiple logically separated streams that are then separated again at the other end of the shared stream.

Những gì ta muốn minh họa trong phần này là làm thế nào có thể dùng một stream Node.js chia sẻ để truyền nhiều stream tách biệt về mặt logic mà sau đó được tách trở lại ở đầu kia của stream chia sẻ.

Building a remote logger — Xây dựng một bộ ghi log từ xa

Let's use an example to drive our discussion. We want a small program that starts a **child process** and redirects both its standard output and standard error to a remote server, which, in turn, saves the two streams in two separate files. So, in this case, the shared medium is a TCP connection, while the two channels to be multiplexed are the `stdout` and `stderr` of a child process. We will leverage a technique called packet switching, the same technique that is used by protocols such as IP, TCP, and UDP. Packet switching involves wrapping the data into packets, allowing us to specify various meta information that's useful for multiplexing, routing, controlling the flow, checking for corrupted data, and so on. The protocol that we are implementing in our example is very minimalist. We wrap our data into simple packets, as illustrated in Figure 6.10:

Hãy dùng một ví dụ để dẫn dắt phần thảo luận của ta. Ta muốn một chương trình nhỏ

khởi động một tiến trình con và chuyển hướng cả đầu ra chuẩn lẫn lỗi chuẩn của nó tới một máy chủ từ xa, máy chủ này đến lượt mình lưu hai stream vào hai tập tin riêng biệt. Vậy nên, trong trường hợp này, phương tiện chia sẻ là một kết nối TCP, trong khi hai kênh cần ghép kênh là `stdout` và `stderr` của một tiến trình con. Ta sẽ tận dụng một kỹ thuật gọi là chuyển mạch gói (packet switching), cùng kỹ thuật được dùng bởi các giao thức như IP, TCP, và UDP. Chuyển mạch gói liên quan tới việc bọc dữ liệu vào các gói (packet), cho phép ta chỉ định nhiều thông tin meta hữu ích cho việc ghép kênh, định tuyến, kiểm soát luồng, kiểm tra dữ liệu hỏng, v.v. Giao thức mà ta hiện thực trong ví dụ rất tối giản. Ta bọc dữ liệu của mình vào các gói đơn giản, như minh họa trong Hình 6.10:

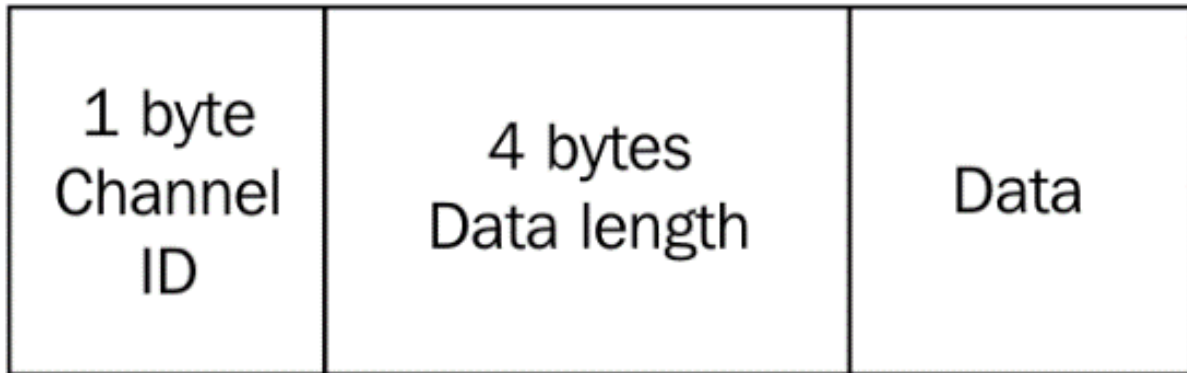


Figure 6.10: Bytes structure of the data packet for our remote logger

Hình 6.10: Cấu trúc byte của gói dữ liệu cho bộ ghi log từ xa của ta

As shown in Figure 6.10, the packet contains the actual data, but also a header (Channel ID + Data length), which will make it possible to differentiate the data of each stream and enable the demultiplexer to route the packet to the right channel.

Như trong Hình 6.10, gói chứa dữ liệu thực tế, nhưng cũng có một header (Channel ID + Data length), vốn sẽ giúp phân biệt dữ liệu của từng stream và cho phép bộ phân kênh định tuyến gói tới đúng kênh.

Client side – multiplexing — Phía client – ghép kênh Let's start to build our application from the client side. With a lot of creativity, we will call the module `client.js`. This represents the part of the application that is responsible for starting a child process and multiplexing its streams.

Hãy bắt đầu xây dựng ứng dụng của ta từ phía client. Với rất nhiều sáng tạo, ta sẽ gọi module là `client.js`. Đây đại diện cho phần của ứng dụng chịu trách nhiệm khởi động một tiến trình con và ghép kênh các stream của nó.

So, let's start by defining the module. First, we need some dependencies:

Vậy nên, hãy bắt đầu bằng việc định nghĩa module. Trước hết, ta cần một số phụ thuộc:

```
import { fork } from 'child_process'
import { connect } from 'net'
```

Now, let's implement a function that performs the multiplexing of a list of sources:

Bây giờ, hãy hiện thực một hàm thực hiện việc ghép kênh một danh sách các nguồn:

```

function multiplexChannels (sources, destination) {
  let openChannels = sources.length
  for (let i = 0; i < sources.length; i++) {
    sources[i]
      .on('readable', function () { // (1)
        let chunk
        while ((chunk = this.read()) !== null) {
          const outBuff = Buffer.alloc(1 + 4 + chunk.length) // (2)
          outBuff.writeUInt8(i, 0)
          outBuff.writeUInt32BE(chunk.length, 1)
          chunk.copy(outBuff, 5)
          console.log(`Sending packet to channel: ${i}`)
          destination.write(outBuff) // (3)
        }
      })
    .on('end', () => { // (4)
      if (--openChannels === 0) {
        destination.end()
      }
    })
  }
}

```

The `multiplexChannels()` function takes in, as input, the source streams to be multiplexed and the destination channel, and then it performs the following steps:

Hàm `multiplexChannels()` nhận vào, làm đầu vào, các stream nguồn cần ghép kênh và kênh đích, rồi nó thực hiện các bước sau:

1. For each source stream, it registers a listener for the `readable` event, where we read the data from the stream using the non-flowing mode.
2. When a chunk is read, we wrap it into a packet that contains, in order, 1 byte (`UInt8`) for the channel ID, 4 bytes (`UInt32BE`) for the packet size, and then the actual data.
3. When the packet is ready, we write it into the destination stream.
4. Finally, we register a listener for the `end` event so that we can terminate the destination stream when all the source streams have ended.

1. Với mỗi stream nguồn, nó đăng ký một listener cho sự kiện `readable`, nơi ta đọc dữ liệu từ stream bằng chế độ non-flowing.
2. Khi một chunk được đọc, ta bọc nó vào một gói chứa, theo thứ tự, 1 byte (`UInt8`) cho channel ID, 4 byte (`UInt32BE`) cho kích thước gói, rồi đến dữ liệu thực tế.
3. Khi gói đã sẵn sàng, ta ghi nó vào stream đích.
4. Cuối cùng, ta đăng ký một listener cho sự kiện `end` để ta có thể kết thúc stream đích khi tất cả các stream nguồn đã kết thúc.

Our protocol is to be able to multiplex up to 256 different source streams because we only have 1 byte to identify the channel.

Giao thức của ta có thể ghép kênh tới 256 stream nguồn khác nhau vì ta chỉ có 1 byte để định danh kênh.

Now, the last part of our client becomes very easy:

Bây giờ, phần cuối cùng của client của ta trở nên rất dễ:

```

const socket = connect(3000, () => {                                // (1)
  const child = fork(                                              // (2)
    process.argv[2],
    process.argv.slice(3),
    { silent: true }
  )
  multiplexChannels([child.stdout, child.stderr], socket) // (3)
})

```

In this last code fragment, we perform the following operations:

Trong đoạn mã cuối cùng này, ta thực hiện các thao tác sau:

1. We create a new TCP client connection to the address `localhost:3000`.
2. We start the child process by using the first command-line argument as the path, while we provide the rest of the `process.argv` array as arguments for the child process. We specify the option `{silent: true}` so that the child process does not inherit `stdout` and `stderr` of the parent.
3. Finally, we take `stdout` and `stderr` of the child process and we multiplex them into the socket's Writable stream using the `multiplexChannels()` function.

1. Ta tạo một kết nối client TCP mới tới địa chỉ `localhost:3000`.
2. Ta khởi động tiến trình con bằng cách dùng đối số dòng lệnh đầu tiên làm đường dẫn, trong khi ta cung cấp phần còn lại của mảng `process.argv` làm đối số cho tiến trình con. Ta chỉ định tùy chọn `{silent: true}` để tiến trình con không kế thừa `stdout` và `stderr` của cha.
3. Cuối cùng, ta lấy `stdout` và `stderr` của tiến trình con và ghép kênh chúng vào Writable stream của socket bằng hàm `multiplexChannels()`.

Server side – demultiplexing — Phía server – phân kênh Now, we can take care of creating the server side of the application (`server.js`), where we demultiplex the streams from the remote connection and pipe them into two different files.

Bây giờ, ta có thể lo việc tạo phía server của ứng dụng (`server.js`), nơi ta phân kênh các stream từ kết nối ở xa và nối ống chúng vào hai tập tin khác nhau.

Let's start by creating a function called `demultiplexChannel()`:

Hãy bắt đầu bằng việc tạo một hàm tên `demultiplexChannel()`:

```

import { createWriteStream } from 'fs'
import { createServer } from 'net'
function demultiplexChannel (source, destinations) {
  let currentChannel = null
  let currentLength = null
  source
    .on('readable', () => {                                       // (1)
      let chunk
      if (currentChannel === null) {                                // (2)
        chunk = source.read(1)
        currentChannel = chunk && chunk.readUInt8(0)
      }
      if (currentLength === null) {                                  // (3)
        chunk = source.read(4)

```

```

    currentLength = chunk && chunk.readUInt32BE(0)
    if (currentLength === null) {
      return null
    }
  }
  chunk = source.read(currentLength) // (4)
  if (chunk === null) {
    return null
  }
  console.log(`Received packet from: ${currentChannel}`)
  destinations[currentChannel].write(chunk) // (5)
  currentChannel = null
  currentLength = null
})
.on('end', () => { // (6)
  destinations.forEach(destination => destination.end())
  console.log('Source channel closed')
})
}

```

The preceding code might look complicated, but it is not. Thanks to the features of Node.js Readable streams, we can easily implement the demultiplexing of our little protocol as follows:

Đoạn mã trên có thể trông phức tạp, nhưng thực ra không phải. Nhờ các tính năng của Readable stream của Node.js, ta có thể dễ dàng hiện thực việc phân kênh cho giao thức nhỏ của mình như sau:

1. We start reading from the stream using the non-flowing mode.
2. First, if we have not read the channel ID yet, we try to read 1 byte from the stream and then transform it into a number.
3. The next step is to read the length of the data. We need 4 bytes for that, so it's possible (even if unlikely) that we don't have enough data in the internal buffer, which will cause the `this.read()` invocation to return `null`. In such a case, we simply interrupt the parsing and retry at the next readable event.
4. When we can finally also read the data size, we know how much data to pull from the internal buffer, so we try to read it all.
5. When we read all the data, we can write it to the right destination channel, making sure that we reset the `currentChannel` and `currentLength` variables (these will be used to parse the next packet).
6. Lastly, we make sure to end all the destination channels when the source channel ends.
 1. Ta bắt đầu đọc từ stream bằng chế độ non-flowing.
 2. Trước hết, nếu ta chưa đọc channel ID, ta thử đọc 1 byte từ stream rồi biến đổi nó thành một số.
 3. Bước tiếp theo là đọc độ dài của dữ liệu. Ta cần 4 byte cho việc đó, nên có thể (dù khó xảy ra) ta không có đủ dữ liệu trong buffer nội bộ, điều này sẽ khiến lời gọi `this.read()` trả về `null`. Trong trường hợp đó, ta đơn giản gián đoạn việc phân tích cú pháp và thử lại ở sự kiện readable tiếp theo.
 4. Khi ta cuối cùng cũng có thể đọc kích thước dữ liệu, ta biết cần kéo bao nhiêu dữ liệu từ buffer nội bộ, nên ta thử đọc tất cả.
 5. Khi ta đọc xong tất cả dữ liệu, ta có thể ghi nó vào đúng kênh đích, đảm bảo rằng ta đặt lại các biến `currentChannel` và `currentLength` (chúng sẽ được dùng để phân

tích cú pháp gói tiếp theo).

6. Cuối cùng, ta đảm bảo kết thúc tất cả các kênh đích khi kênh nguồn kết thúc.

Now that we can demultiplex the source stream, let's put our new function to work:

Giờ đây, sau khi ta có thể phân kênh stream nguồn, hãy đưa hàm mới của ta vào hoạt động:

```
const server = createServer((socket) => {
  const stdoutStream = createWriteStream('stdout.log')
  const stderrStream = createWriteStream('stderr.log')
  demultiplexChannel(socket, [stdoutStream, stderrStream])
})
server.listen(3000, () => console.log('Server started'))
```

In the preceding code, we first start a TCP server on port 3000; then, for each connection that we receive, we create two Writable streams pointing to two different files: one for the standard output and the other for the standard error. These are our destination channels. Finally, we use `demultiplexChannel()` to demultiplex the socket stream into `stdoutStream` and `stderrStream`.

Trong đoạn mã trên, trước hết ta khởi động một máy chủ TCP trên cổng 3000; rồi, với mỗi kết nối ta nhận được, ta tạo hai Writable stream trỏ tới hai tập tin khác nhau: một cho đầu ra chuẩn và một cho lỗi chuẩn. Đây là các kênh đích của ta. Cuối cùng, ta dùng `demultiplexChannel()` để phân kênh stream socket thành `stdoutStream` và `stderrStream`.

Running the mux/demux application — Chạy ứng dụng mux/demux Now, we are ready to try our new mux/demux application, but first, let's create a small Node.js program to produce some sample output; let's call it `generate-data.js`:

Bây giờ, ta đã sẵn sàng thử ứng dụng mux/demux mới của mình, nhưng trước hết, hãy tạo một chương trình Node.js nhỏ để tạo ra một ít đầu ra mẫu; hãy gọi nó là `generate-data.js`:

```
console.log('out1')
console.log('out2')
console.error('err1')
console.log('out3')
console.error('err2')
```

Okay; now, we are ready to try our remote logging application. First, let's start the server:

Được rồi; bây giờ, ta đã sẵn sàng thử ứng dụng ghi log từ xa của mình. Trước hết, hãy khởi động máy chủ:

```
node server.js
```

Then, we'll start the client by providing the file that we want to start as a child process:

Sau đó, ta sẽ khởi động client bằng cách cung cấp tập tin mà ta muốn khởi động làm tiến trình con:

```
node client.js generateData.js
```

The client will run almost immediately, but at the end of the process, the standard input and standard output of the `generate-data.js` application will have traveled through one single TCP connection and then, on the server, be demultiplexed into two separate files. Please make a note that, as we are

using `child_process.fork()` ([nodejsdp.link/fork](https://nodejs.org/docs/latest/api/child_process.html#child_process_fork)), our client will only be able to launch other Node.js modules.

Client sẽ chạy gần như ngay lập tức, nhưng ở cuối tiến trình, đầu vào chuẩn và đầu ra chuẩn của ứng dụng `generate-data.js` sẽ đã đi qua một kết nối TCP duy nhất rồi, trên máy chủ, được phân kênh thành hai tập tin riêng biệt. Xin lưu ý rằng, vì ta dùng `child_process.fork()` ([nodejsdp.link/fork](https://nodejs.org/docs/latest/api/child_process.html#child_process_fork)), client của ta sẽ chỉ có thể khởi chạy các module Node.js khác.

Multiplexing and demultiplexing object streams — Ghép kênh và phân kênh các object stream

The example that we have just shown demonstrates how to multiplex and demultiplex a binary/text stream, but it's worth mentioning that the same rules apply to object streams. The biggest difference is that when using objects, we already have a way to transmit the data using atomic messages (the objects), so multiplexing would be as easy as setting a `channelID` property in each object. Demultiplexing would simply involve reading the `channelID` property and routing each object toward the right destination stream.

Ví dụ mà ta vừa cho thấy minh họa cách ghép kênh và phân kênh một binary/text stream, nhưng cũng đáng nhắc rằng các quy tắc tương tự áp dụng cho object stream. Khác biệt lớn nhất là khi dùng object, ta đã có một cách để truyền dữ liệu bằng các thông điệp nguyên tử (atomic) (các object), nên việc ghép kênh sẽ đơn giản như đặt một thuộc tính `channelID` trong mỗi object. Việc phân kênh sẽ chỉ đơn giản liên quan tới việc đọc thuộc tính `channelID` và định tuyến mỗi object về đúng stream đích.

Another pattern involving only demultiplexing is routing the data coming from a source depending on some condition. With this pattern, we can implement complex flows, such as the one shown in Figure 6.11:

Một pattern khác chỉ liên quan tới phân kênh là định tuyến dữ liệu đến từ một nguồn tùy thuộc vào một điều kiện nào đó. Với pattern này, ta có thể hiện thực các luồng phức tạp, chẳng hạn luồng được hiển thị trong Hình 6.11:

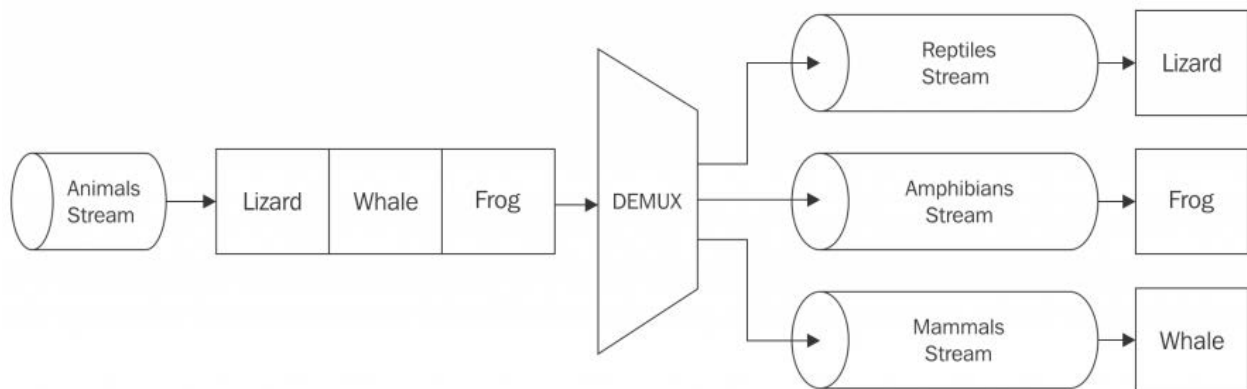


Figure 6.11: Demultiplexing an object stream

Hình 6.11: Phân kênh một object stream

The demultiplexer used in the system in Figure 6.11 takes a stream of objects representing animals and distributes each of them to the right destination stream based on the class of the animal: reptiles, amphibians, or mammals.

Bộ phân kênh được dùng trong hệ thống ở Hình 6.11 nhận một stream các object đại diện cho các loài động vật và phân phối mỗi object tới đúng stream đích dựa trên lớp của loài động vật: bò sát (reptiles), lưỡng cư (amphibians), hoặc động vật có vú (mammals).

Using the same principle, we can also implement an `if...else` statement for streams. For some inspiration, take a look at the `ternary-stream` package ([nodejsdp. link/ternary-stream](https://www.npmjs.com/package/ternary-stream)), which allows us to do exactly that.

Dùng cùng nguyên lý, ta cũng có thể hiện thực một câu lệnh `if...else` cho stream. Để lấy cảm hứng, hãy xem gói `ternary-stream` ([nodejsdp. link/ternary-stream](https://www.npmjs.com/package/ternary-stream)), vốn cho phép ta làm chính xác điều đó.

Summary — Tóm tắt

In this chapter, we have shed some light on Node.js streams and some of their most common use cases. We learned why streams are so acclaimed by the Node.js community and we mastered their basic functionality, enabling us to discover more and navigate comfortably in this new world. We analyzed some advanced patterns and started to understand how to connect streams in different configurations, grasping the importance of interoperability, which is what makes streams so versatile and powerful.

Trong chương này, ta đã làm sáng tỏ stream của Node.js và một số trường hợp sử dụng phổ biến nhất của chúng. Ta đã tìm hiểu vì sao stream được cộng đồng Node.js ca ngợi đến vậy và ta đã nắm vững chức năng cơ bản của chúng, giúp ta khám phá thêm và di chuyển thoải mái trong thế giới mới này. Ta đã phân tích một số pattern nâng cao và bắt đầu hiểu cách kết nối các stream theo các cấu hình khác nhau, nắm bắt tầm quan trọng của khả năng tương tác (interoperability), vốn là thứ làm cho stream linh hoạt và mạnh mẽ đến vậy.

If we can't do something with one stream, we can probably do it by connecting other streams together, and this works great with the one thing per module philosophy. At this point, it should be clear that streams are not just a good to know feature of Node.js; they are an essential part—a crucial pattern to handle binary data, strings, and objects. It's not by chance that we dedicated an entire chapter to them.

Nếu ta không thể làm điều gì đó với một stream, ta có lẽ có thể làm nó bằng cách kết nối các stream khác với nhau, và điều này ăn khớp tuyệt vời với triết lý mỗi module một việc. Tại thời điểm này, lẽ ra đã rõ rằng stream không chỉ là một tính năng “nên biết” của Node.js; chúng là một phần thiết yếu—một pattern then chốt để xử lý dữ liệu nhị phân, chuỗi, và object. Không phải ngẫu nhiên mà ta dành cả một chương cho chúng.

In the next few chapters, we will focus on the traditional object-oriented design patterns. But don't be fooled; even though JavaScript is, to some extent, an objectoriented language, in Node.js, the functional or hybrid approach is often preferred. Get rid of every prejudice before reading the next chapters.

Trong vài chương tới, ta sẽ tập trung vào các mẫu thiết kế hướng đối tượng truyền thống. Nhưng đừng để bị đánh lừa; mặc dù JavaScript ở một mức độ nào đó là một ngôn ngữ hướng đối tượng, trong Node.js, cách tiếp cận hàm (functional) hoặc lai (hybrid) thường được ưa chuộng hơn. Hãy gạt bỏ mọi định kiến trước khi đọc các chương tiếp theo.

Exercises — Bài tập

- 6.1 Data compression efficiency: Write a command-line script that takes a file as input and compresses it using the different algorithms available in the `zlib` module (Brotli, Deflate, Gzip). You want to produce a summary table that compares the algorithm's compression time and compression efficiency on the given file. Hint: This could be a good use case for the fork pattern, but remember that we made some important performance considerations when we discussed it earlier in this chapter.
- 6.2 Stream data processing: On Kaggle, you can find a lot of interesting data sets, such as the London Crime Data (nodejsdp.link/london-crime). You can download the data in CSV format and build a stream processing script that analyzes the data and tries to answer the following questions:
 - Did the number of crimes go up or down over the years?
 - What are the most dangerous areas of London?
 - What is the most common crime per area?
 - What is the least common crime?
- 6.1 Hiệu quả nén dữ liệu: Viết một script dòng lệnh nhận một tập tin làm đầu vào và nén nó bằng các thuật toán khác nhau có sẵn trong module `zlib` (Brotli, Deflate, Gzip). Bạn muốn tạo ra một bảng tóm tắt so sánh thời gian nén và hiệu quả nén của thuật toán trên tập tin cho trước. Gợi ý: Đây có thể là một trường hợp sử dụng tốt cho fork pattern, nhưng hãy nhớ rằng ta đã đưa ra một số cân nhắc quan trọng về hiệu năng khi ta bàn về nó trước đó trong chương này.
- 6.2 Xử lý dữ liệu kiểu stream: Trên Kaggle, bạn có thể tìm thấy rất nhiều tập dữ liệu thú vị, chẳng hạn London Crime Data (nodejsdp.link/london-crime). Bạn có thể tải dữ liệu ở định dạng CSV và xây dựng một script xử lý kiểu stream phân tích dữ liệu và cố gắng trả lời các câu hỏi sau:
 - Số vụ phạm tội tăng hay giảm qua các năm?
 - Đây là những khu vực nguy hiểm nhất của London?
 - Loại tội phạm phổ biến nhất theo từng khu vực là gì?
 - Loại tội phạm ít phổ biến nhất là gì?

Hint: You can use a combination of Transform streams and PassThrough streams to parse and observe the data as it is flowing. Then, you can build in-memory aggregations for the data, which can help you answer the preceding questions. Also, you don't need to do everything in one pipeline; you could build very specialized pipelines (for example, one per question) and use the fork pattern to distribute the parsed data across them.

Gợi ý: Bạn có thể dùng một tổ hợp các Transform stream và PassThrough stream để phân tích cú pháp và quan sát dữ liệu khi nó đang chảy. Sau đó, bạn có thể xây dựng các phép tổng hợp trong bộ nhớ cho dữ liệu, vốn có thể giúp bạn trả lời các câu hỏi trên. Ngoài ra, bạn không cần làm mọi thứ trong một pipeline; bạn có thể xây dựng các pipeline rất chuyên biệt (ví dụ, mỗi câu hỏi một pipeline) và dùng fork pattern để phân phối dữ liệu đã phân tích cú pháp tới chúng.

- 6.3 File share over TCP: Build a client and a server to transfer files over TCP. Extra points if you add a layer of encryption on top of that and if you can transfer multiple files at once. Once you have your implementation ready, give the client code and your IP address to a friend or a colleague, then ask them to send you some files! Hint: You could use mux/demux to receive multiple files at once.
- 6.4 Animations with Readable streams: Did you know you can create amazing terminal animations with just Readable streams? Well, to understand what we are talking about here,

try to run `curl parrot.live` in your terminal and see what happens! If you think that this is cool, why don't you try to create something similar? Hint: If you need some help with figuring out how to implement this, you can check out the actual source code of `parrot.live` by simply accessing its URL through your browser.

- 6.3 Chia sẻ tập tin qua TCP: Xây dựng một client và một server để truyền tập tin qua TCP. Điểm cộng nếu bạn thêm một lớp mã hóa lên trên và nếu bạn có thể truyền nhiều tập tin cùng lúc. Một khi bạn có hiện thực sẵn sàng, hãy đưa mã client và địa chỉ IP của bạn cho một người bạn hoặc đồng nghiệp, rồi nhờ họ gửi cho bạn một số tập tin! Gợi ý: Bạn có thể dùng mux/demux để nhận nhiều tập tin cùng lúc.
- 6.4 Animation với Readable stream: Bạn có biết rằng bạn có thể tạo những animation terminal tuyệt vời chỉ bằng Readable stream không? Vâng, để hiểu chúng ta đang nói về điều gì ở đây, hãy thử chạy `curl parrot.live` trong terminal của bạn và xem điều gì xảy ra! Nếu bạn thấy điều này hay, sao bạn không thử tạo một thứ gì đó tương tự? Gợi ý: Nếu bạn cần một chút trợ giúp để hình dung cách hiện thực điều này, bạn có thể xem mã nguồn thực tế của `parrot.live` bằng cách đơn giản truy cập URL của nó qua trình duyệt của bạn.

Chapter 7: Creational Design Patterns — Chương 7: Các mẫu thiết kế khởi tạo (Creational Design Patterns)

A **design pattern** is a reusable solution to a recurring problem. The term is really broad in its definition and can span multiple domains of an application. However, the term is often associated with a well-known set of object-oriented patterns that were popularized in the 90s by the book, *Design Patterns: Elements of Reusable Object-Oriented Software*, Pearson Education, by the almost legendary Gang of Four (GoF): Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. We will often refer to these specific sets of patterns as traditional design patterns or GoF design patterns.

Mẫu thiết kế (design pattern) là một giải pháp có thể tái sử dụng cho một vấn đề lặp đi lặp lại. Thuật ngữ này có định nghĩa rất rộng và có thể trải dài trên nhiều lĩnh vực của một ứng dụng. Tuy nhiên, thuật ngữ này thường gắn liền với một tập hợp nổi tiếng các mẫu hướng đối tượng đã được phổ biến vào những năm 90 bởi cuốn sách *Design Patterns: Elements of Reusable Object-Oriented Software*, Pearson Education, của bộ tứ gần như huyền thoại Gang of Four (GoF): Erich Gamma, Richard Helm, Ralph Johnson và John Vlissides. Chúng ta sẽ thường gọi tập hợp các mẫu cụ thể này là các mẫu thiết kế truyền thống hay các mẫu thiết kế GoF.

Applying this set of object-oriented design patterns in JavaScript is not as linear and formal as it would be in a classical object-oriented language. As we know, JavaScript is object-oriented, prototype-based, and has dynamic typing. It also treats functions as first-class citizens and allows functional programming styles. These characteristics make JavaScript a very versatile language, which gives tremendous power to the developer but at the same time, it causes fragmentation of programming styles, conventions, techniques, and ultimately the patterns of its ecosystem. With JavaScript, there are so many ways to achieve the same result that each developer has their own opinion on what's the best way to approach a problem. A clear demonstration of this phenomenon is the abundance of frameworks and opinionated libraries in the JavaScript ecosystem; probably no other language has ever seen so many, especially now that Node.js has given new astonishing possibilities to JavaScript and has created so many new scenarios.

Áp dụng tập hợp các mẫu thiết kế hướng đối tượng này trong JavaScript không tuyến tính và chính quy như trong một ngôn ngữ hướng đối tượng cổ điển. Như chúng ta đã biết, JavaScript là ngôn ngữ hướng đối tượng, dựa trên prototype và có kiểu động (dynamic typing). Nó cũng coi hàm là công dân hạng nhất (first-class citizen) và cho phép phong cách lập trình hàm. Những đặc điểm này khiến JavaScript trở thành một ngôn ngữ rất

linh hoạt, trao sức mạnh to lớn cho lập trình viên nhưng đồng thời cũng gây ra sự phân mảnh về phong cách lập trình, quy ước, kỹ thuật, và cuối cùng là các mẫu trong hệ sinh thái của nó. Với JavaScript, có rất nhiều cách để đạt được cùng một kết quả đến mức mỗi lập trình viên có quan điểm riêng của mình về cách tiếp cận tốt nhất cho một vấn đề. Một minh chứng rõ ràng cho hiện tượng này là sự dồi dào của các framework và các thư viện thiên kiến (opinionated) trong hệ sinh thái JavaScript; có lẽ chưa có ngôn ngữ nào khác từng chứng kiến nhiều đến vậy, đặc biệt là bây giờ khi Node.js đã mang đến những khả năng mới đáng kinh ngạc cho JavaScript và tạo ra rất nhiều kịch bản mới.

In this context, the nature of JavaScript affects traditional design patterns too. There are so many ways in which traditional design patterns can be implemented in JavaScript that the traditional, strongly object-oriented implementation stops being relevant. In some cases, the traditional implementation of these design patterns is not even possible because JavaScript, as we know, doesn't have real classes or abstract interfaces. What doesn't change, though, is the original idea at the base of each pattern, the problem it solves, and the concepts at the heart of the solution.

Trong bối cảnh này, bản chất của JavaScript cũng ảnh hưởng đến các mẫu thiết kế truyền thống. Có rất nhiều cách mà các mẫu thiết kế truyền thống có thể được triển khai trong JavaScript đến mức cách triển khai truyền thống, thuần hướng đối tượng, không còn phù hợp nữa. Trong một số trường hợp, cách triển khai truyền thống của các mẫu thiết kế này thậm chí còn không khả thi vì JavaScript, như chúng ta biết, không có lớp (class) thực sự hay giao diện trừu tượng (abstract interface). Tuy nhiên, điều không thay đổi chính là ý tưởng nguyên bản nằm ở nền tảng của mỗi mẫu, vấn đề mà nó giải quyết, và các khái niệm cốt lõi của giải pháp.

In this chapter and in the two that follow, we will see how some of the most important GoF design patterns apply to Node.js and its philosophy, thus rediscovering their importance from another perspective. Among these traditional patterns, we will also have a look at some “less traditional” design patterns born from within the JavaScript ecosystem itself.

Trong chương này và hai chương tiếp theo, chúng ta sẽ thấy một số mẫu thiết kế GoF quan trọng nhất áp dụng vào Node.js và triết lý của nó như thế nào, qua đó khám phá lại tầm quan trọng của chúng từ một góc nhìn khác. Trong số các mẫu truyền thống này, chúng ta cũng sẽ xem xét một số mẫu thiết kế “ít truyền thống hơn” được sinh ra từ chính hệ sinh thái JavaScript.

In this chapter, in particular, we'll take a look at a class of design patterns called creational. As the name suggests, these patterns address problems related to the creation of objects. For example, the **Factory** pattern allows us to encapsulate the creation of an object within a function. The Revealing Constructor pattern allows us to expose private object properties and methods only during the object's creation, while the **Builder** pattern simplifies the creation of complex objects. Finally, the **Singleton** pattern and the **Dependency Injection** pattern help us with wiring the modules within our applications.

Cụ thể trong chương này, chúng ta sẽ xem xét một lớp các mẫu thiết kế gọi là mẫu khởi tạo (creational). Như tên gọi gợi ý, những mẫu này giải quyết các vấn đề liên quan đến việc khởi tạo đối tượng. Ví dụ, mẫu Factory cho phép chúng ta đóng gói việc tạo một đối tượng bên trong một hàm. Mẫu Revealing Constructor cho phép chúng ta phơi bày các thuộc tính và phương thức riêng tư của đối tượng chỉ trong quá trình khởi tạo đối tượng, trong khi mẫu Builder đơn giản hóa việc tạo các đối tượng phức tạp. Cuối cùng, mẫu Singleton và mẫu Dependency Injection giúp chúng ta liên kết các module trong ứng dụng.

This chapter, as well as the following two, assume that you have some notion of how inheritance works in JavaScript. Please also be advised that we will often use generic and more intuitive

diagrams to describe a pattern in place of standard UML. This is because many patterns can have an implementation based not only on classes but also on objects and even functions.

Chương này, cũng như hai chương tiếp theo, giả định rằng bạn đã có một số khái niệm về cách kế thừa hoạt động trong JavaScript. Cũng xin lưu ý rằng chúng ta sẽ thường dùng các sơ đồ tổng quát và trực quan hơn để mô tả một mẫu thay cho UML chuẩn. Điều này là vì nhiều mẫu có thể có cách triển khai dựa trên không chỉ các lớp mà còn dựa trên các đối tượng và thậm chí các hàm.

Factory — Factory

We'll begin our journey from one of the most common design patterns in Node.js: Factory. As you will see, the Factory pattern is very versatile and has more than just one purpose. Its main advantage is its ability to decouple the creation of an object from one particular implementation. This allows us, for example, to create an object whose class is determined at **runtime**. Factory also allows us to expose “a surface area” that is much smaller than that of a class; a class can be extended or manipulated, while a factory, being just a function, offers fewer options to the user, making it more robust and easier to understand. Finally, a factory can also be used to enforce **encapsulation** by leveraging closures.

Chúng ta sẽ bắt đầu hành trình từ một trong những mẫu thiết kế phổ biến nhất trong Node.js: Factory. Như bạn sẽ thấy, mẫu Factory rất linh hoạt và có nhiều hơn một mục đích. Lợi thế chính của nó là khả năng tách rời việc tạo một đối tượng khỏi một cách triển khai cụ thể. Điều này cho phép chúng ta, ví dụ, tạo một đối tượng mà lớp của nó được xác định tại thời điểm chạy (runtime). Factory cũng cho phép chúng ta phơi bày “một bề mặt” nhỏ hơn nhiều so với của một lớp; một lớp có thể được mở rộng hoặc thao tác, trong khi một factory, vốn chỉ là một hàm, cung cấp ít lựa chọn hơn cho người dùng, khiến nó vững chãi hơn và dễ hiểu hơn. Cuối cùng, một factory cũng có thể được dùng để thực thi đóng gói (encapsulation) bằng cách tận dụng closure.

Decoupling object creation and implementation — Tách rời việc tạo đối tượng và cách triển khai

We already stressed the fact that, in JavaScript, the functional paradigm is often preferred to a purely object-oriented design for its simplicity, usability, and small surface area. This is especially true when creating new object instances. In fact, invoking a factory, instead of directly creating a new object from a class using the `new` operator or `Object.create()`, is so much more convenient and flexible in several respects.

Chúng ta đã nhấn mạnh thực tế rằng, trong JavaScript, mô hình hàm thường được ưa chuộng hơn thiết kế thuần hướng đối tượng vì sự đơn giản, khả năng dùng được và bề mặt nhỏ của nó. Điều này đặc biệt đúng khi tạo các thể hiện (instance) đối tượng mới. Thực vậy, gọi một factory, thay vì trực tiếp tạo một đối tượng mới từ một lớp bằng toán tử `new` hay `Object.create()`, thuận tiện và linh hoạt hơn rất nhiều ở nhiều khía cạnh.

First and foremost, a factory allows us to separate the creation of an object from its implementation. Essentially, a factory wraps the creation of a new instance, giving us more flexibility and control in the way we do it. Inside the factory, we can choose to create a new instance of a class using the `new` operator, or leverage closures to dynamically build a stateful object literal, or even return a different object type based on a particular condition. The consumer of the factory is totally agnostic about how the creation of the instance is carried out. The truth is that, by using `new`, we are binding our code

to one specific way of creating an object, while with a factory, we can have much more flexibility, almost for free. As a quick example, let's consider a simple factory that creates an Image object:

Trước hết và quan trọng nhất, một factory cho phép chúng ta tách việc tạo một đối tượng khỏi cách triển khai của nó. Về cơ bản, một factory bao bọc việc tạo một thể hiện mới, cho chúng ta nhiều linh hoạt và kiểm soát hơn trong cách chúng ta làm điều đó. Bên trong factory, chúng ta có thể chọn tạo một thể hiện mới của một lớp bằng toán tử `new`, hoặc tận dụng closure để xây dựng động một object literal có trạng thái, hoặc thậm chí trả về một kiểu đối tượng khác dựa trên một điều kiện cụ thể. Người dùng factory hoàn toàn không cần biết việc tạo thể hiện được thực hiện như thế nào. Sự thật là, bằng cách dùng `new`, chúng ta đang trói buộc mã của mình vào một cách tạo đối tượng cụ thể, trong khi với một factory, chúng ta có thể có nhiều linh hoạt hơn nhiều, gần như miễn phí. Lấy một ví dụ nhanh, hãy xem xét một factory đơn giản tạo ra một đối tượng Image:

```
function createImage (name) {  
  return new Image(name)  
}  
const image = createImage('photo.jpeg')
```

The `createImage()` factory might look totally unnecessary; why not instantiate the Image class by using the `new` operator directly? Why not write something like the following:

Factory `createImage()` trông có vẻ hoàn toàn không cần thiết; tại sao không khởi tạo lớp Image bằng cách dùng toán tử `new` trực tiếp? Tại sao không viết một cái gì đó như sau:

```
const image = new Image(name)
```

As we already mentioned, using `new` binds our code to one particular type of object, which in the preceding case is to the Image object type. A factory, on the other hand, gives us much more flexibility. Imagine that we want to **refactor** the Image class, splitting it into smaller classes, one for each image format that we support. If we exposed a factory as the only means to create new images, we could simply rewrite it as follows, without breaking any of the existing code:

Như chúng ta đã đề cập, dùng `new` trói buộc mã của chúng ta vào một kiểu đối tượng cụ thể, mà trong trường hợp trên là kiểu đối tượng Image. Một factory, mặt khác, cho chúng ta nhiều linh hoạt hơn nhiều. Hãy tưởng tượng chúng ta muốn tái cấu trúc lớp Image, tách nó thành các lớp nhỏ hơn, mỗi lớp cho một định dạng ảnh mà chúng ta hỗ trợ. Nếu chúng ta phơi bày một factory như phương tiện duy nhất để tạo ảnh mới, chúng ta có thể đơn giản viết lại nó như sau, mà không phá vỡ bất kỳ mã hiện có nào:

```
function createImage (name) {  
  if (name.match(/\.(jpe?g$/))) {  
    return new ImageJpeg(name)  
  } else if (name.match(/\.(gif$/))) {  
    return new ImageGif(name)  
  } else if (name.match(/\.(png$/))) {  
    return new ImagePng(name)  
  } else {  
    throw new Error('Unsupported format')  
  }  
}
```

Our factory also allows us to keep the classes hidden and prevents them from being extended or modified (remember the principle of small surface area?). In JavaScript, this can be achieved by exporting only the factory, while keeping the classes private.

Factory của chúng ta cũng cho phép giữ kín các lớp và ngăn chúng bị mở rộng hoặc sửa đổi (nhớ nguyên tắc bề mặt nhỏ chứ?). Trong JavaScript, điều này có thể đạt được bằng cách chỉ xuất (export) factory, trong khi giữ các lớp ở dạng riêng tư.

A mechanism to enforce encapsulation — Một cơ chế để thực thi đóng gói

A factory can also be used as an encapsulation mechanism, thanks to closures.

Một factory cũng có thể được dùng như một cơ chế đóng gói, nhờ vào closure.

Encapsulation refers to controlling the access to some internal details of a component by preventing external code from manipulating them directly. The interaction with the component happens only through its public interface, isolating the external code from the changes in the implementation details of the component. Encapsulation is a fundamental principle of object-oriented design, together with inheritance, polymorphism, and abstraction.

Đóng gói (encapsulation) đề cập đến việc kiểm soát truy cập tới một số chi tiết nội bộ của một thành phần bằng cách ngăn mã bên ngoài thao tác trực tiếp lên chúng. Sự tương tác với thành phần chỉ xảy ra thông qua giao diện công khai của nó, cô lập mã bên ngoài khỏi các thay đổi trong chi tiết triển khai của thành phần. Đóng gói là một nguyên tắc cơ bản của thiết kế hướng đối tượng, cùng với kế thừa, đa hình (polymorphism) và trừu tượng hóa (abstraction).

In JavaScript, one of the main ways to enforce encapsulation is through function scopes and closures. A factory makes it straightforward to enforce private variables. Consider the following, for example:

Trong JavaScript, một trong những cách chính để thực thi đóng gói là thông qua phạm vi hàm (function scope) và closure. Một factory giúp việc thực thi các biến riêng tư trở nên dễ dàng. Hãy xem xét ví dụ sau:

```
function createPerson (name) {  
  const privateProperties = {}  
  const person = {  
    setName (name) {  
      if (!name) {  
        throw new Error('A person must have a name')  
      }  
      privateProperties.name = name  
    },  
    getName () {  
      return privateProperties.name  
    }  
  }  
  person.setName(name)  
  return person  
}
```

In the preceding code, we leverage a **closure** to create two objects: a person object, which represents the public interface returned by the factory, and a group of `privateProperties` that are inaccessible from the outside and that can be manipulated only through the interface provided by the person object. For example, in the preceding code, we make sure that a person's name is never empty; this would not be possible to enforce if name was just a normal property of the person object.

Trong mã trên, chúng ta tận dụng một closure để tạo hai đối tượng: một đối tượng `person`, đại diện cho giao diện công khai do `factory` trả về, và một nhóm `privateProperties` không thể truy cập được từ bên ngoài và chỉ có thể được thao tác thông qua giao diện do đối tượng `person` cung cấp. Ví dụ, trong mã trên, chúng ta đảm bảo rằng `name` của một người không bao giờ rỗng; điều này sẽ không thể thực thi được nếu `name` chỉ là một thuộc tính thông thường của đối tượng `person`.

Using closures is not the only technique that we have for enforcing encapsulation. In fact, other possible approaches are:

Dùng closure không phải là kỹ thuật duy nhất mà chúng ta có để thực thi đóng gói. Thực tế, các cách tiếp cận khả thi khác là:

- Using private class fields (the hashbang `#` prefix syntax), introduced in Node.js 12. More on this at nodejsdp.link/tc39-private-fields. This is the most modern approach, but at the time of writing, the feature is still experimental and has yet to be included in the official ECMAScript specification.
- Using WeakMaps. More on this at nodejsdp.link/weakmaps-private.
- Using symbols, as explained in the following article: nodejsdp.link/symbol-private.
- Defining private variables in a constructor (as recommended by Douglas Crockford: nodejsdp.link/crockford-private). This is the legacy but also the bestknown approach.
- Using conventions, for example, prefixing the name of a property with an underscore “_”. However, this does not technically prevent a member from being read or modified from the outside.
 - Dùng các trường lớp riêng tư (cú pháp tiền tố hashbang `#`), được giới thiệu trong Node.js 12. Tìm hiểu thêm tại nodejsdp.link/tc39-private-fields. Đây là cách tiếp cận hiện đại nhất, nhưng tại thời điểm viết, tính năng này vẫn còn ở dạng thử nghiệm và chưa được đưa vào đặc tả ECMAScript chính thức.
 - Dùng WeakMap. Tìm hiểu thêm tại nodejsdp.link/weakmaps-private.
 - Dùng symbol, như được giải thích trong bài viết sau: nodejsdp.link/symbol-private.
 - Định nghĩa các biến riêng tư trong một constructor (như Douglas Crockford khuyến nghị: nodejsdp.link/crockford-private). Đây là cách tiếp cận cũ nhưng cũng là cách được biết đến nhiều nhất.
 - Dùng quy ước, ví dụ, thêm tiền tố gạch dưới “_” vào tên một thuộc tính. Tuy nhiên, về mặt kỹ thuật điều này không ngăn được một thành viên bị đọc hoặc sửa đổi từ bên ngoài.

Building a simple code profiler — Xây dựng một code profiler đơn giản

Now, let’s work on a complete example using a factory. Let’s build a simple code profiler, an object with the following properties:

Bây giờ, hãy làm việc trên một ví dụ hoàn chỉnh sử dụng một `factory`. Hãy xây dựng một code profiler đơn giản, một đối tượng với các thuộc tính sau:

- A `start()` method that triggers the start of a profiling session
- An `end()` method to terminate the session and log its execution time to the console
 - Một phương thức `start()` kích hoạt việc bắt đầu một phiên profiling
 - Một phương thức `end()` để kết thúc phiên và ghi thời gian thực thi của nó ra console

Let’s start by creating a file named `profiler.js`, which will have the following content:

Hãy bắt đầu bằng cách tạo một tập tin tên là `profiler.js`, sẽ có nội dung sau:

```
class Profiler {
  constructor (label) {
    this.label = label
    this.lastTime = null
  }
  start () {
    this.lastTime = process.hrtime()
  }
  end () {
    const diff = process.hrtime(this.lastTime)
    console.log(`Timer "${this.label}" took ${diff[0]} seconds ` +
      `and ${diff[1]} nanoseconds.`)
  }
}
```

The Profiler class we just defined uses the default high resolution timer of Node.js to save the current time when `start()` is invoked, and then calculate the elapsed time when `end()` is executed, printing the result to the console.

Lớp Profiler mà chúng ta vừa định nghĩa dùng bộ định thời độ phân giải cao mặc định của Node.js để lưu thời điểm hiện tại khi `start()` được gọi, rồi tính thời gian đã trôi qua khi `end()` được thực thi, in kết quả ra console.

Now, if we are going to use such a profiler in a real-world application to calculate the execution time of different routines, we can easily imagine the huge amount of profiling information printed to the console, especially in a production environment. What we may want to do instead is redirect the profiling information to another source, for example, a dedicated log file, or alternatively, disable the profiler altogether if the application is running in production mode. It's clear that if we were to instantiate a Profiler object directly by using the new operator, we would need some extra logic in the client code or in the Profiler object itself in order to switch between the different logics. Alternatively, we can use a factory to abstract the creation of the Profiler object so that, depending on whether the application runs in production or development mode, we can return a fully working Profiler instance or a mock object with the same interface but with empty methods. This is exactly what we are going to do in our `profiler.js` **module**. Instead of exporting the Profiler class, we will export only our factory. The following is its code:

Bây giờ, nếu chúng ta định dùng một profiler như vậy trong một ứng dụng thực tế để tính thời gian thực thi của các thủ tục khác nhau, chúng ta có thể dễ dàng hình dung lượng thông tin profiling khổng lồ được in ra console, đặc biệt trong môi trường production. Điều chúng ta có thể muốn làm thay vào đó là chuyển hướng thông tin profiling tới một nguồn khác, ví dụ một tập tin log chuyên dụng, hoặc cách khác là vô hiệu hóa profiler hoàn toàn nếu ứng dụng đang chạy ở chế độ production. Rõ ràng nếu chúng ta khởi tạo một đối tượng Profiler trực tiếp bằng toán tử `new`, chúng ta sẽ cần thêm một số logic trong mã client hoặc trong chính đối tượng Profiler để chuyển đổi giữa các logic khác nhau. Cách khác, chúng ta có thể dùng một factory để trừu tượng hóa việc tạo đối tượng Profiler sao cho, tùy thuộc vào việc ứng dụng chạy ở chế độ production hay development, chúng ta có thể trả về một thể hiện Profiler hoạt động đầy đủ hoặc một đối tượng giả (mock) với cùng giao diện nhưng các phương thức rỗng. Đây chính xác là điều chúng ta sẽ làm trong module `profiler.js` của mình. Thay vì xuất lớp Profiler, chúng ta sẽ chỉ xuất factory của mình. Sau đây là mã của nó:

```

const noopProfiler = {
  start () {},
  end () {}
}
export function createProfiler (label) {
  if (process.env.NODE_ENV === 'production') {
    return noopProfiler
  }
  return new Profiler(label)
}

```

The `createProfiler()` function is our factory and its role is abstracting the creation of a `Profiler` object from its implementation. If the application is running in production mode, we return `noopProfiler`, which essentially doesn't do anything, effectively disabling any profiling. If the application is not running in production mode, then we create and return a new, fully functional `Profiler` instance.

Hàm `createProfiler()` là factory của chúng ta và vai trò của nó là trừu tượng hóa việc tạo một đối tượng `Profiler` khỏi cách triển khai của nó. Nếu ứng dụng đang chạy ở chế độ production, chúng ta trả về `noopProfiler`, về cơ bản không làm gì cả, vô hiệu hóa hiệu quả mọi việc profiling. Nếu ứng dụng không chạy ở chế độ production, thì chúng ta tạo và trả về một thể hiện `Profiler` mới, hoạt động đầy đủ.

Thanks to JavaScript's dynamic typing, we were able to return an object instantiated with the new operator in one circumstance and a simple object literal in the other (this is also known as duck typing, and you can read more about it at nodejsdp.link/duck-typing). This confirms how we can create objects in any way we like inside the factory function. We could also execute additional initialization steps or return a different type of object based on particular conditions, all of this while isolating the consumer of the object from all these details. We can easily understand the power of this simple pattern.

Nhờ kiểu động của JavaScript, chúng ta có thể trả về một đối tượng được khởi tạo bằng toán tử `new` trong một trường hợp và một object literal đơn giản trong trường hợp khác (điều này còn được gọi là duck typing, và bạn có thể đọc thêm tại nodejsdp.link/duck-typing). Điều này khẳng định cách chúng ta có thể tạo các đối tượng theo bất kỳ cách nào chúng ta thích bên trong hàm factory. Chúng ta cũng có thể thực thi thêm các bước khởi tạo bổ sung hoặc trả về một kiểu đối tượng khác dựa trên các điều kiện cụ thể, tất cả những điều này trong khi cô lập người dùng đối tượng khỏi mọi chi tiết đó. Chúng ta có thể dễ dàng hiểu được sức mạnh của mẫu đơn giản này.

Now, let's play with our profiler factory a bit. Let's create an algorithm to calculate all the factors of a given number and use our profiler to record its running time:

Bây giờ, hãy thử nghịch một chút với factory profiler của chúng ta. Hãy tạo một thuật toán để tính tất cả các thừa số của một số cho trước và dùng profiler của chúng ta để ghi lại thời gian chạy của nó:

```

// index.js
import { createProfiler } from './profiler.js'
function getAllFactors (intNumber) {
  const profiler = createProfiler(
    `Finding all factors of ${intNumber}`)
  profiler.start()
  const factors = []

```

```

    for (let factor = 2; factor <= intNumber; factor++) {
      while ((intNumber % factor) === 0) {
        factors.push(factor)
        intNumber = intNumber / factor
      }
    }
    profiler.end()
    return factors
  }
}
const myNumber = process.argv[2]
const myFactors = getAllFactors(myNumber)
console.log(`Factors of ${myNumber} are: `, myFactors)

```

The profiler variable contains our Profiler object, whose implementation will be decided by the createProfiler() factory at runtime, based on the NODE_ENV environment variable.

Biến profiler chứa đối tượng Profiler của chúng ta, cách triển khai của nó sẽ được factory createProfiler() quyết định tại thời điểm chạy, dựa trên biến môi trường NODE_ENV.

For example, if we run the module in production mode, we will get no profiling information:

Ví dụ, nếu chúng ta chạy module ở chế độ production, chúng ta sẽ không nhận được thông tin profiling nào:

```

NODE_ENV=production node index.js 2201307499

```

While if we run the module in development mode, we will see the profiling information printed to the console:

Trong khi nếu chúng ta chạy module ở chế độ development, chúng ta sẽ thấy thông tin profiling được in ra console:

```

node index.js 2201307499

```

The example that we just presented is just a simple application of the factory function pattern, but it clearly shows the advantages of separating an object's creation from its implementation.

Ví dụ chúng ta vừa trình bày chỉ là một ứng dụng đơn giản của mẫu hàm factory, nhưng nó cho thấy rõ ràng những lợi thế của việc tách việc tạo một đối tượng khỏi cách triển khai của nó.

In the wild — Trong thực tế

As we said, factories are very common in Node.js. We can find one example in the popular Knex (nodejsdp.link/knex) **package**. Knex is a SQL query builder that supports multiple databases. Its package exports just a function, which is a factory. The factory performs various checks, selects the right dialect object to use based on the database engine, and finally creates and returns the Knex object. Take a look at the code at nodejsdp.link/knex-factory.

Như chúng ta đã nói, factory rất phổ biến trong Node.js. Chúng ta có thể tìm thấy một ví dụ trong gói Knex phổ biến (nodejsdp.link/knex). Knex là một SQL query builder hỗ trợ nhiều cơ sở dữ liệu. Gói của nó chỉ xuất một hàm, vốn là một factory. Factory này thực hiện nhiều kiểm tra khác nhau, chọn đối tượng dialect đúng để dùng dựa trên engine cơ sở dữ liệu, và cuối cùng tạo và trả về đối tượng Knex. Hãy xem mã tại nodejsdp.link/knex-factory.

Builder — Builder

Builder is a creational design pattern that simplifies the creation of complex objects by providing a fluent interface, which allows us to build the object step by step. This greatly improves the readability and the general developer experience when creating such complex objects.

Builder là một mẫu thiết kế khởi tạo giúp đơn giản hóa việc tạo các đối tượng phức tạp bằng cách cung cấp một giao diện trôi chảy (fluent interface), cho phép chúng ta xây dựng đối tượng từng bước. Điều này cải thiện đáng kể tính dễ đọc và trải nghiệm chung của lập trình viên khi tạo những đối tượng phức tạp như vậy.

The most apparent situation in which we could benefit from the Builder pattern is a class with a constructor that has a long list of arguments, or takes many complex parameters as input. Usually, these kinds of classes require so many parameters in advance because all of them are necessary to build an instance that is complete and in a consistent **state**, so it's necessary to take this into account when considering potential solutions.

Tình huống rõ ràng nhất mà chúng ta có thể hưởng lợi từ mẫu Builder là một lớp với một constructor có một danh sách dài các đối số, hoặc nhận nhiều tham số phức tạp làm đầu vào. Thông thường, những loại lớp này yêu cầu rất nhiều tham số ngay từ đầu vì tất cả chúng đều cần thiết để xây dựng một thể hiện hoàn chỉnh và ở trạng thái nhất quán, nên cần tính đến điều này khi xem xét các giải pháp tiềm năng.

So, let's see the general structure of the pattern. Imagine having a Boat class with a constructor such as the following:

Vậy, hãy xem cấu trúc tổng quát của mẫu này. Hãy tưởng tượng có một lớp Boat với một constructor như sau:

```
class Boat {
  constructor (hasMotor, motorCount, motorBrand, motorModel,
              hasSails, sailsCount, sailsMaterial, sailsColor,
              hullColor, hasCabin) {
    // ...
  }
}
```

Invoking such a constructor would create some hard to read code, which is easily prone to errors (which argument is what?). Take the following code, for example:

Gọi một constructor như vậy sẽ tạo ra mã khó đọc, dễ phát sinh lỗi (đối số nào là gì?). Lấy ví dụ mã sau:

```
const myBoat = new Boat(true, 2, 'Best Motor Co. ', 'OM123', true, 1,
                        'fabric', 'white', 'blue', false)
```

A first step to improve the design of this constructor is to aggregate all arguments in a single object literal, such as the following:

Bước đầu tiên để cải thiện thiết kế của constructor này là tổng hợp tất cả các đối số vào một object literal duy nhất, chẳng hạn như sau:

```
class Boat {
  constructor (allParameters) {
    // ...
  }
}
```

```

}
const myBoat = new Boat({
  hasMotor: true,
  motorCount: 2,
  motorBrand: 'Best Motor Co. ',
  motorModel: 'OM123',
  hasSails: true,
  sailsCount: 1,
  sailsMaterial: 'fabric',
  sailsColor: 'white',
  hullColor: 'blue',
  hasCabin: false
})

```

As we can note from the previous code, this new constructor is indeed much better than the original one as it allows us to clearly see what is the parameter that receives each value. However, we can do even better than this. One drawback of using a single object literal to pass all inputs at once is that the only way to know what the actual inputs are is to look at the class documentation or, even worse, into the code of the class. In addition to that, there is no enforced protocol that guides the developers toward the creation of a coherent class. For example, if we specify `hasMotor: true`, then we are required to also specify a `motorCount`, a `motorBrand`, and a `motorModel`, but there is nothing in this interface that conveys this information to us.

Như chúng ta có thể nhận thấy từ mã trước, constructor mới này quả thực tốt hơn nhiều so với bản gốc vì nó cho phép chúng ta thấy rõ ràng tham số nào nhận mỗi giá trị. Tuy nhiên, chúng ta có thể làm tốt hơn thế này nữa. Một nhược điểm của việc dùng một object literal duy nhất để truyền tất cả đầu vào cùng một lúc là cách duy nhất để biết các đầu vào thực sự là gì là phải xem tài liệu của lớp hoặc, tệ hơn nữa, xem mã của lớp. Thêm vào đó, không có giao thức bắt buộc nào hướng dẫn các lập trình viên tới việc tạo một lớp mạch lạc. Ví dụ, nếu chúng ta chỉ định `hasMotor: true`, thì chúng ta cũng bắt buộc phải chỉ định một `motorCount`, một `motorBrand` và một `motorModel`, nhưng không có gì trong giao diện này truyền đạt thông tin đó cho chúng ta.

The Builder pattern fixes even these last few flaws and provides a fluent interface that is simple to read, self-documenting, and that provides guidance toward the creation of a coherent object. Let's take a look at the `BoatBuilder` class, which implements the Builder pattern for the `Boat` class:

Mẫu Builder khắc phục cả những thiếu sót cuối cùng này và cung cấp một giao diện trôi chảy đơn giản để đọc, tự tài liệu hóa, và cung cấp hướng dẫn tới việc tạo một đối tượng mạch lạc. Hãy xem lớp `BoatBuilder`, vốn triển khai mẫu Builder cho lớp `Boat`:

```

class BoatBuilder {
  withMotors (count, brand, model) {
    this.hasMotor = true
    this.motorCount = count
    this.motorBrand = brand
    this.motorModel = model
    return this
  }
  withSails (count, material, color) {
    this.hasSails = true
    this.sailsCount = count
    this.sailsMaterial = material
  }
}

```

```

        this.sailsColor = color
        return this
    }
    hullColor (color) {
        this.hullColor = color
        return this
    }
    withCabin () {
        this.hasCabin = true
        return this
    }
    build() {
        return new Boat({
            hasMotor: this.hasMotor,
            motorCount: this.motorCount,
            motorBrand: this.motorBrand,
            motorModel: this.motorModel,
            hasSails: this.hasSails,
            sailsCount: this.sailsCount,
            sailsMaterial: this.sailsMaterial,
            sailsColor: this.sailsColor,
            hullColor: this.hullColor,
            hasCabin: this.hasCabin
        })
    }
}

```

To fully appreciate the positive impact that the Builder pattern has on the way we create our Boat objects, let's see an example of that:

Để hiểu trọn vẹn tác động tích cực mà mẫu Builder có lên cách chúng ta tạo các đối tượng Boat, hãy xem một ví dụ về điều đó:

```

const myBoat = new BoatBuilder()
  .withMotors(2, 'Best Motor Co. ', 'OM123')
  .withSails(1, 'fabric', 'white')
  .withCabin()
  .hullColor('blue')
  .build()

```

As we can see, the role of our BoatBuilder class is to collect all the parameters needed to create a Boat using some helper methods. We usually have a method for each parameter or set of related parameters, but there is not an exact rule to that. It is down to the designer of the Builder class to decide the name and behavior of each method responsible for collecting the input parameters.

Như chúng ta có thể thấy, vai trò của lớp BoatBuilder là thu thập tất cả các tham số cần thiết để tạo một Boat bằng cách dùng một số phương thức trợ giúp. Chúng ta thường có một phương thức cho mỗi tham số hoặc tập hợp các tham số liên quan, nhưng không có quy tắc chính xác nào cho điều đó. Tùy thuộc vào người thiết kế lớp Builder để quyết định tên và hành vi của mỗi phương thức chịu trách nhiệm thu thập các tham số đầu vào.

We can instead summarize some general rules for implementing the Builder pattern, as follows:

Thay vào đó, chúng ta có thể tóm tắt một số quy tắc chung để triển khai mẫu Builder, như

sau:

- The main objective is to break down a complex constructor into multiple, more **readable**, and more manageable steps.
- Try to create builder methods that can set multiple related parameters at once.
- Deduce and implicitly set parameters based on the values received as input by a setter method, and in general, try to encapsulate as much parameter setting related logic into the setter methods so that the consumer of the builder interface is free from doing so.
- If necessary, it's possible to further manipulate the parameters (for example, type casting, normalization, or extra validation) before passing them to the constructor of the class being built to simplify the work left to do by the builder class consumer even more.
 - Mục tiêu chính là chia nhỏ một constructor phức tạp thành nhiều bước dễ đọc và dễ quản lý hơn.
 - Cố gắng tạo các phương thức builder có thể thiết lập nhiều tham số liên quan cùng một lúc.
 - Suy ra và thiết lập ngầm các tham số dựa trên các giá trị nhận được làm đầu vào bởi một phương thức setter, và nói chung, cố gắng đóng gói càng nhiều logic liên quan đến việc thiết lập tham số vào trong các phương thức setter càng tốt sao cho người dùng giao diện builder được giải thoát khỏi việc làm điều đó.
 - Nếu cần, có thể thao tác thêm các tham số (ví dụ, ép kiểu, chuẩn hóa, hoặc xác thực bổ sung) trước khi truyền chúng cho constructor của lớp đang được xây dựng để đơn giản hóa hơn nữa công việc còn lại mà người dùng lớp builder phải làm.

In JavaScript, the Builder pattern can also be applied to invoke functions, not just to build objects using their constructor. In fact, from a technical point of view, the two scenarios are almost identical. The major difference when dealing with functions is that instead of having a `build()` method, we would have an `invoke()` method that invokes the complex function with the parameters collected by the builder object and returns any eventual result to the caller.

Trong JavaScript, mẫu Builder cũng có thể được áp dụng để gọi các hàm, chứ không chỉ để xây dựng các đối tượng bằng constructor của chúng. Thực tế, từ góc độ kỹ thuật, hai kịch bản gần như giống hệt nhau. Sự khác biệt lớn nhất khi làm việc với các hàm là thay vì có một phương thức `build()`, chúng ta sẽ có một phương thức `invoke()` gọi hàm phức tạp với các tham số được thu thập bởi đối tượng builder và trả về bất kỳ kết quả nào cho người gọi.

Next, we will work on a more concrete example that makes use of the Builder pattern we've just learned.

Tiếp theo, chúng ta sẽ làm việc trên một ví dụ cụ thể hơn sử dụng mẫu Builder mà chúng ta vừa học.

Implementing a URL object builder — Triển khai một builder đối tượng URL

We want to implement a `Url` class that can hold all the components of a standard URL, validate them, and format them back into a string. This class is going to be intentionally simple and minimal, so for standard production use, we recommend the built-in `URL` class ([nodejsdp.link/docs-url](https://nodejs.org/docs/latest/api/url)). Now, let's implement our custom `Url` class in a file called `url.js`:

Chúng ta muốn triển khai một lớp `Url` có thể giữ tất cả các thành phần của một URL chuẩn, xác thực chúng, và định dạng chúng trở lại thành một chuỗi. Lớp này được làm đơn giản và tối thiểu một cách có chủ đích, nên đối với việc dùng trong production chuẩn, chúng

tôi khuyến nghị lớp URL tích hợp sẵn ([nodejsdp.link/docs-url](https://nodejs.org/docs/latest/api/url)). Bây giờ, hãy triển khai lớp `Url` tùy chỉnh của chúng ta trong một tập tin tên là `url.js`:

```
export class Url {
  constructor (protocol, username, password, hostname,
    port, pathname, search, hash) {
    this.protocol = protocol
    this.username = username
    this.password = password
    this.hostname = hostname
    this.port = port
    this.pathname = pathname
    this.search = search
    this.hash = hash
    this.validate()
  }
  validate () {
    if (!this.protocol || !this.hostname) {
      throw new Error('Must specify at least a ' +
        'protocol and a hostname')
    }
  }
  toString () {
    let url = ''
    url += `${this.protocol}://`
    if (this.username && this.password) {
      url += `${this.username}:${this.password}@`
    }
    url += this.hostname
    if (this.port) {
      url += this.port
    }
    if (this.pathname) {
      url += this.pathname
    }
    if (this.search) {
      url += `?${this.search}`
    }
    if (this.hash) {
      url += `#${this.hash}`
    }
    return url
  }
}
```

A standard URL is made of several components, so to take them all in, the `Url` class' constructor is inevitably big. Invoking such a constructor can be a challenge, as we have to keep track of the argument position to know what component of the URL we are passing. Take a look at the following example to get an idea of this:

Một URL chuẩn được tạo thành từ nhiều thành phần, nên để nhận hết chúng vào, constructor của lớp `Url` chắc chắn lớn. Gọi một constructor như vậy có thể là một thử

thách, vì chúng ta phải theo dõi vị trí đối số để biết thành phần nào của URL chúng ta đang truyền. Hãy xem ví dụ sau để hình dung điều này:

```
return new Url('https', null, null, 'example.com', null, null, null,
null)
```

This is the perfect situation for applying the Builder pattern we just learned. Let's do that now. The plan is to create a `UrlBuilder` class, which has a setter method for each parameter (or set of related parameters) needed to instantiate the `Url` class. Finally, the builder is going to have a `build()` method to retrieve a new `Url` instance that's been created using all the parameters that have been set in the builder. So, let's implement the builder in a file called `urlBuilder.js`:

Đây là tình huống hoàn hảo để áp dụng mẫu Builder mà chúng ta vừa học. Hãy làm điều đó ngay bây giờ. Kế hoạch là tạo một lớp `UrlBuilder`, có một phương thức setter cho mỗi tham số (hoặc tập hợp các tham số liên quan) cần thiết để khởi tạo lớp `Url`. Cuối cùng, builder sẽ có một phương thức `build()` để lấy về một thể hiện `Url` mới được tạo bằng tất cả các tham số đã được thiết lập trong builder. Vậy, hãy triển khai builder trong một tập tin tên là `urlBuilder.js`:

```
export class UrlBuilder {
  setProtocol (protocol) {
    this.protocol = protocol
    return this
  }
  setAuthentication (username, password) {
    this.username = username
    this.password = password
    return this
  }
  setHostname (hostname) {
    this.hostname = hostname
    return this
  }
  setPort (port) {
    this.port = port
    return this
  }
  setPathname (pathname) {
    this.pathname = pathname
    return this
  }
  setSearch (search) {
    this.search = search
    return this
  }
  setHash (hash) {
    this.hash = hash
    return this
  }
  build () {
    return new Url(this.protocol, this.username, this.password,
      this.hostname, this.port, this.pathname, this.search,
```



```

    this.hash)
  }
}

```

This should be pretty straightforward. Just note the way we coupled together the username and password parameters into a single `setAuthentication()` method. This clearly conveys the fact that if we want to specify any authentication information in the `Url`, we have to provide both username and password.

Điều này khá đơn giản. Chỉ cần lưu ý cách chúng ta ghép username và password lại với nhau thành một phương thức `setAuthentication()` duy nhất. Điều này truyền đạt rõ ràng thực tế rằng nếu chúng ta muốn chỉ định bất kỳ thông tin xác thực nào trong `Url`, chúng ta phải cung cấp cả username và password.

Now, we are ready to try our `UrlBuilder` and witness its benefits over using the `Url` class directly. We can do that in a file called `index.js`:

Bây giờ, chúng ta đã sẵn sàng thử `UrlBuilder` của mình và chứng kiến lợi ích của nó so với việc dùng lớp `Url` trực tiếp. Chúng ta có thể làm điều đó trong một tập tin tên là `index.js`:

```

import { UrlBuilder } from './urlBuilder.js'
const url = new UrlBuilder()
  .setProtocol('https')
  .setAuthentication('user', 'pass')
  .setHostname('example.com')
  .build()
console.log(url.toString())

```

As we can see, the readability of the code has improved dramatically. Each setter method clearly gives us a hint of what parameter we are setting, and on top of that, they provide some guidance on how those parameters must be set (for example, username and password must be set together). The Builder pattern can also be implemented directly into the target class. For example, we could have refactored the `Url` class by adding an empty constructor (and therefore no validation at the object's creation time) and the setter methods for the various components, rather than creating a separate `UrlBuilder` class. However, this approach has a major flaw. Using a builder that is separate from the target class has the advantage of always producing instances that are guaranteed to be in a consistent state. For example, every `Url` object returned by `UrlBuilder.build()` is guaranteed to be valid and in a consistent state; calling `toString()` on such objects will always return a valid URL. The same cannot be said if we implemented the Builder pattern on the `Url` class directly. In fact, in this case, if we invoke `toString()` while we are still setting the various URL components, its return value may not be valid. This can be mitigated by adding extra validations, but at the cost of adding more complexity.

Như chúng ta có thể thấy, tính dễ đọc của mã đã được cải thiện đáng kể. Mỗi phương thức setter cho chúng ta một gợi ý rõ ràng về tham số nào chúng ta đang thiết lập, và trên hết, chúng cung cấp một số hướng dẫn về cách những tham số đó phải được thiết lập (ví dụ, username và password phải được thiết lập cùng nhau). Mẫu Builder cũng có thể được triển khai trực tiếp vào lớp đích. Ví dụ, chúng ta có thể đã tái cấu trúc lớp `Url` bằng cách thêm một constructor rỗng (và do đó không có xác thực tại thời điểm tạo đối tượng) và các phương thức setter cho các thành phần khác nhau, thay vì tạo một lớp `UrlBuilder` riêng. Tuy nhiên, cách tiếp cận này có một nhược điểm lớn. Dùng một builder tách rời khỏi lớp đích có lợi thế là luôn tạo ra các thể hiện được đảm bảo ở trạng thái nhất quán. Ví dụ, mọi đối tượng `Url` được trả về bởi `UrlBuilder.build()` đều được đảm bảo hợp lệ và ở trạng

thái nhất quán; gọi `toString()` trên những đối tượng như vậy sẽ luôn trả về một URL hợp lệ. Không thể nói điều tương tự nếu chúng ta triển khai mẫu Builder trực tiếp trên lớp `Url`. Thực tế, trong trường hợp này, nếu chúng ta gọi `toString()` trong khi vẫn đang thiết lập các thành phần URL khác nhau, giá trị trả về của nó có thể không hợp lệ. Điều này có thể được giảm thiểu bằng cách thêm các xác thực bổ sung, nhưng phải trả giá bằng việc thêm độ phức tạp.

In the wild — Trong thực tế

The Builder pattern is a quite common pattern in Node.js and JavaScript as it provides a very elegant solution to the problem of creating complex objects or invoking complex functions. One perfect example is creating new HTTP(S) client requests with the `request()` API from the `http` and `https` built-in modules. If we look at its documentation (available at [nodejsdp.link/docs-http-request](https://nodejs.org/docs/latest/api/http-request)), we can immediately see it accepts a large amount of options, which is the usual sign that the Builder pattern can potentially provide a better interface. In fact, one of the most popular HTTP(S) request wrappers, `superagent` ([nodejsdp.link/superagent](https://nodejs.org/docs/latest/api/superagent)), aims to simplify the creation of new requests by implementing the Builder pattern, thus providing a fluent interface to create new requests step by step. See the following code fragment for an example:

Mẫu Builder là một mẫu khá phổ biến trong Node.js và JavaScript vì nó cung cấp một giải pháp rất thanh lịch cho vấn đề tạo các đối tượng phức tạp hoặc gọi các hàm phức tạp. Một ví dụ hoàn hảo là tạo các request client HTTP(S) mới bằng API `request()` từ các module tích hợp sẵn `http` và `https`. Nếu chúng ta xem tài liệu của nó (có tại [nodejsdp.link/docs-http-request](https://nodejs.org/docs/latest/api/http-request)), chúng ta có thể ngay lập tức thấy nó chấp nhận một lượng lớn các tùy chọn, đây là dấu hiệu thường thấy rằng mẫu Builder có khả năng cung cấp một giao diện tốt hơn. Thực tế, một trong những wrapper request HTTP(S) phổ biến nhất, `superagent` ([nodejsdp.link/superagent](https://nodejs.org/docs/latest/api/superagent)), nhằm đơn giản hóa việc tạo các request mới bằng cách triển khai mẫu Builder, qua đó cung cấp một giao diện trôi chảy để tạo các request mới từng bước. Xem đoạn mã sau để có một ví dụ:

```
superagent
.post('https://example.com/api/person')
.send({ name: 'John Doe', role: 'user' })
.set('accept', 'json')
.then((response) => {
  // deal with the response
})
```

From the previous code, we can note that this is an unusual builder; in fact, we don't have a `build()` or `invoke()` method (or any other method with a similar purpose), and have not used the `new` operator. What triggers the request instead is an invocation to the `then()` method. It's interesting to note that the `superagent` request object is not a **promise** but rather a custom thenable where the `then()` method triggers the execution of the request we have built with the builder object.

Từ mã trước, chúng ta có thể nhận thấy đây là một builder bất thường; thực tế, chúng ta không có phương thức `build()` hay `invoke()` (hoặc bất kỳ phương thức nào khác có mục đích tương tự), và đã không dùng toán tử `new`. Điều kích hoạt request thay vào đó là một lời gọi tới phương thức `then()`. Điều thú vị cần lưu ý là đối tượng request của `superagent` không phải là một promise mà là một thenable tùy chỉnh, nơi phương thức `then()` kích hoạt việc thực thi request mà chúng ta đã xây dựng với đối tượng builder.

We already discussed thenables in Chapter 5, Asynchronous **Control Flow** Patterns with Promises and **Async/Await**.

Chúng ta đã thảo luận về thenable trong Chương 5, Các mẫu kiểm soát luồng bất đồng bộ với Promise và Async/Await.

If you take a look at the library's code, you will see the Builder pattern in action in the Request class (nodejsdp.link/superagent-src-builder).

Nếu bạn xem mã của thư viện, bạn sẽ thấy mẫu Builder hoạt động trong lớp Request (nodejsdp.link/superagent-src-builder).

This concludes our exploration of the Builder pattern. Next, we'll look at the Revealing Constructor pattern.

Điều này kết thúc việc khám phá mẫu Builder của chúng ta. Tiếp theo, chúng ta sẽ xem mẫu Revealing Constructor.

Revealing Constructor — Revealing Constructor

The Revealing Constructor pattern is one of those patterns that you won't find in the “Gang of Four” book, since it originated directly from the JavaScript and the Node.js community. It solves a very tricky problem, which is: how can we “reveal” some private functionality of an object only at the moment of the object's creation? This is particularly useful when we want to allow an object's internals to be manipulated only during its creation phase. This allows for a few interesting scenarios, such as:

Mẫu Revealing Constructor là một trong những mẫu mà bạn sẽ không tìm thấy trong cuốn sách “Gang of Four”, vì nó bắt nguồn trực tiếp từ cộng đồng JavaScript và Node.js. Nó giải quyết một vấn đề rất hóc búa, đó là: làm thế nào chúng ta có thể “phơi bày” một số chức năng riêng tư của một đối tượng chỉ tại thời điểm tạo đối tượng? Điều này đặc biệt hữu ích khi chúng ta muốn cho phép nội bộ của một đối tượng chỉ được thao tác trong giai đoạn tạo của nó. Điều này cho phép một vài kịch bản thú vị, chẳng hạn như:

- Creating objects that can be modified only at creation time
- Creating objects whose custom behavior can be defined only at creation time
- Creating objects that can be initialized only once at creation time
 - Tạo các đối tượng chỉ có thể được sửa đổi tại thời điểm tạo
 - Tạo các đối tượng mà hành vi tùy chỉnh của chúng chỉ có thể được định nghĩa tại thời điểm tạo
 - Tạo các đối tượng chỉ có thể được khởi tạo một lần tại thời điểm tạo

These are just a few possibilities enabled by the Revealing Constructor pattern. But to better understand all the possible use cases, let's see what the pattern is about by looking at the following code fragment:

Đây chỉ là một vài khả năng được kích hoạt bởi mẫu Revealing Constructor. Nhưng để hiểu rõ hơn tất cả các trường hợp sử dụng có thể, hãy xem mẫu này nói về điều gì bằng cách nhìn vào đoạn mã sau:

```
// (1) (2) (3)
const object = new SomeClass(function executor(revealedMembers) {
  // manipulation code ...
})
```

As we can see from the previous code, the Revealing Constructor pattern is made of three fundamental elements; a constructor (1) that takes a function as input (the executor (2)), which is

invoked at creation time and receives a subset of the object's internals as input (revealed members (3)).

Như chúng ta có thể thấy từ mã trước, mẫu Revealing Constructor được tạo thành từ ba yếu tố cơ bản; một constructor (1) nhận một hàm làm đầu vào (executor (2)), được gọi tại thời điểm tạo và nhận một tập con của nội bộ đối tượng làm đầu vào (các thành viên được phơi bày (3)).

For the pattern to work, the revealed functionality must otherwise be not accessible by the users of the object once it is created. This can be achieved with one of the encapsulation techniques we've mentioned in the previous section regarding the Factory pattern.

Để mẫu này hoạt động, chức năng được phơi bày phải không thể truy cập được bằng cách khác bởi người dùng của đối tượng một khi nó đã được tạo. Điều này có thể đạt được với một trong các kỹ thuật đóng gói mà chúng ta đã đề cập trong phần trước về mẫu Factory.

Domenic Denicola was the first to identify and name the pattern in one of his blog posts, which can be found at nodejsdp.link/domenic-revealing-constructor.

Domenic Denicola là người đầu tiên xác định và đặt tên cho mẫu này trong một trong các bài blog của ông, có thể tìm thấy tại nodejsdp.link/domenic-revealing-constructor.

Now, let's look at a couple of examples to better understand how the Revealing Constructor pattern works.

Bây giờ, hãy xem một vài ví dụ để hiểu rõ hơn cách mẫu Revealing Constructor hoạt động.

Building an immutable buffer — Xây dựng một buffer bất biến

Immutable objects and data structures have many excellent properties that make them ideal to use in countless situations in place of their mutable (or changeable) counterparts. Immutable refers to the property of an object by which its data or state becomes unmodifiable once it's been created.

Các đối tượng và cấu trúc dữ liệu bất biến (immutable) có nhiều tính chất tuyệt vời khiến chúng lý tưởng để dùng trong vô số tình huống thay cho các đối tác có thể thay đổi (mutable) của chúng. Bất biến đề cập đến tính chất của một đối tượng mà theo đó dữ liệu hoặc trạng thái của nó trở nên không thể sửa đổi một khi nó đã được tạo.

With immutable objects, we don't need to create defensive copies before passing them around to other libraries or functions. We simply have a strong guarantee, by definition, that they won't be modified, even when they are passed to code that we don't know or control.

Với các đối tượng bất biến, chúng ta không cần tạo các bản sao phòng thủ trước khi truyền chúng cho các thư viện hoặc hàm khác. Chúng ta đơn giản có một sự đảm bảo mạnh mẽ, theo định nghĩa, rằng chúng sẽ không bị sửa đổi, ngay cả khi chúng được truyền tới mã mà chúng ta không biết hoặc không kiểm soát.

Modifying an immutable object can only be done by creating a new copy and can make the code more maintainable and easier to reason about. We do this to make it easier to keep track of state changes.

Việc sửa đổi một đối tượng bất biến chỉ có thể được thực hiện bằng cách tạo một bản sao mới và có thể khiến mã dễ bảo trì hơn và dễ suy luận hơn. Chúng ta làm điều này để dễ theo dõi các thay đổi trạng thái hơn.

Another common use case for immutable objects is efficient change detection. Since every change requires a copy and if we assume that every copy corresponds to a modification, then detecting a change is as simple as using the strict equality operator (or triple equal `===`). This technique is used extensively in frontend programming to efficiently detect if the UI needs refreshing.

Một trường hợp sử dụng phổ biến khác cho các đối tượng bất biến là phát hiện thay đổi hiệu quả. Vì mọi thay đổi đều yêu cầu một bản sao và nếu chúng ta giả định rằng mọi bản sao tương ứng với một sửa đổi, thì việc phát hiện một thay đổi đơn giản như dùng toán tử bằng nghiêm ngặt (hay ba dấu bằng `===`). Kỹ thuật này được dùng rộng rãi trong lập trình frontend để phát hiện hiệu quả xem giao diện người dùng có cần làm mới hay không.

In this context, let's now create a simple immutable version of the Node.js `Buffer` component (nodejsdp.link/docs-buffer) using the Revealing Constructor pattern. The pattern allows us to manipulate an immutable buffer only at creation time. Let's implement our immutable buffer in a new file called `immutableBuffer.js`, as follows:

Trong bối cảnh này, bây giờ hãy tạo một phiên bản bất biến đơn giản của thành phần `Buffer` của Node.js (nodejsdp.link/docs-buffer) bằng mẫu Revealing Constructor. Mẫu này cho phép chúng ta thao tác một buffer bất biến chỉ tại thời điểm tạo. Hãy triển khai buffer bất biến của chúng ta trong một tập tin mới tên là `immutableBuffer.js`, như sau:

```
const MODIFIER_NAMES = ['swap', 'write', 'fill']
export class ImmutableBuffer {
  constructor (size, executor) {
    const buffer = Buffer.alloc(size) // (1)
    const modifiers = {} // (2)
    for (const prop in buffer) { // (3)
      if (typeof buffer[prop] !== 'function') {
        continue
      }
      if (MODIFIER_NAMES.some(m => prop.startsWith(m))) { // (4)
        modifiers[prop] = buffer[prop].bind(buffer)
      } else {
        this[prop] = buffer[prop].bind(buffer) // (5)
      }
    }
    executor(modifiers) // (6)
  }
}
```

Let's now see how our new `ImmutableBuffer` class works:

Bây giờ hãy xem lớp `ImmutableBuffer` mới của chúng ta hoạt động như thế nào:

1. First, we allocate a new Node.js `Buffer` (`buffer`) of the size specified in the `size` constructor argument.
2. Then, we create an object literal (`modifiers`) to hold all the methods that can mutate the buffer.
3. After that, we iterate over all the properties (own and inherited) of our internal buffer, making sure to skip all those that are not functions.
4. Next, we try to identify if the current `prop` is a method that allows us to modify the buffer. We do that by trying to match its name with one of the strings in the `MODIFIER_NAMES` array. If we have such a method, we bind it to the buffer instance, and then we add it to the `modifiers` object.

5. If our method is not a modifier method, then we add it directly to the current instance (`this`).
6. Finally, we invoke the `executor` function received as input in the constructor and pass the `modifiers` object as an argument, which will allow `executor` to mutate our internal buffer.

1. Đầu tiên, chúng ta cấp phát một Buffer Node.js mới (`buffer`) có kích thước được chỉ định trong đối số constructor `size`.
2. Sau đó, chúng ta tạo một object literal (`modifiers`) để giữ tất cả các phương thức có thể biến đổi `buffer`.
3. Sau đó, chúng ta lặp qua tất cả các thuộc tính (của riêng và được kế thừa) của `buffer` nội bộ của chúng ta, đảm bảo bỏ qua tất cả những thuộc tính không phải là hàm.
4. Tiếp theo, chúng ta cố gắng xác định xem `prop` hiện tại có phải là một phương thức cho phép chúng ta sửa đổi `buffer` hay không. Chúng ta làm điều đó bằng cách cố gắng so khớp tên của nó với một trong các chuỗi trong mảng `MODIFIER_NAMES`. Nếu chúng ta có một phương thức như vậy, chúng ta gắn (`bind`) nó với thể hiện `buffer`, rồi thêm nó vào đối tượng `modifiers`.
5. Nếu phương thức của chúng ta không phải là phương thức `modifier`, thì chúng ta thêm nó trực tiếp vào thể hiện hiện tại (`this`).
6. Cuối cùng, chúng ta gọi hàm `executor` nhận làm đầu vào trong constructor và truyền đối tượng `modifiers` làm đối số, điều này sẽ cho phép `executor` biến đổi `buffer` nội bộ của chúng ta.

In practice, our `ImmutableBuffer` is acting as a **proxy** between its consumers and the internal buffer object. Some of the methods of the buffer instance are exposed directly through the `ImmutableBuffer` interface (mainly the read-only methods), while others are provided to the `executor` function (the modifier methods).

Trong thực tế, `ImmutableBuffer` của chúng ta đang hoạt động như một proxy giữa người dùng của nó và đối tượng `buffer` nội bộ. Một số phương thức của thể hiện `buffer` được phơi bày trực tiếp thông qua giao diện `ImmutableBuffer` (chủ yếu là các phương thức chỉ đọc), trong khi những phương thức khác được cung cấp cho hàm `executor` (các phương thức `modifier`).

We will analyze the Proxy pattern in more detail in Chapter 8, Structural Design Patterns.

Chúng ta sẽ phân tích mẫu Proxy chi tiết hơn trong Chương 8, Các mẫu thiết kế cấu trúc.

Please keep in mind that this is just a demonstration of the Revealing Constructor pattern, so the implementation of the immutable buffer is intentionally kept simple. For example, we are not exposing the size of the buffer or providing other means to initialize the buffer. We'll leave this to you as an exercise.

Hãy nhớ rằng đây chỉ là một minh họa của mẫu Revealing Constructor, nên cách triển khai của `buffer` bất biến được giữ đơn giản một cách có chủ đích. Ví dụ, chúng ta không phơi bày kích thước của `buffer` hoặc cung cấp các phương tiện khác để khởi tạo `buffer`. Chúng tôi sẽ để lại điều này cho bạn như một bài tập.

Now, let's write some code to demonstrate how to use our new `ImmutableBuffer` class. Let's create a new file, `index.js`, containing the following code:

Bây giờ, hãy viết một số mã để minh họa cách dùng lớp `ImmutableBuffer` mới của chúng ta. Hãy tạo một tập tin mới, `index.js`, chứa mã sau:

```
import { ImmutableBuffer } from './immutableBuffer.js'
const hello = 'Hello!'
const immutable = new ImmutableBuffer(hello.length,
```



```

    ({ write }) => {                                     // (1)
      write(hello)
    })
    console.log(String.fromCharCode(immutable.readInt8(0))) // (2)
    // the following line will throw
    // "TypeError: immutable.write is not a function"
    // immutable.write('Hello?')                         // (3)

```

The first thing we can note from the previous code is how the executor function uses the `write()` function (which is part of the modifier methods) to write a string into the buffer (1). In a similar way, the executor function could've used `fill()`, `writeInt8()`, `swap16()` or any other method exposed in the modifiers object.

Điều đầu tiên chúng ta có thể nhận thấy từ mã trước là cách hàm executor dùng hàm `write()` (vốn là một phần của các phương thức modifier) để ghi một chuỗi vào buffer (1). Theo cách tương tự, hàm executor có thể đã dùng `fill()`, `writeInt8()`, `swap16()` hoặc bất kỳ phương thức nào khác được phơi bày trong đối tượng modifiers.

The code we've just seen also demonstrates how the new `ImmutableBuffer` instance exposes only the methods that don't mutate the buffer, such as `readInt8()` (2), while it doesn't provide any method to change the content of the buffer (3).

Mã chúng ta vừa thấy cũng minh họa cách thể hiện `ImmutableBuffer` mới chỉ phơi bày các phương thức không biến đổi buffer, chẳng hạn như `readInt8()` (2), trong khi nó không cung cấp bất kỳ phương thức nào để thay đổi nội dung của buffer (3).

In the wild — Trong thực tế

The Revealing Constructor pattern offers very strong guarantees and for this reason, it's mainly used in contexts where we need to provide foolproof encapsulation. A perfect application of the pattern would be in components used by hundreds of thousands of developers that have to provide unopinionated interfaces and strict encapsulation. However, we can also use the pattern in our projects to improve reliability and simplify code sharing with other people and teams (since it can make an object safer to use by third parties).

Mẫu Revealing Constructor cung cấp những đảm bảo rất mạnh mẽ và vì lý do này, nó chủ yếu được dùng trong các bối cảnh mà chúng ta cần cung cấp đóng gói không thể phá vỡ. Một ứng dụng hoàn hảo của mẫu này sẽ là trong các thành phần được dùng bởi hàng trăm nghìn lập trình viên, vốn phải cung cấp các giao diện không thiên kiến và đóng gói nghiêm ngặt. Tuy nhiên, chúng ta cũng có thể dùng mẫu này trong các dự án của mình để cải thiện độ tin cậy và đơn giản hóa việc chia sẻ mã với người khác và các nhóm khác (vì nó có thể khiến một đối tượng an toàn hơn để được dùng bởi bên thứ ba).

A popular application of the Revealing Constructor pattern is in the JavaScript Promise class. Some of you may have already noticed it. When we create a new Promise from scratch, its constructor accepts as input an executor function that will receive the `resolve()` and `reject()` functions used to mutate the internal state of the Promise. Let's provide a reminder of what this looks like:

Một ứng dụng phổ biến của mẫu Revealing Constructor là trong lớp Promise của JavaScript. Một số bạn có thể đã nhận thấy điều đó. Khi chúng ta tạo một Promise mới từ đầu, constructor của nó chấp nhận làm đầu vào một hàm executor sẽ nhận các hàm `resolve()` và `reject()` được dùng để biến đổi trạng thái nội bộ của Promise. Hãy nhắc lại điều này trông như thế nào:

```
return new Promise((resolve, reject) => {
  // ...
})
```

Once created, the Promise state cannot be altered by any other means. All we can do is receive its fulfillment value or rejection reason using the methods we already learned about in Chapter 5, Asynchronous Control Flow Patterns with Promises and Async/Await.

Một khi được tạo, trạng thái của Promise không thể bị thay đổi bằng bất kỳ phương tiện nào khác. Tất cả những gì chúng ta có thể làm là nhận giá trị hoàn thành (fulfilment) hoặc lý do từ chối (rejection reason) của nó bằng các phương thức mà chúng ta đã học trong Chương 5, Các mẫu kiểm soát luồng bất đồng bộ với Promise và Async/Await.

Singleton — Singleton

Now, we are going to spend a few words on a pattern that is among the most used in object-oriented programming, which is the Singleton pattern. As we will see, Singleton is one of those patterns that has a trivial implementation in Node.js that's almost not worth discussing. However, there are a few caveats and limitations that every good Node.js developer must know.

Bây giờ, chúng ta sẽ dành đôi lời cho một mẫu nằm trong số những mẫu được dùng nhiều nhất trong lập trình hướng đối tượng, đó là mẫu Singleton. Như chúng ta sẽ thấy, Singleton là một trong những mẫu có cách triển khai tầm thường trong Node.js đến mức gần như không đáng để thảo luận. Tuy nhiên, có một vài lưu ý và hạn chế mà mọi lập trình viên Node.js giỏi phải biết.

The purpose of the Singleton pattern is to enforce the presence of only one instance of a class and centralize its access. There are a few reasons for using a single instance across all the components of an application:

Mục đích của mẫu Singleton là thực thi sự hiện diện của chỉ một thể hiện duy nhất của một lớp và tập trung hóa việc truy cập nó. Có một vài lý do để dùng một thể hiện duy nhất xuyên suốt tất cả các thành phần của một ứng dụng:

- For sharing stateful information
- For optimizing resource usage
- To synchronize access to a resource
 - Để chia sẻ thông tin có trạng thái
 - Để tối ưu hóa việc sử dụng tài nguyên
 - Để đồng bộ hóa truy cập tới một tài nguyên

As you can imagine, those are quite common scenarios. Take, for example, a typical Database class, which provides access to a database:

Như bạn có thể hình dung, đó là những kịch bản khá phổ biến. Lấy ví dụ một lớp Database điển hình, cung cấp truy cập tới một cơ sở dữ liệu:

```
// 'Database.js'
export class Database {
  constructor (dbName, connectionDetails) {
    // ...
  }
  // ...
}
```

Typical implementations of such a class usually keep a pool of database connections, so it doesn't make sense to create a new Database instance for each request. Plus, a Database instance may store some stateful information, such as the list of pending transactions. So, our Database class meets two criteria for justifying the Singleton pattern. Therefore, what we usually want is to configure and instantiate one single Database instance at the start of our application and let every component use that single shared Database instance.

Các cách triển khai điển hình của một lớp như vậy thường giữ một pool các kết nối cơ sở dữ liệu, nên việc tạo một thể hiện Database mới cho mỗi request là vô nghĩa. Thêm vào đó, một thể hiện Database có thể lưu một số thông tin có trạng thái, chẳng hạn như danh sách các giao dịch đang chờ xử lý. Vậy, lớp Database của chúng ta đáp ứng hai tiêu chí để biện minh cho mẫu Singleton. Do đó, điều chúng ta thường muốn là cấu hình và khởi tạo một thể hiện Database duy nhất khi bắt đầu ứng dụng và để mọi thành phần dùng thể hiện Database được chia sẻ duy nhất đó.

A lot of people new to Node.js get confused about how to implement the Singleton pattern correctly; however, the answer is easier than what we might think. Simply exporting an instance from a module is already enough to obtain something very similar to the Singleton pattern. Consider, for example, the following lines of code:

Rất nhiều người mới với Node.js bối rối về cách triển khai mẫu Singleton một cách chính xác; tuy nhiên, câu trả lời dễ hơn chúng ta có thể nghĩ. Chỉ cần xuất một thể hiện từ một module đã đủ để có được một thứ rất giống với mẫu Singleton. Ví dụ, hãy xem xét các dòng mã sau:

```
// file 'dbInstance.js'
import { Database } from './Database.js'
export const dbInstance = new Database('my-app-db', {
  url: 'localhost:5432',
  username: 'user',
  password: 'password'
})
```

By simply exporting a new instance of our Database class, we can already assume that within the current package (which can easily be the entire code of our application), we are going to have only one instance of the dbInstance module. This is possible because, as we know from Chapter 2, The **Module System**, Node.js will cache the module, making sure not to execute its code at every import.

Bằng cách chỉ cần xuất một thể hiện mới của lớp Database, chúng ta đã có thể giả định rằng trong gói hiện tại (vốn có thể dễ dàng là toàn bộ mã của ứng dụng của chúng ta), chúng ta sẽ chỉ có một thể hiện của module dbInstance. Điều này khả thi vì, như chúng ta biết từ Chương 2, Hệ thống Module, Node.js sẽ cache module, đảm bảo không thực thi mã của nó tại mỗi lần import.

For example, we can easily obtain a shared instance of the dbInstance module, which we defined earlier, with the following line of code:

Ví dụ, chúng ta có thể dễ dàng có được một thể hiện được chia sẻ của module dbInstance mà chúng ta đã định nghĩa trước đó, với dòng mã sau:

```
import { dbInstance } from './dbInstance.js'
```

But there is a caveat. The module is cached using its full path as the lookup key, so it is only guaranteed to be a singleton within the current package. In fact, each package may have its own set of private dependencies inside its node_modules directory, which might result in multiple instances of the

same package and therefore of the same module, resulting in our singleton not really being unique anymore! This is, of course, a rare scenario, but it's important to understand what its consequences are.

Nhưng có một lưu ý. Module được cache bằng đường dẫn đầy đủ của nó làm khóa tra cứu, nên nó chỉ được đảm bảo là một singleton trong gói hiện tại. Thực tế, mỗi gói có thể có tập hợp các phụ thuộc riêng tư của riêng nó bên trong thư mục `node_modules`, điều này có thể dẫn đến nhiều thể hiện của cùng một gói và do đó của cùng một module, khiến singleton của chúng ta không còn thực sự duy nhất nữa! Đây tất nhiên là một kịch bản hiếm gặp, nhưng quan trọng là hiểu được hậu quả của nó là gì.

Consider, for example, the case in which the `Database.js` and `dbInstance.js` files that we saw earlier are wrapped into a package named `mydb`. The following lines of code would be in its `package.json` file:

Ví dụ, hãy xem xét trường hợp các tập tin `Database.js` và `dbInstance.js` mà chúng ta đã thấy trước đó được đóng gói thành một gói tên là `mydb`. Các dòng mã sau sẽ nằm trong tập tin `package.json` của nó:

```
{
  "name": "mydb",
  "version": "2.0.0",
  "type": "module",
  "main": "dbInstance.js"
}
```

Next, consider two packages (`package-a` and `package-b`), both of which have a single file called `index.js` containing the following code:

Tiếp theo, hãy xem xét hai gói (`package-a` và `package-b`), cả hai đều có một tập tin duy nhất tên là `index.js` chứa mã sau:

```
import { dbInstance } from 'mydb'
export function getDbInstance () {
  return dbInstance
}
```

Both `package-a` and `package-b` have a **dependency** on the `mydb` package. However, `package-a` depends on version `1.0.0` of the `mydb` package, while `package-b` depends on version `2.0.0` of the same package (which, for our example, has an identical implementation, but just a different version specified in its `package.json` file).

Cả `package-a` và `package-b` đều có một phụ thuộc vào gói `mydb`. Tuy nhiên, `package-a` phụ thuộc vào phiên bản `1.0.0` của gói `mydb`, trong khi `package-b` phụ thuộc vào phiên bản `2.0.0` của cùng gói đó (mà, cho ví dụ của chúng ta, có cách triển khai giống hệt, chỉ khác phiên bản được chỉ định trong tập tin `package.json` của nó).

Given the structure we just described, we would end up with the following package dependency tree:

Với cấu trúc mà chúng ta vừa mô tả, chúng ta sẽ kết thúc với cây phụ thuộc gói sau:

```
app/
|-- node_modules
|   |-- package-a
|   |   |-- node_modules
|   |   |   |-- mydb
|   |   |-- package-b
```

```
`-- node_modules
  |-- mydb
```

We end up with a directory structure like the one here because package-a and package-b require two different incompatible versions of the mydb module (for example, 1.0.0 versus 2.0.0). In this case, a typical package manager such as npm or yarn would not “hoist” the dependency to the top node_modules directory, but it will instead install a private copy of each package in an attempt to fix the version incompatibility.

Chúng ta kết thúc với một cấu trúc thư mục như cái ở đây vì package-a và package-b yêu cầu hai phiên bản không tương thích khác nhau của module mydb (ví dụ, 1.0.0 so với 2.0.0). Trong trường hợp này, một trình quản lý gói điển hình như npm hay yarn sẽ không “nâng” (hoist) phụ thuộc lên thư mục node_modules trên cùng, mà thay vào đó sẽ cài đặt một bản sao riêng tư của mỗi gói trong nỗ lực sửa sự không tương thích phiên bản.

With the directory structure we just saw, both package-a and package-b have a dependency on the mydb package; in turn, the app package, which is our root package, depends on both package-a and package-b.

Với cấu trúc thư mục mà chúng ta vừa thấy, cả package-a và package-b đều có một phụ thuộc vào gói mydb; lần lượt, gói app, vốn là gói gốc của chúng ta, phụ thuộc vào cả package-a và package-b.

The scenario we just described will break the assumption about the uniqueness of the database instance. In fact, consider the following file (index.js) located in the root folder of the app package:

Kịch bản chúng ta vừa mô tả sẽ phá vỡ giả định về tính duy nhất của thể hiện cơ sở dữ liệu. Thực tế, hãy xem xét tập tin sau (index.js) nằm trong thư mục gốc của gói app:

```
import { getDbInstance as getDbFromA } from 'package-a'
import { getDbInstance as getDbFromB } from 'package-b'
const isSame = getDbFromA() === getDbFromB()
console.log('Is the db instance in package-a the same ' +
  `as package-b? ${isSame ? 'YES' : 'NO'}`)
```

If you run the previous file, you will notice that the answer to Is the db instance in package-a the same as package-b? is NO. In fact, package-a and package-b will actually load two different instances of the dbInstance object because the mydb module will resolve to a different directory, depending on the package it is required from. This clearly break the assumptions of the Singleton pattern.

Nếu bạn chạy tập tin trước đó, bạn sẽ nhận thấy rằng câu trả lời cho Thể hiện db trong package-a có giống như package-b không? là NO. Thực tế, package-a và package-b thực sự sẽ nạp hai thể hiện khác nhau của đối tượng dbInstance vì module mydb sẽ phân giải tới một thư mục khác nhau, tùy thuộc vào gói mà nó được require từ đó. Điều này rõ ràng phá vỡ các giả định của mẫu Singleton.

If instead, both package-a and package-b required two versions of the mydb package compatible with each other, for example, ^2.0.1 and ^2.0.7, then the package manager would install the mydb package into the top-level node_modules directory (a practice known as dependency hoisting), effectively sharing the same instance with package-a, package-b, and the root package. At this point, we can easily say that the Singleton pattern, as described in the literature, does not exist in Node.js, unless we use a real global variable to store it, such as the following:

Nếu thay vào đó, cả package-a và package-b yêu cầu hai phiên bản của gói mydb tương thích với nhau, ví dụ ^2.0.1 và ^2.0.7, thì trình quản lý gói sẽ cài đặt gói mydb vào thư mục node_modules cấp cao nhất (một thực hành được biết đến là nâng phụ thuộc,

dependency hoisting), chia sẻ hiệu quả cùng một thể hiện với package-a, package-b và gói gốc. Tại điểm này, chúng ta có thể dễ dàng nói rằng mẫu Singleton, như được mô tả trong tài liệu, không tồn tại trong Node.js, trừ khi chúng ta dùng một biến toàn cục (global) thực sự để lưu nó, chẳng hạn như sau:

```
global.dbInstance = new Database('my-app-db', { /* ... */ })
```

This guarantees that the instance is the only one shared across the entire application and not just the same package. However, please consider that most of the time, we don't really need a pure singleton. In fact, we usually create and import singletons within the main package of an application or, at worst, in a subcomponent of the application that has been modularized into a dependency.

Điều này đảm bảo rằng thể hiện là cái duy nhất được chia sẻ trên toàn bộ ứng dụng chứ không chỉ cùng một gói. Tuy nhiên, hãy xem xét rằng hầu hết thời gian, chúng ta không thực sự cần một singleton thuần túy. Thực tế, chúng ta thường tạo và import các singleton trong gói chính của một ứng dụng hoặc, tệ nhất, trong một thành phần con của ứng dụng đã được module hóa thành một phụ thuộc.

If you are creating a package that is going to be used by third parties, try to keep it stateless to avoid the issues we've discussed in this section.

Nếu bạn đang tạo một gói sẽ được dùng bởi các bên thứ ba, hãy cố gắng giữ nó không có trạng thái (stateless) để tránh các vấn đề chúng ta đã thảo luận trong phần này.

Throughout this book, for simplicity, we will use the term singleton to describe a class instance or a stateful object exported by a module, even if this doesn't represent a real singleton in the strict definition of the term.

Xuyên suốt cuốn sách này, để đơn giản, chúng ta sẽ dùng thuật ngữ singleton để mô tả một thể hiện lớp hoặc một đối tượng có trạng thái được xuất bởi một module, ngay cả khi điều này không đại diện cho một singleton thực sự theo định nghĩa nghiêm ngặt của thuật ngữ.

Next, we are going to see the two main approaches for dealing with dependencies between modules, one based on the Singleton pattern and the other leveraging the Dependency Injection pattern.

Tiếp theo, chúng ta sẽ xem hai cách tiếp cận chính để xử lý các phụ thuộc giữa các module, một dựa trên mẫu Singleton và cái còn lại tận dụng mẫu Dependency Injection.

Wiring modules — Liên kết các module

Every application is the result of the aggregation of several components and, as the application grows, the way we connect these components becomes a win or lose factor for the maintainability and success of the project.

Mọi ứng dụng đều là kết quả của việc tổng hợp nhiều thành phần và, khi ứng dụng phát triển, cách chúng ta kết nối các thành phần này trở thành một yếu tố thắng hay thua đối với khả năng bảo trì và sự thành công của dự án.

When a component, A, needs component B to fulfill a given functionality, we say that “A is dependent on B” or, conversely, that “B is a dependency of A.” To appreciate this concept, let's present an example.

Khi một thành phần, A, cần thành phần B để hoàn thành một chức năng nhất định, chúng ta nói rằng “A phụ thuộc vào B” hoặc, ngược lại, rằng “B là một phụ thuộc của A”. Để hiểu rõ khái niệm này, hãy trình bày một ví dụ.

Let's say we want to write an API for a blogging system that uses a database to store its data. We can have a generic module implementing a database connection (`db.js`) and a blog module that exposes the main functionality to create and retrieve blog posts from the database (`blog.js`). The following figure illustrates the relationship between the database module and the blog module:

Giả sử chúng ta muốn viết một API cho một hệ thống blog dùng một cơ sở dữ liệu để lưu trữ dữ liệu của nó. Chúng ta có thể có một module tổng quát triển khai một kết nối cơ sở dữ liệu (`db.js`) và một module blog phơi bày chức năng chính để tạo và lấy các bài viết blog từ cơ sở dữ liệu (`blog.js`). Hình sau minh họa mối quan hệ giữa module cơ sở dữ liệu và module blog:

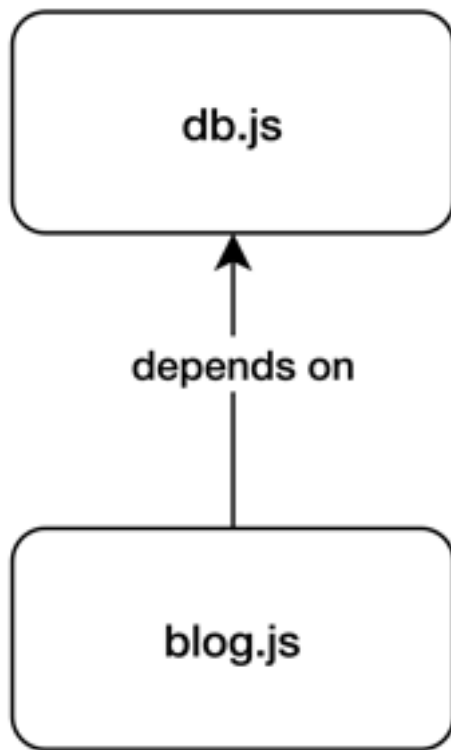


Figure 7.1: Dependency graph between the blog module and the database module

Hình 7.1: Đồ thị phụ thuộc giữa module blog và module cơ sở dữ liệu

In this section, we are going to see how we can model this dependency using two different approaches, one based on the Singleton pattern and the other using the Dependency Injection pattern.

Trong phần này, chúng ta sẽ xem cách chúng ta có thể mô hình hóa phụ thuộc này bằng hai cách tiếp cận khác nhau, một dựa trên mẫu Singleton và cái còn lại dùng mẫu Dependency Injection.

Singleton dependencies — Phụ thuộc Singleton

The simplest way to wire two modules together is by leveraging Node.js' module system. Stateful dependencies wired in this way are de facto singletons, as we discussed in the previous section.

Cách đơn giản nhất để liên kết hai module với nhau là tận dụng hệ thống module của Node.js. Các phụ thuộc có trạng thái được liên kết theo cách này về thực chất là các singleton, như chúng ta đã thảo luận trong phần trước.

To see how this works in practice, we are going to implement the simple blogging application that we described earlier using a singleton instance for the database connection. Let's see a possible implementation of this approach (the file `db.js`):

Để xem cách điều này hoạt động trong thực tế, chúng ta sẽ triển khai ứng dụng blog đơn giản mà chúng ta đã mô tả trước đó bằng cách dùng một thể hiện singleton cho kết nối cơ sở dữ liệu. Hãy xem một cách triển khai khả thi của cách tiếp cận này (tập tin `db.js`):

```
import { dirname, join } from 'path'
import { fileURLToPath } from 'url'
import sqlite3 from 'sqlite3'
const __dirname = dirname(fileURLToPath(import.meta.url))
export const db = new sqlite3.Database(
  join(__dirname, 'data.sqlite'))
```

In the previous code, we are using SQLite (nodejsdp.link/sqlite) as a database to store our posts. To interact with SQLite, we are using the module `sqlite3` (nodejsdp.link/sqlite3) from npm. SQLite is a database system that keeps all the data in a single local file. In our database module, we decided to use a file called `data.sqlite` saved in the same folder as the module. The preceding code creates a new instance of the database pointing to our data file and exports the database connection object as a singleton with the name `db`.

Trong mã trước, chúng ta dùng SQLite (nodejsdp.link/sqlite) làm cơ sở dữ liệu để lưu các bài viết của chúng ta. Để tương tác với SQLite, chúng ta dùng module `sqlite3` (nodejsdp.link/sqlite3) từ npm. SQLite là một hệ cơ sở dữ liệu giữ tất cả dữ liệu trong một tập tin cục bộ duy nhất. Trong module cơ sở dữ liệu của chúng ta, chúng ta quyết định dùng một tập tin tên là `data.sqlite` được lưu trong cùng thư mục với module. Mã trên tạo một thể hiện mới của cơ sở dữ liệu trở tới tập tin dữ liệu của chúng ta và xuất đối tượng kết nối cơ sở dữ liệu như một singleton với tên `db`.

Now, let's see how we can implement the `blog.js` module:

Bây giờ, hãy xem cách chúng ta có thể triển khai module `blog.js`:

```
import { promisify } from 'util'
import { db } from './db.js'
const dbRun = promisify(db.run.bind(db))
const dbAll = promisify(db.all.bind(db))
export class Blog {
  initialize () {
    const initQuery = `CREATE TABLE IF NOT EXISTS posts (
      id TEXT PRIMARY KEY,
      title TEXT NOT NULL,
      content TEXT,
      created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );`
    return dbRun(initQuery)
  }
  createPost (id, title, content, createdAt) {
    return dbRun('INSERT INTO posts VALUES (?, ?, ?, ?)',
```

```

    id, title, content, createdAt)
  }
  getAllPosts () {
    return dbAll('SELECT * FROM posts ORDER BY created_at DESC')
  }
}

```

The `blog.js` module exports a class called `Blog` containing three methods:

Module `blog.js` xuất một lớp tên là `Blog` chứa ba phương thức:

- `initialize()`: Creates the `posts` table if it doesn't exist. The table will be used to store the blog post's data.
- `createPost()`: Takes all the necessary parameters needed to create a post. It will execute an `INSERT` statement to add the new post to the database.
- `getAllPosts()`: Retrieves all the posts available in the database and returns them as an array.
 - `initialize()`: Tạo bảng `posts` nếu nó chưa tồn tại. Bảng sẽ được dùng để lưu trữ dữ liệu của bài viết blog.
 - `createPost()`: Nhận tất cả các tham số cần thiết để tạo một bài viết. Nó sẽ thực thi một câu lệnh `INSERT` để thêm bài viết mới vào cơ sở dữ liệu.
 - `getAllPosts()`: Lấy tất cả các bài viết có sẵn trong cơ sở dữ liệu và trả về chúng dưới dạng một mảng.

Now, let's create a module to try out the functionality of the `blog` module we just created (the file `index.js`):

Bây giờ, hãy tạo một module để thử nghiệm chức năng của module `blog` mà chúng ta vừa tạo (tập tin `index.js`):

```

import { Blog } from './blog.js'
async function main () {
  const blog = new Blog()
  await blog.initialize()
  const posts = await blog.getAllPosts()
  if (posts.length === 0) {
    console.log('No post available. Run `node import-posts.js` +
      ' to load some sample posts')
  }
  for (const post of posts) {
    console.log(post.title)
    console.log('-'.repeat(post.title.length))
    console.log(`Published on ${new Date(post.created_at)
      .toISOString()}`)
    console.log(post.content)
  }
}
main().catch(console.error)

```

This preceding module is very simple. We retrieve the array with all the posts using `blog.getAllPosts()` and then we loop over it and display the data for every single post, giving it a bit of formatting.

Module trên rất đơn giản. Chúng ta lấy mảng với tất cả các bài viết bằng `blog.getAllPosts()` và sau đó chúng ta lặp qua nó và hiển thị dữ liệu cho từng bài viết, định dạng nó một chút.

You can use the provided `import-posts.js` module to load some sample posts into the database before running `index.js`. You can find `import-posts.js` in the code repository of this book, along with the rest of the files for this example.

Bạn có thể dùng module `import-posts.js` được cung cấp để nạp một số bài viết mẫu vào cơ sở dữ liệu trước khi chạy `index.js`. Bạn có thể tìm thấy `import-posts.js` trong kho mã của cuốn sách này, cùng với phần còn lại của các tập tin cho ví dụ này.

As a fun exercise, you could try to modify the `index.js` module to generate HTML files; one for the blog index and then a dedicated file for each blog post. This way, you would build your own minimalistic static website **generator**!

Như một bài tập thú vị, bạn có thể thử sửa đổi module `index.js` để tạo các tập tin HTML; một cho trang chỉ mục của blog và sau đó một tập tin chuyên dụng cho mỗi bài viết blog. Theo cách này, bạn sẽ xây dựng trình tạo website tĩnh tối giản của riêng mình!

As we can see from the preceding code, we can implement a very simple commandline blog management system by leveraging the Singleton pattern to pass the db instance around. Most of the time, this is how we manage stateful dependencies in our application; however, there are situations in which this may not be enough. Using a singleton, as we have done in the previous example, is certainly the most simple, immediate, and readable solution to pass stateful dependencies around. However, what happens if we want to mock our database during our tests? What can we do if we want to let the user of the blogging CLI or the blogging API select another database backend, instead of the standard SQLite backend that we provide by default? For these use cases, a singleton can be an obstacle for implementing a properly structured solution.

Như chúng ta có thể thấy từ mã trên, chúng ta có thể triển khai một hệ thống quản lý blog dòng lệnh rất đơn giản bằng cách tận dụng mẫu Singleton để truyền thể hiện db xung quanh. Hầu hết thời gian, đây là cách chúng ta quản lý các phụ thuộc có trạng thái trong ứng dụng của mình; tuy nhiên, có những tình huống mà điều này có thể không đủ. Dùng một singleton, như chúng ta đã làm trong ví dụ trước, chắc chắn là giải pháp đơn giản nhất, trực tiếp nhất và dễ đọc nhất để truyền các phụ thuộc có trạng thái xung quanh. Tuy nhiên, điều gì xảy ra nếu chúng ta muốn mock cơ sở dữ liệu của mình trong các bài kiểm thử? Chúng ta có thể làm gì nếu chúng ta muốn cho người dùng của blog CLI hoặc blog API chọn một backend cơ sở dữ liệu khác, thay vì backend SQLite tiêu chuẩn mà chúng ta cung cấp mặc định? Đối với những trường hợp sử dụng này, một singleton có thể là một trở ngại cho việc triển khai một giải pháp được cấu trúc đúng đắn.

We could introduce `if` statements in our `db.js` module to pick different implementations based on some environment condition or some configuration. Alternatively, we could fiddle with the Node.js module system to intercept the import of the database file and replace it with something else. But, as you can image, these solutions are far from elegant.

Chúng ta có thể đưa các câu lệnh `if` vào module `db.js` của mình để chọn các cách triển khai khác nhau dựa trên một số điều kiện môi trường hoặc một số cấu hình. Cách khác, chúng ta có thể can thiệp vào hệ thống module của Node.js để chặn việc import tập tin cơ sở dữ liệu và thay thế nó bằng thứ khác. Nhưng, như bạn có thể hình dung, những giải pháp này còn lâu mới thanh lịch.

In the next section, we will learn about another **strategy** for wiring modules, which can be the ideal solution to some of the issues we discussed here.

Trong phần tiếp theo, chúng ta sẽ tìm hiểu về một chiến lược khác để liên kết các module, vốn có thể là giải pháp lý tưởng cho một số vấn đề chúng ta đã thảo luận ở đây.

Dependency Injection — Dependency Injection

The Node.js module system and the Singleton pattern can serve as great tools for organizing and wiring together the components of an application. However, these do not always guarantee success. If, on the one hand, they are simple to use and very practical, then on the other, they might introduce a tighter coupling between components.

Hệ thống module của Node.js và mẫu Singleton có thể đóng vai trò là những công cụ tuyệt vời để tổ chức và liên kết các thành phần của một ứng dụng. Tuy nhiên, những thứ này không phải lúc nào cũng đảm bảo thành công. Nếu, một mặt, chúng đơn giản để dùng và rất thiết thực, thì mặt khác, chúng có thể đưa vào một sự ràng buộc chặt hơn (tighter coupling) giữa các thành phần.

In the previous example, we can see that the `blog.js` module is tightly coupled with the `db.js` module. In fact, our `blog.js` module cannot work without the `database.js` module by design, nor can it use a different database module if necessary. We can easily fix this tight coupling between the two modules by leveraging the Dependency Injection pattern.

Trong ví dụ trước, chúng ta có thể thấy rằng module `blog.js` bị ràng buộc chặt với module `db.js`. Thực tế, module `blog.js` của chúng ta không thể hoạt động mà không có module `database.js` theo thiết kế, cũng không thể dùng một module cơ sở dữ liệu khác nếu cần. Chúng ta có thể dễ dàng sửa sự ràng buộc chặt này giữa hai module bằng cách tận dụng mẫu Dependency Injection.

Dependency Injection (DI) is a very simple pattern in which the dependencies of a component are provided as input by an external entity, often referred to as the injector.

Dependency Injection (DI) là một mẫu rất đơn giản trong đó các phụ thuộc của một thành phần được cung cấp làm đầu vào bởi một thực thể bên ngoài, thường được gọi là injector.

The injector initializes the different components and ties their dependencies together. It can be a simple initialization script or a more sophisticated global container that maps all the dependencies and centralizes the wiring of all the modules of the system. The main advantage of this approach is improved decoupling, especially for modules depending on stateful instances (for example, a database connection). Using DI, each dependency, instead of being hardcoded into the module, is received from the outside. This means that the dependent module can be configured to use any compatible dependency, and therefore the module itself can be reused in different contexts with minimal effort. The following diagram illustrates this idea:

Injector khởi tạo các thành phần khác nhau và buộc các phụ thuộc của chúng lại với nhau. Nó có thể là một script khởi tạo đơn giản hoặc một container toàn cục tinh vi hơn ánh xạ tất cả các phụ thuộc và tập trung hóa việc liên kết tất cả các module của hệ thống. Lợi thế chính của cách tiếp cận này là cải thiện sự tách rời, đặc biệt đối với các module phụ thuộc vào các thể hiện có trạng thái (ví dụ, một kết nối cơ sở dữ liệu). Dùng DI, mỗi phụ thuộc, thay vì được mã hóa cứng vào module, được nhận từ bên ngoài. Điều này có nghĩa là module phụ thuộc có thể được cấu hình để dùng bất kỳ phụ thuộc tương thích nào, và do đó bản thân module có thể được tái sử dụng trong các bối cảnh khác nhau với nỗ lực tối thiểu. Sơ đồ sau minh họa ý tưởng này:

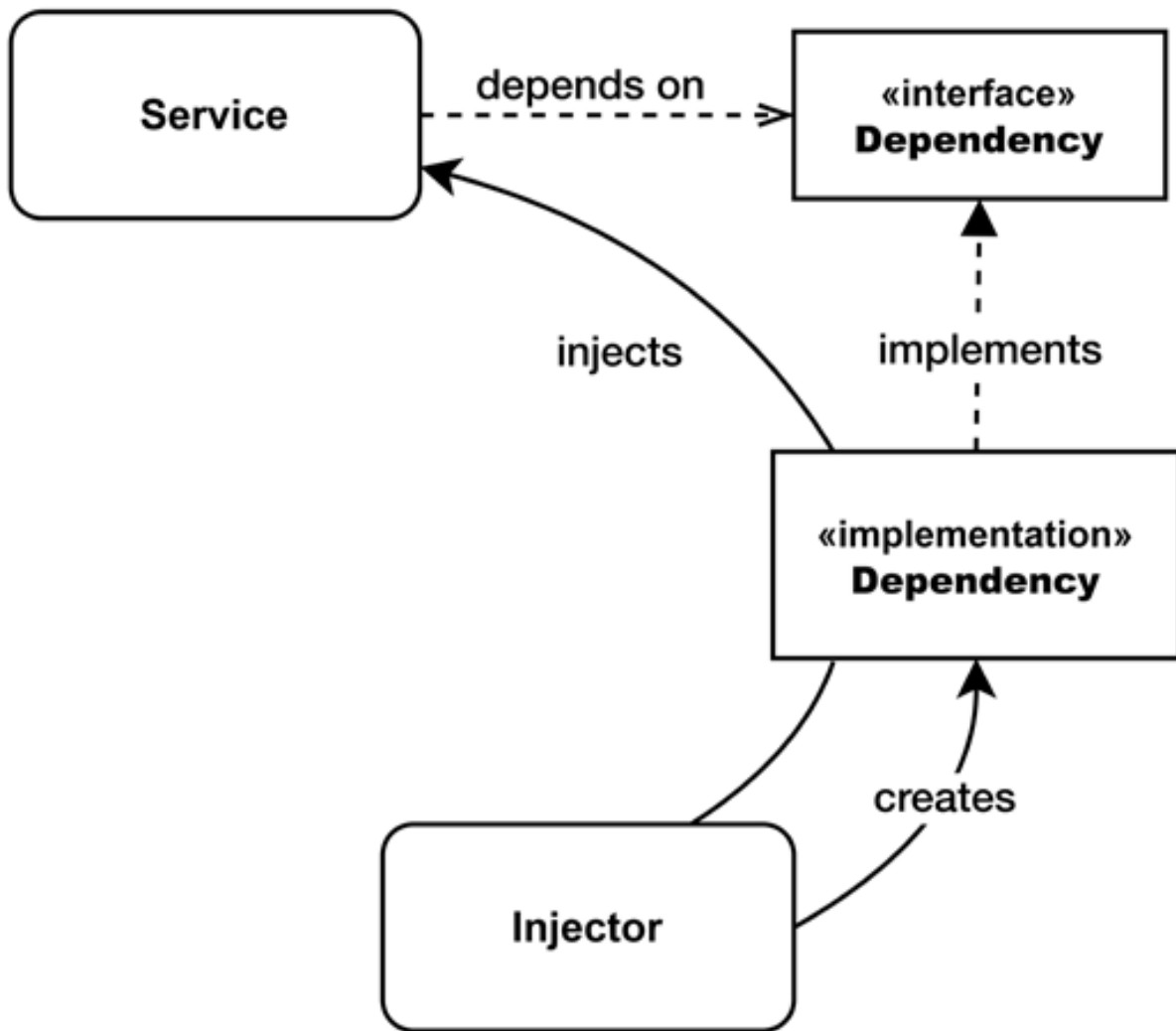


Figure 7.2: Dependency injection schematic

Hình 7.2: Sơ đồ dependency injection

In Figure 7.2, we can see that a generic service expects a dependency with a predetermined interface. It's the responsibility of the injector to retrieve or create an actual concrete instance that implements such an interface and passes it (or “injects it”) into the service. In other words, the injector has the goal of providing an instance that fulfills the dependency for the service.

Trong Hình 7.2, chúng ta có thể thấy rằng một dịch vụ tổng quát mong đợi một phụ thuộc với một giao diện được định trước. Trách nhiệm của injector là lấy hoặc tạo một thể hiện cụ thể thực sự triển khai giao diện đó và truyền nó (hoặc “tiêm nó”) vào dịch vụ. Nói cách khác, injector có mục tiêu cung cấp một thể hiện đáp ứng phụ thuộc cho dịch vụ.

To demonstrate this pattern in practice, let's refactor the simple blogging system that we built in the previous section by using DI to wire its modules. Let's start by refactoring the `blog.js` module:

Để minh họa mẫu này trong thực tế, hãy tái cấu trúc hệ thống blog đơn giản mà chúng ta đã xây dựng trong phần trước bằng cách dùng DI để liên kết các module của nó. Hãy bắt

đầu bằng cách tái cấu trúc module `blog.js`:

```
import { promisify } from 'util'
export class Blog {
  constructor (db) {
    this.db = db
    this.dbRun = promisify(db.run.bind(db))
    this.dbAll = promisify(db.all.bind(db))
  }
  initialize () {
    const initQuery = `CREATE TABLE IF NOT EXISTS posts (
      id TEXT PRIMARY KEY,
      title TEXT NOT NULL,
      content TEXT,
      created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );`
    return this.dbRun(initQuery)
  }
  createPost (id, title, content, createdAt) {
    return this.dbRun('INSERT INTO posts VALUES (?, ?, ?, ?)',
      id, title, content, createdAt)
  }
  getAllPosts () {
    return this.dbAll(
      'SELECT * FROM posts ORDER BY created_at DESC')
  }
}
```

If you compare the new version with the previous one, they are almost identical. There are only two small but important differences:

Nếu bạn so sánh phiên bản mới với phiên bản trước, chúng gần như giống hệt nhau. Chỉ có hai khác biệt nhỏ nhưng quan trọng:

- We are not importing the database module anymore
- The Blog class constructor takes `db` as an argument
 - Chúng ta không còn import module cơ sở dữ liệu nữa
 - Constructor của lớp Blog nhận `db` làm một đối số

The new constructor argument `db` is the expected dependency that needs to be provided at runtime by the client component of the Blog class. This client component is going to be the injector of the dependency. Since JavaScript doesn't have any way to represent abstract interfaces, the provided dependency is expected to implement the `db.run()` and `db.all()` methods. This is called duck typing, as mentioned earlier in this book.

Đối số constructor mới `db` là phụ thuộc được mong đợi cần được cung cấp tại thời điểm chạy bởi thành phần client của lớp Blog. Thành phần client này sẽ là injector của phụ thuộc. Vì JavaScript không có cách nào để biểu diễn các giao diện trừu tượng, phụ thuộc được cung cấp được mong đợi triển khai các phương thức `db.run()` và `db.all()`. Đây được gọi là duck typing, như đã đề cập trước đó trong cuốn sách này.

Let's now rewrite our `db.js` module. The goal here is to get rid of the Singleton pattern and to come up with an implementation that is more reusable and configurable:

Bây giờ hãy viết lại module `db.js` của chúng ta. Mục tiêu ở đây là loại bỏ mẫu Singleton và đưa ra một cách triển khai có thể tái sử dụng và cấu hình được nhiều hơn:

```
import sqlite3 from 'sqlite3'
export function createDb (dbFile) {
  return new sqlite3.Database(dbFile)
}
```

This new implementation of the `db` module provides a factory function called `createDb()`, which allows us to create new instances of the database at runtime. It also allows us to pass the path to the database file at creation time so that we can create independent instances that can write to different files if we have to.

Cách triển khai mới này của module `db` cung cấp một hàm factory tên là `createDb()`, cho phép chúng ta tạo các thể hiện mới của cơ sở dữ liệu tại thời điểm chạy. Nó cũng cho phép chúng ta truyền đường dẫn tới tập tin cơ sở dữ liệu tại thời điểm tạo sao cho chúng ta có thể tạo các thể hiện độc lập có thể ghi vào các tập tin khác nhau nếu cần.

At this point, we have almost all the building blocks in place, we are only missing the injector. We will give an example of the injector by reimplementing the `index.js` module:

Tại điểm này, chúng ta đã có gần như tất cả các khối xây dựng tại chỗ, chúng ta chỉ thiếu injector. Chúng ta sẽ đưa ra một ví dụ về injector bằng cách triển khai lại module `index.js`:

```
import { dirname, join } from 'path'
import { fileURLToPath } from 'url'
import { Blog } from './blog.js'
import { createDb } from './db.js'
const __dirname = dirname(fileURLToPath(import.meta.url))
async function main () {
  const db = createDb(join(__dirname, 'data.sqlite'))
  const blog = new Blog(db)
  await blog.initialize()
  const posts = await blog.getAllPosts()
  if (posts.length === 0) {
    console.log('No post available. Run `node import-posts.js` +
      ' to load some sample posts')
  }
  for (const post of posts) {
    console.log(post.title)
    console.log('-'.repeat(post.title.length))
    console.log(`Published on ${new Date(post.created_at)
      .toISOString()}`)
    console.log(post.content)
  }
}
main().catch(console.error)
```

This code is also quite similar to the previous implementation, except for two important changes (highlighted in the preceding code):

Mã này cũng khá giống với cách triển khai trước, ngoại trừ hai thay đổi quan trọng (được làm nổi bật trong mã trên):

1. We create the database dependency (db) using the factory function `createDb()`.
2. We explicitly “inject” the database instance when we instantiate the `Blog` class.
 1. Chúng ta tạo phụ thuộc cơ sở dữ liệu (db) bằng hàm factory `createDb()`.
 2. Chúng ta “tiêm” thể hiện cơ sở dữ liệu một cách tường minh khi chúng ta khởi tạo lớp `Blog`.

In this implementation of our blogging system, the `blog.js` module is totally decoupled from the actual database implementation, making it more composable and easy to test in isolation.

Trong cách triển khai này của hệ thống blog của chúng ta, module `blog.js` hoàn toàn được tách rời khỏi cách triển khai cơ sở dữ liệu thực sự, khiến nó dễ kết hợp hơn và dễ kiểm thử trong sự cô lập hơn.

We saw how to inject dependencies as constructor arguments (constructor injection), but dependencies can also be passed when invoking a function or method (function injection) or injected explicitly by assigning the relevant properties of an object (property injection).

Chúng ta đã thấy cách tiêm các phụ thuộc làm đối số constructor (constructor injection), nhưng các phụ thuộc cũng có thể được truyền khi gọi một hàm hoặc phương thức (function injection) hoặc được tiêm một cách tường minh bằng cách gán các thuộc tính liên quan của một đối tượng (property injection).

Unfortunately, the advantages in terms of decoupling and reusability offered by the Dependency Injection pattern come with a price to pay. In general, the inability to resolve a dependency at coding time makes it more difficult to understand the relationship between the various components of a system. This is especially true in large applications where we might have a significant amount of services with a complex dependency graph.

Đáng tiếc, những lợi thế về mặt tách rời và khả năng tái sử dụng mà mẫu Dependency Injection mang lại đi kèm với một cái giá phải trả. Nói chung, việc không thể phân giải một phụ thuộc tại thời điểm viết mã khiến việc hiểu mối quan hệ giữa các thành phần khác nhau của một hệ thống trở nên khó khăn hơn. Điều này đặc biệt đúng trong các ứng dụng lớn nơi chúng ta có thể có một lượng đáng kể các dịch vụ với một đồ thị phụ thuộc phức tạp.

Also, if we look at the way we instantiated our database dependency in our preceding example script, we can see that we had to make sure that the database instance was created before we could invoke any function from our `Blog` instance. This means that, when used in its raw form, Dependency Injection forces us to build the dependency graph of the entire application by hand, making sure that we do it in the right order. This can become unmanageable when the number of modules to wire becomes too high.

Ngoài ra, nếu chúng ta nhìn vào cách chúng ta khởi tạo phụ thuộc cơ sở dữ liệu trong script ví dụ trên, chúng ta có thể thấy rằng chúng ta phải đảm bảo thể hiện cơ sở dữ liệu được tạo trước khi chúng ta có thể gọi bất kỳ hàm nào từ thể hiện `Blog` của chúng ta. Điều này có nghĩa là, khi được dùng ở dạng thô của nó, Dependency Injection buộc chúng ta phải xây dựng đồ thị phụ thuộc của toàn bộ ứng dụng bằng tay, đảm bảo rằng chúng ta làm điều đó theo đúng thứ tự. Điều này có thể trở nên không thể quản lý khi số lượng module cần liên kết trở nên quá cao.

Another pattern, called Inversion of Control, allows us to shift the responsibility of wiring the modules of an application to a third-party entity. This entity can be a service locator (a simple component used to retrieve a dependency, for example, `serviceLocator.get('db')`) or a dependency injection container (a system that injects the dependencies into a component based

on some metadata specified in the code itself or in a configuration file). You can find more about these components on Martin Fowler's blog at nodejsdp.link/ioc-containers. Even though these techniques derail a bit from the Node.js way of doing things, some of them have recently gained some popularity. Check out [inversify](https://nodejsdp.link/inversify) (nodejsdp.link/inversify) and [awilix](https://nodejsdp.link/awilix) (nodejsdp.link/awilix) to find out more.

Một mẫu khác, gọi là Inversion of Control, cho phép chúng ta chuyển trách nhiệm liên kết các module của một ứng dụng cho một thực thể bên thứ ba. Thực thể này có thể là một service locator (một thành phần đơn giản được dùng để lấy một phụ thuộc, ví dụ, `serviceLocator.get('db')`) hoặc một dependency injection container (một hệ thống tiêm các phụ thuộc vào một thành phần dựa trên một số metadata được chỉ định trong chính mã hoặc trong một tập tin cấu hình). Bạn có thể tìm hiểu thêm về các thành phần này trên blog của Martin Fowler tại nodejsdp.link/ioc-containers. Mặc dù những kỹ thuật này hơi lệch khỏi cách làm việc của Node.js, một số trong chúng gần đây đã trở nên phổ biến. Hãy xem [inversify](https://nodejsdp.link/inversify) (nodejsdp.link/inversify) và [awilix](https://nodejsdp.link/awilix) (nodejsdp.link/awilix) để tìm hiểu thêm.

Summary — Tóm tắt

In this chapter, you were gently introduced to a set of traditional design patterns concerning the creation of objects. Some of those patterns are so basic, and yet essential at the same time, that you have probably already used them in one way or another.

Trong chương này, bạn đã được giới thiệu một cách nhẹ nhàng về một tập hợp các mẫu thiết kế truyền thống liên quan đến việc tạo các đối tượng. Một số mẫu đó cơ bản đến vậy, nhưng đồng thời lại thiết yếu, đến mức bạn có lẽ đã dùng chúng theo cách này hay cách khác.

Patterns such as Factory and Singleton are, for example, two of the most ubiquitous in object-oriented programming in general. However, in JavaScript, their implementation and significance are very different from what was thought up by the Gang of Four book. For example, Factory becomes a very versatile pattern that works in perfect harmony with the hybrid nature of the JavaScript language, that is, half object-oriented and half functional. On the other hand, Singleton becomes so trivial to implement that it's almost a non-pattern, but it carries a set of caveats that you should have learned to take into account.

Các mẫu như Factory và Singleton, ví dụ, là hai trong số những mẫu phổ biến nhất trong lập trình hướng đối tượng nói chung. Tuy nhiên, trong JavaScript, cách triển khai và ý nghĩa của chúng rất khác so với những gì được nghĩ ra bởi cuốn sách Gang of Four. Ví dụ, Factory trở thành một mẫu rất linh hoạt hoạt động hài hòa hoàn hảo với bản chất lai của ngôn ngữ JavaScript, tức là nửa hướng đối tượng và nửa hàm. Mặt khác, Singleton trở nên tầm thường để triển khai đến mức nó gần như là một non-pattern, nhưng nó mang theo một tập hợp các lưu ý mà bạn nên đã học để tính đến.

Among the patterns you've learned in this chapter, the Builder pattern may seem the one that has retained most of its traditional object-oriented form. However, we've shown you that it can also be used to invoke complex functions and not just to build objects.

Trong số các mẫu bạn đã học trong chương này, mẫu Builder có thể có vẻ là cái đã giữ lại nhiều nhất hình thức hướng đối tượng truyền thống của nó. Tuy nhiên, chúng tôi đã cho bạn thấy rằng nó cũng có thể được dùng để gọi các hàm phức tạp chứ không chỉ để xây dựng các đối tượng.

The Revealing Constructor pattern, on the other hand, deserves a category of its own. Born from necessities arising from the JavaScript language itself, it provides an elegant solution to the problem of “revealing” certain private object properties at construction time only. It provides strong guarantees in a language that is relaxed by nature.

Mặt khác, mẫu Revealing Constructor xứng đáng có một loại riêng của nó. Sinh ra từ những nhu cầu phát sinh từ chính ngôn ngữ JavaScript, nó cung cấp một giải pháp thanh lịch cho vấn đề “phơi bày” một số thuộc tính riêng tư của đối tượng chỉ tại thời điểm xây dựng. Nó cung cấp những đảm bảo mạnh mẽ trong một ngôn ngữ vốn lỏng lẻo theo bản chất.

Finally, you learned about the two main techniques for wiring components together: Singleton and Dependency Injection. We’ve seen how the first is the simplest and most practical approach, while the second is more powerful but also potentially more complex to implement.

Cuối cùng, bạn đã học về hai kỹ thuật chính để liên kết các thành phần với nhau: Singleton và Dependency Injection. Chúng ta đã thấy cách cái đầu tiên là cách tiếp cận đơn giản nhất và thiết thực nhất, trong khi cái thứ hai mạnh mẽ hơn nhưng cũng có khả năng phức tạp hơn để triển khai.

As we already mentioned, this was just the first of a series of three chapters entirely dedicated to traditional design patterns. In these chapters, we will try to teach the right balance between creativity and rigor. We want to show not only that there are patterns that can be reused to improve our code, but also that their implementation is not the most important detail; in fact, it can vary a lot, or even overlap with other patterns. What really matters, however, is the blueprint, the guidelines, and the idea at the base of each pattern. This is the real reusable piece of information that we can exploit to design better Node.js applications in a fun way.

Như chúng ta đã đề cập, đây chỉ là chương đầu tiên trong một loạt ba chương hoàn toàn dành cho các mẫu thiết kế truyền thống. Trong những chương này, chúng ta sẽ cố gắng dạy sự cân bằng đúng đắn giữa sáng tạo và sự nghiêm ngặt. Chúng tôi muốn cho thấy không chỉ rằng có những mẫu có thể được tái sử dụng để cải thiện mã của chúng ta, mà còn rằng cách triển khai của chúng không phải là chi tiết quan trọng nhất; thực tế, nó có thể thay đổi rất nhiều, hoặc thậm chí chồng lấn với các mẫu khác. Tuy nhiên, điều thực sự quan trọng là bản thiết kế, các hướng dẫn, và ý tưởng nằm ở nền tảng của mỗi mẫu. Đây là phần thông tin thực sự có thể tái sử dụng mà chúng ta có thể khai thác để thiết kế các ứng dụng Node.js tốt hơn một cách vui vẻ.

In the next chapter, you will learn about another category of traditional design patterns, called structural patterns. As the name suggests, these patterns are aimed at improving the way we combine objects together to build more complex, yet flexible and reusable structures.

Trong chương tiếp theo, bạn sẽ học về một loại mẫu thiết kế truyền thống khác, gọi là các mẫu cấu trúc (structural patterns). Như tên gọi gợi ý, những mẫu này nhằm cải thiện cách chúng ta kết hợp các đối tượng với nhau để xây dựng các cấu trúc phức tạp hơn, nhưng linh hoạt và có thể tái sử dụng.

Exercises — Bài tập

- 7.1 Console color factory: Create a class called `ColorConsole` that has just one empty method called `log()`. Then, create three subclasses: `RedConsole`, `BlueConsole`, and `GreenConsole`. The `log()` method of every `ColorConsole` subclass will accept a string as input and will print that string to the console using the color that gives the name to the class. Then, create a factory function that takes color as input, such as `'red'`, and returns the related `ColorConsole`

subclass. Finally, write a small command-line script to try the new console color factory. You can use this Stack Overflow answer as a reference for using colors in the console: nodejsdp.link/console-colors.

- 7.2 Request builder: Create your own Builder class around the built-in `http.request()` function. The builder must be able to provide at least basic facilities to specify the HTTP method, the URL, the query component of the URL, the header parameters, and the eventual body data to be sent. To send the request, provide an `invoke()` method that returns a Promise for the invocation. You can find the docs for `http.request()` at the following URL: nodejsdp.link/docs-http-request.
- 7.3 A tamper-free queue: Create a Queue class that has only one publicly accessible method called `dequeue()`. Such a method returns a Promise that resolves with a new element extracted from an internal queue data structure. If the queue is empty, then the Promise will resolve when a new item is added. The Queue class must also have a revealing constructor that provides a function called `enqueue()` to the executor that pushes a new element to the end of the internal queue. The `enqueue()` function can be invoked asynchronously and it must also take care of “unblocking” any eventual Promise returned by the `dequeue()` method. To try out the Queue class, you could build a small HTTP server into the executor function. Such a server would receive messages or tasks from a client and would push them into the queue. A loop would then consume all those messages using the `dequeue()` method.
 - 7.1 Console color factory: Tạo một lớp tên là `ColorConsole` chỉ có một phương thức rỗng tên là `log()`. Sau đó, tạo ba lớp con: `RedConsole`, `BlueConsole` và `GreenConsole`. Phương thức `log()` của mỗi lớp con `ColorConsole` sẽ nhận một chuỗi làm đầu vào và sẽ in chuỗi đó ra console bằng màu đặt tên cho lớp đó. Sau đó, tạo một hàm factory nhận màu làm đầu vào, chẳng hạn như `'red'`, và trả về lớp con `ColorConsole` liên quan. Cuối cùng, viết một script dòng lệnh nhỏ để thử factory console màu mới. Bạn có thể dùng câu trả lời Stack Overflow này làm tài liệu tham khảo cho việc dùng màu trong console: nodejsdp.link/console-colors.
 - 7.2 Request builder: Tạo lớp Builder của riêng bạn xung quanh hàm tích hợp sẵn `http.request()`. Builder phải có khả năng cung cấp ít nhất các tiện ích cơ bản để chỉ định phương thức HTTP, URL, thành phần query của URL, các tham số header, và dữ liệu body cuối cùng cần được gửi. Để gửi request, hãy cung cấp một phương thức `invoke()` trả về một Promise cho lời gọi. Bạn có thể tìm thấy tài liệu cho `http.request()` tại URL sau: nodejsdp.link/docs-http-request.
 - 7.3 A tamper-free queue: Tạo một lớp Queue chỉ có một phương thức công khai có thể truy cập được, tên là `dequeue()`. Phương thức như vậy trả về một Promise được hoàn thành (resolve) với một phần tử mới được trích từ một cấu trúc dữ liệu queue nội bộ. Nếu hàng đợi rỗng, thì Promise sẽ hoàn thành khi một mục mới được thêm vào. Lớp Queue cũng phải có một revealing constructor cung cấp một hàm tên là `enqueue()` cho executor để đẩy một phần tử mới vào cuối hàng đợi nội bộ. Hàm `enqueue()` có thể được gọi bất đồng bộ và nó cũng phải lo việc “bỏ chặn” bất kỳ Promise cuối cùng nào được trả về bởi phương thức `dequeue()`. Để thử nghiệm lớp Queue, bạn có thể xây dựng một máy chủ HTTP nhỏ vào trong hàm executor. Một máy chủ như vậy sẽ nhận các thông điệp hoặc tác vụ từ một client và sẽ đẩy chúng vào hàng đợi. Một vòng lặp sau đó sẽ tiêu thụ tất cả các thông điệp đó bằng phương thức `dequeue()`.

Chapter 8: Structural Design Patterns — Chương 8: Các mẫu thiết kế cấu trúc

In this chapter, we will explore some of the most popular structural design patterns and discover how they apply to Node.js. Structural design patterns are focused on providing ways to realize relationships between entities.

Trong chương này, chúng ta sẽ khám phá một số mẫu thiết kế cấu trúc (structural design pattern) phổ biến nhất và tìm hiểu cách chúng được áp dụng vào Node.js. Các mẫu thiết kế cấu trúc tập trung vào việc cung cấp những cách để hiện thực hóa các mối quan hệ giữa các thực thể.

In particular, in this chapter, we will examine the following patterns:

Cụ thể, trong chương này, chúng ta sẽ xem xét những mẫu sau:

- **Proxy:** A pattern that allows us to control access to another object
- **Decorator:** A common pattern to augment the behavior of an existing object dynamically
- **Adapter:** A pattern that allows us to access the functionality of an object using a different interface
 - Proxy: Một mẫu cho phép chúng ta kiểm soát quyền truy cập tới một đối tượng khác
 - Decorator: Một mẫu phổ biến để bổ sung hành vi cho một đối tượng hiện có một cách động (dynamic)
 - Adapter: Một mẫu cho phép chúng ta truy cập chức năng của một đối tượng thông qua một giao diện khác

Throughout the chapter, we will also explore some interesting concepts such as reactive programming (RP), and we will also spend some time playing with LevelDB, a database technology that is commonly adopted in the Node.js ecosystem.

Xuyên suốt chương này, chúng ta cũng sẽ khám phá một số khái niệm thú vị như lập trình phản ứng (reactive programming - RP), và chúng ta cũng sẽ dành chút thời gian nghịch với LevelDB, một công nghệ cơ sở dữ liệu thường được áp dụng trong hệ sinh thái Node.js.

By the end of this chapter, you will be familiar with several scenarios in which structural design patterns can be useful and you will be able to implement them effectively in your Node.js applications.

Đến cuối chương này, bạn sẽ quen thuộc với một số tình huống mà các mẫu thiết kế cấu trúc có thể hữu ích, và bạn sẽ có khả năng hiện thực chúng một cách hiệu quả trong các ứng dụng Node.js của mình.

Proxy — Proxy

A proxy is an object that controls access to another object, called the subject. The proxy and the subject have an identical interface, and this allows us to swap one for the other transparently; in fact, the alternative name for this pattern is surrogate. A proxy intercepts all or some of the operations that are meant to be executed on the subject, augmenting or complementing their behavior. Figure 8.1 shows a schematic representation of this pattern:

Proxy là một đối tượng kiểm soát quyền truy cập tới một đối tượng khác, gọi là subject. Proxy và subject có giao diện giống hệt nhau, và điều này cho phép chúng ta hoán đổi cái này cho cái kia một cách trong suốt (transparent); thực tế, một tên gọi khác cho mẫu này là surrogate. Một proxy chặn tất cả hoặc một số thao tác dự định được thực thi trên subject, bổ sung hoặc làm hoàn thiện thêm hành vi của chúng. Hình 8.1 biểu diễn sơ đồ của mẫu này:

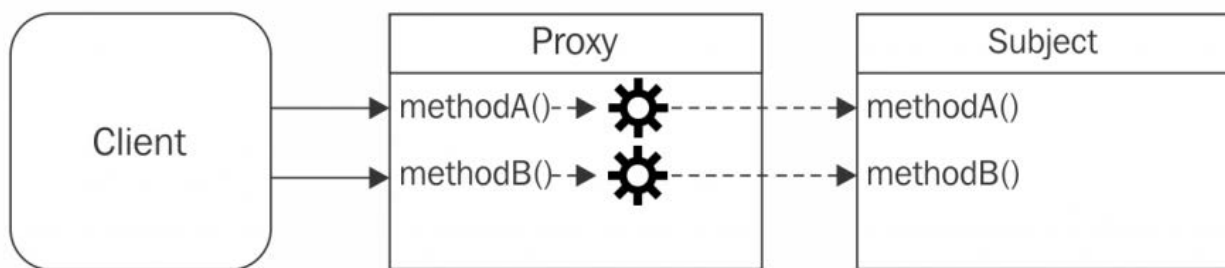


Figure 8.1: Proxy pattern schematic

Hình 8.1: Sơ đồ mẫu Proxy

Figure 8.1 shows us how the proxy and the subject have the same interface, and how this is transparent to the client, who can use one or the other interchangeably. The proxy forwards each operation to the subject, enhancing its behavior with additional preprocessing or postprocessing.

Hình 8.1 cho chúng ta thấy proxy và subject có cùng giao diện như thế nào, và điều này trong suốt với client ra sao, khi client có thể dùng cái này hoặc cái kia một cách thay thế cho nhau. Proxy chuyển tiếp mỗi thao tác tới subject, làm phong phú thêm hành vi của nó bằng các bước tiền xử lý hoặc hậu xử lý bổ sung.

It's important to observe that we are not talking about proxying between classes; the Proxy pattern involves wrapping an actual instance of the subject, thus preserving its internal **state**.

Một điều quan trọng cần lưu ý là chúng ta không nói về việc proxy giữa các class; mẫu Proxy liên quan đến việc bao bọc (wrap) một thể hiện (instance) thực sự của subject, do đó bảo toàn trạng thái nội tại của nó.

A proxy can be useful in several circumstances, for example:

Một proxy có thể hữu ích trong nhiều tình huống, ví dụ:

- Data validation: The proxy validates the input before forwarding it to the subject
- Security: The proxy verifies that the client is authorized to perform the operation, and it passes the **request** to the subject only if the outcome of the check is positive
- Caching: The proxy keeps an internal cache so that the proxied operations are executed on the subject only if the data is not yet present in the cache
- Lazy initialization: If creating the subject is expensive, the proxy can delay it until it's really necessary

- Logging: The proxy intercepts the method invocations and the relative parameters, recoding them as they happen
- Remote objects: The proxy can take a remote object and make it appear local
 - Kiểm tra dữ liệu (data validation): Proxy kiểm tra dữ liệu đầu vào trước khi chuyển tiếp tới subject
 - Bảo mật (security): Proxy xác minh rằng client được phép thực hiện thao tác, và chỉ chuyển yêu cầu tới subject nếu kết quả kiểm tra là tích cực
 - Lưu đệm (caching): Proxy giữ một cache nội bộ sao cho các thao tác được proxy chỉ được thực thi trên subject nếu dữ liệu chưa có sẵn trong cache
 - Khởi tạo lười (lazy initialization): Nếu việc tạo subject tốn kém, proxy có thể trì hoãn nó tới khi thực sự cần thiết
 - Ghi log (logging): Proxy chặn các lời gọi phương thức cùng các tham số liên quan, ghi lại chúng khi chúng xảy ra
 - Đối tượng từ xa (remote objects): Proxy có thể lấy một đối tượng ở xa và làm cho nó trông như cục bộ

There are more Proxy pattern applications, but these should give us an idea of its purpose.

Còn nhiều ứng dụng khác của mẫu Proxy, nhưng những ví dụ này đủ để cho chúng ta hình dung về mục đích của nó.

Techniques for implementing proxies — Các kỹ thuật hiện thực proxy

When proxying an object, we can decide to intercept all of its methods or only some of them, while delegating the rest directly to the subject. There are several ways in which this can be achieved, and in this section, we will present some of them.

Khi proxy một đối tượng, chúng ta có thể quyết định chặn tất cả các phương thức của nó hoặc chỉ một số, đồng thời ủy thác phần còn lại trực tiếp cho subject. Có vài cách để đạt được điều này, và trong phần này, chúng ta sẽ trình bày một số trong đó.

We will be working on a simple example, a StackCalculator class that looks like this:

Chúng ta sẽ làm việc với một ví dụ đơn giản, một class StackCalculator trông như sau:

```
class StackCalculator {
  constructor () {
    this.stack = []
  }
  putValue (value) {
    this.stack.push(value)
  }
  getValue () {
    return this.stack.pop()
  }
  peekValue () {
    return this.stack[this.stack.length - 1]
  }
  clear () {
    this.stack = []
  }
  divide () {
    const divisor = this.getValue()
```

```

    const dividend = this.getValue()
    const result = dividend / divisor
    this.putValue(result)
    return result
  }
  multiply () {
    const multiplicand = this.getValue()
    const multiplier = this.getValue()
    const result = multiplier * multiplicand
    this.putValue(result)
    return result
  }
}

```

This class implements a simplified version of a stack calculator. The idea of this calculator is to keep all operands (values) in a stack. When you perform an operation, for example a multiplication, the multiplicand and the multiplier are extracted from the stack and the result of the multiplication is pushed back into the stack. This is not too different from how the calculator application on your mobile phone is actually implemented.

Class này hiện thực một phiên bản đơn giản hóa của một máy tính dạng ngăn xếp (stack calculator). Ý tưởng của máy tính này là giữ tất cả các toán hạng (giá trị) trong một ngăn xếp. Khi bạn thực hiện một thao tác, ví dụ một phép nhân, số bị nhân (multiplicand) và số nhân (multiplier) được lấy ra khỏi ngăn xếp và kết quả của phép nhân được đẩy trở lại vào ngăn xếp. Điều này không khác mấy so với cách ứng dụng máy tính trên điện thoại di động của bạn thực sự được hiện thực.

Here's an example of how we might use StackCalculator to perform some multiplications and divisions:

Đây là một ví dụ về cách chúng ta có thể dùng StackCalculator để thực hiện vài phép nhân và phép chia:

```

const calculator = new StackCalculator()
calculator.putValue(3)
calculator.putValue(2)
console.log(calculator.multiply()) // 3*2 = 6
calculator.putValue(2)
console.log(calculator.multiply()) // 6*2 = 12

```

There are also some utility methods such as `peekValue()`, which allows us to peek the value at the top of the stack (the last value inserted or the result of the last operation), and `clear()`, which allows us to reset the stack.

Cũng có một số phương thức tiện ích như `peekValue()`, cho phép chúng ta xem giá trị ở đỉnh ngăn xếp (giá trị được chèn cuối cùng hoặc kết quả của thao tác cuối cùng), và `clear()`, cho phép chúng ta đặt lại ngăn xếp.

Fun fact: In JavaScript, when you perform a division by 0, you get back a mysterious value called Infinity. In many other programming languages dividing by 0 is an illegal operation that results in the program panicking or throwing a **runtime** exception.

Sự thật thú vị: Trong JavaScript, khi bạn thực hiện phép chia cho 0, bạn nhận lại một giá trị bí ẩn gọi là Infinity. Trong nhiều ngôn ngữ lập trình khác, chia cho 0 là một thao tác không hợp lệ dẫn đến chương trình panic hoặc ném ra một ngoại lệ runtime.

Our task in the next few sections will be to leverage the Proxy pattern to enhance a `StackCalculator` instance by providing a more conservative behavior for division by 0: rather than returning `Infinity`, we will throw an explicit error.

Nhiệm vụ của chúng ta trong vài phần tiếp theo sẽ là tận dụng mẫu Proxy để nâng cao một thể hiện `StackCalculator` bằng cách cung cấp một hành vi thận trọng hơn cho phép chia cho 0: thay vì trả về `Infinity`, chúng ta sẽ ném ra một lỗi tường minh.

Object composition — Object composition

Composition is a technique whereby an object is combined with another object for the purpose of extending or using its functionality. In the specific case of the Proxy pattern, a new object with the same interface as the subject is created, and a reference to the subject is stored internally in the proxy in the form of an instance variable or a **closure** variable. The subject can be injected from the client at creation time or created by the proxy itself. The following example implements a safe calculator using object composition:

Composition là một kỹ thuật trong đó một đối tượng được kết hợp với một đối tượng khác nhằm mục đích mở rộng hoặc sử dụng chức năng của nó. Trong trường hợp cụ thể của mẫu Proxy, một đối tượng mới có cùng giao diện với subject được tạo ra, và một tham chiếu tới subject được lưu trữ nội bộ trong proxy dưới dạng một biến thể hiện (instance variable) hoặc một biến closure. Subject có thể được tiêm (inject) từ client tại thời điểm tạo hoặc được tạo bởi chính proxy. Ví dụ sau hiện thực một máy tính an toàn bằng cách dùng object composition:

```
class SafeCalculator {
  constructor (calculator) {
    this.calculator = calculator
  }
  // proxied method
  divide () {
    // additional validation logic
    const divisor = this.calculator.peekValue()
    if (divisor === 0) {
      throw Error('Division by 0')
    }
    // if valid delegates to the subject
    return this.calculator.divide()
  }
  // delegated methods
  putValue (value) {
    return this.calculator.putValue(value)
  }
  getValue () {
    return this.calculator.getValue()
  }
  peekValue () {
    return this.calculator.peekValue()
  }
  clear () {
    return this.calculator.clear()
  }
}
```

```

    multiply () {
        return this.calculator.multiply()
    }
}
const calculator = new StackCalculator()
const safeCalculator = new SafeCalculator(calculator)
calculator.putValue(3)
calculator.putValue(2)
console.log(calculator.multiply()) // 3*2 = 6
safeCalculator.putValue(2)
console.log(safeCalculator.multiply()) // 6*2 = 12
calculator.putValue(0)
console.log(calculator.divide()) // 12/0 = Infinity
safeCalculator.clear()
safeCalculator.putValue(4)
safeCalculator.putValue(0)
console.log(safeCalculator.divide()) // 4/0 -> Error

```

The `safeCalculator` object is a proxy for the original `calculator` instance. By invoking `multiply()` on `safeCalculator`, we will end up calling the same method on `calculator`. The same goes for `divide()`, but in this case we can see that, if we try to divide by zero, we will get different outcomes depending on whether we perform the division on the subject or on the proxy.

Đối tượng `safeCalculator` là một proxy cho thể hiện `calculator` gốc. Bằng cách gọi `multiply()` trên `safeCalculator`, cuối cùng chúng ta sẽ gọi cùng phương thức đó trên `calculator`. Điều tương tự cũng đúng với `divide()`, nhưng trong trường hợp này chúng ta có thể thấy rằng, nếu cố chia cho không, chúng ta sẽ nhận được các kết quả khác nhau tùy thuộc vào việc chúng ta thực hiện phép chia trên subject hay trên proxy.

To implement this proxy using composition, we had to intercept the methods that we were interested in manipulating (`divide()`), while simply delegating the rest of them to the subject (`putValue()`, `getValue()`, `peekValue()`, `clear()`, and `multiply()`).

Để hiện thực proxy này bằng composition, chúng ta phải chặn các phương thức mà chúng ta quan tâm thao tác (`divide()`), trong khi đơn thuần ủy thác phần còn lại cho subject (`putValue()`, `getValue()`, `peekValue()`, `clear()`, và `multiply()`).

Note that the `calculator` state (the values in the stack) is still maintained by the `calculator` instance; `safeCalculator` will only invoke methods on `calculator` to read or mutate the state as needed.

Lưu ý rằng trạng thái của `calculator` (các giá trị trong ngăn xếp) vẫn được duy trì bởi thể hiện `calculator`; `safeCalculator` chỉ gọi các phương thức trên `calculator` để đọc hoặc thay đổi trạng thái khi cần.

An alternative implementation of the proxy presented in the preceding code fragment might just use an object literal and a **factory** function:

Một cách hiện thực thay thế cho proxy được trình bày trong đoạn mã trước có thể chỉ dùng một object literal và một hàm factory:

```

function createSafeCalculator (calculator) {
    return {
        // proxied method
        divide () {
            // additional validation logic

```



```

    const divisor = calculator.peekValue()
    if (divisor === 0) {
        throw Error('Division by 0')
    }
    // if valid delegates to the subject
    return calculator.divide()
},
// delegated methods
putValue (value) {
    return calculator.putValue(value)
},
getValue () {
    return calculator.getValue()
},
peekValue () {
    return calculator.peekValue()
},
clear () {
    return calculator.clear()
},
multiply () {
    return calculator.multiply()
}
}
}
const calculator = new StackCalculator()
const safeCalculator = createSafeCalculator(calculator)
// ...

```

This implementation is simpler and more concise than the class-based one, but, once again, it forces us to delegate all the methods to the subject explicitly.

Cách hiện thực này đơn giản và súc tích hơn so với cách dựa trên class, nhưng, một lần nữa, nó buộc chúng ta phải ủy thác tất cả các phương thức cho subject một cách tường minh.

Having to delegate many methods for complex classes can be very tedious and might make it harder to implement these techniques. One way to create a proxy that delegates most of its methods is to use a library that generates all the methods for us, such as `delegates` ([nodejsdp.link/delegates](https://www.npmjs.com/package/delegates)). A more modern and native alternative is to use the Proxy object, which we will discuss later in this chapter.

Việc phải ủy thác nhiều phương thức cho các class phức tạp có thể rất tẻ nhạt và có thể khiến việc hiện thực những kỹ thuật này khó khăn hơn. Một cách để tạo một proxy ủy thác hầu hết các phương thức của nó là dùng một thư viện sinh ra tất cả các phương thức cho chúng ta, chẳng hạn như `delegates` ([nodejsdp.link/delegates](https://www.npmjs.com/package/delegates)). Một lựa chọn thay thế hiện đại hơn và bản địa (native) là dùng đối tượng Proxy, mà chúng ta sẽ bàn tới sau trong chương này.

Object augmentation — Object augmentation

Object augmentation (or monkey patching) is probably the simplest and most common way of proxying just a few methods of an object. It involves modifying the subject directly by replacing a method with its proxied implementation. In the context of our calculator example, this could be done as follows:

Object augmentation (hay monkey patching) có lẽ là cách đơn giản nhất và phổ biến nhất để proxy chỉ một vài phương thức của một đối tượng. Nó liên quan đến việc sửa đổi subject trực tiếp bằng cách thay thế một phương thức bằng hiện thực được proxy của nó. Trong ngữ cảnh ví dụ máy tính của chúng ta, điều này có thể được thực hiện như sau:

```
function patchToSafeCalculator (calculator) {
  const divideOrig = calculator.divide
  calculator.divide = () => {
    // additional validation logic
    const divisor = calculator.peekValue()
    if (divisor === 0) {
      throw Error('Division by 0')
    }
    // if valid delegates to the subject
    return divideOrig.apply(calculator)
  }
  return calculator
}
const calculator = new StackCalculator()
const safeCalculator = patchToSafeCalculator(calculator)
// ...
```

This technique is definitely convenient when we need to proxy only one or a few methods. Did you notice that we didn't have to reimplement the `multiply()` method and all the other delegated methods here?

Kỹ thuật này chắc chắn tiện lợi khi chúng ta chỉ cần proxy một hoặc vài phương thức. Bạn có để ý rằng ở đây chúng ta không phải hiện thực lại phương thức `multiply()` và tất cả các phương thức được ủy thác khác không?

Unfortunately, simplicity comes at the cost of having to mutate the subject object directly, which can be dangerous.

Đáng tiếc, sự đơn giản đi kèm với cái giá là phải biến đổi (mutate) trực tiếp đối tượng subject, điều này có thể nguy hiểm.

Mutations should be avoided at all costs when the subject is shared with other parts of the codebase. In fact, “monkey patching” the subject might create undesirable side effects that affect other components of our application. Use this technique only when the subject exists in a controlled context or in a private **scope**. If you want to appreciate why “monkey patching” is a dangerous practice, you could try to invoke a division by zero in the original calculator instance. If you do so, you will see that the original instance will now throw an error rather than returning Infinity. The original behavior has been altered, and this might have unexpected effects on other parts of the application.

Nên tránh các thay đổi (mutation) bằng mọi giá khi subject được chia sẻ với các phần khác của mã nguồn. Thực tế, “monkey patching” subject có thể tạo ra các tác dụng phụ không mong muốn ảnh hưởng đến các thành phần khác của ứng dụng. Chỉ dùng kỹ thuật này

khi subject tồn tại trong một ngữ cảnh được kiểm soát hoặc trong một phạm vi riêng tư. Nếu bạn muốn thấy rõ tại sao “monkey patching” là một thực hành nguy hiểm, bạn có thể thử gọi một phép chia cho không trên thể hiện calculator gốc. Nếu làm vậy, bạn sẽ thấy rằng thể hiện gốc giờ đây sẽ ném ra một lỗi thay vì trả về Infinity. Hành vi gốc đã bị thay đổi, và điều này có thể gây ra các tác động bất ngờ lên các phần khác của ứng dụng.

In the next section, we will explore the built-in Proxy object, which is a powerful alternative for implementing the Proxy pattern and more.

Trong phần tiếp theo, chúng ta sẽ khám phá đối tượng Proxy có sẵn (built-in), một lựa chọn thay thế mạnh mẽ để hiện thực mẫu Proxy và hơn thế nữa.

The built-in Proxy object — Đối tượng Proxy có sẵn

The ES2015 specification introduced a native way to create powerful proxy objects.

Đặc tả ES2015 đã giới thiệu một cách bản địa để tạo các đối tượng proxy mạnh mẽ.

We are talking about the ES2015 Proxy object, which consists of a Proxy constructor that accepts a target and a handler as arguments:

Chúng ta đang nói về đối tượng Proxy của ES2015, gồm một hàm khởi tạo (constructor) Proxy nhận một target và một handler làm tham số:

```
const proxy = new Proxy(target, handler)
```

Here, target represents the object on which the proxy is applied (the subject for our canonical definition), while handler is a special object that defines the behavior of the proxy.

Ở đây, target đại diện cho đối tượng mà proxy được áp dụng lên (subject theo định nghĩa quy ước của chúng ta), trong khi handler là một đối tượng đặc biệt định nghĩa hành vi của proxy.

The handler object contains a series of optional methods with predefined names called trap methods (for example, apply, get, set, and has) that are automatically called when the corresponding operations are performed on the proxy instance.

Đối tượng handler chứa một loạt các phương thức tùy chọn có tên định trước gọi là các phương thức bẫy (trap method) (ví dụ, apply, get, set, và has) được gọi tự động khi các thao tác tương ứng được thực hiện trên thể hiện proxy.

To better understand how this **API** works, let’s see how we can use the Proxy object to implement our safe calculator proxy:

Để hiểu rõ hơn cách API này hoạt động, hãy xem cách chúng ta có thể dùng đối tượng Proxy để hiện thực proxy máy tính an toàn của mình:

```
const safeCalculatorHandler = {
  get: (target, property) => {
    if (property === 'divide') {
      // proxied method
      return function () {
        // additional validation logic
        const divisor = target.peekValue()
        if (divisor === 0) {
          throw Error('Division by 0')
        }
      }
    }
  }
}
```

```

        // if valid delegates to the subject
        return target.divide()
    }
}
// delegated methods and properties
return target[property]
}
}
const calculator = new StackCalculator()
const safeCalculator = new Proxy(
    calculator,
    safeCalculatorHandler
)
// ...

```

In this implementation of the safe calculator proxy using the Proxy object, we adopted the get trap to intercept access to properties and methods of the original object, including calls to the `divide()` method. When access to `divide()` is intercepted, the proxy returns a modified version of the function that implements the additional logic to check for possible divisions by zero. Note that we can simply return all other methods and properties unchanged by using `target[property]`.

Trong cách hiện thực proxy máy tính an toàn này bằng đối tượng Proxy, chúng ta đã áp dụng bẫy get để chặn việc truy cập tới các thuộc tính và phương thức của đối tượng gốc, bao gồm các lời gọi tới phương thức `divide()`. Khi việc truy cập tới `divide()` bị chặn, proxy trả về một phiên bản đã được sửa đổi của hàm, hiện thực thêm logic kiểm tra các phép chia cho không có thể xảy ra. Lưu ý rằng chúng ta có thể đơn giản trả về tất cả các phương thức và thuộc tính khác mà không thay đổi bằng cách dùng `target[property]`.

Finally, it is important to mention that the Proxy object inherits the prototype of the subject, therefore running `safeCalculator instanceof StackCalculator` will return `true`.

Cuối cùng, một điều quan trọng cần đề cập là đối tượng Proxy kế thừa prototype của subject, do đó chạy `safeCalculator instanceof StackCalculator` sẽ trả về `true`.

With this example, it should be clear that the Proxy object allows us to avoid mutating the subject while giving us an easy way to proxy only the bits that we need to enhance, without having to explicitly delegate all the other properties and methods.

Với ví dụ này, hẳn đã rõ rằng đối tượng Proxy cho phép chúng ta tránh biến đổi subject trong khi cung cấp cho chúng ta một cách dễ dàng để proxy chỉ những phần mà chúng ta cần nâng cao, mà không phải ủy thác tường minh tất cả các thuộc tính và phương thức khác.

Additional capabilities and limitations of the Proxy object — Các khả năng bổ sung và hạn chế của đối tượng Proxy The Proxy object is a feature deeply integrated into the JavaScript language itself, which enables developers to intercept and customize many operations that can be performed on objects. This characteristic opens up new and interesting scenarios that were not easily achievable before, such as meta-programming, operator overloading, and object virtualization.

Đối tượng Proxy là một tính năng được tích hợp sâu vào chính ngôn ngữ JavaScript, cho phép các nhà phát triển chặn và tùy biến nhiều thao tác có thể được thực hiện trên các đối tượng. Đặc tính này mở ra những kịch bản mới mẻ và thú vị mà trước đây không dễ đạt được, chẳng hạn như siêu lập trình (meta-programming), nạp chồng toán tử (operator overloading), và ảo hóa đối tượng (object virtualization).

Let's see another example to clarify this concept:

Hãy xem một ví dụ khác để làm rõ khái niệm này:

```
const evenNumbers = new Proxy([], {
  get: (target, index) => index * 2,
  has: (target, number) => number % 2 === 0
})
console.log(2 in evenNumbers) // true
console.log(5 in evenNumbers) // false
console.log(evenNumbers[7])   // 14
```

In this example, we are creating a virtual array that contains all even numbers. It can be used as a regular array, which means we can access items in the array with the regular array syntax (for example, `evenNumbers[7]`), or check the existence of an element in the array with the `in` operator (for example, `2 in evenNumbers`). The array is considered virtual because we never store data in it.

Trong ví dụ này, chúng ta đang tạo một mảng ảo chứa tất cả các số chẵn. Nó có thể được dùng như một mảng thông thường, nghĩa là chúng ta có thể truy cập các phần tử trong mảng bằng cú pháp mảng thông thường (ví dụ, `evenNumbers[7]`), hoặc kiểm tra sự tồn tại của một phần tử trong mảng bằng toán tử `in` (ví dụ, `2 in evenNumbers`). Mảng này được coi là ảo vì chúng ta không bao giờ lưu trữ dữ liệu trong nó.

It is very important to note that, while the previous code snippet is a very interesting example that aims to showcase some of the advanced capabilities of the Proxy object, it is not implementing the Proxy pattern. This example allows us to see that, even though the Proxy object is commonly used to implement the Proxy pattern (hence the name), it can also be used to implement other patterns and use cases. As an example, we will see later in this chapter how to use the Proxy object—to implement the Decorator pattern.

Một điều rất quan trọng cần lưu ý là, mặc dù đoạn mã trước là một ví dụ rất thú vị nhằm phổ biến một số khả năng nâng cao của đối tượng Proxy, nó không hiện thực mẫu Proxy. Ví dụ này cho phép chúng ta thấy rằng, mặc dù đối tượng Proxy thường được dùng để hiện thực mẫu Proxy (do đó mới có tên này), nó cũng có thể được dùng để hiện thực các mẫu và trường hợp sử dụng khác. Chẳng hạn, ở phần sau của chương này chúng ta sẽ thấy cách dùng đối tượng Proxy—để hiện thực mẫu Decorator.

Looking at the implementation, this proxy uses an empty array as the target and then defines the `get` and `has` traps in the handler:

Nhìn vào hiện thực, proxy này dùng một mảng rỗng làm target và sau đó định nghĩa các bẫy `get` và `has` trong handler:

- The `get` trap intercepts access to the array elements, returning the even number for the given index
- The `has` trap instead intercepts the usage of the `in` operator and checks whether the given number is even or not
 - Bẫy `get` chặn việc truy cập các phần tử của mảng, trả về số chẵn cho chỉ số đã cho
 - Bẫy `has` thì chặn việc sử dụng toán tử `in` và kiểm tra xem số đã cho có chẵn hay không

The Proxy object supports several other interesting traps such as `set`, `delete`, and `construct`, and allows us to create proxies that can be revoked on demand, disabling all the traps and restoring the original behavior of the target object.

Đối tượng Proxy hỗ trợ vài bẫy thú vị khác như `set`, `delete`, và `construct`, và cho phép chúng ta tạo các proxy có thể bị thu hồi (revoke) theo yêu cầu, vô hiệu hóa tất cả các bẫy và khôi phục hành vi gốc của đối tượng target.

Analyzing all these features goes beyond the scope of this chapter; what is important here is understanding that the Proxy object provides a powerful foundation for implementing the Proxy **design pattern**.

Phân tích tất cả các tính năng này nằm ngoài phạm vi của chương này; điều quan trọng ở đây là hiểu rằng đối tượng Proxy cung cấp một nền tảng mạnh mẽ để hiện thực mẫu thiết kế Proxy.

If you are curious to discover all the capabilities and trap methods offered by the Proxy object, you can read more in the related MDN article at nodejsdp.link/mdn-proxy. Another good source is this detailed article from Google at nodejsdp.link/intro-proxy.

Nếu bạn tò mò muốn khám phá tất cả các khả năng và phương thức bẫy mà đối tượng Proxy cung cấp, bạn có thể đọc thêm trong bài viết liên quan của MDN tại nodejsdp.link/mdn-proxy. Một nguồn tốt khác là bài viết chi tiết này từ Google tại nodejsdp.link/intro-proxy.

While the Proxy object is a powerful functionality of the JavaScript language, it suffers from a very important limitation: the Proxy object cannot be fully transpiled or polyfilled. This is because some of the Proxy object traps can be implemented only at the runtime level and cannot be simply rewritten in plain JavaScript. This is something to be aware of if you are working with old browsers or old versions of Node.js that don't support the Proxy object directly. Transpilation: Short for transcompilation. It indicates the action of compiling source code by translating it from one source programming language to another. In the case of JavaScript, this technique is used to convert a program using new capabilities of the language into an equivalent program that can also run on older runtimes that do not support these new capabilities.

Mặc dù đối tượng Proxy là một chức năng mạnh mẽ của ngôn ngữ JavaScript, nó mắc phải một hạn chế rất quan trọng: đối tượng Proxy không thể được transpile hay polyfill hoàn toàn. Điều này là do một số bẫy của đối tượng Proxy chỉ có thể được hiện thực ở cấp độ runtime và không thể chỉ đơn giản viết lại bằng JavaScript thuần. Đây là điều cần lưu ý nếu bạn đang làm việc với các trình duyệt cũ hoặc các phiên bản Node.js cũ không hỗ trợ trực tiếp đối tượng Proxy. Transpilation: Viết tắt của transcompilation. Nó chỉ hành động biên dịch mã nguồn bằng cách dịch nó từ một ngôn ngữ lập trình nguồn này sang một ngôn ngữ khác. Trong trường hợp của JavaScript, kỹ thuật này được dùng để chuyển đổi một chương trình dùng các khả năng mới của ngôn ngữ thành một chương trình tương đương cũng có thể chạy trên các runtime cũ không hỗ trợ các khả năng mới này.

Polyfill: Code that provides an implementation for a standard API in plain JavaScript and that can be imported in environments where this API is not available (generally older browsers or runtimes). `core-js` (nodejsdp.link/corejs) is one of the most complete polyfill libraries for JavaScript.

Polyfill: Mã cung cấp một hiện thực cho một API tiêu chuẩn bằng JavaScript thuần và có thể được import vào các môi trường nơi API này không khả dụng (thường là các trình duyệt hoặc runtime cũ). `core-js` (nodejsdp.link/corejs) là một trong những thư viện polyfill đầy đủ nhất cho JavaScript.

A comparison of the different proxying techniques — So sánh các kỹ thuật proxy khác nhau

Composition can be considered a simple and safe way of creating a proxy because it leaves the subject untouched without mutating its original behavior. Its only drawback is that we have to manually

delegate all the methods, even if we want to proxy only one of them. Also, we might have to delegate access to the properties of the subject.

Composition có thể được coi là một cách đơn giản và an toàn để tạo một proxy vì nó để subject không bị đụng tới mà không biến đổi hành vi gốc của nó. Nhược điểm duy nhất của nó là chúng ta phải ủy thác thủ công tất cả các phương thức, ngay cả khi chúng ta chỉ muốn proxy một trong số chúng. Ngoài ra, chúng ta có thể phải ủy thác việc truy cập tới các thuộc tính của subject.

Object properties can be delegated using `Object.defineProperty()`. Find out more at nodejsdp.link/define-prop.

Các thuộc tính đối tượng có thể được ủy thác bằng `Object.defineProperty()`. Tìm hiểu thêm tại nodejsdp.link/define-prop.

Object augmentation, on the other hand, modifies the subject, which might not always be ideal, but it does not suffer from the various inconveniences related to delegation. For this reason, between these two approaches, object augmentation is generally the preferred technique in all those circumstances in which modifying the subject is an option.

Object augmentation, mặt khác, sửa đổi subject, điều này không phải lúc nào cũng lý tưởng, nhưng nó không gặp phải các bất tiện liên quan đến việc ủy thác. Vì lý do này, giữa hai cách tiếp cận này, object augmentation nhìn chung là kỹ thuật được ưa chuộng hơn trong mọi tình huống mà việc sửa đổi subject là một lựa chọn khả thi.

However, there is at least one situation where composition is almost necessary; this is when we want to control the initialization of the subject, for example, to create it only when needed (lazy initialization).

Tuy nhiên, có ít nhất một tình huống mà composition gần như là bắt buộc; đó là khi chúng ta muốn kiểm soát việc khởi tạo subject, ví dụ, để chỉ tạo nó khi cần (lazy initialization).

Finally, the Proxy object is the go-to approach if you need to intercept function calls or have different types of access to object attributes, even dynamic ones. The Proxy object provides an advanced level of access control that is simply not available with the other techniques. For example, the Proxy object allows us to intercept the deletion of a key in an object and to perform property existence checks. Once again, it's worth highlighting that the Proxy object does not mutate the subject, so it can be safely used in contexts where the subject is shared between different components of the application. We also saw that with the Proxy object, we can easily perform delegation of all the methods and attributes that we want to leave unchanged.

Cuối cùng, đối tượng Proxy là cách tiếp cận nên dùng nếu bạn cần chặn các lời gọi hàm hoặc có các kiểu truy cập khác nhau tới các thuộc tính của đối tượng, kể cả các thuộc tính động. Đối tượng Proxy cung cấp một mức độ kiểm soát truy cập nâng cao mà đơn giản là không có với các kỹ thuật khác. Ví dụ, đối tượng Proxy cho phép chúng ta chặn việc xóa một key trong một đối tượng và thực hiện kiểm tra sự tồn tại của thuộc tính. Một lần nữa, cũng đáng nhấn mạnh rằng đối tượng Proxy không biến đổi subject, nên nó có thể được dùng an toàn trong các ngữ cảnh mà subject được chia sẻ giữa các thành phần khác nhau của ứng dụng. Chúng ta cũng đã thấy rằng với đối tượng Proxy, chúng ta có thể dễ dàng thực hiện việc ủy thác tất cả các phương thức và thuộc tính mà chúng ta muốn để nguyên không thay đổi.

In the next section, we present a more realistic example leveraging the Proxy pattern and use it to compare the different techniques we have discussed so far for implementing this pattern.

Trong phần tiếp theo, chúng ta trình bày một ví dụ thực tế hơn tận dụng mẫu Proxy và

dùng nó để so sánh các kỹ thuật khác nhau mà chúng ta đã bàn đến nay để hiện thực mẫu này.

Creating a logging Writable stream — Tạo một Writable stream ghi log

To see the Proxy pattern applied to a real example, we will now build an object that acts as a proxy to a **Writable stream**, which intercepts all the calls to the `write()` method and logs a message every time this happens. We will use the Proxy object to implement our proxy. Let's write our code in a file called `logging-writable.js`:

Để thấy mẫu Proxy được áp dụng vào một ví dụ thực tế, bây giờ chúng ta sẽ xây dựng một đối tượng đóng vai trò là proxy cho một Writable stream, đối tượng này chặn tất cả các lời gọi tới phương thức `write()` và ghi log một thông điệp mỗi khi điều này xảy ra. Chúng ta sẽ dùng đối tượng Proxy để hiện thực proxy của mình. Hãy viết mã của chúng ta trong một file gọi là `logging-writable.js`:

```
export function createLoggingWritable (writable) {  
  return new Proxy(writable, {                                // (1)  
    get (target, propKey, receiver) {                        // (2)  
      if (propKey === 'write') {                              // (3)  
        return function (...args) {                          // (4)  
          const [chunk] = args  
          console.log('Writing', chunk)  
          return writable.write(...args)  
        }  
      }  
      return target[propKey]                                // (5)  
    }  
  })  
}
```

In the preceding code, we created a factory that returns a proxied version of the `writable` object passed as an argument. Let's see what the main points of the implementation are:

Trong đoạn mã trước, chúng ta đã tạo một factory trả về một phiên bản được proxy của đối tượng `writable` được truyền vào làm tham số. Hãy xem các điểm chính của hiện thực này là gì:

1. We create and return a proxy for the original `writable` object using the ES2015 Proxy constructor.
2. We use the `get` trap to intercept access to the object properties.
3. We check whether the property accessed is the `write` method. If that is the case, we return a function to proxy the original behavior.
4. The proxy implementation logic here is simple: we extract the current chunk from the list of arguments passed to the original function, we log the content of the chunk, and finally, we invoke the original method with the given list of arguments.
5. We return unchanged any other property.
 1. Chúng ta tạo và trả về một proxy cho đối tượng `writable` gốc bằng cách dùng hàm khởi tạo Proxy của ES2015.
 2. Chúng ta dùng bẫy `get` để chặn việc truy cập tới các thuộc tính của đối tượng.
 3. Chúng ta kiểm tra xem thuộc tính được truy cập có phải là phương thức `write` không. Nếu đúng vậy, chúng ta trả về một hàm để proxy hành vi gốc.

4. Logic hiện thực proxy ở đây đơn giản: chúng ta trích chunk hiện tại từ danh sách các tham số được truyền cho hàm gốc, ghi log nội dung của chunk, và cuối cùng, gọi phương thức gốc với danh sách tham số đã cho.
5. Chúng ta trả về không thay đổi mọi thuộc tính khác.

We can now use this newly created function and test our proxy implementation:

Bây giờ chúng ta có thể dùng hàm vừa tạo này và kiểm thử hiện thực proxy của mình:

```
import { createWriteStream } from 'fs'
import { createLoggingWritable } from './logging-writable.js'
const writable = createWriteStream('test.txt')
const writableProxy = createLoggingWritable(writable)
writableProxy.write('First chunk')
writableProxy.write('Second chunk')
writable.write('This is not logged')
writableProxy.end()
```

The proxy did not change the original interface of the **stream** or its external behavior, but if we run the preceding code, we will now see that every chunk that is written into the `writableProxy` stream is transparently logged to the console.

Proxy đã không thay đổi giao diện gốc của stream hay hành vi bên ngoài của nó, nhưng nếu chúng ta chạy đoạn mã trước, bây giờ chúng ta sẽ thấy rằng mỗi chunk được ghi vào stream `writableProxy` được ghi log một cách trong suốt ra console.

Change observer with Proxy — Change observer với Proxy

The Change **Observer** pattern is a design pattern in which an object (the subject) notifies one or more observers of any state changes, so that they can “react” to changes as soon as they happen.

Mẫu Change Observer là một mẫu thiết kế trong đó một đối tượng (subject) thông báo cho một hoặc nhiều observer về bất kỳ thay đổi trạng thái nào, để chúng có thể “phản ứng” với các thay đổi ngay khi chúng xảy ra.

Although very similar, the Change Observer pattern should not be confused with the Observer pattern discussed in Chapter 3, Callbacks and Events. The Change Observer pattern focuses on allowing the detection of property changes, while the Observer pattern is a more generic pattern that adopts an **event emitter** to propagate information about events happening in the system.

Mặc dù rất giống nhau, mẫu Change Observer không nên bị nhầm lẫn với mẫu Observer đã bàn trong Chương 3, Callbacks and Events. Mẫu Change Observer tập trung vào việc cho phép phát hiện các thay đổi thuộc tính, trong khi mẫu Observer là một mẫu tổng quát hơn áp dụng một event emitter để lan truyền thông tin về các sự kiện xảy ra trong hệ thống.

Proxies turn out to be quite an effective tool to create observable objects. Let’s see a possible implementation with `create-observable.js`:

Proxy hóa ra là một công cụ khá hiệu quả để tạo các đối tượng có thể quan sát được (observable). Hãy xem một hiện thực khá dễ với `create-observable.js`:

```
export function createObservable (target, observer) {
  const observable = new Proxy(target, {
    set (obj, prop, value) {
      if (value !== obj[prop]) {
```

```

    const prev = obj[prop]
    obj[prop] = value
    observer({ prop, prev, curr: value })
  }
  return true
}
})
return observable
}

```

In the previous code, `createObservable()` accepts a target object (the object to observe for changes) and an observer (a function to invoke every time a change is detected).

Trong đoạn mã trước, `createObservable()` nhận một đối tượng `target` (đối tượng cần quan sát các thay đổi) và một `observer` (một hàm để gọi mỗi khi phát hiện một thay đổi).

Here, we create the observable instance through an ES2015 Proxy. The proxy implements the set trap, which is triggered every time a property is set. The implementation compares the current value with the new one and, if they are different, the target object is mutated, and the observer gets notified. When the observer is invoked, we pass an object literal that contains information related to the change (the name of the property, the previous value, and the current value).

Ở đây, chúng ta tạo thể hiện `observable` thông qua một Proxy của ES2015. Proxy hiện thực bẫy `set`, được kích hoạt mỗi khi một thuộc tính được đặt giá trị. Hiện thực này so sánh giá trị hiện tại với giá trị mới và, nếu chúng khác nhau, đối tượng `target` được biến đổi, và `observer` được thông báo. Khi `observer` được gọi, chúng ta truyền một object literal chứa thông tin liên quan đến thay đổi (tên của thuộc tính, giá trị trước đó, và giá trị hiện tại).

This is a simplified implementation of the Change Observer pattern. More advanced implementations support multiple observers and use more traps to catch other types of mutation, such as field deletions or changes of prototype. Moreover, our implementation does not recursively create proxies for nested objects or arrays—a more advanced implementation takes care of these cases as well.

Đây là một hiện thực đơn giản hóa của mẫu Change Observer. Các hiện thực nâng cao hơn hỗ trợ nhiều `observer` và dùng nhiều bẫy hơn để bắt các kiểu thay đổi khác, chẳng hạn như xóa trường hoặc thay đổi prototype. Hơn nữa, hiện thực của chúng ta không tạo proxy một cách đệ quy cho các đối tượng hay mảng lồng nhau—một hiện thực nâng cao hơn cũng xử lý các trường hợp này.

Let's see now how we can take advantage of observable objects with a trivial invoice application where the invoice total is updated automatically based on observed changes in the various fields of the invoice:

Bây giờ hãy xem cách chúng ta có thể tận dụng các đối tượng `observable` với một ứng dụng hóa đơn đơn giản, nơi tổng hóa đơn được cập nhật tự động dựa trên các thay đổi được quan sát ở các trường khác nhau của hóa đơn:

```

import { createObservable } from './create-observable.js'
function calculateTotal (invoice) {                                     // (1)
  return invoice.subtotal -
    invoice.discount +
    invoice.tax
}

```

```

const invoice = {
  subtotal: 100,
  discount: 10,
  tax: 20
}
let total = calculateTotal(invoice)
console.log(`Starting total: ${total}`)
const obsInvoice = createObservable(                                     // (2)
  invoice,
  ({ prop, prev, curr }) => {
    total = calculateTotal(invoice)
    console.log(`TOTAL: ${total} (${prop} changed: ${prev} ->
${curr})`)
  }
)
                                                                    // (3)

obsInvoice.subtotal = 200 // TOTAL: 210
obsInvoice.discount = 20  // TOTAL: 200
obsInvoice.discount = 20  // no change: doesn't notify
obsInvoice.tax = 30       // TOTAL: 210
console.log(`Final total: ${total}`)

```

In the previous example, an invoice is composed of a subtotal value, a discount value, and a tax value. The total amount can be calculated from these three values. Let's discuss the implementation in greater detail:

Trong ví dụ trước, một hóa đơn được tạo thành từ một giá trị subtotal, một giá trị discount, và một giá trị tax. Tổng số tiền có thể được tính từ ba giá trị này. Hãy bàn về hiện thực này chi tiết hơn:

1. We declare a function that calculates the total for a given invoice, then we create an invoice object and a value to hold the total for it.
2. Here we create an observable version of the invoice object. Every time there is a change in the original invoice object, we recalculate the total and we also print some logs to keep track of the changes.
3. Finally, we apply some changes to the observable invoice. Every time we mutate the obsInvoice object the observer function is triggered, the total gets updated, and some logs are printed on the screen.
 1. Chúng ta khai báo một hàm tính tổng cho một hóa đơn nhất định, sau đó chúng ta tạo một đối tượng invoice và một giá trị để giữ total của nó.
 2. Ở đây chúng ta tạo một phiên bản observable của đối tượng invoice. Mỗi khi có một thay đổi trong đối tượng invoice gốc, chúng ta tính lại tổng và cũng in một số log để theo dõi các thay đổi.
 3. Cuối cùng, chúng ta áp dụng một số thay đổi cho hóa đơn observable. Mỗi khi chúng ta biến đổi đối tượng obsInvoice, hàm observer được kích hoạt, tổng được cập nhật, và một số log được in ra màn hình.

If we run this example, we will see the following output in the console:

Nếu chúng ta chạy ví dụ này, chúng ta sẽ thấy kết quả sau trong console:

```

Starting total: 110
TOTAL: 210 (subtotal changed: 100 -> 200)

```

TOTAL: 200 (discount changed: 10 -> 20)
TOTAL: 210 (tax changed: 20 -> 30)
Final total: 210

In this example, we could make the total calculation logic arbitrarily complicated, for instance, by introducing new fields in the computation (shipping costs, other taxes, and so on). In this case, it will be fairly trivial to introduce the new fields in the invoice object and update the `calculateTotal()` function. Once we do that, every change to the new properties will be observed and the total will be kept up to date with every change.

Trong ví dụ này, chúng ta có thể làm cho logic tính tổng phức tạp tùy ý, chẳng hạn, bằng cách đưa các trường mới vào phép tính (chi phí vận chuyển, các loại thuế khác, v.v.). Trong trường hợp này, sẽ khá đơn giản để đưa các trường mới vào đối tượng `invoice` và cập nhật hàm `calculateTotal()`. Một khi chúng ta làm vậy, mọi thay đổi đối với các thuộc tính mới sẽ được quan sát và `total` sẽ được giữ cập nhật với mọi thay đổi.

Observables are the cornerstone of reactive programming (RP) and functional reactive programming (FRP). If you are curious to know more about these styles of programming check out the Reactive Manifesto, at nodejsdp.link/reactive-manifesto.

Observable là nền tảng cốt lõi của lập trình phản ứng (reactive programming - RP) và lập trình phản ứng hàm (functional reactive programming - FRP). Nếu bạn tò mò muốn biết thêm về các phong cách lập trình này hãy xem Reactive Manifesto, tại nodejsdp.link/reactive-manifesto.

In the wild — Trong thực tế

The Proxy pattern and more specifically the Change Observer pattern are widely adopted patterns, which can be found on backend projects and libraries as well as in the frontend world. Some popular projects that take advantage of these patterns include the following:

Mẫu Proxy và cụ thể hơn là mẫu Change Observer là những mẫu được áp dụng rộng rãi, có thể được tìm thấy trong các dự án và thư viện backend cũng như trong thế giới frontend. Một số dự án phổ biến tận dụng các mẫu này bao gồm:

- `LoopBack` (nodejsdp.link/loopback) is a popular Node.js web framework that uses the Proxy pattern to provide the capability to intercept and enhance method calls on controllers. This capability can be used to build custom validation or authentication mechanisms.
- Version 3 of `Vue.js` (nodejsdp.link/vue), a very popular JavaScript reactive UI framework, has reimplemented observable properties using the Proxy pattern with the Proxy object.
- `MobX` (nodejsdp.link/mobx) is a famous reactive state management library commonly used in frontend applications in combination with React or Vue.js. Like Vue.js, MobX implements reactive observables using the Proxy object.
 - `LoopBack` (nodejsdp.link/loopback) là một web framework Node.js phổ biến dùng mẫu Proxy để cung cấp khả năng chặn và nâng cao các lời gọi phương thức trên các controller. Khả năng này có thể được dùng để xây dựng các cơ chế kiểm tra hoặc xác thực tùy chỉnh.
 - Phiên bản 3 của `Vue.js` (nodejsdp.link/vue), một framework UI phản ứng JavaScript rất phổ biến, đã hiện thực lại các thuộc tính observable bằng mẫu Proxy với đối tượng Proxy.
 - `MobX` (nodejsdp.link/mobx) là một thư viện quản lý trạng thái phản ứng nổi tiếng thường được dùng trong các ứng dụng frontend kết hợp với React hoặc Vue.js. Giống như Vue.js, MobX hiện thực các observable phản ứng bằng đối tượng Proxy.

Decorator — Decorator

Decorator is a structural design pattern that consists in dynamically augmenting the behavior of an existing object. It's different from classical inheritance, because the behavior is not added to all the objects of the same class, but only to the instances that are explicitly decorated.

Decorator là một mẫu thiết kế cấu trúc gồm việc bổ sung hành vi cho một đối tượng hiện có một cách động. Nó khác với kế thừa (inheritance) cổ điển, vì hành vi không được thêm vào tất cả các đối tượng của cùng một class, mà chỉ vào các thể hiện được trang trí (decorate) một cách tường minh.

Implementation-wise, it is very similar to the Proxy pattern, but instead of enhancing or modifying the behavior of the existing interface of an object, it augments it with new functionalities, as described in Figure 8.2:

Về mặt hiện thực, nó rất giống mẫu Proxy, nhưng thay vì nâng cao hoặc sửa đổi hành vi của giao diện hiện có của một đối tượng, nó bổ sung cho nó các chức năng mới, như mô tả trong Hình 8.2:

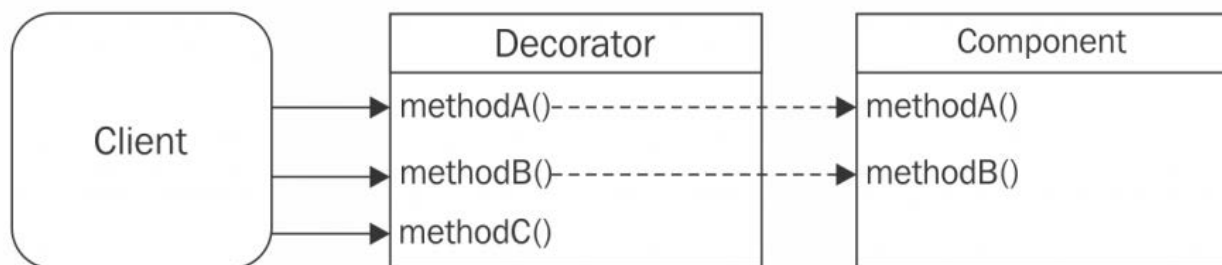


Figure 8.2: Decorator pattern schematic

Hình 8.2: Sơ đồ mẫu Decorator

In Figure 8.2, the Decorator object is extending the Component object by adding the `methodC()` operation. The existing methods are usually delegated to the decorated object without further processing but, in some cases, they might also be intercepted and augmented with extra behaviors.

Trong Hình 8.2, đối tượng Decorator đang mở rộng đối tượng Component bằng cách thêm thao tác `methodC()`. Các phương thức hiện có thường được ủy thác cho đối tượng được trang trí mà không xử lý thêm nhưng, trong một số trường hợp, chúng cũng có thể bị chặn và bổ sung thêm các hành vi bổ trợ.

Techniques for implementing decorators — Các kỹ thuật hiện thực decorator

Although proxy and decorator are conceptually two different patterns with different intents, they practically share the same implementation strategies. We will review them shortly. This time we want to use the Decorator pattern to be able to take an instance of our `StackCalculator` class and “decorate it” so that it also exposes a new method called `add()`, which we can use to perform additions between two numbers. We will also use the decorator to intercept all the calls to the `divide()` method and implement the same division-by-zero check that we already saw in our `SafeCalculator` example.

Mặc dù proxy và decorator về mặt khái niệm là hai mẫu khác nhau với các ý định khác nhau, trên thực tế chúng chia sẻ cùng các chiến lược hiện thực. Chúng ta sẽ điểm qua chúng ngay sau đây. Lần này chúng ta muốn dùng mẫu Decorator để có thể lấy một thể

hiện của class `StackCalculator` và “trang trí nó” để nó cũng phơi bày một phương thức mới gọi là `add()`, mà chúng ta có thể dùng để thực hiện các phép cộng giữa hai số. Chúng ta cũng sẽ dùng decorator để chặn tất cả các lời gọi tới phương thức `divide()` và hiện thực cùng phép kiểm tra chia-cho-không mà chúng ta đã thấy trong ví dụ `SafeCalculator`.

Composition — Composition

Using composition, the decorated component is wrapped around a new object that usually inherits from it. The decorator in this case simply needs to define the new methods, while delegating the existing ones to the original component:

Dùng composition, thành phần được trang trí được bao bọc bởi một đối tượng mới thường kế thừa từ nó. Decorator trong trường hợp này chỉ cần định nghĩa các phương thức mới, trong khi ủy thác các phương thức hiện có cho thành phần gốc:

```
class EnhancedCalculator {
  constructor (calculator) {
    this.calculator = calculator
  }
  // new method
  add () {
    const addend2 = this.getValue()
    const addend1 = this.getValue()
    const result = addend1 + addend2
    this.putValue(result)
    return result
  }
  // modified method
  divide () {
    // additional validation logic
    const divisor = this.calculator.peekValue()
    if (divisor === 0) {
      throw Error('Division by 0')
    }
    // if valid delegates to the subject
    return this.calculator.divide()
  }
  // delegated methods
  putValue (value) {
    return this.calculator.putValue(value)
  }
  getValue () {
    return this.calculator.getValue()
  }
  peekValue () {
    return this.calculator.peekValue()
  }
  clear () {
    return this.calculator.clear()
  }
  multiply () {
```

```

        return this.calculator.multiply()
    }
}
const calculator = new StackCalculator()
const enhancedCalculator = new EnhancedCalculator(calculator)
enhancedCalculator.putValue(4)
enhancedCalculator.putValue(3)
console.log(enhancedCalculator.add()) // 4+3 = 7
enhancedCalculator.putValue(2)
console.log(enhancedCalculator.multiply()) // 7*2 = 14

```

If you remember our composition implementation for the Proxy pattern, you can probably see that the code here looks quite similar.

Nếu bạn nhớ hiện thực composition của chúng ta cho mẫu Proxy, có lẽ bạn có thể thấy rằng mã ở đây trông khá giống.

We created the new `add()` method and enhanced the behavior of the original `divide()` method (effectively replicating the feature we saw in the previous `SafeCalculator` example). Finally, we delegated the `putValue()`, `getValue()`, `peekValue()`, `clear()`, and `multiply()` methods to the original subject.

Chúng ta đã tạo phương thức `add()` mới và nâng cao hành vi của phương thức `divide()` gốc (thực chất là sao chép lại tính năng mà chúng ta đã thấy trong ví dụ `SafeCalculator` trước đó). Cuối cùng, chúng ta ủy thác các phương thức `putValue()`, `getValue()`, `peekValue()`, `clear()`, và `multiply()` cho subject gốc.

Object augmentation — Object augmentation

Object decoration can also be achieved by simply attaching new methods directly to the decorated object (monkey patching), as follows:

Việc trang trí đối tượng cũng có thể đạt được bằng cách đơn giản gắn trực tiếp các phương thức mới vào đối tượng được trang trí (monkey patching), như sau:

```

function patchCalculator (calculator) {
  // new method
  calculator.add = function () {
    const addend2 = calculator.getValue()
    const addend1 = calculator.getValue()
    const result = addend1 + addend2
    calculator.putValue(result)
    return result
  }
  // modified method
  const divideOrig = calculator.divide
  calculator.divide = () => {
    // additional validation logic
    const divisor = calculator.peekValue()
    if (divisor === 0) {
      throw Error('Division by 0')
    }
    // if valid delegates to the subject

```

```

    return divideOrig.apply(calculator)
  }
  return calculator
}
const calculator = new StackCalculator()
const enhancedCalculator = patchCalculator(calculator)
// ...

```

Note that in this example, `calculator` and `enhancedCalculator` reference the same object (`calculator == enhancedCalculator`). This is because `patchCalculator()` is mutating the original calculator object and then returning it. You can confirm this by invoking `calculator.add()` or `calculator.divide()`.

Lưu ý rằng trong ví dụ này, `calculator` và `enhancedCalculator` tham chiếu đến cùng một đối tượng (`calculator == enhancedCalculator`). Điều này là vì `patchCalculator()` đang biến đổi đối tượng `calculator` gốc rồi trả về nó. Bạn có thể xác nhận điều này bằng cách gọi `calculator.add()` hoặc `calculator.divide()`.

Decorating with the Proxy object — Trang trí với đối tượng Proxy

It's possible to implement object decoration by using the Proxy object. A generic example might look like this:

Có thể hiện thực việc trang trí đối tượng bằng cách dùng đối tượng Proxy. Một ví dụ tổng quát có thể trông như sau:

```

const enhancedCalculatorHandler = {
  get (target, property) {
    if (property === 'add') {
      // new method
      return function add () {
        const addend2 = target.getValue()
        const addend1 = target.getValue()
        const result = addend1 + addend2
        target.putValue(result)
        return result
      }
    } else if (property === 'divide') {
      // modified method
      return function () {
        // additional validation logic
        const divisor = target.peekValue()
        if (divisor === 0) {
          throw Error('Division by 0')
        }
        // if valid delegates to the subject
        return target.divide()
      }
    }
    // delegated methods and properties
    return target[property]
  }
}

```

```

}
const calculator = new StackCalculator()
const enhancedCalculator = new Proxy(
  calculator,
  enhancedCalculatorHandler
)
// ...

```

If we were to compare these different implementations, the same caveats discussed during the analysis of the Proxy pattern would also apply for the decorator. Let's focus instead on practicing the pattern with a real-life example!

Nếu chúng ta so sánh các hiện thực khác nhau này, những lưu ý đã bàn trong phần phân tích mẫu Proxy cũng sẽ áp dụng cho decorator. Thay vào đó, hãy tập trung thực hành mẫu này với một ví dụ thực tế!

Decorating a LevelUP database — Trang trí một cơ sở dữ liệu LevelUP

Before we start coding the next example, let's say a few words about LevelUP, the **module** that we are now going to work with.

Trước khi bắt đầu viết mã cho ví dụ tiếp theo, hãy nói vài lời về LevelUP, module mà bây giờ chúng ta sẽ làm việc cùng.

Introducing LevelUP and LevelDB — Giới thiệu LevelUP và LevelDB

LevelUP (nodejsdp.link/levelup) is a Node.js wrapper around Google's LevelDB, a key-value store originally built to implement IndexedDB in the Chrome browser, but it's much more than that. LevelDB has been defined as the “Node.js of databases” because of its minimalism and extensibility. Like Node.js, LevelDB provides blazingly fast performance and only the most basic set of features, allowing developers to build any kind of database on top of it.

LevelUP (nodejsdp.link/levelup) là một bộ bao (wrapper) Node.js quanh LevelDB của Google, một kho key-value ban đầu được xây dựng để hiện thực IndexedDB trong trình duyệt Chrome, nhưng nó còn hơn thế nhiều. LevelDB đã được định nghĩa là “Node.js của các cơ sở dữ liệu” nhờ tính tối giản và khả năng mở rộng của nó. Giống như Node.js, LevelDB cung cấp hiệu năng cực nhanh và chỉ một tập tính năng cơ bản nhất, cho phép các nhà phát triển xây dựng bất kỳ loại cơ sở dữ liệu nào trên nền nó.

The Node.js community, and in this case Rod Vagg, did not miss the chance to bring the power of this database into the Node.js world by creating LevelUP. Born as a wrapper for LevelDB, it then evolved to support several kinds of backends, from in-memory stores, to other NoSQL databases such as Riak and Redis, to web storage engines such as IndexedDB and localStorage, allowing us to use the same API on both the server and the client, opening up some really interesting scenarios.

Cộng đồng Node.js, và trong trường hợp này là Rod Vagg, đã không bỏ lỡ cơ hội đưa sức mạnh của cơ sở dữ liệu này vào thế giới Node.js bằng cách tạo ra LevelUP. Ra đời như một wrapper cho LevelDB, sau đó nó tiến hóa để hỗ trợ vài loại backend, từ các kho trong bộ nhớ (in-memory store), đến các cơ sở dữ liệu NoSQL khác như Riak và Redis, đến các engine lưu trữ web như IndexedDB và localStorage, cho phép chúng ta dùng cùng một API trên cả máy chủ và máy khách, mở ra một số kịch bản thực sự thú vị.

Today, there is a vast ecosystem around LevelUP made of plugins and modules that extend the tiny core to implement features such as replication, secondary indexes, live updates, query

engines, and more. Complete databases were also built on top of LevelUP, including CouchDB clones such as PouchDB (nodejsdp.link/pouchdb), and even a graph database, LevelGraph (nodejsdp.link/levelgraph), which can work both on Node.js and the browser!

Ngày nay, có một hệ sinh thái rộng lớn quanh LevelUP gồm các plugin và module mở rộng phần lõi nhỏ bé để hiện thực các tính năng như sao chép (replication), chỉ mục thứ cấp (secondary index), cập nhật trực tiếp (live update), engine truy vấn, và hơn thế. Các cơ sở dữ liệu hoàn chỉnh cũng được xây dựng trên nền LevelUP, bao gồm các bản sao CouchDB như PouchDB (nodejsdp.link/pouchdb), và thậm chí một cơ sở dữ liệu đồ thị, LevelGraph (nodejsdp.link/levelgraph), có thể chạy được trên cả Node.js và trình duyệt!

Find out more about the LevelUP ecosystem at nodejsdp.link/awesome-level.

Tìm hiểu thêm về hệ sinh thái LevelUP tại nodejsdp.link/awesome-level.

Implementing a LevelUP plugin — Hiện thực một plugin LevelUP

In the next example, we are going to show you how we can create a simple plugin for LevelUP using the Decorator pattern, and in particular, the object augmentation technique, which is the simplest but also the most pragmatic and effective way to decorate objects with additional capabilities.

Trong ví dụ tiếp theo, chúng ta sẽ chỉ cho bạn cách chúng ta có thể tạo một plugin đơn giản cho LevelUP bằng mẫu Decorator, và cụ thể là kỹ thuật object augmentation, vốn là cách đơn giản nhất nhưng cũng thực dụng và hiệu quả nhất để trang trí các đối tượng với các khả năng bổ sung.

For convenience, we are going to use the level **package** (nodejsdp.link/level), which bundles both levelup and the default adapter called leveledown, which uses LevelDB as the backend.

Để thuận tiện, chúng ta sẽ dùng gói level (nodejsdp.link/level), gói này đóng gói cả levelup và adapter mặc định gọi là leveledown, dùng LevelDB làm backend.

What we want to build is a plugin for LevelUP that allows us to receive notifications every time an object with a certain pattern is saved into the database. For example, if we **subscribe** to a pattern such as `{a: 1}`, we want to receive a notification when objects such as `{a: 1, b: 3}` or `{a: 1, c: 'x'}` are saved into the database.

Cái chúng ta muốn xây dựng là một plugin cho LevelUP cho phép chúng ta nhận thông báo mỗi khi một đối tượng có một mẫu (pattern) nhất định được lưu vào cơ sở dữ liệu. Ví dụ, nếu chúng ta đăng ký một mẫu như `{a: 1}`, chúng ta muốn nhận thông báo khi các đối tượng như `{a: 1, b: 3}` hoặc `{a: 1, c: 'x'}` được lưu vào cơ sở dữ liệu.

Let's start to build our small plugin by creating a new module called `level-subscribe.js`. We will then insert the following code:

Hãy bắt đầu xây dựng plugin nhỏ của chúng ta bằng cách tạo một module mới gọi là `level-subscribe.js`. Sau đó chúng ta sẽ chèn đoạn mã sau:

```
export function levelSubscribe (db) {
  db.subscribe = (pattern, listener) => { // (1)
    db.on('put', (key, val) => { // (2)
      const match = Object.keys(pattern).every(
        k => (pattern[k] === val[k]) // (3)
      )
      if (match) {
```



```

        listener(key, val)                                // (4)
    }
  })
}
return db
}

```

That's it for our plugin; it's extremely simple. Let's briefly analyze the preceding code:

Vậy là xong plugin của chúng ta; nó cực kỳ đơn giản. Hãy phân tích ngắn gọn đoạn mã trên:

1. We decorate the db object with a new method named `subscribe()`. We simply attach the method directly to the provided db instance (object augmentation).
2. We listen for any put operation performed on the database.
3. We perform a very simple pattern-matching algorithm, which verifies that all the properties in the provided pattern are also available in the data being inserted.
4. If we have a match, we notify the listener.

1. Chúng ta trang trí đối tượng db bằng một phương thức mới tên là `subscribe()`. Chúng ta đơn giản gắn phương thức trực tiếp vào thể hiện db được cung cấp (object augmentation).
2. Chúng ta lắng nghe bất kỳ thao tác put nào được thực hiện trên cơ sở dữ liệu.
3. Chúng ta thực hiện một thuật toán so khớp mẫu (pattern-matching) rất đơn giản, xác minh rằng tất cả các thuộc tính trong mẫu được cung cấp cũng có sẵn trong dữ liệu đang được chèn.
4. Nếu có một sự khớp, chúng ta thông báo cho listener.

Let's now write some code to try out our new plugin:

Bây giờ hãy viết một ít mã để thử plugin mới của chúng ta:

```

import { dirname, join } from 'path'
import { fileURLToPath } from 'url'
import level from 'level'
import { levelSubscribe } from './level-subscribe.js'
const __dirname = dirname(fileURLToPath(import.meta.url))
const dbPath = join(__dirname, 'db')
const db = level(dbPath, { valueEncoding: 'json' }) // (1)
levelSubscribe(db) // (2)
db.subscribe( // (3)
  { doctype: 'tweet', language: 'en' },
  (k, val) => console.log(val)
)
db.put('1', { // (4)
  doctype: 'tweet',
  text: 'Hi',
  language: 'en'
})
db.put('2', {
  doctype: 'company',
  name: 'ACME Co.'
})

```

This is how the preceding code works:

Đây là cách đoạn mã trên hoạt động:

1. First, we initialize our LevelUP database, choosing the directory where the files are stored and the default encoding for the values.
2. Then, we attach our plugin, which decorates the original db object.
3. At this point, we are ready to use the new feature provided by our plugin, which is the `subscribe()` method, where we specify that we are interested in all the objects with `doctype: 'tweet'` and `language: 'en'`.
4. Finally, we save some values in the database using `put`. The first call triggers the **callback** associated with our subscription and we should see the stored object printed to the console. This is because, in this case, the object matches the subscription. The second call does not generate any output because the stored object does not match the subscription criteria.
 1. Đầu tiên, chúng ta khởi tạo cơ sở dữ liệu LevelUP của mình, chọn thư mục nơi các file được lưu trữ và mã hóa (encoding) mặc định cho các giá trị.
 2. Sau đó, chúng ta gắn plugin của mình, plugin này trang trí đối tượng db gốc.
 3. Tại điểm này, chúng ta đã sẵn sàng dùng tính năng mới do plugin của mình cung cấp, đó là phương thức `subscribe()`, nơi chúng ta chỉ định rằng chúng ta quan tâm đến tất cả các đối tượng có `doctype: 'tweet'` và `language: 'en'`.
 4. Cuối cùng, chúng ta lưu một số giá trị vào cơ sở dữ liệu bằng `put`. Lờ gọi đầu tiên kích hoạt callback liên kết với đăng ký của chúng ta và chúng ta sẽ thấy đối tượng được lưu trữ in ra console. Điều này là vì, trong trường hợp này, đối tượng khớp với đăng ký. Lờ gọi thứ hai không tạo ra kết quả nào vì đối tượng được lưu trữ không khớp với tiêu chí đăng ký.

This example shows a real application of the Decorator pattern in its simplest implementation, which is object augmentation. It may look like a trivial pattern, but it has undoubted power if used appropriately.

Ví dụ này cho thấy một ứng dụng thực tế của mẫu Decorator trong hiện thực đơn giản nhất của nó, đó là object augmentation. Nó có thể trông như một mẫu tầm thường, nhưng nó có sức mạnh không thể chối cãi nếu được dùng đúng cách.

For simplicity, our plugin works only in combination with `put` operations, but it can be easily expanded to work even with batch operations (nodejsdp.link/levelup-batch).

Để cho đơn giản, plugin của chúng ta chỉ hoạt động kết hợp với các thao tác `put`, nhưng nó có thể dễ dàng được mở rộng để hoạt động kể cả với các thao tác batch (nodejsdp.link/levelup-batch).

In the wild — Trong thực tế

For more examples of how decorators are used in the real world, you can inspect the code of some more LevelUP plugins:

Để biết thêm các ví dụ về cách decorator được dùng trong thế giới thực, bạn có thể xem xét mã của một số plugin LevelUP khác:

- `level-inverted-index` (nodejsdp.link/level-inverted-index): This is a plugin that adds inverted indexes to a LevelUP database, allowing us to perform simple text searches across the values stored in the database
- `levelplus` (nodejsdp.link/levelplus): This is a plugin that adds atomic updates to a LevelUP database

- `level-inverted-index` (nodejsdp.link/level-inverted-index): Đây là một plugin thêm chỉ mục đảo ngược (inverted index) vào một cơ sở dữ liệu LevelUP, cho phép chúng ta thực hiện các tìm kiếm văn bản đơn giản trên các giá trị được lưu trong cơ sở dữ liệu
- `levelplus` (nodejsdp.link/levelplus): Đây là một plugin thêm các cập nhật nguyên tử (atomic update) vào một cơ sở dữ liệu LevelUP

Aside from LevelUP plugins, the following projects are also good examples of the adoption of the Decorator pattern:

Ngoài các plugin LevelUP, các dự án sau cũng là những ví dụ tốt về việc áp dụng mẫu Decorator:

- `json-socket` (nodejsdp.link/json-socket): This module makes it easier to send JSON data over a TCP (or a Unix) socket. It is designed to decorate an existing instance of `net.Socket`, which gets enriched with additional methods and behaviors.
- `fastify` (nodejsdp.link/fastify) is a web application framework that exposes an API to decorate a Fastify server instance with additional functionality or configuration. With this approach, the additional functionality is made accessible to different parts of the application. This is a quite generalized implementation of the Decorator pattern. Check out the dedicated documentation page to find out more at nodejsdp.link/fastify-decorators.
 - `json-socket` (nodejsdp.link/json-socket): Module này giúp việc gửi dữ liệu JSON qua một socket TCP (hoặc Unix) dễ dàng hơn. Nó được thiết kế để trang trí một thể hiện hiện có của `net.Socket`, thể hiện này được làm phong phú thêm với các phương thức và hành vi bổ sung.
 - `fastify` (nodejsdp.link/fastify) là một web application framework phơi bày một API để trang trí một thể hiện máy chủ Fastify với chức năng hoặc cấu hình bổ sung. Với cách tiếp cận này, chức năng bổ sung được làm cho có thể truy cập từ các phần khác nhau của ứng dụng. Đây là một hiện thực khá tổng quát hóa của mẫu Decorator. Hãy xem trang tài liệu chuyên dụng để tìm hiểu thêm tại nodejsdp.link/fastify-decorators.

The line between proxy and decorator — Ranh giới giữa proxy và decorator

At this point in the book, you might have some legitimate doubts about the differences between the Proxy and the Decorator patterns. These two patterns are indeed very similar and they can sometimes be used interchangeably.

Tại điểm này trong cuốn sách, bạn có thể có một số nghi ngờ chính đáng về sự khác biệt giữa mẫu Proxy và mẫu Decorator. Hai mẫu này quả thực rất giống nhau và đôi khi chúng có thể được dùng thay thế cho nhau.

In its classic incarnation, the Decorator pattern is defined as a mechanism that allows us to enhance an existing object with new behavior, while the Proxy pattern is used to control access to a concrete or virtual object.

Trong hình thái cổ điển của nó, mẫu Decorator được định nghĩa là một cơ chế cho phép chúng ta nâng cao một đối tượng hiện có với hành vi mới, trong khi mẫu Proxy được dùng để kiểm soát quyền truy cập tới một đối tượng cụ thể hoặc ảo.

There is a conceptual difference between the two patterns, and it's mostly based on the way they are used at runtime.

Có một sự khác biệt về khái niệm giữa hai mẫu, và nó chủ yếu dựa trên cách chúng được dùng tại runtime.

You can look at the Decorator pattern as a wrapper; you can take different types of objects and decide to wrap them with a decorator to enhance their capabilities with extra functionality. A proxy, instead, is used to control the access to an object and it does not change the original interface. For this reason, once you have created a proxy instance, you can pass it over to a context that expects the original object.

Bạn có thể nhìn mẫu Decorator như một bộ bao (wrapper); bạn có thể lấy các kiểu đối tượng khác nhau và quyết định bao chúng bằng một decorator để nâng cao khả năng của chúng với chức năng bổ sung. Một proxy, ngược lại, được dùng để kiểm soát quyền truy cập tới một đối tượng và nó không thay đổi giao diện gốc. Vì lý do này, một khi bạn đã tạo một thể hiện proxy, bạn có thể truyền nó cho một ngữ cảnh kỳ vọng đối tượng gốc.

When it comes to implementation, these differences are generally much more obvious with strongly typed languages where the type of the objects you pass around is checked at compile time. In the Node.js ecosystem, given the dynamic nature of the JavaScript language, the line between the Proxy and the Decorator patterns is quite blurry, and often the two names are used interchangeably. We have also seen how the same techniques can be used to implement both patterns.

Khi đến phần hiện thực, những khác biệt này nhìn chung rõ ràng hơn nhiều với các ngôn ngữ định kiểu mạnh (strongly typed) nơi kiểu của các đối tượng bạn truyền qua lại được kiểm tra tại thời điểm biên dịch. Trong hệ sinh thái Node.js, do bản chất động của ngôn ngữ JavaScript, ranh giới giữa mẫu Proxy và mẫu Decorator khá mờ nhạt, và thường hai cái tên được dùng thay thế cho nhau. Chúng ta cũng đã thấy cách cùng các kỹ thuật có thể được dùng để hiện thực cả hai mẫu.

When dealing with JavaScript and Node.js, our advice is to avoid getting bogged down with the nomenclature and the canonical definition of these two patterns. We encourage you to look at the class of problems that proxy and decorator solve as a whole and treat these two patterns as complementary and sometimes interchangeable tools.

Khi làm việc với JavaScript và Node.js, lời khuyên của chúng tôi là tránh sa lầy vào danh pháp và định nghĩa quy ước của hai mẫu này. Chúng tôi khuyến khích bạn nhìn lớp các vấn đề mà proxy và decorator giải quyết như một tổng thể và coi hai mẫu này như những công cụ hỗ trợ cho nhau và đôi khi có thể thay thế cho nhau.

Adapter — Adapter

The Adapter pattern allows us to access the functionality of an object using a different interface.

Mẫu Adapter cho phép chúng ta truy cập chức năng của một đối tượng thông qua một giao diện khác.

A real-life example of an adapter would be a device that allows you to plug a USB Type-A cable into a USB Type-C port. In a generic sense, an adapter converts an object with a given interface so that it can be used in a context where a different interface is expected. In software, the Adapter pattern is used to take the interface of an object (the adaptee) and make it compatible with another interface that is expected by a given client. Let's have a look at Figure 8.3 to clarify this idea:

Một ví dụ đời thực về adapter là một thiết bị cho phép bạn cắm một cáp USB Type-A vào một cổng USB Type-C. Theo nghĩa tổng quát, một adapter chuyển đổi một đối tượng có một giao diện nhất định để nó có thể được dùng trong một ngữ cảnh kỳ vọng một giao diện khác. Trong phần mềm, mẫu Adapter được dùng để lấy giao diện của một đối tượng (adaptee) và làm cho nó tương thích với một giao diện khác mà một client nhất định kỳ vọng. Hãy xem Hình 8.3 để làm rõ ý tưởng này:

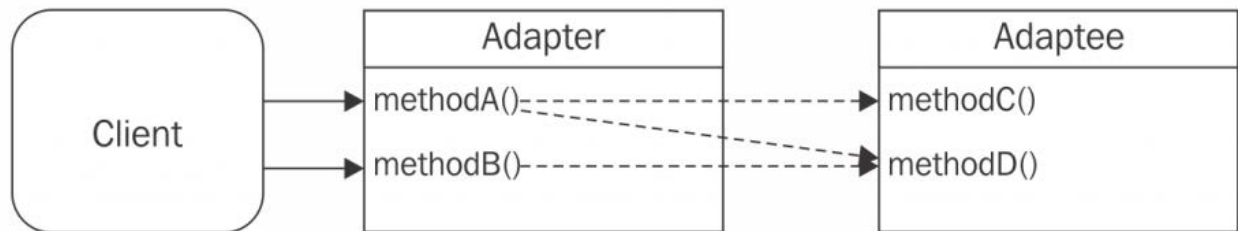


Figure 8.3: Adapter pattern schematic

Hình 8.3: Sơ đồ mẫu Adapter

In Figure 8.3, we can see how the adapter is essentially a wrapper for the adaptee, exposing a different interface. The diagram also highlights the fact that the operations of the adapter can also be a composition of one or more method invocations on the adaptee. From an implementation perspective, the most common technique is composition, where the methods of the adapter provide a bridge to the methods of the adaptee. This pattern is pretty straightforward, so let's immediately work on an example.

Trong Hình 8.3, chúng ta có thể thấy cách adapter về cơ bản là một bộ bao cho adaptee, phơi bày một giao diện khác. Sơ đồ cũng làm nổi bật thực tế rằng các thao tác của adapter cũng có thể là sự kết hợp của một hoặc nhiều lời gọi phương thức trên adaptee. Từ góc độ hiện thực, kỹ thuật phổ biến nhất là composition, nơi các phương thức của adapter cung cấp một cầu nối tới các phương thức của adaptee. Mẫu này khá đơn giản, nên hãy bắt tay ngay vào một ví dụ.

Using LevelUP through the filesystem API — Dùng LevelUP thông qua API filesystem

We are now going to build an adapter around the LevelUP API, transforming it into an interface that is compatible with the core `fs` module. In particular, we will make sure that every call to `readFile()` and `writeFile()` will translate into calls to `db.get()` and `db.put()`. This way we will be able to use a LevelUP database as a storage backend for simple filesystem operations.

Bây giờ chúng ta sẽ xây dựng một adapter quanh API của LevelUP, biến đổi nó thành một giao diện tương thích với module lõi `fs`. Cụ thể, chúng ta sẽ đảm bảo rằng mọi lời gọi tới `readFile()` và `writeFile()` sẽ chuyển thành các lời gọi tới `db.get()` và `db.put()`. Bằng cách này chúng ta sẽ có thể dùng một cơ sở dữ liệu LevelUP làm backend lưu trữ cho các thao tác filesystem đơn giản.

Let's start by creating a new module named `fs-adapter.js`. We will begin by loading the dependencies and exporting the `createFsAdapter()` factory that we are going to use to build the adapter:

Hãy bắt đầu bằng cách tạo một module mới tên là `fs-adapter.js`. Chúng ta sẽ bắt đầu bằng việc nạp các phụ thuộc và export factory `createFsAdapter()` mà chúng ta sẽ dùng để xây dựng adapter:

```

import { resolve } from 'path'
export function createFSAdapter (db) {
  return ({
    readFile (filename, options, callback) {
      // ...
    },
    writeFile (filename, contents, options, callback) {
      // ...
    }
  })
}

```

Next, we will implement the `readFile()` function inside the factory and ensure that its interface is compatible with the one of the original function from the `fs` module:

Tiếp theo, chúng ta sẽ hiện thực hàm `readFile()` bên trong factory và đảm bảo rằng giao diện của nó tương thích với giao diện của hàm gốc từ module `fs`:

```

readFile (filename, options, callback) {
  if (typeof options === 'function') {
    callback = options
    options = {}
  } else if (typeof options === 'string') {
    options = { encoding: options }
  }
  db.get(resolve(filename), {
    valueEncoding: options.encoding
  },
  (err, value) => {
    if (err) {
      if (err.type === 'NotFoundError') {
        err = new Error(`ENOENT, open "${filename}"`)
        err.code = 'ENOENT'
        err.errno = 34
        err.path = filename
      }
      return callback && callback(err)
    }
    callback && callback(null, value)
  })
}

```

In the preceding code, we had to do some extra work to make sure that the behavior of our new function is as close as possible to the original `fs.readFile()` function. The steps performed by the function are described as follows:

Trong đoạn mã trước, chúng ta đã phải làm thêm một ít việc để đảm bảo rằng hành vi của hàm mới của chúng ta càng giống hàm `fs.readFile()` gốc càng tốt. Các bước được hàm thực hiện được mô tả như sau:

1. To retrieve a file from the `db` instance, we invoke `db.get()`, using `filename` as a key, by making sure to always use its full path (using `resolve()`). We set the value of the `valueEncoding` option used by the database to be equal to any eventual encoding option received as an input.

2. If the key is not found in the database, we create an error with ENOENT as the error code, which is the code used by the original `fs` module to indicate a missing file. Any other type of error is forwarded to callback (for the scope of this example, we are adapting only the most common error condition).
3. If the key-value pair is retrieved successfully from the database, we will return the value to the caller using the callback.
 1. Để lấy một file từ thể hiện db, chúng ta gọi `db.get()`, dùng `filename` làm key, đảm bảo luôn dùng đường dẫn đầy đủ của nó (dùng `resolve()`). Chúng ta đặt giá trị của tùy chọn `valueEncoding` mà cơ sở dữ liệu dùng bằng với bất kỳ tùy chọn `encoding` nào nhận được làm đầu vào.
 2. Nếu key không được tìm thấy trong cơ sở dữ liệu, chúng ta tạo một lỗi với ENOENT làm mã lỗi, đây là mã được module `fs` gốc dùng để chỉ một file bị thiếu. Bất kỳ loại lỗi nào khác đều được chuyển tiếp tới callback (trong phạm vi của ví dụ này, chúng ta chỉ thích nghi điều kiện lỗi phổ biến nhất).
 3. Nếu cặp key-value được lấy thành công từ cơ sở dữ liệu, chúng ta sẽ trả giá trị về cho caller bằng callback.

The function that we created does not want to be a perfect replacement for the `fs.readFile()` function, but it definitely does its job in the most common situations.

Hàm chúng ta tạo không nhằm là một bản thay thế hoàn hảo cho hàm `fs.readFile()`, nhưng nó chắc chắn làm tốt việc của nó trong các tình huống phổ biến nhất.

To complete our small adapter, let's now see how to implement the `writeFile()` function:

Để hoàn thành adapter nhỏ của chúng ta, bây giờ hãy xem cách hiện thực hàm `writeFile()`:

```
writeFile (filename, contents, options, callback) {
  if (typeof options === 'function') {
    callback = options
    options = {}
  } else if (typeof options === 'string') {
    options = { encoding: options }
  }
  db.put(resolve(filename), contents, {
    valueEncoding: options.encoding
  }, callback)
}
```

As we can see, we don't have a perfect wrapper in this case either. We are ignoring some options such as file permissions (`options.mode`), and we are forwarding any error that we receive from the database as is.

Như chúng ta có thể thấy, chúng ta cũng không có một bộ bao hoàn hảo trong trường hợp này. Chúng ta đang bỏ qua một số tùy chọn như quyền file (`options.mode`), và chúng ta đang chuyển tiếp nguyên trạng bất kỳ lỗi nào nhận được từ cơ sở dữ liệu.

Our new adapter is now ready. If we now write a small test module, we can try to use it:

Adapter mới của chúng ta giờ đã sẵn sàng. Nếu bây giờ chúng ta viết một module kiểm thử nhỏ, chúng ta có thể thử dùng nó:

```
import fs from 'fs'
fs.writeFile('file.txt', 'Hello!', () => {
```

```

fs.readFile('file.txt', { encoding: 'utf8' }, (err, res) => {
  if (err) {
    return console.error(err)
  }
  console.log(res)
})
})
// try to read a missing file
fs.readFile('missing.txt', { encoding: 'utf8' }, (err, res) => {
  console.error(err)
})

```

The preceding code uses the original `fs` API to perform a few read and write operations on the filesystem, and should print something like the following to the console:

Đoạn mã trước dùng API `fs` gốc để thực hiện vài thao tác đọc và ghi trên filesystem, và sẽ in ra một thứ gì đó như sau ra console:

```

Error: ENOENT, open "missing.txt"
Hello!

```

Now, we can try to replace the `fs` module with our adapter, as follows:

Bây giờ, chúng ta có thể thử thay thế module `fs` bằng adapter của mình, như sau:

```

import { dirname, join } from 'path'
import { fileURLToPath } from 'url'
import level from 'level'
import { createFSAdapter } from './fs-adapter.js'
const __dirname = dirname(fileURLToPath(import.meta.url))
const db = level(join(__dirname, 'db'), {
  valueEncoding: 'binary'
})
const fs = createFSAdapter(db)
// ...

```

Running our program again should produce the same output, except for the fact that no parts of the file that we specified are read or written using the filesystem API directly. Instead, any operation performed using our adapter will be converted into an operation performed on a LevelUP database.

Chạy lại chương trình của chúng ta sẽ tạo ra cùng kết quả, ngoại trừ việc không có phần nào của file mà chúng ta chỉ định được đọc hoặc ghi bằng API filesystem trực tiếp. Thay vào đó, bất kỳ thao tác nào được thực hiện bằng adapter của chúng ta sẽ được chuyển đổi thành một thao tác được thực hiện trên một cơ sở dữ liệu LevelUP.

The adapter that we just created might look silly; what's the purpose of using a database in place of the real filesystem? However, we should remember that LevelUP itself has adapters that enable the database to also run in the browser. One of these adapters is `level-js` (nodejssdp.link/level-js). Now our adapter makes perfect sense. We could use something similar to allow code leveraging the `fs` module to run on both Node.js and a browser. We will soon realize that Adapter is an extremely important pattern when it comes to sharing code with the browser, as we will see in more detail in Chapter 10, **Universal JavaScript** for Web Applications.

Adapter mà chúng ta vừa tạo có thể trông ngớ ngẩn; mục đích của việc dùng một cơ sở dữ liệu thay cho filesystem thực sự là gì? Tuy nhiên, chúng ta nên nhớ rằng bản thân LevelUP

có các adapter cho phép cơ sở dữ liệu cũng chạy được trong trình duyệt. Một trong những adapter đó là `level-js` (nodejsdp.link/level-js). Bây giờ adapter của chúng ta hoàn toàn hợp lý. Chúng ta có thể dùng một thứ tương tự để cho phép mã tận dụng module `fs` chạy được trên cả Node.js và trình duyệt. Chúng ta sẽ sớm nhận ra rằng Adapter là một mẫu cực kỳ quan trọng khi nói đến việc chia sẻ mã với trình duyệt, như chúng ta sẽ thấy chi tiết hơn trong Chương 10, *Universal JavaScript for Web Applications*.

In the wild — Trong thực tế

There are plenty of real-world examples of the Adapter pattern. We've listed some of the most notable examples here for you to explore and analyze:

Có rất nhiều ví dụ thực tế về mẫu Adapter. Chúng tôi đã liệt kê một số ví dụ đáng chú ý nhất ở đây để bạn khám phá và phân tích:

- We already know that LevelUP is able to run with different storage backends, from the default LevelDB to IndexedDB in the browser. This is made possible by the various adapters that are created to replicate the internal (private) LevelUP API. Take a look at some of them to see how they are implemented at nodejsdp.link/level-stores.
- JugglingDB is a multi-database ORM and of course, multiple adapters are used to make it compatible with different databases. Take a look at some of them at nodejsdp.link/jugglingdb-adapters.
- nanoSQL (nodejsdp.link/nanosql) is a modern multi-model database abstraction library that makes heavy usage of the Adapter pattern to support a significant variety of databases.
- The perfect complement to the example that we created is `level-fileSystem` (nodejsdp.link/level-fileSystem), which is the proper implementation of the `fs` API on top of LevelUP.
 - Chúng ta đã biết rằng LevelUP có khả năng chạy với các backend lưu trữ khác nhau, từ LevelDB mặc định đến IndexedDB trong trình duyệt. Điều này được làm cho khả thi bởi các adapter khác nhau được tạo ra để tái tạo API (riêng tư) nội bộ của LevelUP. Hãy xem một số trong đó để biết cách chúng được hiện thực tại nodejsdp.link/level-stores.
 - JugglingDB là một ORM đa cơ sở dữ liệu và tất nhiên, nhiều adapter được dùng để làm cho nó tương thích với các cơ sở dữ liệu khác nhau. Hãy xem một số trong đó tại nodejsdp.link/jugglingdb-adapters.
 - nanoSQL (nodejsdp.link/nanosql) là một thư viện trừu tượng hóa cơ sở dữ liệu đa mô hình hiện đại sử dụng mạnh mẽ mẫu Adapter để hỗ trợ một loạt đáng kể các cơ sở dữ liệu.
 - Phần bổ trợ hoàn hảo cho ví dụ mà chúng ta đã tạo là `level-fileSystem` (nodejsdp.link/level-fileSystem), đây là hiện thực đúng đắn của API `fs` trên nền LevelUP.

Summary — Tóm tắt

Structural design patterns are definitely some of the most widely adopted design patterns in software engineering and it is important to be confident with them. In this chapter, we explored the Proxy, the Decorator, and the Adapter patterns and we discussed different ways to implement these in the context of Node.js.

Các mẫu thiết kế cấu trúc chắc chắn là một trong những mẫu thiết kế được áp dụng rộng rãi nhất trong kỹ thuật phần mềm và điều quan trọng là phải tự tin với chúng. Trong

chương này, chúng ta đã khám phá các mẫu Proxy, Decorator, và Adapter, và chúng ta đã bàn về các cách khác nhau để hiện thực chúng trong ngữ cảnh của Node.js.

We saw how the Proxy pattern can be a very valuable tool to control access to existing objects. In this chapter, we also mentioned how the Proxy pattern can enable different programming paradigms such as reactive programming using the Change Observer pattern.

Chúng ta đã thấy mẫu Proxy có thể là một công cụ rất giá trị để kiểm soát quyền truy cập tới các đối tượng hiện có như thế nào. Trong chương này, chúng ta cũng đã đề cập đến cách mẫu Proxy có thể cho phép các mô hình lập trình khác nhau như lập trình phản ứng (reactive programming) bằng mẫu Change Observer.

In the second part of the chapter, we found out that the Decorator pattern is an invaluable tool to be able to add additional functionality to existing objects. We saw that its implementation doesn't differ much from the Proxy pattern and we explored some examples built around the LevelDB ecosystem.

Trong phần thứ hai của chương, chúng ta đã phát hiện ra rằng mẫu Decorator là một công cụ vô giá để có thể thêm chức năng bổ sung vào các đối tượng hiện có. Chúng ta đã thấy rằng hiện thực của nó không khác mấy so với mẫu Proxy và chúng ta đã khám phá một số ví dụ được xây dựng quanh hệ sinh thái LevelDB.

Finally, we discussed the Adapter pattern, which allows us to wrap an existing object and expose its functionality through a different interface. We saw that this pattern can be useful to expose a piece of existing functionality to a component that expects a different interface. In our examples, we saw how this pattern can be used to implement an alternative storage layer that is compatible with the interface provided by the `fs` module to interact with files. Proxy, decorator and adapter are very similar, the difference between them can be appreciated from the perspective of the interface consumer: proxy provides the same interface as the wrapped object, decorator provides an enhanced interface, and adapter provides a different interface.

Cuối cùng, chúng ta đã bàn về mẫu Adapter, mẫu này cho phép chúng ta bao một đối tượng hiện có và phơi bày chức năng của nó thông qua một giao diện khác. Chúng ta đã thấy rằng mẫu này có thể hữu ích để phơi bày một phần chức năng hiện có cho một thành phần kỳ vọng một giao diện khác. Trong các ví dụ của chúng ta, chúng ta đã thấy cách mẫu này có thể được dùng để hiện thực một tầng lưu trữ thay thế tương thích với giao diện do module `fs` cung cấp để tương tác với các file. Proxy, decorator và adapter rất giống nhau, sự khác biệt giữa chúng có thể được cảm nhận từ góc độ của bên tiêu dùng giao diện: proxy cung cấp cùng giao diện như đối tượng được bao, decorator cung cấp một giao diện được nâng cao, và adapter cung cấp một giao diện khác.

In the next chapter, we will complete our journey through traditional design patterns in Node.js by exploring the category of behavioral design patterns. This category includes important patterns such as the **Strategy** pattern, the **Middleware** pattern, and the **Iterator** pattern. Are you ready to discover behavioral design patterns?

Trong chương tiếp theo, chúng ta sẽ hoàn thành hành trình của mình qua các mẫu thiết kế truyền thống trong Node.js bằng cách khám phá nhóm các mẫu thiết kế hành vi (behavioral design pattern). Nhóm này bao gồm các mẫu quan trọng như mẫu Strategy, mẫu Middleware, và mẫu Iterator. Bạn đã sẵn sàng khám phá các mẫu thiết kế hành vi chưa?

Exercises — Bài tập

- 8.1 HTTP client cache: Write a proxy for your favorite HTTP client library that caches the response of a given HTTP request, so that if you make the same request again, the response is immediately returned from the local cache, rather than being fetched from the remote URL. If you need inspiration, you can check out the `superagent-cache` module ([nodejshq.link/superagent-cache](https://nodejshq.github.io/superagent-cache/)).
 - 8.2 Timestamped logs: Create a proxy for the console object that enhances every logging function (`log()`, `error()`, `debug()`, and `info()`) by prepending the current timestamp to the message you want to print in the logs. For instance, executing `consoleProxy.log('hello')` should print something like `2020-02-18T15:59:30.699Z hello` in the console.
 - 8.3 Colored console output: Write a decorator for the console that adds the `red(message)`, `yellow(message)`, and `green(message)` methods. These methods will have to behave like `console.log(message)` except they will print the message in red, yellow, or green, respectively. In one of the exercises from the previous chapter, we already pointed you to some useful packages to create colored console output. If you want to try something different this time, have a look at `ansi-styles` ([nodejshq.link/ansi-styles](https://nodejshq.github.io/ansi-styles/)).
 - 8.4 Virtual filesystem: Modify our LevelDB filesystem adapter example to write the file data in memory rather than in LevelDB. You can use an object or a Map instance to store the key-value pairs of filenames and the associated data.
 - 8.5 The lazy **buffer**: Can you implement `createLazyBuffer(size)`, a factory function that generates a virtual proxy for a Buffer of the given size? The proxy instance should instantiate a Buffer object (effectively allocating the given amount of memory) only when `write()` is being invoked for the first time. If no attempt to write into the buffer is made, no Buffer instance should be created.
-
- 8.1 Cache cho HTTP client: Viết một proxy cho thư viện HTTP client yêu thích của bạn để lưu đệm phản hồi của một yêu cầu HTTP nhất định, sao cho nếu bạn thực hiện lại cùng yêu cầu đó, phản hồi được trả về ngay lập tức từ cache cục bộ, thay vì được lấy từ URL ở xa. Nếu bạn cần cảm hứng, bạn có thể xem module `superagent-cache` ([nodejshq.link/superagent-cache](https://nodejshq.github.io/superagent-cache/)).
 - 8.2 Log có dấu thời gian: Tạo một proxy cho đối tượng console để nâng cao mọi hàm ghi log (`log()`, `error()`, `debug()`, và `info()`) bằng cách thêm dấu thời gian hiện tại vào trước thông điệp bạn muốn in trong log. Ví dụ, thực thi `consoleProxy.log('hello')` sẽ in ra một thứ gì đó như `2020-02-18T15:59:30.699Z hello` trong console.
 - 8.3 Kết xuất console có màu: Viết một decorator cho console để thêm các phương thức `red(message)`, `yellow(message)`, và `green(message)`. Các phương thức này sẽ phải hành xử như `console.log(message)` ngoại trừ việc chúng sẽ in thông điệp tương ứng bằng màu đỏ, vàng, hoặc xanh lá. Trong một trong các bài tập từ chương trước, chúng tôi đã chỉ cho bạn một số gói hữu ích để tạo kết xuất console có màu. Nếu lần này bạn muốn thử một thứ khác, hãy xem `ansi-styles` ([nodejshq.link/ansi-styles](https://nodejshq.github.io/ansi-styles/)).
 - 8.4 Filesystem ảo: Sửa đổi ví dụ adapter filesystem LevelDB của chúng ta để ghi dữ liệu file trong bộ nhớ thay vì trong LevelDB. Bạn có thể dùng một đối tượng hoặc một thể hiện Map để lưu các cặp key-value của tên file và dữ liệu liên kết.
 - 8.5 Lazy buffer: Bạn có thể hiện thực `createLazyBuffer(size)`, một hàm factory sinh ra một virtual proxy cho một Buffer có kích thước đã cho không? Thể hiện proxy chỉ nên khởi tạo một đối tượng Buffer (thực chất là cấp phát lượng bộ nhớ đã cho) khi `write()` được gọi lần đầu tiên. Nếu không có nỗ lực ghi vào buffer nào được thực hiện, thì không nên tạo thể hiện Buffer nào.

Chapter 9: Behavioral Design Patterns — Chương 9: Các mẫu thiết kế hành vi (Behavioral Design Patterns)

In the last two chapters, we have learned patterns that aid us in the creation of objects and with building complex object structures. Now it's time to move onto another aspect of software design, which concerns the behavior of components. In this chapter, we will learn how to combine objects and how to define the way they communicate so that the behavior of the resulting structure becomes extensible, modular, reusable, and adaptable. Problems such as “How do I change parts of an algorithm at **runtime**?”, “How can I change the behavior of an object based on its **state**?”, and “How can I iterate over a collection without knowing its implementation?” are the typical kinds of problems solved by the patterns presented in this chapter.

Trong hai chương trước, chúng ta đã học các mẫu giúp tạo ra đối tượng và xây dựng những cấu trúc đối tượng phức tạp. Giờ là lúc chuyển sang một khía cạnh khác của thiết kế phần mềm, liên quan đến hành vi của các thành phần. Trong chương này, chúng ta sẽ học cách kết hợp các đối tượng và cách định nghĩa phương thức giao tiếp giữa chúng, sao cho hành vi của cấu trúc tạo thành trở nên có thể mở rộng, mô-đun hoá, tái sử dụng và thích nghi được. Những vấn đề như “Làm sao tôi thay đổi từng phần của một thuật toán tại thời điểm chạy?”, “Làm sao tôi thay đổi hành vi của một đối tượng dựa trên trạng thái của nó?” và “Làm sao tôi duyệt qua một collection mà không cần biết cách nó được hiện thực?” chính là những kiểu vấn đề tiêu biểu được giải quyết bởi các mẫu trình bày trong chương này.

You’ve already met a notable member of this category of patterns, and that is the **Observer** pattern, which we presented in Chapter 3, Callbacks and Events. The Observer pattern is one of the foundational patterns of the Node.js platform as it provides us with a simple interface for dealing with events and subscriptions, which are the life force of Node’s **event-driven** architecture.

Bạn đã từng gặp một thành viên đáng chú ý của nhóm mẫu này, đó là mẫu Observer mà chúng ta đã trình bày ở Chương 3, Callbacks and Events. Mẫu Observer là một trong những mẫu nền tảng của nền tảng Node.js, vì nó cung cấp cho chúng ta một giao diện đơn giản để xử lý sự kiện và đăng ký (subscription), vốn là sinh khí của kiến trúc hướng sự kiện (event-driven) của Node.

If you are already familiar with the Gang of Four (GoF) design patterns, in this chapter, you will witness once again how the implementation of some of those patterns can be radically different in JavaScript compared to a purer object-oriented approach. A great example of this thesis can be found

in the **Iterator** pattern, which you will meet later in the chapter. To implement the Iterator pattern, in fact, we won't need to extend any class or build any complex hierarchy. Instead, we will just need to add a special method to a class. Moreover, one particular pattern in this chapter, **Middleware**, tightly resembles another popular GoF pattern, which is the Chain of Responsibility pattern, but its implementation in Node.js has become such a standard that it can be considered a pattern of its own. Now, it's time to roll up your sleeves and get your hands dirty with some behavioral design patterns. In this chapter, you will learn about the following:

Nếu bạn đã quen thuộc với các mẫu thiết kế của Gang of Four (GoF), trong chương này bạn sẽ một lần nữa chứng kiến cách hiện thực một số mẫu đó có thể khác biệt hoàn toàn trong JavaScript so với cách tiếp cận hướng đối tượng thuần túy hơn. Một ví dụ tuyệt vời cho luận điểm này là mẫu Iterator mà bạn sẽ gặp ở phần sau của chương. Thực tế, để hiện thực mẫu Iterator, chúng ta sẽ không cần kế thừa bất kỳ lớp nào hay xây dựng bất kỳ hệ thống phân cấp phức tạp nào. Thay vào đó, ta chỉ cần thêm một phương thức đặc biệt vào một lớp. Hơn nữa, một mẫu cụ thể trong chương này, Middleware, rất giống với một mẫu GoF phổ biến khác là mẫu Chain of Responsibility, nhưng cách hiện thực của nó trong Node.js đã trở thành một chuẩn mực đến mức có thể coi nó là một mẫu riêng. Giờ thì đến lúc xắn tay áo và bắt tay vào với một số mẫu thiết kế hành vi. Trong chương này, bạn sẽ học về những nội dung sau:

- The **Strategy** pattern, which helps us change parts of a component to adapt it to specific needs
 - The State pattern, which allows us to change the behavior of a component based on its state
 - The Template pattern, which allows us to reuse the structure of a component to define new ones
 - The Iterator pattern, which provides us with a common interface to iterate over a collection
 - The Middleware pattern, which allows us to define a modular chain of processing steps
 - The Command pattern, which materializes the information required to execute a routine, allowing such information to be easily transferred, stored, and processed
- Mẫu Strategy, giúp ta thay đổi từng phần của một thành phần để thích nghi với những nhu cầu cụ thể
 - Mẫu State, cho phép ta thay đổi hành vi của một thành phần dựa trên trạng thái của nó
 - Mẫu Template, cho phép ta tái sử dụng cấu trúc của một thành phần để định nghĩa những thành phần mới
 - Mẫu Iterator, cung cấp cho ta một giao diện chung để duyệt qua một collection
 - Mẫu Middleware, cho phép ta định nghĩa một chuỗi các bước xử lý theo kiểu mô-đun
 - Mẫu Command, hiện thực hoá thông tin cần thiết để thực thi một thủ tục, cho phép thông tin đó dễ dàng được chuyển giao, lưu trữ và xử lý

Strategy — Strategy

The Strategy pattern enables an object, called the context, to support variations in its logic by extracting the variable parts into separate, interchangeable objects called strategies. The context implements the common logic of a family of algorithms, while a strategy implements the mutable parts, allowing the context to adapt its behavior depending on different factors, such as an input value, a system configuration, or user preferences.

Mẫu Strategy cho phép một đối tượng, gọi là context, hỗ trợ các biến thể trong logic của nó bằng cách tách những phần thay đổi được ra thành các đối tượng riêng biệt, có thể hoán đổi cho nhau, gọi là các strategy. Context hiện thực phần logic chung của một họ thuật toán, còn một strategy hiện thực những phần có thể thay đổi, cho phép context điều chỉnh

hành vi của mình tùy theo các yếu tố khác nhau, chẳng hạn một giá trị đầu vào, một cấu hình hệ thống, hoặc tùy chọn của người dùng.

Strategies are usually part of a family of solutions and all of them implement the same interface expected by the context. The following figure shows the situation we just described:

Các strategy thường là một phần của một họ giải pháp và tất cả chúng đều hiện thực cùng một giao diện mà context mong đợi. Hình dưới đây minh họa tình huống mà chúng ta vừa mô tả:

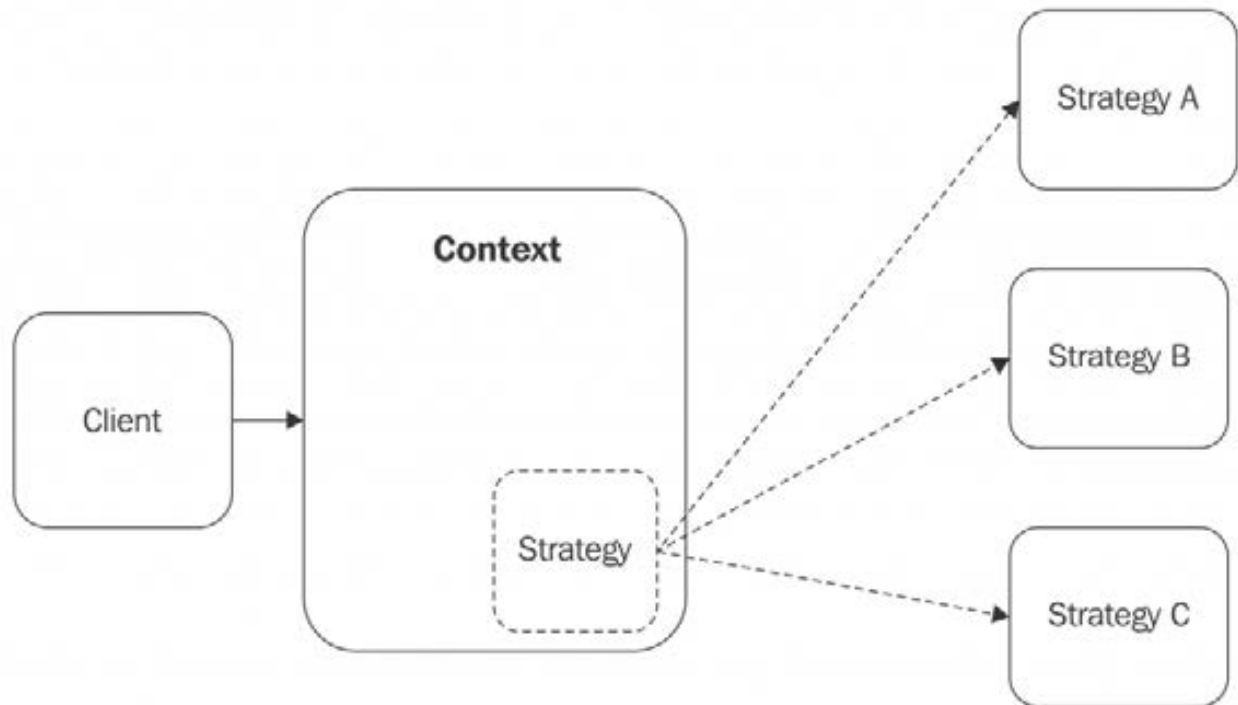


Figure 9.1: General structure of the Strategy pattern

Hình 9.1: Cấu trúc tổng quát của mẫu Strategy

Figure 9.1 shows you how the context object can plug different strategies into its structure as if they were replaceable parts of a piece of machinery. Imagine a car; its tires can be considered its strategy for adapting to different road conditions. We can fit winter tires to go on snowy roads thanks to their studs, while we can decide to fit high-performance tires for traveling mainly on motorways for a long trip. On the one hand, we don't want to change the entire car for this to be possible, and on the other, we don't want a car with eight wheels so that it can go on every possible road.

Hình 9.1 cho bạn thấy cách đối tượng context có thể gắn những strategy khác nhau vào cấu trúc của nó như thể chúng là những bộ phận thay thế được của một cỗ máy. Hãy tưởng tượng một chiếc xe hơi; lốp xe của nó có thể được xem là strategy để thích nghi với các điều kiện đường khác nhau. Ta có thể lắp lốp mùa đông để đi trên đường tuyết nhờ các đinh bám, trong khi ta có thể quyết định lắp lốp hiệu năng cao để di chuyển chủ yếu trên cao tốc trong một chuyến đi dài. Một mặt, ta không muốn phải thay cả chiếc xe để làm được điều đó, và mặt khác, ta cũng không muốn một chiếc xe tám bánh chỉ để có thể chạy trên mọi loại đường có thể.

We quickly understand how powerful this pattern is. Not only does it help with separating the concerns within a given problem, but it also enables our solution to have better flexibility and adapt

to different variations of the same problem.

Chúng ta nhanh chóng hiểu được mẫu này mạnh đến mức nào. Nó không chỉ giúp tách biệt các mối quan tâm trong một bài toán cụ thể, mà còn cho phép giải pháp của chúng ta có tính linh hoạt tốt hơn và thích nghi được với những biến thể khác nhau của cùng một bài toán.

The Strategy pattern is particularly useful in all those situations where supporting variations in the behavior of a component requires complex conditional logic (lots of `if...else` or `switch` statements) or mixing different components of the same family. Imagine an object called `Order` that represents an online order on an e-commerce website. The object has a method called `pay()` that, as it says, finalizes the order and transfers the funds from the user to the online store.

Mẫu Strategy đặc biệt hữu ích trong tất cả những tình huống mà việc hỗ trợ các biến thể trong hành vi của một thành phần đòi hỏi logic điều kiện phức tạp (rất nhiều câu lệnh `if...else` hoặc `switch`) hoặc trộn lẫn các thành phần khác nhau của cùng một họ. Hãy tưởng tượng một đối tượng tên là `Order` biểu diễn một đơn hàng trực tuyến trên một website thương mại điện tử. Đối tượng này có một phương thức tên là `pay()` mà, đúng như tên gọi, hoàn tất đơn hàng và chuyển tiền từ người dùng đến cửa hàng trực tuyến.

To support different payment systems, we have a couple of options:

Để hỗ trợ các hệ thống thanh toán khác nhau, chúng ta có một vài lựa chọn:

- Use an `if...else` statement in the `pay()` method to complete the operation based on the chosen payment option
- Delegate the logic of the payment to a strategy object that implements the logic for the specific payment gateway selected by the user
 - Dùng một câu lệnh `if...else` trong phương thức `pay()` để hoàn tất thao tác dựa trên tùy chọn thanh toán đã chọn
 - Ủy thác logic thanh toán cho một đối tượng strategy hiện thực logic cho cổng thanh toán cụ thể mà người dùng đã chọn

In the first solution, our `Order` object cannot support other payment methods unless its code is modified. Also, this can become quite complex when the number of payment options grows. Instead, using the Strategy pattern enables the `Order` object to support a virtually unlimited number of payment methods and keeps its **scope** limited to only managing the details of the user, the purchased items, and the relative price while delegating the job of completing the payment to another object.

Trong giải pháp thứ nhất, đối tượng `Order` của chúng ta không thể hỗ trợ các phương thức thanh toán khác trừ khi mã của nó được sửa đổi. Ngoài ra, điều này có thể trở nên khá phức tạp khi số lượng tùy chọn thanh toán tăng lên. Thay vào đó, dùng mẫu Strategy cho phép đối tượng `Order` hỗ trợ một số lượng phương thức thanh toán gần như vô hạn và giữ phạm vi của nó chỉ giới hạn ở việc quản lý chi tiết về người dùng, các mặt hàng đã mua và giá tương ứng, trong khi ủy thác công việc hoàn tất thanh toán cho một đối tượng khác.

Let's now demonstrate this pattern with a simple, realistic example.

Giờ hãy minh họa mẫu này bằng một ví dụ đơn giản và thực tế.

Multi-format configuration objects — Đối tượng cấu hình đa định dạng

Let's consider an object called `Config` that holds a set of configuration parameters used by an application, such as the database URL, the listening port of the server, and so on. The `Config` object should be able to provide a simple interface to access these parameters, but also a way to import

and export the configuration using persistent storage, such as a file. We want to be able to support different formats to store the configuration, for example, JSON, INI, or YAML.

Hãy xét một đối tượng tên là Config chứa một tập các tham số cấu hình được một ứng dụng sử dụng, chẳng hạn như URL của cơ sở dữ liệu, cổng lắng nghe của máy chủ, v.v. Đối tượng Config cần có khả năng cung cấp một giao diện đơn giản để truy cập các tham số này, nhưng cũng cần một cách để nhập và xuất cấu hình bằng kho lưu trữ bền vững, chẳng hạn một tập tin. Chúng ta muốn có thể hỗ trợ các định dạng khác nhau để lưu trữ cấu hình, ví dụ JSON, INI, hoặc YAML.

By applying what we learned about the Strategy pattern, we can immediately identify the variable part of the Config object, which is the functionality that allows us to serialize and deserialize the configuration. This is going to be implemented by our strategies.

Bằng cách áp dụng những gì đã học về mẫu Strategy, ta có thể ngay lập tức xác định phần thay đổi được của đối tượng Config, đó là chức năng cho phép ta tuần tự hoá (serialize) và giải tuần tự hoá (deserialize) cấu hình. Phần này sẽ được hiện thực bởi các strategy của chúng ta.

Let's create a new **module** called `config.js`, and let's define the generic part of our configuration manager:

Hãy tạo một module mới tên là `config.js`, và định nghĩa phần tổng quát của trình quản lý cấu hình:

```
import { promises as fs } from 'fs'
import objectPath from 'object-path'
export class Config {
  constructor (formatStrategy) { // (1)
    this.data = {}
    this.formatStrategy = formatStrategy
  }
  get (configPath) { // (2)
    return objectPath.get(this.data, configPath)
  }
  set (configPath, value) { // (2)
    return objectPath.set(this.data, configPath, value)
  }
  async load (filePath) { // (3)
    console.log(`Deserializing from ${filePath}`)
    this.data = this.formatStrategy.deserialize(
      await fs.readFile(filePath, 'utf-8')
    )
  }
  async save (filePath) { // (3)
    console.log(`Serializing to ${filePath}`)
    await fs.writeFile(filePath,
      this.formatStrategy.serialize(this.data))
  }
}
```

This is what's happening in the preceding code:

Đây là những gì đang diễn ra trong đoạn mã trên:

1. In the constructor, we create an instance variable called `data` to hold the configuration data. Then we also store `formatStrategy`, which represents the component that we will use to parse and serialize the data.
2. We provide two methods, `set()` and `get()`, to access the configuration properties using a dotted path notation (for example, `property.subProperty`) by leveraging a library called `object-path` (nodejsdp.link/object-path).
3. The `load()` and `save()` methods are where we delegate, respectively, the deserialization and serialization of the data to our strategy. This is where the logic of the `Config` class is altered based on the `formatStrategy` passed as an input in the constructor.

1. Trong constructor, chúng ta tạo một biến thực thể tên là `data` để chứa dữ liệu cấu hình. Sau đó ta cũng lưu `formatStrategy`, biểu diễn thành phần mà ta sẽ dùng để phân tích (parse) và tuần tự hoá dữ liệu.
2. Chúng ta cung cấp hai phương thức, `set()` và `get()`, để truy cập các thuộc tính cấu hình bằng ký pháp đường dẫn chấm (ví dụ `property.subProperty`) nhờ tận dụng một thư viện tên là `object-path` (nodejsdp.link/object-path).
3. Các phương thức `load()` và `save()` là nơi chúng ta uỷ thác, lần lượt, việc giải tuần tự hoá và tuần tự hoá dữ liệu cho strategy của mình. Đây là chỗ mà logic của lớp `Config` bị thay đổi dựa trên `formatStrategy` được truyền vào làm đầu vào trong constructor.

As we can see, this very simple and neat design allows the `Config` object to seamlessly support different file formats when loading and saving its data. The best part is that the logic to support those various formats is not hardcoded anywhere, so the `Config` class can adapt without any modification to virtually any file format, given the right strategy.

Như ta thấy, thiết kế rất đơn giản và gọn gàng này cho phép đối tượng `Config` hỗ trợ liền mạch các định dạng tập tin khác nhau khi nạp và lưu dữ liệu. Phần hay nhất là logic để hỗ trợ các định dạng khác nhau đó không bị mã hoá cứng (hardcode) ở bất kỳ đâu, nên lớp `Config` có thể thích nghi mà không cần sửa đổi với gần như bất kỳ định dạng tập tin nào, miễn là có strategy phù hợp.

To demonstrate this characteristic, let's now create a couple of format strategies in a file called `strategies.js`. Let's start with a strategy for parsing and serializing data using the INI file format, which is a widely used configuration format (more info about it here: nodejsdp.link/ini-format). For the task, we will use an npm **package** called `ini` (nodejsdp.link/ini):

Để minh hoạ đặc tính này, giờ hãy tạo một vài format strategy trong một tập tin tên là `strategies.js`. Hãy bắt đầu với một strategy để phân tích và tuần tự hoá dữ liệu dùng định dạng tập tin INI, một định dạng cấu hình được sử dụng rộng rãi (thêm thông tin tại đây: nodejsdp.link/ini-format). Cho công việc này, chúng ta sẽ dùng một gói npm tên là `ini` (nodejsdp.link/ini):

```
import ini from 'ini'
export const iniStrategy = {
  deserialize: data => ini.parse(data),
  serialize: data => ini.stringify(data)
}
```

Nothing really complicated! Our strategy simply implements the agreed interface, so that it can be used by the `Config` object.

Chẳng có gì thực sự phức tạp! Strategy của chúng ta chỉ đơn giản hiện thực giao diện đã thống nhất, để nó có thể được dùng bởi đối tượng `Config`.

Similarly, the next strategy that we are going to create allows us to support the JSON file format, widely used in JavaScript and in the web development ecosystem in general:

Tương tự, strategy tiếp theo mà chúng ta sẽ tạo cho phép ta hỗ trợ định dạng tập tin JSON, được dùng rộng rãi trong JavaScript và trong hệ sinh thái phát triển web nói chung:

```
export const jsonStrategy = {
  deserialize: data => JSON.parse(data),
  serialize: data => JSON.stringify(data, null, '  ')
}
```

Now, to show you how everything comes together, let's create a file named `index.js`, and let's try to load and save a sample configuration using different formats:

Giờ, để cho bạn thấy mọi thứ kết hợp với nhau ra sao, hãy tạo một tập tin tên là `index.js`, và thử nạp và lưu một cấu hình mẫu bằng các định dạng khác nhau:

```
import { Config } from './config.js'
import { jsonStrategy, iniStrategy } from './strategies.js'
async function main () {
  const iniConfig = new Config(iniStrategy)
  await iniConfig.load('samples/conf.ini')
  iniConfig.set('book.nodejs', 'design patterns')
  await iniConfig.save('samples/conf_mod.ini')
  const jsonConfig = new Config(jsonStrategy)
  await jsonConfig.load('samples/conf.json')
  jsonConfig.set('book.nodejs', 'design patterns')
  await jsonConfig.save('samples/conf_mod.json')
}
main()
```

Our test module reveals the core properties of the Strategy pattern. We defined only one `Config` class, which implements the common parts of our configuration manager; then, by using different strategies for serializing and deserializing data, we created different `Config` class instances supporting different file formats.

Module kiểm thử của chúng ta bộc lộ những đặc tính cốt lõi của mẫu Strategy. Ta chỉ định nghĩa duy nhất một lớp `Config`, hiện thực các phần chung của trình quản lý cấu hình, rồi bằng cách dùng những strategy khác nhau để tuần tự hoá và giải tuần tự hoá dữ liệu, ta đã tạo ra những thực thể lớp `Config` khác nhau hỗ trợ các định dạng tập tin khác nhau.

The example we've just seen showed us only one of the possible alternatives that we had for selecting a strategy. Other valid approaches might have been the following:

Ví dụ vừa rồi mới chỉ cho ta thấy một trong những phương án khả dĩ để chọn một strategy. Những cách tiếp cận hợp lệ khác có thể là như sau:

- Creating two different strategy families: One for the deserialization and the other for the serialization. This would have allowed reading from a format and saving to another.
- Dynamically selecting the strategy: Depending on the extension of the file provided, the `Config` object could have maintained a map extension $\rightarrow$ strategy and used it to select the right algorithm for the given extension.
 - Tạo hai họ strategy khác nhau: một cho việc giải tuần tự hoá và một cho việc tuần tự hoá. Cách này lẽ ra cho phép đọc từ một định dạng và lưu sang một định dạng khác.

- Chọn strategy một cách động: Tùy theo phần mở rộng của tập tin được cung cấp, đối tượng Config sẽ ra có thể duy trì một ánh xạ extension → strategy và dùng nó để chọn thuật toán phù hợp cho phần mở rộng đã cho.

As we can see, we have several options for selecting the strategy to use, and the right one only depends on your requirements and the tradeoff in terms of features and the simplicity you want to obtain.

Như ta thấy, chúng ta có vài lựa chọn để chọn strategy sử dụng, và lựa chọn đúng chỉ phụ thuộc vào yêu cầu của bạn và sự đánh đổi giữa tính năng với sự đơn giản mà bạn muốn đạt được.

Furthermore, the implementation of the pattern itself can vary a lot as well. For example, in its simplest form, the context and the strategy can both be simple functions:

Hơn nữa, bản thân cách hiện thực của mẫu cũng có thể thay đổi rất nhiều. Ví dụ, ở dạng đơn giản nhất, cả context lẫn strategy đều có thể chỉ là những hàm đơn giản:

```
function context(strategy) { ... }
```

Even though this may seem insignificant, it should not be underestimated in a programming language such as JavaScript, where functions are first-class citizens and used as much as fully-fledged objects.

Mặc dù điều này trông có vẻ không đáng kể, nó không nên bị xem nhẹ trong một ngôn ngữ lập trình như JavaScript, nơi hàm là công dân hạng nhất (first-class citizen) và được dùng nhiều như những đối tượng đầy đủ.

Between all these variations, though, what does not change is the idea behind the pattern; as always, the implementation can slightly change but the core concepts that drive the pattern are always the same.

Tuy nhiên, giữa tất cả những biến thể này, điều không thay đổi chính là ý tưởng đằng sau mẫu; như mọi khi, cách hiện thực có thể thay đổi đôi chút nhưng các khái niệm cốt lõi dẫn dắt mẫu thì luôn như nhau.

The structure of the Strategy pattern may look similar to that of the **Adapter** pattern. However, there is a substantial difference between the two. The adapter object does not add any behavior to the adaptee; it just makes it available under another interface. This can also require some extra logic to be implemented to convert one interface into another, but this logic is limited to this task only. In the Strategy pattern, however, the context and the strategy implement two different parts of an algorithm and therefore both implement some kind of logic and both are essential to build the final algorithm (when combined together).

Cấu trúc của mẫu Strategy có thể trông giống với cấu trúc của mẫu Adapter. Tuy nhiên, có một khác biệt cốt yếu giữa hai mẫu. Đối tượng adapter không thêm bất kỳ hành vi nào cho adaptee; nó chỉ làm cho adaptee khả dụng dưới một giao diện khác. Điều này cũng có thể đòi hỏi hiện thực thêm một chút logic để chuyển đổi giao diện này thành giao diện khác, nhưng logic đó chỉ giới hạn ở nhiệm vụ này. Trong mẫu Strategy thì khác, context và strategy hiện thực hai phần khác nhau của một thuật toán và do đó cả hai đều hiện thực một loại logic nào đó và cả hai đều thiết yếu để xây dựng nên thuật toán cuối cùng (khi kết hợp với nhau).

In the wild — Trong thực tế

Passport (nodejssdp.link/passportjs) is an authentication framework for Node.js, which allows a web server to support different authentication schemes. With Passport, we can provide a login with

Facebook or login with Twitter functionality to our web application with minimal effort. Passport uses the Strategy pattern to separate the common logic used during an authentication process from the parts that can change, namely the actual authentication step. For example, we might want to use OAuth in order to obtain an access token to access a Facebook or Twitter profile, or simply use a local database to verify a username/ password pair. For Passport, these are all different strategies for completing the authentication process and, as we can imagine, this allows the library to support a virtually unlimited number of authentication services. Take a look at the number of different authentication providers supported at [nodejsdp.link/passport- strategies](https://nodejsdp.link/passport-strategies) to get an idea of what the Strategy pattern can do.

Passport (nodejsdp.link/passportjs) là một framework xác thực cho Node.js, cho phép một máy chủ web hỗ trợ các cơ chế xác thực khác nhau. Với Passport, ta có thể cung cấp chức năng đăng nhập bằng Facebook hoặc đăng nhập bằng Twitter cho ứng dụng web của mình với nỗ lực tối thiểu. Passport dùng mẫu Strategy để tách logic chung được sử dụng trong một quá trình xác thực ra khỏi những phần có thể thay đổi, cụ thể là bước xác thực thực tế. Ví dụ, ta có thể muốn dùng OAuth để lấy một access token nhằm truy cập một hồ sơ Facebook hoặc Twitter, hoặc đơn giản dùng một cơ sở dữ liệu cục bộ để xác minh cặp username/password. Đối với Passport, đây đều là những strategy khác nhau để hoàn tất quá trình xác thực và, như ta có thể hình dung, điều này cho phép thư viện hỗ trợ một số lượng dịch vụ xác thực gần như vô hạn. Hãy nhìn vào số lượng nhà cung cấp xác thực khác nhau được hỗ trợ tại [nodejsdp.link/passport- strategies](https://nodejsdp.link/passport-strategies) để hình dung mẫu Strategy có thể làm được gì.

State — State

The State pattern is a specialization of the Strategy pattern where the strategy changes depending on the state of the context.

Mẫu State là một dạng chuyên biệt hoá của mẫu Strategy, trong đó strategy thay đổi tùy theo trạng thái của context.

We have seen in the previous section how a strategy can be selected based on different variables such as a configuration property or an input parameter, and once this selection is done, the strategy remains unchanged for the rest of the lifespan of the context object. In the State pattern, instead, the strategy (also called the state in this circumstance) is dynamic and can change during the lifetime of the context, thus allowing its behavior to adapt depending on its internal state.

Ở phần trước, chúng ta đã thấy cách một strategy có thể được chọn dựa trên các biến khác nhau như một thuộc tính cấu hình hoặc một tham số đầu vào, và một khi lựa chọn này được thực hiện, strategy giữ nguyên không đổi trong suốt phần đời còn lại của đối tượng context. Trong mẫu State thì khác, strategy (trong trường hợp này còn gọi là state) mang tính động và có thể thay đổi trong suốt vòng đời của context, qua đó cho phép hành vi của nó thích nghi tùy theo trạng thái nội tại.

The following figure shows us a representation of the pattern:

Hình dưới đây cho ta một biểu diễn của mẫu:

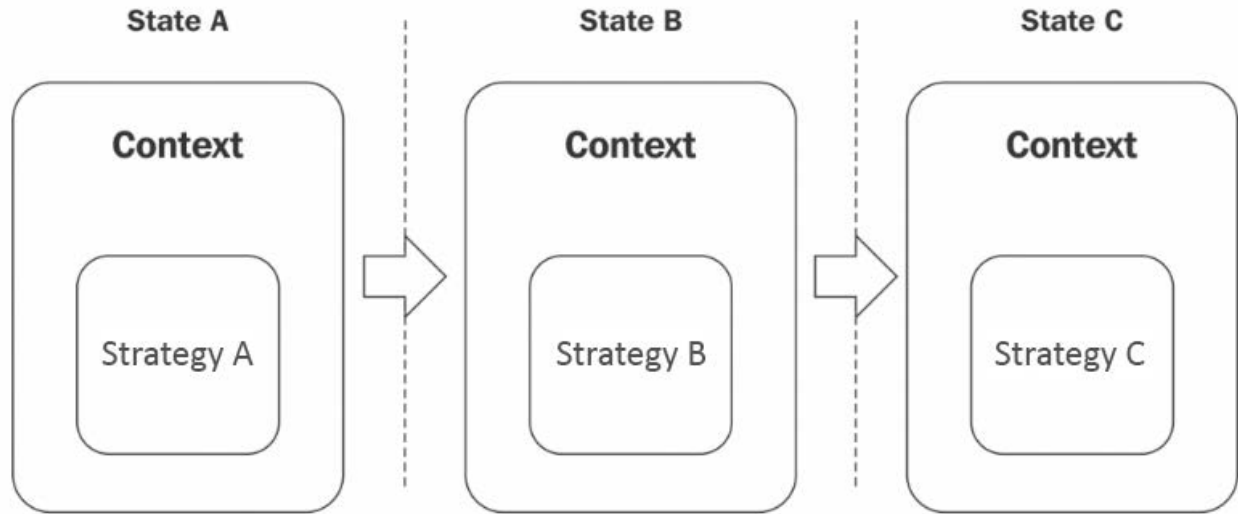


Figure 9.2: The State pattern Figure 9.2 shows how a context object transitions through three states (A, B, and C). With the State pattern, at each different context state, we select a different strategy. This means that the context object will adopt a different behavior based on the state it's in. To make this easier to understand, let's consider an example: imagine we have a hotel booking system and an object called Reservation that models a room reservation. This is a typical situation where we have to adapt the behavior of an object based on its state.

Hình 9.2: Mẫu State. Hình 9.2 cho thấy cách một đối tượng context chuyển tiếp qua ba trạng thái (A, B và C). Với mẫu State, tại mỗi trạng thái khác nhau của context, ta chọn một strategy khác nhau. Điều này có nghĩa là đối tượng context sẽ áp dụng một hành vi khác nhau dựa trên trạng thái mà nó đang ở. Để dễ hiểu hơn, hãy xét một ví dụ: tưởng tượng ta có một hệ thống đặt phòng khách sạn và một đối tượng tên là Reservation mô hình hoá việc đặt một phòng. Đây là một tình huống điển hình mà ta phải điều chỉnh hành vi của một đối tượng dựa trên trạng thái của nó.

Consider the following series of events:

Hãy xét chuỗi sự kiện sau:

- When the reservation is initially created, the user can confirm (using a method called `confirm()`) the reservation. Of course, they cannot cancel it (using `cancel()`), because it's still not confirmed (the caller would receive an exception, for example). They can, however, delete it (using `delete()`) if they change their mind before buying.
 - Once the reservation is confirmed, using the `confirm()` method again does not make any sense; however, now it should be possible to cancel the reservation but no longer delete it, because it has to be kept for the records.
 - On the day before the reservation date, it should not be possible to cancel the reservation anymore; it's too late for that.
- Khi reservation mới được tạo ban đầu, người dùng có thể xác nhận (dùng phương thức tên là `confirm()`) reservation. Tất nhiên, họ không thể huỷ nó (dùng `cancel()`), vì nó vẫn chưa được xác nhận (người gọi sẽ nhận một exception, chẳng hạn). Tuy nhiên họ có thể xoá nó (dùng `delete()`) nếu họ đổi ý trước khi mua.
 - Một khi reservation đã được xác nhận, việc dùng lại phương thức `confirm()` không còn ý nghĩa gì; tuy nhiên, giờ đây thì việc huỷ reservation phải khả thi nhưng không còn được xoá nữa, vì nó phải được giữ lại để lưu hồ sơ.

- Vào ngày trước ngày đặt phòng, việc huỷ reservation không còn khả thi nữa; đã quá muộn cho điều đó.

Now, imagine that we have to implement the reservation system that we just described in one monolithic object. We can already picture all the `if . . . else` or `switch` statements that we would have to write to enable/disable each action depending on the state of the reservation.

Giờ, hãy tưởng tượng rằng ta phải hiện thực hệ thống đặt phòng vừa mô tả trong một đối tượng nguyên khối (monolithic). Ta đã có thể hình dung ra tất cả những câu lệnh `if . . . else` hoặc `switch` mà ta sẽ phải viết để bật/tắt từng hành động tùy theo trạng thái của reservation.

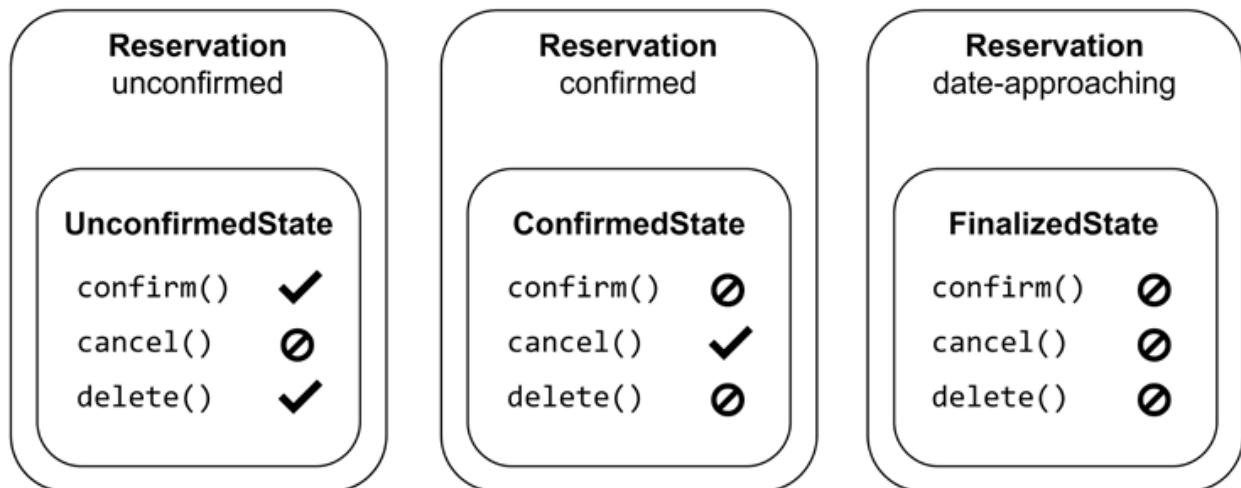


Figure 9.3: An example application of the State pattern

Hình 9.3: Một ví dụ ứng dụng của mẫu State

As illustrated in Figure 9.3, the State pattern is, instead, perfect in this situation: there would be three strategies, all implementing the three methods described (`confirm()`, `cancel()`, and `delete()`) and each one implementing only one behavior—the one corresponding to the modeled state. By using this pattern, it should be very easy for the Reservation object to switch from one behavior to another; this would simply require the activation of a different strategy (state object) on each state change. The state transition can be initiated and controlled by the context object, by the client code, or by the state objects themselves. This last option usually provides the best results in terms of flexibility and decoupling, as the context does not have to know about all the possible states and how to transition between them.

Như được minh họa trong Hình 9.3, mẫu State, ngược lại, lại hoàn hảo cho tình huống này: sẽ có ba strategy, tất cả đều hiện thực ba phương thức đã mô tả (`confirm()`, `cancel()`, và `delete()`) và mỗi cái chỉ hiện thực một hành vi—hành vi tương ứng với trạng thái được mô hình hoá. Bằng cách dùng mẫu này, việc đối tượng Reservation chuyển từ hành vi này sang hành vi khác sẽ rất dễ; điều này chỉ đơn giản đòi hỏi kích hoạt một strategy khác (đối tượng state) tại mỗi lần đổi trạng thái. Việc chuyển trạng thái có thể được khởi tạo và điều khiển bởi đối tượng context, bởi mã client, hoặc bởi chính các đối tượng state. Lựa chọn cuối cùng này thường mang lại kết quả tốt nhất về mặt linh hoạt và giảm phụ thuộc (decoupling), vì context không cần phải biết về tất cả các trạng thái khả dĩ và cách chuyển tiếp giữa chúng.

Let's now work on a more concrete example so that we can apply what we learned about the State pattern.

Giờ hãy làm việc với một ví dụ cụ thể hơn để ta có thể áp dụng những gì đã học về mẫu State.

Implementing a basic failsafe socket — Hiện thực một socket failsafe cơ bản

Let's build a TCP client socket that does not fail when the connection with the server is lost; instead, we want to queue all the data sent during the time in which the server is offline and then try to send it again as soon as the connection is reestablished. We want to leverage this socket in the context of a simple monitoring system, where a set of machines sends some statistics about their resource utilization at regular intervals. If the server that collects these resources goes down, our socket will continue to queue the data locally until the server comes back online.

Hãy xây dựng một TCP client socket mà không thất bại khi kết nối với máy chủ bị mất; thay vào đó, ta muốn xếp vào hàng đợi tất cả dữ liệu được gửi trong khoảng thời gian máy chủ offline rồi thử gửi lại ngay khi kết nối được thiết lập lại. Ta muốn tận dụng socket này trong bối cảnh một hệ thống giám sát đơn giản, nơi một tập các máy gửi một số thống kê về mức sử dụng tài nguyên của chúng theo những khoảng thời gian đều đặn. Nếu máy chủ thu thập các tài nguyên này bị sập, socket của chúng ta sẽ tiếp tục xếp dữ liệu vào hàng đợi cục bộ cho đến khi máy chủ trực tuyến trở lại.

Let's start by creating a new module called `failsafeSocket.js` that defines our context object:

Hãy bắt đầu bằng cách tạo một module mới tên là `failsafeSocket.js` định nghĩa đối tượng context của chúng ta:

```
import { OfflineState } from './offlineState.js'
import { OnlineState } from './onlineState.js'
export class FailsafeSocket {
  constructor (options) { // (1)
    this.options = options
    this.queue = []
    this.currentState = null
    this.socket = null
    this.states = {
      offline: new OfflineState(this),
      online: new OnlineState(this)
    }
    this.changeState('offline')
  }
  changeState (state) { // (2)
    console.log(`Activating state: ${state}`)
    this.currentState = this.states[state]
    this.currentState.activate()
  }
  send (data) { // (3)
    this.currentState.send(data)
  }
}
```

The `FailsafeSocket` class is made of three main elements:

Lớp `FailsafeSocket` được tạo thành từ ba thành phần chính:

1. The constructor initializes various data structures, including the queue that will contain any data sent while the socket is offline. Also, it creates a set of two states: one for implementing the behavior of the socket while it's offline, and another one when the socket is online.
2. The `changeState()` method is responsible for transitioning from one state to another. It simply updates the `currentState` instance variable and calls `activate()` on the target state.
3. The `send()` method contains the main functionality of the `FailsafeSocket` class. This is where we want to have a different behavior based on the offline/online state. As we can see, this is done by delegating the operation to the currently active state. Let's now see what the two states look like, starting from the `offlineState.js` module:

1. Constructor khởi tạo nhiều cấu trúc dữ liệu khác nhau, bao gồm hàng đợi sẽ chứa bất kỳ dữ liệu nào được gửi trong khi socket offline. Ngoài ra, nó tạo một tập gồm hai state: một để hiện thực hành vi của socket khi offline, và một khi socket online.
2. Phương thức `changeState()` chịu trách nhiệm chuyển tiếp từ trạng thái này sang trạng thái khác. Nó chỉ đơn giản cập nhật biến thực thể `currentState` và gọi `activate()` trên trạng thái đích.
3. Phương thức `send()` chứa chức năng chính của lớp `FailsafeSocket`. Đây là nơi ta muốn có một hành vi khác nhau dựa trên trạng thái offline/online. Như ta thấy, điều này được thực hiện bằng cách ủy thác thao tác cho trạng thái đang được kích hoạt. Giờ hãy xem hai trạng thái trông như thế nào, bắt đầu từ module `offlineState.js`:

```
import jsonOverTcp from 'json-over-tcp-2' // (1)
export class OfflineState {
  constructor (failsafeSocket) {
    this.failsafeSocket = failsafeSocket
  }
  send (data) { // (2)
    this.failsafeSocket.queue.push(data)
  }
  activate () { // (3)
    const retry = () => {
      setTimeout(() => this.activate(), 1000)
    }
    console.log('Trying to connect...')
    this.failsafeSocket.socket = jsonOverTcp.connect(
      this.failsafeSocket.options,
      () => {
        console.log('Connection established')
        this.failsafeSocket.socket.removeListener('error', retry)
        this.failsafeSocket.changeState('online')
      }
    )
    this.failsafeSocket.socket.once('error', retry)
  }
}
```

The module that we just created is responsible for managing the behavior of the socket while it's offline. This is how it works:

Module mà ta vừa tạo chịu trách nhiệm quản lý hành vi của socket khi nó offline. Đây là cách nó hoạt động:

1. Instead of using a raw TCP socket, we will use a little library called `json-over-tcp-2`

(nodejsdp.link/json-over-tcp-2). This will greatly simplify our work since the library will take care of all the parsing and formatting of the data going through the socket into JSON objects.

2. The `send()` method is only responsible for queuing any data it receives. We are assuming that we are offline, so we'll save those data objects for later. That's all we need to do here.
3. The `activate()` method tries to establish a connection with the server using the `json-over-tcp-2` socket. If the operation fails, it tries again after one second. It continues trying until a valid connection is established, in which case the state of `failsafeSocket` is transitioned to online.

1. Thay vì dùng một TCP socket thô, chúng ta sẽ dùng một thư viện nhỏ tên là `json-over-tcp-2` (nodejsdp.link/json-over-tcp-2). Điều này sẽ đơn giản hoá đáng kể công việc của ta vì thư viện sẽ lo toàn bộ việc phân tích và định dạng dữ liệu đi qua socket thành các đối tượng JSON.
2. Phương thức `send()` chỉ chịu trách nhiệm xếp vào hàng đợi bất kỳ dữ liệu nào nó nhận được. Ta đang giả định rằng ta đang offline, nên ta sẽ lưu các đối tượng dữ liệu đó để dùng sau. Đó là tất cả những gì ta cần làm ở đây.
3. Phương thức `activate()` cố gắng thiết lập một kết nối với máy chủ bằng socket `json-over-tcp-2`. Nếu thao tác thất bại, nó thử lại sau một giây. Nó tiếp tục thử cho đến khi một kết nối hợp lệ được thiết lập, trong trường hợp đó trạng thái của `failsafeSocket` được chuyển sang online.

Next, let's create the `onlineState.js` module, which is where we will implement the `OnlineState` class:

Tiếp theo, hãy tạo module `onlineState.js`, đây là nơi ta sẽ hiện thực lớp `OnlineState`:

```
export class OnlineState {
  constructor (failsafeSocket) {
    this.failsafeSocket = failsafeSocket
    this.hasDisconnected = false
  }
  send (data) { // (1)
    this.failsafeSocket.queue.push(data)
    this._safeWrite(data)
  }
  _safeWrite (data) { // (2)
    this.failsafeSocket.socket.write(data, (err) => {
      if (!this.hasDisconnected && !err) {
        this.failsafeSocket.queue.shift()
      }
    })
  }
  activate () { // (3)
    this.hasDisconnected = false
    for (const data of this.failsafeSocket.queue) {
      this._safeWrite(data)
    }
    this.failsafeSocket.socket.once('error', () => {
      this.hasDisconnected = true
      this.failsafeSocket.changeState('offline')
    })
  }
}
```

```
}  
}
```

The `OnlineState` class models the behavior of the `FailsafeSocket` when there is an active connection with the server. This is how it works:

Lớp `OnlineState` mô hình hoá hành vi của `FailsafeSocket` khi có một kết nối đang hoạt động với máy chủ. Đây là cách nó hoạt động:

1. The `send()` method queues the data and then immediately tries to write it directly into the socket, as we assume that we are online. It'll use the internal `_safeWrite()` method to do that.
2. The `_safeWrite()` method tries to write the data into the socket **writable stream** (see the official docs at [nodejsdp.link/writable-write](https://nodejs.org/en/docs/guides/writable-stream/)) and it waits for the data to be written into the underlying resource. If no errors are returned and if the socket didn't disconnect in the meantime, it means that the data was sent successfully and therefore we remove it from the queue.
3. The `activate()` method flushes any data that was queued while the socket was offline and it also starts listening for any error event; we will take this as a symptom that the socket went offline (for simplicity). When this happens, we transition to the offline state.

1. Phương thức `send()` xếp dữ liệu vào hàng đợi rồi ngay lập tức cố gắng ghi nó trực tiếp vào socket, vì ta giả định rằng ta đang online. Nó sẽ dùng phương thức nội bộ `_safeWrite()` để làm điều đó.
2. Phương thức `_safeWrite()` cố gắng ghi dữ liệu vào writable stream của socket (xem tài liệu chính thức tại [nodejsdp.link/writable-write](https://nodejs.org/en/docs/guides/writable-stream/)) và nó chờ dữ liệu được ghi vào tài nguyên bên dưới. Nếu không có lỗi nào được trả về và nếu socket không bị ngắt kết nối trong lúc đó, điều đó có nghĩa là dữ liệu đã được gửi thành công và do đó ta xoá nó khỏi hàng đợi.
3. Phương thức `activate()` xả (flush) bất kỳ dữ liệu nào đã được xếp vào hàng đợi trong khi socket offline và nó cũng bắt đầu lắng nghe bất kỳ sự kiện error nào; ta sẽ xem đây là dấu hiệu cho thấy socket đã offline (để cho đơn giản). Khi điều này xảy ra, ta chuyển sang trạng thái offline.

That's it for our `FailsafeSocket`. Now we are ready to build a sample client and a server to try it out. Let's put the server code in a module named `server.js`:

Vậy là xong phần `FailsafeSocket` của chúng ta. Giờ ta đã sẵn sàng xây dựng một client mẫu và một máy chủ để thử nó. Hãy đặt mã máy chủ vào một module tên là `server.js`:

```
import jsonOverTcp from 'json-over-tcp-2'  
const server = jsonOverTcp.createServer({ port: 5000 })  
server.on('connection', socket => {  
  socket.on('data', data => {  
    console.log('Client data', data)  
  })  
})  
server.listen(5000, () => console.log('Server started'))
```

Then, the client-side code, which is what we are really interested in, goes into `client.js`:

Sau đó, mã phía client, vốn là điều ta thực sự quan tâm, đặt vào `client.js`:

```
import { FailsafeSocket } from './failsafeSocket.js'  
const failsafeSocket = new FailsafeSocket({ port: 5000 })  
setInterval(() => {
```

```
// send current memory usage
failsafeSocket.send(process.memoryUsage())
}, 1000)
```

Our server simply prints to the console any JSON message it receives, while our clients are sending a measurement of their memory utilization every second, leveraging a `FailsafeSocket` object.

Máy chủ của chúng ta chỉ đơn giản in ra console bất kỳ thông điệp JSON nào nó nhận được, trong khi các client của ta gửi một số đo về mức sử dụng bộ nhớ của chúng mỗi giây, tận dụng một đối tượng `FailsafeSocket`.

To try the small system that we built, we should run both the client and the server, then we can test the features of `failsafeSocket` by stopping and then restarting the server. We should see that the state of the client changes between `online` and `offline` and that any memory measurement collected while the server is offline is queued and then resent as soon as the server goes back online.

Để thử hệ thống nhỏ mà ta đã xây dựng, ta nên chạy cả client lẫn máy chủ, rồi ta có thể kiểm thử các tính năng của `failsafeSocket` bằng cách dừng rồi khởi động lại máy chủ. Ta sẽ thấy trạng thái của client thay đổi giữa `online` và `offline` và rằng bất kỳ số đo bộ nhớ nào được thu thập trong khi máy chủ `offline` đều được xếp vào hàng đợi rồi gửi lại ngay khi máy chủ trực tuyến trở lại.

This sample should be a clear demonstration of how the State pattern can help increase the modularity and readability of a component that has to adapt its behavior depending on its state.

Ví dụ này nên là một minh họa rõ ràng về cách mẫu State có thể giúp tăng tính mô-đun và khả năng đọc hiểu của một thành phần phải điều chỉnh hành vi tùy theo trạng thái của nó.

The `FailsafeSocket` class that we built in this section is only for demonstrating the State pattern and doesn't want to be a complete and 100% reliable solution for handling connectivity issues with TCP sockets. For example, we are not verifying that all the data written into the socket **stream** is received by the server, which would require some more code not strictly related to the pattern that we wanted to describe. For a production alternative, you can count on ZeroMQ (nodejsdp.link/zeromq). We'll talk about some patterns using ZeroMQ later in the book in Chapter 13, Messaging and Integration Patterns.

Lớp `FailsafeSocket` mà ta xây dựng trong phần này chỉ để minh họa mẫu State và không nhằm trở thành một giải pháp hoàn chỉnh và đáng tin cậy 100% để xử lý các vấn đề kết nối với TCP socket. Ví dụ, ta không kiểm chứng rằng tất cả dữ liệu được ghi vào stream của socket đều được máy chủ nhận, điều này sẽ đòi hỏi thêm một ít mã không liên quan chặt chẽ đến mẫu mà ta muốn mô tả. Đối với một giải pháp thay thế cấp độ production, bạn có thể trông cậy vào ZeroMQ (nodejsdp.link/zeromq). Chúng ta sẽ nói về một số mẫu dùng ZeroMQ ở phần sau của cuốn sách trong Chương 13, Messaging and Integration Patterns.

Template — Template

The next pattern that we are going to analyze is called Template and it has a lot in common with the Strategy pattern. The Template pattern defines an abstract class that implements the skeleton (representing the common parts) of a component, where some of its steps are left undefined. Subclasses can then fill the gaps in the component by implementing the missing parts, called template methods. The intent of this pattern is to make it possible to define a family of classes that

are all variations of a family of components. The following UML diagram shows the structure that we just described:

Mẫu tiếp theo mà ta sẽ phân tích được gọi là Template và nó có rất nhiều điểm chung với mẫu Strategy. Mẫu Template định nghĩa một lớp trừu tượng hiện thực bộ khung (đại diện cho các phần chung) của một thành phần, trong đó một số bước của nó được để chưa định nghĩa. Các lớp con sau đó có thể lấp vào những chỗ trống của thành phần bằng cách hiện thực những phần còn thiếu, gọi là các template method. Mục đích của mẫu này là làm cho việc định nghĩa một họ lớp đều là các biến thể của một họ thành phần trở nên khả thi. Sơ đồ UML dưới đây cho thấy cấu trúc mà ta vừa mô tả:

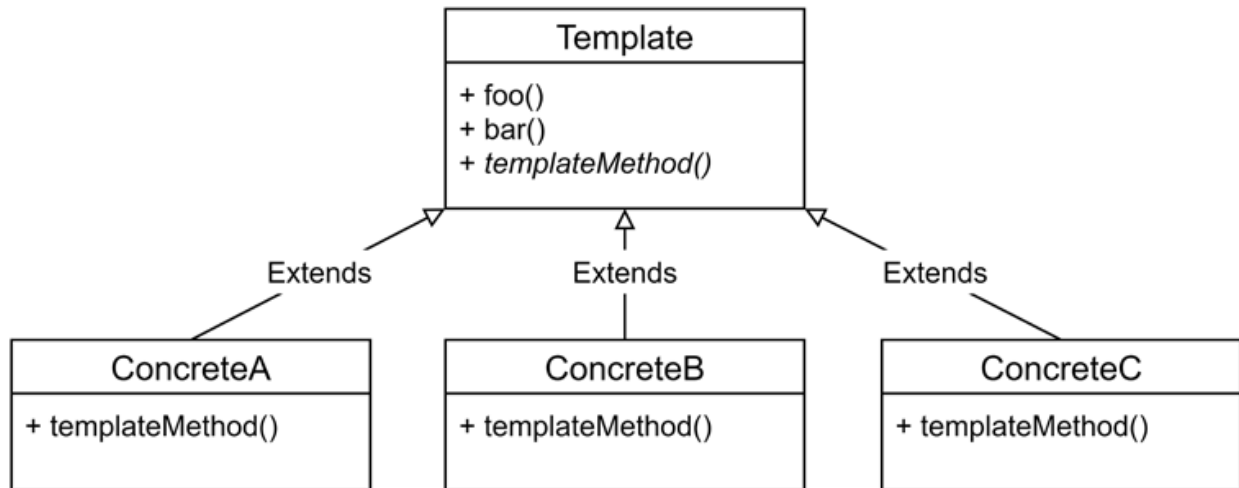


Figure 9.4: UML diagram of the Template pattern

Hình 9.4: Sơ đồ UML của mẫu Template

The three concrete classes shown in Figure 9.4, extend the template class and provide an implementation for `templateMethod()`, which is abstract or pure virtual, to use C++ terminology. In JavaScript, we don't have a formal way to define abstract classes, so all we can do is leave the method undefined or assign it to a function that always throws an exception, indicating that the method has to be implemented. The Template pattern can be considered a more traditionally object-oriented pattern than the other patterns we have seen so far, because inheritance is a core part of its implementation.

Ba lớp cụ thể được thể hiện trong Hình 9.4 kế thừa lớp template và cung cấp một hiện thực cho `templateMethod()`, vốn là trừu tượng hoặc pure virtual, theo thuật ngữ C++. Trong JavaScript, ta không có một cách chính thức để định nghĩa các lớp trừu tượng, nên tất cả những gì ta có thể làm là để phương thức đó chưa được định nghĩa hoặc gán nó cho một hàm luôn ném ra một exception, cho biết rằng phương thức này phải được hiện thực. Mẫu Template có thể được coi là một mẫu mang tính hướng đối tượng truyền thống hơn so với các mẫu mà ta đã thấy cho đến giờ, vì kế thừa là một phần cốt lõi của cách hiện thực nó.

The purpose of Template and Strategy is very similar, but the main difference between the two lies in their structure and implementation. Both allow us to change the variable parts of a component while reusing the common parts. However, while Strategy allows us to do it dynamically at runtime, with Template, the complete component is determined the moment the concrete class is defined. Under these assumptions, the Template pattern might be more suitable in those circumstances where we want to create prepackaged variations of a component. As always, the choice between one pattern and the other is up to the developer, who has to consider the various pros and cons for each use case.

Mục đích của Template và Strategy rất giống nhau, nhưng khác biệt chính giữa hai mẫu nằm ở cấu trúc và cách hiện thực của chúng. Cả hai đều cho phép ta thay đổi những phần thay đổi được của một thành phần trong khi tái sử dụng các phần chung. Tuy nhiên, trong khi Strategy cho phép ta làm điều đó một cách động tại thời điểm chạy, thì với Template, thành phần hoàn chỉnh được xác định ngay khi lớp cụ thể được định nghĩa. Dưới những giả định này, mẫu Template có thể phù hợp hơn trong những hoàn cảnh mà ta muốn tạo ra các biến thể đóng gói sẵn của một thành phần. Như mọi khi, việc chọn mẫu này hay mẫu kia là tùy thuộc nhà phát triển, người phải cân nhắc những điểm lợi và hại khác nhau cho từng trường hợp sử dụng.

Let's now work on an example.

Giờ hãy làm việc với một ví dụ.

A configuration manager template — Một template trình quản lý cấu hình

To have a better idea of the differences between Strategy and Template, let's now reimplement the Config object that we defined in the Strategy pattern section, but this time using Template. As in the previous version of the Config object, we want to have the ability to load and save a set of configuration properties using different file formats.

Để có cái nhìn rõ hơn về sự khác biệt giữa Strategy và Template, giờ hãy tái hiện thực đối tượng Config mà ta đã định nghĩa ở phần mẫu Strategy, nhưng lần này dùng Template. Như trong phiên bản trước của đối tượng Config, ta muốn có khả năng nạp và lưu một tập các thuộc tính cấu hình bằng các định dạng tập tin khác nhau.

Let's start by defining the template class. We will call it ConfigTemplate:

Hãy bắt đầu bằng cách định nghĩa lớp template. Ta sẽ gọi nó là ConfigTemplate:

```
import { promises as fsPromises } from 'fs'
import objectPath from 'object-path'
export class ConfigTemplate {
  async load (file) {
    console.log(`Deserializing from ${file}`)
    this.data = this._deserialize(
      await fsPromises.readFile(file, 'utf-8'))
  }
  async save (file) {
    console.log(`Serializing to ${file}`)
    await fsPromises.writeFile(file, this._serialize(this.data))
  }
  get (path) {
    return objectPath.get(this.data, path)
  }
  set (path, value) {
    return objectPath.set(this.data, path, value)
  }
  _serialize () {
    throw new Error('_serialize() must be implemented')
  }
  _deserialize () {
    throw new Error('_deserialize() must be implemented')
  }
}
```

```
}  
}
```

The ConfigTemplate class implements the common parts of the configuration management logic, namely setting and getting properties, plus loading and saving it to the disk. However, it leaves the implementation of `_serialize()` and `_deserialize()` open; those are in fact our template methods, which will allow the creation of concrete Config classes supporting specific configuration formats. The underscore at the beginning of the template methods' names indicates that they are for internal use only, an easy way to flag protected methods. Since in JavaScript we cannot declare a method as abstract, we simply define them as stubs, throwing an error if they are invoked (in other words, if they are not overridden by a concrete subclass).

Lớp ConfigTemplate hiện thực các phần chung của logic quản lý cấu hình, cụ thể là đặt và lấy thuộc tính, cộng với việc nạp và lưu nó vào đĩa. Tuy nhiên, nó để ngỏ việc hiện thực `_serialize()` và `_deserialize()`; đó thực ra là các template method của ta, sẽ cho phép tạo ra các lớp Config cụ thể hỗ trợ những định dạng cấu hình cụ thể. Dấu gạch dưới ở đầu tên các template method cho biết chúng chỉ dùng nội bộ, một cách dễ dàng để đánh dấu các phương thức protected. Vì trong JavaScript ta không thể khai báo một phương thức là trừu tượng, ta chỉ đơn giản định nghĩa chúng như những stub, ném ra một lỗi nếu chúng được gọi (nói cách khác, nếu chúng không bị ghi đè bởi một lớp con cụ thể).

Let's now create a concrete class using our template, for example, one that allows us to load and save the configuration using the JSON format:

Giờ hãy tạo một lớp cụ thể dùng template của chúng ta, ví dụ một lớp cho phép ta nạp và lưu cấu hình bằng định dạng JSON:

```
import { ConfigTemplate } from './configTemplate.js'  
export class JsonConfig extends ConfigTemplate {  
  _deserialize (data) {  
    return JSON.parse(data)  
  }  
  _serialize (data) {  
    return JSON.stringify(data, null, '  ')  
  }  
}
```

The JsonConfig class extends our template class, ConfigTemplate, and provides a concrete implementation for the `_deserialize()` and `_serialize()` methods.

Lớp JsonConfig kế thừa lớp template của chúng ta, ConfigTemplate, và cung cấp một hiện thực cụ thể cho các phương thức `_deserialize()` và `_serialize()`.

Similarly, we can implement an IniConfig class supporting the .ini file format using the same template class:

Tương tự, ta có thể hiện thực một lớp IniConfig hỗ trợ định dạng tập tin .ini dùng cùng lớp template:

```
import { ConfigTemplate } from './configTemplate.js'  
import ini from 'ini'  
export class IniConfig extends ConfigTemplate {  
  _deserialize (data) {  
    return ini.parse(data)  
  }  
}
```



```

_serialize (data) {
  return ini.stringify(data)
}
}

```

Now we can use our concrete configuration manager classes to load and save some configuration data:

Giờ ta có thể dùng các lớp trình quản lý cấu hình cụ thể của mình để nạp và lưu một số dữ liệu cấu hình:

```

import { JsonConfig } from './jsonConfig.js'
import { IniConfig } from './iniConfig.js'
async function main () {
  const jsonConfig = new JsonConfig()
  await jsonConfig.load('samples/conf.json')
  jsonConfig.set('nodejs', 'design patterns')
  await jsonConfig.save('samples/conf_mod.json')
  const iniConfig = new IniConfig()
  await iniConfig.load('samples/conf.ini')
  iniConfig.set('nodejs', 'design patterns')
  await iniConfig.save('samples/conf_mod.ini')
}
main()

```

Note the difference with the Strategy pattern: the logic for formatting and parsing the configuration data is baked into the class itself, rather than being chosen at runtime.

Lưu ý điểm khác biệt với mẫu Strategy: logic định dạng và phân tích dữ liệu cấu hình được nướng (bake) thẳng vào bản thân lớp, thay vì được chọn tại thời điểm chạy.

With minimal effort, the Template pattern allowed us to obtain a new, fully working configuration manager by reusing the logic and the interface inherited from the parent template class and providing only the implementation of a few abstract methods.

Với nỗ lực tối thiểu, mẫu Template đã cho phép ta có được một trình quản lý cấu hình mới, hoạt động đầy đủ, bằng cách tái sử dụng logic và giao diện kế thừa từ lớp template cha và chỉ cung cấp hiện thực cho một vài phương thức trừu tượng.

In the wild — Trong thực tế

This pattern should not look entirely new to us. We already encountered it in Chapter 6, Coding with Streams, when we were extending the different stream classes to implement our custom streams. In that context, the template methods were the `_write()`, `_read()`, `_transform()`, or `_flush()` methods, depending on the stream class that we wanted to implement. To create a new custom stream, we needed to inherit from a specific abstract stream class, providing an implementation for the template methods.

Mẫu này không nên trông hoàn toàn mới với chúng ta. Ta đã gặp nó ở Chương 6, Coding with Streams, khi ta kế thừa các lớp stream khác nhau để hiện thực các stream tùy chỉnh của mình. Trong bối cảnh đó, các template method là các phương thức `_write()`, `_read()`, `_transform()`, hoặc `_flush()`, tùy thuộc vào lớp stream mà ta muốn hiện thực. Để tạo một stream tùy chỉnh mới, ta cần kế thừa từ một lớp stream trừu tượng cụ thể, cung cấp một hiện thực cho các template method.

Next, we are going to learn about a very important and ubiquitous pattern that is also built into the JavaScript language itself, which is the Iterator pattern.

Tiếp theo, chúng ta sẽ học về một mẫu rất quan trọng và phổ biến khắp nơi, mẫu này cũng được tích hợp sẵn vào chính ngôn ngữ JavaScript, đó là mẫu Iterator.

Iterator — Iterator

The Iterator pattern is a fundamental pattern and it's so important and commonly used that it's usually built into the programming language itself. All major programming languages implement the pattern in one way or another, including, of course, JavaScript (starting from the ECMAScript2015 specification).

Mẫu Iterator là một mẫu nền tảng và nó quan trọng cũng như được dùng phổ biến đến mức thường được tích hợp sẵn vào chính ngôn ngữ lập trình. Tất cả các ngôn ngữ lập trình lớn đều hiện thực mẫu này theo cách này hay cách khác, bao gồm, tất nhiên, cả JavaScript (bắt đầu từ đặc tả ECMAScript2015).

The Iterator pattern defines a common interface or protocol for iterating the elements of a container, such as an array or a tree data structure. Usually, the algorithm for iterating over the elements of a container is different depending on the actual structure of the data. Think about iterating over an array versus traversing a tree: in the first situation, we need just a simple loop; in the second, a more complex tree traversal algorithm is required ([nodejsdp.link/tree-traversal](#)). With the Iterator pattern, we hide the details about the algorithm being used or the structure of the data and provide a common interface for iterating over any type of container. In essence, the Iterator pattern allows us to decouple the implementation of the traversal algorithm from the way we consume the results (the elements) of the traversal operation.

Mẫu Iterator định nghĩa một giao diện hoặc giao thức chung để duyệt qua các phần tử của một container, chẳng hạn một mảng hoặc một cấu trúc dữ liệu cây. Thông thường, thuật toán để duyệt qua các phần tử của một container là khác nhau tùy theo cấu trúc thực tế của dữ liệu. Hãy nghĩ về việc duyệt qua một mảng so với việc duyệt một cây: ở tình huống thứ nhất, ta chỉ cần một vòng lặp đơn giản; ở tình huống thứ hai, cần một thuật toán duyệt cây phức tạp hơn ([nodejsdp.link/tree-traversal](#)). Với mẫu Iterator, ta giấu đi các chi tiết về thuật toán đang được dùng hoặc về cấu trúc của dữ liệu và cung cấp một giao diện chung để duyệt qua bất kỳ loại container nào. Về bản chất, mẫu Iterator cho phép ta tách rời cách hiện thực thuật toán duyệt khỏi cách ta tiêu thụ các kết quả (các phần tử) của thao tác duyệt.

In JavaScript, however, iterators work great even with other types of constructs, which are not necessarily containers, such as event emitters and streams. Therefore, we can say in more general terms that the Iterator pattern defines an interface to iterate over elements produced or retrieved in sequence.

Tuy nhiên, trong JavaScript, các iterator hoạt động tuyệt vời cả với những loại cấu trúc khác mà không nhất thiết là container, chẳng hạn như event emitter và stream. Do đó, ta có thể nói một cách tổng quát hơn rằng mẫu Iterator định nghĩa một giao diện để duyệt qua các phần tử được sinh ra hoặc lấy về theo trình tự.

The iterator protocol — Giao thức iterator

In JavaScript, the Iterator pattern is implemented through protocols rather than through formal constructs, such as inheritance. This essentially means that the interaction between the implementer and the consumer of the Iterator pattern will communicate using interfaces and objects whose shape is agreed in advance.

Trong JavaScript, mẫu Iterator được hiện thực thông qua các giao thức thay vì qua các cấu trúc chính thức, chẳng hạn kế thừa. Điều này về cơ bản có nghĩa là sự tương tác giữa bên hiện thực và bên tiêu thụ mẫu Iterator sẽ giao tiếp bằng các giao diện và đối tượng mà hình dạng của chúng được thống nhất từ trước.

The starting point for implementing the Iterator pattern in JavaScript is the iterator protocol, which defines an interface for producing a sequence of values. So, we'll call iterator an object implementing a `next()` method having the following behavior: each time the method is called, the function returns the next element in the iteration through an object, called the iterator result, having two properties—`done` and `value`:

Điểm khởi đầu để hiện thực mẫu Iterator trong JavaScript là giao thức iterator, định nghĩa một giao diện để sinh ra một trình tự các giá trị. Vậy, ta sẽ gọi là iterator một đối tượng hiện thực phương thức `next()` có hành vi sau: mỗi lần phương thức được gọi, hàm trả về phần tử tiếp theo trong vòng lặp duyệt thông qua một đối tượng, gọi là kết quả iterator (iterator result), có hai thuộc tính—`done` và `value`:

- `done` is set to `true` when the iteration is complete, or in other words, when there are no more elements to return. Otherwise, `done` will be `undefined` or `false`.
- `value` contains the current element of the iteration and it can be left `undefined` if `done` is `true`. If `value` is set even when `done` is `true`, then it is said that `value` contains the return value of the iteration, a value which is not part of the elements being iterated, but it's related to the iteration itself as a whole (for example, the time spent iterating all the elements or the average of all the elements iterated if the elements are numbers).
 - `done` được đặt thành `true` khi vòng duyệt hoàn tất, hay nói cách khác, khi không còn phần tử nào để trả về nữa. Ngược lại, `done` sẽ là `undefined` hoặc `false`.
 - `value` chứa phần tử hiện tại của vòng duyệt và nó có thể để chưa định nghĩa (`undefined`) nếu `done` là `true`. Nếu `value` được đặt ngay cả khi `done` là `true`, thì khi đó người ta nói rằng `value` chứa giá trị trả về của vòng duyệt, một giá trị không phải là một phần của các phần tử đang được duyệt, mà liên quan đến chính vòng duyệt như một tổng thể (ví dụ, thời gian đã dùng để duyệt tất cả các phần tử hoặc giá trị trung bình của tất cả các phần tử đã duyệt nếu các phần tử là số).

Nothing prevents us from adding extra properties to the object returned by an iterator. However, those properties will be simply ignored by the built-in constructs or APIs consuming the iterator (we'll meet those in a moment).

Không có gì ngăn ta thêm các thuộc tính bổ sung vào đối tượng được trả về bởi một iterator. Tuy nhiên, những thuộc tính đó sẽ chỉ đơn giản bị bỏ qua bởi các cấu trúc hoặc API tích hợp sẵn tiêu thụ iterator (ta sẽ gặp chúng trong giây lát).

Let's use a quick example to demonstrate how to implement the iterator protocol. Let's implement a **factory** function called `createAlphabetIterator()`, which creates an iterator that allows us to traverse all the letters of the English alphabet. Such a function would look like this:

Hãy dùng một ví dụ nhanh để minh họa cách hiện thực giao thức iterator. Hãy hiện thực một factory function tên là `createAlphabetIterator()`, tạo ra một iterator cho phép ta

duyệt qua tất cả các chữ cái của bảng chữ cái tiếng Anh. Một hàm như vậy sẽ trông như sau:

```
const A_CHAR_CODE = 65
const Z_CHAR_CODE = 90
function createAlphabetIterator () {
  let currCode = A_CHAR_CODE
  return {
    next () {
      const currChar = String.fromCharCode(currCode)
      if (currCode > Z_CHAR_CODE) {
        return { done: true }
      }
      currCode++
      return { value: currChar, done: false }
    }
  }
}
```

The logic of the iteration is actually very straightforward; at each invocation of the `next()` method, we simply increment a number representing the letter's character code, convert it to a character, and then return it using the object shape defined by the iterator protocol. It's not a requirement for an iterator to ever return `done: true`. In fact, there can be many situations in which an iterator is infinite. An example is an iterator that returns a random number at each iteration. Another example is an iterator that calculates a mathematical series, such as the Fibonacci series or the digits of the constant π (as an exercise, you can try to convert the following algorithm to use iterators: [nodejsdp.link/pi-js](#)). Note that even if an iterator is theoretically infinite, it doesn't mean that it won't have computational or spatial limits. For example, the number returned by the Fibonacci sequence will get very big very soon.

Logic của vòng duyệt thực ra rất đơn giản; tại mỗi lần gọi phương thức `next()`, ta chỉ đơn giản tăng một số biểu diễn mã ký tự của chữ cái, chuyển nó thành một ký tự, rồi trả về nó bằng hình dạng đối tượng được định nghĩa bởi giao thức iterator. Không bắt buộc một iterator phải trả về `done: true` ở bất kỳ thời điểm nào. Thực tế, có thể có nhiều tình huống mà một iterator là vô hạn. Một ví dụ là một iterator trả về một số ngẫu nhiên ở mỗi vòng lặp. Một ví dụ khác là một iterator tính một chuỗi toán học, chẳng hạn chuỗi Fibonacci hoặc các chữ số của hằng số π (như một bài tập, bạn có thể thử chuyển thuật toán sau sang dùng iterator: [nodejsdp.link/pi-js](#)). Lưu ý rằng ngay cả khi một iterator về mặt lý thuyết là vô hạn, điều đó không có nghĩa là nó sẽ không có các giới hạn về tính toán hoặc không gian. Ví dụ, số được chuỗi Fibonacci trả về sẽ rất nhanh trở nên rất lớn.

The important aspect to note is that an iterator is very often a stateful object since we have to keep track in some way of the current position of the iteration. In the previous example, we managed to keep the state in a **closure** (the `currCode` variable) but this is just one of the ways we can do so. We could have, for example, kept the state in an instance variable. This is usually better in terms of debuggability since we can read the status of the iteration from the iterator itself at any time, but on the other side, it does not prevent external code from modifying the instance variable and hence tampering with the status of the iteration. It's up to you to decide the pros and cons of each option.

Khía cạnh quan trọng cần lưu ý là một iterator rất thường là một đối tượng có trạng thái (stateful) vì ta phải theo dõi bằng cách nào đó vị trí hiện tại của vòng duyệt. Trong ví dụ trước, ta đã quản lý được trạng thái trong một closure (biến `currCode`) nhưng đây chỉ là một trong những cách ta có thể làm. Ta có thể, chẳng hạn, đã giữ trạng thái trong một biến

thực thể. Cách này thường tốt hơn về khả năng gỡ lỗi vì ta có thể đọc trạng thái của vòng duyệt từ chính iterator tại bất kỳ thời điểm nào, nhưng mặt khác, nó không ngăn được mã bên ngoài sửa đổi biến thực thể và do đó can thiệp vào trạng thái của vòng duyệt. Tùy bạn quyết định ưu và nhược điểm của mỗi lựa chọn.

Iterators can indeed be fully stateless as well. Examples are iterators returning random elements and either completing randomly or never completing, and iterators stopping at the first iteration.

Iterator quả thực cũng có thể hoàn toàn không trạng thái (stateless). Ví dụ là các iterator trả về các phần tử ngẫu nhiên và hoặc kết thúc ngẫu nhiên hoặc không bao giờ kết thúc, và các iterator dừng lại ở vòng lặp đầu tiên.

Now, let's see how we can use the iterator we just built. Consider the following code fragment:

Giờ, hãy xem cách ta có thể dùng iterator mà ta vừa xây dựng. Xét đoạn mã sau:

```
const iterator = createAlphabetIterator()
let iterationResult = iterator.next()
while (!iterationResult.done) {
  console.log(iterationResult.value)
  iterationResult = iterator.next()
}
```

As we can see from the previous code, the code that consumes an iterator can be considered a pattern itself. However, as we will see later in this section, it's not the only way we have to consume an iterator. JavaScript has, in fact, much more convenient and elegant ways to use iterators. Iterators can optionally specify two additional methods: `return([value])` and `throw(error)`. The first is by convention used to signal to the iterator that the consumer has stopped the iteration before its completion, while the second is used to communicate to the iterator that an error condition has occurred. Both methods are rarely used by built-in iterators.

Như ta thấy từ đoạn mã trước, đoạn mã tiêu thụ một iterator bản thân nó có thể được coi là một mẫu. Tuy nhiên, như ta sẽ thấy ở phần sau của mục này, đó không phải là cách duy nhất để tiêu thụ một iterator. Thực tế, JavaScript có những cách tiện lợi và thanh lịch hơn nhiều để dùng iterator. Iterator có thể tùy chọn xác định thêm hai phương thức: `return([value])` và `throw(error)`. Cái đầu tiên, theo quy ước, được dùng để báo cho iterator biết rằng bên tiêu thụ đã dừng vòng duyệt trước khi nó hoàn tất, còn cái thứ hai được dùng để truyền đạt cho iterator rằng một điều kiện lỗi đã xảy ra. Cả hai phương thức hiếm khi được các iterator tích hợp sẵn sử dụng.

The iterable protocol — Giao thức iterable

The iterable protocol defines a standardized means for an object to return an iterator. Such objects are called iterables. Usually, an iterable is a container of elements, such as a data structure, but it can also be an object virtually representing a set of elements, such as a `Directory` object, which would allow us to iterate over the files in a directory.

Giao thức iterable định nghĩa một phương tiện chuẩn hoá để một đối tượng trả về một iterator. Những đối tượng như vậy được gọi là iterable. Thông thường, một iterable là một container chứa các phần tử, chẳng hạn một cấu trúc dữ liệu, nhưng nó cũng có thể là một đối tượng biểu diễn một cách hư ảo (virtually) một tập các phần tử, chẳng hạn một đối tượng `Directory`, sẽ cho phép ta duyệt qua các tập tin trong một thư mục.

In JavaScript, we can define an iterable by making sure it implements the `@@iterator` method, or in other words, a method accessible through the built-in symbol `Symbol.iterator`.

Trong JavaScript, ta có thể định nghĩa một iterable bằng cách bảo đảm nó hiện thực phương thức `@@iterator`, hay nói cách khác, một phương thức truy cập được thông qua symbol tích hợp sẵn `Symbol.iterator`.

The `@@name` convention indicates a well-known symbol according to the ES6 specification. To find out more, you can check out the relative section of the ES6 specification at nodejsdp.link/es6-well-known-symbols.

Quy ước `@@name` chỉ một well-known symbol theo đặc tả ES6. Để tìm hiểu thêm, bạn có thể xem mục liên quan của đặc tả ES6 tại nodejsdp.link/es6-well-known-symbols.

Such an `@@iterator` method should return an iterator object, which can be used to iterate over the elements represented by the iterable. For example, if our iterable is a class, we would have something like the following:

Một phương thức `@@iterator` như vậy nên trả về một đối tượng iterator, có thể được dùng để duyệt qua các phần tử được biểu diễn bởi iterable. Ví dụ, nếu iterable của ta là một lớp, ta sẽ có thứ gì đó như sau:

```
class MyIterable {
  // other methods...
  [Symbol.iterator] () {
    // return an iterator
  }
}
```

To show how this works in practice, let's build a class to manage information organized in a bidimensional matrix structure. We want this class to be implementing the iterable protocol, so that we can scan all the elements in the matrix using an iterator. Let's create a file called `matrix.js` containing the following content:

Để cho thấy cách điều này hoạt động trong thực tế, hãy xây dựng một lớp để quản lý thông tin được tổ chức trong một cấu trúc ma trận hai chiều. Ta muốn lớp này hiện thực giao thức iterable, để ta có thể quét tất cả các phần tử trong ma trận bằng một iterator. Hãy tạo một tập tin tên là `matrix.js` chứa nội dung sau:

```
export class Matrix {
  constructor (inMatrix) {
    this.data = inMatrix
  }
  get (row, column) {
    if (row >= this.data.length ||
        column >= this.data[row].length) {
      throw new RangeError('Out of bounds')
    }
    return this.data[row][column]
  }
  set (row, column, value) {
    if (row >= this.data.length ||
        column >= this.data[row].length) {
      throw new RangeError('Out of bounds')
    }
    this.data[row][column] = value
  }
}
```



```

[Symbol.iterator] () {
  let nextRow = 0
  let nextCol = 0
  return {
    next: () => {
      if (nextRow === this.data.length) {
        return { done: true }
      }
      const currVal = this.data[nextRow][nextCol]
      if (nextCol === this.data[nextRow].length - 1) {
        nextRow++
        nextCol = 0
      } else {
        nextCol++
      }
      return { value: currVal }
    }
  }
}

```

As we can see, the class contains the basic methods for getting and setting values in the matrix, as well as the @@iterator method, implementing our iterable protocol. The @@iterator method will return an iterator, as specified by the iterable protocol and such an iterator adheres to the iterator protocol. The logic of the iterator is very straightforward: we are simply traversing the matrix's cells from the top left to the bottom right, by scanning each column of each row; we are doing that by leveraging two indexes, nextRow and nextCol.

Như ta thấy, lớp này chứa các phương thức cơ bản để lấy và đặt giá trị trong ma trận, cũng như phương thức @@iterator, hiện thực giao thức iterable của chúng ta. Phương thức @@iterator sẽ trả về một iterator, như được quy định bởi giao thức iterable, và một iterator như vậy tuân theo giao thức iterator. Logic của iterator rất đơn giản: ta chỉ đơn giản duyệt các ô của ma trận từ trên trái xuống dưới phải, bằng cách quét từng cột của từng hàng; ta làm điều đó bằng cách tận dụng hai chỉ số, nextRow và nextCol.

Now, it's time to try out our iterable Matrix class. We can do that in a file called index.js:

Giờ, đã đến lúc thử lớp Matrix iterable của chúng ta. Ta có thể làm điều đó trong một tập tin tên là index.js:

```

import { Matrix } from './matrix.js'
const matrix2x2 = new Matrix([
  ['11', '12'],
  ['21', '22']
])
const iterator = matrix2x2[Symbol.iterator]()
let iterationResult = iterator.next()
while (!iterationResult.done) {
  console.log(iterationResult.value)
  iterationResult = iterator.next()
}

```

All we are doing in the previous code is creating a sample Matrix instance and then obtaining an

iterator using the `@@iterator` method. What comes next, as we already know, is just boilerplate code that iterates over the elements returned by the iterator. The output of the iteration should be `'11', '12', '21', '22'`.

Tất cả những gì ta làm trong đoạn mã trước là tạo một thực thể `Matrix` mẫu rồi lấy một iterator bằng phương thức `@@iterator`. Tiếp theo, như ta đã biết, chỉ là mã rập khuôn (boilerplate) duyệt qua các phần tử được iterator trả về. Đầu ra của vòng duyệt sẽ là `'11', '12', '21', '22'`.

Iterators and iterables as a native JavaScript interface — Iterator và iterable như một giao diện gốc của JavaScript

At this point, you may ask: “what’s the point of having all these protocols for defining iterators and iterables?” Well, having a standardized interface allows third party code as well as the language itself to be modeled around the two protocols we’ve just seen. This way, we can have APIs (even native) as well as syntax constructs accepting iterables as an input. For example, the most obvious syntax construct accepting an iterable is the `for...of` loop. We’ve just seen in the last code sample that iterating over a JavaScript iterator is a pretty standard operation, and its code is mostly boilerplate. In fact, we’ll always have an invocation to `next()` to retrieve the next element and a check to verify if the `done` property of the iteration result is set to `true`, which indicates the end of the iteration. But, worry not, simply pass an iterable to the `for...of` instruction to seamlessly loop over the elements returned by its iterator. This allows us to process the iteration with an intuitive and compact syntax:

Đến đây, bạn có thể hỏi: “có ích gì khi có tất cả các giao thức này để định nghĩa iterator và iterable?” Vâng, việc có một giao diện chuẩn hoá cho phép mã của bên thứ ba cũng như bản thân ngôn ngữ được mô hình hoá xoay quanh hai giao thức mà ta vừa thấy. Bằng cách này, ta có thể có các API (kể cả API gốc) cũng như các cấu trúc cú pháp chấp nhận iterable làm đầu vào. Ví dụ, cấu trúc cú pháp rõ ràng nhất chấp nhận một iterable là vòng lặp `for...of`. Ta vừa thấy ở đoạn mã mẫu cuối rằng việc duyệt qua một iterator JavaScript là một thao tác khá chuẩn, và mã của nó chủ yếu là rập khuôn. Thực tế, ta sẽ luôn có một lần gọi `next()` để lấy phần tử tiếp theo và một lần kiểm tra để xác minh xem thuộc tính `done` của kết quả vòng duyệt có được đặt thành `true` hay không, vốn cho biết kết thúc vòng duyệt. Nhưng đừng lo, chỉ cần truyền một iterable cho lệnh `for...of` để lặp một cách liền mạch qua các phần tử được iterator của nó trả về. Điều này cho phép ta xử lý vòng duyệt với một cú pháp trực quan và súc tích:

```
for (const element of matrix2x2) {  
  console.log(element)  
}
```

Another construct compatible with iterables is the spread operator:

Một cấu trúc khác tương thích với iterable là toán tử spread:

```
const flattenedMatrix = [...matrix2x2]  
console.log(flattenedMatrix)
```

Similarly, we can use an iterable with the destructuring assignment operation:

Tương tự, ta có thể dùng một iterable với thao tác gán bằng destructuring:

```
const [oneOne, oneTwo, twoOne, twoTwo] = matrix2x2  
console.log(oneOne, oneTwo, twoOne, twoTwo)
```

The following are some JavaScript built-in APIs accepting iterables:

Sau đây là một số API tích hợp sẵn của JavaScript chấp nhận iterable:

- `Map([iterable]): nodejsdp.link/map-constructor`
- `WeakMap([iterable]): nodejsdp.link/weakmap-constructor`
- `Set([iterable]): nodejsdp.link/set-constructor`
- `WeakSet([iterable]): nodejsdp.link/weakset-constructor`
- `**Promise*.all(iterable): nodejsdp.link/promise-all`
- `Promise.race(iterable): nodejsdp.link/promise-race`
- `Array.from(iterable): nodejsdp.link/array-from`
 - `Map([iterable]): nodejsdp.link/map-constructor`
 - `WeakMap([iterable]): nodejsdp.link/weakmap-constructor`
 - `Set([iterable]): nodejsdp.link/set-constructor`
 - `WeakSet([iterable]): nodejsdp.link/weakset-constructor`
 - `Promise.all(iterable): nodejsdp.link/promise-all`
 - `Promise.race(iterable): nodejsdp.link/promise-race`
 - `Array.from(iterable): nodejsdp.link/array-from`

On the Node.js side, one notable **API** accepting an iterable is `stream.Readable.from(iterable, [options])` ([nodejsdp.link/readable-from](https://nodejs.org/docs/latest/api/stream.html#stream.Readable.from(iterable, options))), which creates a readable stream out of an iterable object. Note that all the APIs and syntax constructs we've just seen accept as input an iterable and not an iterator. But, what can we do if we have a function returning an iterator, such as in our `createAlphabetIterator()` example? How can we leverage all the built-in APIs and syntax constructs? A possible solution is implementing the `@@iterator` method in the iterator object itself, which will simply return the iterator object itself. This way we'll be able to write something such as the following:

Về phía Node.js, một API đáng chú ý chấp nhận một iterable là `stream.Readable.from(iterable, [options])` ([nodejsdp.link/readable-from](https://nodejs.org/docs/latest/api/stream.html#stream.Readable.from(iterable, options))), tạo ra một readable stream từ một đối tượng iterable. Lưu ý rằng tất cả các API và cấu trúc cú pháp mà ta vừa thấy đều chấp nhận đầu vào là một iterable chứ không phải một iterator. Nhưng, ta có thể làm gì nếu ta có một hàm trả về một iterator, chẳng hạn như trong ví dụ `createAlphabetIterator()` của ta? Làm sao ta có thể tận dụng tất cả các API và cấu trúc cú pháp tích hợp sẵn? Một giải pháp khả dĩ là hiện thực phương thức `@@iterator` ngay trong chính đối tượng iterator, mà sẽ chỉ đơn giản trả về chính đối tượng iterator đó. Bằng cách này ta sẽ có thể viết được thứ gì đó như sau:

```
for (const letter of createAlphabetIterator()) {  
  // ...  
}
```

JavaScript itself defines many iterables that can be used with the APIs and constructs we've just seen. The most notable iterable is `Array`, but also other data structures, such as `Map` and `Set`, and even `String` all implement the `@@iterator` method. In Node.js land, `**Buffer**` is probably the most notable iterable.

Bản thân JavaScript định nghĩa nhiều iterable có thể được dùng với các API và cấu trúc mà ta vừa thấy. Iterable đáng chú ý nhất là `Array`, nhưng cả các cấu trúc dữ liệu khác, chẳng hạn `Map` và `Set`, và thậm chí `String` đều hiện thực phương thức `@@iterator`. Trong thế giới Node.js, `Buffer` có lẽ là iterable đáng chú ý nhất.

A trick to make sure that an array doesn't contain duplicate elements is the following: `const uniqArray = Array.from(new Set(arrayWithDuplicates))`. This also shows us how iterables offer a way for different components to talk to each other using a shared interface.

Một mẹo để bảo đảm một mảng không chứa các phần tử trùng lặp là như sau: `const uniqArray = Array.from(new Set(arrayWithDuplicates))`. Điều này cũng cho ta thấy cách iterable mang lại một cách để các thành phần khác nhau nói chuyện với nhau bằng một giao diện chung.

Generators — Generator

The ES2015 specification introduced a syntax construct that is closely related to iterators. We are talking about generators, also known as semicoroutines. They are a generalization of standard functions, in which there can be different entry points. In a standard function, we can have only one entry point, which corresponds to the invocation of the function itself, but a **generator** can be suspended (using the `yield` statement) and then resumed at a later time. Among other things, generators are very well suited to implement iterators, in fact, as we will see in a bit, the generator object returned by a generator function is indeed both an iterator and an iterable.

Đặc tả ES2015 đã giới thiệu một cấu trúc cú pháp liên quan chặt chẽ đến iterator. Ta đang nói về generator, còn được biết đến với tên semicoroutine. Chúng là một sự tổng quát hoá của các hàm thông thường, trong đó có thể có nhiều điểm vào (entry point) khác nhau. Trong một hàm thông thường, ta chỉ có thể có một điểm vào duy nhất, tương ứng với lần gọi chính hàm đó, nhưng một generator có thể bị tạm dừng (dùng câu lệnh `yield`) rồi được tiếp tục vào một thời điểm sau đó. Trong số những điều khác, generator rất phù hợp để hiện thực iterator, thực tế, như ta sẽ thấy trong giây lát, đối tượng generator được trả về bởi một generator function quả thực vừa là một iterator vừa là một iterable.

Generators in theory — Generator về mặt lý thuyết

To define a generator function, we need to use the `function*` declaration (the `function` keyword followed by an asterisk):

Để định nghĩa một generator function, ta cần dùng khai báo `function*` (từ khoá `function` theo sau bởi một dấu sao):

```
function * myGenerator () {  
  // generator body  
}
```

Invoking a generator function will not execute its body immediately. Rather, it will return a generator object, which, as we already mentioned, is both an iterator and an iterable. But it doesn't end here; invoking `next()` on the generator object will start or resume the execution of the generator until the `yield` instruction is invoked or the generator returns (either implicitly or explicitly with a `return` instruction). Within the generator, using the keyword `yield` followed by a value `x` is equivalent to returning `{done: false, value: x}` from the iterator, while returning a value `x` is equivalent to returning `{done: true, value: x}`.

Việc gọi một generator function sẽ không thực thi thân của nó ngay lập tức. Thay vào đó, nó sẽ trả về một đối tượng generator, mà, như ta đã đề cập, vừa là một iterator vừa là một iterable. Nhưng nó không dừng ở đây; việc gọi `next()` trên đối tượng generator sẽ bắt đầu hoặc tiếp tục việc thực thi generator cho đến khi lệnh `yield` được gọi hoặc generator trả về (hoặc ngầm định hoặc tường minh bằng một lệnh `return`). Bên trong generator, dùng từ khoá `yield` theo sau bởi một giá trị `x` tương đương với việc trả về `{done: false, value: x}` từ iterator, trong khi trả về một giá trị `x` tương đương với việc trả về `{done: true, value: x}`.

A simple generator function — Một generator function đơn giản

To demonstrate what we just learned, let's see a simple generator called `fruitGenerator()`, which will yield two names of fruits and return their ripening season:

Để minh họa những gì ta vừa học, hãy xem một generator đơn giản tên là `fruitGenerator()`, sẽ yield hai tên trái cây và trả về mùa chín của chúng:

```
function * fruitGenerator () {  
  yield 'peach'  
  yield 'watermelon'  
  return 'summer'  
}  
const fruitGeneratorObj = fruitGenerator()  
console.log(fruitGeneratorObj.next()) // (1)  
console.log(fruitGeneratorObj.next()) // (2)  
console.log(fruitGeneratorObj.next()) // (3)
```

The preceding code will print the following text:

Đoạn mã trên sẽ in ra đoạn văn bản sau:

```
{ value: 'peach', done: false }  
{ value: 'watermelon', done: false }  
{ value: 'summer', done: true }
```

This is a short explanation of what happened:

Đây là một giải thích ngắn về những gì đã xảy ra:

1. The first time `fruitGeneratorObj.next()` was invoked, the generator started its execution until it reached the first `yield` command, which put the generator on pause and returned the value 'peach' to the caller.
2. At the second invocation of `fruitGeneratorObj.next()`, the generator resumed, starting from the second `yield` command, which in turn put the execution on pause again, while returning the value 'watermelon' to the caller.
3. The last invocation of `fruitGeneratorObj.next()` caused the execution of the generator to resume from its last instruction, a `return` statement, which terminates the generator, returns the value 'summer', and sets the `done` property to `true` in the result object.

1. Lần đầu tiên `fruitGeneratorObj.next()` được gọi, generator bắt đầu việc thực thi cho đến khi nó đạt đến lệnh `yield` đầu tiên, lệnh này đặt generator vào trạng thái tạm dừng và trả về giá trị 'peach' cho người gọi.
2. Ở lần gọi `fruitGeneratorObj.next()` thứ hai, generator tiếp tục, bắt đầu từ lệnh `yield` thứ hai, lệnh này lần lượt lại đặt việc thực thi vào trạng thái tạm dừng, trong khi trả về giá trị 'watermelon' cho người gọi.
3. Lần gọi `fruitGeneratorObj.next()` cuối cùng khiến việc thực thi generator tiếp tục từ lệnh cuối cùng của nó, một câu lệnh `return`, lệnh này kết thúc generator, trả về giá trị 'summer', và đặt thuộc tính `done` thành `true` trong đối tượng `result`.

Since a generator object is also an iterable, we can use it in a `for...of` loop. For example:

Vì một đối tượng generator cũng là một iterable, ta có thể dùng nó trong một vòng lặp `for...of`. Ví dụ:

```
for (const fruit of fruitGenerator()) {  
  console.log(fruit)}
```

```
}
```

The preceding loop will print:

Vòng lặp trên sẽ in ra:

```
peach
watermelon
```

Why is summer not being printed? Well, summer is not yielded by our generator, but instead, it is returned, which indicates that the iteration is complete with summer as a return value (not as an element).

Tại sao summer không được in ra? Vâng, summer không được generator của ta yield, mà thay vào đó, nó được return, điều này cho biết rằng vòng duyệt hoàn tất với summer là một giá trị trả về (không phải là một phần tử).

Controlling a generator iterator — Điều khiển một generator iterator

Generator objects are more than standard iterators, in fact, their `next()` method optionally accepts an argument (whereas, as specified by the iterator protocol, it does not need to accept one). Such an argument is passed as the return value of the `yield` instruction. To show this, let's create a new simple generator:

Đối tượng generator còn hơn cả những iterator chuẩn, thực tế, phương thức `next()` của chúng có thể tùy chọn chấp nhận một đối số (trong khi, như được quy định bởi giao thức iterator, nó không cần phải chấp nhận một đối số nào). Một đối số như vậy được truyền vào làm giá trị trả về của lệnh `yield`. Để cho thấy điều này, hãy tạo một generator đơn giản mới:

```
function * twoWayGenerator () {
  const what = yield null
  yield 'Hello ' + what
}
const twoWay = twoWayGenerator()
twoWay.next()
console.log(twoWay.next('world'))
```

When executed, the preceding code prints `Hello world`. This means that the following has happened:

Khi được thực thi, đoạn mã trên in ra `Hello world`. Điều này có nghĩa là những điều sau đã xảy ra:

1. The first time the `next()` method is invoked, the generator reaches the first `yield` statement and is then put on pause.
2. When `next('world')` is invoked, the generator resumes from the point where it was put on pause, which is on the `yield` instruction, but this time we have a value that is passed back to the generator. This value will then be set to the `what` variable. The generator then appends the `what` variable to the string `'Hello '` and yields the result.
 1. Lần đầu tiên phương thức `next()` được gọi, generator đạt đến câu lệnh `yield` đầu tiên và sau đó bị đặt vào trạng thái tạm dừng.
 2. Khi `next('world')` được gọi, generator tiếp tục từ điểm mà nó đã bị đặt tạm dừng, vốn là tại lệnh `yield`, nhưng lần này ta có một giá trị được truyền ngược lại vào

generator. Giá trị này sau đó sẽ được đặt vào biến `what`. Generator sau đó nối biến `what` vào chuỗi `'Hello '` và `yield` kết quả.

Two other extra features provided by generator objects are the optional `throw()` and `return()` iterator methods. The first behaves like `next()` but it will also throw an exception within the generator as if it was thrown at the point of the last `yield`, and returns the canonical iterator object with the `done` and `value` properties. The second, the `return()` method, forces the generator to terminate and returns an object such as the following: `{done: true, value: returnArgument}` where `returnArgument` is the argument passed to the `return()` method.

Hai tính năng bổ sung khác được cung cấp bởi đối tượng generator là các phương thức iterator tùy chọn `throw()` và `return()`. Cái đầu tiên hành xử giống `next()` nhưng nó cũng sẽ ném ra một exception bên trong generator như thể nó được ném ra tại điểm của lệnh `yield` cuối cùng, và trả về đối tượng iterator chuẩn mực với các thuộc tính `done` và `value`. Cái thứ hai, phương thức `return()`, buộc generator kết thúc và trả về một đối tượng như sau: `{done: true, value: returnArgument}` trong đó `returnArgument` là đối số được truyền cho phương thức `return()`.

The following code shows a demonstration of these two methods:

Đoạn mã sau cho thấy một minh họa về hai phương thức này:

```
function * twoWayGenerator () {
  try {
    const what = yield null
    yield 'Hello ' + what
  } catch (err) {
    yield 'Hello error: ' + err.message
  }
}
console.log('Using throw():')
const twoWayException = twoWayGenerator()
twoWayException.next()
console.log(twoWayException.throw(new Error('Boom!')))
console.log('Using return():')
const twoWayReturn = twoWayGenerator()
console.log(twoWayReturn.return('myReturnValue'))
```

Running the previous code will print the following to the console:

Việc chạy đoạn mã trước sẽ in ra console những dòng sau:

```
Using throw():
{ value: 'Hello error: Boom!', done: false }
Using return():
{ value: 'myReturnValue', done: true }
```

As we can see, the `twoWayGenerator()` function will receive an exception as soon as the first `yield` instruction returns. This works exactly as if an exception was thrown from inside the generator, and this means that it can be caught and handled like any other exception using a `try...catch` block. The `return()` method, instead, will simply stop the execution of the generator causing the given value to be provided as a return value by the generator.

Như ta thấy, hàm `twoWayGenerator()` sẽ nhận một exception ngay khi lệnh `yield` đầu tiên trả về. Điều này hoạt động chính xác như thể một exception được ném ra từ bên

trong generator, và điều này có nghĩa là nó có thể được bắt và xử lý như bất kỳ exception nào khác bằng một khối `try...catch`. Phương thức `return()`, ngược lại, sẽ chỉ đơn giản dừng việc thực thi generator khiến cho giá trị đã cho được generator cung cấp làm giá trị trả về.

How to use generators in place of iterators — Cách dùng generator thay cho iterator

Generator objects are also iterators. This means that generator functions can be used to implement the `@@iterator` method of iterable objects. To demonstrate this, let's convert our previous `Matrix` iteration example to generators. Let's update our `matrix.js` file as follows:

Đối tượng generator cũng là iterator. Điều này có nghĩa là các generator function có thể được dùng để hiện thực phương thức `@@iterator` của các đối tượng iterable. Để minh họa điều này, hãy chuyển ví dụ duyệt `Matrix` trước đây của ta sang generator. Hãy cập nhật tập tin `matrix.js` của ta như sau:

```
export class Matrix {
  // ...rest of the methods (stay unchanged)
  * [Symbol.iterator] () { // (1)
    let nextRow = 0 // (2)
    let nextCol = 0
    while (nextRow !== this.data.length) { // (3)
      yield this.data[nextRow][nextCol]
      if (nextCol === this.data[nextRow].length - 1) {
        nextRow++
        nextCol = 0
      } else {
        nextCol++
      }
    }
  }
}
```

There are a few interesting aspects in the code fragment we've just seen. Let's analyze them in more detail:

Có một vài khía cạnh thú vị trong đoạn mã mà ta vừa thấy. Hãy phân tích chúng chi tiết hơn:

1. The first thing to notice is that the `@@iterator` method is now a generator (note the asterisk `*` before the method name).
2. The variables we use to maintain the state of the iteration are now just local variables for the generator, while in the previous version of the `Matrix` class, those two variables were part of a closure. This is possible because when a generator is invoked, its local state is preserved between reentries.
3. We are using a standard loop to iterate over the elements of the matrix. This is certainly more intuitive than trying to imagine a loop that invokes the `next()` method of an iterator.
 1. Điều đầu tiên cần lưu ý là phương thức `@@iterator` giờ là một generator (chú ý dấu sao `*` trước tên phương thức).
 2. Các biến mà ta dùng để duy trì trạng thái của vòng duyệt giờ chỉ là các biến cục bộ của generator, trong khi ở phiên bản trước của lớp `Matrix`, hai biến đó là một phần của một closure. Điều này khả thi vì khi một generator được gọi, trạng thái cục bộ của

- nó được bảo toàn giữa các lần quay vào lại (reentry).
3. Ta đang dùng một vòng lặp chuẩn để duyệt qua các phần tử của ma trận. Điều này chắc chắn trực quan hơn việc cố gắng hình dung một vòng lặp gọi phương thức `next()` của một iterator.

As we can see, generators are an excellent alternative to writing iterators from scratch. They will improve the readability of our iteration routine and will offer the same level of functionality (or even better).

Như ta thấy, generator là một lựa chọn tuyệt vời thay cho việc viết iterator từ đầu. Chúng sẽ cải thiện khả năng đọc hiểu của thủ tục duyệt của ta và sẽ mang lại cùng mức chức năng (hoặc thậm chí tốt hơn).

The generator delegation instruction, `yield * iterable`, is another example of a JavaScript built-in syntax accepting an iterable as an argument. The instruction will loop over the elements of the iterable and yield each element one by one.

Lệnh uỷ thác generator (generator delegation), `yield * iterable`, là một ví dụ khác về một cú pháp tích hợp sẵn của JavaScript chấp nhận một iterable làm đối số. Lệnh này sẽ lặp qua các phần tử của iterable và yield từng phần tử một.

Async iterators — Async iterator

The iterators we've seen so far return a value synchronously from their `next()` method. However, in JavaScript and especially in Node.js, it's very common to have iterations over items that require an asynchronous operation to be produced.

Các iterator mà ta đã thấy cho đến giờ trả về một giá trị một cách đồng bộ từ phương thức `next()` của chúng. Tuy nhiên, trong JavaScript và đặc biệt trong Node.js, rất phổ biến việc có những vòng duyệt qua các mục đòi hỏi một thao tác bất đồng bộ để được sinh ra.

Imagine, for example, to iterate over the requests received by an HTTP server, or over the results of an SQL query, or over the elements of a paginated REST API. In all those situations, it would be handy to be able to return a promise from the `next()` method of an iterator, or even better, use the **async/await** construct.

Hãy tưởng tượng, chẳng hạn, việc duyệt qua các yêu cầu nhận được bởi một máy chủ HTTP, hoặc qua các kết quả của một truy vấn SQL, hoặc qua các phần tử của một REST API phân trang. Trong tất cả những tình huống đó, sẽ rất tiện nếu có thể trả về một promise từ phương thức `next()` của một iterator, hoặc tốt hơn nữa, dùng cấu trúc **async/await**.

Well, that's exactly what async iterators are; they are iterators returning a promise, and since that's the only extra requirement, it means that we can also use an async function to define the `next()` method of the iterator. Similarly, async iterables are objects that implement an `@@asyncIterator` method, or in other words, a method accessible through the `Symbol.asyncIterator` key, which returns (synchronously) an **async iterator**. Async iterables can be looped over using the `for await...of` syntax, which can only be used inside an async function. With the `for await...of` syntax, we are essentially implementing a sequential asynchronous execution flow on top of the Iterator pattern. Essentially, it's just syntactic sugar for the following loop:

Vâng, đó chính xác là async iterator; chúng là các iterator trả về một promise, và vì đó là yêu cầu bổ sung duy nhất, điều đó có nghĩa là ta cũng có thể dùng một hàm async để định nghĩa phương thức `next()` của iterator. Tương tự, async iterable là các đối tượng hiện thực phương thức `@@asyncIterator`, hay nói cách khác, một phương thức truy cập được thông qua khoá `Symbol.asyncIterator`, mà trả về (một cách đồng bộ) một async iterator.

Async iterable có thể được lặp bằng cú pháp `for await...of`, vốn chỉ có thể được dùng bên trong một hàm async. Với cú pháp `for await...of`, về cơ bản ta đang hiện thực một luồng thực thi bất đồng bộ tuần tự trên nền mẫu Iterator. Về bản chất, nó chỉ là cú pháp đường (syntactic sugar) cho vòng lặp sau:

```
const asyncIterator = iterable[Symbol.asyncIterator]()
let iterationResult = await asyncIterator.next()
while (!iterationResult.done) {
  console.log(iterationResult.value)
  iterationResult = await asyncIterator.next()
}
```

This means that the `for await...of` syntax can also be used to iterate over a simple iterable (not just async iterables) as, for example, over an array of promises. It will work even if not all the elements (or none) of the iterator are promises.

Điều này có nghĩa là cú pháp `for await...of` cũng có thể được dùng để duyệt qua một iterable đơn giản (không chỉ async iterable), chẳng hạn qua một mảng các promise. Nó sẽ hoạt động ngay cả khi không phải tất cả các phần tử (hoặc không phần tử nào) của iterator là promise.

To quickly demonstrate this, let's build a class that takes a list of URLs as input and allows us to iterate over their availability status (up/down). Let's call the class `CheckUrls`:

Để minh họa nhanh điều này, hãy xây dựng một lớp nhận một danh sách URL làm đầu vào và cho phép ta duyệt qua trạng thái khả dụng của chúng (up/down). Hãy gọi lớp đó là `CheckUrls`:

```
import superagent from 'superagent'
export class CheckUrls {
  constructor (urls) { // (1)
    this.urls = urls
  }
  [Symbol.asyncIterator] () {
    const urlsIterator = this.urls[Symbol.iterator]() // (2)
    return {
      async next () { // (3)
        const iteratorResult = urlsIterator.next() // (4)
        if (iteratorResult.done) {
          return { done: true }
        }
        const url = iteratorResult.value
        try {
          const checkResult = await superagent // (5)
            .head(url)
            .redirects(2)
          return {
            done: false,
            value: `${url} is up, status: ${checkResult.status}`
          }
        } catch (err) {
          return {
            done: false,

```

```

        value: `${url} is down, error: ${err.message}`
      }
    }
  }
}
}
}

```

Let's analyze the previous code's most important parts:

Hãy phân tích những phần quan trọng nhất của đoạn mã trước:

1. The `CheckUrls` class constructor takes as input a list of URLs. Since we now know how to use iterators and iterables, we can say that this list of URLs can be just any iterable.
2. In our `@@asyncIterator` method, we obtain an iterator from the `this.urls` object, which, as we just said, should be an iterable. We can do that by simply invoking its `@@iterable` method.
3. Note how the `next()` method is now an `async` function. This means that it will always return a promise, as requested by the `async` iterable protocol.
4. In the `next()` method, we use the `urlsIterator` to get the next URL in the list, unless there are no more, in which case we simply return `{done: true}`.
5. Note how we can now use the `await` instruction to asynchronously get the result of the **HEAD request** sent to the current URL.

1. Constructor của lớp `CheckUrls` nhận đầu vào là một danh sách URL. Vì giờ ta đã biết cách dùng iterator và iterable, ta có thể nói rằng danh sách URL này có thể chỉ là bất kỳ iterable nào.
2. Trong phương thức `@@asyncIterator` của ta, ta lấy một iterator từ đối tượng `this.urls`, vốn, như ta vừa nói, nên là một iterable. Ta có thể làm điều đó bằng cách chỉ đơn giản gọi phương thức `@@iterable` của nó.
3. Lưu ý cách phương thức `next()` giờ là một hàm `async`. Điều này có nghĩa là nó sẽ luôn trả về một promise, như được yêu cầu bởi giao thức `async` iterable.
4. Trong phương thức `next()`, ta dùng `urlsIterator` để lấy URL tiếp theo trong danh sách, trừ khi không còn URL nào nữa, trong trường hợp đó ta chỉ đơn giản trả về `{done: true}`.
5. Lưu ý cách giờ ta có thể dùng lệnh `await` để lấy một cách bất đồng bộ kết quả của yêu cầu **HEAD** được gửi đến URL hiện tại.

Now, let's use the `for await...of` syntax we mentioned earlier to iterate over a `CheckUrls` object:

Giờ, hãy dùng cú pháp `for await...of` mà ta đã đề cập trước đó để duyệt qua một đối tượng `CheckUrls`:

```

import { CheckUrls } from './checkUrls.js'
async function main () {
  const checkUrls = new CheckUrls([
    'https://nodejsdesignpatterns.com',
    'https://example.com',
    'https://mustbedownforsurehopefully.com'
  ])
  for await (const status of checkUrls) {
    console.log(status)
  }
}
main()

```

As we can see, the `for await...of` syntax is a very intuitive way to iterate over an async iterable and, as we will see in a while, it can be used in conjunction with some interesting built-in iterables to obtain alternative new ways to access asynchronous information.

Như ta thấy, cú pháp `for await...of` là một cách rất trực quan để duyệt qua một async iterable và, như ta sẽ thấy trong chốc lát, nó có thể được dùng kết hợp với một số iterable tích hợp sẵn thú vị để có được những cách mới thay thế nhằm truy cập thông tin bất đồng bộ.

The `for await...of` loop (as well as its synchronous version) will call the optional `return()` method of the iterator if it's prematurely interrupted with a `break`, a `return`, or an exception. This can be used to immediately perform any cleanup task that would usually be performed when the iteration completes.

Vòng lặp `for await...of` (cũng như phiên bản đồng bộ của nó) sẽ gọi phương thức tùy chọn `return()` của iterator nếu nó bị ngắt sớm bằng một `break`, một `return`, hoặc một exception. Điều này có thể được dùng để ngay lập tức thực hiện bất kỳ tác vụ dọn dẹp nào mà thường được thực hiện khi vòng duyệt hoàn tất.

Async generators — Async generator

As well as async iterators, we can also have async generators. To define an async generator function, simply prepend the keyword `async` to the function definition:

Cũng như async iterator, ta cũng có thể có async generator. Để định nghĩa một async generator function, chỉ cần thêm từ khoá `async` vào trước định nghĩa hàm:

```
async function * generatorFunction() {  
  // ...generator body  
}
```

As you can well imagine, async generators allow the use of the `await` instruction within their body and the return value of their `next()` method is a promise that resolves to an object having the canonical `done` and `value` properties. This way, async generator objects are also valid async iterators. They are also valid async iterables, so they can be used in `for await...of` loops.

Như bạn có thể hình dung, async generator cho phép dùng lệnh `await` bên trong thân của chúng và giá trị trả về của phương thức `next()` của chúng là một promise giải quyết (resolve) thành một đối tượng có các thuộc tính chuẩn mực `done` và `value`. Bằng cách này, các đối tượng async generator cũng là những async iterator hợp lệ. Chúng cũng là những async iterable hợp lệ, nên chúng có thể được dùng trong các vòng lặp `for await...of`.

To demonstrate how async generators can simplify the implementation of async iterators, let's convert the `CheckUrls` class we saw in the previous example to use an async generator:

Để minh hoạ cách async generator có thể đơn giản hoá việc hiện thực async iterator, hãy chuyển lớp `CheckUrls` mà ta đã thấy ở ví dụ trước sang dùng một async generator:

```
export class CheckUrls {  
  constructor (urls) {  
    this.urls = urls  
  }  
  async * [Symbol.asyncIterator] () {  
    for (const url of this.urls) {  
      try {
```



```

    const checkResult = await superagent
      .head(url)
      .redirects(2)
    yield `${url} is up, status: ${checkResult.status}`
  } catch (err) {
    yield `${url} is down, error: ${err.message}`
  }
}
}
}

```

Interestingly, using an async generator in place of a bare async iterator allowed us to save a few lines of code and the resulting logic is also more readable and explicit.

Thú vị là, việc dùng một async generator thay cho một async iterator trên đã cho phép ta tiết kiệm được vài dòng mã và logic kết quả cũng dễ đọc và tường minh hơn.

Async iterators and Node.js streams — Async iterator và Node.js stream

If we stop for a second and think about the relationship between async iterators and Node.js readable streams, we would be surprised by how similar they are in both purpose and behavior. In fact, we can say that async iterators are indeed a stream construct, as they can be used to process the data of an asynchronous resource piece by piece, exactly as it happens for readable streams.

Nếu ta dừng lại một giây và nghĩ về mối quan hệ giữa async iterator và readable stream của Node.js, ta sẽ ngạc nhiên trước mức độ giống nhau của chúng cả về mục đích lẫn hành vi. Thực tế, ta có thể nói rằng async iterator quả thực là một cấu trúc stream, vì chúng có thể được dùng để xử lý dữ liệu của một tài nguyên bất đồng bộ từng mảnh một, đúng như cách nó xảy ra với readable stream.

It's not a coincidence that `stream.Readable` implements the `@@asyncIterator` method, making it an async iterable. This provides us with an additional, and probably even more intuitive, mechanism to read data from a readable stream, thanks to the `for await...of` construct.

Không phải ngẫu nhiên mà `stream.Readable` hiện thực phương thức `@@asyncIterator`, biến nó thành một async iterable. Điều này cung cấp cho ta một cơ chế bổ sung, và có lẽ còn trực quan hơn, để đọc dữ liệu từ một readable stream, nhờ cấu trúc `for await...of`.

To quickly demonstrate this, consider the following example where we take the `stdin` stream of the current process and we **pipe** it into the `split()` transform stream, which will emit a new chunk when it finds a newline character. Then, we iterate over each line using the `for await...of` loop:

Để minh họa nhanh điều này, hãy xét ví dụ sau, trong đó ta lấy stream `stdin` của tiến trình hiện tại và ta nối ống (pipe) nó vào transform stream `split()`, stream này sẽ phát ra (emit) một chunk mới khi nó tìm thấy một ký tự xuống dòng. Sau đó, ta duyệt qua từng dòng bằng vòng lặp `for await...of`:

```

import split from 'split2'
async function main () {
  const stream = process.stdin.pipe(split())
  for await (const line of stream) {
    console.log(`You wrote: ${line}`)
  }
}

```

```
}  
main()
```

This sample code will print back whatever we have written to the standard input only after we have pressed the Return key. To quit the program, you can just press Ctrl + C.

Đoạn mã mẫu này sẽ in lại bất cứ thứ gì ta đã viết vào standard input chỉ sau khi ta đã nhấn phím Return. Để thoát chương trình, bạn chỉ cần nhấn Ctrl + C.

As we can see, this alternative way of consuming a readable stream is indeed very intuitive and compact. The previous example also shows us how similar the two paradigms—iterators and streams—are. They are so similar that they can interoperate almost seamlessly. To prove this point even further, just consider that the function `stream.Readable.from(iterable, [options])` takes an iterable as an argument, which can be both synchronous or asynchronous. The function will return a readable stream that wraps the provided iterable, “adapting” its interface to that of a readable stream (this is also a good example of the Adapter pattern, which we have already met in Chapter 8, Structural Design Patterns).

Như ta thấy, cách thay thế này để tiêu thụ một readable stream quả thực rất trực quan và súc tích. Ví dụ trước cũng cho ta thấy hai mô hình—iterator và stream—giống nhau đến mức nào. Chúng giống nhau đến mức có thể tương tác với nhau gần như liền mạch. Để chứng minh điểm này thêm nữa, chỉ cần xét rằng hàm `stream.Readable.from(iterable, [options])` nhận một iterable làm đối số, và iterable này có thể vừa là đồng bộ vừa là bất đồng bộ. Hàm sẽ trả về một readable stream bọc lấy iterable được cung cấp, “thích nghi” giao diện của nó thành giao diện của một readable stream (đây cũng là một ví dụ tốt về mẫu Adapter, mà ta đã gặp ở Chương 8, Structural Design Patterns).

So, if streams and async iterators are so closely related, which one should you actually use? This, as always, depends on the use case and many other factors; however, to help you with the decision, this is a list of aspects that set the two constructs apart:

Vậy, nếu stream và async iterator liên quan chặt chẽ đến vậy, thì thực tế bạn nên dùng cái nào? Điều này, như mọi khi, phụ thuộc vào trường hợp sử dụng và nhiều yếu tố khác; tuy nhiên, để giúp bạn ra quyết định, đây là một danh sách các khía cạnh phân biệt hai cấu trúc:

- Streams are push, meaning that data is pushed into the internal buffers by the stream and then consumed from the buffers. Async iterators are pull by default (unless another logic is explicitly implemented by the iterator), meaning that data is only retrieved/produced on demand by the consumer.
- Streams are better suited to process binary data since they natively provide internal buffering and **backpressure**.
- Streams can be composed using a well-known and streamlined API, `pipe()`, while async iterators do not offer any standardized composition method.
 - Stream là push (đẩy), nghĩa là dữ liệu được đẩy vào các buffer nội bộ bởi stream rồi được tiêu thụ từ các buffer. Async iterator mặc định là pull (kéo) (trừ khi một logic khác được iterator hiện thực một cách tường minh), nghĩa là dữ liệu chỉ được lấy về/sinh ra theo nhu cầu bởi bên tiêu thụ.
 - Stream phù hợp hơn để xử lý dữ liệu nhị phân vì chúng cung cấp sẵn buffer nội bộ và backpressure (áp lực ngược).
 - Stream có thể được kết hợp lại bằng một API nổi tiếng và tinh gọn, `pipe()`, trong khi async iterator không cung cấp bất kỳ phương pháp kết hợp chuẩn hoá nào.

We can iterate an `EventEmitter` as well. Using the `events.on(emitter, eventName)` utility function, we can in fact get an async iterable whose iterator will return all the events matching the specified `eventName`.

Ta cũng có thể duyệt qua một `EventEmitter`. Dùng hàm tiện ích `events.on(emitter, eventName)`, thực tế ta có thể lấy được một async iterable mà iterator của nó sẽ trả về tất cả các sự kiện khớp với `eventName` đã chỉ định.

In the wild — Trong thực tế

Iterators and, in particular, async iterators are quickly gaining popularity in the Node.js ecosystem. In fact, in many circumstances, they are becoming a preferred alternative to streams and are replacing custom-built iteration mechanisms.

Iterator và, đặc biệt, async iterator đang nhanh chóng trở nên phổ biến trong hệ sinh thái Node.js. Thực tế, trong nhiều hoàn cảnh, chúng đang trở thành một lựa chọn được ưa thích hơn so với stream và đang thay thế các cơ chế duyệt tự xây dựng.

For example, the packages `@databases/pg`, `@databases/mysql` and `@databases/sqlite` are popular libraries for accessing Postgres, MySQL, and SQLite databases respectively (more at nodejsdp.link/atdatabases). They all expose a function called `queryStream()`, which returns an async iterable, which can be used to easily iterate over the results of a query. For example:

Ví dụ, các gói `@databases/pg`, `@databases/mysql` và `@databases/sqlite` là những thư viện phổ biến để truy cập lần lượt các cơ sở dữ liệu Postgres, MySQL và SQLite (thêm tại nodejsdp.link/atdatabases). Tất cả chúng đều phơi bày một hàm tên là `queryStream()`, trả về một async iterable, có thể được dùng để duyệt dễ dàng qua các kết quả của một truy vấn. Ví dụ:

```
for await (const record of db.queryStream(sql`SELECT * FROM my_table`))
{
  // do something with record
}
```

Internally, the iterator will automatically handle the cursor for a query result, so all we have to do is simply loop with the `for await...of` construct.

Bên trong, iterator sẽ tự động xử lý con trỏ (cursor) cho một kết quả truy vấn, nên tất cả những gì ta phải làm chỉ đơn giản là lặp với cấu trúc `for await...of`.

Another example of a library heavily relying on iterators for its API is the `zeromq` package (nodejsdp.link/npm-zeromq). We'll see a detailed example of it in the next section, about the Middleware pattern, as we move on to other behavioral patterns.

Một ví dụ khác về một thư viện phụ thuộc nhiều vào iterator cho API của nó là gói `zeromq` (nodejsdp.link/npm-zeromq). Ta sẽ thấy một ví dụ chi tiết về nó ở mục tiếp theo, về mẫu Middleware, khi ta chuyển sang các mẫu hành vi khác.

Middleware — Middleware

One of the most distinctive patterns in Node.js is definitely Middleware. Unfortunately, it's also one of the most confusing for the inexperienced, especially for developers coming from the enterprise programming world. The reason for the disorientation is probably connected to the traditional meaning of the term middleware, which in enterprise architecture jargon represents

the various software suites that help to abstract lower-level mechanisms such as OS APIs, network communications, memory management, and so on, allowing the developer to focus only on the business case of the application. In this context, the term middleware recalls topics such as CORBA, enterprise service bus, Spring, JBoss, and WebSphere, but in its more generic meaning, it can also define any kind of software layer that acts as glue between lower-level services and the application (literally, the software in the middle).

Một trong những mẫu đặc trưng nhất trong Node.js chắc chắn là Middleware. Không may, nó cũng là một trong những mẫu gây bối rối nhất cho người thiếu kinh nghiệm, đặc biệt với các nhà phát triển đến từ thế giới lập trình doanh nghiệp. Lý do của sự mất phương hướng có lẽ liên quan đến nghĩa truyền thống của thuật ngữ middleware, vốn trong biệt ngữ kiến trúc doanh nghiệp đại diện cho các bộ phần mềm khác nhau giúp trừu tượng hoá những cơ chế cấp thấp hơn như API hệ điều hành, giao tiếp mạng, quản lý bộ nhớ, v.v., cho phép nhà phát triển chỉ tập trung vào nghiệp vụ của ứng dụng. Trong bối cảnh này, thuật ngữ middleware gợi nhớ các chủ đề như CORBA, enterprise service bus, Spring, JBoss và WebSphere, nhưng theo nghĩa tổng quát hơn, nó cũng có thể định nghĩa bất kỳ loại lớp phần mềm nào đóng vai trò keo dính giữa các dịch vụ cấp thấp hơn và ứng dụng (theo nghĩa đen, phần mềm ở giữa).

Middleware in Express — Middleware trong Express

Express (`nodejsdp.link/express`) popularized the term middleware in the Node.js world, binding it to a very specific **design pattern**. In Express, in fact, middleware represents a set of services, typically functions, that are organized in a pipeline and are responsible for processing incoming HTTP requests and relative responses.

Express (`nodejsdp.link/express`) đã phổ biến hoá thuật ngữ middleware trong thế giới Node.js, gắn nó với một mẫu thiết kế rất cụ thể. Thực tế, trong Express, middleware đại diện cho một tập các dịch vụ, thường là các hàm, được tổ chức trong một pipeline và chịu trách nhiệm xử lý các yêu cầu HTTP đến và các phản hồi tương ứng.

Express is famous for being a very non-opinionated and minimalist web framework and the Middleware pattern is the main reason for that. Express middleware is, in fact, an effective strategy for allowing developers to easily create and distribute new features that can be easily added to an application, without the need to grow the minimalistic core of the framework. An Express middleware has the following signature:

Express nổi tiếng là một web framework rất không áp đặt quan điểm (non-opinionated) và tối giản, và mẫu Middleware là lý do chính cho điều đó. Thực tế, Express middleware là một chiến lược hiệu quả để cho phép các nhà phát triển dễ dàng tạo và phân phối các tính năng mới có thể dễ dàng được thêm vào một ứng dụng, mà không cần phải mở rộng phần lõi tối giản của framework. Một Express middleware có chữ ký (signature) sau:

```
function (req, res, next) { ... }
```

Here, `req` is the incoming HTTP request, `res` is the response, and `next` is the **callback** to be invoked when the current middleware has completed its tasks, and that in turn triggers the next middleware in the pipeline.

Ở đây, `req` là yêu cầu HTTP đến, `res` là phản hồi, và `next` là callback được gọi khi middleware hiện tại đã hoàn thành nhiệm vụ của nó, và điều đó lần lượt kích hoạt middleware tiếp theo trong pipeline.

Examples of the tasks carried out by Express middleware include the following:

Ví dụ về các tác vụ được thực hiện bởi Express middleware bao gồm những điều sau:

- Parsing the body of the request
- Compressing/decompressing requests and responses
- Producing access logs
- Managing sessions
- Managing encrypted cookies
- Providing Cross-Site Request Forgery (CSRF) protection
 - Phân tích thân (body) của yêu cầu
 - Nén/giải nén các yêu cầu và phản hồi
 - Tạo nhật ký truy cập (access log)
 - Quản lý phiên (session)
 - Quản lý cookie được mã hoá
 - Cung cấp bảo vệ Cross-Site Request Forgery (CSRF)

If we think about it, these are all tasks that are not strictly related to the main business logic of an application, nor are they essential parts of the minimal core of a web server. They are accessories, components providing support to the rest of the application and allowing the actual request handlers to focus only on their main business logic. Essentially, those tasks are “software in the middle.”

Nếu ta nghĩ về điều đó, đây đều là những tác vụ không liên quan chặt chẽ đến nghiệp vụ chính của một ứng dụng, cũng không phải là những phần thiết yếu của phần lõi tối giản của một máy chủ web. Chúng là các phụ kiện, các thành phần cung cấp hỗ trợ cho phần còn lại của ứng dụng và cho phép các trình xử lý yêu cầu thực sự chỉ tập trung vào nghiệp vụ chính của chúng. Về bản chất, những tác vụ đó là “phần mềm ở giữa.”

Middleware as a pattern — Middleware như một mẫu

The technique used to implement middleware in Express is not new, in fact, it can be considered the Node.js incarnation of the Intercepting Filter pattern and the Chain of Responsibility pattern. In more generic terms, it also represents a processing pipeline, which reminds us of streams. Today, in Node.js, the word middleware is used well beyond the boundaries of the Express framework, and indicates a particular pattern whereby a set of processing units, filters, and handlers, under the form of functions, are connected to form an asynchronous sequence in order to perform the preprocessing and postprocessing of any kind of data. The main advantage of this pattern is flexibility. In fact, the Middleware pattern allows us to obtain a plugin infrastructure with incredibly little effort, providing an unobtrusive way to extend a system with new filters and handlers.

Kỹ thuật được dùng để hiện thực middleware trong Express không mới, thực tế, nó có thể được coi là hiện thân Node.js của mẫu Intercepting Filter và mẫu Chain of Responsibility. Theo các thuật ngữ tổng quát hơn, nó cũng đại diện cho một pipeline xử lý, gợi cho ta nhớ đến stream. Ngày nay, trong Node.js, từ middleware được dùng vượt xa ranh giới của framework Express, và biểu thị một mẫu cụ thể trong đó một tập các đơn vị xử lý, các filter và các handler, dưới dạng các hàm, được nối lại với nhau để tạo thành một trình tự bất đồng bộ nhằm thực hiện việc tiền xử lý và hậu xử lý bất kỳ loại dữ liệu nào. Lợi thế chính của mẫu này là tính linh hoạt. Thực tế, mẫu Middleware cho phép ta có được một hạ tầng plugin với nỗ lực nhỏ đến không ngờ, cung cấp một cách không gây xáo trộn để mở rộng một hệ thống với các filter và handler mới.

If you want to know more about the Intercepting Filter pattern, the following article is a good starting point: nodejsdp.link/intercepting-filter. Similarly, a nice overview of the Chain of Responsibility pattern is available at this URL: nodejsdp.link/chain-of-responsibility. The

following diagram shows the components of the Middleware pattern:

Nếu bạn muốn biết thêm về mẫu Intercepting Filter, bài viết sau là một điểm khởi đầu tốt: [nodejsdp.link/ intercepting-filter](https://nodejsdp.link/intercepting-filter). Tương tự, một cái nhìn tổng quan tốt về mẫu Chain of Responsibility có tại URL này: [nodejsdp.link/ chain-of-responsibility](https://nodejsdp.link/chain-of-responsibility). Sơ đồ sau cho thấy các thành phần của mẫu Middleware:

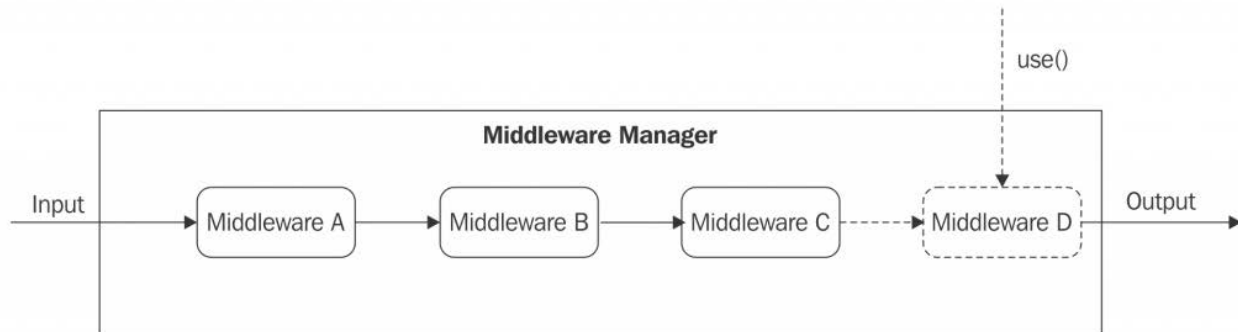


Figure 9.5: The structure of the Middleware pattern

Hình 9.5: Cấu trúc của mẫu Middleware

The essential component of the pattern is the Middleware Manager, which is responsible for organizing and executing the middleware functions. The most important implementation details of the pattern are as follows:

Thành phần thiết yếu của mẫu là Middleware Manager, chịu trách nhiệm tổ chức và thực thi các hàm middleware. Các chi tiết hiện thực quan trọng nhất của mẫu như sau:

- New middleware can be registered by invoking the `use()` function (the name of this function is a common convention in many implementations of the Middleware pattern, but we can choose any name). Usually, new middleware can only be appended at the end of the pipeline, but this is not a strict rule.
- When new data is received for processing, the registered middleware is invoked in an asynchronous **sequential execution** flow. Each unit in the pipeline receives the result of the execution of the previous unit as input.
- Each piece of middleware can decide to stop further processing of the data. This can be done by invoking a special function, by not invoking the callback (in case the middleware uses callbacks), or by propagating an error. An error situation usually triggers the execution of another sequence of middleware that is specifically dedicated to handling errors.
- Middleware mới có thể được đăng ký bằng cách gọi hàm `use()` (tên của hàm này là một quy ước phổ biến trong nhiều cách hiện thực của mẫu Middleware, nhưng ta có thể chọn bất kỳ tên nào). Thông thường, middleware mới chỉ có thể được nối thêm vào cuối pipeline, nhưng đây không phải là một quy tắc cứng nhắc.
- Khi dữ liệu mới được nhận để xử lý, các middleware đã đăng ký được gọi trong một luồng thực thi tuần tự bất đồng bộ. Mỗi đơn vị trong pipeline nhận kết quả của việc thực thi đơn vị trước làm đầu vào.
- Mỗi mảnh middleware có thể quyết định dừng việc xử lý dữ liệu thêm nữa. Điều này có thể được thực hiện bằng cách gọi một hàm đặc biệt, bằng cách không gọi callback (trong trường hợp middleware dùng callback), hoặc bằng cách lan truyền một lỗi. Một tình huống lỗi thường kích hoạt việc thực thi một trình tự middleware khác chuyên dụng riêng cho việc xử lý lỗi.

There is no strict rule on how the data is processed and propagated in the pipeline. The strategies for propagating the data modifications in the pipeline include:

Không có quy tắc cứng nhắc về cách dữ liệu được xử lý và lan truyền trong pipeline. Các chiến lược để lan truyền các sửa đổi dữ liệu trong pipeline bao gồm:

- Augmenting the data received as input with additional properties or functions
- Maintaining the immutability of the data and always return fresh copies as the result of the processing
 - Bổ sung (augment) dữ liệu nhận được làm đầu vào với các thuộc tính hoặc hàm bổ sung
 - Duy trì tính bất biến (immutability) của dữ liệu và luôn trả về các bản sao mới làm kết quả của việc xử lý

The right approach depends on the way the Middleware Manager is implemented and on the type of processing carried out by the middleware itself.

Cách tiếp cận đúng phụ thuộc vào cách Middleware Manager được hiện thực và vào loại xử lý được thực hiện bởi chính middleware.

Creating a middleware framework for ZeroMQ — Tạo một framework middleware cho ZeroMQ

Let's now demonstrate the pattern by building a middleware framework around the ZeroMQ (nodejsdp.link/zeromq) messaging library. ZeroMQ (also known as ZMQ, or ØMQ) provides a simple interface for exchanging atomic messages across the network using a variety of protocols. It shines for its performance, and its basic set of abstractions are specifically built to facilitate the implementation of custom messaging architectures. For this reason, ZeroMQ is often chosen to build complex distributed systems.

Giờ hãy minh họa mẫu bằng cách xây dựng một framework middleware xoay quanh thư viện nhắn tin ZeroMQ (nodejsdp.link/zeromq). ZeroMQ (còn được biết đến là ZMQ, hoặc ØMQ) cung cấp một giao diện đơn giản để trao đổi các thông điệp nguyên tử (atomic message) qua mạng bằng nhiều giao thức khác nhau. Nó nổi bật về hiệu năng, và tập các phép trừu tượng cơ bản của nó được xây dựng đặc biệt để tạo điều kiện cho việc hiện thực các kiến trúc nhắn tin tùy chỉnh. Vì lý do này, ZeroMQ thường được chọn để xây dựng các hệ thống phân tán phức tạp.

In Chapter 13, Messaging and Integration Patterns, we will have the chance to analyze the features of ZeroMQ in more detail.

Trong Chương 13, Messaging and Integration Patterns, ta sẽ có cơ hội phân tích các tính năng của ZeroMQ chi tiết hơn.

The interface of ZeroMQ is pretty low-level as it only allows us to use strings and binary buffers for messages. So, any encoding or custom formatting of data has to be implemented by the users of the library.

Giao diện của ZeroMQ khá cấp thấp vì nó chỉ cho phép ta dùng chuỗi và binary buffer cho các thông điệp. Vậy nên, bất kỳ việc mã hoá hoặc định dạng tùy chỉnh nào của dữ liệu đều phải được hiện thực bởi người dùng thư viện.

In the next example, we are going to build a middleware infrastructure to abstract the preprocessing and postprocessing of the data passing through a ZeroMQ socket, so that we can transparently work with JSON objects, but also seamlessly compress messages traveling over the wire.

Trong ví dụ tiếp theo, ta sẽ xây dựng một hạ tầng middleware để trừu tượng hoá việc tiền xử lý và hậu xử lý dữ liệu đi qua một ZeroMQ socket, sao cho ta có thể làm việc một cách trong suốt với các đối tượng JSON, mà cũng nén liền mạch các thông điệp di chuyển trên đường truyền.

The Middleware Manager — Middleware Manager

The first step toward building a middleware infrastructure around ZeroMQ is to create a component that is responsible for executing the middleware pipeline when a new message is received or sent. For this purpose, let's create a new module called `zmqMiddlewareManager.js` and let's define it:

Bước đầu tiên hướng tới việc xây dựng một hạ tầng middleware xoay quanh ZeroMQ là tạo một thành phần chịu trách nhiệm thực thi pipeline middleware khi một thông điệp mới được nhận hoặc gửi. Vì mục đích này, hãy tạo một module mới tên là `zmqMiddlewareManager.js` và định nghĩa nó:

```
export class ZmqMiddlewareManager {
  constructor (socket) { // (1)
    this.socket = socket
    this.inboundMiddleware = []
    this.outboundMiddleware = []
    this.handleIncomingMessages()
      .catch(err => console.error(err))
  }
  async handleIncomingMessages () { // (2)
    for await (const [message] of this.socket) {
      await this
        .executeMiddleware(this.inboundMiddleware, message)
        .catch(err => {
          console.error('Error while processing the message', err)
        })
    }
  }
  async send (message) { // (3)
    const finalMessage = await this
      .executeMiddleware(this.outboundMiddleware, message)
    return this.socket.send(finalMessage)
  }
  use (middleware) { // (4)
    if (middleware.inbound) {
      this.inboundMiddleware.push(middleware.inbound)
    }
    if (middleware.outbound) {
      this.outboundMiddleware.unshift(middleware.outbound)
    }
  }
  async executeMiddleware (middlewares, initialMessage) { // (5)
    let message = initialMessage
    for await (const middlewareFunc of middlewares) {
      message = await middlewareFunc.call(this, message)
    }
  }
}
```

```

    return message
}
}

```

Let's discuss in detail how we implemented our `ZmqMiddlewareManager`:

Hãy thảo luận chi tiết cách ta đã hiện thực `ZmqMiddlewareManager` của mình:

1. In the first part of the class, we define the constructor that accepts a ZeroMQ socket as an argument. In the constructor, we create two empty lists that will contain our middleware functions, one for inbound messages and another one for outbound messages. Next, we immediately start processing the messages coming from the socket. We do that in the `handleIncomingMessages()` method.
2. In the `handleIncomingMessages()` method, we use the ZeroMQ socket as an async iterable and with a `for await...of` loop, we process any incoming message and we pass it down the `inboundMiddleware` list of middlewares.
3. Similarly to `handleIncomingMessages()`, the `send()` method will pass the message received as an argument down the `outboundMiddleware` pipeline. The result of the processing is stored in the `finalMessage` variable and then sent through the socket.
4. The `use()` method is used for appending new middleware functions to our internal pipelines. In our implementation, each middleware comes in pairs; it's an object that contains two properties, `inbound` and `outbound`. Each property can be used to define the middleware function to be added to the respective list. It's important to observe here that the inbound middleware is pushed to the end of the `inboundMiddleware` list, while the outbound middleware is inserted (using `unshift()`) at the beginning of the `outboundMiddleware` list. This is because complementary inbound/outbound middleware functions usually need to be executed in inverted order. For example, if we want to decompress and then deserialize an inbound message using JSON, it means that for the outbound, we should instead first serialize and then compress. This convention for organizing the middleware in pairs is not strictly part of the general pattern, but only an implementation detail of our specific example.
5. The last method, `executeMiddleware()`, represents the core of our component as it's the part responsible for executing the middleware functions. Each function in the middleware array received as input is executed one after the other, and the result of the execution of a middleware function is passed to the next. Note that we are using the `await` instruction on each result returned by each middleware function; this allows the middleware function to return a value synchronously as well as asynchronously using a promise. Finally, the result of the last middleware function is returned back to the caller.

1. Ở phần đầu của lớp, ta định nghĩa constructor chấp nhận một ZeroMQ socket làm đối số. Trong constructor, ta tạo hai danh sách rỗng sẽ chứa các hàm middleware của ta, một cho các thông điệp đến (inbound) và một cho các thông điệp đi (outbound). Tiếp theo, ta ngay lập tức bắt đầu xử lý các thông điệp đến từ socket. Ta làm điều đó trong phương thức `handleIncomingMessages()`.
2. Trong phương thức `handleIncomingMessages()`, ta dùng ZeroMQ socket như một async iterable và với một vòng lặp `for await...of`, ta xử lý bất kỳ thông điệp đến nào và ta truyền nó xuôi xuống danh sách middleware `inboundMiddleware`.
3. Tương tự như `handleIncomingMessages()`, phương thức `send()` sẽ truyền message nhận được làm đối số xuôi xuống pipeline `outboundMiddleware`. Kết quả của việc xử lý được lưu trong biến `finalMessage` rồi được gửi qua socket.
4. Phương thức `use()` được dùng để nối thêm các hàm middleware mới vào các pipeline nội bộ của ta. Trong cách hiện thực của ta, mỗi middleware đến theo cặp; nó là một đối tượng chứa hai thuộc tính, `inbound` và `outbound`. Mỗi thuộc tính có thể

được dùng để định nghĩa hàm middleware được thêm vào danh sách tương ứng. Điều quan trọng cần quan sát ở đây là inbound middleware được đẩy (push) vào cuối danh sách inboundMiddleware, trong khi outbound middleware được chen (dùng unshift()) vào đầu danh sách outboundMiddleware. Điều này là vì các hàm middleware inbound/outbound hỗ trợ cho nhau thường cần được thực thi theo thứ tự đảo ngược. Ví dụ, nếu ta muốn giải nén rồi giải tuần tự hoá một thông điệp inbound bằng JSON, điều đó có nghĩa là đối với outbound, thay vào đó ta nên trước tiên tuần tự hoá rồi mới nén. Quy ước tổ chức middleware theo cặp này không phải là một phần chặt chẽ của mẫu tổng quát, mà chỉ là một chi tiết hiện thực của ví dụ cụ thể của ta.

5. Phương thức cuối cùng, executeMiddleware(), đại diện cho phần lõi của thành phần của ta vì nó là phần chịu trách nhiệm thực thi các hàm middleware. Mỗi hàm trong mảng middleware nhận làm đầu vào được thực thi lần lượt cái này sau cái kia, và kết quả của việc thực thi một hàm middleware được truyền cho hàm tiếp theo. Lưu ý rằng ta đang dùng lệnh await trên mỗi kết quả được trả về bởi mỗi hàm middleware; điều này cho phép hàm middleware trả về một giá trị một cách đồng bộ cũng như bất đồng bộ bằng một promise. Cuối cùng, kết quả của hàm middleware cuối cùng được trả về lại cho người gọi.

For brevity, we are not supporting an error middleware pipeline. Normally, when a middleware function propagates an error, another set of middleware functions specifically dedicated to handling errors is executed. This can be easily implemented using the same technique that we are demonstrating here. For instance, we could accept an extra (optional) errorMiddleware function in addition to inboundMiddleware and outboundMiddleware.

Để cho ngắn gọn, ta không hỗ trợ một pipeline middleware cho lỗi. Thông thường, khi một hàm middleware lan truyền một lỗi, một tập các hàm middleware khác chuyên dụng riêng cho việc xử lý lỗi được thực thi. Điều này có thể dễ dàng được hiện thực bằng cùng kỹ thuật mà ta đang minh hoạ ở đây. Chẳng hạn, ta có thể chấp nhận một hàm errorMiddleware bổ sung (tuỳ chọn) bên cạnh inboundMiddleware và outboundMiddleware.

Implementing the middleware to process messages — Hiện thực middleware để xử lý thông điệp

Now that we have implemented our Middleware Manager, we can create our first pair of middleware functions to demonstrate how to process inbound and outbound messages. As we said, one of the goals of our middleware infrastructure is to have a filter that serializes and deserializes JSON messages. So, let's create a new middleware to take care of this. In a new module called jsonMiddleware.js, let's include the following code:

Giờ ta đã hiện thực Middleware Manager của mình, ta có thể tạo cặp hàm middleware đầu tiên để minh hoạ cách xử lý các thông điệp inbound và outbound. Như ta đã nói, một trong những mục tiêu của hạ tầng middleware của ta là có một filter tuần tự hoá và giải tuần tự hoá các thông điệp JSON. Vậy, hãy tạo một middleware mới để lo việc này. Trong một module mới tên là jsonMiddleware.js, hãy đưa vào đoạn mã sau:

```
export const jsonMiddleware = function () {
  return {
    inbound (message) {
      return JSON.parse(message.toString())
    },
    outbound (message) {
      return Buffer.from(JSON.stringify(message))
    }
  }
}
```

```

    }
  }
}

```

The inbound part of our middleware deserializes the message received as input, while the outbound part serializes the data into a string, which is then converted into a buffer.

Phần inbound của middleware của ta giải tuần tự hoá thông điệp nhận được làm đầu vào, trong khi phần outbound tuần tự hoá dữ liệu thành một chuỗi, rồi chuỗi này được chuyển thành một buffer.

In a similar way, we can implement a pair of middleware functions in a file called `zlibMiddleware.js`, to inflate/deflate the message using the `zlib` core module (`nodejsdp.link/zlib`):

Theo cách tương tự, ta có thể hiện thực một cặp hàm middleware trong một tập tin tên là `zlibMiddleware.js`, để nén/giải nén (inflate/deflate) thông điệp dùng module lõi `zlib` (`nodejsdp.link/zlib`):

```

import { inflateRaw, deflateRaw } from 'zlib'
import { promisify } from 'util'
const inflateRawAsync = promisify(inflateRaw)
const deflateRawAsync = promisify(deflateRaw)
export const zlibMiddleware = function () {
  return {
    inbound (message) {
      return inflateRawAsync(Buffer.from(message))
    },
    outbound (message) {
      return deflateRawAsync(message)
    }
  }
}

```

Compared to the JSON middleware, our `zlib` middleware functions are asynchronous and return a promise as a result. As we already know, this is perfectly supported by our Middleware Manager.

So với JSON middleware, các hàm `zlib` middleware của ta là bất đồng bộ và trả về một promise làm kết quả. Như ta đã biết, điều này được Middleware Manager của ta hỗ trợ hoàn hảo.

You can note how the middleware used by our framework is quite different from the one used in Express. This is totally normal and a perfect demonstration of how we can adapt this pattern to fit our specific needs.

Bạn có thể lưu ý cách middleware được framework của ta dùng khá khác với cái được dùng trong Express. Điều này hoàn toàn bình thường và là một minh hoạ hoàn hảo về cách ta có thể điều chỉnh mẫu này để phù hợp với nhu cầu cụ thể của mình.

Using the ZeroMQ middleware framework — Sử dụng framework middleware cho ZeroMQ

We are now ready to use the middleware infrastructure that we just created. To do that, we are going to build a very simple application, with a client sending a ping to a server at regular intervals and the server echoing back the message received.

Giờ ta đã sẵn sàng sử dụng hạ tầng middleware mà ta vừa tạo. Để làm điều đó, ta sẽ xây dựng một ứng dụng rất đơn giản, với một client gửi một ping đến một máy chủ theo những

khoảng thời gian đều đặn và máy chủ vọng lại (echo) thông điệp nhận được.

From an implementation perspective, we are going to rely on a Request/**Reply** messaging pattern using the req/rep socket pair provided by ZeroMQ ([nodejsdp. link/zmq-req-rep](#)). We will then wrap the sockets with our ZmqMiddlewareManager to get all the advantages from the middleware infrastructure that we built, including the middleware for serializing/deserializing JSON messages.

Từ góc độ hiện thực, ta sẽ dựa vào một mẫu nhắn tin Request/Reply dùng cặp socket req/rep do ZeroMQ cung cấp ([nodejsdp. link/zmq-req-rep](#)). Sau đó ta sẽ bọc các socket bằng ZmqMiddlewareManager của ta để có được tất cả các lợi thế từ hạ tầng middleware mà ta đã xây dựng, bao gồm middleware để tuần tự hoá/giải tuần tự hoá các thông điệp JSON.

We'll analyze the Request/Reply pattern and other messaging patterns in Chapter 13, Messaging and Integration Patterns.

Ta sẽ phân tích mẫu Request/Reply và các mẫu nhắn tin khác trong Chương 13, Messaging and Integration Patterns.

The server — Máy chủ Let's start by creating the server-side of our application in a file called `server.js`:

Hãy bắt đầu bằng cách tạo phía máy chủ của ứng dụng trong một tập tin tên là `server.js`:

```
import zmq from 'zmq' // (1)
import { ZmqMiddlewareManager } from './zmqMiddlewareManager.js'
import { jsonMiddleware } from './jsonMiddleware.js'
import { zlibMiddleware } from './zlibMiddleware.js'
async function main () {
  const socket = new zmq.Reply() // (2)
  await socket.bind('tcp://127.0.0.1:5000')
  const zmqm = new ZmqMiddlewareManager(socket) // (3)
  zmqm.use(zlibMiddleware())
  zmqm.use(jsonMiddleware())
  zmqm.use({ // (4)
    async inbound (message) {
      console.log('Received', message)
      if (message.action === 'ping') {
        await this.send({ action: 'pong', echo: message.echo })
      }
      return message
    }
  })
  console.log('Server started')
}
main()
```

The server-side of our application works as follows:

Phía máy chủ của ứng dụng hoạt động như sau:

1. We first load the necessary dependencies. The `zmq` package is essentially a JavaScript interface over the native ZeroMQ library. See [nodejsdp. link/ npm-zmq](#).
2. Next, in the `main()` function, we create a new ZeroMQ Reply socket and bind it to port 5000 on localhost.

3. Then comes the part where we wrap ZeroMQ with our middleware manager and then add the zlib and JSON middlewares.
4. Finally, we are ready to handle a request coming from the client. We will do this by simply adding another middleware, this time using it as a request handler.
 1. Trước tiên ta nạp các phụ thuộc cần thiết. Gói zeromq về cơ bản là một giao diện JavaScript trên nền thư viện ZeroMQ gốc. Xem [nodejsdp.link/ npm-zeromq](https://nodejsdp.link/npm-zeromq).
 2. Tiếp theo, trong hàm main(), ta tạo một ZeroMQ Reply socket mới và bind nó vào cổng 5000 trên localhost.
 3. Rồi đến phần ta bọc ZeroMQ bằng middleware manager của mình rồi thêm các middleware zlib và JSON.
 4. Cuối cùng, ta đã sẵn sàng xử lý một yêu cầu đến từ client. Ta sẽ làm điều này bằng cách chỉ đơn giản thêm một middleware khác, lần này dùng nó như một request handler.

Since our request handler comes after the zlib and JSON middlewares, we will receive a decompressed and deserialized version of the received message. On the other hand, any data passed to send() will be processed by the outbound middleware, which in our case will serialize and then compress the data.

Vì request handler của ta đến sau các middleware zlib và JSON, ta sẽ nhận được một phiên bản đã được giải nén và giải tuần tự hoá của thông điệp nhận được. Mặt khác, bất kỳ dữ liệu nào được truyền cho send() sẽ được xử lý bởi outbound middleware, trong trường hợp của ta sẽ tuần tự hoá rồi nén dữ liệu.

The client — Client On the client-side of our little application, in a file called client.js, we will have the following code:

Ở phía client của ứng dụng nhỏ của ta, trong một tập tin tên là client.js, ta sẽ có đoạn mã sau:

```
import zeromq from 'zeromq'
import { ZmqMiddlewareManager } from './zmqMiddlewareManager.js'
import { jsonMiddleware } from './jsonMiddleware.js'
import { zlibMiddleware } from './zlibMiddleware.js'
async function main () {
  const socket = new zeromq.Request() // (1)
  await socket.connect('tcp://127.0.0.1:5000')
  const zmqm = new ZmqMiddlewareManager(socket)
  zmqm.use(zlibMiddleware())
  zmqm.use(jsonMiddleware())
  zmqm.use({
    inbound (message) {
      console.log('Echoed back', message)
      return message
    }
  })
  setInterval(() => { // (2)
    zmqm.send({ action: 'ping', echo: Date.now() })
    .catch(err => console.error(err))
  }, 1000)
  console.log('Client connected')
}
main()
```

Most of the code of the client application is very similar to that of the server. The notable differences are:

Phần lớn mã của ứng dụng client rất giống với mã của máy chủ. Những khác biệt đáng chú ý là:

1. We create a `Request` socket, rather than a `Reply` socket, and we connect it to a remote (or local) host rather than binding it on a local port. The rest of the middleware setup is exactly the same as in the server, except for the fact that our request handler now just prints any message it receives. Those messages should be the pong reply to our ping requests.
2. The core logic of the client application is a timer that sends a ping message every second.
 1. Ta tạo một `Request` socket, thay vì một `Reply` socket, và ta kết nối nó đến một host từ xa (hoặc cục bộ) thay vì bind nó vào một cổng cục bộ. Phần còn lại của việc thiết lập middleware giống hệt như ở máy chủ, ngoại trừ việc request handler của ta giờ chỉ in ra bất kỳ thông điệp nào nó nhận được. Những thông điệp đó nên là phản hồi pong cho các yêu cầu ping của ta.
 2. Logic lõi của ứng dụng client là một bộ định thời gửi một thông điệp ping mỗi giây.

Now, we're ready to try our client/server pair and see the application in action. First, start the server:

Giờ, ta đã sẵn sàng thử cặp client/server của mình và xem ứng dụng hoạt động. Trước tiên, khởi động máy chủ:

```
node server.js
```

We can then start the client in another terminal with the following command:

Sau đó ta có thể khởi động client trong một terminal khác bằng lệnh sau:

```
node client.js
```

At this point, we should see the client sending messages and the server echoing them back.

Lúc này, ta sẽ thấy client gửi các thông điệp và máy chủ vọng chúng lại.

Our middleware framework did its job. It allowed us to decompress/compress and deserialize/serialize our messages transparently, leaving the handlers free to focus on their business logic.

Framework middleware của ta đã hoàn thành nhiệm vụ của nó. Nó cho phép ta giải nén/nén và giải tuần tự hoá/tuần tự hoá các thông điệp của mình một cách trong suốt, để các handler được tự do tập trung vào nghiệp vụ của chúng.

In the wild — Trong thực tế

We opened this section by saying that the library that popularized the Middleware pattern in Node.js is Express (nodejsdp.link/express). So, we can easily say that Express is also the most notable example of the Middleware pattern out there.

Ta đã mở đầu mục này bằng cách nói rằng thư viện đã phổ biến hoá mẫu Middleware trong Node.js là Express (nodejsdp.link/express). Vậy, ta có thể dễ dàng nói rằng Express cũng là ví dụ đáng chú ý nhất về mẫu Middleware hiện có.

Two other interesting examples are:

Hai ví dụ thú vị khác là:

- Koa (nodejsdp.link/koa), which is known as the successor of Express. It was created by the same team behind Express and it shares with it its philosophy and main design principles.

Koa's middleware is slightly different than that of Express since it uses modern programming techniques such as `async/await` instead of callbacks.

- Middy (`nodejsdp.link/middy`) is a classic example of the Middleware pattern applied to something different than a web framework. Middy is, in fact, a middleware engine for AWS Lambda functions.
 - Koa (`nodejsdp.link/koa`), được biết đến là kế thừa của Express. Nó được tạo ra bởi cùng đội ngũ đứng sau Express và nó chia sẻ với Express triết lý cùng các nguyên tắc thiết kế chính. Middleware của Koa hơi khác với middleware của Express vì nó dùng các kỹ thuật lập trình hiện đại như `async/await` thay vì `callback`.
 - Middy (`nodejsdp.link/middy`) là một ví dụ kinh điển về mẫu Middleware được áp dụng cho thứ gì đó khác với một web framework. Middy thực tế là một middleware engine cho các hàm AWS Lambda.

Next, we are going to explore the Command pattern, which, as we will see shortly, is a very flexible and multiform pattern.

Tiếp theo, chúng ta sẽ khám phá mẫu Command, mà, như ta sẽ thấy ngay sau đây, là một mẫu rất linh hoạt và đa hình.

Command — Command

Another design pattern with huge importance in Node.js is Command. In its most generic definition, we can consider a command any object that encapsulates all the information necessary to perform an action at a later time. So, instead of invoking a method or a function directly, we create an object representing the intention to perform such an invocation. It will then be the responsibility of another component to materialize the intent, transforming it into an actual action. Traditionally, this pattern is built around four major components, as shown in Figure 9.6:

Một mẫu thiết kế khác có tầm quan trọng lớn trong Node.js là Command. Theo định nghĩa tổng quát nhất của nó, ta có thể coi là command bất kỳ đối tượng nào đóng gói toàn bộ thông tin cần thiết để thực hiện một hành động vào một thời điểm sau đó. Vậy, thay vì gọi một phương thức hoặc một hàm trực tiếp, ta tạo một đối tượng biểu diễn ý định thực hiện lần gọi đó. Sau đó sẽ là trách nhiệm của một thành phần khác để hiện thực hoá ý định, biến nó thành một hành động thực sự. Theo truyền thống, mẫu này được xây dựng xoay quanh bốn thành phần chính, như được thể hiện trong Hình 9.6:

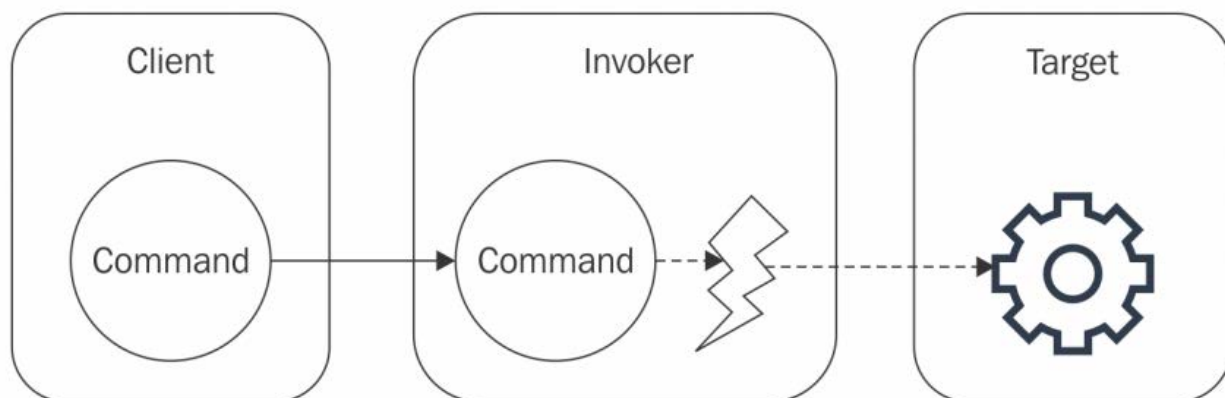


Figure 9.6: The components of the Command pattern The typical configuration of the Command pattern can be described as follows:

Hình 9.6: Các thành phần của mẫu Command. Cấu hình điển hình của mẫu Command có thể được mô tả như sau:

- Command is the object encapsulating the information necessary to invoke a method or function.
- Client is the component that creates the command and provides it to the invoker.
- Invoker is the component responsible for executing the command on the target.
- Target (or receiver) is the subject of the invocation. It can be a lone function or a method of an object.
 - Command là đối tượng đóng gói thông tin cần thiết để gọi một phương thức hoặc hàm.
 - Client là thành phần tạo ra command và cung cấp nó cho invoker.
 - Invoker là thành phần chịu trách nhiệm thực thi command trên target.
 - Target (hoặc receiver) là đối tượng của lần gọi. Nó có thể là một hàm đơn lẻ hoặc một phương thức của một đối tượng.

As we will see, these four components can vary a lot depending on the way we want to implement the pattern. This should not sound new at this point.

Như ta sẽ thấy, bốn thành phần này có thể thay đổi rất nhiều tùy theo cách ta muốn hiện thực mẫu. Điều này không nên nghe có vẻ mới mẻ vào lúc này.

Using the Command pattern instead of directly executing an operation has several applications:

Việc dùng mẫu Command thay vì trực tiếp thực thi một thao tác có một số ứng dụng:

- A command can be scheduled for execution at a later time.
- A command can be easily serialized and sent over the network. This simple property allows us to distribute jobs across remote machines, transmit commands from the browser to the server, create remote procedure call (RPC) systems, and so on.
- Commands make it easy to keep a history of all the operations executed on a system.
- Commands are an important part of some algorithms for data synchronization and conflict resolution.
- A command scheduled for execution can be canceled if it's not yet executed. It can also be reverted (undone), bringing the state of the application to the point before the command was executed.
- Several commands can be grouped together. This can be used to create atomic transactions or to implement a mechanism whereby all the operations in the group are executed at once.
- Different kinds of transformation can be performed on a set of commands, such as duplicate removal, joining and splitting, or applying more complex algorithms such as operational transformation (OT), which is the base for most of today's real-time collaborative software, such as collaborative text editing.
 - Một command có thể được lên lịch để thực thi vào một thời điểm sau đó.
 - Một command có thể dễ dàng được tuần tự hoá và gửi qua mạng. Đặc tính đơn giản này cho phép ta phân phối các công việc trên các máy từ xa, truyền các command từ trình duyệt đến máy chủ, tạo các hệ thống gọi thủ tục từ xa (RPC), v.v.
 - Command giúp dễ dàng giữ một lịch sử của tất cả các thao tác đã được thực thi trên một hệ thống.
 - Command là một phần quan trọng của một số thuật toán để đồng bộ hoá dữ liệu và giải quyết xung đột.
 - Một command được lên lịch để thực thi có thể bị huỷ nếu nó chưa được thực thi. Nó cũng có thể được hoàn tác (undo), đưa trạng thái của ứng dụng về điểm trước khi command được thực thi.
 - Nhiều command có thể được nhóm lại với nhau. Điều này có thể được dùng để tạo

các giao dịch nguyên tử (atomic transaction) hoặc để hiện thực một cơ chế trong đó tất cả các thao tác trong nhóm được thực thi cùng một lúc.

- Các loại biến đổi khác nhau có thể được thực hiện trên một tập command, chẳng hạn loại bỏ trùng lặp, ghép và tách, hoặc áp dụng các thuật toán phức tạp hơn như operational transformation (OT), vốn là nền tảng cho hầu hết phần mềm cộng tác thời gian thực ngày nay, chẳng hạn việc soạn thảo văn bản cộng tác.

A great explanation of how OT works can be found at [nodejsdp.link/operational-transformation](#). The preceding list clearly shows us how important this pattern is, especially on a platform such as Node.js where networking and asynchronous execution are essential players.

Một giải thích tuyệt vời về cách OT hoạt động có thể tìm thấy tại [nodejsdp.link/operational-transformation](#). Danh sách trên cho ta thấy rõ mẫu này quan trọng đến mức nào, đặc biệt trên một nền tảng như Node.js, nơi mạng và thực thi bất đồng bộ là những nhân tố thiết yếu.

Now, we are going to explore in more detail a couple of different implementations of the Command pattern, just to give you an idea of its scope.

Giờ, ta sẽ khám phá chi tiết hơn một vài cách hiện thực khác nhau của mẫu Command, chỉ để cho bạn một ý niệm về phạm vi của nó.

The Task pattern — Mẫu Task

We can start off with the most basic and trivial implementation of the Command pattern: the Task pattern. The easiest way in JavaScript to create an object representing an invocation is, of course, by creating a closure around a function definition or a bound function:

Ta có thể bắt đầu với cách hiện thực cơ bản và tầm thường nhất của mẫu Command: mẫu Task. Cách dễ nhất trong JavaScript để tạo một đối tượng biểu diễn một lần gọi, tất nhiên, là tạo một closure xoay quanh một định nghĩa hàm hoặc một hàm đã được bind:

```
function createTask(target, ...args) {  
  return () => {  
    target(...args)  
  }  
}
```

This is (mostly) equivalent to doing:

Điều này (về cơ bản) tương đương với việc làm:

```
const task = target.bind(null, ...args)
```

This should not look new at all. In fact, we have used this pattern already so many times throughout the book, and in particular in Chapter 4, Asynchronous **Control Flow** Patterns with Callbacks. This technique allowed us to use a separate component to control and schedule the execution of our tasks, which is essentially equivalent to the invoker of the Command pattern.

Điều này không nên trông mới mẻ chút nào. Thực tế, ta đã dùng mẫu này rất nhiều lần xuyên suốt cuốn sách, và đặc biệt trong Chương 4, Asynchronous Control Flow Patterns with Callbacks. Kỹ thuật này cho phép ta dùng một thành phần riêng biệt để điều khiển và lên lịch việc thực thi các task của mình, điều này về cơ bản tương đương với invoker của mẫu Command.

A more complex command — Một command phức tạp hơn

Let's now work on a more articulated example leveraging the Command pattern. This time, we want to support undo and serialization. Let's start with the target of our commands, a little object that is responsible for sending status updates to a Twitter-like service. We will use a mockup of such a service for simplicity (the `statusUpdateService.js` file):

Giờ hãy làm việc với một ví dụ được trình bày kỹ lưỡng hơn tận dụng mẫu Command. Lần này, ta muốn hỗ trợ undo và tuần tự hoá. Hãy bắt đầu với target của các command của ta, một đối tượng nhỏ chịu trách nhiệm gửi các cập nhật trạng thái đến một dịch vụ kiểu Twitter. Ta sẽ dùng một bản mô phỏng (mockup) của dịch vụ như vậy cho đơn giản (tập tin `statusUpdateService.js`):

```
const statusUpdates = new Map()
// The Target
export const statusUpdateService = {
  postUpdate (status) {
    const id = Math.floor(Math.random() * 10000000)
    statusUpdates.set(id, status)
    console.log(`Status posted: ${status}`)
    return id
  },
  destroyUpdate (id) => {
    statusUpdates.delete(id)
    console.log(`Status removed: ${id}`)
  }
}
```

The `statusUpdateService` we just created represents the target of our Command pattern. Now, let's implement a factory function that creates a command to represent the posting of a new status update. We'll do that in a file called `createPostStatusCmd.js`:

`statusUpdateService` mà ta vừa tạo đại diện cho target của mẫu Command của ta. Giờ, hãy hiện thực một factory function tạo ra một command để biểu diễn việc đăng một cập nhật trạng thái mới. Ta sẽ làm điều đó trong một tập tin tên là `createPostStatusCmd.js`:

```
export function createPostStatusCmd (service, status) {
  let postId = null
  // The Command
  return {
    run () {
      postId = service.postUpdate(status)
    },
    undo () {
      if (postId) {
        service.destroyUpdate(postId)
        postId = null
      }
    },
    serialize () {
      return { type: 'status', action: 'post', status: status }
    }
  }
}
```



```
}
```

The preceding function is a factory that produces commands to model “post status” intentions. Each command implements the following three functionalities:

Hàm trên là một factory tạo ra các command để mô hình hoá các ý định “post status”. Mỗi command hiện thực ba chức năng sau:

- A `run()` method that, when invoked, will trigger the action. In other words, it implements the Task pattern that we have seen before. The command, when executed, will post a new status update using the methods of the target service.
- An `undo()` method that reverts the effects of the post operation. In our case, we are simply invoking the `destroyUpdate()` method on the target service.
- A `serialize()` method that builds a JSON object that contains all the necessary information to reconstruct the same command object.
 - Một phương thức `run()` mà, khi được gọi, sẽ kích hoạt hành động. Nói cách khác, nó hiện thực mẫu Task mà ta đã thấy trước đó. Command, khi được thực thi, sẽ đăng một cập nhật trạng thái mới bằng các phương thức của dịch vụ target.
 - Một phương thức `undo()` đảo ngược các tác động của thao tác post. Trong trường hợp của ta, ta chỉ đơn giản gọi phương thức `destroyUpdate()` trên dịch vụ target.
 - Một phương thức `serialize()` xây dựng một đối tượng JSON chứa tất cả thông tin cần thiết để tái dựng lại cùng một đối tượng command.

After this, we can build an invoker. We can start by implementing its constructor and its `run()` method (the `invoker.js` file):

Sau điều này, ta có thể xây dựng một invoker. Ta có thể bắt đầu bằng cách hiện thực constructor của nó và phương thức `run()` của nó (tập tin `invoker.js`):

```
import superagent from 'superagent'
// The Invoker
export class Invoker {
  constructor () {
    this.history = []
  }
  run (cmd) {
    this.history.push(cmd)
    cmd.run()
    console.log('Command executed', cmd.serialize())
  }
  // ...rest of the class
```

The `run()` method is the basic functionality of our Invoker. It is responsible for saving the command into the history instance variable and then triggering the execution of the command itself.

Phương thức `run()` là chức năng cơ bản của Invoker của ta. Nó chịu trách nhiệm lưu command vào biến thực thể history rồi kích hoạt việc thực thi chính command đó.

Next, we can add to the Invoker a new method that delays the execution of a command:

Tiếp theo, ta có thể thêm vào Invoker một phương thức mới làm trì hoãn việc thực thi một command:

```
delay (cmd, delay) {
  setTimeout(() => {
    console.log('Executing delayed command', cmd.serialize())
```

```

    this.run(cmd)
  }, delay)
}

```

Then, we can implement an `undo()` method that reverts the last command:

Sau đó, ta có thể hiện thực một phương thức `undo()` đảo ngược command cuối cùng:

```

undo () {
  const cmd = this.history.pop()
  cmd.undo()
  console.log('Command undone', cmd.serialize())
}

```

Finally, we also want to be able to run a command on a remote server, by serializing and then transferring it over the network using a web service:

Cuối cùng, ta cũng muốn có thể chạy một command trên một máy chủ từ xa, bằng cách tuần tự hoá rồi truyền nó qua mạng bằng một dịch vụ web:

```

async runRemotely (cmd) {
  await superagent
    .post('http://localhost:3000/cmd')
    .send({ json: cmd.serialize() })
  console.log('Command executed remotely', cmd.serialize())
}

```

Now that we have the command, the invoker, and the target, the only component missing is the client, which we will implement in a file called `client.js`. Let's start by importing all the necessary dependencies and by instantiating `Invoker`:

Giờ ta đã có command, invoker và target, thành phần duy nhất còn thiếu là client, mà ta sẽ hiện thực trong một tập tin tên là `client.js`. Hãy bắt đầu bằng cách nạp tất cả các phụ thuộc cần thiết và bằng cách khởi tạo `Invoker`:

```

import { createPostStatusCmd } from './createPostStatusCmd.js'
import { statusUpdateService } from './statusUpdateService.js'
import { Invoker } from './invoker.js'
const invoker = new Invoker()

```

Then, we can create a command using the following line of code:

Sau đó, ta có thể tạo một command bằng dòng mã sau:

```

const command = createPostStatusCmd(statusUpdateService, 'HI!')

```

We now have a command representing the posting of a status message. We can then decide to dispatch it immediately:

Giờ ta đã có một command biểu diễn việc đăng một thông điệp trạng thái. Ta có thể quyết định gửi đi (dispatch) nó ngay lập tức:

```

invoker.run(command)

```

Oops, we made a mistake, let's revert our timeline to the state it was before posting the last message:

Ồi, ta đã phạm một sai lầm, hãy hoàn nguyên dòng thời gian của ta về trạng thái trước khi đăng thông điệp cuối cùng:

```

invoker.undo()

```

We can also decide to schedule the message to be sent in 3 seconds from now:

Ta cũng có thể quyết định lên lịch gửi thông điệp sau 3 giây kể từ bây giờ:

```
invoker.delay(command, 1000 * 3)
```

Alternatively, we can distribute the load of the application by migrating the task to another machine:

Hoặc, ta có thể phân tán tải của ứng dụng bằng cách di trú (migrate) task sang một máy khác:

```
invoker.runRemotely(command)
```

The little example that we have just implemented shows how wrapping an operation in a command can open a world of possibilities, and that's just the tip of the iceberg.

Ví dụ nhỏ mà ta vừa hiện thực cho thấy việc bọc một thao tác trong một command có thể mở ra cả một thế giới khả năng, và đó mới chỉ là phần nổi của tảng băng.

As the last remarks, it is worth noting that a fully-fledged Command pattern should be used only when strictly necessary. We saw, in fact, how much additional code we had to write to simply invoke a method of the `statusUpdateService`. If all that we need is only an invocation, then a complex command would be overkill. If, however, we need to schedule the execution of a task or run an asynchronous operation, then the simpler Task pattern offers the best compromise. If instead, we need more advanced features such as undo support, transformations, conflict resolution, or one of the other fancy use cases that we described previously, using a more complex representation for the command is almost necessary.

Như những nhận xét cuối cùng, đáng lưu ý rằng một mẫu Command đầy đủ chỉ nên được dùng khi thực sự cần thiết. Thực tế, ta đã thấy ta phải viết thêm bao nhiêu mã chỉ để gọi một phương thức của `statusUpdateService`. Nếu tất cả những gì ta cần chỉ là một lần gọi, thì một command phức tạp sẽ là quá mức cần thiết. Tuy nhiên, nếu ta cần lên lịch việc thực thi một task hoặc chạy một thao tác bất đồng bộ, thì mẫu Task đơn giản hơn mang lại sự thoả hiệp tốt nhất. Còn nếu, thay vào đó, ta cần các tính năng nâng cao hơn như hỗ trợ undo, biến đổi, giải quyết xung đột, hoặc một trong những trường hợp sử dụng hoa mỹ khác mà ta đã mô tả trước đó, thì việc dùng một biểu diễn phức tạp hơn cho command gần như là cần thiết.

Summary — Tóm tắt

We opened this chapter with three closely related patterns, which are Strategy, State, and Template.

Chúng ta đã mở đầu chương này với ba mẫu liên quan chặt chẽ với nhau, đó là Strategy, State và Template.

Strategy allows us to extract the common parts of a family of closely related components into a component called the context and allows us to define strategy objects that the context can use to implement specific behaviors. The State pattern is a variation of the Strategy pattern where the strategies are used to model the behavior of a component when under different states. The Template pattern, instead, can be considered the “static” version of the Strategy pattern, where the different specific behaviors are implemented as subclasses of the template class, which models the common parts of the component.

Strategy cho phép ta tách các phần chung của một họ các thành phần liên quan chặt chẽ thành một thành phần gọi là context và cho phép ta định nghĩa các đối tượng strategy mà context có thể dùng để hiện thực các hành vi cụ thể. Mẫu State là một biến thể của mẫu

Strategy trong đó các strategy được dùng để mô hình hoá hành vi của một thành phần khi ở dưới các trạng thái khác nhau. Mẫu Template, ngược lại, có thể được coi là phiên bản “tĩnh” của mẫu Strategy, trong đó các hành vi cụ thể khác nhau được hiện thực dưới dạng các lớp con của lớp template, lớp mô hình hoá các phần chung của thành phần.

Next, we learned about what has now become a core pattern in Node.js, which is Iterator. We learned how JavaScript offers native support for the pattern (with the iterator and iterable protocols), and how async iterators can be used as an alternative to complex async iteration patterns and even to Node.js streams.

Tiếp theo, chúng ta học về một mẫu giờ đã trở thành một mẫu cốt lõi trong Node.js, đó là Iterator. Ta học cách JavaScript cung cấp hỗ trợ gốc cho mẫu này (với các giao thức iterator và iterable), và cách async iterator có thể được dùng như một lựa chọn thay thế cho các mẫu duyệt bất đồng bộ phức tạp và thậm chí cho cả Node.js stream.

Then, we examined Middleware, which is a very distinctive pattern born from within the Node.js ecosystem. We learned how it can be used to preprocess and postprocess data and requests.

Sau đó, chúng ta xem xét Middleware, một mẫu rất đặc trưng sinh ra từ bên trong hệ sinh thái Node.js. Ta học cách nó có thể được dùng để tiền xử lý và hậu xử lý dữ liệu và yêu cầu.

Finally, we had a taste of the possibilities offered by the Command pattern, which can be used to implement a myriad of functionality, from simple undo/redo and serialization, to more complex operational transformation algorithms.

Cuối cùng, chúng ta nếm thử các khả năng do mẫu Command mang lại, mẫu này có thể được dùng để hiện thực vô số chức năng, từ undo/redo và tuần tự hoá đơn giản, đến các thuật toán operational transformation phức tạp hơn.

We have now arrived at the end of the last chapter dedicated to “traditional” design patterns. By now, you should have added to your toolbelt a series of patterns that will be enormously useful in your everyday programming endeavors. In the next chapter, we’ll shift our attention to a topic that goes beyond the boundaries of server-side development. Thanks to Node.js, in fact, we can create “Universal” JavaScript applications, or in other words, applications that can run as seamlessly on the server as they run on the browser. Stay tuned, then, to learn about the most useful **Universal JavaScript** patterns.

Giờ chúng ta đã đến cuối chương cuối cùng dành cho các mẫu thiết kế “truyền thống”. Đến lúc này, bạn đã thêm vào bộ đồ nghề của mình một loạt các mẫu sẽ vô cùng hữu ích trong những nỗ lực lập trình hằng ngày của bạn. Trong chương tiếp theo, chúng ta sẽ chuyển sự chú ý sang một chủ đề vượt ra ngoài ranh giới của phát triển phía máy chủ. Thực tế, nhờ Node.js, ta có thể tạo các ứng dụng JavaScript “Universal”, hay nói cách khác, các ứng dụng có thể chạy liên mạch trên máy chủ cũng như chạy trên trình duyệt. Vậy thì, hãy đón xem để học về những mẫu Universal JavaScript hữu ích nhất.

Exercises — Bài tập

- Exercise 9.1 Logging with Strategy: Implement a logging component having at least the following methods: `debug()`, `info()`, `warn()`, and `error()`. The logging component should also accept a strategy that defines where the log messages are sent. For example, we might have a `ConsoleStrategy` to send the messages to the console, or a `FileStrategy` to save the log messages to a file.
- Exercise 9.2 Logging with Template: Implement the same logging component we defined in the previous exercise, but this time using the Template pattern. We would then obtain a

ConsoleLogger class to log to the console or FileLogger class to log to a file. Appreciate the differences between the Template and the Strategy approaches.

- Exercise 9.3 Warehouse item: Imagine we are working on a warehouse management program. Our next task is to create a class to model a warehouse item and help track it. Such a WarehouseItem class has a constructor, which accepts an id and the initial state of the item (which can be one of arriving, stored, or delivered). It has three public methods:
 - store(locationId) moves the item into the stored state and records the locationId where it's stored.
 - deliver(address) changes the state of the item to delivered, sets the delivery address, and clears the locationId.
 - describe() returns a string representation of the current state of the item (for example, "Item 5821 is on its way to the warehouse," or "Item 3647 is stored in location 1ZH3," or "Item 3452 was delivered to John Smith, 1st Avenue, New York." The arriving state can be set only when the object is created as it cannot be transitioned to from the other states. An item can't move back to the arriving state once it's stored or delivered, it cannot be moved back to stored once it's delivered, and it cannot be delivered if it's not stored first. Use the State pattern to implement the WarehouseItem class.
- Exercise 9.4 Logging with Middleware: Rewrite the logging component you implemented for exercises 9.1 and 9.2, but this time use the Middleware pattern to postprocess each log message allowing different middlewares to customize how to handle the messages and how to output them. We could, for example, add a serialize() middleware to convert the log messages to a string representation ready to be sent over the wire or saved somewhere. Then, we could add a saveToFile() middleware that saves each message to a file. This exercise should highlight the flexibility and universality of the Middleware pattern.
- Exercise 9.5 Queues with iterators: Implement an AsyncQueue class similar to one of the TaskQueue classes we defined in Chapter 5, Asynchronous Control Flow Patterns with Promises and Async/Await, but with a slightly different behavior and interface. Such an AsyncQueue class will have a method called enqueue() to append new items to the queue and then expose an @@asyncIterable method, which should provide the ability to process the elements of the queue asynchronously, one at a time (so, with a **concurrency** of 1). The async iterator returned from AsyncQueue should terminate only after the done() method of AsyncQueue is invoked and only after all items in the queue are consumed. Consider that the @@asyncIterable method could be invoked in more than one place, thus returning an additional async iterator, which would allow you to increase the concurrency with which the queue is consumed.
 - Bài tập 9.1 Logging với Strategy: Hiện thực một thành phần logging có ít nhất các phương thức sau: debug(), info(), warn(), và error(). Thành phần logging này cũng nên chấp nhận một strategy định nghĩa nơi các thông điệp log được gửi đến. Ví dụ, ta có thể có một ConsoleStrategy để gửi các thông điệp đến console, hoặc một FileStrategy để lưu các thông điệp log vào một tập tin.
 - Bài tập 9.2 Logging với Template: Hiện thực cùng thành phần logging mà ta đã định nghĩa ở bài tập trước, nhưng lần này dùng mẫu Template. Khi đó ta sẽ thu được một lớp ConsoleLogger để log ra console hoặc lớp FileLogger để log vào một tập tin. Hãy cảm nhận sự khác biệt giữa cách tiếp cận Template và Strategy.
 - Bài tập 9.3 Mặt hàng trong kho: Hãy tưởng tượng ta đang làm việc trên một chương trình quản lý kho. Nhiệm vụ tiếp theo của ta là tạo một lớp để mô hình hoá một mặt hàng trong kho và giúp theo dõi nó. Lớp WarehouseItem như vậy có một constructor, chấp nhận một id và state ban đầu của mặt hàng (có thể là một trong các trạng thái arriving, stored, hoặc delivered). Nó có ba phương thức công khai:
 - store(locationId) chuyển mặt hàng vào trạng thái stored và ghi lại locationId

nơi nó được lưu.

- `deliver(address)` đổi trạng thái của mặt hàng thành `delivered`, đặt `address` giao hàng, và xoá `locationId`.
- `describe()` trả về một biểu diễn chuỗi của trạng thái hiện tại của mặt hàng (ví dụ, “Item 5821 is on its way to the warehouse,” hoặc “Item 3647 is stored in location 1ZH3,” hoặc “Item 3452 was delivered to John Smith, 1st Avenue, New York.” Trạng thái `arriving` chỉ có thể được đặt khi đối tượng được tạo vì không thể chuyển tiếp sang nó từ các trạng thái khác. Một mặt hàng không thể quay lại trạng thái `arriving` một khi nó đã `stored` hoặc `delivered`, nó không thể được chuyển trở lại `stored` một khi nó đã `delivered`, và nó không thể được `delivered` nếu nó không được `stored` trước. Dùng mẫu `State` để hiện thực lớp `WarehouseItem`.
- Bài tập 9.4 Logging với `Middleware`: Viết lại thành phần logging mà bạn đã hiện thực cho bài tập 9.1 và 9.2, nhưng lần này dùng mẫu `Middleware` để hậu xử lý mỗi thông điệp log cho phép các `middleware` khác nhau tùy biến cách xử lý các thông điệp và cách xuất chúng. Ta có thể, ví dụ, thêm một `middleware` `serialize()` để chuyển các thông điệp log thành một biểu diễn chuỗi sẵn sàng để được gửi qua đường truyền hoặc lưu ở đâu đó. Sau đó, ta có thể thêm một `middleware` `saveToFile()` lưu mỗi thông điệp vào một tập tin. Bài tập này nên làm nổi bật tính linh hoạt và tính phổ quát của mẫu `Middleware`.
- Bài tập 9.5 Hàng đợi với `iterator`: Hiện thực một lớp `AsyncQueue` tương tự một trong các lớp `TaskQueue` mà ta đã định nghĩa ở Chương 5, `Asynchronous Control Flow Patterns with Promises and Async/Await`, nhưng với hành vi và giao diện hơi khác. Lớp `AsyncQueue` như vậy sẽ có một phương thức tên là `enqueue()` để nối thêm các mục mới vào hàng đợi rồi phơi bày một phương thức `@@asyncIterable`, phương thức này nên cung cấp khả năng xử lý các phần tử của hàng đợi một cách bất đồng bộ, mỗi lần một phần tử (vậy, với một mức đồng thời là 1). `Async iterator` được trả về từ `AsyncQueue` chỉ nên kết thúc sau khi phương thức `done()` của `AsyncQueue` được gọi và chỉ sau khi tất cả các mục trong hàng đợi được tiêu thụ. Hãy lưu ý rằng phương thức `@@asyncIterable` có thể được gọi ở nhiều hơn một nơi, do đó trả về một `async iterator` bổ sung, điều này sẽ cho phép bạn tăng mức đồng thời mà hàng đợi được tiêu thụ.

Chapter 10: Universal JavaScript for Web Applications — Chương 10: Universal JavaScript cho ứng dụng web

JavaScript was born with the goal of giving web developers the power to execute code directly on the browser and build dynamic and interactive websites.

JavaScript ra đời với mục tiêu trao cho các nhà phát triển web khả năng thực thi mã trực tiếp trên trình duyệt và xây dựng những website động, có tính tương tác.

Since its inception, JavaScript has grown up a lot. If, at the very beginning, JavaScript was a very simple and limited language, today, it can be considered a complete general-purpose language that can be used even outside the browser to build almost any kind of application. In fact, JavaScript now powers frontend applications, web servers, and mobile applications, as well as embedded devices such as wearable devices, thermostats, and flying drones.

Kể từ khi ra đời, JavaScript đã trưởng thành rất nhiều. Nếu thuở ban đầu JavaScript chỉ là một ngôn ngữ rất đơn giản và hạn chế, thì ngày nay nó có thể được xem là một ngôn ngữ đa dụng (general-purpose) hoàn chỉnh, thậm chí có thể dùng bên ngoài trình duyệt để xây dựng gần như mọi loại ứng dụng. Thực tế, JavaScript hiện đang vận hành các ứng dụng frontend, máy chủ web và ứng dụng di động, cũng như các thiết bị nhúng (embedded) như thiết bị đeo (wearable), bộ điều nhiệt và drone bay.

The language's availability across platforms and devices is fostering a new trend among JavaScript developers: being able to simplify code reuse across different environments in the same project. With Node.js, developers have the opportunity to build web applications where it is easy to share code between the server (backend) and the browser (frontend). This quest for code reuse was originally identified with the term Isomorphic JavaScript, but today, it's mostly recognized as **Universal JavaScript**.

Sự sẵn có của ngôn ngữ này trên nhiều nền tảng và thiết bị đang nuôi dưỡng một xu hướng mới trong cộng đồng lập trình viên JavaScript: khả năng đơn giản hoá việc tái sử dụng mã giữa các môi trường khác nhau trong cùng một dự án. Với Node.js, lập trình viên có cơ hội xây dựng các ứng dụng web nơi việc chia sẻ mã giữa máy chủ (backend) và trình duyệt (frontend) trở nên dễ dàng. Cuộc truy tìm khả năng tái sử dụng mã này ban đầu được gọi bằng thuật ngữ Isomorphic JavaScript, nhưng ngày nay nó chủ yếu được biết đến với tên Universal JavaScript.

In this chapter, we are going to explore the wonders of Universal JavaScript, specifically in the field of web development, and discover many tools and techniques we can use to share code between the server and the browser. We will explore what a **module bundler** is and why we need one. We will then learn how module bundlers work and we will practice with one of the most popular, webpack. Then, we will discuss some generic patterns that can help us with code reuse across platforms.

Trong chương này, chúng ta sẽ khám phá những điều kỳ diệu của Universal JavaScript, cụ thể trong lĩnh vực phát triển web, và tìm hiểu nhiều công cụ cùng kỹ thuật có thể dùng để chia sẻ mã giữa máy chủ và trình duyệt. Chúng ta sẽ khám phá module bundler là gì và tại sao chúng ta cần đến nó. Sau đó, chúng ta sẽ tìm hiểu cách các module bundler hoạt động và thực hành với một trong những công cụ phổ biến nhất, webpack. Tiếp theo, chúng ta sẽ bàn về một số mẫu (pattern) tổng quát có thể giúp ích cho việc tái sử dụng mã giữa các nền tảng.

Finally, we will learn the basic functionalities of React and we will use it to build a complete Universal JavaScript application that features universal rendering, universal routing, and universal data loading.

Cuối cùng, chúng ta sẽ tìm hiểu các chức năng cơ bản của React và dùng nó để xây dựng một ứng dụng Universal JavaScript hoàn chỉnh có khả năng kết xuất universal (universal rendering), định tuyến universal (universal routing) và tải dữ liệu universal (universal data loading).

To summarize, here's a list of topics we will be covering in this chapter:

Tóm lại, dưới đây là danh sách các chủ đề chúng ta sẽ đề cập trong chương này:

- How to share code between the browser and Node.js
- Fundamentals of cross-platform development (code branching, module swapping, and other useful patterns)
- A brief introduction to React
- How to build a complete Universal JavaScript application using React and Node.js
 - Cách chia sẻ mã giữa trình duyệt và Node.js
 - Những nền tảng cốt lõi của phát triển đa nền tảng (cross-platform) (rẽ nhánh mã, hoán đổi module và các mẫu hữu ích khác)
 - Giới thiệu ngắn gọn về React
 - Cách xây dựng một ứng dụng Universal JavaScript hoàn chỉnh bằng React và Node.js

Sit tight, this is going to be an exciting chapter!

Ngồi vững nhé, đây sẽ là một chương đầy hứng khởi!

Sharing code with the browser — Chia sẻ mã với trình duyệt

One of the main selling points of Node.js is the fact that it's based on JavaScript and runs on V8, a JavaScript engine that actually powers some of the most popular browsers: Google Chrome and Microsoft Edge. We might think that sharing the same JavaScript engine is enough to make sharing code between Node.js and the browser an easy task; however, as we will see in this chapter, this is not always true, unless we want to share only simple, self-contained, and generic fragments of code.

Một trong những điểm hấp dẫn chính của Node.js là việc nó dựa trên JavaScript và chạy trên V8, một JavaScript engine vốn đang vận hành một số trình duyệt phổ biến nhất: Google Chrome và Microsoft Edge. Chúng ta có thể nghĩ rằng việc dùng chung một JavaScript engine là đủ để biến việc chia sẻ mã giữa Node.js và trình duyệt thành một

nhệm vụ dễ dàng; tuy nhiên, như chúng ta sẽ thấy trong chương này, điều đó không phải lúc nào cũng đúng, trừ khi chúng ta chỉ muốn chia sẻ những đoạn mã đơn giản, khép kín (self-contained) và tổng quát.

Developing code for both the client and the server requires a non-negligible level of effort in making sure that the same code can run properly in two environments that are intrinsically different. For example, in Node.js, we don't have the DOM or longliving views, while on the browser, we surely don't have the filesystem and many other interfaces to interact with the underlying operating system.

Phát triển mã cho cả phía client lẫn máy chủ đòi hỏi một mức công sức không nhỏ để đảm bảo rằng cùng một đoạn mã có thể chạy đúng đắn trong hai môi trường vốn khác biệt về bản chất. Ví dụ, trong Node.js, chúng ta không có DOM hay các khung nhìn (view) tồn tại lâu dài, trong khi trên trình duyệt, chắc chắn chúng ta không có hệ thống tập tin (filesystem) và nhiều giao diện khác để tương tác với hệ điều hành bên dưới.

Another contention point is the level of support for modern JavaScript features. When we target Node.js, we can safely adopt modern language features because we know which Node.js version runs on our servers. For instance, for our server code, we can safely decide to adopt **async/await** if we know it will run on Node.js version 8 (or on a more recent version). Unfortunately, we can't have the same confidence when writing JavaScript code for the browser. This is because different users will have different browsers with different levels of compatibility with the latest language features. Some users might be using a modern browser with full support for async/await, while other users might still be using an old device with an old browser that does not support async/await.

Một điểm gây tranh cãi khác là mức độ hỗ trợ các tính năng JavaScript hiện đại. Khi nhắm tới Node.js, chúng ta có thể yên tâm áp dụng các tính năng ngôn ngữ hiện đại vì biết rõ phiên bản Node.js nào đang chạy trên máy chủ của mình. Chẳng hạn, với mã máy chủ, chúng ta có thể yên tâm quyết định dùng async/await nếu biết nó sẽ chạy trên Node.js phiên bản 8 (hoặc phiên bản mới hơn). Tiếc thay, chúng ta không thể có sự tự tin tương tự khi viết mã JavaScript cho trình duyệt. Lý do là vì những người dùng khác nhau sẽ dùng những trình duyệt khác nhau với các mức độ tương thích khác nhau đối với những tính năng ngôn ngữ mới nhất. Một số người dùng có thể đang dùng trình duyệt hiện đại hỗ trợ đầy đủ async/await, trong khi những người khác có thể vẫn đang dùng thiết bị cũ với trình duyệt cũ không hỗ trợ async/await.

So, most of the effort required when developing for both platforms is to make sure to reduce those differences to a minimum. This can be done with the help of abstractions, patterns, and tools that enable the application to switch, dynamically or at build time, between browser-compatible code and Node.js code.

Vì vậy, phần lớn công sức cần bỏ ra khi phát triển cho cả hai nền tảng là đảm bảo giảm thiểu những khác biệt đó tới mức tối thiểu. Điều này có thể thực hiện với sự trợ giúp của các lớp trừu tượng, mẫu thiết kế và công cụ cho phép ứng dụng chuyển đổi, một cách động hoặc tại thời điểm build, giữa mã tương thích trình duyệt và mã Node.js.

Luckily, with the rising interest in this new mind-blowing possibility, many libraries and frameworks in the ecosystem have started to support both environments. This evolution is also backed by a growing number of tools supporting this new kind of workflow, which, over the years, have been refined and perfected. This means that if we are using an npm **package** on Node.js, there is a good probability that it will work seamlessly on the browser as well. However, this is often not enough to guarantee that our application can run without problems on both the browser and Node.js. As we will see, a careful design is always needed when developing cross-platform code.

May thay, với sự quan tâm ngày càng tăng dành cho khả năng mới mẻ đầy hấp dẫn này,

nhiều thư viện và framework trong hệ sinh thái đã bắt đầu hỗ trợ cả hai môi trường. Sự tiến hoá này còn được củng cố bởi số lượng ngày càng nhiều các công cụ hỗ trợ kiểu quy trình làm việc mới này, vốn qua nhiều năm đã được tinh chỉnh và hoàn thiện. Điều này có nghĩa là nếu chúng ta đang dùng một gói npm trên Node.js, nhiều khả năng nó cũng sẽ hoạt động trơn tru trên trình duyệt. Tuy nhiên, điều đó thường vẫn chưa đủ để đảm bảo ứng dụng của chúng ta có thể chạy không vấn đề trên cả trình duyệt lẫn Node.js. Như chúng ta sẽ thấy, một thiết kế cẩn thận luôn là cần thiết khi phát triển mã đa nền tảng.

In this section, we are going to explore the fundamental problems we might encounter when writing code for both Node.js and the browser, and we are going to propose some tools and patterns that can help us with tackling this new and exciting challenge.

Trong phần này, chúng ta sẽ khám phá những vấn đề cốt lõi có thể gặp khi viết mã cho cả Node.js và trình duyệt, đồng thời đề xuất một số công cụ và mẫu thiết kế có thể giúp chúng ta đương đầu với thử thách mới mẻ và hấp dẫn này.

JavaScript modules in a cross-platform context — Module JavaScript trong ngữ cảnh đa nền tảng

The first wall we hit when we want to share some code between the browser and the server is the mismatch between the **module system** used by Node.js and the heterogeneous landscape of the module systems used on the browser. Another problem is that on the browser, we don't have a `require()` function or the filesystem from which we can resolve modules. Most modern browsers support `import` and ES modules, but again, some of the users visiting our website might not have already adopted one of those modern browsers.

Bức tường đầu tiên chúng ta đụng phải khi muốn chia sẻ mã giữa trình duyệt và máy chủ là sự lệch pha giữa hệ thống module mà Node.js dùng và bức tranh đa dạng của các hệ thống module được dùng trên trình duyệt. Một vấn đề khác là trên trình duyệt, chúng ta không có hàm `require()` hay hệ thống tập tin để từ đó phân giải các module. Hầu hết các trình duyệt hiện đại hỗ trợ `import` và ES module, nhưng một lần nữa, một số người dùng truy cập website của chúng ta có thể chưa chuyển sang dùng một trong các trình duyệt hiện đại đó.

In addition to these problems, we have to take into account the differences in distributing code for the server and the browser. On the server, modules are loaded directly from the filesystem. This is generally a performant operation and therefore developers are encouraged to split their code into small modules to keep the different logic units small and organized. On the browser, the script loading model is totally different. The process generally starts with the browser downloading an HTML page from a remote endpoint. The HTML code is parsed by the browser, which might find references to script files that need to be downloaded and executed. If we are dealing with a large application, there might be many scripts to download, so the browser will have to issue a significant number of HTTP requests and download and parse multiple script files before the application can be fully initialized. The higher the number of script files, the larger the performance penalty that we will have to pay to run an application on the browser, especially on slow networks. Even though some of this performance penalty can be mitigated with the adoption of HTTP/2 Server Push (`nodejsdp.link/http2-server-push`), client-side caching, preloading, or similar techniques, the underlying problem still stands: having to receive and parse a large number of files is generally worse than having to deal with a few optimized files.

Bên cạnh những vấn đề này, chúng ta phải tính đến những khác biệt trong việc phân phối mã cho máy chủ và trình duyệt. Trên máy chủ, các module được tải trực tiếp từ hệ thống tập tin. Đây nói chung là một thao tác hiệu năng cao, nên lập trình viên được khuyến khích

chia mã thành các module nhỏ để giữ cho các đơn vị logic khác nhau nhỏ gọn và có tổ chức. Trên trình duyệt, mô hình tải script hoàn toàn khác. Quá trình thường bắt đầu với việc trình duyệt tải về một trang HTML từ một endpoint từ xa. Mã HTML được trình duyệt phân tích cú pháp, và nó có thể tìm thấy các tham chiếu tới các tập tin script cần được tải về và thực thi. Nếu chúng ta đang xử lý một ứng dụng lớn, có thể có rất nhiều script cần tải về, nên trình duyệt sẽ phải phát ra một số lượng đáng kể các yêu cầu HTTP, tải về và phân tích cú pháp nhiều tập tin script trước khi ứng dụng có thể được khởi tạo đầy đủ. Số lượng tập tin script càng nhiều thì hình phạt hiệu năng mà chúng ta phải trả để chạy ứng dụng trên trình duyệt càng lớn, đặc biệt trên các mạng chậm. Dù một phần hình phạt hiệu năng này có thể được giảm nhẹ bằng việc áp dụng HTTP/2 Server Push ([nodejsdp . link/http2-server-push](https://nodejs.org/en/docs/guides/http2-server-push/)), bộ nhớ đệm phía client (client-side caching), preloading hoặc các kỹ thuật tương tự, vấn đề cốt lõi vẫn còn đó: phải nhận và phân tích cú pháp một số lượng lớn tập tin nói chung tệ hơn so với phải xử lý một vài tập tin đã được tối ưu.

A common practice to address this problem is to “build” packages (or bundles) for the browser. A typical build process will collate all the source files into a very small number of bundles (for instance, one JavaScript file per page) so that the browser won’t have to download a huge number of scripts for each page visit. A build process is not limited to just reducing the number of files, in fact, it can perform other interesting optimizations. Another common optimization is code minification, which allows us to reduce the number of characters to a minimum without altering the functionality. This is generally done by removing comments, removing unused code, and renaming function and variable names.

Một thực hành phổ biến để giải quyết vấn đề này là “build” các gói (hoặc bundle) cho trình duyệt. Một quy trình build điển hình sẽ gom tất cả các tập tin mã nguồn lại thành một số rất ít các bundle (ví dụ, một tập tin JavaScript cho mỗi trang) sao cho trình duyệt không phải tải về một số lượng khổng lồ các script cho mỗi lượt truy cập trang. Một quy trình build không chỉ giới hạn ở việc giảm số lượng tập tin, thực tế nó còn có thể thực hiện những tối ưu thú vị khác. Một tối ưu phổ biến khác là rút gọn mã (code minification), cho phép chúng ta giảm số ký tự xuống mức tối thiểu mà không làm thay đổi chức năng. Điều này thường được thực hiện bằng cách loại bỏ chú thích, loại bỏ mã không dùng đến, và đổi tên các hàm và biến.

Module bundlers — Module bundler

If we want to write large portions of code that can work as seamlessly as possible both on the server and on the browser, we need a tool to help us with “bundling” all the dependencies together at build time. These tools are generally called module bundlers. Let’s visualize this with an example of how shared code can be loaded on to the server and the client using a module bundler:

Nếu chúng ta muốn viết những phần mã lớn có thể hoạt động trơn tru nhất có thể trên cả máy chủ và trình duyệt, chúng ta cần một công cụ giúp “đóng gói” (bundling) tất cả các phụ thuộc lại với nhau tại thời điểm build. Những công cụ này thường được gọi là module bundler. Hãy hình dung điều này qua một ví dụ về cách mã chia sẻ có thể được tải lên máy chủ và client bằng một module bundler:

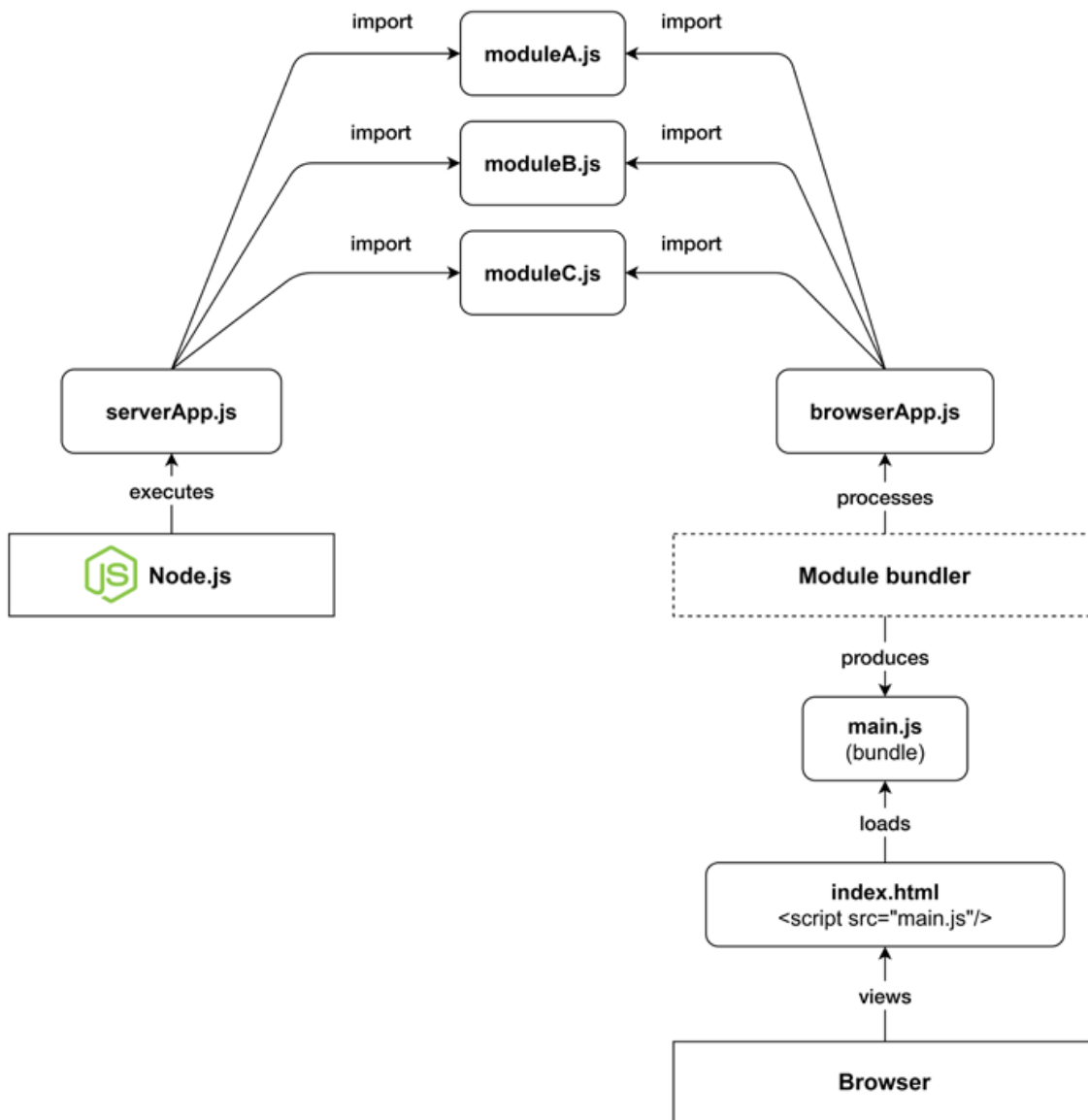


Figure 10.1: Loading shared modules on the server and on the browser (using a module bundler) By looking at Figure 10.1, we can see that the code is processed and loaded differently on the server side and on the browser:

Hình 10.1: Tải các module chia sẻ trên máy chủ và trên trình duyệt (dùng một module bundler) Nhìn vào Hình 10.1, chúng ta có thể thấy mã được xử lý và tải khác nhau ở phía máy chủ và trên trình duyệt:

- On the server side: Node.js can directly execute our `serverApp.js`, which, in turn, will import the modules `moduleA.js`, `moduleB.js`, and `moduleC.js`.
- On the browser: We have `browserApp.js`, which also imports `moduleA.js`, `moduleB.js`, and `moduleC.js`. If our index file were to include `browserApp.js` directly, we would have to download a total of five files (`index.html`, `browserApp.js`, and the three **dependency** modules) before the app would be fully initialized. The module bundler allows us to reduce the total number of files to only two by preprocessing `browserApp.js` and all its dependencies and

producing a single equivalent bundle called `main.js`, which is then referenced by `index.html` and therefore loaded by the browser.

- Ở phía máy chủ: Node.js có thể trực tiếp thực thi `serverApp.js` của chúng ta, mà đến lượt nó sẽ import các module `moduleA.js`, `moduleB.js` và `moduleC.js`.
- Trên trình duyệt: Chúng ta có `browserApp.js`, vốn cũng import `moduleA.js`, `moduleB.js` và `moduleC.js`. Nếu tập tin `index` của chúng ta nhúng trực tiếp `browserApp.js`, chúng ta sẽ phải tải về tổng cộng năm tập tin (`index.html`, `browserApp.js`, và ba module phụ thuộc) trước khi ứng dụng được khởi tạo đầy đủ. Module bundler cho phép chúng ta giảm tổng số tập tin xuống chỉ còn hai bằng cách tiền xử lý `browserApp.js` cùng tất cả các phụ thuộc của nó và tạo ra một bundle tương đương duy nhất tên là `main.js`, sau đó được tham chiếu bởi `index.html` và do đó được trình duyệt tải về.

To summarize, on the browser, we generally have to deal with two logical phases, build and **runtime**, while on the server, we generally don't need a build phase and we can execute our source code directly.

Tóm lại, trên trình duyệt, chúng ta thường phải xử lý hai pha logic, build và runtime, trong khi trên máy chủ, chúng ta thường không cần pha build và có thể thực thi trực tiếp mã nguồn của mình.

When it comes to picking a module bundler, the most popular option is probably webpack (nodejsdp.link/webpack). Webpack is one of the most complete and mature module bundlers currently available and it is the one we are going to use in this chapter. It's worth mentioning, though, that there is a quite prosperous ecosystem full of alternatives, each one with its own strengths. If you are curious, here are some of the most well-known alternatives to webpack:

Khi nói đến việc chọn một module bundler, lựa chọn phổ biến nhất có lẽ là webpack (nodejsdp.link/webpack). Webpack là một trong những module bundler hoàn thiện và trưởng thành nhất hiện có, và nó là công cụ chúng ta sẽ dùng trong chương này. Tuy vậy, đáng nói là có một hệ sinh thái khá thịnh vượng đầy các lựa chọn thay thế, mỗi cái có thể mạnh riêng. Nếu bạn tò mò, dưới đây là một số lựa chọn thay thế webpack nổi tiếng nhất:

- Parcel (nodejsdp.link/parcel): Aims to be fast and to work “auto-magically” without any configuration.
- Rollup (nodejsdp.link/rollup): One of the first module bundlers to fully support **ESM** and to offer a number of optimizations like tree shaking and dead code elimination.
- Browserify (nodejsdp.link/browserify): The first module bundler with support for **CommonJS** and is still widely adopted.
 - Parcel (nodejsdp.link/parcel): Hướng tới sự nhanh chóng và hoạt động “tự động như có phép màu” mà không cần bất kỳ cấu hình nào.
 - Rollup (nodejsdp.link/rollup): Một trong những module bundler đầu tiên hỗ trợ đầy đủ ESM và cung cấp một số tối ưu như tree shaking và loại bỏ mã chết (dead code elimination).
 - Browserify (nodejsdp.link/browserify): Module bundler đầu tiên hỗ trợ CommonJS và vẫn còn được sử dụng rộng rãi.

Other trending module bundlers are FuseBox (nodejsdp.link/fusebox), Brunch (nodejsdp.link/brunch), and Microbundle (nodejsdp.link/microbundle).

Các module bundler đang thịnh hành khác là FuseBox (nodejsdp.link/fusebox), Brunch (nodejsdp.link/brunch), và Microbundle (nodejsdp.link/microbundle).

In the next section, we will discuss in greater detail how a module bundler works.

Trong phần tiếp theo, chúng ta sẽ bàn chi tiết hơn về cách một module bundler hoạt động.

How a module bundler works — Cách một module bundler hoạt động

We can define a module bundler as a tool that takes the source code of an application (in the form of an entry module and its dependencies) and produces one or more bundle files. The bundling process doesn't change the business logic of the app; it just creates files that are optimized to run on the browser. In a way, we can think of a bundler as a compiler for the browser.

Chúng ta có thể định nghĩa module bundler là một công cụ nhận mã nguồn của một ứng dụng (dưới dạng một module điểm vào (entry module) cùng các phụ thuộc của nó) và tạo ra một hoặc nhiều tập tin bundle. Quá trình đóng gói không làm thay đổi logic nghiệp vụ của ứng dụng; nó chỉ tạo ra các tập tin được tối ưu để chạy trên trình duyệt. Theo một cách nào đó, chúng ta có thể coi một bundler như một trình biên dịch (compiler) cho trình duyệt.

In the previous section, we saw how a bundler can help to reduce the total number of files that the browser will need to load, but in reality, a bundler can do so much more than that. For instance, it can use a transpiler like Babel ([nodejsdp.link/babel](https://nodejs.org/en/packages/babel/)). A transpiler is a tool that processes the source code and makes sure that modern JavaScript syntax is converted into equivalent ECMAScript 5 syntax so that a large variety of browsers (including older ones) can run the application correctly. Some module bundlers allow us to preprocess and optimize not just JavaScript code but also other assets such as images and stylesheets.

Ở phần trước, chúng ta đã thấy một bundler có thể giúp giảm tổng số tập tin mà trình duyệt cần tải như thế nào, nhưng thực tế một bundler có thể làm được nhiều hơn thế. Chẳng hạn, nó có thể dùng một transpiler như Babel ([nodejsdp.link/babel](https://nodejs.org/en/packages/babel/)). Một transpiler là công cụ xử lý mã nguồn và đảm bảo cú pháp JavaScript hiện đại được chuyển đổi thành cú pháp ECMAScript 5 tương đương sao cho nhiều loại trình duyệt (kể cả những trình duyệt cũ) có thể chạy ứng dụng đúng đắn. Một số module bundler cho phép chúng ta tiền xử lý và tối ưu không chỉ mã JavaScript mà còn cả những tài nguyên (asset) khác như hình ảnh và stylesheet.

In this section, we will provide a simplified view of how a module bundler works and how it navigates the code of a given application to produce an equivalent bundle optimized for the browser. The work of a module bundler can be divided into two steps that we will call dependency resolution and packing.

Trong phần này, chúng ta sẽ cung cấp một cái nhìn đơn giản hoá về cách một module bundler hoạt động và cách nó duyệt qua mã của một ứng dụng cho trước để tạo ra một bundle tương đương được tối ưu cho trình duyệt. Công việc của một module bundler có thể chia thành hai bước mà chúng ta sẽ gọi là phân giải phụ thuộc (dependency resolution) và đóng gói (packing).

Dependency resolution — Phân giải phụ thuộc (Dependency resolution) The dependency resolution step has the goal of traversing the codebase, starting from the main module (also called the entry point), and discovering all the dependencies. The way a bundler can do this is by representing dependencies as an acyclic direct graph, known as a dependency graph.

Bước phân giải phụ thuộc có mục tiêu duyệt khắp codebase, bắt đầu từ module chính (còn gọi là điểm vào, entry point), và khám phá tất cả các phụ thuộc. Cách một bundler có thể

làm điều này là biểu diễn các phụ thuộc dưới dạng một đồ thị có hướng phi chu trình (acyclic direct graph), gọi là đồ thị phụ thuộc (dependency graph).

Let's explore this concept with an example: a fictional calculator application. The implementation is intentionally incomplete as we only want to focus on the module structure, how the different modules depend on each other, and how the module bundler can build the dependency graph of this application:

Hãy khám phá khái niệm này qua một ví dụ: một ứng dụng máy tính (calculator) hư cấu. Phần triển khai cố ý không hoàn chỉnh vì chúng ta chỉ muốn tập trung vào cấu trúc module, cách các module khác nhau phụ thuộc lẫn nhau, và cách module bundler có thể xây dựng đồ thị phụ thuộc của ứng dụng này:

```
// app.js (1)
import { calculator } from './calculator.js'
import { display } from './display.js'
display(calculator('2 + 2 / 4'))
// display.js (5)
export function display () {
  // ...
}
// calculator.js (2)
import { parser } from './parser.js'
import { resolver } from './resolver.js'
export function calculator (expr) {
  return resolver(parser(expr))
}
// parser.js (3)
export function parser (expr) {
  // ...
}
// resolver.js (4)
export function resolver (tokens) {
  // ...
}
```

Let's see how the module bundler will walk through this code to figure out the dependency graph:

Hãy xem module bundler sẽ duyệt qua mã này như thế nào để tìm ra đồ thị phụ thuộc:

1. The module bundler starts its analysis from the entry point of the application, the module `app.js`. In this phase, the module bundler will discover dependencies by looking at `import` statements. The bundler starts to scan the code of the entry point and the first `import` it finds references the `calculator.js` module. Now, the bundler suspends the analysis of `app.js` and jumps immediately into `calculator.js`. The bundler will keep tabs on the open files: it will remember that the first line of `app.js` was already scanned so that when it eventually restarts processing this file, it will continue from the second line.
2. In `calculator.js`, the bundler immediately finds a new `import` for `parser.js` so that the processing of `calculator.js` is interrupted to move into `parser.js`.
3. In `parser.js`, there's no `import` statement, so after the file has been scanned entirely, the bundler goes back into `calculator.js`, where the next `import` statement refers to `resolver.js`. Again, the analysis of `calculator.js` is suspended and the bundler jumps immediately into `resolver.js`.

4. The `module resolver.js` does not contain any imports, so the control goes back to `calculator.js`. The `calculator.js` module does not contain other imports, so the control goes back to `app.js`. In `app.js`, the next import is `display.js` and the bundler jumps straight into it.
5. `display.js` does not contain any imports. So, again the control goes back to `app.js`. There are no more imports in `app.js`, so the code has been fully explored, and the dependency graph has been fully constructed.

1. Module bundler bắt đầu phân tích từ điểm vào của ứng dụng, module `app.js`. Trong pha này, module bundler sẽ khám phá các phụ thuộc bằng cách nhìn vào các câu lệnh `import`. Bundler bắt đầu quét mã của điểm vào và `import` đầu tiên nó tìm thấy tham chiếu tới module `calculator.js`. Lúc này, bundler tạm dừng phân tích `app.js` và lập tức nhảy sang `calculator.js`. Bundler sẽ ghi nhớ các tập tin đang mở: nó sẽ nhớ rằng dòng đầu tiên của `app.js` đã được quét xong, để khi cuối cùng quay lại xử lý tập tin này, nó sẽ tiếp tục từ dòng thứ hai.
2. Trong `calculator.js`, bundler lập tức tìm thấy một `import` mới cho `parser.js`, nên việc xử lý `calculator.js` bị gián đoạn để chuyển sang `parser.js`.
3. Trong `parser.js`, không có câu lệnh `import` nào, nên sau khi tập tin được quét xong hoàn toàn, bundler quay lại `calculator.js`, nơi câu lệnh `import` tiếp theo tham chiếu tới `resolver.js`. Một lần nữa, việc phân tích `calculator.js` bị tạm dừng và bundler lập tức nhảy sang `resolver.js`.
4. Module `resolver.js` không chứa `import` nào, nên quyền điều khiển quay lại `calculator.js`. Module `calculator.js` không chứa `import` nào khác, nên quyền điều khiển quay lại `app.js`. Trong `app.js`, `import` tiếp theo là `display.js` và bundler nhảy thẳng vào nó.
5. `display.js` không chứa `import` nào. Vậy nên, một lần nữa quyền điều khiển quay lại `app.js`. Không còn `import` nào trong `app.js`, nên mã đã được khám phá đầy đủ, và đồ thị phụ thuộc đã được xây dựng hoàn chỉnh.

Every time the module bundler jumps from one file to another, it means we are discovering a new dependency and adding a new node to the dependency graph. A visual representation of the steps described in the preceding list can be found in Figure 10.2:

Mỗi lần module bundler nhảy từ tập tin này sang tập tin khác, điều đó nghĩa là chúng ta đang khám phá một phụ thuộc mới và thêm một nút mới vào đồ thị phụ thuộc. Một biểu diễn trực quan của các bước được mô tả trong danh sách trên có thể thấy ở Hình 10.2:

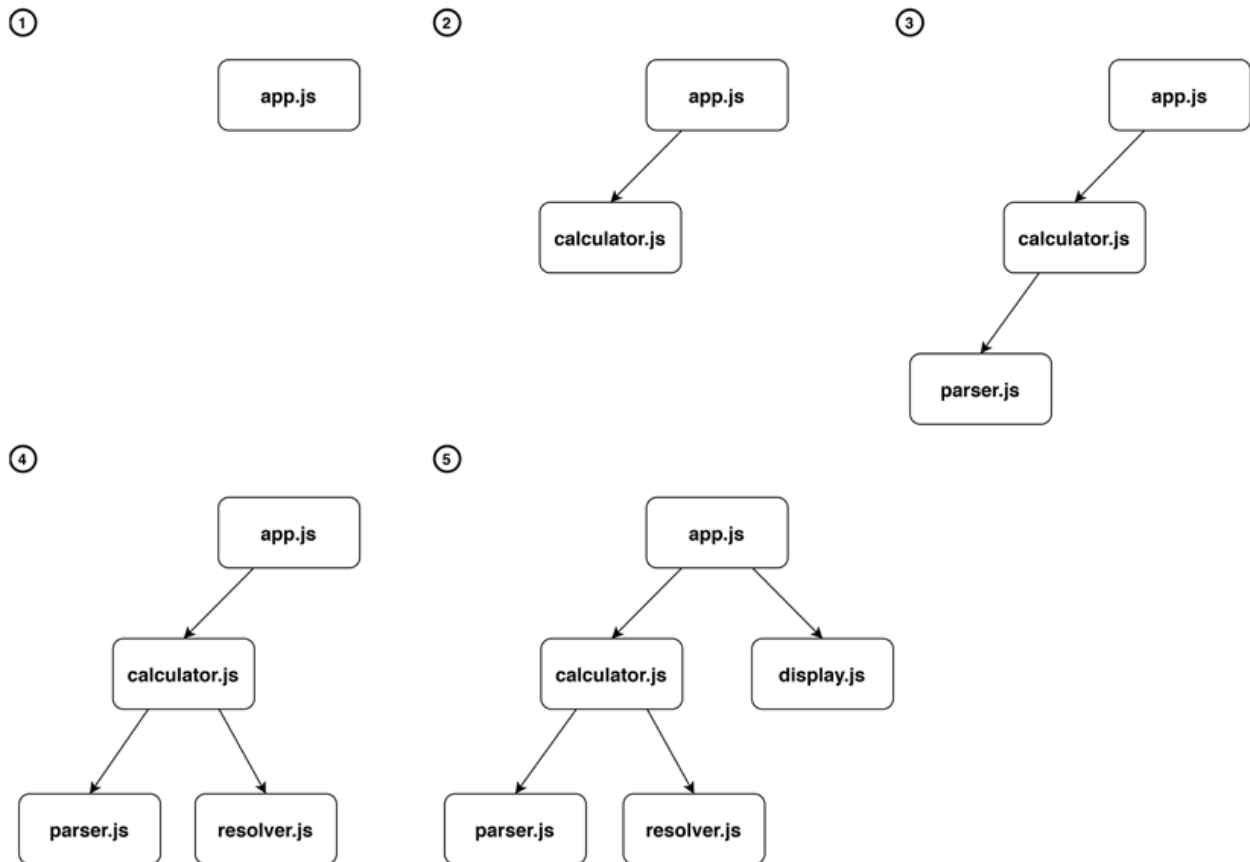


Figure 10.2: Dependency graph resolution

Hình 10.2: Phân giải đồ thị phụ thuộc

This way of resolving dependencies also works with cyclic dependencies. In fact, if the bundler encounters the same dependency for a second time, the dependency will be skipped because it's already present in the dependency graph. Tree shaking

Cách phân giải phụ thuộc này cũng hoạt động với các phụ thuộc vòng (cyclic dependency). Thực tế, nếu bundler gặp lại cùng một phụ thuộc lần thứ hai, phụ thuộc đó sẽ bị bỏ qua vì nó đã có mặt trong đồ thị phụ thuộc. Tree shaking

It's worth noting that if we have entities (for example, functions, classes, or variables) in our project modules that are never imported, then these won't appear in this dependency graph, so they won't be included in the final bundle.

Đáng lưu ý rằng nếu chúng ta có các thực thể (ví dụ, hàm, lớp, hoặc biến) trong các module của dự án mà không bao giờ được import, thì chúng sẽ không xuất hiện trong đồ thị phụ thuộc này, nên chúng sẽ không được đưa vào bundle cuối cùng.

A more advanced module bundler could also keep track of the entities imported from every module and the exported entities found in the dependency graph. This allows the bundle to figure out if there are exported functionalities that are never used in the application so that they can be pruned from the final bundle. This optimization technique is called tree shaking (nodejsdp.link/tree-shaking).

Một module bundler tân tiến hơn còn có thể theo dõi các thực thể được import từ mỗi module và các thực thể được export tìm thấy trong đồ thị phụ thuộc. Điều này cho phép

bundle nhận ra liệu có những chức năng được export mà không bao giờ được dùng đến trong ứng dụng hay không, để chúng có thể bị lược bỏ khỏi bundle cuối cùng. Kỹ thuật tối ưu này được gọi là tree shaking ([nodejsdp.link/ tree-shaking](https://nodejsdp.link/tree-shaking)).

During the dependency resolution phase, the module bundler builds a data structure called modules map. This data structure is a hash map that has unique module identifiers (for example, file paths) as keys and a representation of the module source code as values. In our example, a simplified representation of the modules map might look like this:

Trong pha phân giải phụ thuộc, module bundler xây dựng một cấu trúc dữ liệu gọi là bản đồ module (modules map). Cấu trúc dữ liệu này là một hash map có các định danh module duy nhất (ví dụ, đường dẫn tập tin) làm khoá (key) và một biểu diễn của mã nguồn module làm giá trị (value). Trong ví dụ của chúng ta, một biểu diễn đơn giản hoá của modules map có thể trông như sau:

```
{
  'app.js': (module, require) => { /* ... */ },
  'calculator.js': (module, require) => { /* ... */ },
  'display.js': (module, require) => { /* ... */ },
  'parser.js': (module, require) => { /* ... */ },
  'resolver.js': (module, require) => { /* ... */ }
}
```

Every module in the modules map is a **factory** function that accepts two arguments: module and require. We will see in more detail what those arguments are in the next section. What is important to understand now is that every module here is a complete representation of the code in the original source module. If we take, for example, the code for the calculator.js module, it might be represented as follows:

Mỗi module trong modules map là một hàm factory (factory function) nhận hai tham số: module và require. Chúng ta sẽ xem chi tiết hơn về các tham số này trong phần tiếp theo. Điều quan trọng cần hiểu lúc này là mỗi module ở đây là một biểu diễn hoàn chỉnh của mã trong module mã nguồn gốc. Lấy ví dụ mã cho module calculator.js, nó có thể được biểu diễn như sau:

```
(module, require) => {
  const { parser } = require('parser.js')
  const { resolver } = require('resolver.js')
  module.exports.calculator = function (expr) {
    return resolver(parser(expr))
  }
}
```

Note how the ESM syntax has been converted into something that resembles the syntax of the CommonJS module system. Remember that the browser does not support CommonJS and that these variables are not global, so there is no risk of a naming collision here. In this simplified implementation, we decided to use exactly the same identifiers as in CommonJS (module, require, and module.exports) to make the similarity with CommonJS look more apparent. In reality, every module bundler will use its own unique identifiers. For instance, webpack uses identifiers such as __webpack_require__ and __webpack_exports__.

Hãy lưu ý cách cú pháp ESM đã được chuyển đổi thành thứ gì đó giống cú pháp của hệ thống module CommonJS. Hãy nhớ rằng trình duyệt không hỗ trợ CommonJS và rằng những biến này không phải biến toàn cục (global), nên không có rủi ro va chạm

tên (naming collision) ở đây. Trong cách triển khai đơn giản hoá này, chúng ta quyết định dùng đúng những định danh giống như trong CommonJS (module, require, và module.exports) để làm cho sự tương đồng với CommonJS trông rõ ràng hơn. Trên thực tế, mỗi module bundler sẽ dùng các định danh duy nhất của riêng nó. Chẳng hạn, webpack dùng các định danh như `__webpack_require__` và `__webpack_exports__`.

Packing — Đóng gói (Packing) The modules map is the final output of the dependency resolution phase. In the packing phase, the module bundler takes the modules map and converts it into an executable bundle: a single JavaScript file that contains all the business logic of the original application.

Modules map là kết quả đầu ra cuối cùng của pha phân giải phụ thuộc. Trong pha đóng gói, module bundler nhận modules map và chuyển đổi nó thành một bundle có thể thực thi: một tập tin JavaScript duy nhất chứa toàn bộ logic nghiệp vụ của ứng dụng gốc.

The idea is simple: we already have a representation of the original codebase of our application inside the modules map; we have to find a way to convert it into something that the browser can execute correctly and save it into the resulting bundle file.

Ý tưởng rất đơn giản: chúng ta đã có một biểu diễn của codebase gốc của ứng dụng bên trong modules map; chúng ta phải tìm cách chuyển đổi nó thành thứ gì đó mà trình duyệt có thể thực thi đúng đắn và lưu nó vào tập tin bundle kết quả.

Given the structure of our modules map, this can actually be done with just a few lines of code wrapping the modules map:

Với cấu trúc của modules map của chúng ta, điều này thực ra có thể làm được chỉ với vài dòng mã bọc quanh modules map:

```
((modulesMap) => {                                     // (1)
  const require = (name) => {                             // (2)
    const module = { exports: {} }                       // (3)
    modulesMap[name](module, require)                     // (4)
    return module.exports                                 // (5)
  }
  require('app.js')                                       // (6)
})(
{
  'app.js': (module, require) => { /* ... */ },
  'calculator.js': (module, require) => { /* ... */ },
  'display.js': (module, require) => { /* ... */ },
  'parser.js': (module, require) => { /* ... */ },
  'resolver.js': (module, require) => { /* ... */ },
}
)
```

This is not a lot of code, but there's a lot happening here, so let's go through it together, step by step:

Đây không phải là nhiều mã, nhưng có rất nhiều thứ đang diễn ra ở đây, nên hãy cùng nhau đi qua từng bước:

1. In this code snippet, we have an Immediately Invoked Function Expression (IIFE) that receives the entire modules map as an argument.
2. When the function is executed, it defines a custom `require` function. This function receives a module name as input and it will load and execute the corresponding module from `modulesMap`.

3. In the `require` function, a module object is initialized. This object has only one property called `exports`, which is an object with no attributes.
4. At this point, the factory function of the given module is invoked and we pass to it the module object we just created and a reference to the `require` function itself. Note that this is essentially an implementation of the Service Locator pattern (nodejsdp.link/service-locator-pattern). Here, the factory function, once executed, modifies the module object by attaching to it the functionality that the module exports. The factory function can also recursively require other modules by using the `require` function passed as an argument.
5. Finally, the `require` function returns the `module.exports` object, which was populated by the factory function that was invoked in the previous step.
6. The last step is to require the entry point of our dependency graph, which in our case is the module `app.js`. This last step is what actually bootstraps the entire application. In fact, by loading the entry point, it will, in turn, load and execute all its dependencies in the right order and then execute its own business logic.

1. Trong đoạn mã này, chúng ta có một Immediately Invoked Function Expression (IIFE) nhận toàn bộ modules map làm tham số.
2. Khi hàm được thực thi, nó định nghĩa một hàm `require` tùy chỉnh. Hàm này nhận một name module làm đầu vào và sẽ tải và thực thi module tương ứng từ `modulesMap`.
3. Trong hàm `require`, một đối tượng module được khởi tạo. Đối tượng này chỉ có một thuộc tính tên là `exports`, là một đối tượng không có thuộc tính nào.
4. Tại điểm này, hàm `factory` của module cho trước được gọi và chúng ta truyền vào nó đối tượng module vừa tạo cùng một tham chiếu tới chính hàm `require`. Lưu ý rằng đây về cơ bản là một cách triển khai của mẫu Service Locator (nodejsdp.link/service-locator-pattern). Ở đây, hàm `factory`, một khi được thực thi, sẽ chỉnh sửa đối tượng module bằng cách gắn vào nó chức năng mà module export ra. Hàm `factory` cũng có thể đệ quy `require` các module khác bằng cách dùng hàm `require` được truyền vào làm tham số.
5. Cuối cùng, hàm `require` trả về đối tượng `module.exports`, vốn đã được điền dữ liệu bởi hàm `factory` được gọi ở bước trước.
6. Bước cuối cùng là `require` điểm vào của đồ thị phụ thuộc, trong trường hợp của chúng ta là module `app.js`. Bước cuối này chính là thứ thực sự khởi động (bootstrap) toàn bộ ứng dụng. Thực tế, bằng việc tải điểm vào, đến lượt nó sẽ tải và thực thi tất cả các phụ thuộc của nó theo đúng thứ tự rồi thực thi logic nghiệp vụ của chính nó.

With this process, we essentially created a self-sufficient module system that is capable of loading modules that have been properly organized within the same file. In other words, we managed to convert an app originally organized in multiple files into an equivalent app where all the code has been moved into a single file. This is the resulting bundle file.

Với quy trình này, về cơ bản chúng ta đã tạo ra một hệ thống module tự cấp (self-sufficient) có khả năng tải các module đã được tổ chức hợp lý trong cùng một tập tin. Nói cách khác, chúng ta đã chuyển đổi thành công một ứng dụng vốn được tổ chức trong nhiều tập tin thành một ứng dụng tương đương nơi tất cả mã đã được dồn vào một tập tin duy nhất. Đây chính là tập tin bundle kết quả.

Note that the preceding code has been intentionally simplified just to illustrate how module bundlers work. There are many edge cases that we did not take into account. For instance, what happens if we require a module that does not exist in the modules map?

Lưu ý rằng mã ở trên đã được cố ý đơn giản hoá chỉ để minh họa cách các module bundler hoạt động. Có nhiều trường hợp biên (edge case) mà chúng ta đã không tính đến. Chẳng hạn, điều gì xảy ra nếu chúng ta `require` một module không tồn tại trong `modules map`?

Using webpack — Sử dụng webpack

Now that we know how a module bundler works, let's build a simple application that can work both on Node.js and on the browser. Throughout this exercise, we will learn how to write a simple library that can be used without changes from the browser app and the server app. We will be using webpack to build the browser bundle.

Giờ khi đã biết một module bundler hoạt động ra sao, hãy xây dựng một ứng dụng đơn giản có thể chạy trên cả Node.js và trình duyệt. Qua bài tập này, chúng ta sẽ học cách viết một thư viện đơn giản có thể dùng mà không cần thay đổi gì từ cả ứng dụng trình duyệt lẫn ứng dụng máy chủ. Chúng ta sẽ dùng webpack để build bundle cho trình duyệt.

To keep things simple, our application will be nothing more than a simple “hello world” for now, but don't worry, we will be building a more realistic application in the Creating a Universal JavaScript app section, later in this chapter.

Để giữ mọi thứ đơn giản, ứng dụng của chúng ta tạm thời sẽ không là gì khác ngoài một “hello world” đơn giản, nhưng đừng lo, chúng ta sẽ xây dựng một ứng dụng thực tế hơn ở phần Tạo một ứng dụng Universal JavaScript, ở phần sau của chương này.

Let's start by installing the webpack CLI in our system with:

Hãy bắt đầu bằng việc cài đặt webpack CLI vào hệ thống với:

```
npm install --global webpack-cli
```

Let's now initialize a new project in a new folder with:

Bây giờ hãy khởi tạo một dự án mới trong một thư mục mới với:

```
npm init
```

Once the guided project initialization is complete, since we want to use ESM in Node.js, we need to add the property "type": "module" to our package.json.

Một khi quá trình khởi tạo dự án có hướng dẫn hoàn tất, vì chúng ta muốn dùng ESM trong Node.js, chúng ta cần thêm thuộc tính "type": "module" vào package.json.

Now, we can run:

Giờ chúng ta có thể chạy:

```
webpack-cli init
```

This guided procedure will install webpack in your project and it will help you to automatically generate a webpack configuration file. At the time of writing, using webpack 4, the guided procedure does not realize that we want to use ESM in Node.js, so we have to apply two small changes to the generated files:

Quy trình có hướng dẫn này sẽ cài đặt webpack vào dự án của bạn và giúp bạn tự động tạo ra một tập tin cấu hình webpack. Tại thời điểm viết sách, dùng webpack 4, quy trình có hướng dẫn không nhận ra rằng chúng ta muốn dùng ESM trong Node.js, nên chúng ta phải áp dụng hai thay đổi nhỏ cho các tập tin được tạo ra:

- Rename webpack.config.js to webpack.config.cjs
- Change the following npm scripts in package.json:
 - Đổi tên webpack.config.js thành webpack.config.cjs
 - Thay đổi các npm script sau trong package.json:

```
"build": "webpack --config webpack.config.cjs"
"start": "webpack-dev-server --config webpack.config.cjs"
```

Now, we are ready to start writing our application. Let's first write the module we want to share in `src/say-hello.js`:

Giờ chúng ta đã sẵn sàng bắt đầu viết ứng dụng. Trước hết hãy viết module mà chúng ta muốn chia sẻ trong `src/say-hello.js`:

```
import nunjucks from 'nunjucks'
const template = '<h1>Hello <i>{{ name }}</i></h1>'
export function sayHello (name) {
  return nunjucks.renderString(template, { name })
}
```

In this code, we are using the nunjucks template library ([nodejsdp.link/nunjucks](https://www.npmjs.com/package/nunjucks)), which must be installed with npm. This module is exporting a simple `sayHello` function that accepts a `name` as the only argument and uses it to construct an HTML string.

Trong mã này, chúng ta đang dùng thư viện template `nunjucks` ([nodejsdp.link/nunjucks](https://www.npmjs.com/package/nunjucks)), phải được cài bằng npm. Module này export một hàm `sayHello` đơn giản nhận một `name` làm tham số duy nhất và dùng nó để dựng một chuỗi HTML.

Let's now write the browser application that will use this module (`src/index.js`):

Giờ hãy viết ứng dụng trình duyệt sẽ dùng module này (`src/index.js`):

```
import { sayHello } from './say-hello.js'
const body = document.getElementsByTagName('body')[0]
body.innerHTML = sayHello('Browser')
```

This code uses the `sayHello` function to build an HTML fragment saying Hello Browser and then inserts it into the `body` section of the current HTML page.

Mã này dùng hàm `sayHello` để dựng một đoạn HTML nói Hello Browser rồi chèn nó vào phần `body` của trang HTML hiện tại.

If you want to preview this application, you can run `npm start` in your terminal. This should open your default browser and you should see the application running.

Nếu bạn muốn xem trước ứng dụng này, bạn có thể chạy `npm start` trong terminal. Lệnh này sẽ mở trình duyệt mặc định của bạn và bạn sẽ thấy ứng dụng đang chạy.

If you want to generate a static version of the application, you can run:

Nếu bạn muốn tạo một phiên bản tĩnh của ứng dụng, bạn có thể chạy:

```
npm run build
```

This will generate a folder called `dist` containing two files: an `index.html` and our bundle file (whose name will look like `main.12345678901234567890.js`).

Lệnh này sẽ tạo một thư mục tên `dist` chứa hai tập tin: một `index.html` và tập tin `bundle` của chúng ta (có tên trông giống `main.12345678901234567890.js`).

The file name of the bundle is generated by using a hash of the file content. This way, every time our source code changes, we will obtain a new bundle with a different name. This is a useful optimization technique, called *cache busting*, that webpack adopts by default and it is particularly convenient when deploying our assets to a content delivery network (CDN). With CDNs, it is generally quite expensive to override files that are geographically distributed across multiple servers and already

cached in multiple layers, possibly including our users' browsers. By generating new files with every change, we avoid cache invalidation entirely. You can open the `index.html` file with your browser to see a preview of your application.

Tên tập tin của bundle được tạo bằng cách dùng một hash của nội dung tập tin. Theo cách này, mỗi lần mã nguồn của chúng ta thay đổi, chúng ta sẽ nhận được một bundle mới với tên khác. Đây là một kỹ thuật tối ưu hữu ích, gọi là cache busting, mà webpack áp dụng theo mặc định và nó đặc biệt tiện lợi khi triển khai (deploy) các asset của chúng ta tới một content delivery network (CDN). Với các CDN, việc ghi đè các tập tin được phân tán địa lý trên nhiều máy chủ và đã được cache ở nhiều tầng, có thể bao gồm cả trình duyệt của người dùng, nói chung khá tốn kém. Bằng việc tạo các tập tin mới với mỗi thay đổi, chúng ta tránh được hoàn toàn việc vô hiệu hoá cache (cache invalidation). Bạn có thể mở tập tin `index.html` bằng trình duyệt để xem trước ứng dụng của mình.

If you are curious, you can have a look at the generated bundle file. You will notice that it is a bit more convoluted and verbose than the sample bundle we illustrated in the previous section. However, you should be able to recognize the structure and notice that the entire `nunjucks` library, as well as our `sayHello` module, have been embedded in the bundle code.

Nếu tò mò, bạn có thể xem qua tập tin bundle được tạo ra. Bạn sẽ nhận thấy nó có phần rắc rối và dài dòng hơn bundle mẫu mà chúng ta minh hoạ ở phần trước. Tuy nhiên, bạn vẫn có thể nhận ra cấu trúc và để ý rằng toàn bộ thư viện `nunjucks`, cũng như module `sayHello` của chúng ta, đã được nhúng vào mã bundle.

Now, what if we want to build an equivalent application that runs in Node.js? For instance, we could use the `sayHello` function and display the resulting code in the terminal:

Bây giờ, nếu chúng ta muốn xây dựng một ứng dụng tương đương chạy trong Node.js thì sao? Chẳng hạn, chúng ta có thể dùng hàm `sayHello` và hiển thị mã kết quả trong terminal:

```
// src/server.js
import { sayHello } from './say-hello.js'
console.log(sayHello('Node.js'))
```

That's it!

Vậy đó!

If we run this code with:

Nếu chúng ta chạy mã này với:

```
node src/server.js
```

We will see the following output:

Chúng ta sẽ thấy đầu ra sau:

```
<h1>Hello <i>Node.js</i></h1>
```

Yes, displaying HTML in the terminal is not particularly useful, but right now we achieved our goal of being able to use a library from both the browser and the server without any changes in the library codebase.

Đúng vậy, hiển thị HTML trong terminal không đặc biệt hữu ích, nhưng ngay lúc này chúng ta đã đạt mục tiêu là có thể dùng một thư viện từ cả trình duyệt và máy chủ mà không cần bất kỳ thay đổi nào trong codebase của thư viện.

In the next sections, we will discuss some patterns that allow us to actually change the code where necessary if we want to provide more specialized behaviors on the browser or Node.js.

Trong các phần tiếp theo, chúng ta sẽ bàn về một số mẫu cho phép chúng ta thực sự thay đổi mã ở nơi cần thiết nếu muốn cung cấp các hành vi chuyên biệt hơn trên trình duyệt hoặc Node.js.

Fundamentals of cross-platform development — Những nền tảng cốt lõi của phát triển đa nền tảng

When developing for different platforms, the most common problem we face is how can we reuse as much code as possible and, at the same time, provide specialized implementations for details that are platform-specific. We will now explore some of the principles and the patterns to use when facing this challenge, such as code branching and module swapping.

Khi phát triển cho các nền tảng khác nhau, vấn đề phổ biến nhất mà chúng ta đối mặt là làm sao tái sử dụng càng nhiều mã càng tốt và đồng thời cung cấp các cách triển khai chuyên biệt cho những chi tiết đặc thù theo nền tảng. Giờ chúng ta sẽ khám phá một số nguyên tắc và mẫu để dùng khi đối mặt với thử thách này, chẳng hạn rẽ nhánh mã (code branching) và hoán đổi module (module swapping).

Runtime code branching — Rẽ nhánh mã tại runtime (Runtime code branching)

The most simple and intuitive technique for providing different implementations based on the host platform is to dynamically branch our code. This requires that we have a mechanism to recognize the host platform at runtime and then dynamically switch the implementation with an `if...else` statement. Some generic approaches involve checking global variables that are available only on Node.js or only on the browser.

Kỹ thuật đơn giản và trực quan nhất để cung cấp các cách triển khai khác nhau dựa trên nền tảng máy chủ là rẽ nhánh mã một cách động. Điều này đòi hỏi chúng ta có một cơ chế để nhận biết nền tảng máy chủ tại runtime rồi chuyển đổi cách triển khai một cách động bằng câu lệnh `if...else`. Một số cách tiếp cận tổng quát liên quan đến việc kiểm tra các biến toàn cục chỉ có sẵn trên Node.js hoặc chỉ có trên trình duyệt.

For example, we can check the existence of the `window` global variable. Let's modify our `say-hello.js` module to use this technique to provide a slightly different functionality depending on whether the module is running on the browser or on the server:

Ví dụ, chúng ta có thể kiểm tra sự tồn tại của biến toàn cục `window`. Hãy chỉnh sửa module `say-hello.js` để dùng kỹ thuật này nhằm cung cấp chức năng hơi khác nhau tùy vào việc module đang chạy trên trình duyệt hay máy chủ:

```
import nunjucks from 'nunjucks'
const template = '<h1>Hello <i>{{ name }}</i></h1>'
export function sayHello (name) {
  if (typeof window !== 'undefined' && window.document) {
    // client-side code
    return nunjucks.renderString(template, { name })
  }
  // Node.js code
```



```

return `Hello \u001b[1m${name}\u001b[0m`
}

```

The escape sequence `\u001b[1m` is a special terminal formatting indicator that sets the text to bold. The sequence `\u001b[0m` instead resets the formatting to normal. If you are curious to find out more about escape sequences and their history, check out ANSI escape sequences: [nodejsdp.link/ansi-escape-sequences](https://nodejs.org/en/docs/faq/ansi-escape-sequences).

Chuỗi escape `\u001b[1m` là một chỉ thị định dạng đặc biệt của terminal đặt văn bản thành đậm (bold). Chuỗi `\u001b[0m` thì đặt lại định dạng về bình thường. Nếu bạn tò mò tìm hiểu thêm về các chuỗi escape và lịch sử của chúng, hãy xem ANSI escape sequences: [nodejsdp.link/ansi-escape-sequences](https://nodejs.org/en/docs/faq/ansi-escape-sequences).

Try again to run our application on Node.js and on the browser and see the differences! If you do that, you will not see HTML code on the terminal when running the Node.js application. Instead, you will see a string with proper terminal formatting. The frontend application on the browser remains unchanged.

Hãy thử chạy lại ứng dụng của chúng ta trên Node.js và trên trình duyệt rồi xem sự khác biệt! Nếu làm vậy, bạn sẽ không thấy mã HTML trên terminal khi chạy ứng dụng Node.js. Thay vào đó, bạn sẽ thấy một chuỗi có định dạng terminal phù hợp. Ứng dụng frontend trên trình duyệt vẫn không thay đổi.

Challenges of runtime code branching — Những thách thức của rẽ nhánh mã tại runtime

Using a runtime branching approach for switching between Node.js and the browser is definitely the most intuitive and simple pattern we can use for this purpose; however, there are some inconveniences:

Dùng cách rẽ nhánh tại runtime để chuyển đổi giữa Node.js và trình duyệt chắc chắn là mẫu trực quan và đơn giản nhất chúng ta có thể dùng cho mục đích này; tuy nhiên, có một số bất tiện:

- The code for both platforms is included in the same module and therefore in the final bundle. This increases the bundle size, adding unreachable and unnecessary code. It is also possible that the unreachable code contains sensitive information like encryption keys or **API** keys that are not meant to be sent to a user's browser. In this case, this approach might also raise significant security concerns.
- If used too extensively, it can considerably reduce the readability of the code, as the business logic would be mixed with logic meant only to add crossplatform compatibility.
- Using dynamic branching to load a different module, depending on the platform, will result in all the modules being added to the final bundle, regardless of their target platform. For example, if we consider the following code fragment, both `clientModule` and `serverModule` will be included in a bundle generated with webpack, unless we explicitly exclude one of them from the build:
 - Mã cho cả hai nền tảng đều nằm trong cùng một module và do đó trong bundle cuối cùng. Điều này làm tăng kích thước bundle, thêm vào mã không thể chạm tới (unreachable) và không cần thiết. Cũng có khả năng mã không thể chạm tới chứa thông tin nhạy cảm như khoá mã hoá (encryption key) hoặc API key vốn không nhằm gửi tới trình duyệt của người dùng. Trong trường hợp này, cách tiếp cận này còn có thể làm dấy lên những lo ngại nghiêm trọng về bảo mật.
 - Nếu dùng quá rộng rãi, nó có thể làm giảm đáng kể khả năng đọc của mã, vì logic nghiệp vụ sẽ bị trộn lẫn với logic chỉ nhằm thêm khả năng tương thích đa nền tảng.

- Dùng rẽ nhánh động để tải một module khác nhau, tùy vào nền tảng, sẽ khiến tất cả các module bị thêm vào bundle cuối cùng, bất kể nền tảng đích của chúng. Ví dụ, nếu xét đoạn mã sau, cả `clientModule` lẫn `serverModule` sẽ được đưa vào một bundle được tạo bằng webpack, trừ khi chúng ta loại trừ rõ ràng một trong số chúng khỏi quá trình build:

```
import { clientFunctionality } from 'clientModule'
import { serverFunctionality } from 'serverModule'
if (typeof window !== 'undefined' && window.document) {
  clientFunctionality()
} else {
  serverFunctionality()
}
```

This last inconvenience happens because of the following reasons:

Bất tiện cuối cùng này xảy ra do những lý do sau:

- Bundlers have no sure way of knowing the value of a runtime variable at build time (unless the variable is a constant), so, in the preceding example, both branches of the `if...else` statement are always included in the final bundle, even though it is obvious that the browser will always execute only one of them.
- ES module imports are always defined declaratively at the top of the file and we don't have a way to filter the imports based on the current environment. The bundler will not try to understand whether you are conditionally using only a subset of the imported feature and it will include all the imported code anyway.
 - Các bundler không có cách nào chắc chắn để biết giá trị của một biến runtime tại thời điểm build (trừ khi biến đó là hằng số), nên trong ví dụ trên, cả hai nhánh của câu lệnh `if...else` luôn được đưa vào bundle cuối cùng, dù rõ ràng trình duyệt sẽ luôn chỉ thực thi một trong hai nhánh.
 - Các import ES module luôn được định nghĩa một cách khai báo (declaratively) ở đầu tập tin và chúng ta không có cách nào lọc các import dựa trên môi trường hiện tại. Bundler sẽ không cố hiểu liệu bạn có đang dùng một cách có điều kiện chỉ một tập con của tính năng được import hay không, và nó sẽ đưa vào toàn bộ mã được import bất kể thế nào.

A consequence of this last property is that modules imported dynamically using variables are not included in the bundle. For example, from the following code, no module will be bundled:

Một hệ quả của tính chất cuối cùng này là các module được import động bằng biến sẽ không được đưa vào bundle. Ví dụ, từ mã sau, không module nào được đóng gói:

```
moduleList.forEach(function(module) {
  import(module)
})
```

It's worth underlining that webpack overcomes some of these limitations and, under certain specific circumstances, it is able to guess all the possible values for a dynamic requirement. For instance, if you have a snippet of code like the following:

Đáng nhấn mạnh rằng webpack vượt qua được một số hạn chế này, và trong một số trường hợp cụ thể nhất định, nó có thể đoán tất cả các giá trị khả dĩ cho một yêu cầu động. Chẳng hạn, nếu bạn có một đoạn mã như sau:

```
function getControllerModule (controllerName) {
  return import(`./controller/${controllerName}`)
}
```

Webpack will include all the modules available in the `controller` folder in the final bundle.

Webpack sẽ đưa tất cả các module có sẵn trong thư mục `controller` vào bundle cuối cùng.

It's highly recommended to have a look at the official documentation to understand all the supported cases (nodejsdp.link/webpack-dynamic-imports).

Rất khuyến khích xem qua tài liệu chính thức để hiểu tất cả các trường hợp được hỗ trợ (nodejsdp.link/webpack-dynamic-imports).

Build-time code branching — Rẽ nhánh mã tại thời điểm build (Build-time code branching)

In this section, we are going to see how to use webpack plugins to remove, at build time, all parts of the code that we want to run only on the server. This allows us to obtain lighter bundle files and to avoid accidentally exposing code containing sensible information (for instance, secrets, passwords, or API keys) that should only live on the server.

Trong phần này, chúng ta sẽ thấy cách dùng các plugin của webpack để loại bỏ, tại thời điểm build, tất cả các phần mã mà chúng ta chỉ muốn chạy trên máy chủ. Điều này cho phép chúng ta thu được các tập tin bundle nhẹ hơn và tránh vô tình phơi bày mã chứa thông tin nhạy cảm (chẳng hạn, secret, mật khẩu, hoặc API key) vốn chỉ nên tồn tại trên máy chủ.

Webpack offers support for plugins, which allows us to extend webpack's capabilities and add new processing steps that can be used to produce the bundle file. To perform build-time code branching, we can leverage a built-in plugin called `DefinePlugin` and a third-party plugin called `terser-webpack-plugin` (nodejsdp.link/terser-webpack).

Webpack cung cấp hỗ trợ cho các plugin, cho phép chúng ta mở rộng khả năng của webpack và thêm các bước xử lý mới có thể dùng để tạo ra tập tin bundle. Để thực hiện rẽ nhánh mã tại thời điểm build, chúng ta có thể tận dụng một plugin tích hợp sẵn tên là `DefinePlugin` và một plugin bên thứ ba tên là `terser-webpack-plugin` (nodejsdp.link/terser-webpack).

`DefinePlugin` can be used to replace specific code occurrences in our source files with custom code or variables. `terser-webpack-plugin` allows us to compress the resulting code and remove unreachable statements (dead code elimination).

`DefinePlugin` có thể dùng để thay thế những lần xuất hiện mã cụ thể trong các tập tin nguồn của chúng ta bằng mã hoặc biến tùy chỉnh. `terser-webpack-plugin` cho phép chúng ta nén mã kết quả và loại bỏ các câu lệnh không thể chạm tới (loại bỏ mã chết, dead code elimination).

Let's start by rewriting our `say-hello.js` module to explore these concepts:

Hãy bắt đầu bằng việc viết lại module `say-hello.js` để khám phá các khái niệm này:

```
import nunjucks from 'nunjucks'
export function sayHello (name) {
  if (typeof __BROWSER__ !== 'undefined') {
```

```

    // client-side code
    const template = '<h1>Hello <i>{{ name }}</i></h1>'
    return nunjucks.renderString(template, { name })
  }
  // Node.js code
  return `Hello \u001b[1m${name}\u001b[0m`
}

```

Note that we are checking for the existence of a generic variable called `__BROWSER__` to enable the browser code. This is the variable that we will replace at build time using `DefinePlugin`.

Lưu ý rằng chúng ta đang kiểm tra sự tồn tại của một biến tổng quát tên là `__BROWSER__` để kích hoạt mã trình duyệt. Đây là biến mà chúng ta sẽ thay thế tại thời điểm build bằng `DefinePlugin`.

Now, let's install `terser-webpack-plugin` with:

Bây giờ, hãy cài `terser-webpack-plugin` với:

```
npm install --save-dev terser-webpack-plugin
```

Finally, let's update our `webpack.config.cjs` file:

Cuối cùng, hãy cập nhật tập tin `webpack.config.cjs`:

```

// ...
const TerserPlugin = require('terser-webpack-plugin')
module.exports = {
  mode: 'production',
  // ...
  plugins: [
    // ...
    new webpack.DefinePlugin({
      __BROWSER__: true
    })
  ],
  // ...
  optimization: {
    // ...
    minimize: true,
    minimizer: [new TerserPlugin()]
  }
}

```

The first change here is to set the option `mode` to `production`. This option will enable optimizations such as code minification (or minimization). Optimization options are defined in the dedicated optimization object. Here, we are enabling minification by setting `minimize` to `true` and we are providing a new instance of `terser-webpack-plugin` as the minimizer. Finally, we are also adding `webpack.DefinePlugin` and configuring it to replace the string `__BROWSER__` with the value `true`. Every value in the configuration object of `DefinePlugin` represents a piece of code that will be evaluated by webpack at build time and then used to replace the currently matched snippet of code. This allows us to add external dynamic values containing, for instance, the content of an environment variable, the current timestamp, or the hash of the last git commit to the bundle.

Thay đổi đầu tiên ở đây là đặt tùy chọn `mode` thành `production`. Tùy chọn này sẽ kích hoạt

các tối ưu như rút gọn mã (minification hay minimization). Các tùy chọn tối ưu được định nghĩa trong đối tượng `optimization` chuyên dụng. Ở đây, chúng ta kích hoạt minification bằng cách đặt `minimize` thành `true` và cung cấp một instance mới của `terser-webpack-plugin` làm minimizer. Cuối cùng, chúng ta cũng thêm `webpack.DefinePlugin` và cấu hình nó để thay thế chuỗi `__BROWSER__` bằng giá trị `true`. Mỗi giá trị trong đối tượng cấu hình của `DefinePlugin` đại diện cho một đoạn mã sẽ được webpack đánh giá tại thời điểm build rồi dùng để thay thế đoạn mã đang khớp hiện tại. Điều này cho phép chúng ta thêm các giá trị động bên ngoài chứa, chẳng hạn, nội dung của một biến môi trường, dấu thời gian hiện tại, hoặc hash của commit git cuối cùng vào bundle.

With this configuration, when we build a new bundle, every occurrence of `__BROWSER__` is replaced with `true`. The first `if` statement will internally look like `if (true !== 'undefined')`, but webpack is smart enough to understand that this expression will always be evaluated as `true`, so it transforms the resulting code again into `if (true)`.

Với cấu hình này, khi chúng ta build một bundle mới, mọi lần xuất hiện của `__BROWSER__` được thay thế bằng `true`. Câu lệnh `if` đầu tiên về mặt nội bộ sẽ trông như `if (true !== 'undefined')`, nhưng webpack đủ thông minh để hiểu rằng biểu thức này sẽ luôn được đánh giá là `true`, nên nó lại biến đổi mã kết quả thành `if (true)`.

Once webpack has finished processing all the code, it will invoke `terser-webpack-plugin` to minimize the resulting code. `terser-webpack-plugin` is a wrapper around `Terser` (`nodejsdp.link/terser`), a modern JavaScript minifier. Terser is capable of removing dead code as part of its minimization algorithm, so given that, at this stage, our code will look like this:

Một khi webpack đã xử lý xong toàn bộ mã, nó sẽ gọi `terser-webpack-plugin` để rút gọn mã kết quả. `terser-webpack-plugin` là một wrapper bao quanh `Terser` (`nodejsdp.link/terser`), một trình rút gọn JavaScript hiện đại. Terser có khả năng loại bỏ mã chết như một phần của thuật toán rút gọn của nó, nên với điều đó, ở giai đoạn này, mã của chúng ta sẽ trông như sau:

```
if (true) {
  const template = '<h1>Hello <i>{{ name }}</i></h1>'
  return nunjucks.renderString(template, { name })
}
return `Hello \u001b[1m${name}\u001b[0m`
```

Terser will reduce it to:

Terser sẽ rút gọn nó thành:

```
const template = '<h1>Hello <i>{{ name }}</i></h1>'
return nunjucks.renderString(template, { name })
```

This way, we got rid of all the server-side code in our browser bundle.

Theo cách này, chúng ta đã loại bỏ tất cả mã phía máy chủ trong bundle trình duyệt của mình.

Even if build-time code branching is way better than runtime code branching because it produces much leaner bundle files, it can still make our source code cumbersome when abused. In fact, if you overuse this technique, you will end up with code that contains too many `if` statements, which will be hard to understand and debug.

Dù rẽ nhánh mã tại thời điểm build tốt hơn nhiều so với rẽ nhánh mã tại runtime vì nó tạo ra các tập tin bundle gọn gàng hơn nhiều, nó vẫn có thể khiến mã nguồn của chúng ta

công kênh khi bị lạm dụng. Thực tế, nếu bạn lạm dụng kỹ thuật này, bạn sẽ kết thúc với mã chứa quá nhiều câu lệnh `if`, vốn sẽ khó hiểu và khó debug.

When this happens, it is generally better to move all the platform-specific code into dedicated modules. We will discuss this alternative approach in the next section.

Khi điều này xảy ra, nói chung tốt hơn là chuyển tất cả mã đặc thù theo nền tảng vào các module chuyên dụng. Chúng ta sẽ bàn về cách tiếp cận thay thế này trong phần tiếp theo.

Module swapping — Hoán đổi module (Module swapping)

Most of the time, we already know at build time what code has to be included in the client bundle and what shouldn't. This means that we can take this decision upfront and instruct the bundler to replace the implementation of an entire module at build time. This often results in a leaner bundle, as we are excluding unnecessary modules, and more **readable** code because we don't have all the `if...else` statements required by runtime and build-time branching.

Hầu hết thời gian, chúng ta đã biết ngay tại thời điểm build mã nào phải được đưa vào bundle client và mã nào không nên. Điều này nghĩa là chúng ta có thể đưa ra quyết định này ngay từ đầu và chỉ thị cho bundler thay thế cách triển khai của cả một module tại thời điểm build. Điều này thường dẫn đến một bundle gọn gàng hơn, vì chúng ta loại trừ các module không cần thiết, và mã dễ đọc hơn vì chúng ta không có tất cả các câu lệnh `if...else` mà rẽ nhánh runtime và build-time đòi hỏi.

Let's find out how to adopt module swapping with webpack by updating our example.

Hãy tìm hiểu cách áp dụng hoán đổi module với webpack bằng cách cập nhật ví dụ của chúng ta.

The main idea is that we want to have two separate implementations of our `sayHello` functionality: one optimized for the server (`say-hello.js`) and one optimized for the browser (`say-hello-browser.js`). We will then tell webpack to replace any import of `say-hello.js` with `say-hello-browser.js`. Let's see what our new implementation looks like now:

Ý tưởng chính là chúng ta muốn có hai cách triển khai riêng biệt của chức năng `sayHello`: một được tối ưu cho máy chủ (`say-hello.js`) và một được tối ưu cho trình duyệt (`say-hello-browser.js`). Sau đó chúng ta sẽ bảo webpack thay thế bất kỳ import nào của `say-hello.js` bằng `say-hello-browser.js`. Hãy xem cách triển khai mới của chúng ta trông như thế nào bây giờ:

```
// src/say-hello.js
import chalk from 'chalk'
export function sayHello (name) {
  return `Hello ${chalk.green(name)}`
}

// src/say-hello-browser.js
import nunjucks from 'nunjucks'
const template = '<h1>Hello <i>{{ name }}</i></h1>'
export function sayHello (name) {
  return nunjucks.renderString(template, { name })
}
```

Note that, on the server-side version, we introduced a new dependency, `chalk` (`nodejsdp.link/chalk`), a utility library that allows us to format text for the terminal. This is to demonstrate one of the main advantages of this approach. Now that we've separated our server-side code from the client-side

code, we can introduce new functionalities and libraries without worrying about the impact that those might have on the frontend-only bundle. At this point, in order to tell webpack to swap the modules at build time, we have to replace `webpack.DefinePlugin` with a new plugin in our `webpack.config.cjs`, as follows:

Lưu ý rằng, ở phiên bản phía máy chủ, chúng ta đã giới thiệu một phụ thuộc mới, `chalk` (nodejsdp.link/chalk), một thư viện tiện ích cho phép chúng ta định dạng văn bản cho terminal. Điều này nhằm minh họa một trong những ưu điểm chính của cách tiếp cận này. Giờ khi đã tách mã phía máy chủ khỏi mã phía client, chúng ta có thể giới thiệu các chức năng và thư viện mới mà không phải lo lắng về ảnh hưởng mà chúng có thể gây ra cho bundle chỉ dành cho frontend. Tại điểm này, để bảo webpack hoán đổi các module tại thời điểm build, chúng ta phải thay thế `webpack.DefinePlugin` bằng một plugin mới trong `webpack.config.cjs`, như sau:

```
plugins: [  
  // ...  
  new webpack.NormalModuleReplacementPlugin(  
    /src\/say-hello\.js$/,  
    path.resolve(__dirname, 'src', 'say-hello-browser.js')  
  )  
]
```

We are using `webpack.NormalModuleReplacementPlugin`, which accepts two arguments. The first argument is a regular expression and the second one is a string representing a path to a resource. At build time, if a module path matches the given regular expression, it is replaced with the one provided in the second argument.

Chúng ta đang dùng `webpack.NormalModuleReplacementPlugin`, nhận hai tham số. Tham số thứ nhất là một biểu thức chính quy (regular expression) và tham số thứ hai là một chuỗi đại diện cho đường dẫn tới một tài nguyên. Tại thời điểm build, nếu đường dẫn của một module khớp biểu thức chính quy cho trước, nó được thay thế bằng đường dẫn được cung cấp trong tham số thứ hai.

Note that this technique is not limited to our internal modules, but it can also be used with external libraries in our `node_modules` folder.

Lưu ý rằng kỹ thuật này không giới hạn ở các module nội bộ của chúng ta, mà còn có thể dùng với các thư viện bên ngoài trong thư mục `node_modules` của chúng ta.

Thanks to webpack and the module replacement plugin, we can easily deal with structural differences between platforms. We can focus on writing separate modules that are meant to provide platform-specific code and we can then swap Node.js-only modules with browser-specific ones in the final bundle.

Nhờ webpack và plugin thay thế module, chúng ta có thể dễ dàng xử lý những khác biệt mang tính cấu trúc giữa các nền tảng. Chúng ta có thể tập trung vào việc viết các module riêng biệt nhằm cung cấp mã đặc thù theo nền tảng và sau đó có thể hoán đổi các module chỉ dành cho Node.js bằng các module dành riêng cho trình duyệt trong bundle cuối cùng.

Design patterns for cross-platform development — Các mẫu thiết kế cho phát triển đa nền tảng

Let's now revise some of the design patterns we discussed in the previous chapters to see how we can leverage those for cross-platform development:

Giờ hãy ôn lại một số mẫu thiết kế mà chúng ta đã bàn ở các chương trước để xem chúng ta có thể tận dụng chúng cho phát triển đa nền tảng như thế nào:

- **Strategy** and **template**: These two are probably the most useful patterns when sharing code with the browser. Their intent is, in fact, to define the common steps of an algorithm, allowing some of its parts to be replaced, which is exactly what we need! In cross-platform development, these patterns allow us to share the platform-agnostic part of our components, while allowing their platform-specific parts to be changed using a different strategy or template method (which can be changed using code branching (runtime or build-time) or module swapping).
- **Adapter**: This pattern is probably the most useful when we need to swap an entire component. We have already seen several examples in Chapter 8, Structural Design Patterns. If your server application is using a database like SQLite, you could use the Adapter pattern to provide an alternative data storage implementation that works in the browser. For instance you could use the `localStorage` API (nodejsdp.link/localstorage) or the `IndexedDB` API (nodejsdp.link/indexdb).
- **Proxy**: When code meant to run on the server runs on the browser, we often need functionality that is used on the server to be available on the browser as well. This is where the remote Proxy pattern is useful. Imagine if we wanted to access the filesystem of the server from the browser: we could think of creating an `fs` object on the client that proxies every call to the `fs` module living on the server, using Ajax or WebSockets as a way of exchanging commands and return values.
- **Dependency injection** and **service locator**: Both dependency injection and service locator can be useful to replace the implementation of a module at the moment of its injection. When we introduced the concept of modules maps, in the Packing section, we also saw how the Service Locator pattern was intrinsically used by module bundlers to collate all the code from different modules into one file.
 - Strategy và template: Hai mẫu này có lẽ là hữu ích nhất khi chia sẻ mã với trình duyệt. Ý đồ của chúng, thực ra, là định nghĩa các bước chung của một thuật toán, cho phép một số phần của nó được thay thế, đúng là thứ chúng ta cần! Trong phát triển đa nền tảng, các mẫu này cho phép chúng ta chia sẻ phần không phụ thuộc nền tảng (platform-agnostic) của các thành phần của mình, đồng thời cho phép thay đổi các phần đặc thù theo nền tảng của chúng bằng một strategy hoặc template method khác (vốn có thể được thay đổi bằng rẽ nhánh mã (runtime hoặc build-time) hoặc hoán đổi module).
 - Adapter: Mẫu này có lẽ hữu ích nhất khi chúng ta cần hoán đổi cả một thành phần. Chúng ta đã thấy vài ví dụ ở Chương 8, Các mẫu thiết kế cấu trúc (Structural Design Patterns). Nếu ứng dụng máy chủ của bạn đang dùng một cơ sở dữ liệu như SQLite, bạn có thể dùng mẫu Adapter để cung cấp một cách triển khai lưu trữ dữ liệu thay thế hoạt động trong trình duyệt. Chẳng hạn bạn có thể dùng API `localStorage` (nodejsdp.link/localstorage) hoặc API `IndexedDB` (nodejsdp.link/indexdb).
 - Proxy: Khi mã nhằm chạy trên máy chủ lại chạy trên trình duyệt, chúng ta thường cần một chức năng vốn được dùng trên máy chủ phải có sẵn cả trên trình duyệt. Đây là nơi mẫu remote Proxy phát huy tác dụng. Hãy hình dung nếu chúng ta muốn truy cập hệ thống tập tin của máy chủ từ trình duyệt: chúng ta có thể nghĩ đến việc tạo một đối tượng `fs` trên client làm proxy cho mọi lời gọi tới module `fs` đang sống trên máy chủ, dùng Ajax hoặc WebSockets như một phương tiện để trao đổi lệnh và giá trị trả về.
 - Dependency injection và service locator: Cả dependency injection lẫn service locator đều có thể hữu ích để thay thế cách triển khai của một module tại thời điểm tiêm (inject) nó. Khi chúng ta giới thiệu khái niệm modules map, trong phần Đóng gói

(Packing), chúng ta cũng đã thấy mẫu Service Locator được module bundler dùng một cách nội tại như thế nào để dồn tất cả mã từ các module khác nhau vào một tập tin.

As we can see, the arsenal of patterns at our disposal is quite powerful, but the most powerful weapon is still the ability of the developer to choose the best approach and adapt it to the specific problem at hand.

Như chúng ta có thể thấy, kho vũ khí các mẫu trong tay chúng ta khá mạnh mẽ, nhưng vũ khí mạnh nhất vẫn là khả năng của lập trình viên trong việc chọn cách tiếp cận tốt nhất và điều chỉnh nó cho vấn đề cụ thể đang phải giải quyết.

Now that we understand the fundamentals of module bundlers and we have learned a number of useful patterns to write cross-platform code, we are ready to move into the second part of this chapter, where we will learn about React and write our first universal JavaScript application.

Giờ khi đã hiểu các nền tảng cốt lõi của module bundler và đã học một số mẫu hữu ích để viết mã đa nền tảng, chúng ta đã sẵn sàng chuyển sang phần thứ hai của chương này, nơi chúng ta sẽ tìm hiểu về React và viết ứng dụng Universal JavaScript đầu tiên của mình.

A brief introduction to React — Giới thiệu ngắn gọn về React

React is a popular JavaScript library created and maintained by Facebook. React is focused on providing a comprehensive set of functions and tools to build the view layer in web applications. React offers a view abstraction focused on the concept of a component. A component could be a button, a form input, a simple container such as an HTML div, or any other element in your user interface. The idea is that you should be able to construct the user interface of your application by just defining and composing highly reusable components with specific responsibilities.

React là một thư viện JavaScript phổ biến được tạo và bảo trì bởi Facebook. React tập trung vào việc cung cấp một bộ hàm và công cụ toàn diện để xây dựng tầng khung nhìn (view layer) trong các ứng dụng web. React cung cấp một lớp trừu tượng khung nhìn tập trung vào khái niệm thành phần (component). Một component có thể là một nút bấm, một ô nhập của form, một vùng chứa đơn giản như một div HTML, hoặc bất kỳ phần tử nào khác trong giao diện người dùng của bạn. Ý tưởng là bạn có thể dựng nên giao diện người dùng của ứng dụng chỉ bằng cách định nghĩa và tổ hợp các component có tính tái sử dụng cao với những trách nhiệm cụ thể.

What makes React different from other view libraries for the web is that it is not bound to the DOM by design. In fact, it provides a high-level abstraction called the virtual DOM (nodejsdp.link/virtual-dom) that fits very well with the web but that can also be used in other contexts, for example, for building mobile apps, modeling 3D environments, or even defining the interaction between hardware components. In simple terms, the virtual DOM can be seen as an efficient way to re-render data organized in a tree-like structure.

Điều khiến React khác với các thư viện khung nhìn khác cho web là về mặt thiết kế nó không bị ràng buộc với DOM. Thực tế, nó cung cấp một lớp trừu tượng cấp cao gọi là virtual DOM (nodejsdp.link/virtual-dom) vốn rất hợp với web nhưng cũng có thể dùng trong các ngữ cảnh khác, ví dụ, để xây dựng ứng dụng di động, mô hình hoá môi trường 3D, hoặc thậm chí định nghĩa sự tương tác giữa các thành phần phần cứng. Nói đơn giản, virtual DOM có thể được xem như một cách hiệu quả để kết xuất lại (re-render) dữ liệu được tổ chức theo cấu trúc dạng cây.

“Learn it once, use it everywhere.” —Facebook This is the motto used by Facebook to introduce React. It intentionally mocks the famous Java motto Write it once, run it everywhere with the clear intention to underline a fundamental shift from the Java philosophy. The original design goal of Java was to allow developers to write applications once and run them on as many platforms as possible without changes. Conversely, the React philosophy acknowledges that every platform is inherently different and therefore encourages developers to write different applications that are optimized for the related target platform. React, as a library, shifts its focus on providing convenient design and architecture principles and tools that, once mastered, can be easily used to write platform-specific code.

“Học một lần, dùng ở mọi nơi.” —Facebook Đây là khẩu hiệu Facebook dùng để giới thiệu React. Nó cố ý nhại lại khẩu hiệu Java nổi tiếng Viết một lần, chạy ở mọi nơi với ý đồ rõ ràng nhằm nhấn mạnh một sự chuyển dịch căn bản khỏi triết lý của Java. Mục tiêu thiết kế ban đầu của Java là cho phép lập trình viên viết ứng dụng một lần và chạy chúng trên càng nhiều nền tảng càng tốt mà không cần thay đổi. Ngược lại, triết lý của React thừa nhận rằng mỗi nền tảng vốn dĩ khác nhau và do đó khuyến khích lập trình viên viết những ứng dụng khác nhau được tối ưu cho nền tảng đích liên quan. React, với tư cách một thư viện, chuyển trọng tâm sang việc cung cấp các nguyên tắc thiết kế và kiến trúc cùng các công cụ tiện lợi mà, một khi thành thạo, có thể dễ dàng dùng để viết mã đặc thù theo nền tảng.

If you are curious to learn about the applications of React in contexts not strictly related to the field of web development, you can have a look at the following projects: React Native for mobile apps (nodejsdp.link/react-native), React PIXI for 2D rendering with OpenGL (nodejsdp.link/react-pixi), reactthree-fiber to create 3D scenes (nodejsdp.link/react-three-fiber), and React Hardware (nodejsdp.link/react-hardware).

Nếu bạn tò mò muốn tìm hiểu về các ứng dụng của React trong những ngữ cảnh không gắn chặt với lĩnh vực phát triển web, bạn có thể xem qua các dự án sau: React Native cho ứng dụng di động (nodejsdp.link/react-native), React PIXI cho kết xuất 2D với OpenGL (nodejsdp.link/react-pixi), react-three-fiber để tạo các cảnh 3D (nodejsdp.link/react-three-fiber), và React Hardware (nodejsdp.link/react-hardware).

The main reason why React is so interesting in the context of Universal JavaScript development is because it allows us to render React components both on the client and on the server using almost the same code. To put it another way, with React, we are able to render the HTML code that is required to display the page directly from Node.js. Then, when the page is loaded on the browser, React will perform a process called hydration (nodejsdp.link/hydration), which will add all the frontend-only side effects like click handlers, animations, additional asynchronous data fetching, dynamic routing, and so on. Hydration converts a static markup into a fully interactive experience.

Lý do chính khiến React rất thú vị trong ngữ cảnh phát triển Universal JavaScript là vì nó cho phép chúng ta kết xuất các component React trên cả client lẫn máy chủ bằng gần như cùng một đoạn mã. Nói cách khác, với React, chúng ta có thể kết xuất mã HTML cần thiết để hiển thị trang trực tiếp từ Node.js. Sau đó, khi trang được tải trên trình duyệt, React sẽ thực hiện một quá trình gọi là hydration (nodejsdp.link/hydration), vốn sẽ thêm tất cả các tác dụng phụ (side effect) chỉ dành cho frontend như trình xử lý click, hoạt ảnh, tải dữ liệu bất đồng bộ bổ sung, định tuyến động, v.v. Hydration chuyển đổi một markup tĩnh thành một trải nghiệm có tính tương tác đầy đủ.

This approach allows us to build single-page applications (SPAs), where the first render happens mostly on the server, but then, once the page is loaded on the browser and the user starts to click around, only the parts of the page that need to be changed are dynamically refreshed, without

requiring a full page reload.

Cách tiếp cận này cho phép chúng ta xây dựng các ứng dụng single-page (SPA), nơi lần kết xuất đầu tiên xảy ra chủ yếu trên máy chủ, nhưng sau đó, một khi trang được tải trên trình duyệt và người dùng bắt đầu click loanh quanh, chỉ những phần của trang cần thay đổi mới được làm mới một cách động, mà không cần tải lại toàn trang.

This design offers two main advantages:

Thiết kế này mang lại hai ưu điểm chính:

- Better search engine optimization (SEO): Since the page markup is prerendered by the server, various search engines can make sense of the content of the page by just looking at the HTML returned by the server. They won't need to simulate a browser environment and wait for the page to be fully loaded to see what a given page is about.
- Better performance: Since we are pre-rendering the markup, this will be already available and visible on the browser, even while the browser is still downloading, parsing, and executing the JavaScript code included with the page. This approach can lead to a better user experience as the content is perceived to load faster and there are less browser “flashes” during rendering.
 - Tối ưu hoá cho công cụ tìm kiếm (SEO) tốt hơn: Vì markup của trang được kết xuất trước (pre-render) bởi máy chủ, các công cụ tìm kiếm khác nhau có thể hiểu nội dung của trang chỉ bằng cách nhìn vào HTML mà máy chủ trả về. Chúng sẽ không cần mô phỏng một môi trường trình duyệt và chờ trang được tải đầy đủ để xem một trang cho trước nói về điều gì.
 - Hiệu năng tốt hơn: Vì chúng ta kết xuất trước markup, nó sẽ đã có sẵn và hiển thị trên trình duyệt, ngay cả khi trình duyệt vẫn đang tải về, phân tích cú pháp và thực thi mã JavaScript đi kèm với trang. Cách tiếp cận này có thể dẫn đến trải nghiệm người dùng tốt hơn vì nội dung được cảm nhận là tải nhanh hơn và có ít “nhấp nháy” (flash) của trình duyệt hơn trong quá trình kết xuất.

It is worth mentioning that the React virtual DOM is capable of optimizing the way changes are rendered. This means that the DOM is not rendered in full after every change, but instead React uses a smart in-memory diffing algorithm that is able to precalculate the minimum number of changes to apply to the DOM in order to update the view. This results in a very efficient mechanism for fast browser rendering.

Đáng nói rằng virtual DOM của React có khả năng tối ưu cách kết xuất các thay đổi. Điều này nghĩa là DOM không được kết xuất toàn bộ sau mỗi thay đổi, mà thay vào đó React dùng một thuật toán so khác (diffing) thông minh trong bộ nhớ, có khả năng tính toán trước số lượng thay đổi tối thiểu cần áp dụng lên DOM để cập nhật khung nhìn. Điều này tạo ra một cơ chế rất hiệu quả cho việc kết xuất nhanh trên trình duyệt.

Now that we know what React is, in the next section, we will write our first React component!

Giờ khi đã biết React là gì, trong phần tiếp theo, chúng ta sẽ viết component React đầu tiên của mình!

Hello React — Hello React

Without further ado, let's start to use React and jump to a concrete example. This will be a “Hello World” type of example but it will help us to illustrate the main ideas behind React, before we move onto more realistic examples.

Không dài dòng thêm, hãy bắt đầu dùng React và nhảy vào một ví dụ cụ thể. Đây sẽ là một ví dụ kiểu “Hello World” nhưng nó sẽ giúp chúng ta minh họa những ý tưởng chính đằng sau React, trước khi chuyển sang các ví dụ thực tế hơn.

Let’s start by creating a new webpack project in a new folder with:

Hãy bắt đầu bằng việc tạo một dự án webpack mới trong một thư mục mới với:

```
npm init -y
npm install --save-dev webpack webpack-cli
node_modules/.bin/webpack init
```

Then, follow the guided instructions. Now, let’s install React:

Sau đó, làm theo các hướng dẫn được dẫn dắt. Giờ hãy cài React:

```
npm install --save react react-dom
```

Now, let’s create a file, `src/index.js`, with the following content:

Bây giờ, hãy tạo một tập tin, `src/index.js`, với nội dung sau:

```
import react from 'react'
import ReactDOM from 'react-dom'
const h = react.createElement // (1)
class Hello extends react.Component { // (2)
  render () { // (3)
    return h('h1', null, [ // (4)
      'Hello ',
      this.props.name || 'World' // (5)
    ])
  }
}
ReactDOM.render( // (6)
  h(Hello, { name: 'React' }),
  document.getElementsByTagName('body')[0]
)
```

Let’s review what’s happening with this code:

Hãy điểm lại những gì đang diễn ra với mã này:

1. The first thing that we do is to create a handy alias for the `react.createElement` function. We will be using this function a couple of times in this example to create React elements. These could be plain DOM nodes (regular HTML tags) or instances of React components.
2. Now, we define our `Hello` component, which has to extend the `react.Component` class.
3. Every React component has to implement a `render()` method. This method defines how the component will be displayed on the screen when it is rendered on the DOM and it has to return a React element.
4. We are using the `react.createElement` function to create an `h1` DOM element. This method expects three or more arguments. The first argument is the tag name (as a string) or a React component class. The second argument is an object used to pass attributes (or props) to the component (or `null` if we don’t need to specify any attribute). Finally, the third argument is an array (or you can pass multiple arguments as well) of children elements. Elements can also be text (text nodes), as in our current example.

5. Here, we are using `this.props` to access the attributes that are passed to this component at runtime. In this specific case, we are looking for the `name` attribute. If this is passed, we use it to construct a text node; otherwise, we default to the string `"World"`.
6. In this last block of code, we use `ReactDOM.render()` to initialize our application. This function is responsible for attaching a React application to the existing page. An application is nothing more than an instance of a React component. Here, we are instantiating our `Hello` component and passing the string `"React"` for the `name` attribute. Finally, as the last argument, we have to specify which DOM node in the page will be the parent element of our application. In this case, we are using the `body` element of the page, but you can target any existing DOM element in the page.

1. Điều đầu tiên chúng ta làm là tạo một bí danh (alias) tiện lợi cho hàm `react.createElement`. Chúng ta sẽ dùng hàm này vài lần trong ví dụ này để tạo các phần tử React (React element). Chúng có thể là các nút DOM thuần (thẻ HTML thông thường) hoặc các instance của các component React.
2. Bây giờ, chúng ta định nghĩa component `Hello` của mình, vốn phải kế thừa lớp `react.Component`.
3. Mỗi component React phải triển khai một phương thức `render()`. Phương thức này định nghĩa cách component sẽ được hiển thị trên màn hình khi nó được kết xuất lên DOM và nó phải trả về một phần tử React.
4. Chúng ta đang dùng hàm `react.createElement` để tạo một phần tử DOM `h1`. Phương thức này mong đợi ba tham số trở lên. Tham số thứ nhất là tên thẻ (dưới dạng chuỗi) hoặc một lớp component React. Tham số thứ hai là một đối tượng dùng để truyền thuộc tính (hay props) vào component (hoặc `null` nếu chúng ta không cần chỉ định thuộc tính nào). Cuối cùng, tham số thứ ba là một mảng (hoặc bạn cũng có thể truyền nhiều tham số) các phần tử con. Các phần tử cũng có thể là văn bản (nút văn bản, text node), như trong ví dụ hiện tại của chúng ta.
5. Ở đây, chúng ta đang dùng `this.props` để truy cập các thuộc tính được truyền vào component này tại runtime. Trong trường hợp cụ thể này, chúng ta đang tìm thuộc tính `name`. Nếu nó được truyền vào, chúng ta dùng nó để dựng một nút văn bản; nếu không, chúng ta mặc định về chuỗi `"World"`.
6. Trong khối mã cuối cùng này, chúng ta dùng `ReactDOM.render()` để khởi tạo ứng dụng của mình. Hàm này chịu trách nhiệm gắn một ứng dụng React vào trang hiện có. Một ứng dụng không là gì khác ngoài một instance của một component React. Ở đây, chúng ta đang khởi tạo component `Hello` và truyền chuỗi `"React"` cho thuộc tính `name`. Cuối cùng, làm tham số cuối, chúng ta phải chỉ định nút DOM nào trong trang sẽ là phần tử cha của ứng dụng. Trong trường hợp này, chúng ta đang dùng phần tử `body` của trang, nhưng bạn có thể nhắm vào bất kỳ phần tử DOM nào hiện có trong trang.

Now, you can see a preview of your application by running:

Giờ, bạn có thể xem trước ứng dụng của mình bằng cách chạy:

```
npm start
```

You should now see “Hello React” in your browser window. Congratulations, you have built your first React application!

Bây giờ bạn sẽ thấy “Hello React” trong cửa sổ trình duyệt. Chúc mừng, bạn đã xây dựng ứng dụng React đầu tiên của mình!

Alternatives to `react.createElement` — Các lựa chọn thay thế cho `react.createElement`

Repeated usage of `react.createElement()` might compromise the readability of our React components. In fact, nesting many invocations of `react.createElement()`, even with our `h()` alias, will make it hard to understand the HTML structure we want our components to render.

Việc dùng lặp đi lặp lại `react.createElement()` có thể làm tổn hại khả năng đọc của các component React của chúng ta. Thực tế, lồng nhiều lời gọi `react.createElement()`, ngay cả với bí danh `h()` của chúng ta, sẽ khiến khó hiểu cấu trúc HTML mà chúng ta muốn các component của mình kết xuất.

For this reason, it is not very common to use `react.createElement()` directly. To address this problem, the React team offers and encourages an alternative syntax called JSX (nodejsdp.link/jsx).

Vì lý do này, việc dùng trực tiếp `react.createElement()` không phổ biến lắm. Để giải quyết vấn đề này, đội ngũ React cung cấp và khuyến khích một cú pháp thay thế gọi là JSX (nodejsdp.link/jsx).

JSX is a superset of JavaScript that allows you to embed HTML-like code into JavaScript code. JSX makes the creation of React elements similar to writing HTML code. With JSX, React components are generally more readable and easier to write. It is easier to see what we mean here by looking at a concrete example, so let's rewrite our “Hello React” application using JSX:

JSX là một tập cha (superset) của JavaScript cho phép bạn nhúng mã giống HTML vào mã JavaScript. JSX làm cho việc tạo các phần tử React giống với việc viết mã HTML. Với JSX, các component React nói chung dễ đọc và dễ viết hơn. Ý chúng ta dễ thấy hơn khi nhìn vào một ví dụ cụ thể, nên hãy viết lại ứng dụng “Hello React” của chúng ta bằng JSX:

```
import react from 'react'
import ReactDOM from 'react-dom'
class Hello extends react.Component {
  render () {
    return <h1>Hello {this.props.name || 'World'}</h1>
  }
}
ReactDOM.render(
  <Hello name="React"/>,
  document.getElementsByTagName('body')[0]
)
```

Much more readable, isn't it? Unfortunately, since JSX is not a standard JavaScript feature, adopting JSX would require us to “compile” JSX code into standard equivalent JavaScript code. In the context of Universal JavaScript applications, we would have to do this both on the client-side code and the server-side code, so, for the sake of simplicity, we are not going to use JSX throughout the rest of this chapter.

Dễ đọc hơn nhiều, phải không? Tiếc thay, vì JSX không phải là một tính năng JavaScript tiêu chuẩn, việc áp dụng JSX sẽ đòi hỏi chúng ta phải “biên dịch” (compile) mã JSX thành mã JavaScript tiêu chuẩn tương đương. Trong ngữ cảnh các ứng dụng Universal JavaScript, chúng ta sẽ phải làm điều này cho cả mã phía client lẫn mã phía máy chủ, nên, để đơn giản, chúng ta sẽ không dùng JSX trong phần còn lại của chương này.

There are some relatively new JSX alternatives that rely on standard JavaScript tagged template literals (you can read more about JavaScript tagged template literals at nodejsdp.link/template-literals). Using template literals seems to be a good compromise between code that is still quite

easy to read and write and not having to perform an intermediate compilation process. Two of the most promising libraries providing this functionality are `htm` (nodejsdp.link/htm) and `esx` (nodejsdp.link/esx).

Có một số lựa chọn thay thế JSX khá mới dựa vào tagged template literal tiêu chuẩn của JavaScript (bạn có thể đọc thêm về tagged template literal của JavaScript tại nodejsdp.link/template-literals). Dùng template literal có vẻ là một sự thoả hiệp tốt giữa mã vẫn khá dễ đọc và dễ viết với việc không phải thực hiện một quá trình biên dịch trung gian. Hai trong số các thư viện hứa hẹn nhất cung cấp chức năng này là `htm` (nodejsdp.link/htm) và `esx` (nodejsdp.link/esx).

In the rest of this chapter, we will be using `htm`, so let's rewrite once more our “Hello React” example, this time using `htm`:

Trong phần còn lại của chương này, chúng ta sẽ dùng `htm`, nên hãy viết lại một lần nữa ví dụ “Hello React” của chúng ta, lần này dùng `htm`:

```
import react from 'react'
import ReactDOM from 'react-dom'
import htm from 'htm'
const html = htm.bind(react.createElement) // (1)
class Hello extends react.Component {
  render () { // (2)
    return html`<h1>
      Hello ${this.props.name || 'World'}
    </h1>`
  }
}
ReactDOM.render(
  html`<${Hello} name="React"/>`, // (3)
  document.getElementsByTagName('body')[0]
)
```

This code looks quite readable, but let's quickly clarify how we are using `htm` here:

Mã này trông khá dễ đọc, nhưng hãy nhanh chóng làm rõ cách chúng ta đang dùng `htm` ở đây:

1. The first thing that we have to do is create the template tag function `html`. This function allows us to use template literals to generate React elements. At runtime, this template tag function will be calling `react.createElement()` for us when needed.
2. Here, we use a tagged template literal with the `html` tag function to create an `h1` tag. Note that, as this is a standard tagged template literal, we can use the regular placeholder syntax (`${expression}`) to insert dynamic expressions into the string. Remember that template literals and tagged template literals use backticks (") instead of single quotes (') to delimit the template string.
3. Similarly, we can use the placeholder syntax to create instances of React components (`<${ComponentClass}>`). Note that, if a component instance contains children elements, we can use the special `</>` tag to indicate the end of the component (for example, `<${Component}><child/></>`). Finally, we can pass props to the component as normal HTML attributes.

1. Điều đầu tiên chúng ta phải làm là tạo hàm thẻ template (template tag function) `html`. Hàm này cho phép chúng ta dùng template literal để tạo các phần tử React.

Tại runtime, hàm thẻ template này sẽ gọi `react.createElement()` thay cho chúng ta khi cần.

- Ở đây, chúng ta dùng một tagged template literal với hàm thẻ `html` để tạo một thẻ `h1`. Lưu ý rằng, vì đây là một tagged template literal tiêu chuẩn, chúng ta có thể dùng cú pháp placeholder thông thường (`${expression}`) để chèn các biểu thức động vào chuỗi. Hãy nhớ rằng template literal và tagged template literal dùng dấu backtick (") thay vì dấu nháy đơn (') để phân định chuỗi template.
- Tương tự, chúng ta có thể dùng cú pháp placeholder để tạo các instance của component React (`<${ComponentClass}>`). Lưu ý rằng, nếu một instance component chứa các phần tử con, chúng ta có thể dùng thẻ đặc biệt `</>` để chỉ điểm kết thúc của component (ví dụ, `<${Component}><child/></>`). Cuối cùng, chúng ta có thể truyền props vào component như các thuộc tính HTML thông thường.

At this point, we should be able to understand the basic structure of a simple “Hello World” React component. In the next section, we will show you how to manage states in a React component, an important concept for most real-world applications.

Tại điểm này, chúng ta đã có thể hiểu cấu trúc cơ bản của một component React “Hello World” đơn giản. Trong phần tiếp theo, chúng ta sẽ chỉ cho bạn cách quản lý trạng thái (state) trong một component React, một khái niệm quan trọng đối với hầu hết các ứng dụng thực tế.

Stateful components — Component có trạng thái (Stateful components)

In the previous example, we saw how to build a stateless React component. By stateless, we mean that the component only receives input from the outside (in our example, it was receiving a name property) and it doesn’t need to calculate or manage any internal information to be able to render itself to the DOM.

Trong ví dụ trước, chúng ta đã thấy cách xây dựng một component React không trạng thái (stateless). Bằng từ stateless, chúng ta muốn nói rằng component chỉ nhận đầu vào từ bên ngoài (trong ví dụ của chúng ta, nó nhận một thuộc tính name) và nó không cần tính toán hay quản lý bất kỳ thông tin nội bộ nào để có thể tự kết xuất ra DOM.

While it’s great to have stateless components, sometimes, you have to manage some kind of **state**. React allows us to do that, so let’s learn how with an example.

Dù có các component không trạng thái thì rất tốt, đôi khi bạn phải quản lý một loại trạng thái nào đó. React cho phép chúng ta làm điều đó, nên hãy học cách làm qua một ví dụ.

Let’s build a React application that displays a list of projects that have been recently updated on GitHub.

Hãy xây dựng một ứng dụng React hiển thị một danh sách các dự án vừa được cập nhật gần đây trên GitHub.

We can encapsulate all the logic for asynchronously fetching the data from GitHub and displaying it on a dedicated component: the `RecentGithubProjects` component. This component is configurable through the `query` prop, which allows us to filter the projects on GitHub. The `query` prop will receive a keyword such as “javascript” or “react”, and this value will be used to construct the API call to GitHub.

Chúng ta có thể đóng gói toàn bộ logic cho việc lấy dữ liệu bất đồng bộ từ GitHub và hiển thị nó vào một component chuyên dụng: component `RecentGithubProjects`. Component này có thể cấu hình thông qua prop `query`, cho phép chúng ta lọc các dự án trên GitHub.

Prop query sẽ nhận một từ khoá như “javascript” hoặc “react”, và giá trị này sẽ được dùng để dựng lời gọi API tới GitHub.

Let’s finally have a look at the code of the RecentGithubProjects component:

Cuối cùng hãy xem mã của component RecentGithubProjects:

```
// src/RecentGithubProjects.js
import react from 'react'
import htm from 'htm'
const html = htm.bind(react.createElement)
function createRequestUri (query) {
  return `https://api.github.com/search/repositories?q=${
    encodeURIComponent(query)
  }&sort=updated`
}
export class RecentGithubProjects extends react.Component {
  constructor (props) { // (1)
    super(props) // (2)
    this.state = { // (3)
      loading: true,
      projects: []
    }
  }
  async loadData () { // (4)
    this.setState({ loading: true, projects: [] })
    const response = await fetch(
      createRequestUri(this.props.query),
      { mode: 'cors' }
    )
    const responseBody = await response.json()
    this.setState({
      projects: responseBody.items,
      loading: false
    })
  }
  componentDidMount () { // (5)
    this.loadData()
  }
  componentDidUpdate (prevProps) { // (6)
    if (this.props.query !== prevProps.query) {
      this.loadData()
    }
  }
  render () { // (7)
    if (this.state.loading) {
      return 'Loading ...'
    }
    // (8)
    return html`<ul>
      ${this.state.projects.map(project => html`
        <li key=${project.id}>`
```

```

        <a href=${project.html_url}>${project.full_name}</a>:
        ${' '}${project.description}
    </li>
    `)}
</ul>`
}
}

```

There are some new React concepts in this component, so let's discuss the main details here:

Có một số khái niệm React mới trong component này, nên hãy bàn về các chi tiết chính ở đây:

1. In this new component, we are overriding the default constructor. A constructor accepts the props passed to the component as an argument.
2. The first thing we have to do is call the original constructor and propagate the props so that the component can be initialized correctly by React.
3. Now, we can define the initial component state. Our final state is going to be a list of GitHub projects, but those won't be available immediately as we will need to load them dynamically. Therefore, we define the initial state as a boolean flag, indicating that we are loading the data and the list of projects as an empty array.
4. The function `loadData()` is the function that is responsible for making the API **request**, fetching the necessary data, and updating the internal state using `this.setState()`. Note that `this.setState()` is called twice: before we issue the HTTP request (to activate the loading state) and when the request is completed (to unset the loading flag and populate the list of projects). React will re-render the component automatically when the state changes.
5. Here, we are introducing another new concept: the `componentDidMount` lifecycle function. This function is automatically invoked by React once the component has been successfully instantiated and attached (or mounted) to the DOM. This is the perfect place to load our data for the first time.
6. The function `componentDidUpdate` is another React lifecycle function and it is automatically invoked every time the component is updated (for instance, if new props have been passed to the component). Here, we check if the `query` prop has changed since the last update. If that's the case, then we need to reload the list of projects.
7. Finally, let's see what happens in our `render()` function. The main thing to note is that here we have to handle the two different states of the component: the loading state and the state where we have the list of projects available for display. Since React will invoke the `render()` function every time the state or the props change, just having an `if` statement here will be enough. This technique is often called conditional rendering.
8. In this final step, we are rendering a list of elements using `Array.map()` to create a list element for every project fetched using the GitHub API. Note that every list element receives a value for the `key` prop. The `key` prop is a special prop that is recommended whenever you are rendering an array of elements. Every element should provide a unique key. This prop helps the virtual DOM optimize every rendering pass (If you are curious to understand in detail what React does in this situation you can have a look at nodejsdp.link/react-reconciliation).

1. Trong component mới này, chúng ta đang ghi đè constructor mặc định. Một constructor nhận các props được truyền vào component làm tham số.
2. Điều đầu tiên chúng ta phải làm là gọi constructor gốc và truyền tiếp (propagate) các props để component có thể được React khởi tạo đúng đắn.
3. Bây giờ, chúng ta có thể định nghĩa trạng thái khởi đầu của component. Trạng thái cuối cùng của chúng ta sẽ là một danh sách các dự án GitHub, nhưng chúng sẽ không

có sẵn ngay lập tức vì chúng ta sẽ cần tải chúng một cách động. Do đó, chúng ta định nghĩa trạng thái khởi đầu là một cờ boolean, cho biết rằng chúng ta đang tải dữ liệu, và danh sách dự án là một mảng rỗng.

4. Hàm `loadData()` là hàm chịu trách nhiệm thực hiện yêu cầu API, lấy dữ liệu cần thiết, và cập nhật trạng thái nội bộ bằng `this.setState()`. Lưu ý rằng `this.setState()` được gọi hai lần: trước khi chúng ta phát ra yêu cầu HTTP (để kích hoạt trạng thái loading) và khi yêu cầu hoàn tất (để bỏ cờ loading và điền danh sách dự án). React sẽ tự động kết xuất lại component khi trạng thái thay đổi.
5. Ở đây, chúng ta đang giới thiệu một khái niệm mới khác: hàm vòng đời (lifecycle function) `componentDidMount`. Hàm này được React tự động gọi một khi component đã được khởi tạo thành công và gắn (hay mount) vào DOM. Đây là nơi hoàn hảo để tải dữ liệu của chúng ta lần đầu tiên.
6. Hàm `componentDidUpdate` là một hàm vòng đời khác của React và nó được tự động gọi mỗi lần component được cập nhật (chẳng hạn, nếu các props mới đã được truyền vào component). Ở đây, chúng ta kiểm tra xem prop `query` có thay đổi kể từ lần cập nhật trước hay không. Nếu đúng vậy, thì chúng ta cần tải lại danh sách dự án.
7. Cuối cùng, hãy xem điều gì xảy ra trong hàm `render()` của chúng ta. Điều chính cần lưu ý là ở đây chúng ta phải xử lý hai trạng thái khác nhau của component: trạng thái loading và trạng thái nơi chúng ta có danh sách dự án sẵn sàng để hiển thị. Vì React sẽ gọi hàm `render()` mỗi lần trạng thái hoặc props thay đổi, chỉ cần có một câu lệnh `if` ở đây là đủ. Kỹ thuật này thường được gọi là kết xuất có điều kiện (conditional rendering).
8. Ở bước cuối cùng này, chúng ta đang kết xuất một danh sách các phần tử bằng `Array.map()` để tạo một phần tử danh sách cho mỗi dự án được lấy về bằng API GitHub. Lưu ý rằng mỗi phần tử danh sách nhận một giá trị cho prop `key`. Prop `key` là một prop đặc biệt được khuyến nghị mỗi khi bạn kết xuất một mảng các phần tử. Mỗi phần tử nên cung cấp một key duy nhất. Prop này giúp virtual DOM tối ưu mỗi lượt kết xuất (Nếu bạn tò mò muốn hiểu chi tiết React làm gì trong tình huống này bạn có thể xem qua nodejsdp.link/react-reconciliation).

You might have noticed that we are not handling potential errors while fetching the data. There are several ways we can do this in React. The most elegant solution is probably implementing an `ErrorBoundary` component (nodejsdp.link/error-boundary), but we will leave that as an exercise for you.

Có lẽ bạn đã nhận thấy rằng chúng ta không xử lý các lỗi tiềm tàng khi lấy dữ liệu. Có vài cách chúng ta có thể làm điều này trong React. Giải pháp thanh lịch nhất có lẽ là triển khai một component `ErrorBoundary` (nodejsdp.link/error-boundary), nhưng chúng ta sẽ để điều đó như một bài tập cho bạn.

Let's now write the main application component. Here, we want to display a navigation menu where the user can select different queries ("JavaScript", "Node.js", and "React") to filter for different types of GitHub projects:

Giờ hãy viết component ứng dụng chính. Ở đây, chúng ta muốn hiển thị một menu điều hướng nơi người dùng có thể chọn các truy vấn khác nhau ("JavaScript", "Node.js" và "React") để lọc các loại dự án GitHub khác nhau:

```
// src/App.js
import React from 'react'
import htm from 'htm'
import { RecentGithubProjects } from './RecentGithubProjects.js'
const html = htm.bind(React.createElement)
```

```

export class App extends react.Component {
  constructor (props) {
    super(props)
    this.state = {
      query: 'javascript',
      label: 'JavaScript'
    }
    this.setQuery = this.setQuery.bind(this)
  }
  setQuery (e) {
    e.preventDefault()
    const label = e.currentTarget.text
    this.setState({ label, query: label.toLowerCase() })
  }
  render () {
    return html`<div>
      <nav>
        <a href="#" onClick=${this.setQuery}>JavaScript</a>
        ${' '}
        <a href="#" onClick=${this.setQuery}>Node.js</a>
        ${' '}
        <a href="#" onClick=${this.setQuery}>React</a>
      </nav>
      <h1>Recently updated ${this.state.label} projects</h1>
      <${RecentGithubProjects} query=${this.state.query}/>
    </div>`
  }
}

```

This component is using its internal state to track the currently selected query. Initially, the “javascript” query is set and passed down to the RecentGithubProjects component. Then, every time a keyword in the navigation menu is clicked, we update the state with the new selected keyword. When this happens, the render() method will be automatically invoked and it will pass the new value for the query prop to RecentGithubProjects. In turn, RecentGithubProjects will be marked as updated, and it will internally reload and eventually update the list of projects for the new query.

Component này dùng trạng thái nội bộ của nó để theo dõi truy vấn đang được chọn. Ban đầu, truy vấn “javascript” được đặt và truyền xuống component RecentGithubProjects. Sau đó, mỗi lần một từ khoá trong menu điều hướng được click, chúng ta cập nhật trạng thái với từ khoá mới được chọn. Khi điều này xảy ra, phương thức render() sẽ được tự động gọi và nó sẽ truyền giá trị mới cho prop query tới RecentGithubProjects. Đến lượt, RecentGithubProjects sẽ được đánh dấu là đã cập nhật, và nó sẽ tải lại một cách nội bộ và cuối cùng cập nhật danh sách dự án cho truy vấn mới.

One interesting detail to underline is that, in the constructor, we are explicitly binding the setQuery() function to the current component instance. The reason why we do this is because this function is used directly as an event handler for the click event. In this case, the reference to this would be undefined without the bind and it would not be possible to call this.setState() from the handler.

Một chi tiết thú vị cần nhấn mạnh là, trong constructor, chúng ta đang ràng buộc (bind)

một cách tường minh hàm `setQuery()` với instance component hiện tại. Lý do chúng ta làm điều này là vì hàm này được dùng trực tiếp làm trình xử lý sự kiện (event handler) cho sự kiện click. Trong trường hợp này, tham chiếu tới `this` sẽ là `undefined` nếu không có bind và sẽ không thể gọi `this.setState()` từ trình xử lý.

At this point, we only need to attach the App component to the DOM to run our application. Let's do this:

Tại điểm này, chúng ta chỉ cần gắn component App vào DOM để chạy ứng dụng của mình. Hãy làm điều này:

```
// src/index.js
import react from 'react'
import ReactDOM from 'react-dom'
import htm from 'htm'
import { App } from './App.js'
const html = htm.bind(react.createElement)
ReactDOM.render(
  html`<${App}/>`,
  document.getElementsByTagName('body')[0]
)
```

Finally, let's just run the application with `npm start` and test it on the browser.

Cuối cùng, hãy chạy ứng dụng với `npm start` và kiểm thử nó trên trình duyệt.

Note that since we used `async/await` in our application, the default configuration generated by webpack might not work straight away. If you have any issues, compare your configuration file with the one in the code examples provided with this book (nodejsdp.link/wpconf).

Lưu ý rằng vì chúng ta đã dùng `async/await` trong ứng dụng của mình, cấu hình mặc định được webpack tạo ra có thể không hoạt động ngay. Nếu bạn gặp bất kỳ vấn đề nào, hãy so sánh tập tin cấu hình của bạn với tập tin trong các ví dụ mã đi kèm cuốn sách này (nodejsdp.link/wpconf).

Try to refresh the page and click on the various keywords on the navigation menu. After a few seconds, you should see the list of projects being refreshed.

Hãy thử làm mới trang và click vào các từ khoá khác nhau trên menu điều hướng. Sau vài giây, bạn sẽ thấy danh sách dự án được làm mới.

At this point, it should be quite clear to you how React works, how to compose components together, and how to take advantage of state and props. Hopefully, this simple exercise will also help you to find new, interesting, open source JavaScript projects that you might want to contribute to!

Tại điểm này, hẳn bạn đã khá rõ React hoạt động ra sao, cách tổ hợp các component với nhau, và cách tận dụng state và props. Hy vọng bài tập đơn giản này cũng sẽ giúp bạn tìm thấy những dự án JavaScript mã nguồn mở mới, thú vị mà bạn có thể muốn đóng góp!

We've covered just enough ground for us to be able to build our first Universal React application. But if you want to be proficient with React, we recommend that you read the official React documentation (nodejsdp.link/react-docs) for a more exhaustive overview of the library.

Chúng ta vừa đi qua đủ nền tảng để có thể xây dựng ứng dụng Universal React đầu tiên của mình. Nhưng nếu bạn muốn thành thạo với React, chúng tôi khuyên bạn đọc tài liệu chính thức của React (nodejsdp.link/react-docs) để có cái nhìn đầy đủ hơn về thư viện.

We are finally ready to take what we learned about webpack and React to create a simple, yet complete, universal JavaScript application.

Cuối cùng chúng ta đã sẵn sàng lấy những gì đã học về webpack và React để tạo một ứng dụng Universal JavaScript đơn giản nhưng hoàn chỉnh.

Creating a Universal JavaScript app — Tạo một ứng dụng Universal JavaScript

Now that we’ve covered the basics, let’s start to build a more complete Universal JavaScript application. We are going to build a simple “book library” application where we can list different authors and see their biography and some of their masterpieces. Although this is going to be a very simple application, it will allow us to cover more advanced topics such as universal routing, universal rendering, and universal data fetching. The idea is that you can later use this application as a scaffold for a real project and build on top of it your next universal JavaScript application.

Giờ khi đã đề cập đến những điều cơ bản, hãy bắt đầu xây dựng một ứng dụng Universal JavaScript hoàn chỉnh hơn. Chúng ta sẽ xây dựng một ứng dụng “thư viện sách” (book library) đơn giản nơi chúng ta có thể liệt kê các tác giả khác nhau và xem tiểu sử cùng một số kiệt tác của họ. Dù đây sẽ là một ứng dụng rất đơn giản, nó sẽ cho phép chúng ta đề cập đến những chủ đề tân tiến hơn như định tuyến universal, kết xuất universal, và lấy dữ liệu universal. Ý tưởng là sau này bạn có thể dùng ứng dụng này làm khung sườn (scaffold) cho một dự án thực và xây dựng trên đó ứng dụng Universal JavaScript tiếp theo của mình.

In this experiment, we are going to use the following technologies:

Trong thử nghiệm này, chúng ta sẽ dùng các công nghệ sau:

- React (nodejsdp.link/react), which we just introduced
- React Router (nodejsdp.link/react-router), a companion routing layer for React
- Fastify (nodejsdp.link/fastify), a fast and ergonomic framework to build web servers in Node.js
- Webpack as the module bundler
 - React (nodejsdp.link/react), thứ chúng ta vừa giới thiệu
 - React Router (nodejsdp.link/react-router), một tầng định tuyến đồng hành cho React
 - Fastify (nodejsdp.link/fastify), một framework nhanh và tiện dụng để xây dựng máy chủ web trong Node.js
 - Webpack làm module bundler

For practical reasons, we selected a very specific set of technologies for this exercise, but we will try to focus as much as possible on the design principles and patterns rather than the technologies themselves. As you learn these patterns, you should be able to use the acquired knowledge with any other combination of technologies and achieve similar results.

Vì lý do thực tiễn, chúng ta đã chọn một bộ công nghệ rất cụ thể cho bài tập này, nhưng chúng ta sẽ cố gắng tập trung nhiều nhất có thể vào các nguyên tắc và mẫu thiết kế thay vì bản thân các công nghệ. Khi học các mẫu này, bạn sẽ có thể dùng kiến thức thu được với bất kỳ tổ hợp công nghệ nào khác và đạt được kết quả tương tự.

In order to keep things simple, we will be using webpack only to process the frontend code and we will leave the backend code unchanged, leveraging the native Node.js support for ESM.

Để giữ mọi thứ đơn giản, chúng ta sẽ chỉ dùng webpack để xử lý mã frontend và sẽ giữ nguyên mã backend, tận dụng hỗ trợ ESM native của Node.js.

At the time of writing, there are some subtle discrepancies between how webpack interprets the semantics of ESM imports as opposed to how Node.js does it, especially when importing modules written using the CommonJS syntax. For this reason, we recommend running the examples in the rest of this chapter using `esm` (nodejsdp.link/esm), a Node.js library that will preprocess ESM imports in a way that minimizes those differences. Once you have installed the `esm` module in your project, you can run a script with `esm` as follows:

Tại thời điểm viết sách, có một số khác biệt tinh tế giữa cách webpack diễn giải ngữ nghĩa của các import ESM so với cách Node.js làm, đặc biệt khi import các module được viết bằng cú pháp CommonJS. Vì lý do này, chúng tôi khuyên chạy các ví dụ ở phần còn lại của chương này bằng `esm` (nodejsdp.link/esm), một thư viện Node.js sẽ tiền xử lý các import ESM theo cách giảm thiểu những khác biệt đó. Một khi bạn đã cài module `esm` trong dự án của mình, bạn có thể chạy một script với `esm` như sau:

```
node -r esm script.js
```

Frontend-only app — Ứng dụng chỉ frontend (Frontend-only app)

In this section, we are going to focus on building our app on the frontend only, using webpack as a development web server. In the next sections, we will expand and update this basic app to convert it to a full Universal JavaScript application.

Trong phần này, chúng ta sẽ tập trung xây dựng ứng dụng của mình chỉ trên frontend, dùng webpack làm máy chủ web phát triển (development web server). Trong các phần tiếp theo, chúng ta sẽ mở rộng và cập nhật ứng dụng cơ bản này để chuyển đổi nó thành một ứng dụng Universal JavaScript đầy đủ.

This time, we will be using a custom webpack configuration, so let's start by creating a new folder and copying the `package.json` and `webpack.config.js` files from the code repository provided with this book (nodejsdp.link/frontend-only-app), then install all the necessary dependencies with:

Lần này, chúng ta sẽ dùng một cấu hình webpack tùy chỉnh, nên hãy bắt đầu bằng việc tạo một thư mục mới và sao chép các tập tin `package.json` và `webpack.config.js` từ kho mã đi kèm cuốn sách này (nodejsdp.link/frontend-only-app), rồi cài đặt tất cả các phụ thuộc cần thiết với:

```
npm install
```

The data we will be using is stored in a JavaScript file (as a simple substitute for a database), so make sure you copy the file `data/authors.js` into your project as well. This file contains some sample data in the following format:

Dữ liệu chúng ta sẽ dùng được lưu trong một tập tin JavaScript (như một sự thay thế đơn giản cho một cơ sở dữ liệu), nên hãy đảm bảo bạn cũng sao chép tập tin `data/authors.js` vào dự án của mình. Tập tin này chứa một số dữ liệu mẫu theo định dạng sau:

```
export const authors = [
  {
    id: 'author\'s unique id',
    name: 'author\'s name',
    bio: 'author\'s biography',
  }
]
```

```

books: [ // author's books
  {
    id: 'book unique id',
    title: 'book title',
    year: 1914 // book publishing year
  },
  // ... more books
]
},
// ... more authors
]

```

Of course, feel free to change the data in this file if you want to add your favorite authors and books!

Tất nhiên, hãy thoải mái thay đổi dữ liệu trong tập tin này nếu bạn muốn thêm các tác giả và sách yêu thích của mình!

Now that we have all the configuration in place, let's quickly discuss what we want our application to look like.

Giờ khi đã có tất cả cấu hình tại chỗ, hãy nhanh chóng bàn về việc chúng ta muốn ứng dụng của mình trông như thế nào.

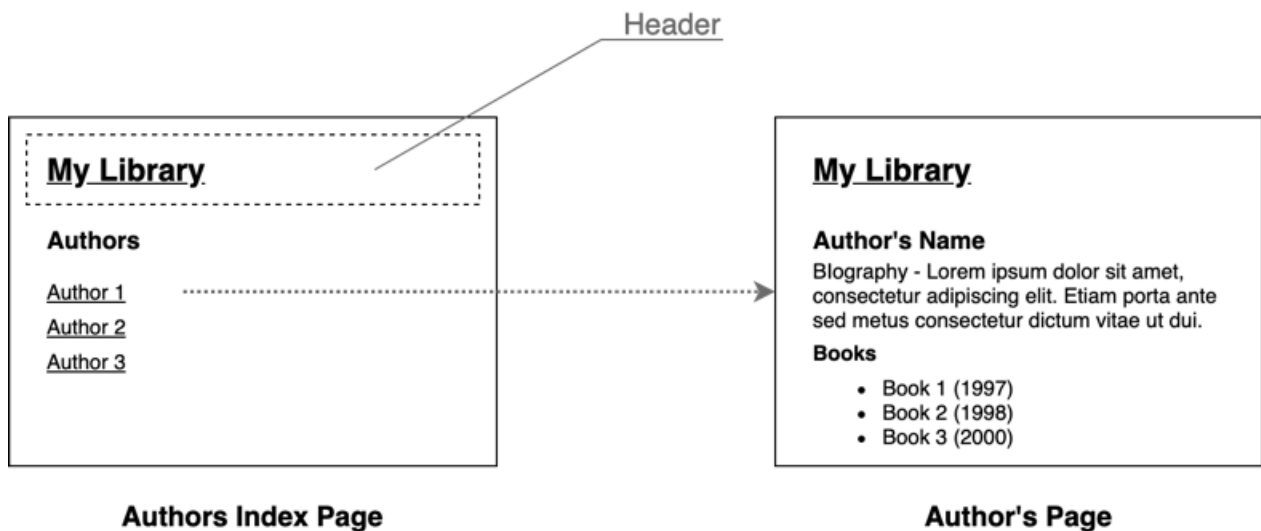


Figure 10.3: Application mockup

Hình 10.3: Bản phác thảo (mockup) ứng dụng

Figure 10.3 shows that our application will have two different types of page: an index page, where we list all the authors available in our data store, and then a page to visualize the details of a given author, where we will see their biography and some of their books.

Hình 10.3 cho thấy ứng dụng của chúng ta sẽ có hai loại trang khác nhau: một trang index, nơi chúng ta liệt kê tất cả các tác giả có sẵn trong kho dữ liệu, và rồi một trang để hiển thị chi tiết của một tác giả cho trước, nơi chúng ta sẽ thấy tiểu sử và một số sách của họ.

These two types of page will only have a header in common. This will allow us to go back to the index page at any time.

Hai loại trang này sẽ chỉ có chung một header. Điều này sẽ cho phép chúng ta quay lại trang index bất cứ lúc nào.

We will be exposing the index page at the root path of our server (/), while we will be using the path /author/:authorId for the author's page.

Chúng ta sẽ phơi bày trang index ở đường dẫn gốc của máy chủ (/), trong khi sẽ dùng đường dẫn /author/:authorId cho trang của tác giả.

Finally, we will also have a 404 page.

Cuối cùng, chúng ta cũng sẽ có một trang 404.

In terms of file structure, we will organize our project as follows:

Về mặt cấu trúc tập tin, chúng ta sẽ tổ chức dự án của mình như sau:

```
src
├── data
│   └── authors.js - data file
├── frontend
│   ├── App.js - application component
│   ├── components
│   │   ├── Header.js - header component
│   │   └── pages
│   │       ├── Author.js - author page
│   │       ├── AuthorsIndex.js - index page
│   │       └── FourOhFour.js - 404 page
└── index.js - project entry point
```

Let's start by writing the index.js module, which will serve as the entry point for loading our frontend application and attaching it to the DOM:

Hãy bắt đầu bằng việc viết module index.js, vốn sẽ đóng vai trò điểm vào để tải ứng dụng frontend của chúng ta và gắn nó vào DOM:

```
import react from 'react'
import ReactDOM from 'react-dom'
import htm from 'htm'
import { BrowserRouter } from 'react-router-dom'
import { App } from './App.js'
const html = htm.bind(react.createElement)
ReactDOM.render(
  html`<${BrowserRouter}><${App}/></>`,
  document.getElementById('root')
)
```

This code is quite simple as we are mainly importing the App component and attaching it to the DOM in an element with the ID equal to root. The only detail that stands out is that we are wrapping the application into a BrowserRouter component. This component comes from the react-router-dom library and it provides our app with client-side routing capabilities. Some of the components we will be writing next will showcase how to fully take advantage of these routing capabilities and how to connect different pages together using links. Later on, we will revisit this routing configuration to make it available on the server side as well.

Mã này khá đơn giản vì chúng ta chủ yếu đang import component App và gắn nó vào DOM trong một phần tử có ID bằng root. Chi tiết duy nhất nổi bật là chúng ta đang bọc

ứng dụng vào một component `BrowserRouter`. Component này đến từ thư viện `react-router-dom` và nó cung cấp cho ứng dụng của chúng ta khả năng định tuyến phía client. Một số component chúng ta sẽ viết tiếp theo sẽ trình diễn cách tận dụng tối đa các khả năng định tuyến này và cách kết nối các trang khác nhau với nhau bằng liên kết (link). Về sau, chúng ta sẽ xem lại cấu hình định tuyến này để làm nó có sẵn cả ở phía máy chủ.

Right now, let's focus on the source code for `App.js`:

Ngay bây giờ, hãy tập trung vào mã nguồn cho `App.js`:

```
import react from 'react'
import htm from 'htm'
import { Switch, Route } from 'react-router-dom'
import { AuthorsIndex } from './components/pages/AuthorsIndex.js'
import { Author } from './components/pages/Author.js'
import { FourOhFour } from './components/pages/FourOhFour.js'
const html = htm.bind(react.createElement)
export class App extends react.Component {
  render () {
    return html`
      <${Switch}>
        <${Route}
          path="/"
          exact=${true}
          component=${AuthorsIndex}
        />
        <${Route}
          path="/author/:authorId"
          component=${Author}
        />
        <${Route}
          path="*"
          component=${FourOhFour}
        />
      </>
    `
  }
}
```

As you can tell from this code, the `App` component is responsible for loading all the page components and configuring the routing for them.

Như bạn có thể nhận ra từ mã này, component `App` chịu trách nhiệm tải tất cả các component trang và cấu hình định tuyến cho chúng.

Here, we are using the `Switch` component from `react-router-dom`. This component allows us to define `Route` components. Every `Route` component needs to have a `path` and a `component` prop associated with it. At render time, `Switch` will check the current URL against the paths defined by the routes, and it will render the component associated to the first `Route` component that matches.

Ở đây, chúng ta đang dùng component `Switch` từ `react-router-dom`. Component này cho phép chúng ta định nghĩa các component `Route`. Mỗi component `Route` cần có một prop `path` và một prop `component` gắn với nó. Tại thời điểm kết xuất, `Switch` sẽ kiểm tra URL hiện tại với các đường dẫn được định nghĩa bởi các route, và nó sẽ kết xuất component

gắn với component Route đầu tiên khớp.

As in a JavaScript `switch` statement, where the order of case statements is important, here, the order of the Route components is important too. Our last route is a catch-all route, which will always match if none of the previous routes matches..

Như trong một câu lệnh `switch` của JavaScript, nơi thứ tự các câu lệnh `case` quan trọng, ở đây, thứ tự các component Route cũng quan trọng. Route cuối cùng của chúng ta là một route bắt-tất-cả (catch-all), vốn sẽ luôn khớp nếu không route nào trước đó khớp.

Note also that we are setting the prop `exact` for the first Route. This is needed because `react-router-dom` will match based on prefixes, so a plain `/` will match any URL. By specifying `exact: true`, we are telling the router to only match this path if it is exactly `/` (and not if it just starts with `/`).

Cũng lưu ý rằng chúng ta đang đặt prop `exact` cho Route đầu tiên. Điều này cần thiết vì `react-router-dom` sẽ khớp dựa trên tiền tố (prefix), nên một `/` thuần sẽ khớp bất kỳ URL nào. Bằng việc chỉ định `exact: true`, chúng ta đang bảo router chỉ khớp đường dẫn này nếu nó đúng là `/` (và không nếu nó chỉ bắt đầu bằng `/`).

Let's now have a quick look at our Header component:

Giờ hãy xem nhanh component Header của chúng ta:

```
import react from 'react'
import htm from 'htm'
import { Link } from 'react-router-dom'
const html = htm.bind(react.createElement)
export class Header extends react.Component {
  render () {
    return html`<header>
      <h1>
        <${Link} to="/">My library</>
      </h1>
    </header>`
  }
}
```

This is a very simple component that just renders an `h1` title containing “My library.” The only detail worth discussing here is that the title is wrapped by a `Link` component from the `react-router-dom` library. This component is responsible for rendering a clickable link that can interact with the application router to switch to a new route dynamically, without refreshing the entire page.

Đây là một component rất đơn giản chỉ kết xuất một tiêu đề `h1` chứa “My library.” Chi tiết duy nhất đáng bàn ở đây là tiêu đề được bọc bởi một component `Link` từ thư viện `react-router-dom`. Component này chịu trách nhiệm kết xuất một liên kết có thể click được, có thể tương tác với router của ứng dụng để chuyển sang một route mới một cách động, mà không cần làm mới toàn trang.

Now, we have to write, one by one, our page components. Let's start with the `AuthorsIndex` component:

Bây giờ, chúng ta phải viết, lần lượt từng cái một, các component trang của mình. Hãy bắt đầu với component `AuthorsIndex`:

```
import react from 'react'
import htm from 'htm'
```

```

import { Link } from 'react-router-dom'
import { Header } from '../Header.js'
import { authors } from '../../../data/authors.js'
const html = htm.bind(react.createElement)
export class AuthorsIndex extends react.Component {
  render () {
    return html`<div>
      <${Header}/>
      <div>${authors.map((author) =>
        html`<div key=${author.id}>
          <p>
            <${Link} to="${`/author/${author.id}`}">
              ${author.name}
            </>
          </p>
        </div>`)}
      </div>
    </div>`
  }
}

```

Yet another very simple component. Here, we are rendering some markup dynamically based on the list of authors available in our data file. Note that we are using, once again, the Link component from react-router-dom to create dynamic links to the author page.

Lại một component rất đơn giản nữa. Ở đây, chúng ta đang kết xuất một số markup một cách động dựa trên danh sách các tác giả có sẵn trong tập tin dữ liệu của chúng ta. Lưu ý rằng chúng ta đang dùng, một lần nữa, component Link từ react-router-dom để tạo các liên kết động tới trang tác giả.

Now, let's have a look at the Author component code:

Giờ, hãy xem mã của component Author:

```

import react from 'react'
import htm from 'htm'
import { FourOhFour } from '../FourOhFour.js'
import { Header } from '../Header.js'
import { authors } from '../../../data/authors.js'
const html = htm.bind(react.createElement)
export class Author extends react.Component {
  render () {
    const author = authors.find(
      author => author.id === this.props.match.params.authorId
    )
    if (!author) {
      return html`<${FourOhFour} error="Author not found"/>`
    }
    return html`<div>
      <${Header}/>
      <h2>${author.name}</h2>
      <p>${author.bio}</p>
      <h3>Books</h3>
    </div>`
  }
}

```

```

    <ul>
      ${author.books.map((book) =>
        html`<li key=${book.id}>${book.title} (${book.year})</li>`
      )}
    </ul>
  </div>`
}
}

```

This component has a little bit of logic in it. In the `render()` method, we filter the authors dataset to find the current author. Notice that we are using `props.match.params.authorId` to get the current author ID. The `match` prop will be passed to the component by the router at render time and the nested `params` object will be populated if the current path has dynamic parameters. It is common practice to memoize (nodejsdp.link/memoization) the result of any complex calculation performed in the `render()` method. This prevents the complex calculation from running again in case its inputs haven't changed since the last render. In our example, a possible target for this type of optimization is the call to `authors.find()`. We leave this to you as an exercise. If you want to know more about this technique take a look at nodejsdp.link/react-memoization.

Component này có một chút logic trong nó. Trong phương thức `render()`, chúng ta lọc tập dữ liệu `authors` để tìm tác giả hiện tại. Hãy lưu ý rằng chúng ta đang dùng `props.match.params.authorId` để lấy ID tác giả hiện tại. Prop `match` sẽ được router truyền vào component tại thời điểm kết xuất và đối tượng `params` lồng bên trong sẽ được điền dữ liệu nếu đường dẫn hiện tại có các tham số động. Một thực hành phổ biến là ghi nhớ (memoize) (nodejsdp.link/memoization) kết quả của bất kỳ tính toán phức tạp nào được thực hiện trong phương thức `render()`. Điều này ngăn tính toán phức tạp chạy lại trong trường hợp đầu vào của nó không thay đổi kể từ lần kết xuất trước. Trong ví dụ của chúng ta, một mục tiêu khả dĩ cho loại tối ưu này là lời gọi `authors.find()`. Chúng ta để điều này cho bạn như một bài tập. Nếu bạn muốn biết thêm về kỹ thuật này hãy xem nodejsdp.link/react-memoization.

There's a chance that we are receiving an ID that doesn't match any author in our dataset, so in this case, `author` will be `undefined`. This is clearly a 404, so instead of rendering the author data, we delegate the render logic to the `FourOhFour` component, which is responsible for rendering the 404 error page.

Có khả năng chúng ta đang nhận một ID không khớp tác giả nào trong tập dữ liệu của mình, nên trong trường hợp này, `author` sẽ là `undefined`. Đây rõ ràng là một 404, nên thay vì kết xuất dữ liệu tác giả, chúng ta uỷ thác logic kết xuất cho component `FourOhFour`, vốn chịu trách nhiệm kết xuất trang lỗi 404.

Finally, let's see the source code for the `FourOhFour` component:

Cuối cùng, hãy xem mã nguồn cho component `FourOhFour`:

```

import react from 'react'
import htm from 'htm'
import { Link } from 'react-router-dom'
import { Header } from '../Header.js'
const html = htm.bind(react.createElement)
export class FourOhFour extends react.Component {
  render () {
    return html`<div>
      <${Header}/>
    </div>`
  }
}

```

```

    <div>
      <h2>404</h2>
      <h3>${this.props.error} || 'Page not found'</h3>
      <${Link} to="/">Go back to the home page</>
    </div>
  </div>`
}
}

```

This component is responsible for rendering the 404 page. Note that we made the error message configurable through the `error` prop and also that we are using a `Link` from the `react-router-dom` library to allow the user to travel back to the home page when landing on this error page. This was quite a lot of code, but we are finally ready to run our frontend-only React application: just type `npm start` in your console and you should see the application running in your browser. Pretty barebones, but if we did everything correctly, it should work as expected and allow us to see our favorite authors and their masterpieces.

Component này chịu trách nhiệm kết xuất trang 404. Lưu ý rằng chúng ta đã làm cho thông báo lỗi có thể cấu hình thông qua prop `error` và cũng rằng chúng ta đang dùng một `Link` từ thư viện `react-router-dom` để cho phép người dùng quay lại trang chủ khi đáp xuống trang lỗi này. Đây là khá nhiều mã, nhưng cuối cùng chúng ta đã sẵn sàng chạy ứng dụng React chỉ-frontend của mình: chỉ cần gõ `npm start` trong console và bạn sẽ thấy ứng dụng đang chạy trong trình duyệt. Khá trơ trụi, nhưng nếu chúng ta làm đúng mọi thứ, nó sẽ hoạt động như mong đợi và cho phép chúng ta xem các tác giả yêu thích và các kiệt tác của họ.

It is worth using the app with the browser developer tools open so that we can verify that our dynamic routing is working correctly, that is, once the first page is loaded, transitions to other pages happen without any page refresh.

Đáng dùng ứng dụng với công cụ dành cho nhà phát triển (developer tools) của trình duyệt được mở để chúng ta có thể xác minh rằng định tuyến động của mình đang hoạt động đúng, nghĩa là, một khi trang đầu tiên được tải, các chuyển tiếp sang trang khác xảy ra mà không cần làm mới trang nào.

For a better understanding of what happens when you interact with a React application, you can install and use the React Developer Tools browser extension on Chrome (nodejsdp.link/react-dev-tools-chrome) or Firefox (nodejsdp.link/react-dev-tools-firefox).

Để hiểu rõ hơn điều gì xảy ra khi bạn tương tác với một ứng dụng React, bạn có thể cài và dùng tiện ích mở rộng trình duyệt React Developer Tools trên Chrome (nodejsdp.link/react-dev-tools-chrome) hoặc Firefox (nodejsdp.link/react-dev-tools-firefox).

Server-side rendering — Kết xuất phía máy chủ (Server-side rendering)

Our application works and this is great news. However, the app is running only on the client side, which means that if we try to `curl` one of the pages, we will see something like this:

Ứng dụng của chúng ta hoạt động và đây là tin tuyệt vời. Tuy nhiên, ứng dụng chỉ đang chạy ở phía client, nghĩa là nếu chúng ta thử `curl` một trong các trang, chúng ta sẽ thấy thứ gì đó như sau:

```
<!DOCTYPE html>
```



```

<html>
  <head>
    <meta charset="UTF-8">
    <title>My library</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="text/javascript" src="/main.js"></script></body>
</html>

```

No content whatsoever! There's only an empty container (the root div), which is where our application is mounted at runtime.

Hoàn toàn không có nội dung gì! Chỉ có một vùng chứa rỗng (root div), là nơi ứng dụng của chúng ta được mount tại runtime.

In this section, we will modify our application to be able to render the content also from the server. Let's start by adding fastify and esm to our project:

Trong phần này, chúng ta sẽ chỉnh sửa ứng dụng để có thể kết xuất nội dung cả từ máy chủ. Hãy bắt đầu bằng việc thêm fastify và esm vào dự án của chúng ta:

```
npm install --save fastify fastify-static esm
```

Now, we can create our server application in `src/server.js`:

Bây giờ, chúng ta có thể tạo ứng dụng máy chủ của mình trong `src/server.js`:

```

import { resolve, dirname } from 'path'
import { fileURLToPath } from 'url'
import react from 'react'
import reactServer from 'react-dom/server.js'
import htm from 'htm'
import fastify from 'fastify'
import fastifyStatic from 'fastify-static'
import { StaticRouter } from 'react-router-dom'
import { App } from './frontend/App.js'
const __dirname = dirname(fileURLToPath(import.meta.url))
const html = htm.bind(react.createElement)
// (1)
const template = ({ content }) => `<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>My library</title>
  </head>
  <body>
    <div id="root">${content}</div>
    <script type="text/javascript" src="/public/main.js"></script>
  </body>
</html>`
const server = fastify({ logger: true }) // (2)
server.register(fastifyStatic, { // (3)
  root: resolve(__dirname, '..', 'public'),

```

```

    prefix: '/public/'
  })
  server.get('*', async (req, reply) => {                                // (4)
    const location = req.raw.originalUrl                                // (5)

    const serverApp = html`
      <${StaticRouter} location=${location}>
      <${App}/>
    </>
    ,

    const content = reactServer.renderToString(serverApp)              // (6)
    const responseHtml = template({ content })
    reply.code(200).type('text/html').send(responseHtml)
  })
  const port = Number.parseInt(process.env.PORT) || 3000              // (7)
  const address = process.env.ADDRESS || '127.0.0.1'
  server.listen(port, address, function (err) {
    if (err) {
      console.error(err)
      process.exit(1)
    }
  })
})

```

There's a lot of code here, so let's discuss step by step the main concepts introduced here:

Có rất nhiều mã ở đây, nên hãy bàn từng bước những khái niệm chính được giới thiệu ở đây:

1. Since we are not going to use the webpack dev server, we need to return the full HTML code of the page from our server. Here, we are defining the HTML template for all our pages using a function and a template literal. We will be passing the result of our server-rendered React application as content to this template to get the final HTML to return to the client.
2. Here, we create a Fastify server instance and enable logging.
3. As you might have noticed from our template code, our web application will load the script `/public/main.js`. This file is the frontend bundle that is generated by webpack. Here, we are letting the Fastify server instance serve all static assets from the public folder using the `fastify-static` plugin.
4. In this line, we define a catch-all route for every GET request to the server. The reason why we are doing a catch-all route is because the actual routing logic is already contained in the React application. When we render the React application, it will display the correct page component based on the current URL.
5. On the server side, we have to use an instance of `StaticRouter` from `react-router-dom` and wrap our application component with it. `StaticRouter` is a version of React Router that can be used for **server-side rendering**. This router, rather than taking the current URL from the browser window, allows us to pass the current URL directly from the server through the `location` prop.
6. Here, we can finally generate the HTML code for our `serverApp` component by using React's `renderToString()` function. The generated HTML is the same as the one generated by the client-side application on a given URL. In the next few lines, we wrap this code with our page layout using the `template()` function and finally, we send the result to the client.
7. In the last few lines of code, we tell our Fastify server instance to listen on a given address and port defaulting to `localhost:3000`.

1. Vì chúng ta sẽ không dùng webpack dev server, chúng ta cần trả về toàn bộ mã HTML của trang từ máy chủ của mình. Ở đây, chúng ta đang định nghĩa template HTML cho tất cả các trang bằng một hàm và một template literal. Chúng ta sẽ truyền kết quả của ứng dụng React được kết xuất phía máy chủ làm content cho template này để có được HTML cuối cùng trả về cho client.
2. Ở đây, chúng ta tạo một instance máy chủ Fastify và bật ghi log (logging).
3. Như bạn có thể đã nhận thấy từ mã template của chúng ta, ứng dụng web của chúng ta sẽ tải script `/public/main.js`. Tập tin này là bundle frontend được webpack tạo ra. Ở đây, chúng ta đang để instance máy chủ Fastify phục vụ tất cả các asset tĩnh từ thư mục public bằng plugin `fastify-static`.
4. Ở dòng này, chúng ta định nghĩa một route bắt-tất-cả cho mọi yêu cầu GET tới máy chủ. Lý do chúng ta làm một route bắt-tất-cả là vì logic định tuyến thực sự đã được chứa trong ứng dụng React. Khi chúng ta kết xuất ứng dụng React, nó sẽ hiển thị component trang đúng dựa trên URL hiện tại.
5. Ở phía máy chủ, chúng ta phải dùng một instance của `StaticRouter` từ `react-router-dom` và bọc component ứng dụng của chúng ta bằng nó. `StaticRouter` là một phiên bản của `React Router` có thể dùng cho kết xuất phía máy chủ. Router này, thay vì lấy URL hiện tại từ cửa sổ trình duyệt, cho phép chúng ta truyền URL hiện tại trực tiếp từ máy chủ thông qua prop `location`.
6. Ở đây, cuối cùng chúng ta có thể tạo mã HTML cho component `serverApp` của mình bằng cách dùng hàm `renderToString()` của React. HTML được tạo ra giống với HTML được ứng dụng phía client tạo ra trên một URL cho trước. Trong vài dòng tiếp theo, chúng ta bọc mã này bằng layout trang của mình dùng hàm `template()` và cuối cùng, chúng ta gửi kết quả tới client.
7. Trong vài dòng mã cuối cùng, chúng ta bảo instance server Fastify của mình lắng nghe trên một địa chỉ và cổng cho trước, mặc định là `localhost:3000`.

Now, we can run `npm run build` to create the frontend bundle and finally, we can run our server, as follows:

Giờ, chúng ta có thể chạy `npm run build` để tạo bundle frontend và cuối cùng, chúng ta có thể chạy máy chủ của mình, như sau:

```
node -r esm src/server.js
```

Let's open our browser on `http://localhost:3000/` and see if our app is still working as expected. All good, right? Great! Now, let's try to `curl` our home page to see if the server-generated code looks different:

Hãy mở trình duyệt tại `http://localhost:3000/` và xem ứng dụng của chúng ta còn hoạt động như mong đợi không. Mọi thứ ổn cả, phải không? Tuyệt! Giờ, hãy thử `curl` trang chủ của chúng ta để xem mã do máy chủ tạo ra có khác đi không:

```
curl http://localhost:3000/
```

This time, this is what we should see:

Lần này, đây là thứ chúng ta sẽ thấy:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>My library</title>
  </head>
```

```

<body>
  <div id="root"><div><header><h1><a href="/">My library</a></h1></
header><div><h2>Authors</h2><div><div><a href="/author/joyce"><p>James
Joyce</p></a></div><div><a href="/author/h-g-wells"><p>Herbert George
Wells</p></a></div><div><a href="/author/orwell"><p>George Orwell</p></
a></div></div></div></div></div>
  <script type="text/javascript" src="/public/main.js"></script>
</body>
</html>

```

Great! This time, our root container is not empty: we are rendering the list of authors directly from the server. You should also try some author pages and see that it works correctly for those as well. Mission complete! Well, almost... what happens if we try to render a page that does not exist? Let's have a look:

Tuyệt! Lần này, vùng chứa root của chúng ta không rỗng: chúng ta đang kết xuất danh sách tác giả trực tiếp từ máy chủ. Bạn cũng nên thử một số trang tác giả và thấy rằng nó cũng hoạt động đúng cho các trang đó. Nhiệm vụ hoàn thành! Ồ, gần như vậy... điều gì xảy ra nếu chúng ta thử kết xuất một trang không tồn tại? Hãy xem:

```
curl -i http://localhost:3000/blah
```

This will print:

Lệnh này sẽ in ra:

```

HTTP/1.1 200 OK
content-type: text/html
content-length: 367
Date: Sun, 05 Apr 2020 18:38:47 GMT
Connection: keep-alive
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>My library</title>
  </head>
  <body>
    <div id="root"><div><header><h1><a href="/">My library</a></h1></
header><div><h2>404</h2><h3>Page not found</h3><a href="/">Go back to
the home page</a></div></div></div>
    <script type="text/javascript" src="/public/main.js"></script>
  </body>
</html>

```

At first glance, this might seem correct because we are rendering our 404 page, but we are actually returning a 200 status code... not good!

Thoạt nhìn, điều này có vẻ đúng vì chúng ta đang kết xuất trang 404 của mình, nhưng thực ra chúng ta đang trả về một mã trạng thái 200... không tốt!

We can actually fix this with just a little extra effort, so let's do it.

Chúng ta thực ra có thể sửa điều này chỉ với một chút công sức thêm, nên hãy làm thôi.

React `StaticRouter` allows us to pass a generic context prop that can be used to exchange information between the React application and the server. We can leverage this utility to allow our 404 page to inject some information into this shared context so that on the server side, we are aware of whether we should return a 200 or a 404 status code. Let's update the catch-all route on the server side first:

`StaticRouter` của React cho phép chúng ta truyền một prop context tổng quát có thể dùng để trao đổi thông tin giữa ứng dụng React và máy chủ. Chúng ta có thể tận dụng tiện ích này để cho phép trang 404 của mình tiêm một số thông tin vào ngữ cảnh chia sẻ này sao cho ở phía máy chủ, chúng ta biết liệu nên trả về mã trạng thái 200 hay 404. Trước hết hãy cập nhật route bắt-tất-cả ở phía máy chủ:

```
server.get('*', async (req, reply) => {
  const location = req.raw.originalUrl
  const staticContext = {}
  const serverApp = html`
    <${StaticRouter}
      location=${location}
      context=${staticContext}
    >
    <${App} />
  </>
  ,

  const content = reactServer.renderToString(serverApp)
  const responseHtml = template({ content })
  let code = 200
  if (staticContext.statusCode) {
    code = staticContext.statusCode
  }
  reply.code(code).type('text/html').send(responseHtml)
})
```

The changes from the previous version are highlighted in bold. As you can see, we create an empty object called `staticContext` and pass it to the router instance in the context prop. Later on, after the server-side rendering is completed, we check if `staticContext.statusCode` was populated during the rendering process. If it was, it will now contain the status code that we have to return to the client, together with the rendered HTML code.

Các thay đổi so với phiên bản trước được tô đậm. Như bạn có thể thấy, chúng ta tạo một đối tượng rỗng tên là `staticContext` và truyền nó cho instance router trong prop context. Sau đó, sau khi kết xuất phía máy chủ hoàn tất, chúng ta kiểm tra xem `staticContext.statusCode` có được điền dữ liệu trong quá trình kết xuất hay không. Nếu có, giờ nó sẽ chứa mã trạng thái mà chúng ta phải trả về cho client, cùng với mã HTML đã kết xuất.

Let's now change the `FourOhFour` component to actually populate this value. To do this, we just need to update the `render()` function with the following code before we return the elements to render:

Giờ hãy thay đổi component `FourOhFour` để thực sự điền giá trị này. Để làm điều này, chúng ta chỉ cần cập nhật hàm `render()` với mã sau trước khi trả về các phần tử để kết xuất:

```
if (this.props.staticContext) {
  this.props.staticContext.statusCode = 404
}
```

```
}
```

Note that the context prop passed to `StaticRouter` is passed only to direct children of `Route` components using the prop `staticContext`. For this reason, if we rebuild the frontend bundle and relaunch our server, this time, we will see a correct 404 status for `http://localhost:3000/blah`, but it won't work for URLs that match the author page such as `http://localhost:3000/author/blah`. In order to make this work, we also need to propagate `staticContext` from the `Author` component into the `FourOhFour` component. To do this, in the `Author` component's `render()` method, we have to apply the following change:

Lưu ý rằng prop context được truyền cho `StaticRouter` chỉ được truyền tới các con trực tiếp của các component `Route` thông qua prop `staticContext`. Vì lý do này, nếu chúng ta build lại bundle frontend và khởi động lại máy chủ, lần này, chúng ta sẽ thấy một trạng thái 404 đúng cho `http://localhost:3000/blah`, nhưng nó sẽ không hoạt động cho các URL khớp trang tác giả như `http://localhost:3000/author/blah`. Để làm điều này hoạt động, chúng ta cũng cần truyền tiếp `staticContext` từ component `Author` vào component `FourOhFour`. Để làm điều này, trong phương thức `render()` của component `Author`, chúng ta phải áp dụng thay đổi sau:

```
if (!author) {  
  return html`<${FourOhFour}  
    staticContext=${this.props.staticContext}  
    error="Author not found"  
  />`  
}  
// ...
```

Now, the 404 status code will be returned correctly from the server, even on author pages for non-existent authors.

Giờ, mã trạng thái 404 sẽ được trả về đúng đắn từ máy chủ, ngay cả trên các trang tác giả cho những tác giả không tồn tại.

Great—we now have a fully functional React application that uses server-side rendering! But don't celebrate just yet, we still have some work to do...

Tuyệt—giờ chúng ta đã có một ứng dụng React hoạt động đầy đủ dùng kết xuất phía máy chủ! Nhưng đừng ăn mừng vội, chúng ta vẫn còn một số việc phải làm...

Asynchronous data retrieval — Truy xuất dữ liệu bất đồng bộ (Asynchronous data retrieval)

Now, imagine for a second that we are asked to build the website for the Library of Trinity College in Dublin, one of the most famous libraries in the world. It has about 300 years of history and about 7 million books. Ok, now let's imagine we have to allow the users to browse this massive collection of books. Yes, all 7 million of them... a simple data file is not going to be a great idea here!

Giờ, hãy hình dung trong giây lát rằng chúng ta được yêu cầu xây dựng website cho Thư viện Trinity College ở Dublin, một trong những thư viện nổi tiếng nhất thế giới. Nó có khoảng 300 năm lịch sử và khoảng 7 triệu cuốn sách. Ok, giờ hãy hình dung chúng ta phải cho phép người dùng duyệt bộ sưu tập sách khổng lồ này. Vâng, cả 7 triệu cuốn... một tập tin dữ liệu đơn giản sẽ không phải là một ý tưởng hay ở đây!

A better approach would be to have a dedicated API to retrieve the data about the books and use it to dynamically fetch only the minimum amount of data needed to render a given page. More data

will be fetched as the user navigates through the various pages of the website.

Một cách tiếp cận tốt hơn sẽ là có một API chuyên dụng để truy xuất dữ liệu về sách và dùng nó để lấy động chỉ lượng dữ liệu tối thiểu cần thiết để kết xuất một trang cho trước. Nhiều dữ liệu hơn sẽ được lấy khi người dùng điều hướng qua các trang khác nhau của website.

This approach is valid for most web applications, so let's try to apply the same principle to our demo application. We will be using an API with two endpoints:

Cách tiếp cận này hợp lệ cho hầu hết các ứng dụng web, nên hãy thử áp dụng cùng nguyên tắc cho ứng dụng demo của chúng ta. Chúng ta sẽ dùng một API với hai endpoint:

- `/api/authors`, to get the list of authors
- `/api/author/:authorId`, to get the information for a given author
 - `/api/authors`, để lấy danh sách tác giả
 - `/api/author/:authorId`, để lấy thông tin cho một tác giả cho trước

For the sake of this demo application, we will keep things very simple. We only want to demonstrate how our application is going to change as soon as we introduce asynchronous data fetching, so we are not going to bother with using a real database to back our API or with introducing more advanced features like pagination, filtering, or search. Since building such an API server leveraging our existing data file is a rather trivial exercise (one that doesn't add much value in the context of this chapter), we are going to skip the walkthrough of the API implementation. You can get the source code of the API server from the code repository of this book (nodejsdp.link/authors-api-server).

Vì mục đích của ứng dụng demo này, chúng ta sẽ giữ mọi thứ rất đơn giản. Chúng ta chỉ muốn trình diễn cách ứng dụng của chúng ta sẽ thay đổi như thế nào ngay khi chúng ta giới thiệu việc lấy dữ liệu bất đồng bộ, nên chúng ta sẽ không bận tâm dùng một cơ sở dữ liệu thực để hậu thuẫn cho API hay giới thiệu các tính năng tân tiến hơn như phân trang (pagination), lọc, hoặc tìm kiếm. Vì xây dựng một máy chủ API như vậy tận dụng tập tin dữ liệu hiện có của chúng ta là một bài tập khá tầm thường (một bài tập không thêm nhiều giá trị trong ngữ cảnh của chương này), chúng ta sẽ bỏ qua phần hướng dẫn triển khai API. Bạn có thể lấy mã nguồn của máy chủ API từ kho mã của cuốn sách này (nodejsdp.link/authors-api-server).

This simple API server runs independently from our backend server, so it uses another port (or potentially even on another domain). In order to allow the browser to make asynchronous HTTP requests to a different port or domain, we need our API server to support cross-origin resource sharing or CORS (nodejsdp.link/cors), a mechanism that allows secure crossorigin requests. Thankfully, enabling CORS with Fastify is as easy as installing the `fastify-cors` (nodejsdp.link/fastify-cors) plugin.

Máy chủ API đơn giản này chạy độc lập với máy chủ backend của chúng ta, nên nó dùng một cổng khác (hoặc có thể thậm chí trên một domain khác). Để cho phép trình duyệt thực hiện các yêu cầu HTTP bất đồng bộ tới một cổng hoặc domain khác, chúng ta cần máy chủ API của mình hỗ trợ cross-origin resource sharing hay CORS (nodejsdp.link/cors), một cơ chế cho phép các yêu cầu cross-origin an toàn. May thay, bật CORS với Fastify dễ dàng như việc cài plugin `fastify-cors` (nodejsdp.link/fastify-cors).

We are also going to need an HTTP client that works seamlessly on both the browser and Node.js. A good option is `superagent` (nodejsdp.link/superagent).

Chúng ta cũng sẽ cần một HTTP client hoạt động trơn tru trên cả trình duyệt và Node.js. Một lựa chọn tốt là `superagent` (nodejsdp.link/superagent).

Let's install the new dependencies then:

Vậy hãy cài các phụ thuộc mới:

```
npm install --save fastify-cors superagent
```

Now we are ready to run our API server:

Giờ chúng ta đã sẵn sàng chạy máy chủ API của mình:

```
node -r esm src/api.js
```

And let's try some requests with curl, for instance:

Và hãy thử một số yêu cầu với curl, chẳng hạn:

```
curl -i http://localhost:3001/api/authors
curl -i http://localhost:3001/api/author/joyce
curl -i http://localhost:3001/api/author/invalid
```

If everything worked as expected, we are now ready to update our React components to use these new API endpoints rather than reading directly from the authors dataset. Let's start by updating the AuthorsIndex component:

Nếu mọi thứ hoạt động như mong đợi, giờ chúng ta đã sẵn sàng cập nhật các component React của mình để dùng các endpoint API mới này thay vì đọc trực tiếp từ tập dữ liệu authors. Hãy bắt đầu bằng việc cập nhật component AuthorsIndex:

```
import react from 'react'
import htm from 'htm'
import { Link } from 'react-router-dom'
import superagent from 'superagent'
import { Header } from '../Header.js'
const html = htm.bind(react.createElement)
export class AuthorsIndex extends react.Component {
  constructor (props) {
    super(props)
    this.state = {
      authors: [],
      loading: true
    }
  }
  async componentDidMount () {
    const { body } = await superagent.get('http://localhost:3001/api/authors')
    this.setState({ loading: false, authors: body })
  }
  render () {
    if (this.state.loading) {
      return html`<${Header}/><div>Loading ...</div>`
    }
    return html`<div>
      <${Header}/>
      <div>${this.state.authors.map((author) =>
        html`<div key=${author.id}>
          <p>
```

```

        <${Link} to="${`}/author/${author.id}`">
            ${author.name}
        </>
    </p>
</div>`))}
</div>
</div>`
}
}

```

The main changes from the previous version are highlighted in bold. Essentially, we converted our React component into a stateful component. At construction time, we initialized the state to an empty array of authors and we set the loading flag to true. Then, we used the `componentDidMount` lifecycle method to load the authors data using the new API endpoint. Finally, we updated the `render()` method to display a loading message while the data was being loaded asynchronously. Now, we have to update our Author component:

Các thay đổi chính so với phiên bản trước được tô đậm. Về cơ bản, chúng ta đã chuyển đổi component React của mình thành một component có trạng thái. Tại thời điểm khởi tạo, chúng ta khởi tạo trạng thái thành một mảng tác giả rỗng và đặt cờ loading thành true. Sau đó, chúng ta dùng phương thức vòng đời `componentDidMount` để tải dữ liệu tác giả bằng endpoint API mới. Cuối cùng, chúng ta cập nhật phương thức `render()` để hiển thị thông báo loading trong khi dữ liệu đang được tải bất đồng bộ. Giờ, chúng ta phải cập nhật component Author của mình:

```

import react from 'react'
import htm from 'htm'
import superagent from 'superagent'
import { FourOhFour } from './FourOhFour.js'
import { Header } from './Header.js'
const html = htm.bind(react.createElement)
export class Author extends react.Component {
  constructor (props) {
    super(props)
    this.state = {
      author: null,
      loading: true
    }
  }
  async loadData () {
    let author = null
    this.setState({ loading: false, author })
    try {
      const { body } = await superagent.get(
        `http://localhost:3001/api/author/${
          this.props.match.params.authorId
        }`)
      author = body
    } catch (e) {}
    this.setState({ loading: false, author })
  }
  componentDidMount () {

```

```

    this.loadData()
  }
  componentDidUpdate (prevProps) {
    if (prevProps.match.params.authorId !==
        this.props.match.params.authorId) {
      this.loadData()
    }
  }
  render () {
    if (this.state.loading) {
      return html`<${Header}/><div>Loading ...</div>`
    }
    if (!this.state.author) {
      return html`<${FourOhFour}
        staticContext=${this.props.staticContext}
        error="Author not found"
      />`
    }
    return html`<div>
      <${Header}/>
      <h2>${this.state.author.name}</h2>
      <p>${this.state.author.bio}</p>
      <h3>Books</h3>
      <ul>
        ${this.state.author.books.map((book) =>
          html`<li key=${book.id}>
            ${book.title} (${book.year})
          </li>`
        )}
      </ul>
    </div>`
  }
}

```

The changes here are quite similar to the ones we applied to the previous component. In this component, we also generalized the data loading operation into the `loadData()` method. We did this because this component implements not just the `componentDidMount()` but also the `componentDidUpdate()` lifecycle method. This is necessary because if we end up passing new props to the same component instance, we want the component to update correctly. This will happen, for instance, if we have a link in the author page that points to another author page, something that could happen if we implement a “related authors” feature in our application.

Các thay đổi ở đây khá tương tự các thay đổi chúng ta đã áp dụng cho component trước. Trong component này, chúng ta cũng đã tổng quát hoá thao tác tải dữ liệu vào phương thức `loadData()`. Chúng ta làm điều này vì component này triển khai không chỉ phương thức vòng đời `componentDidMount()` mà còn cả `componentDidUpdate()`. Điều này cần thiết vì nếu chúng ta cuối cùng truyền các props mới vào cùng một instance component, chúng ta muốn component cập nhật đúng đắn. Điều này sẽ xảy ra, chẳng hạn, nếu chúng ta có một liên kết trong trang tác giả trở tới một trang tác giả khác, điều có thể xảy ra nếu chúng ta triển khai một tính năng “tác giả liên quan” trong ứng dụng của mình.

At this point, we are ready to try this new version of the code. Let’s regenerate the frontend bundle

with `npm run build` and start both our backend server and our API server, then point our browser to `http://localhost:3000/`. If you navigate around the various pages, everything should work as expected. You might also notice that page content gets loaded interactively as you navigate through the pages.

Tại điểm này, chúng ta đã sẵn sàng thử phiên bản mới này của mã. Hãy tạo lại bundle frontend với `npm run build` và khởi động cả máy chủ backend lẫn máy chủ API của chúng ta, rồi trở trình duyệt tới `http://localhost:3000/`. Nếu bạn điều hướng loanh quanh các trang khác nhau, mọi thứ sẽ hoạt động như mong đợi. Bạn cũng có thể nhận thấy nội dung trang được tải một cách tương tác khi bạn điều hướng qua các trang.

But what happens to our server-side rendering? If we try to use `curl` on our home page, we should see the following HTML markup being returned:

Nhưng điều gì xảy ra với kết xuất phía máy chủ của chúng ta? Nếu chúng ta thử dùng `curl` trên trang chủ, chúng ta sẽ thấy markup HTML sau được trả về:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>My library</title>
  </head>
  <body>
    <div id="root"><div><header><h1><a href="/">My library</a></h1></
header><div>Loading ...</div></div></div>
    <script type="text/javascript" src="/public/main.js"></script>
  </body>
</html>
```

Did you notice that there's no content anymore, but just a quite useless "Loading ..." indicator? This is not good. Also, this is not the only problem here. If you try to use `curl` on an invalid author page, you will notice that you will get the same HTML markup with the loading indicator and no content and that the returned status code is 200 rather than 404!

Bạn có nhận thấy rằng không còn nội dung nữa, mà chỉ có một chỉ báo "Loading ..." khá vô dụng? Điều này không tốt. Ngoài ra, đây không phải là vấn đề duy nhất ở đây. Nếu bạn thử dùng `curl` trên một trang tác giả không hợp lệ, bạn sẽ nhận thấy rằng bạn sẽ nhận được cùng markup HTML với chỉ báo loading và không có nội dung, và mã trạng thái được trả về là 200 chứ không phải 404!

We don't see any real content on the server-side rendered markup because the `componentDidMount` lifecycle method is executed only on the browser while it's ignored by React during server-side rendering. Furthermore, server-side rendering is a synchronous operation, so even if we move our loading code somewhere else, we still won't be able to perform any asynchronous data loading while rendering on the server.

Chúng ta không thấy bất kỳ nội dung thực nào trong markup được kết xuất phía máy chủ vì phương thức vòng đời `componentDidMount` chỉ được thực thi trên trình duyệt trong khi nó bị React bỏ qua trong quá trình kết xuất phía máy chủ. Hơn nữa, kết xuất phía máy chủ là một thao tác đồng bộ, nên ngay cả khi chúng ta chuyển mã loading của mình sang nơi khác, chúng ta vẫn không thể thực hiện bất kỳ việc tải dữ liệu bất đồng bộ nào trong khi kết xuất trên máy chủ.

In the next section of this chapter, we will explore a pattern that can help us to achieve full universal rendering and data loading.

Trong phần tiếp theo của chương này, chúng ta sẽ khám phá một mẫu có thể giúp chúng ta đạt được kết xuất và tải dữ liệu universal đầy đủ.

Universal data retrieval — Truy xuất dữ liệu universal (Universal data retrieval)

Server-side rendering is a synchronous operation and this makes it tricky to preload all the necessary data effectively. Being able to avoid the problems we underlined at the end of the previous section is not as straightforward as you might expect.

Kết xuất phía máy chủ là một thao tác đồng bộ và điều này làm cho việc tải trước (preload) tất cả dữ liệu cần thiết một cách hiệu quả trở nên khó khăn. Có khả năng tránh các vấn đề mà chúng ta đã nhấn mạnh ở cuối phần trước không đơn giản như bạn có thể mong đợi.

The root of the problem is that we are keeping our routing logic within the React application, so, on the server, we cannot know which page we are actually going to render before we call `renderToString()`. This is why the server cannot establish whether we need to preload some data for a particular page.

Gốc rễ của vấn đề là chúng ta đang giữ logic định tuyến của mình trong ứng dụng React, nên, trên máy chủ, chúng ta không thể biết chúng ta thực sự sẽ kết xuất trang nào trước khi gọi `renderToString()`. Đây là lý do máy chủ không thể xác định liệu chúng ta có cần tải trước một số dữ liệu cho một trang cụ thể hay không.

Universal data retrieval is still quite a nebulous area in React, and different frameworks or libraries that facilitate React server-side rendering have come up with different solutions to this problem.

Truy xuất dữ liệu universal vẫn còn là một lĩnh vực khá mơ hồ trong React, và các framework hoặc thư viện khác nhau hỗ trợ kết xuất phía máy chủ với React đã đưa ra những giải pháp khác nhau cho vấn đề này.

As of today, the two patterns that we believe are worth discussing are two-pass rendering and async pages. These two techniques have different ways of figuring out which data needs to be preloaded. In both cases, once the data is fully loaded on the server, the generated HTML page will provide an inline script block to inject all the data into the global **scope** (the window object) so that when the application runs on the browser, the same data already loaded on the server won't have to be reloaded from the client.

Tính đến hôm nay, hai mẫu mà chúng tôi tin là đáng bàn là kết xuất hai-lượt (two-pass rendering) và các trang bất đồng bộ (async pages). Hai kỹ thuật này có những cách khác nhau để tìm ra dữ liệu nào cần được tải trước. Trong cả hai trường hợp, một khi dữ liệu được tải đầy đủ trên máy chủ, trang HTML được tạo ra sẽ cung cấp một khối script inline để tiêm tất cả dữ liệu vào phạm vi toàn cục (đối tượng window) sao cho khi ứng dụng chạy trên trình duyệt, cùng dữ liệu đã được tải trên máy chủ sẽ không phải tải lại từ client.

Two-pass rendering — Kết xuất hai-lượt (Two-pass rendering)

The idea of two-pass rendering is to use the React router static context as a vector to exchange information between React and the server. Figure 10.4 shows us how this works:

Ý tưởng của kết xuất hai-lượt là dùng static context của React router làm phương tiện để trao đổi thông tin giữa React và máy chủ. Hình 10.4 cho chúng ta thấy cách điều này hoạt động:

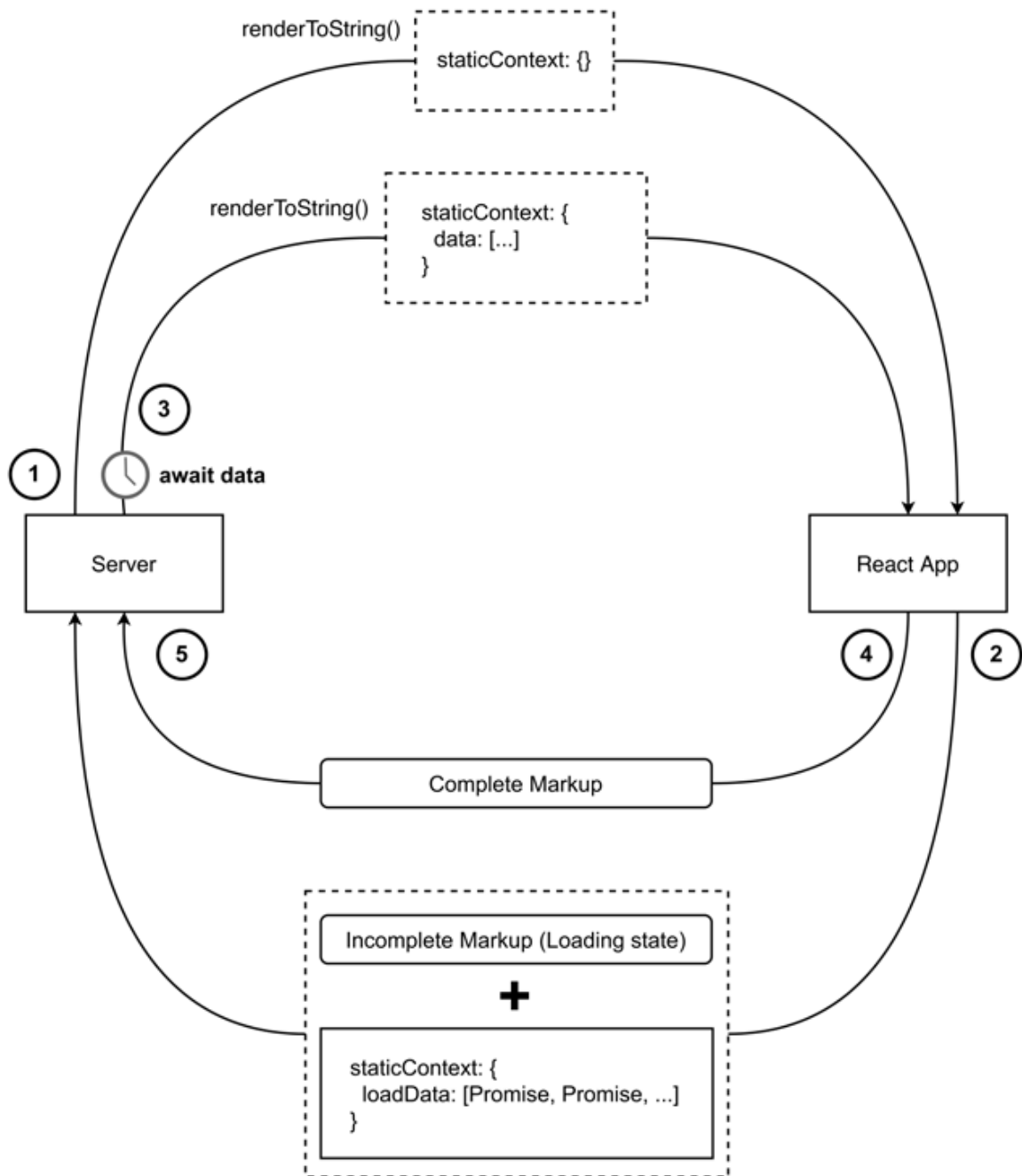


Figure 10.4: Two-pass rendering schematic

Hình 10.4: Sơ đồ kết xuất hai-lượt

The steps of two-pass rendering are as follows:

Các bước của kết xuất hai-lượt như sau:

1. The server calls `renderToString()`, passing the URL received from the client and an empty static context object to the React application.
2. The React application will perform the routing process and select the components that need to be rendered for the given URL. Every component that requires to load some data asynchronously will need to implement some extra logic to allow such data to be preloaded on the server as well. This can be done by attaching a **promise** representing the result of the data loading operation to the router static context. This way, at the end of the rendering process, the server will receive an incomplete markup (representing the current loading state) and the static context will contain a number of promises representing data loading operations.
3. At this point, the server can look at the static context and wait for all the promises to settle to make sure that all the data has been preloaded completely. During this process, the server builds a new static context that contains the results returned by the promises. This new static context is used for a second round of rendering. This is why this technique is called two-pass rendering.
4. Now, the ball is again on the React side of the field. The routing process should pick the same components used during the first rendering pass, since the URL has not changed. This time, the components that need data preloading should see that such data is already available in the static context and they can render the view straight away. This step produces a complete static markup that the server can now use.
5. At this point, the server has the complete markup and it uses it to render the final HTML page. The server can also include all the preloaded data in a `script` tag so that, on the browser, the data will be already available so there won't be any need to load it again while visiting the first page of the application.

1. Máy chủ gọi `renderToString()`, truyền URL nhận từ client và một đối tượng static context rỗng cho ứng dụng React.
2. Ứng dụng React sẽ thực hiện quá trình định tuyến và chọn các component cần được kết xuất cho URL cho trước. Mỗi component cần tải một số dữ liệu bất đồng bộ sẽ cần triển khai một số logic bổ sung để cho phép dữ liệu đó cũng được tải trước trên máy chủ. Điều này có thể làm được bằng cách gắn một promise đại diện cho kết quả của thao tác tải dữ liệu vào static context của router. Theo cách này, ở cuối quá trình kết xuất, máy chủ sẽ nhận một markup chưa hoàn chỉnh (đại diện cho trạng thái loading hiện tại) và static context sẽ chứa một số promise đại diện cho các thao tác tải dữ liệu.
3. Tại điểm này, máy chủ có thể nhìn vào static context và chờ tất cả các promise được giải quyết (settle) để đảm bảo rằng tất cả dữ liệu đã được tải trước hoàn toàn. Trong quá trình này, máy chủ xây dựng một static context mới chứa các kết quả được các promise trả về. Static context mới này được dùng cho một vòng kết xuất thứ hai. Đây là lý do kỹ thuật này được gọi là kết xuất hai-lượt.
4. Giờ, quả bóng lại ở phía sân React. Quá trình định tuyến sẽ chọn cùng các component được dùng trong lượt kết xuất đầu tiên, vì URL không thay đổi. Lần này, các component cần tải trước dữ liệu sẽ thấy rằng dữ liệu đó đã có sẵn trong static context và chúng có thể kết xuất khung nhìn ngay lập tức. Bước này tạo ra một markup tĩnh hoàn chỉnh mà máy chủ giờ có thể dùng.
5. Tại điểm này, máy chủ có markup hoàn chỉnh và nó dùng markup đó để kết xuất trang HTML cuối cùng. Máy chủ cũng có thể đưa tất cả dữ liệu đã tải trước vào một thẻ `script` sao cho, trên trình duyệt, dữ liệu sẽ đã có sẵn nên sẽ không cần tải lại trong khi truy cập trang đầu tiên của ứng dụng.

This technique is very powerful and has some interesting advantages. For instance, it allows you to organize your React components tree in a very flexible way. You can have multiple components requesting asynchronous data, and they can be placed at any level of the components tree.

Kỹ thuật này rất mạnh mẽ và có một số ưu điểm thú vị. Chẳng hạn, nó cho phép bạn tổ chức cây component React của mình theo một cách rất linh hoạt. Bạn có thể có nhiều component yêu cầu dữ liệu bất đồng bộ, và chúng có thể được đặt ở bất kỳ tầng nào của cây component.

In more advanced use cases, you can also have data being loaded over multiple rendering passes. For instance, during the second pass, a new component in the tree might be rendered and this component might also need to load data asynchronously so it can just add new promises to the static context. To support this particular case, the server will have to continue the rendering loop until there are no more promises left in the static context. This particular variation of the two-pass rendering technique is referred to as multi-pass rendering.

Trong các trường hợp dùng nâng cao hơn, bạn cũng có thể có dữ liệu được tải qua nhiều lượt kết xuất. Chẳng hạn, trong lượt thứ hai, một component mới trong cây có thể được kết xuất và component này cũng có thể cần tải dữ liệu bất đồng bộ nên nó có thể chỉ cần thêm các promise mới vào static context. Để hỗ trợ trường hợp đặc biệt này, máy chủ sẽ phải tiếp tục vòng lặp kết xuất cho đến khi không còn promise nào trong static context. Biến thể đặc biệt này của kỹ thuật kết xuất hai-lượt được gọi là kết xuất đa-lượt (multi-pass rendering).

The biggest disadvantage of this technique is that every call to `renderToString()` is not cheap and in real-life applications, this technique might force the server to go through multiple rendering passes, making the whole process very slow. This might lead to severe performance degradation on the entire application, which can dramatically affect the user experience.

Nhược điểm lớn nhất của kỹ thuật này là mỗi lời gọi `renderToString()` đều không rẻ và trong các ứng dụng thực tế, kỹ thuật này có thể buộc máy chủ phải đi qua nhiều lượt kết xuất, làm cho toàn bộ quá trình rất chậm. Điều này có thể dẫn đến sự suy giảm hiệu năng nghiêm trọng trên toàn bộ ứng dụng, có thể ảnh hưởng đột ngột đến trải nghiệm người dùng.

A simpler but potentially more performant alternative will be discussed in the next section.

Một lựa chọn thay thế đơn giản hơn nhưng có khả năng có hiệu năng cao hơn sẽ được bàn trong phần tiếp theo.

Async pages — Các trang bất đồng bộ (Async pages)

The technique we are going to describe here, which we are going to call “async pages,” is based on a more constrained structure of the React application.

Kỹ thuật mà chúng ta sẽ mô tả ở đây, mà chúng ta sẽ gọi là “async pages”, dựa trên một cấu trúc bị ràng buộc chặt hơn của ứng dụng React.

The idea is to structure the top layers of the application components tree in a very specific way. Let’s have a look at a possible structure first, then it will be easier to discuss how this specific approach can help us with asynchronous data loading.

Ý tưởng là cấu trúc các tầng trên cùng của cây component ứng dụng theo một cách rất cụ thể. Hãy xem một cấu trúc khả dĩ trước đã, rồi sẽ dễ hơn để bàn về cách tiếp cận cụ thể này có thể giúp chúng ta với việc tải dữ liệu bất đồng bộ.

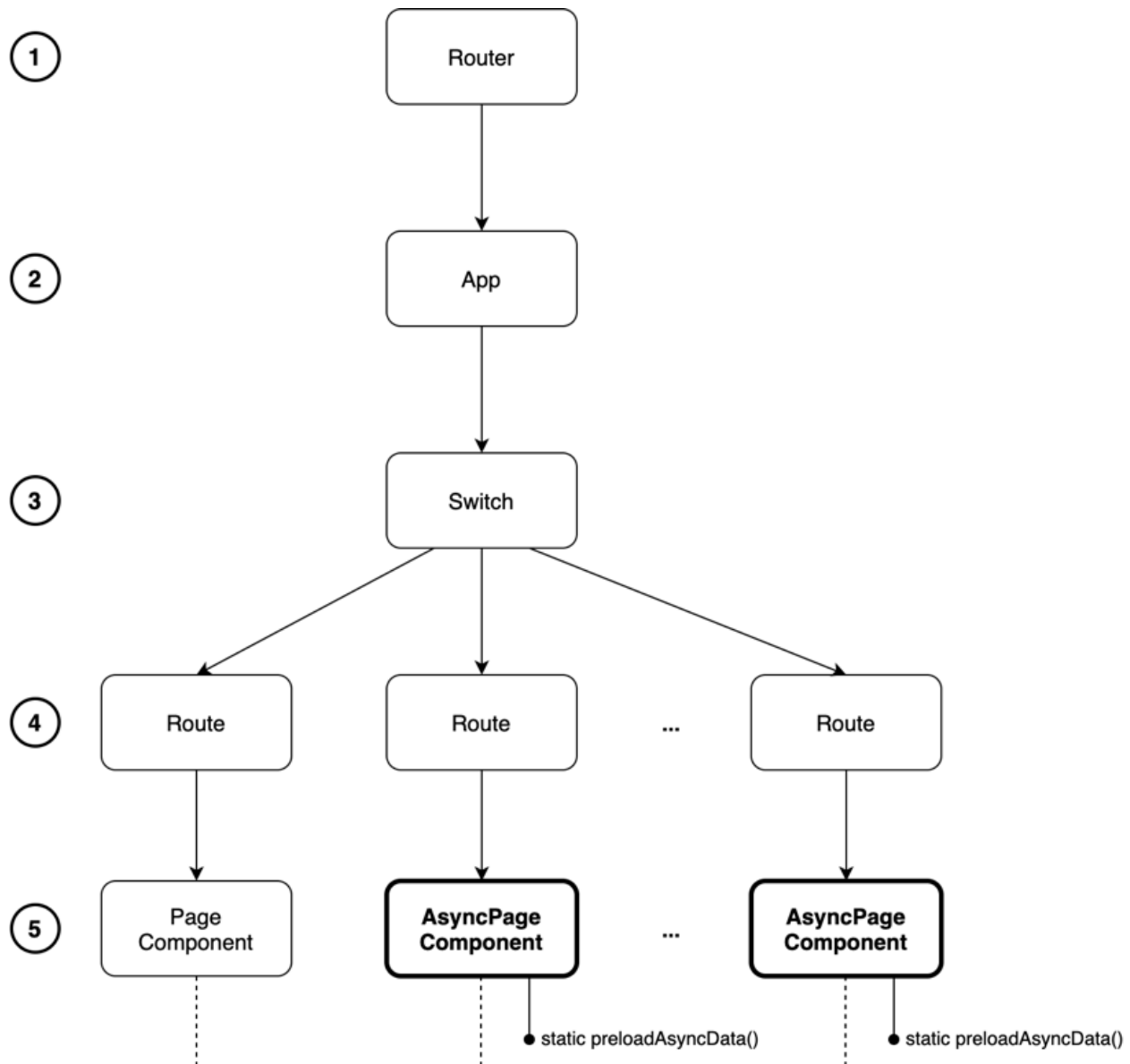


Figure 10.5: Async pages components tree structure In Figure 10.5, we have represented the structure that allows us to apply the async pages technique. Let’s discuss in detail the scope of every layer in the components tree:

Hình 10.5: Cấu trúc cây component của các trang bất đồng bộ Trong Hình 10.5, chúng ta đã biểu diễn cấu trúc cho phép chúng ta áp dụng kỹ thuật async pages. Hãy bàn chi tiết phạm vi của mỗi tầng trong cây component:

1. The root of the application is always a Router component (StaticRouter on the server and BrowserRouter on the client).
2. The application component is the only child of the Router component.
3. The only child of the application component is a Switch component from the react-router-dom package.
4. The Switch component has one or more Route components as children. These are used to define all the possible routes and which component should be rendered for every route.
5. This is the most interesting layer as we actually introduce the concept of a “page component.”

The idea is that a page component is responsible for the look and feel of an entire page. A page component can have an arbitrary subtree of components used to render the current view; for instance, a header, a body, and a footer. We can have two types of page components: regular page components that behave as any other React components and AsyncPage components. Async pages are special stateful components that need to preload data for the page to be rendered both on the server- and the client side. They implement a special static method called `preloadAsyncData()` that contains the logic necessary to preload the data for the given page.

1. Gốc của ứng dụng luôn là một component Router (`StaticRouter` trên máy chủ và `BrowserRouter` trên client).
2. Component ứng dụng là con duy nhất của component Router.
3. Con duy nhất của component ứng dụng là một component Switch từ gói `react-router-dom`.
4. Component Switch có một hoặc nhiều component Route làm con. Chúng được dùng để định nghĩa tất cả các route khả dĩ và component nào nên được kết xuất cho mỗi route.
5. Đây là tầng thú vị nhất vì chúng ta thực sự giới thiệu khái niệm “component trang” (page component). Ý tưởng là một component trang chịu trách nhiệm về diện mạo (look and feel) của cả một trang. Một component trang có thể có một cây con (subtree) tùy ý các component được dùng để kết xuất khung nhìn hiện tại; chẳng hạn, một header, một body, và một footer. Chúng ta có thể có hai loại component trang: các component trang thông thường vốn hành xử như bất kỳ component React nào khác và các component AsyncPage. Async pages là các component có trạng thái đặc biệt cần tải trước dữ liệu để trang được kết xuất trên cả phía máy chủ lẫn phía client. Chúng triển khai một phương thức tĩnh đặc biệt tên là `preloadAsyncData()` chứa logic cần thiết để tải trước dữ liệu cho trang cho trước.

You can see that layers 1 to 4 are responsible for the routing logic, while level 5 is responsible for data loading and for actually rendering the current page. There are no other nested layers for additional routing and data loading.

Bạn có thể thấy rằng các tầng 1 đến 4 chịu trách nhiệm về logic định tuyến, trong khi tầng 5 chịu trách nhiệm về việc tải dữ liệu và thực sự kết xuất trang hiện tại. Không có tầng lồng nào khác cho việc định tuyến và tải dữ liệu bổ sung.

Technically, there could be additional layers for routing and data loading after level 5, but those won't be universally available as they will be resolved only on the client side after the page has been rendered.

Về mặt kỹ thuật, có thể có thêm các tầng cho việc định tuyến và tải dữ liệu sau tầng 5, nhưng những tầng đó sẽ không có sẵn một cách universal vì chúng sẽ chỉ được phân giải ở phía client sau khi trang đã được kết xuất.

Now that we've discussed this more rigid structure, let's see how it can be useful to avoid multiple rendering passes and achieve universal data retrieval.

Giờ khi đã bàn về cấu trúc cứng nhắc hơn này, hãy xem nó có thể hữu ích thế nào để tránh nhiều lượt kết xuất và đạt được truy xuất dữ liệu universal.

Here's the idea: if we have our routes defined in a dedicated file as an array of paths and components, we can easily reuse this file on the server side and determine, before the React rendering phase, which page component we will actually end up rendering. Then, we can see if this page component is an AsyncPage. If it is, it means we have to preload some data on the server side before the rendering. We can do this by calling the `preloadAsyncData()` method from the given component.

Đây là ý tưởng: nếu chúng ta có các route được định nghĩa trong một tập tin chuyên dụng dưới dạng một mảng các đường dẫn và component, chúng ta có thể dễ dàng tái sử dụng tập tin này ở phía máy chủ và xác định, trước pha kết xuất React, component trang nào chúng ta thực sự sẽ kết xuất. Sau đó, chúng ta có thể xem liệu component trang này có phải là một `AsyncPage` hay không. Nếu phải, nghĩa là chúng ta phải tải trước một số dữ liệu ở phía máy chủ trước khi kết xuất. Chúng ta có thể làm điều này bằng cách gọi phương thức `preloadAsyncData()` từ component cho trước.

Once the data has been preloaded, this can be added in the static context and we can render the entire application. During the rendering phase, the `AsyncPage` component will see that its data is already preloaded and available in the static context and it will be able to render straight away, skipping the loading state.

Một khi dữ liệu đã được tải trước, nó có thể được thêm vào static context và chúng ta có thể kết xuất toàn bộ ứng dụng. Trong pha kết xuất, component `AsyncPage` sẽ thấy rằng dữ liệu của nó đã được tải trước và có sẵn trong static context và nó sẽ có thể kết xuất ngay lập tức, bỏ qua trạng thái loading.

Once the rendering is finished, the server can add the same preloaded data in a script tag so that, on the browser side, the user won't have to wait for the data to be loaded again.

Một khi việc kết xuất hoàn tất, máy chủ có thể thêm cùng dữ liệu đã tải trước vào một thẻ `script` sao cho, ở phía trình duyệt, người dùng sẽ không phải chờ dữ liệu được tải lại lần nữa.

The Next.js framework (nodejsdp.link/nextjs) is a popular framework for Universal JavaScript applications and adopts a similar technique to the one described here, so it is a good example of this pattern in the wild.

Framework Next.js (nodejsdp.link/nextjs) là một framework phổ biến cho các ứng dụng Universal JavaScript và áp dụng một kỹ thuật tương tự với kỹ thuật được mô tả ở đây, nên nó là một ví dụ tốt về mẫu này trong thực tế.

Implementing async pages — Triển khai các trang bất đồng bộ

Now that we know how to solve our data fetching problems, let's implement the async pages technique in our application.

Giờ khi đã biết cách giải quyết các vấn đề lấy dữ liệu của mình, hãy triển khai kỹ thuật async pages trong ứng dụng của chúng ta.

Our components tree is already structured in a way that it's compliant to what's expected by this technique. Our pages are the `AuthorsIndex` component, the `Author` component, and the `FourOhFour` component. The first two require universal data loading, so we will have to convert them into async pages.

Cây component của chúng ta đã được cấu trúc theo một cách tuân thủ những gì kỹ thuật này mong đợi. Các trang của chúng ta là component `AuthorsIndex`, component `Author`, và component `FourOhFour`. Hai cái đầu cần tải dữ liệu universal, nên chúng ta sẽ phải chuyển đổi chúng thành async pages.

Let's start to update our application by extrapolating the route definitions into a dedicated file, `src/frontend/routes.js`:

Hãy bắt đầu cập nhật ứng dụng của chúng ta bằng cách trích xuất các định nghĩa route vào một tập tin chuyên dụng, `src/frontend/routes.js`:


```

import { AuthorsIndex } from './components/pages/AuthorsIndex.js'
import { Author } from './components/pages/Author.js'
import { FourOhFour } from './components/pages/FourOhFour.js'
export const routes = [
  {
    path: '/',
    exact: true,
    component: AuthorsIndex
  },
  {
    path: '/author/:authorId',
    component: Author
  },
  {
    path: '*',
    component: FourOhFour
  }
]

```

We want this configuration file to be the source of truth for the router configuration across the various parts of the application, so let's **refactor** the frontend App component to use this file as well:

Chúng ta muốn tập tin cấu hình này là nguồn chân lý (source of truth) cho cấu hình router xuyên suốt các phần khác nhau của ứng dụng, nên hãy tái cấu trúc component App frontend để cũng dùng tập tin này:

```

// src/frontend/App.js
import react from 'react'
import htm from 'htm'
import { Switch, Route } from 'react-router-dom'
import { routes } from './routes.js'
const html = htm.bind(react.createElement)
export class App extends react.Component {
  render () {
    return html`<${Switch}>
      ${routes.map(routeConfig =>
        html`<${Route}
          key=${routeConfig.path}
          ...${routeConfig}
        />`
      )}
    </>`
  }
}

```

As you can see, the only change here is that, rather than defining the various Route components inline, we build them dynamically starting from the routes configuration array. Any change in the routes.js file will be automatically reflected in the application as well.

Như bạn có thể thấy, thay đổi duy nhất ở đây là, thay vì định nghĩa các component Route khác nhau inline, chúng ta xây dựng chúng một cách động bắt đầu từ mảng cấu hình routes. Bất kỳ thay đổi nào trong tập tin routes.js cũng sẽ tự động được phản ánh

trong ứng dụng.

At this point, we can update the server-side logic in `src/server.js`.

Tại điểm này, chúng ta có thể cập nhật logic phía máy chủ trong `src/server.js`.

The first thing that we want to do is import a utility function from the `react-router-dom` package that allows us to see if a given URL matches a given React router path definition. We also need to import the routes array from the `new routes.js` module.

Điều đầu tiên chúng ta muốn làm là import một hàm tiện ích từ gói `react-router-dom` cho phép chúng ta xem liệu một URL cho trước có khớp một định nghĩa đường dẫn React router cho trước hay không. Chúng ta cũng cần import mảng routes từ module `routes.js` mới.

```
// ...
import { StaticRouter, matchPath } from 'react-router-dom'
import { routes } from '../frontend/routes.js'
// ...
```

Now, let's update our server-side HTML template generation function to be able to embed preloaded data in our page:

Giờ, hãy cập nhật hàm tạo template HTML phía máy chủ của chúng ta để có thể nhúng dữ liệu đã tải trước vào trang của mình:

```
// ...
const template = ({ content, serverData }) => `<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>My library</title>
  </head>
  <body>
    <div id="root">${content}</div>
    ${serverData ? `<script type="text/javascript">
window.__STATIC_CONTEXT__=${JSON.stringify(serverData)}
</script>` : ''}
    <script type="text/javascript" src="/public/main.js"></script>
  </body>
</html>`
// ...
```

As you can see, our template now accepts a new argument called `serverData`. If this argument is passed to the template function, it will render a script tag that will inject this data into a global variable called `window.__STATIC_CONTEXT__`.

Như bạn có thể thấy, template của chúng ta giờ nhận một tham số mới tên là `serverData`. Nếu tham số này được truyền cho hàm `template`, nó sẽ kết xuất một thẻ script sẽ tiêm dữ liệu này vào một biến toàn cục tên là `window.__STATIC_CONTEXT__`.

Now, let's get into the meaty bit; let's rewrite the server-side rendering logic:

Giờ, hãy đi vào phần cốt lõi; hãy viết lại logic kết xuất phía máy chủ:

```
// ...
server.get('*', async (req, reply) => {
```

```

const location = req.raw.originalUrl
let component // (1)
let match
for (const route of routes) {
  component = route.component
  match = matchPath(location, route)
  if (match) {
    break
  }
}
let staticData // (2)
let staticError
let hasStaticContext = false
if (typeof component.preloadAsyncData === 'function') {
  hasStaticContext = true
  try {
    const data = await component.preloadAsyncData({ match })
    staticData = data
  } catch (err) {
    staticError = err
  }
}
const staticContext = {
  [location]: {
    data: staticData,
    err: staticError
  }
}

const serverApp = html`
  <${StaticRouter}
    location=${location}
    context=${staticContext}
  >
  <${App}/>
</>
`

const content = reactServer.renderToString(serverApp)
const serverData = hasStaticContext ? staticContext : null
const responseHtml = template({ content, serverData })
const code = staticContext.statusCode
  ? staticContext.statusCode
  : 200
reply.code(code).type('text/html').send(responseHtml)
// ...

```

There are quite some changes here. Let's discuss the main blocks one by one:

Có khá nhiều thay đổi ở đây. Hãy bàn các khối chính từng cái một:

1. The first change aims to detect which page will be rendered for the current URL. We loop through the defined routes and we use the `matchPath` utility to verify if location matches

the current route definition. If it does, we stop the loop and record which component will be rendered in the component variable. We can be sure a component will be matched here because our last route (the 404 page) will always match. The match variable will contain information about the match. For instance, if the route contains some parameters, match will contain the path fragment that matched every parameter. For instance, for the URL `/author/joyce`, match will have the property `params` equal to `{ authorId: 'joyce' }`. This is the same prop that a page component will receive from the router when rendered.

2. In the second block of changes, we check if the selected component is an `AsyncPage`. We do that by checking if the component has a static method called `preloadAsyncData`. If that's the case, we invoke that function by passing an object that contains the match object as an argument (this way, we propagate any parameter that might be needed to fetch the data, such as `authorId`). This function should return a promise. If the promise resolves, we have successfully preloaded the data for this component. If it rejects, we make sure to record the error. Finally, we create the `staticContext` object. This object maps the preloaded data (or the rejection error) to the current location. The reason why we keep the location as a key is to be sure that if, for any reason, the browser renders another page from the one we preloaded (because of a programmatic error or because of a user action, like hitting the back button on the browser before the page is fully loaded), we won't end up using preloaded data that is not relevant to the current page on the browser.
3. In the last block of changes, we invoke the `renderToString()` function to get the rendered HTML of the application. Note that since we are passing a static context containing the preloaded data, we expect that the application will be able to completely render the page without returning a loading state view. This does not happen magically, of course. We will need to add some logic to our React component to check if the necessary data is already available in the static context. Once we have the generated HTML, we use our `template()` function to generate the complete page markup and we return it to the browser. We also make sure to respect the status code. For instance, if we ended up rendering the `FourOhFour` component, we will have the `statusCode` property in the static context changed, so if that's the case, we use that value for the final status code; otherwise, we default to `200`.

1. Thay đổi đầu tiên nhằm phát hiện trang nào sẽ được kết xuất cho URL hiện tại. Chúng ta lặp qua các routes đã định nghĩa và dùng tiện ích `matchPath` để xác minh liệu `location` có khớp định nghĩa route hiện tại hay không. Nếu có, chúng ta dừng vòng lặp và ghi lại component nào sẽ được kết xuất vào biến `component`. Chúng ta có thể chắc chắn một component sẽ được khớp ở đây vì route cuối cùng của chúng ta (trang 404) sẽ luôn khớp. Biến `match` sẽ chứa thông tin về sự khớp. Chẳng hạn, nếu route chứa một số tham số, `match` sẽ chứa đoạn đường dẫn (path fragment) khớp với mỗi tham số. Chẳng hạn, với URL `/author/joyce`, `match` sẽ có thuộc tính `params` bằng `{ authorId: 'joyce' }`. Đây là cùng prop mà một component trang sẽ nhận từ router khi được kết xuất.
2. Trong khối thay đổi thứ hai, chúng ta kiểm tra xem component được chọn có phải là một `AsyncPage` hay không. Chúng ta làm điều đó bằng cách kiểm tra xem component có một phương thức tĩnh tên là `preloadAsyncData` hay không. Nếu đúng vậy, chúng ta gọi hàm đó bằng cách truyền một đối tượng chứa đối tượng `match` làm tham số (theo cách này, chúng ta truyền tiếp bất kỳ tham số nào có thể cần để lấy dữ liệu, chẳng hạn `authorId`). Hàm này sẽ trả về một promise. Nếu promise được giải quyết (resolve), chúng ta đã tải trước thành công dữ liệu cho component này. Nếu nó bị từ chối (reject), chúng ta đảm bảo ghi lại lỗi. Cuối cùng, chúng ta tạo đối tượng `staticContext`. Đối tượng này ánh xạ dữ liệu đã tải trước (hoặc lỗi từ chối) tới `location` hiện tại. Lý do chúng ta giữ `location` làm khoá là để chắc chắn rằng nếu, vì bất kỳ lý do nào, trình duyệt kết xuất một trang khác với trang mà chúng ta đã tải

trước (vì lỗi lập trình hoặc vì hành động của người dùng, như nhấn nút quay lại trên trình duyệt trước khi trang được tải đầy đủ), chúng ta sẽ không kết thúc bằng việc dùng dữ liệu đã tải trước không liên quan đến trang hiện tại trên trình duyệt.

3. Trong khối thay đổi cuối cùng, chúng ta gọi hàm `renderToString()` để lấy HTML đã kết xuất của ứng dụng. Lưu ý rằng vì chúng ta đang truyền một static context chứa dữ liệu đã tải trước, chúng ta mong đợi rằng ứng dụng sẽ có thể kết xuất trang hoàn chỉnh mà không trả về một khung nhìn trạng thái loading. Tất nhiên điều này không xảy ra một cách kỳ diệu. Chúng ta sẽ cần thêm một số logic vào component React của mình để kiểm tra xem dữ liệu cần thiết đã có sẵn trong static context hay chưa. Một khi chúng ta có HTML được tạo ra, chúng ta dùng hàm `template()` của mình để tạo markup trang hoàn chỉnh và trả nó về trình duyệt. Chúng ta cũng đảm bảo tôn trọng mã trạng thái. Chẳng hạn, nếu chúng ta cuối cùng kết xuất component `FourOhFour`, chúng ta sẽ có thuộc tính `statusCode` trong static context bị thay đổi, nên nếu đúng vậy, chúng ta dùng giá trị đó cho mã trạng thái cuối cùng; nếu không, chúng ta mặc định về 200.

That's it for our server-side rendering. Now, it's time to create the async page abstraction in our React application. Since we are going to have two distinct async pages, a good way to reuse some code is to create a base class and to use the Template pattern that we already discussed in Chapter 9, Behavioral Design Patterns. Let's define this class in `src/frontend/components/pages/AsyncPage.js`:

Vậy là xong với phần kết xuất phía máy chủ của chúng ta. Giờ, đã đến lúc tạo lớp trừu tượng async page trong ứng dụng React của chúng ta. Vì chúng ta sẽ có hai async page riêng biệt, một cách tốt để tái sử dụng một số mã là tạo một lớp cơ sở (base class) và dùng mẫu Template mà chúng ta đã bàn ở Chương 9, Các mẫu thiết kế hành vi (Behavioral Design Patterns). Hãy định nghĩa lớp này trong `src/frontend/components/pages/AsyncPage.js`:

```
import react from 'react'
export class AsyncPage extends react.Component {
  static async preloadAsyncData (props) { // (1)
    throw new Error('Must be implemented by sub class')
  }
  render () {
    throw new Error('Must be implemented by sub class')
  }
  constructor (props) { // (2)
    super(props)
    const location = props.match.url
    this.hasData = false
    let staticData
    let staticError
    const staticContext = typeof window !== 'undefined'
      ? window.__STATIC_CONTEXT__ // client-side
      : this.props.staticContext // server-side
    if (staticContext && staticContext[location]) {
      const { data, err } = staticContext[location]
      staticData = data
      staticError = err
      this.hasStaticData = true
      typeof window !== 'undefined' &&
        delete staticContext[location]
    }
  }
}
```

```

    }
    this.state = {
      ...staticData,
      staticError,
      loading: !this.hasStaticData
    }
  }
  async componentDidMount () { // (3)
    if (!this.hasStaticData) {
      let staticData
      let staticError
      try {
        const data = await this.constructor.preloadAsyncData(
          this.props
        )
        staticData = data
      } catch (err) {
        staticError = err
      }
      this.setState({
        ...staticData,
        loading: false,
        staticError
      })
    }
  }
}

```

This class provides helper code for building a stateful component that can handle three possible scenarios:

Lớp này cung cấp mã trợ giúp để xây dựng một component có trạng thái có thể xử lý ba kịch bản khả dĩ:

- We are rendering on the server and we already have the data preloaded (no need to load the data).
- We are rendering on the client and the data is already available in the page through the `__STATIC_CONTEXT__` variable (no need to load the data).
- We are rendering on the client and the data is not available (for instance, if this page was not rendered by the server, but a page the user navigated to after the first load). In this case, the data has to be dynamically loaded from the client when the component is mounted.
 - Chúng ta đang kết xuất trên máy chủ và đã có dữ liệu được tải trước (không cần tải dữ liệu).
 - Chúng ta đang kết xuất trên client và dữ liệu đã có sẵn trong trang thông qua biến `__STATIC_CONTEXT__` (không cần tải dữ liệu).
 - Chúng ta đang kết xuất trên client và dữ liệu không có sẵn (chẳng hạn, nếu trang này không được kết xuất bởi máy chủ, mà là một trang người dùng điều hướng tới sau lần tải đầu tiên). Trong trường hợp này, dữ liệu phải được tải động từ client khi component được mount.

Let's review the main points of this implementation together:

Hãy cùng nhau điểm lại các điểm chính của cách triển khai này:

1. This component class should not be instantiated directly but only extended when implementing async pages. When this class is extended, the async page component will need to implement the methods `static async preloadAsyncData(props)` and `render()`.
2. In the constructor, we have to initialize the component state. There are two possible outcomes here: the data is already available (so we can set it in the state) or the data is not available (so we need to set the state to “loading” and let the component load the data once it’s mounted on the page). If we are on the browser and we load the data from the static context, we also make sure to delete this data from the context. This will allow the user to see fresh data if they happen to go back to this page during the navigation.
3. The method `componentDidMount()` is executed by React only on the browser. Here, we handle the case where the data was not preloaded and we have to dynamically load it at runtime.
 1. Lớp component này không nên được khởi tạo trực tiếp mà chỉ được kế thừa khi triển khai các async page. Khi lớp này được kế thừa, component async page sẽ cần triển khai các phương thức `static async preloadAsyncData(props)` và `render()`.
 2. Trong constructor, chúng ta phải khởi tạo trạng thái component. Có hai kết quả khả dĩ ở đây: dữ liệu đã có sẵn (nên chúng ta có thể đặt nó vào state) hoặc dữ liệu không có sẵn (nên chúng ta cần đặt state thành “loading” và để component tải dữ liệu một khi nó được mount lên trang). Nếu chúng ta đang ở trên trình duyệt và tải dữ liệu từ static context, chúng ta cũng đảm bảo xóa dữ liệu này khỏi context. Điều này sẽ cho phép người dùng thấy dữ liệu mới nếu họ tình cờ quay lại trang này trong quá trình điều hướng.
 3. Phương thức `componentDidMount()` được React thực thi chỉ trên trình duyệt. Ở đây, chúng ta xử lý trường hợp dữ liệu không được tải trước và chúng ta phải tải nó động tại runtime.

Now that we have this useful abstraction in place, we can rewrite our `AuthorsIndex` and `Author` components and convert them into async pages. Let’s start with `AuthorsIndex`:

Giờ khi đã có lớp trừu tượng hữu ích này tại chỗ, chúng ta có thể viết lại các component `AuthorsIndex` và `Author` của mình và chuyển đổi chúng thành async pages. Hãy bắt đầu với `AuthorsIndex`:

```
import react from 'react'
import htm from 'htm'
import { Link } from 'react-router-dom'
import superagent from 'superagent'
import { AsyncPage } from './AsyncPage.js'
import { Header } from '../Header.js'
const html = htm.bind(react.createElement)
export class AuthorsIndex extends AsyncPage {
  static async preloadAsyncData (props) {
    const { body } = await superagent.get(
      'http://localhost:3001/api/authors'
    )
    return { authors: body }
  }
  render () {
    // unchanged...
  }
}
```

As you can see here, our `AuthorsIndex` component now extends `AsyncPage`. Since the `AsyncPage` template will take care of all the state management in its constructor, we don't need a constructor here anymore; we just need to specify the business logic to load the data in the `preloadAsyncData()` method. If you compare this implementation with the previous one, you might notice that the logic of this method is almost the same as what we had previously in `componentDidMount()`. The method `componentDidMount()` has been removed from here because the one we inherit from `AsyncPage` will suffice. The only difference between the previous version of `componentDidMount()` and `preloadAsyncData()` is that in `preloadAsyncData()`, we don't set the internal state directly; we just need to return the data. The underlying code in `AsyncPage` will update the state as needed for us.

Như bạn có thể thấy ở đây, component `AuthorsIndex` của chúng ta giờ kế thừa `AsyncPage`. Vì template `AsyncPage` sẽ lo tất cả việc quản lý trạng thái trong constructor của nó, chúng ta không cần một constructor ở đây nữa; chúng ta chỉ cần chỉ định logic nghiệp vụ để tải dữ liệu trong phương thức `preloadAsyncData()`. Nếu bạn so sánh cách triển khai này với cách trước, bạn có thể nhận thấy rằng logic của phương thức này gần như giống với những gì chúng ta đã có trước đây trong `componentDidMount()`. Phương thức `componentDidMount()` đã bị loại bỏ khỏi đây vì phương thức chúng ta kế thừa từ `AsyncPage` là đủ. Khác biệt duy nhất giữa phiên bản trước của `componentDidMount()` và `preloadAsyncData()` là trong `preloadAsyncData()`, chúng ta không đặt trạng thái nội bộ trực tiếp; chúng ta chỉ cần trả về dữ liệu. Mã bên dưới trong `AsyncPage` sẽ cập nhật trạng thái khi cần thay cho chúng ta.

Let's now rewrite the `Author` component:

Giờ hãy viết lại component `Author`:

```
import react from 'react'
import htm from 'htm'
import superagent from 'superagent'
import { AsyncPage } from './AsyncPage.js'
import { FourOhFour } from './FourOhFour.js'
import { Header } from '../Header.js'
const html = htm.bind(react.createElement)
export class Author extends AsyncPage {
  static async preloadAsyncData (props) {
    const { body } = await superagent.get(
      `http://localhost:3001/api/author/${
        props.match.params.authorId
      }`
    )
    return { author: body }
  }
  render () {
    // unchanged...
  }
}
```

The changes here are perfectly in line with the changes we made for the `AuthorsIndex` component. We are only moving the data loading logic into `preloadAsyncData()` and letting the underlying abstraction manage the state transition for us. Now, we can apply just a last small optimization in our `src/frontend/index.js` file. We can swap the `ReactDOM.render()` function call with `ReactDOM.hydrate()`. Since we will produce exactly the same markup from both the server side

and the client side, this will make React a bit faster to initialize during the first browser load.

Các thay đổi ở đây hoàn toàn nhất quán với các thay đổi chúng ta đã làm cho component `AuthorsIndex`. Chúng ta chỉ đang chuyển logic tải dữ liệu vào `preloadAsyncData()` và để lớp trừu tượng bên dưới quản lý chuyển tiếp trạng thái thay cho chúng ta. Giờ, chúng ta có thể áp dụng một tối ưu nhỏ cuối cùng trong tập tin `src/frontend/index.js` của mình. Chúng ta có thể thay lời gọi hàm `ReactDOM.render()` bằng `ReactDOM.hydrate()`. Vì chúng ta sẽ tạo ra đúng cùng markup từ cả phía máy chủ lẫn phía client, điều này sẽ làm cho React khởi tạo nhanh hơn một chút trong lần tải trình duyệt đầu tiên.

We are finally ready to try all these changes. Make sure to rebuild the frontend bundle and relaunch the server. Have a look at the application and the code that is generated by the server; it should contain all the preloaded data for every page. Also, 404 errors should be reported correctly for every 404 page, including the ones for missing authors.

Cuối cùng chúng ta đã sẵn sàng thử tất cả những thay đổi này. Hãy đảm bảo build lại bundle frontend và khởi động lại máy chủ. Hãy xem ứng dụng và mã được máy chủ tạo ra; nó sẽ chứa tất cả dữ liệu đã tải trước cho mọi trang. Ngoài ra, các lỗi 404 sẽ được báo cáo đúng đắn cho mọi trang 404, kể cả những trang cho các tác giả bị thiếu.

Great! We finally managed to build an application that efficiently shares code, logic, and data between the client and the server: a true Universal JavaScript application!

Tuyệt! Cuối cùng chúng ta đã xoay sở để xây dựng một ứng dụng chia sẻ mã, logic và dữ liệu một cách hiệu quả giữa client và máy chủ: một ứng dụng Universal JavaScript thực thụ!

Summary — Tóm tắt

In this chapter, we explored the innovative and fast-moving world of Universal JavaScript. Universal JavaScript opens up a lot of new opportunities in the field of web development and it can help you build single-page applications that load fast, are accessible, and are optimized for search engines.

Trong chương này, chúng ta đã khám phá thế giới đổi mới và chuyển động nhanh của Universal JavaScript. Universal JavaScript mở ra rất nhiều cơ hội mới trong lĩnh vực phát triển web và nó có thể giúp bạn xây dựng các ứng dụng single-page tải nhanh, dễ tiếp cận và được tối ưu cho các công cụ tìm kiếm.

In this chapter, we focused on introducing all the basics of this subject. We started from exploring module bundlers, why we need them, and how they work. We learned how to use webpack, and then we introduced React and discussed some of its functionality. We learned how to build component-oriented user interfaces and then started to build an application from scratch to explore universal rendering, universal routing, and universal data retrieval.

Trong chương này, chúng ta tập trung vào việc giới thiệu tất cả những điều cơ bản của chủ đề này. Chúng ta bắt đầu từ việc khám phá các module bundler, tại sao chúng ta cần chúng, và cách chúng hoạt động. Chúng ta đã học cách dùng webpack, rồi giới thiệu React và bàn về một số chức năng của nó. Chúng ta đã học cách xây dựng giao diện người dùng hướng-component rồi bắt đầu xây dựng một ứng dụng từ đầu để khám phá kết xuất universal, định tuyến universal, và truy xuất dữ liệu universal.

Even though we discussed a lot of topics, we barely scratched the surface of this wide topic, but you should have gained all the necessary knowledge to keep exploring this world on your own if you are interested in knowing more. Given that this field is still evolving quite rapidly, tools and libraries will

probably change a lot in the next few years, but all the basic concepts should stay there, so don't be afraid to keep exploring and experimenting. Becoming an expert on this topic is now just a matter of using the acquired knowledge to build a first real-world app with real, business-driven use cases. It's also worth underlining that the knowledge acquired here might be useful for projects that cross the boundaries of web development, like mobile app development. If you are interested in this topic, React Native might be a good starting point.

Dù đã bàn rất nhiều chủ đề, chúng ta mới chỉ chạm bề mặt của chủ đề rộng lớn này, nhưng bạn nên đã thu được tất cả kiến thức cần thiết để tiếp tục tự mình khám phá thế giới này nếu bạn quan tâm muốn biết thêm. Vì lĩnh vực này vẫn còn tiến hoá khá nhanh, các công cụ và thư viện có lẽ sẽ thay đổi rất nhiều trong vài năm tới, nhưng tất cả các khái niệm cơ bản nên vẫn còn đó, nên đừng ngại tiếp tục khám phá và thử nghiệm. Trở thành chuyên gia về chủ đề này giờ chỉ còn là vấn đề dùng kiến thức thu được để xây dựng một ứng dụng thực tế đầu tiên với các trường hợp dùng thực, do nghiệp vụ thúc đẩy. Cũng đáng nhấn mạnh rằng kiến thức thu được ở đây có thể hữu ích cho các dự án vượt ra ngoài ranh giới của phát triển web, như phát triển ứng dụng di động. Nếu bạn quan tâm chủ đề này, React Native có thể là một điểm khởi đầu tốt.

In the next chapter, we are going to take a problem-solution approach to explore some more advanced topics. Are you ready?

Trong chương tiếp theo, chúng ta sẽ áp dụng cách tiếp cận bài toán-lời giải để khám phá một số chủ đề tân tiến hơn. Bạn đã sẵn sàng chưa?

Exercises — Bài tập

- 10.1 A matter of style: Our little library application looks very barebone. It could look a lot better with some style and images. Why don't you try to improve the look of the app? If you get stuck or need some inspiration, you can check our version of this exercise on GitHub (nodejsdp.link/univ).
 - 10.2 Proper data management: As we said, keeping a lot of data in a file is not a great idea. Why don't you try to move all the data into a real database backend of your choice? At this point, you might also want to take this application to the next level and write some script to import data from a big collection of books like the Open Library archive (nodejsdp.link/open-library-api).
 - 10.3 Pagination and search: Now that you have a more significant database, it's probably time to add some important features like search and pagination.
 - 10.4 A Universal... Blog!: Build a new universal JavaScript app from scratch that implements a blog. Then, try the same exercise using a framework like Next.js (nodejsdp.link/nextjs) or Gatsby (nodejsdp.link/gatsby).
- 10.1 Vấn đề về phong cách: Ứng dụng thư viện nhỏ của chúng ta trông rất trơ trụi. Nó có thể trông đẹp hơn nhiều với một chút phong cách và hình ảnh. Sao bạn không thử cải thiện diện mạo của ứng dụng nhỉ? Nếu bạn bí hoặc cần chút cảm hứng, bạn có thể xem phiên bản của chúng tôi cho bài tập này trên GitHub (nodejsdp.link/univ).
 - 10.2 Quản lý dữ liệu đúng cách: Như chúng ta đã nói, giữ nhiều dữ liệu trong một tập tin không phải là một ý tưởng hay. Sao bạn không thử chuyển tất cả dữ liệu vào một backend cơ sở dữ liệu thực tự chọn nhỉ? Tại điểm này, bạn cũng có thể muốn đưa ứng dụng này lên tầm cao mới và viết một script nào đó để import dữ liệu từ một bộ sưu tập sách lớn như kho lưu trữ Open Library (nodejsdp.link/open-library-api).
 - 10.3 Phân trang và tìm kiếm: Giờ khi bạn đã có một cơ sở dữ liệu đáng kể hơn, có lẽ

đã đến lúc thêm một số tính năng quan trọng như tìm kiếm và phân trang.

- 10.4 Một... Blog Universal!: Xây dựng một ứng dụng Universal JavaScript mới từ đầu triển khai một blog. Sau đó, thử cùng bài tập đó bằng một framework như Next.js (nodejsdp.link/nextjs) hoặc Gatsby (nodejsdp.link/gatsby).

Chapter 11: Advanced Recipes —

Chương 11: Các công thức nâng cao

In this chapter, we'll take a problem-solution approach and, like in a cookbook, we'll present a set of ready-to-use recipes to solve some common Node.js programming problems.

Trong chương này, chúng ta sẽ tiếp cận theo hướng bài toán - giải pháp và, giống như trong một cuốn sách dạy nấu ăn, sẽ trình bày một tập hợp các công thức sẵn dùng để giải quyết một số bài toán lập trình Node.js phổ biến.

You shouldn't be surprised by the fact that most of the problems presented in this chapter arise when we try to do things asynchronously. In fact, as we've seen repeatedly in the previous chapters of this book, tasks that are trivial in traditional synchronous programming can become more complicated when applied to asynchronous programming. A typical example is trying to use a component that requires an asynchronous initialization step. In this case, we have the inconvenience of delaying any attempt to use the component until the initialization completes. We'll show you how to solve this elegantly later.

Bạn không nên ngạc nhiên khi hầu hết các bài toán được trình bày trong chương này phát sinh khi ta cố gắng làm mọi việc một cách bất đồng bộ. Thực tế, như ta đã thấy nhiều lần trong các chương trước của cuốn sách này, những tác vụ vốn rất tầm thường trong lập trình đồng bộ truyền thống có thể trở nên phức tạp hơn khi áp dụng cho lập trình bất đồng bộ. Một ví dụ điển hình là việc cố gắng sử dụng một thành phần đòi hỏi một bước khởi tạo bất đồng bộ. Trong trường hợp này, ta gặp phải sự bất tiện là phải trì hoãn mọi nỗ lực sử dụng thành phần đó cho đến khi quá trình khởi tạo hoàn tất. Chúng ta sẽ chỉ cho bạn cách giải quyết điều này một cách thanh lịch ở phần sau.

But this chapter is not just about recipes involving asynchronous programming. You will also learn the best ways to run CPU-intensive tasks in Node.js.

Nhưng chương này không chỉ nói về các công thức liên quan đến lập trình bất đồng bộ. Bạn cũng sẽ học được những cách tốt nhất để chạy các tác vụ tốn nhiều CPU (CPU-intensive) trong Node.js.

These are the recipes you will learn in this chapter:

Đây là những công thức bạn sẽ học trong chương này:

- Dealing with asynchronously initialized components
- Asynchronous **request** batching and caching
- Canceling asynchronous operations
- Running CPU-bound tasks
 - Xử lý các thành phần được khởi tạo bất đồng bộ

- Gộp lô (batching) và lưu đệm (caching) các yêu cầu bất đồng bộ
- Hủy các thao tác bất đồng bộ
- Chạy các tác vụ chiếm dụng CPU (CPU-bound)

Let's get started.

Bắt đầu thôi.

Dealing with asynchronously initialized components — Xử lý các thành phần được khởi tạo bất đồng bộ

One of the reasons for the existence of synchronous APIs in the Node.js core modules and many npm packages is because they are handy to use for implementing initialization tasks. For simple programs, using synchronous APIs at initialization time can streamline things a lot and the drawbacks associated with their use remain contained because they are used only once, which is when the program or a particular component is initialized.

Một trong những lý do tồn tại của các API đồng bộ trong các module lõi của Node.js và nhiều gói npm là vì chúng tiện dùng để hiện thực các tác vụ khởi tạo. Với những chương trình đơn giản, dùng API đồng bộ tại thời điểm khởi tạo có thể giúp mọi thứ gọn gàng hơn rất nhiều, và những nhược điểm khi sử dụng chúng vẫn nằm trong tầm kiểm soát vì chúng chỉ được dùng một lần duy nhất, đó là khi chương trình hoặc một thành phần cụ thể được khởi tạo.

Unfortunately, this is not always possible. A synchronous **API** might not always be available, especially for components using the network during their initialization phase to, for example, perform handshake protocols or to retrieve configuration parameters. This is the case for many database drivers and clients for **middleware** systems such as message queues.

Tiếc thay, điều này không phải lúc nào cũng khả thi. Một API đồng bộ có thể không phải lúc nào cũng sẵn có, đặc biệt là với các thành phần dùng mạng trong giai đoạn khởi tạo để, ví dụ, thực hiện các giao thức bắt tay (handshake) hoặc lấy về các tham số cấu hình. Đây là trường hợp của nhiều trình điều khiển cơ sở dữ liệu (database driver) và client cho các hệ thống middleware như hàng đợi thông điệp (message queue).

The issue with asynchronously initialized components — Vấn đề với các thành phần được khởi tạo bất đồng bộ

Let's consider an example where a **module** called `db` is used to interact with a remote database. The `db` module will accept API requests only after the connection and handshake with the database server have been successfully completed. Therefore, no queries or other commands can be sent until the initialization phase is complete. The following is the code for such a sample module (the `db.js` file):

Hãy xét một ví dụ trong đó một module tên là `db` được dùng để tương tác với một cơ sở dữ liệu từ xa. Module `db` sẽ chỉ chấp nhận các yêu cầu API sau khi kết nối và bắt tay với máy chủ cơ sở dữ liệu đã hoàn tất thành công. Do đó, không thể gửi truy vấn hay lệnh nào khác cho đến khi giai đoạn khởi tạo hoàn thành. Sau đây là mã cho một module mẫu như vậy (tệp `db.js`):

```
import { EventEmitter } from 'events'
class DB extends EventEmitter {
```

```

connected = false
connect () {
  // simulate the delay of the connection
  setTimeout(() => {
    this.connected = true
    this.emit('connected')
  }, 500)
}
async query (queryString) {
  if (!this.connected) {
    throw new Error('Not connected yet')
  }
  console.log(`Query executed: ${queryString}`)
}
}
export const db = new DB()

```

This is a typical example of an asynchronously initialized component. Under these assumptions, we usually have two quick and easy solutions to this problem, which we can call local initialization check and delayed startup. Let's analyze them in more detail.

Đây là một ví dụ điển hình của thành phần được khởi tạo bất đồng bộ. Theo các giả định này, ta thường có hai giải pháp nhanh và dễ cho bài toán này, mà ta có thể gọi là kiểm tra khởi tạo cục bộ (local initialization check) và khởi động trì hoãn (delayed startup). Hãy phân tích chúng chi tiết hơn.

Local initialization check — Kiểm tra khởi tạo cục bộ

The first solution makes sure that the module is initialized before any of its APIs are invoked; otherwise, we wait for its initialization. This check has to be done every time we want to invoke an operation on the asynchronous module:

Giải pháp đầu tiên đảm bảo rằng module đã được khởi tạo trước khi bất kỳ API nào của nó được gọi; nếu chưa, ta đợi quá trình khởi tạo của nó. Việc kiểm tra này phải được thực hiện mỗi khi ta muốn gọi một thao tác trên module bất đồng bộ:

```

import { once } from 'events'
import { db } from './db.js'
db.connect()
async function updateLastAccess () {
  if (!db.connected) {
    await once(db, 'connected')
  }
  await db.query(`INSERT (${Date.now()}) INTO "LastAccesses"`)
}
updateLastAccess()
setTimeout(() => {
  updateLastAccess()
}, 600)

```

As we already anticipated, any time we want to invoke the `query()` method on the `db` component, we have to check if the module is initialized; otherwise, we wait for its initialization by listening for the `'connected'` event. A variation of this technique performs the check inside the `query()`

method itself, which shifts the burden of the boilerplate code from the consumer to the provider of the service.

Như đã dự đoán trước, mỗi khi ta muốn gọi phương thức `query()` trên thành phần `db`, ta phải kiểm tra xem module đã được khởi tạo chưa; nếu chưa, ta đợi quá trình khởi tạo của nó bằng cách lắng nghe sự kiện `'connected'`. Một biến thể của kỹ thuật này thực hiện việc kiểm tra ngay bên trong chính phương thức `query()`, qua đó dịch chuyển gánh nặng mã soạn sẵn (boilerplate) từ phía bên tiêu thụ sang phía bên cung cấp dịch vụ.

Delayed startup — Khởi động trì hoãn

The second quick and dirty solution to the problem of asynchronously initialized components involves delaying the execution of any code relying on the asynchronously initialized component until the component has finished its initialization routine. We can see an example of such a technique in the following code fragment:

Giải pháp nhanh và thô thứ hai cho bài toán các thành phần được khởi tạo bất đồng bộ liên quan đến việc trì hoãn việc thực thi mọi đoạn mã phụ thuộc vào thành phần được khởi tạo bất đồng bộ cho đến khi thành phần đó hoàn tất quy trình khởi tạo của mình. Ta có thể thấy một ví dụ của kỹ thuật này trong đoạn mã sau:

```
import { db } from './db.js'
import { once } from 'events'
async function initialize () {
  db.connect()
  await once(db, 'connected')
}
async function updateLastAccess () {
  await db.query(`INSERT (${Date.now()}) INTO "LastAccesses"`)
}
initialize()
  .then(() => {
    updateLastAccess()
    setTimeout(() => {
      updateLastAccess()
    }, 600)
  })
```

As we can see from the preceding code, we first wait for the initialization to complete, and then we proceed with executing any routine that uses the `db` object.

Như ta thấy từ đoạn mã trên, trước tiên ta đợi quá trình khởi tạo hoàn tất, rồi sau đó mới tiến hành thực thi bất kỳ routine nào sử dụng đối tượng `db`.

The main disadvantage of this technique is that it requires us to know, in advance, which components will make use of the asynchronously initialized component, which makes our code fragile and exposed to mistakes. One solution to this problem is delaying the startup of the entire application until all the asynchronous services are initialized. This has the advantage of being simple and effective; however, it can add a significant delay to the overall startup time of the application and moreover, it won't take into account the case in which the asynchronously initialized component has to be reinitialized.

Nhược điểm chính của kỹ thuật này là nó đòi hỏi ta phải biết trước những thành phần nào sẽ sử dụng thành phần được khởi tạo bất đồng bộ, điều này khiến mã của ta trở nên

mong manh và dễ mắc lỗi. Một giải pháp cho vấn đề này là trì hoãn việc khởi động toàn bộ ứng dụng cho đến khi tất cả các dịch vụ bất đồng bộ đã được khởi tạo. Cách này có lợi thế là đơn giản và hiệu quả; tuy nhiên, nó có thể thêm một độ trễ đáng kể vào tổng thời gian khởi động của ứng dụng, và hơn nữa, nó sẽ không tính đến trường hợp thành phần được khởi tạo bất đồng bộ phải được khởi tạo lại.

As we will see in the next section, there is a third alternative that allows us to transparently and efficiently delay every operation until the asynchronous initialization step has completed.

Như ta sẽ thấy trong phần tiếp theo, có một lựa chọn thứ ba cho phép ta trì hoãn mọi thao tác một cách minh bạch và hiệu quả cho đến khi bước khởi tạo bất đồng bộ đã hoàn tất.

Pre-initialization queues — Hàng đợi tiền khởi tạo

Another recipe to make sure that the services of a component are invoked only after the component is initialized involves the use of queues and the Command pattern. The idea is to queue the method invocations (only those requiring the component to be initialized) received while the component is not yet initialized, and then execute them as soon as all the initialization steps have been completed.

Một công thức khác để đảm bảo các dịch vụ của một thành phần chỉ được gọi sau khi thành phần đã được khởi tạo liên quan đến việc dùng hàng đợi và mẫu Command. Ý tưởng là xếp hàng các lời gọi phương thức (chỉ những lời gọi đòi hỏi thành phần phải được khởi tạo) nhận được trong khi thành phần chưa được khởi tạo, rồi thực thi chúng ngay khi tất cả các bước khởi tạo đã hoàn tất.

Let's see how this technique can be applied to our sample db component:

Hãy xem kỹ thuật này có thể được áp dụng cho thành phần db mẫu của ta như thế nào:

```
import { EventEmitter } from 'events'
class DB extends EventEmitter {
  connected = false
  commandsQueue = []
  async query (queryString) {
    if (!this.connected) {
      console.log(`Request queued: ${queryString}`)
      return new Promise((resolve, reject) => { // (1)
        const command = () => {
          this.query(queryString)
            .then(resolve, reject)
        }
        this.commandsQueue.push(command)
      })
    }
    console.log(`Query executed: ${queryString}`)
  }
  connect () {
    // simulate the delay of the connection
    setTimeout(() => {
      this.connected = true
      this.emit('connected')
      this.commandsQueue.forEach(command => command()) // (2)
      this.commandsQueue = []
    }, 1000)
  }
}
```

```

    }, 500)
  }
}
export const db = new DB()

```

As we already mentioned, the technique described here consists of two parts:

Như đã đề cập, kỹ thuật được mô tả ở đây gồm hai phần:

1. If the component has not been initialized—which, in our case, is when the `connected` property is `false`—we create a command from the parameters received with the current invocation and push it to the `commandsQueue` array. When the command is executed, it will run the original `query()` method again and forward the result to the `**Promise**` we are returning to the caller.
2. When the initialization of the component is completed—which, in our case, means that the connection with the database server is established—we go through the `commandsQueue`, executing all the commands that have been previously queued.
 1. Nếu thành phần chưa được khởi tạo - trong trường hợp của ta là khi thuộc tính `connected` là `false` - ta tạo một command từ các tham số nhận được với lời gọi hiện tại và đẩy nó vào mảng `commandsQueue`. Khi command được thực thi, nó sẽ chạy lại phương thức `query()` gốc và chuyển tiếp kết quả tới `Promise` mà ta đang trả về cho phía gọi.
 2. Khi quá trình khởi tạo của thành phần hoàn tất - trong trường hợp của ta nghĩa là kết nối với máy chủ cơ sở dữ liệu đã được thiết lập - ta duyệt qua `commandsQueue`, thực thi tất cả các command đã được xếp hàng trước đó.

With the `DB` class we just implemented, there is no need to check if the component is initialized before invoking its methods. In fact, all the logic is embedded in the component itself and any consumer can just transparently use it without worrying about its initialization status.

Với lớp `DB` ta vừa hiện thực, không cần phải kiểm tra xem thành phần đã được khởi tạo chưa trước khi gọi các phương thức của nó. Thực tế, toàn bộ logic được nhúng ngay trong chính thành phần và bất kỳ bên tiêu thụ nào cũng có thể dùng nó một cách minh bạch mà không cần bận tâm đến trạng thái khởi tạo của nó.

We can also go a step further and try to reduce the boilerplate of the `DB` class we just created and, at the same time, improve its modularity. We can achieve that by applying the **State** pattern, which we learned about in Chapter 9, Behavioral Design Patterns, with two states:

Ta cũng có thể tiến thêm một bước nữa và cố gắng giảm bớt mã soạn sẵn của lớp `DB` ta vừa tạo, đồng thời cải thiện tính module hóa của nó. Ta có thể đạt được điều đó bằng cách áp dụng mẫu `State`, mà ta đã học ở Chương 9, Behavioral Design Patterns, với hai trạng thái:

- The first state implements all the methods that require the component to be initialized, and it's activated only when there is a successful initialization. Each of these methods implements its own business logic without worrying about the initialization status of the `db` component
- The second state is activated before the initialization has completed and it implements the same methods as the first state, but their only role here is to add a new command to the queue using the parameters passed to the invocation.
 - Trạng thái thứ nhất hiện thực tất cả các phương thức đòi hỏi thành phần phải được khởi tạo, và nó chỉ được kích hoạt khi việc khởi tạo thành công. Mỗi phương thức này hiện thực logic nghiệp vụ riêng mà không cần bận tâm đến trạng thái khởi tạo của thành phần `db`

- Trạng thái thứ hai được kích hoạt trước khi việc khởi tạo hoàn tất và nó hiện thực cùng các phương thức như trạng thái thứ nhất, nhưng vai trò duy nhất của chúng ở đây là thêm một command mới vào hàng đợi dùng các tham số được truyền cho lời gọi.

Let's see how we can apply the structure we just described to our db component. First, we create the `InitializedState`, which implements the actual business logic of our component:

Hãy xem ta có thể áp dụng cấu trúc vừa mô tả vào thành phần db của mình như thế nào. Trước tiên, ta tạo `InitializedState`, lớp hiện thực logic nghiệp vụ thực sự của thành phần:

```
class InitializedState {
  async query (queryString) {
    console.log(`Query executed: ${queryString}`)
  }
}
```

As we can see, the only method that we need to implement in the `InitializedState` class is the `query()` method, which will print a message to the console when it receives a new query. Next, we implement the `QueuingState`, the core of our recipe. This state implements the queuing logic:

Như ta thấy, phương thức duy nhất ta cần hiện thực trong lớp `InitializedState` là phương thức `query()`, sẽ in một thông báo ra console khi nó nhận được một truy vấn mới. Tiếp theo, ta hiện thực `QueuingState`, phần cốt lõi của công thức. Trạng thái này hiện thực logic xếp hàng:

```
const METHODS_REQUIRING_CONNECTION = ['query']
const deactivate = Symbol('deactivate')
class QueuingState {
  constructor (db) {
    this.db = db
    this.commandsQueue = []
    METHODS_REQUIRING_CONNECTION.forEach(methodName => {
      this[methodName] = function (...args) {
        console.log('Command queued:', methodName, args)
        return new Promise((resolve, reject) => {
          const command = () => {
            db[methodName](...args)
              .then(resolve, reject)
          }
          this.commandsQueue.push(command)
        })
      }
    })
  }
  [deactivate] () {
    this.commandsQueue.forEach(command => command())
    this.commandsQueue = []
  }
}
```

It's interesting to note how the `QueuingState` is mostly built dynamically at creation time. For each method that requires an active connection, we create a new method for the current instance, which

queues a new command representing the function invocation. When the command is executed at a later time, when a connection is established, the result of the invocation of the method on the db instance is forwarded to the caller (through the returned promise).

Thật thú vị khi lưu ý cách `QueuingState` chủ yếu được xây dựng động tại thời điểm tạo. Với mỗi phương thức đòi hỏi một kết nối đang hoạt động, ta tạo một phương thức mới cho thực thể (instance) hiện tại, phương thức này xếp hàng một command mới đại diện cho lời gọi hàm. Khi command được thực thi vào lúc sau, khi một kết nối đã được thiết lập, kết quả của việc gọi phương thức trên thực thể db được chuyển tiếp tới phía gọi (thông qua promise được trả về).

The other important part of this state class is `[deactivate]()`. This method is invoked when the state is deactivated (which is when the component is initialized) and it executes all the commands in the queue. Note how we used a `Symbol` to name the method. This will avoid any name clashes in the future if we add more methods to the state (for example, what if we need to decorate a hypothetical `deactivate()` method of the DB class?).

Phần quan trọng khác của lớp trạng thái này là `[deactivate]()`. Phương thức này được gọi khi trạng thái bị vô hiệu hóa (đó là khi thành phần được khởi tạo) và nó thực thi tất cả các command trong hàng đợi. Lưu ý cách ta dùng một `Symbol` để đặt tên cho phương thức. Điều này sẽ tránh mọi xung đột tên trong tương lai nếu ta thêm các phương thức khác vào trạng thái (ví dụ, sẽ ra sao nếu ta cần trang trí (decorate) một phương thức `deactivate()` giả định của lớp DB?).

Now, it's time to reimplement the DB class using the two states we just described:

Bây giờ, đã đến lúc hiện thực lại lớp DB dùng hai trạng thái ta vừa mô tả:

```
class DB extends EventEmitter {
  constructor () {
    super()
    this.state = new QueuingState(this) // (1)
  }
  async query (queryString) {
    return this.state.query(queryString) // (2)
  }
  connect () {
    // simulate the delay of the connection
    setTimeout(() => {
      this.connected = true
      this.emit('connected')
      const oldState = this.state // (3)
      this.state = new InitializedState(this)
      oldState[deactivate] && oldState[deactivate]()
    }, 500)
  }
}
export const db = new DB()
```

Let's further analyze the most important parts of the new DB class:

Hãy phân tích kỹ hơn những phần quan trọng nhất của lớp DB mới:

1. In the constructor, we initialize the current state of the instance. It's going to be the `QueuingState` as the asynchronous initialization of the component hasn't been completed

yet.

2. The only method of our class implementing some (stub) business logic is the `query()` method. Here, all we have to do is invoke the homonymous method on the currently active state.
 3. Finally, when we establish the connection with the database (initialization complete), we switch the current state to the `InitializedState` and we deactivate the old one. The effect of deactivating the `QueuedState`, as we've seen previously, is that any command that had been queued is now executed.
1. Trong hàm khởi tạo, ta khởi tạo state hiện tại của thực thể. Nó sẽ là `QueuingState` vì quá trình khởi tạo bất đồng bộ của thành phần chưa hoàn tất.
 2. Phương thức duy nhất của lớp ta hiện thực một chút logic nghiệp vụ (dạng stub) là phương thức `query()`. Ở đây, tất cả những gì ta phải làm là gọi phương thức cùng tên trên trạng thái đang được kích hoạt.
 3. Cuối cùng, khi ta thiết lập kết nối với cơ sở dữ liệu (khởi tạo hoàn tất), ta chuyển trạng thái hiện tại sang `InitializedState` và vô hiệu hóa trạng thái cũ. Hệ quả của việc vô hiệu hóa `QueuedState`, như ta đã thấy trước đó, là mọi command đã được xếp hàng giờ đây được thực thi.

We can immediately see how this approach allows us to reduce the boilerplate and, at the same time, create a class that is purely business logic (the `InitializedState`) free from any repetitive initialization check.

Ta có thể ngay lập tức thấy cách tiếp cận này cho phép ta giảm mã soạn sẵn, đồng thời tạo ra một lớp thuần túy là logic nghiệp vụ (lớp `InitializedState`) không còn bất kỳ kiểm tra khởi tạo lặp đi lặp lại nào.

The approach we've just seen will only work if we can modify the code of our asynchronously initialized component. In all those cases in which we can't make modifications to the component, we will need to create a wrapper or **proxy**, but the technique will be mostly similar to what we've seen here.

Cách tiếp cận ta vừa thấy chỉ hoạt động nếu ta có thể chỉnh sửa mã của thành phần được khởi tạo bất đồng bộ của mình. Trong tất cả những trường hợp ta không thể thay đổi thành phần, ta sẽ cần tạo một wrapper hoặc proxy, nhưng kỹ thuật sẽ phần lớn tương tự với những gì ta đã thấy ở đây.

In the wild — Trong thực tế

The pattern we just presented is used by many database drivers and ORM libraries. The most notable is Mongoose ([nodejsdp.link/mongoose](https://mongoosejs.com/)), which is an ORM for MongoDB. With Mongoose, it's not necessary to wait for the database connection to open in order to be able to send queries. This is because each operation is queued and then executed later when the connection with the database is fully established, exactly as we've described in this section. This is clearly a must for any API that wants to provide a good developer experience (DX).

Mẫu ta vừa trình bày được nhiều trình điều khiển cơ sở dữ liệu và thư viện ORM sử dụng. Nổi bật nhất là Mongoose ([nodejsdp.link/mongoose](https://mongoosejs.com/)), một ORM cho MongoDB. Với Mongoose, không cần phải đợi kết nối cơ sở dữ liệu mở ra để có thể gửi truy vấn. Đó là vì mỗi thao tác được xếp hàng rồi thực thi sau đó khi kết nối với cơ sở dữ liệu đã được thiết lập đầy đủ, đúng như ta đã mô tả trong phần này. Đây rõ ràng là điều bắt buộc với bất kỳ API nào muốn cung cấp trải nghiệm lập trình viên (developer experience - DX) tốt.

Take a look at the code of Mongoose to see how every method in the native driver is proxied to add the pre-initialization queue. This also demonstrates an alternative way of implementing the recipe

we presented in this section. You can find the relevant code fragment at nodejsdp.link/mongoose-init-queue.

Hãy xem mã của Mongoose để thấy cách mọi phương thức trong trình điều khiển gốc (native driver) được proxy để thêm hàng đợi tiền khởi tạo. Điều này cũng minh họa một cách thay thế để hiện thực công thức ta đã trình bày trong phần này. Bạn có thể tìm đoạn mã liên quan tại nodejsdp.link/mongoose-init-queue.

Similarly, the **pg package** (nodejsdp.link/pg), which is a client for the PostgreSQL database, leverages pre-initialization queues, but in a slightly different fashion. **pg** queues every query, regardless of the initialization status of the database, and then immediately tries to execute all the commands in the queue. Take a look at the relevant code line at nodejsdp.link/pg-queue.

Tương tự, gói **pg** (nodejsdp.link/pg), một client cho cơ sở dữ liệu PostgreSQL, tận dụng hàng đợi tiền khởi tạo, nhưng theo một cách hơi khác. **pg** xếp hàng mọi truy vấn, bất kể trạng thái khởi tạo của cơ sở dữ liệu, rồi ngay lập tức cố gắng thực thi tất cả các command trong hàng đợi. Hãy xem dòng mã liên quan tại nodejsdp.link/pg-queue.

Asynchronous request batching and caching — Gộp lô và lưu đệm các yêu cầu bất đồng bộ

In high-load applications, caching plays a critical role and it's used almost everywhere on the web, from static resources such as web pages, images, and stylesheets, to pure data such as the result of database queries. In this section, we are going to learn how caching applies to asynchronous operations and how a high request throughput can be turned to our advantage.

Trong các ứng dụng tải cao, caching đóng vai trò then chốt và nó được dùng gần như ở khắp mọi nơi trên web, từ các tài nguyên tĩnh như trang web, hình ảnh và stylesheet, đến dữ liệu thuần túy như kết quả của các truy vấn cơ sở dữ liệu. Trong phần này, ta sẽ học cách caching áp dụng cho các thao tác bất đồng bộ và cách một thông lượng (throughput) yêu cầu cao có thể được biến thành lợi thế của ta.

What's asynchronous request batching? — Gộp lô yêu cầu bất đồng bộ là gì?

When dealing with asynchronous operations, the most basic level of caching can be achieved by batching together a set of invocations to the same API. The idea is very simple: if we invoke an asynchronous function while there is still another one pending, we can piggyback on the already running operation instead of creating a brand new request. Take a look at the following diagram:

Khi xử lý các thao tác bất đồng bộ, mức caching cơ bản nhất có thể đạt được bằng cách gộp lô (batching) một tập hợp các lời gọi tới cùng một API lại với nhau. Ý tưởng rất đơn giản: nếu ta gọi một hàm bất đồng bộ trong khi vẫn còn một hàm khác đang chờ xử lý, ta có thể bám theo (piggyback) thao tác đang chạy thay vì tạo một yêu cầu hoàn toàn mới. Hãy xem sơ đồ sau:

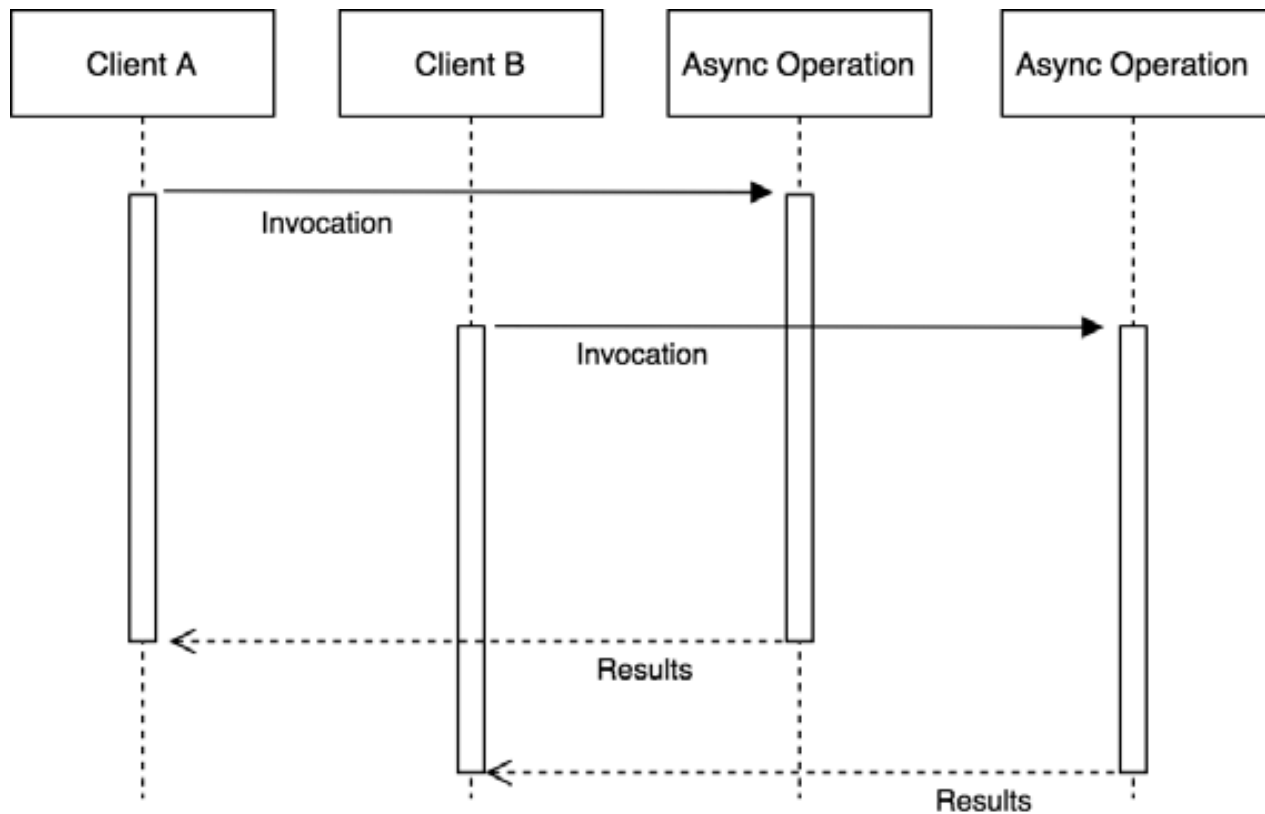


Figure 11.1: Two asynchronous requests with no batching

Hình 11.1: Hai yêu cầu bất đồng bộ không gộp lô

The previous diagram shows two clients invoking the same asynchronous operation with exactly the same input. Of course, the natural way to picture this situation is with the two clients starting two separate operations that will complete at two different moments. Now, consider the following scenario:

Sơ đồ trên cho thấy hai client gọi cùng một thao tác bất đồng bộ với chính xác cùng một đầu vào. Tất nhiên, cách tự nhiên để hình dung tình huống này là hai client bắt đầu hai thao tác riêng biệt sẽ hoàn tất ở hai thời điểm khác nhau. Bây giờ, hãy xét kịch bản sau:

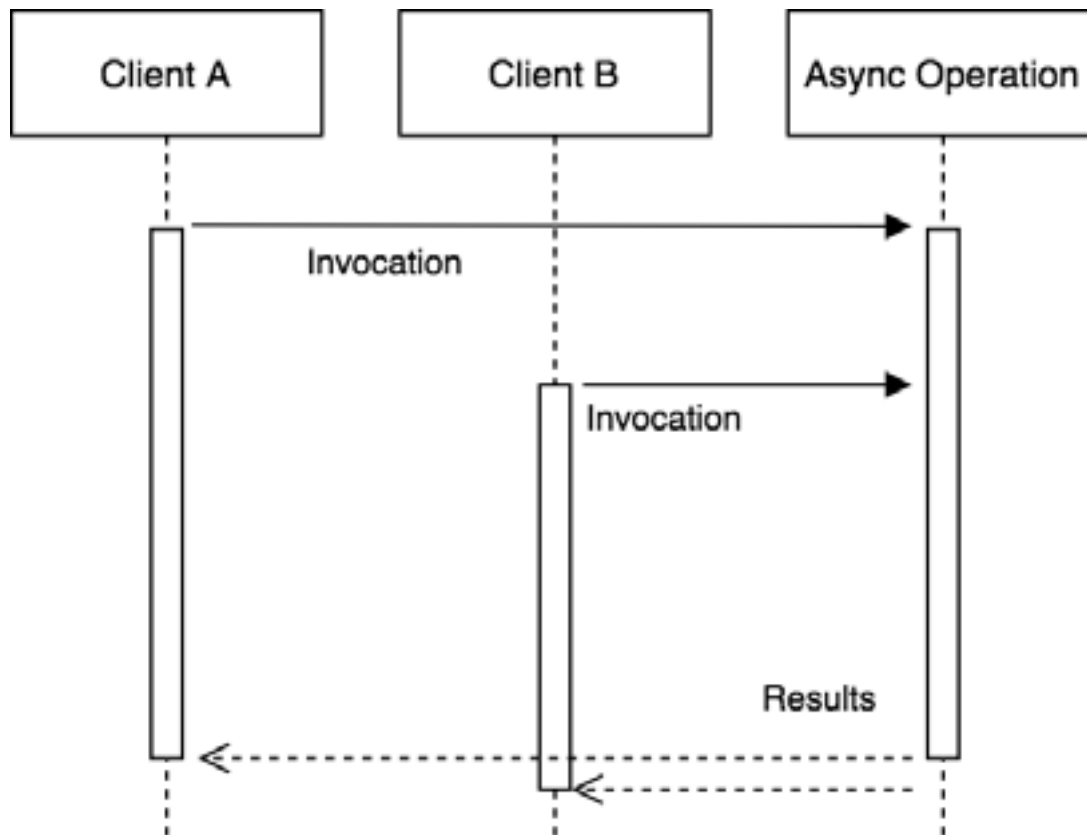


Figure 11.2: Batching of two asynchronous requests

Hình 11.2: Gộp lô hai yêu cầu bất đồng bộ

Figure 11.2 shows us how two identical requests—which invoke the same API with the same input—can be batched, or in other words, appended to the same running operation. By doing this, when the operation completes, both clients are notified, even though the **async** operation is actually executed only once. This represents a simple, yet extremely powerful, way to optimize the load of an application while not having to deal with more complex caching mechanisms, which usually require an adequate memory management and invalidation **strategy**.

Hình 11.2 cho ta thấy cách hai yêu cầu giống hệt nhau - cùng gọi một API với cùng đầu vào - có thể được gộp lô, hay nói cách khác, được gắn vào cùng một thao tác đang chạy. Bằng cách này, khi thao tác hoàn tất, cả hai client đều được thông báo, dù thao tác bất đồng bộ thực ra chỉ được thực thi một lần. Điều này đại diện cho một cách đơn giản nhưng cực kỳ mạnh mẽ để tối ưu tải của một ứng dụng mà không phải đối phó với các cơ chế caching phức tạp hơn, vốn thường đòi hỏi một chiến lược quản lý bộ nhớ và vô hiệu hóa (invalidation) thích hợp.

Optimal asynchronous request caching — Lưu đệm yêu cầu bất đồng bộ tối ưu

Request batching is less effective if the operation is fast enough or if matching requests are spread across a longer period of time. Also, most of the time, we can safely assume that the result of two identical API invocations will not change so often, so simple request batching will not provide the best performance. In all these circumstances, the best candidate to reduce the load of an application and increase its responsiveness is definitely a more aggressive caching mechanism. The idea is simple:

as soon as a request completes, we store its result in the cache, which can be an in-memory variable or an item in a specialized caching server (such as Redis). Hence, the next time the API is invoked, the result can be retrieved immediately from the cache, instead of spawning another request.

Gộp lô yêu cầu kém hiệu quả hơn nếu thao tác đủ nhanh hoặc nếu các yêu cầu trùng khớp trải dài trên một khoảng thời gian dài hơn. Ngoài ra, hầu hết thời gian, ta có thể giả định an toàn rằng kết quả của hai lời gọi API giống hệt nhau sẽ không thay đổi quá thường xuyên, do đó gộp lô yêu cầu đơn thuần sẽ không cho hiệu năng tốt nhất. Trong tất cả những trường hợp này, ứng cử viên tốt nhất để giảm tải cho ứng dụng và tăng khả năng đáp ứng của nó chắc chắn là một cơ chế caching quyết liệt hơn. Ý tưởng rất đơn giản: ngay khi một yêu cầu hoàn tất, ta lưu kết quả của nó vào cache, có thể là một biến trong bộ nhớ hoặc một mục trong máy chủ caching chuyên dụng (như Redis). Do đó, lần sau khi API được gọi, kết quả có thể được lấy ngay lập tức từ cache, thay vì sinh ra một yêu cầu khác.

The idea of caching should not be new to an experienced developer, but what makes this technique different in asynchronous programming is that it should be combined with request batching to be optimal. The reason for this is because multiple requests might run concurrently while the cache is not set and when those requests complete, the cache would be set multiple times.

Ý tưởng về caching không nên là điều mới mẻ với một lập trình viên có kinh nghiệm, nhưng điều làm cho kỹ thuật này khác biệt trong lập trình bất đồng bộ là nó nên được kết hợp với gộp lô yêu cầu để đạt tối ưu. Lý do là vì nhiều yêu cầu có thể chạy đồng thời trong khi cache chưa được thiết lập, và khi các yêu cầu đó hoàn tất, cache sẽ bị thiết lập nhiều lần.

Based on these assumptions, we can illustrate the Combined Request Batching and Caching pattern as follows:

Dựa trên các giả định này, ta có thể minh họa mẫu Gộp lô và Lưu đệm Yêu cầu Kết hợp (Combined Request Batching and Caching) như sau:

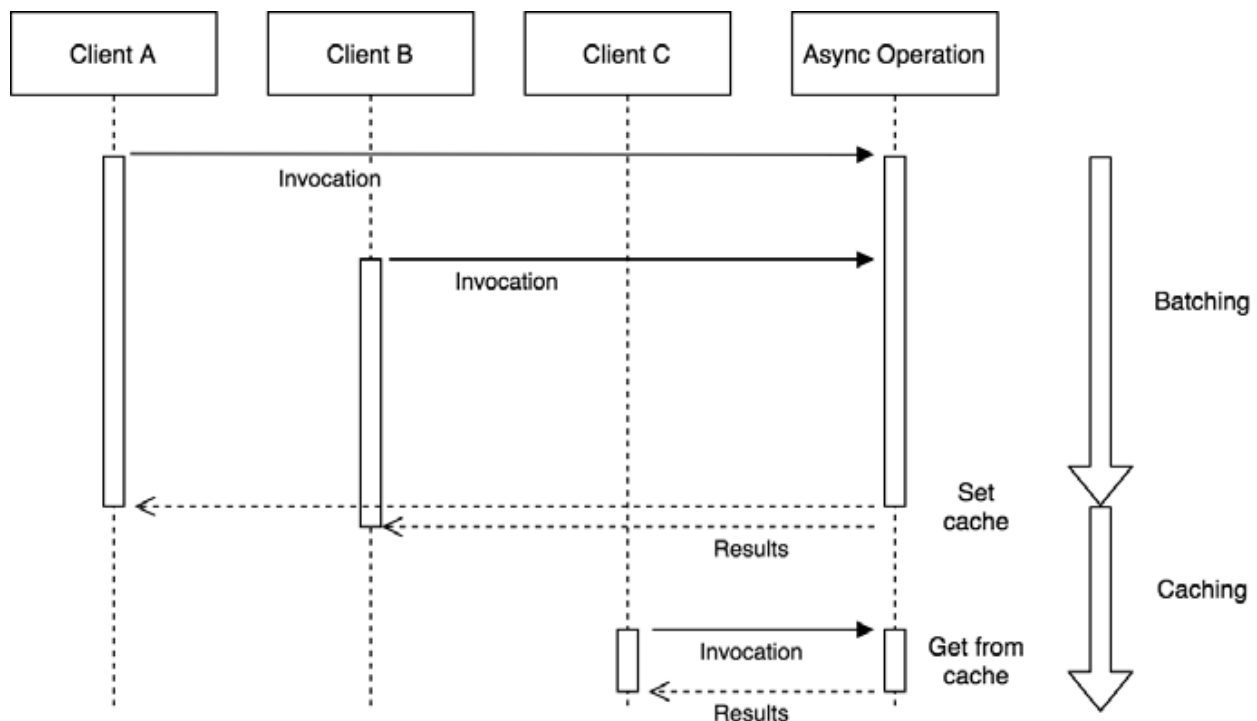


Figure 11.3: Combined batching and caching

Hình 11.3: Gộp lô và lưu đệm kết hợp

The preceding figure shows the two phases of an optimal asynchronous caching algorithm:

Hình trên cho thấy hai giai đoạn của một thuật toán caching bất đồng bộ tối ưu:

- The first phase is totally identical to the batching pattern. Any request received while the cache is not set will be batched together. When the request completes, the cache is set, once.
- When the cache is finally set, any subsequent request will be served directly from it.
 - Giai đoạn thứ nhất hoàn toàn giống với mẫu gộp lô. Bất kỳ yêu cầu nào nhận được trong khi cache chưa được thiết lập sẽ được gộp lô lại với nhau. Khi yêu cầu hoàn tất, cache được thiết lập một lần.
 - Khi cache cuối cùng đã được thiết lập, bất kỳ yêu cầu nào sau đó sẽ được phục vụ trực tiếp từ nó.

Another crucial detail to consider is the Zalgo anti-pattern (as we saw in Chapter 3, Callbacks and Events). Since we are dealing with asynchronous APIs, we must be sure to always return the cached value asynchronously, even if accessing the cache involves only a synchronous operation, such as in the case in which the cached value is retrieved from an in-memory variable.

Một chi tiết quan trọng khác cần xem xét là antipattern Zalgo (như ta đã thấy ở Chương 3, Callbacks and Events). Vì ta đang xử lý các API bất đồng bộ, ta phải đảm bảo luôn trả về giá trị đã cache một cách bất đồng bộ, ngay cả khi việc truy cập cache chỉ liên quan đến một thao tác đồng bộ, chẳng hạn như trong trường hợp giá trị đã cache được lấy từ một biến trong bộ nhớ.

An API server without caching or batching — Một máy chủ API không có caching hay batching

Before we start diving into this new challenge, let's implement a small demo server that we will use as a reference to measure the impact of the various techniques we are going to implement.

Trước khi bắt đầu đi sâu vào thử thách mới này, hãy hiện thực một máy chủ demo nhỏ mà ta sẽ dùng làm tham chiếu để đo lường tác động của các kỹ thuật khác nhau mà ta sắp hiện thực.

Let's consider an API server that manages the sales of an e-commerce company. In particular, we want to query our server for the sum of all the transactions of a particular type of merchandise. For this purpose, we are going to use a LevelUP database through the `level` npm package (nodejsdp.link/level). The data model that we are going to use is a simple list of transactions stored in the `sales` sublevel (a subsection of the database), which is organized in the following format:

Hãy xét một máy chủ API quản lý doanh số của một công ty thương mại điện tử. Cụ thể, ta muốn truy vấn máy chủ để lấy tổng tất cả các giao dịch của một loại hàng hóa cụ thể. Với mục đích này, ta sẽ dùng một cơ sở dữ liệu LevelUP thông qua gói npm `level` (nodejsdp.link/level). Mô hình dữ liệu ta sẽ dùng là một danh sách giao dịch đơn giản được lưu trong `sublevel sales` (một phần con của cơ sở dữ liệu), được tổ chức theo định dạng sau:

```
transactionId {amount, product}
```

The key is represented by `transactionId` and the value is a JSON object that contains the amount of the sale (`amount`) and the product type (`product`).

Khóa được biểu diễn bởi `transactionId` và giá trị là một đối tượng JSON chứa số tiền của giao dịch bán (`amount`) và loại sản phẩm (`product`).

The data to process is really basic, so let's implement a simple query over the database that we can use for our experiments. Let's say that we want to get the total amount of sales for a particular product. The routine would look as follows (file `totalSales.js`):

Dữ liệu cần xử lý thực sự rất cơ bản, nên hãy hiện thực một truy vấn đơn giản trên cơ sở dữ liệu mà ta có thể dùng cho các thí nghiệm của mình. Giả sử ta muốn lấy tổng số tiền bán hàng cho một sản phẩm cụ thể. Routine sẽ trông như sau (tệp `totalSales.js`):

```
import level from 'level'
import sublevel from 'subleveldown'
const db = level('example-db')
const salesDb = sublevel(db, 'sales', { valueEncoding: 'json' })
export async function totalSales (product) {
  const now = Date.now()
  let sum = 0
  for await (const transaction of salesDb.createValueStream()) {
    if (!product || transaction.product === product) {
      sum += transaction.amount
    }
  }
  console.log(`totalSales() took: ${Date.now() - now}ms`)
  return sum
}
```

The `totalSales()` function iterates over all the transactions of the sales sublevel and calculates the sum of the amounts of a particular product. The algorithm is intentionally slow as we want to highlight the effect of batching and caching later on. In a real-world application, we would have used an index to query the transactions by product or, even better, we could have used an incremental map/reduce algorithm to continuously calculate the sum for every product

Hàm `totalSales()` lặp qua tất cả các giao dịch của sublevel sales và tính tổng số tiền của một sản phẩm cụ thể. Thuật toán cố ý chậm vì ta muốn làm nổi bật tác động của batching và caching ở phần sau. Trong một ứng dụng thực tế, ta sẽ dùng một chỉ mục (index) để truy vấn các giao dịch theo product hoặc, tốt hơn nữa, ta có thể dùng một thuật toán map/reduce tăng dần để liên tục tính tổng cho mỗi sản phẩm

We can now expose the `totalSales()` API through a simple HTTP server (the `server.js` file):

Giờ ta có thể phơi bày API `totalSales()` thông qua một máy chủ HTTP đơn giản (tệp `server.js`):

```
import { createServer } from 'http'
import { totalSales } from './totalSales.js'
createServer(async (req, res) => {
  const url = new URL(req.url, 'http://localhost')
  const product = url.searchParams.get('product')
  console.log(`Processing query: ${url.search}`)
  const sum = await totalSales(product)
  res.setHeader('Content-Type', 'application/json')
  res.writeHead(200)
  res.end(JSON.stringify({
```

```
    product,  
    sum  
  ))  
}).listen(8000, () => console.log('Server started'))
```

Before we start the server for the first time, we need to populate the database with some sample data. We can do this with the `populateDb.js` script, which can be found in this book's code repository in the folder dedicated to this section. This script creates 100,000 random sales transactions in the database so that our query spends some time crunching data:

Trước khi khởi động máy chủ lần đầu tiên, ta cần điền vào cơ sở dữ liệu một số dữ liệu mẫu. Ta có thể làm điều này với script `populateDb.js`, có thể tìm thấy trong kho mã của cuốn sách này ở thư mục dành riêng cho phần này. Script này tạo 100.000 giao dịch bán hàng ngẫu nhiên trong cơ sở dữ liệu để truy vấn của ta tốn một chút thời gian để xử lý dữ liệu:

```
node populateDb.js
```

Okay! Now, everything is ready. Let's start the server:

Được rồi! Giờ thì mọi thứ đã sẵn sàng. Hãy khởi động máy chủ:

```
node server.js
```

To query the server, you can simply navigate with a browser to the following URL:

Để truy vấn máy chủ, bạn có thể chỉ cần dùng trình duyệt điều hướng tới URL sau:

```
http://localhost:8000?product=book
```

However, to have a better idea of the performance of our server, we will need more than one request. So, we will use a small script named `loadTest.js`, which sends 20 requests at intervals of 200 ms. The script can be found in the code repository of this book and it's already configured to connect to the local URL of the server, so, to run it, just execute the following command:

Tuy nhiên, để có hình dung tốt hơn về hiệu năng của máy chủ, ta sẽ cần nhiều hơn một yêu cầu. Vì vậy, ta sẽ dùng một script nhỏ tên là `loadTest.js`, gửi 20 yêu cầu cách nhau 200 ms. Script này có thể tìm thấy trong kho mã của cuốn sách và nó đã được cấu hình sẵn để kết nối tới URL cục bộ của máy chủ, nên để chạy nó, chỉ cần thực thi lệnh sau:

```
node loadTest.js
```

We will see that the 20 requests will take a while to complete. Take note of the total execution time of the test. Now, we are going to apply our optimizations and measure how much time we can save. We'll start by implementing both batching and caching by leveraging the properties of promises.

Ta sẽ thấy 20 yêu cầu sẽ mất một lúc để hoàn tất. Hãy ghi lại tổng thời gian thực thi của bài kiểm tra. Bây giờ, ta sẽ áp dụng các tối ưu hóa của mình và đo xem ta có thể tiết kiệm bao nhiêu thời gian. Ta sẽ bắt đầu bằng việc hiện thực cả batching và caching bằng cách tận dụng các thuộc tính của promise.

Batching and caching with promises — Batching và caching với promise

Promises are a great tool for implementing asynchronous batching and caching of requests. Let's see why.

Promise là một công cụ tuyệt vời để hiện thực batching và caching yêu cầu bất đồng bộ. Hãy xem tại sao.

If we recall what we learned about promises in Chapter 5, Asynchronous **Control Flow** Patterns with Promises and Async/**Await**, there are two properties that can be exploited to our advantage in this circumstance:

Nếu ta nhớ lại những gì đã học về promise ở Chương 5, Asynchronous Control Flow Patterns with Promises and Async/Await, có hai thuộc tính có thể được khai thác để có lợi cho ta trong tình huống này:

- Multiple `then()` listeners can be attached to the same promise.
- The `then()` listener is guaranteed to be invoked (only once), and it works even if it's attached after the promise is already resolved. Moreover, `then()` is guaranteed to always be invoked asynchronously.
 - Nhiều listener `then()` có thể được gắn vào cùng một promise.
 - Listener `then()` được đảm bảo sẽ được gọi (chỉ một lần), và nó hoạt động ngay cả khi nó được gắn sau khi promise đã được giải quyết (resolved). Hơn nữa, `then()` được đảm bảo luôn được gọi một cách bất đồng bộ.

In short, the first property is exactly what we need for batching requests, while the second means that a promise is already a cache for the resolved value and offers a natural mechanism for returning a cached value in a consistent, asynchronous way. In other words, this means that batching and caching become extremely simple and concise with promises.

Tóm lại, thuộc tính thứ nhất chính là điều ta cần để gộp lô yêu cầu, trong khi thuộc tính thứ hai nghĩa là một promise vốn đã là một cache cho giá trị đã resolve và cung cấp một cơ chế tự nhiên để trả về giá trị đã cache một cách nhất quán, bất đồng bộ. Nói cách khác, điều này có nghĩa là batching và caching trở nên cực kỳ đơn giản và ngắn gọn với promise.

Batching requests in the total sales web server — Gộp lô yêu cầu trong máy chủ web tổng doanh số

Let's now add a batching layer on top of our `totalSales` API. The pattern we are going to use is very simple: if there is another identical request pending when the API is invoked, we will wait for that request to complete instead of launching a new one. As we will see, this can easily be implemented with promises. In fact, all we have to do is save the promise in a map, associating it to the specified request parameters (the product type, in our case) every time we launch a new request. Then, at every subsequent request, we check if there is already a promise for the specified product and if there is one, we just return it; otherwise, we launch a new request.

Giờ hãy thêm một lớp batching lên trên API `totalSales` của ta. Mẫu ta sẽ dùng rất đơn giản: nếu có một yêu cầu giống hệt khác đang chờ xử lý khi API được gọi, ta sẽ đợi yêu cầu đó hoàn tất thay vì khởi chạy một yêu cầu mới. Như ta sẽ thấy, điều này có thể dễ dàng được hiện thực với promise. Thực tế, tất cả những gì ta phải làm là lưu promise vào một map, liên kết nó với các tham số yêu cầu được chỉ định (loại product, trong trường hợp của ta) mỗi khi ta khởi chạy một yêu cầu mới. Sau đó, ở mỗi yêu cầu tiếp theo, ta kiểm tra xem đã có một promise cho product được chỉ định chưa và nếu có, ta chỉ cần trả về nó; nếu không, ta khởi chạy một yêu cầu mới.

Now, let's see how this translates into code. Let's create a new module named `totalSalesBatch.js`. Here, we're going to implement a batching layer on top of the original `totalSales()` API:

Giờ hãy xem điều này được dịch thành mã như thế nào. Hãy tạo một module mới tên là `totalSalesBatch.js`. Ở đây, ta sẽ hiện thực một lớp batching lên trên API `totalSales()` gốc:

```
import { totalSales as totalSalesRaw } from './totalSales.js'
const runningRequests = new Map()
export function totalSales (product) {
  if (runningRequests.has(product)) { // (1)
    console.log('Batching')
    return runningRequests.get(product)
  }
  const resultPromise = totalSalesRaw(product) // (2)
  runningRequests.set(product, resultPromise)
  resultPromise.finally(() => {
    runningRequests.delete(product)
  })
  return resultPromise
}
```

The `totalSales()` function of the `totalSalesBatch` module is a proxy for the original `totalSales()` API, and it works as follows:

Hàm `totalSales()` của module `totalSalesBatch` là một proxy cho API `totalSales()` gốc, và nó hoạt động như sau:

1. If a promise for the given product already exists, we just return it. This is where we piggyback on an already running request.
2. If there is no request running for the given product, we execute the original `totalSales()` function and we save the resulting promise into the `runningRequests` map. Next, we make sure to remove the same promise from the `runningRequests` map as soon as the request completes.
 1. Nếu một promise cho product đã cho đã tồn tại, ta chỉ cần trả về nó. Đây là nơi ta bám theo (piggyback) một yêu cầu đang chạy.
 2. Nếu không có yêu cầu nào đang chạy cho product đã cho, ta thực thi hàm `totalSales()` gốc và lưu promise kết quả vào map `runningRequests`. Tiếp theo, ta đảm bảo xóa chính promise đó khỏi map `runningRequests` ngay khi yêu cầu hoàn tất.

The behavior of the new `totalSales()` function is identical to that of the original `totalSales()` API, with the difference that, now, multiple calls to the API using the same input are batched, thus saving us time and resources.

Hành vi của hàm `totalSales()` mới giống hệt với hành vi của API `totalSales()` gốc, với điểm khác biệt là giờ đây, nhiều lời gọi API dùng cùng một đầu vào được gộp lô, qua đó tiết kiệm cho ta thời gian và tài nguyên.

Curious to know what the performance improvement compared to the raw, non-batched version of the `totalSales()` API is? Let's then replace the `totalSales` module used by the HTTP server with the one we just created (the `app.js` file):

Tò mò muốn biết mức cải thiện hiệu năng so với phiên bản `totalSales()` API thô, không gộp lô là gì? Vậy thì hãy thay thế module `totalSales` được máy chủ HTTP dùng bằng module ta vừa tạo (tệp `app.js`):

```
// import { totalSales } from './totalSales.js'
import { totalSales } from './totalSalesBatch.js'
createServer(async (req, res) => {
  // ...
```

If we now try to start the server again and run the load test against it, the first thing we will see is that the requests are returned in batches. This is the effect of the recipe we just implemented, and it's a great practical demonstration of how it works.

Nếu giờ ta thử khởi động lại máy chủ và chạy bài kiểm tra tải đối với nó, điều đầu tiên ta sẽ thấy là các yêu cầu được trả về theo từng lô. Đây là hệ quả của công thức ta vừa hiện thực, và nó là một minh chứng thực tế tuyệt vời về cách nó hoạt động.

Besides that, we should also observe a considerable reduction in the total time for executing the test. It should be at least four times faster than the original test performed against the plain `totalSales()` API!

Bên cạnh đó, ta cũng sẽ quan sát thấy một sự giảm đáng kể trong tổng thời gian thực thi bài kiểm tra. Nó phải nhanh hơn ít nhất bốn lần so với bài kiểm tra gốc thực hiện đối với API `totalSales()` thuần!

This result substantiates the huge performance boost that we can obtain by just applying a simple batching layer, without all the complexity of managing a full-fledged cache and, more importantly, without worrying about invalidation strategies.

Kết quả này chứng thực cú hích hiệu năng to lớn mà ta có thể đạt được chỉ bằng cách áp dụng một lớp batching đơn giản, mà không cần toàn bộ sự phức tạp của việc quản lý một cache đầy đủ, và quan trọng hơn, mà không phải lo lắng về các chiến lược vô hiệu hóa.

The Request Batching pattern reaches its best potential in highload applications and with slow APIs. This is because it's exactly in these circumstances that we can batch together a high number of requests.

Mẫu Gộp lô Yêu cầu (Request Batching) đạt tiềm năng tốt nhất trong các ứng dụng tải cao và với các API chậm. Đó là vì chính trong những hoàn cảnh này, ta có thể gộp lô một số lượng lớn yêu cầu lại với nhau.

Let's now see how we can implement both batching and caching using a slight variation of the technique we've just explored.

Giờ hãy xem cách ta có thể hiện thực cả batching và caching bằng một biến thể nhỏ của kỹ thuật ta vừa khám phá.

Caching requests in the total sales web server — Lưu đệm yêu cầu trong máy chủ web tổng doanh số

Adding a caching layer to our batching API is straightforward, thanks to promises. All we have to do is leave the promise in our request map, even after the request has completed. Let's implement the `totalSalesCache.js` module straightaway:

Thêm một lớp caching vào API batching của ta rất dễ dàng, nhờ promise. Tất cả những gì ta phải làm là để promise lại trong map yêu cầu của ta, ngay cả sau khi yêu cầu đã hoàn tất. Hãy hiện thực module `totalSalesCache.js` ngay:

```
import { totalSales as totalSalesRaw } from './totalSales.js'
const CACHE_TTL = 30 * 1000 // 30 seconds TTL
const cache = new Map()
export function totalSales (product) {
  if (cache.has(product)) {
    console.log('Cache hit')
    return cache.get(product)
```



```

}
const resultPromise = totalSalesRaw(product)
cache.set(product, resultPromise)
resultPromise.then(() => {
  setTimeout(() => {
    cache.delete(product)
  }, CACHE_TTL)
}, err => {
  cache.delete(product)
  throw err
})
return resultPromise
}

```

The relevant code that enables caching is highlighted. All we have to do is remove the promise from the cache after a certain time (CACHE_TTL) after the request has completed, or immediately if the request has failed. This is a very basic cache invalidation technique, but it works perfectly for our demonstration.

Mã liên quan kích hoạt caching được tô sáng. Tất cả những gì ta phải làm là xóa promise khỏi cache sau một khoảng thời gian nhất định (CACHE_TTL) sau khi yêu cầu đã hoàn tất, hoặc ngay lập tức nếu yêu cầu thất bại. Đây là một kỹ thuật vô hiệu hóa cache rất cơ bản, nhưng nó hoạt động hoàn hảo cho phần minh họa của ta.

Now, we are ready to try the `totalSales()` caching wrapper we just created. To do that, we only need to update the `app.js` module, as follows:

Giờ ta đã sẵn sàng thử wrapper caching của `totalSales()` mà ta vừa tạo. Để làm điều đó, ta chỉ cần cập nhật module `app.js`, như sau:

```

// import { totalSales } from './totalSales.js'
// import { totalSales } from './totalSalesBatch.js'
import { totalSales } from './totalSalesCache.js'
createServer(async (req, res) => {
  // ...

```

Now, the server can be started again and profiled using the `loadTest.js` script, as we did in the previous examples. With the default test parameters, we should see a 10% reduction in the execution time compared to simple batching. Of course, this is highly dependent on a lot of factors; for example, the number of requests received and the delay between one request and the other. The advantages of using caching over batching will be much more substantial when the number of requests is higher and spans a longer period of time.

Giờ máy chủ có thể được khởi động lại và đo profile bằng script `loadTest.js`, như ta đã làm trong các ví dụ trước. Với các tham số kiểm tra mặc định, ta sẽ thấy mức giảm 10% thời gian thực thi so với batching đơn thuần. Tất nhiên, điều này phụ thuộc rất nhiều vào nhiều yếu tố; ví dụ, số lượng yêu cầu nhận được và độ trễ giữa yêu cầu này với yêu cầu khác. Lợi thế của việc dùng caching so với batching sẽ đáng kể hơn nhiều khi số lượng yêu cầu cao hơn và trải dài trên một khoảng thời gian dài hơn.

Notes about implementing caching mechanisms — Ghi chú về việc hiện thực các cơ chế caching

We must remember that in real-life applications, we may want to use more advanced cache invalidation techniques and storage mechanisms. This is necessary for the following reasons:

Ta phải nhớ rằng trong các ứng dụng thực tế, ta có thể muốn dùng các kỹ thuật vô hiệu hóa cache và cơ chế lưu trữ tiên tiến hơn. Điều này cần thiết vì những lý do sau:

- A large amount of cached values can easily consume a lot of memory. In this case, a least recently used (LRU) or a first in first out (FIFO) policy can be applied to maintain constant memory utilization.
- When the application is distributed across multiple processes, keeping the cache in memory may produce different results across each server instance. If that's undesired for the particular application we are implementing, the solution is to use a shared store for the cache. This is also more performant than a simple in-memory solution as the cache is shared across multiple instances. Popular caching solutions include Redis (nodejsdp.link/redis) and Memcached (nodejsdp.link/memcached).
- A manual cache invalidation (for example, when the related non-cached value changes), as opposed to a timed expiry, can enable a longer-living cache and at the same time provide more up-to-date data, but, of course, it would be a lot more complex to manage. Let's not forget the famous quote by Phil Karlton (principal engineer at Netscape, Silicon Graphics, and more): "There are only two hard things in Computer Science: cache invalidation and naming things." With this, we conclude this section on request batching and caching. Next, we are going to learn how to tackle a tricky business: canceling asynchronous operations.
 - Một lượng lớn giá trị đã cache có thể dễ dàng tiêu tốn rất nhiều bộ nhớ. Trong trường hợp này, một chính sách ít được dùng gần đây nhất (least recently used - LRU) hoặc vào trước ra trước (first in first out - FIFO) có thể được áp dụng để duy trì mức sử dụng bộ nhớ không đổi.
 - Khi ứng dụng được phân tán trên nhiều tiến trình, việc giữ cache trong bộ nhớ có thể tạo ra các kết quả khác nhau giữa mỗi thực thể máy chủ. Nếu điều đó không mong muốn cho ứng dụng cụ thể ta đang hiện thực, giải pháp là dùng một kho lưu trữ chung (shared store) cho cache. Cách này cũng có hiệu năng cao hơn so với một giải pháp trong bộ nhớ đơn thuần vì cache được chia sẻ giữa nhiều thực thể. Các giải pháp caching phổ biến bao gồm Redis (nodejsdp.link/redis) và Memcached (nodejsdp.link/memcached).
 - Vô hiệu hóa cache thủ công (ví dụ, khi giá trị không cache liên quan thay đổi), trái ngược với hết hạn theo thời gian, có thể cho phép một cache tồn tại lâu hơn và đồng thời cung cấp dữ liệu cập nhật hơn, nhưng, tất nhiên, nó sẽ phức tạp hơn rất nhiều để quản lý. Đừng quên câu nói nổi tiếng của Phil Karlton (kỹ sư chính tại Netscape, Silicon Graphics, và hơn thế nữa): "Chỉ có hai điều khó trong Khoa học Máy tính: vô hiệu hóa cache và đặt tên cho mọi thứ." Với điều này, ta kết thúc phần về gộp lô và lưu đệm yêu cầu. Tiếp theo, ta sẽ học cách giải quyết một việc khó nhằn: hủy các thao tác bất đồng bộ.

Canceling asynchronous operations — Hủy các thao tác bất đồng bộ

Being able to stop a long-running operation is particularly useful if the operation has been canceled by the user or if it has become redundant. In multithreaded programming, we can just terminate

the **thread**, but on a **single-threaded** platform such as Node.js, things can get a little bit more complicated. In this section, we'll be talking about canceling asynchronous operations and not about canceling promises, which is a different matter altogether. By the way, the Promises/A+ standard doesn't include an API for canceling promises. However, you can use a third-party promise library such as bluebird if you need such a feature (more at nodejsdp.link/bluebird-cancelation). Note that canceling a promise doesn't mean that the operation the promise refers to will also be canceled; in fact, bluebird offers an `onCancel` **callback** in the promise constructor, in addition to `resolve` and `reject`, which can be used to cancel the underlying async operation when the promise is canceled. This is actually what this section is about.

Khả năng dừng một thao tác chạy lâu đặc biệt hữu ích nếu thao tác đã bị người dùng hủy hoặc nếu nó đã trở nên dư thừa. Trong lập trình đa luồng, ta chỉ cần chấm dứt luồng (thread), nhưng trên một nền tảng đơn luồng như Node.js, mọi thứ có thể trở nên phức tạp hơn một chút. Trong phần này, ta sẽ nói về việc hủy các thao tác bất đồng bộ chứ không phải về việc hủy promise, vốn là một vấn đề hoàn toàn khác. Nhân tiện, chuẩn Promises/A+ không bao gồm một API để hủy promise. Tuy nhiên, bạn có thể dùng một thư viện promise của bên thứ ba như bluebird nếu bạn cần tính năng đó (thêm tại nodejsdp.link/bluebird-cancelation). Lưu ý rằng việc hủy một promise không có nghĩa là thao tác mà promise đề cập đến cũng sẽ bị hủy; thực tế, bluebird cung cấp một callback `onCancel` trong hàm khởi tạo promise, ngoài `resolve` và `reject`, có thể được dùng để hủy thao tác bất đồng bộ bên dưới khi promise bị hủy. Đây thực ra chính là điều phần này nói đến.

A basic recipe for creating cancelable functions — Một công thức cơ bản để tạo các hàm có thể hủy

Actually, in asynchronous programming, the basic principle for canceling the execution of a function is very simple: we check if the operation has been canceled after every asynchronous call, and if that's the case, we prematurely quit the operation. Consider, for example, the following code:

Thực ra, trong lập trình bất đồng bộ, nguyên tắc cơ bản để hủy việc thực thi một hàm rất đơn giản: ta kiểm tra xem thao tác đã bị hủy chưa sau mỗi lời gọi bất đồng bộ, và nếu đúng vậy, ta thoát khỏi thao tác sớm. Hãy xét, ví dụ, đoạn mã sau:

```
import { asyncRoutine } from './asyncRoutine.js'
import { CancelError } from './cancelError.js'
async function cancelable (cancelObj) {
  const resA = await asyncRoutine('A')
  console.log(resA)
  if (cancelObj.cancelRequested) {
    throw new CancelError()
  }
  const resB = await asyncRoutine('B')
  console.log(resB)
  if (cancelObj.cancelRequested) {
    throw new CancelError()
  }
  const resC = await asyncRoutine('C')
  console.log(resC)
}
```

The `cancelable()` function receives, as input, an object (`cancelObj`) containing a single property

called `cancelRequested`. In the function, we check the `cancelRequested` property after every asynchronous call, and if that's true, we throw a special `CancelError` exception to interrupt the execution of the function.

Hàm `cancelable()` nhận vào, làm đầu vào, một đối tượng (`cancelObj`) chứa một thuộc tính duy nhất tên là `cancelRequested`. Trong hàm, ta kiểm tra thuộc tính `cancelRequested` sau mỗi lời gọi bất đồng bộ, và nếu nó là `true`, ta ném một ngoại lệ đặc biệt `CancelError` để ngắt việc thực thi của hàm.

The `asyncRoutine()` function is just a demo function that prints a string to the console and returns another string after 100 ms. You will find its full implementation, along with that of `CancelError`, in the code repository for this book.

Hàm `asyncRoutine()` chỉ là một hàm demo in một chuỗi ra console và trả về một chuỗi khác sau 100 ms. Bạn sẽ tìm thấy phần hiện thực đầy đủ của nó, cùng với phần hiện thực của `CancelError`, trong kho mã của cuốn sách này.

It's important to note that any code external to the `cancelable()` function will be able to set the `cancelRequested` property only after the `cancelable()` function gives back control to the **event loop**, which is usually when an asynchronous operation is awaited. This is why it's worth checking the `cancelRequested` property only after the completion of an asynchronous operation and not more often.

Điều quan trọng cần lưu ý là bất kỳ đoạn mã nào bên ngoài hàm `cancelable()` sẽ chỉ có thể thiết lập thuộc tính `cancelRequested` sau khi hàm `cancelable()` trả lại quyền điều khiển cho vòng lặp sự kiện (event loop), điều này thường xảy ra khi một thao tác bất đồng bộ được `await`. Đây là lý do tại sao đáng để kiểm tra thuộc tính `cancelRequested` chỉ sau khi hoàn tất một thao tác bất đồng bộ chứ không phải thường xuyên hơn.

The following code demonstrates how we can cancel the `cancelable()` function:

Đoạn mã sau minh họa cách ta có thể hủy hàm `cancelable()`:

```
const cancelObj = { cancelRequested: false }
cancelable(cancelObj)
  .catch(err => {
    if (err instanceof CancelError) {
      console.log('Function canceled')
    } else {
      console.error(err)
    }
  })
setTimeout(() => {
  cancelObj.cancelRequested = true
}, 100)
```

As we can see, all we have to do to cancel the function is set the `cancelObj.cancelRequested` property to true. This will cause the function to stop and throw a `CancelError`.

Như ta thấy, tất cả những gì ta phải làm để hủy hàm là thiết lập thuộc tính `cancelObj.cancelRequested` thành `true`. Điều này sẽ khiến hàm dừng lại và ném ra một `CancelError`.

Wrapping asynchronous invocations — Bao bọc các lời gọi bất đồng bộ

Creating and using a basic asynchronous cancelable function is very easy, but there is a lot of boilerplate involved. In fact, it involves so much extra code that it becomes hard to identify the actual business logic of the function.

Tạo và sử dụng một hàm có thể hủy bất đồng bộ cơ bản rất dễ, nhưng có rất nhiều mã soạn sẵn liên quan. Thực tế, nó liên quan đến nhiều mã thừa đến mức trở nên khó nhận ra logic nghiệp vụ thực sự của hàm.

We can reduce the boilerplate by including the cancelation logic inside a wrapping function, which we can use to invoke asynchronous routines. Such a wrapper would look as follows (the `cancelWrapper.js` file):

Ta có thể giảm bớt mã soạn sẵn bằng cách đưa logic hủy vào bên trong một hàm bao bọc (wrapping function), mà ta có thể dùng để gọi các routine bất đồng bộ. Một wrapper như vậy sẽ trông như sau (tệp `cancelWrapper.js`):

```
import { CancelError } from './cancelError.js'
export function createCancelWrapper () {
  let cancelRequested = false
  function cancel () {
    cancelRequested = true
  }
  function cancelWrapper (func, ...args) {
    if (cancelRequested) {
      return Promise.reject(new CancelError())
    }
    return func(...args)
  }
  return { cancelWrapper, cancel }
}
```

Our wrapper is created through a **factory** function called `createCancelWrapper()`. The factory returns two functions: the wrapper function (`cancelWrapper`) and a function to trigger the cancelation of the asynchronous operation (`cancel`). This allows us to create a wrapper function to wrap multiple asynchronous invocations and then use a single `cancel()` function to cancel all of them.

Wrapper của ta được tạo thông qua một hàm factory tên là `createCancelWrapper()`. Factory này trả về hai hàm: hàm wrapper (`cancelWrapper`) và một hàm để kích hoạt việc hủy thao tác bất đồng bộ (`cancel`). Điều này cho phép ta tạo một hàm wrapper để bao bọc nhiều lời gọi bất đồng bộ rồi dùng một hàm `cancel()` duy nhất để hủy tất cả chúng.

The `cancelWrapper()` function takes, as input, a function to invoke (`func`) and a set of parameters to pass to the function (`args`). The wrapper simply checks if a cancelation has been requested and if positive, it will return a promise rejected with a `CancelError` object as the rejection reason; otherwise, it will invoke `func`.

Hàm `cancelWrapper()` nhận vào, làm đầu vào, một hàm để gọi (`func`) và một tập hợp các tham số để truyền cho hàm (`args`). Wrapper chỉ đơn giản kiểm tra xem việc hủy có được yêu cầu hay không và nếu có, nó sẽ trả về một promise bị reject với một đối tượng `CancelError` làm lý do reject; nếu không, nó sẽ gọi `func`.

Let's now see how our wrapper factory can greatly improve the readability and modularity of our

cancelable() function:

Giờ hãy xem cách factory wrapper của ta có thể cải thiện đáng kể tính dễ đọc và tính module hóa của hàm cancelable():

```
import { asyncRoutine } from './asyncRoutine.js'
import { createCancelWrapper } from './cancelWrapper.js'
import { CancelError } from './cancelError.js'
async function cancelable (cancelWrapper) {
  const resA = await cancelWrapper(asyncRoutine, 'A')
  console.log(resA)
  const resB = await cancelWrapper(asyncRoutine, 'B')
  console.log(resB)
  const resC = await cancelWrapper(asyncRoutine, 'C')
  console.log(resC)
}
const { cancelWrapper, cancel } = createCancelWrapper()
cancelable(cancelWrapper)
  .catch(err => {
    if (err instanceof CancelError) {
      console.log('Function canceled')
    } else {
      console.error(err)
    }
  })
setTimeout(() => {
  cancel()
}, 100)
```

We can immediately see the benefits of using a wrapper function for implementing our cancelation logic. In fact, the cancelable() function is now much more concise and **readable**.

Ta có thể ngay lập tức thấy lợi ích của việc dùng một hàm wrapper để hiện thực logic hủy của mình. Thực tế, hàm cancelable() giờ đây ngắn gọn và dễ đọc hơn nhiều.

Cancelable async functions with generators — Các hàm bất đồng bộ có thể hủy với generator

The cancelable wrapper function we just created is already a big step ahead compared to embedding the cancelation logic directly in our code. However, it's still not ideal for two reasons: it is error prone (what if we forget to wrap one function?) and it still affects the readability of our code, which makes it not ideal for implementing cancelable asynchronous operations that are already large and complex.

Hàm wrapper cancelable ta vừa tạo đã là một bước tiến lớn so với việc nhúng logic hủy trực tiếp vào mã của ta. Tuy nhiên, nó vẫn chưa lý tưởng vì hai lý do: nó dễ gây lỗi (sẽ ra sao nếu ta quên bao bọc một hàm?) và nó vẫn ảnh hưởng đến tính dễ đọc của mã, điều khiến nó không lý tưởng để hiện thực các thao tác bất đồng bộ có thể hủy vốn đã lớn và phức tạp.

An even neater solution involves the use of generators. In Chapter 9, Behavioral Design Patterns, we introduced generators as a means to implement iterators. However, they are a very versatile tool and can be used to implement all sorts of algorithms. In this case, we will be using generators to build

a supervisor to control the asynchronous flow of a function. The result will be an asynchronous function that is transparently cancelable, whose behavior resembles an async function in which the `await` instruction is replaced by `yield`. Let's see how we can implement this cancelable function using generators (the `createAsyncCancelable.js` file):

Một giải pháp gọn gàng hơn nữa liên quan đến việc dùng generator. Ở Chương 9, Behavioral Design Patterns, ta đã giới thiệu generator như một phương tiện để hiện thực iterator. Tuy nhiên, chúng là một công cụ rất đa năng và có thể dùng để hiện thực đủ loại thuật toán. Trong trường hợp này, ta sẽ dùng generator để xây dựng một bộ giám sát (supervisor) nhằm điều khiển luồng bất đồng bộ của một hàm. Kết quả sẽ là một hàm bất đồng bộ có thể hủy một cách minh bạch, có hành vi giống một hàm async trong đó lệnh `await` được thay bằng `yield`. Hãy xem cách ta có thể hiện thực hàm có thể hủy này dùng generator (tệp `createAsyncCancelable.js`):

```
import { CancelError } from './cancelError.js'
export function createAsyncCancelable (generatorFunction) { // (1)
  return function asyncCancelable (...args) {
    const generatorObject = generatorFunction(...args) // (3)
    let cancelRequested = false
    function cancel () {
      cancelRequested = true
    }
    const promise = new Promise((resolve, reject) => {
      async function nextStep (prevResult) { // (4)
        if (cancelRequested) {
          return reject(new CancelError())
        }
        if (prevResult.done) {
          return resolve(prevResult.value)
        }
        try { // (5)
          nextStep(generatorObject.next(await prevResult.value))
        } catch (err) {
          try { // (6)
            nextStep(generatorObject.throw(err))
          } catch (err2) {
            reject(err2)
          }
        }
      }
      nextStep({})
    })
    return { promise, cancel } // (2)
  }
}
```

The `createAsyncCancelable()` function may seem complex, so let's analyze it in more detail:

Hàm `createAsyncCancelable()` có thể trông phức tạp, vậy hãy phân tích nó chi tiết hơn:

1. First, we should note that the `createAsyncCancelable()` function takes, as input, a **generator** function (the supervised function) and returns another function (`asyncCancelable()`) that wraps the generator function with our supervising logic. The `asyncCancelable()`

function is what we will use to invoke the asynchronous operation.

2. The `asyncCancelable()` function returns an object with two properties: a. The `promise` property, which contains the promise representing the eventual resolution (or rejection) of the asynchronous operation. b. The `cancel` property, which is a function that can be used to cancel the supervised asynchronous flow.
3. When invoked, the first task of `asyncCancelable()` is to invoke the generator function with the arguments received as input (`args`) and obtain a generator object, which we can use to control the execution flow of the running coroutine.
4. The entire logic of the supervisor is implemented within the `nextStep()` function, which is responsible for iterating over the values yielded by the supervised coroutine (`prevResult`). Those can be actual values or promises. If a cancelation is requested, we throw the usual `CancelError`; otherwise, if the coroutine has been terminated (for example, `prevResult.done` is `true`), we immediately resolve the outer promise and complete the return.
5. The core part of the `nextStep()` function is where we retrieve the next value yielded by the supervised coroutine (which, let's not forget, it's a generator). We `await` on that value so we can make sure we get the actual resolution value in case we are dealing with a promise. This also makes sure that if `prevResult.value` is a promise and it rejects, we end up in the `catch` statement. We can end up in the `catch` statement even if the supervised coroutine actually throws an exception.
6. In the `catch` statement, we throw the caught error inside the coroutine. This is redundant if that error was already thrown by the coroutine, but not if it's the result of a promise rejection. Even if not optimal, this trick can simplify our code a bit for the sake of this demonstration. We invoke `nextStep()` using whatever value is yielded next after throwing it inside the coroutine, but if the result is another exception (for example, the exception is not caught inside the coroutine or another one is thrown), we immediately reject the outer promise and complete the asynchronous operation.

1. Trước tiên, ta nên lưu ý rằng hàm `createAsyncCancelable()` nhận vào, làm đầu vào, một hàm generator (hàm được giám sát) và trả về một hàm khác (`asyncCancelable()`) bao bọc hàm generator với logic giám sát của ta. Hàm `asyncCancelable()` là thứ ta sẽ dùng để gọi thao tác bất đồng bộ.
2. Hàm `asyncCancelable()` trả về một đối tượng với hai thuộc tính: a. Thuộc tính `promise`, chứa promise đại diện cho việc giải quyết (hoặc reject) cuối cùng của thao tác bất đồng bộ. b. Thuộc tính `cancel`, là một hàm có thể dùng để hủy luồng bất đồng bộ được giám sát.
3. Khi được gọi, nhiệm vụ đầu tiên của `asyncCancelable()` là gọi hàm generator với các đối số nhận được làm đầu vào (`args`) và lấy về một đối tượng generator, mà ta có thể dùng để điều khiển luồng thực thi của coroutine đang chạy.
4. Toàn bộ logic của bộ giám sát được hiện thực bên trong hàm `nextStep()`, chịu trách nhiệm lặp qua các giá trị được `yield` bởi coroutine được giám sát (`prevResult`). Đó có thể là các giá trị thực hoặc promise. Nếu một yêu cầu hủy được đưa ra, ta ném `CancelError` quen thuộc; nếu không, nếu coroutine đã kết thúc (ví dụ, `prevResult.done` là `true`), ta ngay lập tức `resolve` promise bên ngoài và hoàn tất việc trả về.
5. Phần cốt lõi của hàm `nextStep()` là nơi ta lấy về giá trị tiếp theo được `yield` bởi coroutine được giám sát (mà, đừng quên, nó là một generator). Ta `await` trên giá trị đó để có thể đảm bảo ta nhận được giá trị giải quyết thực sự trong trường hợp ta đang xử lý một promise. Điều này cũng đảm bảo rằng nếu `prevResult.value` là một promise và nó `reject`, ta sẽ rơi vào câu lệnh `catch`. Ta cũng có thể rơi vào câu lệnh `catch` ngay cả khi coroutine được giám sát thực sự ném ra một ngoại lệ.

- Trong câu lệnh `catch`, ta ném lỗi đã bắt được vào bên trong `coroutine`. Điều này dư thừa nếu lỗi đó đã được ném bởi `coroutine`, nhưng không dư thừa nếu nó là kết quả của một `promise` bị `reject`. Dù không tối ưu, mẹo này có thể đơn giản hóa mã của ta một chút cho mục đích minh họa này. Ta gọi `nextStep()` dùng bất kỳ giá trị nào được `yield` tiếp theo sau khi ném nó vào bên trong `coroutine`, nhưng nếu kết quả là một ngoại lệ khác (ví dụ, ngoại lệ không được bắt bên trong `coroutine` hoặc một ngoại lệ khác được ném ra), ta ngay lập tức `reject promise` bên ngoài và hoàn tất thao tác bất đồng bộ.

As we saw, there are a lot of moving parts in the `createAsyncCancelable()` function. But we should appreciate the fact that, in just a few lines of code, we were able to create a cancelable function that doesn't require any manual cancelation logic. As we will see now, the results are impressive.

Như ta đã thấy, có rất nhiều thành phần chuyển động trong hàm `createAsyncCancelable()`. Nhưng ta nên trân trọng thực tế rằng, chỉ trong vài dòng mã, ta đã có thể tạo một hàm có thể hủy không đòi hỏi bất kỳ logic hủy thủ công nào. Như ta sẽ thấy ngay bây giờ, kết quả thật ấn tượng.

Let's rewrite our sample asynchronous cancelable operation using the supervisor we implemented in the `createAsyncCancelable()` function:

Hãy viết lại thao tác bất đồng bộ có thể hủy mẫu của ta dùng bộ giám sát ta đã hiện thực trong hàm `createAsyncCancelable()`:

```
import { asyncRoutine } from './asyncRoutine.js'
import { createAsyncCancelable } from './createAsyncCancelable.js'
import { CancelError } from './cancelError.js'
const cancelable = createAsyncCancelable(function * () {
  const resA = yield asyncRoutine('A')
  console.log(resA)
  const resB = yield asyncRoutine('B')
  console.log(resB)
  const resC = yield asyncRoutine('C')
  console.log(resC)
})
const { promise, cancel } = cancelable()
promise.catch(err => {
  if (err instanceof CancelError) {
    console.log('Function canceled')
  } else {
    console.error(err)
  }
})
setTimeout(() => {
  cancel()
}, 100)
```

We can immediately see that the generator wrapped by `createAsyncCancelable()` closely resembles an async function, but we are using `yield` instead of `await`. Also, there is no visible cancelation logic at all. The generator function maintains the excellent properties of async functions (for example, to make asynchronous code look like synchronous code), but unlike the async function and thanks to the supervisor introduced by `createAsyncCancelable()`, it's also possible to cancel the operation. The second interesting aspect is that `createAsyncCancelable()` creates a function

(called cancelable) that can be invoked like any other function but at the same time returns a promise representing the result of the operation and a function to cancel the operation.

Ta có thể ngay lập tức thấy rằng generator được bao bọc bởi `createAsyncCancelable()` rất giống một hàm async, nhưng ta đang dùng `yield` thay vì `await`. Ngoài ra, hoàn toàn không có logic hủy nào hiện rõ. Hàm generator duy trì các thuộc tính tuyệt vời của các hàm async (ví dụ, làm cho mã bất đồng bộ trông như mã đồng bộ), nhưng khác với hàm async và nhờ bộ giám sát được giới thiệu bởi `createAsyncCancelable()`, nó cũng có thể hủy thao tác. Khía cạnh thú vị thứ hai là `createAsyncCancelable()` tạo ra một hàm (gọi là cancelable) có thể được gọi như bất kỳ hàm nào khác nhưng đồng thời trả về một promise đại diện cho kết quả của thao tác và một hàm để hủy thao tác.

This technique of using generators represents the best option we have to implement cancelable asynchronous operations.

Kỹ thuật dùng generator này đại diện cho lựa chọn tốt nhất ta có để hiện thực các thao tác bất đồng bộ có thể hủy.

For use in production, most of the time, we can rely on a widely used package from the Node.js ecosystem such as `caf` (the acronym means Cancelable Async Flows), which you can find at `nodejsdp.link/caf`.

Để dùng trong production, hầu hết thời gian, ta có thể dựa vào một gói được sử dụng rộng rãi từ hệ sinh thái Node.js như `caf` (viết tắt của Cancelable Async Flows), mà bạn có thể tìm thấy tại `nodejsdp.link/caf`.

Running CPU-bound tasks — Chạy các tác vụ chiếm dụng CPU

The `totalSales()` API that we implemented in the Asynchronous request batching and caching section was (intentionally) expensive in terms of resources and took a few hundred milliseconds to run. Nonetheless, invoking the `totalSales()` function did not affect the ability of the application to process concurrent incoming requests. What we learned about the event loop in Chapter 1, The Node.js Platform, should explain this behavior: invoking an asynchronous operation always causes the stack to unwind back to the event loop, leaving it free to handle other requests.

API `totalSales()` mà ta đã hiện thực trong phần Gộp lô và lưu đệm yêu cầu bất đồng bộ vốn (cố ý) tốn kém về mặt tài nguyên và mất vài trăm mili giây để chạy. Tuy nhiên, việc gọi hàm `totalSales()` không ảnh hưởng đến khả năng xử lý các yêu cầu đến đồng thời của ứng dụng. Những gì ta đã học về vòng lặp sự kiện ở Chương 1, The Node.js Platform, sẽ giải thích hành vi này: việc gọi một thao tác bất đồng bộ luôn khiến ngăn xếp (stack) cuộn lại về vòng lặp sự kiện, để nó rảnh rỗi xử lý các yêu cầu khác.

But what happens when we run a synchronous task that takes a long time to complete and that never gives back the control to the event loop until it has finished? This kind of task is also known as CPU-bound, because its main characteristic is that it is heavy on CPU utilization rather than being heavy on I/O operations.

Nhưng điều gì xảy ra khi ta chạy một tác vụ đồng bộ mất nhiều thời gian để hoàn tất và không bao giờ trả lại quyền điều khiển cho vòng lặp sự kiện cho đến khi nó kết thúc? Loại tác vụ này còn được gọi là chiếm dụng CPU (CPU-bound), vì đặc điểm chính của nó là nó nặng về mức sử dụng CPU chứ không phải nặng về các thao tác I/O.

Let's work immediately on an example to see how these types of task behave in Node.js.

Hãy bắt tay ngay vào một ví dụ để xem các loại tác vụ này hành xử như thế nào trong Node.js.

Solving the subset sum problem — Giải bài toán tổng tập con

Let's now choose a computationally expensive problem to use as a base for our experiment. A good candidate is the subset sum problem, which decides whether a set (or multiset) of integers contains a non-empty subset that has a sum equal to zero. For example, if we had as input the set [1, 2, -4, 5, -3], the subsets satisfying the problem are [1, 2, -3] and [2, -4, 5, -3]. The simplest algorithm is the one that checks every possible combination of subsets of any size and has a computational cost of $O(2^n)$, or in other words, it grows exponentially with the size of the input. This means that a set of 20 integers would require up to 1,048,576 combinations to be checked, which is not bad for testing our assumptions. For our example, we are going to consider the following variation of the subset sum problem: given a set of integers, we want to calculate all the possible combinations whose sum is equal to a given arbitrary integer, not just zero.

Giờ hãy chọn một bài toán tốn kém về mặt tính toán để dùng làm cơ sở cho thí nghiệm của ta. Một ứng cử viên tốt là bài toán tổng tập con (subset sum problem), quyết định xem một tập hợp (hoặc đa tập - multiset) các số nguyên có chứa một tập con không rỗng có tổng bằng không hay không. Ví dụ, nếu ta có đầu vào là tập hợp [1, 2, -4, 5, -3], các tập con thỏa mãn bài toán là [1, 2, -3] và [2, -4, 5, -3]. Thuật toán đơn giản nhất là thuật toán kiểm tra mọi tổ hợp có thể của các tập con với bất kỳ kích thước nào và có chi phí tính toán $O(2^n)$, hay nói cách khác, nó tăng theo cấp số mũ với kích thước của đầu vào. Điều này nghĩa là một tập hợp gồm 20 số nguyên sẽ đòi hỏi kiểm tra tới 1.048.576 tổ hợp, điều không tệ để kiểm chứng các giả định của ta. Cho ví dụ của mình, ta sẽ xét biến thể sau của bài toán tổng tập con: cho một tập hợp các số nguyên, ta muốn tính tất cả các tổ hợp có thể có tổng bằng một số nguyên tùy ý cho trước, chứ không chỉ bằng không.

Now, let's work to implement such an algorithm. First, let's create a new module called `subsetSum.js`. We will start by creating a class called `SubsetSum`:

Giờ hãy bắt tay vào hiện thực một thuật toán như vậy. Trước tiên, hãy tạo một module mới tên là `subsetSum.js`. Ta sẽ bắt đầu bằng việc tạo một lớp tên là `SubsetSum`:

```
export class SubsetSum extends EventEmitter {
  constructor (sum, set) {
    super()
    this.sum = sum
    this.set = set
    this.totalSubsets = 0
  }
  //...
```

The `SubsetSum` class is extending `EventEmitter`. This allows us to produce an event every time we find a new subset matching the sum received as input. As we will see, this will give us a lot of flexibility.

Lớp `SubsetSum` mở rộng `EventEmitter`. Điều này cho phép ta phát ra một sự kiện mỗi khi ta tìm thấy một tập con mới khớp với tổng nhận được làm đầu vào. Như ta sẽ thấy, điều này sẽ cho ta rất nhiều sự linh hoạt.

Next, let's see how we can generate all the possible combinations of subsets:

Tiếp theo, hãy xem cách ta có thể sinh tất cả các tổ hợp có thể của các tập con:

```

_combine (set, subset) {
  for (let i = 0; i < set.length; i++) {
    const newSubset = subset.concat(set[i])
    this._combine(set.slice(i + 1), newSubset)
    this._processSubset(newSubset)
  }
}

```

We will not go into too much detail about the algorithm, but there are two important things to notice:

Ta sẽ không đi quá sâu vào chi tiết về thuật toán, nhưng có hai điều quan trọng cần lưu ý:

- The `_combine()` method is completely synchronous. It recursively generates every possible subset without ever giving back control to the event loop.
- Every time a new combination is generated, we provide it to the `_processSubset()` method for further processing.
 - Phương thức `_combine()` hoàn toàn đồng bộ. Nó đệ quy sinh ra mọi tập con có thể mà không bao giờ trả lại quyền điều khiển cho vòng lặp sự kiện.
 - Mỗi khi một tổ hợp mới được sinh ra, ta cung cấp nó cho phương thức `_processSubset()` để xử lý thêm.

The `_processSubset()` method is responsible for verifying that the sum of the elements of the given subset is equal to the number we are looking for:

Phương thức `_processSubset()` chịu trách nhiệm kiểm chứng rằng tổng các phần tử của tập con đã cho bằng với số ta đang tìm kiếm:

```

_processSubset (subset) {
  console.log('Subset', ++this.totalSubsets, subset)
  const res = subset.reduce((prev, item) => (prev + item), 0)
  if (res === this.sum) {
    this.emit('match', subset)
  }
}

```

Trivially, the `_processSubset()` method applies a `reduce` operation to the subset in order to calculate the sum of its elements. Then, it emits an event of the type `match` when the resulting sum is equal to the one we are interested in finding (`this.sum`).

Một cách đơn giản, phương thức `_processSubset()` áp dụng một thao tác `reduce` lên tập con để tính tổng các phần tử của nó. Sau đó, nó phát ra một sự kiện loại `match` khi tổng kết quả bằng với tổng ta muốn tìm (`this.sum`).

Finally, the `start()` method puts all the preceding pieces together:

Cuối cùng, phương thức `start()` ghép tất cả các phần trên lại với nhau:

```

start () {
  this._combine(this.set, [])
  this.emit('end')
}

```

The `start()` method triggers the generation of all the combinations by invoking `_combine()`, and lastly, emits an end event, signaling that all the combinations were checked and any possible match has already been emitted. This is possible because `_combine()` is synchronous; therefore, the end

event is emitted as soon as the function returns, which means that all the combinations have been calculated.

Phương thức `start()` kích hoạt việc sinh tất cả các tổ hợp bằng cách gọi `_combine()`, và cuối cùng, phát ra một sự kiện `end`, báo hiệu rằng tất cả các tổ hợp đã được kiểm tra và bất kỳ kết quả khớp nào có thể đã được phát ra. Điều này khả thi vì `_combine()` là đồng bộ; do đó, sự kiện `end` được phát ra ngay khi hàm trả về, nghĩa là tất cả các tổ hợp đã được tính toán.

Next, we have to expose the algorithm we just created over the network. As always, we can use a simple HTTP server for this task. In particular, we want to create an endpoint that accepts requests in the format `/subsetSum?data=<Array>&sum=<Integer>` that invokes the `SubsetSum` algorithm with the given array of integers and sum to match.

Tiếp theo, ta phải phơi bày thuật toán ta vừa tạo qua mạng. Như mọi khi, ta có thể dùng một máy chủ HTTP đơn giản cho tác vụ này. Cụ thể, ta muốn tạo một endpoint chấp nhận các yêu cầu theo định dạng `/subsetSum?data=<Array>&sum=<Integer>` gọi thuật toán `SubsetSum` với mảng số nguyên đã cho và tổng cần khớp.

Let's implement this simple server in a module named `index.js`:

Hãy hiện thực máy chủ đơn giản này trong một module tên là `index.js`:

```
import { createServer } from 'http'
import { SubsetSum } from './subsetSum.js'
createServer((req, res) => {
  const url = new URL(req.url, 'http://localhost')
  if (url.pathname !== '/subsetSum') {
    res.writeHead(200)
    return res.end('I\'m alive!\n')
  }
  const data = JSON.parse(url.searchParams.get('data'))
  const sum = JSON.parse(url.searchParams.get('sum'))
  res.writeHead(200)
  const subsetSum = new SubsetSum(sum, data)
  subsetSum.on('match', match => {
    res.write(`Match: ${JSON.stringify(match)}\n`)
  })
  subsetSum.on('end', () => res.end())
  subsetSum.start()
}).listen(8000, () => console.log('Server started'))
```

Thanks to the fact that the `SubsetSum` object returns its results using events, we can **stream** the matching subsets as soon as they are generated by the algorithm, in real time. Another detail to mention is that our server responds with the text `I'm alive!` every time we hit a URL different than `/subsetSum`. We will use this for checking the responsiveness of our server, as we will see in a moment.

Nhờ thực tế là đối tượng `SubsetSum` trả về kết quả của nó bằng sự kiện, ta có thể truyền (stream) các tập con khớp ngay khi chúng được sinh ra bởi thuật toán, trong thời gian thực. Một chi tiết khác cần đề cập là máy chủ của ta phản hồi với văn bản `I'm alive!` mỗi khi ta truy cập một URL khác với `/subsetSum`. Ta sẽ dùng điều này để kiểm tra khả năng đáp ứng của máy chủ, như ta sẽ thấy trong giây lát.

We are now ready to try our subset sum algorithm. Curious to know how our server will handle it? Let's fire it up, then:

Giờ ta đã sẵn sàng thử thuật toán tổng tập con của mình. Tò mò muốn biết máy chủ của ta sẽ xử lý nó như thế nào? Vậy thì hãy khởi động nó lên:

```
node index.js
```

As soon as the server starts, we are ready to send our first request. Let's try it with a multiset of 17 random numbers, which will result in the generation of 131,071 combinations, a nice amount to keep our server busy for a while:

Ngay khi máy chủ khởi động, ta đã sẵn sàng gửi yêu cầu đầu tiên. Hãy thử nó với một đa tập gồm 17 số ngẫu nhiên, điều sẽ dẫn đến việc sinh ra 131.071 tổ hợp, một lượng vừa đủ để giữ máy chủ của ta bận rộn một lúc:

```
curl -G http://localhost:8000/subsetSum --data-urlencode "data=[16,19,1,1,-16,9,1,-5,-2,17,-15,-97,19,-16,-4,-5,15]" --data-urlencode "sum=0"
```

After a few seconds, we should see the results coming from the server. But if we try the following command in another terminal while the first request is still running, we will spot a huge problem:

Sau vài giây, ta sẽ thấy các kết quả đến từ máy chủ. Nhưng nếu ta thử lệnh sau trong một terminal khác trong khi yêu cầu đầu tiên vẫn đang chạy, ta sẽ phát hiện ra một vấn đề lớn:

```
curl -G http://localhost:8000
```

We will immediately see that this last request hangs until the subset sum algorithm of the first request has finished: the server is unresponsive! This was actually expected. The Node.js event loop runs in a single thread, and if this thread is blocked by a long synchronous computation, it will be unable to execute even a single cycle in order to respond with a simple I'm alive! We quickly understand that this behavior does not work for any kind of application meant to process multiple concurrent requests. But don't despair. In Node.js, we can tackle this type of situation in several ways. So, let's analyze the three most popular methods, which are “interleaving with setImmediate,” “using external processes,” and “using worker threads.”

Ta sẽ ngay lập tức thấy rằng yêu cầu cuối này bị treo cho đến khi thuật toán tổng tập con của yêu cầu đầu tiên kết thúc: máy chủ không phản hồi! Điều này thực ra đã được dự đoán. Vòng lặp sự kiện của Node.js chạy trong một luồng đơn, và nếu luồng này bị chặn bởi một phép tính đồng bộ dài, nó sẽ không thể thực thi dù chỉ một chu kỳ để phản hồi với một câu I'm alive! đơn giản. Ta nhanh chóng hiểu rằng hành vi này không phù hợp với bất kỳ loại ứng dụng nào nhằm xử lý nhiều yêu cầu đồng thời. Nhưng đừng tuyệt vọng. Trong Node.js, ta có thể giải quyết loại tình huống này theo nhiều cách. Vậy, hãy phân tích ba phương pháp phổ biến nhất, đó là “ đan xen với setImmediate”, “dùng tiến trình bên ngoài”, và “dùng worker thread”.

Interleaving with setImmediate — Đan xen với setImmediate

Usually, a CPU-bound algorithm is built upon a set of steps. This can be a set of recursive invocations, a loop, or any variation/combination of these. So, a simple solution to our problem would be to give back the control to the event loop after each of these steps completes (or after a certain number of them). This way, any pending I/O can still be processed by the event loop in those intervals in which the long-running algorithm yields the CPU. A simple way to achieve this is to schedule the next step

of the algorithm to run after any pending I/O requests. This sounds like the perfect use case for the `setImmediate()` function (we already introduced this API in Chapter 3, *Callbacks and Events*).

Thông thường, một thuật toán chiếm dụng CPU được xây dựng dựa trên một tập hợp các bước. Đây có thể là một tập hợp các lời gọi đệ quy, một vòng lặp, hoặc bất kỳ biến thể/kết hợp nào của những thứ này. Vậy, một giải pháp đơn giản cho bài toán của ta là trả lại quyền điều khiển cho vòng lặp sự kiện sau khi mỗi bước này hoàn tất (hoặc sau một số bước nhất định). Bằng cách này, bất kỳ I/O đang chờ nào vẫn có thể được vòng lặp sự kiện xử lý trong những khoảng thời gian mà thuật toán chạy lâu nhường lại CPU. Một cách đơn giản để đạt được điều này là lên lịch chạy bước tiếp theo của thuật toán sau bất kỳ yêu cầu I/O đang chờ nào. Điều này nghe có vẻ như là trường hợp dùng hoàn hảo cho hàm `setImmediate()` (ta đã giới thiệu API này ở Chương 3, *Callbacks and Events*).

Interleaving the steps of the subset sum algorithm — Đan xen các bước của thuật toán tổng tập con

Let's now see how this technique applies to our subset sum algorithm. All we have to do is slightly modify the `subsetSum.js` module. For convenience, we are going to create a new module called `subsetSumDefer.js`, taking the code of the original `subsetSum` class as a starting point.

Giờ hãy xem kỹ thuật này áp dụng cho thuật toán tổng tập con của ta như thế nào. Tất cả những gì ta phải làm là chỉnh sửa nhẹ module `subsetSum.js`. Cho tiện, ta sẽ tạo một module mới tên là `subsetSumDefer.js`, lấy mã của lớp `subsetSum` gốc làm điểm khởi đầu.

The first change we are going to make is to add a new method called `_combineInterleaved()`, which is the core of the technique we are implementing:

Thay đổi đầu tiên ta sẽ thực hiện là thêm một phương thức mới tên là `_combineInterleaved()`, phần cốt lõi của kỹ thuật ta đang hiện thực:

```
_combineInterleaved (set, subset) {
  this.runningCombine++
  setImmediate(() => {
    this._combine(set, subset)
    if (--this.runningCombine === 0) {
      this.emit('end')
    }
  })
}
```

As we can see, all we had to do is defer the invocation of the original (synchronous) `_combine()` method with `setImmediate()`. However, now, it becomes more difficult to know when the function has finished generating all the combinations, because the algorithm is not synchronous anymore. To fix this, we have to keep track of all the running instances of the `_combine()` method using a pattern very similar to the asynchronous **parallel execution** flow that we saw in Chapter 4, *Asynchronous Control Flow Patterns with Callbacks*. When all the instances of the `_combine()` method have finished running, we can emit the end event, notifying any listener that the process has completed.

Như ta thấy, tất cả những gì ta phải làm là trì hoãn việc gọi phương thức `_combine()` (đồng bộ) gốc bằng `setImmediate()`. Tuy nhiên, giờ đây, việc biết khi nào hàm đã hoàn tất việc sinh tất cả các tổ hợp trở nên khó hơn, vì thuật toán không còn đồng bộ nữa. Để khắc phục điều này, ta phải theo dõi tất cả các thực thể đang chạy của phương thức `_combine()`

dùng một mẫu rất giống với luồng thực thi song song bất đồng bộ mà ta đã thấy ở Chương 4, Asynchronous Control Flow Patterns with Callbacks. Khi tất cả các thực thể của phương thức `_combine()` đã chạy xong, ta có thể phát ra sự kiện `end`, thông báo cho bất kỳ listener nào rằng tiến trình đã hoàn tất.

To finish refactoring the subset sum algorithm, we need to make a couple more tweaks. First, we need to replace the recursive step in the `_combine()` method with its deferred counterpart:

Để hoàn tất việc tái cấu trúc thuật toán tổng tập con, ta cần thực hiện thêm một vài điều chỉnh. Trước tiên, ta cần thay thế bước đệ quy trong phương thức `_combine()` bằng phiên bản trì hoãn của nó:

```
_combine (set, subset) {
  for (let i = 0; i < set.length; i++) {
    const newSubset = subset.concat(set[i])
    this._combineInterleaved(set.slice(i + 1), newSubset)
    this._processSubset(newSubset)
  }
}
```

With the preceding change, we make sure that each step of the algorithm will be queued in the event loop using `setImmediate()` and, therefore, executed after any pending I/O request instead of being run synchronously.

Với thay đổi trên, ta đảm bảo rằng mỗi bước của thuật toán sẽ được xếp hàng trong vòng lặp sự kiện dùng `setImmediate()` và, do đó, được thực thi sau bất kỳ yêu cầu I/O đang chờ nào thay vì được chạy đồng bộ.

The other small tweak is in the `start()` method:

Điều chỉnh nhỏ khác là trong phương thức `start()`:

```
start () {
  this.runningCombine = 0
  this._combineInterleaved(this.set, [])
}
```

In the preceding code, we initialized the number of running instances of the `_combine()` method to `0`. We also replaced the call to `_combine()` with a call to `_combineInterleaved()` and removed the emission of the end event because this is now handled asynchronously in `_combineInterleaved()`.

Trong đoạn mã trên, ta khởi tạo số lượng thực thể đang chạy của phương thức `_combine()` thành `0`. Ta cũng thay thế lời gọi tới `_combine()` bằng một lời gọi tới `_combineInterleaved()` và xóa việc phát ra sự kiện `end` vì điều này giờ được xử lý bất đồng bộ trong `_combineInterleaved()`.

With this last change, our subset sum algorithm should now be able to run its CPU-bound code in steps interleaved by intervals, where the event loop can run and process any other pending request.

Với thay đổi cuối này, thuật toán tổng tập con của ta giờ sẽ có thể chạy mã chiếm dụng CPU của nó theo các bước được đan xen bởi các khoảng thời gian, nơi vòng lặp sự kiện có thể chạy và xử lý bất kỳ yêu cầu đang chờ nào khác.

The last missing bit is updating the `index.js` module so that it can use the new version of the SubsetSum API. This is actually a trivial change:

Phần cuối còn thiếu là cập nhật module `index.js` để nó có thể dùng phiên bản mới của API SubsetSum. Đây thực ra là một thay đổi tầm thường:

```
import { createServer } from 'http'
// import { SubsetSum } from './subsetSum.js'
import { SubsetSum } from './subsetSumDefer.js'
createServer((req, res) => {
  // ...

```

We are now ready to try this new version of the subset sum server. Start the server again and then try to send a request to calculate all the subsets matching a given sum:

Giờ ta đã sẵn sàng thử phiên bản mới này của máy chủ tổng tập con. Khởi động lại máy chủ rồi thử gửi một yêu cầu để tính tất cả các tập con khớp với một tổng cho trước:

```
curl -G http://localhost:8000/subsetSum --data-urlencode "data=[16,
19,1,1,-16,9,1,-5,-2,17,-15,-97,19,-16,-4,-5,15]" --data-urlencode
"sum=0"
```

While the request is running, check again whether the server is responsive:

Trong khi yêu cầu đang chạy, hãy kiểm tra lại xem máy chủ có phản hồi không:

```
curl -G http://localhost:8000
```

Cool! The second request should return almost immediately, even while the SubsetSum task is still running, confirming that our technique is working well.

Tuyệt! Yêu cầu thứ hai sẽ trả về gần như ngay lập tức, ngay cả khi tác vụ SubsetSum vẫn đang chạy, xác nhận rằng kỹ thuật của ta hoạt động tốt.

Considerations on the interleaving approach — Những cân nhắc về cách tiếp cận đan xen

As we saw, running a CPU-bound task while preserving the responsiveness of an application is not that complicated; it just requires the use of `setImmediate()` to schedule the next step of an algorithm to run after any pending I/O. However, this is not the best recipe in terms of efficiency. In fact, deferring a task introduces a small overhead that, multiplied by all the steps that an algorithm has to run, can have a significant impact on the overall running time. This is usually the last thing we want when running a CPU-bound task. A possible solution to mitigate this problem would be using `setImmediate()` only after a certain number of steps—instead of using it at every single step—but still, this would not solve the root of the problem.

Như ta đã thấy, chạy một tác vụ chiếm dụng CPU trong khi vẫn giữ khả năng đáp ứng của một ứng dụng không quá phức tạp; nó chỉ đòi hỏi dùng `setImmediate()` để lên lịch chạy bước tiếp theo của thuật toán sau bất kỳ I/O đang chờ nào. Tuy nhiên, đây không phải là công thức tốt nhất về mặt hiệu quả. Thực tế, việc trì hoãn một tác vụ tạo ra một chi phí phụ nhỏ mà, khi nhân với tất cả các bước mà một thuật toán phải chạy, có thể có tác động đáng kể lên tổng thời gian chạy. Đây thường là điều cuối cùng ta muốn khi chạy một tác vụ chiếm dụng CPU. Một giải pháp khả thi để giảm nhẹ vấn đề này là chỉ dùng `setImmediate()` sau một số bước nhất định - thay vì dùng nó ở mỗi bước đơn lẻ - nhưng dù vậy, điều này sẽ không giải quyết được gốc rễ của vấn đề.

Also, this technique doesn't work very well if each step takes a long time to run. In this case, in fact, the event loop would lose responsiveness, and the whole application would start lagging, which is undesirable in a production environment.

Ngoài ra, kỹ thuật này không hoạt động tốt lắm nếu mỗi bước mất nhiều thời gian để chạy. Trong trường hợp này, thực tế, vòng lặp sự kiện sẽ mất khả năng đáp ứng, và toàn bộ ứng dụng sẽ bắt đầu bị lag, điều không mong muốn trong môi trường production.

Bear in mind that this does not mean that the technique we have just seen should be avoided at all costs. In certain situations in which the synchronous task is executed sporadically and doesn't take too long to run, using `setImmediate()` to interleave its execution is sometimes the simplest and most effective way to avoid blocking the event loop. Note that `process.nextTick()` cannot be used to interleave a long-running task. As we saw in Chapter 3, *Callbacks and Events*, `nextTick()` schedules a task to run before any pending I/O, and this will cause I/O starvation in case of repeated calls. You can verify this yourself by replacing `setImmediate()` with `process.nextTick()` in the previous example.

Hãy nhớ rằng điều này không có nghĩa là kỹ thuật ta vừa thấy nên bị tránh bằng mọi giá. Trong một số tình huống nhất định khi tác vụ đồng bộ được thực thi không thường xuyên và không mất quá nhiều thời gian để chạy, dùng `setImmediate()` để đan xen việc thực thi của nó đôi khi là cách đơn giản và hiệu quả nhất để tránh chặn vòng lặp sự kiện. Lưu ý rằng `process.nextTick()` không thể dùng để đan xen một tác vụ chạy lâu. Như ta đã thấy ở Chương 3, *Callbacks and Events*, `nextTick()` lên lịch chạy một tác vụ trước bất kỳ I/O đang chờ nào, và điều này sẽ gây ra tình trạng đói I/O (I/O starvation) trong trường hợp gọi lặp lại. Bạn có thể tự kiểm chứng điều này bằng cách thay thế `setImmediate()` bằng `process.nextTick()` trong ví dụ trước.

Using external processes — Dùng tiến trình bên ngoài

Deferring the steps of an algorithm is not the only option we have for running CPU-bound tasks. Another pattern for preventing the event loop from blocking is using child processes. We already know that Node.js gives its best when running I/O-intensive applications such as web servers, which allows us to optimize resource utilization thanks to its asynchronous architecture. So, the best way we have to maintain the responsiveness of an application is to not run expensive CPU-bound tasks in the context of the main application and, instead, use separate processes. This has three main advantages:

Tri hoãn các bước của một thuật toán không phải là lựa chọn duy nhất ta có để chạy các tác vụ chiếm dụng CPU. Một mẫu khác để ngăn vòng lặp sự kiện bị chặn là dùng các tiến trình con (child process). Ta đã biết rằng Node.js cho ra điều tốt nhất của nó khi chạy các ứng dụng nặng I/O như máy chủ web, điều cho phép ta tối ưu mức sử dụng tài nguyên nhờ kiến trúc bất đồng bộ của nó. Vậy, cách tốt nhất ta có để duy trì khả năng đáp ứng của một ứng dụng là không chạy các tác vụ chiếm dụng CPU tốn kém trong ngữ cảnh của ứng dụng chính mà, thay vào đó, dùng các tiến trình riêng biệt. Điều này có ba lợi thế chính:

- The synchronous task can run at full speed, without the need to interleave the steps of its execution.
- Working with processes in Node.js is simple, probably easier than modifying an algorithm to use `setImmediate()`, and allows us to easily use multiple processors without the need to scale the main application itself.
- If we really need maximum performance, the external process could be created in lower-level languages, such as good old C or more modern compiled languages like Go or Rust. Always use the best tool for the job!
 - Tác vụ đồng bộ có thể chạy với tốc độ tối đa, mà không cần phải đan xen các bước thực thi của nó.
 - Làm việc với các tiến trình trong Node.js rất đơn giản, có lẽ dễ hơn việc sửa đổi một thuật toán để dùng `setImmediate()`, và cho phép ta dễ dàng dùng nhiều bộ xử lý mà không cần mở rộng quy mô chính ứng dụng chính.
 - Nếu ta thực sự cần hiệu năng tối đa, tiến trình bên ngoài có thể được tạo bằng các

ngôn ngữ cấp thấp hơn, như C cũ kỹ hoặc các ngôn ngữ biên dịch hiện đại hơn như Go hay Rust. Luôn dùng công cụ tốt nhất cho công việc!

Node.js has an ample toolbelt of APIs for interacting with external processes. We can find all we need in the `child_process` module. Moreover, when the external process is just another Node.js program, connecting it to the main application is extremely easy and allows seamless communication with the local application. This magic happens thanks to the `child_process.fork()` function, which creates a new child Node.js process and also automatically creates a communication channel with it, allowing us to exchange information using an interface very similar to the `EventEmitter`. Let's see how this works by refactoring our subset sum server again.

Node.js có một bộ công cụ API phong phú để tương tác với các tiến trình bên ngoài. Ta có thể tìm thấy tất cả những gì ta cần trong module `child_process`. Hơn nữa, khi tiến trình bên ngoài chỉ là một chương trình Node.js khác, việc kết nối nó với ứng dụng chính cực kỳ dễ dàng và cho phép giao tiếp liền mạch với ứng dụng cục bộ. Điều kỳ diệu này xảy ra nhờ hàm `child_process.fork()`, vốn tạo ra một tiến trình con Node.js mới và cũng tự động tạo một kênh giao tiếp với nó, cho phép ta trao đổi thông tin dùng một giao diện rất giống `EventEmitter`. Hãy xem cách điều này hoạt động bằng cách tái cấu trúc lại máy chủ tổng tập con của ta một lần nữa.

Delegating the subset sum task to an external process — Ủy thác tác vụ tổng tập con cho một tiến trình bên ngoài

The goal of refactoring the `SubsetSum` task is to create a separate **child process** responsible for handling the synchronous processing, leaving the event loop of the main server free to handle requests coming from the network. This is the recipe we are going to follow to make this possible:

Mục tiêu của việc tái cấu trúc tác vụ `SubsetSum` là tạo một tiến trình con riêng biệt chịu trách nhiệm xử lý phần xử lý đồng bộ, để vòng lặp sự kiện của máy chủ chính rảnh rỗi xử lý các yêu cầu đến từ mạng. Đây là công thức ta sẽ làm theo để biến điều này thành khả thi:

1. We will create a new module named `processPool.js` that will allow us to create a pool of running processes. Starting a new process is expensive and requires time, so keeping them constantly running and ready to handle requests allows us to save time and CPU cycles. Also, the pool will help us limit the number of processes running at the same time to prevent exposing the application to denial-of-service (DoS) attacks.
2. Next, we will create a module called `subsetSumFork.js` responsible for abstracting a `SubsetSum` task running in a child process. Its role will be communicating with the child process and forwarding the results of the task as if they were coming from the current application.
3. Finally, we need a worker (our child process), a new Node.js program with the only goal of running the subset sum algorithm and forwarding its results to the parent process.
 1. Ta sẽ tạo một module mới tên là `processPool.js` cho phép ta tạo một pool các tiến trình đang chạy. Khởi động một tiến trình mới tốn kém và đòi hỏi thời gian, nên giữ chúng liên tục chạy và sẵn sàng xử lý yêu cầu cho phép ta tiết kiệm thời gian và chu kỳ CPU. Ngoài ra, pool sẽ giúp ta giới hạn số lượng tiến trình chạy cùng lúc để ngăn ứng dụng bị phơi bày trước các cuộc tấn công từ chối dịch vụ (denial-of-service - DoS).
 2. Tiếp theo, ta sẽ tạo một module tên là `subsetSumFork.js` chịu trách nhiệm trừu tượng hóa một tác vụ `SubsetSum` chạy trong một tiến trình con. Vai trò của nó sẽ là giao tiếp với tiến trình con và chuyển tiếp các kết quả của tác vụ như thể chúng đến từ ứng dụng hiện tại.

3. Cuối cùng, ta cần một worker (tiến trình con của ta), một chương trình Node.js mới với mục tiêu duy nhất là chạy thuật toán tổng tập con và chuyển tiếp kết quả của nó cho tiến trình cha.

The purpose of a DoS attack is to make a machine unavailable to its users. This is usually achieved by exhausting the capacity of such a machine by exploiting a vulnerability or massively overloading it with requests (DDoS – distributed DoS).

Mục đích của một cuộc tấn công DoS là làm cho một máy không khả dụng đối với người dùng của nó. Điều này thường đạt được bằng cách làm cạn kiệt năng lực của máy đó bằng cách khai thác một lỗ hổng hoặc làm quá tải nó hàng loạt với các yêu cầu (DDoS - DoS phân tán).

Implementing a process pool — Hiện thực một process pool Let's start by building the `processPool.js` module piece by piece:

Hãy bắt đầu xây dựng module `processPool.js` từng phần một:

```
import { fork } from 'child_process'
export class ProcessPool {
  constructor (file, poolMax) {
    this.file = file
    this.poolMax = poolMax
    this.pool = []
    this.active = []
    this.waiting = []
  }
  //...
```

In the first part of the module, we import the `fork()` function from the `child_process` module, which we will use to create new processes. Then, we define the `ProcessPool` constructor, which accepts a `file` parameter representing the Node.js program to run, and the maximum number of running instances in the pool (`poolMax`). We then define three instance variables:

Trong phần đầu của module, ta import hàm `fork()` từ module `child_process`, mà ta sẽ dùng để tạo các tiến trình mới. Sau đó, ta định nghĩa hàm khởi tạo `ProcessPool`, nhận một tham số `file` đại diện cho chương trình Node.js để chạy, và số lượng tối đa các thực thể đang chạy trong pool (`poolMax`). Sau đó ta định nghĩa ba biến thực thể:

- `pool` is the set of running processes ready to be used.
- `active` contains the list of the processes currently being used.
- `waiting` contains a queue of callbacks for all those requests that could not be fulfilled immediately because of the lack of an available process.
 - `pool` là tập hợp các tiến trình đang chạy sẵn sàng để dùng.
 - `active` chứa danh sách các tiến trình hiện đang được dùng.
 - `waiting` chứa một hàng đợi các callback cho tất cả những yêu cầu không thể được đáp ứng ngay lập tức do thiếu một tiến trình khả dụng.

The next piece of the `ProcessPool` class is the `acquire()` method, which is responsible for eventually returning a process ready to be used, when one becomes available:

Phần tiếp theo của lớp `ProcessPool` là phương thức `acquire()`, chịu trách nhiệm cuối cùng trả về một tiến trình sẵn sàng để dùng, khi có một tiến trình trở nên khả dụng:

```

acquire () {
  return new Promise((resolve, reject) => {
    let worker
    if (this.pool.length > 0) { // (1)
      worker = this.pool.pop()
      this.active.push(worker)
      return resolve(worker)
    }
    if (this.active.length >= this.poolMax) { // (2)
      return this.waiting.push({ resolve, reject })
    }
    worker = fork(this.file) // (3)
    worker.once('message', message => {
      if (message === 'ready') {
        this.active.push(worker)
        return resolve(worker)
      }
      worker.kill()
      reject(new Error('Improper process start'))
    })
    worker.once('exit', code => {
      console.log(`Worker exited with code ${code}`)
      this.active = this.active.filter(w => worker !== w)
      this.pool = this.pool.filter(w => worker !== w)
    })
  })
}

```

The logic of `acquire()` is very simple and is explained as follows:

Logic của `acquire()` rất đơn giản và được giải thích như sau:

1. If we have a process in the pool ready to be used, we simply move it to the active list and then use it to fulfill the outer promise with `resolve()`.
2. If there are no available processes in the pool and we have already reached the maximum number of running processes, we have to wait for one to be available. We achieve this by queuing the `resolve()` and `reject()` callbacks of the outer promise, for later use.
3. If we haven't reached the maximum number of running processes yet, we create a new one using `child_process.fork()`. Then, we wait for the ready message coming from the newly launched process, which indicates that the process has started and is ready to accept new jobs. This message-based channel is automatically provided with all processes started with `child_process.fork()`. The last method of the `ProcessPool` class is `release()`, whose purpose is to put a process back into the pool once we are done with it:

1. Nếu ta có một tiến trình trong pool sẵn sàng để dùng, ta chỉ cần chuyển nó sang danh sách active rồi dùng nó để đáp ứng promise bên ngoài bằng `resolve()`.
2. Nếu không có tiến trình khả dụng nào trong pool và ta đã đạt số lượng tối đa các tiến trình đang chạy, ta phải đợi một tiến trình trở nên khả dụng. Ta đạt được điều này bằng cách xếp hàng các callback `resolve()` và `reject()` của promise bên ngoài, để dùng sau.
3. Nếu ta chưa đạt số lượng tối đa các tiến trình đang chạy, ta tạo một tiến trình mới dùng `child_process.fork()`. Sau đó, ta đợi thông điệp `ready` đến từ tiến trình vừa

khởi chạy, báo hiệu rằng tiến trình đã khởi động và sẵn sàng chấp nhận các công việc mới. Kênh dựa trên thông điệp này được cung cấp tự động với tất cả các tiến trình được khởi động bằng `child_process.fork()`. Phương thức cuối cùng của lớp `ProcessPool` là `release()`, mục đích của nó là đưa một tiến trình trở lại pool khi ta đã dùng xong nó:

```
release (worker) {  
  if (this.waiting.length > 0) { // (1)  
    const { resolve } = this.waiting.shift()  
    return resolve(worker)  
  }  
  this.active = this.active.filter(w => worker !== w) // (2)  
  this.pool.push(worker)  
}
```

This is how the `release()` method works:

Đây là cách phương thức `release()` hoạt động:

1. If there is a request in the waiting list, we simply reassign the worker we are releasing by passing it to the `resolve()` callback at the head of the waiting queue.
2. Otherwise, we remove the worker that we are releasing from the active list and put it back into the pool.
 1. Nếu có một yêu cầu trong danh sách `waiting`, ta chỉ cần tái phân công `worker` ta đang giải phóng bằng cách truyền nó cho callback `resolve()` ở đầu hàng đợi `waiting`.
 2. Nếu không, ta xóa `worker` mà ta đang giải phóng khỏi danh sách `active` và đưa nó trở lại `pool`.

As we can see, the processes are never stopped but just reassigned, allowing us to save time by not restarting them at each request. However, it's important to observe that this might not always be the best choice, and this greatly depends on the requirements of your application. Other possible tweaks for reducing long-term memory usage and adding resilience to our process pool are:

Như ta thấy, các tiến trình không bao giờ bị dừng mà chỉ được tái phân công, cho phép ta tiết kiệm thời gian bằng cách không khởi động lại chúng ở mỗi yêu cầu. Tuy nhiên, điều quan trọng cần lưu ý là điều này có thể không phải lúc nào cũng là lựa chọn tốt nhất, và điều này phụ thuộc rất nhiều vào các yêu cầu của ứng dụng của bạn. Các điều chỉnh khả thi khác để giảm mức sử dụng bộ nhớ dài hạn và thêm tính bền bỉ cho process pool của ta là:

- Terminate idle processes to free memory after a certain time of inactivity.
- Add a mechanism to kill non-responsive processes or restart those that have crashed.
 - Chấm dứt các tiến trình nhàn rỗi để giải phóng bộ nhớ sau một khoảng thời gian không hoạt động nhất định.
 - Thêm một cơ chế để giết các tiến trình không phản hồi hoặc khởi động lại những tiến trình đã sập.

In this example, we will keep the implementation of our process pool simple as the details we could add are really endless.

Trong ví dụ này, ta sẽ giữ phần hiện thực process pool của mình đơn giản vì các chi tiết ta có thể thêm vào thực sự là vô tận.

Communicating with a child process — Giao tiếp với một tiến trình con Now that our `ProcessPool` class is ready, we can use it to implement the `SubsetSumFork` class, whose role is to communicate with the worker and forward the results it produces. As we already mentioned, starting a process with `child_process.fork()` also gives us a simple message-based communication channel, so let's see how this works by implementing the `subsetSumFork.js` module:

Giờ khi lớp `ProcessPool` của ta đã sẵn sàng, ta có thể dùng nó để hiện thực lớp `SubsetSumFork`, vai trò của nó là giao tiếp với worker và chuyển tiếp các kết quả mà nó tạo ra. Như đã đề cập, khởi động một tiến trình bằng `child_process.fork()` cũng cho ta một kênh giao tiếp dựa trên thông điệp đơn giản, vậy hãy xem cách điều này hoạt động bằng cách hiện thực module `subsetSumFork.js`:

```
import { EventEmitter } from 'events'
import { dirname, join } from 'path'
import { fileURLToPath } from 'url'
import { ProcessPool } from './processPool.js'
const __dirname = dirname(fileURLToPath(import.meta.url))
const workerFile = join(__dirname,
  'workers', 'subsetSumProcessWorker.js')
const workers = new ProcessPool(workerFile, 2)
export class SubsetSum extends EventEmitter {
  constructor (sum, set) {
    super()
    this.sum = sum
    this.set = set
  }
  async start () {
    const worker = await workers.acquire() // (1)
    worker.send({ sum: this.sum, set: this.set })
    const onMessage = msg => {
      if (msg.event === 'end') { // (3)
        worker.removeListener('message', onMessage)
        workers.release(worker)
      }
      this.emit(msg.event, msg.data) // (4)
    }
    worker.on('message', onMessage) // (2)
  }
}
```

The first thing to note is that we created a new `ProcessPool` object using the file `./workers/subsetSumProcessWorker.js` as the child worker. We also set the maximum capacity of the pool to 2.

Điều đầu tiên cần lưu ý là ta đã tạo một đối tượng `ProcessPool` mới dùng tệp `./workers/subsetSumProcessWorker.js` làm worker con. Ta cũng đặt năng lực tối đa của pool là 2.

Another point worth mentioning is that we tried to maintain the same public API of the original `SubsetSum` class. In fact, `SubsetSumFork` is an `EventEmitter` whose constructor accepts `sum` and `set`, while the `start()` method triggers the execution of the algorithm, which, this time, runs on a separate process. This is what happens when the `start()` method is invoked:

Một điểm khác đáng đề cập là ta đã cố gắng duy trì cùng API công khai của lớp `SubsetSum` gốc. Thực tế, `SubsetSumFork` là một `EventEmitter` có hàm khởi tạo nhận `sum` và `set`, trong khi phương thức `start()` kích hoạt việc thực thi thuật toán, lần này chạy trên một tiến trình riêng biệt. Đây là những gì xảy ra khi phương thức `start()` được gọi:

1. We try to acquire a new child process from the pool. When the operation completes, we immediately use the worker handle to send a message to the child process with the data of the job to run. The `send()` API is provided automatically by Node.js to all processes started with `child_process.fork()`. This is essentially the communication channel that we were talking about.
2. We then start listening for any message sent by the worker process using the `on()` method to attach a new listener (this is also a part of the communication channel provided by all processes started with `child_process.fork()`).
3. In the `onMessage` listener, we first check if we received an end event, which means that the `SubsetSum` task has finished, in which case we remove the `onMessage` listener and release the worker, putting it back into the pool.
4. The worker produces messages in the format `{event, data}`, allowing us to seamlessly forward (re-emit) any event produced by the child process.

1. Ta cố gắng lấy (acquire) một tiến trình con mới từ pool. Khi thao tác hoàn tất, ta ngay lập tức dùng `handle worker` để gửi một thông điệp tới tiến trình con với dữ liệu của công việc cần chạy. API `send()` được Node.js cung cấp tự động cho tất cả các tiến trình được khởi động bằng `child_process.fork()`. Đây thực chất là kênh giao tiếp mà ta đã nói đến.
2. Sau đó ta bắt đầu lắng nghe bất kỳ thông điệp nào được gửi bởi tiến trình worker dùng phương thức `on()` để gắn một listener mới (đây cũng là một phần của kênh giao tiếp được cung cấp bởi tất cả các tiến trình được khởi động bằng `child_process.fork()`).
3. Trong listener `onMessage`, trước tiên ta kiểm tra xem ta có nhận được một sự kiện `end` không, nghĩa là tác vụ `SubsetSum` đã hoàn tất, trong trường hợp đó ta xóa listener `onMessage` và giải phóng worker, đưa nó trở lại pool.
4. Worker tạo ra các thông điệp theo định dạng `{event, data}`, cho phép ta chuyển tiếp (phát lại) một cách liền mạch bất kỳ sự kiện nào được tạo ra bởi tiến trình con.

That's it for the `SubsetSumFork` wrapper. Let's now implement the worker (our child process). It is good to know that the `send()` method available on a child process instance can also be used to propagate a socket handle from the main application to a child process (look at the documentation at [nodejsdp.link/childprocess-send](https://nodejs.org/en/docs/child-process/childprocess/#send)). This is actually the technique used by the `cluster` module to distribute the load of an HTTP server across multiple processes. We will see this in more detail in the next chapter.

Vậy là xong với wrapper `SubsetSumFork`. Giờ hãy hiện thực worker (tiến trình con của ta). Cũng nên biết rằng phương thức `send()` có sẵn trên một thực thể tiến trình con cũng có thể được dùng để truyền một handle socket từ ứng dụng chính tới một tiến trình con (hãy xem tài liệu tại [nodejsdp.link/childprocess-send](https://nodejs.org/en/docs/child-process/childprocess/#send)). Đây thực ra là kỹ thuật được module `cluster` dùng để phân phối tải của một máy chủ HTTP trên nhiều tiến trình. Ta sẽ xem điều này chi tiết hơn ở chương tiếp theo.

Implementing the worker — Hiện thực worker Let's now create the `workers/subsetSumProcessWorker.js` module, our worker process:

Giờ hãy tạo module `workers/subsetSumProcessWorker.js`, tiến trình worker của ta:


```
import { SubsetSum } from '../subsetSum.js'
process.on('message', msg => {                                     // (1)
  const subsetSum = new SubsetSum(msg.sum, msg.set)
  subsetSum.on('match', data => {                                   // (2)
    process.send({ event: 'match', data: data })
  })
  subsetSum.on('end', data => {
    process.send({ event: 'end', data: data })
  })
  subsetSum.start()
})
process.send('ready')
```

We can immediately see that we are reusing the original (and synchronous) `SubsetSum` as it is. Now that we are in a separate process, we don't have to worry about blocking the event loop anymore; all the HTTP requests will continue to be handled by the event loop of the main application without disruptions.

Ta có thể ngay lập tức thấy rằng ta đang tái sử dụng `SubsetSum` gốc (và đồng bộ) y nguyên. Giờ khi ta ở trong một tiến trình riêng biệt, ta không phải lo lắng về việc chặn vòng lặp sự kiện nữa; tất cả các yêu cầu HTTP sẽ tiếp tục được xử lý bởi vòng lặp sự kiện của ứng dụng chính mà không bị gián đoạn.

When the worker is started as a child process, this is what happens:

Khi worker được khởi động như một tiến trình con, đây là những gì xảy ra:

1. It immediately starts listening for messages coming from the parent process. This can be easily done with the `process.on()` function (a part of the communication API provided when the process is started with `child_process.fork()`).

1. Nó ngay lập tức bắt đầu lắng nghe các thông điệp đến từ tiến trình cha. Điều này có thể dễ dàng thực hiện với hàm `process.on()` (một phần của API giao tiếp được cung cấp khi tiến trình được khởi động bằng `child_process.fork()`).

The only message we expect from the parent process is the one providing the input to a new `SubsetSum` task. As soon as such a message is received, we create a new instance of the `SubsetSum` class and register the listeners for the `match` and `end` events. Lastly, we start the computation with `subsetSum.start()`.

Thông điệp duy nhất ta mong đợi từ tiến trình cha là thông điệp cung cấp đầu vào cho một tác vụ `SubsetSum` mới. Ngay khi một thông điệp như vậy được nhận, ta tạo một thực thể mới của lớp `SubsetSum` và đăng ký các listener cho các sự kiện `match` và `end`. Cuối cùng, ta bắt đầu phép tính với `subsetSum.start()`.

2. Every time an event is received from the running algorithm, we wrap it in an object having the format `{event, data}` and send it to the parent process. These messages are then handled in the `subsetSumFork.js` module, as we have seen in the previous section.

2. Mỗi khi một sự kiện được nhận từ thuật toán đang chạy, ta bọc nó trong một đối tượng có định dạng `{event, data}` và gửi nó tới tiến trình cha. Các thông điệp này sau đó được xử lý trong module `subsetSumFork.js`, như ta đã thấy trong phần trước.

As we can see, we just had to wrap the algorithm we already built, without modifying its internals. This clearly shows that any portion of an application can be easily put in an external process by simply using the technique we have just seen.

Như ta thấy, ta chỉ phải bọc thuật toán mà ta đã xây dựng, mà không cần sửa đổi phần bên trong của nó. Điều này cho thấy rõ ràng rằng bất kỳ phần nào của một ứng dụng có thể dễ dàng được đưa vào một tiến trình bên ngoài chỉ bằng cách dùng kỹ thuật ta vừa thấy.

When the child process is not a Node.js program, the simple communication channel we just described (`on()`, `send()`) is not available. In these situations, we can still establish an interface with the child process by implementing our own protocol on top of the standard input and standard output streams, which are exposed to the parent process. To find out more about all the capabilities of the `child_process` API, you can refer to the official Node.js documentation at [nodejsdp.link/child_process](https://nodejs.org/en/docs/api/child-process/).

Khi tiến trình con không phải là một chương trình Node.js, kênh giao tiếp đơn giản ta vừa mô tả (`on()`, `send()`) không có sẵn. Trong những tình huống này, ta vẫn có thể thiết lập một giao diện với tiến trình con bằng cách hiện thực giao thức riêng của ta trên nền các luồng đầu vào chuẩn (standard input) và đầu ra chuẩn (standard output), vốn được phơi bày cho tiến trình cha. Để tìm hiểu thêm về tất cả các năng lực của API `child_process`, bạn có thể tham khảo tài liệu Node.js chính thức tại [nodejsdp.link/child_process](https://nodejs.org/en/docs/api/child-process/).

Considerations for the multi-process approach — Những cân nhắc cho cách tiếp cận đa tiến trình

As always, to try this new version of the subset sum algorithm, we simply have to replace the module used by the HTTP server (the `index.js` file):

Như mọi khi, để thử phiên bản mới này của thuật toán tổng tập con, ta chỉ cần thay thế module được máy chủ HTTP dùng (tệp `index.js`):

```
import { createServer } from 'http'
// import { SubsetSum } from './subsetSum.js'
// import { SubsetSum } from './subsetSumDefer.js'
import { SubsetSum } from './subsetSumFork.js'
createServer((req, res) => {
  //...
```

We can now start the server again and try to send a sample request:

Giờ ta có thể khởi động lại máy chủ và thử gửi một yêu cầu mẫu:

```
curl -G http://localhost:8000/subsetSum --data-urlencode "data=[16,
19,1,1,-16,9,1,-5,-2,17,-15,-97,19,-16,-4,-5,15]" --data-urlencode
"sum=0"
```

As for the interleaving approach that we saw previously, with this new version of the `SubsetSum` module, the event loop is not blocked while running the CPU-bound task. This can be confirmed by sending another concurrent request, as follows:

Cũng như cách tiếp cận đan xen mà ta đã thấy trước đó, với phiên bản mới này của module `SubsetSum`, vòng lặp sự kiện không bị chặn trong khi chạy tác vụ chiếm dụng CPU. Điều này có thể được xác nhận bằng cách gửi một yêu cầu đồng thời khác, như sau:

```
curl -G http://localhost:8000
```

The preceding command should immediately return the text `I'm alive!`

Lệnh trên sẽ ngay lập tức trả về văn bản `I'm alive!`

More interestingly, we can also try to start two `SubsetSum` tasks concurrently and we will be able to see that they will use the full power of two different processors in order to run (if your system has more than one processor, of course). Instead, if we try to run three `SubsetSum` tasks concurrently, the result should be that the last one to start will hang. This is not because the event loop of the main process is blocked, but because we set a **concurrency** limit of two processes for the `SubsetSum` task, which means that the third request will be handled as soon as at least one of the two processes in the pool becomes available again.

Thú vị hơn, ta cũng có thể thử khởi động hai tác vụ `SubsetSum` đồng thời và ta sẽ có thể thấy rằng chúng sẽ dùng toàn bộ sức mạnh của hai bộ xử lý khác nhau để chạy (tất nhiên, nếu hệ thống của bạn có nhiều hơn một bộ xử lý). Ngược lại, nếu ta thử chạy ba tác vụ `SubsetSum` đồng thời, kết quả sẽ là tác vụ khởi động cuối cùng sẽ bị treo. Điều này không phải vì vòng lặp sự kiện của tiến trình chính bị chặn, mà vì ta đã đặt giới hạn đồng thời là hai tiến trình cho tác vụ `SubsetSum`, nghĩa là yêu cầu thứ ba sẽ được xử lý ngay khi ít nhất một trong hai tiến trình trong pool trở nên khả dụng trở lại.

As we saw, the multi-process approach has many advantages compared to the interleaving approach. First, it doesn't introduce any computational penalty when running the algorithm. Second, it can take full advantage of a multi-processor machine.

Như ta đã thấy, cách tiếp cận đa tiến trình có nhiều lợi thế so với cách tiếp cận đan xen. Thứ nhất, nó không đưa ra bất kỳ hình phạt tính toán nào khi chạy thuật toán. Thứ hai, nó có thể tận dụng tối đa một máy đa bộ xử lý.

Now, let's see an alternative approach that uses threads instead of processes.

Giờ hãy xem một cách tiếp cận thay thế dùng luồng (thread) thay vì tiến trình.

Using worker threads — Dùng worker thread

Since Node 10.5.0, we have a new mechanism for running CPU-intensive algorithms outside of the main event loop called worker threads. Worker threads can be seen as a lightweight alternative to `child_process.fork()` with some extra goodies. Compared to processes, worker threads have a smaller memory footprint and a faster startup time since they run within the main process but inside different threads.

Kể từ Node 10.5.0, ta có một cơ chế mới để chạy các thuật toán nặng CPU bên ngoài vòng lặp sự kiện chính gọi là worker thread. Worker thread có thể được xem như một lựa chọn thay thế nhẹ cho `child_process.fork()` với một số tính năng bổ sung. So với các tiến trình, worker thread có dung lượng bộ nhớ nhỏ hơn và thời gian khởi động nhanh hơn vì chúng chạy bên trong tiến trình chính nhưng trong các luồng khác nhau.

Even though worker threads are based on real threads, they don't allow the deep synchronization and sharing capabilities supported by other languages such as Java or Python. This is because JavaScript is a single-threaded language and it doesn't have any built-in mechanism to synchronize access to variables from multiple threads. JavaScript with threads simply wouldn't be JavaScript. The solution to bring all the advantages of threads within Node.js without actually changing the language is worker threads. Worker threads are essentially threads that, by default, don't share anything with the main application thread; they run within their own V8 instance, with an independent Node.js **runtime** and event loop. Communication with the main thread is possible thanks to message-based communication channels, the transfer of `ArrayBuffer` objects, and the use of `SharedArrayBuffer` objects whose synchronization is managed by the user (usually with the help of `Atomics`).

Mặc dù worker thread dựa trên các luồng thực, chúng không cho phép việc đồng bộ hóa sâu và các năng lực chia sẻ được hỗ trợ bởi các ngôn ngữ khác như Java hay Python. Đó là vì JavaScript là một ngôn ngữ đơn luồng và nó không có bất kỳ cơ chế tích hợp sẵn nào để đồng bộ hóa việc truy cập các biến từ nhiều luồng. JavaScript với các luồng đơn giản là sẽ không còn là JavaScript. Giải pháp để mang tất cả các lợi thế của luồng vào trong Node.js mà không thực sự thay đổi ngôn ngữ là worker thread. Worker thread về cơ bản là các luồng mà, theo mặc định, không chia sẻ bất cứ thứ gì với luồng ứng dụng chính; chúng chạy bên trong thực thể V8 riêng của mình, với một runtime Node.js và vòng lặp sự kiện độc lập. Giao tiếp với luồng chính khả thi nhờ các kênh giao tiếp dựa trên thông điệp, việc chuyển các đối tượng `ArrayBuffer`, và việc dùng các đối tượng `SharedArrayBuffer` mà việc đồng bộ hóa được quản lý bởi người dùng (thường với sự trợ giúp của `Atomics`).

You can read more about `SharedArrayBuffer` and `Atomics` in the following article: [nodejsdp.link/shared-array-**buffer**](#). Even though the article focuses on Web Workers, a lot of concepts are similar to Node.js's worker threads.

Bạn có thể đọc thêm về `SharedArrayBuffer` và `Atomics` trong bài viết sau: [nodejsdp.link/shared-array-buffer](#). Mặc dù bài viết tập trung vào Web Workers, rất nhiều khái niệm tương tự với worker thread của Node.js.

This extensive level of isolation of worker threads from the main thread preserves the integrity of the language. At the same time, the basic communication facilities and data-sharing capabilities are more than enough for 99% of use cases.

Mức độ cô lập rộng rãi này của worker thread khỏi luồng chính bảo toàn tính toàn vẹn của ngôn ngữ. Đồng thời, các phương tiện giao tiếp cơ bản và các năng lực chia sẻ dữ liệu là quá đủ cho 99% các trường hợp sử dụng.

Now, let's use worker threads in our `SubsetSum` example.

Giờ hãy dùng worker thread trong ví dụ `SubsetSum` của ta.

Running the subset sum task in a worker thread — Chạy tác vụ tổng tập con trong một worker thread

The worker threads API has a lot in common with that of `ChildProcess`, so the changes to our code will be minimal.

API worker thread có rất nhiều điểm chung với API của `ChildProcess`, nên các thay đổi với mã của ta sẽ là tối thiểu.

First, we need to create a new class called `ThreadPool`, which is our `ProcessPool` adapted to operate with worker threads instead of processes. The following code shows the differences between the new `ThreadPool` class and the `ProcessPool` class. There are only a few differences in the `acquire()` method, which are highlighted; the rest of the code is identical:

Trước tiên, ta cần tạo một lớp mới tên là `ThreadPool`, đó là `ProcessPool` của ta được điều chỉnh để hoạt động với worker thread thay vì tiến trình. Đoạn mã sau cho thấy các điểm khác biệt giữa lớp `ThreadPool` mới và lớp `ProcessPool`. Chỉ có một vài điểm khác biệt trong phương thức `acquire()`, được tô sáng; phần còn lại của mã thì giống hệt:

```
import { Worker } from 'worker_threads'
export class ThreadPool {
  // ...
  acquire () {
    return new Promise((resolve, reject) => {
```

```

let worker
if (this.pool.length > 0) {
  worker = this.pool.pop()
  this.active.push(worker)
  return resolve(worker)
}
if (this.active.length >= this.poolMax) {
  return this.waiting.push({ resolve, reject })
}
worker = new Worker(this.file)
worker.once('online', () => {
  this.active.push(worker)
  resolve(worker)
})
worker.once('exit', code => {
  console.log(`Worker exited with code ${code}`)
  this.active = this.active.filter(w => worker !== w)
  this.pool = this.pool.filter(w => worker !== w)
})
})
}
//...
}

```

Next, we need to adapt the worker and place it in a new file called `subsetSumThreadWorker.js`. The main difference from our old worker is that instead of using `process.send()` and `process.on()`, we'll have to use `parentPort.postMessage()` and `parentPort.on()`:

Tiếp theo, ta cần điều chỉnh worker và đặt nó trong một tệp mới tên là `subsetSumThreadWorker.js`. Điểm khác biệt chính so với worker cũ của ta là thay vì dùng `process.send()` và `process.on()`, ta sẽ phải dùng `parentPort.postMessage()` và `parentPort.on()`:

```

import { parentPort } from 'worker_threads'
import { SubsetSum } from '../subsetSum.js'
parentPort.on('message', msg => {
  const subsetSum = new SubsetSum(msg.sum, msg.set)
  subsetSum.on('match', data => {
    parentPort.postMessage({ event: 'match', data: data })
  })
  subsetSum.on('end', data => {
    parentPort.postMessage({ event: 'end', data: data })
  })
  subsetSum.start()
})

```

Similarly, the module `subsetSumThreads.js` is essentially the same as the `subsetSumFork.js` module except for a couple of lines of code, which are highlighted in the following code fragment:

Tương tự, module `subsetSumThreads.js` về cơ bản giống với module `subsetSumFork.js` ngoại trừ một vài dòng mã, được tô sáng trong đoạn mã sau:

```

import { EventEmitter } from 'events'
import { dirname, join } from 'path'

```

```

import { fileURLToPath } from 'url'
import { ThreadPool } from './threadPool.js'
const __dirname = dirname(fileURLToPath(import.meta.url))
const workerFile = join(__dirname,
  'workers', 'subsetSumThreadWorker.js')
const workers = new ThreadPool(workerFile, 2)
export class SubsetSum extends EventEmitter {
  constructor (sum, set) {
    super()
    this.sum = sum
    this.set = set
  }
  async start () {
    const worker = await workers.acquire()
    worker.postMessage({ sum: this.sum, set: this.set })
    const onMessage = msg => {
      if (msg.event === 'end') {
        worker.removeListener('message', onMessage)
        workers.release(worker)
      }
      this.emit(msg.event, msg.data)
    }
    worker.on('message', onMessage)
  }
}

```

As we've seen, adapting an existing application to use worker threads instead of forked processes is a trivial operation. This is because the API of the two components are very similar, but also because a **worker thread** has a lot in common with a fullfledged Node.js process. Finally, we need to update the `index.js` module so that it can use the new `subsetSumThreads.js` module, as we've seen for the other implementations of the algorithm:

Như ta đã thấy, điều chỉnh một ứng dụng hiện có để dùng worker thread thay vì các tiến trình được fork là một thao tác tầm thường. Đó là vì API của hai thành phần rất giống nhau, nhưng cũng vì một worker thread có rất nhiều điểm chung với một tiến trình Node.js đầy đủ. Cuối cùng, ta cần cập nhật module `index.js` để nó có thể dùng module `subsetSumThreads.js` mới, như ta đã thấy với các phần hiện thực khác của thuật toán:

```

import { createServer } from 'http'
// import { SubsetSum } from './subsetSum.js'
// import { SubsetSum } from './subsetSumDefer.js'
// import { SubsetSum } from './subsetSumFork.js'
import { SubsetSum } from './subsetSumThreads.js'
createServer((req, res) => {
  // ...

```

Now, you can try the new version of the subset sum server using worker threads. As for the previous two implementations, the event loop of the main application is not blocked by the subset sum algorithm as it runs in a separate thread.

Giờ, bạn có thể thử phiên bản mới của máy chủ tổng tập con dùng worker thread. Cũng như hai phần hiện thực trước, vòng lặp sự kiện của ứng dụng chính không bị chặn bởi thuật toán tổng tập con vì nó chạy trong một luồng riêng biệt.

The example we've seen uses only a small subset of all the capabilities offered by worker threads. For more advanced topics such as transferring `ArrayBuffer` objects or `SharedArrayBuffer` objects, you can read the official API documentation at nodejsdp.link/worker-threads.

Ví dụ ta đã thấy chỉ dùng một tập con nhỏ trong tất cả các năng lực mà worker thread cung cấp. Về các chủ đề nâng cao hơn như chuyển các đối tượng `ArrayBuffer` hoặc các đối tượng `SharedArrayBuffer`, bạn có thể đọc tài liệu API chính thức tại nodejsdp.link/worker-threads.

Running CPU-bound tasks in production — Chạy các tác vụ chiếm dụng CPU trong production

The examples we've seen so far should give you an idea of the tools at our disposal for running CPU-intensive operations in Node.js. However, components such as process pools and thread pools are complex pieces of machinery that require proper mechanisms to deal with timeouts, errors, and other types of failures, which, for brevity, we left out from our implementation. Therefore, unless you have special requirements, it's better to rely on more battle-tested libraries for production use. Two of those libraries are `workerpool` (nodejsdp.link/workerpool) and `piscina` (nodejsdp.link/piscina), which are based on the same concepts we've seen in this section. They allow us to coordinate the execution of CPU-intensive tasks using external processes or worker threads.

Các ví dụ ta đã thấy cho đến nay sẽ cho bạn một ý niệm về các công cụ ta có sẵn để chạy các thao tác nặng CPU trong Node.js. Tuy nhiên, các thành phần như process pool và thread pool là những bộ máy phức tạp đòi hỏi các cơ chế thích hợp để xử lý timeout, lỗi, và các loại thất bại khác, mà, cho ngắn gọn, ta đã bỏ qua trong phần hiện thực của mình. Do đó, trừ khi bạn có các yêu cầu đặc biệt, tốt hơn là dựa vào các thư viện đã được kiểm chứng kỹ hơn để dùng trong production. Hai trong số các thư viện đó là `workerpool` (nodejsdp.link/workerpool) và `piscina` (nodejsdp.link/piscina), vốn dựa trên cùng các khái niệm ta đã thấy trong phần này. Chúng cho phép ta điều phối việc thực thi các tác vụ nặng CPU dùng các tiến trình bên ngoài hoặc worker thread.

One last observation is that we must consider that if we have particularly complex algorithms to run or if the number of CPU-bound tasks exceeds the capacity of a single node, we may have to think about scaling out the computation across multiple nodes. This is a completely different problem and we'll learn more about this in the next two chapters.

Một quan sát cuối cùng là ta phải xem xét rằng nếu ta có các thuật toán đặc biệt phức tạp để chạy hoặc nếu số lượng tác vụ chiếm dụng CPU vượt quá năng lực của một node đơn lẻ, ta có thể phải nghĩ đến việc mở rộng ra (scaling out) phép tính trên nhiều node. Đây là một bài toán hoàn toàn khác và ta sẽ tìm hiểu thêm về điều này trong hai chương tiếp theo.

Summary — Tóm tắt

This chapter added some great new weapons to our toolbelt, and as you can see, our journey is getting more focused on advanced problems. Due to this, we have started to delve deeply into more complex solutions. This chapter gave us not only a set of recipes to reuse and customize for our needs, but also some great demonstrations of how mastering a few principles and patterns can help us tackle the most complex problems in Node.js development.

Chương này đã thêm một số vũ khí mới tuyệt vời vào bộ công cụ của ta, và như bạn thấy, hành trình của ta đang trở nên tập trung hơn vào các bài toán nâng cao. Vì vậy, ta đã bắt đầu đi sâu vào các giải pháp phức tạp hơn. Chương này cho ta không chỉ một tập hợp các công thức để tái sử dụng và tùy biến theo nhu cầu của mình, mà còn một số minh chứng tuyệt vời về cách việc thành thạo một vài nguyên tắc và mẫu có thể giúp ta giải quyết những bài toán phức tạp nhất trong phát triển Node.js.

The next two chapters represent the peak of our journey. After studying the various tactics of Node.js development, we are now ready to move on to the strategies and explore the architectural patterns for scaling and distributing our Node.js applications.

Hai chương tiếp theo đại diện cho đỉnh cao của hành trình của ta. Sau khi nghiên cứu các chiến thuật khác nhau của phát triển Node.js, giờ ta đã sẵn sàng chuyển sang các chiến lược và khám phá các mẫu kiến trúc để mở rộng quy mô và phân tán các ứng dụng Node.js của mình.

Exercises — Bài tập

- 11.1 Proxy with pre-initialization queues: Using a JavaScript Proxy, create a wrapper for adding pre-initialization queues to any object. You should allow the consumer of the wrapper to decide which methods to augment and the name of the property/event that indicates if the component is initialized.
 - 11.2 Batching and caching with callbacks: Implement batching and caching for the `totalSales` API examples using only callbacks, streams, and events (without using promises or `async/await`). Hint: Pay attention to `Zalgo` when returning cached values!
 - 11.3 Deep async cancelable: Extend the `createAsyncCancelable()` function so that it's possible to invoke other cancelable functions from within the main cancelable function. Canceling the main operation should also cancel all nested operations. Hint: Allow to yield the result of an `asyncCancelable()` from within the generator function.
 - 11.4 Compute farm: Create an HTTP server with a POST endpoint that receives, as input, the code of a function (as a string) and an array of arguments, executes the function with the given arguments in a worker thread or in a separate process, and returns the result back to the client. Hint: You can use `eval()`, `vm.runInContext()`, or neither of the two. Note: Whatever code you produce for this exercise, please be aware that allowing users to run arbitrary code in a production setting can pose serious security risks, and you should never do it unless you know exactly what the implications are.
-
- 11.1 Proxy với hàng đợi tiền khởi tạo: Dùng một Proxy của JavaScript, hãy tạo một wrapper để thêm hàng đợi tiền khởi tạo cho bất kỳ đối tượng nào. Bạn nên cho phép bên tiêu thụ wrapper quyết định những phương thức nào cần bổ sung và tên của thuộc tính/sự kiện chỉ ra liệu thành phần đã được khởi tạo hay chưa.
 - 11.2 Batching và caching với callback: Hiện thực batching và caching cho các ví dụ API `totalSales` chỉ dùng callback, stream, và sự kiện (không dùng promise hay `async/await`). Gợi ý: Hãy chú ý đến `Zalgo` khi trả về các giá trị đã cache!
 - 11.3 Deep async cancelable: Mở rộng hàm `createAsyncCancelable()` để có thể gọi các hàm có thể hủy khác từ bên trong hàm có thể hủy chính. Hủy thao tác chính cũng nên hủy tất cả các thao tác lồng nhau. Gợi ý: Cho phép `yield` kết quả của một `asyncCancelable()` từ bên trong hàm generator.
 - 11.4 Compute farm: Tạo một máy chủ HTTP với một endpoint POST nhận vào, làm đầu vào, mã của một hàm (dưới dạng chuỗi) và một mảng đối số, thực thi hàm với các đối số đã cho trong một worker thread hoặc trong một tiến trình riêng biệt, và

trả về kết quả cho client. Gợi ý: Bạn có thể dùng `eval()`, `vm.runInContext()`, hoặc không dùng cả hai. Lưu ý: Bất kể bạn tạo ra mã nào cho bài tập này, hãy lưu ý rằng việc cho phép người dùng chạy mã tùy ý trong môi trường production có thể gây ra các rủi ro bảo mật nghiêm trọng, và bạn không bao giờ nên làm điều đó trừ khi bạn biết chính xác các hệ quả là gì.

Chapter 12: Scalability and Architectural Patterns — Chương 12: Khả năng mở rộng và các mẫu kiến trúc

In its early days, Node.js was just a non-blocking web server written in C++ and JavaScript and was called web.js. Its creator, Ryan Dahl, soon realized the potential of the platform and started extending it with tools to enable the creation of different types of server-side applications on top of JavaScript and the non-blocking paradigm.

Trong những ngày đầu, Node.js chỉ là một web server không chặn (non-blocking) được viết bằng C++ và JavaScript và được gọi là web.js. Người tạo ra nó, Ryan Dahl, nhanh chóng nhận ra tiềm năng của nền tảng này và bắt đầu mở rộng nó với các công cụ để cho phép tạo ra nhiều loại ứng dụng phía máy chủ khác nhau dựa trên JavaScript và mô hình không chặn.

The characteristics of Node.js are perfect for the implementation of distributed systems, ranging from a few nodes to thousands of nodes communicating through the network: Node.js was born to be distributed.

Các đặc điểm của Node.js là hoàn hảo cho việc triển khai các hệ thống phân tán, từ một vài node đến hàng nghìn node giao tiếp qua mạng: Node.js sinh ra để được phân tán.

Unlike in other web platforms, **scalability** is a topic that gets explored rather quickly in Node.js while developing an application. This is often because of the singlethreaded nature of Node.js, which is incapable of exploiting all the resources of a multi-core machine. But this is just one side of the coin. In reality, there are more profound reasons for talking about scalability with Node.js.

Khác với các nền tảng web khác, khả năng mở rộng (scalability) là một chủ đề được khám phá khá sớm trong Node.js khi phát triển một ứng dụng. Điều này thường là vì bản chất đơn luồng (single-threaded) của Node.js, vốn không có khả năng khai thác toàn bộ tài nguyên của một máy đa lõi. Nhưng đây chỉ là một mặt của vấn đề. Trên thực tế, có những lý do sâu sắc hơn để bàn về khả năng mở rộng với Node.js.

As we will see in this chapter, scaling an application does not only mean increasing its capacity, enabling it to handle more requests faster: it's also a crucial path to achieving high availability and tolerance to errors.

Như chúng ta sẽ thấy trong chương này, việc mở rộng một ứng dụng không chỉ có nghĩa là tăng năng lực của nó, cho phép nó xử lý nhiều yêu cầu nhanh hơn: đó còn là con đường

thiết yếu để đạt được tính sẵn sàng cao và khả năng chịu lỗi.

Sometimes, we even refer to scalability when we talk about ways to split the complexity of an application into more manageable pieces. Scalability is a concept with multiple faces, six to be precise, as many as there are faces on a cube—the scale cube. In this chapter, you will learn the following topics:

Đôi khi, chúng ta thậm chí còn nhắc đến khả năng mở rộng khi nói về cách chia nhỏ độ phức tạp của một ứng dụng thành các phần dễ quản lý hơn. Khả năng mở rộng là một khái niệm có nhiều mặt, chính xác là sáu mặt, đúng bằng số mặt của một khối lập phương—khối lập phương mở rộng (scale cube). Trong chương này, bạn sẽ học các chủ đề sau:

- Why you should care about scalability
- What the scale cube is and why it is useful to understand scalability
- How to scale by running multiple instances of the same application
- How to leverage a load balancer when scaling an application
- What a service registry is and how it can be used
- Running and scaling Node.js applications using container orchestration platforms like Kubernetes
- How to design a microservice architecture out of a monolithic application
- How to integrate a large number of services through the use of some simple architectural patterns
 - Tại sao bạn nên quan tâm đến khả năng mở rộng
 - Khối lập phương mở rộng (scale cube) là gì và tại sao nó hữu ích để hiểu về khả năng mở rộng
 - Cách mở rộng bằng cách chạy nhiều instance của cùng một ứng dụng
 - Cách tận dụng một bộ cân bằng tải (load balancer) khi mở rộng một ứng dụng
 - Service registry là gì và nó có thể được sử dụng như thế nào
 - Chạy và mở rộng các ứng dụng Node.js bằng các nền tảng điều phối container (container orchestration) như Kubernetes
 - Cách thiết kế một kiến trúc microservice từ một ứng dụng monolithic
 - Cách tích hợp một số lượng lớn các dịch vụ thông qua việc sử dụng một số mẫu kiến trúc đơn giản

An introduction to application scaling — Giới thiệu về việc mở rộng ứng dụng

Scalability can be described as the capability of a system to grow and adapt to everchanging conditions. Scalability is not limited to pure technical growth; it is also dependent on the growth of a business and the organization behind it.

Khả năng mở rộng có thể được mô tả là khả năng của một hệ thống để phát triển và thích ứng với các điều kiện luôn thay đổi. Khả năng mở rộng không chỉ giới hạn ở sự tăng trưởng thuần túy về mặt kỹ thuật; nó còn phụ thuộc vào sự tăng trưởng của một doanh nghiệp và tổ chức đứng sau nó.

If you are building the next “unicorn startup” and you expect your product to rapidly reach millions of users worldwide, you will face serious scalability challenges. How is your application going to sustain ever-increasing demand? Is the system going to get slower over time or crash often? How can you store high volumes of data and keep I/O under control? As you hire more people, how can

you organize the different teams effectively and make them able to work autonomously, without contention across the different parts of the codebase?

Nếu bạn đang xây dựng “startup kỳ lân” tiếp theo và bạn kỳ vọng sản phẩm của mình nhanh chóng đạt đến hàng triệu người dùng trên toàn thế giới, bạn sẽ phải đối mặt với những thách thức nghiêm trọng về khả năng mở rộng. Ứng dụng của bạn sẽ duy trì như thế nào trước nhu cầu ngày càng tăng? Hệ thống có chậm dần theo thời gian hay sập thường xuyên không? Làm thế nào bạn có thể lưu trữ khối lượng dữ liệu lớn và giữ I/O trong tầm kiểm soát? Khi bạn tuyển thêm nhiều người, làm thế nào bạn có thể tổ chức các nhóm khác nhau một cách hiệu quả và khiến họ có thể làm việc tự chủ, mà không có tranh chấp giữa các phần khác nhau của codebase?

Even if you are not working on a high-scale project, that doesn’t mean that you will be free from scalability concerns. You will just face different types of scalability challenges. Being unprepared for these challenges might seriously hinder the success of the project and ultimately damage the company behind it. It’s important to approach scalability in the context of the specific project and understand the expectations for current and future business needs.

Ngay cả khi bạn không làm việc trên một dự án quy mô lớn, điều đó không có nghĩa là bạn sẽ thoát khỏi các mối quan tâm về khả năng mở rộng. Bạn chỉ phải đối mặt với các loại thách thức về khả năng mở rộng khác nhau. Việc không chuẩn bị cho những thách thức này có thể cản trở nghiêm trọng sự thành công của dự án và cuối cùng gây tổn hại cho công ty đứng sau nó. Điều quan trọng là tiếp cận khả năng mở rộng trong bối cảnh của dự án cụ thể và hiểu các kỳ vọng đối với nhu cầu kinh doanh hiện tại và tương lai.

Since scalability is such a broad topic, in this chapter, we will focus our attention on discussing the role of Node.js in the context of scalability. We will discuss several useful patterns and architectures used to scale Node.js applications.

Vì khả năng mở rộng là một chủ đề rất rộng, trong chương này, chúng ta sẽ tập trung sự chú ý vào việc thảo luận vai trò của Node.js trong bối cảnh khả năng mở rộng. Chúng ta sẽ thảo luận một số mẫu và kiến trúc hữu ích được sử dụng để mở rộng các ứng dụng Node.js.

With these in your toolbelt and a solid understanding of your business context, you will be able to design and implement Node.js applications that can adapt and satisfy your business needs and keep your customers happy.

Với những thứ này trong bộ công cụ của bạn và một hiểu biết vững chắc về bối cảnh kinh doanh của mình, bạn sẽ có thể thiết kế và triển khai các ứng dụng Node.js có thể thích ứng và đáp ứng nhu cầu kinh doanh của bạn và giữ cho khách hàng của bạn hài lòng.

Scaling Node.js applications — Mở rộng các ứng dụng Node.js

We already know that most of the workload of a typical Node.js application runs in the context of a single **thread**. In Chapter 1, The Node.js Platform, we learned that this is not necessarily a limitation but rather an advantage, because it allows the application to optimize the usage of the resources necessary to handle concurrent requests, thanks to the **non-blocking I/O** paradigm. This model works wonderfully for applications handling a moderate number of requests per second (usually a few hundred per second), especially if the application is mostly performing I/O-bound tasks (for example, reading and writing from the filesystem and the network) rather than CPU-bound ones (for example, number crunching and data processing).

Chúng ta đã biết rằng phần lớn khối lượng công việc của một ứng dụng Node.js điển hình

chạy trong bối cảnh của một luồng (thread) đơn lẻ. Trong Chương 1, Nền tảng Node.js, chúng ta đã học rằng đây không nhất thiết là một hạn chế mà đúng hơn là một lợi thế, bởi vì nó cho phép ứng dụng tối ưu hóa việc sử dụng các tài nguyên cần thiết để xử lý các yêu cầu đồng thời, nhờ vào mô hình I/O không chặn (non-blocking I/O). Mô hình này hoạt động tuyệt vời cho các ứng dụng xử lý một số lượng yêu cầu vừa phải mỗi giây (thường là vài trăm mỗi giây), đặc biệt nếu ứng dụng chủ yếu thực hiện các tác vụ thiên về I/O (I/O-bound) (ví dụ, đọc và ghi từ filesystem và mạng) thay vì các tác vụ thiên về CPU (CPU-bound) (ví dụ, tính toán số học và xử lý dữ liệu).

In any case, assuming we are using commodity hardware, the capacity that a single thread can support is limited. This is regardless of how powerful a server can be, so if we want to use Node.js for high-load applications, the only way is to scale it across multiple processes and machines.

Dù sao đi nữa, giả sử chúng ta đang sử dụng phần cứng phổ thông, năng lực mà một luồng đơn có thể hỗ trợ là có giới hạn. Điều này bất kể một máy chủ có thể mạnh đến đâu, vì vậy nếu chúng ta muốn sử dụng Node.js cho các ứng dụng tải cao, cách duy nhất là mở rộng nó trên nhiều tiến trình và máy.

However, workload is not the only reason to scale a Node.js application. In fact, with the same techniques that allow us to scale workloads, we can obtain other desirable properties such as high availability and tolerance to failures. Scalability is also a concept applicable to the size and complexity of an application. In fact, building architectures that can grow as much as needed over time is another important factor when designing software.

Tuy nhiên, khối lượng công việc không phải là lý do duy nhất để mở rộng một ứng dụng Node.js. Trên thực tế, với cùng các kỹ thuật cho phép chúng ta mở rộng khối lượng công việc, chúng ta có thể đạt được những thuộc tính mong muốn khác như tính sẵn sàng cao và khả năng chịu lỗi. Khả năng mở rộng cũng là một khái niệm áp dụng được cho kích thước và độ phức tạp của một ứng dụng. Trên thực tế, việc xây dựng các kiến trúc có thể phát triển nhiều như cần thiết theo thời gian là một yếu tố quan trọng khác khi thiết kế phần mềm.

JavaScript is a tool to be used with caution. The lack of type checking and its many gotchas can be an obstacle to the growth of an application, but with discipline and accurate design, we can turn some of its downsides into precious advantages. With JavaScript, we are often pushed to keep the application simple and to split its components into small, manageable pieces. This mindset can make it easier to build applications that are distributed and scalable, but also easy to evolve over time.

JavaScript là một công cụ cần được sử dụng một cách thận trọng. Việc thiếu kiểm tra kiểu (type checking) và nhiều cạm bẫy của nó có thể là một trở ngại cho sự phát triển của một ứng dụng, nhưng với kỷ luật và thiết kế chính xác, chúng ta có thể biến một số nhược điểm của nó thành những lợi thế quý giá. Với JavaScript, chúng ta thường bị thúc đẩy để giữ cho ứng dụng đơn giản và chia các thành phần của nó thành các phần nhỏ, dễ quản lý. Tư duy này có thể giúp dễ dàng hơn trong việc xây dựng các ứng dụng phân tán và có khả năng mở rộng, nhưng cũng dễ dàng phát triển theo thời gian.

The three dimensions of scalability — Ba chiều của khả năng mở rộng

When talking about scalability, the first fundamental principle to understand is load distribution, which is the science of splitting the load of an application across several processes and machines. There are many ways to achieve this, and the book *The Art of Scalability* by Martin L. Abbott and Michael T. Fisher proposes an ingenious model to represent them, called the scale cube. This model describes scalability in terms of the following three dimensions:

Khi nói về khả năng mở rộng, nguyên tắc cơ bản đầu tiên cần hiểu là phân phối tải (load distribution), tức là khoa học chia tải của một ứng dụng trên nhiều tiến trình và máy. Có nhiều cách để đạt được điều này, và cuốn sách *The Art of Scalability* của Martin L. Abbott và Michael T. Fisher đề xuất một mô hình tài tình để biểu diễn chúng, gọi là khối lập phương mở rộng (scale cube). Mô hình này mô tả khả năng mở rộng theo ba chiều sau:

- X-axis — Cloning
- Y-axis — Decomposing by service/functionality
- Z-axis — Splitting by data partition
 - Trục X — Nhân bản (Cloning)
 - Trục Y — Phân tách theo dịch vụ/chức năng (Decomposing by service/functionality)
 - Trục Z — Chia tách theo phân vùng dữ liệu (Splitting by data partition)

These three dimensions can be represented as a cube, as shown in Figure 12.1:

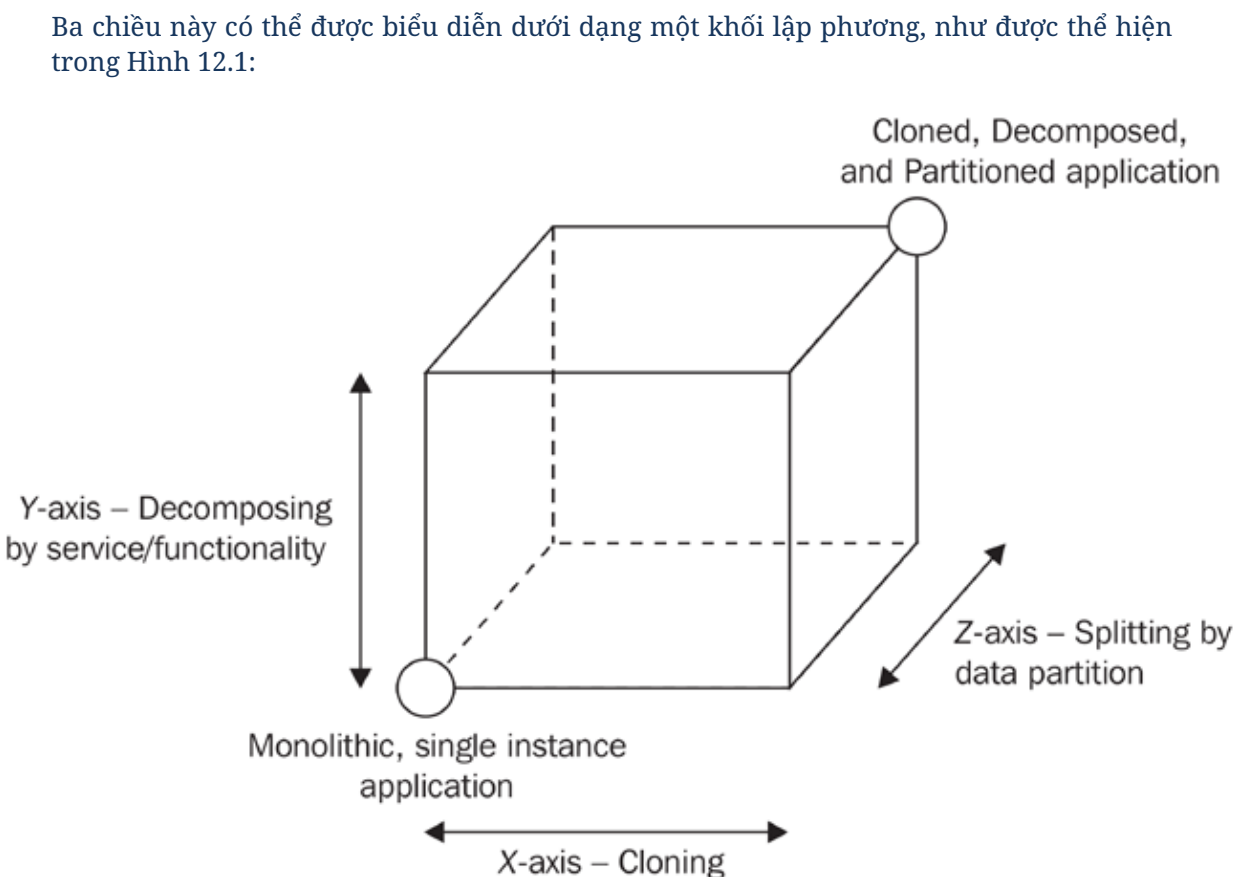


Figure 12.1: The scale cube

Hình 12.1: Khối lập phương mở rộng (scale cube)

The bottom-left corner of the cube (that is, the intersection between the X-axis and the Y-axis) represents the application having all the functionality in a single code base and running on a single instance. This is what we generally call a monolithic application. This is a common situation for applications handling small workloads or at the early stages of their development. Given a monolithic application, there are three different strategies for scaling it. By looking at the scale cube, these strategies are represented as growth along the different axes of the cube: X, Y, and Z:

Góc dưới bên trái của khối lập phương (tức là giao điểm giữa trục X và trục Y) đại diện cho ứng dụng có tất cả chức năng trong một codebase duy nhất và chạy trên một instance duy nhất. Đây là cái mà chúng ta thường gọi là ứng dụng monolithic. Đây là tình huống phổ biến cho các ứng dụng xử lý khối lượng công việc nhỏ hoặc ở giai đoạn đầu của quá trình phát triển. Cho một ứng dụng monolithic, có ba chiến lược khác nhau để mở rộng nó. Bằng cách nhìn vào khối lập phương mở rộng, các chiến lược này được biểu diễn dưới dạng sự tăng trưởng dọc theo các trục khác nhau của khối lập phương: X, Y và Z:

- X-axis — Cloning: The most intuitive evolution of a monolithic, unscaled application is moving right along the X-axis, which is simple, most of the time inexpensive (in terms of development cost), and highly effective. The principle behind this technique is elementary, that is, cloning the same application n times and letting each instance handle $1/n$ th of the workload.
- Y-axis — Decomposing by service/functionality: Scaling along the Y-axis means decomposing the application based on its functionalities, services, or use cases. In this instance, decomposing means creating different, standalone applications, each with its own codebase, possibly with its own dedicated database, and even with a separate UI. For example, a common situation is separating the part of an application responsible for the administration from the public-facing product. Another example is extracting the services responsible for user authentication, thereby creating a dedicated authentication server.
 - Trục X — Nhân bản (Cloning): Sự tiến hóa trực quan nhất của một ứng dụng monolithic, chưa được mở rộng là di chuyển sang phải dọc theo trục X, điều này đơn giản, hầu hết thời gian không tốn kém (về chi phí phát triển) và rất hiệu quả. Nguyên tắc đằng sau kỹ thuật này là cơ bản, đó là nhân bản cùng một ứng dụng n lần và để mỗi instance xử lý $1/n$ khối lượng công việc.
 - Trục Y — Phân tách theo dịch vụ/chức năng (Decomposing by service/functionality): Mở rộng dọc theo trục Y nghĩa là phân tách ứng dụng dựa trên các chức năng, dịch vụ hoặc trường hợp sử dụng (use case) của nó. Trong trường hợp này, phân tách nghĩa là tạo ra các ứng dụng độc lập, riêng biệt, mỗi ứng dụng có codebase riêng, có thể có cơ sở dữ liệu riêng, và thậm chí có một UI riêng. Ví dụ, một tình huống phổ biến là tách phần của ứng dụng chịu trách nhiệm quản trị khỏi sản phẩm hướng đến công chúng. Một ví dụ khác là trích xuất các dịch vụ chịu trách nhiệm xác thực người dùng, từ đó tạo ra một máy chủ xác thực chuyên dụng.

The criteria to split an application by its functionalities depend mostly on its business requirements, the use cases, the data, and many other factors, as we will see later in this chapter. Interestingly, this is the scaling dimension with the biggest repercussions, not only on the architecture of an application but also on the way it is managed from a development and an operational perspective. As we will see, microservice is a term that is most commonly associated with a fine-grained Y-axis scaling.

Tiêu chí để chia một ứng dụng theo chức năng của nó phụ thuộc chủ yếu vào các yêu cầu kinh doanh, các trường hợp sử dụng, dữ liệu và nhiều yếu tố khác, như chúng ta sẽ thấy sau trong chương này. Thú vị là, đây là chiều mở rộng có tác động lớn nhất, không chỉ đến kiến trúc của một ứng dụng mà còn đến cách nó được quản lý từ góc độ phát triển và vận hành. Như chúng ta sẽ thấy, microservice là một thuật ngữ thường được liên kết nhất với việc mở rộng trục Y ở mức độ chi tiết.

- Z-axis — Splitting by data partition: The last scaling dimension is the Z-axis, where the application is split in such a way that each instance is responsible for only a portion of the whole data. This is a technique often used in databases, also known as horizontal/vertical partitioning. In this setup, there are multiple instances of the same application, each of them operating on a partition of the data, which is determined using different criteria. For example, we could partition the users of an application based on their country (list partitioning), or

based on the starting letter of their surname (range partitioning), or by letting a hash function decide the partition each user belongs to (hash partitioning). Each partition can then be assigned to a particular instance of the application. The use of data partitions requires each operation to be preceded by a lookup step to determine which instance of the application is responsible for a given datum. As we said, data partitioning is usually applied and handled at the data storage level because its main purpose is overcoming the problems related to handling large monolithic datasets (limited disk space, memory, and network capacity). Applying it at the application level is worth considering only for complex, distributed architectures or for very particular use cases such as, for example, when building applications relying on custom solutions for data persistence, when using databases that don't support partitioning, or when building applications at Google scale. Considering its complexity, scaling an application along the Z-axis should be taken into consideration only after the X and Y axes of the scale cube have been fully exploited.

- Trục Z — Chia tách theo phân vùng dữ liệu (Splitting by data partition): Chiều mở rộng cuối cùng là trục Z, nơi ứng dụng được chia theo cách mà mỗi instance chỉ chịu trách nhiệm cho một phần của toàn bộ dữ liệu. Đây là một kỹ thuật thường được sử dụng trong cơ sở dữ liệu, còn được gọi là phân vùng ngang/dọc (horizontal/vertical partitioning). Trong thiết lập này, có nhiều instance của cùng một ứng dụng, mỗi instance hoạt động trên một phân vùng của dữ liệu, được xác định bằng các tiêu chí khác nhau. Ví dụ, chúng ta có thể phân vùng người dùng của một ứng dụng dựa trên quốc gia của họ (phân vùng theo danh sách - list partitioning), hoặc dựa trên chữ cái đầu của họ (phân vùng theo phạm vi - range partitioning), hoặc bằng cách để một hàm băm (hash function) quyết định phân vùng mà mỗi người dùng thuộc về (phân vùng theo băm - hash partitioning). Mỗi phân vùng sau đó có thể được gán cho một instance cụ thể của ứng dụng. Việc sử dụng các phân vùng dữ liệu yêu cầu mỗi thao tác phải được đặt trước bởi một bước tra cứu để xác định instance nào của ứng dụng chịu trách nhiệm cho một dữ liệu nhất định. Như chúng ta đã nói, phân vùng dữ liệu thường được áp dụng và xử lý ở cấp độ lưu trữ dữ liệu vì mục đích chính của nó là khắc phục các vấn đề liên quan đến việc xử lý các tập dữ liệu monolithic lớn (không gian đĩa, bộ nhớ và năng lực mạng giới hạn). Việc áp dụng nó ở cấp độ ứng dụng chỉ đáng cân nhắc đối với các kiến trúc phân tán phức tạp hoặc cho các trường hợp sử dụng rất đặc biệt như, ví dụ, khi xây dựng các ứng dụng dựa trên các giải pháp tùy chỉnh cho việc lưu trữ dữ liệu, khi sử dụng các cơ sở dữ liệu không hỗ trợ phân vùng, hoặc khi xây dựng các ứng dụng ở quy mô Google. Xét đến độ phức tạp của nó, việc mở rộng một ứng dụng dọc theo trục Z chỉ nên được xem xét sau khi các trục X và Y của khối lập phương mở rộng đã được khai thác đầy đủ.

In the following sections, we will focus on the two most common and effective techniques used to scale Node.js applications, namely, cloning and decomposing by functionality/service.

Trong các phần tiếp theo, chúng ta sẽ tập trung vào hai kỹ thuật phổ biến và hiệu quả nhất được sử dụng để mở rộng các ứng dụng Node.js, đó là nhân bản (cloning) và phân tách theo chức năng/dịch vụ.

Cloning and load balancing — Nhân bản và cân bằng tải

Traditional, multithreaded web servers are usually only scaled horizontally when the resources assigned to a machine cannot be upgraded any more, or when doing so would involve a higher cost than simply launching another machine. By using multiple threads, traditional web servers can take advantage of all the processing power of a server, using all the available processors and memory.

Conversely, Node.js applications, being **single-threaded**, are usually scaled much sooner compared to traditional web servers. Even in the context of a single machine, we need to find ways to “scale” an application in order to take advantage of all the available resources.

Các web server đa luồng truyền thống thường chỉ được mở rộng theo chiều ngang khi các tài nguyên được gán cho một máy không thể được nâng cấp thêm nữa, hoặc khi làm như vậy sẽ tốn chi phí cao hơn so với việc chỉ đơn giản khởi chạy một máy khác. Bằng cách sử dụng nhiều luồng, các web server truyền thống có thể tận dụng toàn bộ năng lực xử lý của một máy chủ, sử dụng tất cả các bộ xử lý và bộ nhớ có sẵn. Ngược lại, các ứng dụng Node.js, vốn đơn luồng, thường được mở rộng sớm hơn nhiều so với các web server truyền thống. Ngay cả trong bối cảnh của một máy đơn lẻ, chúng ta cần tìm cách “mở rộng” một ứng dụng để tận dụng tất cả các tài nguyên có sẵn.

In Node.js, vertical scaling (adding more resources to a single machine) and horizontal scaling (adding more machines to the infrastructure) are almost equivalent concepts: both, in fact, involve similar techniques to leverage all the available processing power.

Trong Node.js, mở rộng theo chiều dọc (vertical scaling - thêm nhiều tài nguyên hơn cho một máy đơn lẻ) và mở rộng theo chiều ngang (horizontal scaling - thêm nhiều máy hơn vào hạ tầng) gần như là các khái niệm tương đương nhau: cả hai, thực tế, đều liên quan đến các kỹ thuật tương tự để tận dụng tất cả năng lực xử lý có sẵn.

Don’t be fooled into thinking about this as a disadvantage. On the contrary, being almost forced to scale has beneficial effects on other attributes of an application, in particular, availability and fault-tolerance. In fact, scaling a Node.js application by cloning is relatively simple and it’s often implemented even if there is no need to harvest more resources, just for the purpose of having a redundant, fail-tolerant setup.

Đừng bị lừa nghĩ về điều này như một nhược điểm. Ngược lại, việc gần như bị buộc phải mở rộng có tác động có lợi đến các thuộc tính khác của một ứng dụng, đặc biệt là tính sẵn sàng và khả năng chịu lỗi. Trên thực tế, việc mở rộng một ứng dụng Node.js bằng cách nhân bản là tương đối đơn giản và nó thường được triển khai ngay cả khi không cần thu hoạch thêm tài nguyên, chỉ với mục đích có một thiết lập dự thừa, chịu lỗi.

This also pushes the developer to take into account scalability from the early stages of an application, making sure the application does not rely on any resource that cannot be shared across multiple processes or machines. In fact, an absolute prerequisite to scaling an application is that each instance does not have to store common information on resources that cannot be shared, such as memory or disk. For example, in a web server, storing the session data in memory or on disk is a practice that does not work well with scaling. Instead, using a shared database will ensure that each instance will have access to the same session information, wherever it is deployed.

Điều này cũng thúc đẩy nhà phát triển tính đến khả năng mở rộng ngay từ giai đoạn đầu của một ứng dụng, đảm bảo rằng ứng dụng không phụ thuộc vào bất kỳ tài nguyên nào không thể chia sẻ giữa nhiều tiến trình hoặc máy. Trên thực tế, một điều kiện tiên quyết tuyệt đối để mở rộng một ứng dụng là mỗi instance không phải lưu trữ thông tin chung trên các tài nguyên không thể chia sẻ, chẳng hạn như bộ nhớ hoặc đĩa. Ví dụ, trong một web server, việc lưu trữ dữ liệu phiên (session data) trong bộ nhớ hoặc trên đĩa là một thực hành không hoạt động tốt với việc mở rộng. Thay vào đó, việc sử dụng một cơ sở dữ liệu chia sẻ sẽ đảm bảo rằng mỗi instance sẽ có quyền truy cập vào cùng thông tin phiên, bất kể nó được triển khai ở đâu.

Let’s now introduce the most basic mechanism for scaling Node.js applications: the **`cluster`** module.

Bây giờ chúng ta hãy giới thiệu cơ chế cơ bản nhất để mở rộng các ứng dụng Node.js: module `cluster`.

The cluster module — Module cluster

In Node.js, the simplest pattern to distribute the load of an application across different instances running on a single machine is by using the `cluster` module, which is part of the core libraries. The `cluster` module simplifies the forking of new instances of the same application and automatically distributes incoming connections across them, as shown in Figure 12.2:

Trong Node.js, mẫu đơn giản nhất để phân phối tải của một ứng dụng trên các instance khác nhau chạy trên một máy đơn lẻ là sử dụng module `cluster`, vốn là một phần của các thư viện lõi. Module `cluster` đơn giản hóa việc fork các instance mới của cùng một ứng dụng và tự động phân phối các kết nối đến cho chúng, như được thể hiện trong Hình 12.2:

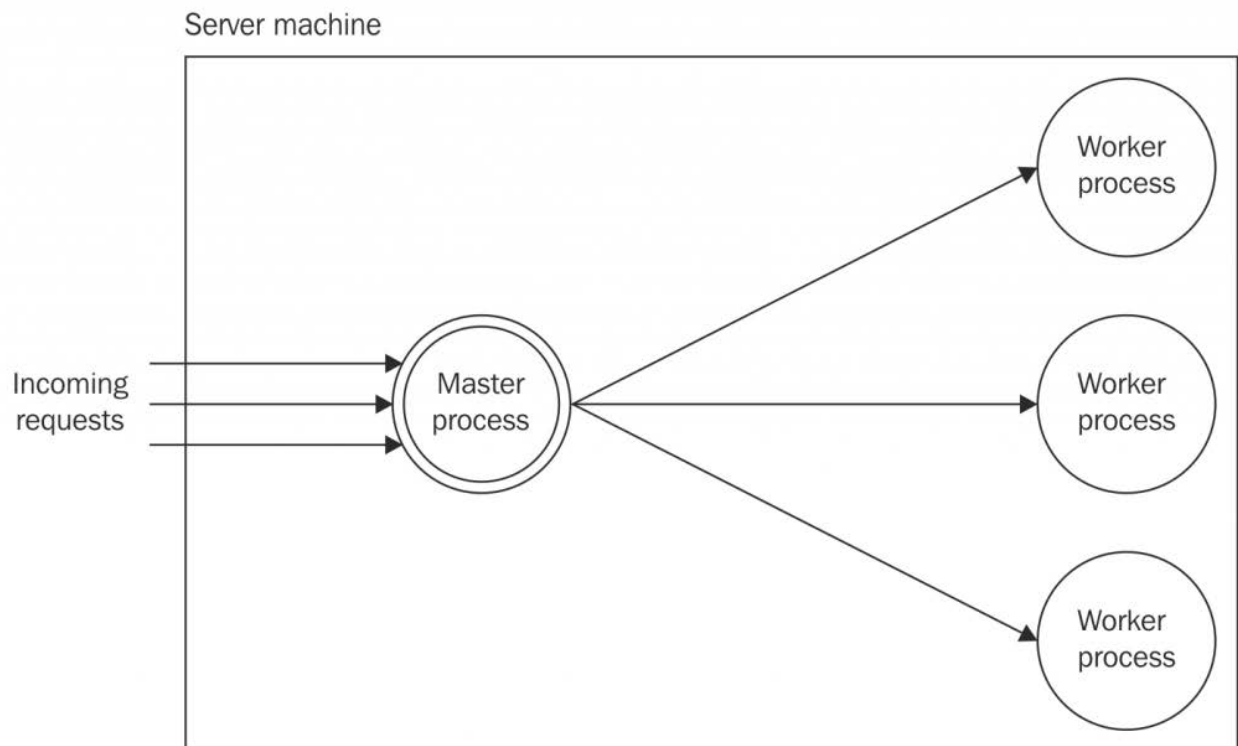


Figure 12.2: Cluster module schematic

Hình 12.2: Sơ đồ module cluster

The master process is responsible for spawning a number of processes (workers), each representing an instance of the application we want to scale. Each incoming connection is then distributed across the cloned workers, spreading the load across them.

Tiến trình master chịu trách nhiệm sinh ra một số tiến trình (worker), mỗi tiến trình đại diện cho một instance của ứng dụng mà chúng ta muốn mở rộng. Mỗi kết nối đến sau đó được phân phối trên các worker được nhân bản, dàn trải tải lên chúng.

Since every worker is an independent process, you can use this approach to spawn as many workers as the number of CPUs available in the system. With this approach, you can easily allow a Node.js

application to take advantage of all the computing power available in the system.

Vì mỗi worker là một tiến trình độc lập, bạn có thể sử dụng cách tiếp cận này để sinh ra số lượng worker bằng số CPU có sẵn trong hệ thống. Với cách tiếp cận này, bạn có thể dễ dàng cho phép một ứng dụng Node.js tận dụng toàn bộ năng lực tính toán có sẵn trong hệ thống.

Notes on the behavior of the cluster module — Ghi chú về hành vi của module cluster

In most systems, the cluster module uses an explicit round-robin **load balancing** algorithm. This algorithm is used inside the master process, which makes sure the requests are evenly distributed across all the workers. Round-robin scheduling is enabled by default on all platforms except Windows, and it can be globally modified by setting the variable `cluster.schedulingPolicy` and using the constants `cluster.SCHED_RR` (round robin) or `cluster.SCHED_NONE` (handled by the operating system). The round-robin algorithm distributes the load evenly across the available servers on a rotational basis. The first **request** is forwarded to the first server, the second to the next server in the list, and so on. When the end of the list is reached, the iteration starts again from the beginning. In the cluster module, the roundrobin logic is a little bit smarter than the traditional implementation. In fact, it is enriched with some extra behaviors that aim to avoid overloading a given worker process.

Trong hầu hết các hệ thống, module cluster sử dụng một thuật toán cân bằng tải round-robin tường minh. Thuật toán này được sử dụng bên trong tiến trình master, vốn đảm bảo rằng các yêu cầu được phân phối đều trên tất cả các worker. Lập lịch round-robin được bật theo mặc định trên tất cả các nền tảng ngoại trừ Windows, và nó có thể được sửa đổi toàn cục bằng cách đặt biến `cluster.schedulingPolicy` và sử dụng các hằng số `cluster.SCHED_RR` (round robin) hoặc `cluster.SCHED_NONE` (do hệ điều hành xử lý). Thuật toán round-robin phân phối tải đều trên các máy chủ có sẵn theo cơ chế luân phiên. Yêu cầu đầu tiên được chuyển tiếp đến máy chủ đầu tiên, yêu cầu thứ hai đến máy chủ tiếp theo trong danh sách, và cứ thế. Khi đến cuối danh sách, quá trình lặp lại bắt đầu từ đầu. Trong module cluster, logic round-robin thông minh hơn một chút so với cách triển khai truyền thống. Trên thực tế, nó được bổ sung một số hành vi thêm nhằm tránh quá tải một tiến trình worker nhất định.

When we use the cluster module, every invocation to `server.listen()` in a worker process is delegated to the master process. This allows the master process to receive all the incoming messages and distribute them to the pool of workers. The cluster module makes this delegation process very simple for most use cases, but there are several edge cases in which calling `server.listen()` in a worker module might not do what you expect:

Khi chúng ta sử dụng module cluster, mỗi lần gọi `server.listen()` trong một tiến trình worker được ủy thác cho tiến trình master. Điều này cho phép tiến trình master nhận tất cả các thông điệp đến và phân phối chúng cho nhóm các worker. Module cluster làm cho quá trình ủy thác này rất đơn giản cho hầu hết các trường hợp sử dụng, nhưng có một số trường hợp đặc biệt mà việc gọi `server.listen()` trong một module worker có thể không làm những gì bạn mong đợi:

- `server.listen({fd})`: If a worker listens using a specific file descriptor, for instance, by invoking `server.listen({fd: 17})`, this operation might produce unexpected results. File descriptors are mapped at the process level, so if a worker process maps a file descriptor, this won't match the same file in the master process. One way to overcome this limitation is to create the file descriptor in the master process and then pass it to the worker process. This way, the worker process can invoke `server.listen()` using a descriptor that is known to the master.

- `server.listen(handle)`: Listening using handle objects (FileHandle) explicitly in a worker process will cause the worker to use the supplied handle directly, rather than delegating the operation to the master process.
- `server.listen(0)`: Calling `server.listen(0)` will generally cause servers to listen on a random port. However, in a cluster, each worker will receive the same “random” port each time they call `server.listen(0)`. In other words, the port is random only the first time; it will be fixed from the second call on. If you want every worker to listen on a different random port, you have to generate the port numbers by yourself.
 - `server.listen({fd})`: Nếu một worker lắng nghe bằng một file descriptor cụ thể, ví dụ, bằng cách gọi `server.listen({fd: 17})`, thao tác này có thể tạo ra kết quả không mong muốn. File descriptor được ánh xạ ở cấp độ tiến trình, vì vậy nếu một tiến trình worker ánh xạ một file descriptor, nó sẽ không khớp với cùng file đó trong tiến trình master. Một cách để khắc phục hạn chế này là tạo file descriptor trong tiến trình master và sau đó chuyển nó cho tiến trình worker. Bằng cách này, tiến trình worker có thể gọi `server.listen()` bằng một descriptor mà master đã biết.
 - `server.listen(handle)`: Lắng nghe bằng cách sử dụng các đối tượng handle (FileHandle) một cách tường minh trong một tiến trình worker sẽ khiến worker sử dụng trực tiếp handle được cung cấp, thay vì ủy thác thao tác cho tiến trình master.
 - `server.listen(0)`: Gọi `server.listen(0)` thường sẽ khiến các máy chủ lắng nghe trên một cổng ngẫu nhiên. Tuy nhiên, trong một cluster, mỗi worker sẽ nhận cùng một cổng “ngẫu nhiên” mỗi lần chúng gọi `server.listen(0)`. Nói cách khác, cổng chỉ ngẫu nhiên trong lần đầu tiên; nó sẽ cố định từ lần gọi thứ hai trở đi. Nếu bạn muốn mỗi worker lắng nghe trên một cổng ngẫu nhiên khác nhau, bạn phải tự tạo các số cổng.

Building a simple HTTP server — Xây dựng một HTTP server đơn giản

Let’s now start working on an example. Let’s build a small HTTP server, cloned and load balanced using the `cluster` module. First of all, we need an application to scale, and for this example, we don’t need too much, just a very basic HTTP server. So, let’s create a file called `app.js` containing the following code:

Bây giờ chúng ta hãy bắt đầu làm việc với một ví dụ. Hãy xây dựng một HTTP server nhỏ, được nhân bản và cân bằng tải bằng module `cluster`. Trước hết, chúng ta cần một ứng dụng để mở rộng, và cho ví dụ này, chúng ta không cần quá nhiều, chỉ là một HTTP server rất cơ bản. Vậy nên, hãy tạo một file gọi là `app.js` chứa đoạn code sau:

```
import { createServer } from 'http'
const { pid } = process
const server = createServer((req, res) => {
  // simulates CPU intensive work
  let i = 1e7; while (i > 0) { i-- }
  console.log(`Handling request from ${pid}`)
  res.end(`Hello from ${pid}\n`)
})
server.listen(8080, () => console.log(`Started at ${pid}`))
```

The HTTP server we just built responds to any request by sending back a message containing its process identifier (PID); this is useful for identifying which instance of the application is handling the request. In this version of the application, we have only one process, so the PID that you see in the responses and the logs will always be the same.

HTTP server mà chúng ta vừa xây dựng phản hồi bất kỳ yêu cầu nào bằng cách gửi lại một thông điệp chứa định danh tiến trình (PID) của nó; điều này hữu ích để xác định instance nào của ứng dụng đang xử lý yêu cầu. Trong phiên bản này của ứng dụng, chúng ta chỉ có một tiến trình, vì vậy PID mà bạn thấy trong các phản hồi và log sẽ luôn giống nhau.

Also, to simulate some actual CPU work, we perform an empty loop 10 million times: without this, the server load would be almost insignificant and it will be quite hard to draw conclusions from the benchmarks we are going to run.

Ngoài ra, để mô phỏng một số công việc CPU thực tế, chúng ta thực hiện một vòng lặp rỗng 10 triệu lần: nếu không có điều này, tải của máy chủ sẽ gần như không đáng kể và sẽ khá khó để rút ra kết luận từ các benchmark mà chúng ta sắp chạy.

The app module we create here is just a simple abstraction for a generic web server. We are not using a web framework like Express or Fastify for simplicity, but feel free to rewrite these examples using your web framework of choice.

Module app mà chúng ta tạo ở đây chỉ là một trừu tượng đơn giản cho một web server tổng quát. Chúng ta không sử dụng một web framework như Express hay Fastify để cho đơn giản, nhưng bạn hãy thoải mái viết lại các ví dụ này bằng web framework mà bạn lựa chọn.

You can now check if all works as expected by running the application as usual and sending a request to `http://localhost:8080` using either a browser or `curl`.

Giờ bạn có thể kiểm tra xem mọi thứ có hoạt động như mong đợi không bằng cách chạy ứng dụng như bình thường và gửi một yêu cầu đến `http://localhost:8080` bằng trình duyệt hoặc `curl`.

You can also try to measure the requests per second that the server is able to handle on one process. For this purpose, you can use a network benchmarking tool such as `autocannon` (nodejsdp.link/autocannon):

Bạn cũng có thể thử đo số yêu cầu mỗi giây mà máy chủ có thể xử lý trên một tiến trình. Với mục đích này, bạn có thể sử dụng một công cụ benchmark mạng như `autocannon` (nodejsdp.link/autocannon):

```
npx autocannon -c 200 -d 10 http://localhost:8080
```

The preceding command will load the server with 200 concurrent connections for 10 seconds. As a reference, the result we got on our machine (a 2.5 GHz quad-core Intel Core i7 using Node.js v14) is in the order of 300 transactions per second. Please remember that the load tests we will perform in this chapter are intentionally simple and minimal and are provided only for reference and learning purposes. Their results cannot provide a 100% accurate evaluation of the performance of the various techniques we are analyzing. When you are trying to optimize a real production application, make sure to always run your own benchmarks after every change. You might find out that, among the different techniques we are going to illustrate here, some can be more effective than others for your specific application.

Lệnh trên sẽ tải máy chủ với 200 kết nối đồng thời trong 10 giây. Để tham khảo, kết quả chúng tôi thu được trên máy của mình (một Intel Core i7 bốn lõi 2.5 GHz sử dụng Node.js v14) nằm trong khoảng 300 giao dịch mỗi giây. Hãy nhớ rằng các bài kiểm tra tải mà chúng ta sẽ thực hiện trong chương này được thiết kế đơn giản và tối thiểu một cách có chủ ý và chỉ được cung cấp cho mục đích tham khảo và học tập. Kết quả của chúng không thể cung cấp một đánh giá chính xác 100% về hiệu năng của các kỹ thuật khác nhau mà chúng ta đang phân tích. Khi bạn đang cố gắng tối ưu hóa một ứng dụng production thực tế, hãy

đảm bảo luôn chạy các benchmark của riêng bạn sau mỗi thay đổi. Bạn có thể phát hiện ra rằng, trong số các kỹ thuật khác nhau mà chúng ta sẽ minh họa ở đây, một số có thể hiệu quả hơn những kỹ thuật khác cho ứng dụng cụ thể của bạn.

Now that we have a simple test web application and some reference benchmarks, we are ready to try some techniques to improve the performance of the application.

Giờ chúng ta đã có một web application kiểm thử đơn giản và một số benchmark tham khảo, chúng ta đã sẵn sàng thử một số kỹ thuật để cải thiện hiệu năng của ứng dụng.

Scaling with the cluster module — Mở rộng với module cluster

Let's now update `app.js` to scale our application using the `cluster` module:

Bây giờ hãy cập nhật `app.js` để mở rộng ứng dụng của chúng ta bằng module `cluster`:

```
import { createServer } from 'http'
import { cpus } from 'os'
import cluster from 'cluster'
if (cluster.isMaster) {                                     // (1)
  const availableCpus = cpus()
  console.log(`Clustering to ${availableCpus.length} processes`)
  availableCpus.forEach(() => cluster.fork())
} else {                                                    // (2)
  const { pid } = process
  const server = createServer((req, res) => {
    let i = 1e7; while (i > 0) { i-- }
    console.log(`Handling request from ${pid}`)
    res.end(`Hello from ${pid}\n`)
  })
  server.listen(8080, () => console.log(`Started at ${pid}`))
}
```

As we can see, using the `cluster` module requires very little effort. Let's analyze what is happening:

Như chúng ta có thể thấy, việc sử dụng module `cluster` đòi hỏi rất ít công sức. Hãy phân tích những gì đang diễn ra:

1. When we launch `app.js` from the command line, we are actually executing the master process. In this case, the `cluster.isMaster` variable is set to `true` and the only work we are required to do is forking the current process using `cluster.fork()`. In the preceding example, we are starting as many workers as there are logical CPU cores in the system to take advantage of all the available processing power.
2. When `cluster.fork()` is executed from the master process, the current module (`app.js`) is run again, but this time in worker mode (`cluster.isWorker` is set to `true`, while `cluster.isMaster` is `false`). When the application runs as a worker, it can start doing some actual work. In this case, it starts a new HTTP server.
 1. Khi chúng ta khởi chạy `app.js` từ dòng lệnh, chúng ta thực sự đang thực thi tiến trình master. Trong trường hợp này, biến `cluster.isMaster` được đặt thành `true` và công việc duy nhất chúng ta cần làm là fork tiến trình hiện tại bằng `cluster.fork()`. Trong ví dụ trên, chúng ta đang khởi động số lượng worker bằng số lõi CPU logic trong hệ thống để tận dụng toàn bộ năng lực xử lý có sẵn.
 2. Khi `cluster.fork()` được thực thi từ tiến trình master, module hiện tại (`app.js`) được chạy lại, nhưng lần này ở chế độ worker (`cluster.isWorker` được đặt thành

true, trong khi `cluster.isMaster` là false). Khi ứng dụng chạy như một worker, nó có thể bắt đầu thực hiện một số công việc thực tế. Trong trường hợp này, nó khởi động một HTTP server mới.

It's important to remember that each worker is a different Node.js process with its own **event loop**, memory space, and loaded modules.

Điều quan trọng cần nhớ là mỗi worker là một tiến trình Node.js khác nhau với vòng lặp sự kiện (event loop), không gian bộ nhớ và các module đã nạp của riêng nó.

It's interesting to note that the usage of the `cluster` module is based on a recurring pattern, which makes it very easy to run multiple instances of an application:

Điều thú vị cần lưu ý là việc sử dụng module `cluster` dựa trên một mẫu lặp lại, khiến cho việc chạy nhiều instance của một ứng dụng trở nên rất dễ dàng:

```
if (cluster.isMaster) {  
  // fork()  
} else {  
  // do work  
}
```

Under the hood, the `cluster.fork()` function uses the `child_process.fork()` **API**, therefore, we also have a communication channel available between the master and the workers. The worker processes can be accessed from the variable `cluster.workers`, so broadcasting a message to all of them would be as easy as running the following line of code:

Bên dưới, hàm `cluster.fork()` sử dụng API `child_process.fork()`, do đó, chúng ta cũng có một kênh giao tiếp khả dụng giữa master và các worker. Các tiến trình worker có thể được truy cập từ biến `cluster.workers`, vì vậy việc phát một thông điệp đến tất cả chúng sẽ dễ dàng như chạy dòng code sau:

```
Object.values(cluster.workers).forEach(worker =>  
worker.send('Hello from the master'))
```

Now, let's try to run our HTTP server in cluster mode. If our machine has more than one core, we should see a number of workers being started by the master process, one after the other. For example, in a system with four logical cores, the terminal should look like this:

Bây giờ, hãy thử chạy HTTP server của chúng ta ở chế độ cluster. Nếu máy của chúng ta có nhiều hơn một lõi, chúng ta sẽ thấy một số worker được khởi động bởi tiến trình master, từng cái một. Ví dụ, trong một hệ thống có bốn lõi logic, terminal sẽ trông như thế này:

```
Started 14107  
Started 14099  
Started 14102  
Started 14101
```

If we now try to hit our server again using the URL `http://localhost:8080`, we should notice that each request will return a message with a different PID, which means that these requests have been handled by different workers, confirming that the load is being distributed among them.

Nếu bây giờ chúng ta thử truy cập máy chủ lại bằng URL `http://localhost:8080`, chúng ta sẽ nhận thấy rằng mỗi yêu cầu sẽ trả về một thông điệp với một PID khác nhau, điều đó có nghĩa là các yêu cầu này đã được xử lý bởi các worker khác nhau, xác nhận rằng tải đang được phân phối giữa chúng.

Now, we can try to load test our server again:

Bây giờ, chúng ta có thể thử kiểm tra tải máy chủ của mình lại:

```
npx autocannon -c 200 -d 10 http://localhost:8080
```

This way, we should be able to discover the performance increase obtained by scaling our application across multiple processes. As a reference, in our machine, we saw a performance increase of about 3.3x (1,000 trans/sec versus 300 trans/sec).

Bằng cách này, chúng ta sẽ có thể khám phá mức tăng hiệu năng đạt được bằng cách mở rộng ứng dụng của mình trên nhiều tiến trình. Để tham khảo, trên máy của chúng tôi, chúng tôi đã thấy mức tăng hiệu năng khoảng 3.3 lần (1.000 giao dịch/giây so với 300 giao dịch/giây).

Resiliency and availability with the cluster module — Khả năng phục hồi và tính sẵn sàng với module cluster

Because workers are all separate processes, they can be killed or respawned depending on a program's needs, without affecting other workers. As long as there are some workers still alive, the server will continue to accept connections. If no workers are alive, existing connections will be dropped, and new connections will be refused. Node.js does not automatically manage the number of workers; however, it is the application's responsibility to manage the worker pool based on its own needs.

Vì các worker đều là các tiến trình riêng biệt, chúng có thể bị kill hoặc được respawn tùy theo nhu cầu của một chương trình, mà không ảnh hưởng đến các worker khác. Miễn là vẫn còn một số worker còn sống, máy chủ sẽ tiếp tục chấp nhận các kết nối. Nếu không có worker nào còn sống, các kết nối hiện có sẽ bị hủy, và các kết nối mới sẽ bị từ chối. Node.js không tự động quản lý số lượng worker; tuy nhiên, đó là trách nhiệm của ứng dụng để quản lý nhóm worker dựa trên nhu cầu của chính nó.

As we already mentioned, scaling an application also brings other advantages, in particular, the ability to maintain a certain level of service, even in the presence of malfunctions or crashes. This property is also known as resiliency and it contributes to the availability of a system.

Như chúng ta đã đề cập, việc mở rộng một ứng dụng cũng mang lại những lợi thế khác, đặc biệt là khả năng duy trì một mức độ dịch vụ nhất định, ngay cả khi có sự cố hoặc sập. Thuộc tính này còn được gọi là khả năng phục hồi (resiliency) và nó góp phần vào tính sẵn sàng của một hệ thống.

By starting multiple instances of the same application, we are creating a redundant system, which means that if one instance goes down for whatever reason, we still have other instances ready to serve requests. This pattern is pretty straightforward to implement using the `cluster` module. Let's see how it works! Let's take the code from the previous section as a starting point. In particular, let's modify the `app.js` module so that it crashes after a random interval of time:

Bằng cách khởi động nhiều instance của cùng một ứng dụng, chúng ta đang tạo ra một hệ thống dư thừa, nghĩa là nếu một instance bị sập vì bất kỳ lý do gì, chúng ta vẫn có các instance khác sẵn sàng phục vụ các yêu cầu. Mẫu này khá đơn giản để triển khai bằng module `cluster`. Hãy xem nó hoạt động như thế nào! Hãy lấy đoạn code từ phần trước làm điểm xuất phát. Cụ thể, hãy sửa đổi module `app.js` để nó sập sau một khoảng thời gian ngẫu nhiên:

```
// ...  
} else {  
  // Inside our worker block
```



```

setTimeout(
  () => { throw new Error('Oops') },
  Math.ceil(Math.random() * 3) * 1000
)
// ...

```

With this change in place, our server exits with an error after a random number of seconds between 1 and 3. In a real-life situation, this would eventually cause our application to stop serving requests, unless we use some external tool to monitor its status and restart it automatically. However, if we only have one instance, there may be a non-negligible delay between restarts caused by the startup time of the application. This means that during those restarts, the application is not available. Having multiple instances instead will make sure we always have a backup process to serve an incoming request, even when one of the workers fails.

Với thay đổi này, máy chủ của chúng ta thoát ra với một lỗi sau một số giây ngẫu nhiên từ 1 đến 3. Trong tình huống thực tế, điều này cuối cùng sẽ khiến ứng dụng của chúng ta ngừng phục vụ các yêu cầu, trừ khi chúng ta sử dụng một số công cụ bên ngoài để giám sát trạng thái của nó và khởi động lại nó tự động. Tuy nhiên, nếu chúng ta chỉ có một instance, có thể có một độ trễ không nhỏ giữa các lần khởi động lại do thời gian khởi động của ứng dụng. Điều này có nghĩa là trong những lần khởi động lại đó, ứng dụng không khả dụng. Thay vào đó, việc có nhiều instance sẽ đảm bảo chúng ta luôn có một tiến trình dự phòng để phục vụ một yêu cầu đến, ngay cả khi một trong các worker bị lỗi.

With the `cluster` module, all we have to do is spawn a new worker as soon as we detect that one is terminated with an error code. Let's modify `app.js` to take this into account:

Với module `cluster`, tất cả những gì chúng ta phải làm là sinh ra một worker mới ngay khi chúng ta phát hiện một worker bị kết thúc với mã lỗi. Hãy sửa đổi `app.js` để tính đến điều này:

```

// ...
if (cluster.isMaster) {
  // ...
  cluster.on('exit', (worker, code) => {
    if (code !== 0 && !worker.exitedAfterDisconnect) {
      console.log(
        `Worker ${worker.process.pid} crashed. ` +
        'Starting a new worker'
      )
      cluster.fork()
    }
  })
} else {
  // ...
}

```

In the preceding code, as soon as the master process receives an `'exit'` event, we check whether the process is terminated intentionally or as the result of an error. We do this by checking the status code and the flag `worker.exitedAfterDisconnect`, which indicates whether the worker was terminated explicitly by the master. If we confirm that the process was terminated because of an error, we start a new worker. It's interesting to note that while the crashed worker gets replaced, the other workers can still serve requests, thus not affecting the availability of the application.

Trong đoạn code trên, ngay khi tiến trình master nhận được sự kiện `'exit'`, chúng ta kiểm

tra xem tiến trình có bị kết thúc một cách có chủ ý hay là kết quả của một lỗi. Chúng ta làm điều này bằng cách kiểm tra mã trạng thái và cờ `worker.exitedAfterDisconnect`, vốn cho biết liệu worker có bị kết thúc một cách tường minh bởi master hay không. Nếu chúng ta xác nhận rằng tiến trình bị kết thúc vì một lỗi, chúng ta khởi động một worker mới. Điều thú vị cần lưu ý là trong khi worker bị sập được thay thế, các worker khác vẫn có thể phục vụ các yêu cầu, do đó không ảnh hưởng đến tính sẵn sàng của ứng dụng.

To test this assumption, we can try to stress our server again using autocannon. When the stress test completes, we will notice that among the various metrics in the output, there is also an indication of the number of failures. In our case, it is something like this:

Để kiểm tra giả định này, chúng ta có thể thử gây căng thẳng cho máy chủ của mình lần nữa bằng autocannon. Khi bài kiểm tra căng thẳng hoàn tất, chúng ta sẽ nhận thấy rằng trong số các số liệu khác nhau ở đầu ra, cũng có một chỉ báo về số lần thất bại. Trong trường hợp của chúng ta, nó giống như thế này:

```
[...]  
8k requests in 10.07s, 964 kB read  
674 errors (7 timeouts)
```

This should amount to about 92% availability. Bear in mind that this result can vary a lot as it greatly depends on the number of running instances and how many times they crash during the test, but it should give us a good indicator of how our solution works. The preceding numbers tell us that despite the fact that our application is constantly crashing, we only had 674 failed requests over 8,000 hits.

Điều này tương đương với khoảng 92% tính sẵn sàng. Hãy nhớ rằng kết quả này có thể thay đổi rất nhiều vì nó phụ thuộc rất lớn vào số lượng instance đang chạy và số lần chúng sập trong quá trình kiểm tra, nhưng nó sẽ cho chúng ta một chỉ báo tốt về cách giải pháp của chúng ta hoạt động. Các con số trên cho chúng ta biết rằng mặc dù ứng dụng của chúng ta liên tục sập, chúng ta chỉ có 674 yêu cầu thất bại trên 8.000 lượt truy cập.

In the example scenario that we just built, most of the failing requests will be caused by the interruption of already established connections during a crash. Unfortunately, there is very little we can do to prevent these types of failures, especially when the application terminates because of a crash. Nonetheless, our solution proves to be working and its availability is not bad at all for an application that crashes so often!

Trong kịch bản ví dụ mà chúng ta vừa xây dựng, hầu hết các yêu cầu thất bại sẽ do sự gián đoạn của các kết nối đã được thiết lập trong một lần sập. Thật không may, có rất ít điều chúng ta có thể làm để ngăn chặn các loại thất bại này, đặc biệt khi ứng dụng kết thúc vì một lần sập. Tuy nhiên, giải pháp của chúng ta đã chứng tỏ là hoạt động và tính sẵn sàng của nó không tệ chút nào đối với một ứng dụng sập thường xuyên như vậy!

Zero-downtime restart — Khởi động lại không gián đoạn (zero-downtime restart)

A Node.js application might also need to be restarted when we want to release a new version to our production servers. So, also in this scenario, having multiple instances can help maintain the availability of our application.

Một ứng dụng Node.js cũng có thể cần được khởi động lại khi chúng ta muốn phát hành một phiên bản mới lên các máy chủ production của mình. Vậy nên, cũng trong kịch bản này, việc có nhiều instance có thể giúp duy trì tính sẵn sàng của ứng dụng.

When we have to intentionally restart an application to update it, there is a small window in which the application restarts and is unable to serve requests. This can be acceptable if we are updating our

personal blog, but it's not even an option for a professional application with a service-level agreement (SLA) or one that is updated very often as part of a continuous delivery process. The solution is to implement a zero-downtime restart, where the code of an application is updated without affecting its availability. With the `cluster` module, this is, again, a pretty easy task: the pattern involves restarting the workers one at a time. This way, the remaining workers can continue to operate and maintain the services of the application available.

Khi chúng ta phải khởi động lại một ứng dụng một cách có chủ ý để cập nhật nó, có một cửa sổ nhỏ trong đó ứng dụng khởi động lại và không thể phục vụ các yêu cầu. Điều này có thể chấp nhận được nếu chúng ta đang cập nhật blog cá nhân của mình, nhưng nó thậm chí không phải là một lựa chọn cho một ứng dụng chuyên nghiệp có thỏa thuận mức dịch vụ (service-level agreement - SLA) hoặc một ứng dụng được cập nhật rất thường xuyên như một phần của quy trình giao hàng liên tục (continuous delivery). Giải pháp là triển khai một khởi động lại không gián đoạn (zero-downtime restart), trong đó code của một ứng dụng được cập nhật mà không ảnh hưởng đến tính sẵn sàng của nó. Với module `cluster`, đây, một lần nữa, là một tác vụ khá dễ dàng: mẫu này liên quan đến việc khởi động lại các worker từng cái một. Bằng cách này, các worker còn lại có thể tiếp tục hoạt động và duy trì các dịch vụ của ứng dụng khả dụng.

Let's add this new feature to our clustered server. All we have to do is add some new code to be executed by the master process:

Hãy thêm tính năng mới này vào máy chủ được phân cụm của chúng ta. Tất cả những gì chúng ta phải làm là thêm một số code mới để được thực thi bởi tiến trình master:

```
import { once } from 'events'
// ...
if (cluster.isMaster) {
  // ...
  process.on('SIGUSR2', async () => { // (1)
    const workers = Object.values(cluster.workers)
    for (const worker of workers) { // (2)
      console.log(`Stopping worker: ${worker.process.pid}`)
      worker.disconnect() // (2)
      await once(worker, 'exit')
      if (!worker.exitedAfterDisconnect) continue
      const newWorker = cluster.fork() // (4)
      await once(newWorker, 'listening') // (5)
    }
  })
} else {
  // ...
}
```

This is how the preceding code block works:

Đây là cách đoạn code trên hoạt động:

1. The restarting of the workers is triggered on receiving the SIGUSR2 signal. Note that we are using an `async` function to implement the event handler as we will need to perform some asynchronous tasks here.
2. When a SIGUSR2 signal is received, we iterate over all the values of the `cluster.workers` object. Every element is a worker object that we can use to interact with a given worker currently active in the pool of workers.

3. The first thing we do for the current worker is invoke `worker.disconnect()`, which stops the worker gracefully. This means that if the worker is currently handling a request, this won't be interrupted abruptly; instead, it will be completed. The worker exits only after the completion of all inflight requests.
4. When the terminated process exits, we can spawn a new worker.
5. We wait for the new worker to be ready and listening for new connections before we proceed with restarting the next worker.
 1. Việc khởi động lại các worker được kích hoạt khi nhận tín hiệu SIGUSR2. Lưu ý rằng chúng ta đang sử dụng một hàm `async` để triển khai trình xử lý sự kiện vì chúng ta sẽ cần thực hiện một số tác vụ bất đồng bộ ở đây.
 2. Khi một tín hiệu SIGUSR2 được nhận, chúng ta lặp qua tất cả các giá trị của đối tượng `cluster.workers`. Mỗi phần tử là một đối tượng worker mà chúng ta có thể sử dụng để tương tác với một worker nhất định hiện đang hoạt động trong nhóm worker.
 3. Điều đầu tiên chúng ta làm cho worker hiện tại là gọi `worker.disconnect()`, vốn dừng worker một cách nhẹ nhàng (gracefully). Điều này có nghĩa là nếu worker hiện đang xử lý một yêu cầu, yêu cầu này sẽ không bị gián đoạn đột ngột; thay vào đó, nó sẽ được hoàn thành. Worker chỉ thoát ra sau khi hoàn thành tất cả các yêu cầu đang trong tiến trình.
 4. Khi tiến trình bị kết thúc thoát ra, chúng ta có thể sinh ra một worker mới.
 5. Chúng ta chờ worker mới sẵn sàng và lắng nghe các kết nối mới trước khi chúng ta tiếp tục khởi động lại worker tiếp theo.

Since our program makes use of Unix signals, it will not work properly on Windows systems (unless you are using the Windows Subsystem for Linux). Signals are the simplest mechanism to implement our solution. However, this isn't the only one. In fact, other approaches include listening for a command coming from a socket, a **pipe**, or the standard input.

Vì chương trình của chúng ta sử dụng các tín hiệu Unix, nó sẽ không hoạt động đúng trên các hệ thống Windows (trừ khi bạn đang sử dụng Windows Subsystem for Linux). Tín hiệu là cơ chế đơn giản nhất để triển khai giải pháp của chúng ta. Tuy nhiên, đây không phải là cơ chế duy nhất. Trên thực tế, các cách tiếp cận khác bao gồm lắng nghe một lệnh đến từ một socket, một pipe, hoặc đầu vào chuẩn (standard input).

Now, we can test our zero-downtime restart by running the application and then sending a SIGUSR2 signal. However, we first need to obtain the PID of the master process. The following command can be useful to identify it from the list of all the running processes:

Bây giờ, chúng ta có thể kiểm tra khởi động lại không gián đoạn của mình bằng cách chạy ứng dụng và sau đó gửi một tín hiệu SIGUSR2. Tuy nhiên, trước tiên chúng ta cần lấy PID của tiến trình master. Lệnh sau có thể hữu ích để xác định nó từ danh sách tất cả các tiến trình đang chạy:

```
ps -af
```

The master process should be the parent of a set of node processes. Once we have the PID we are looking for, we can send the signal to it:

Tiến trình master nên là tiến trình cha của một tập hợp các tiến trình node. Một khi chúng ta có PID mà chúng ta đang tìm kiếm, chúng ta có thể gửi tín hiệu đến nó:

```
kill -SIGUSR2 <PID>
```

Now, the output of the application should display something like this:

Bây giờ, đầu ra của ứng dụng sẽ hiển thị một cái gì đó như thế này:

```
Restarting workers
Stopping worker: 19389
Started 19407
Stopping worker: 19390
Started 19409
```

We can try to use autocannon again to verify that we don't have any considerable impact on the availability of our application during the restart of the workers.

Chúng ta có thể thử sử dụng autocannon lần nữa để xác minh rằng chúng ta không có bất kỳ tác động đáng kể nào đến tính sẵn sàng của ứng dụng trong quá trình khởi động lại các worker.

pm2 (nodejsdp.link/pm2) is a small utility, based on cluster, which offers load balancing, process monitoring, zero-downtime restarts, and other goodies.

pm2 (nodejsdp.link/pm2) là một tiện ích nhỏ, dựa trên cluster, cung cấp cân bằng tải, giám sát tiến trình, khởi động lại không gián đoạn, và các tính năng hữu ích khác.

Dealing with stateful communications — Xử lý các giao tiếp có trạng thái (stateful)

The cluster module does not work well with stateful communications where the application **state** is not shared between the various instances. This is because different requests belonging to the same stateful session may potentially be handled by a different instance of the application. This is not a problem limited only to the cluster module, but, in general, it applies to any kind of stateless, load balancing algorithm. Consider, for example, the situation described by Figure 12.3:

Module cluster không hoạt động tốt với các giao tiếp có trạng thái (stateful communications) nơi trạng thái ứng dụng không được chia sẻ giữa các instance khác nhau. Điều này là vì các yêu cầu khác nhau thuộc cùng một phiên có trạng thái có thể được xử lý bởi một instance khác nhau của ứng dụng. Đây không phải là vấn đề chỉ giới hạn ở module cluster, mà nói chung, nó áp dụng cho bất kỳ loại thuật toán cân bằng tải không trạng thái (stateless) nào. Hãy xem xét, ví dụ, tình huống được mô tả bởi Hình 12.3:

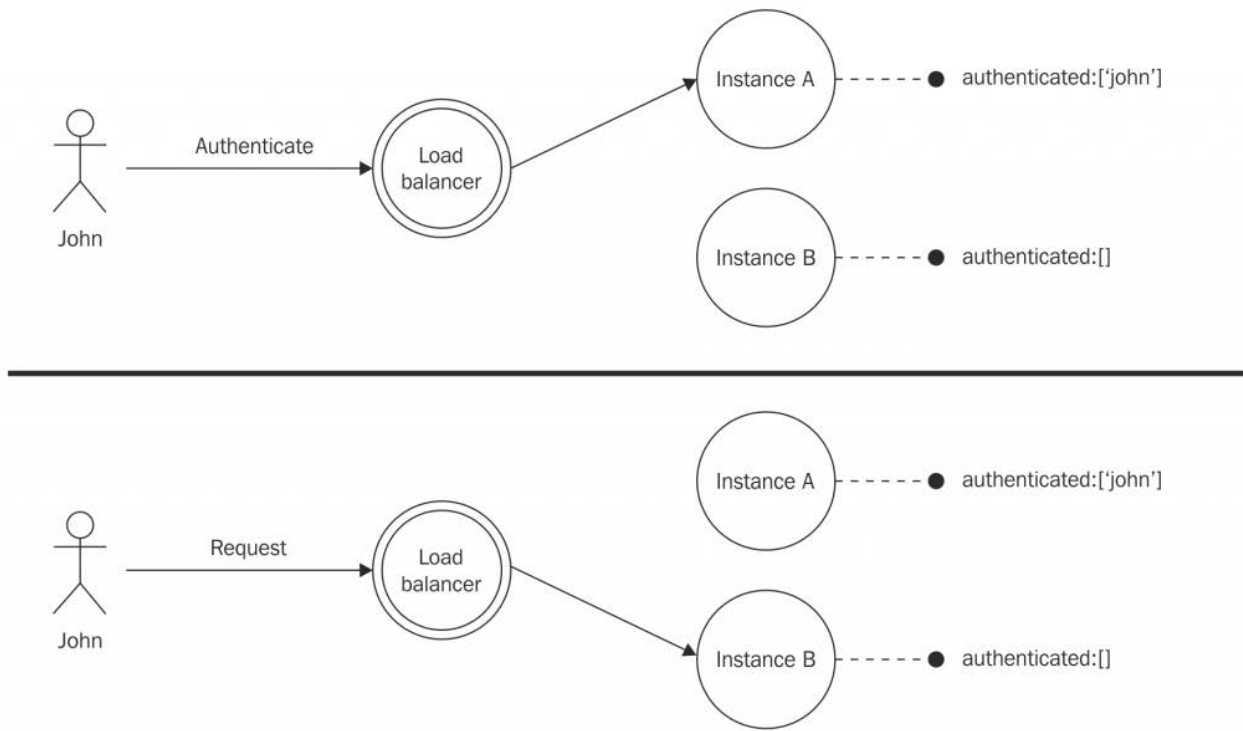


Figure 12.3: An example issue with a stateful application behind a load balancer

Hình 12.3: Một ví dụ về vấn đề với một ứng dụng có trạng thái đứng sau một bộ cân bằng tải

The user John initially sends a request to our application to authenticate himself, but the result of the operation is registered locally (for example, in memory), so only the instance of the application that receives the authentication request (Instance A) knows that John is successfully authenticated. When John sends a new request, the load balancer might forward it to a different instance of the application, which actually doesn't possess the authentication details of John, hence refusing to perform the operation. The application we just described cannot be scaled as it is, but luckily, there are two easy solutions we can apply to solve this problem.

Người dùng John ban đầu gửi một yêu cầu đến ứng dụng của chúng ta để xác thực bản thân, nhưng kết quả của thao tác được đăng ký cục bộ (ví dụ, trong bộ nhớ), vì vậy chỉ instance của ứng dụng nhận được yêu cầu xác thực (Instance A) biết rằng John đã được xác thực thành công. Khi John gửi một yêu cầu mới, bộ cân bằng tải có thể chuyển tiếp nó đến một instance khác của ứng dụng, vốn thực sự không sở hữu các chi tiết xác thực của John, do đó từ chối thực hiện thao tác. Ứng dụng mà chúng ta vừa mô tả không thể được mở rộng như hiện tại, nhưng may mắn thay, có hai giải pháp dễ dàng mà chúng ta có thể áp dụng để giải quyết vấn đề này.

Sharing the state across multiple instances — Chia sẻ trạng thái trên nhiều instance

The first option we have to scale an application using stateful communications is sharing the state across all the instances.

Lựa chọn đầu tiên chúng ta có để mở rộng một ứng dụng sử dụng các giao tiếp có trạng thái là chia sẻ trạng thái trên tất cả các instance.

This can be easily achieved with a shared datastore, such as, for example, a database like

PostgreSQL (nodejsdp.link/postgresql), MongoDB (nodejsdp.link/mongodb), or CouchDB (nodejsdp.link/couchdb), or, even better, we can use an in-memory store such as Redis (nodejsdp.link/redis) or Memcached (nodejsdp.link/memcached). Figure 12.4 outlines this simple and effective solution:

Điều này có thể dễ dàng đạt được với một datastore chia sẻ, chẳng hạn như, ví dụ, một cơ sở dữ liệu như PostgreSQL (nodejsdp.link/postgresql), MongoDB (nodejsdp.link/mongodb), hoặc CouchDB (nodejsdp.link/couchdb), hoặc, tốt hơn nữa, chúng ta có thể sử dụng một kho lưu trữ trong bộ nhớ (in-memory store) như Redis (nodejsdp.link/redis) hoặc Memcached (nodejsdp.link/memcached). Hình 12.4 phác họa giải pháp đơn giản và hiệu quả này:

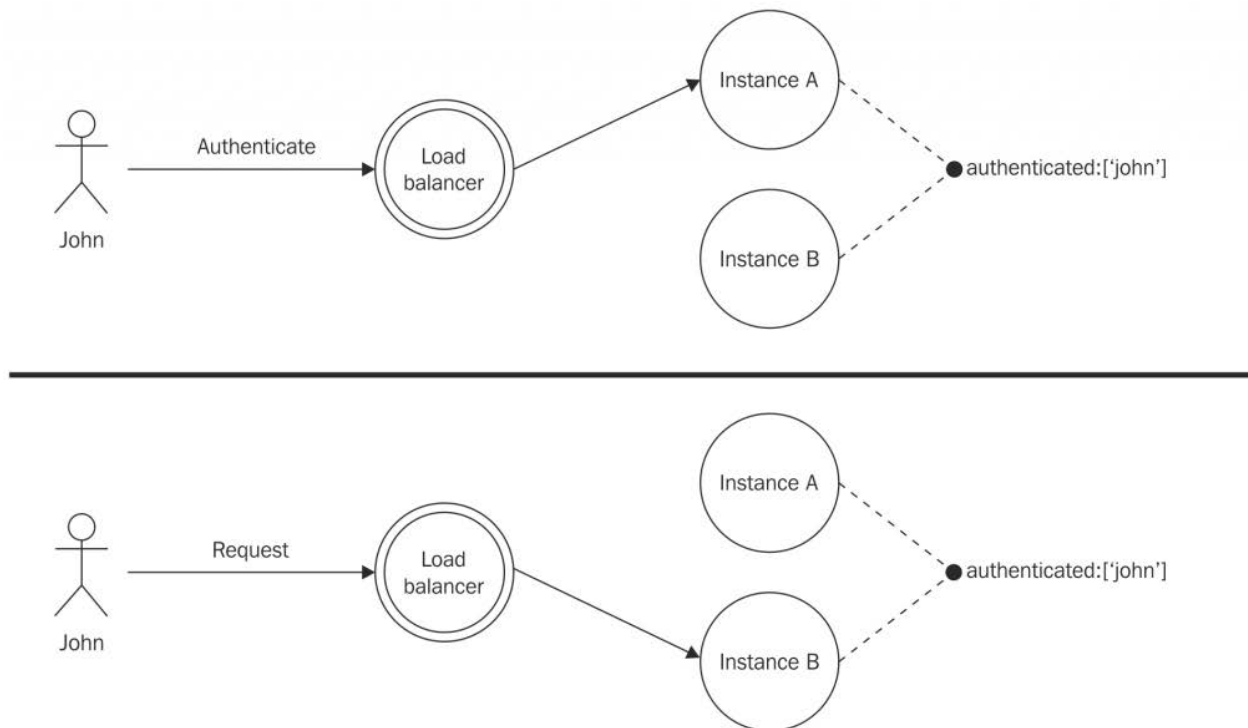


Figure 12.4: Application behind a load balancer using a shared data store

Hình 12.4: Ứng dụng đứng sau một bộ cân bằng tải sử dụng một kho dữ liệu chia sẻ

The only drawback of using a shared store for the communication state is that applying this pattern might require a significant amount of refactoring of the code base. For example, we might be using an existing library that keeps the communication state in memory, so we have to figure out how to configure, replace, or reimplement this library to use a shared store.

Nhược điểm duy nhất của việc sử dụng một kho chia sẻ cho trạng thái giao tiếp là việc áp dụng mẫu này có thể đòi hỏi một lượng tái cấu trúc (refactor) đáng kể codebase. Ví dụ, chúng ta có thể đang sử dụng một thư viện hiện có giữ trạng thái giao tiếp trong bộ nhớ, vì vậy chúng ta phải tìm ra cách cấu hình, thay thế, hoặc triển khai lại thư viện này để sử dụng một kho chia sẻ.

In cases where refactoring might not be feasible, for instance, because of too many changes required or stringent time constraints in making the application more scalable, we can rely on a less invasive solution: sticky load balancing (or sticky sessions).

Trong các trường hợp mà việc tái cấu trúc có thể không khả thi, ví dụ, vì quá nhiều thay

đổi cần thiết hoặc các ràng buộc thời gian nghiêm ngặt trong việc làm cho ứng dụng có khả năng mở rộng hơn, chúng ta có thể dựa vào một giải pháp ít xâm phạm hơn: cân bằng tải dính (sticky load balancing) (hoặc các phiên dính - sticky sessions).

Sticky load balancing — Cân bằng tải dính (sticky load balancing)

The other alternative we have to support stateful communications is having the load balancer always routing all of the requests associated with a session to the same instance of the application. This technique is also called sticky load balancing. Figure 12.5 illustrates a simplified scenario involving this technique:

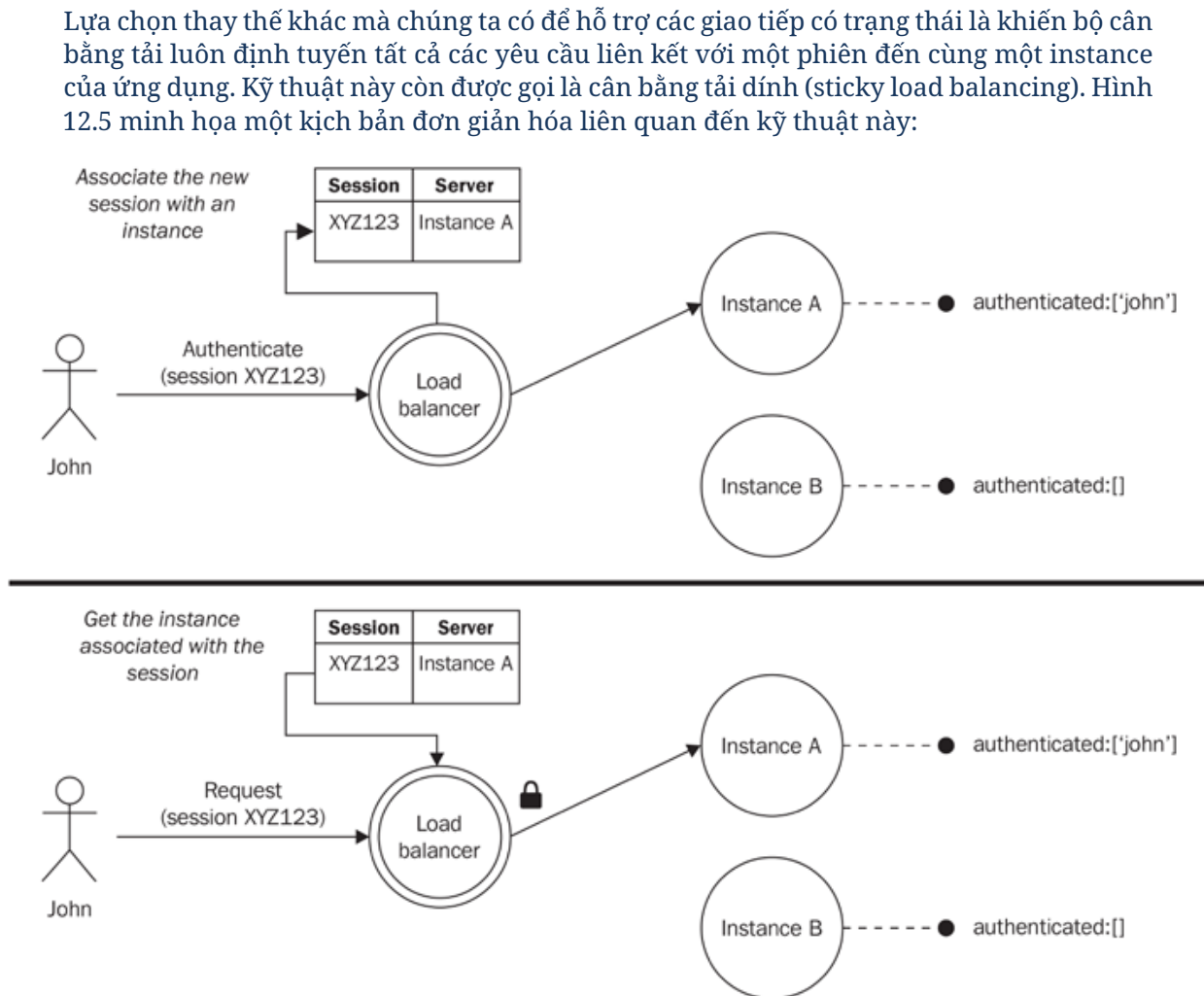


Figure 12.5: An example illustrating how sticky load balancing works

Hình 12.5: Một ví dụ minh họa cách cân bằng tải dính hoạt động

As we can see from Figure 12.5, when the load balancer receives a request associated with a new session, it creates a mapping with one particular instance selected by the load balancing algorithm. The next time the load balancer receives a request from that same session, it bypasses the load balancing algorithm, selecting the application instance that was previously associated with the session. The particular technique we just described involves inspecting the session ID associated with the requests (usually included in a cookie by the application or the load balancer itself).

Như chúng ta có thể thấy từ Hình 12.5, khi bộ cân bằng tải nhận được một yêu cầu liên kết với một phiên mới, nó tạo ra một ánh xạ với một instance cụ thể được chọn bởi thuật toán cân bằng tải. Lần tiếp theo bộ cân bằng tải nhận được một yêu cầu từ cùng phiên đó, nó bỏ qua thuật toán cân bằng tải, chọn instance ứng dụng đã được liên kết trước đó với phiên. Kỹ thuật cụ thể mà chúng ta vừa mô tả liên quan đến việc kiểm tra session ID liên kết với các yêu cầu (thường được bao gồm trong một cookie bởi ứng dụng hoặc chính bộ cân bằng tải).

A simpler alternative to associate a stateful connection to a single server is by using the IP address of the client performing the request. Usually, the IP is provided to a hash function that generates an ID representing the application instance designated to receive the request. This technique has the advantage of not requiring the association to be remembered by the load balancer. However, it doesn't work well with devices that frequently change IP, for example, when roaming on different networks. Sticky load balancing is not supported by default by the `cluster` module, but it can be added with an npm library called `sticky-session` ([nodejsdp.link/sticky-session](https://www.npmjs.com/package/sticky-session)).

Một lựa chọn thay thế đơn giản hơn để liên kết một kết nối có trạng thái với một máy chủ duy nhất là sử dụng địa chỉ IP của client thực hiện yêu cầu. Thông thường, IP được cung cấp cho một hàm băm tạo ra một ID đại diện cho instance ứng dụng được chỉ định để nhận yêu cầu. Kỹ thuật này có lợi thế là không yêu cầu bộ cân bằng tải phải ghi nhớ liên kết. Tuy nhiên, nó không hoạt động tốt với các thiết bị thường xuyên thay đổi IP, ví dụ, khi chuyển vùng (roaming) trên các mạng khác nhau. Cân bằng tải dính không được hỗ trợ theo mặc định bởi module `cluster`, nhưng nó có thể được thêm vào với một thư viện npm gọi là `sticky-session` ([nodejsdp.link/sticky-session](https://www.npmjs.com/package/sticky-session)).

One big problem with sticky load balancing is the fact that it nullifies most of the advantages of having a redundant system, where all the instances of the application are the same, and where an instance can eventually replace another one that stopped working. For these reasons, it is recommended to always try to avoid sticky load balancing and building applications that maintain session state in a shared store. Alternatively, where feasible, you can try to build applications that don't require stateful communications at all; for example, by including the state in the request itself.

Một vấn đề lớn với cân bằng tải dính là việc nó vô hiệu hóa hầu hết các lợi thế của việc có một hệ thống dư thừa, nơi tất cả các instance của ứng dụng đều giống nhau, và nơi một instance cuối cùng có thể thay thế một instance khác đã ngừng hoạt động. Vì những lý do này, được khuyến nghị là luôn cố gắng tránh cân bằng tải dính và xây dựng các ứng dụng duy trì trạng thái phiên trong một kho chia sẻ. Hoặc, khi khả thi, bạn có thể thử xây dựng các ứng dụng hoàn toàn không yêu cầu giao tiếp có trạng thái; ví dụ, bằng cách bao gồm trạng thái trong chính yêu cầu.

For a real example of a library requiring sticky load balancing, we can mention `Socket.IO` ([nodejsdp.link/socket-io](https://www.npmjs.com/package/socket.io)).

Để có một ví dụ thực tế về một thư viện yêu cầu cân bằng tải dính, chúng ta có thể nhắc đến `Socket.IO` ([nodejsdp.link/socket-io](https://www.npmjs.com/package/socket.io)).

Scaling with a reverse proxy — Mở rộng với một reverse proxy

The `cluster` module, although very convenient and simple to use, is not the only option we have to scale a Node.js web application. Traditional techniques are often preferred because they offer more control and power in highly-available production environments.

Module `cluster`, mặc dù rất tiện lợi và đơn giản để sử dụng, không phải là lựa chọn duy nhất chúng ta có để mở rộng một web application Node.js. Các kỹ thuật truyền thống

thường được ưa chuộng hơn vì chúng cung cấp nhiều quyền kiểm soát và sức mạnh hơn trong các môi trường production có tính sẵn sàng cao.

The alternative to using `cluster` is to start multiple standalone instances of the same application running on different ports or machines, and then use a reverse **proxy** (or gateway) to provide access to those instances, distributing the traffic across them. In this configuration, we don't have a master process distributing requests to a set of workers, but a set of distinct processes running on the same machine (using different ports) or scattered across different machines inside a network. To provide a single access point to our application, we can use a reverse proxy, a special device or service placed between the clients and the instances of our application, which takes any request and forwards it to a destination server, returning the result to the client as if it was itself the origin. In this scenario, the reverse proxy is also used as a load balancer, distributing the requests among the instances of the application. For a clear explanation of the differences between a reverse proxy and a forward proxy, you can refer to the Apache HTTP server documentation at nodejsdp.link/forward-reverse.

Lựa chọn thay thế cho việc sử dụng `cluster` là khởi động nhiều instance độc lập của cùng một ứng dụng chạy trên các cổng hoặc máy khác nhau, và sau đó sử dụng một reverse proxy (hoặc gateway) để cung cấp quyền truy cập vào các instance đó, phân phối lưu lượng trên chúng. Trong cấu hình này, chúng ta không có một tiến trình master phân phối các yêu cầu đến một tập hợp các worker, mà là một tập hợp các tiến trình riêng biệt chạy trên cùng một máy (sử dụng các cổng khác nhau) hoặc nằm rải rác trên các máy khác nhau trong một mạng. Để cung cấp một điểm truy cập duy nhất đến ứng dụng của chúng ta, chúng ta có thể sử dụng một reverse proxy, một thiết bị hoặc dịch vụ đặc biệt được đặt giữa các client và các instance của ứng dụng của chúng ta, nhận bất kỳ yêu cầu nào và chuyển tiếp nó đến một máy chủ đích, trả kết quả về cho client như thể chính nó là nguồn gốc. Trong kịch bản này, reverse proxy cũng được sử dụng như một bộ cân bằng tải, phân phối các yêu cầu giữa các instance của ứng dụng. Để có một giải thích rõ ràng về sự khác biệt giữa một reverse proxy và một forward proxy, bạn có thể tham khảo tài liệu của Apache HTTP server tại nodejsdp.link/forward-reverse.

Figure 12.6 shows a typical multi-process, multi-machine configuration with a reverse proxy acting as a load balancer on the front:

Hình 12.6 cho thấy một cấu hình đa tiến trình, đa máy điển hình với một reverse proxy đóng vai trò là bộ cân bằng tải ở phía trước:

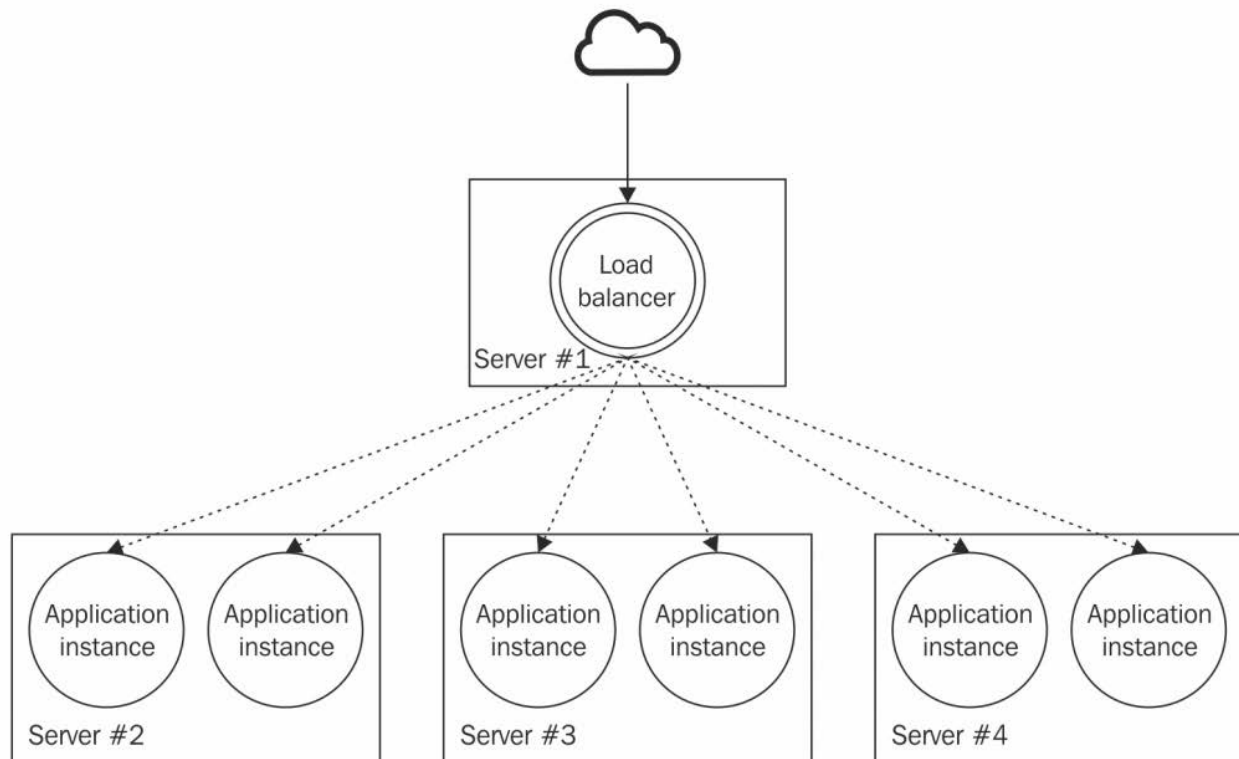


Figure 12.6: A typical multi-process, multi-machine configuration with a reverse proxy acting as a load balancer

Hình 12.6: Một cấu hình đa tiến trình, đa máy điển hình với một reverse proxy đóng vai trò là bộ cân bằng tải

For a Node.js application, there are many reasons to choose this approach in place of the `cluster` module:

Đối với một ứng dụng Node.js, có nhiều lý do để chọn cách tiếp cận này thay cho module `cluster`:

- A reverse proxy can distribute the load across several machines, not just several processes.
- The most popular reverse proxies on the market support sticky load balancing out of the box.
- A reverse proxy can route a request to any available server, regardless of its programming language or platform.
- We can choose more powerful load balancing algorithms.
- Many reverse proxies offer additional powerful features such as URL rewrites, caching, SSL termination point, security features (for example, denial-of-service protection), or even the functionality of fully-fledged web servers that can be used to, for example, serve static files.
 - Một reverse proxy có thể phân phối tải trên nhiều máy, không chỉ nhiều tiến trình.
 - Các reverse proxy phổ biến nhất trên thị trường hỗ trợ cân bằng tải dính ngay từ đầu (out of the box).
 - Một reverse proxy có thể định tuyến một yêu cầu đến bất kỳ máy chủ khả dụng nào, bất kể ngôn ngữ lập trình hay nền tảng của nó.
 - Chúng ta có thể chọn các thuật toán cân bằng tải mạnh mẽ hơn.
 - Nhiều reverse proxy cung cấp các tính năng mạnh mẽ bổ sung như viết lại URL (URL rewrites), caching, điểm kết thúc SSL (SSL termination point), các tính năng bảo mật

(ví dụ, bảo vệ chống tấn công từ chối dịch vụ - denial-of-service), hoặc thậm chí chức năng của các web server đầy đủ tính năng có thể được sử dụng để, ví dụ, phục vụ các file tĩnh.

That said, the `cluster` module could also be easily combined with a reverse proxy if necessary, for example, by using `cluster` to scale vertically inside a single machine and then using the reverse proxy to scale horizontally across different nodes.

Điều đó nói rằng, module `cluster` cũng có thể được kết hợp dễ dàng với một reverse proxy nếu cần, ví dụ, bằng cách sử dụng `cluster` để mở rộng theo chiều dọc bên trong một máy đơn lẻ và sau đó sử dụng reverse proxy để mở rộng theo chiều ngang trên các node khác nhau.

Pattern

Mẫu (Pattern)

Use a reverse proxy to balance the load of an application across multiple instances running on different ports or machines.

Sử dụng một reverse proxy để cân bằng tải của một ứng dụng trên nhiều instance chạy trên các cổng hoặc máy khác nhau.

We have many options to implement a load balancer using a reverse proxy. The following is a list of the most popular solutions:

Chúng ta có nhiều lựa chọn để triển khai một bộ cân bằng tải bằng một reverse proxy. Sau đây là danh sách các giải pháp phổ biến nhất:

- Nginx ([nodejsdp.link/nginx](#)): This is a web server, reverse proxy, and load balancer, built upon the non-**blocking** I/O model.
- HAProxy ([nodejsdp.link/haproxy](#)): This is a fast load balancer for TCP/ HTTP traffic.
- Node.js-based proxies: There are many solutions for the implementation of reverse proxies and load balancers directly in Node.js. This might have advantages and disadvantages, as we will see later.
- Cloud-based proxies: In the era of cloud computing, it's not rare to utilize a load balancer as a service. This can be convenient because it requires minimal maintenance, it's usually highly scalable, and sometimes it can support dynamic configurations to enable on-demand scalability.
 - Nginx ([nodejsdp.link/nginx](#)): Đây là một web server, reverse proxy, và bộ cân bằng tải, được xây dựng trên mô hình I/O không chặn.
 - HAProxy ([nodejsdp.link/haproxy](#)): Đây là một bộ cân bằng tải nhanh cho lưu lượng TCP/HTTP.
 - Các proxy dựa trên Node.js: Có nhiều giải pháp để triển khai các reverse proxy và bộ cân bằng tải trực tiếp trong Node.js. Điều này có thể có ưu điểm và nhược điểm, như chúng ta sẽ thấy sau.
 - Các proxy dựa trên đám mây (cloud-based): Trong kỷ nguyên điện toán đám mây, không hiếm khi sử dụng một bộ cân bằng tải như một dịch vụ. Điều này có thể tiện lợi vì nó yêu cầu bảo trì tối thiểu, thường có khả năng mở rộng cao, và đôi khi nó có thể hỗ trợ các cấu hình động để cho phép khả năng mở rộng theo nhu cầu.

In the next few sections of this chapter, we will analyze a sample configuration using Nginx. Later on, we will work on building our very own load balancer using nothing but Node.js!

Trong một vài phần tiếp theo của chương này, chúng ta sẽ phân tích một cấu hình mẫu sử dụng Nginx. Sau đó, chúng ta sẽ làm việc trên việc xây dựng bộ cân bằng tải của riêng mình bằng cách không sử dụng gì khác ngoài Node.js!

Load balancing with Nginx — Cân bằng tải với Nginx

To give you an idea of how reverse proxies work, we will now build a scalable architecture based on Nginx, but first, we need to install it. We can do that by following the instructions at nodejsdp.link/nginx-install. On the latest Ubuntu system, you can quickly install Nginx with the command `sudo apt-get install nginx`. On macOS, you can use `brew` (nodejsdp.link/brew): `brew install nginx`. Note that for the following examples, we will be using the latest version of Nginx available at the time of writing (1.17.10).

Để cho bạn một ý tưởng về cách các reverse proxy hoạt động, bây giờ chúng ta sẽ xây dựng một kiến trúc có khả năng mở rộng dựa trên Nginx, nhưng trước tiên, chúng ta cần cài đặt nó. Chúng ta có thể làm điều đó bằng cách làm theo các hướng dẫn tại nodejsdp.link/nginx-install. Trên hệ thống Ubuntu mới nhất, bạn có thể nhanh chóng cài đặt Nginx với lệnh `sudo apt-get install nginx`. Trên macOS, bạn có thể sử dụng `brew` (nodejsdp.link/brew): `brew install nginx`. Lưu ý rằng cho các ví dụ sau, chúng ta sẽ sử dụng phiên bản Nginx mới nhất có sẵn tại thời điểm viết (1.17.10).

Since we are not going to use `cluster` to start multiple instances of our server, we need to slightly modify the code of our application so that we can specify the listening port using a command-line argument. This will allow us to launch multiple instances on different ports. Let's consider the main module of our example application (`app.js`):

Vì chúng ta sẽ không sử dụng `cluster` để khởi động nhiều instance của máy chủ của mình, chúng ta cần sửa đổi một chút code của ứng dụng để chúng ta có thể chỉ định cổng lắng nghe bằng một đối số dòng lệnh. Điều này sẽ cho phép chúng ta khởi chạy nhiều instance trên các cổng khác nhau. Hãy xem xét module chính của ứng dụng ví dụ của chúng ta (`app.js`):

```
import { createServer } from 'http'
const { pid } = process
const server = createServer((req, res) => {
  let i = 1e7; while (i > 0) { i-- }
  console.log(`Handling request from ${pid}`)
  res.end(`Hello from ${pid}\n`)
})
const port = Number.parseInt(
  process.env.PORT || process.argv[2]
) || 8080
server.listen(port, () => console.log(`Started at ${pid}`))
```

The only difference between this version and the first version of our web server is that here, we are making the port number configurable through the `PORT` environment variable or a command-line argument. This is needed because we want to be able to start multiple instances of the server and allow them to listen on different ports.

Sự khác biệt duy nhất giữa phiên bản này và phiên bản đầu tiên của web server của chúng ta là ở đây, chúng ta đang làm cho số cổng có thể cấu hình thông qua biến môi trường `PORT` hoặc một đối số dòng lệnh. Điều này là cần thiết vì chúng ta muốn có thể khởi động nhiều instance của máy chủ và cho phép chúng lắng nghe trên các cổng khác nhau.

Another important feature that we won't have available without `cluster` is the automatic restart in case of a crash. Luckily, this is easy to fix by using a dedicated supervisor, that is, an external process that monitors our application and restarts it if necessary. The following are some possible choices:

Một tính năng quan trọng khác mà chúng ta sẽ không có sẵn nếu không có `cluster` là

khởi động lại tự động trong trường hợp sập. May mắn thay, điều này dễ khắc phục bằng cách sử dụng một trình giám sát (supervisor) chuyên dụng, tức là một tiến trình bên ngoài giám sát ứng dụng của chúng ta và khởi động lại nó nếu cần. Sau đây là một số lựa chọn có thể có:

- Node.js-based supervisors such as `forever` ([nodejsdp.link/forever](#)) or `pm2` ([nodejsdp.link/pm2](#))
- OS-based monitors such as `systemd` ([nodejsdp.link/systemd](#)) or `runit` ([nodejsdp.link/runit](#))
- More advanced monitoring solutions such as `monit` ([nodejsdp.link/monit](#)) or `supervisord` ([nodejsdp.link/supervisord](#))
- Container-based runtimes such as Kubernetes ([nodejsdp.link/kubernetes](#)), Nomad ([nodejsdp.link/nomad](#)), or Docker Swarm ([nodejsdp.link/swarm](#)).
 - Các trình giám sát dựa trên Node.js như `forever` ([nodejsdp.link/forever](#)) hoặc `pm2` ([nodejsdp.link/pm2](#))
 - Các trình giám sát dựa trên OS như `systemd` ([nodejsdp.link/systemd](#)) hoặc `runit` ([nodejsdp.link/runit](#))
 - Các giải pháp giám sát nâng cao hơn như `monit` ([nodejsdp.link/monit](#)) hoặc `supervisord` ([nodejsdp.link/supervisord](#))
 - Các runtime dựa trên container như Kubernetes ([nodejsdp.link/kubernetes](#)), Nomad ([nodejsdp.link/nomad](#)), hoặc Docker Swarm ([nodejsdp.link/swarm](#)).

For this example, we are going to use `forever`, which is the simplest and most immediate for us to use. We can install it globally by running the following command:

Cho ví dụ này, chúng ta sẽ sử dụng `forever`, vốn là đơn giản nhất và tức thì nhất để chúng ta sử dụng. Chúng ta có thể cài đặt nó toàn cục bằng cách chạy lệnh sau:

```
npm install forever -g
```

The next step is to start the four instances of our application, all on different ports and supervised by `forever`:

Bước tiếp theo là khởi động bốn instance của ứng dụng của chúng ta, tất cả trên các cổng khác nhau và được giám sát bởi `forever`:

```
forever start app.js 8081
forever start app.js 8082
forever start app.js 8083
forever start app.js 8084
```

We can check the list of the started processes using the command:

Chúng ta có thể kiểm tra danh sách các tiến trình đã khởi động bằng lệnh:

```
forever list
```

You can use `forever stopall` to stop all the Node.js processes previously started with `forever`. Alternatively, you can use `forever stop <id>` to stop a specific process from the ones shown with `forever list`.

Bạn có thể sử dụng `forever stopall` để dừng tất cả các tiến trình Node.js đã được khởi động trước đó với `forever`. Hoặc, bạn có thể sử dụng `forever stop <id>` để dừng một tiến trình cụ thể trong số những tiến trình được hiển thị với `forever list`.

Now, it's time to configure the Nginx server as a load balancer.

Bây giờ, đã đến lúc cấu hình máy chủ Nginx như một bộ cân bằng tải.

First, we need to create a minimal configuration file in our working directory that we will call `nginx.conf`.

Trước tiên, chúng ta cần tạo một file cấu hình tối thiểu trong thư mục làm việc của mình mà chúng ta sẽ gọi là `nginx.conf`.

Note that, because Nginx allows you to run multiple applications behind the same server instance, it is more common to use a global configuration file, which, in Unix systems, is generally located under `/usr/local/nginx/conf`, `/etc/nginx` or `/usr/local/etc/nginx`. Here, by having a configuration file in our working folder, we are taking a simpler approach. This is ok for the sake of this demo as we want to run just one application locally, but we advise you follow the recommended best practices for production deployments. Next, let's write the `nginx.conf` file and apply the following configuration, which is the very minimum required to get a working load balancer for our Node.js processes:

Lưu ý rằng, vì Nginx cho phép bạn chạy nhiều ứng dụng đứng sau cùng một instance máy chủ, việc sử dụng một file cấu hình toàn cục là phổ biến hơn, mà trong các hệ thống Unix, thường nằm dưới `/usr/local/nginx/conf`, `/etc/nginx` hoặc `/usr/local/etc/nginx`. Ở đây, bằng cách có một file cấu hình trong thư mục làm việc của chúng ta, chúng ta đang áp dụng một cách tiếp cận đơn giản hơn. Điều này ổn cho mục đích của bản demo này vì chúng ta muốn chỉ chạy một ứng dụng cục bộ, nhưng chúng tôi khuyên bạn nên tuân theo các thực hành tốt nhất được khuyến nghị cho việc triển khai production. Tiếp theo, hãy viết file `nginx.conf` và áp dụng cấu hình sau, vốn là mức tối thiểu cần thiết để có được một bộ cân bằng tải hoạt động cho các tiến trình Node.js của chúng ta:

```
daemon off;                                ## (1)
error_log /dev/stderr info;                 ## (2)
events {                                    ## (3)
    worker_connections 2048;
}
http {                                      ## (4)
    access_log /dev/stdout;
    upstream my-load-balanced-app {
        server 127.0.0.1:8081;
        server 127.0.0.1:8082;
        server 127.0.0.1:8083;
        server 127.0.0.1:8084;
    }
    server {
        listen 8080;
        location / {
            proxy_pass http://my-load-balanced-app;
        }
    }
}
```

Let's discuss this configuration together:

Hãy cùng thảo luận về cấu hình này:

1. The declaration `daemon off` allows us to run Nginx as a standalone process using the current unprivileged user and by keeping the process running in the foreground of the current terminal (which allows us to shut it down using `Ctrl + C`).

2. We use `error_log` (and later in the `http` block, `access_log`) to **stream** errors and access logs respectively to the standard output and standard error, so we can read the logs in real time straight from our terminal.
3. The `events` block allows us to configure how network connections are managed by Nginx. Here, we are setting the maximum number of simultaneous connections that can be opened by an Nginx worker process to 2048.
4. The `http` block allows us to define the configuration for a given application. In the `upstream my-load-balanced-app` section, we are defining the list of backend servers used to handle the network requests. In the `server` section, we use `listen 8080` to instruct the server to listen on port 8080 and finally, we specify the `proxy_pass` directive, which essentially tells Nginx to forward any request to the server group we defined before (`my-load-balanced-app`).
 1. Khai báo `daemon off` cho phép chúng ta chạy Nginx như một tiến trình độc lập sử dụng người dùng không có đặc quyền hiện tại và bằng cách giữ tiến trình chạy ở tiền cảnh (foreground) của terminal hiện tại (điều này cho phép chúng ta tắt nó bằng Ctrl + C).
 2. Chúng ta sử dụng `error_log` (và sau đó trong khối `http`, `access_log`) để truyền (stream) các log lỗi và log truy cập tương ứng đến đầu ra chuẩn và lỗi chuẩn, để chúng ta có thể đọc các log theo thời gian thực trực tiếp từ terminal của mình.
 3. Khối `events` cho phép chúng ta cấu hình cách các kết nối mạng được quản lý bởi Nginx. Ở đây, chúng ta đang đặt số lượng kết nối đồng thời tối đa có thể được mở bởi một tiến trình worker của Nginx thành 2048.
 4. Khối `http` cho phép chúng ta định nghĩa cấu hình cho một ứng dụng nhất định. Trong phần `upstream my-load-balanced-app`, chúng ta đang định nghĩa danh sách các máy chủ backend được sử dụng để xử lý các yêu cầu mạng. Trong phần `server`, chúng ta sử dụng `listen 8080` để chỉ thị máy chủ lắng nghe trên cổng 8080 và cuối cùng, chúng ta chỉ định chỉ thị `proxy_pass`, vốn về cơ bản nói với Nginx chuyển tiếp bất kỳ yêu cầu nào đến nhóm máy chủ mà chúng ta đã định nghĩa trước đó (`my-load-balanced-app`).

That's it! Now, we only need to start Nginx using our configuration file with the following command:

Vậy là xong! Bây giờ, chúng ta chỉ cần khởi động Nginx bằng file cấu hình của mình với lệnh sau:

```
nginx -c ${PWD}/nginx.conf
```

Our system should now be up and running, ready to accept requests and balance the traffic across the four instances of our Node.js application. Simply point your browser to the address `http://localhost:8080` to see how the traffic is balanced by our Nginx server. You can also try again to load test this application using autocannon. Since we are still running all the processes in one local machine, your results should not diverge much from what you got when benchmarking the version using the cluster module approach.

Hệ thống của chúng ta giờ sẽ hoạt động, sẵn sàng chấp nhận các yêu cầu và cân bằng lưu lượng trên bốn instance của ứng dụng Node.js của chúng ta. Chỉ cần trở trình duyệt của bạn đến địa chỉ `http://localhost:8080` để xem cách lưu lượng được cân bằng bởi máy chủ Nginx của chúng ta. Bạn cũng có thể thử lại để kiểm tra tải ứng dụng này bằng autocannon. Vì chúng ta vẫn đang chạy tất cả các tiến trình trên một máy cục bộ, kết quả của bạn không nên khác nhiều so với những gì bạn nhận được khi benchmark phiên bản sử dụng cách tiếp cận module cluster.

This example demonstrated how to use Nginx to load balance traffic. For simplicity, we kept everything locally on our machine, but nonetheless, this was a great exercise to get us ready to

deploy an application on multiple remote servers. If you want to try to do that, you will essentially have to follow this recipe:

Ví dụ này đã minh họa cách sử dụng Nginx để cân bằng tải lưu lượng. Để cho đơn giản, chúng ta đã giữ mọi thứ cục bộ trên máy của mình, nhưng dù sao đi nữa, đây là một bài tập tuyệt vời để chuẩn bị cho chúng ta triển khai một ứng dụng trên nhiều máy chủ từ xa. Nếu bạn muốn thử làm điều đó, về cơ bản bạn sẽ phải làm theo công thức này:

1. Provision n backend servers running the Node.js application (running multiple instances with a service monitor like `forever` or by using the `cluster` module).
 2. Provision a load balancer machine that has Nginx installed and all the necessary configuration to route the traffic to the n backend servers. Every process in every server should be listed in the `upstream` block of your Nginx configuration file using the correct address of the various machines in the network.
 3. Make your load balancer publicly available on the internet by using a public IP and possibly a public domain name.
 4. Try to send some traffic to the load balancer's public address by using a browser or a benchmarking tool like `autocannon`.
1. Cấp phát n máy chủ backend chạy ứng dụng Node.js (chạy nhiều instance với một trình giám sát dịch vụ như `forever` hoặc bằng cách sử dụng module `cluster`).
 2. Cấp phát một máy bộ cân bằng tải đã cài đặt Nginx và tất cả cấu hình cần thiết để định tuyến lưu lượng đến n máy chủ backend. Mỗi tiến trình trong mỗi máy chủ nên được liệt kê trong khối `upstream` của file cấu hình Nginx của bạn sử dụng địa chỉ chính xác của các máy khác nhau trong mạng.
 3. Làm cho bộ cân bằng tải của bạn khả dụng công khai trên internet bằng cách sử dụng một IP công khai và có thể là một tên miền công khai.
 4. Thử gửi một số lưu lượng đến địa chỉ công khai của bộ cân bằng tải bằng cách sử dụng một trình duyệt hoặc một công cụ benchmark như `autocannon`.

For simplicity, you can perform all these steps manually by spinning servers through your cloud provider admin interface and by using SSH to log in to those. Alternatively, you could choose tools that allow you to automate these tasks by writing infrastructure as code such as Terraform (nodejsdp.link/terraform), Ansible (nodejsdp.link/ansible), and Packer (nodejsdp.link/packer).

Để cho đơn giản, bạn có thể thực hiện tất cả các bước này thủ công bằng cách khởi tạo các máy chủ thông qua giao diện quản trị của nhà cung cấp đám mây của bạn và bằng cách sử dụng SSH để đăng nhập vào chúng. Hoặc, bạn có thể chọn các công cụ cho phép bạn tự động hóa các tác vụ này bằng cách viết hạ tầng dưới dạng code (infrastructure as code) như Terraform (nodejsdp.link/terraform), Ansible (nodejsdp.link/ansible), và Packer (nodejsdp.link/packer).

In this example, we used a predefined number of backend servers. In the next section, we will explore a technique that allows us to load balance traffic to a dynamic set of backend servers.

Trong ví dụ này, chúng ta đã sử dụng một số lượng máy chủ backend được xác định trước. Trong phần tiếp theo, chúng ta sẽ khám phá một kỹ thuật cho phép chúng ta cân bằng tải lưu lượng đến một tập hợp động các máy chủ backend.

Dynamic horizontal scaling — Mở rộng theo chiều ngang động (dynamic horizontal scaling)

One important advantage of modern cloud-based infrastructure is the ability to dynamically adjust the capacity of an application based on the current or predicted traffic. This is also known as dynamic scaling. If implemented properly, this practice can reduce the cost of the IT infrastructure enormously while still keeping the application highly available and responsive.

Một lợi thế quan trọng của hạ tầng hiện đại dựa trên đám mây là khả năng điều chỉnh năng lực của một ứng dụng một cách động dựa trên lưu lượng hiện tại hoặc được dự đoán. Điều này còn được gọi là mở rộng động (dynamic scaling). Nếu được triển khai đúng cách, thực hành này có thể giảm chi phí của hạ tầng IT một cách to lớn trong khi vẫn giữ ứng dụng có tính sẵn sàng cao và phản hồi nhanh.

The idea is simple: if our application is experiencing a performance degradation caused by a peak in traffic, the system automatically spawns new servers to cope with the increased load. Similarly, if we see that the allocated resources are underutilized, we can shut some servers down to reduce the cost of the running infrastructure. We could also decide to perform scaling operations based on a schedule; for instance, we could shut down some servers during certain hours of the day when we know that the traffic will be lighter, and restart them again just before the peak hours. These mechanisms require the load balancer to always be up-to-date with the current network topology, knowing at any time which server is up.

Ý tưởng rất đơn giản: nếu ứng dụng của chúng ta đang trải qua sự suy giảm hiệu năng do một đỉnh điểm lưu lượng, hệ thống tự động sinh ra các máy chủ mới để đối phó với tải tăng lên. Tương tự, nếu chúng ta thấy rằng các tài nguyên được cấp phát đang bị sử dụng dưới mức, chúng ta có thể tắt một số máy chủ để giảm chi phí của hạ tầng đang chạy. Chúng ta cũng có thể quyết định thực hiện các thao tác mở rộng dựa trên một lịch trình; ví dụ, chúng ta có thể tắt một số máy chủ trong những giờ nhất định trong ngày khi chúng ta biết rằng lưu lượng sẽ nhẹ hơn, và khởi động lại chúng ngay trước giờ cao điểm. Các cơ chế này yêu cầu bộ cân bằng tải luôn được cập nhật với topo mạng hiện tại, biết bất kỳ lúc nào máy chủ nào đang hoạt động.

Using a service registry — Sử dụng một service registry

A common pattern to solve this problem is to use a central repository called a service registry, which keeps track of the running servers and the services they provide. Figure 12.7 shows a multiservice architecture with a load balancer on the front, dynamically configured using a service registry:

Một mẫu phổ biến để giải quyết vấn đề này là sử dụng một kho lưu trữ trung tâm gọi là service registry, vốn theo dõi các máy chủ đang chạy và các dịch vụ mà chúng cung cấp. Hình 12.7 cho thấy một kiến trúc đa dịch vụ với một bộ cân bằng tải ở phía trước, được cấu hình động bằng một service registry:

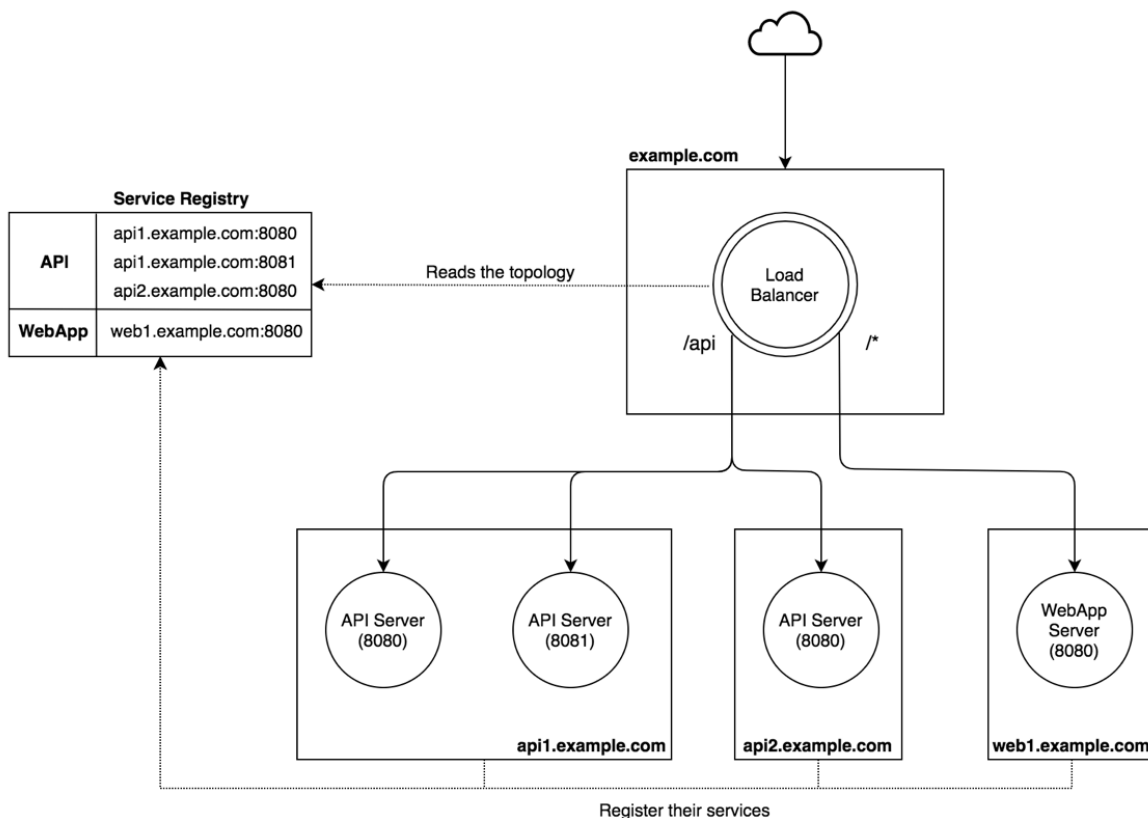


Figure 12.7: A multiservice architecture with a load balancer on the front, dynamically configured using a service registry

Hình 12.7: Một kiến trúc đa dịch vụ với một bộ cân bằng tải ở phía trước, được cấu hình động bằng một service registry

The architecture in Figure 12.7 assumes the presence of two services, API and WebApp. There can be one or many instances of each service, spread across multiple servers.

Kiến trúc trong Hình 12.7 giả định sự hiện diện của hai dịch vụ, API và WebApp. Có thể có một hoặc nhiều instance của mỗi dịch vụ, trải rộng trên nhiều máy chủ.

When a request to `example.com` is received, the load balancer checks the prefix of the request path. If the prefix is `/api`, the request is load balanced between the available instances of the API service. In Figure 12.7, we have two instances running on the server `api1.example.com` and one instance running on `api2.example.com`. For all the other path prefixes, the request is load balanced between the available instances of the WebApp service. In the diagram, we have only one WebApp instance, which is running on the server `web1.example.com`. The load balancer obtains the list of servers and service instances running on every server using the service registry. For this to work in complete automation, each application instance has to register itself to the service registry the moment it comes up online and unregister itself when it stops. This way, the load balancer can always have an up-to-date view of the servers and the services available on the network.

Khi một yêu cầu đến `example.com` được nhận, bộ cân bằng tải kiểm tra tiền tố của đường dẫn yêu cầu. Nếu tiền tố là `/api`, yêu cầu được cân bằng tải giữa các instance khả dụng của dịch vụ API. Trong Hình 12.7, chúng ta có hai instance chạy trên máy chủ

api1.example.com và một instance chạy trên api2.example.com. Đối với tất cả các tiền tố đường dẫn khác, yêu cầu được cân bằng tải giữa các instance khả dụng của dịch vụ WebApp. Trong sơ đồ, chúng ta chỉ có một instance WebApp, vốn đang chạy trên máy chủ web1.example.com. Bộ cân bằng tải lấy danh sách các máy chủ và các instance dịch vụ chạy trên mỗi máy chủ bằng cách sử dụng service registry. Để điều này hoạt động trong sự tự động hoàn toàn, mỗi instance ứng dụng phải tự đăng ký vào service registry vào lúc nó trực tuyến và hủy đăng ký chính nó khi nó dừng lại. Bằng cách này, bộ cân bằng tải luôn có thể có một cái nhìn được cập nhật về các máy chủ và các dịch vụ khả dụng trên mạng.

Pattern (service registry)

Mẫu (service registry)

Use a central repository to store an always up-to-date view of the servers and the services available in a system.

Sử dụng một kho lưu trữ trung tâm để lưu trữ một cái nhìn luôn được cập nhật về các máy chủ và các dịch vụ khả dụng trong một hệ thống.

While this pattern is useful to load balance traffic, it has the added benefit of being able to decouple service instances from the servers on which they are running. We can look at the Service Registry pattern as an implementation of the Service Locator **Design pattern** applied to network services.

Mặc dù mẫu này hữu ích để cân bằng tải lưu lượng, nó có thêm lợi ích là có thể tách rời các instance dịch vụ khỏi các máy chủ mà chúng đang chạy. Chúng ta có thể nhìn vào mẫu Service Registry như một sự triển khai của mẫu thiết kế Service Locator áp dụng cho các dịch vụ mạng.

Implementing a dynamic load balancer with http-proxy and Consul — Triển khai một bộ cân bằng tải động với http-proxy và Consul

To support a dynamic network infrastructure, we can use a reverse proxy such as Nginx or HAProxy: all we need to do is update their configuration using an automated service and then force the load balancer to pick the changes. For Nginx, this can be done using the following command line:

Để hỗ trợ một hạ tầng mạng động, chúng ta có thể sử dụng một reverse proxy như Nginx hoặc HAProxy: tất cả những gì chúng ta cần làm là cập nhật cấu hình của chúng bằng một dịch vụ tự động và sau đó buộc bộ cân bằng tải nhận các thay đổi. Đối với Nginx, điều này có thể được thực hiện bằng dòng lệnh sau:

```
nginx -s reload
```

The same result can be achieved with a cloud-based solution, but we have a third and more familiar alternative that makes use of our favorite platform.

Cùng kết quả này có thể đạt được với một giải pháp dựa trên đám mây, nhưng chúng ta có một lựa chọn thứ ba và quen thuộc hơn sử dụng nền tảng yêu thích của chúng ta.

We all know that Node.js is a great tool for building any sort of network application and, as we said throughout this book, this is exactly one of its main design goals. So, why not build a load balancer using nothing but Node.js? This would give us much more freedom and power and would allow us to implement any sort of pattern or algorithm straight into our custom-built load balancer, including the one we are now going to explore: dynamic load balancing using a service registry. Furthermore, working on this exercise will definitely help us to understand even better how **production-grade** products such as Nginx and HAProxy actually work. In this example, we are going to use Consul

(`nodejsdp.link/consul`) as the service registry to replicate the multiservice architecture we saw in Figure 12.7. To do that, we are going to mainly use three npm packages:

Tất cả chúng ta đều biết rằng Node.js là một công cụ tuyệt vời để xây dựng bất kỳ loại ứng dụng mạng nào và, như chúng ta đã nói xuyên suốt cuốn sách này, đây chính là một trong những mục tiêu thiết kế chính của nó. Vậy, tại sao không xây dựng một bộ cân bằng tải bằng cách không sử dụng gì khác ngoài Node.js? Điều này sẽ cho chúng ta nhiều tự do và sức mạnh hơn nhiều và sẽ cho phép chúng ta triển khai bất kỳ loại mẫu hoặc thuật toán nào trực tiếp vào bộ cân bằng tải tùy chỉnh của chúng ta, bao gồm cả cái mà chúng ta sắp khám phá: cân bằng tải động sử dụng một service registry. Hơn nữa, làm việc trên bài tập này chắc chắn sẽ giúp chúng ta hiểu rõ hơn nữa cách các sản phẩm cấp độ production như Nginx và HAProxy thực sự hoạt động. Trong ví dụ này, chúng ta sẽ sử dụng Consul (`nodejsdp.link/consul`) làm service registry để tái tạo kiến trúc đa dịch vụ mà chúng ta đã thấy trong Hình 12.7. Để làm điều đó, chúng ta sẽ chủ yếu sử dụng ba gói npm:

- `http-proxy` (`nodejsdp.link/http-proxy`): To simplify the creation of a reverse proxy/load balancer in Node.js
- `portfinder` (`nodejsdp.link/portfinder`): To find a free port in the system
- `consul` (`nodejsdp.link/consul-lib`): To interact with Consul
 - `http-proxy` (`nodejsdp.link/http-proxy`): Để đơn giản hóa việc tạo một reverse proxy/bộ cân bằng tải trong Node.js
 - `portfinder` (`nodejsdp.link/portfinder`): Để tìm một cổng trống trong hệ thống
 - `consul` (`nodejsdp.link/consul-lib`): Để tương tác với Consul

Let's start by implementing our services. These are simple HTTP servers like the ones we have used so far to test `cluster` and Nginx, but this time, we want each server to register itself into the service registry the moment it starts.

Hãy bắt đầu bằng cách triển khai các dịch vụ của chúng ta. Đây là các HTTP server đơn giản như những cái mà chúng ta đã sử dụng cho đến nay để kiểm thử `cluster` và Nginx, nhưng lần này, chúng ta muốn mỗi máy chủ tự đăng ký vào service registry vào lúc nó khởi động.

Let's see how this looks (file `app.js`):

Hãy xem điều này trông như thế nào (file `app.js`):

```
import { createServer } from 'http'
import consul from 'consul'
import portfinder from 'portfinder'
import { nanoid } from 'nanoid'
const serviceType = process.argv[2]
const { pid } = process
async function main () {
  const consulClient = consul()
  const port = await portfinder.getPortPromise() // (1)
  const address = process.env.ADDRESS || 'localhost'
  const serviceId = nanoid()
  function registerService () { // (2)
    consulClient.agent.service.register({
      id: serviceId,
      name: serviceType,
      address,
```

```

    port,
    tags: [serviceType]
  }, () => {
    console.log(`${serviceType} registered successfully`)
  })
}
function unregisterService (err) { // (3)
  err && console.error(err)
  console.log(`deregistering ${serviceId}`)
  consulClient.agent.service.deregister(serviceId, () => {
    process.exit(err ? 1 : 0)
  })
}
process.on('exit', unregisterService) // (4)
process.on('uncaughtException', unregisterService)
process.on('SIGINT', unregisterService)
const server = createServer((req, res) => { // (5)
  let i = 1e7; while (i > 0) { i-- }
  console.log(`Handling request from ${pid}`)
  res.end(`${serviceType} response from ${pid}\n`)
})
server.listen(port, address, () => {
  registerService()
  console.log(`Started ${serviceType} at ${pid} on port ${port}`)
})
}
main().catch((err) => {
  console.error(err)
  process.exit(1)
})

```

In the preceding code, there are some parts that deserve our attention:

Trong đoạn code trên, có một số phần đáng để chúng ta chú ý:

1. First, we use `portfinder.getPortPromise()` to discover a free port in the system (by default, `portfinder` starts to search from port 8000). We also allow the user to configure the address based on the environment variable `ADDRESS`. Finally, we generate a random ID to identify this service using `nanoid` (`nodejsdp.link/nanoid`).
2. Next, we declare the `registerService()` function, which uses the `consul` library to register a new service in the registry. The service definition needs several attributes: `id` (a unique identifier for the service), `name` (a generic name that identifies the service), `address` and `port` (to identify how to access the service), and `tags` (an optional array of tags that can be used to filter and group services). We are using `serviceType` (which we get from the commandline arguments) to specify the service name and to add a tag. This will allow us to identify all the services of the same type available in the cluster.
3. At this point, we define a function called `unregisterService()`, which allows us to remove the service we just registered in Consul.
4. We use `unregisterService()` as a cleanup function so that when the program is closed (either intentionally or by accident), the service is unregistered from Consul.
5. Finally, we start the HTTP server for our service on the port discovered by `portfinder` and the address configured for the current service. Note that when the server is started, we make

sure to invoke the `registerService()` function to make sure that the service is registered for discovery.

1. Đầu tiên, chúng ta sử dụng `portfinder.getPortPromise()` để khám phá một cổng trống trong hệ thống (theo mặc định, `portfinder` bắt đầu tìm kiếm từ cổng 8000). Chúng ta cũng cho phép người dùng cấu hình địa chỉ dựa trên biến môi trường `ADDRESS`. Cuối cùng, chúng ta tạo một ID ngẫu nhiên để xác định dịch vụ này bằng `nanoid` (`nodejsdp.link/nanoid`).
2. Tiếp theo, chúng ta khai báo hàm `registerService()`, vốn sử dụng thư viện `consul` để đăng ký một dịch vụ mới trong registry. Định nghĩa dịch vụ cần một số thuộc tính: `id` (một định danh duy nhất cho dịch vụ), `name` (một tên chung xác định dịch vụ), `address` và `port` (để xác định cách truy cập dịch vụ), và `tags` (một mảng tag tùy chọn có thể được sử dụng để lọc và nhóm các dịch vụ). Chúng ta đang sử dụng `serviceType` (cái mà chúng ta nhận từ các đối số dòng lệnh) để chỉ định tên dịch vụ và để thêm một tag. Điều này sẽ cho phép chúng ta xác định tất cả các dịch vụ cùng loại khả dụng trong cluster.
3. Tại thời điểm này, chúng ta định nghĩa một hàm gọi là `unregisterService()`, vốn cho phép chúng ta xóa dịch vụ mà chúng ta vừa đăng ký trong Consul.
4. Chúng ta sử dụng `unregisterService()` như một hàm dọn dẹp (cleanup) để khi chương trình bị đóng (dù có chủ ý hay do tai nạn), dịch vụ được hủy đăng ký khỏi Consul.
5. Cuối cùng, chúng ta khởi động HTTP server cho dịch vụ của chúng ta trên cổng được khám phá bởi `portfinder` và địa chỉ được cấu hình cho dịch vụ hiện tại. Lưu ý rằng khi máy chủ được khởi động, chúng ta đảm bảo gọi hàm `registerService()` để đảm bảo rằng dịch vụ được đăng ký để khám phá.

With this script, we will be able to start and register different types of applications.

Với script này, chúng ta sẽ có thể khởi động và đăng ký các loại ứng dụng khác nhau.

Now, it's time to implement the load balancer. Let's do that by creating a new module called `loadBalancer.js`:

Bây giờ, đã đến lúc triển khai bộ cân bằng tải. Hãy làm điều đó bằng cách tạo một module mới gọi là `loadBalancer.js`:

```
import { createServer } from 'http'
import httpProxy from 'http-proxy'
import consul from 'consul'
const routing = [ // (1)
  {
    path: '/api',
    service: 'api-service',
    index: 0
  },
  {
    path: '/',
    service: 'webapp-service',
    index: 0
  }
]
const consulClient = consul() // (2)
const proxy = httpProxy.createProxyServer()
```

```

const server = createServer((req, res) => {
  const route = routing.find((route) =>                // (3)
    req.url.startsWith(route.path))
  consulClient.agent.service.list((err, services) => {   // (4)
    const servers = !err && Object.values(services)
      .filter(service => service.Tags.includes(route.service))
    if (err || !servers.length) {
      res.writeHead(502)
      return res.end('Bad gateway')
    }
    route.index = (route.index + 1) % servers.length    // (5)
    const server = servers[route.index]
    const target = `http://${server.Address}:${server.Port}`
    proxy.web(req, res, { target })
  })
})
server.listen(8080, () => {
  console.log('Load balancer started on port 8080')
})

```

This is how we implemented our Node.js-based load balancer:

Đây là cách chúng ta đã triển khai bộ cân bằng tải dựa trên Node.js của mình:

1. First, we define our load balancer routes. Each item in the routing array contains the service used to handle the requests arriving on the mapped path. The index property will be used to round-robin the requests of a given service.
2. We need to instantiate a consul client so that we can have access to the registry. Next, we instantiate an http-proxy server.
3. In the request handler of the server, the first thing we do is match the URL against our routing table. The result will be a descriptor containing the service name.
4. We obtain from consul the list of servers implementing the required service. If this list is empty or there was an error retrieving it, then we return an error to the client. We use the Tags attribute to filter all the available services and find the address of the servers that implement the current service type.
5. At last, we can route the request to its destination. We update route.index to point to the next server in the list, following a round-robin approach. We then use the index to select a server from the list, passing it to proxy.web(), along with the request (req) and the response (res) objects. This will simply forward the request to the server we chose.

1. Đầu tiên, chúng ta định nghĩa các route của bộ cân bằng tải của mình. Mỗi mục trong mảng routing chứa service được sử dụng để xử lý các yêu cầu đến trên path được ánh xạ. Thuộc tính index sẽ được sử dụng để round-robin các yêu cầu của một dịch vụ nhất định.
2. Chúng ta cần khởi tạo một client consul để chúng ta có thể có quyền truy cập vào registry. Tiếp theo, chúng ta khởi tạo một máy chủ http-proxy.
3. Trong trình xử lý yêu cầu của máy chủ, điều đầu tiên chúng ta làm là so khớp URL với bảng định tuyến của mình. Kết quả sẽ là một descriptor chứa tên dịch vụ.
4. Chúng ta lấy từ consul danh sách các máy chủ triển khai dịch vụ cần thiết. Nếu danh sách này trống hoặc có lỗi khi lấy nó, thì chúng ta trả về một lỗi cho client. Chúng ta sử dụng thuộc tính Tags để lọc tất cả các dịch vụ khả dụng và tìm địa chỉ của các máy chủ triển khai loại dịch vụ hiện tại.

5. Cuối cùng, chúng ta có thể định tuyến yêu cầu đến đích của nó. Chúng ta cập nhật `route.index` để trở đến máy chủ tiếp theo trong danh sách, theo cách tiếp cận round-robin. Sau đó chúng ta sử dụng `index` để chọn một máy chủ từ danh sách, chuyển nó cho `proxy.web()`, cùng với các đối tượng yêu cầu (`req`) và phản hồi (`res`). Điều này sẽ đơn giản chuyển tiếp yêu cầu đến máy chủ mà chúng ta đã chọn.

It is now clear how simple it is to implement a load balancer using only Node.js and a service registry, as well as how much flexibility we can have by doing so. Note that in order to keep the implementation simple, we intentionally left out some interesting optimization opportunities. For instance, in this implementation, we are interrogating `consul` to get the list of registered services for every single request. This is something that can add a significant overhead, especially if our load balancer receives requests with a high frequency. It would be more efficient to cache the list of services and refresh it on a regular basis (for instance, every 10 seconds). Another optimization could be to use the `cluster` module to run multiple instances of our load balancer and distribute the load across all the available cores in the machine.

Bây giờ thì đã rõ việc triển khai một bộ cân bằng tải chỉ bằng Node.js và một service registry đơn giản như thế nào, cũng như chúng ta có thể có bao nhiêu sự linh hoạt khi làm như vậy. Lưu ý rằng để giữ cho việc triển khai đơn giản, chúng ta đã cố ý bỏ qua một số cơ hội tối ưu hóa thú vị. Ví dụ, trong cách triển khai này, chúng ta đang truy vấn `consul` để lấy danh sách các dịch vụ đã đăng ký cho từng yêu cầu đơn lẻ. Đây là điều có thể thêm overhead đáng kể, đặc biệt nếu bộ cân bằng tải của chúng ta nhận các yêu cầu với tần suất cao. Sẽ hiệu quả hơn nếu cache danh sách các dịch vụ và làm mới nó một cách định kỳ (ví dụ, mỗi 10 giây). Một tối ưu hóa khác có thể là sử dụng module `cluster` để chạy nhiều instance của bộ cân bằng tải của chúng ta và phân phối tải trên tất cả các lõi khả dụng trong máy.

Now, we should be ready to give our system a try, but first, let's install the Consul server by following the official documentation at `nodejsdp.link/consul-install`.

Bây giờ, chúng ta nên sẵn sàng để thử hệ thống của mình, nhưng trước tiên, hãy cài đặt máy chủ Consul bằng cách làm theo tài liệu chính thức tại `nodejsdp.link/consul-install`.

This allows us to start the Consul service registry on our development machine with this simple command line:

Điều này cho phép chúng ta khởi động service registry Consul trên máy phát triển của mình với dòng lệnh đơn giản này:

```
consul agent -dev
```

Now, we are ready to start the load balancer (using `forever` to make sure the application is restarted in case of a crash):

Bây giờ, chúng ta đã sẵn sàng để khởi động bộ cân bằng tải (sử dụng `forever` để đảm bảo ứng dụng được khởi động lại trong trường hợp sập):

```
forever start loadBalancer.js
```

Now, if we try to access some of the services exposed by the load balancer, we will notice that it returns an HTTP 502 error, because we didn't start any servers yet. Try it yourself:

Bây giờ, nếu chúng ta thử truy cập một số dịch vụ được phơi bày bởi bộ cân bằng tải, chúng ta sẽ nhận thấy rằng nó trả về một lỗi HTTP 502, vì chúng ta chưa khởi động bất kỳ máy chủ nào. Hãy tự thử:

```
curl localhost:8080/api
```

The preceding command should return the following output:

Lệnh trên sẽ trả về đầu ra sau:

Bad Gateway

The situation will change if we spawn some instances of our services, for example, two `api-service` and one `webapp-service`:

Tình huống sẽ thay đổi nếu chúng ta sinh ra một số instance của các dịch vụ của mình, ví dụ, hai `api-service` và một `webapp-service`:

```
forever start --killSignal=SIGINT app.js api-service
forever start --killSignal=SIGINT app.js api-service
forever start --killSignal=SIGINT app.js webapp-service
```

Now, the load balancer should automatically see the new servers and start distributing requests across them. Let's try again with the following command:

Bây giờ, bộ cân bằng tải sẽ tự động thấy các máy chủ mới và bắt đầu phân phối các yêu cầu trên chúng. Hãy thử lại với lệnh sau:

```
curl localhost:8080/api
```

The preceding command should now return this:

Lệnh trên giờ sẽ trả về cái này:

api-service response **from 6972**

By running this again, we should now receive a message from another server, confirming that the requests are being distributed evenly among the different servers:

Bằng cách chạy lại lệnh này, chúng ta giờ sẽ nhận được một thông điệp từ một máy chủ khác, xác nhận rằng các yêu cầu đang được phân phối đều giữa các máy chủ khác nhau:

api-service response **from 6979**

If you want to see the instances managed by `forever` and stop some of them you can use the commands `forever list` and `forever stop`. To stop all running instances you can use `forever stopall`. Why don't you try to stop one of the running instances of the `api-service` to see what happens to the whole application?

Nếu bạn muốn xem các instance được quản lý bởi `forever` và dừng một số trong số chúng, bạn có thể sử dụng các lệnh `forever list` và `forever stop`. Để dừng tất cả các instance đang chạy, bạn có thể sử dụng `forever stopall`. Tại sao bạn không thử dừng một trong các instance đang chạy của `api-service` để xem điều gì xảy ra với toàn bộ ứng dụng?

The advantages of this pattern are immediate. We can now scale our infrastructure dynamically, on demand, or based on a schedule, and our load balancer will automatically adjust with the new configuration without any extra effort!

Các lợi thế của mẫu này là tức thì. Bây giờ chúng ta có thể mở rộng hạ tầng của mình một cách động, theo nhu cầu, hoặc dựa trên một lịch trình, và bộ cân bằng tải của chúng ta sẽ tự động điều chỉnh với cấu hình mới mà không cần bất kỳ nỗ lực thêm nào!

Consul offers a convenient web UI available at `localhost:8500` by default. Check it out while playing with this example to see how services appear and disappear as they get registered or unregistered.

Consul cung cấp một web UI tiện lợi có sẵn tại `localhost:8500` theo mặc định. Hãy xem qua nó trong khi chơi với ví dụ này để thấy cách các dịch vụ xuất hiện và biến mất khi chúng được đăng ký hoặc hủy đăng ký.

Consul also offers a health check feature to monitor registered services. This feature could be integrated within our example to make our infrastructure even more resilient to failures. In fact, if a service does not respond to a health check, it gets automatically removed from the registry and therefore, it won't receive traffic anymore. If you are curious to see how you can implement this feature, you can check out the official documentation for Checks at nodejsdp.link/consul-checks.

Consul cũng cung cấp một tính năng kiểm tra sức khỏe (health check) để giám sát các dịch vụ đã đăng ký. Tính năng này có thể được tích hợp trong ví dụ của chúng ta để làm cho hạ tầng của chúng ta thậm chí còn có khả năng phục hồi tốt hơn trước các thất bại. Trên thực tế, nếu một dịch vụ không phản hồi một health check, nó tự động bị xóa khỏi registry và do đó, nó sẽ không nhận lưu lượng nữa. Nếu bạn tò mò muốn xem cách bạn có thể triển khai tính năng này, bạn có thể xem tài liệu chính thức về Checks tại nodejsdp.link/consul-checks.

Now that we know how to perform dynamic load balancing using a load balancer and a service registry, we are ready to explore some interesting alternative approaches, like peer-to-peer load balancing.

Bây giờ chúng ta đã biết cách thực hiện cân bằng tải động bằng một bộ cân bằng tải và một service registry, chúng ta đã sẵn sàng để khám phá một số cách tiếp cận thay thế thú vị, như cân bằng tải peer-to-peer.

Peer-to-peer load balancing — Cân bằng tải peer-to-peer

Using a reverse proxy is almost a necessity when we want to expose a complex internal network architecture to a public network such as the Internet. It helps hide the complexity, providing a single access point that external applications can easily use and rely on. However, if we need to scale a service that is for internal use only, we can have much more flexibility and control.

Việc sử dụng một reverse proxy gần như là một điều cần thiết khi chúng ta muốn phơi bày một kiến trúc mạng nội bộ phức tạp cho một mạng công khai như Internet. Nó giúp che giấu độ phức tạp, cung cấp một điểm truy cập duy nhất mà các ứng dụng bên ngoài có thể dễ dàng sử dụng và dựa vào. Tuy nhiên, nếu chúng ta cần mở rộng một dịch vụ chỉ dành cho sử dụng nội bộ, chúng ta có thể có nhiều linh hoạt và kiểm soát hơn nhiều.

Let's imagine having a service, Service A, that relies on Service B to implement its functionality. Service B is scaled across multiple machines and it's available only in the internal network. What we have learned so far is that Service A will connect to Service B using a load balancer, which will distribute the traffic to all the servers implementing Service B.

Hãy tưởng tượng có một dịch vụ, Service A, vốn dựa vào Service B để triển khai chức năng của nó. Service B được mở rộng trên nhiều máy và nó chỉ khả dụng trong mạng nội bộ. Những gì chúng ta đã học cho đến nay là Service A sẽ kết nối đến Service B bằng một bộ cân bằng tải, vốn sẽ phân phối lưu lượng đến tất cả các máy chủ triển khai Service B.

However, there is an alternative. We can remove the load balancer from the picture and distribute the requests directly from the client (Service A), which now becomes directly responsible for load balancing its requests across the various instances of Service B. This is possible only if Server A knows

the details about the servers exposing Service B, and in an internal network, this is usually known information. With this approach, we are essentially implementing peer-to-peer load balancing.

Tuy nhiên, có một cách thay thế. Chúng ta có thể loại bỏ bộ cân bằng tải khỏi bức tranh và phân phối các yêu cầu trực tiếp từ client (Service A), vốn bây giờ trở nên trực tiếp chịu trách nhiệm cho việc cân bằng tải các yêu cầu của nó trên các instance khác nhau của Service B. Điều này chỉ có thể nếu Server A biết các chi tiết về các máy chủ phơi bày Service B, và trong một mạng nội bộ, đây thường là thông tin đã biết. Với cách tiếp cận này, về cơ bản chúng ta đang triển khai cân bằng tải peer-to-peer.

Figure 12.8 compares the two alternatives we just described:

Hình 12.8 so sánh hai lựa chọn thay thế mà chúng ta vừa mô tả:

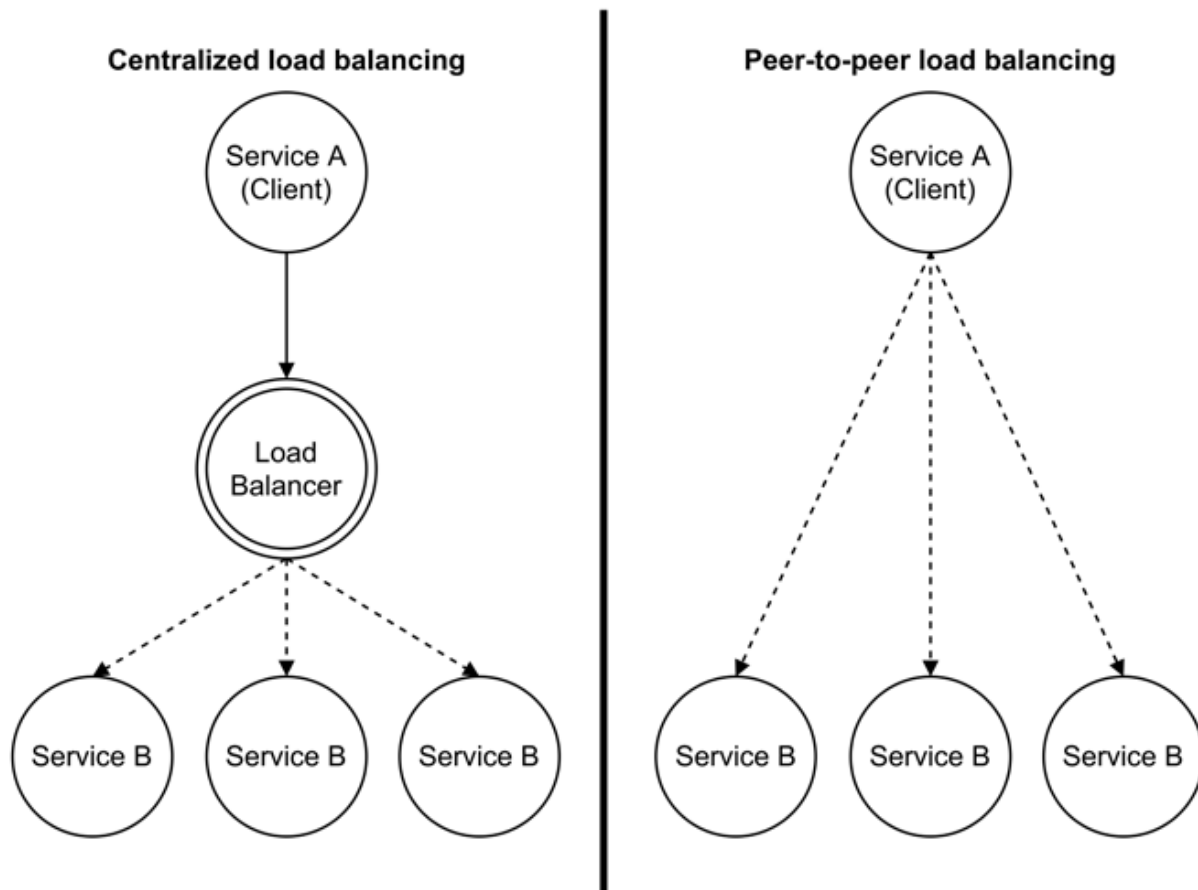


Figure 12.8: Centralized load balancing versus peer-to-peer load balancing This is an extremely simple and effective pattern that enables truly distributed communications without bottlenecks or single points of failure. Besides that, it also has the following properties:

Hình 12.8: Cân bằng tải tập trung so với cân bằng tải peer-to-peer Đây là một mẫu cực kỳ đơn giản và hiệu quả cho phép các giao tiếp thực sự phân tán mà không có nút thắt cổ chai hoặc điểm lỗi đơn lẻ (single point of failure). Bên cạnh đó, nó cũng có các thuộc tính sau:

- Reduces the infrastructure complexity by removing a network node
- Allows faster communications because messages will travel through one fewer node
- Scales better because performances are not limited by what the load balancer can handle

- Giảm độ phức tạp của hạ tầng bằng cách loại bỏ một node mạng
- Cho phép các giao tiếp nhanh hơn vì các thông điệp sẽ đi qua ít hơn một node
- Mở rộng tốt hơn vì hiệu năng không bị giới hạn bởi những gì bộ cân bằng tải có thể xử lý

On the other hand, by removing the load balancer, we are actually exposing the complexity of its underlying infrastructure. Also, each client has to be smarter by implementing a load balancing algorithm and, possibly, also a way to keep its knowledge of the infrastructure up to date.

Mặt khác, bằng cách loại bỏ bộ cân bằng tải, chúng ta thực sự đang phơi bày độ phức tạp của hạ tầng bên dưới của nó. Ngoài ra, mỗi client phải thông minh hơn bằng cách triển khai một thuật toán cân bằng tải và, có thể, cũng là một cách để giữ cho kiến thức của nó về hạ tầng được cập nhật.

Peer-to-peer load balancing is a pattern used extensively in the ZeroMQ (nodejsdp.link/zeromq) library, which we will use in the next chapter.

Cân bằng tải peer-to-peer là một mẫu được sử dụng rộng rãi trong thư viện ZeroMQ (nodejsdp.link/zeromq), mà chúng ta sẽ sử dụng trong chương tiếp theo.

In the next section, we will showcase an example implementing peer-to-peer load balancing in an HTTP client.

Trong phần tiếp theo, chúng ta sẽ trình bày một ví dụ triển khai cân bằng tải peer-to-peer trong một HTTP client.

Implementing an HTTP client that can balance requests across multiple servers — Triển khai một HTTP client có thể cân bằng các yêu cầu trên nhiều máy chủ

We already know how to implement a load balancer using only Node.js and distribute incoming requests across the available servers, so implementing the same mechanism on the client side should not be that different. All we have to do, in fact, is wrap the client API and augment it with a load balancing mechanism. Take a look at the following module (`balancedRequest.js`):

Chúng ta đã biết cách triển khai một bộ cân bằng tải chỉ bằng Node.js và phân phối các yêu cầu đến trên các máy chủ khả dụng, vì vậy việc triển khai cùng cơ chế ở phía client sẽ không khác nhiều như vậy. Tất cả những gì chúng ta phải làm, thực ra, là bọc client API và bổ sung nó với một cơ chế cân bằng tải. Hãy xem module sau (`balancedRequest.js`):

```
import { request } from 'http'
import getStream from 'get-stream'
const servers = [
  { host: 'localhost', port: 8081 },
  { host: 'localhost', port: 8082 }
]
let i = 0
export function balancedRequest (options) {
  return new Promise((resolve) => {
    i = (i + 1) % servers.length
    options.hostname = servers[i].host
    options.port = servers[i].port
    request(options, (response) => {
      resolve(getStream(response))
    }).end()
  })
}
```

```

    })
  }
}

```

The preceding code is very simple and needs little explanation. We wrapped the original `http.request` API so that it overrides the hostname and port of the request with those selected from the list of available servers using a round-robin algorithm. Note that, for simplicity, we used the module `get-stream` (nodejsdp.link/get-stream) to “accumulate” the response stream into a **buffer** that will contain the full response body.

Đoạn code trên rất đơn giản và cần ít giải thích. Chúng ta đã bọc API `http.request` gốc để nó ghi đè hostname và port của yêu cầu bằng những cái được chọn từ danh sách các máy chủ khả dụng bằng một thuật toán round-robin. Lưu ý rằng, để cho đơn giản, chúng ta đã sử dụng module `get-stream` (nodejsdp.link/get-stream) để “tích lũy” stream phản hồi vào một buffer sẽ chứa toàn bộ body của phản hồi.

The new wrapped API can then be used seamlessly (`client.js`):

API được bọc mới sau đó có thể được sử dụng một cách liền mạch (`client.js`):

```

import { balancedRequest } from './balancedRequest.js'
async function main () {
  for (let i = 0; i < 10; i++) {
    const body = await balancedRequest({
      method: 'GET',
      path: '/'
    })
    console.log(`Request ${i} completed:`, body)
  }
}
main().catch((err) => {
  console.error(err)
  process.exit(1)
})

```

To run the preceding code, we have to start two instances of the sample server provided:

Để chạy đoạn code trên, chúng ta phải khởi động hai instance của máy chủ mẫu được cung cấp:

```

node app.js 8081
node app.js 8082

```

This is followed by the client application we just built:

Tiếp theo là ứng dụng client mà chúng ta vừa xây dựng:

```

node client.js

```

We should note that each request is sent to a different server, confirming that we are now able to balance the load without a dedicated load balancer!

Chúng ta nên lưu ý rằng mỗi yêu cầu được gửi đến một máy chủ khác nhau, xác nhận rằng giờ chúng ta có thể cân bằng tải mà không cần một bộ cân bằng tải chuyên dụng!

An obvious improvement to the wrapper we created previously would be to integrate a service registry directly into the client and obtain the server list dynamically.

Một cải tiến hiển nhiên cho wrapper mà chúng ta đã tạo trước đó sẽ là tích hợp một service registry trực tiếp vào client và lấy danh sách máy chủ một cách động.

In the next section, we will explore the field of containers and container orchestration and see how, in this specific context, the **runtime** takes ownership of many scalability concerns.

Trong phần tiếp theo, chúng ta sẽ khám phá lĩnh vực của các container và điều phối container (container orchestration) và xem, trong bối cảnh cụ thể này, runtime đảm nhận nhiều mối quan tâm về khả năng mở rộng như thế nào.

Scaling applications using containers — Mở rộng các ứng dụng bằng các container

In this section, we will demonstrate how using containers and container orchestration platforms, such as Kubernetes, can help us to write simpler Node.js applications that can delegate most of the scaling concerns like load balancing, elastic scaling, and high availability to the underlying container platform.

Trong phần này, chúng ta sẽ minh họa cách sử dụng các container và các nền tảng điều phối container, chẳng hạn như Kubernetes, có thể giúp chúng ta viết các ứng dụng Node.js đơn giản hơn có thể ủy thác hầu hết các mối quan tâm về mở rộng như cân bằng tải, mở rộng đàn hồi (elastic scaling), và tính sẵn sàng cao cho nền tảng container bên dưới.

Containers and container orchestration platforms constitute a quite broad topic, largely outside the **scope** of this book. For this reason, here, we aim to provide only some basic examples to get you started with this technology using Node.js. Ultimately, our goal is to encourage you to explore new modern patterns in order to run and scale Node.js applications.

Các container và các nền tảng điều phối container tạo thành một chủ đề khá rộng, phần lớn nằm ngoài phạm vi của cuốn sách này. Vì lý do này, ở đây, chúng ta nhằm mục đích chỉ cung cấp một số ví dụ cơ bản để giúp bạn bắt đầu với công nghệ này bằng Node.js. Cuối cùng, mục tiêu của chúng ta là khuyến khích bạn khám phá các mẫu hiện đại mới để chạy và mở rộng các ứng dụng Node.js.

What is a container? — Container là gì?

A container, specifically a Linux container, as standardized by the Open Container Initiative (OCI) (nodejsdp.link/opencontainers), is defined as “a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another.”

Một container, cụ thể là một Linux container, như được chuẩn hóa bởi Open Container Initiative (OCI) (nodejsdp.link/opencontainers), được định nghĩa là “một đơn vị phần mềm chuẩn đóng gói code và tất cả các phụ thuộc của nó để ứng dụng chạy nhanh chóng và đáng tin cậy từ một môi trường tính toán này sang môi trường khác.”

In other words, by using containers, you can seamlessly **package** and run applications on different machines, from a local development laptop on your desk to a production server in the cloud.

Nói cách khác, bằng cách sử dụng các container, bạn có thể đóng gói và chạy các ứng dụng một cách liền mạch trên các máy khác nhau, từ một laptop phát triển cục bộ trên bàn của bạn đến một máy chủ production trên đám mây.

Other than being extremely portable, applications running as containers have the advantage of having very little overhead when executed. In fact, containers run almost as fast as running the

native application directly on the operating system. In simple terms, you can see a container as a standard unit of software that allows you to define and run an isolated process directly on a Linux operating system.

Ngoài việc cực kỳ di động, các ứng dụng chạy như các container có lợi thế là có rất ít overhead khi được thực thi. Trên thực tế, các container chạy gần như nhanh bằng việc chạy ứng dụng native trực tiếp trên hệ điều hành. Nói một cách đơn giản, bạn có thể xem một container như một đơn vị phần mềm chuẩn cho phép bạn định nghĩa và chạy một tiến trình bị cô lập trực tiếp trên một hệ điều hành Linux.

For their portability and performance, containers are considered a huge step forward when compared to virtual machines.

Vì tính di động và hiệu năng của chúng, các container được coi là một bước tiến lớn so với các máy ảo (virtual machine).

There are different ways and tools to create and run an OCI compliant container for an application. The most popular of them is Docker (nodejsdp.link/docker).

Có nhiều cách và công cụ khác nhau để tạo và chạy một container tuân thủ OCI cho một ứng dụng. Phổ biến nhất trong số đó là Docker (nodejsdp.link/docker).

You can install Docker in your system by following the instructions for your operating system on the official documentation: nodejsdp.link/docker-docs.

Bạn có thể cài đặt Docker trong hệ thống của mình bằng cách làm theo các hướng dẫn cho hệ điều hành của bạn trong tài liệu chính thức: nodejsdp.link/docker-docs.

Creating and running a container with Docker — Tạo và chạy một container với Docker

Let's rewrite our simple web server application with some minor changes (`app.js`):

Hãy viết lại ứng dụng web server đơn giản của chúng ta với một số thay đổi nhỏ (`app.js`):

```
import { createServer } from 'http'
import { hostname } from 'os'
const version = 1
const server = createServer((req, res) => {
  let i = 1e7; while (i > 0) { i-- }
  res.end(`Hello from ${hostname()} (v${version})`)
})
server.listen(8080)
```

Compared to the previous versions of this web server, here, we send the machine hostname and the application version back to the user. If you run this server and make a request, you should get back something like this:

So với các phiên bản trước của web server này, ở đây, chúng ta gửi tên máy chủ (hostname) của máy và phiên bản ứng dụng trở lại cho người dùng. Nếu bạn chạy máy chủ này và thực hiện một yêu cầu, bạn sẽ nhận lại một cái gì đó như thế này:

Hello from my-amazing-laptop.local (v1)

Let's see how we can run this application as a container. The first thing we need to do is create a `package.json` file for the project:

Hãy xem cách chúng ta có thể chạy ứng dụng này như một container. Điều đầu tiên chúng ta cần làm là tạo một file `package.json` cho dự án:

```
{
  "name": "my-simple-app",
  "version": "1.0.0",
  "main": "app.js",
  "type": "module",
  "scripts": {
    "start": "node app.js"
  }
}
```

In order to dockerize our application, we need to follow a two-step process:

Để dockerize ứng dụng của chúng ta, chúng ta cần làm theo một quy trình hai bước:

- Build a container image
- Run a container instance from the image
 - Build một container image
 - Chạy một instance container từ image

To create the container image for our application, we have to define a Dockerfile. A container image (or Docker image) is the actual package and conforms to the OCI standard. It contains all the source code and the necessary dependencies and describes how the application must be executed. A Dockerfile is a file (actually named Dockerfile) that defines the build script used to build a container image for an application. So, without further ado, let's write the Dockerfile for our application:

Để tạo container image cho ứng dụng của chúng ta, chúng ta phải định nghĩa một Dockerfile. Một container image (hoặc Docker image) là gói thực tế và tuân thủ chuẩn OCI. Nó chứa tất cả source code và các phụ thuộc cần thiết và mô tả cách ứng dụng phải được thực thi. Một Dockerfile là một file (thực tế được đặt tên là Dockerfile) định nghĩa script build được sử dụng để build một container image cho một ứng dụng. Vậy, không nói thêm nữa, hãy viết Dockerfile cho ứng dụng của chúng ta:

```
FROM node:14-alpine
EXPOSE 8080
COPY app.js package.json /app/
WORKDIR /app
CMD ["npm", "start"]
```

Our Dockerfile is quite short, but there are a lot of interesting things here, so let's discuss them one by one:

Dockerfile của chúng ta khá ngắn, nhưng có rất nhiều điều thú vị ở đây, vậy hãy thảo luận về chúng từng cái một:

- `FROM node:14-alpine` indicates the base image that we want to use. A base image allows us to build “on top” of an existing image. In this specific case, we are starting from an image that already contains version 14 of Node.js. This means we don't have to be worried about describing how Node.js needs to be packaged into the container image.
- `EXPOSE 8080` informs Docker that the application will be listening for TCP connections on the port 8080.
- `COPY app.js package.json /app/` copies the files `app.js` and `package.json` into the `/app` folder of the container filesystem. Containers are isolated, so, by default, they can't share files with the host operating system; therefore, we need to copy the project files into the container

to be able to access and execute them.

- `WORKDIR /app` sets the working directory for the container to `/app`.
- `CMD ["npm", "start"]` specifies the command that is executed to start the application when we run a container from an image. Here, we are just running `npm start`, which, in turn, will run `node app.js`, as specified in our `package.json`. Remember that we are able to run both `node` and `npm` in the container only because those two executables are made available through the base image.
 - `FROM node:14-alpine` chỉ ra base image mà chúng ta muốn sử dụng. Một base image cho phép chúng ta build “lên trên” một image hiện có. Trong trường hợp cụ thể này, chúng ta đang bắt đầu từ một image đã chứa phiên bản 14 của Node.js. Điều này có nghĩa là chúng ta không phải lo lắng về việc mô tả cách Node.js cần được đóng gói vào container image.
 - `EXPOSE 8080` thông báo cho Docker rằng ứng dụng sẽ lắng nghe các kết nối TCP trên cổng 8080.
 - `COPY app.js package.json /app/` sao chép các file `app.js` và `package.json` vào thư mục `/app` của filesystem container. Các container bị cô lập, vì vậy, theo mặc định, chúng không thể chia sẻ các file với hệ điều hành host; do đó, chúng ta cần sao chép các file dự án vào container để có thể truy cập và thực thi chúng.
 - `WORKDIR /app` đặt thư mục làm việc cho container là `/app`.
 - `CMD ["npm", "start"]` chỉ định lệnh được thực thi để khởi động ứng dụng khi chúng ta chạy một container từ một image. Ở đây, chúng ta chỉ chạy `npm start`, mà, lần lượt, sẽ chạy `node app.js`, như được chỉ định trong `package.json` của chúng ta. Hãy nhớ rằng chúng ta có thể chạy cả `node` và `npm` trong container chỉ vì hai file thực thi đó được làm cho khả dụng thông qua base image.

Now, we can use the Dockerfile to build the container image with the following command:

Bây giờ, chúng ta có thể sử dụng Dockerfile để build container image với lệnh sau:

```
docker build .
```

This command will look for a Dockerfile in the current working directory and execute it to build our image.

Lệnh này sẽ tìm một Dockerfile trong thư mục làm việc hiện tại và thực thi nó để build image của chúng ta.

The output of this command should be something like this:

Đầu ra của lệnh này sẽ là một cái gì đó như thế này:

```
Sending build context to Docker daemon 7.168kB
Step 1/5 : FROM node:14-alpine
--> ea308280893e
Step 2/5 : EXPOSE 8080
--> Running in 61c34f4064ab
Removing intermediate container 61c34f4064ab
--> 6abfcd0e750
Step 3/5 : COPY app.js package.json /app/
--> 9d498d7dbf8b
Step 4/5 : WORKDIR /app
--> Running in 70ea26158cbe
Removing intermediate container 70ea26158cbe
--> fc075a421b91
```

```
Step 5/5 : CMD ["npm", "start"]
---> Running in 3642a01224e8
Removing intermediate container 3642a01224e8
---> bb3bd34bac55
Successfully built bb3bd34bac55
```

Note that if you have never used the `node:14-alpine` image before (or if you have recently wiped your Docker cache), you will also see some additional output, indicating the download of this container image.

Lưu ý rằng nếu bạn chưa bao giờ sử dụng image `node:14-alpine` trước đây (hoặc nếu bạn vừa xóa Docker cache của mình), bạn cũng sẽ thấy một số đầu ra bổ sung, cho biết việc tải xuống container image này.

The final hash is the ID of our container image. We can use it to run an instance of the container with the following command:

Hash cuối cùng là ID của container image của chúng ta. Chúng ta có thể sử dụng nó để chạy một instance của container với lệnh sau:

```
docker run -it -p 8080:8080 bb3bd34bac55
```

This command is essentially telling Docker to run the application from image `bb3bd34bac55` in “interactive mode” (which means that it will not go in the background) and that port `8080` of the container will be mapped to port `8080` of the host machine (our operating system).

Lệnh này về cơ bản nói với Docker chạy ứng dụng từ image `bb3bd34bac55` trong “chế độ tương tác” (interactive mode) (nghĩa là nó sẽ không chạy nền) và rằng cổng `8080` của container sẽ được ánh xạ đến cổng `8080` của máy host (hệ điều hành của chúng ta).

Now, we can access the application at `localhost:8080`. So, if we use `curl` to send a request to the web server, we should get a response similar to the following:

Bây giờ, chúng ta có thể truy cập ứng dụng tại `localhost:8080`. Vậy, nếu chúng ta sử dụng `curl` để gửi một yêu cầu đến web server, chúng ta sẽ nhận được một phản hồi tương tự như sau:

```
Hello from f2ffa85c8ff8 (v1)
```

Note that the hostname is now different. This is because every container is running in a sandboxed environment that, by default, doesn’t have access to most of the resources in the underlying operating system.

Lưu ý rằng hostname giờ đã khác. Điều này là vì mỗi container đang chạy trong một môi trường sandbox mà, theo mặc định, không có quyền truy cập vào hầu hết các tài nguyên trong hệ điều hành bên dưới.

At this point, you can stop the container by just pressing `Ctrl + C` in the terminal window where the container is running.

Tại thời điểm này, bạn có thể dừng container bằng cách chỉ nhấn `Ctrl + C` trong cửa sổ terminal nơi container đang chạy.

When building an image, we can use the `-t` flag to tag the resulting image. A tag can be used as a more predictable alternative to a generated hash to identify and run container images. For instance, if we want to call our container image `hello-web:v1`, we can use the following commands:

Khi build một image, chúng ta có thể sử dụng cờ `-t` để gắn tag (tag) cho image kết quả. Một tag có thể được sử dụng như một lựa chọn thay thế dễ dự đoán hơn cho một hash được tạo

để xác định và chạy các container image. Ví dụ, nếu chúng ta muốn gọi container image của mình là `hello-web:v1`, chúng ta có thể sử dụng các lệnh sau:

```
docker build -t hello-web:v1 .  
docker run -it -p 8080:8080 hello-web:v1
```

When using tags, you might want to follow the conventional format of `image-name:version`.

Khi sử dụng các tag, bạn có thể muốn tuân theo định dạng thông thường của `image-name:version`.

What is Kubernetes? — Kubernetes là gì?

We just ran a Node.js application using containers, hooray! Even though this seems like a particularly exciting achievement, we have just scratched the surface here. The real power of containers comes out when building more complicated applications. For instance, when building applications composed by multiple independent services that needs to be deployed and coordinated across multiple cloud servers. In this situation, Docker alone is not sufficient anymore. We need a more complex system that allows us to orchestrate all the running container instances over the available machines in our cloud cluster: we need a container orchestration tool. A container orchestration tool has a number of responsibilities:

Chúng ta vừa chạy một ứng dụng Node.js bằng các container, hoan hô! Mặc dù điều này dường như là một thành tựu đặc biệt thú vị, chúng ta mới chỉ chạm vào bề mặt ở đây. Sức mạnh thực sự của các container bộc lộ khi xây dựng các ứng dụng phức tạp hơn. Ví dụ, khi xây dựng các ứng dụng được tạo thành từ nhiều dịch vụ độc lập cần được triển khai và phối hợp trên nhiều máy chủ đám mây. Trong tình huống này, chỉ riêng Docker không còn đủ nữa. Chúng ta cần một hệ thống phức tạp hơn cho phép chúng ta điều phối tất cả các instance container đang chạy trên các máy khả dụng trong cluster đám mây của chúng ta: chúng ta cần một công cụ điều phối container. Một công cụ điều phối container có một số trách nhiệm:

- It allows us to join multiple cloud servers (nodes) into one logical cluster, where nodes can be added and removed dynamically without affecting the availability of the services running in every node.
- It makes sure that there is no downtime. If a container instance stops or becomes unresponsive to health checks, it will be automatically restarted. Also, if a node in the cluster fails, the workload running in that node will be automatically migrated to another node.
- Provides functionalities to implement service discovery and load balancing.
- Provides orchestrated access to durable storage so that data can be persisted as needed.
- Automatic rollouts and rollbacks of applications with zero downtime.
- Secret storage for sensitive data and configuration management systems.
 - Nó cho phép chúng ta nối nhiều máy chủ đám mây (node) thành một cluster logic, nơi các node có thể được thêm và xóa một cách động mà không ảnh hưởng đến tính sẵn sàng của các dịch vụ chạy trong mỗi node.
 - Nó đảm bảo rằng không có thời gian chết. Nếu một instance container dừng hoặc trở nên không phản hồi với các health check, nó sẽ được khởi động lại tự động. Ngoài ra, nếu một node trong cluster bị lỗi, khối lượng công việc chạy trong node đó sẽ được di chuyển tự động sang một node khác.
 - Cung cấp các chức năng để triển khai khám phá dịch vụ (service discovery) và cân bằng tải.

- Cung cấp quyền truy cập được điều phối vào bộ lưu trữ bền vững (durable storage) để dữ liệu có thể được duy trì khi cần.
- Triển khai (rollout) và rollback tự động các ứng dụng với không có thời gian chết.
- Lưu trữ bí mật (secret storage) cho dữ liệu nhạy cảm và các hệ thống quản lý cấu hình.

One of the most popular container orchestration systems is Kubernetes ([nodejsdp. link/kubernetes](https://kubernetes.io)), originally open sourced by Google in 2014. The name Kubernetes originates from the Greek “κυβερνήτης”, meaning “helmsman” or “pilot”, but also “governor” or more generically, “the one in command”. Kubernetes incorporates years of experience from Google engineers running workloads in the cloud at scale.

Một trong những hệ thống điều phối container phổ biến nhất là Kubernetes ([nodejsdp. link/kubernetes](https://kubernetes.io)), ban đầu được Google mã nguồn mở vào năm 2014. Cái tên Kubernetes bắt nguồn từ tiếng Hy Lạp “κυβερνήτης”, có nghĩa là “người lái tàu” hoặc “phi công”, nhưng cũng có nghĩa là “thống đốc” hoặc tổng quát hơn, “người chỉ huy”. Kubernetes kết hợp nhiều năm kinh nghiệm từ các kỹ sư Google chạy các khối lượng công việc trên đám mây ở quy mô lớn.

One of its peculiarities is the declarative configuration system that allows you to define an “end state” and let the orchestrator figure out the sequence of steps necessary to reach the desired state, without disrupting the stability of the services running on the cluster.

Một trong những điểm đặc biệt của nó là hệ thống cấu hình khai báo (declarative) cho phép bạn định nghĩa một “trạng thái cuối” (end state) và để trình điều phối tìm ra trình tự các bước cần thiết để đạt được trạng thái mong muốn, mà không làm gián đoạn tính ổn định của các dịch vụ chạy trên cluster.

The whole idea of Kubernetes configuration revolves around the concept of “objects”. An object is an element in your cloud deployment, which can be added, removed, and have its configuration changed over time. Some good examples of Kubernetes objects are:

Toàn bộ ý tưởng về cấu hình Kubernetes xoay quanh khái niệm “đối tượng” (object). Một đối tượng là một phần tử trong việc triển khai đám mây của bạn, có thể được thêm, xóa, và có cấu hình của nó được thay đổi theo thời gian. Một số ví dụ tốt về các đối tượng Kubernetes là:

- Containerized applications
- Resources for the containers (CPU and memory allocations, persistent storage, access to devices such as network interfaces or GPU, and so on)
- Policies for the application behavior (restart policies, upgrades, faulttolerance)
 - Các ứng dụng được container hóa
 - Các tài nguyên cho các container (cấp phát CPU và bộ nhớ, lưu trữ bền vững, quyền truy cập vào các thiết bị như giao diện mạng hoặc GPU, v.v.)
 - Các chính sách cho hành vi ứng dụng (chính sách khởi động lại, nâng cấp, chịu lỗi)

A Kubernetes object is a sort of “record of intent”, which means that once you create one in a cluster, Kubernetes will constantly monitor (and change, if needed) the state of the object to make sure it stays compliant with the defined expectation. A Kubernetes cluster is generally managed through a command-line tool called `kubectl` ([nodejsdp. link/kubectl-install](https://kubernetes.io)).

Một đối tượng Kubernetes là một loại “bản ghi ý định” (record of intent), nghĩa là một khi bạn tạo một đối tượng trong một cluster, Kubernetes sẽ liên tục giám sát (và thay đổi, nếu cần) trạng thái của đối tượng để đảm bảo nó vẫn tuân thủ kỳ vọng được định nghĩa. Một

cluster Kubernetes thường được quản lý thông qua một công cụ dòng lệnh gọi là `kubectl` ([nodejsdp.link/kubectl-install](https://kubernetes.io/docs/tasks/tools/install-kubectl/)).

There are several ways to create a Kubernetes cluster for development, testing, and production purposes. The easiest way to start experimenting with Kubernetes is through a local single-node cluster, which can be easily created by a tool called minikube ([nodejsdp.link/minikube-install](https://kubernetes.io/docs/tasks/tools/install-minikube/)).

Có một số cách để tạo một cluster Kubernetes cho các mục đích phát triển, kiểm thử và production. Cách dễ nhất để bắt đầu thử nghiệm với Kubernetes là thông qua một cluster đơn-node cục bộ, vốn có thể dễ dàng được tạo bởi một công cụ gọi là minikube ([nodejsdp.link/minikube-install](https://kubernetes.io/docs/tasks/tools/install-minikube/)).

Make sure to install both `kubectl` and `minikube` on your system, as we will be deploying our sample containerized app on a local Kubernetes cluster in the next section!

Hãy đảm bảo cài đặt cả `kubectl` và `minikube` trên hệ thống của bạn, vì chúng ta sẽ triển khai ứng dụng được container hóa mẫu của mình trên một cluster Kubernetes cục bộ trong phần tiếp theo!

Another great way to learn about Kubernetes is by using the official interactive tutorials ([nodejsdp.link/kubernetes-tutorials](https://kubernetes.io/docs/tutorials/interactive/)).

Một cách tuyệt vời khác để học về Kubernetes là sử dụng các hướng dẫn tương tác chính thức ([nodejsdp.link/kubernetes-tutorials](https://kubernetes.io/docs/tutorials/interactive/)).

Deploying and scaling an application on Kubernetes — Triển khai và mở rộng một ứng dụng trên Kubernetes

In this section, we will be running our simple web server application on a local minikube cluster. So, make sure you have `kubectl` and `minikube` correctly installed and started.

Trong phần này, chúng ta sẽ chạy ứng dụng web server đơn giản của mình trên một cluster minikube cục bộ. Vậy, hãy đảm bảo bạn đã cài đặt và khởi động `kubectl` và `minikube` đúng cách.

On macOS and Linux environments, make sure to run `minikube start` and `eval $(minikube docker-env)` to initialize the working environment. The second command makes sure that when you use `docker` and `kubectl` in your current terminal you will interact with the local Minikube cluster. If you open multiple terminals you should run `eval $(minikube docker-env)` on every terminal. You can also run `minikube dashboard` to run a convenient web dashboard that allows you to visualize and interact with all the objects in your cluster.

Trên các môi trường macOS và Linux, hãy đảm bảo chạy `minikube start` và `eval $(minikube docker-env)` để khởi tạo môi trường làm việc. Lệnh thứ hai đảm bảo rằng khi bạn sử dụng `docker` và `kubectl` trong terminal hiện tại của mình, bạn sẽ tương tác với cluster Minikube cục bộ. Nếu bạn mở nhiều terminal, bạn nên chạy `eval $(minikube docker-env)` trên mỗi terminal. Bạn cũng có thể chạy `minikube dashboard` để chạy một web dashboard tiện lợi cho phép bạn trực quan hóa và tương tác với tất cả các đối tượng trong cluster của bạn.

The first thing that we want to do is build our Docker image and give it a meaningful name:

Điều đầu tiên mà chúng ta muốn làm là build Docker image của chúng ta và đặt cho nó một cái tên có ý nghĩa:

```
docker build -t hello-web:v1 .
```

If you have configured your environment correctly, the `hello-web` image will be available to be used in your local Kubernetes cluster. Using local images is sufficient for local development. When you are ready to go to production, the best option is to **publish** your images to a Docker container registry such as Docker Hub (nodejsdp.link/docker-hub), Docker Registry (nodejsdp.link/docker-registry), Google Cloud Container Registry (nodejsdp.link/gc-container-registry), or Amazon Elastic Container Registry (nodejsdp.link/ecr). Once you have your images published to a container registry, you can easily deploy your application to different hosts without having to rebuild the corresponding images each time.

Nếu bạn đã cấu hình môi trường của mình một cách chính xác, image `hello-web` sẽ khả dụng để được sử dụng trong cluster Kubernetes cục bộ của bạn. Sử dụng các image cục bộ là đủ cho phát triển cục bộ. Khi bạn đã sẵn sàng để chuyển sang production, lựa chọn tốt nhất là xuất bản các image của bạn lên một Docker container registry như Docker Hub (nodejsdp.link/docker-hub), Docker Registry (nodejsdp.link/docker-registry), Google Cloud Container Registry (nodejsdp.link/gc-container-registry), hoặc Amazon Elastic Container Registry (nodejsdp.link/ecr). Một khi bạn đã xuất bản các image của mình lên một container registry, bạn có thể dễ dàng triển khai ứng dụng của mình lên các host khác nhau mà không phải build lại các image tương ứng mỗi lần.

Creating a Kubernetes deployment — Tạo một Kubernetes deployment Now, in order to run an instance of this container in the Minikube cluster, we have to create a deployment (which is a Kubernetes object) using the following command:

Bây giờ, để chạy một instance của container này trong cluster Minikube, chúng ta phải tạo một deployment (vốn là một đối tượng Kubernetes) bằng lệnh sau:

```
kubectl create deployment hello-web --image=hello-web:v1
```

This should produce the following output:

Điều này sẽ tạo ra đầu ra sau:

```
deployment.apps/hello-web created
```

This command is basically telling Kubernetes to run an instance of the `hello-web:v1` container as an application called `hello-web`.

Lệnh này về cơ bản nói với Kubernetes chạy một instance của container `hello-web:v1` như một ứng dụng gọi là `hello-web`.

You can verify that the deployment is running with the following command:

Bạn có thể xác minh rằng deployment đang chạy với lệnh sau:

```
kubectl get deployments
```

This should print something like this:

Điều này sẽ in ra một cái gì đó như thế này:

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
hello-web	1/1	1	1	7s

This table is basically saying that our `hello-web` deployment is alive and that there is one pod allocated for it. A pod is a basic unit in Kubernetes and represents a set of containers that have to run together in the same Kubernetes node. Containers in the same pod have shared resources like storage and network. Generally, a pod contains only one container, but it's not uncommon to see more than one container in a pod when these containers are running tightly coupled applications.

Bảng này về cơ bản nói rằng deployment hello-web của chúng ta đang hoạt động và rằng có một pod được cấp phát cho nó. Một pod là một đơn vị cơ bản trong Kubernetes và đại diện cho một tập hợp các container phải chạy cùng nhau trong cùng một node Kubernetes. Các container trong cùng một pod có các tài nguyên chia sẻ như lưu trữ và mạng. Thông thường, một pod chỉ chứa một container, nhưng không hiếm khi thấy nhiều hơn một container trong một pod khi các container này đang chạy các ứng dụng được ghép chặt (tightly coupled).

You can list all the pods running in the cluster with:

Bạn có thể liệt kê tất cả các pod đang chạy trong cluster với:

```
kubectl get pods
```

This should print something like:

Điều này sẽ in ra một cái gì đó như:

NAME	READY	STATUS	RESTARTS	AGE
hello-web-65f47d9997-df7nr	1/1	Running	0	2m19s

Now, in order to be able to access the web server from our local machine, we need to expose the deployment:

Bây giờ, để có thể truy cập web server từ máy cục bộ của chúng ta, chúng ta cần phơi bày (expose) deployment:

```
kubectl expose deployment hello-web --type=LoadBalancer --port=8080
minikube service hello-web
```

The first command tells Kubernetes to create a LoadBalancer object that exposes the instances of the hello-web app, connecting to port 8080 of every container.

Lệnh đầu tiên nói với Kubernetes tạo một đối tượng LoadBalancer phơi bày các instance của ứng dụng hello-web, kết nối đến cổng 8080 của mọi container.

The second command is a minikube helper command that allows us to get the local address to access the load balancer. This command will also open a browser window for you, so now you should see the container response in the browser, which should look like this:

Lệnh thứ hai là một lệnh trợ giúp của minikube cho phép chúng ta lấy địa chỉ cục bộ để truy cập bộ cân bằng tải. Lệnh này cũng sẽ mở một cửa sổ trình duyệt cho bạn, vậy giờ bạn sẽ thấy phản hồi của container trong trình duyệt, vốn sẽ trông như thế này:

```
Hello from hello-web-65f47d9997-df7nr (v1)
```

Scaling a Kubernetes deployment — Mở rộng một Kubernetes deployment Now that our application is running and is accessible, let's actually start to experiment with some of the capabilities of Kubernetes. For instance, why not try to scale our application by running five instances instead of just one? This is as easy as running:

Bây giờ ứng dụng của chúng ta đang chạy và có thể truy cập được, hãy thực sự bắt đầu thử nghiệm với một số khả năng của Kubernetes. Ví dụ, tại sao không thử mở rộng ứng dụng của chúng ta bằng cách chạy năm instance thay vì chỉ một? Điều này dễ dàng như chạy:

```
kubectl scale --replicas=5 deployment hello-web
```

Now, kubectl get deployments should show us the following status:

Bây giờ, `kubectl get deployments` sẽ hiển thị cho chúng ta trạng thái sau:

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
hello-web	5/5	5	5	9m18s

And `kubectl get pods` should produce something like this:

Và `kubectl get pods` sẽ tạo ra một cái gì đó như thế này:

NAME	READY	STATUS	RESTARTS	AGE
hello-web-65f47d9997-df7nr	1/1	Running	0	9m24s
hello-web-65f47d9997-g98jb	1/1	Running	0	14s
hello-web-65f47d9997-hbdkx	1/1	Running	0	14s
hello-web-65f47d9997-jnfd7	1/1	Running	0	14s
hello-web-65f47d9997-s54g6	1/1	Running	0	14s

If you try to hit the load balancer now, chances are you will see different hostnames as the traffic gets distributed across the available instances. This should be even more apparent if you try to hit the load balancer while putting the application under stress, for instance, by running an autocannon load test against the load balancer URL.

Nếu bạn thử truy cập bộ cân bằng tải bây giờ, có khả năng bạn sẽ thấy các hostname khác nhau khi lưu lượng được phân phối trên các instance khả dụng. Điều này sẽ thậm chí còn rõ ràng hơn nếu bạn thử truy cập bộ cân bằng tải trong khi đặt ứng dụng dưới áp lực, ví dụ, bằng cách chạy một bài kiểm tra tải autocannon đối với URL của bộ cân bằng tải.

Kubernetes rollouts — Kubernetes rollouts Now, let's try out another feature of Kubernetes: rollouts. What if we want to release a new version of our app?

Bây giờ, hãy thử một tính năng khác của Kubernetes: rollouts. Sẽ thế nào nếu chúng ta muốn phát hành một phiên bản mới của ứng dụng của mình?

We can set `const version = 2` in our `app.js` file and create a new image:

Chúng ta có thể đặt `const version = 2` trong file `app.js` của mình và tạo một image mới:

```
docker build -t hello-web:v2 .
```

At this point, in order to upgrade all the running pods to this new version, we have to run the following command:

Tại thời điểm này, để nâng cấp tất cả các pod đang chạy lên phiên bản mới này, chúng ta phải chạy lệnh sau:

```
kubectl set image deployment/hello-web hello-web=hello-web:v2 --record
```

The output of this command should be as follows:

Đầu ra của lệnh này sẽ như sau:

```
deployment.apps/hello-web image updated
```

If everything worked as expected, you should now be able to refresh your browser page and see something like the following:

Nếu mọi thứ hoạt động như mong đợi, bây giờ bạn sẽ có thể làm mới trang trình duyệt của mình và thấy một cái gì đó như sau:

Hello from hello-web-567b986bfb-qjvfw (v2)

Note the v2 flag there.

Lưu ý cờ v2 ở đó.

What just happened behind the scenes is that Kubernetes started to roll out the new version of our image by replacing the containers one by one. When a container is replaced, the running instance is stopped gracefully. This way requests that are currently in progress can be completed before the container is shut down.

Những gì vừa xảy ra ở hậu trường là Kubernetes bắt đầu triển khai phiên bản mới của image của chúng ta bằng cách thay thế các container từng cái một. Khi một container được thay thế, instance đang chạy được dừng một cách nhẹ nhàng. Bằng cách này, các yêu cầu hiện đang trong tiến trình có thể được hoàn thành trước khi container bị tắt.

This completes our mini Kubernetes tutorial. The lesson here is that, when using a container orchestrator platform like Kubernetes, we can keep our application code quite simple, as we won't have to include concerns such as scaling to multiple instances or deal with soft rollouts and application restarts. This is the major advantage of this approach. Of course, this simplicity does not come for free. It is paid by having to learn and manage the orchestration platform. If you are running small applications in production, it is probably not worth to incur the complexity and the cost of having to install and manage a container orchestrator platform like Kubernetes. However, if you are serving millions of users every day, there is definitely a lot of value in building and maintaining such a powerful infrastructure.

Điều này hoàn thành hướng dẫn Kubernetes mini của chúng ta. Bài học ở đây là, khi sử dụng một nền tảng điều phối container như Kubernetes, chúng ta có thể giữ cho code ứng dụng của mình khá đơn giản, vì chúng ta sẽ không phải bao gồm các mối quan tâm như mở rộng đến nhiều instance hoặc xử lý các rollout mềm và khởi động lại ứng dụng. Đây là lợi thế chính của cách tiếp cận này. Tất nhiên, sự đơn giản này không miễn phí. Nó được trả giá bằng việc phải học và quản lý nền tảng điều phối. Nếu bạn đang chạy các ứng dụng nhỏ trong production, có lẽ không đáng để chịu độ phức tạp và chi phí của việc phải cài đặt và quản lý một nền tảng điều phối container như Kubernetes. Tuy nhiên, nếu bạn đang phục vụ hàng triệu người dùng mỗi ngày, chắc chắn có rất nhiều giá trị trong việc xây dựng và duy trì một hạ tầng mạnh mẽ như vậy.

Another interesting observation is that, when running containers in Kubernetes, containers are often considered “disposable,” which basically means that they could be killed and restarted at any time. While this might seem like a non-relevant detail, you should actually take this behavior into account and try to keep your applications as stateless as possible. In fact, containers, by default, won't retain any change in the local filesystem, so every time you have to store some persistent information, you will have to rely on external storage mechanisms such as databases or persistent volumes.

Một quan sát thú vị khác là, khi chạy các container trong Kubernetes, các container thường được coi là “có thể vứt bỏ” (disposable), về cơ bản nghĩa là chúng có thể bị kill và khởi động lại bất kỳ lúc nào. Mặc dù điều này có vẻ như là một chi tiết không liên quan, bạn thực sự nên tính đến hành vi này và cố gắng giữ cho các ứng dụng của bạn không trạng thái (stateless) càng nhiều càng tốt. Trên thực tế, các container, theo mặc định, sẽ không giữ lại bất kỳ thay đổi nào trong filesystem cục bộ, vì vậy mỗi khi bạn phải lưu trữ một số thông tin bền vững, bạn sẽ phải dựa vào các cơ chế lưu trữ bên ngoài như cơ sở dữ liệu hoặc các persistent volume.

If you want to clean up your system from the containers you just ran in the preceding examples and stop minikube, you can do so with the following commands:

Nếu bạn muốn dọn dẹp hệ thống của mình khỏi các container mà bạn vừa chạy trong các

ví dụ trên và dùng minikube, bạn có thể làm như vậy với các lệnh sau:

```
kubectl scale --replicas=0 deployment hello-web
kubectl delete -n default service hello-web
minikube stop
```

In the next and last part of this chapter, we will explore some interesting patterns to decompose a monolithic application into a set of decoupled microservices, something that is critically important if you have built a monolithic application and are now suffering from scalability issues.

Trong phần tiếp theo và cuối cùng của chương này, chúng ta sẽ khám phá một số mẫu thú vị để phân tách một ứng dụng monolithic thành một tập hợp các microservice được tách rời, điều gì đó cực kỳ quan trọng nếu bạn đã xây dựng một ứng dụng monolithic và giờ đang chịu các vấn đề về khả năng mở rộng.

Decomposing complex applications — Phân tách các ứng dụng phức tạp

So far in this chapter, we have mainly focused our analysis on the X-axis of the scale cube. We saw how it represents the easiest and most immediate way to distribute the load and scale an application, also improving its availability. In the following section, we are going to focus on the Y-axis of the scale cube, where applications are scaled by decomposing them by functionality and service. As we will learn, this technique allows us to scale not only the capacity of an application, but also, and most importantly, its complexity.

Cho đến nay trong chương này, chúng ta đã chủ yếu tập trung phân tích vào trục X của khối lập phương mở rộng. Chúng ta đã thấy nó đại diện như thế nào cho cách dễ nhất và tức thì nhất để phân phối tải và mở rộng một ứng dụng, đồng thời cải thiện tính sẵn sàng của nó. Trong phần sau, chúng ta sẽ tập trung vào trục Y của khối lập phương mở rộng, nơi các ứng dụng được mở rộng bằng cách phân tách chúng theo chức năng và dịch vụ. Như chúng ta sẽ học, kỹ thuật này cho phép chúng ta mở rộng không chỉ năng lực của một ứng dụng, mà còn, và quan trọng nhất, độ phức tạp của nó.

Monolithic architecture — Kiến trúc monolithic

The term monolithic might make us think of a system without modularity, where all the services of an application are interconnected and almost indistinguishable. However, this is not always the case. Often, monolithic systems have a highly modular architecture and a good level of decoupling between their internal components.

Thuật ngữ monolithic có thể khiến chúng ta nghĩ đến một hệ thống không có tính module, nơi tất cả các dịch vụ của một ứng dụng được kết nối với nhau và gần như không thể phân biệt. Tuy nhiên, điều này không phải lúc nào cũng đúng. Thường thì, các hệ thống monolithic có một kiến trúc module hóa cao và một mức độ tách rời tốt giữa các thành phần nội bộ của chúng.

A perfect example is the Linux OS kernel, which is part of a category called monolithic kernels (in perfect opposition with its ecosystem and the Unix philosophy). Linux has thousands of services and modules that we can load and unload dynamically, even while the system is running. However, they all run in kernel mode, which means that a failure in any of them could bring the entire OS down (have you ever seen a kernel panic?). This approach is opposite to the microkernel architecture, where only the core services of the operating system run in kernel mode, while the rest run in

user mode, usually each one with its own process. The main advantage of this approach is that a problem in any of these services would more likely cause it to crash in isolation, instead of affecting the stability of the entire system.

Một ví dụ hoàn hảo là kernel của hệ điều hành Linux, vốn là một phần của một loại gọi là monolithic kernel (đối lập hoàn hảo với hệ sinh thái của nó và triết lý Unix). Linux có hàng nghìn dịch vụ và module mà chúng ta có thể nạp và gỡ nạp một cách động, thậm chí trong khi hệ thống đang chạy. Tuy nhiên, tất cả chúng đều chạy ở chế độ kernel (kernel mode), nghĩa là một lỗi trong bất kỳ cái nào trong số chúng có thể đánh sập toàn bộ OS (bạn đã bao giờ thấy một kernel panic chưa?). Cách tiếp cận này đối lập với kiến trúc microkernel, nơi chỉ có các dịch vụ lõi của hệ điều hành chạy ở chế độ kernel, trong khi phần còn lại chạy ở chế độ người dùng (user mode), thường mỗi cái với tiến trình riêng của nó. Lợi thế chính của cách tiếp cận này là một vấn đề trong bất kỳ dịch vụ nào trong số này sẽ có nhiều khả năng khiến nó sập một cách cô lập hơn, thay vì ảnh hưởng đến tính ổn định của toàn bộ hệ thống.

The Torvalds-Tanenbaum debate on kernel design is probably one of the most famous flame wars in the history of computer science, where one of the main points of dispute was exactly monolithic versus microkernel design. You can find a web version of the discussion (it originally appeared on Usenet) at nodejsdp.link/torvalds-tanenbaum.

Cuộc tranh luận Torvalds-Tanenbaum về thiết kế kernel có lẽ là một trong những cuộc khẩu chiến (flame war) nổi tiếng nhất trong lịch sử khoa học máy tính, nơi một trong những điểm tranh chấp chính chính xác là thiết kế monolithic so với microkernel. Bạn có thể tìm thấy một phiên bản web của cuộc thảo luận (nó ban đầu xuất hiện trên Usenet) tại nodejsdp.link/torvalds-tanenbaum.

It's remarkable how these design principles, which are more than 30 years old, can still be applied today and in totally different environments. Modern monolithic applications are comparable to monolithic kernels: if any of their components fail, the entire system is affected, which, translated into Node.js terms, means that all the services are part of the same code base and run in a single process (when not cloned). Figure 12.9 shows an example monolithic architecture:

Thật đáng chú ý là những nguyên tắc thiết kế này, vốn đã hơn 30 năm tuổi, vẫn có thể được áp dụng ngày nay và trong những môi trường hoàn toàn khác nhau. Các ứng dụng monolithic hiện đại có thể so sánh với các monolithic kernel: nếu bất kỳ thành phần nào của chúng bị lỗi, toàn bộ hệ thống bị ảnh hưởng, mà, dịch sang thuật ngữ Node.js, có nghĩa là tất cả các dịch vụ đều là một phần của cùng một codebase và chạy trong một tiến trình duy nhất (khi không được nhân bản). Hình 12.9 cho thấy một ví dụ kiến trúc monolithic:

E-commerce application

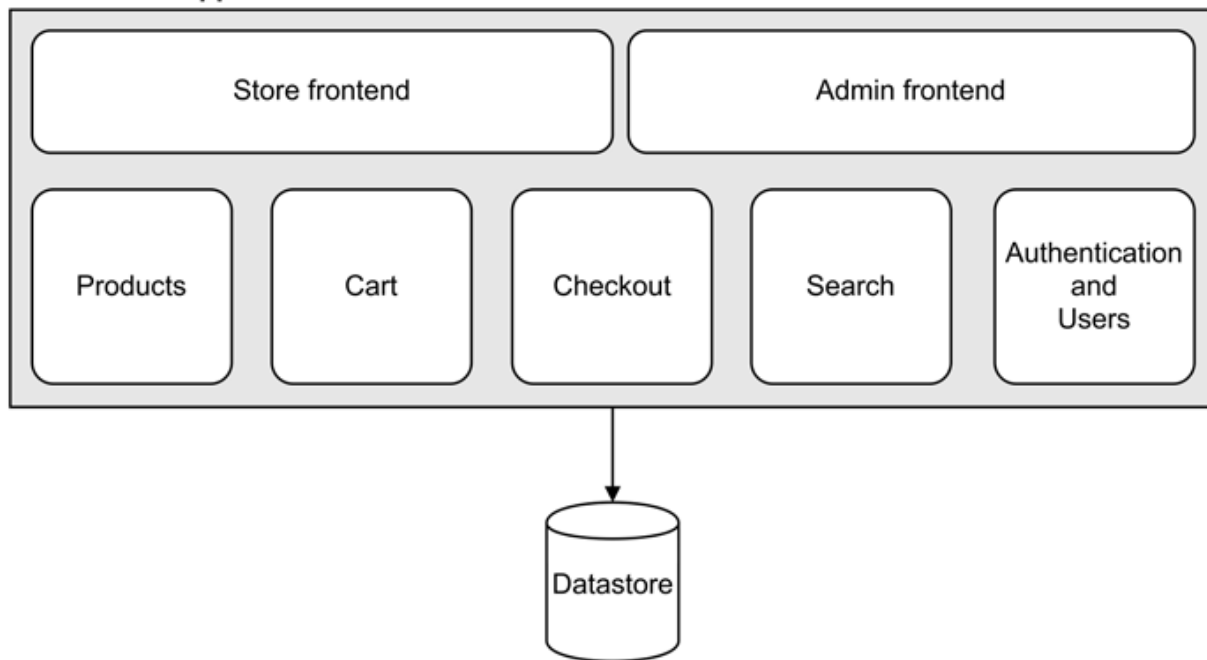


Figure 12.9: Example of a monolithic architecture

Hình 12.9: Ví dụ về một kiến trúc monolithic

Figure 12.9 shows the architecture of a typical e-commerce application. Its structure is modular: we have two different frontends, one for the main store and another for the administration interface. Internally, we have a clear separation of the services implemented by the application. Each service is responsible for a specific portion of the application business logic: Products, Cart, Checkout, Search, and Authentication and Users. However, the preceding architecture is monolithic since every module is part of the same codebase and runs as part of a single application. A failure in any of its components can potentially tear down the entire online store.

Hình 12.9 cho thấy kiến trúc của một ứng dụng thương mại điện tử điển hình. Cấu trúc của nó là module hóa: chúng ta có hai frontend khác nhau, một cho cửa hàng chính và một cho giao diện quản trị. Bên trong, chúng ta có một sự phân tách rõ ràng các dịch vụ được triển khai bởi ứng dụng. Mỗi dịch vụ chịu trách nhiệm cho một phần cụ thể của logic nghiệp vụ ứng dụng: Products, Cart, Checkout, Search, và Authentication and Users. Tuy nhiên, kiến trúc trên là monolithic vì mỗi module là một phần của cùng codebase và chạy như một phần của một ứng dụng duy nhất. Một lỗi trong bất kỳ thành phần nào của nó có thể tiềm tàng đánh sập toàn bộ cửa hàng trực tuyến.

Another problem with this type of architecture is the interconnection between its modules; the fact that they all live inside the same application makes it very easy for a developer to build interactions and coupling between modules. For example, consider the use case of when a product is being purchased: the Checkout module has to update the availability of a Product object, and if those two modules are in the same application, it's too easy for a developer to just obtain a reference to a Product object and update its availability directly. Maintaining a low coupling between internal modules is very hard in a monolithic application, partly because the boundaries between them are not always clear or properly enforced. A high coupling is often one of the main obstacles to the growth of an application and prevents its scalability in terms of complexity. In fact, an intricate

dependency graph means that every part of the system is a liability, it has to be maintained for the entire life of the product, and any change should be carefully evaluated because every component is like a wooden block in a Jenga tower: moving or removing one of them can cause the entire tower to collapse. This often results in building conventions and development processes to cope with the increasing complexity of the project.

Một vấn đề khác với loại kiến trúc này là sự liên kết giữa các module của nó; việc tất cả chúng cùng tồn tại bên trong cùng một ứng dụng khiến cho một nhà phát triển rất dễ dàng xây dựng các tương tác và sự ghép nối (coupling) giữa các module. Ví dụ, hãy xem xét trường hợp sử dụng khi một sản phẩm đang được mua: module Checkout phải cập nhật tình trạng có sẵn của một đối tượng Product, và nếu hai module đó nằm trong cùng một ứng dụng, một nhà phát triển quá dễ dàng để chỉ cần lấy một tham chiếu đến một đối tượng Product và cập nhật tình trạng có sẵn của nó trực tiếp. Việc duy trì một sự ghép nối thấp giữa các module nội bộ là rất khó trong một ứng dụng monolithic, một phần vì các ranh giới giữa chúng không phải lúc nào cũng rõ ràng hoặc được thực thi đúng cách. Một sự ghép nối cao thường là một trong những trở ngại chính cho sự phát triển của một ứng dụng và ngăn cản khả năng mở rộng của nó về mặt độ phức tạp. Trên thực tế, một đồ thị phụ thuộc rối rắm nghĩa là mọi phần của hệ thống đều là một gánh nặng, nó phải được duy trì trong suốt vòng đời của sản phẩm, và bất kỳ thay đổi nào cũng nên được đánh giá cẩn thận vì mỗi thành phần giống như một khối gỗ trong một tháp Jenga: di chuyển hoặc loại bỏ một trong số chúng có thể khiến toàn bộ tháp sụp đổ. Điều này thường dẫn đến việc xây dựng các quy ước và quy trình phát triển để đối phó với độ phức tạp ngày càng tăng của dự án.

The microservice architecture — Kiến trúc microservice

Now, we are going to reveal the most important pattern in Node.js for writing big applications: avoid writing big applications. This seems like a trivial statement, but it's an incredibly effective **strategy** to scale both the complexity and the capacity of a software system. So, what's the alternative to writing big applications? The answer is in the Y-axis of the scale cube: decomposition and splitting by service and functionality. The idea is to break down an application into its essential components, creating separate, independent applications. It is practically the opposite of a monolithic architecture. This fits perfectly with the Unix philosophy and the Node.js principles we discussed at the beginning of the book; in particular, the motto “make each program do one thing well.”

Bây giờ, chúng ta sẽ tiết lộ mẫu quan trọng nhất trong Node.js để viết các ứng dụng lớn: tránh viết các ứng dụng lớn. Điều này có vẻ như là một tuyên bố tầm thường, nhưng nó là một chiến lược cực kỳ hiệu quả để mở rộng cả độ phức tạp và năng lực của một hệ thống phần mềm. Vậy, lựa chọn thay thế cho việc viết các ứng dụng lớn là gì? Câu trả lời nằm ở trục Y của khối lập phương mở rộng: phân tách và chia tách theo dịch vụ và chức năng. Ý tưởng là chia nhỏ một ứng dụng thành các thành phần thiết yếu của nó, tạo ra các ứng dụng riêng biệt, độc lập. Nó thực tế là đối lập của một kiến trúc monolithic. Điều này hoàn toàn phù hợp với triết lý Unix và các nguyên tắc Node.js mà chúng ta đã thảo luận ở đầu cuốn sách; đặc biệt, phương châm “làm cho mỗi chương trình làm tốt một việc.”

Microservice architecture is, today, the main reference pattern for this type of approach, where a set of self-sufficient services replace big monolithic applications. The prefix “micro” means that the services should be as small as possible, but always within reasonable limits. Don't be misled by thinking that creating an architecture with a hundred different applications exposing only one web service is necessarily a good choice. In reality, there is no strict rule on how small or big a service should be. It's not the size that matters in the design of a microservice architecture; instead, it's a combination of different factors, mainly loose coupling, high cohesion, and integration complexity.

Kiến trúc microservice, ngày nay, là mẫu tham chiếu chính cho loại cách tiếp cận này, nơi một tập hợp các dịch vụ tự cung tự cấp thay thế các ứng dụng monolithic lớn. Tiền tố “micro” nghĩa là các dịch vụ nên càng nhỏ càng tốt, nhưng luôn trong các giới hạn hợp lý. Đừng bị nhầm lẫn bởi việc nghĩ rằng tạo ra một kiến trúc với một trăm ứng dụng khác nhau chỉ phơi bày một web service nhất thiết là một lựa chọn tốt. Trên thực tế, không có quy tắc nghiêm ngặt nào về việc một dịch vụ nên nhỏ hay lớn đến mức nào. Không phải kích thước mới là điều quan trọng trong thiết kế của một kiến trúc microservice; thay vào đó, đó là một sự kết hợp của các yếu tố khác nhau, chủ yếu là sự ghép nối lỏng (loose coupling), tính gắn kết cao (high cohesion), và độ phức tạp tích hợp.

An example of a microservice architecture — Một ví dụ về kiến trúc microservice

Let's now see what the monolithic e-commerce application would look like using a microservice architecture:

Bây giờ hãy xem ứng dụng thương mại điện tử monolithic sẽ trông như thế nào khi sử dụng một kiến trúc microservice:

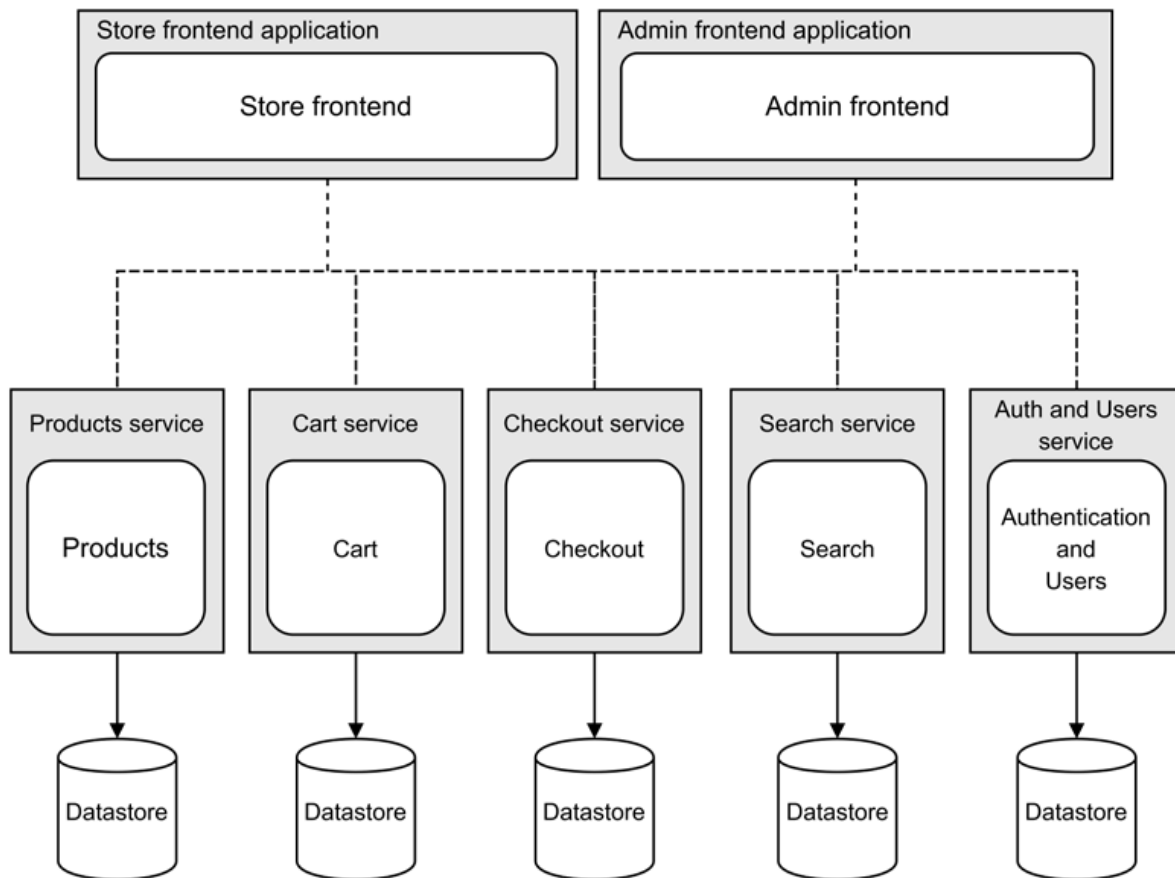


Figure 12.10: An example implementation of an e-commerce system using the Microservice pattern

Hình 12.10: Một ví dụ triển khai một hệ thống thương mại điện tử bằng mẫu Microservice

As we can see from Figure 12.10, each fundamental component of the e-commerce application is now a self-sustaining and independent entity, living in its own context, with its own database. In practice,

they are all independent applications exposing a set of related services.

Như chúng ta có thể thấy từ Hình 12.10, mỗi thành phần cơ bản của ứng dụng thương mại điện tử giờ là một thực thể độc lập và tự duy trì, sống trong bối cảnh riêng của nó, với cơ sở dữ liệu riêng của nó. Trên thực tế, tất cả chúng đều là các ứng dụng độc lập phối hợp một tập hợp các dịch vụ liên quan.

The data ownership of a service is an important characteristic of the microservice architecture. This is why the database also has to be split to maintain the proper level of isolation and independence. If a unique shared database is used, it would become much easier for the services to work together; however, this would also introduce a coupling between the services (based on data), nullifying some of the advantages of having different applications. The dashed lines connecting all the nodes tells us that, in some way, they have to communicate and exchange information for the entire system to be fully functional. As the services do not share the same database, there is more communication involved to maintain the consistency of the whole system. For example, the Checkout service needs to know some information about Products, such as the price and restrictions on shipping, and at the same time, it needs to update the data stored in the Products service such as the product's availability when the checkout is complete. In Figure 12.10, we tried to represent the way the nodes communicate generic. Surely, the most popular strategy is using web services, but as we will see later, this is not the only option.

Quyền sở hữu dữ liệu của một dịch vụ là một đặc điểm quan trọng của kiến trúc microservice. Đây là lý do tại sao cơ sở dữ liệu cũng phải được chia tách để duy trì mức độ cô lập và độc lập phù hợp. Nếu một cơ sở dữ liệu chia sẻ duy nhất được sử dụng, sẽ trở nên dễ dàng hơn nhiều cho các dịch vụ làm việc cùng nhau; tuy nhiên, điều này cũng sẽ đưa vào một sự ghép nối giữa các dịch vụ (dựa trên dữ liệu), vô hiệu hóa một số lợi thế của việc có các ứng dụng khác nhau. Các đường nét đứt kết nối tất cả các node nói cho chúng ta biết rằng, bằng một cách nào đó, chúng phải giao tiếp và trao đổi thông tin để toàn bộ hệ thống hoàn toàn hoạt động. Vì các dịch vụ không chia sẻ cùng một cơ sở dữ liệu, có nhiều giao tiếp liên quan hơn để duy trì tính nhất quán của toàn bộ hệ thống. Ví dụ, dịch vụ Checkout cần biết một số thông tin về Products, chẳng hạn như giá và các hạn chế về vận chuyển, và đồng thời, nó cần cập nhật dữ liệu được lưu trữ trong dịch vụ Products như tình trạng có sẵn của sản phẩm khi quá trình checkout hoàn tất. Trong Hình 12.10, chúng ta đã cố gắng biểu diễn cách các node giao tiếp một cách chung chung. Chắc chắn, chiến lược phổ biến nhất là sử dụng các web service, nhưng như chúng ta sẽ thấy sau, đây không phải là lựa chọn duy nhất.

Pattern (microservice architecture)

Mẫu (kiến trúc microservice)

Split a complex application by creating several small, selfcontained services.

Chia tách một ứng dụng phức tạp bằng cách tạo ra một số dịch vụ nhỏ, tự chứa.

Microservices – advantages and disadvantages — Microservice – ưu điểm và nhược điểm

In this section, we are going to highlight some of the advantages and disadvantages of implementing a microservice architecture. As we will see, this approach promises to bring a radical change in the way we develop our applications, revolutionizing the way we see scalability and complexity, but on the other hand, it introduces new nontrivial challenges.

Trong phần này, chúng ta sẽ làm nổi bật một số ưu điểm và nhược điểm của việc triển khai một kiến trúc microservice. Như chúng ta sẽ thấy, cách tiếp cận này hứa hẹn mang lại một

sự thay đổi triệt để trong cách chúng ta phát triển các ứng dụng của mình, cách mạng hóa cách chúng ta nhìn nhận khả năng mở rộng và độ phức tạp, nhưng mặt khác, nó đưa vào những thách thức mới không hề tầm thường.

Martin Fowler wrote a great article about microservices that you can find at nodejsdp.link/microservices.

Martin Fowler đã viết một bài báo tuyệt vời về microservice mà bạn có thể tìm thấy tại nodejsdp.link/microservices.

Every service is expendable — Mỗi dịch vụ đều có thể bỏ đi (expendable) The main technical advantage of having each service living in its own application context is that crashes do not propagate to the entire system. The goal is to build truly independent services that are smaller, easier to change, or can even be rebuilt from scratch. If, for example, the Checkout service of our e-commerce application suddenly crashes because of a serious bug, the rest of the system would continue to work as normal. Some functionality may be affected; for example, the ability to purchase a product, but the rest of the system would continue to work. Also, imagine if we suddenly realized that the database or the programming language we used to implement a component was not a good design decision. In a monolithic application, there would be very little we could do to change things without affecting the entire system. Instead, in a microservice architecture, we could more easily reimplement the entire service from scratch, using a different database or platform, and the rest of the system would not even notice it, as long as the new implementation maintains the same interface to the rest of the system.

Lợi thế kỹ thuật chính của việc có mỗi dịch vụ sống trong bối cảnh ứng dụng riêng của nó là các lỗi sập không lan truyền đến toàn bộ hệ thống. Mục tiêu là xây dựng các dịch vụ thực sự độc lập nhỏ hơn, dễ thay đổi hơn, hoặc thậm chí có thể được xây dựng lại từ đầu. Nếu, ví dụ, dịch vụ Checkout của ứng dụng thương mại điện tử của chúng ta đột nhiên sập vì một lỗi nghiêm trọng, phần còn lại của hệ thống sẽ tiếp tục hoạt động như bình thường. Một số chức năng có thể bị ảnh hưởng; ví dụ, khả năng mua một sản phẩm, nhưng phần còn lại của hệ thống sẽ tiếp tục hoạt động. Ngoài ra, hãy tưởng tượng nếu chúng ta đột nhiên nhận ra rằng cơ sở dữ liệu hoặc ngôn ngữ lập trình mà chúng ta đã sử dụng để triển khai một thành phần không phải là một quyết định thiết kế tốt. Trong một ứng dụng monolithic, sẽ có rất ít điều chúng ta có thể làm để thay đổi mọi thứ mà không ảnh hưởng đến toàn bộ hệ thống. Thay vào đó, trong một kiến trúc microservice, chúng ta có thể dễ dàng hơn triển khai lại toàn bộ dịch vụ từ đầu, sử dụng một cơ sở dữ liệu hoặc nền tảng khác, và phần còn lại của hệ thống thậm chí sẽ không nhận thấy điều đó, miễn là việc triển khai mới duy trì cùng một giao diện đến phần còn lại của hệ thống.

Reusability across platforms and languages — Khả năng tái sử dụng giữa các nền tảng và ngôn ngữ Splitting a big monolithic application into many small services allows us to create independent units that can be reused much more easily. Elasticsearch (nodejsdp.link/elasticsearch) is a great example of a reusable search service. ORY (nodejsdp.link/ory) is another example of a reusable open source technology that provides a complete authentication and authorization service that can be easily integrated into a microservice architecture.

Việc chia tách một ứng dụng monolithic lớn thành nhiều dịch vụ nhỏ cho phép chúng ta tạo ra các đơn vị độc lập có thể được tái sử dụng dễ dàng hơn nhiều. Elasticsearch (nodejsdp.link/elasticsearch) là một ví dụ tuyệt vời về một dịch vụ tìm kiếm có thể tái sử dụng. ORY (nodejsdp.link/ory) là một ví dụ khác về một công nghệ mã nguồn mở có thể tái sử dụng cung cấp một dịch vụ xác thực và ủy quyền hoàn chỉnh có thể được tích hợp dễ dàng vào một kiến trúc microservice.

The main advantage of the microservice approach is that the level of information hiding is usually much higher compared to monolithic applications. This is possible because the interactions usually happen through a remote interface such as a web API or a message **broker**, which makes it much easier to hide implementation details and shield the client from changes in the way the service is implemented or deployed. For example, if all we have to do is invoke a web service, we are shielded from the way the infrastructure behind is scaled, from what programming language it uses, from what database it uses to store its data, and so on. All these decisions can be revisited and adjusted as needed, with potentially no impact on the rest of the system.

Lợi thế chính của cách tiếp cận microservice là mức độ che giấu thông tin (information hiding) thường cao hơn nhiều so với các ứng dụng monolithic. Điều này có thể vì các tương tác thường xảy ra thông qua một giao diện từ xa như một web API hoặc một message broker, vốn khiến cho việc che giấu các chi tiết triển khai và bảo vệ client khỏi các thay đổi trong cách dịch vụ được hiện thực hoặc triển khai (deploy) trở nên dễ dàng hơn nhiều. Ví dụ, nếu tất cả những gì chúng ta phải làm là gọi một web service, chúng ta được bảo vệ khỏi cách hạ tầng phía sau được mở rộng, khỏi ngôn ngữ lập trình nào nó sử dụng, khỏi cơ sở dữ liệu nào nó sử dụng để lưu trữ dữ liệu của nó, và cứ thế. Tất cả các quyết định này có thể được xem xét lại và điều chỉnh khi cần, với khả năng không có tác động đến phần còn lại của hệ thống.

A way to scale the application — Một cách để mở rộng ứng dụng Going back to the scale cube, it's clear that microservices are equivalent to scaling an application along the Y-axis, so it's already a solution for distributing the load across multiple machines. Also, we should not forget that we can combine microservices with the other two dimensions of the cube to scale the application even further. For example, each service could be cloned to handle more traffic, and the interesting aspect is that they can be scaled independently, allowing better resource management.

Quay lại khối lập phương mở rộng, rõ ràng là microservice tương đương với việc mở rộng một ứng dụng dọc theo trục Y, vì vậy nó đã là một giải pháp để phân phối tải trên nhiều máy. Ngoài ra, chúng ta không nên quên rằng chúng ta có thể kết hợp microservice với hai chiều còn lại của khối lập phương để mở rộng ứng dụng thậm chí xa hơn nữa. Ví dụ, mỗi dịch vụ có thể được nhân bản để xử lý nhiều lưu lượng hơn, và khía cạnh thú vị là chúng có thể được mở rộng một cách độc lập, cho phép quản lý tài nguyên tốt hơn.

At this point, it would look like microservices are the solution to all our problems. However, this is far from being true. Let's see the challenges we face using microservices.

Tại thời điểm này, có vẻ như microservice là giải pháp cho tất cả các vấn đề của chúng ta. Tuy nhiên, điều này còn xa mới đúng. Hãy xem các thách thức mà chúng ta phải đối mặt khi sử dụng microservice.

The challenges of microservices — Các thách thức của microservice Having more nodes to manage introduces a higher complexity in terms of integration, deployment, and code sharing: it fixes some of the pains of traditional architectures, but it also opens up many new questions. How do we make the services interact? How can we keep sanity with deploying, scaling, and monitoring such a high number of applications? How can we share and reuse code between services?

Việc có nhiều node hơn để quản lý đưa vào một độ phức tạp cao hơn về mặt tích hợp, triển khai, và chia sẻ code: nó khắc phục một số nỗi đau của các kiến trúc truyền thống, nhưng nó cũng mở ra nhiều câu hỏi mới. Làm thế nào chúng ta khiến các dịch vụ tương tác? Làm thế nào chúng ta có thể giữ sự tinh tảo với việc triển khai, mở rộng, và giám sát một số

lượng cao các ứng dụng như vậy? Làm thế nào chúng ta có thể chia sẻ và tái sử dụng code giữa các dịch vụ?

Fortunately, cloud services and modern DevOps methodologies can provide some answers to those questions, and also, using Node.js can help a lot. Its **module system** is a perfect companion to share code between different projects. Node.js was made to be a node in a distributed system such as those of a microservice architecture.

May mắn thay, các dịch vụ đám mây và các phương pháp luận DevOps hiện đại có thể cung cấp một số câu trả lời cho những câu hỏi đó, và ngoài ra, việc sử dụng Node.js có thể giúp ích rất nhiều. Hệ thống module của nó là một người bạn đồng hành hoàn hảo để chia sẻ code giữa các dự án khác nhau. Node.js được tạo ra để là một node trong một hệ thống phân tán như những hệ thống của một kiến trúc microservice.

In the following sections, we will introduce some integration patterns that can help with managing and integrating services in a microservice architecture.

Trong các phần sau, chúng ta sẽ giới thiệu một số mẫu tích hợp có thể giúp ích với việc quản lý và tích hợp các dịch vụ trong một kiến trúc microservice.

Integration patterns in a microservice architecture — Các mẫu tích hợp trong một kiến trúc microservice

One of the toughest challenges of microservices is connecting all the nodes to make them collaborate. For example, the Cart service of our e-commerce application would make little sense without some Products to add, and the Checkout service would be useless without a list of products to buy (a cart). As we already mentioned, there are also other factors that necessitate an interaction between the various services. For example, the Search service has to know which Products are available and must also ensure it keeps its information up to date. The same can be said about the Checkout service, which has to update the information about Product availability when a purchase is completed.

Một trong những thách thức khó khăn nhất của microservice là kết nối tất cả các node để khiến chúng cộng tác. Ví dụ, dịch vụ Cart của ứng dụng thương mại điện tử của chúng ta sẽ ít có ý nghĩa nếu không có một số Products để thêm vào, và dịch vụ Checkout sẽ vô dụng nếu không có một danh sách sản phẩm để mua (một giỏ hàng - cart). Như chúng ta đã đề cập, cũng có những yếu tố khác đòi hỏi một sự tương tác giữa các dịch vụ khác nhau. Ví dụ, dịch vụ Search phải biết những Products nào khả dụng và cũng phải đảm bảo nó giữ thông tin của nó được cập nhật. Điều tương tự có thể nói về dịch vụ Checkout, vốn phải cập nhật thông tin về tình trạng có sẵn của Product khi một giao dịch mua được hoàn thành.

When designing an integration strategy, it's also important to consider the coupling that it's going to introduce between the services in the system. We should not forget that designing a distributed architecture involves the same practices and principles we use locally when designing a module or subsystem. Therefore, we also need to take into consideration properties such as the reusability and extensibility of the service.

Khi thiết kế một chiến lược tích hợp, điều quan trọng là cũng phải xem xét sự ghép nối mà nó sẽ đưa vào giữa các dịch vụ trong hệ thống. Chúng ta không nên quên rằng việc thiết kế một kiến trúc phân tán liên quan đến cùng các thực hành và nguyên tắc mà chúng ta sử dụng cục bộ khi thiết kế một module hoặc subsystem. Do đó, chúng ta cũng cần tính đến các thuộc tính như khả năng tái sử dụng và khả năng mở rộng của dịch vụ.

The API proxy — API proxy

The first pattern we are going to show makes use of an API proxy (also commonly identified as an API gateway), a server that proxies the communications between a client and a set of remote APIs. In a microservice architecture, its main purpose is to provide a single access point for multiple API endpoints, but it can also offer load balancing, caching, authentication, and traffic limiting, all of which are features that prove to be very useful to implement a solid API solution.

Mẫu đầu tiên mà chúng ta sẽ trình bày sử dụng một API proxy (cũng thường được xác định là một API gateway), một máy chủ làm proxy cho các giao tiếp giữa một client và một tập hợp các API từ xa. Trong một kiến trúc microservice, mục đích chính của nó là cung cấp một điểm truy cập duy nhất cho nhiều API endpoint, nhưng nó cũng có thể cung cấp cân bằng tải, caching, xác thực, và giới hạn lưu lượng, tất cả đều là các tính năng chứng tỏ rất hữu ích để triển khai một giải pháp API vững chắc.

This pattern should not be new to us since we already saw it in action in this chapter when we built the custom load balancer with `http-proxy` and `consul`. For that example, our load balancer was exposing only two services, and then, thanks to a service registry, it was able to map a URL path to a service and hence to a list of servers. An API proxy works in the same way; it is essentially a reverse proxy and often also a load balancer, specifically configured to handle API requests. Figure 12.11 shows how we can apply such a solution to our e-commerce application:

Mẫu này không nên là mới với chúng ta vì chúng ta đã thấy nó trong hành động trong chương này khi chúng ta xây dựng bộ cân bằng tải tùy chỉnh với `http-proxy` và `consul`. Đối với ví dụ đó, bộ cân bằng tải của chúng ta chỉ phơi bày hai dịch vụ, và sau đó, nhờ một service registry, nó có thể ánh xạ một đường dẫn URL đến một dịch vụ và do đó đến một danh sách máy chủ. Một API proxy hoạt động theo cùng cách; về cơ bản nó là một reverse proxy và thường cũng là một bộ cân bằng tải, được cấu hình cụ thể để xử lý các yêu cầu API. Hình 12.11 cho thấy cách chúng ta có thể áp dụng một giải pháp như vậy cho ứng dụng thương mại điện tử của mình:

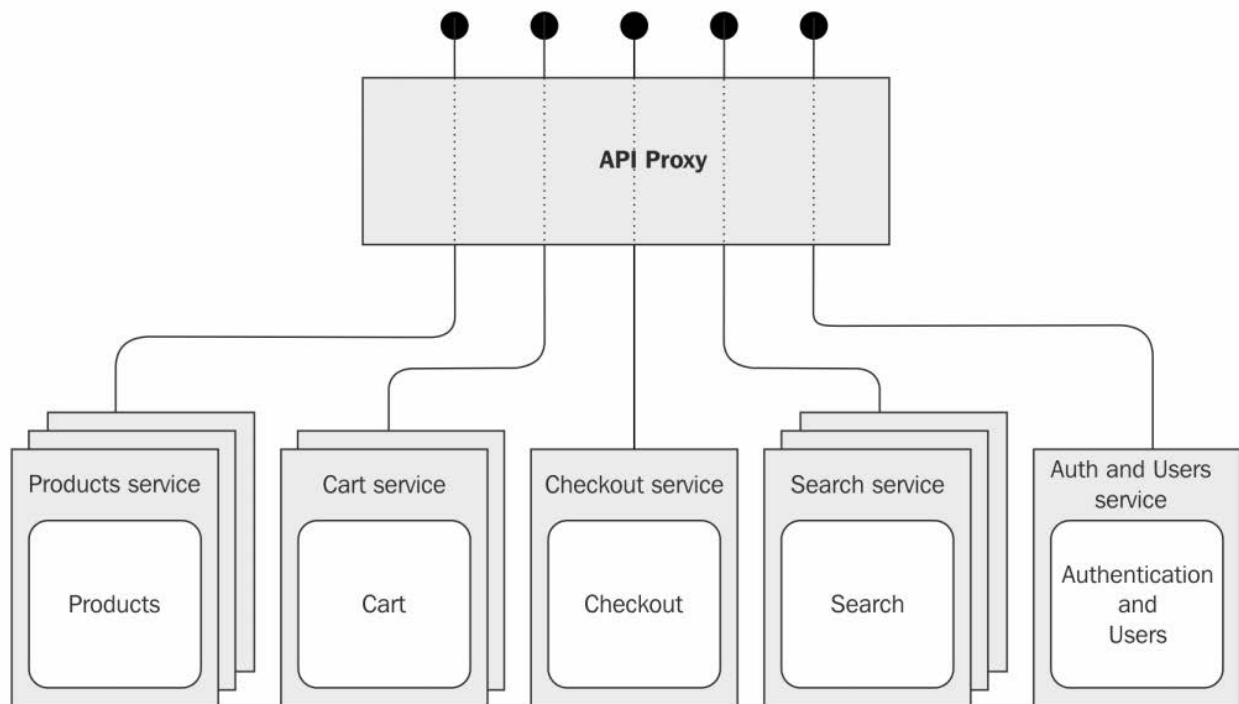


Figure 12.11: Using the API Proxy pattern in an e-commerce application

Hình 12.11: Sử dụng mẫu API Proxy trong một ứng dụng thương mại điện tử

From the preceding diagram, it should be clear how an API proxy can hide the complexity of its underlying infrastructure. This is really handy in a microservice infrastructure, as the number of nodes may be high, especially if each service is scaled across multiple machines. The integration achieved by an API proxy is therefore only structural since there is no semantic mechanism. It simply provides a familiar monolithic view of a complex microservice infrastructure. Since the API Proxy pattern essentially abstracts the complexity of connecting to all the different APIs in the system, it might also allow for some freedom to restructure the various services. Maybe, as your requirements change, you will need to split an existing microservice into two or more decoupled microservices or, conversely, you might realize that, in your business context, it's better to join two or more services together. In both cases, the API Proxy pattern will allow you to make all the necessary changes with potentially no impact on the upstream systems accessing the data through the proxy.

Từ sơ đồ trên, nên rõ ràng cách một API proxy có thể che giấu độ phức tạp của hạ tầng bên dưới của nó. Điều này thực sự tiện lợi trong một hạ tầng microservice, vì số lượng node có thể cao, đặc biệt nếu mỗi dịch vụ được mở rộng trên nhiều máy. Sự tích hợp đạt được bởi một API proxy do đó chỉ là về mặt cấu trúc vì không có cơ chế ngữ nghĩa. Nó chỉ đơn giản cung cấp một cái nhìn monolithic quen thuộc về một hạ tầng microservice phức tạp. Vì mẫu API Proxy về cơ bản trừu tượng hóa độ phức tạp của việc kết nối đến tất cả các API khác nhau trong hệ thống, nó cũng có thể cho phép một số tự do để tái cấu trúc các dịch vụ khác nhau. Có thể, khi các yêu cầu của bạn thay đổi, bạn sẽ cần chia một microservice hiện có thành hai hoặc nhiều microservice được tách rời hoặc, ngược lại, bạn có thể nhận ra rằng, trong bối cảnh kinh doanh của bạn, sẽ tốt hơn nếu gộp hai hoặc nhiều dịch vụ lại với nhau. Trong cả hai trường hợp, mẫu API Proxy sẽ cho phép bạn thực hiện tất cả các thay đổi cần thiết với khả năng không có tác động đến các hệ thống thượng nguồn (upstream) truy cập dữ liệu thông qua proxy.

The ability to enable incremental change in an architecture over time is a very important characteristic in modern distributed systems. If you are interested in studying this broad subject in greater depth, we recommend the book *Building Evolutionary Architectures*: nodejsdp.link/evolutionary-architectures.

Khả năng cho phép thay đổi tăng dần (incremental change) trong một kiến trúc theo thời gian là một đặc điểm rất quan trọng trong các hệ thống phân tán hiện đại. Nếu bạn quan tâm đến việc nghiên cứu chủ đề rộng này sâu hơn, chúng tôi khuyên dùng cuốn sách *Building Evolutionary Architectures*: nodejsdp.link/evolutionary-architectures.

API orchestration — Điều phối API (API orchestration)

The pattern we are going to describe next is probably the most natural and explicit way to integrate and compose a set of services, and it's called API orchestration. Daniel Jacobson, VP of Engineering for the Netflix API, in one of his blog posts (nodejsdp.link/orchestration-layer), defines API orchestration as follows:

Mẫu mà chúng ta sẽ mô tả tiếp theo có lẽ là cách tự nhiên và tường minh nhất để tích hợp và soạn (compose) một tập hợp các dịch vụ, và nó được gọi là điều phối API (API orchestration). Daniel Jacobson, Phó Chủ tịch Kỹ thuật của Netflix API, trong một trong các bài đăng blog của ông (nodejsdp.link/orchestration-layer), định nghĩa điều phối API như sau:

“An API Orchestration Layer (OL) is an abstraction layer that takes generically modeled data elements and/or features and prepares them in a more specific way for a targeted developer or application.”

“Một Lớp Điều phối API (Orchestration Layer - OL) là một lớp trừu tượng nhận các phần tử dữ liệu và/hoặc tính năng được mô hình hóa một cách chung chung và chuẩn bị chúng theo một cách cụ thể hơn cho một nhà phát triển hoặc ứng dụng mục tiêu.”

The “generically modeled elements and/or features” fit the description of a service in a microservice architecture perfectly. The idea is to create an abstraction to connect those bits and pieces to implement new services specific to a particular application. Let’s see an example using the e-commerce application. Refer to Figure 12.12:

“Các phần tử và/hoặc tính năng được mô hình hóa chung chung” phù hợp hoàn hảo với mô tả về một dịch vụ trong một kiến trúc microservice. Ý tưởng là tạo ra một trừu tượng để kết nối những mảnh ghép đó để triển khai các dịch vụ mới cụ thể cho một ứng dụng cụ thể. Hãy xem một ví dụ sử dụng ứng dụng thương mại điện tử. Tham khảo Hình 12.12:

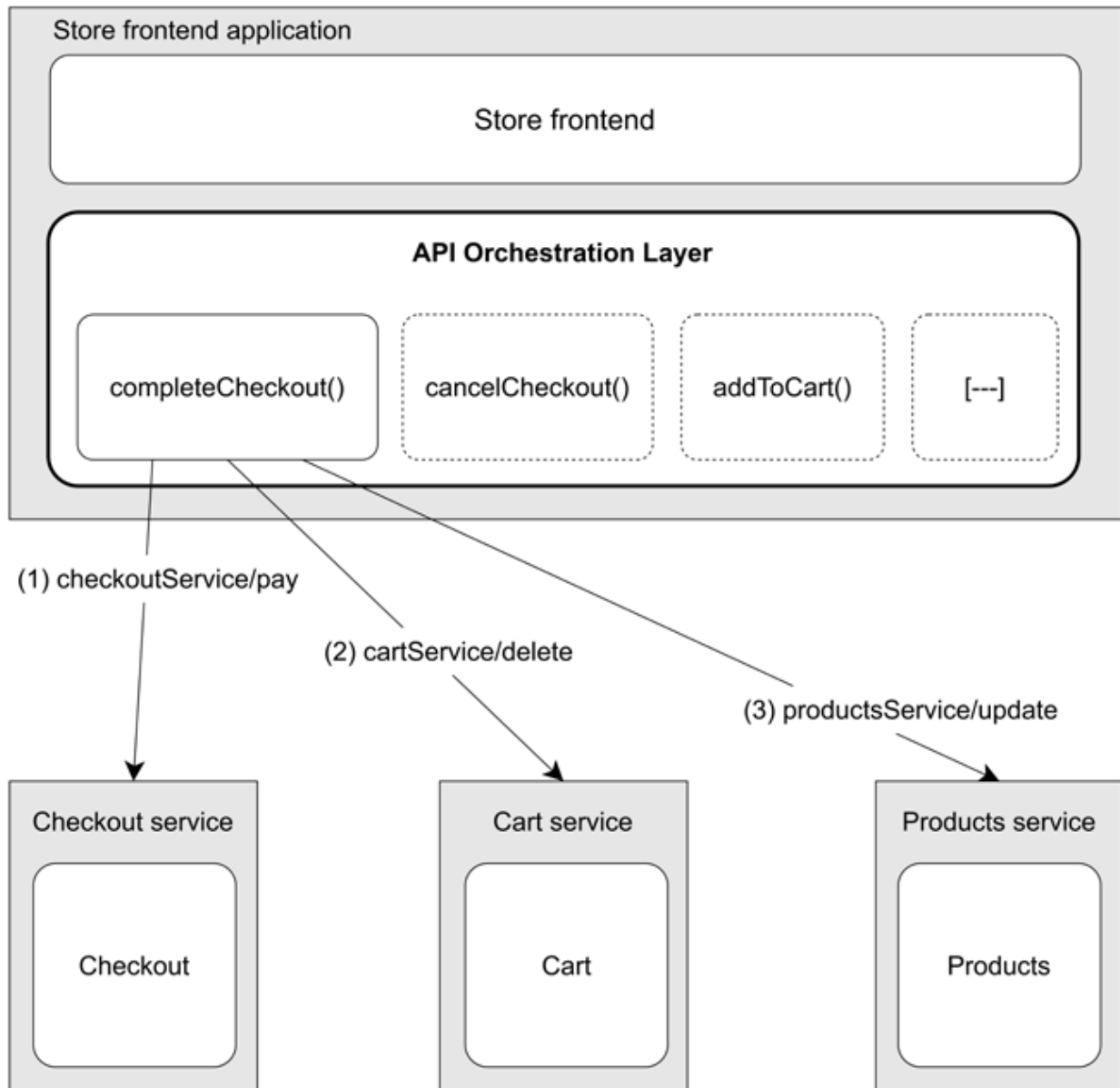


Figure 12.12: An example usage of an orchestration layer to interact with multiple microservices

Hình 12.12: Một ví dụ sử dụng một lớp điều phối để tương tác với nhiều microservice

Figure 12.12 shows how the Store frontend application uses an orchestration layer to build more complex and specific features by composing and orchestrating existing services. The described scenario takes, as an example, a hypothetical `completeCheckout()` service that is invoked the moment a customer clicks the Pay button at the end of the checkout. The figure shows how `completeCheckout()` is a composite operation made of three different steps:

Hình 12.12 cho thấy cách ứng dụng frontend Store sử dụng một lớp điều phối để xây dựng các tính năng phức tạp và cụ thể hơn bằng cách soạn và điều phối các dịch vụ hiện có. Kịch bản được mô tả lấy, làm ví dụ, một dịch vụ `completeCheckout()` giả định được gọi vào lúc một khách hàng nhấp vào nút Pay ở cuối quá trình checkout. Hình cho thấy `completeCheckout()` là một thao tác tổng hợp được tạo thành từ ba bước khác nhau như

thế nào:

1. First, we complete the transaction by invoking `checkoutService/pay`.
 2. Then, when the payment is successfully processed, we need to tell the Cart service that the items were purchased and that they can be removed from the cart. We do that by invoking `cartService/delete`.
 3. Also, when the payment is complete, we need to update the availability of the products that were just purchased. This is done through `productsService/update`.
1. Đầu tiên, chúng ta hoàn thành giao dịch bằng cách gọi `checkoutService/pay`.
 2. Sau đó, khi khoản thanh toán được xử lý thành công, chúng ta cần nói với dịch vụ Cart rằng các mặt hàng đã được mua và rằng chúng có thể được xóa khỏi giỏ hàng. Chúng ta làm điều đó bằng cách gọi `cartService/delete`.
 3. Ngoài ra, khi khoản thanh toán hoàn tất, chúng ta cần cập nhật tình trạng có sẵn của các sản phẩm vừa được mua. Điều này được thực hiện thông qua `productsService/update`.

As we can see, we took three operations from three different services and we built a new API that coordinates the services to maintain the entire system in a consistent state.

Như chúng ta có thể thấy, chúng ta đã lấy ba thao tác từ ba dịch vụ khác nhau và chúng ta đã xây dựng một API mới phối hợp các dịch vụ để duy trì toàn bộ hệ thống ở trạng thái nhất quán.

Another common operation performed by the API Orchestration Layer is data aggregation, or in other words, combining data from different services into a single response. Imagine we wanted to list all the products contained in a cart. In this case, the orchestration would need to retrieve the list of product IDs from the Cart service, and then retrieve the complete information about the products from the Products service. The ways in which we can combine and coordinate services is infinite, but the important pattern to remember is the role of the orchestration layer, which acts as an abstraction between a number of services and a specific application.

Một thao tác phổ biến khác được thực hiện bởi Lớp Điều phối API là tổng hợp dữ liệu (data aggregation), hay nói cách khác, kết hợp dữ liệu từ các dịch vụ khác nhau vào một phản hồi duy nhất. Hãy tưởng tượng chúng ta muốn liệt kê tất cả các sản phẩm có trong một giỏ hàng. Trong trường hợp này, việc điều phối sẽ cần lấy danh sách các ID sản phẩm từ dịch vụ Cart, và sau đó lấy thông tin đầy đủ về các sản phẩm từ dịch vụ Products. Các cách mà chúng ta có thể kết hợp và phối hợp các dịch vụ là vô hạn, nhưng mẫu quan trọng cần nhớ là vai trò của lớp điều phối, vốn đóng vai trò như một trừu tượng giữa một số dịch vụ và một ứng dụng cụ thể.

The orchestration layer is a great candidate for a further functional splitting. It is, in fact, very common to have it implemented as a dedicated, independent service, in which case it takes the name of API Orchestrator. This practice is perfectly in line with the microservice philosophy. Figure 12.13 shows this further improvement of our architecture:

Lớp điều phối là một ứng cử viên tuyệt vời cho việc chia tách chức năng hơn nữa. Trên thực tế, rất phổ biến khi nó được triển khai như một dịch vụ chuyên dụng, độc lập, trong trường hợp đó nó mang tên API Orchestrator. Thực hành này hoàn toàn phù hợp với triết lý microservice. Hình 12.13 cho thấy sự cải tiến hơn nữa này của kiến trúc của chúng ta:

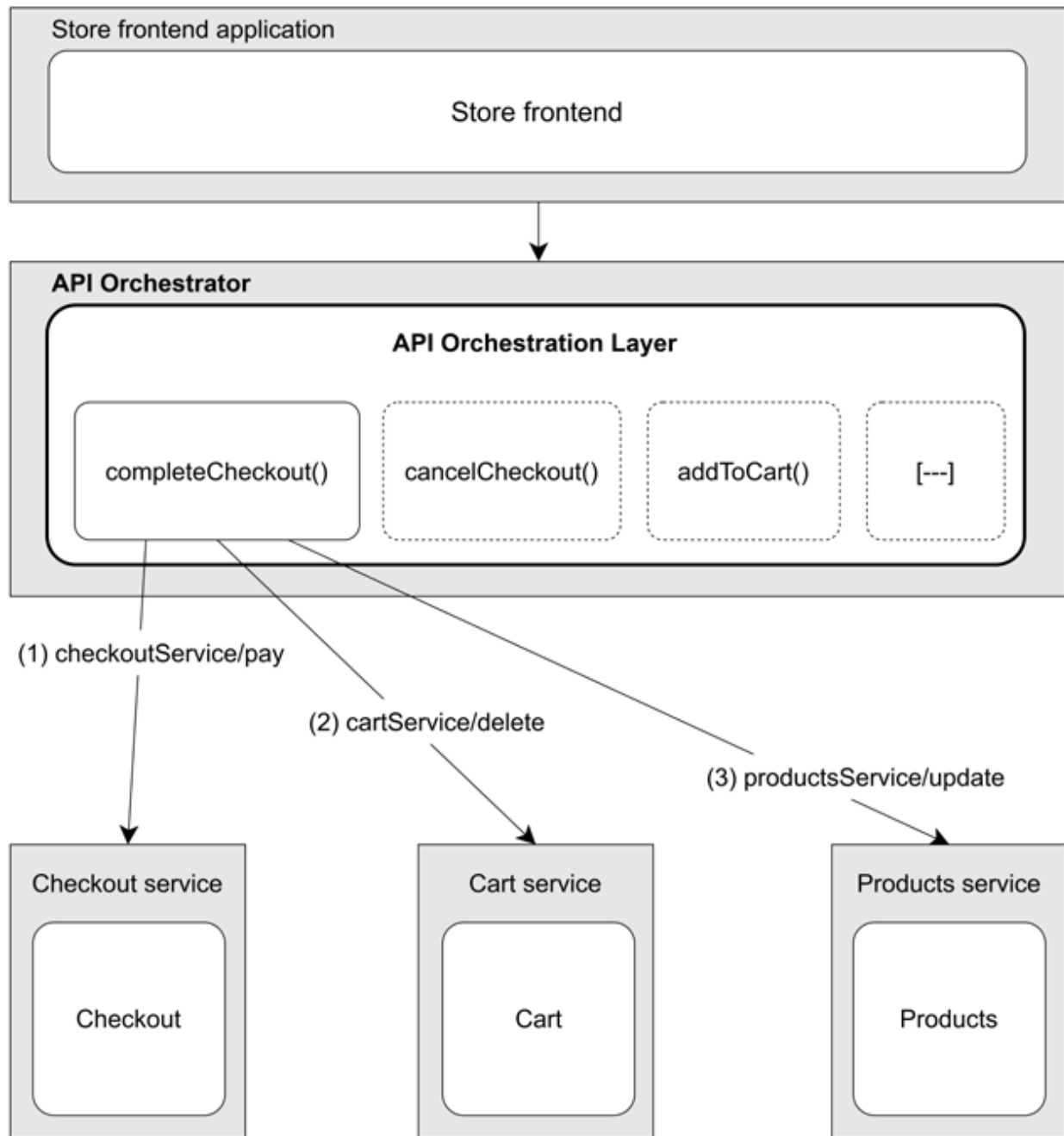


Figure 12.13: An application of the API Orchestrator pattern for our e-commerce example. Creating a standalone orchestrator, as shown in the previous figure, can help in decoupling the client application (in our case, the Store frontend) from the complexity of the microservice infrastructure. This is similar to the API proxy, but there is a crucial difference: an orchestrator performs a semantic integration of the various services, it's not just a naïve proxy, and it often exposes an API that is different from the one exposed by the underlying services.

Hình 12.13: Một ứng dụng của mẫu API Orchestrator cho ví dụ thương mại điện tử của chúng ta. Việc tạo ra một orchestrator độc lập, như được thể hiện trong hình trước, có thể giúp ích trong việc tách rời ứng dụng client (trong trường hợp của chúng ta, frontend Store) khỏi độ phức tạp của hạ tầng microservice. Điều này tương tự như API proxy, nhưng

có một sự khác biệt quan trọng: một orchestrator thực hiện một sự tích hợp ngữ nghĩa của các dịch vụ khác nhau, nó không chỉ là một proxy ngây thơ, và nó thường phơi bày một API khác với cái được phơi bày bởi các dịch vụ bên dưới.

Integration with a message broker — Tích hợp với một message broker

The Orchestrator pattern gave us a mechanism to integrate the various services in an explicit way. This has both advantages and disadvantages. It is easy to design, easy to debug, and easy to scale, but unfortunately, it has to have a complete knowledge of the underlying architecture and how each service works. If we were talking about objects instead of architectural nodes, the orchestrator would be an **antipattern** called God object, which defines an object that knows and does too much, which usually results in high coupling, low cohesion, but most importantly, high complexity.

Mẫu Orchestrator đã cho chúng ta một cơ chế để tích hợp các dịch vụ khác nhau theo một cách tường minh. Điều này có cả ưu điểm và nhược điểm. Nó dễ thiết kế, dễ debug, và dễ mở rộng, nhưng tiếc là, nó phải có một kiến thức hoàn chỉnh về kiến trúc bên dưới và cách mỗi dịch vụ hoạt động. Nếu chúng ta đang nói về các đối tượng thay vì các node kiến trúc, orchestrator sẽ là một antipattern gọi là God object, vốn định nghĩa một đối tượng biết và làm quá nhiều, mà thường dẫn đến sự ghép nối cao, tính gắn kết thấp, nhưng quan trọng nhất, độ phức tạp cao.

The pattern we are now going to show tries to distribute, across the services, the responsibility of synchronizing the information of the entire system. However, the last thing we want to do is create direct relationships between services, which would result in high coupling and a further increase in the complexity of the system, due to the increasing number of interconnections between nodes. The goal is to keep every service decoupled: every service should be able to work, even without the rest of the services in the system or in combination with new services and nodes.

Mẫu mà chúng ta sẽ trình bày bây giờ cố gắng phân phối, trên các dịch vụ, trách nhiệm đồng bộ hóa thông tin của toàn bộ hệ thống. Tuy nhiên, điều cuối cùng chúng ta muốn làm là tạo ra các mối quan hệ trực tiếp giữa các dịch vụ, mà sẽ dẫn đến sự ghép nối cao và một sự gia tăng hơn nữa độ phức tạp của hệ thống, do số lượng các kết nối giữa các node ngày càng tăng. Mục tiêu là giữ cho mọi dịch vụ được tách rời: mỗi dịch vụ nên có thể hoạt động, ngay cả khi không có phần còn lại của các dịch vụ trong hệ thống hoặc kết hợp với các dịch vụ và node mới.

The solution is to use a message broker, a system capable of decoupling the sender from the receiver of a message, allowing us to implement a Centralized Publish/ **Subscribe** pattern. This is, in practice, an implementation of the **Observer** pattern for distributed systems. We will talk more about this pattern later in Chapter 13, Messaging and Integration Patterns. Figure 12.14 shows an example of how this applies to the e-commerce application:

Giải pháp là sử dụng một message broker, một hệ thống có khả năng tách rời người gửi khỏi người nhận của một thông điệp, cho phép chúng ta triển khai một mẫu Publish/Subscribe Tập trung (Centralized Publish/Subscribe). Đây, trên thực tế, là một sự triển khai của mẫu Observer cho các hệ thống phân tán. Chúng ta sẽ nói thêm về mẫu này sau trong Chương 13, Các mẫu Nhắn tin và Tích hợp. Hình 12.14 cho thấy một ví dụ về cách điều này áp dụng cho ứng dụng thương mại điện tử:

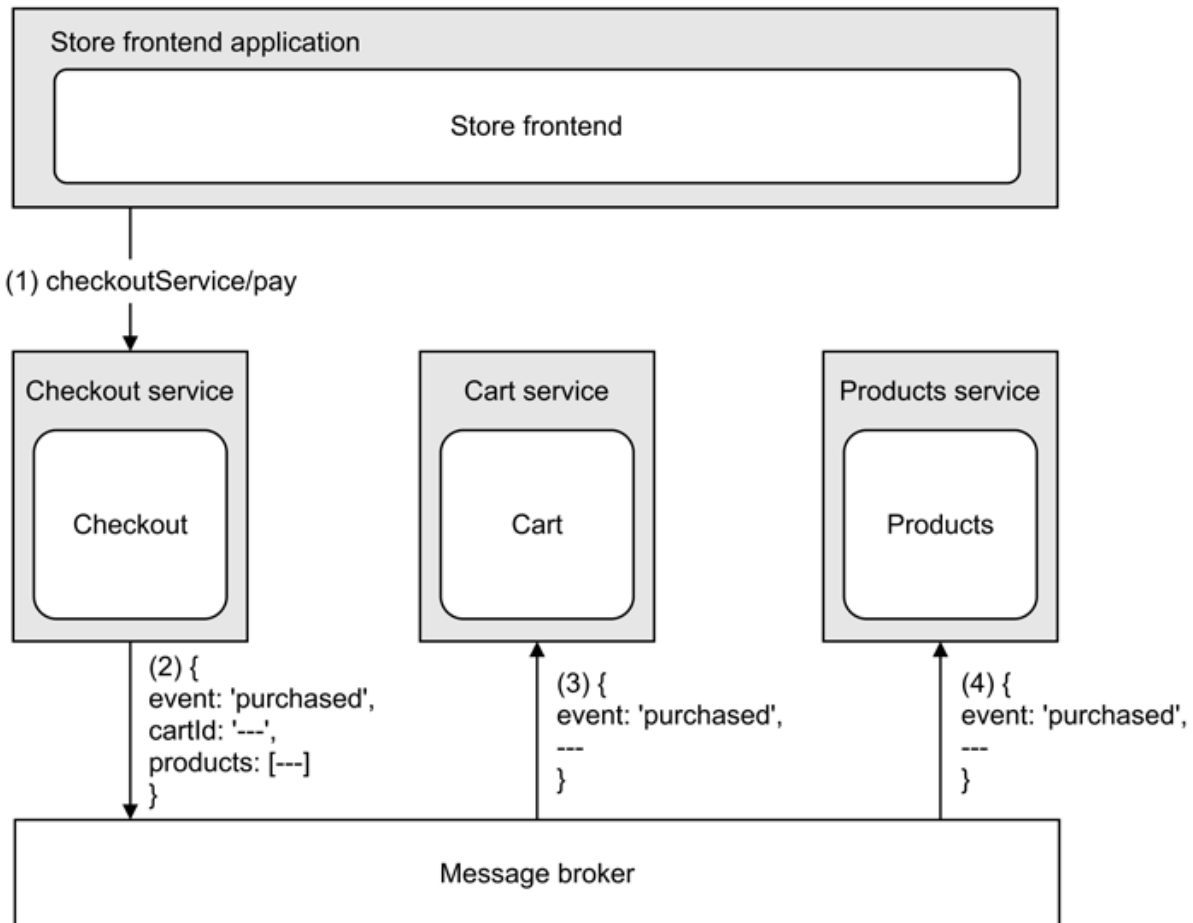


Figure 12.14: Using a message broker to distribute events in our e-commerce application

Hình 12.14: Sử dụng một message broker để phân phối các sự kiện trong ứng dụng thương mại điện tử của chúng ta

As we can see from Figure 12.14, the client of the Checkout service, which is the frontend application, does not need to carry out any explicit integration with the other services. All it has to do is invoke `checkoutService/pay` to complete the checkout process and take the money from the customer; all the integration work happens in the background:

Như chúng ta có thể thấy từ Hình 12.14, client của dịch vụ Checkout, vốn là ứng dụng frontend, không cần thực hiện bất kỳ sự tích hợp tường minh nào với các dịch vụ khác. Tất cả những gì nó phải làm là gọi `checkoutService/pay` để hoàn thành quá trình checkout và lấy tiền từ khách hàng; tất cả công việc tích hợp xảy ra ở hậu trường:

1. The Store frontend invokes the `checkoutService/pay` operation on the Checkout service.
2. When the operation completes, the Checkout service generates an event, attaching the details of the operation, that is, the `cartId` and the list of `products` that were just purchased. The event is published into the message broker. At this point, the Checkout service does not know who is going to receive the message.
3. The Cart service is subscribed to the broker, so it's going to receive the purchased event that was just published by the Checkout service. The Cart service reacts by removing the cart identified with the ID contained in the message from its database.

4. The Products service was subscribed to the message broker as well, so it receives the same purchased event. It then updates its database based on this new information, adjusting the availability of the products included in the message.

1. Frontend Store gọi thao tác `checkoutService/pay` trên dịch vụ Checkout.
2. Khi thao tác hoàn thành, dịch vụ Checkout tạo ra một sự kiện, đính kèm các chi tiết của thao tác, đó là `cartId` và danh sách `products` vừa được mua. Sự kiện được publish vào message broker. Tại thời điểm này, dịch vụ Checkout không biết ai sẽ nhận thông điệp.
3. Dịch vụ Cart đã subscribe vào broker, vì vậy nó sẽ nhận sự kiện `purchased` vừa được publish bởi dịch vụ Checkout. Dịch vụ Cart phản ứng bằng cách xóa giỏ hàng được xác định với ID có trong thông điệp khỏi cơ sở dữ liệu của nó.
4. Dịch vụ Products cũng đã subscribe vào message broker, vì vậy nó nhận cùng sự kiện `purchased`. Sau đó nó cập nhật cơ sở dữ liệu của nó dựa trên thông tin mới này, điều chỉnh tình trạng có sẵn của các sản phẩm có trong thông điệp.

This whole process happens without any explicit intervention from external entities such as an orchestrator. The responsibility of spreading the knowledge and keeping information in sync is distributed across the services themselves. There is no god service that has to know how to move the gears of the entire system, since each service is in charge of its own part of the integration.

Toàn bộ quá trình này xảy ra mà không có bất kỳ sự can thiệp tường minh nào từ các thực thể bên ngoài như một orchestrator. Trách nhiệm lan truyền kiến thức và giữ thông tin đồng bộ được phân phối trên chính các dịch vụ. Không có dịch vụ god nào phải biết cách di chuyển các bánh răng của toàn bộ hệ thống, vì mỗi dịch vụ phụ trách phần tích hợp của riêng nó.

The message broker is a fundamental element used to decouple the services and reduce the complexity of their interaction. It might also offer other interesting features, such as persistent message queues and guaranteed ordering of the messages. We will talk more about this in the next chapter.

Message broker là một phần tử cơ bản được sử dụng để tách rời các dịch vụ và giảm độ phức tạp của sự tương tác của chúng. Nó cũng có thể cung cấp các tính năng thú vị khác, chẳng hạn như các hàng đợi thông điệp bền vững (persistent message queue) và đảm bảo thứ tự của các thông điệp. Chúng ta sẽ nói thêm về điều này trong chương tiếp theo.

Summary — Tóm tắt

In this chapter, we learned how to design Node.js architectures that scale both in capacity and complexity. We saw how scaling an application is not only about handling more traffic or reducing the response time, but it's also a practice to apply whenever we want better availability and tolerance to failures. We saw how these properties often are on the same wavelength, and we understood that scaling early is not a bad practice, especially in Node.js, which allows us to do it easily and with few resources. The scale cube taught us that applications can be scaled across three dimensions. Throughout this chapter, we focused on the two most important dimensions, the X-and Y-axes, allowing us to discover two essential architectural patterns, namely, load balancing and microservices. You should now know how to start multiple instances of the same Node.js application, how to distribute traffic across them, and how to exploit this setup for other purposes, such as fail tolerance and zero-downtime restarts. We also analyzed how to handle the problem of dynamic and auto-scaled infrastructures. With this, we saw that a service registry can really come in useful for those situations. We learned how to achieve these goals by using plain Node.js, external

load balancers like Nginx, and service discovery systems like Consul. We also learned the basics of Kubernetes.

Trong chương này, chúng ta đã học cách thiết kế các kiến trúc Node.js mở rộng cả về năng lực và độ phức tạp. Chúng ta đã thấy việc mở rộng một ứng dụng không chỉ là về việc xử lý nhiều lưu lượng hơn hoặc giảm thời gian phản hồi, mà nó còn là một thực hành để áp dụng bất cứ khi nào chúng ta muốn tính sẵn sàng và khả năng chịu lỗi tốt hơn. Chúng ta đã thấy các thuộc tính này thường ở trên cùng một bước sóng, và chúng ta đã hiểu rằng việc mở rộng sớm không phải là một thực hành tồi, đặc biệt trong Node.js, vốn cho phép chúng ta làm điều đó dễ dàng và với ít tài nguyên. Khối lập phương mở rộng đã dạy chúng ta rằng các ứng dụng có thể được mở rộng trên ba chiều. Xuyên suốt chương này, chúng ta đã tập trung vào hai chiều quan trọng nhất, các trục X và Y, cho phép chúng ta khám phá hai mẫu kiến trúc thiết yếu, đó là cân bằng tải và microservice. Bây giờ bạn nên biết cách khởi động nhiều instance của cùng một ứng dụng Node.js, cách phân phối lưu lượng trên chúng, và cách khai thác thiết lập này cho các mục đích khác, chẳng hạn như khả năng chịu lỗi và khởi động lại không gián đoạn. Chúng ta cũng đã phân tích cách xử lý vấn đề của các hạ tầng động và tự động mở rộng. Với điều này, chúng ta đã thấy rằng một service registry thực sự có thể trở nên hữu ích cho những tình huống đó. Chúng ta đã học cách đạt được những mục tiêu này bằng cách sử dụng Node.js thuần túy, các bộ cân bằng tải bên ngoài như Nginx, và các hệ thống khám phá dịch vụ như Consul. Chúng ta cũng đã học những điều cơ bản về Kubernetes.

At this point, we should have got to grips with some very practical approaches to be able to face scalability much more fearlessly than before.

Tại thời điểm này, chúng ta nên đã nắm vững một số cách tiếp cận rất thực tế để có thể đối mặt với khả năng mở rộng một cách ít sợ hãi hơn nhiều so với trước đây.

However, cloning and load balancing cover only one dimension of the scale cube, so we moved our analysis to another dimension, studying in more detail what it means to split an application by its constituent services by building a microservice architecture. We saw how microservices enable a complete revolution in how a project is developed and managed, providing a natural way to distribute the load of an application and split its complexity. However, we learned that this also means shifting the complexity from how to build a big monolithic application to how to integrate a set of services. This last aspect is where we focused the last part of our analysis, showing some of the architectural solutions to integrate a set of independent services.

Tuy nhiên, nhân bản và cân bằng tải chỉ bao phủ một chiều của khối lập phương mở rộng, vì vậy chúng ta đã chuyển phân tích của mình sang một chiều khác, nghiên cứu chi tiết hơn việc chia một ứng dụng theo các dịch vụ cấu thành của nó bằng cách xây dựng một kiến trúc microservice có nghĩa là gì. Chúng ta đã thấy cách microservice cho phép một cuộc cách mạng hoàn chỉnh trong cách một dự án được phát triển và quản lý, cung cấp một cách tự nhiên để phân phối tải của một ứng dụng và chia tách độ phức tạp của nó. Tuy nhiên, chúng ta đã học rằng điều này cũng có nghĩa là chuyển độ phức tạp từ cách xây dựng một ứng dụng monolithic lớn sang cách tích hợp một tập hợp các dịch vụ. Khía cạnh cuối cùng này là nơi chúng ta đã tập trung phần cuối của phân tích của mình, trình bày một số giải pháp kiến trúc để tích hợp một tập hợp các dịch vụ độc lập.

In the next and last chapter of this book, we will have the chance to complete our Node.js Design Patterns journey by analyzing the messaging patterns we discussed in this chapter, in addition to more advanced integration techniques that are useful when implementing complex distributed architectures.

Trong chương tiếp theo và cuối cùng của cuốn sách này, chúng ta sẽ có cơ hội hoàn thành

hành trình Node.js Design Patterns của mình bằng cách phân tích các mẫu nhần tin mà chúng ta đã thảo luận trong chương này, cùng với các kỹ thuật tích hợp nâng cao hơn hữu ích khi triển khai các kiến trúc phân tán phức tạp.

Exercises — Bài tập

- 12.1 A scalable book library: Revisit the book library application we built in Chapter 10, **Universal JavaScript** for Web Applications, reconsidering it after what we learned in this chapter. Can you make our original implementation more scalable? Some ideas might be to use the `cluster` module to run multiple instances of the server, making sure you handle failures by restarting workers that might accidentally die. Alternatively, why not try to run the entire application on Kubernetes?
- 12.2 Exploring the Z-axis: Throughout this chapter, we did not show you any examples about how to shard data across multiple instances, but we explored all the necessary patterns to build an application that achieves scalability along the Z-axis of the scale cube. In this exercise, you are challenged to build a REST API that allows you to get a list of (randomly generated) people whose first name starts with a given letter. You could use a library like `faker` (nodejsdp.link/faker) to generate a sample of random people, and then you could store this data in different JSON files (or different databases), splitting the data into three different groups. For instance, you might have three groups called A-D, E-P, and Q-Z. Ada will go in the first group, Peter in the second, and Ugo in the third. Now, you can run one or more instances of a web server for every group, but you should expose only one public API endpoint to be able to retrieve all the people whose names starts with a given letter (for instance, `/api/people/byFirstName/{letter}`). Hint: You could use just a load balancer and map all the possible letters to the respective backend of the instances that are responsible for the associated group. Alternatively, you could create an API orchestration layer that encodes the mapping logic and redirects the traffic accordingly. Can you also throw a service discovery tool into the mix and apply dynamic load balancing, so that groups receiving more traffic can scale as needed?
- 12.3 Music addiction: Imagine you have to design the architecture of a service like Spotify or Apple Music. Can you try to design this service as a collection of microservices by applying some of the principles discussed in this chapter? Bonus points if you can actually implement a minimal version of this idea with Node.js! If this turns out to be the next big startup idea and makes you a millionaire, well... don't forget to thank the authors of this book. :)
- 12.1 Một thư viện sách có khả năng mở rộng: Xem lại ứng dụng thư viện sách mà chúng ta đã xây dựng trong Chương 10, **Universal JavaScript** cho các Ứng dụng Web, xem xét lại nó sau những gì chúng ta đã học trong chương này. Bạn có thể làm cho cách triển khai gốc của chúng ta có khả năng mở rộng hơn không? Một số ý tưởng có thể là sử dụng module `cluster` để chạy nhiều instance của máy chủ, đảm bảo bạn xử lý các thất bại bằng cách khởi động lại các worker có thể vô tình chết. Hoặc, tại sao không thử chạy toàn bộ ứng dụng trên Kubernetes?
- 12.2 Khám phá trục Z: Xuyên suốt chương này, chúng ta không cho bạn thấy bất kỳ ví dụ nào về cách shard dữ liệu trên nhiều instance, nhưng chúng ta đã khám phá tất cả các mẫu cần thiết để xây dựng một ứng dụng đạt được khả năng mở rộng dọc theo trục Z của khối lập phương mở rộng. Trong bài tập này, bạn được thử thách xây dựng một REST API cho phép bạn lấy một danh sách những người (được tạo ngẫu nhiên) có tên bắt đầu bằng một chữ cái cho trước. Bạn có thể sử dụng một thư viện như `faker` (nodejsdp.link/faker) để tạo một mẫu những người ngẫu nhiên, và sau đó bạn có thể lưu trữ dữ liệu này trong các file JSON khác nhau (hoặc các cơ sở dữ

liệu khác nhau), chia dữ liệu thành ba nhóm khác nhau. Ví dụ, bạn có thể có ba nhóm gọi là A-D, E-P, và Q-Z. Ada sẽ vào nhóm đầu tiên, Peter vào nhóm thứ hai, và Ugo vào nhóm thứ ba. Bây giờ, bạn có thể chạy một hoặc nhiều instance của một web server cho mỗi nhóm, nhưng bạn nên chỉ phơi bày một API endpoint công khai duy nhất để có thể truy xuất tất cả những người có tên bắt đầu bằng một chữ cái cho trước (ví dụ, `/api/people/byFirstName/{letter}`). Gợi ý: Bạn có thể chỉ sử dụng một bộ cân bằng tải và ánh xạ tất cả các chữ cái có thể đến backend tương ứng của các instance chịu trách nhiệm cho nhóm liên kết. Hoặc, bạn có thể tạo một lớp điều phối API mã hóa logic ánh xạ và chuyển hướng lưu lượng tương ứng. Bạn cũng có thể đưa một công cụ khám phá dịch vụ vào hỗn hợp và áp dụng cân bằng tải động, để các nhóm nhận nhiều lưu lượng hơn có thể mở rộng khi cần không?

- 12.3 Nghiệm âm nhạc: Hãy tưởng tượng bạn phải thiết kế kiến trúc của một dịch vụ như Spotify hoặc Apple Music. Bạn có thể thử thiết kế dịch vụ này như một tập hợp các microservice bằng cách áp dụng một số nguyên tắc được thảo luận trong chương này không? Điểm thưởng nếu bạn thực sự có thể triển khai một phiên bản tối thiểu của ý tưởng này với Node.js! Nếu điều này hóa ra là ý tưởng startup lớn tiếp theo và biến bạn thành triệu phú, thì... đừng quên cảm ơn các tác giả của cuốn sách này. :)

Chapter 13: Messaging and Integration Patterns — Chương 13: Các mẫu nhắn tin và tích hợp

If **scalability** is about distributing systems, integration is about connecting them. In the previous chapter, we learned how to distribute an application, fragmenting it across several processes and machines. For this to work properly, all those pieces have to communicate in some way and, hence, they have to be integrated.

Nếu khả năng mở rộng (scalability) là chuyện phân tán các hệ thống, thì tích hợp là chuyện kết nối chúng lại. Ở chương trước, chúng ta đã học cách phân tán một ứng dụng, chia nhỏ nó ra trên nhiều tiến trình và máy khác nhau. Để mọi thứ vận hành đúng đắn, tất cả các mảnh đó phải giao tiếp với nhau theo một cách nào đó, và do vậy, chúng phải được tích hợp.

There are two main techniques to integrate a distributed application: one is to use shared storage as a central coordinator and keeper of all the information, the other one is to use messages to disseminate data, events, and commands across the nodes of the system. This last option is what really makes the difference when scaling distributed systems, and it's also what makes this topic so fascinating and sometimes complex.

Có hai kỹ thuật chính để tích hợp một ứng dụng phân tán: một là dùng kho lưu trữ chia sẻ (shared storage) làm bộ điều phối trung tâm và nơi giữ toàn bộ thông tin; cách còn lại là dùng các thông điệp (message) để lan truyền dữ liệu, sự kiện và lệnh qua các node của hệ thống. Lựa chọn sau mới thực sự tạo ra khác biệt khi mở rộng các hệ thống phân tán, và đó cũng là điều khiến chủ đề này hấp dẫn và đôi khi phức tạp đến vậy.

Messages are used in every layer of a software system. We exchange messages to communicate on the Internet; we can use messages to send information to other processes using pipes; we can use messages within an application as an alternative to direct function invocation (the Command pattern), and also device drivers use messages to communicate with the hardware. Any discrete and structured data that is used as a way to exchange information between components and systems can be seen as a message. However, when dealing with distributed architectures, the term messaging system is used to describe a specific class of solutions, patterns, and architectures that are meant to facilitate the exchange of information over the network. As we will see, several traits characterize these types of systems. We might choose to use a **broker** versus a peer-to-peer structure, we might use a **request/reply** message exchange or one-way type of communication, or we might use queues to deliver our messages more reliably; the **scope** of the topic is really broad. The book Enterprise Integration Patterns by Gregor Hohpe and Bobby Woolf gives us an idea about the vastness of the topic. Historically, it is considered the Bible of messaging and integration patterns and has more than

700 pages describing 65 different integration patterns. In this final chapter, we will explore the most important of those wellknown patterns—plus some more modern alternatives—considering them from the perspective of Node.js and its ecosystem.

Thông điệp được dùng ở mọi tầng của một hệ thống phần mềm. Chúng ta trao đổi thông điệp để giao tiếp trên Internet; ta có thể dùng thông điệp để gửi thông tin tới các tiến trình khác thông qua pipe; ta có thể dùng thông điệp bên trong một ứng dụng như một giải pháp thay thế cho việc gọi hàm trực tiếp (Command pattern); và các trình điều khiển thiết bị cũng dùng thông điệp để giao tiếp với phần cứng. Bất kỳ dữ liệu rời rạc và có cấu trúc nào được dùng như một phương tiện trao đổi thông tin giữa các thành phần và hệ thống đều có thể xem là một thông điệp. Tuy nhiên, khi nói về kiến trúc phân tán, thuật ngữ messaging system (hệ thống nhắn tin) được dùng để mô tả một lớp giải pháp, mẫu hình và kiến trúc cụ thể nhằm tạo thuận lợi cho việc trao đổi thông tin qua mạng. Như chúng ta sẽ thấy, có vài đặc trưng định hình nên những loại hệ thống này. Ta có thể chọn dùng một broker thay vì cấu trúc peer-to-peer, có thể dùng kiểu trao đổi thông điệp request/reply hay kiểu giao tiếp một chiều (one-way), hoặc có thể dùng hàng đợi (queue) để chuyển giao thông điệp một cách đáng tin cậy hơn; phạm vi của chủ đề này thực sự rất rộng. Cuốn sách Enterprise Integration Patterns của Gregor Hohpe và Bobby Woolf cho ta một ý niệm về sự bao la của chủ đề này. Về mặt lịch sử, nó được coi là kinh thánh của các mẫu nhắn tin và tích hợp, dày hơn 700 trang mô tả 65 mẫu tích hợp khác nhau. Trong chương cuối này, chúng ta sẽ khám phá những mẫu quan trọng nhất trong số các mẫu nổi tiếng đó — cùng một vài lựa chọn thay thế hiện đại hơn — xem xét chúng từ góc nhìn của Node.js và hệ sinh thái của nó.

To sum up, in this chapter, we will learn about the following topics:

Tóm lại, trong chương này, chúng ta sẽ tìm hiểu về các chủ đề sau:

- The fundamentals of a messaging system
- The **Publish/Subscribe** pattern
- Task distribution patterns and pipelines
- Request/reply patterns
 - Các nguyên lý nền tảng của một messaging system
 - Mẫu Publish/Subscribe
 - Các mẫu phân phối tác vụ (task distribution) và pipeline
 - Các mẫu request/reply

Let's begin with the fundamentals.

Hãy bắt đầu từ những nền tảng cơ bản.

Fundamentals of a messaging system — Các nguyên lý nền tảng của một messaging system

When talking about messages and messaging systems, there are four fundamental elements to take into consideration:

Khi nói về thông điệp và các messaging system, có bốn yếu tố nền tảng cần cân nhắc:

- The direction of the communication, which can be one-way only or a request/reply exchange
- The purpose of the message, which also determines its content
- The timing of the message, which can be sent and received in-context (synchronously) or out-of-context (asynchronously)

- The delivery of the message, which can happen directly or via a broker
 - Hướng của giao tiếp, có thể là một chiều (one-way) hoặc trao đổi request/reply
 - Mục đích của thông điệp, yếu tố này cũng quyết định nội dung của nó
 - Thời điểm của thông điệp, có thể được gửi và nhận trong-ngữ-cảnh (đồng bộ) hoặc ngoài-ngữ-cảnh (bất đồng bộ)
 - Cách chuyển giao thông điệp, có thể diễn ra trực tiếp hoặc thông qua một broker

In the sections that follow, we are going to formalize these aspects to provide a base for our later discussions.

Trong các phần tiếp theo, chúng ta sẽ hình thức hóa những khía cạnh này để tạo nền tảng cho các thảo luận về sau.

One way versus request/reply patterns — Mẫu one-way so với request/reply

The most fundamental aspect in a messaging system is the direction of the communication, which often also determines its semantics. The simplest communication pattern is when the message is pushed one way from a source to a destination; this is a trivial situation, and it doesn't need much explanation:

Khía cạnh nền tảng nhất trong một messaging system là hướng của giao tiếp, yếu tố thường cũng quyết định ngữ nghĩa của nó. Mẫu giao tiếp đơn giản nhất là khi thông điệp được đẩy một chiều từ nguồn tới đích; đây là tình huống đơn giản và không cần giải thích nhiều:

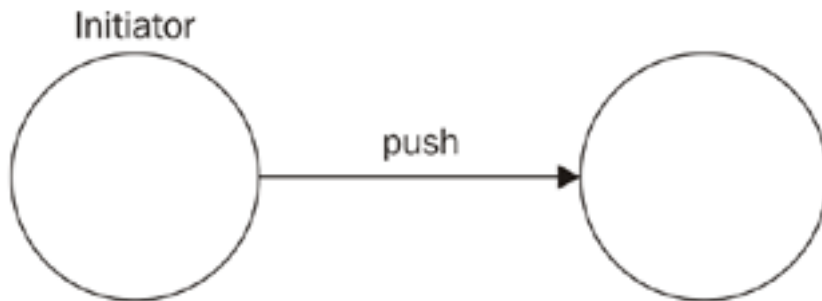


Figure 13.1: One-way communication

Hình 13.1: Giao tiếp một chiều

A typical example of one-way communication is an email or a web server that sends a message to a connected browser using WebSockets, or a system that distributes tasks to a set of workers.

Một ví dụ điển hình của giao tiếp một chiều là email, hoặc một web server gửi thông điệp tới trình duyệt đang kết nối bằng WebSockets, hoặc một hệ thống phân phối tác vụ tới một tập các worker.

On the other side, we have the Request/Reply exchange pattern, where the message in one direction is always matched (excluding error conditions) by a message in the opposite direction. A typical example of this exchange pattern is the invocation of a web service or sending a query to a database. The following diagram shows this simple and well-known scenario:

Ở phía bên kia, ta có mẫu trao đổi Request/Reply, trong đó thông điệp theo một hướng luôn được khớp (trừ các trường hợp lỗi) bởi một thông điệp theo hướng ngược lại. Ví dụ điển hình của mẫu trao đổi này là việc gọi một web service hay gửi một truy vấn tới database. Sơ đồ sau minh họa kịch bản đơn giản và quen thuộc này:

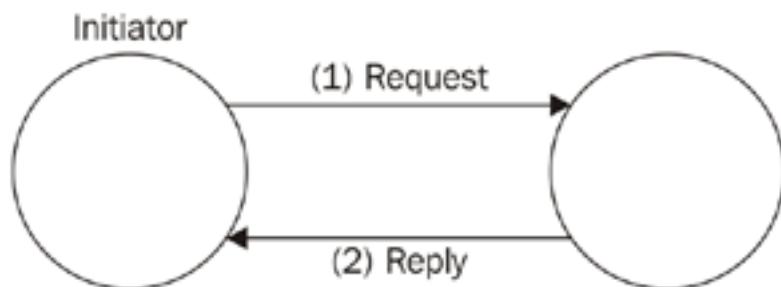


Figure 13.2: Request/Reply message exchange pattern

Hình 13.2: Mẫu trao đổi thông điệp Request/Reply

The Request/Reply pattern might seem a trivial pattern to implement, however, as we will see later, it becomes more complicated when the communication channel is asynchronous or involves multiple nodes. Take a look at the example represented in the next diagram:

Mẫu Request/Reply trông có vẻ là một mẫu tầm thường để triển khai, tuy nhiên, như ta sẽ thấy sau này, nó trở nên phức tạp hơn khi kênh giao tiếp là bất đồng bộ hoặc liên quan tới nhiều node. Hãy xem ví dụ được biểu diễn trong sơ đồ tiếp theo:

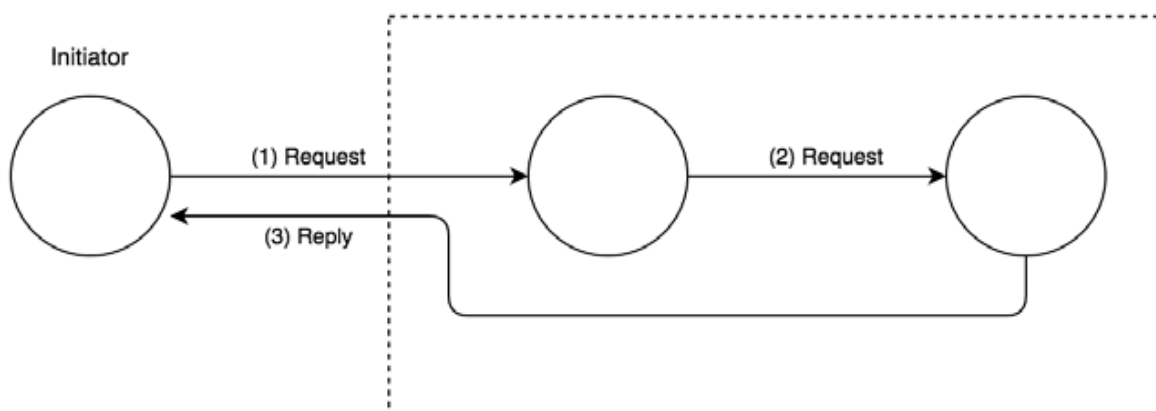


Figure 13.3: Multi-node request/reply communication With the setup shown in Figure 13.3, we can better appreciate the complexity of some request/reply patterns. If we consider the direction of the communication between any two nodes, we can surely say that it is one way. However, from a global point of view, the initiator sends a request and in turn receives an associated response, even if from a different node. In these situations, what really differentiates a Request/Reply pattern from a bare one-way loop is the relationship between the request and the reply, which is kept in the initiator. The reply is usually handled in the same context as the request.

Hình 13.3: Giao tiếp request/reply đa node. Với thiết lập như trong Hình 13.3, ta có thể đánh giá rõ hơn sự phức tạp của một số mẫu request/reply. Nếu xét hướng giao tiếp giữa hai node bất kỳ, ta chắc chắn có thể nói rằng đó là một chiều. Tuy nhiên, từ góc nhìn toàn cục, bên khởi tạo gửi một request và đến lượt mình nhận được một response liên quan, dù là từ một node khác. Trong những tình huống này, điều thực sự phân biệt mẫu Request/Reply với một vòng lặp một chiều thuần túy chính là mối quan hệ giữa request và reply, được giữ lại ở bên khởi tạo. Reply thường được xử lý trong cùng ngữ cảnh với request.

Message types — Các loại thông điệp

A message is essentially a means to connect different software components and there are different reasons for doing so: it might be because we want to obtain some information held by another system or component, to execute operations remotely, or to notify some peers that something has just happened.

Một thông điệp về cơ bản là phương tiện để kết nối các thành phần phần mềm khác nhau, và có nhiều lý do để làm điều này: có thể là vì ta muốn lấy thông tin do một hệ thống hoặc thành phần khác đang giữ, để thực thi các thao tác từ xa, hoặc để thông báo cho một số peer rằng điều gì đó vừa xảy ra.

The message content will also vary depending on the reason for the communication. In general, we can identify three types of messages, depending on their purpose:

Nội dung của thông điệp cũng sẽ thay đổi tùy theo lý do giao tiếp. Nhìn chung, ta có thể xác định ba loại thông điệp, tùy theo mục đích của chúng:

- Command Messages
- Event Messages
- Document Messages
 - Command Message
 - Event Message
 - Document Message

Command Messages — Command Message

You should already be familiar with the Command Message as it's essentially a serialized Command object (we learned about this in the Command section in Chapter 9, Behavioral Design Patterns)

Bạn hẳn đã quen thuộc với Command Message, vì về cơ bản nó là một Command object đã được tuần tự hóa (serialize) (chúng ta đã học về điều này trong phần Command ở Chương 9, Behavioral Design Patterns).

The purpose of this type of message is to trigger the execution of an action or a task on the receiver. For this to be possible, the Command Message has to contain the essential information to run the task, which usually includes the name of the operation and a list of arguments. The Command Message can be used to implement remote procedure call (RPC) systems, distributed computations, or can be more simply used to request some data. RESTful HTTP calls are simple examples of commands; each HTTP verb has a specific meaning and is associated with a precise operation: GET, to retrieve the resource; POST, to create a new one; PUT/PATCH, to update it; and DELETE, to destroy it.

Mục đích của loại thông điệp này là kích hoạt việc thực thi một hành động hay một tác vụ trên bên nhận. Để làm được điều này, Command Message phải chứa thông tin thiết yếu để chạy tác vụ, thường bao gồm tên của thao tác và một danh sách đối số. Command Message có thể được dùng để triển khai các hệ thống remote procedure call (RPC), tính toán phân tán, hoặc đơn giản hơn là được dùng để yêu cầu một số dữ liệu. Các lời gọi RESTful HTTP là các ví dụ đơn giản về command; mỗi HTTP verb có một ý nghĩa cụ thể và gắn với một thao tác chính xác: GET để lấy resource; POST để tạo mới; PUT/PATCH để cập nhật; và DELETE để hủy nó.

Event Messages — Event Message

An Event Message is used to notify another component that something has occurred. It usually contains the type of the event and sometimes also some details such as the context, the subject, or the actor involved.

Một Event Message được dùng để thông báo cho một thành phần khác rằng điều gì đó đã xảy ra. Nó thường chứa loại của sự kiện và đôi khi cả một số chi tiết như ngữ cảnh, chủ thể (subject), hoặc đối tượng (actor) liên quan.

In web development, we are using an Event Message when, for example, we leverage WebSockets to send notifications from the server to the client to communicate changes to some data or mutations in the **state** of the system.

Trong phát triển web, ta đang dùng Event Message khi, ví dụ, ta tận dụng WebSockets để gửi thông báo từ server tới client nhằm truyền đạt các thay đổi đối với một số dữ liệu hoặc các biến đổi trong trạng thái của hệ thống.

Events are a very important integration mechanism in distributed applications, as they enable us to keep all the nodes of the system on the same page.

Sự kiện là một cơ chế tích hợp rất quan trọng trong các ứng dụng phân tán, vì chúng cho phép ta giữ cho tất cả các node của hệ thống ở cùng một trang (cùng nắm thông tin như nhau).

Document Messages — Document Message

The Document Message is primarily meant to transfer data between components and machines. A typical example is a message used to transfer the results of a database query.

Document Message chủ yếu nhằm chuyển dữ liệu giữa các thành phần và máy móc. Một ví dụ điển hình là thông điệp dùng để chuyển kết quả của một truy vấn database.

The main characteristic that differentiates a Document Message from a Command Message (which might also contain data) is that the message does not contain any information that tells the receiver what to do with the data. On the other hand, the main difference between a Document Message and an Event Message is the absence of an association with a particular occurrence with something that happened. Often, the replies to Command Messages are Document Messages, as they usually contain only the data that was requested or the result of an operation.

Đặc điểm chính phân biệt Document Message với Command Message (vốn cũng có thể chứa dữ liệu) là thông điệp không chứa bất kỳ thông tin nào cho bên nhận biết phải làm gì với dữ liệu. Mặt khác, khác biệt chính giữa Document Message và Event Message là sự vắng mặt của mối liên hệ với một sự việc cụ thể nào đó đã xảy ra. Thông thường, các reply cho Command Message chính là Document Message, vì chúng thường chỉ chứa dữ liệu được yêu cầu hoặc kết quả của một thao tác.

Now that we know how to categorize the semantics of a message, let's learn about the semantic of the communication channel used to move our messages around.

Giờ khi đã biết cách phân loại ngữ nghĩa của một thông điệp, hãy tìm hiểu về ngữ nghĩa của kênh giao tiếp dùng để di chuyển thông điệp của chúng ta đi quanh hệ thống.

Asynchronous messaging, queues, and streams — Nhắn tin bất đồng bộ, hàng đợi và stream

At this point in the book, you should already be familiar with the characteristics of an asynchronous operation. Well, it turns out that the same principles can be applied to messaging and communications.

Đến thời điểm này trong cuốn sách, bạn hẳn đã quen thuộc với các đặc tính của một thao tác bất đồng bộ. Hóa ra cùng những nguyên lý đó có thể áp dụng cho việc nhắn tin và giao tiếp.

We can compare synchronous communications to a phone call: the two peers must be connected to the same channel at the same time and they should exchange messages in real time. Normally, if we want to call someone else, we either need another phone or terminate the ongoing communication to start a new one. Asynchronous communication is similar to an SMS: it doesn't require the recipient to be connected to the network the moment we send it; we might receive a response immediately or after an unknown delay, or we might not receive a response at all. We might send multiple SMSes to multiple recipients one after the other and receive their responses (if any) in any order. In short, we have better parallelism with the use of fewer resources.

Ta có thể so sánh giao tiếp đồng bộ với một cuộc gọi điện thoại: hai bên phải kết nối vào cùng một kênh tại cùng một thời điểm và họ phải trao đổi thông điệp theo thời gian thực. Thông thường, nếu ta muốn gọi cho ai khác, ta hoặc cần một điện thoại khác, hoặc phải kết thúc cuộc gọi đang diễn ra để bắt đầu cuộc mới. Giao tiếp bất đồng bộ thì giống như tin nhắn SMS: nó không đòi hỏi người nhận phải kết nối vào mạng đúng vào lúc ta gửi; ta có thể nhận phản hồi ngay lập tức hoặc sau một độ trễ không xác định, hoặc cũng có thể không nhận phản hồi nào cả. Ta có thể gửi nhiều SMS tới nhiều người nhận lần lượt và nhận phản hồi của họ (nếu có) theo bất kỳ thứ tự nào. Tóm lại, ta có được tính song song tốt hơn với việc dùng ít tài nguyên hơn.

Another important characteristic of asynchronous communications is that the messages can be stored and then delivered as soon as possible or at a later time. This can be useful when the receiver is too busy to handle new messages or when we want to guarantee delivery. In messaging systems, this is made possible using a **message queue**, a component that mediates the communication between the producer of the messages and the consumer, storing any message before it gets delivered to its destination, as shown in the following diagram:

Một đặc tính quan trọng khác của giao tiếp bất đồng bộ là các thông điệp có thể được lưu trữ rồi chuyển giao ngay khi có thể hoặc vào lúc muộn hơn. Điều này hữu ích khi bên nhận quá bận để xử lý thông điệp mới, hoặc khi ta muốn đảm bảo việc chuyển giao. Trong các messaging system, điều này được thực hiện nhờ một message queue, một thành phần làm trung gian giao tiếp giữa bên sản xuất (producer) thông điệp và bên tiêu thụ (consumer), lưu trữ mọi thông điệp trước khi nó được chuyển tới đích, như thể hiện trong sơ đồ sau:

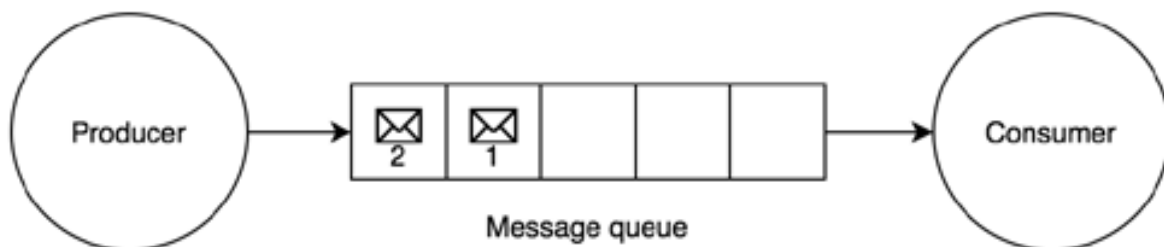


Figure 13.4: A message queue

Hình 13.4: Một message queue

If for any reason the consumer crashes, disconnects from the network, or experiences a slowdown, the messages are accumulated in the queue and dispatched as soon as the consumer comes back online. The queue can be located in the producer, or be split between the producer and the consumer (in peer-to-peer architectures), or live in a dedicated external system acting as **middleware** for the communication (broker).

Nếu vì bất kỳ lý do gì mà consumer bị crash, ngắt kết nối khỏi mạng, hoặc bị chậm đi, các thông điệp sẽ được tích lũy trong queue và phân phát ngay khi consumer trở lại trực tuyến. Queue có thể nằm tại producer, hoặc được tách ra giữa producer và consumer (trong các kiến trúc peer-to-peer), hoặc nằm trong một hệ thống ngoài chuyên dụng đóng vai trò middleware cho việc giao tiếp (broker).

Another data structure that has a similar (but not the same!) goal as a message queue is the log. A log is an append-only data structure, which is durable and whose messages can be read as they arrive or by accessing its history. In the context of messaging and integration systems, this is also known as a data **stream**.

Một cấu trúc dữ liệu khác có mục tiêu tương tự (nhưng không hề giống nhau!) với message queue là log. Một log là cấu trúc dữ liệu chỉ-thêm (append-only), bền vững (durable), mà các thông điệp của nó có thể được đọc khi chúng đến hoặc bằng cách truy cập lịch sử của nó. Trong ngữ cảnh các hệ thống nhắn tin và tích hợp, cấu trúc này cũng được gọi là một data stream.

Compared to a queue, in a stream, messages are not removed when they are retrieved or processed. This way, consumers can retrieve the messages as they arrive or can query the stream at any time to retrieve past messages. This means that a stream provides more freedom when it comes to accessing the messages, while queues usually expose only one message at a time to their consumers. Most importantly, a stream can be shared by more than one consumer, which can access the messages (even the same messages) using different approaches. Figure 13.5 gives you an idea of the structure of a stream compared to that of a message queue:

So với queue, trong một stream, các thông điệp không bị xóa khi chúng được truy xuất hay xử lý. Theo cách này, các consumer có thể truy xuất thông điệp khi chúng đến, hoặc có thể truy vấn stream vào bất kỳ lúc nào để lấy lại các thông điệp trong quá khứ. Điều này nghĩa là một stream cung cấp nhiều tự do hơn khi truy cập thông điệp, trong khi queue thường chỉ phơi bày một thông điệp tại một thời điểm cho các consumer của nó. Quan trọng nhất, một stream có thể được chia sẻ bởi nhiều hơn một consumer, mỗi consumer có thể truy cập các thông điệp (kể cả cùng những thông điệp đó) theo các cách tiếp cận khác nhau. Hình 13.5 cho bạn một ý niệm về cấu trúc của một stream so với một message queue:

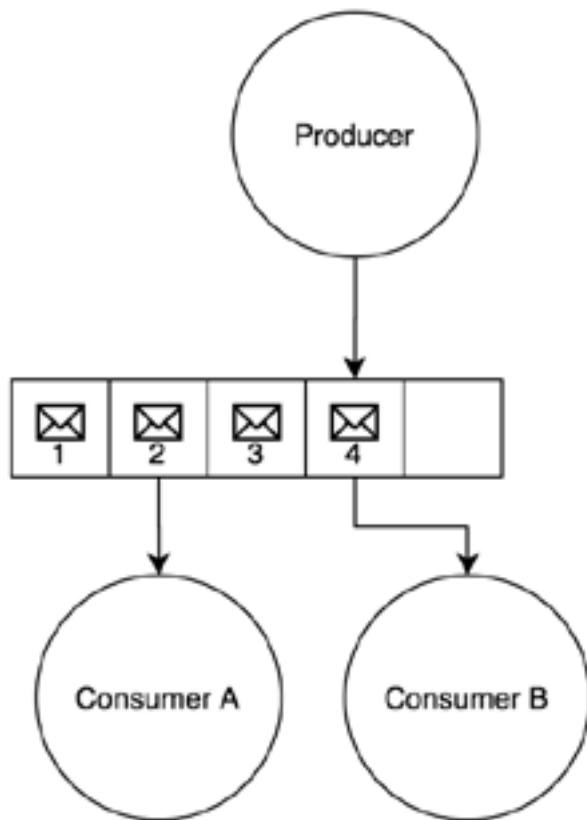


Figure 13.5: A stream

Hình 13.5: Một stream

You will be able to better appreciate the difference between a queue and a stream later in the chapter when we implement a sample application using both approaches.

Bạn sẽ có thể đánh giá rõ hơn sự khác biệt giữa một queue và một stream về sau trong chương này, khi chúng ta triển khai một ứng dụng mẫu dùng cả hai cách tiếp cận.

The final fundamental element to consider in a messaging system is the way the nodes of the system are connected together, which can be directly or through an intermediary.

Yếu tố nền tảng cuối cùng cần xem xét trong một messaging system là cách các node của hệ thống được kết nối với nhau, có thể là trực tiếp hoặc thông qua một bên trung gian.

Peer-to-peer or broker-based messaging — Nhắn tin peer-to-peer hay dựa trên broker

Messages can be delivered directly to the receiver in a peer-to-peer fashion, or through a centralized intermediary system called a message broker. The main role of the broker is to decouple the receiver of the message from the sender. The following diagram shows the architectural difference between the two approaches:

Thông điệp có thể được chuyển giao trực tiếp tới bên nhận theo kiểu peer-to-peer, hoặc thông qua một hệ thống trung gian tập trung gọi là message broker. Vai trò chính của

broker là tách rời (decouple) bên nhận thông điệp khỏi bên gửi. Sơ đồ sau cho thấy sự khác biệt về kiến trúc giữa hai cách tiếp cận:

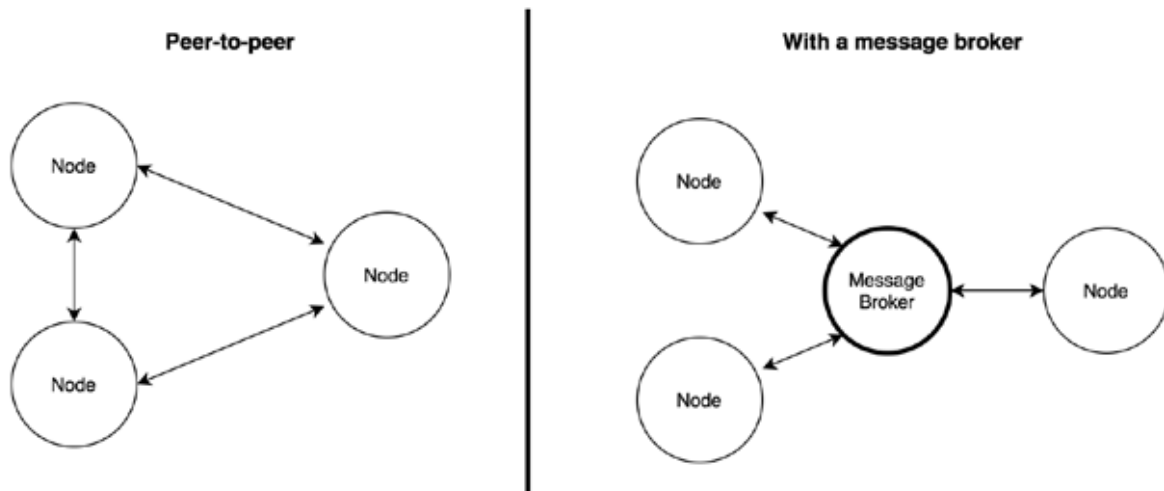


Figure 13.6: Peer-to-peer communication versus message brokering In a peer-to-peer architecture, every node is directly responsible for the delivery of the message to the receiver. This implies that the nodes have to know the address and port of the receiver and they have to agree on a protocol and message format. The broker eliminates these complexities from the equation: each node can be totally independent and can communicate with an unspecified number of peers without directly knowing their details.

Hình 13.6: Giao tiếp peer-to-peer so với message brokering. Trong kiến trúc peer-to-peer, mỗi node chịu trách nhiệm trực tiếp cho việc chuyển giao thông điệp tới bên nhận. Điều này hàm ý rằng các node phải biết địa chỉ và cổng của bên nhận, và chúng phải thống nhất về một giao thức và định dạng thông điệp. Broker loại bỏ những phức tạp này khỏi phương trình: mỗi node có thể hoàn toàn độc lập và có thể giao tiếp với một số lượng peer không xác định mà không cần biết trực tiếp chi tiết của chúng.

A broker can also act as a bridge between different communication protocols. For example, the popular RabbitMQ broker (nodejsdp.link/rabbitmq) supports Advanced Message Queuing Protocol (AMQP), Message Queue Telemetry Transport (MQTT), and Simple/Streaming Text Orientated Messaging Protocol (STOMP), enabling multiple applications supporting different messaging protocols to interact.

Một broker cũng có thể đóng vai trò cầu nối giữa các giao thức giao tiếp khác nhau. Ví dụ, broker phổ biến RabbitMQ (nodejsdp.link/rabbitmq) hỗ trợ Advanced Message Queuing Protocol (AMQP), Message Queue Telemetry Transport (MQTT) và Simple/Streaming Text Orientated Messaging Protocol (STOMP), cho phép nhiều ứng dụng hỗ trợ các giao thức nhắn tin khác nhau tương tác với nhau.

MQTT (nodejsdp.link/mqtt) is a lightweight messaging protocol, specifically designed for machine-to-machine communications (such as the Internet of things). AMQP (nodejsdp.link/amqp) is a more complex messaging protocol, designed to be an open source alternative to proprietary messaging middleware. STOMP (nodejsdp.link/stomp) is a lightweight text-based protocol, which comes from “the HTTP school of design”. All three are application layer protocols and are based on TCP/IP.

MQTT (nodejsdp.link/mqtt) là một giao thức nhắn tin nhẹ, được thiết kế riêng cho giao tiếp máy-với-máy (machine-to-machine, chẳng hạn như Internet of Things). AMQP

(`nodejsdp.link/amqp`) là một giao thức nhắn tin phức tạp hơn, được thiết kế để trở thành một giải pháp mã nguồn mở thay thế cho các messaging middleware độc quyền. STOMP (`nodejsdp.link/stomp`) là một giao thức nhẹ dựa trên văn bản, xuất thân từ “trường phái thiết kế HTTP”. Cả ba đều là giao thức tầng ứng dụng và dựa trên TCP/IP.

Besides the advantages in terms of decoupling and interoperability, a broker can offer additional features such as persistent queues, routing, message transformations, and monitoring, without mentioning the broad range of messaging patterns that many brokers support out of the box.

Bên cạnh các lợi thế về mặt tách rời (decoupling) và khả năng tương tác (interoperability), một broker có thể cung cấp thêm các tính năng như queue bền vững, định tuyến (routing), biến đổi thông điệp và giám sát, chưa kể đến hàng loạt mẫu nhắn tin mà nhiều broker hỗ trợ sẵn.

Of course, nothing prevents us from implementing all these features using a peer-to-peer architecture, but unfortunately, there is much more effort involved. Nonetheless, there might be different reasons for choosing a peer-to-peer approach instead of a broker:

Tất nhiên, không có gì ngăn cản ta triển khai tất cả những tính năng này bằng kiến trúc peer-to-peer, nhưng tiếc thay, công sức bỏ ra sẽ nhiều hơn rất nhiều. Tuy vậy, vẫn có thể có những lý do khác nhau để chọn cách tiếp cận peer-to-peer thay vì broker:

- By removing the broker, we are removing a single point of failure from the system
- A broker has to be scaled, while in a peer-to-peer architecture we only need to scale the single nodes of the application
- Exchanging messages without intermediaries can greatly reduce the latency of the communication
 - Bằng cách loại bỏ broker, ta loại bỏ một điểm hỏng đơn (single point of failure) khỏi hệ thống
 - Một broker phải được mở rộng quy mô, trong khi với kiến trúc peer-to-peer ta chỉ cần mở rộng từng node riêng lẻ của ứng dụng
 - Trao đổi thông điệp mà không qua trung gian có thể giảm đáng kể độ trễ (latency) của giao tiếp

By using a peer-to-peer messaging system we can have much more flexibility and power because we are not bound to any particular technology, protocol, or architecture. Now that we know the basics of a messaging system, let's explore some of the most important messaging patterns. Let's start with the Publish/Subscribe pattern.

Bằng cách dùng một messaging system peer-to-peer, ta có thể có nhiều tính linh hoạt và sức mạnh hơn vì ta không bị ràng buộc với bất kỳ công nghệ, giao thức hay kiến trúc cụ thể nào. Giờ khi đã biết những điều cơ bản về một messaging system, hãy khám phá một số mẫu nhắn tin quan trọng nhất. Hãy bắt đầu với mẫu Publish/Subscribe.

Publish/Subscribe pattern — Mẫu Publish/Subscribe

Publish/Subscribe (often abbreviated to Pub/Sub) is probably the best-known one-way messaging pattern. We should already be familiar with it, as it's nothing more than a distributed **Observer** pattern. As in the case of Observer, we have a set of subscribers registering their interest in receiving a specific category of messages. On the other side, the publisher produces messages that are distributed across all the relevant subscribers. Figure 13.7 shows the two main variants of the Pub/Sub pattern; the first is based on a peer-to-peer architecture, and the second uses a broker to mediate the communication:

Publish/Subscribe (thường viết tắt là Pub/Sub) có lẽ là mẫu nhắn tin một chiều nổi tiếng nhất. Ta hẳn đã quen thuộc với nó, vì nó chẳng qua chỉ là một Observer pattern được phân tán. Như trong trường hợp của Observer, ta có một tập các subscriber đăng ký quan tâm tới việc nhận một loại thông điệp cụ thể. Ở phía bên kia, publisher sản xuất các thông điệp được phân phối tới tất cả các subscriber liên quan. Hình 13.7 cho thấy hai biến thể chính của mẫu Pub/Sub; biến thể thứ nhất dựa trên kiến trúc peer-to-peer, còn biến thể thứ hai dùng một broker để làm trung gian giao tiếp:

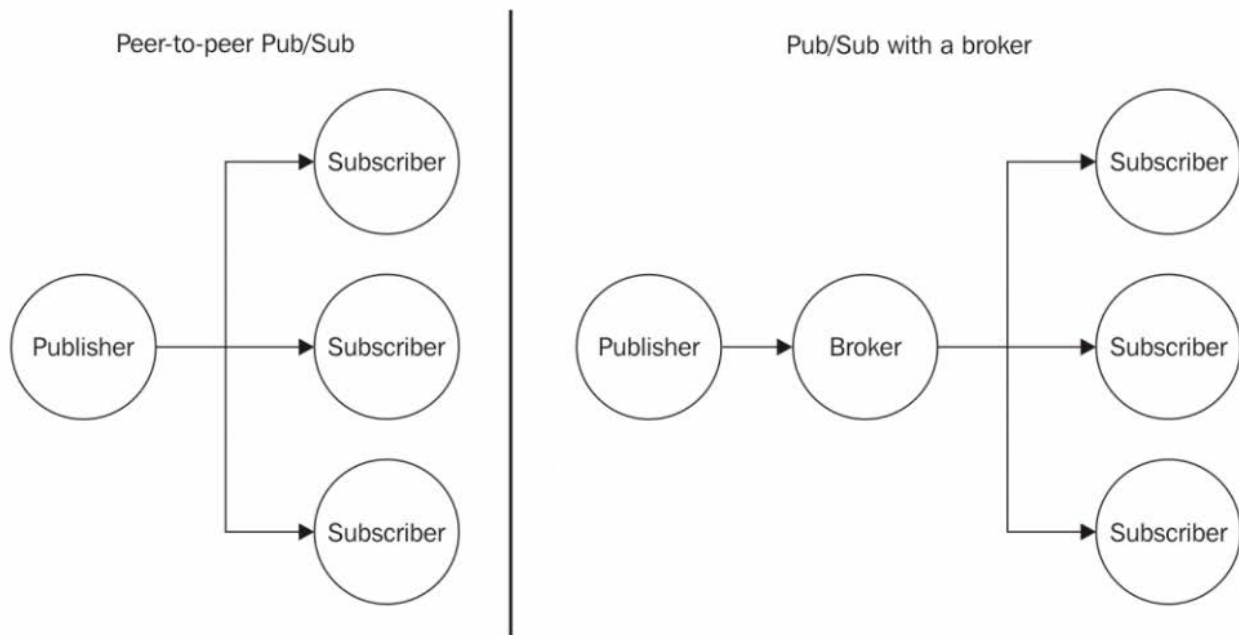


Figure 13.7: Publish/Subscribe messaging pattern

Hình 13.7: Mẫu nhắn tin Publish/Subscribe

What makes Pub/Sub so special is the fact that the publisher doesn't know in advance who the recipients of the messages are. As we said, it's the subscriber that has to register its interest to receive a particular message, allowing the publisher to work with an unspecified number of receivers. In other words, the two sides of the Pub/Sub pattern are loosely coupled, which makes this an ideal pattern to integrate the nodes of an evolving distributed system.

Điều khiến Pub/Sub đặc biệt là việc publisher không hề biết trước ai là người nhận của các thông điệp. Như ta đã nói, chính subscriber phải đăng ký mối quan tâm của mình để nhận một thông điệp cụ thể, cho phép publisher làm việc với một số lượng người nhận không xác định. Nói cách khác, hai phía của mẫu Pub/Sub được liên kết lỏng (loosely coupled), khiến đây trở thành một mẫu lý tưởng để tích hợp các node của một hệ thống phân tán đang tiến hóa.

The presence of a broker further improves the decoupling between the nodes of the system because the subscribers interact only with the broker, not knowing which node is the publisher of a message. As we will see later, a broker can also provide a message queuing system, allowing reliable delivery even in the presence of connectivity problems between the nodes. Now, let's work on an example to demonstrate this pattern.

Sự hiện diện của một broker còn cải thiện hơn nữa sự tách rời giữa các node của hệ thống, vì các subscriber chỉ tương tác với broker, không cần biết node nào là publisher của một thông điệp. Như ta sẽ thấy sau này, một broker cũng có thể cung cấp một hệ thống message

queuing, cho phép chuyển giao đáng tin cậy ngay cả khi có vấn đề kết nối giữa các node. Bây giờ, hãy cùng làm một ví dụ để minh họa mẫu này.

Building a minimalist real-time chat application — Xây dựng một ứng dụng chat thời gian thực tối giản

To show a real-life example of how the Pub/Sub pattern can help us integrate a distributed architecture, we are now going to build a very basic real-time chat application using pure WebSockets. Then, we will scale it by running multiple instances, and finally, using a messaging system, we will build a communication channel between all the server instances.

Để cho thấy một ví dụ thực tế về cách mẫu Pub/Sub có thể giúp ta tích hợp một kiến trúc phân tán, bây giờ chúng ta sẽ xây dựng một ứng dụng chat thời gian thực rất cơ bản chỉ dùng WebSockets thuần. Sau đó, ta sẽ mở rộng nó bằng cách chạy nhiều instance, và cuối cùng, bằng cách dùng một messaging system, ta sẽ xây dựng một kênh giao tiếp giữa tất cả các instance của server.

Implementing the server side — Triển khai phía server

Now, let's take one step at a time. Let's first build a basic chat application, then we'll scale it to multiple instances.

Bây giờ, hãy đi từng bước một. Trước tiên hãy xây dựng một ứng dụng chat cơ bản, rồi ta sẽ mở rộng nó ra nhiều instance.

To implement the real-time capabilities of a typical chat application, we will rely on the **ws package** (nodejsdp.link/ws), which is a pure WebSocket implementation for Node.js. Implementing real-time applications in Node.js is pretty simple, and the code we are going to write will confirm this assumption. So, let's create the server side of our chat application in a file called `index.js`:

Để triển khai khả năng thời gian thực của một ứng dụng chat điển hình, ta sẽ dựa vào gói **ws** (nodejsdp.link/ws), một bản triển khai WebSocket thuần cho Node.js. Triển khai các ứng dụng thời gian thực trong Node.js khá đơn giản, và đoạn code mà ta sắp viết sẽ xác nhận giả định này. Vậy, hãy tạo phía server của ứng dụng chat trong một file tên là `index.js`:

```
import { createServer } from 'http'
import staticHandler from 'serve-handler'
import ws from 'ws'
// serve static files
const server = createServer((req, res) => { // (1)
  return staticHandler(req, res, { public: 'www' })
})
const wss = new ws.Server({ server }) // (2)
wss.on('connection', client => {
  console.log('Client connected')
  client.on('message', msg => { // (3)
    console.log(`Message: ${msg}`)
    broadcast(msg)
  })
})
function broadcast (msg) { // (4)
  for (const client of wss.clients) {
```

```

    if (client.readyState === ws.OPEN) {
        client.send(msg)
    }
}
server.listen(process.argv[2] || 8080)

```

That's it! That's all we need to implement the server-side component of our chat application. This is how it works:

Vậy đó! Đó là tất cả những gì ta cần để triển khai thành phần phía server của ứng dụng chat. Đây là cách nó hoạt động:

1. We first create an HTTP server and forward every request to a special handler (`nodejsdp.link/serve-handler`), which will take care to serve all the static files from the `www` directory. This is needed to access the client-side resources of our application (for example, HTML, JavaScript, and CSS files).
2. We then create a new instance of the WebSocket server, and we attach it to our existing HTTP server. Next, we start listening for incoming WebSocket client connections by attaching an event listener for the `connection` event.
3. Each time a new client connects to our server, we start listening for incoming messages. When a new message arrives, we broadcast it to all the connected clients.
4. The `broadcast()` function is a simple iteration over all the known clients, where the `send()` function is invoked on each connected client.
 1. Trước tiên ta tạo một HTTP server và chuyển tiếp mọi request tới một handler đặc biệt (`nodejsdp.link/serve-handler`), handler này sẽ lo việc phục vụ tất cả các file tĩnh từ thư mục `www`. Điều này cần thiết để truy cập các tài nguyên phía client của ứng dụng (ví dụ, các file HTML, JavaScript và CSS).
 2. Sau đó ta tạo một instance mới của WebSocket server, và gắn nó vào HTTP server hiện có. Tiếp theo, ta bắt đầu lắng nghe các kết nối WebSocket client tới bằng cách gắn một event listener cho sự kiện `connection`.
 3. Mỗi lần một client mới kết nối tới server, ta bắt đầu lắng nghe các thông điệp đến. Khi một thông điệp mới đến, ta broadcast nó tới tất cả các client đang kết nối.
 4. Hàm `broadcast()` đơn giản là một vòng lặp qua tất cả các client đã biết, trong đó hàm `send()` được gọi trên mỗi client đang kết nối.

This is the magic of Node.js! Of course, the server that we just implemented is very minimal and basic, but as we will see, it does its job.

Đây chính là điều kỳ diệu của Node.js! Tất nhiên, server mà ta vừa triển khai rất tối giản và cơ bản, nhưng như ta sẽ thấy, nó làm tròn nhiệm vụ.

Implementing the client side — Triển khai phía client

Next, it's time to implement the client side of our chat application. This can be done with another compact and simple fragment of code, essentially a minimal HTML page with some basic JavaScript code. Let's create this page in a file named `www/index.html` as follows:

Tiếp theo, đến lúc triển khai phía client của ứng dụng chat. Việc này có thể làm được với một đoạn code nhỏ gọn và đơn giản khác, về cơ bản là một trang HTML tối giản với một ít code JavaScript cơ bản. Hãy tạo trang này trong một file tên là `www/index.html` như sau:

```

<!DOCTYPE html>
<html>

```

```

<body>
  Messages:
  <div id="messages"></div>
  <form id="msgForm">
    <input type="text" placeholder="Send a message" id="msgBox"/>
    <input type="submit" value="Send"/>
  </form>
  <script>
    const ws = new WebSocket(
      `ws://${window.document.location.host}`
    )
    ws.onmessage = function (message) {
      const msgDiv = document.createElement('div')
      msgDiv.innerHTML = message.data
      document.getElementById('messages').appendChild(msgDiv)
    }
    const form = document.getElementById('msgForm')
    form.addEventListener('submit', (event) => {
      event.preventDefault()
      const message = document.getElementById('msgBox').value
      ws.send(message)
      document.getElementById('msgBox').value = ''
    })
  </script>
</body>
</html>

```

The HTML page we just created doesn't really need many comments, it's just a piece of straightforward web development. We use the native WebSocket object to initialize a connection to our Node.js server, and then start listening for messages from the server, displaying them in new div elements as they arrive. For sending messages, instead, we use a simple textbox and a button within a form.

Trang HTML mà ta vừa tạo thực ra không cần nhiều chú thích, nó chỉ là một mẫu phát triển web đơn giản. Ta dùng đối tượng WebSocket gốc (native) để khởi tạo một kết nối tới Node.js server, rồi bắt đầu lắng nghe các thông điệp từ server, hiển thị chúng trong các phần tử div mới khi chúng đến. Còn để gửi thông điệp, ta dùng một hộp văn bản đơn giản và một nút bấm trong một form.

Please note that when stopping or restarting the chat server, the WebSocket connection is closed and the client will not try to reconnect automatically (as we might expect from a productiongrade application). This means that it is necessary to refresh the browser after a server restart to reestablish the connection (or implement a reconnection mechanism, which we will not cover here for brevity). Also, in this initial version of our app, the clients will not receive any message sent while they were not connected to the server.

Xin lưu ý rằng khi dừng hoặc khởi động lại chat server, kết nối WebSocket sẽ bị đóng và client sẽ không tự động thử kết nối lại (như ta có thể mong đợi ở một ứng dụng cấp độ production). Điều này nghĩa là cần phải refresh trình duyệt sau khi khởi động lại server để thiết lập lại kết nối (hoặc triển khai một cơ chế kết nối lại, điều mà ta sẽ không đề cập ở đây cho ngắn gọn). Ngoài ra, trong phiên bản đầu tiên này của ứng dụng, các client sẽ không nhận được bất kỳ thông điệp nào được gửi khi chúng chưa kết nối tới server.

Running and scaling the chat application — Chạy và mở rộng ứng dụng chat

We can try to run our application immediately. Just launch the server with the following command:

Ta có thể thử chạy ứng dụng ngay lập tức. Chỉ cần khởi chạy server với lệnh sau:

```
node index.js 8080
```

Then, open a couple of browser tabs or even two different browsers, point them at `http://localhost:8080`, and start chatting:

Sau đó, mở vài tab trình duyệt hoặc thậm chí hai trình duyệt khác nhau, trỏ chúng tới `http://localhost:8080`, và bắt đầu chat:

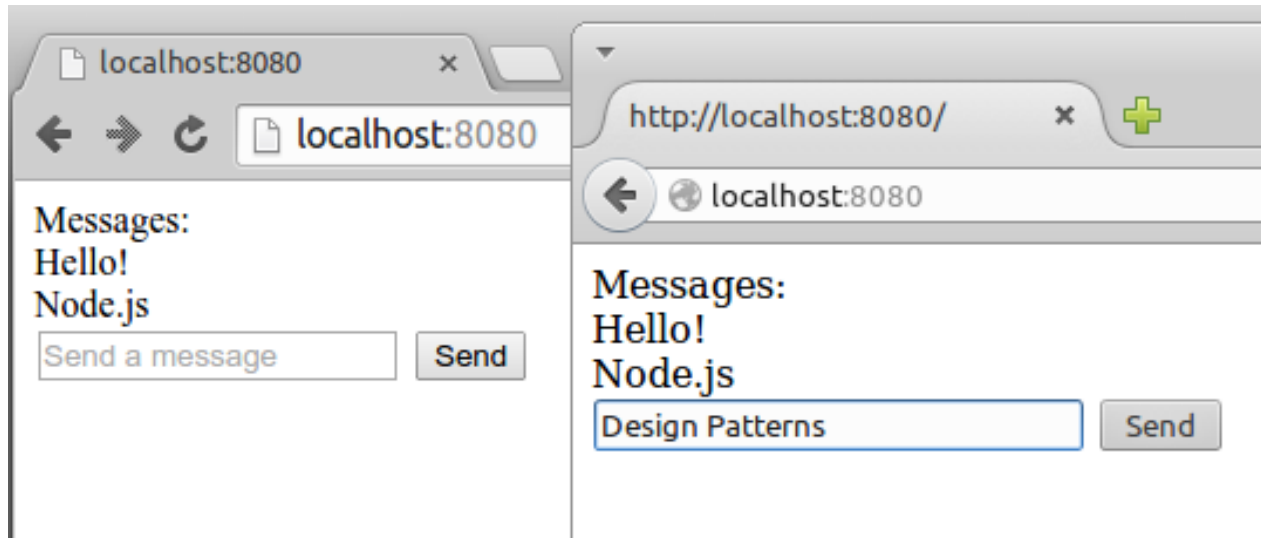


Figure 13.8: Our new chat application in action

Hình 13.8: Ứng dụng chat mới của chúng ta đang hoạt động

Now, we want to see what happens when we try to scale our application by launching multiple instances. Let's try to do that. Let's start another server on another port:

Bây giờ, ta muốn xem điều gì xảy ra khi ta thử mở rộng ứng dụng bằng cách khởi chạy nhiều instance. Hãy thử làm điều đó. Hãy khởi chạy một server khác trên một cổng khác:

```
node index.js 8081
```

The desired outcome should be that two different clients, connected to two different servers, should be able to exchange chat messages. Unfortunately, this is not what happens with our current implementation. We can test this by opening another browser tab to `http://localhost:8081`.

Kết quả mong muốn là hai client khác nhau, kết nối tới hai server khác nhau, phải có thể trao đổi thông điệp chat với nhau. Đáng tiếc, đây không phải là điều xảy ra với bản triển khai hiện tại của chúng ta. Ta có thể kiểm chứng điều này bằng cách mở một tab trình duyệt khác tới `http://localhost:8081`.

In a real-world application, we would use a load balancer to distribute the load across our instances, but for this demo we will not use one. This allows us to access each server instance in a deterministic way to verify how it interacts with the other instances. When sending a chat message on one instance, we only broadcast the message locally, distributing it only to the clients connected to that particular server. In practice, the two servers are not talking to each other. We need to integrate them, and that's exactly what we are going to see next.

Trong một ứng dụng thực tế, ta sẽ dùng một load balancer để phân phối tải qua các instance, nhưng cho bản demo này ta sẽ không dùng. Điều này cho phép ta truy cập từng instance của server theo cách xác định để kiểm chứng xem nó tương tác với các instance khác ra sao. Khi gửi một thông điệp chat trên một instance, ta chỉ broadcast thông điệp đó cục bộ, phân phối nó chỉ tới các client kết nối tới server cụ thể đó. Trên thực tế, hai server không nói chuyện với nhau. Ta cần tích hợp chúng, và đó chính xác là điều ta sắp xem tiếp theo.

Using Redis as a simple message broker — Dùng Redis làm một message broker đơn giản

We start our analysis of the most common Pub/Sub implementations by introducing Redis (nodejsdp.link/redis), which is a very fast and flexible in-memory data structure store. Redis is often used as a database or a cache server, however, among its many features there is a pair of commands specifically designed to implement a centralized Pub/Sub message exchange pattern.

Ta bắt đầu phân tích các bản triển khai Pub/Sub phổ biến nhất bằng cách giới thiệu Redis (nodejsdp.link/redis), một kho lưu trữ cấu trúc dữ liệu trong bộ nhớ (in-memory) rất nhanh và linh hoạt. Redis thường được dùng làm database hoặc cache server, tuy nhiên, trong số nhiều tính năng của nó có một cặp lệnh được thiết kế riêng để triển khai một mẫu trao đổi thông điệp Pub/Sub tập trung.

Redis' message brokering capabilities are (intentionally) very simple and basic, especially if we compare them to those of more advanced message-oriented middleware. However, this is one of the main reasons for its popularity. Often, Redis is already available in an existing infrastructure, for example, used as a cache server or as a session data store. Its speed and flexibility make it a very popular choice for sharing data in a distributed system. So, as soon as the need for a publish/subscribe broker arises in a project, the most simple and immediate choice is to reuse Redis itself, avoiding the need to install and maintain a dedicated message broker.

Khả năng message brokering của Redis (cố ý) rất đơn giản và cơ bản, đặc biệt khi so với những middleware hướng-thông-điệp tiên tiến hơn. Tuy nhiên, đây lại là một trong những lý do chính tạo nên sự phổ biến của nó. Thường thì Redis đã có sẵn trong một hạ tầng hiện hữu, ví dụ, được dùng làm cache server hoặc làm kho lưu trữ liệu session. Tốc độ và sự linh hoạt của nó khiến nó trở thành một lựa chọn rất phổ biến để chia sẻ dữ liệu trong một hệ thống phân tán. Vì vậy, ngay khi nhu cầu về một broker publish/subscribe nảy sinh trong một dự án, lựa chọn đơn giản và tức thì nhất là tái sử dụng chính Redis, tránh được nhu cầu cài đặt và bảo trì một message broker chuyên dụng.

Let's now work on an example to demonstrate the simplicity and power of using Redis as a message broker.

Bây giờ hãy làm một ví dụ để minh họa sự đơn giản và sức mạnh của việc dùng Redis làm message broker.

This example requires a working installation of Redis, listening on its default port. You can find more details at nodejsdp.link/redis-quickstart.

Ví dụ này yêu cầu một bản cài đặt Redis đang hoạt động, lắng nghe trên cổng mặc định của nó. Bạn có thể tìm thêm chi tiết tại nodejsdp.link/redis-quickstart.

Our plan of action is to integrate our chat servers using Redis as a message broker. Each instance publishes any message received from its clients to the broker, and at the same time, it subscribes for any message coming from other server instances. As we can see, each server in our architecture is

both a subscriber and a publisher. The following diagram shows a representation of the architecture that we want to obtain:

Kế hoạch hành động của ta là tích hợp các chat server bằng cách dùng Redis làm message broker. Mỗi instance publish bất kỳ thông điệp nào nhận được từ client của nó tới broker, đồng thời, nó subscribe để nhận bất kỳ thông điệp nào đến từ các instance server khác. Như ta thấy, mỗi server trong kiến trúc của ta vừa là subscriber vừa là publisher. Sơ đồ sau cho thấy biểu diễn của kiến trúc mà ta muốn đạt được:

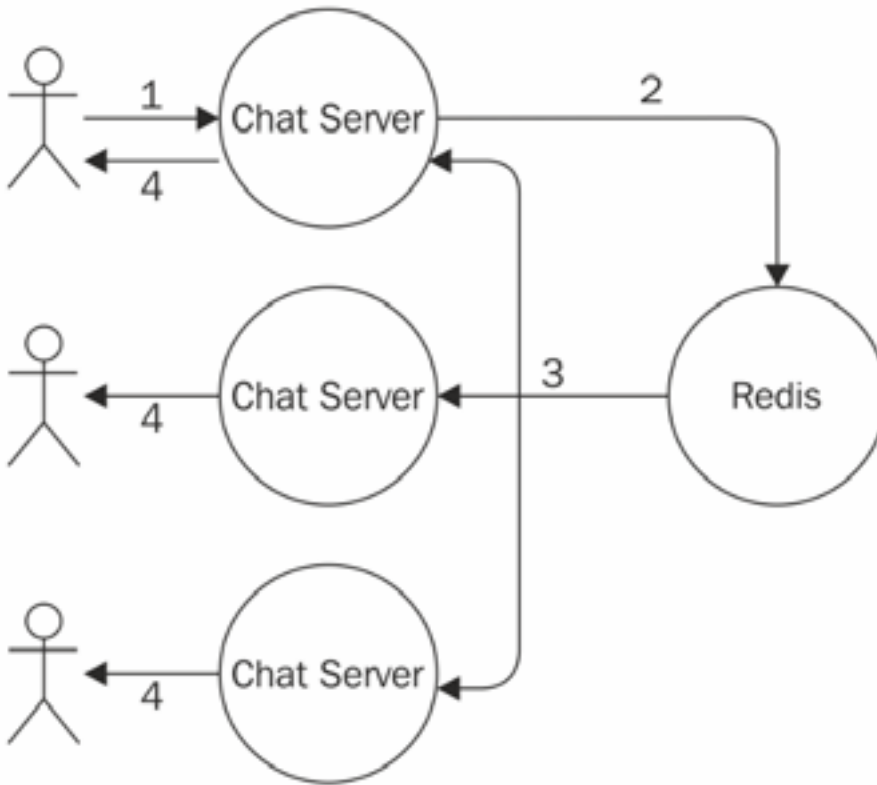


Figure 13.9: Using Redis as a message broker for our chat application

Hình 13.9: Dùng Redis làm message broker cho ứng dụng chat của chúng ta

Based on the architecture described in Figure 13.9, we can sum up the journey of a message as follows:

Dựa trên kiến trúc được mô tả trong Hình 13.9, ta có thể tóm tắt hành trình của một thông điệp như sau:

1. The message is typed into the textbox of the web page and sent to the connected instance of our chat server.
2. The message is then published to the broker.
3. The broker dispatches the message to all the subscribers, which in our architecture are all the instances of the chat server.
4. In each instance, the message is distributed to all the connected clients.
 1. Thông điệp được gõ vào hộp văn bản của trang web và gửi tới instance chat server đang kết nối.
 2. Thông điệp sau đó được publish tới broker.

3. Broker phân phát thông điệp tới tất cả các subscriber, mà trong kiến trúc của ta là tất cả các instance của chat server.
4. Trong mỗi instance, thông điệp được phân phối tới tất cả các client đang kết nối.

Let's see in practice how this works. Let's modify the server code by adding the publish/subscribe logic:

Hãy xem trong thực tế điều này hoạt động ra sao. Hãy sửa code của server bằng cách thêm logic publish/subscribe:

```
import { createServer } from 'http'
import staticHandler from 'serve-handler'
import ws from 'ws'
import Redis from 'ioredis' // (1)
const redisSub = new Redis()
const redisPub = new Redis()
// serve static files
const server = createServer((req, res) => {
  return staticHandler(req, res, { public: 'www' })
})
const wss = new ws.Server({ server })
wss.on('connection', client => {
  console.log('Client connected')
  client.on('message', msg => {
    console.log(`Message: ${msg}`)
    redisPub.publish('chat_messages', msg) // (2)
  })
})
redisSub.subscribe('chat_messages') // (3)
redisSub.on('message', (channel, msg) => {
  for (const client of wss.clients) {
    if (client.readyState === ws.OPEN) {
      client.send(msg)
    }
  }
})
server.listen(process.argv[2] || 8080)
```

The changes that we made to our original chat server are highlighted in the preceding code. This how the new implementation works:

Các thay đổi mà ta thực hiện trên chat server ban đầu được tô sáng trong đoạn code trên. Đây là cách bản triển khai mới hoạt động:

1. To connect our Node.js application to the Redis server, we use the ioredis package (nodejsdp.link/ioredis), which is a complete Node.js client supporting all the available Redis commands. Next, we instantiate two different connections, one used to subscribe to a channel, the other to publish messages. This is necessary in Redis, because once a connection is put in subscriber mode, only commands related to the subscription can be used. This means that we need a second connection for publishing messages.
2. When a new message is received from a connected client, we publish the message in the chat_messages channel. We don't directly broadcast the message to our clients because our server is subscribed to the same channel (as we will see in a moment), so it will come back to us

through Redis. For the scope of this example, this is a simple and effective mechanism. However, depending on the requirements of your application, you may instead want to broadcast the message immediately and ignore any message arriving from Redis and originating from the current server instance. We leave this to you as an exercise.

3. As we said, our server also has to subscribe to the `chat_messages` channel, so we register a listener to receive all the messages published into that channel (either by the current server instance or any other chat server instance). When a message is received, we simply broadcast it to all the clients connected to the current WebSocket server.

1. Để kết nối ứng dụng Node.js của ta tới Redis server, ta dùng gói `ioredis` (nodejsdp.link/ioredis), một client Node.js hoàn chỉnh hỗ trợ tất cả các lệnh Redis hiện có. Tiếp theo, ta khởi tạo hai kết nối khác nhau, một dùng để subscribe vào một channel, kết nối còn lại để publish thông điệp. Điều này cần thiết trong Redis, vì một khi một kết nối được đặt vào chế độ subscriber, chỉ các lệnh liên quan tới subscription mới có thể được dùng. Điều này nghĩa là ta cần một kết nối thứ hai để publish thông điệp.
2. Khi một thông điệp mới được nhận từ một client đang kết nối, ta publish thông điệp đó vào channel `chat_messages`. Ta không broadcast trực tiếp thông điệp tới các client vì server của ta cũng subscribe vào chính channel đó (như ta sẽ thấy trong giây lát), nên nó sẽ quay lại với ta thông qua Redis. Với phạm vi của ví dụ này, đây là một cơ chế đơn giản và hiệu quả. Tuy nhiên, tùy theo yêu cầu của ứng dụng, bạn có thể thay vào đó muốn broadcast thông điệp ngay lập tức và bỏ qua mọi thông điệp đến từ Redis có nguồn gốc từ chính instance server hiện tại. Chúng tôi để lại điều này cho bạn như một bài tập.
3. Như đã nói, server của ta cũng phải subscribe vào channel `chat_messages`, nên ta đăng ký một listener để nhận tất cả các thông điệp được publish vào channel đó (dù bởi instance server hiện tại hay bất kỳ instance chat server nào khác). Khi một thông điệp được nhận, ta đơn giản broadcast nó tới tất cả các client đang kết nối tới WebSocket server hiện tại.

These few changes are enough to integrate all the chat server instances that we might decide to start. To prove this, you can try starting multiple instances of our application:

Vài thay đổi này là đủ để tích hợp tất cả các instance chat server mà ta có thể quyết định khởi chạy. Để chứng minh điều này, bạn có thể thử khởi chạy nhiều instance của ứng dụng:

```
node index.js 8080
node index.js 8081
node index.js 8082
```

You can then connect multiple browser tabs to each instance and verify that the messages you send to one instance are successfully received by all the other clients connected to the other instances.

Sau đó bạn có thể kết nối nhiều tab trình duyệt tới mỗi instance và kiểm chứng rằng các thông điệp bạn gửi tới một instance được nhận thành công bởi tất cả các client khác đang kết nối tới các instance khác.

Congratulations! We just integrated multiple nodes of a distributed real-time application using the Publish/Subscribe pattern.

Xin chúc mừng! Chúng ta vừa tích hợp nhiều node của một ứng dụng thời gian thực phân tán bằng cách dùng mẫu Publish/Subscribe.

Redis allows us to publish and subscribe to channels identified by a string, for example, `chat.nodejs`. But it also allows us to use globstyle patterns to define subscriptions that can

potentially match multiple channels, for example, `chat.*`.

Redis cho phép ta publish và subscribe vào các channel được định danh bằng một chuỗi, ví dụ, `chat.nodejs`. Nhưng nó cũng cho phép ta dùng các mẫu kiểu glob để định nghĩa các subscription có thể khớp với nhiều channel, ví dụ, `chat.*`.

Peer-to-peer Publish/Subscribe with ZeroMQ — Publish/Subscribe peer-to-peer với ZeroMQ

The presence of a broker can considerably simplify the architecture of a messaging system. However, in some circumstances, this may not be the best solution. This includes all the situations where a low latency is critically important, or when scaling complex distributed systems, or when the presence of a single point of failure is not an option. The alternative to using a broker is, of course, implementing a peer-to-peer messaging system.

Sự hiện diện của một broker có thể đơn giản hóa đáng kể kiến trúc của một messaging system. Tuy nhiên, trong một số trường hợp, đây có thể không phải là giải pháp tốt nhất. Điều này bao gồm tất cả các tình huống mà độ trễ thấp là cực kỳ quan trọng, hoặc khi mở rộng các hệ thống phân tán phức tạp, hoặc khi sự hiện diện của một điểm hỏng đơn là không thể chấp nhận. Giải pháp thay thế cho việc dùng broker, tất nhiên, là triển khai một messaging system peer-to-peer.

Introducing ZeroMQ — Giới thiệu ZeroMQ

If our project is a good candidate for a peer-to-peer architecture, one of the best solutions to evaluate is certainly ZeroMQ (nodejsdp.link/zeromq, also known as `zmq` or `ØMQ`). ZeroMQ is a networking library that provides the basic tools to build a large variety of messaging patterns. It is low-level, extremely fast, and has a minimalistic **API**, but it offers all the basic building blocks to create a solid messaging system, such as atomic messages, **load balancing**, queues, and many more. It supports many types of transport, such as in-process channels (`inproc://`), inter-process communication (`ipc://`), multicast using the PGM protocol (`pgm://` or `epgm://`), and, of course, the classic TCP (`tcp://`). Among the features of ZeroMQ, we can also find tools to implement a Publish/Subscribe pattern, which is exactly what we need for our example. So, what we are going to do now is remove the broker (Redis) from the architecture of our chat application and let the various nodes communicate in a peer-to-peer fashion, leveraging the publish/subscribe sockets of ZeroMQ.

Nếu dự án của ta là một ứng viên tốt cho kiến trúc peer-to-peer, một trong những giải pháp tốt nhất để cân nhắc chắc chắn là ZeroMQ (nodejsdp.link/zeromq, còn được biết đến là `zmq` hay `ØMQ`). ZeroMQ là một thư viện mạng cung cấp các công cụ cơ bản để xây dựng đa dạng các mẫu nhắn tin. Nó cấp thấp (low-level), cực kỳ nhanh, và có một API tối giản, nhưng cung cấp tất cả các khối xây dựng cơ bản để tạo ra một messaging system vững chắc, như thông điệp nguyên tử (atomic message), load balancing, queue và nhiều thứ khác. Nó hỗ trợ nhiều loại transport, như các kênh trong-tiến-trình (`inproc://`), giao tiếp liên-tiến-trình (`ipc://`), multicast dùng giao thức PGM (`pgm://` hoặc `epgm://`), và tất nhiên, TCP cổ điển (`tcp://`). Trong số các tính năng của ZeroMQ, ta cũng có thể tìm thấy các công cụ để triển khai một mẫu Publish/Subscribe, chính là thứ ta cần cho ví dụ của mình. Vậy, điều ta sắp làm bây giờ là loại bỏ broker (Redis) khỏi kiến trúc của ứng dụng chat và để cho các node khác nhau giao tiếp theo kiểu peer-to-peer, tận dụng các socket publish/subscribe của ZeroMQ.

A ZeroMQ socket can be considered as a network socket on steroids, which provides additional

abstractions to help implement the most common messaging patterns. For example, we can find sockets designed to implement publish/subscribe, request/reply, or one-way push communications.

Một socket của ZeroMQ có thể được coi như một network socket được tăng cường sức mạnh, cung cấp thêm các lớp trừu tượng để giúp triển khai các mẫu nhắn tin phổ biến nhất. Ví dụ, ta có thể tìm thấy các socket được thiết kế để triển khai publish/subscribe, request/reply, hoặc giao tiếp push một chiều.

Designing a peer-to-peer architecture for the chat server — Thiết kế một kiến trúc peer-to-peer cho chat server

When we remove the broker from our architecture, each instance of the chat server has to directly connect to the other available instances in order to receive the messages they publish. In ZeroMQ, we have two types of sockets specifically designed for this purpose: PUB and SUB. The typical pattern is to bind a PUB socket to a local port where it will start listening for incoming subscription requests from sockets of type SUB.

Khi ta loại bỏ broker khỏi kiến trúc, mỗi instance của chat server phải kết nối trực tiếp tới các instance khả dụng khác để nhận các thông điệp mà chúng publish. Trong ZeroMQ, ta có hai loại socket được thiết kế riêng cho mục đích này: PUB và SUB. Mẫu hình điển hình là bind một socket PUB vào một cổng cục bộ, nơi nó sẽ bắt đầu lắng nghe các yêu cầu subscription đến từ các socket loại SUB.

A subscription can have a filter that specifies what messages are delivered to the connected SUB sockets. The filter is a simple binary **buffer** (so it can also be a string), which will be matched against the beginning of the message (which is also a binary buffer). When a message is sent through the PUB socket it is broadcast to all the connected SUB sockets, but only after their subscription filters are applied. The filters will be applied to the publisher side only if a connected protocol is used, such as, for example, TCP.

Một subscription có thể có một filter chỉ định những thông điệp nào được phân phát tới các socket SUB đang kết nối. Filter là một buffer nhị phân đơn giản (nên nó cũng có thể là một chuỗi), sẽ được khớp với phần đầu của thông điệp (cũng là một buffer nhị phân). Khi một thông điệp được gửi qua socket PUB, nó được broadcast tới tất cả các socket SUB đang kết nối, nhưng chỉ sau khi các filter subscription của chúng được áp dụng. Các filter sẽ được áp dụng ở phía publisher chỉ khi một giao thức có kết nối (connected) được dùng, chẳng hạn như TCP.

The following diagram shows the pattern applied to our distributed chat server architecture (with only two instances, for simplicity):

Sơ đồ sau cho thấy mẫu được áp dụng vào kiến trúc chat server phân tán của ta (chỉ với hai instance, cho đơn giản):

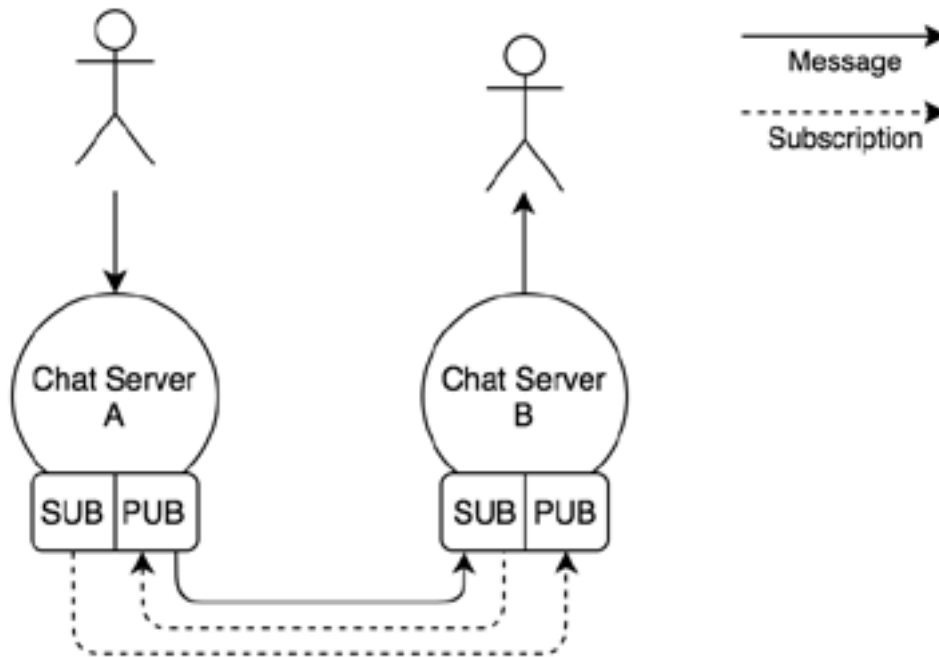


Figure 13.10: Chat server messaging architecture using ZeroMQ PUB/SUB sockets Figure 13.10 shows us the flow of information when we have two instances of the chat application, but the same concept can be applied to N instances. This architecture tells us that each node must be aware of the other nodes in the system to be able to establish all the necessary connections. It also shows us how the subscriptions go from a SUB socket to a PUB socket, while messages travel in the opposite direction.

Hình 13.10: Kiến trúc nhắn tin của chat server dùng các socket PUB/SUB của ZeroMQ. Hình 13.10 cho ta thấy luồng thông tin khi có hai instance của ứng dụng chat, nhưng cùng khái niệm đó có thể áp dụng cho N instance. Kiến trúc này cho ta biết rằng mỗi node phải biết về các node khác trong hệ thống để có thể thiết lập tất cả các kết nối cần thiết. Nó cũng cho ta thấy cách các subscription đi từ một socket SUB tới một socket PUB, trong khi các thông điệp di chuyển theo hướng ngược lại.

Using the ZeroMQ PUB/SUB sockets — Dùng các socket PUB/SUB của ZeroMQ

Let's see how the ZeroMQ PUB/SUB sockets work in practice by modifying our chat server:

Hãy xem các socket PUB/SUB của ZeroMQ hoạt động ra sao trong thực tế bằng cách sửa chat server của ta:

```
import { createServer } from 'http'
import staticHandler from 'serve-handler'
import ws from 'ws'
import yargs from 'yargs' // (1)
import zmq from 'zeromq'
// serve static files
const server = createServer((req, res) => {
  return staticHandler(req, res, { public: 'www' })
})
let pubSocket
async function initializeSockets () {
```

```

pubSocket = new zmq.Publisher() // (2)
await pubSocket.bind(`tcp://127.0.0.1:${yargs.argv.pub}`)
const subSocket = new zmq.Subscriber() // (3)
const subPorts = [].concat(yargs.argv.sub)
for (const port of subPorts) {
  console.log(`Subscribing to ${port}`)
  subSocket.connect(`tcp://127.0.0.1:${port}`)
}
subSocket.subscribe('chat')
for await (const [msg] of subSocket) { // (4)
  console.log(`Message from another server: ${msg}`)
  broadcast(msg.toString().split(' ')[1])
}
}
initializeSockets()
const wss = new ws.Server({ server })
wss.on('connection', client => {
  console.log('Client connected')
  client.on('message', msg => {
    console.log(`Message: ${msg}`)
    broadcast(msg)
    pubSocket.send(`chat ${msg}`) // (5)
  })
})
function broadcast (msg) {
  for (const client of wss.clients) {
    if (client.readyState === ws.OPEN) {
      client.send(msg)
    }
  }
}
server.listen(yargs.argv.http || 8080)

```

The preceding code clearly shows that the logic of our application became slightly more complicated, however, it's still straightforward considering that we are implementing a peer-to-peer Publish/Subscribe pattern. Let's see how all the pieces come together:

Đoạn code trên cho thấy rõ rằng logic của ứng dụng đã trở nên phức tạp hơn một chút, tuy nhiên, nó vẫn khá đơn giản nếu xét rằng ta đang triển khai một mẫu Publish/Subscribe peer-to-peer. Hãy xem tất cả các mảnh ghép kết hợp với nhau ra sao:

1. We import two new packages. First, we import `yargs` (nodejsdp.link/yargs), which is a command-line argument parser; we need this to easily accept named arguments. Secondly, we import the `zeromq` package ([nodejsdp.link/ zeromq](https://nodejsdp.link/zeromq)), which is a Node.js client for ZeroMQ.
2. In the `initializeSockets()` function, we immediately create our Publisher socket and bind it to the port provided in the `--pub` command-line argument.
3. We create the Subscriber socket and we connect it to the Publisher sockets of the other instances of our application. The ports of the target Publisher sockets are provided in the `--sub` command-line arguments (there might be more than one). We then create the actual subscription, by providing `chat` as a filter, which means that we will receive only the messages beginning with `chat`.
4. We start listening for messages arriving at our Subscriber socket using a `for **await**`...of

loop, since `subSocket` is an **async** iterable. With each message we receive, we do some simple parsing to remove the chat prefix, and then we `broadcast()` the actual payload to all the clients connected to the current WebSocket server.

5. When a new message is received by the WebSocket server of the current instance, we broadcast it to all the connected clients but we also publish it through our Publisher socket. We use chat as a prefix followed by a space, so that the message will be published to all the subscriptions using chat as a filter.

1. Ta import hai gói mới. Đầu tiên, ta import `yargs` (`nodejsdp.link/yargs`), một bộ phân tích đối số dòng lệnh (command-line argument parser); ta cần cái này để dễ dàng chấp nhận các đối số có tên. Thứ hai, ta import gói `zeromq` (`nodejsdp.link/zeromq`), một client Node.js cho ZeroMQ.
2. Trong hàm `initializeSockets()`, ta lập tức tạo socket Publisher và bind nó vào cổng được cung cấp trong đối số dòng lệnh `--pub`.
3. Ta tạo socket Subscriber và kết nối nó tới các socket Publisher của các instance khác của ứng dụng. Các cổng của các socket Publisher đích được cung cấp trong các đối số dòng lệnh `--sub` (có thể có nhiều hơn một). Sau đó ta tạo subscription thực sự, bằng cách cung cấp chat làm filter, nghĩa là ta sẽ chỉ nhận các thông điệp bắt đầu bằng chat.
4. Ta bắt đầu lắng nghe các thông điệp đến socket Subscriber của mình bằng một vòng lặp `for await...of`, vì `subSocket` là một async iterable. Với mỗi thông điệp nhận được, ta thực hiện một chút phân tích đơn giản để loại bỏ tiền tố chat, rồi `broadcast()` payload thực sự tới tất cả các client đang kết nối tới WebSocket server hiện tại.
5. Khi một thông điệp mới được nhận bởi WebSocket server của instance hiện tại, ta broadcast nó tới tất cả các client đang kết nối nhưng cũng publish nó qua socket Publisher của ta. Ta dùng chat làm tiền tố theo sau bởi một dấu cách, để thông điệp sẽ được publish tới tất cả các subscription dùng chat làm filter.

We have now built a simple distributed system, integrated using a peer-to-peer Publish/Subscribe pattern!

Giờ ta đã xây dựng một hệ thống phân tán đơn giản, được tích hợp bằng một mẫu Publish/Subscribe peer-to-peer!

Let's fire it up, let's start three instances of our application by making sure to connect their Publisher and Subscriber sockets properly:

Hãy khởi động nó lên, hãy chạy ba instance của ứng dụng, đảm bảo kết nối các socket Publisher và Subscriber của chúng cho đúng:

```
node index.js --http 8080 --pub 5000 --sub 5001 --sub 5002
node index.js --http 8081 --pub 5001 --sub 5000 --sub 5002
node index.js --http 8082 --pub 5002 --sub 5000 --sub 5001
```

The first command will start an instance with an HTTP server listening on port 8080, while binding its Publisher socket on port 5000 and connecting the Subscriber socket to ports 5001 and 5002, which is where the Publisher sockets of the other two instances should be listening at. The other two commands work in a similar way.

Lệnh đầu tiên sẽ khởi chạy một instance với một HTTP server lắng nghe trên cổng 8080, đồng thời bind socket Publisher của nó vào cổng 5000 và kết nối socket Subscriber tới các cổng 5001 và 5002, là nơi các socket Publisher của hai instance kia sẽ lắng nghe. Hai lệnh kia hoạt động theo cách tương tự.

Now, the first thing you will see is that ZeroMQ will not complain if a Subscriber socket can't establish a connection to a Publisher socket. For example, at the time of the first command, there are no Publisher sockets listening on ports 5001 and 5002, however, ZeroMQ is not throwing any error. This is because ZeroMQ is built to be resilient to faults and it implements a built-in connection retry mechanism. This feature also comes in particularly handy if any node goes down or is restarted. The same forgiving logic applies to the Publisher socket: if there are no subscriptions, it will simply drop all the messages, but it will continue working.

Bây giờ, điều đầu tiên bạn sẽ thấy là ZeroMQ sẽ không phàn nàn nếu một socket Subscriber không thể thiết lập kết nối tới một socket Publisher. Ví dụ, tại thời điểm chạy lệnh đầu tiên, không có socket Publisher nào lắng nghe trên các cổng 5001 và 5002, tuy nhiên, ZeroMQ không ném ra lỗi nào. Đây là vì ZeroMQ được xây dựng để chịu lỗi (resilient to faults) và nó triển khai sẵn một cơ chế thử kết nối lại. Tính năng này cũng đặc biệt hữu ích nếu một node nào đó sập hoặc được khởi động lại. Cùng một logic khoan dung như vậy áp dụng cho socket Publisher: nếu không có subscription nào, nó sẽ đơn giản loại bỏ tất cả các thông điệp, nhưng vẫn tiếp tục hoạt động.

At this point, we can try to navigate with a browser to any of the server instances that we started and verify that the messages are properly propagated to all the chat servers. In the previous example, we assumed a static architecture where the number of instances and their addresses are known in advance. We can introduce a service registry, as explained in Chapter 12, Scalability and Architectural Patterns, to connect our instances dynamically. It is also important to point out that ZeroMQ can be used to implement a broker using the same primitives we demonstrated here.

Đến đây, ta có thể thử dùng trình duyệt điều hướng tới bất kỳ instance server nào mà ta đã khởi chạy và kiểm chứng rằng các thông điệp được lan truyền đúng cách tới tất cả các chat server. Trong ví dụ trước, ta giả định một kiến trúc tĩnh trong đó số lượng instance và địa chỉ của chúng được biết trước. Ta có thể đưa vào một service registry, như đã giải thích trong Chương 12, Scalability and Architectural Patterns, để kết nối các instance một cách động. Cũng quan trọng cần chỉ ra rằng ZeroMQ có thể được dùng để triển khai một broker bằng chính những primitive mà ta đã minh họa ở đây.

Reliable message delivery with queues — Chuyển giao thông điệp đáng tin cậy với hàng đợi

An important abstraction in a messaging system is the message queue (MQ). With a message queue, the sender and the receiver(s) of the message don't necessarily need to be active and connected at the same time to establish a communication, because the queuing system takes care of storing the messages until the destination is able to receive them. This behavior is opposed to the fire-and-forget paradigm, where a subscriber can receive messages only during the time it is connected to the messaging system.

Một lớp trừu tượng quan trọng trong một messaging system là message queue (MQ). Với một message queue, bên gửi và (các) bên nhận thông điệp không nhất thiết phải cùng hoạt động và kết nối tại cùng một thời điểm để thiết lập một giao tiếp, vì hệ thống queuing lo việc lưu trữ thông điệp cho tới khi đích có thể nhận chúng. Hành vi này trái ngược với mô hình fire-and-forget, trong đó một subscriber chỉ có thể nhận thông điệp trong khoảng thời gian nó kết nối tới messaging system.

A subscriber that is able to always reliably receive all the messages, even those sent when it's not listening for them, is called a durable subscriber.

Một subscriber có khả năng luôn nhận một cách đáng tin cậy tất cả các thông điệp, kể cả

những thông điệp được gửi khi nó không lắng nghe, được gọi là một durable subscriber.

We can summarize the delivery semantic of a messaging system in three categories:

Ta có thể tóm tắt ngữ nghĩa chuyển giao của một messaging system thành ba loại:

- At most once: Also known as fire-and-forget, the message is not persisted, and the delivery is not acknowledged. This means that the message can be lost in cases of crashes or disconnections of the receiver.
- At least once: The message is guaranteed to be received at least once, but duplicates might occur if, for example, the receiver crashes before notifying the sender of the reception. This implies that the message has to be persisted in the eventuality it has to be sent again.
- Exactly once: This is the most reliable delivery semantic. It guarantees that the message is received once and only once. This comes at the expense of a slower and more data-intensive mechanism for acknowledging the delivery of messages.
 - At most once (nhiều nhất một lần): Còn được gọi là fire-and-forget, thông điệp không được lưu bền, và việc chuyển giao không được xác nhận. Điều này nghĩa là thông điệp có thể bị mất trong các trường hợp crash hoặc ngắt kết nối của bên nhận.
 - At least once (ít nhất một lần): Thông điệp được đảm bảo nhận ít nhất một lần, nhưng có thể xảy ra trùng lặp nếu, ví dụ, bên nhận crash trước khi thông báo cho bên gửi về việc đã nhận. Điều này hàm ý rằng thông điệp phải được lưu bền phòng trường hợp phải gửi lại.
 - Exactly once (chính xác một lần): Đây là ngữ nghĩa chuyển giao đáng tin cậy nhất. Nó đảm bảo rằng thông điệp được nhận một và chỉ một lần. Điều này phải trả giá bằng một cơ chế xác nhận chuyển giao thông điệp chậm hơn và tốn nhiều dữ liệu hơn.

We have a durable subscriber when our messaging system can achieve an “at least once” or an “exactly once” delivery semantic and to do that, the system has to use a message queue to accumulate the messages while the subscriber is disconnected. The queue can be stored in memory or persisted on disk to allow the recovery of its messages even if the queuing system restarts or crashes. The following diagram shows a graphical representation of a durable subscriber backed by a message queue:

Ta có một durable subscriber khi messaging system của ta có thể đạt được ngữ nghĩa chuyển giao “at least once” hoặc “exactly once”, và để làm điều đó, hệ thống phải dùng một message queue để tích lũy các thông điệp trong khi subscriber bị ngắt kết nối. Queue có thể được lưu trong bộ nhớ hoặc lưu bền trên đĩa để cho phép khôi phục các thông điệp của nó ngay cả khi hệ thống queuing khởi động lại hay crash. Sơ đồ sau cho thấy biểu diễn đồ họa của một durable subscriber được hỗ trợ bởi một message queue:

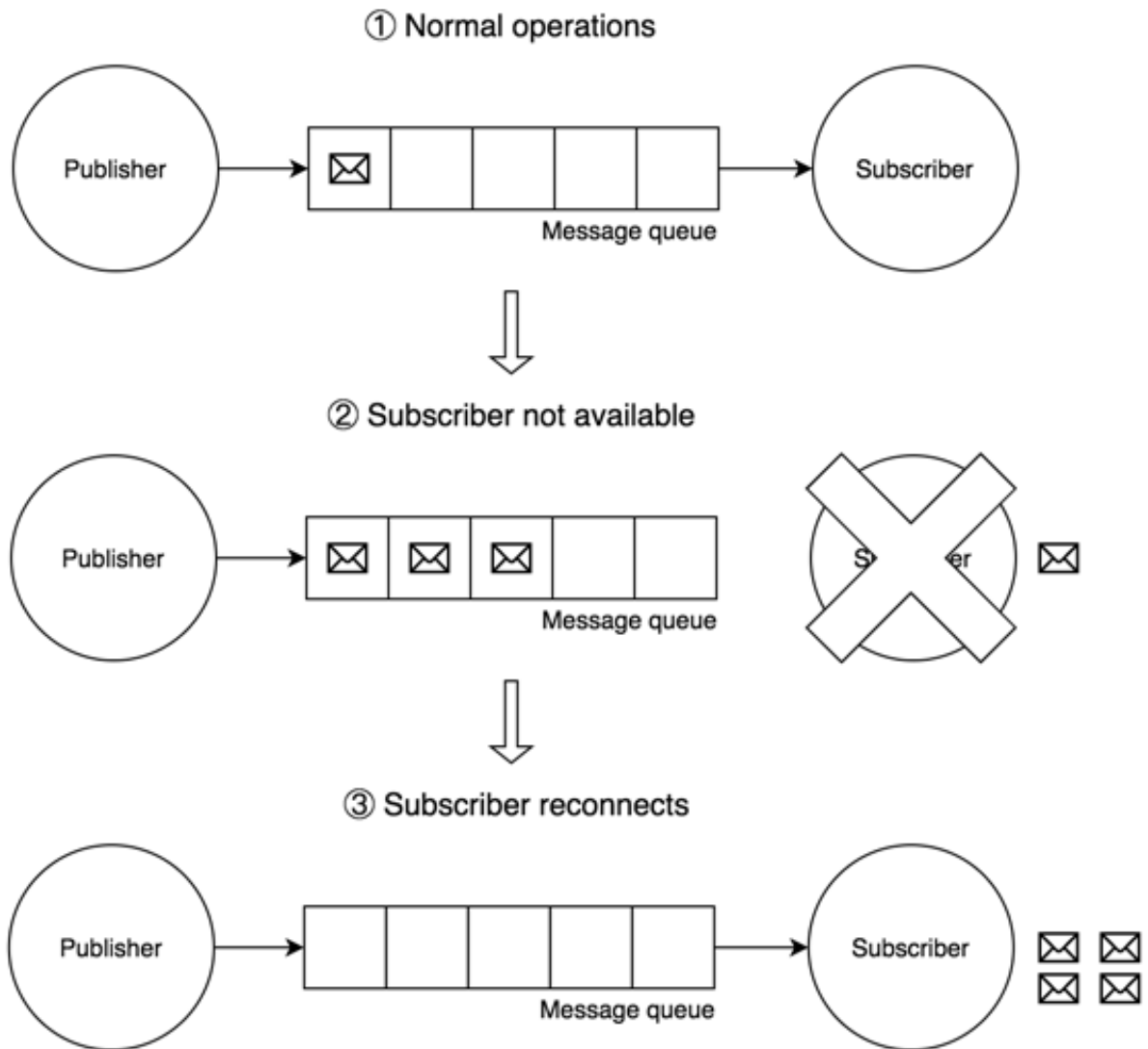


Figure 13.11: Example behavior of a messaging system backed by a queue

Hình 13.11: Ví dụ hành vi của một messaging system được hỗ trợ bởi một queue

Figure 13.11 shows us how a message queue can help us implement the Durable Subscriber pattern. As we can see, during normal operations (1) messages travel from the publisher to the subscriber through the message queue. When the subscriber goes offline (2) because of a crash, a malfunction, or simply a planned maintenance period, any message sent by the publisher is stored and accumulated safely in the message queue. Afterward, when the subscriber comes back online (3), all messages accumulated in the queue are sent to the subscriber, so no message is lost.

Hình 13.11 cho ta thấy một message queue có thể giúp ta triển khai mẫu Durable Subscriber ra sao. Như ta thấy, trong quá trình hoạt động bình thường (1), các thông điệp di chuyển từ publisher tới subscriber thông qua message queue. Khi subscriber ngoại tuyến (2) vì một crash, một trục trặc, hoặc đơn giản là một kỳ bảo trì có kế hoạch, bất kỳ thông điệp nào được gửi bởi publisher đều được lưu trữ và tích lũy an toàn trong message queue. Sau đó, khi subscriber trở lại trực tuyến (3), tất cả các thông điệp đã tích lũy trong queue được gửi tới subscriber, nên không có thông điệp nào bị mất.

The durable subscriber is probably the most important pattern enabled by a message queue, but it's certainly not the only one, as we will see later in the chapter.

Durable subscriber có lẽ là mẫu quan trọng nhất được message queue làm khả thi, nhưng nó chắc chắn không phải là mẫu duy nhất, như ta sẽ thấy về sau trong chương.

Next, we are going to learn about AMQP, which is the protocol we are going to use throughout the rest of the chapter to implement our message queue examples.

Tiếp theo, ta sẽ tìm hiểu về AMQP, giao thức mà ta sẽ dùng xuyên suốt phần còn lại của chương để triển khai các ví dụ message queue của mình.

Introducing AMQP — Giới thiệu AMQP

A message queue is normally used in situations where messages must not be lost, which includes mission-critical applications such as banking systems, air traffic management and control systems, medical applications, and so on. This usually means that the typical enterprise-grade message queue is a very complex piece of software, which utilizes bulletproof protocols and persistent storage to guarantee the delivery of the message even in the presence of malfunctions. For this reason, enterprise messaging middleware has been, for many years, a prerogative of tech giants such as Oracle and IBM, each one of them usually implementing their own proprietary protocol, resulting in a strong customer lock-in. Fortunately, it's been a few years now since messaging systems entered the mainstream, thanks to the growth of open protocols such as AMQP, STOMP, and MQTT. Throughout the rest of the chapter we are going to use AMQP as the messaging protocol for our queuing system, so let's give it a proper introduction.

Một message queue thường được dùng trong các tình huống mà thông điệp không được phép bị mất, bao gồm các ứng dụng trọng yếu (mission-critical) như hệ thống ngân hàng, hệ thống quản lý và điều khiển không lưu, ứng dụng y tế, v.v. Điều này thường nghĩa là một message queue cấp doanh nghiệp điển hình là một phần mềm rất phức tạp, sử dụng các giao thức chống đạn (bulletproof) và lưu trữ bền vững để đảm bảo việc chuyển giao thông điệp ngay cả khi có trục trặc. Vì lý do này, trong nhiều năm, các messaging middleware cấp doanh nghiệp đã là đặc quyền của các gã khổng lồ công nghệ như Oracle và IBM, mỗi hãng thường triển khai giao thức độc quyền của riêng mình, dẫn tới việc khóa khách hàng (customer lock-in) mạnh mẽ. May thay, đã vài năm nay các messaging system trở nên phổ biến trong giới chính thống, nhờ sự phát triển của các giao thức mở như AMQP, STOMP và MQTT. Trong suốt phần còn lại của chương, ta sẽ dùng AMQP làm giao thức nhắn tin cho hệ thống queuing của mình, nên hãy giới thiệu nó một cách đàng hoàng.

AMQP is an open standard protocol supported by many message-queuing systems. Besides defining a common communication protocol, it also provides a model to describe routing, filtering, queuing, reliability, and security.

AMQP là một giao thức chuẩn mở được hỗ trợ bởi nhiều hệ thống message-queuing. Bên cạnh việc định nghĩa một giao thức giao tiếp chung, nó cũng cung cấp một mô hình để mô tả định tuyến, lọc, hàng đợi, độ tin cậy và bảo mật.

The following diagram shows us all the AMQP components at a glance:

Sơ đồ sau cho ta thấy tất cả các thành phần AMQP trong một cái nhìn:

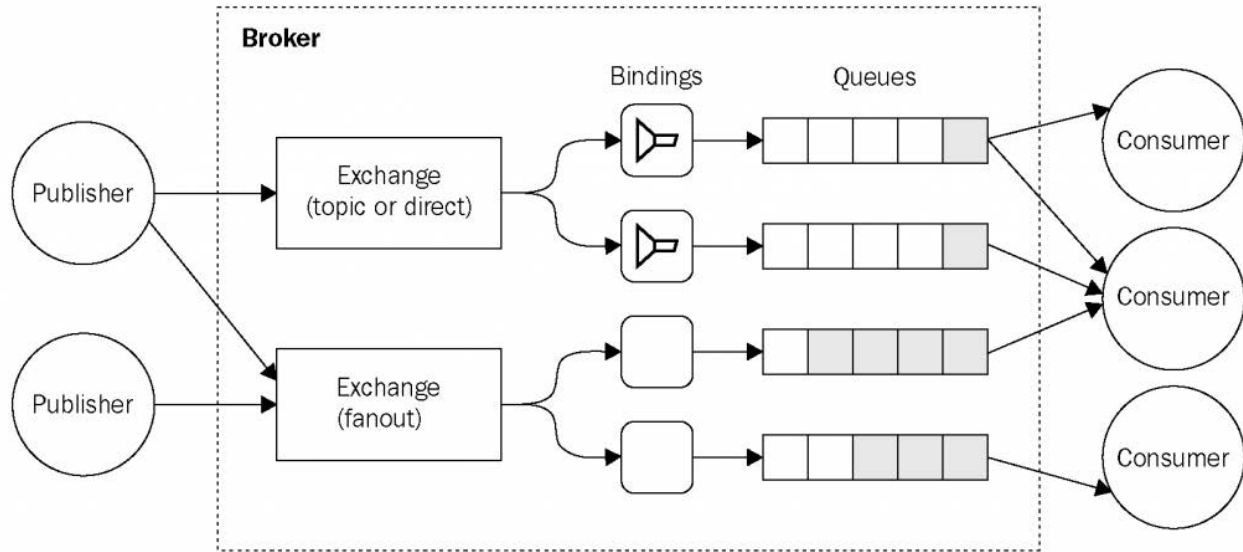


Figure 13.12: Example of an AMQP-based messaging system

Hình 13.12: Ví dụ về một messaging system dựa trên AMQP

As shown in Figure 13.12, in AMQP there are three essential components:

Như thể hiện trong Hình 13.12, trong AMQP có ba thành phần thiết yếu:

- **Queue:** The data structure responsible for storing the messages consumed by the clients. The messages from a queue are pushed (or pulled) to one or more consumers. If multiple consumers are attached to the same queue, the messages are load balanced across them. A queue can be any of the following:
 - **Durable:** This means that the queue is automatically recreated if the broker restarts. A durable queue does not imply that its contents are preserved as well; in fact, only messages that are marked as persistent are saved to the disk and restored in case of a restart.
 - **Exclusive:** This means that the queue is bound to only one particular subscriber connection. When the connection is closed, the queue is destroyed.
 - **Auto-delete:** This will cause the queue to be deleted when the last subscriber disconnects.
- **Exchange:** This is where a message is published. An exchange routes the messages to one or more queues depending on the algorithm it implements:
 - **Direct exchange:** It routes the messages by matching an entire routing key (for example, `chat.msg`)
 - **Topic exchange:** It distributes the messages using a glob-like pattern matched against the routing key (for example, `chat.#` matches all the routing keys starting with `chat.`)
 - **Fanout exchange:** It broadcasts a message to all the connected queues, ignoring any routing key provided
- **Binding:** This is the link between exchanges and queues. It also defines the routing key or the pattern used to filter the messages that arrive from the exchange.
 - **Queue:** Cấu trúc dữ liệu chịu trách nhiệm lưu trữ các thông điệp được tiêu thụ bởi client. Các thông điệp từ một queue được đẩy (hoặc kéo) tới một hoặc nhiều consumer. Nếu nhiều consumer gắn vào cùng một queue, các thông điệp được load balance qua chúng. Một queue có thể là một trong những loại sau:
 - **Durable:** Điều này nghĩa là queue được tự động tái tạo nếu broker khởi động lại. Một durable queue không hàm ý rằng nội dung của nó cũng được bảo toàn; thực tế, chỉ những thông điệp được đánh dấu là persistent mới được lưu vào đĩa và khôi phục

trong trường hợp khởi động lại.

- Exclusive: Điều này nghĩa là queue chỉ gắn với một kết nối subscriber cụ thể. Khi kết nối bị đóng, queue bị hủy.
- Auto-delete: Điều này sẽ khiến queue bị xóa khi subscriber cuối cùng ngắt kết nối.
- Exchange: Đây là nơi một thông điệp được publish. Một exchange định tuyến các thông điệp tới một hoặc nhiều queue tùy theo thuật toán mà nó triển khai:
- Direct exchange: Nó định tuyến các thông điệp bằng cách khớp toàn bộ một routing key (ví dụ, chat.msg)
- Topic exchange: Nó phân phối các thông điệp dùng một mẫu kiểu glob khớp với routing key (ví dụ, chat.# khớp với tất cả các routing key bắt đầu bằng chat.)
- Fanout exchange: Nó broadcast một thông điệp tới tất cả các queue đang kết nối, bỏ qua mọi routing key được cung cấp
- Binding: Đây là liên kết giữa các exchange và các queue. Nó cũng định nghĩa routing key hoặc mẫu dùng để lọc các thông điệp đến từ exchange.

These components are managed by a broker, which exposes an API for creating and manipulating them. When connecting to a broker, a client creates a channel—an abstraction of a connection—which is responsible for maintaining the state of the communication with the broker.

Các thành phần này được quản lý bởi một broker, broker phơi bày một API để tạo và thao tác với chúng. Khi kết nối tới một broker, một client tạo ra một channel — một lớp trừu tượng của một kết nối — chịu trách nhiệm duy trì trạng thái của giao tiếp với broker.

In AMQP, we can obtain the Durable Subscriber pattern by creating any type of queue that is not exclusive or auto-delete.

Trong AMQP, ta có thể đạt được mẫu Durable Subscriber bằng cách tạo bất kỳ loại queue nào không phải là exclusive hay auto-delete.

The AMQP model is way more complex than the messaging systems we have used so far (Redis and ZeroMQ). However, it offers a set of features and a level of reliability that would be very hard to obtain using only primitive publish/subscribe mechanisms.

Mô hình AMQP phức tạp hơn nhiều so với các messaging system mà ta đã dùng cho tới giờ (Redis và ZeroMQ). Tuy nhiên, nó cung cấp một tập tính năng và một mức độ tin cậy mà sẽ rất khó đạt được nếu chỉ dùng các cơ chế publish/subscribe nguyên thủy.

You can find a detailed introduction to the AMQP model on the RabbitMQ website at nodejsdp.link/amqp-components.

Bạn có thể tìm thấy một phần giới thiệu chi tiết về mô hình AMQP trên trang web của RabbitMQ tại nodejsdp.link/amqp-components.

Durable subscribers with AMQP and RabbitMQ — Durable subscriber với AMQP và RabbitMQ

Let's now practice what we learned about durable subscribers and AMQP and work on a small example. A typical scenario where it's important to not lose any message is when we want to keep the different services of a microservice architecture in sync (we already described this integration pattern in the previous chapter). If we want to use a broker to keep all our services on the same page, it's important that we don't lose any information, otherwise we might end up in an inconsistent state.

Bây giờ hãy thực hành những gì ta đã học về durable subscriber và AMQP và làm một ví dụ nhỏ. Một kịch bản điển hình mà việc không được mất thông điệp là quan trọng chính là khi ta muốn giữ cho các service khác nhau của một kiến trúc microservice đồng bộ với

nhau (ta đã mô tả mẫu tích hợp này ở chương trước). Nếu ta muốn dùng một broker để giữ cho tất cả các service của mình ở cùng một trạng, điều quan trọng là ta không được làm mất bất kỳ thông tin nào, nếu không ta có thể rơi vào một trạng thái không nhất quán.

Designing a history service for the chat application — Thiết kế một history service cho ứng dụng chat Let's now extend our small chat application using a microservice approach. Let's add a history service that persists our chat messages inside a database, so that when a client connects, we can query the service and retrieve the entire chat history. We are going to integrate the history service with the chat server using the RabbitMQ broker (`nodejsdp.link/rabbitmq`) and AMQP.

Bây giờ hãy mở rộng ứng dụng chat nhỏ của ta bằng cách tiếp cận microservice. Hãy thêm một history service lưu bền các thông điệp chat của ta bên trong một database, để khi một client kết nối, ta có thể truy vấn service và lấy lại toàn bộ lịch sử chat. Ta sẽ tích hợp history service với chat server bằng cách dùng broker RabbitMQ (`nodejsdp.link/rabbitmq`) và AMQP.

The following diagram shows our planned architecture:

Sơ đồ sau cho thấy kiến trúc dự kiến của ta:

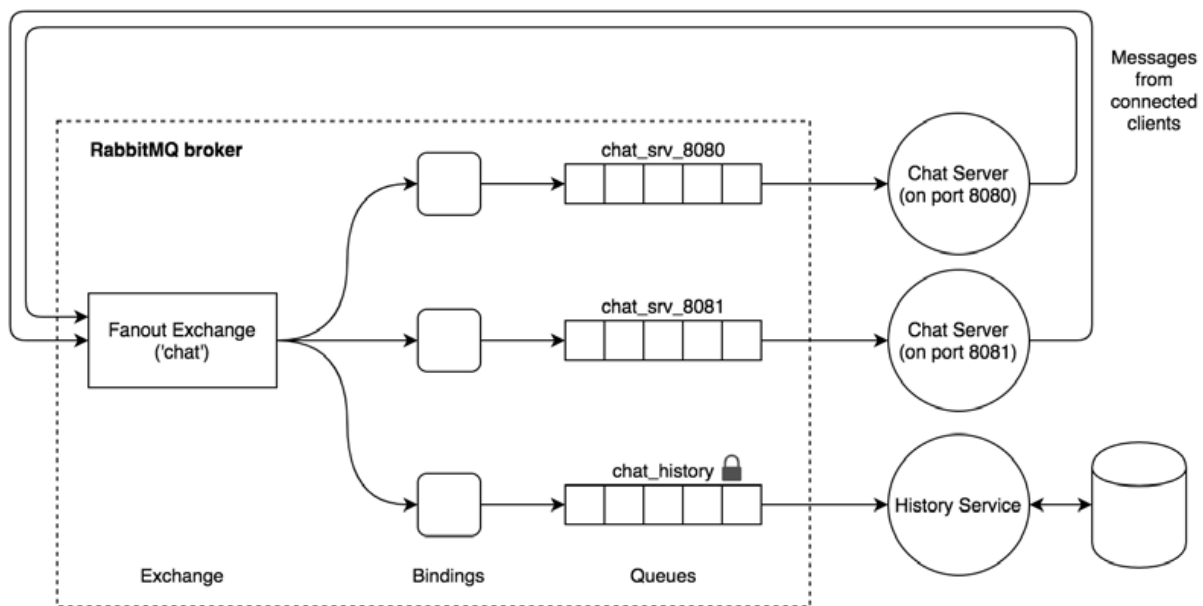


Figure 13.13: Architecture of our chat application with AMQP and history service

Hình 13.13: Kiến trúc ứng dụng chat của chúng ta với AMQP và history service

As shown in Figure 13.13, we are going to use a single fanout exchange; we don't need any complicated routing logic, so our scenario does not require any exchange more complex than that. Next, we will create one queue for each instance of the chat server. These queues are exclusive since we are not interested in receiving any messages missed while a chat server is offline; that's the job of our history service, which can eventually also implement more complicated queries against the stored messages. In practice, this means that our chat servers are not durable subscribers and their queues will be destroyed as soon as the connection is closed. The history service instead cannot afford to lose any messages, otherwise it would not fulfill its very purpose. Therefore, the queue we

are going to create for it has to be durable, so that any message that is published while the history service is disconnected will be kept in the queue and delivered when it comes back online.

Như thể hiện trong Hình 13.13, ta sẽ dùng một fanout exchange duy nhất; ta không cần bất kỳ logic định tuyến phức tạp nào, nên kịch bản của ta không đòi hỏi một exchange nào phức tạp hơn thế. Tiếp theo, ta sẽ tạo một queue cho mỗi instance của chat server. Các queue này là exclusive vì ta không quan tâm tới việc nhận bất kỳ thông điệp nào bị bỏ lỡ trong khi một chat server ngoại tuyến; đó là việc của history service, service này cuối cùng cũng có thể triển khai các truy vấn phức tạp hơn trên các thông điệp đã lưu. Trên thực tế, điều này nghĩa là các chat server của ta không phải là durable subscriber và queue của chúng sẽ bị hủy ngay khi kết nối bị đóng. Trái lại, history service không thể chịu mất bất kỳ thông điệp nào, nếu không nó sẽ không hoàn thành đúng mục đích của mình. Do đó, queue mà ta sẽ tạo cho nó phải là durable, để bất kỳ thông điệp nào được publish trong khi history service ngắt kết nối sẽ được giữ trong queue và chuyển giao khi nó trở lại trực tuyến.

We are going to use the familiar LevelUP as the storage engine for the history service, while we will use the amqplib package (nodejsdp.link/amqplib) to connect to RabbitMQ using the AMQP protocol.

Ta sẽ dùng LevelUP quen thuộc làm storage engine cho history service, còn ta sẽ dùng gói amqplib (nodejsdp.link/amqplib) để kết nối tới RabbitMQ bằng giao thức AMQP.

The example that follows requires a working RabbitMQ server, listening on its default port. For more information, please refer to its official installation guide at nodejsdp.link/rabbitmq-getstarted.

Ví dụ tiếp theo yêu cầu một RabbitMQ server đang hoạt động, lắng nghe trên cổng mặc định của nó. Để biết thêm thông tin, vui lòng tham khảo hướng dẫn cài đặt chính thức của nó tại nodejsdp.link/rabbitmq-getstarted.

Implementing a history service using AMQP — Triển khai một history service dùng AMQP

Let's now implement our history service! We are going to create a standalone application (a typical microservice), which is implemented in the `historySvc.js` **module**. The module is made up of two parts: an HTTP server to expose the chat history to clients, and an AMQP consumer responsible for capturing the chat messages and storing them in a local database.

Bây giờ hãy triển khai history service của ta! Ta sẽ tạo một ứng dụng độc lập (một microservice điển hình), được triển khai trong module `historySvc.js`. Module gồm hai phần: một HTTP server để phơi bày lịch sử chat tới client, và một AMQP consumer chịu trách nhiệm bắt các thông điệp chat và lưu chúng trong một database cục bộ.

Let's see what this looks like in the code that follows:

Hãy xem nó trông ra sao trong đoạn code sau:

```
import { createServer } from 'http'
import level from 'level'
import timestamp from 'monotonic-timestamp'
import JSONStream from 'JSONStream'
import amqp from 'amqplib'
async function main () {
  const db = level('./msgHistory')
  const connection = await amqp.connect('amqp://localhost') // (1)
  const channel = await connection.createChannel()
```



```

await channel.assertExchange('chat', 'fanout') // (2)
const { queue } = channel.assertQueue('chat_history') // (3)
await channel.bindQueue(queue, 'chat') // (4)
channel.consume(queue, async msg => { // (5)
  const content = msg.content.toString()
  console.log(`Saving message: ${content}`)
  await db.put(timestamp(), content)
  channel.ack(msg)
})
createServer((req, res) => {
  res.writeHead(200)
  db.createValueStream()
    .pipe(JSONStream.stringify())
    .pipe(res)
}).listen(8090)
}
main().catch(err => console.error(err))

```

We can immediately see that AMQP requires a little bit of setting up, which is necessary to create and connect all the components of the model. Let's see in detail how it works:

Ta có thể thấy ngay rằng AMQP đòi hỏi một chút thiết lập, điều cần thiết để tạo và kết nối tất cả các thành phần của mô hình. Hãy xem chi tiết cách nó hoạt động:

1. We first establish a connection with the AMQP broker, which in our case is RabbitMQ. Then, we create a channel, which is similar to a session that will maintain the state of our communications.
2. Next, we set up an exchange, named chat. As we already mentioned, it is a fanout exchange. The `assertExchange()` command will make sure that the exchange exists on the broker, otherwise it will create it.
3. We also create a queue called chat_history. By default, the queue is durable (not exclusive and not auto-delete), so we don't need to pass any extra options to support durable subscribers.
4. Next, we bind the queue to the exchange we previously created. Here, we don't need any other particular option (such as a routing key or pattern), as the exchange is of the type fanout, so it doesn't perform any filtering.
5. Finally, we can begin to listen for messages coming from the queue we just created. We save every message that we receive in a LevelDB database using a monotonic timestamp as the key (see nodejsdp.link/monotonic-timestamp) to keep the messages sorted by date. It's also interesting to see that we are acknowledging every message using `channel.ack(msg)`, but only after the message is successfully saved into the database. If the ACK (acknowledgment) is not received by the broker, the message is kept in the queue to be processed again.
 1. Trước tiên ta thiết lập một kết nối với AMQP broker, trong trường hợp của ta là RabbitMQ. Sau đó, ta tạo một channel, tương tự như một session sẽ duy trì trạng thái giao tiếp của ta.
 2. Tiếp theo, ta thiết lập một exchange, tên là chat. Như đã đề cập, nó là một fanout exchange. Lệnh `assertExchange()` sẽ đảm bảo rằng exchange tồn tại trên broker, nếu không nó sẽ tạo ra.
 3. Ta cũng tạo một queue tên là chat_history. Theo mặc định, queue là durable (không exclusive và không auto-delete), nên ta không cần truyền thêm tùy chọn nào để hỗ trợ durable subscriber.
 4. Tiếp theo, ta bind queue vào exchange mà ta đã tạo trước đó. Ở đây, ta không cần

bất kỳ tùy chọn cụ thể nào khác (như routing key hay mẫu), vì exchange thuộc loại fanout, nên nó không thực hiện việc lọc nào.

5. Cuối cùng, ta có thể bắt đầu lắng nghe các thông điệp đến từ queue mà ta vừa tạo. Ta lưu mỗi thông điệp nhận được vào một database LevelDB dùng một monotonic timestamp làm key (xem nodejsdp.link/monotonic-timestamp) để giữ các thông điệp được sắp xếp theo ngày. Cũng thú vị khi thấy rằng ta đang xác nhận mỗi thông điệp bằng `channel.ack(msg)`, nhưng chỉ sau khi thông điệp được lưu thành công vào database. Nếu ACK (acknowledgment) không được broker nhận, thông điệp sẽ được giữ trong queue để được xử lý lại.

If we are not interested in sending explicit acknowledgments, we can pass the `{ noAck: true }` option to the `channel.consume()` API.

Nếu ta không quan tâm tới việc gửi các acknowledgment tường minh, ta có thể truyền tùy chọn `{ noAck: true }` cho API `channel.consume()`.

Integrating the chat application with AMQP — Tích hợp ứng dụng chat với AMQP To integrate the chat servers using AMQP, we have to use a setup very similar to the one we implemented in the history service, but with some small variations. So, let's see how the new `index.js` module looks with the introduction of AMQP:

Để tích hợp các chat server bằng AMQP, ta phải dùng một thiết lập rất giống với thiết lập đã triển khai trong history service, nhưng với một vài biến đổi nhỏ. Vậy, hãy xem module `index.js` mới trông ra sao với sự bổ sung của AMQP:

```
import { createServer } from 'http'
import staticHandler from 'serve-handler'
import ws from 'ws'
import amqp from 'amqplib'
import JSONStream from 'JSONStream'
import superagent from 'superagent'
const httpPort = process.argv[2] || 8080
async function main () {
  const connection = await amqp.connect('amqp://localhost')
  const channel = await connection.createChannel()
  await channel.assertExchange('chat', 'fanout')
  const { queue } = await channel.assertQueue( // (1)
    `chat_srv_${httpPort}`,
    { exclusive: true }
  )
  await channel.bindQueue(queue, 'chat')
  channel.consume(queue, msg => { // (2)
    msg = msg.content.toString()
    console.log(`From queue: ${msg}`)
    broadcast(msg)
  }, { noAck: true })
  // serve static files
  const server = createServer((req, res) => {
    return staticHandler(req, res, { public: 'www' })
  })
  const wss = new ws.Server({ server })
  wss.on('connection', client => {
```

```

console.log('Client connected')
client.on('message', msg => {
  console.log(`Message: ${msg}`)
  channel.publish('chat', '', Buffer.from(msg)) // (3)
})
// query the history service
superagent // (4)
  .get('http://localhost:8090')
  .on('error', err => console.error(err))
  .pipe(JSONStream.parse('*'))
  .on('data', msg => client.send(msg))
})
function broadcast (msg) {
  for (const client of wss.clients) {
    if (client.readyState === ws.OPEN) {
      client.send(msg)
    }
  }
}
server.listen(httpPort)
}
main().catch(err => console.error(err))

```

As we can see, AMQP made the code a little bit more verbose on this occasion too, but at this point we should already be familiar with most of it. There are just a few aspects to be aware of:

Như ta thấy, AMQP cũng làm code dài dòng hơn một chút trong dịp này, nhưng đến giờ ta hẳn đã quen thuộc với hầu hết các phần. Chỉ có vài khía cạnh cần lưu ý:

1. As we mentioned, our chat server doesn't need to be a durable subscriber: a fire-and-forget paradigm is enough. So when we create our queue, we pass the { `exclusive: true` } option, indicating that the queue is scoped to the current connection and therefore it will be destroyed as soon as the chat server shuts down.
2. For the same reason as in the previous point, we don't need to send back any acknowledgement when we read a message from the queue. So, to make things easier, we pass the { `noAck: true` } option when starting to consume the messages from the queue.
3. Publishing a new message is also very easy. We simply have to specify the target exchange (chat) and a routing key, which in our case is empty (`'`) because we are using a fanout exchange, so there is no routing to perform.
4. The other peculiarity of this version of our chat server is that we can now present to the user the full history of the chat, thanks to our history microservice. We do that by querying the history microservice and sending every past message to the client as soon as a new connection is established.

1. Như đã đề cập, chat server của ta không cần là một durable subscriber: mô hình fire-and-forget là đủ. Vậy nên khi tạo queue, ta truyền tùy chọn { `exclusive: true` }, cho biết rằng queue được giới hạn trong phạm vi kết nối hiện tại và do đó sẽ bị hủy ngay khi chat server tắt.
2. Vì lý do tương tự như ở điểm trước, ta không cần gửi lại bất kỳ acknowledgment nào khi đọc một thông điệp từ queue. Vậy nên, để cho mọi thứ dễ dàng hơn, ta truyền tùy chọn { `noAck: true` } khi bắt đầu tiêu thụ các thông điệp từ queue.
3. Publish một thông điệp mới cũng rất dễ. Ta chỉ cần chỉ định exchange đích (chat) và

một routing key, mà trong trường hợp của ta là rỗng (' ') vì ta đang dùng một fanout exchange, nên không có việc định tuyến nào để thực hiện.

4. Điểm đặc thù khác của phiên bản chat server này là giờ ta có thể trình bày cho người dùng toàn bộ lịch sử chat, nhờ history microservice. Ta làm điều đó bằng cách truy vấn history microservice và gửi mỗi thông điệp trong quá khứ tới client ngay khi một kết nối mới được thiết lập.

We can now run our new improved chat application. To do that, first make sure to have RabbitMQ running locally on your machine, then let's start two chat servers and the history service in three different terminals:

Bây giờ ta có thể chạy ứng dụng chat mới được cải tiến của mình. Để làm điều đó, trước tiên hãy đảm bảo RabbitMQ đang chạy cục bộ trên máy của bạn, rồi hãy khởi chạy hai chat server và history service trong ba terminal khác nhau:

```
node index.js 8080
node index.js 8081
node historySvc.js
```

We should now focus our attention on how our system, and in particular the history service, behaves in case of downtime. If we stop the history server and continue to send messages using the web UI of the chat application, we will see that when the history server is restarted, it will immediately receive all the messages it missed. This is a perfect demonstration of how the Durable Subscriber pattern works!

Bây giờ ta nên tập trung sự chú ý vào cách hệ thống của ta, và đặc biệt là history service, hành xử trong trường hợp downtime. Nếu ta dừng history server và tiếp tục gửi thông điệp bằng web UI của ứng dụng chat, ta sẽ thấy rằng khi history server được khởi động lại, nó sẽ ngay lập tức nhận tất cả các thông điệp mà nó đã bỏ lỡ. Đây là một minh chứng hoàn hảo về cách mẫu Durable Subscriber hoạt động!

It is interesting to see how the microservice approach allows our system to survive even without one of its components—the history service. There would be a temporary reduction of functionality (no chat history available) but people would still be able to exchange chat messages in real time. Awesome!

Thú vị khi thấy cách tiếp cận microservice cho phép hệ thống của ta sống sót ngay cả khi thiếu một trong các thành phần của nó — history service. Sẽ có một sự giảm sút chức năng tạm thời (không có lịch sử chat khả dụng) nhưng mọi người vẫn có thể trao đổi thông điệp chat theo thời gian thực. Tuyệt vời!

Reliable messaging with streams — Nhắn tin đáng tin cậy với stream

At the beginning of this chapter, we mentioned that a possible alternative to message queues are streams. The two paradigms are similar in scope, but fundamentally different in their approach to messaging. In this section, we are going to unveil the power of streams by leveraging Redis Streams to implement our chat application.

Ở đầu chương này, ta đã đề cập rằng một giải pháp thay thế khả dĩ cho message queue là stream. Hai mô hình này tương tự nhau về phạm vi, nhưng khác biệt căn bản trong cách tiếp cận việc nhắn tin. Trong phần này, ta sẽ hé lộ sức mạnh của stream bằng cách tận dụng Redis Streams để triển khai ứng dụng chat của mình.

Characteristics of a streaming platform — Đặc tính của một nền tảng streaming

In the context of system integration, a stream (or log) is an ordered, append-only, durable data structure. Messages—which in the context of streams would be more appropriately called records—are always added at the end of the stream and, unlike queues, they are not automatically deleted when they are consumed. Essentially, this characteristic makes a stream more similar to a data store than to a message broker. And like a data store, a stream can be queried to retrieve a batch of past records or replayed starting from a specific record. Another important characteristic of streams is that records are pulled by the consumer from the stream. This intrinsically allows the consumer to process the records at its own pace without risking being overwhelmed.

Trong ngữ cảnh tích hợp hệ thống, một stream (hay log) là một cấu trúc dữ liệu có thứ tự, chỉ-thêm (append-only) và bền vững (durable). Các thông điệp — mà trong ngữ cảnh stream sẽ được gọi đúng hơn là các record (bản ghi) — luôn được thêm vào cuối stream và, không giống queue, chúng không tự động bị xóa khi được tiêu thụ. Về cơ bản, đặc tính này khiến một stream giống một data store hơn là một message broker. Và giống như một data store, một stream có thể được truy vấn để lấy một lô (batch) các record trong quá khứ, hoặc được phát lại (replay) bắt đầu từ một record cụ thể. Một đặc tính quan trọng khác của stream là các record được consumer kéo (pull) từ stream. Điều này về bản chất cho phép consumer xử lý các record theo nhịp độ của riêng nó mà không có nguy cơ bị quá tải.

Based on these features, a stream allows us to implement reliable message delivery out of the box, since no data is ever lost from the stream (even though data can still be removed explicitly or can be deleted after an optional retention period). In fact, as Figure 13.14 shows, if a consumer crashes, all it has to do is start reading the stream from where it left off:

Dựa trên những tính năng này, một stream cho phép ta triển khai việc chuyển giao thông điệp đáng tin cậy ngay từ đầu (out of the box), vì không có dữ liệu nào bị mất khỏi stream (mặc dù dữ liệu vẫn có thể bị xóa tường minh hoặc có thể bị xóa sau một khoảng thời gian giữ lại (retention period) tùy chọn). Thực tế, như Hình 13.14 cho thấy, nếu một consumer crash, tất cả những gì nó phải làm là bắt đầu đọc stream từ chỗ nó đã dừng:

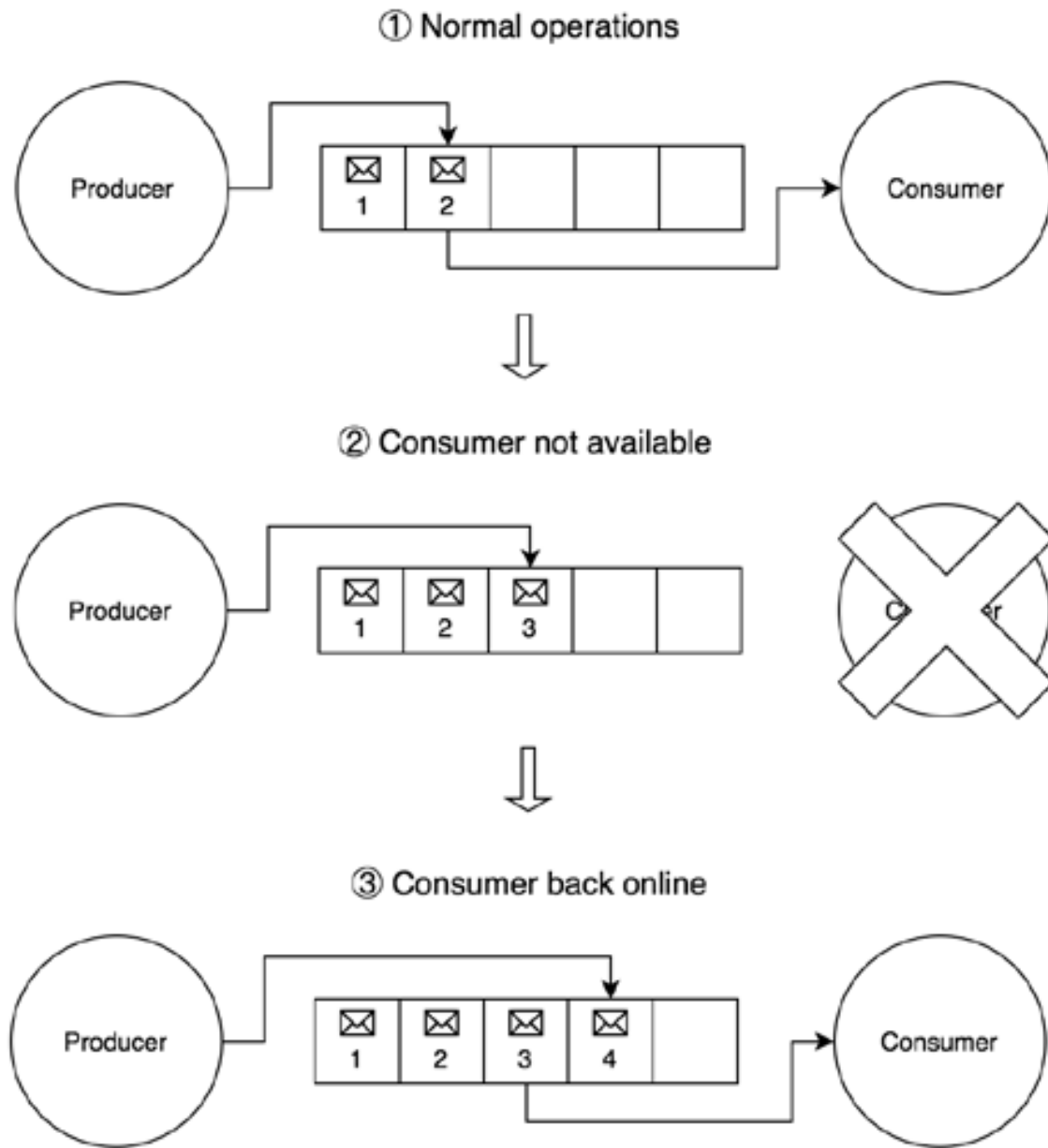


Figure 13.14: Reliable message delivery with streams

Hình 13.14: Chuyển giao thông điệp đáng tin cậy với stream

As Figure 13.14 shows, during normal operations (1) the consumer processes the records in the stream as soon as they are added by the producer. When the consumer becomes unavailable (2) because of a problem or a scheduled maintenance, the producer simply continues to add records to the stream as normal. When the consumer comes back online (3), it starts processing the records from the point where it left. The main aspect of this mechanism is that it's very simple and barebone, but it's quite effective at making sure that no message is lost even when the consumer is not available.

Như Hình 13.14 cho thấy, trong quá trình hoạt động bình thường (1) consumer xử lý các record trong stream ngay khi chúng được producer thêm vào. Khi consumer không khả dụng (2) vì một vấn đề hay một kỳ bảo trì theo lịch, producer chỉ đơn giản tiếp tục thêm

record vào stream như bình thường. Khi consumer trở lại trực tuyến (3), nó bắt đầu xử lý các record từ điểm nó đã dừng lại. Khía cạnh chính của cơ chế này là nó rất đơn giản và sơ khai (barebone), nhưng nó khá hiệu quả trong việc đảm bảo không có thông điệp nào bị mất ngay cả khi consumer không khả dụng.

Streams versus message queues — Stream so với message queue

As we have seen so far, there are a lot of differences, but also a lot of similarities between a message queue and a stream. So, when should you use one in place of the other?

Như ta đã thấy cho tới giờ, có rất nhiều khác biệt, nhưng cũng có rất nhiều điểm tương đồng giữa một message queue và một stream. Vậy, khi nào ta nên dùng cái này thay cho cái kia?

Well, the obvious use case for streams is when we have to process sequential data (streaming data) that may also require the consumer to process messages in batch or to look for correlations in past messages. Also, modern streaming platforms allow the ingestion of gigabytes of data per second and the distribution of both the data and the processing of the data across multiple nodes.

Vâng, trường hợp sử dụng hiển nhiên cho stream là khi ta phải xử lý dữ liệu tuần tự (streaming data) mà cũng có thể đòi hỏi consumer xử lý các thông điệp theo lô (batch) hoặc tìm các mối tương quan trong các thông điệp quá khứ. Ngoài ra, các nền tảng streaming hiện đại cho phép thu nạp (ingest) hàng gigabyte dữ liệu mỗi giây và phân phối cả dữ liệu lẫn việc xử lý dữ liệu qua nhiều node.

Both message queues and streams are well suited to implement simple Publish/Subscribe patterns, even with reliable message delivery. However, message queues are better suited for complex system integration tasks, since they provide advanced routing of messages and allow us to have different priorities for different messages (in streams, the order of the records is always preserved).

Cả message queue lẫn stream đều phù hợp để triển khai các mẫu Publish/Subscribe đơn giản, kể cả với chuyển giao thông điệp đáng tin cậy. Tuy nhiên, message queue phù hợp hơn cho các tác vụ tích hợp hệ thống phức tạp, vì chúng cung cấp việc định tuyến thông điệp nâng cao và cho phép ta có các mức ưu tiên khác nhau cho các thông điệp khác nhau (trong stream, thứ tự của các record luôn được bảo toàn).

As we will see later, both can also be used to implement task distribution patterns, even though, in a standard architecture, message queues could be more suitable thanks to message priorities and more advanced routing mechanisms.

Như ta sẽ thấy về sau, cả hai cũng có thể được dùng để triển khai các mẫu phân phối tác vụ, mặc dù, trong một kiến trúc tiêu chuẩn, message queue có thể phù hợp hơn nhờ các mức ưu tiên thông điệp và các cơ chế định tuyến nâng cao hơn.

Implementing the chat application using Redis Streams — Triển khai ứng dụng chat dùng Redis Streams

At the moment of writing, the most popular streaming platforms out there are Apache Kafka (nodejsdp.link/kafka) and Amazon Kinesis (nodejsdp.link/kinesis). However, for simpler tasks, we can rely again on Redis, which implements a log data structure called Redis Streams.

Tại thời điểm viết bài, các nền tảng streaming phổ biến nhất ngoài kia là Apache Kafka (nodejsdp.link/kafka) và Amazon Kinesis (nodejsdp.link/kinesis). Tuy nhiên, với các tác vụ đơn giản hơn, ta lại có thể dựa vào Redis, vốn triển khai một cấu trúc dữ liệu log gọi là Redis Streams.

In the next code sample, we are going to see Redis Streams in action by adapting our chat application. The immediate advantage of using a stream over a message queue is that we don't need to rely on a dedicated component to store and retrieve the history of the messages exchanged in a chat room, but we can simply query the stream every time we need to access older messages. As we will see, this simplifies a lot the architecture of our application and certainly makes streams a better choice than message queues, at least for our very simple use case. So, let's dive into some code. Let's update the `index.js` of our chat application to use Redis Streams:

Trong mẫu code tiếp theo, ta sẽ thấy Redis Streams trong hành động bằng cách điều chỉnh ứng dụng chat của mình. Lợi thế tức thì của việc dùng stream thay vì message queue là ta không cần dựa vào một thành phần chuyên dụng để lưu và lấy lại lịch sử các thông điệp được trao đổi trong một phòng chat, mà ta chỉ cần truy vấn stream mỗi khi ta cần truy cập các thông điệp cũ hơn. Như ta sẽ thấy, điều này đơn giản hóa rất nhiều kiến trúc của ứng dụng và chắc chắn khiến stream trở thành lựa chọn tốt hơn message queue, ít nhất là cho trường hợp sử dụng rất đơn giản của ta. Vậy, hãy đi sâu vào một ít code. Hãy cập nhật `index.js` của ứng dụng chat để dùng Redis Streams:

```
import { createServer } from 'http'
import staticHandler from 'serve-handler'
import ws from 'ws'
import Redis from 'ioredis'
const redisClient = new Redis()
const redisClientXRead = new Redis()
// serve static files
const server = createServer((req, res) => {
  return staticHandler(req, res, { public: 'www' })
})
const wss = new ws.Server({ server })
wss.on('connection', async client => {
  console.log('Client connected')
  client.on('message', msg => {
    console.log(`Message: ${msg}`)
    redisClient.xadd('chat_stream', '*', 'message', msg) // (1)
  })
  // Load message history
  const logs = await redisClient.xrange('chat_stream', '-', '+') // (2)
  for (const [, [, message]] of logs) {
    client.send(message)
  }
})
function broadcast (msg) {
  for (const client of wss.clients) {
    if (client.readyState === ws.OPEN) {
      client.send(msg)
    }
  }
}
let lastRecordId = '$'
async function processStreamMessages () { // (3)
  while (true) {
```

```

const [[, records]] = await redisClientXRead.xread(
  'BLOCK', '0', 'STREAMS', 'chat_stream', lastRecordId)
for (const [recordId, [, message]] of records) {
  console.log(`Message from stream: ${message}`)
  broadcast(message)
  lastRecordId = recordId
}
}
}
processStreamMessages().catch(err => console.error(err))
server.listen(process.argv[2] || 8080)

```

As always, the overall structure of the application has remained the same; what changed is the API we used to exchange messages with the other instances of the application.

Như mọi khi, cấu trúc tổng thể của ứng dụng vẫn giữ nguyên; cái thay đổi là API mà ta dùng để trao đổi thông điệp với các instance khác của ứng dụng.

Let's take a look at those APIs more closely:

Hãy xem các API đó kỹ hơn:

1. The first command we want to analyze is `xadd`. This command appends a new record to a stream, and we are using it to add a new chat message as it arrives from a connected client. We pass to `xadd` the following arguments: a. The name of the stream, which in our case is `chat_stream`. b. The ID of the record. In our case, we provide an asterisk (*), which is a special ID that asks Redis to generate an ID for us. This is usually what we want, as IDs have to be monotonic to preserve the lexicographic order of the records and Redis takes care of that for us. c. It follows a list of key-value pairs. In our case, we specify only a 'message' key of the value `msg` (which is the message we receive from the client).
2. This is one of the most interesting aspects of using streams: we query the past records of the stream to retrieve the chat history. We do this every time a client connects. We use the `xrange` command for that, which, as the name implies, allows us to retrieve all the records in the stream within the two specified IDs. In our case we are using the special IDs '-' (minus) and '+' (plus) which indicate the lowest possible ID and the highest possible ID. This essentially means that we want to retrieve all the records currently in the stream.
3. The last interesting part of our new chat application is where we wait for new records to be added to the stream. This allows each application instance to read new chat messages as they are added into the queue, and it's an essential part for the integration to work. We use an infinite loop and the `xread` command for the task, providing the following arguments: a. `BLOCK` means that we want the call to block until new messages arrive. b. Next, we specify the timeout after which the command will simply return with a null result. In our case, `0` means that we want to wait forever. c. `STREAMS` is a keyword that tells Redis that we are now going to specify the details of the streams we want to read. d. `chat_stream` is the name of the stream we want to read. e. Finally, we supply the record ID (`lastRecordId`) after which we want to start reading the new messages. Initially, this is set to `$` (dollar sign), which is a special ID indicating the highest ID currently in the stream, which should essentially start to read the stream after the last record currently in the stream. After we read the first record, we update the `lastRecordId` variable with the ID of the last record read.

1. **Lệnh đầu tiên mà ta muốn phân tích là `xadd`.** **Lệnh này thêm một record mới vào một stream, và ta đang dùng nó để thêm một thông điệp chat mới khi nó đến từ một client đang kết nối.** Ta truyền cho `xadd` các đối số sau: a. Tên của stream, trong trường hợp

của ta là `chat_stream`. b. ID của record. Trong trường hợp của ta, ta cung cấp một dấu hoa thị (*), là một ID đặc biệt yêu cầu Redis tạo một ID cho ta. Đây thường là điều ta muốn, vì các ID phải đơn điệu (monotonic) để bảo toàn thứ tự từ điển (lexicographic) của các record và Redis lo điều đó cho ta. c. Theo sau là một danh sách các cặp key-value. Trong trường hợp của ta, ta chỉ chỉ định một key 'message' với giá trị msg (là thông điệp ta nhận từ client).

2. Đây là một trong những khía cạnh thú vị nhất của việc dùng stream: ta truy vấn các record trong quá khứ của stream để lấy lại lịch sử chat. Ta làm điều này mỗi khi một client kết nối. Ta dùng lệnh `xrange` cho việc đó, lệnh này, đúng như tên gọi, cho phép ta lấy lại tất cả các record trong stream nằm trong khoảng giữa hai ID được chỉ định. Trong trường hợp của ta, ta dùng các ID đặc biệt '-' (dấu trừ) và '+' (dấu cộng) chỉ ID thấp nhất có thể và ID cao nhất có thể. Điều này về cơ bản nghĩa là ta muốn lấy lại tất cả các record hiện có trong stream.
3. Phần thú vị cuối cùng của ứng dụng chat mới của ta là nơi ta chờ các record mới được thêm vào stream. Điều này cho phép mỗi instance ứng dụng đọc các thông điệp chat mới khi chúng được thêm vào queue, và đó là phần thiết yếu để việc tích hợp hoạt động. Ta dùng một vòng lặp vô hạn và lệnh `xread` cho nhiệm vụ này, cung cấp các đối số sau: a. BLOCK nghĩa là ta muốn lờ gọi chặn (block) cho tới khi các thông điệp mới đến. b. Tiếp theo, ta chỉ định timeout, sau đó lệnh sẽ đơn giản trả về với kết quả null. Trong trường hợp của ta, 0 nghĩa là ta muốn chờ mãi mãi. c. STREAMS là một từ khóa báo cho Redis biết rằng ta sắp chỉ định chi tiết của các stream mà ta muốn đọc. d. `chat_stream` là tên của stream mà ta muốn đọc. e. Cuối cùng, ta cung cấp ID của record (`lastRecordId`) mà sau đó ta muốn bắt đầu đọc các thông điệp mới. Ban đầu, cái này được đặt thành '\$' (dấu đô la), một ID đặc biệt chỉ ID cao nhất hiện có trong stream, về cơ bản sẽ bắt đầu đọc stream sau record cuối cùng hiện có trong stream. Sau khi ta đọc record đầu tiên, ta cập nhật biến `lastRecordId` với ID của record cuối cùng đã đọc.

Within the previous example, we also made use of some clever destructuring instructions. Consider for example the following code:

Trong ví dụ trước, ta cũng đã sử dụng một số chỉ thị destructuring khéo léo. Hãy xem ví dụ đoạn code sau:

```
for (const [, [, message]] of logs) {...}
```

This instruction could be expanded to something like the following:

Chỉ thị này có thể được khai triển thành dạng tương tự như sau:

```
for (const [recordId, [propertyId, message]] of logs) {...}
```

But since we are not interested in getting the `recordId` and the `propertyId`, we are simply keeping them out of the destructuring instruction. This particular destructuring, in combination with the `for...of` loop, is necessary to parse the data returned from the `xrange` command, which in our case is in the following form:

Nhưng vì ta không quan tâm tới việc lấy `recordId` và `propertyId`, ta chỉ đơn giản để chúng ra ngoài chỉ thị destructuring. Kiểu destructuring đặc biệt này, kết hợp với vòng lặp `for...of` loop, là cần thiết để phân tích dữ liệu trả về từ lệnh `xrange`, mà trong trường hợp của ta có dạng sau:

```
[  
  ["1588590110918-0", ["message", "This is a message"]],  
  ...  
]
```

```
[["1588590130852-0", ["message", "This is another message"]]]
```

We applied a similar principle to parse the return value of `xread`. Please refer to the API documentation of those instructions for a detailed explanation of their return value.

Ta áp dụng một nguyên tắc tương tự để phân tích giá trị trả về của `xread`. Vui lòng tham khảo tài liệu API của các chỉ thị đó để có giải thích chi tiết về giá trị trả về của chúng.

You can read more about the `xadd` command and the format of record IDs in the official Redis documentation at nodejsdp.link/xadd. The `xread` command has also a fairly complicated arguments list and return value that you can read more about at nodejsdp.link/xread. Also, check out the documentation for `xrange` at nodejsdp.link/xrange.

Bạn có thể đọc thêm về lệnh `xadd` và định dạng của các record ID trong tài liệu Redis chính thức tại nodejsdp.link/xadd. Lệnh `xread` cũng có một danh sách đối số và giá trị trả về khá phức tạp mà bạn có thể đọc thêm tại nodejsdp.link/xread. Ngoài ra, hãy xem tài liệu cho `xrange` tại nodejsdp.link/xrange.

Now, you can start a couple of server instances again and test the application to see how the new implementation works.

Bây giờ, bạn có thể khởi chạy lại vài instance server và kiểm thử ứng dụng để xem bản triển khai mới hoạt động ra sao.

It's interesting to highlight again the fact that we didn't need to rely on a dedicated component to manage our chat history, but instead, all we needed to do was to retrieve the past records from the stream with `xrange`. This aspect of streams makes them intrinsically reliable as no message is lost unless explicitly deleted.

Thú vị khi nhấn mạnh lại một lần nữa thực tế rằng ta không cần dựa vào một thành phần chuyên dụng để quản lý lịch sử chat của mình, mà thay vào đó, tất cả những gì ta cần làm là lấy lại các record trong quá khứ từ stream bằng `xrange`. Khía cạnh này của stream khiến chúng vốn dĩ đáng tin cậy vì không có thông điệp nào bị mất trừ khi bị xóa tường minh.

Records can be removed from the stream with the `xdel` (nodejsdp.link/xdel) or `xtrim` (nodejsdp.link/xtrim) or with the `MAXLEN` option of `xadd` (nodejsdp.link/xadd-maxlen).

Các record có thể được loại bỏ khỏi stream bằng lệnh `xdel` (nodejsdp.link/xdel) hoặc `xtrim` (nodejsdp.link/xtrim) hoặc bằng tùy chọn `MAXLEN` của `xadd` (nodejsdp.link/xadd-maxlen).

This concludes our exploration of the Publish/Subscribe pattern. Now, it's time to discover another important category of messaging patterns: task distribution patterns.

Điều này khép lại việc khám phá mẫu Publish/Subscribe của chúng ta. Bây giờ, đến lúc khám phá một loại mẫu nhắn tin quan trọng khác: các mẫu phân phối tác vụ (task distribution patterns).

Task distribution patterns — Các mẫu phân phối tác vụ

In Chapter 11, Advanced Recipes, you learned how to delegate costly tasks to multiple local processes. Even though this was an effective approach, it cannot be scaled beyond the boundaries of a single

machine, so in this section, we are going to see how it's possible to use a similar pattern in a distributed architecture, using remote workers located anywhere in a network.

Trong Chương 11, Advanced Recipes, bạn đã học cách ủy thác các tác vụ tốn kém cho nhiều tiến trình cục bộ. Mặc dù đây là một cách tiếp cận hiệu quả, nó không thể mở rộng vượt ra ngoài ranh giới của một máy duy nhất, nên trong phần này, ta sẽ xem làm thế nào có thể dùng một mẫu tương tự trong một kiến trúc phân tán, sử dụng các remote worker nằm ở bất kỳ đâu trong một mạng.

The idea is to have a messaging pattern that allows us to spread tasks across multiple machines. These tasks might be individual chunks of work or pieces of a bigger task split using a divide and conquer approach. If we look at the logical architecture represented in the following diagram, we should be able to recognize a familiar pattern:

Ý tưởng là có một mẫu nhắn tin cho phép ta rải các tác vụ ra nhiều máy. Các tác vụ này có thể là các phần công việc riêng lẻ hoặc các mảnh của một tác vụ lớn hơn được chia nhỏ bằng cách tiếp cận chia để trị (divide and conquer). Nếu ta nhìn vào kiến trúc logic được biểu diễn trong sơ đồ sau, ta sẽ nhận ra một mẫu quen thuộc:

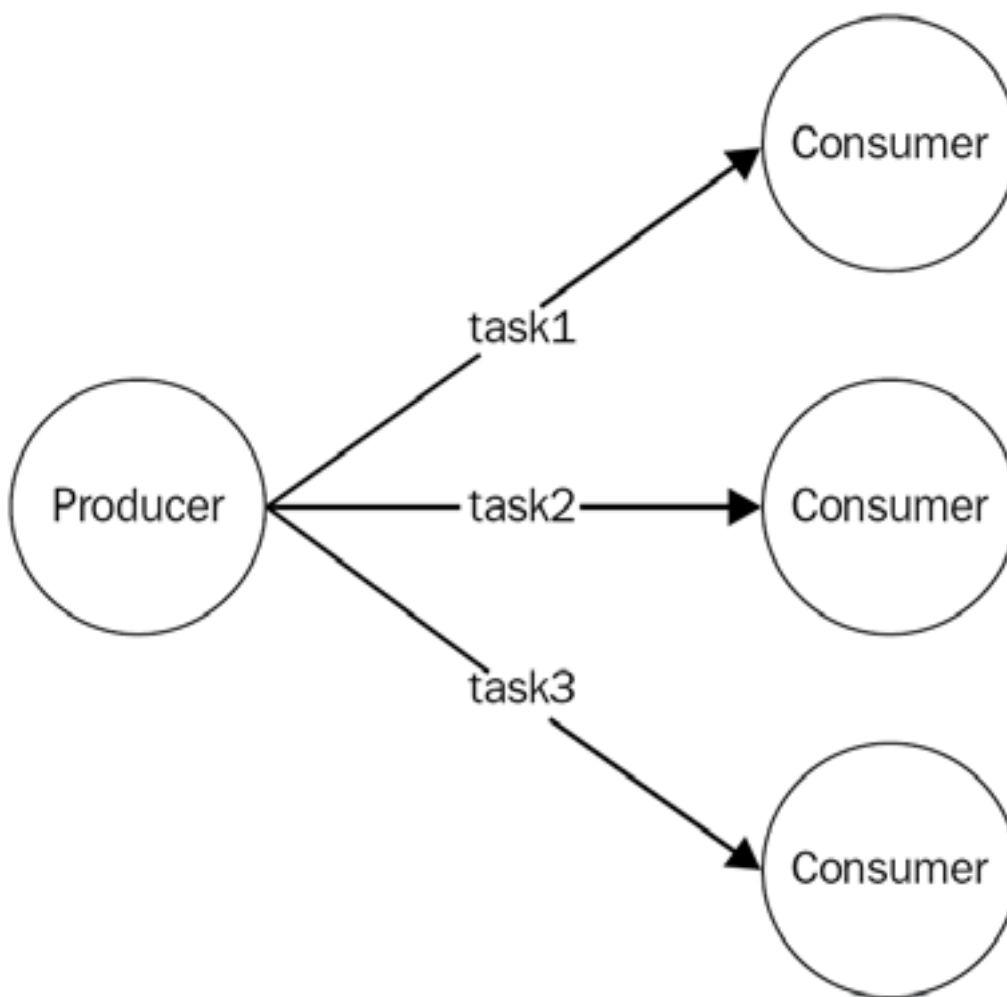


Figure 13.15: Distributing tasks to a set of consumers

Hình 13.15: Phân phối tác vụ tới một tập các consumer

As we can see from the diagram of Figure 13.15, the Publish/Subscribe pattern is not suitable for this type of application, as we absolutely don't want a task to be received by multiple workers. What we need instead, is a message distribution pattern similar to a load balancer that dispatches each message to a different consumer (also called a worker, in this case). In messaging systems terminology, this pattern is also known as competing consumers, fanout distribution, or ventilator.

Như ta thấy từ sơ đồ của Hình 13.15, mẫu Publish/Subscribe không phù hợp cho loại ứng dụng này, vì ta tuyệt đối không muốn một tác vụ được nhận bởi nhiều worker. Thay vào đó, cái ta cần là một mẫu phân phối thông điệp tương tự như một load balancer, phân phát mỗi thông điệp tới một consumer khác nhau (trong trường hợp này còn gọi là một worker). Trong thuật ngữ của các messaging system, mẫu này còn được gọi là competing consumers, fanout distribution, hay ventilator.

One important difference to the HTTP load balancers that we saw in the previous chapter is that, here, the consumers have a more active role. In fact, as we will see later, most of the time it's not the producer that connects to the consumers, but the consumers themselves that connect to the task producer or to the task queue in order to receive new jobs. This is a great advantage in a scalable system as it allows us to seamlessly increase the number of workers without modifying the producer or adopting a service registry.

Một khác biệt quan trọng so với các HTTP load balancer mà ta đã thấy ở chương trước là, ở đây, các consumer có một vai trò chủ động hơn. Thực tế, như ta sẽ thấy về sau, hầu hết thời gian không phải producer kết nối tới các consumer, mà chính các consumer kết nối tới task producer hoặc tới task queue để nhận các công việc mới. Đây là một lợi thế lớn trong một hệ thống có khả năng mở rộng, vì nó cho phép ta tăng số lượng worker một cách liền mạch mà không cần sửa đổi producer hay áp dụng một service registry.

Also, in a generic messaging system, we don't necessarily have request/reply communication between the producer and the workers. Instead, most of the time, the preferred approach is to use one-way asynchronous communication, which enables better parallelism and scalability. In such an architecture, messages can potentially always travel in one direction, creating pipelines, as shown in the following diagram:

Ngoài ra, trong một messaging system tổng quát, ta không nhất thiết phải có giao tiếp request/reply giữa producer và các worker. Thay vào đó, hầu hết thời gian, cách tiếp cận được ưa chuộng là dùng giao tiếp bất đồng bộ một chiều, điều này cho phép tính song song và khả năng mở rộng tốt hơn. Trong một kiến trúc như vậy, các thông điệp có thể luôn di chuyển theo một hướng, tạo ra các pipeline, như thể hiện trong sơ đồ sau:

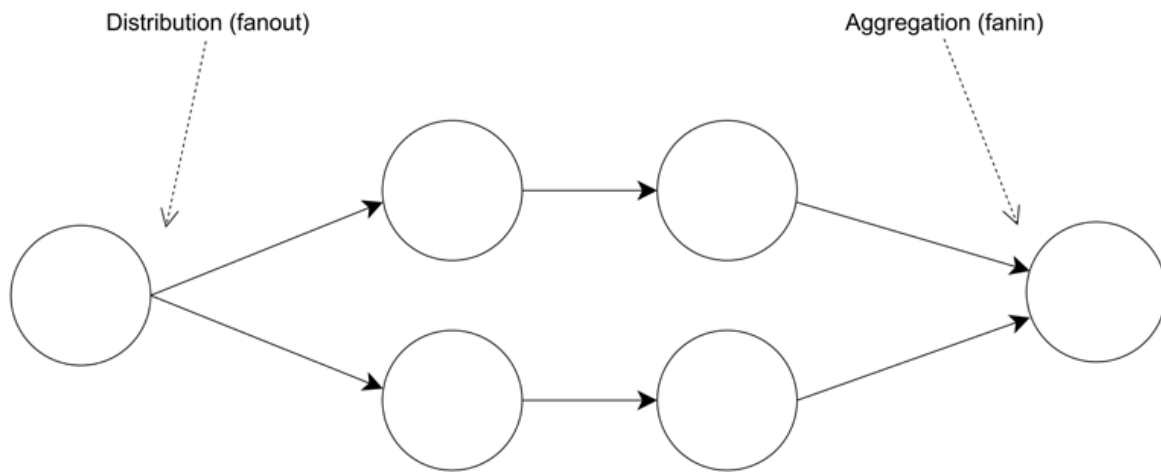


Figure 13.16: A messaging pipeline

Hình 13.16: Một pipeline nhắn tin

Pipelines allow us to build very complex processing architectures without the overhead of a synchronous request/reply communication, often resulting in lower latency and higher throughput. In Figure 13.16, we can see how messages can be distributed across a set of workers (fanout), forwarded to other processing units, and then aggregated into a single node (fanin), usually called the sink.

Pipeline cho phép ta xây dựng các kiến trúc xử lý rất phức tạp mà không có overhead của một giao tiếp request/reply đồng bộ, thường dẫn tới độ trễ thấp hơn và thông lượng (throughput) cao hơn. Trong Hình 13.16, ta có thể thấy các thông điệp có thể được phân phối qua một tập các worker (fanout), chuyển tiếp tới các đơn vị xử lý khác, rồi tổng hợp vào một node duy nhất (fanin), thường được gọi là sink.

In this section, we are going to focus on the building blocks of these kinds of architectures, by analyzing the two most important variations: peer-to-peer and broker-based.

Trong phần này, ta sẽ tập trung vào các khối xây dựng của những loại kiến trúc này, bằng cách phân tích hai biến thể quan trọng nhất: peer-to-peer và dựa trên broker.

The combination of a pipeline with a task distribution pattern is also called a parallel pipeline.

Sự kết hợp của một pipeline với một mẫu phân phối tác vụ còn được gọi là một parallel pipeline (pipeline song song).

The ZeroMQ Fanout/Fanin pattern — Mẫu Fanout/Fanin của ZeroMQ

We have already discovered some of the capabilities of ZeroMQ for building peer-to-peer distributed architectures. In the previous section, in fact, we used PUB and SUB sockets to disseminate a single message to multiple consumers, and now, we are going to see how it's possible to build parallel pipelines using another pair of sockets called PUSH and PULL.

Ta đã khám phá một số khả năng của ZeroMQ cho việc xây dựng các kiến trúc phân tán peer-to-peer. Thực tế, ở phần trước, ta đã dùng các socket PUB và SUB để phát tán một thông

điệp duy nhất tới nhiều consumer, và bây giờ, ta sẽ xem làm thế nào có thể xây dựng các parallel pipeline bằng một cặp socket khác gọi là PUSH và PULL.

PUSH/PULL sockets — Các socket PUSH/PULL

Intuitively, we can say that the PUSH sockets are made for sending messages, while the PULL sockets are meant for receiving. It might seem a trivial combination, however, they have some extra features that make them perfect for building one-way communication systems:

Một cách trực giác, ta có thể nói rằng các socket PUSH được tạo ra để gửi thông điệp, còn các socket PULL để nhận. Điều này có vẻ là một sự kết hợp tầm thường, tuy nhiên, chúng có một số tính năng bổ sung khiến chúng hoàn hảo cho việc xây dựng các hệ thống giao tiếp một chiều:

- Both can work in connect mode or bind mode. In other words, we can create a PUSH socket and bind it to a local port listening for the incoming connections from a PULL socket, or vice versa, a PULL socket might listen for connections from a PUSH socket. The messages always travel in the same direction, from PUSH to PULL, it's only the initiator of the connection that can be different. The bind mode is the best solution for durable nodes, such as, for example, the task producer and the sink, while the connect mode is perfect for transient nodes, such as the task workers. This allows the number of transient nodes to vary arbitrarily without affecting the more stable, durable nodes.
- If there are multiple PULL sockets connected to a single PUSH socket, the messages are evenly distributed across all the PULL sockets. In practice, they are load balanced (peer-to-peer load balancing!). On the other hand, a PULL socket that receives messages from multiple PUSH sockets will process the messages using a fair queuing system, which means that they are consumed evenly from all the sources—a round-robin applied to inbound messages.
- The messages sent over a PUSH socket that doesn't have any connected PULL sockets do not get lost. They are instead queued until a node comes online and starts pulling the messages.
 - Cả hai đều có thể hoạt động ở chế độ connect hoặc chế độ bind. Nói cách khác, ta có thể tạo một socket PUSH và bind nó vào một cổng cục bộ để lắng nghe các kết nối đến từ một socket PULL, hoặc ngược lại, một socket PULL có thể lắng nghe các kết nối từ một socket PUSH. Các thông điệp luôn di chuyển theo cùng một hướng, từ PUSH tới PULL, chỉ có bên khởi tạo kết nối là có thể khác. Chế độ bind là giải pháp tốt nhất cho các node bền vững (durable), chẳng hạn như task producer và sink, trong khi chế độ connect là hoàn hảo cho các node tạm thời (transient), chẳng hạn như các task worker. Điều này cho phép số lượng node tạm thời thay đổi tùy ý mà không ảnh hưởng tới các node ổn định, bền vững hơn.
 - Nếu có nhiều socket PULL kết nối tới một socket PUSH duy nhất, các thông điệp được phân phối đều qua tất cả các socket PULL. Trên thực tế, chúng được load balance (cân bằng tải peer-to-peer!). Mặt khác, một socket PULL nhận thông điệp từ nhiều socket PUSH sẽ xử lý các thông điệp bằng một hệ thống fair queuing (xếp hàng công bằng), nghĩa là chúng được tiêu thụ đều từ tất cả các nguồn — một kiểu round-robin áp dụng cho các thông điệp đến.
 - Các thông điệp được gửi qua một socket PUSH không có socket PULL nào kết nối thì không bị mất. Thay vào đó, chúng được xếp hàng cho tới khi một node trực tuyến và bắt đầu kéo (pull) các thông điệp.

We are now starting to understand how ZeroMQ is different from traditional web services and why it's a perfect tool for building a distributed messaging system.

Bây giờ ta bắt đầu hiểu cách ZeroMQ khác với các web service truyền thống và tại sao nó

là một công cụ hoàn hảo để xây dựng một messaging system phân tán.

Building a distributed hashsum cracker with ZeroMQ — Xây dựng một bộ phá hashsum phân tán với ZeroMQ

Now it's time to build a sample application to see the properties of the PUSH/ PULL sockets we just described in action.

Bây giờ đến lúc xây dựng một ứng dụng mẫu để thấy các đặc tính của các socket PUSH/PULL mà ta vừa mô tả trong hành động.

A simple and fascinating application to work with would be a hashsum cracker: A system that uses a brute-force approach to try to match a given hashsum (such as MD5 or SHA1) to the hashsum of every possible variation of characters of a given alphabet, thus discovering the original string the given hashsum was created from. This is an embarrassingly parallel workload (nodejsdp.link/embarrassingly-parallel), which is perfect for building an example demonstrating the power of parallel pipelines.

Một ứng dụng đơn giản và hấp dẫn để thử nghiệm sẽ là một bộ phá hashsum: Một hệ thống dùng cách tiếp cận brute-force để cố khớp một hashsum cho trước (như MD5 hay SHA1) với hashsum của mọi biến thể có thể của các ký tự thuộc một bảng chữ cái cho trước, từ đó tìm ra chuỗi gốc mà hashsum đã cho được tạo ra từ đó. Đây là một khối công việc song song một cách hiển nhiên ([embarrassingly parallel](https://nodejsdp.link/embarrassingly-parallel)) (nodejsdp.link/embarrassingly-parallel), hoàn hảo để xây dựng một ví dụ minh họa sức mạnh của các parallel pipeline.

Never use plain hashsums to encrypt passwords as they are very easy to crack. Use instead a purpose-built algorithm such as `bcrypt` (nodejsdp.link/bcrypt), `scrypt` (nodejsdp.link/scrypt), `PBKDF2` (nodejsdp.link/pbkdf2), or `Argon2` (nodejsdp.link/argon2).

Đừng bao giờ dùng hashsum thuần để mã hóa mật khẩu vì chúng rất dễ bị phá. Thay vào đó hãy dùng một thuật toán được xây dựng riêng cho mục đích đó như `bcrypt` (nodejsdp.link/bcrypt), `scrypt` (nodejsdp.link/scrypt), `PBKDF2` (nodejsdp.link/pbkdf2), hoặc `Argon2` (nodejsdp.link/argon2).

For our application, we want to implement a typical parallel pipeline where we have the following:

Cho ứng dụng của ta, ta muốn triển khai một parallel pipeline điển hình trong đó ta có những thứ sau:

- A node to create and distribute tasks across multiple workers
- Multiple worker nodes (where the actual computation happens)
- A node to collect all the results
 - Một node để tạo và phân phối các tác vụ qua nhiều worker
 - Nhiều node worker (nơi việc tính toán thực sự diễn ra)
 - Một node để thu thập tất cả các kết quả

The system we just described can be implemented in ZeroMQ using the following architecture:

Hệ thống mà ta vừa mô tả có thể được triển khai trong ZeroMQ bằng kiến trúc sau:

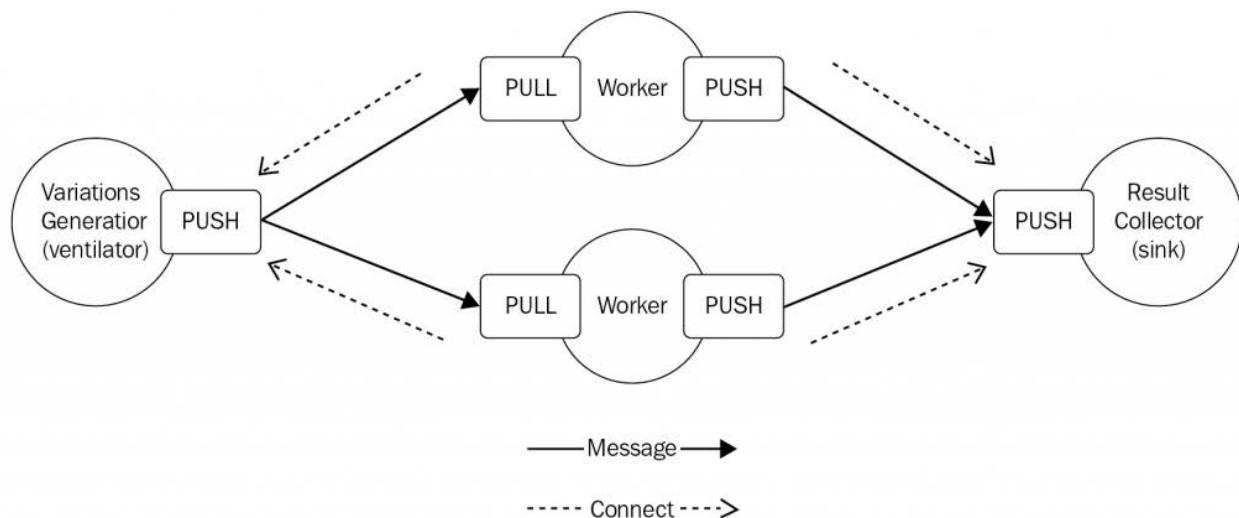


Figure 13.17: The architecture of a typical pipeline with ZeroMQ

Hình 13.17: Kiến trúc của một pipeline điển hình với ZeroMQ

In our architecture, we have a ventilator generating intervals of variations of characters in the given alphabet (for example, the interval ‘aa’ to ‘bb’ includes the variations ‘aa’, ‘ab’, ‘ba’, ‘bb’) and distributing those intervals to the workers as tasks. Each worker, then, calculates the hashsum of every variation in the given interval, trying to match each resulting hashsum against the control hashsum given as input. If a match is found, the result is sent to a results collector node (sink). The durable nodes of our architecture are the ventilator and the sink, while the transient nodes are the workers. This means that each worker connects its PULL socket to the ventilator and its PUSH socket to the sink, this way we can start and stop as many workers as we want without changing any parameter in the ventilator or the sink.

Trong kiến trúc của ta, ta có một ventilator sinh ra các khoảng biến thể của các ký tự trong bảng chữ cái cho trước (ví dụ, khoảng từ ‘aa’ tới ‘bb’ bao gồm các biến thể ‘aa’, ‘ab’, ‘ba’, ‘bb’) và phân phối các khoảng đó tới các worker dưới dạng các tác vụ. Mỗi worker, sau đó, tính hashsum của mọi biến thể trong khoảng cho trước, cố khớp mỗi hashsum kết quả với control hashsum được cho làm đầu vào. Nếu tìm thấy một khớp, kết quả được gửi tới một node thu thập kết quả (sink). Các node bền vững trong kiến trúc của ta là ventilator và sink, trong khi các node tạm thời là các worker. Điều này nghĩa là mỗi worker kết nối socket PULL của nó tới ventilator và socket PUSH của nó tới sink, theo cách này ta có thể khởi động và dừng bao nhiêu worker tùy thích mà không cần thay đổi bất kỳ tham số nào trong ventilator hay sink.

Implementing the producer — Triển khai producer To represent intervals of variations, we are going to use indexed n-ary trees. If we imagine having a tree in which each node has exactly n children, where each child is one of the n elements of the given alphabet and we assign an index to each node in breadth-first order, then, given the alphabet [a, b] we should obtain a tree such as the following:

Để biểu diễn các khoảng biến thể, ta sẽ dùng các cây n-ary được đánh chỉ số (indexed n-ary tree). Nếu ta hình dung có một cây trong đó mỗi node có đúng n con, mỗi con là một trong n phần tử của bảng chữ cái cho trước và ta gán một chỉ số cho mỗi node theo thứ tự duyệt theo bề rộng (breadth-first), thì, với bảng chữ cái [a, b], ta sẽ thu được một cây như sau:

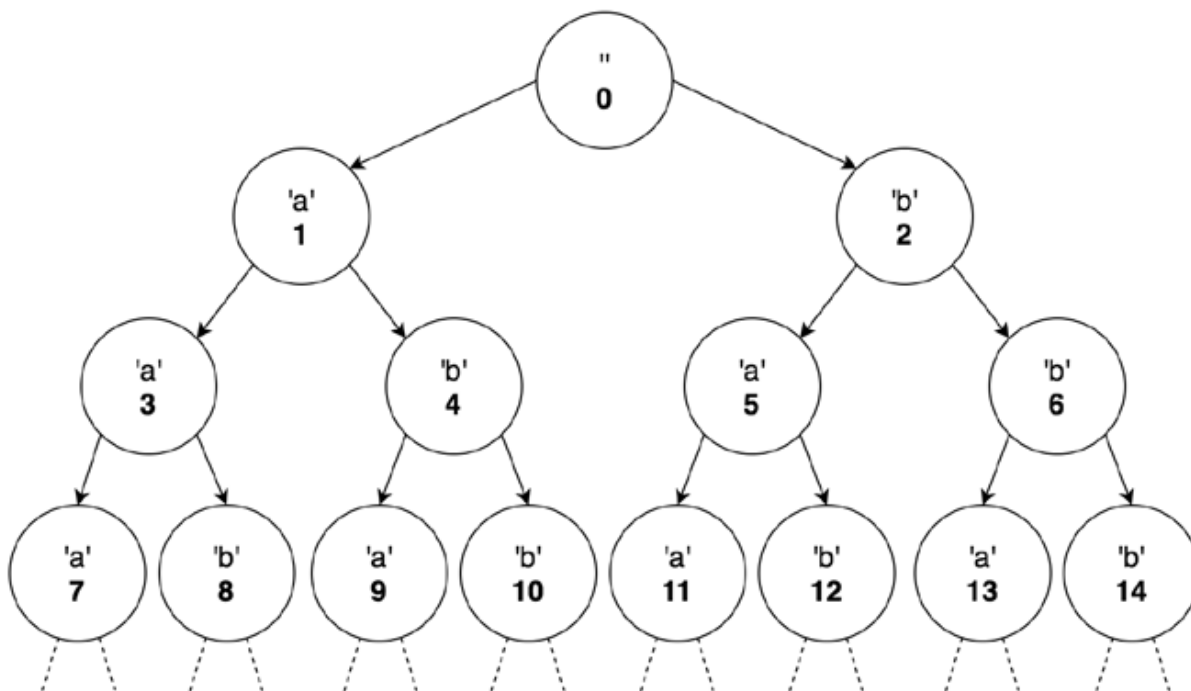


Figure 13.18: Indexed n-ary tree for alphabet [a, b]

Hình 13.18: Cây n-ary được đánh chỉ số cho bảng chữ cái [a, b]

It's then possible to obtain the variation corresponding to an index by traversing the tree from the root to the given index, appending the element of the nodes found along the way to the variation being calculated. For example, given the tree in Figure 13.18, the variation corresponding to the index 13 will be 'bba'.

Khi đó ta có thể thu được biến thể tương ứng với một chỉ số bằng cách duyệt cây từ gốc tới chỉ số cho trước, nối thêm phần tử của các node gặp trên đường vào biến thể đang được tính. Ví dụ, với cây trong Hình 13.18, biến thể tương ứng với chỉ số 13 sẽ là 'bba'.

We'll leverage the `indexed-string-variation` package (nodejsdp.link/indexed-string-variation) to aid us in calculating the corresponding variation given its index in the n-ary tree. This operation is done in the workers, so all we have to do in the ventilator is to produce intervals of indexes to give to the workers, which in turn will calculate all the variations of characters represented by those intervals. Now, after the necessary theory, let's start to build our system by implementing the component responsible to generate the tasks to distribute (`generateTasks.js`):

Ta sẽ tận dụng gói `indexed-string-variation` (nodejsdp.link/indexed-string-variation) để hỗ trợ tính biến thể tương ứng với chỉ số của nó trong cây n-ary. Thao tác này được thực hiện trong các worker, nên tất cả những gì ta phải làm trong ventilator là sinh ra các khoảng chỉ số để giao cho các worker, các worker sau đó sẽ tính tất cả các biến thể của ký tự được biểu diễn bởi các khoảng đó. Bây giờ, sau phần lý thuyết cần thiết, hãy bắt đầu xây dựng hệ thống của ta bằng cách triển khai thành phần chịu trách nhiệm sinh ra các tác vụ để phân phối (`generateTasks.js`):

```
export function * generateTasks (searchHash, alphabet,
  maxWordLength, batchSize) {
```



```

let nVariations = 0
for (let n = 1; n <= maxWordLength; n++) {
  nVariations += Math.pow(alphabet.length, n)
}
console.log('Finding the hashsum source string over ' +
  `${nVariations} possible variations`)
let batchStart = 1
while (batchStart <= nVariations) {
  const batchEnd = Math.min(
    batchStart + batchSize - 1, nVariations)
  yield {
    searchHash,
    alphabet: alphabet,
    batchStart,
    batchEnd
  }
  batchStart = batchEnd + 1
}
}

```

The `generateTasks()` **generator** creates intervals of integers of `batchSize` size, starting from 1 (we exclude 0, which is the root of the tree, corresponding to the empty variation) and ending at the largest possible index (`nVariations`) for the given alphabet and the maximum word length provided (`maxLength`). Then, we pack all the data about the task into an object and `yield` it to the caller.

Generator `generateTasks()` tạo ra các khoảng số nguyên có kích thước `batchSize`, bắt đầu từ 1 (ta loại trừ 0, là gốc của cây, tương ứng với biến thể rỗng) và kết thúc ở chỉ số lớn nhất có thể (`nVariations`) cho alphabet cho trước và độ dài từ tối đa được cung cấp (`maxLength`). Sau đó, ta đóng gói tất cả dữ liệu về tác vụ vào một object và `yield` nó cho bên gọi.

Please consider that to generate longer strings it may be necessary to switch to `BigInt` ([nodejsdp.link/bigint](https://nodejs.org/en/docs/faq/bigint/)) to represent their indexes, since the maximum safe integer manageable by JavaScript is currently $2^{53} - 1$, which is the value of `Number.MAX_SAFE_INTEGER`. Note that using very large integers may have a negative impact on the performances of the variations generator. Now, we need to implement the logic of our producer, which is responsible for distributing the tasks across all workers (in the `producer.js` file):

Xin lưu ý rằng để sinh các chuỗi dài hơn, có thể cần chuyển sang dùng `BigInt` ([nodejsdp.link/bigint](https://nodejs.org/en/docs/faq/bigint/)) để biểu diễn các chỉ số của chúng, vì số nguyên an toàn lớn nhất mà JavaScript hiện có thể xử lý là $2^{53} - 1$, là giá trị của `Number.MAX_SAFE_INTEGER`. Lưu ý rằng việc dùng các số nguyên rất lớn có thể có tác động tiêu cực tới hiệu năng của bộ sinh biến thể. Bây giờ, ta cần triển khai logic của producer, vốn chịu trách nhiệm phân phối các tác vụ qua tất cả các worker (trong file `producer.js`):

```

import zmq from 'zeromq'
import delay from 'delay'
import { generateTasks } from './generateTasks.js'
const ALPHABET = 'abcdefghijklmnopqrstuvwxyz'
const BATCH_SIZE = 10000
const [, , maxLength, searchHash] = process.argv
async function main () {

```

```

const ventilator = new zmq.Push() // (1)
await ventilator.bind('tcp://*:5016')
await delay(1000) // wait for all the workers to connect
const generatorObj = generateTasks(searchHash, ALPHABET,
    maxLength, BATCH_SIZE)
for (const task of generatorObj) {
    await ventilator.send(JSON.stringify(task)) // (2)
}
}
main().catch(err => console.error(err))

```

To avoid generating too many variations, our generator uses only the lowercase letters of the English alphabet and sets a limit on the size of the words generated. This limit is provided as an input in the command-line arguments (maxLength) together with the hashsum to match (searchHash).

Để tránh sinh ra quá nhiều biến thể, bộ sinh của ta chỉ dùng các chữ cái thường của bảng chữ cái tiếng Anh và đặt một giới hạn về kích thước của các từ được sinh ra. Giới hạn này được cung cấp làm đầu vào trong các đối số dòng lệnh (maxLength) cùng với hashsum cần khớp (searchHash).

But the part that we are most interested in analyzing is how we distribute the tasks across the workers:

Nhưng phần mà ta quan tâm nhất khi phân tích là cách ta phân phối các tác vụ qua các worker:

1. We first create a PUSH socket and we bind it to the local port 5016, which is where the PULL socket of the workers will connect to receive their tasks. We then wait 1 second for all the workers to connect: we do this because if the producer starts while the workers are already running, the workers may connect at different times (because of their timer-based reconnection algorithm) and that may cause the first connecting worker to receive most of the tasks.
2. For each generated task, we stringify it and send it to a worker using the send() function of the ventilator socket. Each connected worker will receive a different task following a round-robin approach.
 1. Trước tiên ta tạo một socket PUSH và bind nó vào cổng cục bộ 5016, là nơi socket PULL của các worker sẽ kết nối để nhận các tác vụ của chúng. Sau đó ta chờ 1 giây để tất cả các worker kết nối: ta làm vậy vì nếu producer khởi động trong khi các worker đã đang chạy, các worker có thể kết nối vào các thời điểm khác nhau (vì thuật toán kết nối lại dựa trên timer của chúng) và điều đó có thể khiến worker kết nối đầu tiên nhận hầu hết các tác vụ.
 2. Với mỗi tác vụ được sinh ra, ta stringify nó và gửi nó tới một worker bằng hàm send() của socket ventilator. Mỗi worker đang kết nối sẽ nhận một tác vụ khác nhau theo cách tiếp cận round-robin.

Implementing the worker — Triển khai worker Now it's time to implement the worker, but first, let's create a component to process the incoming tasks (in the processTask.js file):

Bây giờ đến lúc triển khai worker, nhưng trước tiên, hãy tạo một thành phần để xử lý các tác vụ đến (trong file processTask.js):

```

import isv from 'indexed-string-variation'
import { createHash } from 'crypto'
export function processTask (task) {

```

```

const variationGen = isv.generator(task.alphabet)
console.log('Processing from ' +
  `${variationGen(task.batchStart)} (${task.batchStart}) ` +
  `to ${variationGen(task.batchEnd)} (${task.batchEnd})`)
for (let idx = task.batchStart; idx <= task.batchEnd; idx++) {
  const word = variationGen(idx)
  const shasum = createHash('sha1')
  shasum.update(word)
  const digest = shasum.digest('hex')
  if (digest === task.searchHash) {
    return word
  }
}
}

```

The logic of the `processTask()` function is quite simple: it iterates over the indexes within the given interval, then for each index it generates the corresponding variation of characters (word). Next, it calculates the SHA1 checksum for the word and it tries to match it against the `searchHash` passed within the task object. If the two digests match, then it returns the source word to the caller.

Logic của hàm `processTask()` khá đơn giản: nó lặp qua các chỉ số trong khoảng cho trước, rồi với mỗi chỉ số nó sinh biến thể ký tự tương ứng (word). Tiếp theo, nó tính SHA1 checksum cho word và cố khớp nó với `searchHash` được truyền vào trong object task. Nếu hai digest khớp nhau, nó trả về word nguồn cho bên gọi.

Now we are ready to implement the main logic of our worker (`worker.js`):

Bây giờ ta đã sẵn sàng triển khai logic chính của worker (`worker.js`):

```

import zmq from 'zermq'
import { processTask } from './processTask.js'
async function main () {
  const fromVentilator = new zmq.Pull()
  const toSink = new zmq.Push()
  fromVentilator.connect('tcp://localhost:5016')
  toSink.connect('tcp://localhost:5017')
  for await (const rawMessage of fromVentilator) {
    const found = processTask(JSON.parse(rawMessage.toString()))
    if (found) {
      console.log(`Found! => ${found}`)
      await toSink.send(`Found: ${found}`)
    }
  }
}
main().catch(err => console.error(err))

```

As we said, our worker represents a transient node in our architecture, therefore, its sockets should connect to a remote node instead of listening for the incoming connections. That's exactly what we do in our worker, we create two sockets:

Như đã nói, worker của ta biểu diễn một node tạm thời trong kiến trúc, do đó, các socket của nó nên kết nối tới một node từ xa thay vì lắng nghe các kết nối đến. Đó chính xác là điều ta làm trong worker, ta tạo hai socket:

- A PULL socket that connects to the ventilator, for receiving the tasks
- A PUSH socket that connects to the sink, for propagating the results
 - Một socket PULL kết nối tới ventilator, để nhận các tác vụ
 - Một socket PUSH kết nối tới sink, để lan truyền các kết quả

Besides this, the job done by our worker is very simple: it processes every task received, and if a match is found, we send a message to the results collector through the toSink socket.

Bên cạnh đó, công việc mà worker của ta làm rất đơn giản: nó xử lý mọi tác vụ nhận được, và nếu tìm thấy một khớp, ta gửi một thông điệp tới bộ thu thập kết quả thông qua socket toSink.

Implementing the results collector — Triển khai bộ thu thập kết quả For our example, the results collector (sink) is a very basic program that simply prints the messages received by the workers to the console. The contents of the collector.js file are as follows:

Cho ví dụ của ta, bộ thu thập kết quả (sink) là một chương trình rất cơ bản chỉ đơn giản in các thông điệp nhận được từ các worker ra console. Nội dung của file collector.js như sau:

```
import zmq from 'zermq'
async function main () {
  const sink = new zmq.Pull()
  await sink.bind('tcp://*:5017')
  for await (const rawMessage of sink) {
    console.log('Message from worker: ', rawMessage.toString())
  }
}
main().catch(err => console.error(err))
```

It's interesting to see that the results collector (as the producer) is also a durable node of our architecture and therefore we bind its PULL socket instead of connecting it explicitly to the PUSH socket of the workers.

Thú vị khi thấy rằng bộ thu thập kết quả (cũng như producer) cũng là một node bền vững trong kiến trúc của ta và do đó ta bind socket PULL của nó thay vì kết nối nó một cách tường minh tới socket PUSH của các worker.

Running the application — Chạy ứng dụng We are now ready to launch our application; let's start a couple of workers and the results collector (each one in a different terminal):

Bây giờ ta đã sẵn sàng khởi chạy ứng dụng; hãy khởi động vài worker và bộ thu thập kết quả (mỗi cái trong một terminal khác nhau):

```
node worker.js
node worker.js
node collector.js
```

Then it's time to start the producer, specifying the maximum length of the words to generate and the SHA1 checksum that we want to match. The following is a sample command line:

Sau đó đến lúc khởi động producer, chỉ định độ dài tối đa của các từ cần sinh và SHA1 checksum mà ta muốn khớp. Sau đây là một dòng lệnh mẫu:

```
node producer.js 4 f8e966d1e207d02c44511a58dccff2f5429e9a3b
```

When the preceding command is run, the producer will start generating tasks and distributing them to the set of workers we started. We are telling the producer to generate all possible words with 4 lowercase letters (because our alphabet comprises only lowercase letters) and we also provide a sample SHA1 checksum that corresponds to a secret 4-letter word.

Khi lệnh trên được chạy, producer sẽ bắt đầu sinh các tác vụ và phân phối chúng tới tập các worker mà ta đã khởi động. Ta đang bảo producer sinh tất cả các từ có thể với 4 chữ cái thường (vì bảng chữ cái của ta chỉ gồm các chữ cái thường) và ta cũng cung cấp một SHA1 checksum mẫu tương ứng với một từ bí mật 4 chữ cái.

The results of the computation, if any, will appear in the terminal of the results collector application.

Kết quả của việc tính toán, nếu có, sẽ xuất hiện trong terminal của ứng dụng thu thập kết quả.

Please note that given the low-level nature of PUSH/PULL sockets in ZeroMQ and in particular the lack of message acknowledgments, if a node crashes, then all the tasks it was processing will be lost. It's possible to implement a custom acknowledgment mechanism on top of ZeroMQ but we'll leave that as an exercise for the reader.

Xin lưu ý rằng do bản chất cấp thấp của các socket PUSH/PULL trong ZeroMQ và đặc biệt là việc thiếu các message acknowledgment, nếu một node crash, thì tất cả các tác vụ mà nó đang xử lý sẽ bị mất. Có thể triển khai một cơ chế acknowledgment tùy chỉnh trên nền ZeroMQ nhưng ta sẽ để lại điều đó như một bài tập cho người đọc.

Another known limitation of this implementation is the fact that the workers won't stop processing tasks if a match is found. This feature was intentionally left out to make the examples as focused as possible on the pattern being discussed. You can try adding this "stopping" mechanism as an exercise.

Một hạn chế đã biết khác của bản triển khai này là việc các worker sẽ không dừng xử lý các tác vụ khi tìm thấy một khớp. Tính năng này được cố ý bỏ qua để giữ cho các ví dụ tập trung nhất có thể vào mẫu đang được thảo luận. Bạn có thể thử thêm cơ chế "dừng" này như một bài tập.

Pipelines and competing consumers in AMQP — Pipeline và competing consumers trong AMQP

In the previous section, we saw how a parallel pipeline can be implemented in a peer-to-peer context. Now, we are going to explore this pattern when applied in a broker-based architecture using RabbitMQ.

Ở phần trước, ta đã thấy cách một parallel pipeline có thể được triển khai trong ngữ cảnh peer-to-peer. Bây giờ, ta sẽ khám phá mẫu này khi áp dụng trong một kiến trúc dựa trên broker dùng RabbitMQ.

Point-to-point communications and competing consumers — Giao tiếp điểm-tới-điểm và competing consumers

In a peer-to-peer configuration, a pipeline is a very straightforward concept to imagine. With a message broker in the middle, though, the relationships between the various nodes of the system are a little bit harder to understand: the broker itself acts as an intermediary for our communications and, often, we don't really know who is on the other side listening for messages. For example, when we send a message using AMQP, we don't deliver it directly to its destination, but instead to an

exchange and then to a queue. Finally, it will be for the broker to decide where to route the message, based on the rules defined in the exchange, the bindings, and the destination queues.

Trong một cấu hình peer-to-peer, một pipeline là một khái niệm rất dễ hình dung. Tuy nhiên, với một message broker ở giữa, mối quan hệ giữa các node khác nhau của hệ thống hơi khó hiểu hơn một chút: bản thân broker đóng vai trò trung gian cho các giao tiếp của ta và, thường thì, ta không thực sự biết ai đang ở phía bên kia lắng nghe các thông điệp. Ví dụ, khi ta gửi một thông điệp bằng AMQP, ta không chuyển giao nó trực tiếp tới đích của nó, mà thay vào đó tới một exchange rồi tới một queue. Cuối cùng, sẽ là việc của broker để quyết định định tuyến thông điệp đi đâu, dựa trên các quy tắc được định nghĩa trong exchange, các binding, và các queue đích.

If we want to implement a pipeline and a task distribution pattern using a system like AMQP, we have to make sure that each message is received by only one consumer, but this is impossible to guarantee if an exchange can potentially be bound to more than one queue. The solution, then, is to send a message directly to the destination queue, bypassing the exchange altogether. This way, we can make sure that only one queue will ever receive the message. This communication pattern is called point-to-point.

Nếu ta muốn triển khai một pipeline và một mẫu phân phối tác vụ dùng một hệ thống như AMQP, ta phải đảm bảo rằng mỗi thông điệp chỉ được nhận bởi một consumer, nhưng điều này không thể đảm bảo nếu một exchange có khả năng được bind vào nhiều hơn một queue. Giải pháp, khi đó, là gửi một thông điệp trực tiếp tới queue đích, bỏ qua hoàn toàn exchange. Theo cách này, ta có thể đảm bảo rằng chỉ một queue duy nhất sẽ nhận thông điệp. Mẫu giao tiếp này được gọi là point-to-point (điểm-tới-điểm).

Once we are able to send a set of messages directly to a single queue, we are already half-way to implementing our task distribution pattern. In fact, the next step comes naturally: when multiple consumers are listening on the same queue, the messages will be distributed evenly across them, following a fanout distribution pattern. As we already mentioned, in the context of message brokers this is better known as the Competing Consumers pattern.

Một khi ta có thể gửi một tập các thông điệp trực tiếp tới một queue duy nhất, ta đã đi được nửa đường để triển khai mẫu phân phối tác vụ của mình. Thực tế, bước tiếp theo đến một cách tự nhiên: khi nhiều consumer cùng lắng nghe trên cùng một queue, các thông điệp sẽ được phân phối đều qua chúng, theo một mẫu fanout distribution. Như ta đã đề cập, trong ngữ cảnh các message broker, điều này được biết đến nhiều hơn với tên mẫu Competing Consumers.

Next, we are going to reimplement our simple hashsum cracker using AMQP, so we can appreciate the differences to the peer-to-peer approach we have discussed in the previous section.

Tiếp theo, ta sẽ triển khai lại bộ phá hashsum đơn giản của mình bằng AMQP, để ta có thể đánh giá những khác biệt so với cách tiếp cận peer-to-peer mà ta đã thảo luận ở phần trước.

Implementing the hashsum cracker using AMQP — Triển khai bộ phá hashsum dùng AMQP

We just learned that exchanges are the point in a broker where a message is multicast to a set of consumers, while queues are the place where messages are load balanced. With this knowledge in mind, let's now implement our brute-force hashsum cracker on top of an AMQP broker (which in our case is RabbitMQ). The following figure gives you an overview of the system we want to implement:

Ta vừa học rằng các exchange là điểm trong một broker nơi một thông điệp được multicast tới một tập các consumer, trong khi các queue là nơi các thông điệp được load balance. Với

kiến thức này trong đầu, bây giờ hãy triển khai bộ phá hashsum brute-force của ta trên nền một AMQP broker (trong trường hợp của ta là RabbitMQ). Hình sau cho bạn cái nhìn tổng quan về hệ thống mà ta muốn triển khai:

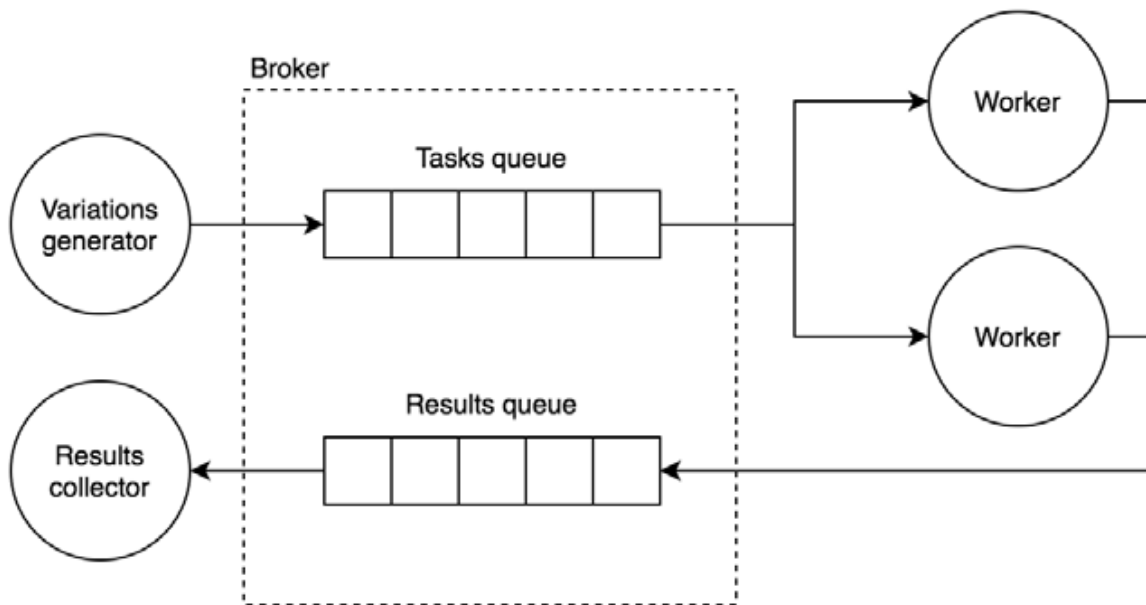


Figure 13.19: Task distribution architecture using a message queue broker

Hình 13.19: Kiến trúc phân phối tác vụ dùng một message queue broker

As we discussed, to distribute a set of tasks across multiple workers, we need to use a single queue. In Figure 13.19, we called this the tasks queue. On the other side of the tasks queue, we have a set of workers, which are competing consumers: in other words, each one will receive a different message from the queue. The effect is that multiple tasks will execute in parallel on different workers.

Như ta đã thảo luận, để phân phối một tập các tác vụ qua nhiều worker, ta cần dùng một queue duy nhất. Trong Hình 13.19, ta gọi nó là tasks queue. Ở phía bên kia của tasks queue, ta có một tập các worker, là các competing consumer: nói cách khác, mỗi worker sẽ nhận một thông điệp khác nhau từ queue. Hiệu ứng là nhiều tác vụ sẽ thực thi song song trên các worker khác nhau.

The results generated by the workers are published into another queue, which we called the results queue, and then consumed by the results collector, which is actually equivalent to a sink. In the entire architecture, we don't make use of any exchange, we only send messages directly to their destination queue, implementing a point-to-point type of communication.

Các kết quả được sinh ra bởi các worker được publish vào một queue khác, mà ta gọi là results queue, rồi được tiêu thụ bởi bộ thu thập kết quả, vốn thực ra tương đương với một sink. Trong toàn bộ kiến trúc, ta không dùng bất kỳ exchange nào, ta chỉ gửi các thông điệp trực tiếp tới queue đích của chúng, triển khai một kiểu giao tiếp point-to-point.

Implementing the producer — Triển khai producer Let's see how to implement such a system, starting from the producer (in the `producer.js` file):

Hãy xem cách triển khai một hệ thống như vậy, bắt đầu từ producer (trong file `producer.js`):

```

import amqp from 'amqplib'
import { generateTasks } from './generateTasks.js'
const ALPHABET = 'abcdefghijklmnopqrstuvwxyz'
const BATCH_SIZE = 10000
const [, , maxLength, searchHash] = process.argv
async function main () {
  const connection = await amqp.connect('amqp://localhost')
  const channel = await connection.createConfirmChannel() // (1)
  await channel.assertQueue('tasks_queue')
  const generatorObj = generateTasks(searchHash, ALPHABET,
    maxLength, BATCH_SIZE)
  for (const task of generatorObj) {
    channel.sendToQueue('tasks_queue', // (2)
      Buffer.from(JSON.stringify(task)))
  }
  await channel.waitForConfirms()
  channel.close()
  connection.close()
}
main().catch(err => console.error(err))

```

As we can see, the absence of any exchange or binding makes the setup of an AMQPbased application much simpler. There are however a few details to note:

Như ta thấy, sự vắng mặt của bất kỳ exchange hay binding nào khiến việc thiết lập một ứng dụng dựa trên AMQP đơn giản hơn nhiều. Tuy nhiên, có vài chi tiết cần lưu ý:

1. Instead of creating a standard channel, we are creating a `confirmChannel`. This is necessary as it creates a channel with some extra functionality, in particular, it provides the `waitForConfirms()` function that we use later in the code to wait until the broker confirms the reception of all the messages. This is necessary to prevent the application from closing the connection to the broker too soon, before all the messages have been dispatched from the local queue.
2. The core of the producer is the `channel.sendToQueue()` API, which is actually new to us. As its name says, that's the API responsible for delivering a message straight to a queue—the `tasks_queue` in our example—bypassing any exchange or routing.
 1. Thay vì tạo một channel tiêu chuẩn, ta đang tạo một `confirmChannel`. Điều này cần thiết vì nó tạo ra một channel với một số chức năng bổ sung, cụ thể, nó cung cấp hàm `waitForConfirms()` mà ta dùng về sau trong code để chờ cho tới khi broker xác nhận việc đã nhận tất cả các thông điệp. Điều này cần thiết để ngăn ứng dụng đóng kết nối tới broker quá sớm, trước khi tất cả các thông điệp đã được phân phát từ queue cục bộ.
 2. Cốt lõi của producer là API `channel.sendToQueue()`, vốn thực ra mới với chúng ta. Như tên gọi, đó là API chịu trách nhiệm chuyển giao một thông điệp thẳng tới một queue — `tasks_queue` trong ví dụ của ta — bỏ qua bất kỳ exchange hay định tuyến nào.

Implementing the worker — Triển khai worker On the other side of the `tasks_queue`, we have the workers listening for the incoming tasks. Let's update the code of our existing `worker.js` module to use AMQP:

Ở phía bên kia của `tasks_queue`, ta có các worker lắng nghe các tác vụ đến. Hãy cập nhật code của module `worker.js` hiện có của ta để dùng AMQP:

```
import amqp from 'amqplib'
import { processTask } from './processTask.js'
async function main () {
  const connection = await amqp.connect('amqp://localhost')
  const channel = await connection.createChannel()
  const { queue } = await channel.assertQueue('tasks_queue')
  channel.consume(queue, async (rawMessage) => {
    const found = processTask(
      JSON.parse(rawMessage.content.toString()))
    if (found) {
      console.log(`Found! => ${found}`)
      await channel.sendToQueue('results_queue',
        Buffer.from(`Found: ${found}`))
    }
    await channel.ack(rawMessage)
  })
}
main().catch(err => console.error(err))
```

Our new worker is also very similar to the one we implemented in the previous section using ZeroMQ, except for the parts related to the exchange of messages. In the preceding code, we can see how we first get a reference to the queue called `tasks_queue` and then we start listening for incoming tasks using `channel.consume()`. Then, every time a match is found, we send the result to the collector via the `results_queue`, again using point-to-point communication. It's also important to note how we are acknowledging every message with `channel.ack()` after the message has been completely processed.

Worker mới của ta cũng rất giống với worker mà ta đã triển khai ở phần trước dùng ZeroMQ, ngoại trừ các phần liên quan tới việc trao đổi thông điệp. Trong đoạn code trên, ta có thể thấy cách ta trước tiên lấy một tham chiếu tới queue tên là `tasks_queue` rồi bắt đầu lắng nghe các tác vụ đến bằng `channel.consume()`. Sau đó, mỗi lần tìm thấy một khớp, ta gửi kết quả tới bộ thu thập thông qua `results_queue`, lại dùng giao tiếp point-to-point. Cũng quan trọng cần lưu ý cách ta xác nhận mỗi thông điệp bằng `channel.ack()` sau khi thông điệp đã được xử lý hoàn toàn.

If multiple workers are started, they will all listen on the same queue, resulting in the messages being load balanced between them (they become competing consumers).

Nếu nhiều worker được khởi động, chúng sẽ đều lắng nghe trên cùng một queue, dẫn tới việc các thông điệp được load balance giữa chúng (chúng trở thành các competing consumer).

Implementing the result collector — Triển khai bộ thu thập kết quả The results collector is again a trivial module, simply printing any message received to the console. This is implemented in the `collector.js` file, as follows:

Bộ thu thập kết quả lại là một module tầm thường, đơn giản in mọi thông điệp nhận được ra console. Điều này được triển khai trong file `collector.js`, như sau:

```
import amqp from 'amqplib'
async function main () {
```

```

const connection = await amqp.connect('amqp://localhost')
const channel = await connection.createChannel()
const { queue } = await channel.assertQueue('results_queue')
channel.consume(queue, msg => {
  console.log(`Message from worker: ${msg.content.toString()}`)
})
}
main().catch(err => console.error(err))

```

Running the application — Chạy ứng dụng Now everything is ready to give our new system a try. First, make sure that the RabbitMQ server is running, then you can launch a couple of workers (in two separate terminals), which will both connect to the same queue (tasks_queue) so that every message will be load balanced between them:

Bây giờ mọi thứ đã sẵn sàng để thử hệ thống mới của ta. Trước tiên, hãy đảm bảo rằng RabbitMQ server đang chạy, rồi bạn có thể khởi chạy vài worker (trong hai terminal riêng biệt), cả hai sẽ kết nối tới cùng một queue (tasks_queue) để mỗi thông điệp sẽ được load balance giữa chúng:

```

node worker.js
node worker.js

```

Then, you can run the collector module and then the producer (by providing the maximum word length and the hash to crack):

Sau đó, bạn có thể chạy module collector rồi tới producer (bằng cách cung cấp độ dài từ tối đa và hash cần phá):

```

node collector.js
node producer.js 4 f8e966d1e207d02c44511a58dccff2f5429e9a3b

```

With this, we implemented a message pipeline and the Competing Consumers pattern using AMQP.

Với điều này, ta đã triển khai một message pipeline và mẫu Competing Consumers dùng AMQP.

It's interesting to note that our new version of the hashsum cracker based on AMQP takes slightly longer (compared to the ZeroMQbased version) to execute all the tasks and find a match. This is a practical demonstration of how a broker can actually introduce a negative performance impact, compared to a more low-level peerto-peer approach. However, let's not forget that with AMQP we are getting much more out of the box compared to our ZeroMQ implementation. For example, with the AMQP implementation, if a worker crashes, the messages it was processing won't be lost and will eventually be passed to another worker. So, remember to always look at the bigger picture when choosing the right approach to use for your application: a small delay may mean nothing compared to a massive increase in the overall complexity of the system or to a lack of some important features.

Thú vị khi lưu ý rằng phiên bản mới của bộ phá hashsum dựa trên AMQP của ta mất nhiều thời gian hơn một chút (so với phiên bản dựa trên ZeroMQ) để thực thi tất cả các tác vụ và tìm một khớp. Đây là một minh chứng thực tế về cách một broker có thể thực sự đưa vào một tác động tiêu cực tới hiệu năng, so với cách tiếp cận peer-to-peer cấp thấp hơn. Tuy nhiên, đừng quên rằng với AMQP ta nhận được nhiều thứ sẵn có hơn nhiều so với bản triển khai ZeroMQ của ta. Ví dụ, với bản triển khai AMQP, nếu một worker crash, các thông điệp mà nó đang xử lý sẽ không bị mất và cuối cùng sẽ được chuyển cho một worker khác. Vậy nên, hãy nhớ luôn nhìn vào bức tranh lớn hơn khi chọn cách tiếp cận đúng để

dùng cho ứng dụng của bạn: một độ trễ nhỏ có thể chẳng là gì so với một sự gia tăng khổng lồ về độ phức tạp tổng thể của hệ thống hay so với việc thiếu một số tính năng quan trọng.

Now, let's consider another broker-based approach for implementing task distribution patterns, this time built on top of Redis Streams.

Bây giờ, hãy cân nhắc một cách tiếp cận dựa trên broker khác để triển khai các mẫu phân phối tác vụ, lần này được xây dựng trên nền Redis Streams.

Distributing tasks with Redis Streams — Phân phối tác vụ với Redis Streams

After seeing how the Task Distribution pattern can be implemented using ZeroMQ and AMQP, we are now going to see how we can implement this pattern leveraging Redis Streams.

Sau khi xem cách mẫu Task Distribution có thể được triển khai dùng ZeroMQ và AMQP, bây giờ ta sẽ xem làm thế nào có thể triển khai mẫu này bằng cách tận dụng Redis Streams.

Redis consumer groups — Redis consumer group

Before diving into some code, we need to learn about a critical feature of Redis that allows us to implement a Task Distribution pattern using Redis Streams. This feature is called consumer groups and is an implementation of the Competing Consumer pattern (with the addition of some useful accessories) on top of Redis Streams.

Trước khi đi sâu vào code, ta cần học về một tính năng quan trọng của Redis cho phép ta triển khai mẫu Task Distribution dùng Redis Streams. Tính năng này được gọi là consumer group và là một bản triển khai của mẫu Competing Consumer (với một số phụ kiện hữu ích bổ sung) trên nền Redis Streams.

A consumer group is a stateful entity, identified by a name, which comprises a set of consumers identified by a name. When the consumers in the group try to read the stream, they will receive the records in a round-robin configuration.

Một consumer group là một thực thể có trạng thái (stateful), được định danh bằng một tên, gồm một tập các consumer được định danh bằng tên. Khi các consumer trong group cố đọc stream, chúng sẽ nhận các record theo cấu hình round-robin.

Each record has to be explicitly acknowledged, otherwise, the record will be kept in a pending state. Each consumer can only access its own history of pending records unless it explicitly claims the records of another consumer. This is useful if a consumer crashes while processing a record. When the consumer comes back online, the first thing it should do is retrieve its list of pending records and process those before requesting new records from the stream. Figure 13.20 provides a visual representation of how consumer groups work in Redis.

Mỗi record phải được xác nhận tường minh, nếu không, record sẽ được giữ ở trạng thái pending (chờ xử lý). Mỗi consumer chỉ có thể truy cập lịch sử các record pending của riêng nó trừ khi nó claim (đòi) các record của một consumer khác một cách tường minh. Điều này hữu ích nếu một consumer crash trong khi xử lý một record. Khi consumer trở lại trực tuyến, điều đầu tiên nó nên làm là lấy lại danh sách các record pending của mình và xử lý chúng trước khi yêu cầu các record mới từ stream. Hình 13.20 cung cấp một biểu diễn trực quan về cách các consumer group hoạt động trong Redis.

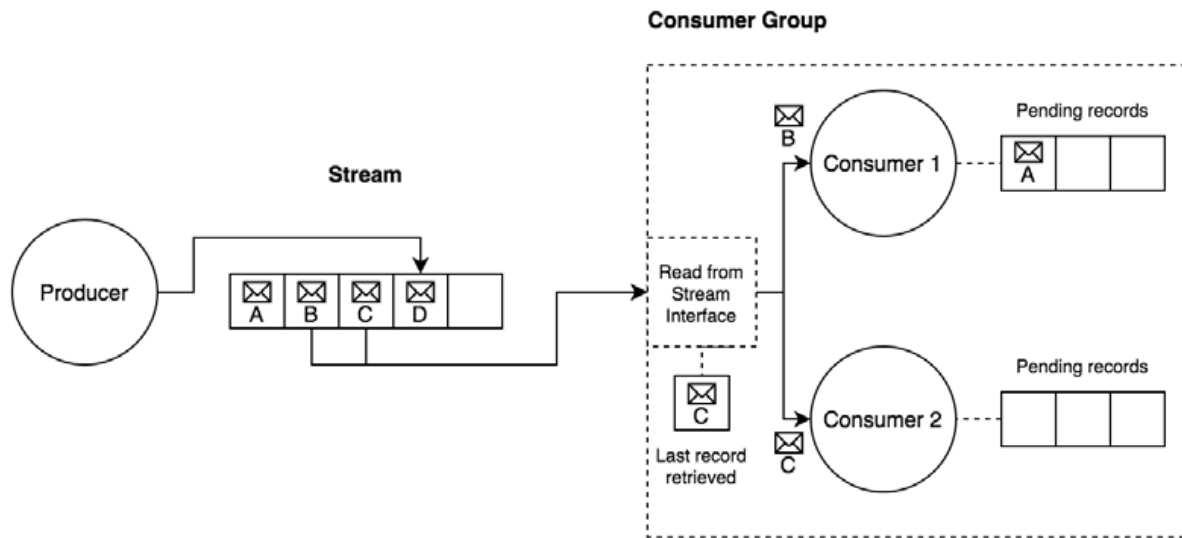


Figure 13.20: A Redis Stream consumer group

Hình 13.20: Một consumer group của Redis Stream

We can note how the two consumers in the group receive two different records (B for Consumer 1 and C for Consumer 2) when they try to read from the stream. The consumer group also stores the ID of the last retrieved record (record C), so that at the successive read operation the consumer group knows what's the next record to read. We can also note how Consumer 1 has a pending record (A), which is a record that it's still processing or couldn't process. Consumer 1 can implement a retry algorithm to make sure to process all the pending records assigned to itself. A Redis Stream can have multiple consumer groups. This way it's possible to simultaneously apply different types of processing to the same data.

Ta có thể lưu ý cách hai consumer trong group nhận hai record khác nhau (B cho Consumer 1 và C cho Consumer 2) khi chúng cố đọc từ stream. Consumer group cũng lưu ID của record được lấy lại cuối cùng (record C), để ở thao tác đọc kế tiếp consumer group biết được record tiếp theo cần đọc là gì. Ta cũng có thể lưu ý cách Consumer 1 có một record pending (A), là một record mà nó vẫn đang xử lý hoặc không thể xử lý. Consumer 1 có thể triển khai một thuật toán thử lại (retry) để đảm bảo xử lý tất cả các record pending được gán cho chính nó. Một Redis Stream có thể có nhiều consumer group. Theo cách này, ta có thể đồng thời áp dụng các loại xử lý khác nhau cho cùng một dữ liệu.

Now let's put into practice what we just learned about Redis consumer groups to implement our hashsum cracker.

Bây giờ hãy đưa vào thực hành những gì ta vừa học về Redis consumer group để triển khai bộ phá hashsum của mình.

Implementing the hashsum cracker using Redis Streams — Triển khai bộ phá hashsum dùng Redis Streams

The architecture of our hashsum cracker with Redis Streams is going to resemble closely that of the previous AMQP example. In fact, we are going to have two different streams (in the AMQP examples they were queues): one stream to hold the tasks to be processed (`tasks_stream`) and another stream to hold the results coming from the workers (`results_stream`).

Kiến trúc của bộ phá hashsum dùng Redis Streams của ta sẽ giống chặt chẽ với kiến trúc của ví dụ AMQP trước đó. Thực tế, ta sẽ có hai stream khác nhau (trong các ví dụ AMQP chúng là các queue): một stream để giữ các tác vụ cần xử lý (`tasks_stream`) và một stream khác để giữ các kết quả đến từ các worker (`results_stream`).

Then, we are going to use a consumer group to distribute the tasks from the `tasks_stream` to the workers of our application (our workers are the consumers).

Sau đó, ta sẽ dùng một consumer group để phân phối các tác vụ từ `tasks_stream` tới các worker của ứng dụng (các worker của ta là các consumer).

Implementing the producer — Triển khai producer Let's start by implementing the producer (in the `producer.js` file):

Hãy bắt đầu bằng việc triển khai producer (trong file `producer.js`):

```
import Redis from 'ioredis'
import { generateTasks } from './generateTasks.js'
const ALPHABET = 'abcdefghijklmnopqrstuvwxyz'
const BATCH_SIZE = 10000
const redisClient = new Redis()
const [, , maxLength, searchHash] = process.argv
async function main () {
  const generatorObj = generateTasks(searchHash, ALPHABET,
    maxLength, BATCH_SIZE)
  for (const task of generatorObj) {
    await redisClient.xadd('tasks_stream', '*',
      'task', JSON.stringify(task))
  }
  redisClient.disconnect()
}
main().catch(err => console.error(err))
```

As we can see, there is nothing new to us in the implementation of the new `producer.js` module. In fact, we already know very well how to add records to a stream; all we have to do is invoke `xadd()` as discussed in the Reliable messaging with streams section.

Như ta thấy, không có gì mới với chúng ta trong bản triển khai module `producer.js` mới. Thực tế, ta đã biết rất rõ cách thêm các record vào một stream; tất cả những gì ta phải làm là gọi `xadd()` như đã thảo luận trong phần Nhấn tin đáng tin cậy với stream.

Implementing the worker — Triển khai worker Next, we need to adapt our worker so it can interface with a Redis Stream using a consumer group. This is the core of all the architecture, as in here, in the worker, we leverage consumer groups and their features. So, let's implement the new `worker.js` module:

Tiếp theo, ta cần điều chỉnh worker của mình để nó có thể giao tiếp với một Redis Stream dùng một consumer group. Đây là cốt lõi của toàn bộ kiến trúc, vì ở đây, trong worker, ta tận dụng các consumer group và các tính năng của chúng. Vậy, hãy triển khai module `worker.js` mới:

```
import Redis from 'ioredis'
import { processTask } from './processTask.js'
const redisClient = new Redis()
```

```

const [, , consumerName] = process.argv
async function main () {
  await redisClient.xgroup('CREATE', 'tasks_stream',           // (1)
    'workers_group', '$', 'MKSTREAM')
  .catch(() => console.log('Consumer group already exists'))
  const [[, records]] = await redisClient.xreadgroup(           // (2)
    'GROUP', 'workers_group', consumerName, 'STREAMS',
    'tasks_stream', '0')
  for (const [recordId, [, rawTask]] of records) {
    await processAndAck(recordId, rawTask)
  }
  while (true) {
    const [[, records]] = await redisClient.xreadgroup(         // (3)
      'GROUP', 'workers_group', consumerName, 'BLOCK', '0',
      'COUNT', '1', 'STREAMS', 'tasks_stream', '>')
    for (const [recordId, [, rawTask]] of records) {
      await processAndAck(recordId, rawTask)
    }
  }
}
async function processAndAck (recordId, rawTask) {              // (4)
  const found = processTask(JSON.parse(rawTask))
  if (found) {
    console.log(`Found! => ${found}`)
    await redisClient.xadd('results_stream', '*', 'result',
      `Found: ${found}`)
  }
  await redisClient.xack('tasks_stream', 'workers_group', recordId)
}
main().catch(err => console.error(err))

```

OK, there are a lot of moving parts in the new worker code. So, let's analyze it one step at a time:

OK, có rất nhiều phần chuyển động trong code worker mới. Vậy nên, hãy phân tích nó từng bước một:

1. First, we need to make sure that the consumer group exists before we can use it. We can do that with the `xgroup` command, which we invoke with the following parameters: a. `'CREATE'` is the keyword to use when we want to create a consumer group. In fact, with the `xgroup` command, we can also destroy the consumer group, remove a consumer, or update the last read record ID, using different subcommands. b. `'tasks_stream'` is the name of the stream we want to read from. c. `'workers_group'` is the name of the consumer group. d. The fourth argument represents the record ID from where the consumer group should start consuming records from the stream. Using `'$'` (dollar sign) means that the consumer group should start reading the stream from the ID of the last record currently in the stream. e. `'MKSTREAM'` is an extra parameter that instructs Redis to create the stream if it doesn't exist already.
2. Next, we read all the pending records belonging to the current consumer. Those are the leftover records from a previous run of the consumer that weren't processed because of an abrupt interruption of the application (such as a crash). If the same consumer (with the same name) terminated properly during the last run, without errors, then this list would most likely be empty. As we already mentioned, each consumer has access only to its own pending records. We retrieve this list with a `xreadgroup` command and the following arguments:

1. Trước tiên, ta cần đảm bảo rằng consumer group tồn tại trước khi ta có thể dùng nó. Ta có thể làm điều đó bằng lệnh `xgroup`, mà ta gọi với các tham số sau: a. `'CREATE'` là từ khóa cần dùng khi ta muốn tạo một consumer group. Thực tế, với lệnh `xgroup`, ta cũng có thể hủy consumer group, loại bỏ một consumer, hoặc cập nhật ID của record được đọc cuối cùng, dùng các subcommand khác nhau. b. `'tasks_stream'` là tên của stream mà ta muốn đọc. c. `'workers_group'` là tên của consumer group. d. Đối số thứ tư biểu diễn record ID mà từ đó consumer group nên bắt đầu tiêu thụ các record từ stream. Dùng `'$'` (dấu đô la) nghĩa là consumer group nên bắt đầu đọc stream từ ID của record cuối cùng hiện có trong stream. e. `'MKSTREAM'` là một tham số bổ sung chỉ thị Redis tạo stream nếu nó chưa tồn tại.
2. Tiếp theo, ta đọc tất cả các record pending thuộc về consumer hiện tại. Đó là các record còn sót lại từ một lần chạy trước của consumer mà chưa được xử lý vì một sự gián đoạn đột ngột của ứng dụng (như một crash). Nếu cùng consumer đó (với cùng tên) kết thúc đúng cách trong lần chạy trước, không có lỗi, thì danh sách này nhiều khả năng sẽ rỗng. Như ta đã đề cập, mỗi consumer chỉ có quyền truy cập các record pending của riêng nó. Ta lấy lại danh sách này bằng một lệnh `xreadgroup` và các đối số sau:
 - a. `'GROUP'`, `'workers_group'`, `consumerName` là một bộ ba bắt buộc trong đó ta chỉ định tên của consumer group (`'workers_group'`) và tên của consumer (`consumerName`) mà ta đọc từ các đầu vào dòng lệnh. b. Sau đó ta chỉ định stream mà ta muốn đọc bằng `'STREAMS'`, `'tasks_stream'`. c. Cuối cùng, ta chỉ định `'0'` làm đối số cuối, là ID mà từ đó ta nên bắt đầu đọc. Về cơ bản, ta đang nói rằng ta muốn đọc tất cả các thông điệp pending thuộc về consumer hiện tại bắt đầu từ thông điệp đầu tiên.
3. Then, we have another call to `xreadgroup()`, but this time it has a completely different semantic. In this case, in fact, we want to start reading new records from the stream (and not access the consumer's own history). This is possible with the following list of arguments: a. As in the previous call of `xreadgroup()`, we specify the consumer group that we want to use for the read operation with the three arguments: `'GROUP'`, `'workers_group'`, `consumerName`. b. Then we indicate that the call should block if there are no new records currently available instead of returning an empty list. We do that with the following two arguments: `'BLOCK'`, `'0'`. The last argument is the timeout after which the function returns anyway, even without results. `'0'` means that we want to wait indefinitely. c. The next two arguments, `'COUNT'` and `'1'`, tell Redis that we are interested in getting one record per call. d. Next, we specify the stream we want to read from with `'STREAMS'`, `'tasks_stream'`. e. Finally, with the special ID `'>'` (greater than symbol), we indicate that we are interested in any record not yet retrieved by this consumer group.
4. Finally, in the `processAndAck()` function, we check if we have a match and if that's the case, we append a new record to the `results_stream`. At last, when all the processing for the record returned by `xreadgroup()` completes, we invoke the Redis `xack` command to acknowledge that the record has been successfully consumed, which results in the record being removed from the pending list for the current consumer.

3. Sau đó, ta có một lời gọi khác tới `xreadgroup()`, nhưng lần này nó có một ngữ nghĩa hoàn toàn khác. Trong trường hợp này, thực tế, ta muốn bắt đầu đọc các record mới từ stream (chứ không phải truy cập lịch sử của riêng consumer). Điều này khả thi với danh sách đối số sau: a. Như trong lời gọi trước của `xreadgroup()`, ta chỉ định consumer group mà ta muốn dùng cho thao tác đọc với ba đối số: `'GROUP'`, `'workers_group'`, `consumerName`. b. Sau đó ta báo rằng lời gọi nên chặn nếu hiện không có record mới nào khả dụng thay vì trả về một danh sách rỗng. Ta làm điều đó với hai đối số sau: `'BLOCK'`, `'0'`. Đối số cuối là timeout, sau đó hàm vẫn trả về, ngay cả khi không có kết quả. `'0'` nghĩa là ta muốn chờ vô hạn. c. Hai đối số tiếp theo, `'COUNT'` và `'1'`, báo cho Redis biết rằng ta quan tâm tới việc lấy một record mỗi lời gọi. d. Tiếp theo, ta chỉ định stream mà ta muốn đọc với `'STREAMS'`, `'tasks_stream'`. e. Cuối cùng, với ID đặc biệt `'>'` (ký hiệu lớn hơn), ta báo rằng ta quan tâm tới bất kỳ record nào chưa được consumer group này lấy lại.
4. Cuối cùng, trong hàm `processAndAck()`, ta kiểm tra xem có một khớp không và nếu có, ta thêm một record mới vào `results_stream`. Sau hết, khi tất cả việc xử lý cho record được `xreadgroup()` trả về hoàn tất, ta gọi lệnh Redis `xack` để xác nhận rằng record đã được tiêu thụ thành công, dẫn tới việc record bị loại khỏi danh sách pending cho consumer hiện tại.

Phew! There was a lot going on in the `worker.js` module. It's interesting to note that most of the complexity comes from the large amount of arguments required by the various Redis commands. You may be surprised to know that this example just scratches the surface, as there is a lot more to know about Redis Streams, and in particular, consumer groups. Check out the official Redis introduction to Streams for more details at nodejsdp.link/redis-streams.

Phù! Có rất nhiều thứ diễn ra trong module `worker.js`. Thú vị khi lưu ý rằng phần lớn độ phức tạp đến từ số lượng lớn đối số mà các lệnh Redis khác nhau yêu cầu. Bạn có thể ngạc nhiên khi biết rằng ví dụ này chỉ mới chạm bề mặt, vì còn rất nhiều điều để biết về Redis Streams, và đặc biệt là các consumer group. Hãy xem phần giới thiệu chính thức của Redis về Streams để biết thêm chi tiết tại nodejsdp.link/redis-streams.

Now, everything should be ready for us to try out this new version of the hashsum cracker. Let's start a couple of workers, but this time remember to assign them a name, which will be used to identify them in the consumer group:

Bây giờ, mọi thứ hẳn đã sẵn sàng để ta thử phiên bản mới này của bộ phá hashsum. Hãy khởi động vài worker, nhưng lần này nhớ gán cho chúng một tên, sẽ được dùng để định danh chúng trong consumer group:

```
node worker.js workerA
node worker.js workerB
```

Then, you can run the collector and the producer as we did in the previous examples:

Sau đó, bạn có thể chạy collector và producer như ta đã làm trong các ví dụ trước:

```
node collector.js
node producer.js 4 f8e966d1e207d02c44511a58dccff2f5429e9a3b
```

This concludes our exploration of the task distribution patterns, so now, we'll take a closer look at the request/reply patterns.

Điều này khép lại việc khám phá các mẫu phân phối tác vụ của ta, vậy nên bây giờ, ta sẽ xem xét kỹ hơn các mẫu request/reply.

Request/Reply patterns — Các mẫu Request/Reply

One-way communications can give us great advantages in terms of parallelism and efficiency, but alone they are not able to solve all our integration and communication problems. Sometimes, a good old request/reply pattern might just be the perfect tool for the job. But, there are situations in which all we have is an asynchronous one-way channel. It's therefore important to know the various patterns and approaches required to build an abstraction that would allow us to exchange messages in a request/reply fashion on top of a one-way channel. That's exactly what we are going to learn next.

Giao tiếp một chiều có thể cho ta những lợi thế lớn về mặt tính song song và hiệu quả, nhưng một mình chúng không thể giải quyết tất cả các vấn đề tích hợp và giao tiếp của ta. Đôi khi, một mẫu request/reply tốt cũ kỹ có thể chính là công cụ hoàn hảo cho công việc. Nhưng, có những tình huống mà tất cả những gì ta có là một kênh một chiều bất đồng bộ. Do đó, điều quan trọng là biết các mẫu và cách tiếp cận khác nhau cần thiết để xây dựng một lớp trừu tượng cho phép ta trao đổi thông điệp theo kiểu request/reply trên nền một kênh một chiều. Đó chính xác là điều ta sắp học tiếp theo.

Correlation Identifier — Correlation Identifier

The first Request/Reply pattern that we are going to learn is called the Correlation Identifier and it represents the basic block for building a request/reply abstraction on top of a one-way channel.

Mẫu Request/Reply đầu tiên mà ta sẽ học được gọi là Correlation Identifier và nó biểu diễn khối cơ bản để xây dựng một lớp trừu tượng request/reply trên nền một kênh một chiều.

The pattern involves marking each request with an identifier, which is then attached to the response by the receiver: this way, the sender of the request can correlate the two messages and return the response to the right handler. This elegantly solves the problem in the context of a one-way asynchronous channel, where messages can travel in any direction at any time. Let's take a look at the example in the following diagram:

Mẫu này liên quan tới việc đánh dấu mỗi request bằng một định danh (identifier), định danh này sau đó được bên nhận gắn vào response: theo cách này, bên gửi request có thể tương quan (correlate) hai thông điệp và trả response về đúng handler. Điều này giải quyết một cách thanh lịch vấn đề trong ngữ cảnh một kênh bất đồng bộ một chiều, nơi các thông điệp có thể di chuyển theo bất kỳ hướng nào vào bất kỳ lúc nào. Hãy xem ví dụ trong sơ đồ sau:

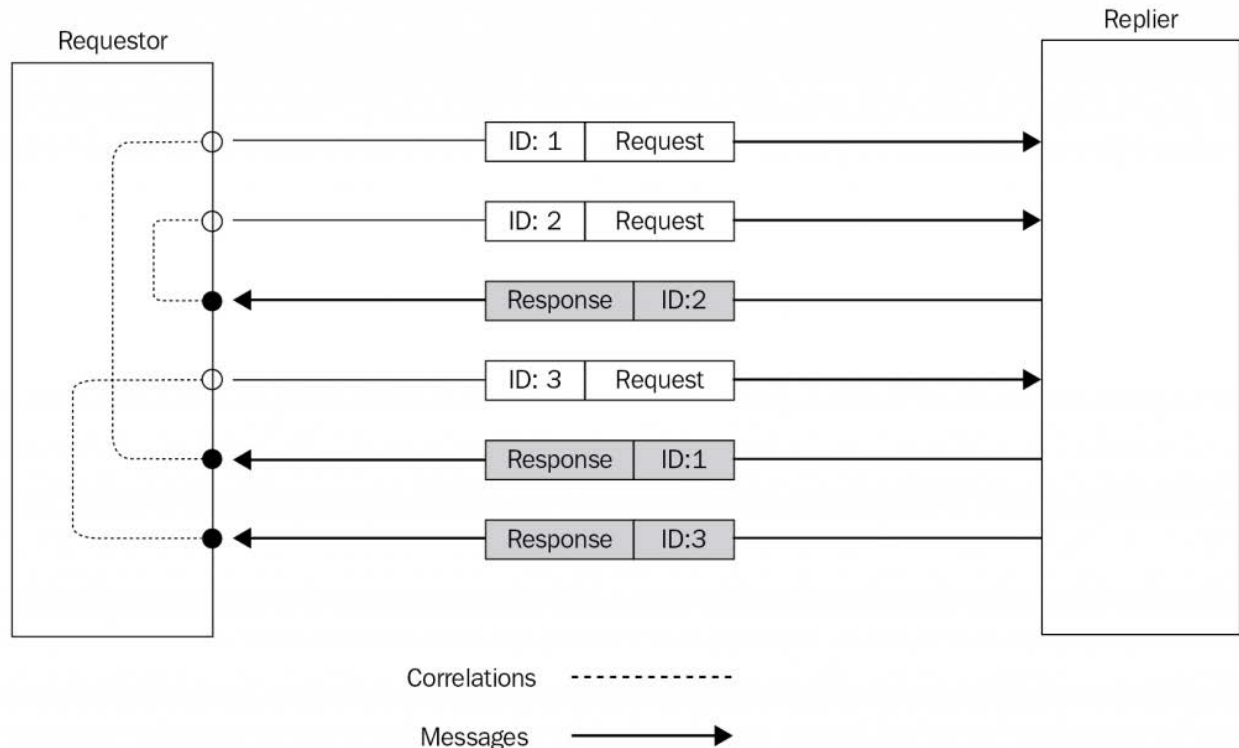


Figure 13.21: Request/reply message exchange using correlation identifiers

Hình 13.21: Trao đổi thông điệp request/reply dùng correlation identifier

The scenario depicted in Figure 13.21 shows how using a correlation ID allows us to match each response with the right request, even if those are sent and then received in a different order. The way this works will be much clearer once we start working on our next example.

Kịch bản được mô tả trong Hình 13.21 cho thấy cách dùng một correlation ID cho phép ta khớp mỗi response với đúng request, ngay cả khi chúng được gửi rồi nhận theo thứ tự khác nhau. Cách thức hoạt động của điều này sẽ rõ ràng hơn nhiều một khi ta bắt đầu làm ví dụ tiếp theo.

Implementing a request/reply abstraction using correlation identifiers — Triển khai một lớp trừu tượng request/reply dùng correlation identifier

Let's now start working on an example by choosing the simplest type of one-way channel; one that is point-to-point (which directly connects two nodes of the system) and fully duplex (messages can travel in both directions).

Bây giờ hãy bắt đầu làm một ví dụ bằng cách chọn loại kênh một chiều đơn giản nhất; một kênh là point-to-point (kết nối trực tiếp hai node của hệ thống) và song công hoàn toàn (full duplex — các thông điệp có thể di chuyển theo cả hai hướng).

In this simple channel category, we can find, for example, WebSockets: they establish a point-to-point connection between the server and browser, and the messages can travel in any direction. Another example is the communication channel that is created when a **child process** is spawned using `child_process.fork()` (we already met this API in Chapter 11, Advanced Recipes). This channel too is asynchronous, point-to-point, and duplex since it connects the parent only with the child process and it allows messages to travel in any direction. This is probably the most basic

channel of this category, so that's what we are going to use in the next example. The plan for the next application is to build an abstraction in order to wrap the channel created between the parent process and the child process. This abstraction should provide a request/reply communication channel by automatically marking each request with a correlation identifier and then matching the ID of any incoming reply against the list of request handlers awaiting a response.

Trong loại kênh đơn giản này, ta có thể tìm thấy, ví dụ, WebSockets: chúng thiết lập một kết nối point-to-point giữa server và trình duyệt, và các thông điệp có thể di chuyển theo bất kỳ hướng nào. Một ví dụ khác là kênh giao tiếp được tạo ra khi một tiến trình con được spawn bằng `child_process.fork()` (ta đã gặp API này trong Chương 11, Advanced Recipes). Kênh này cũng bất đồng bộ, point-to-point và song công vì nó chỉ kết nối tiến trình cha với tiến trình con và nó cho phép các thông điệp di chuyển theo bất kỳ hướng nào. Đây có lẽ là kênh cơ bản nhất của loại này, nên đó là cái ta sẽ dùng trong ví dụ tiếp theo. Kế hoạch cho ứng dụng tiếp theo là xây dựng một lớp trừu tượng để bọc kênh được tạo ra giữa tiến trình cha và tiến trình con. Lớp trừu tượng này nên cung cấp một kênh giao tiếp request/reply bằng cách tự động đánh dấu mỗi request bằng một correlation identifier rồi khớp ID của bất kỳ reply đến nào với danh sách các request handler đang chờ một response.

From Chapter 11, Advanced Recipes, we should remember that the parent process can send a message to a child with `child.send(message)`, while receiving messages is possible with the `child.on('message', **callback**)` event handler.

Từ Chương 11, Advanced Recipes, ta nên nhớ rằng tiến trình cha có thể gửi một thông điệp tới một tiến trình con với `child.send(message)`, trong khi việc nhận thông điệp khả thi với event handler `child.on('message', callback)`.

In a similar way, the child process can send a message to the parent process using `process.send(message)` and receive messages with `process.on('message', callback)`.

Theo cách tương tự, tiến trình con có thể gửi một thông điệp tới tiến trình cha bằng `process.send(message)` và nhận thông điệp với `process.on('message', callback)`.

This means that the interface of the channel available in the parent process is identical to the one available in the child. This will allow us to build a common abstraction that can be used from both ends of the channel.

Điều này nghĩa là giao diện của kênh khả dụng trong tiến trình cha giống hệt với giao diện khả dụng trong tiến trình con. Điều này sẽ cho phép ta xây dựng một lớp trừu tượng chung có thể được dùng từ cả hai đầu của kênh.

Abstracting the request — Trừu tượng hóa request Let's start building this abstraction by considering the part responsible for sending new requests. Let's create a new file called `createRequestChannel.js` with the following content:

Hãy bắt đầu xây dựng lớp trừu tượng này bằng cách xem xét phần chịu trách nhiệm gửi các request mới. Hãy tạo một file mới tên là `createRequestChannel.js` với nội dung sau:

```
import { nanoid } from 'nanoid'
export function createRequestChannel (channel) { // (1)
  const correlationMap = new Map()
  function sendRequest (data) { // (2)
    console.log('Sending request', data)
    return new Promise((resolve, reject) => {
```

```

    const correlationId = nanoid()
    const replyTimeout = setTimeout(() => {
      correlationMap.delete(correlationId)
      reject(new Error('Request timeout'))
    }, 10000)
    correlationMap.set(correlationId, (replyData) => {
      correlationMap.delete(correlationId)
      clearTimeout(replyTimeout)
      resolve(replyData)
    })
    channel.send({
      type: 'request',
      data,
      id: correlationId
    })
  })
}
channel.on('message', message => { // (3)
  const callback = correlationMap.get(message.inReplyTo)
  if (callback) {
    callback(message.data)
  }
})
return sendRequest
}

```

This is how our request abstraction works:

Đây là cách lớp trừu tượng request của ta hoạt động:

1. The `createRequestChannel()` is a **factory** that wraps the input channel and returns a `sendRequest()` function used to send a request and receive a reply. The magic of the pattern lies in the `correlationMap` variable, which stores the association between the outgoing requests and their reply handlers.
2. The `sendRequest()` function is used to send new requests. Its job is to generate a correlation ID using the `nanoid` package (nodejsdp.link/nanoid) and then wrap the request data in an envelope that allows us to specify the correlation ID and the type of the message. The correlation ID and the handler responsible for returning the reply data to the caller (which uses `resolve()` under the hood) are then added to the `correlationMap` so that the handler can be retrieved later using the correlation ID. We also implemented a very simple request timeout logic.
3. When the factory is invoked, we also start listening for incoming messages. If the correlation ID of the message (contained in the `inReplyTo` property) matches any of the IDs contained in the `correlationMap` map, we know that we just received a reply, so we obtain the reference to the associated response handler and we invoke it with the data contained in the message.

1. `createRequestChannel()` là một factory bọc input channel và trả về một hàm `sendRequest()` dùng để gửi một request và nhận một reply. Điều kỳ diệu của mẫu này nằm ở biến `correlationMap`, biến này lưu trữ sự liên kết giữa các request đi ra và các reply handler của chúng.
2. Hàm `sendRequest()` được dùng để gửi các request mới. Công việc của nó là sinh ra một correlation ID dùng gói `nanoid` (nodejsdp.link/nanoid) rồi bọc dữ liệu request trong một phong bì (envelope) cho phép ta chỉ định correlation ID và loại

của thông điệp. Correlation ID và handler chịu trách nhiệm trả dữ liệu reply về cho bên gọi (sử dụng `resolve()` ở bên dưới) sau đó được thêm vào `correlationMap` để handler có thể được lấy lại sau này bằng correlation ID. Ta cũng triển khai một logic `timeout request` rất đơn giản.

3. Khi factory được gọi, ta cũng bắt đầu lắng nghe các thông điệp đến. Nếu correlation ID của thông điệp (chứa trong thuộc tính `inReplyTo`) khớp với bất kỳ ID nào chứa trong map `correlationMap`, ta biết rằng ta vừa nhận một reply, nên ta lấy tham chiếu tới response handler liên quan và gọi nó với dữ liệu chứa trong thông điệp.

That's it for the `createRequestChannel.js` module. Let's move on to the next part.

Vậy là xong với module `createRequestChannel.js`. Hãy chuyển sang phần tiếp theo.

Abstracting the reply — Trừu tượng hóa reply We are just a step away from implementing the full pattern, so let's see how the counterpart of the request channel, which is the reply channel, works. Let's create another file called `createReplyChannel.js`, which will contain the abstraction for wrapping the reply handler:

Ta chỉ còn cách một bước nữa là triển khai đầy đủ mẫu này, nên hãy xem đối tác của request channel, là reply channel, hoạt động ra sao. Hãy tạo một file khác tên là `createReplyChannel.js`, file này sẽ chứa lớp trừu tượng để bọc reply handler:

```
export function createReplyChannel (channel) {
  return function registerHandler (handler) {
    channel.on('message', async message => {
      if (message.type !== 'request') {
        return
      }
      const replyData = await handler(message.data) // (1)
      channel.send({                                // (2)
        type: 'response',
        data: replyData,
        inReplyTo: message.id
      })
    })
  }
}
```

Our `createReplyChannel()` function is again a factory that returns another function used to register new reply handlers. This is what happens when a new handler is registered:

Hàm `createReplyChannel()` của ta lại là một factory trả về một hàm khác dùng để đăng ký các reply handler mới. Đây là điều xảy ra khi một handler mới được đăng ký:

1. When we receive a new request, we immediately invoke the handler by passing the data contained in the message.
2. Once the handler has done its work and returned its reply, we build an envelope around the data and include the type of the message and the correlation ID of the request (the `inReplyTo` property), then we put everything back into the channel.
 1. Khi ta nhận một request mới, ta lập tức gọi handler bằng cách truyền dữ liệu chứa trong thông điệp.
 2. Một khi handler đã làm xong công việc của nó và trả về reply của nó, ta xây một phong bì quanh dữ liệu và bao gồm loại của thông điệp và correlation ID của request (thuộc

tính `inReplyTo`), rồi ta đưa mọi thứ trở lại vào kênh.

The amazing thing about this pattern is that in Node.js it comes very easily: everything for us is already asynchronous, so an asynchronous request/reply communication built on top of a one-way channel is not very different from any other asynchronous operation, especially if we build an abstraction to hide its implementation details.

Điều tuyệt vời về mẫu này là trong Node.js nó đến rất dễ dàng: mọi thứ với ta đều đã bất đồng bộ, nên một giao tiếp request/reply bất đồng bộ được xây dựng trên nền một kênh một chiều không khác mấy so với bất kỳ thao tác bất đồng bộ nào khác, đặc biệt nếu ta xây một lớp trừu tượng để ẩn đi các chi tiết triển khai của nó.

Trying the full request/reply cycle — Thử toàn bộ chu trình request/reply Now we are ready to try our new asynchronous request/reply abstraction. Let's create a sample replier in a file named `replier.js`:

Bây giờ ta đã sẵn sàng thử lớp trừu tượng request/reply bất đồng bộ mới của mình. Hãy tạo một replier mẫu trong một file tên là `replier.js`:

```
import { createReplyChannel } from './createReplyChannel.js'
const registerReplyHandler = createReplyChannel(process)
registerReplyHandler(req => {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve({ sum: req.a + req.b })
    }, req.delay)
  })
})
process.send('ready')
```

Our replier simply calculates the sum between the two numbers received in the request and returns the result after a certain delay (which is also specified in the request). This will allow us to verify that the order of the responses can be different from the order in which we sent the requests, to confirm that our pattern is working. With the last instruction of the module, we send a message back to the parent process to indicate that the child is ready to accept requests.

Replier của ta chỉ đơn giản tính tổng giữa hai số nhận được trong request và trả về kết quả sau một độ trễ nhất định (cũng được chỉ định trong request). Điều này sẽ cho phép ta kiểm chứng rằng thứ tự của các response có thể khác với thứ tự mà ta gửi các request, để xác nhận rằng mẫu của ta đang hoạt động. Với chỉ thị cuối của module, ta gửi một thông điệp về cho tiến trình cha để cho biết rằng tiến trình con đã sẵn sàng chấp nhận các request.

The final step to complete the example is to create the requestor in a file named `requestor.js`, which also has the task of starting the replier using `child_process.fork()`:

Bước cuối để hoàn thành ví dụ là tạo requestor trong một file tên là `requestor.js`, file này cũng có nhiệm vụ khởi động replier bằng `child_process.fork()`:

```
import { fork } from 'child_process'
import { dirname, join } from 'path'
import { fileURLToPath } from 'url'
import { once } from 'events'
import { createRequestChannel } from './createRequestChannel.js'
const __dirname = dirname(fileURLToPath(import.meta.url))
async function main () {
```

```

const channel = fork(join(__dirname, 'replier.js'))           // (1)
const request = createRequestChannel(channel)
try {
  const [message] = await once(channel, 'message')           // (2)
  console.log(`Child process initialized: ${message}`)
  const p1 = request({ a: 1, b: 2, delay: 500 })              // (3)
    .then(res => {
      console.log(`Reply: 1 + 2 = ${res.sum}`)
    })
  const p2 = request({ a: 6, b: 1, delay: 100 })              // (4)
    .then(res => {
      console.log(`Reply: 6 + 1 = ${res.sum}`)
    })
  await Promise.all([p1, p2])                                  // (5)
} finally {
  channel.disconnect()                                         // (6)
}
}
main().catch(err => console.error(err))

```

The requestor starts the replier (1) and then passes its reference to our `createRequestChannel()` abstraction. We then wait for the child process to be available (2) and run a couple of sample requests (3, 4). Finally, we wait for both requests to complete (5) and we disconnect the channel (6) to allow the child process (and therefore the parent process) to exit gracefully.

Requestor khởi động replier (1) rồi truyền tham chiếu của nó cho lớp trừu tượng `createRequestChannel()` của ta. Sau đó ta chờ tiến trình con khả dụng (2) và chạy vài request mẫu (3, 4). Cuối cùng, ta chờ cả hai request hoàn thành (5) và ta ngắt kết nối kênh (6) để cho phép tiến trình con (và do đó tiến trình cha) thoát một cách duyên dáng.

To try out the sample, simply launch the `requestor.js` module. The output should be something similar to the following:

Để thử ví dụ, chỉ cần khởi chạy module `requestor.js`. Output sẽ là gì đó tương tự như sau:

```

Child process initialized: ready
Sending request { a: 1, b: 2, delay: 500 }
Sending request { a: 6, b: 1, delay: 100 }
Reply: 6 + 1 = 7
Reply: 1 + 2 = 3

```

This confirms that our implementation of the Request/Reply messaging pattern works perfectly and that the replies are correctly associated with their respective requests, no matter in what order they are sent or received.

Điều này xác nhận rằng bản triển khai mẫu nhắn tin Request/Reply của ta hoạt động hoàn hảo và rằng các reply được liên kết đúng với các request tương ứng của chúng, bất kể chúng được gửi hay nhận theo thứ tự nào.

The technique we've discussed in this section works great when we have a single point-to-point channel. But what happens if we have a more complex architecture with multiple channels or queues? That's what we are going to see next.

Kỹ thuật mà ta đã thảo luận trong phần này hoạt động tuyệt vời khi ta có một kênh point-

to-point duy nhất. Nhưng điều gì xảy ra nếu ta có một kiến trúc phức tạp hơn với nhiều kênh hoặc nhiều queue? Đó là điều ta sắp xem tiếp theo.

Return address — Return address

The Correlation Identifier is the fundamental pattern for creating a request/reply communication on top of a one-way channel. However, it's not enough when our messaging architecture has more than one channel or queue, or when there can be potentially more than one requestor. In these situations, in addition to a correlation ID, we also need to know the return address, a piece of information that allows the replier to send the response back to the original sender of the request.

Correlation Identifier là mẫu nền tảng để tạo ra một giao tiếp request/reply trên nền một kênh một chiều. Tuy nhiên, nó không đủ khi kiến trúc nhắn tin của ta có nhiều hơn một kênh hoặc một queue, hoặc khi có khả năng nhiều hơn một requestor. Trong những tình huống này, ngoài một correlation ID, ta cũng cần biết return address, một mẫu thông tin cho phép replier gửi response trở lại cho bên gửi gốc của request.

Implementing the Return Address pattern in AMQP — Triển khai mẫu Return Address trong AMQP

In the context of an AMQP-based architecture, the return address is the queue where the requestor is listening for incoming replies. Because the response is meant to be received by only one requestor, it's important that the queue is private and not shared across different consumers. From these properties, we can infer that we are going to need a transient queue scoped to the connection of the requestor, and that the replier has to establish a point-to-point communication with the return queue to be able to deliver its responses.

Trong ngữ cảnh một kiến trúc dựa trên AMQP, return address là queue nơi requestor lắng nghe các reply đến. Vì response chỉ được dành để nhận bởi một requestor duy nhất, điều quan trọng là queue phải riêng tư (private) và không được chia sẻ qua các consumer khác nhau. Từ những đặc tính này, ta có thể suy ra rằng ta sẽ cần một queue tạm thời (transient) giới hạn trong phạm vi kết nối của requestor, và rằng replier phải thiết lập một giao tiếp point-to-point với return queue để có thể chuyển giao các response của nó.

The following diagram gives us an example of this scenario:

Sơ đồ sau cho ta một ví dụ về kịch bản này:

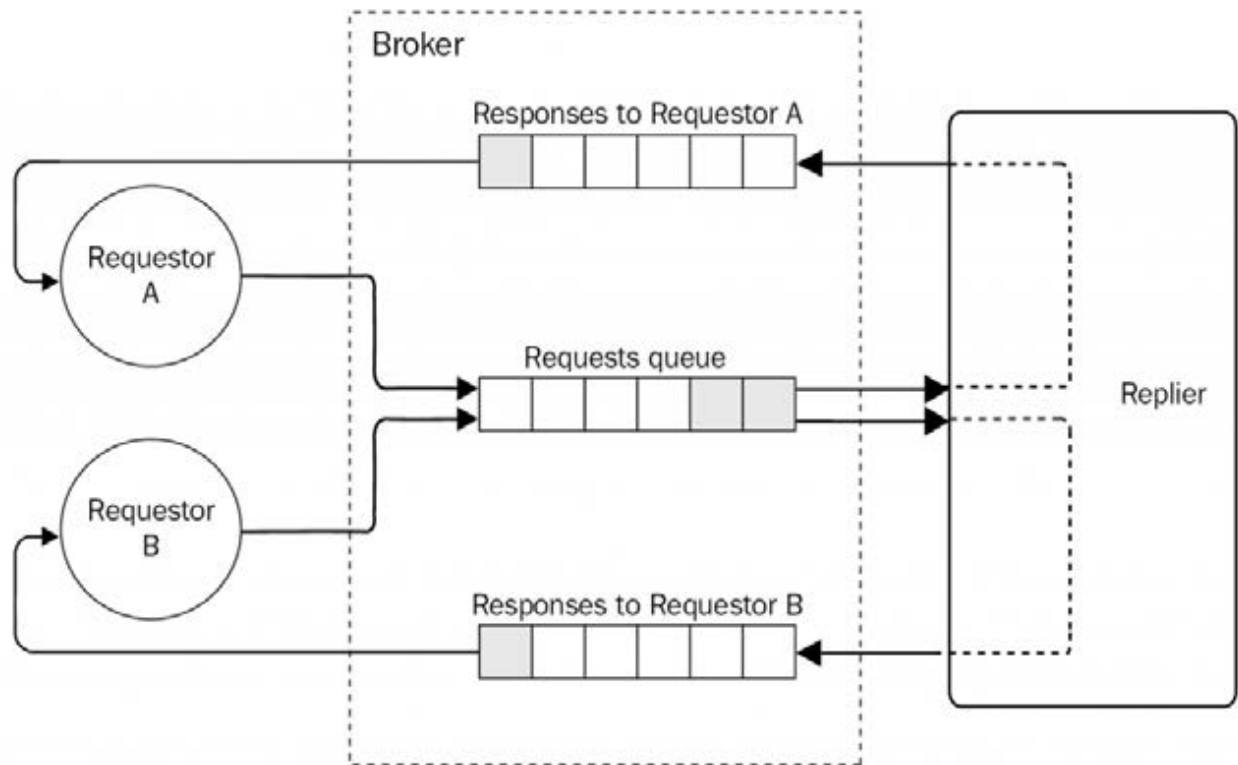


Figure 13.22: Request/reply messaging architecture using AMQP

Hình 13.22: Kiến trúc nhắn tin request/reply dùng AMQP

Figure 13.22 shows us how each requestor has its own private queue, specifically intended to handle the replies to their requests. All requests are sent instead to a single queue, which is then consumed by the replier. The replier will route the replies to the correct response queue thanks to the return address information specified in the request. In fact, to create a Request/Reply pattern on top of AMQP, all we need to do is to specify the name of the response queue in the message properties, so that the replier knows where the response message has to be delivered.

Hình 13.22 cho ta thấy cách mỗi requestor có queue riêng tư của riêng nó, được dành riêng để xử lý các reply cho các request của nó. Thay vào đó, tất cả các request đều được gửi tới một queue duy nhất, queue này sau đó được tiêu thụ bởi replier. Replier sẽ định tuyến các reply tới đúng response queue nhờ thông tin return address được chỉ định trong request. Thực tế, để tạo một mẫu Request/Reply trên nền AMQP, tất cả những gì ta cần làm là chỉ định tên của response queue trong các thuộc tính của thông điệp, để replier biết response message phải được chuyển giao đi đâu.

The theory seems very straightforward, so let's see how to implement this in a real application.

Lý thuyết có vẻ rất đơn giản, vậy hãy xem cách triển khai điều này trong một ứng dụng thực tế.

Implementing the request abstraction — Triển khai lớp trừu tượng request Let's now build a request/reply abstraction on top of AMQP. We will use RabbitMQ as a broker, but any compatible AMQP broker should do the job. Let's start with the request abstraction, implemented in the `amqpRequest.js` module. We will show the code here one piece at a time to make the explanation easier. Let's start from the constructor of the `AMQPRequest` class:

Bây giờ hãy xây dựng một lớp trừu tượng request/reply trên nền AMQP. Ta sẽ dùng RabbitMQ làm broker, nhưng bất kỳ AMQP broker tương thích nào cũng làm được công việc. Hãy bắt đầu với lớp trừu tượng request, được triển khai trong module `amqpRequest.js`. Ta sẽ hiển thị code ở đây từng mảnh một để việc giải thích dễ dàng hơn. Hãy bắt đầu từ constructor của lớp `AMQPRequest`:

```
export class AMQPRequest {
  constructor () {
    this.correlationMap = new Map()
  }
  //...
```

As we can see from the preceding code, we will again be using the Correlation Identifier pattern, so we are going to need a map to hold the association between the message ID and the relative handler.

Như ta thấy từ đoạn code trên, ta lại sẽ dùng mẫu Correlation Identifier, nên ta sẽ cần một map để giữ sự liên kết giữa ID của thông điệp và handler tương ứng.

Then, we need a method to initialize the AMQP connection and its objects:

Sau đó, ta cần một phương thức để khởi tạo kết nối AMQP và các object của nó:

```
async initialize () {
  this.connection = await amqp.connect('amqp://localhost')
  this.channel = await this.connection.createChannel()
  const { queue } = await this.channel.assertQueue('',           // (1)
    { exclusive: true })
  this.replyQueue = queue
  this.channel.consume(this.replyQueue, msg => {                // (2)
    const correlationId = msg.properties.correlationId
    const handler = this.correlationMap.get(correlationId)
    if (handler) {
      handler(JSON.parse(msg.content.toString()))
    }
  }, { noAck: true })
}
```

The interesting thing to observe here is how we create the queue to hold the replies (1). The peculiarity is that we don't specify any name, which means that a random one will be chosen for us. In addition to this, the queue is exclusive, which means that it's bound to the currently active AMQP connection and it will be destroyed when the connection closes. There is no need to bind the queue to an exchange as we don't need any routing or distribution to multiple queues, which means that the messages have to be delivered straight into our response queue. In the second part of the function (2), we start to consume the messages from the replyQueue. Here we match the ID of the incoming message with the one we have in our correlationMap and invoke the associated handler.

Điều thú vị cần quan sát ở đây là cách ta tạo queue để giữ các reply (1). Điểm đặc thù là ta không chỉ định tên nào, nghĩa là một tên ngẫu nhiên sẽ được chọn cho ta. Ngoài ra, queue là exclusive, nghĩa là nó gắn với kết nối AMQP hiện đang hoạt động và nó sẽ bị hủy khi kết nối đóng. Không cần bind queue vào một exchange vì ta không cần định tuyến hay phân phối tới nhiều queue, nghĩa là các thông điệp phải được chuyển giao thẳng vào response queue của ta. Trong phần thứ hai của hàm (2), ta bắt đầu tiêu thụ các thông điệp từ replyQueue. Ở đây ta khớp ID của thông điệp đến với ID mà ta có trong correlationMap và gọi handler

liên quan.

Next, let's see how it's possible to send new requests:

Tiếp theo, hãy xem làm thế nào có thể gửi các request mới:

```
send (queue, message) {
  return new Promise((resolve, reject) => {
    const id = nanoid() // (1)
    const replyTimeout = setTimeout(() => {
      this.correlationMap.delete(id)
      reject(new Error('Request timeout'))
    }, 10000)
    this.correlationMap.set(id, (replyData) => { // (2)
      this.correlationMap.delete(id)
      clearTimeout(replyTimeout)
      resolve(replyData)
    })
    this.channel.sendToQueue(queue, // (3)
      Buffer.from(JSON.stringify(message)),
      { correlationId: id, replyTo: this.replyQueue }
    )
  })
}
```

The `send()` method accepts as input the name of the requests queue and the message to send. As we learned in the previous section, we need to generate a correlation ID (1) and associate it to a handler responsible for returning the reply to the caller (2). Finally, we send the message (3), specifying the `correlationId` and the `replyTo` property as metadata. In AMQP, in fact, we can specify a set of properties (or metadata) to be passed to the consumer, together with the main message. The metadata object is passed as the third argument of the `sendToQueue()` method. It's important to note that we are using the `channel.sendToQueue()` API instead of `channel.publish()` to send the message. This is because we are not interested in implementing a publish/subscribe distribution pattern using exchanges, but a more basic point-to-point delivery straight into the destination queue.

Phương thức `send()` nhận làm đầu vào tên của queue request và message cần gửi. Như ta đã học ở phần trước, ta cần sinh ra một correlation ID (1) và liên kết nó với một handler chịu trách nhiệm trả reply về cho bên gọi (2). Cuối cùng, ta gửi thông điệp (3), chỉ định thuộc tính `correlationId` và `replyTo` làm metadata. Thực tế, trong AMQP, ta có thể chỉ định một tập các thuộc tính (hay metadata) để truyền tới consumer, cùng với thông điệp chính. Object metadata được truyền làm đối số thứ ba của phương thức `sendToQueue()`. Cũng quan trọng cần lưu ý rằng ta đang dùng API `channel.sendToQueue()` thay vì `channel.publish()` để gửi thông điệp. Đây là vì ta không quan tâm tới việc triển khai một mẫu phân phối publish/subscribe dùng exchange, mà là một kiểu chuyển giao point-to-point cơ bản hơn thẳng vào queue đích.

The last piece of our `AMQPRequest` class is where we implement the `destroy()` method, which is used to close the connection and the channel:

Mảnh cuối của lớp `AMQPRequest` của ta là nơi ta triển khai phương thức `destroy()`, dùng để đóng kết nối và channel:

```
destroy () {
  this.channel.close()
}
```

```

    this.connection.close()
  }
}

```

That's it for the `amqpRequest.js` module.

Vậy là xong với module `amqpRequest.js`.

Implementing the reply abstraction — Triển khai lớp trừu tượng reply Now it's time to implement the reply abstraction in a new module named `amqpReply.js`:

Bây giờ đến lúc triển khai lớp trừu tượng reply trong một module mới tên là `amqpReply.js`:

```

import amqp from 'amqplib'
export class AMQPReply {
  constructor (requestsQueueName) {
    this.requestsQueueName = requestsQueueName
  }
  async initialize () {
    const connection = await amqp.connect('amqp://localhost')
    this.channel = await connection.createChannel()
    const { queue } = await this.channel.assertQueue(           // (1)
      this.requestsQueueName)
    this.queue = queue
  }
  handleRequests (handler) {                                   // (2)
    this.channel.consume(this.queue, async msg => {
      const content = JSON.parse(msg.content.toString())
      const replyData = await handler(content)
      this.channel.sendToQueue(                                 // (3)
        msg.properties.replyTo,
        Buffer.from(JSON.stringify(replyData)),
        { correlationId: msg.properties.correlationId }
      )
      this.channel.ack(msg)
    })
  }
}

```

In the `initialize()` method of the `AMQPReply` class, we create the queue that will receive the incoming requests (1): we can use a simple durable queue for this purpose. The `handleRequests()` method (2) is used to register new request handlers from where new replies can be sent. When sending back a reply (3), we use `channel.sendToQueue()` to publish the message straight into the queue specified in the `replyTo` property of the message (our return address). We also set the `correlationId` in the reply, so that the receiver can match the message with the list of pending requests.

Trong phương thức `initialize()` của lớp `AMQPReply`, ta tạo queue sẽ nhận các request đến (1): ta có thể dùng một durable queue đơn giản cho mục đích này. Phương thức `handleRequests()` (2) được dùng để đăng ký các request handler mới từ đó các reply mới có thể được gửi. Khi gửi lại một reply (3), ta dùng `channel.sendToQueue()` để publish thông điệp thẳng vào queue được chỉ định trong thuộc tính `replyTo` của thông

điệp (return address của ta). Ta cũng đặt `correlationId` trong reply, để bên nhận có thể khớp thông điệp với danh sách các request đang chờ.

Implementing the requestor and the replier — Triển khai requestor và replier Everything is now ready to give our system a try, but first, let's build a pair sample requestor and replier to see how to use our new abstraction.

Bây giờ mọi thứ đã sẵn sàng để thử hệ thống của ta, nhưng trước tiên, hãy xây một cặp requestor và replier mẫu để xem cách dùng lớp trừu tượng mới của ta.

Let's start with the replier.js module:

Hãy bắt đầu với module replier.js:

```
import { AMQPReply } from './amqpReply.js'
async function main () {
  const reply = new AMQPReply('requests_queue')
  await reply.initialize()
  reply.handleRequests(req => {
    console.log('Request received', req)
    return { sum: req.a + req.b }
  })
}
main().catch(err => console.error(err))
```

It's nice to see how the abstraction we built allows us to hide all the mechanisms to handle the correlation ID and the return address. All we need to do is initialize a new reply object, specifying the name of the queue where we want to receive our requests ('requests_queue'). The rest of the code is just trivial; in practice, our sample replier simply calculates the sum of the two numbers received as the input and sends back the result in an object. On the other side, we have a sample requestor implemented in the requestor.js file:

Thật hay khi thấy cách lớp trừu tượng mà ta đã xây cho phép ta ẩn đi tất cả các cơ chế để xử lý correlation ID và return address. Tất cả những gì ta cần làm là khởi tạo một object reply mới, chỉ định tên của queue nơi ta muốn nhận các request của mình ('requests_queue'). Phần còn lại của code chỉ là tầm thường; trên thực tế, replier mẫu của ta chỉ đơn giản tính tổng của hai số nhận được làm đầu vào và gửi lại kết quả trong một object. Ở phía bên kia, ta có một requestor mẫu được triển khai trong file requestor.js:

```
import { AMQPRequest } from './amqpRequest.js'
import delay from 'delay'
async function main () {
  const request = new AMQPRequest()
  await request.initialize()
  async function sendRandomRequest () {
    const a = Math.round(Math.random() * 100)
    const b = Math.round(Math.random() * 100)
    const reply = await request.send('requests_queue', { a, b })
    console.log(`${a} + ${b} = ${reply.sum}`)
  }
  for (let i = 0; i < 20; i++) {
    await sendRandomRequest()
    await delay(1000)
  }
}
```

```
    request.destroy()
  }
  main().catch(err => console.error(err))
```

Our sample requestor sends 20 random requests at one-second intervals to the `requests_queue` queue. In this case, also, it's interesting to see that our abstraction is doing its job perfectly, hiding all the details behind the implementation of the asynchronous Request/Reply pattern.

Requestor mẫu của ta gửi 20 request ngẫu nhiên tại các khoảng cách một giây tới queue `requests_queue`. Trong trường hợp này, cũng thú vị khi thấy rằng lớp trừu tượng của ta đang làm tốt công việc của nó, ẩn đi tất cả các chi tiết đằng sau bản triển khai của mẫu Request/Reply bất đồng bộ.

Now, to try out the system, simply run the `replier` module followed by a couple of `requestor` instances:

Bây giờ, để thử hệ thống, chỉ cần chạy module `replier` rồi tới vài instance `requestor`:

```
node replier.js
node requestor.js
node requestor.js
```

You will see a set of operations published by the requestors and then received by the replier, which in turn will send back the responses to the right requestor. Now we can try other experiments. Once the replier is started for the first time, it creates a durable queue and this means that if we now stop it and then run the replier again, no request will be lost. All the messages will be stored in the queue until the replier is started again!

Bạn sẽ thấy một tập các thao tác được publish bởi các requestor rồi được nhận bởi replier, replier đến lượt mình sẽ gửi lại các response cho đúng requestor. Bây giờ ta có thể thử các thí nghiệm khác. Một khi replier được khởi động lần đầu tiên, nó tạo một durable queue và điều này nghĩa là nếu ta dừng nó rồi chạy replier lại, không có request nào bị mất. Tất cả các thông điệp sẽ được lưu trong queue cho tới khi replier được khởi động lại!

Note that based on how we implemented the application, a request will time out after 10 seconds. So, in order for a reply to reach the requestor in time, the replier can afford to have only a limited downtime (certainly less than 10 seconds).

Lưu ý rằng dựa trên cách ta đã triển khai ứng dụng, một request sẽ timeout sau 10 giây. Vậy nên, để một reply tới được requestor kịp thời, replier chỉ có thể có một khoảng downtime hạn chế (chắc chắn ít hơn 10 giây).

Another nice feature that we get for free by using AMQP is the fact that our replier is scalable out of the box. To test this assumption, we can try to start two or more instances of the replier, and watch the requests being load balanced between them. This works because, every time a requestor starts, it attaches itself as a listener to the same durable queue, and as a result, the broker will load balance the messages across all the consumers of the queue (remember the Competing Consumers pattern?). Sweet!

Một tính năng hay khác mà ta nhận được miễn phí khi dùng AMQP là việc replier của ta có khả năng mở rộng sẵn (scalable out of the box). Để kiểm chứng giả định này, ta có thể thử khởi động hai hay nhiều instance của replier, và quan sát các request được load balance giữa chúng. Điều này hoạt động vì, mỗi lần một requestor khởi động, nó tự gắn mình như một listener vào cùng durable queue đó, và kết quả là, broker sẽ load balance các thông điệp qua tất cả các consumer của queue (nhớ mẫu Competing Consumers chứ?). Tuyệt!

ZeroMQ has a pair of sockets specifically meant for implementing request/reply patterns, called REQ/REP, however, they are synchronous (only one request/response at a time). More complex request/reply patterns are possible with more sophisticated techniques. For more information, you can read the official guide at nodejsdp.link/zeromq-reqrep.

ZeroMQ có một cặp socket được dành riêng để triển khai các mẫu request/reply, gọi là REQ/REP, tuy nhiên, chúng đồng bộ (chỉ một request/response tại một thời điểm). Các mẫu request/reply phức tạp hơn khả thi với các kỹ thuật tinh vi hơn. Để biết thêm thông tin, bạn có thể đọc hướng dẫn chính thức tại nodejsdp.link/zeromq-reqrep.

A Request/Reply pattern with a return address is also possible on top of Redis Streams and resembles very closely the system we implemented with AMQP. We'll leave this to you to implement as an exercise.

Một mẫu Request/Reply với return address cũng khả thi trên nền Redis Streams và giống rất chặt chẽ với hệ thống mà ta đã triển khai với AMQP. Ta sẽ để lại điều này cho bạn triển khai như một bài tập.

Summary — Tóm tắt

You have reached the end of this chapter. Here, you learned the most important messaging and integration patterns and the role they play in the design of distributed systems. You should now have mastered the three most important types of message exchange patterns: Publish/Subscribe, Task Distribution, and Request/Reply, implemented either on top of a peer-to-peer architecture or using a broker. We analyzed the pros and cons of each pattern and architecture, and we saw that by using a broker (implementing either a message queue or data stream), it's possible to implement reliable and scalable applications with little effort, but at the cost of having one more system to maintain and scale. You have also learned how ZeroMQ allows you to build distributed systems where you can have total control over every aspect of the architecture, fine tuning its properties around your very own requirements.

Bạn đã đến cuối chương này. Ở đây, bạn đã học các mẫu nhắn tin và tích hợp quan trọng nhất và vai trò mà chúng đóng trong việc thiết kế các hệ thống phân tán. Giờ bạn hẳn đã nắm vững ba loại mẫu trao đổi thông điệp quan trọng nhất: Publish/Subscribe, Task Distribution và Request/Reply, được triển khai hoặc trên nền một kiến trúc peer-to-peer hoặc dùng một broker. Ta đã phân tích các ưu nhược điểm của mỗi mẫu và kiến trúc, và ta đã thấy rằng bằng cách dùng một broker (triển khai hoặc một message queue hoặc một data stream), có thể triển khai các ứng dụng đáng tin cậy và có khả năng mở rộng với ít công sức, nhưng phải trả giá bằng việc có thêm một hệ thống nữa để bảo trì và mở rộng. Bạn cũng đã học cách ZeroMQ cho phép bạn xây dựng các hệ thống phân tán nơi bạn có thể có toàn quyền kiểm soát mọi khía cạnh của kiến trúc, tinh chỉnh các đặc tính của nó quanh chính các yêu cầu của bạn.

Ultimately, both approaches will give you all the tools that you need to build any type of distributed systems, from basic chat applications to web-scale platforms used by millions of people.

Cuối cùng, cả hai cách tiếp cận sẽ cho bạn tất cả các công cụ bạn cần để xây dựng bất kỳ loại hệ thống phân tán nào, từ các ứng dụng chat cơ bản tới các nền tảng quy mô web được hàng triệu người dùng sử dụng.

This chapter also closes the book. By now, you should have a toolbox full of patterns and techniques that you can go and apply in your projects. You should also have a deeper understanding of how Node.js development works and what its strengths and weaknesses are. Throughout the book,

you also had the chance to work with a myriad of packages and solutions developed by many extraordinary developers. In the end, this is the most beautiful aspect of Node.js: its people, a community where everybody plays their part in giving something back.

Chương này cũng khép lại cuốn sách. Đến giờ, bạn hẳn đã có một bộ đồ nghề đầy ắp các mẫu và kỹ thuật mà bạn có thể đi và áp dụng vào các dự án của mình. Bạn cũng nên có một hiểu biết sâu sắc hơn về cách phát triển Node.js hoạt động và đâu là điểm mạnh và điểm yếu của nó. Xuyên suốt cuốn sách, bạn cũng đã có cơ hội làm việc với vô số gói và giải pháp được phát triển bởi nhiều lập trình viên phi thường. Cuối cùng, đây là khía cạnh đẹp đẽ nhất của Node.js: con người của nó, một cộng đồng nơi mọi người đóng vai trò của mình trong việc cho đi điều gì đó.

We hope you enjoyed our small contribution and we look forward to seeing yours.

Chúng tôi hy vọng bạn đã thích đóng góp nhỏ của chúng tôi và chúng tôi mong chờ được thấy đóng góp của bạn.

Sincerely, Mario Casciaro and Luciano Mammino.

Trân trọng, Mario Casciaro và Luciano Mammino.

Exercises — Bài tập

- 13.1 History service with streams: In our publish/subscribe example with Redis Stream, we didn't need a history service (as we did instead in the related AMQP example) because all the message history was saved in the stream anyway. Now, implement such a history service, storing all the incoming messages in a separate database and use this service to retrieve the chat history when a new client connects. Hint: the history service will need to remember the ID of the last message retrieved across restarts.
- 13.2 Multiroom chat: Update the chat application example we created in this chapter to be able to support multiple chat rooms. The application should also support displaying the message history when the client connects. You can choose the messaging system you prefer, and even mix different ones.
- 13.3 Tasks that stop: Update the hashsum cracker examples we implemented in this chapter and add the necessary logic to stop the computation on all nodes once a match has been found.
- 13.4 Reliable task processing with ZeroMQ: Implement a mechanism to make our hashsum cracker example with ZeroMQ more reliable. As we already mentioned, with the implementation we saw in this chapter, if a worker crashes, all the tasks it was processing are lost. Implement a peer-to-peer queuing system and an acknowledgment mechanism to make sure that the message is always processed at least once (excluding errors due to hypothetical unprocessable tasks).
- 13.5 Data aggregator: Create an abstraction that can be used to send a request to all the nodes connected to the system and then returns an aggregation of all the replies received by those nodes. Hint: you can use publish/reply to send the request, and any one-way channel to send back the replies. Use any combination of the technologies we have learned.
- 13.6 Worker status CLI: Use the data aggregator component defined in Exercise 13.5 to implement a command-line application that, when invoked, displays the current status of all the workers of the hashsum cracker application (for example, which chunk they are processing, whether they found a match, and so on).
- 13.7 Worker status UI: Implement a web application (from client to server) to expose the status of the workers of the hashsum cracker application through a web UI that can report in real time when a match is found.

- 13.8 Pre-initialization queues are back: In the AMQP request/reply example, we implemented a Delayed Startup pattern to deal with the fact that the `initialize()` method is asynchronous. Now, **refactor** that example by adding pre-initialization queues as we learned in Chapter 11, Advanced Recipes.
- 13.9 Request/reply with Redis Streams: Build a request/reply abstraction on top of Redis Streams.
- 13.10 Kafka: If you are brave enough, try to reimplement all relevant examples in this chapter using Apache Kafka (`nodejsdp.link/kafka`) instead of Redis Streams.
 - 13.1 History service với stream: Trong ví dụ publish/subscribe của ta với Redis Stream, ta không cần một history service (như ta đã cần trong ví dụ AMQP liên quan) vì toàn bộ lịch sử thông điệp dù sao cũng được lưu trong stream. Bây giờ, hãy triển khai một history service như vậy, lưu trữ tất cả các thông điệp đến trong một database riêng và dùng service này để lấy lại lịch sử chat khi một client mới kết nối. Gợi ý: history service sẽ cần ghi nhớ ID của thông điệp được lấy lại cuối cùng qua các lần khởi động lại.
 - 13.2 Chat đa phòng: Cập nhật ví dụ ứng dụng chat mà ta đã tạo trong chương này để có thể hỗ trợ nhiều phòng chat. Ứng dụng cũng nên hỗ trợ hiển thị lịch sử thông điệp khi client kết nối. Bạn có thể chọn messaging system mà bạn thích, và thậm chí trộn các loại khác nhau.
 - 13.3 Tác vụ biết dừng: Cập nhật các ví dụ bộ phá hashsum mà ta đã triển khai trong chương này và thêm logic cần thiết để dừng việc tính toán trên tất cả các node một khi tìm thấy một khớp.
 - 13.4 Xử lý tác vụ đáng tin cậy với ZeroMQ: Triển khai một cơ chế để làm cho ví dụ bộ phá hashsum với ZeroMQ của ta đáng tin cậy hơn. Như ta đã đề cập, với bản triển khai mà ta đã thấy trong chương này, nếu một worker crash, tất cả các tác vụ mà nó đang xử lý đều bị mất. Hãy triển khai một hệ thống queuing peer-to-peer và một cơ chế acknowledgment để đảm bảo rằng thông điệp luôn được xử lý ít nhất một lần (loại trừ các lỗi do các tác vụ giả định không thể xử lý được).
 - 13.5 Bộ tổng hợp dữ liệu: Tạo một lớp trừu tượng có thể được dùng để gửi một request tới tất cả các node kết nối tới hệ thống rồi trả về một bản tổng hợp của tất cả các reply nhận được từ các node đó. Gợi ý: bạn có thể dùng publish/reply để gửi request, và bất kỳ kênh nào để gửi lại các reply. Dùng bất kỳ tổ hợp nào của các công nghệ mà ta đã học.
 - 13.6 CLI trạng thái worker: Dùng thành phần bộ tổng hợp dữ liệu được định nghĩa trong Bài tập 13.5 để triển khai một ứng dụng dòng lệnh mà, khi được gọi, hiển thị trạng thái hiện tại của tất cả các worker của ứng dụng bộ phá hashsum (ví dụ, chúng đang xử lý chunk nào, chúng có tìm thấy một khớp không, v.v.).
 - 13.7 UI trạng thái worker: Triển khai một ứng dụng web (từ client tới server) để phơi bày trạng thái của các worker của ứng dụng bộ phá hashsum thông qua một web UI có thể báo cáo theo thời gian thực khi tìm thấy một khớp.
 - 13.8 Pre-initialization queue trở lại: Trong ví dụ request/reply AMQP, ta đã triển khai một mẫu Delayed Startup để đối phó với việc phương thức `initialize()` là bất đồng bộ. Bây giờ, hãy tái cấu trúc (refactor) ví dụ đó bằng cách thêm các pre-initialization queue như ta đã học trong Chương 11, Advanced Recipes.
 - 13.9 Request/reply với Redis Streams: Xây dựng một lớp trừu tượng request/reply trên nền Redis Streams.
 - 13.10 Kafka: Nếu bạn đủ can đảm, hãy thử triển khai lại tất cả các ví dụ liên quan trong chương này dùng Apache Kafka (`nodejsdp.link/kafka`) thay vì Redis Streams.